Engineering Computations and Modeling in MATLAB®/Simulink®

Second Edition

Oleg A. Yakimenko

AIAA EDUCATION SERIES

Joseph A. Schetz
Editor-In-Chief

Engineering Computations and Modeling in MATLAB® / Simulink®

Second Edition

Engineering Computations and Modeling in MATLAB® / Simulink®

Second Edition

Oleg A. Yakimenko
Naval Postgraduate School
Monterey, California

AIAA EDUCATION SERIES

Joseph A. Schetz, Editor-in-Chief
Virginia Polytechnic Institute and State University
Blacksburg, Virginia

American Institute of Aeronautics and Astronautics, Inc.

American Institute of Aeronautics and Astronautics, Inc., Reston, Virginia

1 2 3 4 5

Cover images: Courtesy NASA/JPL-Caltech.

Library of Congress Cataloging-in-Publication Data

Names: Yakimenko, Oleg A., author.
Title: Engineering computations and modeling in MATLAB/Simulink / Oleg A. Yakimenko, Naval Postgraduate School Monterey, California.
Description: Second edition. | Reston, Virginia : American Institute of Aeronautics and Astronautics, Inc., 2019. | Series: AIAA education series | Includes bibliographical references and index. | Summary: "Engineering Computations and Modeling in MATLAB/Simulink, Second Edition provides a broad overview of The MathWorks' MATLAB/Simulink/GUIDE/App Designer development and modeling environment. This textbook shows how to use these software tools effectively in a variety of engineering applications, including modeling real-world dynamic systems. This book is used as a textbook by undergraduate and graduate students taking both introductory and advanced courses in applied programming and numerical analysis, and also as a reference for working professionals using MATLAB/Simulink"—Provided by publisher.
Identifiers: LCCN 2019050008 | ISBN 9781624105159 (hardcover)
Subjects: LCSH: Engineering mathematics—Data processing. | Computer programming. | MATLAB. | SIMULINK.
Classification: LCC TA345 .Y35 2019 | DDC 620.00285/53–dc23
LC record available at https://lccn.loc.gov/2019050008

MATLAB® and Simulink® are registered trademarks of the MathWorks, Inc., 3 Apple Hill Drive, Natick, MA 01760-2098, USA; www.mathworks.com

Copyright © 2020 by the American Institute of Aeronautics and Astronautics, Inc. The U.S. Government has a world-wide, royalty-free license to exercise all rights under the copyright claimed herein for governmental purposes. All other rights are reserved by the copyright owner. Printed in the United States of America. No part of this publication may be reproduced, distributed, or transmitted, in any form or by any means, or stored in a database or retrieval system, without the prior written permission of the publisher.

Data and information appearing in this book are for informational purposes only. AIAA is not responsible for any injury or damage resulting from use or reliance, nor does AIAA warrant that use or reliance will be free from privately owned rights.

AIAA EDUCATION SERIES

Editor-in-Chief

Joseph A. Schetz

Virginia Polytechnic Institute and State University

Editorial Board

João Luiz F. Azevedo
Comando-Geral De Tecnologia Aeroespacial

Marty Bradley
The Boeing Company

James R. DeBonis
NASA Glenn Research Center

Kajal K. Gupta
NASA Dryden Flight Research Center

Rakesh K. Kapania
Virginia Polytechnic Institute and State University

Brian Landrum
University of Alabama, Huntsville

Michael Mohaghegh
The Boeing Company

Conrad F. Newberry
Naval Postgraduate School; California State Polytechnic University, Pomona

Brett Newman
Old Dominion University

Hanspeter Schaub
University of Colorado

David M. Van Wie
Johns Hopkins University

To Alena, Marina, Irina, Eugene

CONTENTS

Preface xiii

Conventions xix

List of Acronyms xxi

Chapter 1 MATLAB/Simulink as a Technical Computing Language 1

1.1 Introduction 1
1.2 History of MATLAB Creation and Development 1
1.3 Capabilities and Resources 4
1.4 Aerospace Application Tools 7
1.5 Overview of MathWorks Products 8

Chapter 2 Development Environment and Basic Operations 21

2.1 Introduction 21
2.2 MATLAB Development Environment and MATLAB Online 21
2.3 Help System 46
2.4 Function Collections 49
2.5 Order of Precedence 56
2.6 Managing Work Session 57
2.7 Changing Preferences 63
2.8 Problem-Solving Methodology 68
Problems 69

Chapter 3 Arrays and Array Operations 71

3.1 Introduction 71
3.2 Types of Arrays and Indexing Their Elements 72
3.3 Array Operations 92
3.4 Array Functions 101
3.5 Using MATLAB Matrix Formalism To Handle Polynomials 110

3.6 Handling Text as Character and String Arrays 114
Problems 122

Chapter 4 Mixed-Type Data "Containers" and Input/Output Management 125

4.1 Introduction 125
4.2 Data Types 126
4.3 Cell Arrays 129
4.4 Structure Arrays 134
4.5 Tables 139
4.6 Types of MATLAB Files 145
4.7 Recording MATLAB Sessions and Storing Workspace Variables 148
4.8 Importing and Exporting Data 150
4.9 Writing and Displaying Formatted Data 168
4.10 Interactive Input and Output 172
Problems 177

Chapter 5 Programming in MATLAB 179

5.1 Introduction 179
5.2 Relational Operators 180
5.3 Logical (Boolean) Operators and Functions 181
5.4 Conditional Statements 184
5.5 Loops 188
5.6 The `switch` Construct 191
5.7 The Script Files 193
5.8 Functions 195
5.9 Developing Professional-Looking Functions 215
5.10 Editing and Debugging 225
5.11 Code Analyzer and Profiler 233
5.12 Formatting MATLAB Comments for Publishing 238
Problems 245

Chapter 6 2-D and 3-D Plotting and Animation 247

6.1 Introduction 247
6.2 Anatomy of the 2-D (x–y) Plots in MATLAB 248
6.3 Line Specifications 255
6.4 Accessing Plot Properties via the `get` and `set` Functions 258
6.5 Text Strings 268
6.6 Overlays, Legends, Subplots, and Multiple Figures 271
6.7 Other Types of the 2-D Plots 282
6.8 The 3-D Plots 292
6.9 Changing the Color Palette 310

6.10 Function-Plotting Functions 318
6.11 Plot Editing 320
6.12 Interactive Plotting and Animation 326
6.13 Requirements to the Engineering Plots 336
Problems 337

Chapter 7 Symbolic Math Toolbox 339

7.1 Introduction 339
7.2 Symbolic Math Toolbox Overview 340
7.3 Getting Started 340
7.4 Linear Algebra Functions 348
7.5 Calculus Functions 348
7.6 Solving Algebraic and Differential Equations 355
7.7 Simplification and Substitution 359
7.8 Other Functions 363
7.9 Plotting Symbolic Expressions and Using Symbolic Notations 366
7.10 Using Symbolic Expressions in Numerical Operations 372
7.11 Symbolic Calculators 379
7.12 Units of Measurement 382
Problems 383

Chapter 8 Accuracy of Digital Computations 385

8.1 Introduction 385
8.2 Accuracy and Precision of Calculations and Measurements 385
8.3 Computer Number Representation 386
8.4 Round-Off Errors 403
8.5 Truncation Errors 407
8.6 Control of Numerical Errors 416
Problems 418

Chapter 9 Numerical Linear Algebra and Eigenvalue Problems 419

9.1 Introduction 419
9.2 Classification of LAEs 420
9.3 Inhomogeneous Sets of LAEs 421
9.4 Matrix Decompositions 434
9.5 Special Cases of LAEs 437
9.6 Applying Linear Algebra To Solve Systems of ODEs 448
9.7 Using the MATLAB Functions 452
Problems 472

Chapter 10 Root Finding and Introduction to Optimization 475

10.1 Introduction 475
10.2 Bracketing Methods 476
10.3 Open Methods 485
10.4 Zeroin Algorithm 498
10.5 Multiple Roots and Value Finding 501
10.6 Roots of Polynomials 506
10.7 From Root Finding to 1-D Unconstrained Optimization 507
10.8 Introduction to Multidimensional Optimization 510
10.9 Using the MATLAB Functions 514
Problems 542

Chapter 11 Curve Fitting to Measured Data 545

11.1 Introduction 545
11.2 Least-Squares Regression 546
11.3 Full-Degree Polynomial Approximation 557
11.4 Piecewise Approximation 561
11.5 Fourier Approximation 576
11.6 Using the MATLAB Functions 579
Problems 609

Chapter 12 Numerical Differentiation 611

12.1 Introduction 611
12.2 Two-Point Difference Approximations of the First-Order Derivative 612
12.3 Error Estimates from the Taylor Series Expansion 613
12.4 More Accurate Multiple-Point Approximations 617
12.5 Second- and Higher-Order Derivatives 621
12.6 Alternative Ways of Obtaining Approximation Formulas 627
12.7 Partial Derivatives 629
12.8 Using the MATLAB Functions 630
Problems 637

Chapter 13 Numerical Integration 639

13.1 Introduction 639
13.2 Rectangle Method 640
13.3 Trapezoidal Rule 642
13.4 Simpson's 1/3 Rule 645
13.5 Simpson's 3/8 Rule 649
13.6 Newton–Cotes Formulas 650
13.7 Multiple-Application Scheme 652
13.8 Gauss Quadrature 654
13.9 Using the MATLAB Functions 663
Problems 680

Chapter 14 Initial-Value Problem 683

14.1 Introduction 683
14.2 Initial-Value Problem Definition 684
14.3 Single-Step Methods 687
14.4 Adaptive Single-Step Algorithms 698
14.5 Multistep Methods 701
14.6 Stiff Problems and Other Issues 710
14.7 Using MATLAB IVP Solvers 716
14.8 Other MATLAB Solvers 730
Problems 740

Chapter 15 Simulink Basics 743

15.1 Introduction 743
15.2 Simulink Development Environment 744
15.3 Modeling Continuous Dynamics 750
15.4 Modeling Discrete Systems 774
15.5 Other Libraries of Simulink 782
15.6 Integration of MATLAB and Simulink 782
15.7 Practical Examples 794
15.8 Model Reviewing and Profiling 798
Problems 801

Appendix A *LaTeX Symbols, Math Constructs, and Matrices* 803

Appendix B *Functions for Handling Character and String Arrays* 817

Appendix C *Datetime and Categorical Data Types in Support of Tables and Timetables* 821

Appendix D *Hooke–Jeeves Minimization Function* 851

Appendix E *Aerospace Systems Modeling Tools* 861

Appendix F *Building and Sharing GUIs and Apps* 883

Bibliography 913

Index 917

Supporting Materials 931

PREFACE

This book is a resource for teaching an introductory course in programming and numerical engineering computations as a part of aeronautical and astronautical engineering curricula and is based on the course currently taught at the Naval Postgraduate School in Monterey, CA. This course has been taught in different formats (one quarter, one semester, and even two semesters) to undergraduate and graduate students for over 30 years. The two major parts of the course are the basics of a high-level programming language and its usage in engineering computations, that is, writing efficient code for applied engineering problems, which includes employing the appropriate numerical methods. The purpose of including both parts is that if you concentrate only on the language basics, students will lack experience in using this programming language in practical applications and therefore will forget it quickly. Hence, when they are introduced to numerical methods later, they will need to review programming techniques once again. It is the same concept as in learning a foreign language—you have to use it, that is, read, write, and most importantly speak. Otherwise, it will fade away in no time.

In older days, the programming portion of the course was based on FORTRAN, Pascal, or BASIC, but for the last 20 years it has been based on MathWorks' MATLAB and Simulink. MATLAB is a unique programming language and development environment that features many advantages over other programming languages. It is very easy to use, offers a wide variety of options to represent the results of computations graphically, utilizes the latest trends in computer architecture, allows easy integration with other languages (C/C++, Java), and enables real-time implementation for data acquisition, image processing, and control of autonomous vehicles and robots, that is, applications that many engineering students need for their graduate work. These factors have made MATLAB/Simulink quite popular in engineering universities worldwide. In the case of MATLAB, the aforementioned reason for introducing students to a high-level programming language and numerical methods simultaneously is even stronger because MATLAB represents a natural engineering environment that has a variety of the best numerical methods programmed effectively in its functions and toolboxes (collection of functions). Above all, the open-source paradigm

allows a user to look at the source code of a majority of these functions and, understanding the method coded within, to explore code from the standpoint of programming constructions.

The goal of this book is to teach students how to use the MathWorks' products, MATLAB and Simulink, in applied engineering computations and effective modeling by progressing from the basics of programming to writing complex scripts fully utilizing specific features of MATLAB. Therefore, the intended audience of this book is not only undergraduate and graduate students, who are using or intend to use MATLAB for the first time and have little or no experience in computer programming, but also working professionals, who have some experience in using other programming languages and wish to learn quickly about the major features of the MATLAB/Simulink development environment. This book can also serve as a quick reference for professionals, who use MATLAB/Simulink on a regular basis, and complement to its rich embedded Help system.

The content of the book is subdivided into four parts as shown in Fig. 0.1. The basics of programming in MATLAB are considered in Part I. This part introduces MATLAB/Simulink as a technical computing language, describes its development environment and basic operations, and emphasizes its uniqueness in handling multidimensional numeric, cell, and structure arrays. Part I then proceeds with teaching how to write foolproof professional scripts in MATLAB, by progressing from the simple code constructs to sophisticated programs taking advantage of different types of functions. Special attention is given to effective usage of MATLAB's plotting capabilities. This includes not only numerous 2-D and 3-D plotting functions but also manipulations with the properties of graphics objects after they have been created. The latter allows a user to alter any attribute of a plot programmatically, which in turn enables creating animations.

Part II introduces one of MATLAB's toolboxes, the Symbolic Math Toolbox. The reason for introducing this toolbox so early is that its functions

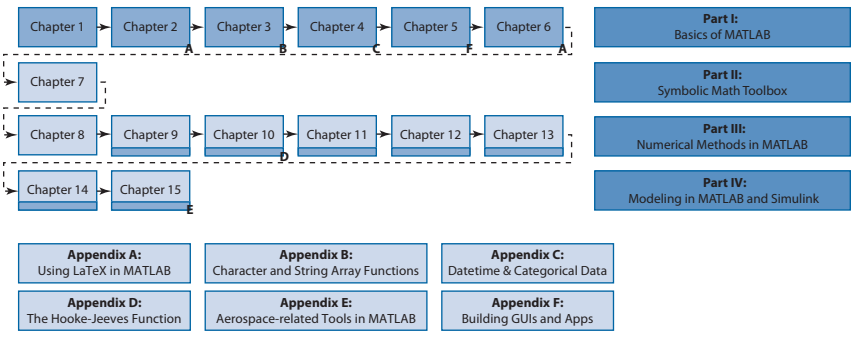


Fig. 0.1 Content of the book subdivided into parts.

can be used for developing and plotting analytical formulas for numerical methods considered in the following parts. The single chapter of this part starts with basic symbolic operations, followed by a discussion of a variety of the functions supporting symbolic computations in different areas including linear algebra, calculus, and equation solving. It ends with showing how to use symbolic expressions in numerical computations.

Part III is devoted to an overview of basic numerical methods coded in the MATLAB functions. It starts by discussing precision of digital computations in Chapter 8 and then follows the same pattern of introducing the basic principles behind a numerical method, using the Symbolic Math Toolbox to derive the corresponding analytical formulas, showing the advantages and disadvantages of different approaches, and finally discussing the available MATLAB functions realizing these methods (which are represented by the dark horizontal bar at the bottom of Chapters 9–15 in Fig. 0.1). The types of numerical computations covered in this manner include numerical linear algebra and eigenvalue problems, root finding and optimization, curve fitting, numerical differentiation, and integration.

Numerical modeling in MATLAB and Simulink is discussed in Part IV. First, the initial-value problem is introduced in Chapter 14, showing a variety of different methods to address it. Then, it follows the same pattern as in Part III, utilizing the Symbolic Math Toolbox to derive mathematical expressions for different methods. A variety of MATLAB solvers is introduced and thoroughly discussed. These solvers are also utilized in Simulink, the specialized modeling environment introduced in Chapter 15. Using several simple examples, the latter chapter covers the basics of modeling continuous and discrete systems in Simulink as well as interacting with the MATLAB development environment, leaving further exploration to readers.

All chapters in this book end with a set of three simple problems. Depending on the specific application, each instructor can come up with a variety of additional problems for his or her students, but these simple examples are intended as part of the educational process. A Solution Manual is available for a free download at the book's website. The hope is that by working through these solutions, readers will solidify their programming skills.

One of the features of MATLAB is that it allows using TeX and LaTeX interpreters to convert specific sequences of characters onto special symbols and math constructs to be used in the comment sections of the scripts (showing up when published) and their graphical output. While these capabilities, first mentioned in Sec. 2.2.3, are gradually introduced in Secs. 5.12, 6.5, and 7.9.1, Appendix A provides a full list of available LaTeX commands ("A" by Chapter 2 in Fig. 0.1 shows where Appendix A notionally belongs to).

Appendix B complements a discussion on handling text as character and string arrays started in Sec. 3.6 and for completeness lists available functions for manipulating with such the arrays.

Appendix C continues a discussion started in Sec. 4.2 on several newly introduced data types, datetime and categorical arrays, and shows how to effectively use them in manipulating with tables and timetables (another two recently introduced data types).

Appendix D complements the collection of MATLAB's Optimization Toolbox functions, considered in Chapter 10, with one more, very effective function, written and widely used by the author. This function realizes the Hooke–Jeeves method that searches for the minimum of a nonlinear function of multiple arguments without calculating its derivatives. It is given here as an example of professional-looking code that mimics inputs and outputs of any other optimization function of MATLAB.

Besides basic MATLAB and Simulink, MathWorks offers a variety of supplementary products addressing the specific needs of many industries. Surely, this includes the Aerospace industry. For completeness, the tools supporting technical computations and simulations of aerospace systems are presented in Appendix E.

A couple of useful MATLAB tools, which are included in the basic version of MATLAB—the App Designer (as of R2016a) and Graphical User Interface Development Environment (GUIDE)—are introduced in Appendix F. Readers who are interested in further improving their programming skills will find this part of the book very interesting. It takes a little bit more effort but allows enhancing the scripts with good-looking GUIs, simplifying input/ output operations and making it easier to share code with others who may not necessarily want to understand how code works but rather intend to use it with no hassle. Speaking of sharing, it is important to note that MATLAB allows the conversion of MATLAB software to MATLAB or deployed web apps, standalone desktop applications, or shared libraries. This topic is addressed in Appendix F as well.

The reference section includes publications that were used in the preparation of this book and a list of other textbooks that may be used to further enhance programming skills or to dive deeper into numerical methods. Of course, the embedded MATLAB Help system was a major source of information about different MATLAB functions and numerical methods programmed in them.

Depending on the duration of a course, an instructor may choose to cover only the material in Chapters 1-6 (basic programming techniques). If time permits, he or she may also present examples of MATLAB functions utilizing different numerical methods without talking about actual algorithms, that is, covering the very last section of each of Chapters 9–14. For a one-semester or two-semester course (depending on students' background), it may be worth dedicating more time to explaining the advantages and disadvantages of different numerical methods, that is, helping students work through the material of Chapters 9–14. The GUIDE tool is usually introduced at the end of the course, if at all, by showing a simple example and leaving further exploration to students.

To cover the Symbolic Math Toolbox, discussed in Chapter 7, I usually dedicate two hours of lab time, rather than lecture time, so that students can follow the material of this chapter and familiarize themselves with this toolbox at their own pace. The same applies to Chapter 15, which introduces Simulink. I conclude a three-hour Simulink lab with a one-hour lecture summarizing basic concepts and approaches of the Simulink modeling environment and providing a guide for further study.

No doubt, it would be impossible to fully describe all the functions of MATLAB in one book. Moreover, because MathWorks has followed a twice-yearly general release schedule since 2002 (with an "a" release in the March timeframe and "b" in the September timeframe), in order to introduce new functions, features, concepts, and toolboxes, such a book would only have a 6-month life time. However, being a proficient applied programmer means knowing the basics and being familiar with a variety of available resources so that, if needed, knowledge in certain areas can be built up fast and new topics can be quickly incorporated. That is exactly what this book intends to provide.

The book is based on the R2020a release of MATLAB, but for those readers using previous releases, Table 0.1 provides highlights of the releases that followed the first edition of this book (omitting the changes in toolboxes and blocksets). The three major updates, R2012b, R2014b and R2019b (in bold in Table 0.1), which introduced the new MATLAB Desktop, new graphics system, and new Simulink Editor with Toolstrip, caused a total rewrite of Chapters 2, 6, 15 and Appendix F. The description of some of the new data types, specifically the tables, were added to Chapter 4, but the remaining new data types are introduced in Appendix C in order to avoid overloading the main body of the book. To eliminate confusion when talking about the new concepts and functions introduced lately, the specific release with which they became available is clearly specified throughout the book.

Table 0.1 Highlights of the Latest Releases of MATLAB

Release	Highlights
R2012b	New MATLAB Desktop features (Toolstrip interface that replaced menus and toolbars, Apps gallery presenting apps from the MATLAB product family, redesigned Help system), command line suggestions, new Simulink Editor, smart signal routing and simulation tools
R2013b	New types of data (`table` data container, `categorical` array)
R2014a	New way to display Command History window (pop-up window rather than static window)
R2014b	New graphics system, new types of data (`datetime`, `duration`, `calendarDuration`), suggested corrections for syntax errors in the Command Window, packaging and sharing tool, big data analysis tools (`datastore` and others)
R2016a	Live Editor to create and run live scripts with embedded output, App Designer

Release	Highlights
R2016b	New types of data (`timetable` data container, `timeseries` objects, `string` array), working with missing data (`fillmissing` and others) and big data (`tall`)
R2017a	MATLAB Online to use MATLAB through the web browser, working with outliers (`filloutliers` and others)
R2017b	MATLAB Drive providing a common, free cloud-based storage of 250 Mb, plotting in geographic coordinates, `wordcloud` function, contextual hints for function arguments in Live Editor
R2018a	Improved graphics (`axes`, `legend`)
R2018b	New plotting functions (`xline`, `yline`, `stackedplot`, `geoaxis`, `scatterhistogram`, and others), axes toolbar, Deep Learning Toolbox
R2019a	New tabular data reading functions (`readmatrix` and others), `parallelplot`, graphics export, object detection using you-only-look-once (YOLO) v2 detectors, Reinforcement Learning Toolbox
R2019b	Simulink Toolstrip and other tools, Git integration with MATLAB, map-based data visualization, Live Editor Tasks, function argument validation, Navigation, Robotics System and Robot Operating System (ROS) Toolboxes

To conclude, I would like to thank many of my students for all the why's and how's that allowed me to shape this book. I am also grateful to my sponsors, who challenge me with complex, real-world problems that require me to constantly perfect my programming skills. Finally and most importantly, I would like to thank my family for continuous encouragement and support.

Oleg A. Yakimenko
January 2020

CONVENTIONS

To make it easier for you to follow the content, this book utilizes the following conventions.

Variables in symbolic mathematical expressions appear in *italics,* for example, $y = \pi x - 1$. However, if the symbol represents a vector or matrix, rather than a scalar variable, **boldface type** is used, for example, $\mathbf{Ax} = \mathbf{b}$. Sometimes, such an expression may use subscripts to explicitly define the size of variables (number of rows and columns), for example, $\mathbf{A}_{n\times m}\mathbf{x}_{m\times 1} = \mathbf{b}_{n\times 1}$.

The newly introduced terms will appear in *italics,* whereas **boldface type** is used to represent the name of an icon (button) menu or item that appears in a pop-up menu (for example, **Run** button or **Show Code** option). The names of the keys on a computer keyboard are enclosed in less than and greater than signs, for example, < Enter > .

The `typewriter font` represents the names of MATLAB functions and commands, that is, any text you type in the Command Window, and any MATLAB response that appears on the screen. To distinguish the actual command you enter from a MATLAB response, the command appears in **typewriter bold font.** For example, the following represents the command you enter (starting with a screen prompt >>) and the response you get:

```
>> z=6*5/2+1
z = 16
```

(It is assumed that you press the <Enter> key after you type a command, so this action is not shown with a separate symbol.) The larger fragments of MATLAB programs, scripts and functions, appear in `typewriter font` as well. Usually, they result in a plot and/or the output is suppressed anyway, so there is no need to use a bold font to distinguish them. For example,

```
A=[1 1]; b=[2];
x1=0:5; x2=(b-A(1)*x1)/A(2);  % define the equation
plot(x1,x2), hold
xs=A\b;                       % find a particular solution
plot(xs(1),xs(2),'rp')
xlabel('x_1'), ylabel('x_2'), axis equal
```

```
s=rref([A b]); xh(2)=1; xh(1)=-s(2)*xh(2); xp(2)=0; xp(1)=s(3);
% Display the set of LAEs, general and particular solutions
syms r, x=xp'+r*xh'; H=latex(x);
text(3.5,1,['$$' H '$$'],'interpreter','latex','fontsize',14)
text(xs(1)+0.3,xs(2),'Particular solution provided by A\\b')
```

To highlight different constructions of code, MATLAB uses different colors. By default, green is used for the comments (text following the percent sign), and magenta—for the character and string arrays text strings (text embraced by single quotation marks and double quotes, respectively). In order to maintain this pattern even with a limited capability, this textbook employs gray and light blue colors for these two constructions (as seen in the above fragment).

The names of the files and directories appear in *italic Courier,* for example, *myfile.m* or *. . .\MATLAB\R2020a\rtw).*

Finally, it should be noted that while the MATLAB development environment makes a very good use of deferent colors – to highlight different features, panels and icons, this book utilizes a simplified graphics based on a few shades of gray and blue colors. Similarly, while the MATLAB graphical output allows millions and millions of colors, most of the figures in this textbook are simplified to the aforementioned shades of gray and blue. Occasionally however, when it is critical, the figures are shown in true colors.

LIST OF ACRONYMS

2-D	two-dimensional
3-D	three-dimensional
5G	5th generation
A2C	advantage actor critic
ADAS	Advanced Driver Assistance Systems
AMI	algorithmic modeling interface
ANSI	American National Standards Institute
ASIC	application-specific integrated circuits
AUTOSAR	AUTomotive Open System Architecture
BC	boundary condition
BDF	backward differentiation formula
BVP	boundary-value problem
CAN	controller-area network
CG	center of gravity
COM	Microsoft component object model
CPU	central processing unit
CTF	component technology file
CTFS	continuous time Fourier series
DA	data access
DATCOM	U.S. Air Force Digital Data Compendium
DDE	differential equation with delay
DDPG	deep deterministic policy gradients
DFT	discrete Fourier transform
DoF	degree of freedom
DPI	dots per inch
DQN	data quality network
DSP	digital signal processing
FFT	fast Fourier transform
FPGA	field-programmable gate array
FPU	floating point unit
FSAL	first step as last step (principle)
GNC	guidance, navigation and control
GPS	Global Positioning System

GPU	graphics processing unit
GUI	graphical user interface
GUIDE	graphical user interface development environment
HDA	historical data access
HDL	hardware description language
HOT	higher-order term
HSB	hue-saturation-brightness
HSL	hue-saturation-lightness
HSV	hue-saturation-value
HTML	hypertext markup language
IBIS	International Benchmarking of the Information Society
IC	initial condition, or integrated circuit
IEEE	Institute of Electrical and Electronics Engineers
INS	inertial navigation system
IoT	Internet of Things
IQI	inverse quadratic interpolation
IVP	initial-value problem
LAE	linear algebraic equation
LMI	linear matrix inequality
LSB	least significant bit
LTE	long-term evolution
LU	lower-upper
MATLAB	matrix laboratory
MBD	model-based design
MCR	MATLAB Compiler Runtime
MEX	MATLAB external interface
MSB	most significant bit
NDF	numerical differentiation formula
ODE	ordinary differential equation
OPC	open packaging conventions
OLE	object linking and embedding
PC	personal computer
PDE	partial differential equation
PDF	Portable Document Format
PID	proportional-integral-derivative (controller)
RF	radio-frequency
RGB	red-green-blue
RK	Runge-Kutta (method)
ROS	robot operating system
SerDes	serializer/deserializer
SLAM	simultaneous localization and mapping
SMT	Symbolic Math Toolbox
SoC	systems on a chip

SSL	Secure Sockets Layer
SV	singular value
SVD	singular value decomposition
UI	user interface
URL	Uniform Resource Locator
VHDL	Very High Speed Integrated Circuit HDL
VPA	variable-precision arithmetic
VRML	virtual reality modeling language
WLAN	wireless local area network
WGS84	1984 World Geodetic System
XML	extensible markup language

Chapter 1 MATLAB/Simulink as a Technical Computing Language

- History of MATLAB Development
- Overview of MATLAB Family
- Overview of Simulink Family
- Student Versions of MATLAB and Simulink

MATLAB Toolboxes, Simulink Blocksets

1.1 Introduction

Originally developed as a MATrix LABoratory over three decades ago, today MATLAB integrates computation, visualization, and programming in a powerful full-featured, self-contained, easy-to-use, technical computing environment where problems and solutions are expressed in familiar mathematical notation. Typical uses for MATLAB include math and computation; algorithm development; modeling, simulation, and prototyping; data analysis, exploration, and visualization; scientific and engineering graphics; graphical user interface (GUI) building; application development and deployment. The continuous enhancement of MATLAB makes it an ideal vehicle for exploring problems in all disciplines that manipulate mathematical content. This ability is multiplied by application-specific solutions using toolboxes, application development tools, and other products. The aim of this chapter is to provide an overview and interrelation of MathWorks' products and, by exploring the history of MATLAB development and its enhanced capabilities, present the MATLAB/Simulink product as a perfect tool for both introductory and enhanced programming involving engineering computations.

1.2 History of MATLAB Creation and Development

Officially, The MathWorks, Inc., was founded on 7 December 1984 in California by Jack Little, Cleve Moler, and Steve Bangert, but the basic concepts behind MATLAB were invented much earlier.

According to Moler, who maintains Cleve's Corner on the company's website, *www.mathworks.com* (Fig. 1.1), which was one of the first 75 registered commercial websites, MATLAB's history started in the 1960s when he wrote one of his first codes for solving the system of linear algebraic equations. The code was written in FORTRAN and used punched cards. When working on finite-difference methods in 1967, Forsythe and Moler published a textbook containing working code in ALGOL, FORTRAN, and PL/I that became an iconic book for many programmers. Later on, these algorithms, including those for matrix eigenvalue computation, were translated into FORTRAN to produce EISPACK, a collection of FORTRAN subroutines that compute the eigenvalues and eigenvectors for nine classes of matrices: complex general, complex Hermitian, real general, real symmetric, real symmetric banded, real symmetric tridiagonal, special real tridiagonal, generalized real, and generalized real symmetric matrices. In addition, it included two routines that used a singular value decomposition to solve certain least-squares problems. This was followed by LINPACK, a package of FORTRAN subroutines that analyzed and solved linear equations and linear least-squares problems. This package was specifically designed for supercomputers used in the 1970s and early 1980s. Later on, it was superseded by LAPACK, which was written in FORTRAN 90 and designed to run efficiently on shared-memory computers.

During the late 1970s, following the desire to allow users to use two packages, EISPACK and LINPACK, without writing FORTRAN programs, Moler used portions of these packages to develop the first version of

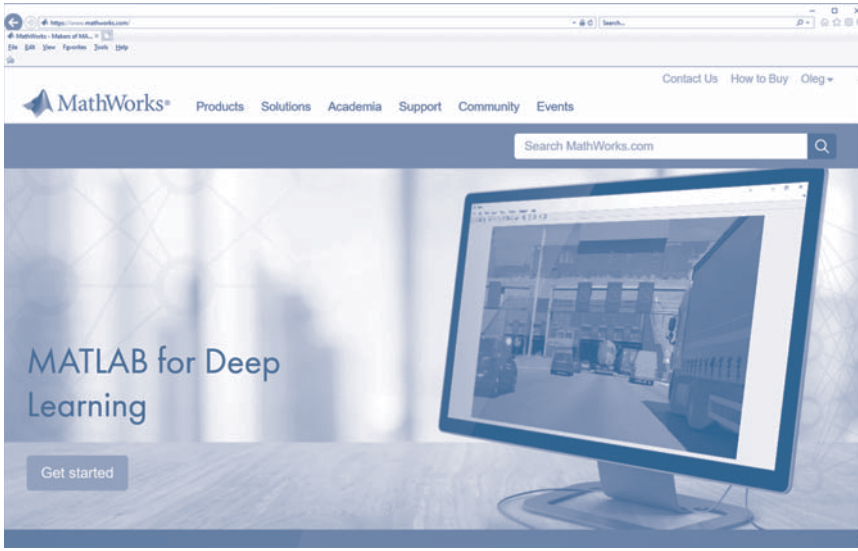


Fig. 1.1 Home page of MathWorks' website.

Table 1.1 Functions Available in the First Version of FORTRAN MATLAB

ABS	ANS	ATAN	BASE	CHAR	CHOL	CHOP	CLEA	COND	CONJ	COS
DET	DIAG	DIAR	DISP	EDIT	EIG	ELSE	END	EPS	EXEC	EXIT
EXP	EYE	FILE	FLOP	FLPS	FOR	FUN	HESS	HILB	IF	IMAG
INV	KRON	LINE	LOAD	LOG	LONG	LU	MACR	MAGI	NORM	ONES
ORTH	PINV	PLOT	POLY	PRIN	PROD	QR	RAND	RANK	RCON	RAT
REAL	RETU	RREF	ROOT	ROUN	SAVE	SCHU	SHOR	SEMI	SIN	SIZE
SQRT	STOP	SUM	SVD	TRIL	TRIU	USER	WHAT	WHIL	WHO	WHY

MATLAB. The only data type was "matrix," and the `HELP` command listed all of the available functions with their names abbreviated (Table 1.1). There were no *m*-files or toolboxes, and the plots were very primitive, made by printing asterisks on the teletypes and typewriters that served as terminals. If you wanted to add a function, you had to modify FORTRAN source code and recompile the entire program. Even so, all the functionalities of future MATLAB versions were present.

It is indicative that Stanford University students, who used this initial version, split into two groups. Those from math and computer science departments were not impressed because it was based on FORTRAN, not a particularly powerful programming language, and because it did not represent the current research work in numerical analysis (C and C++ on the one hand, and Lisp and PROLOG on the other hand, seemed to be much more innovative). However, engineering students liked the concept of MATLAB for its ability to handle matrices. Upon graduation, these students extended FORTRAN MATLAB to have more capability in control analysis and signal processing and, in the early 1980s, offered the resulting software as a commercial product.

At about the same time, in August 1981, IBM announced their first PC. Jack Little, the principal developer of the commercial FORTRAN MATLAB, suggested making MATLAB a matrix-based programming language, to which one could easily add new functions organized into toolboxes. So he and his colleague, Steve Bangert, reprogrammed MATLAB in C, wrote the parser/interpreter, and added *m*-files, the first few toolboxes, and more powerful graphics. This constituted the initial version of MATLAB, and some of the original code, proven by time, is still used today.

This version, MATLAB 1, implemented in C for MS-DOS PCs, was released by the newly established company, MathWorks, and presented at the 23rd IEEE Conference on Decision and Control in Las Vegas, Nevada, 12–14 December 1984.

Since then, MATLAB has been developed in parallel to PC. MATLAB 2, released in 1986, was enhanced with UNIX support. In 1990, the third version of MATLAB was enriched with Simulink, software for modeling, simulating, and analyzing dynamic systems. In 1992, 2-D and 3-D color graphics

were added to MATLAB 4. The MATLAB Student Edition was also released at that time. Since 1993, MATLAB has run in Microsoft Windows; since 1995, it has supported Linux. MATLAB 5 included a compiler, allowing conversion of MATLAB programs to C source code, featured data types, advanced visualization, a debugger, a profiler, and a GUI builder. MATLAB 6 featured a new desktop and provided Macintosh support. MATLAB 7 enabled single-precision and integer math, nested and anonymous functions, a plot editor, and interactive algorithm development. It also featured the Distributed Computing Toolbox (later renamed the Parallel Computing Toolbox).

Since 2006, MathWorks has been releasing two updates a year, in March (release a) and in September (release b), adjusting the development environment, improving the effectiveness and robustness of the existing code, piloting new concepts, and fixing bugs, including those reported by users of MATLAB and Simulink. You may learn about the new features and changes introduced in the latest versions of MATLAB at *www.mathworks.com/help/doc-archives.html*.

This textbook is based on the R2020a release of MATLAB/Simulink, but a vast majority of the functions and code samples presented in this book will run in the previous versions as well.

1.3 Capabilities and Resources

As discussed in the previous section, MATLAB was designed for engineers and is very easy to use and interpret. Along with the basic core of functions, MATLAB features a variety of special-use libraries and toolboxes enabling the most efficient usage of modern numerical methods, something that, say, FORTRAN or C++ lacks. In general, by design, MATLAB suffers few of the disadvantages of other programming languages. Its code is compact, yet it is learning intuitive. It avoids the standard separate compilation and link sequence by using an interpreter. To make problems work it requires only a few lines of code compared with traditional languages. Because of interaction with general purpose code instead of specific computation, MATLAB is slower in terms of computational speed, but the usage of modern, fast computers mitigates this issue. The open-source paradigm, unlike programming from the ground up, allows standard MATLAB pieces of code to be threaded together to translate into a new algorithm. The interactive features and built-in support of MATLAB are used to translate algorithms into functioning code in a fraction of time needed in other languages. Another important feature is code portability: Code written for the PC version of MATLAB does not have to be changed for Macintosh or UNIX systems (of course, this does not hold if your code relies on system-dependent resources).

With MATLAB as the foundation, MathWorks offers a variety of other products building on it. Figure 1.2 presents a general overview of these

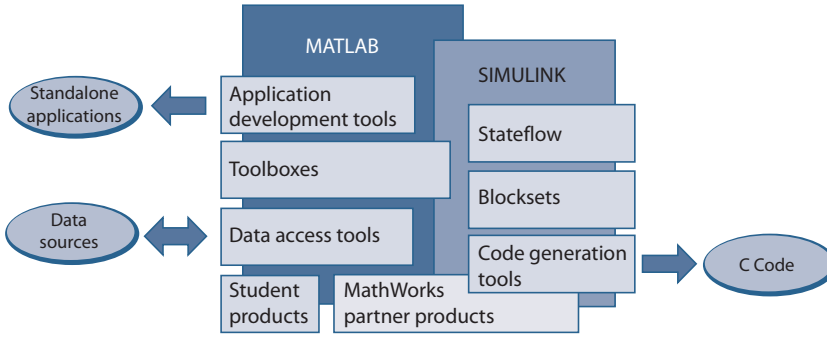


Fig. 1.2 One of the first depictions of MathWorks' products.

products, with a more detailed discussion in the following section. It is important that MATLAB allows you to convert its programs to C code. Reciprocally, it allows C, C++, and FORTRAN users to use their programs in MATLAB by compiling the so-called MATLAB external interface (MEX)-functions from the source code written in these languages. Also, MATLAB and Simulink allow you to get data from other software or external devices and compile standalone executable code.

To summarize, the family of MathWorks' MATLAB/Simulink products

- Presents a natural language for scientists and engineers all over the world
- Supports an open-source paradigm
- Features many toolboxes with the best numerical algorithms
- Has powerful visualization features to present the results of calculations and simulations
- Is a dynamic and constantly growing software
- Is well documented and supported [including a very good support via the Web (Fig. 1.1)]
- Allows you to use programs written in other languages
- Converts easily to C code, executable files, and standalone applications
- Supports Windows (32-bit and 64-bit), Mac OS X (Intel 64-bit), and Linux (64-bit) operating systems and the latest advances in computers (parallel, cluster, cloud, and grid computations)

For these reasons, MATLAB has become a standard tool for introductory and advanced courses in mathematics, engineering, and science in 6,500+ colleges and universities around the world. In industry, it is a tool of choice for research, development, and analysis. Presenting a piece of code in MATLAB is academically equivalent to presenting a formula or algorithm; for example, the function presented in Fig. 1.3 represents a complete algorithm for converting a latitude–longitude–height geodetic triad to the Earth-centered Earth-fixed coordinates. This particular code features one of the most powerful advantages of MATLAB—matrix computations,

```
function [X,Y,Z] = llh2xyz(lat,long,h)
% This function converts lat-long-height in WGS84 to ECEF X-Y-Z
%   lat and long should be given in decimal degrees
lat  = lat/180*pi;          % convert to radians
long = long/180*pi;         % convert to radians
a = 6378137;                % Earth's semimajor axis in meters
f = 1/298.257223563;        % reciprocal flattening
e2 = 2*f-f^2;               % eccentricity squared
N = a./sqrt(1-e2*sin(lat).^2);
X = (N+h).*cos(lat).*cos(long);
Y = (N+h).*cos(lat).*sin(long);
Z = (N*(1-e2)+h).*sin(lat);
```

Fig. 1.3 Example of MATLAB self-explanatory *m*-script.

allowing handling of multidimensional arrays of data like scalars. The input and output arguments of the function of Fig. 1.3 may be scalars, matrices, or arrays, and yet the computational code is as compact as it could possibly be (without involving the `for` loops as would be expected in other programming languages).

As of R2020a, more than 2,000 books on or based on MATLAB and Simulink are available for teachers, students, and professionals (the current list is available at *www.mathworks.com/support/books*). Among them are several books on numerical methods that cover the best techniques and algorithms using MATLAB code and application-specific manuscripts

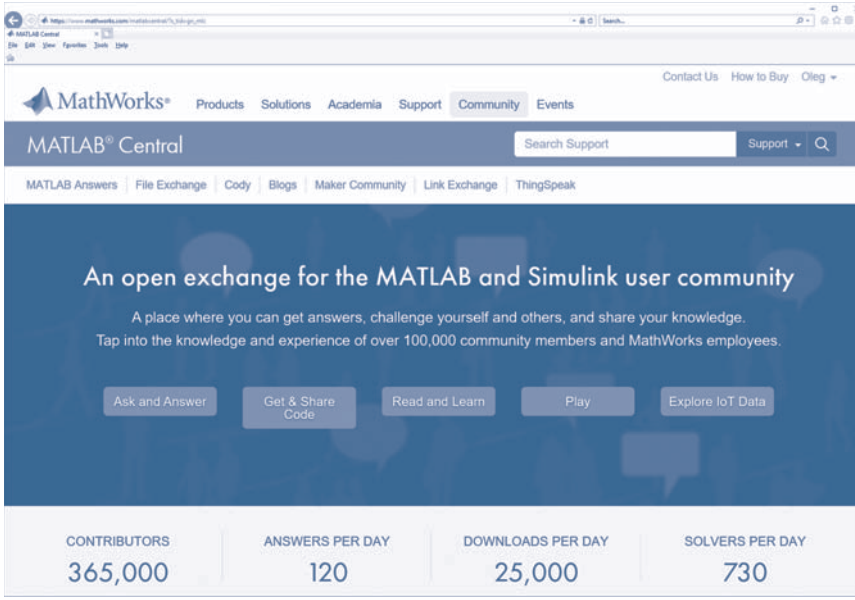


Fig. 1.4 MATLAB user community.

concentrating on a specific area, such as classical feedback control, image processing, and financial analysis. The MathWorks' website features a user community page, where users all over the world post and discuss their algorithms (Fig. 1.4) (over 235 thousand visits per day). To this end, MATLAB Central File Exchange (*www.mathworks.com/matlabcentral/fileexchange*) features over 36,000 files, so it is safe to say that, before writing a function, you may want to check if such a function has been developed already. If so, you may simply download it (thanks to the open-source paradigm) and modify it to serve your specific needs.

1.4 Aerospace Application Tools

The variety of toolboxes and blocksets available to extend MATLAB and Simulink (Fig. 1.2) will be addressed in the following section. Many of these tools can be and are widely used in the aerospace industry. However, MATLAB features one toolbox and one blockset that were specifically designed to be used by aerospace scientists and engineers. These tools are mentioned briefly here to emphasize attractiveness of the MathWorks product for specialists of this industry (more details can be found in Appendix E).

The Aerospace Toolbox extends the MATLAB computing environment by providing standards-based environmental models. For instance, it features the 1984 World Geodetic System (WGS84) representation of Earth's gravity, EGM96 Geopotential Model of Earth, World Magnetic Model, International Standard Atmosphere model, climatic data, and Spherical Harmonic Gravity models of Earth's Moon and Mars. You can also use it to import aerodynamic and control coefficients directly from the U.S. Air Force Digital Data Compendium (DATCOM) to carry out preliminary control design and vehicle performance assessment. The toolbox also provides an interface to the FlightGear Simulator (*www.flightgear.org*) to visualize flight data and vehicle dynamics in a 3-D environment and reconstruct behavioral anomalies in the flight-test results. To ensure design consistency, the Aerospace Toolbox software provides utilities for unit conversions, coordinate transformations (such as the ones presented in Fig. 1.3), and quaternion math. Finally, it allows you to compute a variety of gas dynamics functions, including isentropic flow, normal shock, Rayleigh flow, Fanno flow, and Prandtl–Meyer flow.

The Aerospace Blockset expands Simulink with blocks for modeling and simulating three- and six-degree-of-freedom dynamics of manned and unmanned aircraft, spacecraft, and rocket-craft, including simulation of their propulsion systems, control systems, mass properties, and actuators. It also includes standards-based environmental models for the atmosphere, gravity, wind, geoid height, and magnetic field based on the corresponding functions of the Aerospace Toolbox. In a similar manner, the Aerospace Blockset allows you to use Aerospace Toolbox utilities for converting units, transforming coordinates and spatial representations, and performing

common aerospace math operations. Once you have developed the model of your specific vehicle and tuned its guidance, navigation, and control algorithms, you can automatically generate code for production deployment and real-time execution in rapid prototyping and hardware-in-the-loop systems using the tools briefly mentioned in Sec. 1.5.2.

1.5 Overview of MathWorks Products

The list of more than 100 available MathWorks products is provided at *www.mathworks.com/products.html*. They are subdivided into the MATLAB, Simulink, and Polyspace product families and applications products. Within these product families, they are grouped into several similar-functionality categories or application domains. The further extension of Fig. 1.2, Fig. 1.5, features one of the recent attempts to represent the aforementioned variety of products in an intelligent way. Even though the number of categories has grown and keeps growing (e.g., 13 new tools were added in the two 2019 releases), Fig. 1.5 gives a sense of the interdependency of different products, so that the very same tool may belong to several different groups. Formally, though, each product is placed in just one of the groups. With time, some of these products migrate from one group to another. For example, the Aerospace Toolbox and Aerospace Blockset, introduced in the previous section, used to belong to the Control Systems group of

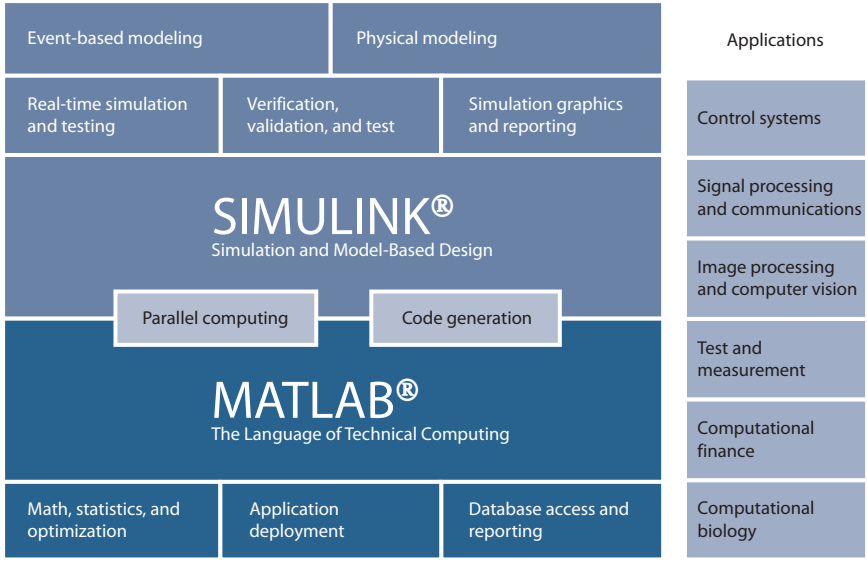


Fig. 1.5 Product families of MathWorks.

products portrayed in the top-right corner of Fig. 1.5, but they now form a separate Aerospace application domain. The Control Systems application domain includes the Control System Toolbox, one of the toolboxes featured by the very first version of MATLAB, and other tools.

Although this book basically describes the two major products of MathWorks, MATLAB and Simulink, for the sake of completeness, the following section provides a brief overview of other products available in the R2020a release.

1.5.1 MATLAB Product Family

Being the kernel of all product families of MathWorks, MATLAB is an intuitive programming language and a user-friendly technical computing environment. MATLAB is easy to use and learn, and it provides core mathematics and advanced graphical tools for data analysis, visualization, and algorithm and application development. With more than 1,000 mathematical, statistical, and engineering functions, satisfying a majority of engineers' and scientists' basic necessities, there might be no need to go beyond the MATLAB environment.

Toolboxes, collections of the highly optimized, application-specific functions, extend MATLAB functionality farther. MATLAB toolboxes support applications involving signal and image processing, control system design, optimization, financial engineering, symbolic math, neural networks, and others, as shown in Fig. 1.5. In addition to the basis functions of MATLAB, the toolbox functions are built in the MATLAB language and can easily be viewed, copied and modified (if needed), as the basic MATLAB functions. Tables 1.2–1.11 list available toolboxes by groups/domains.

Table 1.2 Math and Optimization Toolboxes

Toolbox	Brief Description
Optimization Toolbox	To solve standard and large-scale optimization problems using linear programming, mixed-integer linear programming, quadratic programming, nonlinear optimization, and nonlinear least squares
Global Optimization Toolbox	To find global solutions to optimization problems that contain multiple maxima or minima using global search, multistart, pattern search, genetic algorithm, and simulated annealing solvers
Curve Fitting Toolbox	To explore different options of fitting curves and surfaces to data
Symbolic Math Toolbox	To perform computations using symbolic mathematics and variable-precision arithmetic
Mapping Toolbox	To analyze and visualize geographic information
Partial Differential Equation Toolbox	To solve and analyze partial differential equations

Table 1.3 Artificial Intelligence, Data Science and Statistics Toolboxes

Toolbox	Brief Description
Statistics and Machine Learning Toolbox	To describe, analyze, and model data using statistics and machine learning
Deep Learning Toolbox	To model complex nonlinear systems that are not easily modeled with a closed-form equation using neural networks, including their design, training, visualization, and simulation
Text Analytics Toolbox	To import, process, analyze, visualize, and model text data
Predictive Maintenance Toolbox	To label data, design condition indicators, and estimate the remaining useful life of a machine
Reinforcement Learning Toolbox	To design and train policies using reinforcement learning algorithms including data quality network (DQN), advantage actor critic (A2C), and deep deterministic policy gradients (DDPG). These policies can be used in controllers and decision-making algorithms for complex systems such as robots and autonomous systems

Table 1.4 Control System Toolboxes

Toolbox	Brief Description
Control System Toolbox	To design and analyze controllers for dynamic closed-loop systems
System Identification Toolbox	To create linear dynamic models from measured input–output data
Fuzzy Logic Toolbox	To design and simulate fuzzy logic systems
Robust Control Toolbox	To design robust multivariable feedback control systems [includes linear matrix inequality (LMI) Control Toolbox and μ-Analysis and Synthesis Toolbox featured in the earlier versions]
Model Predictive Control Toolbox	To control large, multivariable processes in the presence of constraints

Table 1.5 Signal Processing Toolboxes

Toolbox	Brief Description
Signal Processing Toolbox	To generate, measure, transform, filter, and visualize signals
Phased Array System Toolbox	To design, simulate, and analyze sensor array systems in radar, sonar, wireless communications, and medical imaging applications
DSP System Toolbox	To create and test digital signal processing (DSP) algorithms for audio, communications, medical, and other real-time signal processing and Internet of Things (IoT) applications
Audio Toolbox	To design, simulate, and prototype audio processing systems
Wavelet Toolbox	To analyze, synthesize, denoise, and compress signals and images using wavelet-based algorithms

Table 1.6 Wireless Communications, Radio Frequency, and Mixed-Signal Toolboxes

Toolbox	Brief Description
Communications System Toolbox	To design, analyze, and verify communications algorithms
LTE Toolbox	To design, simulate, and verify long-term evolution (LTE) and LTE-advanced communications systems
WLAN Toolbox	To design, simulate, analyze, and test wireless local area network (WLAN) communications systems
5G Toolbox	To simulate, analyze, and test the physical layer of 5th generation (5G) communications systems
RF Toolbox	To design, model, and analyze networks of radio-frequency (RF) components
Antenna Toolbox	To design, analyze, and visualize antenna elements and antenna arrays
SerDes Toolbox	To design and verify serializer/deserializer (SerDes) systems and generate International Benchmarking of the Information Society algorithmic modeling interface (IBIS-AMI) models for high-speed digital interconnects

Table 1.7 Autonomous Systems and Aerospace Toolboxes

Toolbox	Brief Description
Robotics System Toolbox	To develop autonomous mobile robotics applications including map representation, path planning, and path following for differential drive robots, enabling an interface to the robot operating system (ROS and ROS 2)
Automated Driving Toolbox	To design and test Advanced Driver Assistance Systems (ADAS) and autonomous driving systems
Sensor Fusion and Tracking Toolbox	To design, simulate, and analyze systems that fuse data from multiple sensors to maintain position, orientation, and situational awareness
Aerospace Toolbox	To provide aerospace reference standards and environmental models and to allow importing aerodynamic coefficients
Navigation Toolbox	To calibrate and simulate motion and range sensors
ROS Toolbox	To design and analyze navigation systems, including sensor modeling/calibration, 2-D and 3-D simultaneous localization and mapping (SLAM), path planning, and path following

Table 1.8 Image Processing and Computer Vision Toolboxes

Toolbox	Brief Description
Image Processing Toolbox	To perform image processing, analysis, and algorithm development
Computer Vision Toolbox	To design and simulate computer vision and video-processing algorithms including object detection and tracking, feature detection and extraction, feature matching, stereo vision, camera calibration, and motion detection

Table 1.9 Test and Measurement Toolboxes

Toolbox	Brief Description
Data Acquisition Toolbox	To acquire and send out data from the plug-in data acquisition boards
Instrument Control Toolbox	To control and communicate with test and measurement instruments
Image Acquisition Toolbox	To enable connecting industrial and scientific cameras to MATLAB and Simulink
OPC Toolbox	To read, write, and log live data and historical Object Linking and Embedding for Process Control (OPC) data from OPC data access (DA) and OPC historical data access (HDA) servers
Vehicle Network Toolbox	To send and receive controller-area network (CAN) packets directly from MATLAB or Simulink, allowing you to encode, decode, and filter them
Model-Based Calibration Toolbox	To calibrate complex powertrain systems using statistical modeling and numeric optimization
ThingSpeak	To enable aggregating, visualizing, and analyzing the live data streams in the cloud

Table 1.10 Computational Finance Toolboxes

Toolbox	Brief Description
Financial Toolbox	To analyze financial data and develop financial algorithms
Econometrics Toolbox	To select, calibrate, and analyze economic models for use in simulation and forecasting
Datafeed Toolbox	To acquire financial data from data service providers
Database Toolbox	To work with relational and nonrelational databases
Financial Instruments Toolbox	To perform cash-flow modeling and yield curve fitting analyses, compute prices and sensitivities, view price evolutions, and perform hedging analyses using common equity and fixed-income modeling method
Trading Toolbox	To integrate streaming and event-based data into MATLAB, enabling development of financial trading strategies and algorithms that analyze and react to the market in real time
Risk Management Toolbox	To model and simulate the credit and market risks

Table 1.11 Computational Biology Toolboxes

Toolbox	Brief Description
Bioinformatics Toolbox	To read, analyze, and visualize genomic, proteomic, and microarray data
SimBiology	To model, simulate, and analyze biochemical pathways

The two toolboxes in Table 1.12 allow getting your data into MATLAB for analysis and usage in the models and integrating reporting capabilities into MATLAB applications.

Table 1.12 Database Access and Reporting Toolboxes

Toolbox	Brief Description
Database Toolbox	To exchange data with relational and nonrelational databases
MATLAB Report Generator	To develop programs that generate reports in Portable Document Format (PDF), Microsoft Word, Microsoft PowerPoint, and HyperText Markup Language (HTML)

To fully exploit modern computers architecture, MathWorks provides two toolboxes shown in Table 1.13.

Once your code is developed, you might need to either generate readable and portable C/C++ code from your MATLAB algorithms or create a standalone application. The MathWorks' code-generation tools (Table 1.14) and application deployment tools (Table 1.15) will assist you in accomplishing these tasks.

Table 1.13 Parallel and Distributed Computing Toolboxes

Toolbox	Brief Description
Parallel Computing Toolbox	To allow performing parallel computations on multicore processors, graphics processing units, and computer clusters
MATLAB Distributed Computing Server	To enable computationally intensive MATLAB and Simulink computations on computer clusters, clouds, and grids, scaling up the parallel computing process developed with the Parallel Computing Toolbox to many computers

Table 1.14 Code-Generation and Verification Toolboxes

Toolbox	Brief Description
MATLAB Coder	To generate standalone C and C++ code from MATLAB code
GPU Coder	To generate optimized CUDA code for NVIDIA graphics processing units (GPUs) from MATLAB code for deep learning, embedded vision, and autonomous systems
HDL Coder	To generate portable, synthesizable Verilog and Very High Speed Integrated Circuit Hardware Description Language (VHDL) code from the MATLAB functions, Simulink models, and Stateflow charts
HDL Verifier	To automate Verilog and VHDL design verification using HDL simulators and field-programmable gate array (FPGA) hardware-in-the-loop testbenches
Vision HDL Toolbox	To design image processing, video, and computer vision systems for FPGA and application-specific integrated circuits (ASIC) using an architecture appropriate for HDL implementations
Filter Design HDL Coder	To generate efficient, synthesizable, and portable VHDL and Verilog code for fixed-point filters that are designed with DSP System Toolbox software, for implementation in FPGAs or ASICs
Fixed-Point Designer	To develop fixed-point algorithms in MATLAB code, Simulink models, and Stateflow charts or convert floating-point algorithms to fixed point

Table 1.15 Application Deployment Toolboxes

Toolbox	Brief Description
MATLAB Compiler	To convert MATLAB code to standalone applications (executables or software components) and create Microsoft Excel add-ins
MATLAB Compiler SDK	To extend the functionality of MATLAB Compiler and build C/C++ shared libraries, Microsoft .NET assemblies, and Java classes from MATLAB programs
MATLAB Production Server	To incorporate custom analytics in enterprise applications running on MATLAB Production Server using a lightweight client library protected with Secure Sockets Layer (SSL) encryption
Spreadsheet Link	To connect Excel spreadsheet software with the MATLAB workspace and exchange data between MATLAB and Excel

Table 1.16 MATLAB Coder versus MATLAB Compiler as Code Deployment Solution

Specification	MATLAB Coder	MATLAB Compiler
Output	Portable and readable C/C++ source code	Executable file or software component/library
MATLAB language support	Subset	Full
Additional libraries	None	MCR
Supported toolboxes	Some	Most

The reason for developing C/C++ code is that you might want to integrate MATLAB algorithms with existing C environments (using source code or static libraries), prototype MATLAB algorithms on desktops as standalone executables, accelerate MATLAB algorithms (using binary *mex*-files, dynamically linked subroutines that the MATLAB interpreter loads and executes), implement C/C++ code on processors, or simply hand it off to software engineers. Application deployment tools allow building standalone executables and software components from MATLAB code and sharing these standalone versions of your MATLAB applications with colleagues who use other development environments. Table 1.16 shows the key differences between two aforementioned deployment solutions. In this table, MATLAB Compiler Runtime (MCR) is a free standalone set of shared libraries that enables the execution of compiled MATLAB applications or components on computers that do not have MATLAB installed. Appendix F shows an example of using the MATLAB Compiler to convert a piece of code developed throughout this book into a standalone application.

1.5.2 Simulink Product Family

MathWorks' *Simulink*, introduced in the third version of MATLAB, is a simulation and prototyping environment that provides a block diagram interface built on the core MATLAB numeric, graphics, and programming functionality. Using a drag-and-drop paradigm and an extensive block

Table 1.17 Event-Based Modeling Blocksets

Blockset	Brief Description
Stateflow	To model and simulate combinatorial and sequential decision logic based on state machines and flow charts
SimEvents	To design event-driven processes and distributed control systems, hardware architectures, and sensor and communication networks for aerospace, automotive, and electronics applications

library, you can build a graphical block diagram of any dynamic system. Once the system is diagrammed, you can simulate and analyze its behavior.

Blocksets are collections of application-specific blocks that support multiple design areas, including electrical power-system modeling, digital signal processing, fixed-point algorithm development, and some others, as shown in Fig. 1.5 and Tables 1.17–1.21. These blocks can be incorporated directly into your Simulink models. (Note that some of the toolboxes introduced in the previous section may include Simulink blocks in addition to MATLAB scripts.)

These days, engineering of complex control, signal-processing, and communications systems with industrial, aerospace, and automotive applications relies on model-based design (MBD), which provides an efficient approach

Table 1.18 Physical Modeling Blocksets

Blockset	Brief Description
Simscape	To model and simulate multidomain systems, spanning mechanical, electrical, hydraulic, and other domains
Simscape Multibody	To model and simulate multibody mechanical systems, such as robots, vehicle suspensions, construction equipment, and aircraft landing gear
Simscape Driveline	To model and simulate mechanical driveline systems
Simscape Fluids	To model and simulate fluid systems, such as hydraulic pumps, valves, actuators, pipelines, and heat exchanger systems
Simscape Electronics	To model and simulate electronic and electromechanical systems
Simscape Power Systems	To model and simulate electrical power systems

Table 1.19 Control Systems Blocksets

Blockset	Brief Description
Simulink Control Design	To design and analyze control systems in Simulink
Simulink Design Optimization	To improve designs by estimating and tuning model parameters using numerical optimization
Aerospace Blockset	To model and simulate aircraft, spacecraft, and propulsion systems
Powertrain Blockset	To model automotive powertrains, including gasoline, diesel, hybrid, and electric systems
Vehicle Dynamics Blockset	To simulate driving maneuvers in a 3-D environment

Table 1.20 Various Systems Design Blocksets

Blockset	Brief Description
RF Blockset	To design RF communications and radar systems
Mixed-Signal Blockset	To design, simulate and verify mixed-signal integrated circuits (ICs)
SoC Blockset	To design, model, evaluate, and implement hardware and software architectures for ASICs, FPGAs, and systems on a chip (SoC)

Table 1.21 Simulation Graphics and Reporting Blocksets

Blockset	Brief Description
Simulink 3-D Animation	To visualize and animate Simulink models in a 3-D virtual environment using Virtual Reality Modeling Language (VRML)
Simulink Report Generator	To automatically create Microsoft Word, HTML, and PDF reports for Simulink models and simulations, including model-based design system design descriptions and generated code, requirements traceability, and testing reports

to establishing a common framework for communication throughout the design process while supporting the development cycle (Fig. 1.6). Table 1.22 presents a couple of systems engineering tools that allow managing

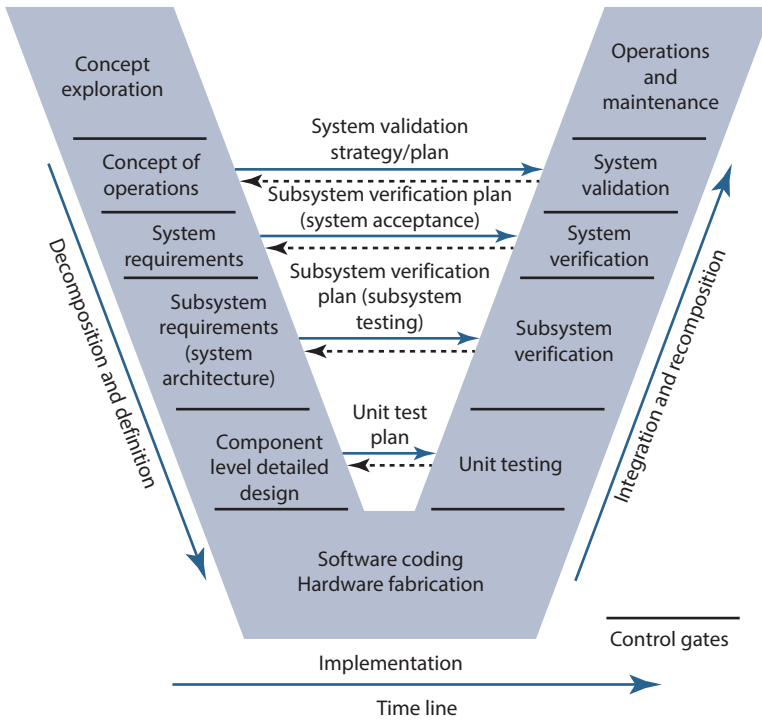


Fig. 1.6 The V-model of the systems engineering process.

Table 1.22 Systems Engineering Blocksets

Blockset	Brief Description
Simulink Requirements	To author, analyze, and manage rich text requirements with custom attributes within Simulink
System Composer	To define, design, analyze, and specify architectures and compositions for model-based systems engineering and software design

requirements and designing system architecture. Moreover, by design, Simulink is very well suited to support MBD by defining the models with advanced functional characteristics using continuous-time and discrete-time building blocks. From these easy-to-build models used with simulation tools, you can generate code to prototype, test in real time, and deploy it on your specific embedded system. To support this rapid prototyping paradigm, MathWorks offers the code-generation and real-time simulation and testing tools presented in Tables 1.23 and 1.24, respectively. Verification, validation, and testing tools (Table 1.25) allow simulating, testing, and analyzing models to verify designs and validate requirements.

In addition to the aforementioned toolboxes and blocksets, more than 500 complementary third-party hardware and software products are compatible with MATLAB/Simulink. These partner offerings include toolboxes, blocksets, and real-time workshop targets. MathWorks' website keeps track of

Table 1.23 Simulink Code-Generation Tools

Tool	Brief Description
Simulink Coder	To generate and execute C and C++ code from Simulink diagrams, Stateflow charts, and MATLAB functions (formerly Real-Time Workshop)
Embedded Coder	To generate readable, compact, and fast C and C++ code optimized for use on embedded processors, on-target rapid prototyping boards, and microprocessors used in mass production
AUTOSAR Blockset	To develop classic and adaptive AUTOSAR software using Simulink models (AUTOSAR stands for AUTomotive Open System Architecture, a global development partnership consisting of automobile manufacturers, component suppliers, and other companies)
HDL Coder	See Table 1.14
Wireless HDL Toolbox	To design and implement LTE wireless communications subsystems on FPGAs and ASICs
Simulink PLC Coder	To generate hardware-independent structured text in PLCopen extensible markup language (XML) and other file formats
Fixed-Point Designer	See Table 1.14
DO Qualification Kit	To qualify Simulink and Polyspace software verification tools for projects based on certain software standards (DO-178 and DO-278)
IEC Certification Kit	To qualify code-generation and verification products and streamline certification of your embedded systems to specific functional safety standards (ISO 26262, IEC 61508, EN 50128, and IEC 62304)

Table 1.24 Real-Time Simulation and Testing Tools

Tool	Brief Description
Simulink Real-Time	To create real-time applications from Simulink models and run them on dedicated target computer hardware connected to your physical system
Simulink Desktop Real-Time	To create and tune a real-time system for rapid prototyping or hardware-in-the-loop simulation with your computer

them and allows you to find a specific product or application by industry, function, type, product, or company name (*www.mathworks.com/products/connections*).

1.5.3 MATLAB and Simulink Student Version

The last, but not least, powerful feature of MathWorks' product families is that it gives special attention to beginner programmers—college and university students—by providing them with the student versions of MATLAB and Simulink.

The two options are unbundled MATLAB Student (MATLAB only) and MATLAB and Simulink Student Suite. The latter includes full-featured versions of MATLAB and Simulink along with 10 of the most widely used add-on products:

Table 1.25 Verification, Validation, and Testing Blocksets

Blockset	Brief Description
Simulink Check	To enable industry-recognized checks and metrics that identify standard and guideline violations during development (including software development standards DO-178, ISO 26262, IEC 61508, and IEC 62304)
Simulink Coverage	To perform model and code coverage analysis measuring testing completeness in models and generated code
Simulink Design Verifier	To generate tests and prove model properties using formal methods
Simulink Test	To manage tests and analyze results for system verification and validation
Simulink Code Inspector	To systematically examine blocks, state diagrams, parameters, and settings in a generated model to determine whether they are structurally equivalent to operations, operators, and data in the generated code
HDL Verifier	See Table 1.14
Polyspace Bug Finder Polyspace Bug Finder Access Polyspace Bug Finder Server	To identify runtime errors, data flow problems, and other defects in C and C++ embedded software
Polyspace Code Prover Polyspace Code Prover Access Polyspace Code Prover Server	To prove the absence of overflow, divide-by-zero, and out-of-bounds array access and certain other runtime errors in C and C++ source code

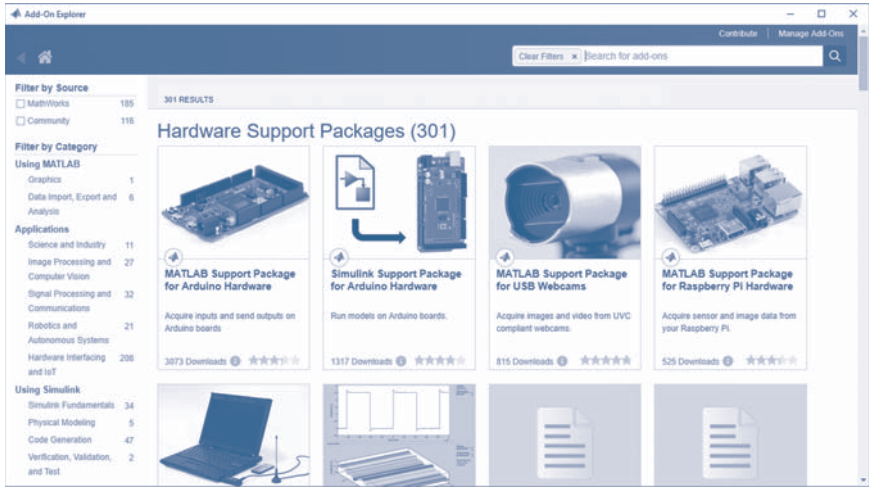


Fig. 1.7 Supported low-cost hardware.

- Control System Toolbox
- Curve Fitting Toolbox
- DSP System Toolbox
- Image Processing Toolbox
- Instrument Control Toolbox
- Optimization Toolbox
- Parallel Computing Toolbox
- Signal Processing Toolbox
- Statistics and Machine Learning Toolbox
- Symbolic Math Toolbox

The suite also includes built-in support for prototyping, testing, deploying, and running models on low-cost target hardware such as Arduino, Raspberry Pi, LEGO Mindstorms NXT, and EV3 (Fig. 1.7).

The low price of both options encourages students to install the programs on their own laptops, so that they can enjoy playing with MATLAB and Simulink and use them for a variety of engineering assignments (both options also allow installing any additional add-on products you might need at a nominal price). For example, you can use MATLAB to acquire accelerometer, magnetometer, orientation, and location data from built-in sensors on your Android device (via streaming sensor data over Wi-Fi to a computer on the same network running MATLAB); utilize Simulink to develop and deploy an app on your iPhone or iPad; or automatically build and deploy flight control algorithms to Parrot Minidrones and AR.Drone 2.0, as well as Pixhawk Autopilot. It is no wonder that MATLAB and Simulink software have become a premier technical computing resource in thousands of universities worldwide.

Chapter 2 Development Environment and Basic Operations

- Desktop Overview and Help System
- Basic Operations and Function Libraries
- Managing Work Session
- Problem-Solving Methodology

`plus`, `minus`, `mtimes`, `mpower`, `sin`, `cos`, `tan`, `atan`, `exp`, `log`, `sqrt`, `abs`, `round`, `fix`, `pi`, `i`, `Inf`, `NaN`, `format`, `disp`, `clear`, `close`, `clc`, and many others

2.1 Introduction

This chapter provides a "quick start" introduction to MATLAB as an interactive calculator. The chapter starts with presenting an overall description of the MATLAB development environment, including its Desktop, Toolstrip, multiple panels, basics of entering commands in the Command Window, and Editor. It then introduces the extended Help system, presents a variety of basic functions, and discusses the order of precedence of mathematical operations. It further presents more details on managing the work session and addresses the way to change many development environment preferences if needed. The chapter concludes with the commonly accepted methodology for proficient programming and problem solving.

2.2 MATLAB Development Environment and MATLAB Online

To start MATLAB, double-click the MATLAB icon . You will then see the MATLAB *Desktop* (Fig. 2.1) featuring a typical Microsoft Office–type multitab Toolstrip (which was a major change introduced in R2012b) with the *Quick Access* toolbar and *Help browser*, multiple detachable panels, *Command Window*, *Workspace*, and *Current Folder*.

Before we proceed with a detailed discussion of the MATLAB Desktop and all available tools, let us mention a very convenient option introduced in R2017a, *MATLAB Online*, which allows using the online version of MATLAB through any standard supported web browser

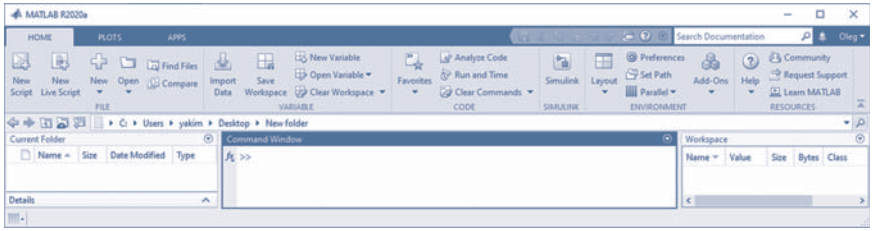


Fig. 2.1 Default desktop of MATLAB.

(Google Chrome, Microsoft Edge, Mozilla Firefox, Safari). Just go to *matlab.mathworks.com* and sign in with your MathWorks credentials. In addition, *MATLAB Drive*, introduced in R2017b, provides a common cloud-based storage location for your MATLAB files (up to 5 Gb, with the first 250 Mb free). As of R2019a, the MATLAB Desktop features the **Access MATLAB Drive** button, , the fourth button above the Current Folder panel, which enables MATLAB Drive on your desktop. All you have to do once MATLAB is installed on your computer is click this button to install the MATLAB Drive Connector, eliminating the need for manual uploading or downloading (all files on MATLAB Drive are synchronized with you computer automatically when connected to the Internet). (You can also share your MATLAB Drive folders with a view-only link or invite individual collaborators and set their editing permissions.)

2.2.1 Entering Instructions in the Command Window

As you can imagine, most of the time you will communicate with the MATLAB by the typing instructions of various types in the Command Window. As seen in Fig. 2.1, MATLAB displays the prompt (>>) to indicate that it is ready to receive instructions. MATLAB is command-line oriented (i.e., it uses an interpretative language), meaning that no compiling is required. Programming errors are flagged by error messages.

To see how simple it is to use MATLAB, try entering a few commands on your computer. Make sure the cursor is at the prompt in the Command Window and type in, for example, the following mathematical expression:

```
>> 3 * pi / 3.141592 \ 3 - 1 + 2 ^ 2
```

Press <Enter> to receive the response

```
ans = 4.0000
```

The expression you entered contains only *scalars*, that is, single elements of the default data type. They include rational numbers (i.e., integers like 2

and fractions like $-1/3$), along with irrational numbers, such as π and $\sqrt{2}$. However, one of the advantages of MATLAB is that it will handle arrays, groups of numbers, in exactly the same way, making code readable and very compact. For example, the following command involves a four-element row vector (enclosed by square brackets):

```
>> z = [1 2 3 4] * 5 / 2 - 1
```

returning

```
z = 1.5000 4.0000 6.5000 9.0000
```

More details on arrays and array operations will be provided in Chapter 3.

Note that in the first example we did not specify a *variable* name to which the result should have been assigned. Hence, MATLAB used the default name, `ans` (abbreviation for answer). In the second example, the variable was specified. When typing in expressions with the assignment operator `=`, MATLAB assigns the value generated on the right to the variable on the left. For example,

```
>> x = 3
x = 3
```

assigns the value of 3 to `x`. If followed by

```
>> x = x + 1
x = 4
```

it further assigns a new value of 4 to `x` (this new value is computed using the old value of `x`).

There is nothing wrong with the automatically created variable `ans`; you can use it for further calculations, similar to what we just did with `x`. For example, if you type

```
>> 50 - ans
```

you can observe that the previous value of `ans` (4) changes to 46 (50 − 4)

```
ans = 46.0000
```

However, you should keep in mind that `ans` is a temporary variable, containing the most recent value, which is overwritten every time you do not specify an output variable name explicitly. That is why using a specific variable name to keep the result of computation is preferable.

The valid variable name may contain 1–63 characters (defined by the `namelengthmax` function of MATLAB). It must start with a letter, but it may use letters, digits, and underscores after the first letter. The only expectation is that the variable name does not coincide with one of 20 keywords (type in `iskeyword` to see what these keywords are) or with the names of MATLAB functions and constants. Try to give your variables meaningful names, so that anyone who looks at your code understands what it does. Also,

be aware of the fact that MATLAB is case sensitive; therefore, `Rate`, `rate`, and `RATE` represent three different variables.

An advantage of specifying meaningful variable names and using them rather than their values in mathematical expressions is that you can reuse the same formula by simply changing the values of the variables it consists of. For instance, to compute the circumference of a circle with a radius of 10 units, it is wise to issue two meaningful statements like

```
>> R = 10; circumference = 2 * pi * R
circumference = 62.8319
```

as opposed to

```
>> y = 2 * pi * 10
y = 62.8319
```

because you can reuse the formula for `circumference` by simply assigning different values for `R` (this idea will be explored further in Sec. 5.8, introducing the MATLAB functions). Note that in the first case, both statements appeared on the same line, which is a valid syntax. All you have to do is to separate multiple statements by a comma or semicolon. The latter also suppresses the output (see Sec. 2.6). If you want to see the results of both statements, substitute the semicolon with a comma.

If you make a typing mistake, just press the <Enter> key, wait until you get the prompt, and then retype your command. Because MATLAB retains your previous keystrokes in a command file, you can use the up-arrow key <↑> to scroll back through the commands. Press the key once to see the previous entry, twice to see the entry before that, and so on. Use the down-arrow key <↓> to scroll forward through the commands. Scrolling back and forth enables you to correct typing mistakes quickly. When you find the line you want (it shows up at the command line), you can edit it using the left- and right-arrow keys (<←> and <→>), <Back Space> key, and <Delete> key. Press the <Enter> key to execute the corrected command.

All previous keystrokes are conveniently available in the *Command History* panel, which pops up once you use the up-arrow key <↑> (Fig. 2.2). (Note, the upper portion of the Command History panel of Fig. 2.2 will not show up exactly as shown—what is shown in Fig. 2.2 also features the ClearStart favorite created as discussed in Sec. 2.2.3.) This panel disappears when you hit the <Enter> key (otherwise, you can close it by hitting the <Esc> key).

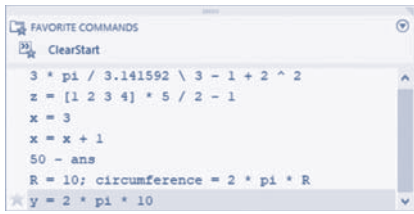


Fig. 2.2 A pop-up Command History panel.

For a function or variable name mistyped in the Command Window,

MATLAB may automatically suggest a correction (this feature was introduced in R2012b). If you agree with it, press <Enter> to run the suggested command (otherwise, press <Esc> to delete it). For example, typing

```
>> data
```

returns

```
Undefined function or variable 'data'.
Did you mean:
>> date
```

Hitting <Enter> produces the desired result:

```
ans = '24-Sep-2019'
```

(`date` is one of the MATLAB functions; it returns a character vector containing the current date).

Note that to improve readability of your code, you may use the space characters between operands (as we did in all preceding examples). When executing the commands, MATLAB simply ignores them. It would be a mistake, however, to break a constant name, such as `pi`, or number, such as `3.141592` (as in the very first example in this section). In this case, MATLAB would return an error message, specifically pointing at its source—for example,

```
>> 3 * p i / 3.141592 \ 3 - 1 + 2 ^ 2
  3 * p i / 3.141592 \ 3 - 1 + 2 ^ 2
       ↑
Error: Invalid expression. Check for missing multiplication operator,
missing or unbalanced delimiters, or other syntax error...
```

In the preceding examples, we introduced all arithmetic operations available in MATLAB. They are formally listed in Table 2.1. Note that, MATLAB has two division operators. A good way to remember the difference between them is to note that a slash always slants toward a denominator. For scalar operations, they are basically the same; hence, the right division operator is usually used. The left division operator is useful for solving sets of linear algebraic equations, as will be discussed in Chapter 9.

Now, let us type

```
>> pi * 1000
```

which, after hitting <Enter>, yields

```
ans = 3.1416e+003
```

Note that as in all previous examples, the displayed result has only four decimal places. Does that mean that it is the precision MATLAB can deliver? Of course not. As will be discussed in detail in Chapter 8, MATLAB uses so-called *double precision* for computations (meaning two

Table 2.1 Scalar Arithmetic Operations

Symbol	Operation	MATLAB Form	Result
^	Exponentiation	a^b	a^b
*	Multiplication	a*b	ab
/	Right division	a/b	a/b
\	Left division	a\b	b/a
+	Addition	a+b	a+b
–	Subtraction	a-b	a–b

four-byte words—eight bytes in total—are allocated for every variable). However, by default, it displays the results using the *short format*, rounding to just four decimal places. If it bothers you, do not worry; there are several ways to change it, and they will be considered in Sec. 2.6, as well as in Sec. 4.9. To fit the results into the short format, MATLAB uses exponentiation to a power of 10, using the notation e. Thus, the last result should be treated as 3.1415×10^3.

2.2.2 Desktop Overview

Now that you have done your first command-entering exercise, let us formally review the MATLAB Desktop (hereafter referred to as Desktop) as presented in Fig. 2.3 (which is what you will have after typing all the commands of the previous section).

By default, Desktop is composed of three docked panels. You already know that in the middle there is the Command Window, where you were typing all your commands. The variables you are creating are displayed in the Workspace panel to the right (which is a default layout). To the left, there is another panel, the Current Folder. Before R2012b, the Command History panel of Fig. 2.2 was the fourth panel docked to Desktop by default.

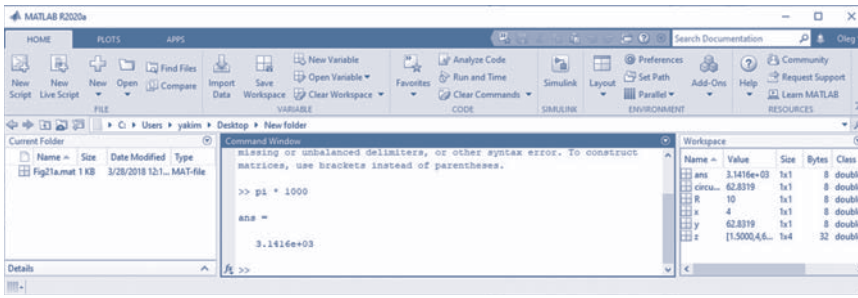


Fig. 2.3 Example of a working session in MATLAB.

You can navigate between Desktop panels by clicking them (the borders of the *active* panel become dark blue then). When you are trying to type in a command and do not see a response in the Command Window, you need to click this window first to make it active. You can also navigate between the panels by holding the <Ctrl> key on your keyboard and pressing the <0> key (for the Command Window), <1> (for the Command History panel), <2> (for the Current Folder panel), and <3> (for the Workspace panel). Alternatively, you can navigate between the panels programmatically by typing in commands—`commandwindow`, `commandhistory`, `filebrowser`, and `workspace`—to open the corresponding Desktop panel when it is closed or to make it active when it is open. The capability to achieve the same result by manipulating Desktop tools (navigating through Toolstrip icons and pop-up menus) and programmatically (issuing the commands in the Command Window) constitutes a very powerful and useful feature of the MATLAB development environment.

Clicking the small upside-down triangle located in the upper-right corner of any of the four panels, including the Command History panel in Fig. 2.2, or right-clicking anywhere on the panel name bar brings up a pop-up *action menu* that allows you to do some relevant manipulations within the corresponding panel (to be covered in more detail throughout this chapter). The pop-up action menus for all four Desktop panels are shown in Fig. 2.4 and feature the same bottom portion, allowing you to minimize, maximize, undock (from Desktop), and close the panel. Note that you can undock any panel by dragging it out as well. If the panel is undocked, then the **Undock** option in the action menu changes to **Dock** (as it is the case for the Command History panel action menu of Fig. 2.4d).

You can resize the MATLAB Desktop panels and move them around within the Desktop, even without undocking them, by simply pulling their edges or clicking their upper portion and dragging them to the desired location. Therefore, MATLAB allows you to adjust the development environment to meet your personal preferences. If you want to return to the default layout, you should select the **Default** option in the drop-down menu of the **Layout** icon (Fig. 2.5a) residing on the Home tab toolstrip (see Fig. 2.3).

The three tabs of the Desktop's Toolstrip—Home, Plots, and Apps—organize commonly used functionalities. New tabs appear when additional functionality is needed.

The six icons of the File group of the Home tab allow you to find, open, and compare different files; for example, clicking the **New** icon brings up a drop-down menu shown in Fig. 2.5b. The **New Script** and **New Live Script** icons of the Home tab open the Editor and Live Editor, respectively (to be considered in Secs. 5.10.2 and 5.10.3), and cause some additional tabs to appear (to be discussed in Sec. 2.2.3).

Fig. 2.4 Pop-up action menus for the a) Current Folder, b) Command Window, c) Workspace, and d) Command History panels.

The Variable group of the Home tab allows importing data (to be discussed in Sec. 4.8.2), creating and editing new variables (including manipulationing with the already existing ones, shown in the Workspace panel), saving all workspace variables in a binary *mat*-file for future use, and clearing workspace variables and functions. The drop-down menu of the **Clear Workspace** icon is shown in Fig. 2.5c (note that you can also utilize the **Clear Workspace** option of the Workspace action menu of Fig. 2.4c).

The Code group allows you to run some common pieces of code stored as favorites (known as the command shortcuts prior to R2018a), analyze MATLAB code in a current folder for inefficiencies and potential bugs, launch a code profiler that helps determining where you can modify your code to make performance improvements, and clear Command Window and Command History entries. Specifically, the drop-down menu of the **Clear Commands** icon is shown in Fig. 2.5d (you can also use the **Clear**

Command Window and **Clear Command History** options of the Command Window and Command History action menus in Fig. 2.4b and Fig. 2.4d, respectively).

The **Start Simulink** icon of the Home tab (if Simulink is installed) launches Simulink (considered in Chapter 15). The Environment group features the icons that allow you to change the Desktop layout (Fig. 2.5a), adjust many default preferences as desired (to be discussed in Sec. 2.7), set the (order of the) search paths used to look for files, take advantage of parallel computation (if the Parallel Computing Toolbox is installed) (Fig. 2.5e), and get some additional capabilities such as new add-ons (toolboxes), management of existing add-ons, and installation of hardware support tools (Fig. 2.5f). Finally, the Resources group of the Home tab provides you with a quick access to all available help resources and allows you to check for product updates (see the drop-down menu of the **Help** icon in Fig. 2.5g).

It should be noted that MathWorks issues multiple bug-fixing updates between the normal twice-yearly releases of MATLAB and Simulink (e.g., R2016a featured seven updates; R2017b – nine updates; R2018a – six updates; R2018b – three updates). As of R2019a, if a more recent version (update) is available, a notification icon, located to the left of the **Signed in as** . . .

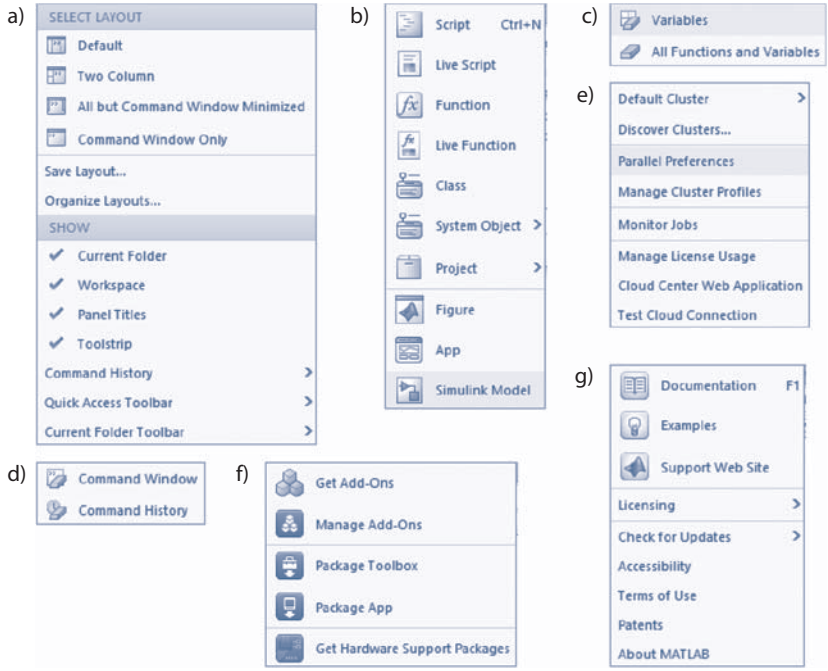


Fig. 2.5 Drop-down menus for the selected icons of the Home tab.

icon in the upper-right corner of the MATLAB Desktop, changes to . Clicking it allows you to install the update (prompting to close all active MATLAB sessions first). To check for interrelease updates manually, choose the **Check for Updates** option of the **Help** button drop-down menu (Fig. 2.5g). (Note that this latter feature has only been available since R2018b.) You can check for known bugs or even submit your own bug report to MathWorks at *www.mathworks.com/support/bugreports*.

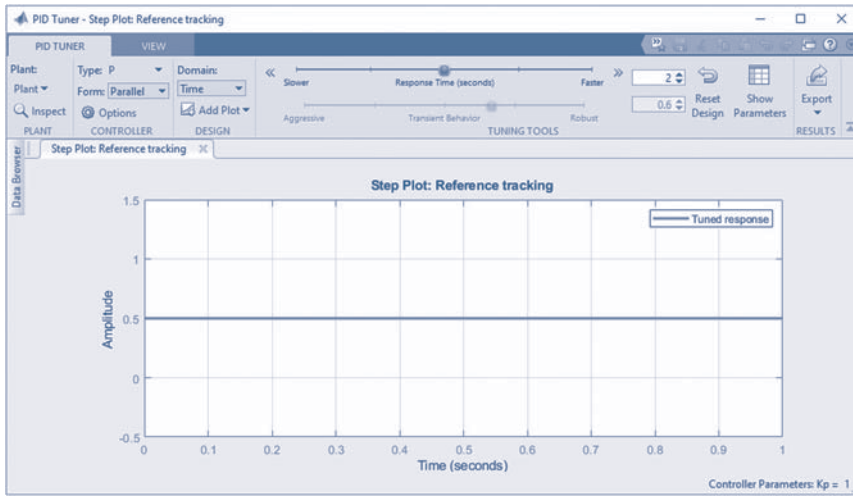
Right-clicking any of the Toolstrip icons allows you to add it to the Quick Access toolbar appearing in the upper-right corner of MATLAB Desktop, to the left of the Help window, where you can type the name of the function or some keywords to get assistance provided by the embedded Help system.

The Plots tab of Desktop shows plotting opportunities for a selected variable. For example, with the `z` variable (a four-element vector) selected in Workspace as shown in Fig. 2.3, MATLAB offers a variety of options (Fig. 2.6a). Clicking any of them results in issuing the corresponding command in the Command Window (e.g., `pie(z)`) and producing a plot (the graphics functions of MATLAB will be discussed in Chapter 6). To see what other plotting options might be available, click the upside-down triangle to open a drop-down menu listing all available plotting options by categories.

Some options behind the **Add-Ons** icon of the Home tab are also repeated in the Apps tab of Toolstrip (Fig. 2.6b). This tab shows the *Apps Gallery*—a collection of available apps. Apps are the interactive MATLAB programs that include GUI and allow rapid testing of different approaches to analyze specific data and prototype need-specific algorithms for unsupervised batch processing of similar data sets. These apps are included in many MATLAB products. Many professionally developed apps are provided by the user community. Depending on the list of toolboxes installed on your computer, the Apps Gallery may look somewhat like the one shown in Fig. 2.6b. If you click the upside-down triangle at the very right of gallery, you will see an extended menu listing all available apps by category, which

Fig. 2.6 MATLAB Toolstrip tabs: a) Plots and b) Apps.

a)



b)

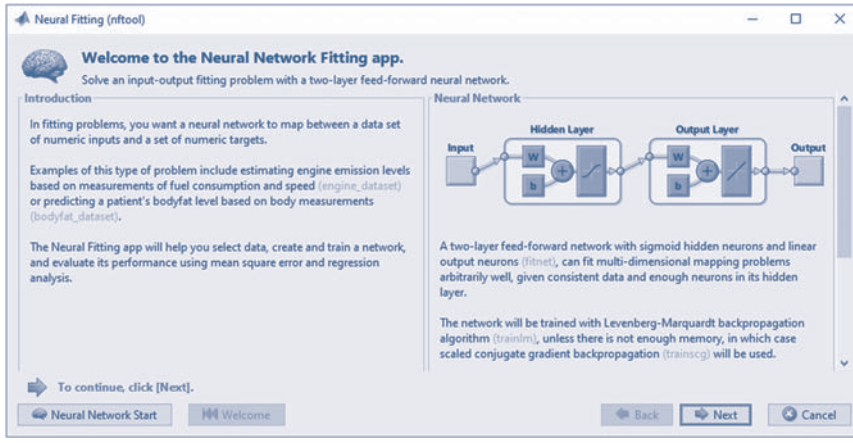


Fig. 2.7 a) PID Tuner and b) Neural Net Fitting apps.

makes them easy to find. Clicking any of them launches the corresponding app (see examples of the main GUIs for two apps in Figs. 2.7a and 2.7b). These apps communicate with the base MATLAB workspace utilizing available data.

To get more apps and/or install additional toolboxes, you can click the **Get More Apps** icon of the Apps tab (Fig. 2.6b), which opens the Add-On Explorer window shown in Fig. 2.8. You can create your own apps, and MATLAB provides you with a convenient single-file application packaging and sharing tool, accessible if you click the fourth icon of the Apps tab, **Package App** (Fig. 2.6b). This tool allows you to include all necessary

Fig. 2.8 Add-On Explorer.

files, document required products, and create a single installation file for easy distribution (more options for sharing apps are discussed in Secs. F.6, F.7, and F.9 of Appendix F). The packaged app can then be added to the Apps Gallery using **Install App**, the third icon of the File group of icons. The very first icon added in R2019a, **Design App**, opens the App Designer, the development environment for designing modern, full-featured interactive applications and writing code that controls app behavior (as discussed in Sec. F.8).

2.2.3 Favorites, Editor, and Live Editor

As mentioned already, the Command History panel (Fig. 2.2) shows all previous keystrokes you entered in the Command Window. Try to dock it to the Desktop (using the action menu of Fig. 2.4d), so that it appears as a regular panel. This panel keeps track of all the sessions you have opened in MATLAB, sorted by date/time, making it very easy to find what you are looking for and to collapse the branches you do not currently need. You

may also re-execute any previous command stored in the Command History panel by double-clicking it or by dragging and dropping it at the Command Window as was mentioned earlier. You can delete, cut, copy, evaluate, and save any portion of the previously issued commands. All you have to do is select the commands you need (use the <Ctrl> key to select the nonsequential commands) and click the right mouse button. As a result, a pop-up menu appears (Fig. 2.9a). Among others, this menu features three useful options allowing you to create (and save) the selected commands as a separate source-code file, live script (introduced in R2016a), or favorite.

If you choose the last option, **Create Favorite**, the *Favorite Command Editor* window opens (Fig. 2.9b). You can modify the selected commands or type in the brand new ones; then, give your code a name (label) and click **Save**. This favorite (group of commands) will be saved under the **Favorites** icon of the Home tab toolstrip (Fig. 2.9c) and will also show up in the Command History panel (Fig. 2.2). If you check the **Add to quick access toolbar** box (Fig. 2.9b), the favorite you created will also be added to the Quick Access toolbar (compare Quick Access toolbars of Fig. 2.1 and Fig. 2.2, where this favorite has already been created). You can bring up the Favorite Command Editor by choosing the **New Favorite** option of the **Favorites** icon (Fig. 2.9c). Dragging the selected commands over to the **Favorites** icon opens the Favorite Command Editor window as well. Figure 2.9c also features three icons to the right of the favorite's name allowing you to remove this favorite from the Quick Access Toolbar, edit or delete it (note that you have to hover over the favorite's name with the mouse pointer for these icons to show up).

The three specific commands shown in Fig. 2.9b (discussed in more detail in Sec. 2.6) allow closing all figure windows (discussed in Chapter 6), clearing (deleting) all variables from the workspace, and returning the cursor to the upper-left corner of the Command Window. While mastering your MATLAB skills, you will repeat these three operations quite often, so

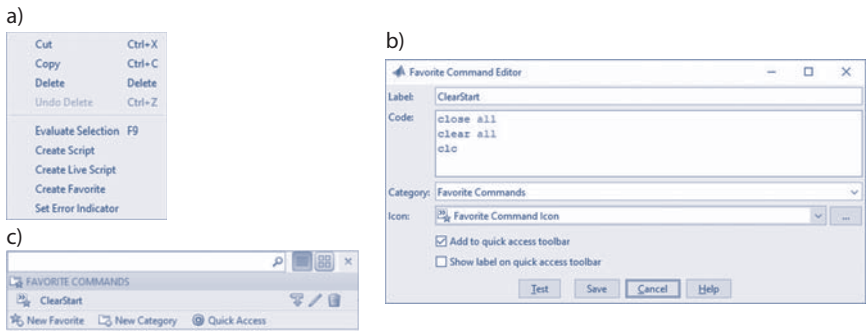


Fig. 2.9 a) Right-click options for the Command History panel, b) Favorite Command Editor, and c) drop-down menu of the Favorites icon of the Home tab toolstrip.

saving them as a favorite allows you to avoid typing them over and over again. (Clicking separately on two icons of the Home tab, **Clear Workspace** and **Clear Commands**, causes the same effect as the last two commands, but there is no icon allowing you to close all open figure windows.)

Now, let us consider the **Create Script** option of the pop-up menu shown in Fig. 2.9a. Choosing this option copies the selected commands to the *Editor*. You can open a blank Editor window by clicking the first icon of the Home tab toolstrip, **New Script**, or choosing the **Script** option when clicking the **New** icon of the Home tab (Fig. 2.5b). If you have some source-code files (hereinafter referred to as *m*-files because of their default extension) created already, you can open the Editor by choosing one of them using the **Open** icon or simply double-clicking its name in the Current Folder panel. Finally, you can open the Editor programmatically by typing `edit` in the Command Window.

The MATLAB Editor acts as a basic word processor for writing your script files and functions; that is, it provides GUI for text editing. Figures 2.10a and 2.10b feature the Editor action menu (which shows up when you click the upside-down triangle in the upper-right corner) and right-click options menu (popping up when you right-click anywhere inside the Editor window). The options these menus provide are also repeated within the three tabs that are added to the Toolstrip when the Editor window is open (Fig. 2.11).

The tools provided by the first two groups of icons of the Editor tab, File and Navigate (Fig. 2.11a) allow creating a new file, opening an existing file,

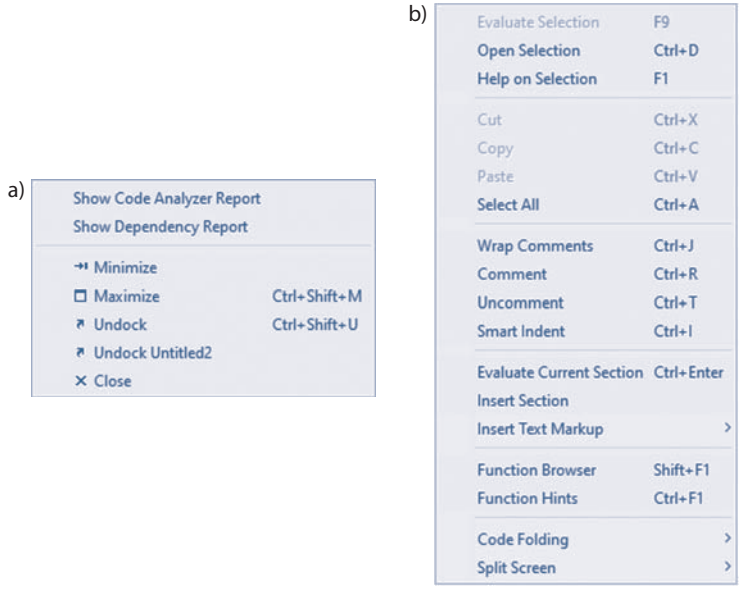


Fig. 2.10 a) Editor action menu and b) right-click menu.

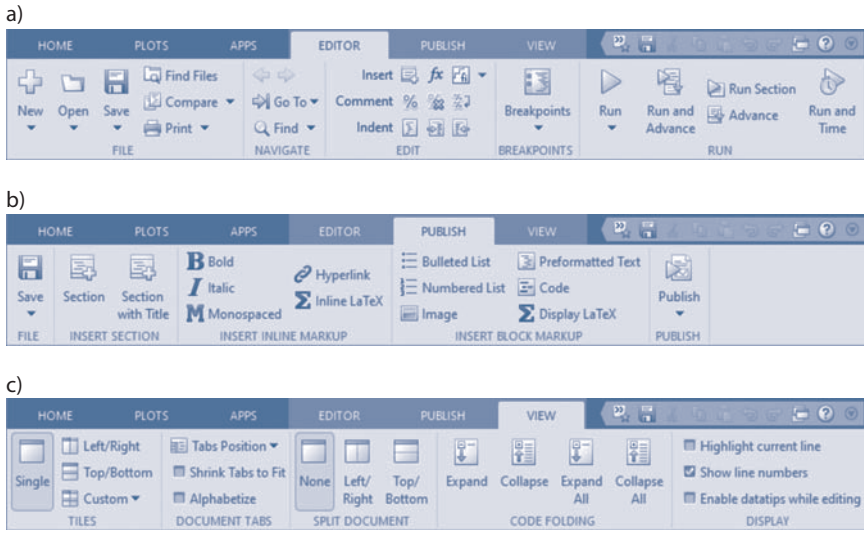


Fig. 2.11 a) Editor, b) Publish and c) View tabs associated with the Editor.

saving the current file, printing, and navigating through the file. They are virtually standard Microsoft Office icons. The icons of the Edit and Breakpoints groups assist in making your code more readable and bug-free, respectively (to be considered in more detail in Secs. 5.10.2 – 5.10.4).

Typically, you would want to create an *m*-file and save it, so that you can run it later in the MATLAB Command Window by simply typing its name. However, if your newly developed *m*-file contains some errors, instead of the expected results you will see a warning message. Therefore, you will need to go back to the Editor, find and fix the errors, save the corrected version of your *m*-file, and then run it from the Command Window again. To avoid this back-and-forth procedure, the MATLAB Editor allows you to run and debug the program directly from the Editor window with the Run group of icons. You can execute the entire script using the **Run** icon, execute portions of the script using the next three icons, or even launch the *Profiler* (considered in Sec. 5.11). You can also select a portion of your script you want to execute and hit the <F9> key (Sec. 5.10.3).

The tools of the Publish tab (Fig. 2.11b) allow you to enhance your script using hypertext markup language (HTML) and extensible markup language (XML) to get it ready to be published on the web (and/or printed) (see Sec. 5.12). MATLAB also allows publishing in other formats—LaTeX, Word, PowerPoint, and PDF. Appendix A lists a variety of mathematical symbols and constructs that can enhance your code and its graphical output using the LaTeX interpreter (Sec. 5.12).

The icons of the last tab, View (Fig. 2.11c), help in managing the Editor window when single and multiple scripts are opened (Sec. 5.10.2).

Finally, let us consider the **Create Live Script** option on the pop-up menu shown in Fig. 2.9a. This option is very similar to **Create Script**, with the only difference being that selecting this option copies the selected commands to the *Live Editor*. Live Editor is an interactive environment that allows combining output and graphics with the MATLAB code that produces them. You can open a blank Live Editor window by clicking **New Live Script** (the second icon of the Home tab toolstrip) or choosing the **Live Script** option when clicking the **New** icon of the Home tab (Fig. 2.5b). For example, Fig. 2.12 shows one of the expressions used in Sec. 2.2.1 typed in Live Editor (on the left). Running it (clicking the **Run** icon of the Live Script toolstrip), produces the result shown to the right of the command within the same interactive environment. Note that the default extension of the live scripts (and live functions) is *.mlx* (as opposed to *.m* for *m*-scripts). [Live script format uses Open Packaging Conventions technology, which is an extension of the zip file format, allowing code and formatted content to be stored in an Office Open XML (ECMA-376) document separately from the output.] You can save a live script as a MATLAB live code file (live script) or *m*-file (ignoring the output part). You can also save it as a PDF, Word, HTML or LaTeX document.)

Note that compared to the Editor tab (Fig. 2.11a), the Live Editor tab (Fig. 2.12) features three new icons, **Task**, **Control** and **Refactor** (residing in the Code group of icons). These tools allow you to automate code creation (reduce the development time) and add interactive controls to set and change the values of variables. As of R2020a, the **Task** menu included eleven app-like interfaces to assist programmer in data preprocessing and table/timetable handling coding as shown in Fig. 2.13a. (The data preprocessing/cleaning workflow is addressed in Secs. 11.6.4 and C.3, while tables and timetables are introduced in Secs. 4.5 and C.2, respectively.) More interfaces (like those two shown for the Symbolic Math Toolbox) could be available if other toolboxes are installed. The **Control** menu offers standard tools (Fig. 2.13b) to produce an interactive narrative that can be shared with others. The last one, **Refactor**, helps to automatically convert selected areas of code into functions or local functions (to be covered in Secs. 5.8.2, 5.8.3).

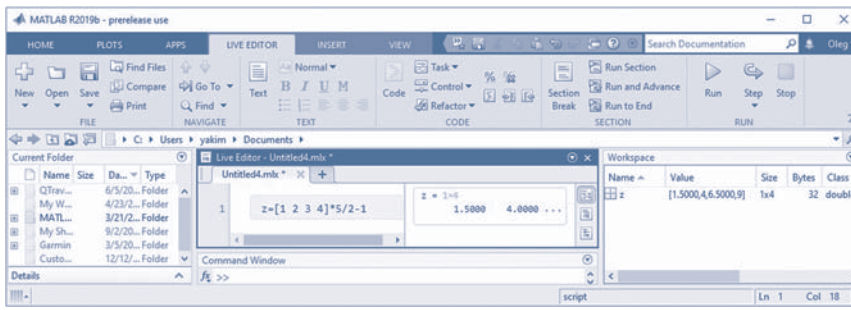


Fig. 2.12 Illustration of the Live Editor environment.

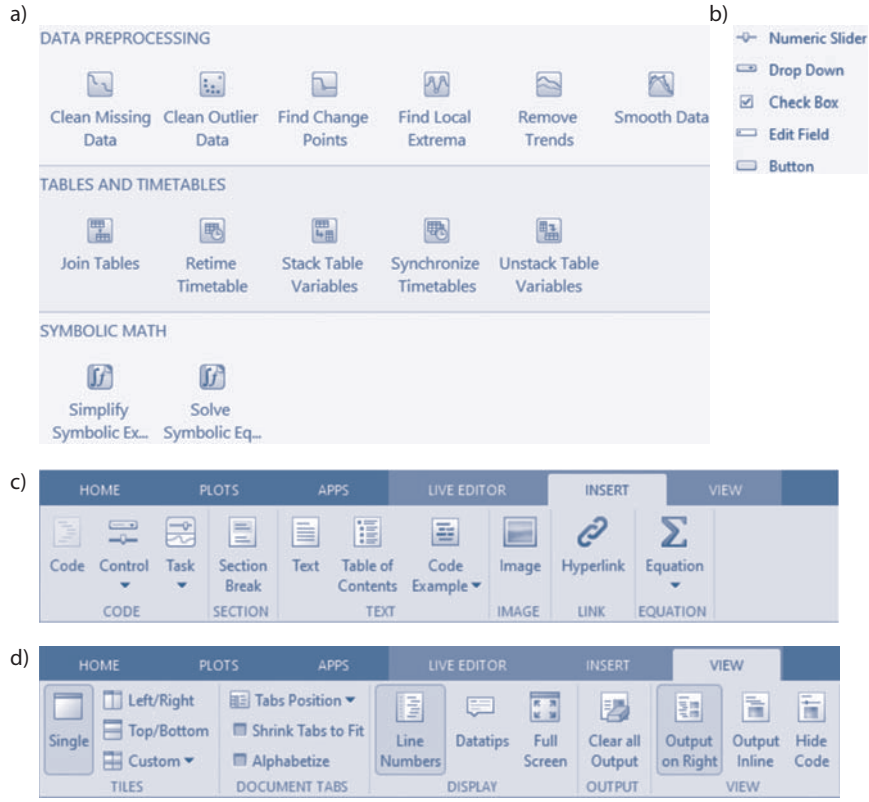


Fig. 2.13 a) Task and b) Control menus; c) Insert and d) View tabs associated with Live Editor.

The second additional tab appearing together with the Live Script tab, Insert (Fig. 2.13c), helps enhance your live code in a manner similar to enhancing a regular script (Fig. 2.11b). The third tab, View (Fig. 2.13d), allows you to change the layout of the Live Editor environment.

If you have worked on *mlx*-files recently, you can open any of them in Live Editor by clicking the **Open** icon, either the Live Script tab or Home tab toolstrip, and choosing the file. You can also double-click one of *mlx*-files in the Current Folder panel. Right-clicking any *m*-file in the Current Folder panel opens a menu that gives you the option to open this *m*-file as a live script rather than a regular script. Finally, you can open Live Editor programmatically by typing in `edit name.mlx` in the Command Window.

Table 2.2 highlights the differences between live code and regular code, and a fragment of a MATLAB-provided live script is shown in Fig. 2.14. You can type

```
>> openExample('matlab/LiveEditorInteractiveNarrative')
```

Table 2.2 Differences between Live Script and Plain Code

Operation	Live Scripts and Functions	Plain Code Scripts and Functions
File Format	Live Code file format (binary)	Plain Text file format (ASCII)
File Extension	.mlx	.m
Output Display	With code in Live Editor	In Command Window
Internationalization	Interoperable across locales	Non-7-bit ASCII characters are not compatible across all locales
Text Formatting	Add and view formatted text in Live Editor	Use publishing markup to add formatted text, publish to view
Visual Representation	Looks like a webpage	Plain code (colored using color preferences)

to open it (note, although the fragment shown in Fig. 2.14 is slightly modified from the original live script). You will be able to fully understand all code in this example after covering the first six chapters of this textbook. The point here is that, compared with the regular *m*-files, live scripts help visually explore and analyze problems while writing, executing, and testing code in a single interactive environment. As such, they can be especially useful for teaching, specifically for creating step-by-step interactive lectures and then evaluating them incrementally to illustrate a topic, modify a piece of code on-the-fly to answer questions or explore related topics, or distribute partially completed files as assignments, among other purposes.

In the future, MathWorks plans to merge both editors into a single one taking advantage of all the tools developed for the live scripts.

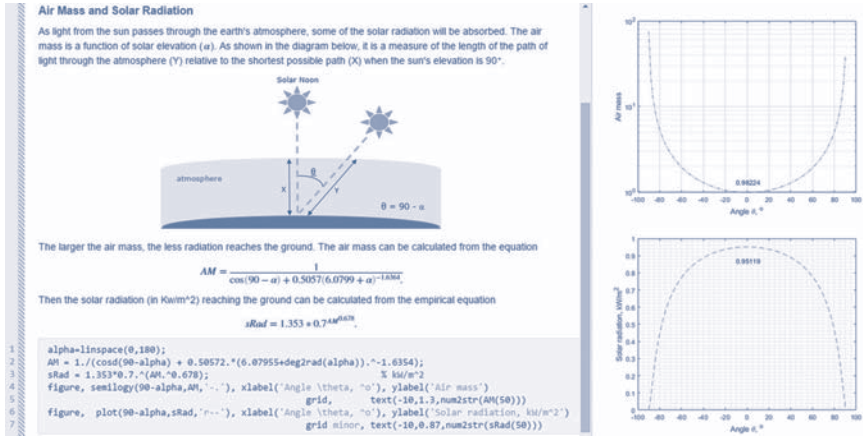


Fig. 2.14 Example of a live script in action.

2.2.4 Workspace Panel and Variables Editor

Let us clear the workspace (you may use the ClearStart favorite you created in the previous section or click the corresponding icons of the Home tab toolstrip) and then type in the following commands:

```
>> a=zeros(5); b=num2cell(ones(3,2)); c=rand(5,6);
>> h=single(ones(2,9)); t='aircraft';
>> s=struct(t,{'B787','A380'},'developer', {'US','EU'});
>> y=int8(45); z=pi/2; 6>3;
```

These commands create several variables (scalars and arrays) of different classes (to be discussed in the next two chapters), showing up in the Workspace panel as presented in Fig. 2.15a (compared with Fig. 2.3, Fig. 2.15a features an undocked Workspace panel). This Workspace panel is in fact a GUI that allows you to view and manage the contents of the MATLAB workspace (the allocation of memory for all variables). The Workspace browser shows the name of each variable (by the icon distinguishing its class), its value, dimensions, size in bytes, and class. These are the default columns of the Workspace panel, but you can choose more (or fewer) of them by right-clicking the name of any column, which opens a pop-up menu (shown in Fig. 2.15b) allowing you to choose what columns (properties) to display.

You can move the entire column (left–right) to a new location by clicking on and dragging its header. You can resize the width of any column by clicking on and moving the divider between the columns. You can sort the contents of the Workspace panel by any column—simply click this column header and observe the reverse order of items in this column (the second click reverses the sort order back to the original). The only exception is the Value column, which you cannot sort. You can achieve the same result by choosing the column from the pop-up menu when selecting the **Sort By** option of the Workspace action menu (Fig. 2.4c). (If you right-click

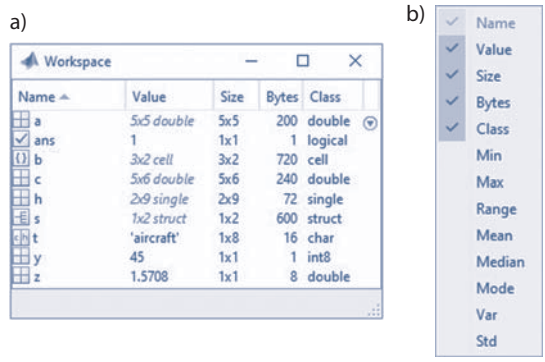


Fig. 2.15 a) Workspace panel and b) available columns.

anywhere in the browser, the same action menu will appear without the bottom part.) You can also use this menu to refresh the content of the browser (the **Refresh** option) and create a new variable (**New**). You can drag any variable to the Command Window to use it in any expression.

Right-clicking the name of a variable or its value brings up another menu, the full version of which is shown in Fig. 2.16a. For numeric arrays, several plotting options are offered automatically. You can also choose other plotting options from the *Plot Catalog* (Fig. 2.16b), which offers exactly the same options as presented in Fig. 2.6a but in a slightly different format, allowing you to see the help topic for the highlighted plotting function. For non-numeric variables or scalars, there are no automatic prompts, and so the **Plot Catalog** option is the only one showing up in the Fig. 2.16a menu.

Rather than plotting, you can use the Fig. 2.16a menu to bring up the *Variables Editor* (**Open Selection**) and save, copy, duplicate, delete, rename, and even edit the value of a chosen variable right in the Workspace browser. For char-class variables and scalars, you can do this by selecting the variable (that is, clicking it) and then clicking the value again. (You can use this approach to rename any variable as well.) Note, when you select a scalar for editing, its exact value shows up, as opposed to a short-format display version of it (to be discussed in Sec. 2.6).

The Variables Editor can also be opened by a simple double-click the name or value of any Workspace variable. Programmatically, you can open it by using the `openvar` function. For example,

```
>> openvar c
```

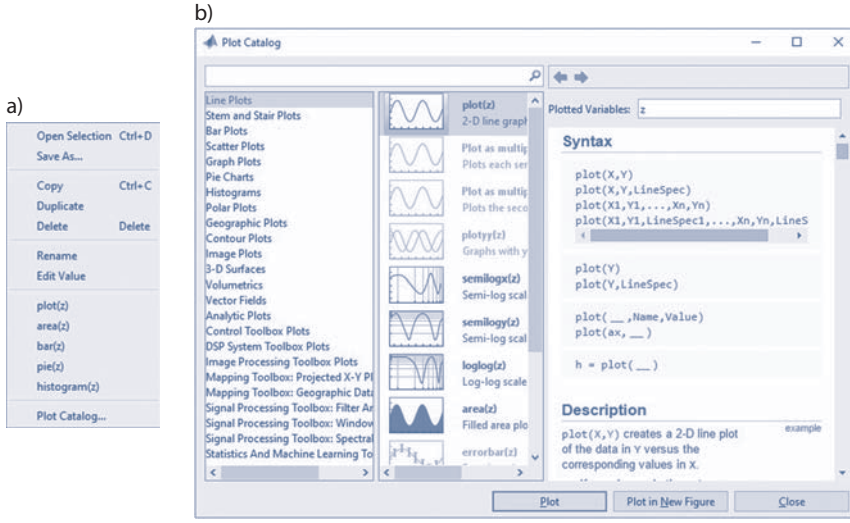


Fig. 2.16 a) Right-click options for Workspace variables and b) Plot Catalog.

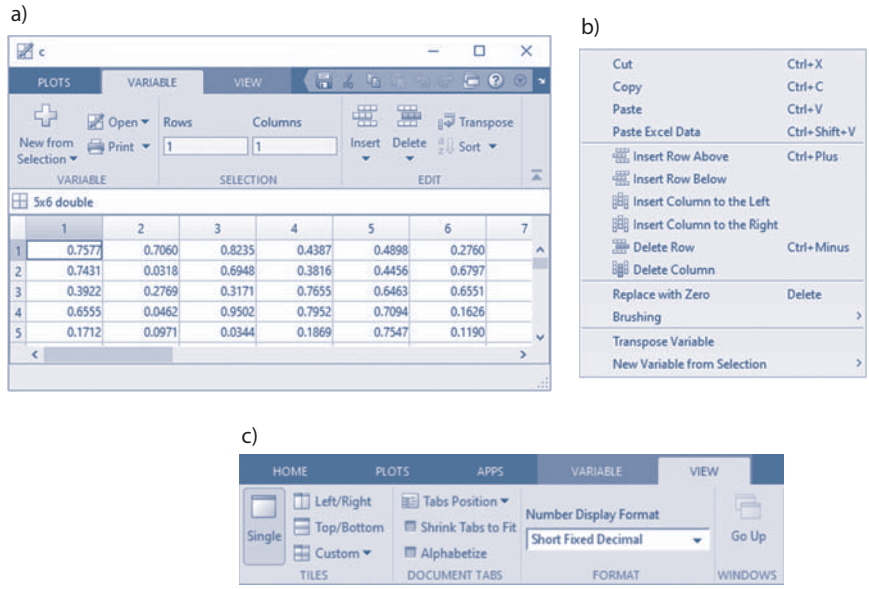


Fig. 2.17 a) Variables Editor, b) right-click menu, and c) View tab.

brings up the window shown in Fig. 2.17a (this figure features an undocked version with three relevant tabs). The Variables Editor allows a variety of options on editing and viewing variables in the workspace. In some sense, working in the Variables Editor is similar to working within a spreadsheet. Figure 2.17b shows a menu that pops up when you right-click anywhere within the editor. Using the Variables Editor, you can change size, content, and format of array elements; cut, copy, paste, and delete them; exchange data with the Command Window and Microsoft Excel; and create graphs (which is why the Plots tab is relevant) and new variables from the current selection. The **Brush** option allows you to interactively mark, delete, modify, and save portions of the array. The View tab (Fig. 2.17c) offers some options for multiple array tiling and number display format (to be addressed in Sec. 2.6).

2.2.5 Current Folder Panel and Search Path Setting

The main purpose of the Current Folder panel and the toolbar right above it (Figs. 2.1, 2.3, 2.12), which can be moved inside the Current Folder panel if necessary, is to navigate to the directory where your files are stored and show its content in a way similar to Microsoft Windows. Right-clicking anywhere within the Current Folder panel brings a pop-up menu shown in Fig. 2.18a. As seen, it shares many options with the action menu of the Current Folder panel shown in Fig. 2.4a. Figure 2.18b presents

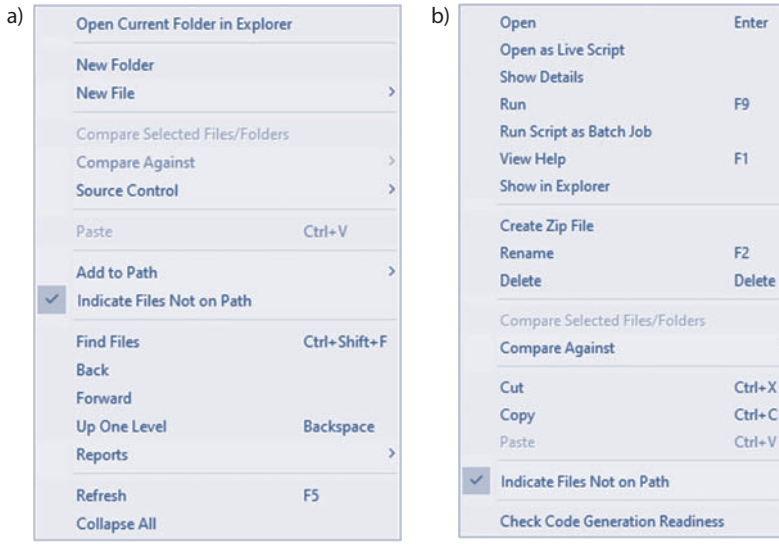


Fig. 2.18 a) Current Folder right-click pop-up menu and b) file-dependent right-click pop-up menu.

an example of a file-type-dependent menu popping up when you right-click any specific file or group of files (in this particular case, it was an *m*-file that was right-clicked).

Figure 2.19a shows that beyond the name of the file and date it was modified (as seen in Figs. 2.1, 2.3, 2.12), you can also add some additional information, including size, type, and description (the latter option adds the first line of each *m*-file in the current folder after its name). This menu is available by either right-clicking the name of any Current Folder browser column or choosing the **Show** option from the action menu (Fig. 2.4a).

Choosing the **New File** option of the Fig. 2.18a menu pops up another menu, as shown in Fig. 2.19b. This is somewhat similar to clicking the **New** icon of the Home tab toolstrip (Fig. 2.5b). The **Add to Path** option makes the current directory "visible" during the variable and function search (to be addressed later in this section). If you add it to the search path, the **Remove from Path** option would be added to this menu. The **Find Files**, **Back**, **Forward**, and **Up One Level** options allow finding files (using extensive search options), going back and forth between the current and previous directories, and switching to the parent folder. The **Reports** option invokes a pop-up menu shown in Fig. 2.19c, offering the option of conducting several types of analysis and reporting within the current folder. The **Sort By** and **Group By** options of the Current Folder action menu of Fig. 2.4a bring self-explanatory submenus shown in Figs. 2.19d and 2.19e, respectively.

Examples of three reports from the Fig. 2.19c menu, namely *Code Compatibility Report*, *Code Analyzer Report* (also known as M-Lint Code Check in the previous releases), and *Dependency Report*, are shown in Fig. 2.20. Beginner students can simply ignore these reports, but later on when writing professional scripts and functions, you may want to explore what these reports have to offer in debugging your code. For example, for every file in the current directory, Code Analyzer Report displays a message suggesting the corrections and improvements you may want to consider (Fig. 2.20b). Common Code Analyzer warning messages are that a variable is defined but has never used or that you should end a line with a semicolon to suppress the output (see a example in Fig. 2.20b). If you type `help codetools`, you can see the full list of commands that can be useful for creating and debugging code (some of which have already been introduced in this chapter).

When you open MATLAB, it sets your current folder to be at some default location. Therefore, unless you change the current directory, every time you save something—it saves in this default folder. Using the default folder is perfectly fine, but our suggestion is that you create a new directory at the known location and navigate to it every time you open MATLAB, so that you know where your files are saved.

When you create your first *m*-file using Editor and try to run it, MATLAB will suggest that you save it first (if it was not saved already). Then, you will have a chance to choose the directory in which to save it. If

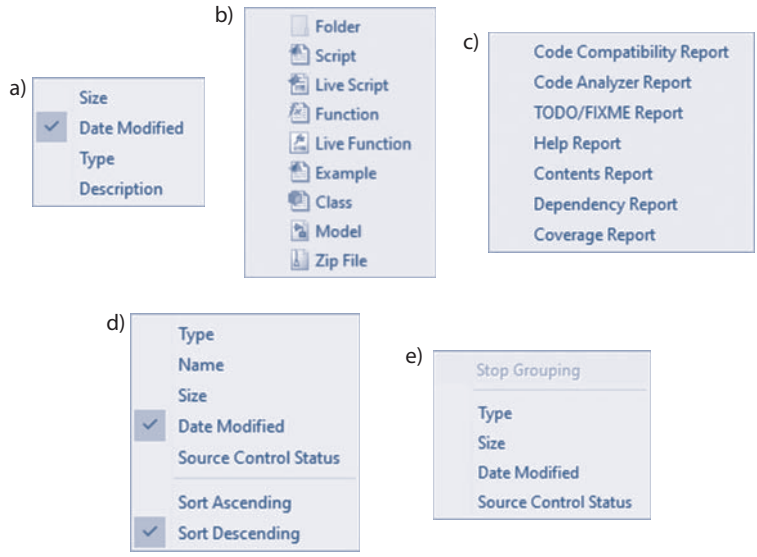


Fig. 2.19 a) **Show**, b) **New File**, c) **Reports**, d) **Sort By**, and e) **Group By** submenus of the Current Folder action menu of Fig. 2.4a.

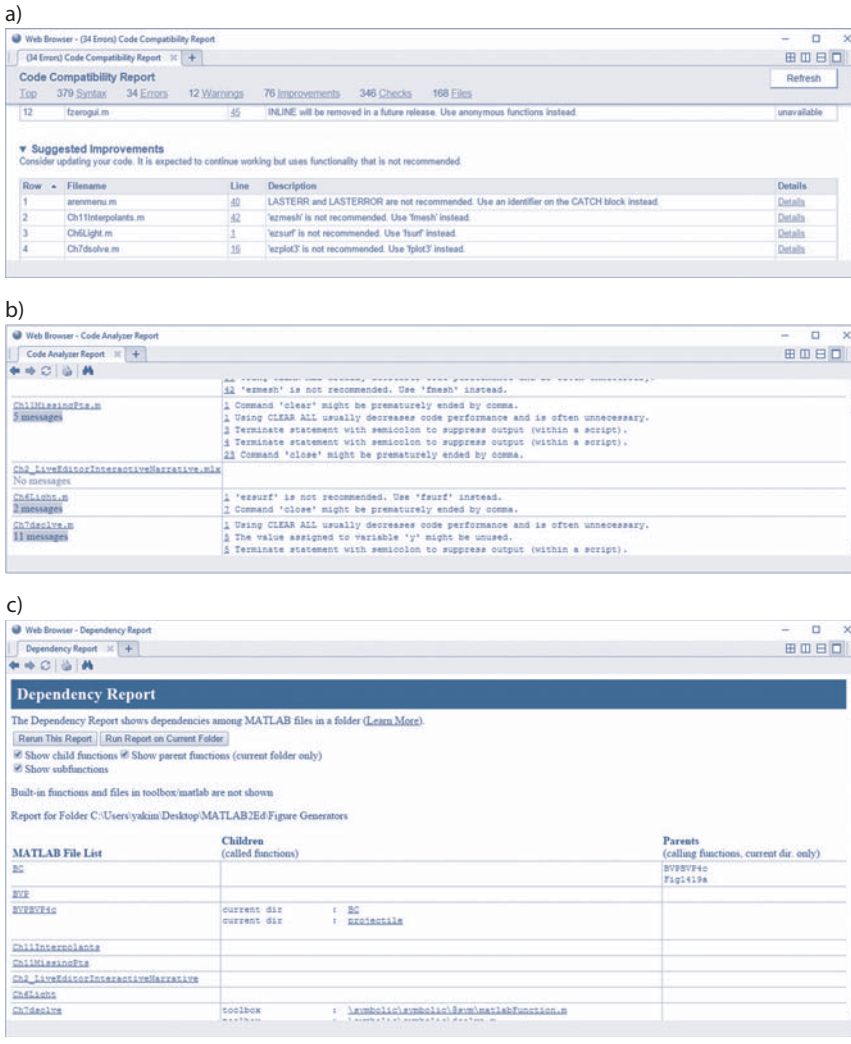


Fig. 2.20 a) Code Compatibility Report, b) Code Analyzer Report, and c) Dependency Report windows of the **Reports** option of the actions menu.

this directory happens to be different from the current folder, then the Editor will suggest you to make this directory a current directory (see example in Fig. 2.21).

This is important because you can only run the files from the current directory. When you are in the Command Window, you can run the files that are located either in the current folder or within the set of other directories MATLAB knows about, so that it can search through them looking for the file you are trying to execute. This set of "other directories" is referred to as a *search path*. To see which directories are on the search

Fig. 2.21 Attempting to run a script from a not-current folder.

path or to change the search path, you should select the **Set Path** icon of the Home tab toolstrip, which results in the dialog window shown in Fig. 2.22. Alternatively, you can open this window programmatically using the `pathtool` command.

The default search path shown in Fig. 2.22 allows you to use all the functions available in basic MATLAB and toolboxes that are installed on your computer from any directory. For instance, the `pi` function, *pi.m*, which we used in Sec. 2.2.1, resides in the . . . *MATLAB\R20xxx\toolbox\matlab\elmat* directory (highlighted in Fig. 2.22). Nevertheless, you were able to use it with a different directory set as the Current Folder. If you want to, you can use the Set Path dialog box to add your own directories to the search path, so that the files stored there are visible from any directory. Be warned though, that if the names of your files or variables happen to be the same as those of MATLAB files or variables and your folders are located prior to those of MATLAB, you will be in danger of not being able

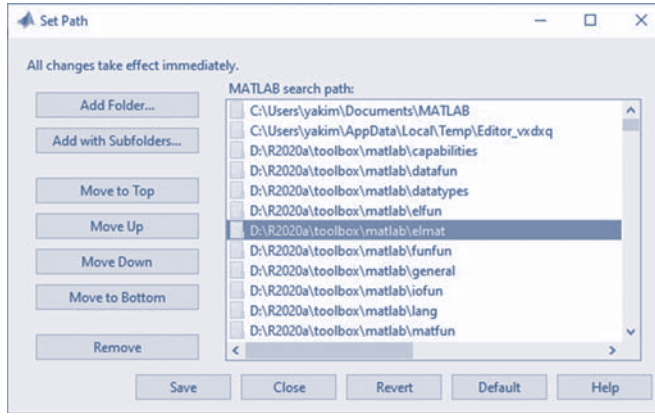


Fig. 2.22 Set Path dialog box.

to use the MATLAB functions and variables because they will be overshadowed by your own.

2.3 Help System

Even two decades ago, the MATLAB documentation occupied several volumes. These days, every new release introduces some changes that make documentation on the previous release somewhat obsolete or at least incomplete. For this reason, an extensive embedded MATLAB Help system has become a major source of information. Once MATLAB has been installed, this help system immediately starts to work, offering you the Getting Started notes (Fig. 2.23) to read.

MATLAB has several options to provide you with the relevant help when needed:

Help browser	This GUI (shown in Fig. 2.23) allows you to find the most complete information and view online documentation for all MathWorks products
Help functions	The `help`, `lookfor` and `doc` functions can be used to display syntax information for a specified function (Table 2.3)
Other resources	Additional online help can be obtained by running various demos, contacting technical support, searching documentation, viewing a list of available books, and participating in a newsgroup

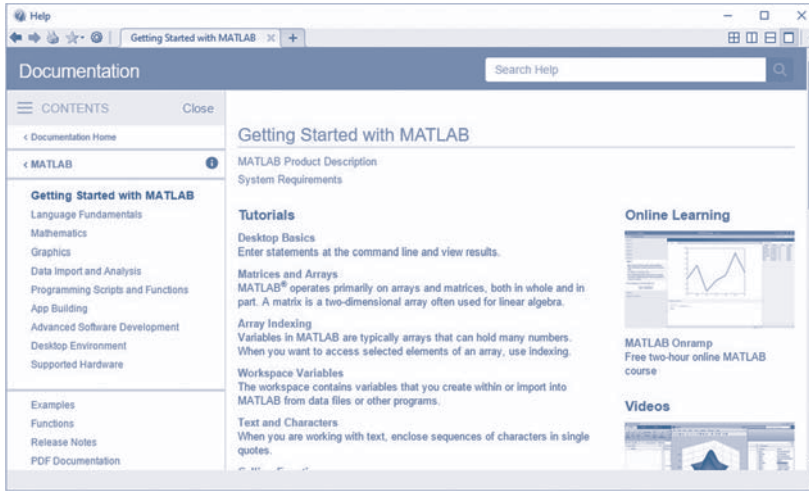


Fig. 2.23 Getting Started notes in the Help browser.

Table 2.3 The Help Functions

Syntax	Description
`help function_name`	Displays a description of the specific function `function_name` (displays the comment lines of the so-called *help text* the corresponding function starts with)
`lookfor topic`	Allows you to search for functions based on a keyword `topic` (lists all functions, the first comment line of the help text of which contains the word `topic`); if the suffix `-all` is added, the `lookfor` function searches the entire help text of all the functions, not just the first line of it
`doc function_name`	Opens documentation on a `function_name` in the Help browser (`doc` displays the start page of the Help browser)

Although the last option in Table 2.3 opens a help topic on a selected function in the *Help browser* window (shown in Fig. 2.23), you can always open the start page of the Help browser by simply typing `doc` in the Command Window (previous releases had the `helpbrowser` command), by clicking the **Help** icon of the Home tab toolstrip, or by selecting the **Documentation** option from this icon's drop-down menu (Fig. 2.5g). As seen in Fig. 2.23, the Help browser has two panes: the Contents pane on the left, and the Display pane on the right. Using these panes along with the **Search Help** tool above the Display pane, you can get the most complete help on any topic you may be interested in.

Figure 2.24 provides an example of the Help browser viewing area in response to a `bin2dec` function search. The display area on the right lists all occurrences of this function in different help topics. The filter on the left allows you to narrow your search. Figure 2.24 also features five icons in the upper-left corner that allow you to go back and forward between the previous searches, print the current help topic, add the topic to the list of favorites, and open the Help browser preferences window. Five icons in the upper-right corner allow rearranging multiple Search Result tabs within the Help browser window and docking it to the Desktop. Figure 2.25 shows an example of a help page on a specific function (the first option in the Fig. 2.24 list).

The MATLAB Help system provides you with supporting materials on almost every major topic, including Desktop and its tools. These materials are quite helpful, especially for self-studying and self-perfecting your programming skills. You can choose the **Support Web Site** option from the drop-down menu of the **Help** icon of the Home tab toolstrip (Fig. 2.5g) and then click Tutorials to get to the webpage offering a variety of introductory videos, including the so-called MATLAB Onramp and Simulink Onramp (*www.mathworks.com/support/learn-with-matlab-tutorials.html*). Clicking the **Learn MATLAB** icon (residing at the lower-right corner of the Resources group of icons of the Toolstrip) opens the MATLAB and Simulink Training webpage, offering even more *self-paced courses* on

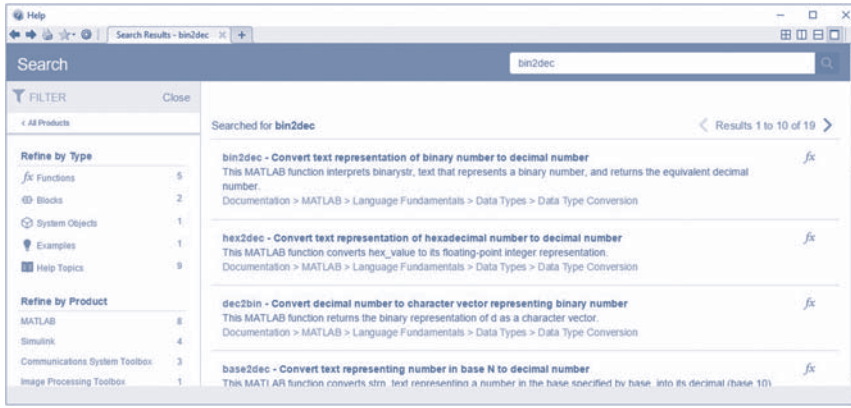


Fig. 2.24 Viewing search results in the Help browser.

different subjects. You can also take advantage of numerous *Examples*, live scripts (Fig. 2.26) accessible if you search for MATLAB Examples or simply Examples in the Help browser or choose the **Examples** option from the drop-down menu of the **Help** icon of the Home tab toolstrip (Fig. 2.5g). You can get to the same page programmatically from the Command Window by typing in `demo` or `demo matlab` [it also works for Simulink (`demo simulink`) and other products (e.g., `demo symbolic` or `demo aerospace`)].

One more option to get quick help while working in the Command Window—the **Function Browser** button—will be introduced in Sec. 2.6.

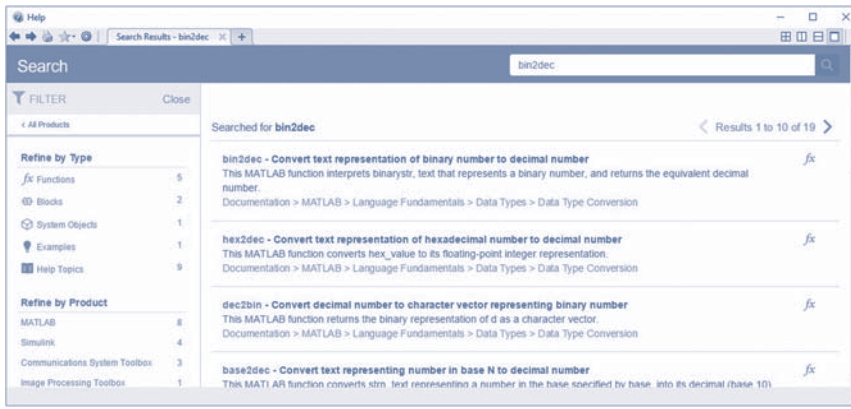


Fig. 2.25 Viewing help documentation for the selected topic (`bin2dec` function).

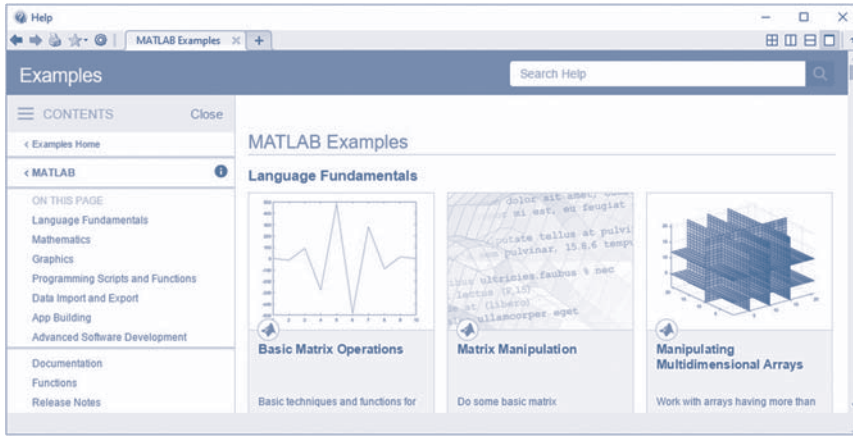


Fig. 2.26 MATLAB Examples page of the Help system.

2.4 Function Collections

MATLAB provides you with a large number of standard mathematical functions along with a variety of specialized functions to create and handle scalars, vectors, and multidimensional arrays; solve systems of algebraic equations; produce 2-D and 3-D plots; and more. Each installed toolbox enhances this list further adding a collection of the specialized functions.

2.4.1 Elementary Math Functions

You can see the list of available elementary mathematical functions by issuing the following instruction:

```
>> help elfun
```

These functions are presented in Tables 2.4–2.8 by groups.

2.4.2 Function Usage

The generalized syntax of the MATLAB functions is

```
[outputs] = function_name(inputs)
```

Here, `inputs` and `outputs` may have multiple entries, but typically any function permits a simple straightforward syntax

```
x = function_name(y)
```

where `inputs` and `outputs` are represented by a single, most important, entry each. For all functions introduced in Tables 2.4–2.8, this simple format is the only format—for example,

```
>> x = sin(pi/2)
```

Table 2.4 Trigonometric Functions

Function	Description	Function	Description
Trigonometric Functions (Operating with Radians)			
`sin(X)`	Sine[a]	`sec(X)`	Secant (1/cos(*x*))
`asin(X)`	Inverse sine	`asec(X)`	Inverse secant
`cos(X)`	Cosine[a]	`csc(X)`	Cosecant (1/sin(*x*))
`acos(X)`	Inverse cosine	`acsc(X)`	Inverse cosecant
`tan(X)`	Tangent	`cot(X)`	Cotangent (1/tan(*x*))
`atan(X)`	Inverse tangent	`acot(X)`	Inverse cotangent
`atan2(Y,X)`	Four-quadrant inverse tangent		
Trigonometric Functions (Operating with Degrees)			
`sind(X)`	Sine	`secd(X)`	Secant
`asind(X)`	Inverse sine	`asecd(X)`	Inverse secant
`cosd(X)`	Cosine	`cscd(X)`	Cosecant
`cosd(X)`	Inverse cosine	`acscd(X)`	Inverse cosecant
`atand(X)`	Tangent	`cotd(X)`	Cotangent
`atan2d(Y,X)`	Four-quadrant inverse tangent	`acotd(X)`	Inverse cotangent
Angle Unit Conversions			
`deg2rad(X)`	Degrees to radians	`rad2deg(X)`	Radians to degrees

[a]As of R2018b, there are two more functions, `sinpi` and `cospi`, computing sin(π^*x) and cos(π^*x) more accurately (refer to Fig. 8.16).

which returns

```
x = 1
```

Note that to enclose the function's input argument(s) operated on by the function, MATLAB uses a pair of parentheses. If function syntax permits more than one output, which is the case for the majority of functions, a pair of square brackets must be used to enclose the list of multiple output parameters. If you want to explore more options (if available) on how to use any specific function, you can now use the MATLAB Help system introduced in the previous section.

Table 2.5 Hyperbolic Trigonometric Functions

Function	Description	Function	Description
`sinh(X)`	Hyperbolic sine	`sech(X)`	Hyperbolic secant
`asinh(X)`	Inverse hyperbolic sine	`asech(X)`	Inverse hyperbolic secant
`cosh(X)`	Hyperbolic cosine	`csch(X)`	Hyperbolic cosecant
`acosh(X)`	Inverse hyperbolic cosine	`acsch(X)`	Inverse hyperbolic cosecant
`tanh(X)`	Hyperbolic tangent	`coth(X)`	Hyperbolic cotangent
`atanh(X)`	Inverse hyperbolic tangent	`acoth(X)`	Inverse hyperbolic cotangent

Table 2.6 Exponential Functions

Function	Description
`exp(X)`	Exponential (`exp(1)` represents a mathematical constant, sometimes called Euler's number, that is the base of the natural logarithm)
`expm1(X)`*	Accurate value of $\exp(x) - 1$
`log(X)`	Natural logarithm ($\ln(x)$)
`log1p(X)`*	Accurate value of $\ln(1 + x)$
`log10(X)`	Common (base-10) logarithm ($\log_{10}(x)$)
`log2(X)`	Base-2 logarithm ($\log_2(x)$)
`reallog(X)`	Natural logarithm for nonnegative real arrays
`pow2(X)`	Base-2 power
`realpow(X,Y)`	Raises each element of array `X` to the power of its corresponding element in array `Y`
`sqrt(X)`	Square root
`realsqrt(X)`	Square root of for nonnegative real arrays
`hypot(A,B)`	Square root of sum of squares (hypotenuse)
`nthroot(X)`	Real *n*th root of real numbers
`nextpow2(X)`	Next higher power of 2

* Special functions `expm1` and `log1p` allow you to compute certain quantities more accurately to avoid underflow and overflow and compensate for the round-off errors (to be discussed in Chapter 8).

The usage of the basic functions is as simple and intuitive as it can possibly be. For example, to compute the square root of 9, you type `sqrt(9)`. Taking the square root or logarithm of a negative number is not an error—the appropriate complex result will be produced automatically. For example,

```
>> sqrt(-3)
```

returns

```
ans = 0.0000 + 1.7321i
```

Table 2.7 Complex Functions

Function	Description
`abs(X)`	Absolute value and complex magnitude
`angle(Z)`	Phase angle in the interval $[-\pi, \pi]$
`complex(X,Y)(A,B)`	Constructs complex data from the real and imaginary parts
`conj(Z)`	Complex conjugate
`imag(Z)`	Imaginary part of complex number
`real(Z)`	Real part of complex number
`unwrap(X)`	`Unwraps` the radian phase angles (whenever the jump between consecutive angles is greater than or equal to π radians, unwrap shifts the angles by adding multiples of $+2\pi$ until the jump is less than π)
`isreal(X)`	Determines whether `X` is real
`cplxpair(Z)`	Sorts complex numbers into complex conjugate pairs

Table 2.8 Rounding and Remainder Functions

Function	Description
`fix(X)`	Rounds toward zero
`floor(X)`	Rounds toward negative infinity
`ceil(X)`	Rounds toward positive infinity
`rounds(X)`	Rounds toward nearest integer (`round(X,N)` rounds to `N` digits)
`mod(a,m)`	Executes a modulo operation (signed remainder after division of `a` by `m`)
`rem(a,b)`	Computes a remainder after division of `a` by `b`
`sign(X)`	Returns the signum function of the argument (returns `1` if the element is greater than 0, `0` if it equals 0, and `-1` if it is less than 0)

The imaginary part of a complex number can be represented by either letter `i` (like in the example above) or letter `j`. The multiplication symbol in front of `i` or `j` may be omitted. Most of the elementary mathematical functions accept complex numbers as arguments as well. For example,

```
>> sin(5+2*j)
```

yields

```
ans = -3.6077 + 1.0288i
```

Elementary functions of Table 2.7 support complex-number-specific operations, such as

```
>> rad2deg(angle(ans))
```

which returns

```
 164.0833
```

(Note how we used a nested format in this example, applying the `rad2deg` function of Table 2.4 to the result of the `angle` function of Table 2.7.)

2.4.3 Function Libraries

If you type

```
>> dir([matlabroot '\toolbox\matlab'])
```

you will see dozens of additional subdirectories beyond the *..\elfun* that store other functions used by the basic MATLAB. They include *..\elmat*, storing the functions handling elementary matrices and matrix manipulation (to be considered in Chapter 3), *..\datatypes*, featuring a group of functions that allow management of different data types and structures; three directories with functions enabling the rich plotting capabilities of MATLAB, *..\graphics*, *..\graph2d*, and *..\graph3d* (to be considered in Chapter 6); and *..\optimfun*, containing major optimization and root-finding functions (to be considered in Chapter 10). You may then explore the content of these

subfolders by typing in `help Subfolder_name`. For example, typing `help ops` results in a list of operators and special characters, which includes

```
Arithmetic operators
  plus      - Plus                          +
  minus     - Minus                         -
  mtimes    - Matrix multiply               *
  times     - Array multiply                .*
  mpower    - Matrix power                  ^
  power     - Array power                   .^
  mldivide  - Backslash or left matrix divide   \
  mrdivide  - Slash or right matrix divide      /
  ldivide   - Left array divide             .\
  rdivide   - Right array divide            ./
Relational operators
  eq        - Equal                         ==
  ne        - Not equal                     ~=
  lt        - Less than                     <
  gt        - Greater than                  >
  le        - Less than or equal            <=
  ge        - Greater than or equal         >=
Logical operators
  and       - Element-wise logical AND      &
  or        - Element-wise logical OR       |
  not       - Logical NOT                   ~
```

and other functions.

For better efficiency, many of the MATLAB core functions, including those of Tables 2.4–2.8, are built in. This means that they are stored as executable files and, as such, no computational details are available. However, most of them still have an *m*-file associated with them, which only supplies documentation for the function.

Typing

```
>> dir([matlabroot '\toolbox'])
```

results in the list of function collections associated with installed toolboxes. For example, it may look like this:

```
aero        daq         glee          matlabxl    lrtw          slde
aeroblks    diagram     hdlcoder      mlhadoop    shared        sldv
autoblks    dig         idelink       mps         signal        sltp
bioinfo     distcomp    images        multisim    simulink      stateflow
coder       dsp         instrument    nnet        simulinktest  stats
comm        eml         local         optim       sl3d          symbolic
compiler    fixedpoint  map           realtime    slcontrol     target
control     fixpoint    matlab        robotics    slcoverage    vision
```

You can still use the `help` function to access the content of these subfolders; for example, the `help map` yields a long list of functions available within the Mapping Toolbox (if you have it installed), including some you might find very useful:

```
Three-dimensional coordinate transformations between global systems
  geodetic2ecef - Transform geodetic to geocentric (ECEF) coordinates
  ecef2geodetic - Transform geocentric (ECEF) to geodetic coordinates
Three-dimensional global-local coordinate transformations
  geodetic2enu - Geodetic to local Cartesian ENU
  geodetic2ned - Geodetic to local Cartesian NED
  enu2geodetic - Local Cartesian ENU to geodetic
  ned2geodetic - Local Cartesian NED to geodetic
Spherical distance conversions
  deg2km    - Convert distance from degrees to kilometers
  deg2nm    - Convert distance from degrees to nautical miles
  deg2sm    - Convert distance from degrees to statute miles
  km2deg    - Convert distance from kilometers to degrees
  nm2deg    - Convert distance from nautical miles to degrees
  sm2deg    - Convert distance from statute miles to degrees
Length unit conversions
  km2nm     - Convert kilometers to nautical miles
  km2sm     - Convert kilometers to statute miles
  nm2km     - Convert nautical miles to kilometers
  nm2sm     - Convert nautical to statute miles
  sm2km     - Convert statute miles to kilometers
  sm2nm     - Convert statute to nautical miles
Longitude or azimuth wrapping
  wrapTo180   - Wrap angle in degrees to [-180 180]
  wrapTo360   - Wrap angle in degrees to [0 360]
  wrapToPi    - Wrap angle in radians to [-pi pi]
  wrapTo2Pi   - Wrap angle in radians to [0 2*pi]
```

If some of these functions are used by multiple toolboxes, they actually reside in the *..\toolbox\shared* subfolder (as is the case for the last four utilities).

As opposed to the core functions, the majority of the MATLAB toolbox functions are implemented as full-fledged *m*-files, allowing you to see the source code and even modify it if you need to (this will be addressed further in Sec. 5.9.1). When one of the *m*-file functions is called, MATLAB parses and executes each line of code in the *m*-file. It saves the parsed version of the function in memory, eliminating parsing time on any further calls to this function.

2.4.4 Constants

MATLAB utilizes several constants. Three of them have been introduced in this chapter already. The full list is presented in Table 2.9

Table 2.9 MATLAB Constants

Constant	Brief Description
`pi`	Generates the numerical equivalent to a numerical constant $\pi = 3.14159\ldots$ (also called Archimedes' constant), the ratio of a circle's circumference to its diameter
`i` and `j`	Represents the imaginary unit, $\sqrt{-1}$ (for complex numbers you may use an asterisk as in `2+3*i` or have it implied as in `2+3i`)
`realmax`	Returns the largest double-precision floating-point number representable on your computer (anything larger overflows), $1.7977e^{+308}$ (this number is one bit less than 2^{1024}, as will be discussed in detail in Chapter 8)
`realmin`	Returns the smallest positive normalized double-precision floating-point number, $2.2251\ e^{-308}$ or 2^{-1022}
`eps`	Returns the distance from 1 (by default) to the next double-precision floating-point number, $2.2204\ e^{-16}$ or 2^{-52}
`Inf`	Represents infinity, ∞, when the result of operations like division by zero and overflow exceeds `realmax`
`NaN`	Represents an undefined numerical result, "Not-a-Number," produced by expressions like `0/0`, `Inf/Inf`, or `Inf-Inf`, as well as from arithmetic operations involving an `NaN`
`NaT`	Represents Not-a-Time, a value that can be stored in a `datetime` array to indicate an unknown or missing datetime value (see Sec. 4.2 and Appendix C)

The truth of the matter is that these "constants" are in fact the regular (built-in) functions appearing as constants (meaning they allow syntax with no input arguments). Except for the first two, these functions allow multiple syntax, including the option to pass one or even more input arguments. For example,

```
>> eps(realmax)
```

returns

```
ans = 1.9958e+292
```

while

```
>> eps(realmin)
```

yields

```
ans = 4.9407e-324
```

This particular function, `eps`, shows the positive distance between the number represented by the input argument (its absolute value) and the next larger-in-magnitude floating-point number. As seen, these distances are different for different numbers, and this has to do with the floating-point representation of a number. Smaller numbers have better precision (Chapter 8). Note how we used the nested format again, passing on the result of one function as an input argument for another.

The last two functions of Table 2.9, `Inf` and `NaN`, are very important. If your code produces some unexpected results involving one of these

"constants," computations continue without an error. Of course, all consecutive results are meaningless, but at least the program does not crash.

Let us conclude this section with a warning to be creative but careful in choosing the names for your variables so that none of them coincides with the names of MATLAB functions or constants. Otherwise, you may overwrite any of them with a new variable and get yourself in trouble. For example, if, by mistake, you introduce a variable `pi` equal `2` (i.e., `pi=2;`), the following command will never give you an area of a 3-unit radius circle:

```
>> Area=pi*3^2
```

Instead, it will return `18`. Similarly, if you unintentionally create a variable `cos=3`, then an attempt to use the `cos(pi)` call will result in an error message

```
??? Index exceeds array bounds.
```

meaning that from now on MATLAB will treat `cos` as a scalar, not a function. Obviously, when dealing with the complex numbers, do not use `i` and `j` as variables, and so on. You will learn more about this in Sec. 5.8.6, but for now let us show how to restore the original constant value or function programmatically. If you get yourself in trouble, simply clear (delete) wrong-name variables using the `clear` function:

```
>> clear pi cos i j
```

This function will be formally introduced in Sec. 2.6.

2.5 Order of Precedence

You can build expressions that use any combination of arithmetic, relational, and logical operators (the latter two will be discussed further in Secs. 5.2 and 5.3, respectively). As in any programming language, mathematical operations represented by the symbols +, −, *, /, \, and ^ follow a set of rules called *precedence*. Precedence levels determine the order in which MATLAB evaluates an expression. Mathematical expressions are evaluated starting from the left, with the exponentiation operation having the highest order of precedence, followed by multiplication and division with equal precedence, followed by addition and subtraction with equal precedence as well.

To alter the order of precedence you may use parentheses, `( )`. Remember not to confuse them with the square, `[ ]`, or curly brackets, `{ }`, which are not allowed in MATLAB mathematical expressions because they have a special usage. Evaluation of mathematical expression begins with the innermost MATLAB pair of parentheses and proceeds outward.

To better understand these rules, let us consider a simple example. Suppose you are dealing with the following mathematical expression:

$$\frac{\left(\sin\left(0.3\pi - \frac{1}{32}\pi\right) + 3\right)^2}{2\pi^3} - \frac{3}{4} \tag{2.1}$$

In MATLAB, it should appear like `(sin(0.3*pi-pi/32)+3)^2/2/pi^3-3/4`. Of course, to avoid mistakes, you should feel free to insert parentheses wherever you are unsure of the effect precedence will have on the calculation. Besides, using parentheses may also improve the readability of your MATLAB expressions. For example, Eq. (2.1) can also be represented as `(sin(0.3*pi-(1/32)*pi)+3)^2/(2*pi^3)-(3/4)`.

To summarize, the precedence rules for MATLAB operators, ordered from highest precedence level to lowest precedence level, are

1. Parentheses `()`
2. Power `^`
3. Multiplication `*`, right division `/`, left division `\`
4. Addition `*`, subtraction `-`

This list will be updated in Sec. 5.3 (Table 5.2) to include precedence of relational and logical operators.

2.6 Managing Work Session

Now that you have some practice in typing your commands in the Command Window, let us explore some additional options on expedient management of your work session in MATLAB. We start by considering different display formats, followed by some useful hints and tricks for fast typing, and end by introducing special symbols and some relevant functions.

As mentioned earlier, although all computations in MATLAB are done in double precision, by default, the results are displayed in the short format with only four decimal digits. For your convenience, MATLAB allows you to change the output display format by using the `format style` command, where all available styles are shown in Table 2.10.

Using the `format` command, you may also change the output display spacing:

`format compact`	Suppresses excess line feeds to show more output in a simple screen
`format loose`	Adds extra line feeds to make output more readable

Note that the above calls use the so-called *command syntax* of the `format` function (without parentheses) to pass along a single input

Table 2.10 Controlling the Output Display Format

Style	Description	Example
`short`	Short, fixed-decimal format with 4 digits after the decimal point (ADP)	`3.1416`
`long`	Long, fixed-decimal format with 15 and 7 digits ADP for `double` and `single` values, respectively	`3.141592653589793`
`shortE`	Short scientific floating-point format with 4 digits ADP	`3.1416e+00`
`longE`	Long scientific floating-point format with 15 and 7 digits ADP for `double` and `single` values, respectively)	`3.141592653589793e+00`
`shortEng`	The same as `shortE`, but with three digits dedicated to store exponent	`3.1416e+000`
`longEng`	The same as `longE`, but with three digits dedicated to store exponent at the expense of having only 14 and 6 digits ADP for `double` and `single` values, respectively	`3.14159265358979e+000`
`shortG`	`short` or `shortE`, whichever is more compact	`3.1416`
`longG`	`long` or `longE`, whichever is more compact	`3.14159265358979`
`hex`	Hexadecimal format	`400921fb54442d18`
`+`	The symbols `+`, `-`, and blank are printed for positive, negative, and zero elements, whereas imaginary parts are ignored	`+`
`bank`	Fixed format for dollars and cents (two decimal digits)	`3.14`
`rat`	Approximation by ratio of small integers	`355/113`

argument (as you will see later, some basic functions do allow it). However, if you prefer to, you may still use the full-fledged *function syntax* [e.g. `format('rat')`]. In this case, the input argument you are passing on is a character vector, and that is why it is enclosed in single quotes (to be covered in Sec. 3.6).

You may always check the current display format setup by typing

```
>> get(0,'Format')
```

If you want to return to the default format, which is `loose` and `short`, type

```
>> format
```

Now, let us consider a few typing hints and tricks. They are related to the use of the keyboard arrow buttons, <Ctrl>, and <Tab> keys to recall, edit, and reuse functions and variables you typed earlier. For example, suppose you mistakenly entered the line

```
>> GoldenRatio = 0.5 * (sqr(5) – 1)
```

MATLAB responds with the error message

```
Undefined function or variable 'sqr'
```

because you misspelled the name of the `sqrt` function (see Table 2.6). Instead of retyping the entire line, press the up-arrow key <↑> once to display the previously typed line. Press the left-arrow key <←> several times to move the cursor and add the missing `t`. You can also use the *smart recall* feature to recall a previously typed function or variable whose first few characters you specify. For example, after you have entered the line starting with `GoldenRatio`, typing `Gol` and pressing the up-arrow key <↑> once recalls the last-typed line that starts with the function or variable whose name begins with `gol` (this feature is not case sensitive).

You can use the *tab completion* feature to reduce the amount of typing, too. MATLAB automatically completes the name of a function, variable, or file if you type the first few letters of the name and press the <Tab> key. If the name is unique, it is automatically completed. For example, if you have a variable `GoldenRatio` and you type `Gol` and press <Tab>, MATLAB completes the name and displays `GoldenRatio`. Then, you can either press <Enter> to display the value of the variable or continue editing to create a new executable line that uses the variable `GoldenRatio`. If the few letters you typed in are not unique (i.e., there are other functions, filenames, variables, etc. on the search path starting with the letters you typed), upon pressing the <Tab> key, a pop-up menu shows up suggesting a variety of options you can choose from (see example in Fig. 2.27a with the pop-up window in response to just two symbols, `si`, typed in).

You can also enjoy having quick, constantly available help—the **Function Browser** button, . This icon can be made always readily available to the left of the prompt as shown in Fig. 2.27b. Clicking this icon (which is equivalent to pushing <Shift>+<F1>) invokes a miniaturized version of the Help browser (also shown in Fig. 2.27b), making it much easier to find any function you are looking for. To have this button by the left edge of the Command Window, you should enable it by right-clicking anywhere within the Command Window and checking the **Show Function Browser Button** option (Fig. 2.28).

Along with using the keyboard arrow keys by themselves, you can also use them in combination with the <Ctrl> key. To move through one word at a time as opposed to one character, press and hold <Ctrl> and then press <→> to move one word to the right (<←> to move one word to the left). Press <Home> to move to the beginning of a line and press <End> to move to the end of a line. Press <Del> to delete the character at the cursor and press <Backspace> to delete the character before the cursor. Press <Esc> to clear the entire line; press and hold <Ctrl> and then press <k> to delete (kill) to the end of the line.

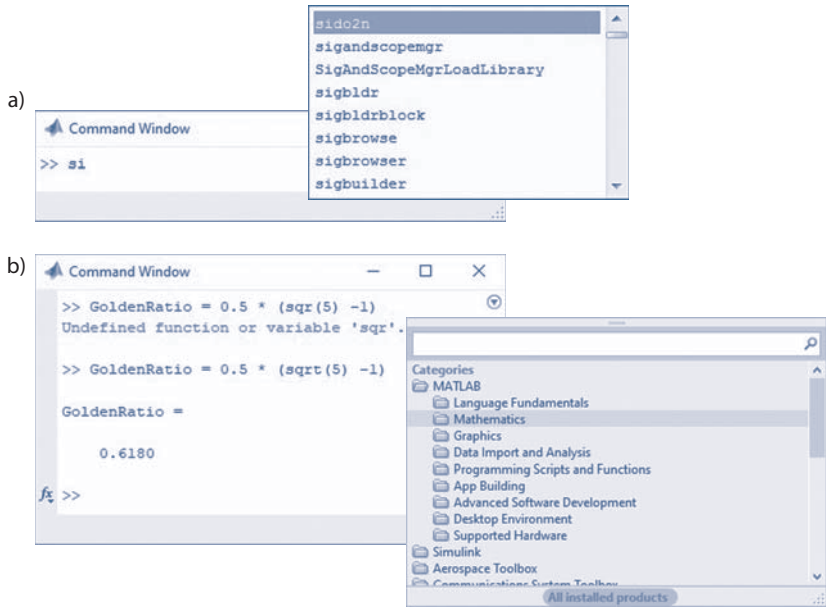


Fig. 2.27 Using a) the <Tab> key and b) the Function Browser button.

Next, Table 2.11 summarizes special symbols that may be useful when typing your commands. Once again, if a line does not end with a semicolon, MATLAB displays the result in the Command Window. That is because, the `display` function (residing in the *..\toolbox\matlab\lang* folder) is being called internally. Even if you suppress displaying the output of a command with the semicolon, MATLAB still computes and retains the variable's value in the workspace.

Fig. 2.28 Command Window right-click menu.

If, however, you want to see a value of a variable without displaing its name or compute the result of some operation and just display it without creating a new variable in the workspace, you might want to use the `disp` function (residing in the same folder as the `display` function). For instance, the command

```
>> disp(10-pi^2)
```

returns

```
0.1304
```

Table 2.11 Use of Some Special Symbols

Symbol	Effect
, (comma)	Separates expressions on one line
; (semicolon)	Suppresses printing in the Command Window
... (ellipsis)	Continues a line

You can also use the `disp` function to display a character vector or even an active hyperlink in the Command Window. For example, try

```
>> disp('Click the link below')
>> disp('<a href = "http://www.wikipedia.org">Wikipedia</a>')
```

which returns the line of text and clickable hyperlink to the Wikipedia webpage

```
Click the link below
Wikipedia
```

You can put several commands on the same line only if you separate them with a comma (if you want to see the result of each command) or semicolon (if you want to suppress displaying the results). For example,

```
>> x=2; y=6+x, x=y+7
```

returns

```
y = 8
x = 15
```

(Note that the first value of x was not displayed. Also, note that the value of x has changed from 2 to 15.)

If you need to type a long line, you can use an ellipsis (a row of three periods), to delay execution. For example,

```
>> Number_of_AC_Damaged = 2;
>> Number_of_AC_Maintenance = 15;
>> Number_of_Aircraft_Grounded = Number_of_AC_Damaged...
+ Number_of_AC_Maintenance
Number_of_Aircraft_Grounded = 17
```

Earlier in Sec. 2.2.3, we introduced three functions, `clear`, `close`, and `clc`, claiming that they are quite useful. Now, it is time to formally introduce them, along with some other functions used to manage the work session. They are shown in Table 2.12. As mentioned before, these functions allow using both the command and function syntaxes. Even though the use of these functions is intuitive, let us make a few comments.

MATLAB retains the last value of a variable until you quit MATLAB or clear its value. Overlooking this fact commonly causes errors. For example, you may choose to use a variable `x` in a number of different calculations. If

Table 2.12 Commands for Managing the Work Session

Command	Effect
`home`	Sends the cursor to its home position (the upper-left corner of the Command Window)
`clc`	Clears the Command Window and homes the cursor
`exist('name')`	Determines if a file or variable `name` exists
`who`	Lists currently existing variables in memory
`whos`	Lists the current variables and their sizes indicating if they have imaginary parts
`clear` (or `clearvars`)	Removes all variables from memory (workspace)
`clear var1 var2`	Removes variables `var1` and `var2` from memory
`clearvars -except... keepVar`	Removes all variables, except for those specified by `keepVar`
`memory`	Displays how much memory is available and how much MATLAB software is currently using
`close('name')`	Closes the named window
`close all`	Closes all the open figure windows
`pwd`	Displays the current working directory
`cd('directory')`	Sets the current working directory to `directory`
`dir` or `ls`	Lists all files in the current working directory
`what`	Lists all MATLAB files in the current directory
`which('name')`	Locates the function or file `name`
`quit`	Stops MATLAB

you forget to enter the correct value for `x`, MATLAB uses the last value, and you get an incorrect result. You can use the `clear` (or `clearvars`) function to remove the values of all variables from memory, or you can use the form specifying what variables to clear specifically. For example,

```
>> clear var1 var2
```

(or `clearvars var1 var2`) clears just two variables named `var1` and `var2`. (There is some difference between the `clear` and `clearvars` functions, but it is not essential for this discussion.) The `clear all` call used in Fig. 2.9b clears more than just all variables from the current workspace and is excessive in our context, but we still use it to be consistent with the `close all` call.

The effect of the `clc` command is different. It clears the Command Window of everything in the window display, but the values of the variables remain in the memory (workspace). After using `clc`, you cannot use the scroll bar to see previously displayed text, but you can still use the up arrow (↑) to recall statements from the Command History window. In addition to that, the `home` command also moves the cursor to the upper-left corner of the Command Window and clears the visible portion of the

window. However, as opposed to the `clc` command, you can still use the scroll bar to see what was displayed within the screen before.

You know already that typing the name of a variable and pressing <Enter> results in displaying its current value. If, however, the variable does not have a value, that is, if it does not exist, you will see an error message. To avoid getting an error of this sort, when writing code, it sometimes makes sense to check whether the variable you are trying to manipulate exists. If you type

```
>> exist('x')
```

to see whether the variable `x` is in use, and you get a `1` as a return, then the variable exists. A `0` indicates that it does not exist.

The `who` function lists the names of all the variables in memory, but it does not give their values. The `who var1 var2` call restricts the display to only those two variables specified. The wildcard character `*` can be used to display variables that match a pattern. For instance, `who N*` finds all variables in the current workspace that start with `N`. If variables from the previous example were still in workspace, then the `who N*` command would return

```
Your variables are:
Number_of_AC_Damaged    Number_of_Aircraft_Grounded
Number_of_AC_Maintenance
```

The `whos` function lists the variable names in the currently active workspace along with their size and class. Other syntaxes allowing you to specify what information you would like to display are also available.

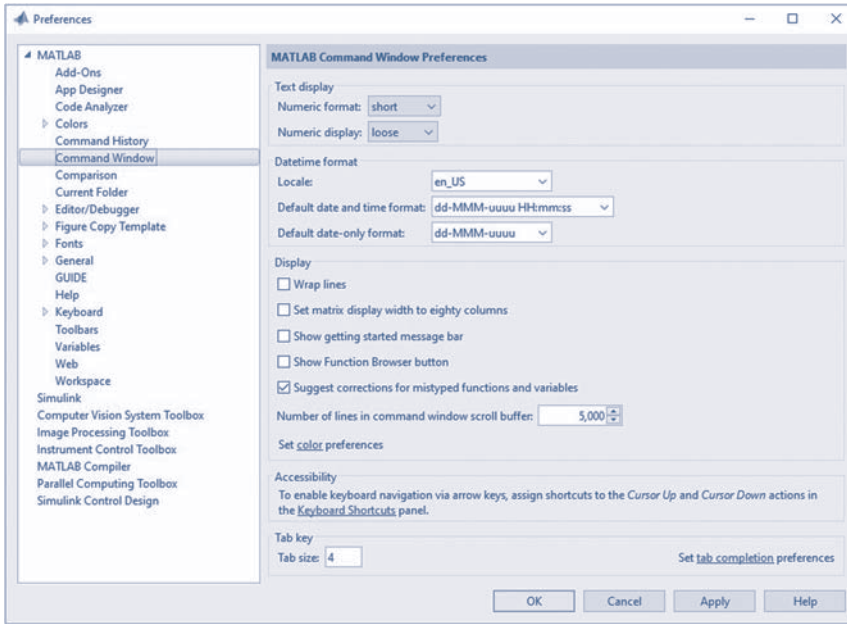
You can quit MATLAB by typing `quit`. Alternative ways are holding the <Ctrl> key and pressing the <Q> key or simply clicking the standard Window's Close icon in the upper-right corner of the MATLAB Desktop.

2.7 Changing Preferences

Quite often throughout this chapter, we have talked about the default settings or default values. To match your personal preferences, MATLAB allows you to change any of them. To access these settings, click the **Preferences** icon of the Environment group of the Home tab toolstrip, . To familiarize yourself with this tool, we present some screens available within the **Preferences** dialog box. As usual, there is an alternative way to open the **Preferences** window programmatically, by calling the corresponding `preferences` command.

Depending on which window is active when you click this icon, you may see something like what is shown in Fig. 2.29 for the Command Window and Command History window, which gives you an idea of what parameters can be modified. For instance, in the Command Window preferences (Fig. 2.29a), you can change the numeric format and display the same way the `format`

a)



b)

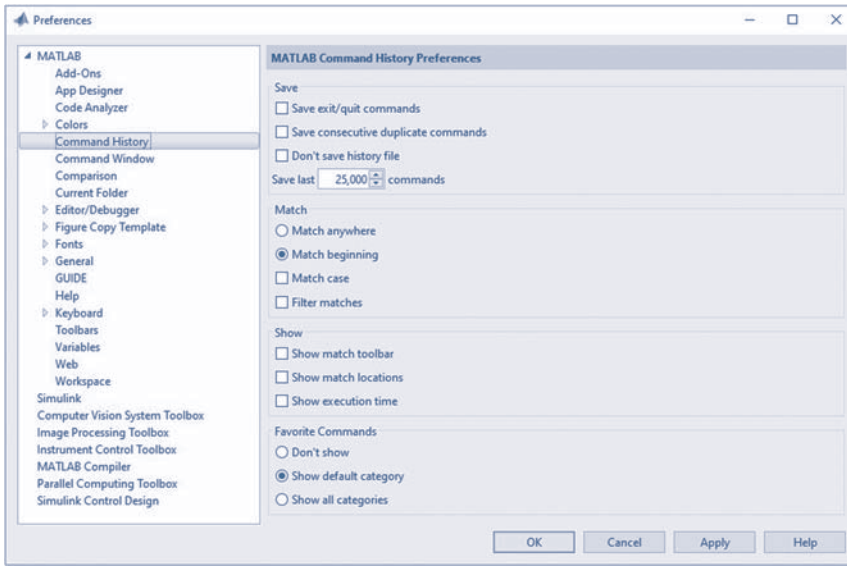
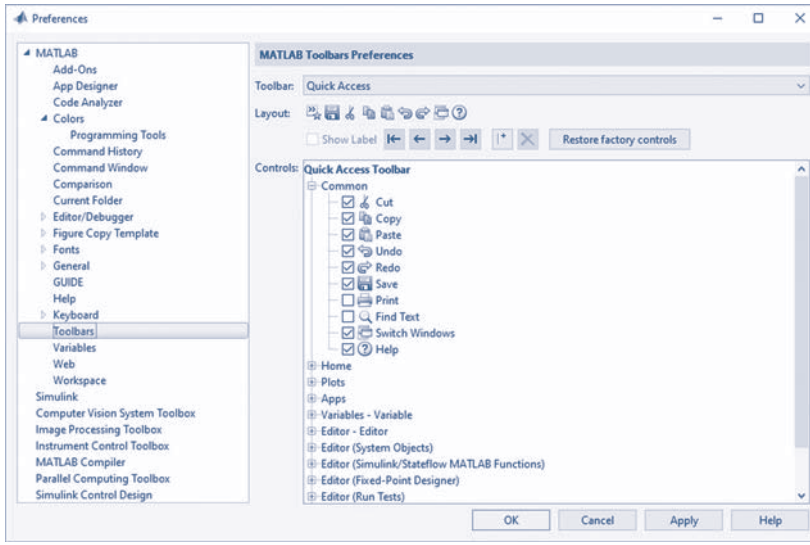


Fig. 2.29 Preferences for a) Command Window and b) Command History.

function would do it programmatically, as described in the previous section. From Fig. 2.29b, you can learn that 25,000 is the default number of commands stored in the Command History window stores by default (25,000).

Figure 2.30a explores the Toolbars preferences, showing why your Desktop may look different from that presented throughout this chapter. You can populate any toolbar of the MATLAB development environment

a)



b)

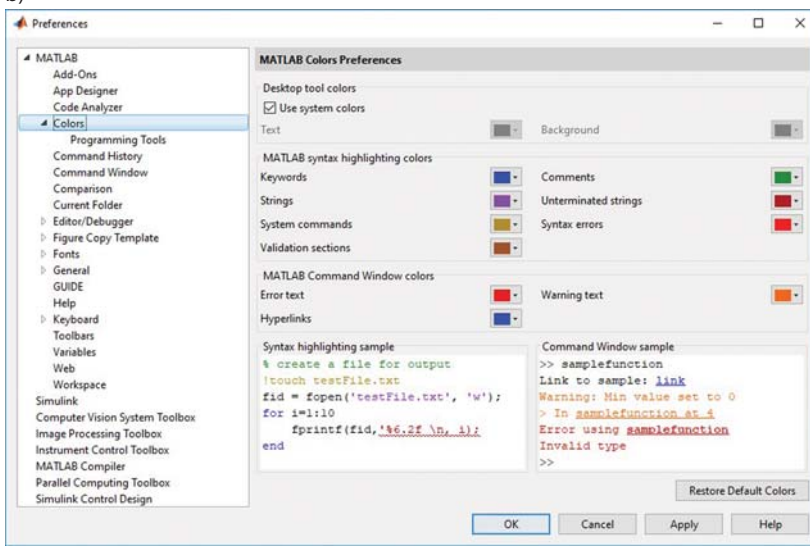
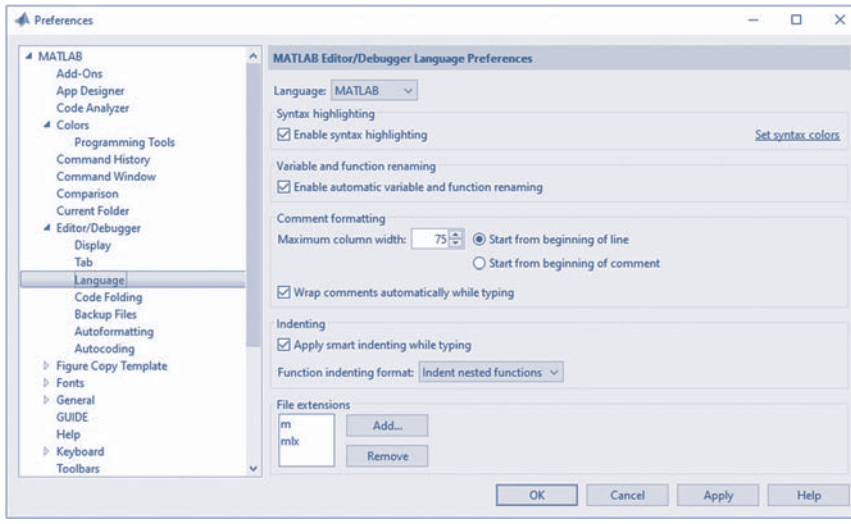


Fig. 2.30 Preferences for a) Toolbars and b) MATLAB Colors.

with the icons you want. Figure 2.30b shows Colors preferences, explaining the rules for applying the highlighting colors.

Figure 2.31 shows examples of available changeable options from the expandable Editor/Debugger preferences list. As seen from Fig. 2.31a, you can, for example, change the maximum number of symbols per line/column width (although we would not recommend going beyond 82 symbols to avoid problems with printing your scripts on a Letter-size paper). Figure 2.31b shows a screen where you might want to make sure that the **Automatically delete backup files when the Editor closes** checkbox is checked. When you open the Editor, MATLAB creates and

a)



b)

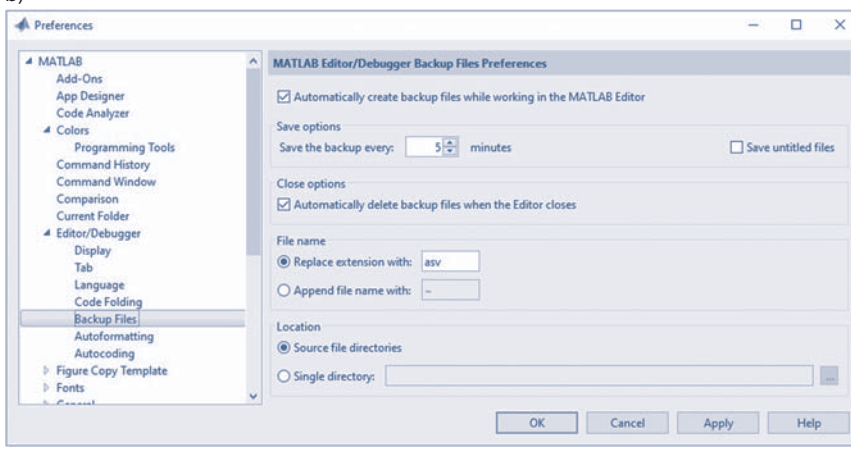
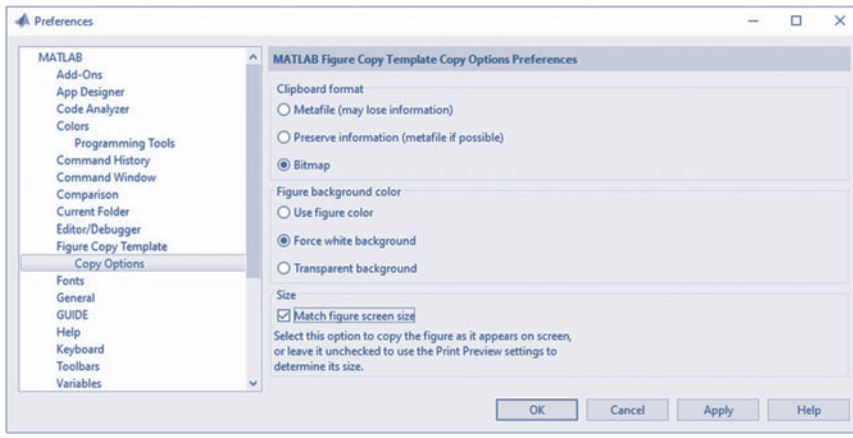


Fig. 2.31 Editor/Debugger a) Language and b) Backup Files preferences.

periodically saves an *autosave file* to avoid loosing your code in the case of power outage or MATLAB crash. Unless this checkbox is checked, these backup files will not be deleted upon exiting MATLAB, and pretty soon you will find your working directory filled with *asv*-files. Of course, that assumes that you will want to save your files before exiting if you really need them or that you have no intentions of saving these files anyway if you choose not to save them.

Another thing you may want to check and change, if appropriate, is settings for figure copying (to be discussed further at the end of Sec. 6.11). The recommended options are shown in Fig. 2.32a. Figure 2.32b shows how to change the default array format in the Variables Editor. As seen, this format may be different from that of the Command Window (Fig. 2.29a).

a)



b)

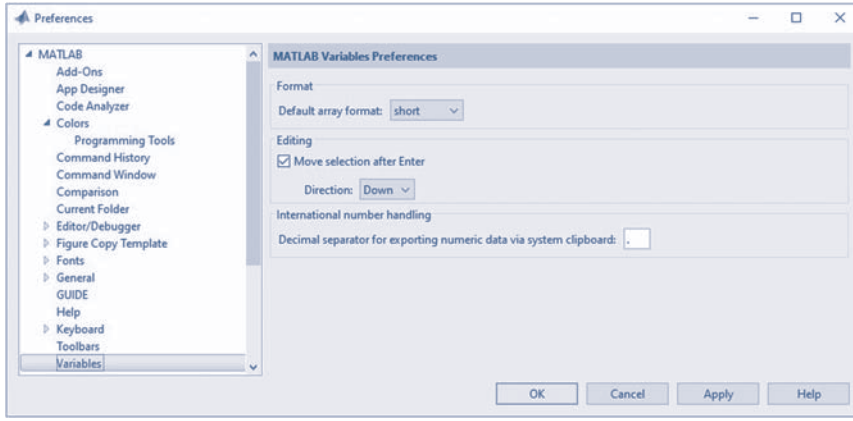


Fig. 2.32 Preferences for a) Figure Copy Options and b) Variables Editor.

2.8 Problem-Solving Methodology

To conclude this chapter, let us introduce a collective list of common steps involved in solving a problem on a computer. It might be more appropriate later in this textbook, when you have enough knowledge to write sophisticated programs, but introducing it as early as possible can eliminate a lot of troubles when writing even the simplest pieces of code, making programming enjoyable and rewarding. The engineering problem-solving methodology assumes that you stick to the following routine:

1. Think about the main goal you want to achieve.
2. Define required inputs and desired outputs.
3. Collect all necessary information.
4. Start with a simplified version of the problem and clearly state your assumptions.
5. Draw a sketch of the problem and think of the names of your variables.
6. Check the dimensions and units of your variables.
7. Think about how you are going to proceed toward a solution and write down the general steps, maybe even labeling them (which will further be converted to the comments line in your code).
8. Make sure your program is robust enough to handle a variety of the expected and unexpected (wrong or missing) inputs as well as interface properly with any applications you use.
9. Start writing code, checking its pieces if appropriate by displaying intermediate results as needed.
10. Devote some time to developing a concise, yet informative output structure, which later on will include producing informative graphics.
11. Check the outputs of your program on some specific cases with the known (expected) results.
12. Perform a "reality check" of the outputs your program produces in a general case.
13. If your algorithm involves some iterative computations (such as optimization), try to limit the number of cycles first to be sure your program works properly.
14. Do not start batch computing unless you are absolutely sure your program runs properly for a single data set (having lots of pretty output does not necessarily mean it is not garbage).
15. After debugging your program, make sure to suppress displaying all unnecessary intermediate results, so that your program produces only those outputs you actually need.

Follow these rules closely at least once or twice to develop healthy programming habits. These habits will help you become a proficient applied programmer and make you an efficient scientist or engineer.

Problems

2.1 Calculate what speed it would take your model aircraft, equipped with a wing having, say, $S = 4\ \text{m}^2$ area to get enough lift to take off. Assume that the total mass is $m = 100$ kg, air density – $\rho = 1\ \text{kg/m}^3$, gravitation due to gravity – $g = 9.8\ \text{m/s}^2$, and lift coefficient – $C_L = 2$, and use the following formula:

$$V = \sqrt{\frac{2mg}{C_L \rho S}}$$

2.2 For a cruising aircraft, thrust T equals drag D and lift L equals weight. Mathematically, it means

$$T = D = C_D \frac{\rho V^2}{2} S \quad \text{and} \quad L = C_L \frac{\rho V^2}{2} S = mg$$

where, in addition to notations of Problem 2.1, C_D denotes the drag coefficient. Combining them, you can write $T = mgC_D/C_L$. Flying dt seconds with thrust T and specific fuel consumption c_T (a ratio of consumed fuel per unit of thrust per unit of time), aircraft burns $dm = c_T T dt$ of fuel and covers $dR = Vdt = V/(c_T T)dm$ distance. Hence, we can write a general range equation as

$$R = \int_{m_0}^{m_f} \frac{V}{c_T} \frac{C_L}{C_D} \frac{1}{mg} dm$$

where m_0 and m_f are the aircraft's initial and final masses, respectively.

Assuming constant altitude (ρ) and angle of attack (C_L and C_D), integration of this equation (with the expression for V taken from Problem 2.1) yields

$$R = \frac{2}{c_T C_D} \sqrt{\frac{2C_L}{\rho S g}} \left(\sqrt{m_0} - \sqrt{m_f} \right)$$

and results in a gradual decrease of cruise speed. Alternatively, for constant speed and angle of attack integration yields the well-known Breguet range equation

$$R = \frac{V}{g c_T} \frac{C_L}{C_D} \ln \frac{m_0}{m_f} = \frac{1}{c_T C_D} \sqrt{\frac{2C_L m_0}{\rho_0 S g}} \ln \frac{m_0}{m_f}$$

and causes the aircraft to climb.

Estimate both range equations using the following values: $C_L = 0.4$, $C_D = 0.04$, $c_T = 8 \times 10^{-6}\ \text{kg/N/s}$, $S = 16\ \text{m}^2$, $m_0 = 1000$ kg, $m_f = 900$ kg, $\rho = \rho_0 = 0.6\ \text{kg/m}^3$. Compare the results.

2.3 Use MATLAB Editor to develop and run your first *m*-script allowing you to define the weekday based on its date. Employ the formula derived by the German clergyman Reverend Christian Zeller for the Gregorian calendar:

$$WD = \left(D + \left[\frac{1}{5}(13m - 1)\right] + Y + \left[\frac{Y}{4}\right] + \left[\frac{c}{4}\right] - 2c\right) \bmod 7$$

This formula computes the weekday as the remainder of the Euclidean division of an expression by 7 (use the `mod` function for modulo operation and the `fix` function for computing the integer parts of the expressions enclosed in square brackets). In this formula, d is the day of month (1, 2, . . ., 31), m is the month's number in the Ancient Roman system ($m = 1$ for March, $m = 2$ for April, etc., starting with March as 1 and ending with February as 12; i.e., January and February of any year are considered to be in the previous year), Y is the number of the year in a century (0–99), and c is the number of centuries. For example, for 22 January 1963, the inputs would be $d = 22$, $m = 11$, $Y = 63$, and $c = 19$. The value WD defined using the preceding formula is treated as follows: "0" means Sunday, "1" means Monday, "2" means Tuesday, etc.

To make your script more useful, develop it as an interactive program. The following provides an example of all but one line (which you need to develop yourself):

```
% This is my first m-script
clear all, clc
  Year =input('\nEnter the year (YYYY): ');
  Month=input('Enter the month: ');
  Day =input('Enter the day: ');
  fprintf('n\You entered %3.0f/%2.0f/%4.0f',Month,Day,Year)
d=Day;
  m=Month-2;
  if  m <= 0
   m=m+12; Year=Year-1;      % month's number in Ancient Rome
  end
Y=mod(Year,100);            % year's number in the century
c=(Year-Y)/100;             % the number of centuries
   WeekDay=__________;      % HERE GOES YOUR FORMULA
Week={'Sunday' 'Monday' 'Tuesday' 'Wednesday' 'Thursday'...
                                        'Friday' 'Saturday'};
fprintf('\n\nYour weekday is: %s\n', Week{WeekDay+1}); % the result
```

Determine the weekday when Sputnik 1 was launched (4 October 1957), when Yuri Gagarin became the first human in outer space (12 April 1961), and when Neil Armstrong set his boot on the surface of the Moon (21 July 1969).

Chapter 3 Arrays and Array Operations

- Classification of Arrays and Their Indexing
- Standard and Nonstandard Matrix and Array Operations
- Handling Polynomials Using the Vectors of Their Coefficients
- Handling Text as the Character Vectors and String Arrays

`linspace`, `logspace`, `cat`, `diag`, `zeros`, `ones`, `rand`, `eye`, `magic`, `pascal`, `rand`, `randn`, `randi`, `rng`, `repmat`, `randperm`, `circshift`, `ndims`, `size`, `length`, `numel`, `reshape`, `spy`, `imagesc`, `cross`, `dot`, `sort`, `sortrows`, `flip`, `rot90`, `min`, `max`, `median`, `sum`, `cumsum`, `find`, `poly`, `roots`, `conv`, `residue`, `poly2sym`, `sym2poly`, `char`, `string`, `num2str`, `str2num`, `int2str`, and others

3.1 Introduction

To acquaint you with the MATLAB development environment, Chapter 2 used simple arithmetic involving some basic operations with single variables (scalars). Now it is time to explore the MATLAB commands in more depth. As mentioned in Chapter 1, one of the MATLAB's strengths is its capability to handle collections of numbers, called arrays, as if they were single variables. For example, to add two same-size arrays, `A` and `B`, you can issue a single command, `C=A+B`, and MATLAB will add up all corresponding elements in `A` and `B` to produce `C`. In most other programming languages, this operation would require more than one line of code. Even if, up to this point, Microsoft Excel remained your favorite tool for data analysis, upon completion of this chapter you may change your mind—you will find MATLAB to be a much easier and more powerful instrument for such work, making it a natural choice for addressing a variety of engineering problems. With an array being the basic building block in MATLAB, this chapter explains how to create, address, and edit different types of arrays, and how to use standard array operations. It also introduces nonstandard array operations and some specific commonly used array-handling functions. Finally, this chapter explains how MATLAB array formalism is used to handle polynomials and textual data.

3.2 Types of Arrays and Indexing Their Elements

By definition, an *array* is a group of elements (values or variables) that are related and can be referred to by one or more *indices*. Such a collection is usually called an array variable or simply an array. For example, the airport weather stations provide hourly weather data so that hourly temperature readings on any specific day can be stored as a one-dimensional (1-D) array, composed of 24 values of temperature. Such an array is also referred to as a *vector*. If, along with a temperature, you would like to store other parameters, like air density, dew point, visibility, etc., totaling n variables per hour, you will need to add another dimension, that is, you will end up dealing with a 2-D array holding $24\times n$ variables.

Now, suppose you need to store these data for the first day of each month. Adding another dimension enables storing, your $24\times n\times 12$ pieces of data in a 3-D array. So far, we talked about only one airport, but what if we are gathering data from 100 airports? To effectively handle $24\times n\times 12\times 100$ values, you will now need a 4-D array. For storing this type of data for 10 consecutive years, you may need a 5-D array, $24\times n\times 12\times 100\times 10$, and so on.

By analogy with the mathematical concepts of vector and matrix, an array type with one or two dimensions is often called a *vector* type or *matrix* type, respectively. The MATLAB classification of arrays featuring two broad classes of arrays—2-D arrays or matrices and multidimensional (n-D) arrays—is shown in Fig. 3.1. In this case, the vectors (traditionally, 1-D arrays) and even scalars are considered as subclasses of the 2-D arrays. (Figure 3.1 also shows some other subclasses of matrix to be also discussed in this chapter.)

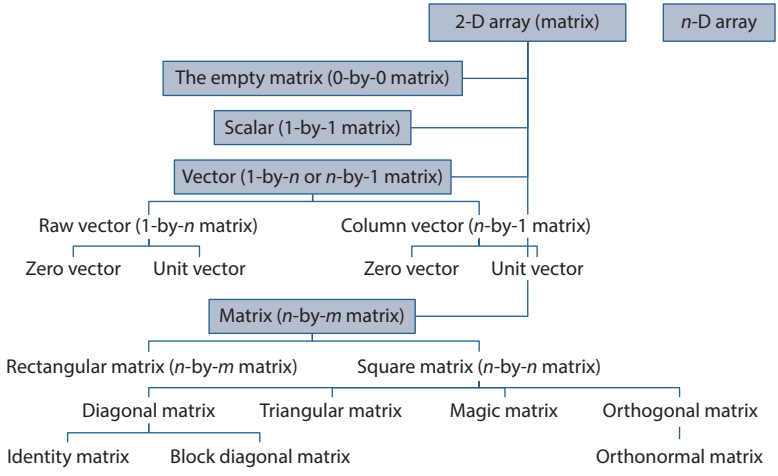


Fig. 3.1 MATLAB classification of arrays.

3.2.1 Matrices and Multidimensional Arrays

The first broad class of arrays in MATLAB is the 2-D arrays or matrices exemplified in Fig. 3.2. A matrix composed of n rows and m columns is called an n-by-m matrix, so that these numbers, n and m, define a *size* of the matrix along each *dimension*. For example, the matrix shown in Fig. 3.2 is the 4-by-6 matrix and its symbolic representation looks like follows:

$$A = [a_{ij}] = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} & a_{15} & a_{16} \\ a_{21} & a_{22} & a_{23} & a_{24} & a_{25} & a_{26} \\ a_{31} & a_{32} & a_{33} & a_{34} & a_{35} & a_{36} \\ a_{41} & a_{42} & a_{43} & a_{44} & a_{45} & a_{46} \end{bmatrix} \tag{3.1}$$

The first subscript, i, changes in the vertical direction (refers to a *row* number) and the second subscript, j, changes in the horizontal direction (refers to a *column* number).

A vector can be regarded as a special case of matrix, where the size of one of the dimensions equals 1. A row vector, with its elements stored "horizontally," is considered to be a 1-by-n matrix, whereas a column vector with one element following another in a column format is considered to be an n-by-1 matrix, as shown in Fig. 3.3.

From Fig. 3.1, it follows that MATLAB scalars are treated as 1-by-1 matrices. For the sake of consistency, the empty (0-by-0) matrix is introduced. The command

```
>> A = [];
```

uses the square bracket operators with no value specified to create a 0-by-0 matrix. The 0-by-0 matrix concept happens to be very helpful when the entire row(s) and/or column(s) need to be deleted, and it will be discussed later.

Figure 3.4 presents an example of a 3-D array, where elements are organized as pages (slices or layers). Geometrical interpretation for arrays with a

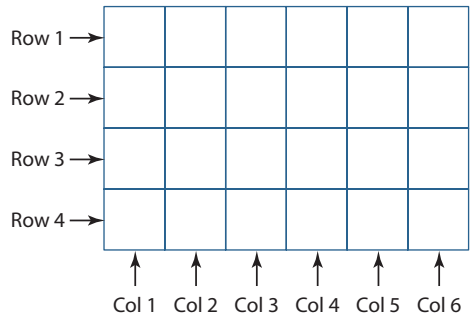


Fig. 3.2 Example of a 4-by-6 matrix.

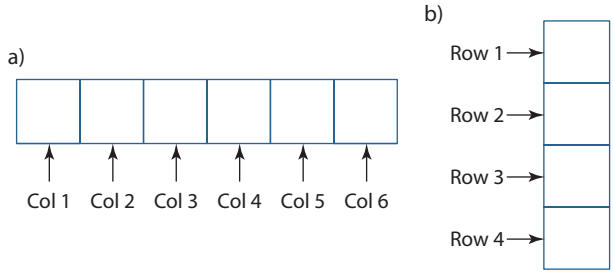


Fig. 3.3 Examples of a) row and b) column vectors.

higher number of dimensions is not possible (we live in three dimensions ourselves), but you may consider a 4-D array to be a collection of several 3-D arrays, a 5-D array to be a collection of 4-D arrays, and so on.

3.2.2 Addressing Array Elements

Storing ordered data in arrays has a huge advantage because you can easily access any specific piece of a data set. Addressing the elements, and groups of elements, in the arrays is quite evident. Table 3.1 provides with some examples of addressing the elements of a 24-element 1-D array `x`.

These six examples cover all possible ways of constructing an *indexing expression* to address array elements by their indices $\text{ind}_1, \ldots, \text{ind}_{\text{last}}$ (positive integers). The general format to compute these indices is

```
first : increment : last
```

with the index of the first element to address, ind_1, being `first`; the next one, ind_2, `first+increment`; followed by ind_3, being `first+2*increment`; etc., all the way up (or down) until $\text{ind}_{\text{last}} \leq$ `last` (in the case of positive `increment`) or $\text{ind}_{\text{last}} \geq$ `last` (in the case of negative `increment`). The `first` and `last` entries must be positive integers (1, 2, 3, ...), `increment`—a nonzero integer. The default value of

Table 3.1 Indexing the Elements of a 24-element 1-D Array

Syntax	Result
`x(:)`	Addresses the entire row (or column) of vector `x`, that is, all 24 elements
`x(2:6)`	Addresses five elements, with the indices 2–6, that is, `x(2)`, `x(3)`, `x(4)`, `x(5)`, and `x(6)`
`x(20:end-2)`	Addresses three elements, `x(20)`, `x(21)`, and `x(22)`
`x(2:5:20)`	Addresses four elements, `x(2)`, `x(7)`, `x(12)`, and `x(17)`
`x(6:-2:2)`	Addresses three elements, `x(6)`, `x(4)`, and `x(2)`
`x([1:3,5,22])`	Addresses five elements, `x(1)`, `x(2)`, `x(3)`, `x(5)`, and `x(22)`

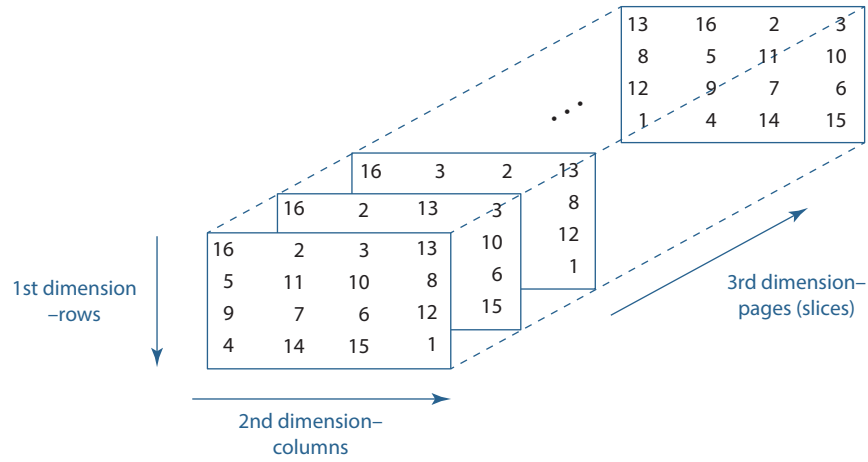


Fig. 3.4 Example of a 3-D array.

`increment` (if it is omitted, like in all but the 4th and 5th examples of Table 3.1) is 1. The `first` and `last` entries may be omitted as well, and in this case the list of indices includes all of the available range (the 1st syntax of Table 3.1). The function `end` serves as the last available index and can be used by itself or in algebraic expression (the 3rd syntax of Table 3.1). Obviously, the following inequalities should hold: `first` $\leq$ `last` (in the case when `increment` > 0) and `first` $\geq$ `last` (in the case when `increment` < 0). That being said, any of the following addressing

```
x(4:4)
x(4:2:4)
x(4:-1:4)
```

is totally acceptable and returns just a single element, `x(4)`. The next three

```
x(6:2:5)
x(8:0:10)
x(20: -1:22)
```

are acceptable (although make no sense), returning an empty 0-by-0 matrix. However, the following two

```
x(-2)
x(25)
```

return an error notifying that the index exceeds matrix dimensions.

The syntax employing brackets (the last example in Table 3.1) allows you to address a group of irregularly spaced elements. As shown in the previous example, a regular indexing expression can be a part of such addressing as well. Expressions in the brackets can be separated by a space, comma, or semicolon.

Table 3.2 Indexing the Elements of the 2-D and 3-D Arrays

Syntax	Result
`A(:,4)`	Addresses all the elements in the fourth column of a matrix `A`
`A([2 5],1:4)`	Addresses the elements of the first through fourth column in the second and fifth rows of a matrix `A`
`A(end,end-1)`	Addresses the last but one element in the last row of a matrix `A` (note that, the `end` function computes the last available index for each dimension separately)
`B(2,5,:)`	Addresses the fifth element in the second row on all pages of a 3-D array `B`

The same rules apply to arrays with more than just one dimension as well. The only difference is that you need to provide an indexing expression for each dimension, separated by comma(s) (Table 3.2),

Keeping in mind that a vector can be regarded as a special case of a matrix, where one of the dimensions is equal to 1 (Fig. 3.1), the addressing `x(5)`, the so-called *linear indexing*, is equivalent to `x(1,5)` for the row vector, and addressing `x(3)` is equivalent to `x(3,1)` for the column vector (Fig. 3.5). This latter type of indexing uses *row–column subscripts* (row–column–page for the 3-D arrays, etc.).

In computer's memory, arrays are stored not in the shape they appear when displayed in the MATLAB Command Window, but as a single column of elements, one element following another, with the indices of the leftmost dimension cycling the fastest. It means that in the case of matrix, the elements are stored column by column (the columns index cycles the first), each appended to the previous one. In the case of a 3-D array, they are stored page by page with each page stored by columns, and so on. Hence, MATLAB permits linear indexing even for multidimensional arrays. For instance, the easiest way to address the element located in the `i`th row and `j`th column of an `n`-by-`m` matrix `A` is obviously `A(i,j)`. However, if needed, you may also use its linear indexing equivalent `A(n*(j-1)+i)`. For the example shown in Fig. 3.6, addressing `A(3,2)` is equivalent to that of `A(7)`.

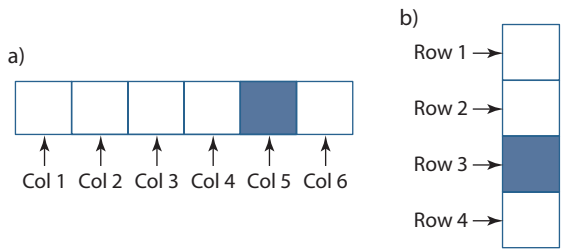


Fig. 3.5 Addressing elements of a) row and b) column vectors.

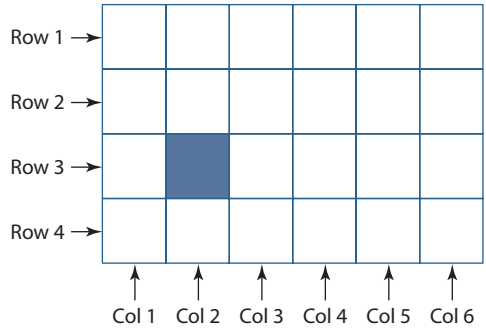


Fig. 3.6 Addressing elements of matrix `A`.

The two useful MATLAB functions, `sub2ind` and `ind2sub`, help to convert subscripts to linear index and vice versa. For example, for the case shown in Fig. 3.6, the command

```
>> i=sub2ind(size(A),3,2)
```

returns `i=7`, and the command

```
>> [r,c]=ind2sub(size(A),6)
```

returns `r = 2` and `c = 2`. These functions are applicable to multidimensional arrays as well (you simply increase the number of input/output arguments as appropriate).

Figure 3.7 summarizes different ways of addressing array elements and presents some more self-explanatory examples.

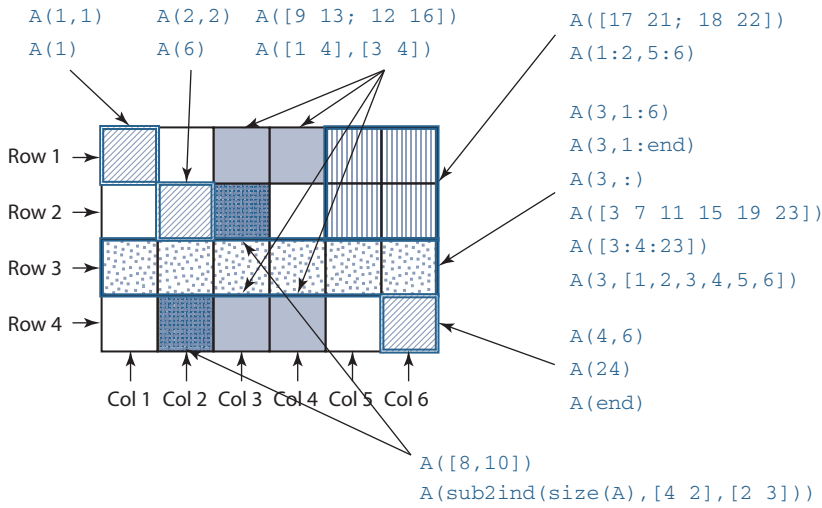


Fig. 3.7 Examples of matrix element(s) indexing.

3.2.3 Creating Arrays

Usually, you only create and fill in simple vector arrays, while the larger arrays are obtained as a result of computations. So let us proceed with a couple of simple examples of creating arrays manually and then introduce several useful functions simplifying this process.

Let us start with creating the vectors. For example, the command `a=[1,2,3]` or `a=[1 2 3]`, where comma(s) or space(s) are used to separate elements, creates the same 1-by-3 row vector `a`:

```
>> a = [1,2,3]
a = 1   2   3
```

The command `b = [4;5;6]` (where a semicolon is used to separate elements) or `b = [4,5,6]'` (where the apostrophe is used to transpose a row vector) creates the same 3-by-1 column vector `b`:

```
>> b=[4;5;6]
a = 4
    5
    6
```

The following two commands create examples of the *unit vectors* (with a vector magnitude or, to be more precise, Euclidian norm being 1):

```
>> c = [0, 1, 0]
c = 0   1   0
>> d = [sqrt(2)^-1; sqrt(2)^-1]'
d = 0.7071   0.7071
```

In the previous examples, we used a comma or space to separate elements in a row and a semicolon to separate them in a column. On top of that, we used a transpose operation to transpose a row vector to a column vector and vice versa. Now we can use the same syntax to create matrices. For example, the same 3-by-2 matrix `C` can be created by issuing any of the following commands: `C = [1,4;2,5;3,6]`, `C = [1,2,3;4,5,6]'`, `C = [1:3;4:6]'`, `C = [a',b]`, and `C = [a;b']'` (in the latter two cases, we would be using the vectors `a` and `b` created earlier):

```
>> C = [1:3;4:6]'
C = 1   4
    2   5
    3   6
```

If it is more convenient, you may also enter elements of this matrix row by row—MATLAB will recognize what you are trying to do:

```
>> C = [1   4
        2   5
        3   6];
```

(you will need to end each line hitting the <Enter> key).

Similarly, to create the first two pages of the 3-D array shown in Fig. 3.4, each page must be assigned individually. For example,

```
>> A(:,:,1) = [16,2,3,13;5,11,10,8;9,7,6,12;4,14,15,1]
>> A(:,:,2) = [16,2,13,3;5,11,8,10;9,7,12,6;4,14,1,15]
```

yields

```
A(:,:,1) = 16     2     3    13
            5    11    10     8
            9     7     6    12
            4    14    15     1
A(:,:,2) = 16     2    13     3
            5    11     8    10
            9     7    12     6
            4    14     1    15
```

Then, you can duplicate these pages as follows:

```
>> A(:,:,3:4) = A(:,:,1:2);
```

In addition to inputting elements or values manually, as shown earlier, several other methods can be used. Implicitly, we have already introduced one of them when discussing indexing. The simplest way to create the regularly spaced vectors is to use a colon operator. Consider the following examples:

```
>> x = 1:10
x = 1   2   3   4   5   6   7   8   9   10
>> y = -1:0.1:1
y = Columns 1 through 8
   -1.0000  -0.9000  -0.8000  -0.7000  -0.6000  -0.5000  -0.4000  -0.3000
   Columns 9 through 16
    ...
```

Note that you can also use syntax with brackets, like `x = [1 : 10]`, which is especially useful if you want to create a column vector:

```
>> x = [1 : 2.5 : 10.2]'
x = 1.0000
   3.5000
   6.0000
   8.5000
```

Obviously, all rules introduced in Sec. 3.2.2 apply, and the only difference is that all three values, the beginning value, increment, and last value, can be given as real numbers, not necessarily integer numbers as it was the case for indexing.

Figure 3.8 shows the Workspace window with all arrays created so far in this section. As seen, to store each element of an array, MATLAB dedicates eight bytes of memory (by default). For instance, to store $4 \times 4 \times 4 = 64$ elements of the array `A`, 512 bytes are required. Also, Fig. 3.8 features some additional columns of the Workspace window, allowing you to

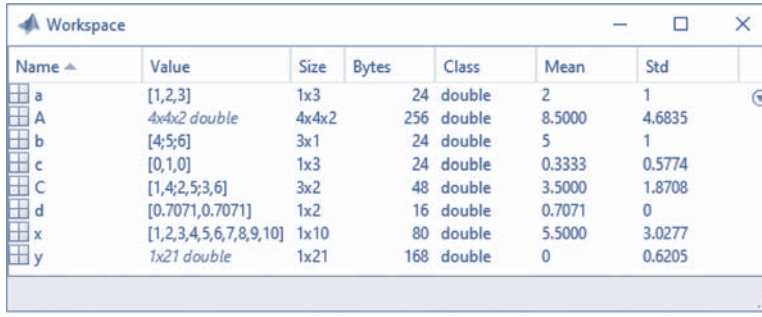


Fig. 3.8 Workspace with a variety of arrays.

compute a mean value and standard deviation for each array (compare it with two of those given in Fig. 2.15a).

An alternative way of doing essentially the same as a pair of brackets is to use the `linspace` function. Specifically, the `linspace(a,b,n)` call returns `n` numbers equally spaced between `a` and `b`. For example,

```
>> xlin = linspace(1,100,30);
```

creates a row vector with 30 elements spread from 1 to 100. The number of points `n` can be omitted resulting in a default number of 100. The increment is computed internally as

$$\text{incr} = \frac{\text{b} - \text{a}}{\text{n} - 1} \tag{3.2}$$

Another command to create a vector with regularly spaced elements, but in the logarithm base—10 domain, is `logspace`. Syntax `logspace(a,b,n)` creates `n` logarithmically spaced points between 10^{a} and 10^{b} with the logarithmic increment computed internally using the same equation (3.2). In the physical domain, it means a geometric progression, i.e. a sequence of numbers where each term after the first is found by multiplying the previous one by a fixed number, the common ratio

$$\text{comRatio} = 10^{\text{incr}} \tag{3.3}$$

For example,

```
>> xlog = linspace(0,2,30);
```

creates a row vector with 30 elements spread from 1 (10^0) to 100 (10^2). The number of points `n` can be omitted assuming the default value of 50. For better understanding, the distribution of elements in both vectors, `xlin` and `xlog`, is shown side by side in Fig. 3.9.

Once you have some vectors and matrices created, you may use a variety of the functions to modify them. For instance, (block) diagonal matrices

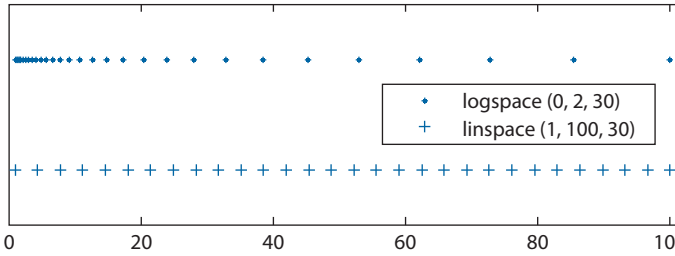


Fig. 3.9 Distribution of 30 elements in the vectors generated using the `linspace` and `logspace` functions.

and diagonals of matrix can be created using the functions of Table 3.3 (see illustrations in Fig. 3.10).

When creating the matrix `C` in the beginning of this section, we introduced a couple of simple examples of how you could concatenate two vectors into a matrix: `C = [a',b]` and `C = [a;b']'`. Note that we had to use a transpose operation to bring both vectors to the same form (column vector or row vector). Moreover, it worked only because they had the same number of elements. In the general case, to append one matrix to another, MATLAB offers several special concatenation functions: `cat`, `vertcat`, and `horzcat`.

By definition, `cat(n,A,B,C,...)` creates a new array by concatenating the arrays `A`, `B`, and `C` along the `n`th dimension (their length in the corresponding dimension should match). The `vertcat` and `horzcat` functions are subsets of the `cat` function and simplify things in the case of matrices as follows (see a graphical illustration in Fig. 3.11):

`vertcat(A,B)` Same as `cat(1,A,B)` and equivalent to `[A;B]`
`horzcat(A,B)` Same as `cat(2,A,B)` and equivalent to `[A,B]`

Table 3.3 Functions for Creating Diagonal Arrays

Function	Brief Description
`diag(v)`	Puts elements of the vector `v` on the main diagonal of a *square* matrix, that is, the matrix that has the same number of rows and columns
`diag(v,k)`	Puts `n` elements of the vector `v` on the `k`th diagonal of a square matrix of the order `n+abs(k)` (for the main diagonal `k` = 0)
`diag(X)`	Returns a column vector formed by elements of the main diagonal of matrix `X`
`diag(X,k)`	Returns a column vector formed by elements of the `k`th diagonal of matrix `X`
`blkdiag(A,B,C,...)`	Outputs a block diagonal matrix, constructed from matrices `A, B, C, ...`
`isdiag(X)`	Determines if matrix `A` is diagonal

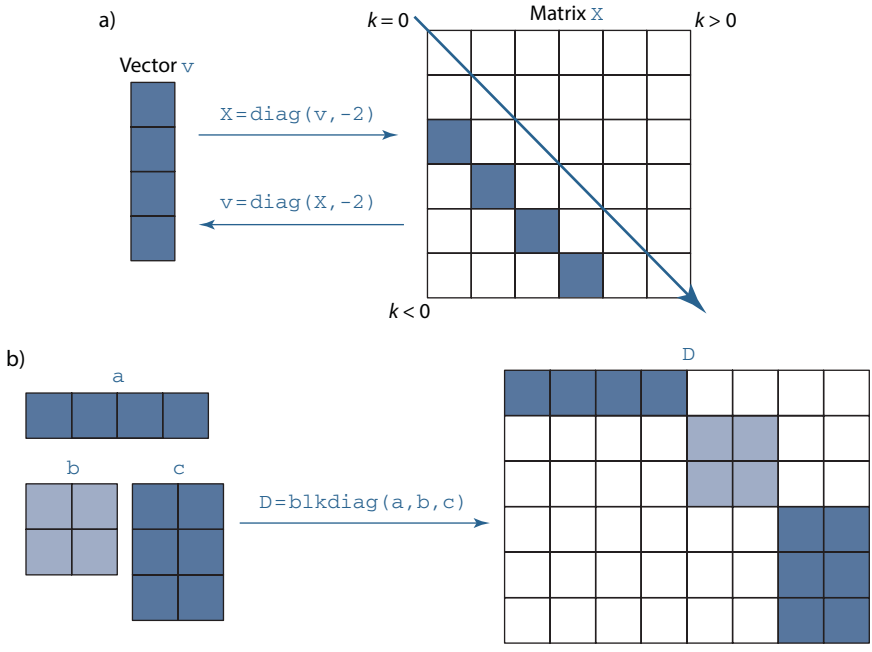


Fig. 3.10 Illustration of a) `diag` and b) `blkdiag` functions.

MATLAB also features several useful functions to create special arrays (Table 3.4). Examples of using these functions are shown in Tables 3.5 and 3.6. Note that the functions `eye`, `magic`, and `pascal` are only applicable to create matrices, whereas the remaining functions can also be used to produce multidimensional arrays (applying an obvious syntax change). A variety of additional specialized test matrices (like Cauchy, Chebyshev, Krylov, Toeplitz, Vandermonde, and others) is also available using the `gallery` function of MATLAB.

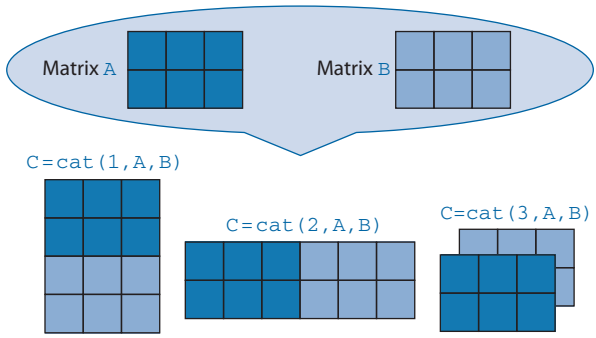


Fig. 3.11 Illustration of the `cat` function options.

Table 3.4 Functions for Creating Special Arrays

Function	Brief Description
`ones(m,n,p,...)`	Creates an `m`-by-`n`-by-`p`-by-... array of ones (`ones(n)` creates a square `n`-by-`n` matrix of ones)
`zeros(m,n,p,...)`	Creates an `m`-by-`n`-by-`p`-by-... array of zeros (`zeros(n)` creates a square `n`-by-`n` matrix, called a *null matrix* and denoted by **0**)
`eye(m,n)`	Creates an `m`-by-`n` matrix with ones on the main diagonal and zeros elsewhere (`eye(n)` creates a square `n`-by-`n` matrix, called an *identity matrix* and denoted by **I**)
`rand(m,n,p,...)`	Creates an `m`-by-`n`-by-`p`-by-... array with uniformly distributed (on the interval [0; 1]) pseudorandom entries (`rand(n)` creates a square `n`-by-`n` matrix)
`randn(m,n,p,...)`	Creates an `m`-by-`n`-by-`p`-by-... array with normally distributed (chosen from a normal distribution with a mean value of 0, variance of 1, and standard deviation of 1) random entries (`randn(n)` creates a square `n`-by-`n` matrix)
`randi(imax,m,n,p,...)`	Creates an `m`-by-`n`-by-`p`-by- ... array containing pseudorandom integer values drawn from the discrete uniform distribution on the interval `[1;imax]` (`randi(imax)` generates a single number, whereas `randi(imax,n)` creates a square `n`-by-`n` matrix)
`randi([imin,imax],...)`	Returns an array containing integers drawn from the discrete uniform distribution on the interval `[imin;imax]`
`magic(n)`	Creates a "magic" square matrix constructed from the integers 1 through n^2 with the equal row, column, and diagonal sums (produces valid magic squares for all $n > 0$ except $n = 2$)
`pascal(n)`	Creates the Pascal matrix of the order `n` (a symmetric positive-definite matrix with the integer entries), where any element is computed by adding the value of the element above it to the value of the element to its left, that is, $a_{ij} = a_{i-1;j} + a_{i;j-1}$, with the very first element $a_{11} = 1$

Table 3.5 Examples of Natural-Number Arrays

Command	`ones(3,4)`	`zeros(3)`	`eye(3,4)`	`magic(3)`	`pascal(3)`
Output	1 1 1 1 1 1 1 1 1 1 1 1	0 0 0 0 0 0 0 0 0	1 0 0 0 0 1 0 0 0 0 1 0	8 1 6 3 5 7 4 9 2	1 1 1 1 2 3 1 3 6

Table 3.6 Examples of the `rand` and `randn` Outputs

Command	`rand(3)`	`randn(3)`
Output	0.8147 0.9134 0.2785 0.9058 0.6324 0.5469 0.1270 0.0975 0.9575	-0.4326 0.2877 1.1892 -1.6656 -1.1465 -0.0376 0.1253 1.1909 0.3273

Be aware that when using the random number generating functions (`rand`, `randn`, `randi`), MATLAB generates these "random" numbers using a deterministic algorithm that is designed to be sufficiently complicated, so that its output appears to be an independent random sequence and can pass various statistical tests for randomness. Every time you start the MATLAB session, this algorithm is initialized the same way, so that the sequence of the random numbers is the same as well. The `rng` function provides ways to take advantage of this determinism to 1) repeat calculations that involve random numbers and get the same results or 2) guarantee that different random numbers are used in repeated calculations. The `rng(seed)` call, where `seed` is a nonnegative integer, allows you to initialize the random number generator differently. The easiest way to ensure that you always have different random numbers is to reseed the generator using the current time. The call

```
>> rng('shuffle')
```

does exactly that. If for some reason you need the repetitive results, you can store the current settings of the random number generator and then reuse them as needed:

```
>> s = rng;          % get generator settings
>> x = rand
x = 0.1576
>> rng(s);           % restore previous generator settings
>> y = rand
y = 0.1576
```

To continue the matrix creation discussion, Table 3.7 presents the functions useful for creating triangular matrices (Fig. 3.12) as well as replicating and tiling the arrays (Fig. 3.13).

For instance, in the beginning of this section, we used the following command to replicate two layers of the 4-by-4-by-2 array `A`:

```
>> A(:,:,3:4) = A(:,:,1:2);
```

Table 3.7 Functions for Creating Triangular Matrices and Tiling

Function	Brief Description
`triu(B,k)`	Returns the upper triangular part of matrix `B` (on and above the `k`th diagonal of `B`)
`istriu(A)`	Determines if matrix `A` is upper triangular
`tril(B,k)`	Returns the lower triangular part of matrix `B` (on and below the `k`th diagonal of `B`)
`istril(A)`	Determines if matrix `A` is lower triangular
`repmat(A,n)`	Creates an `n`-by-`n` tiling of `A`
`repmat(A,[m,n])`	Creates a matrix consisting of an `m`-by-`n` tiling of copies of `A`

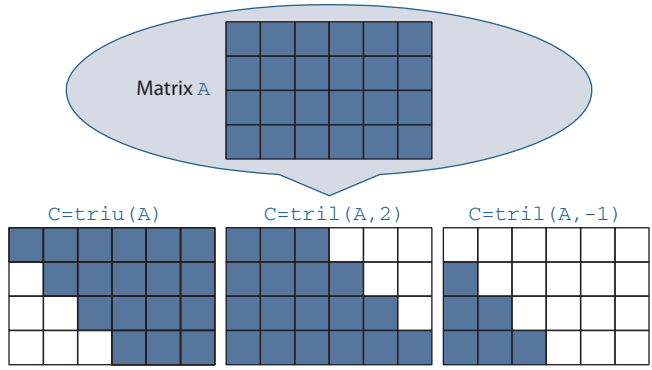


Fig. 3.12 Illustration of creating the lower and upper triangular matrices.

Instead, we could have used

```
>> A=repmat(A,[1,1,2])
```

The `repmat` function can also be used to create signals. For example, the following commands:

```
>> Ts=0.1;
>> triangle = repmat([0:Ts:(1-Ts),1:-Ts:(0+Ts)],1,4);
>> plot(triangle)
```

produce the plot shown in Fig. 3.14. Of course, the Signal Processing Toolbox of MATLAB features a variety of built-in MATLAB functions for creating different signals, such as `square`, `sawtooth`, `gauspuls`, `rectpuls`, `sinc`, `diric`, and `tripuls`. However, if you do not have this toolbox or need to create some specific signal, then the `repmat` function could definitely help.

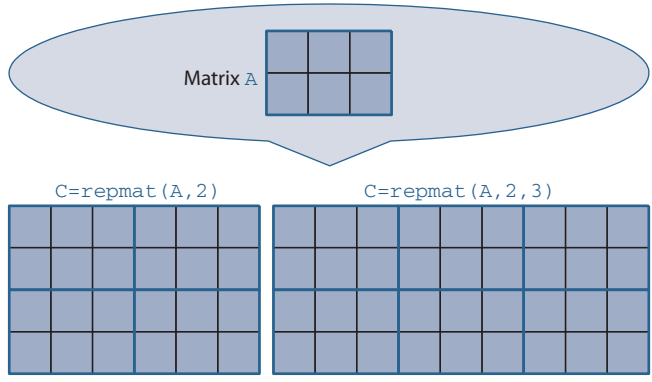


Fig. 3.13 Illustration of the `repmat` function.

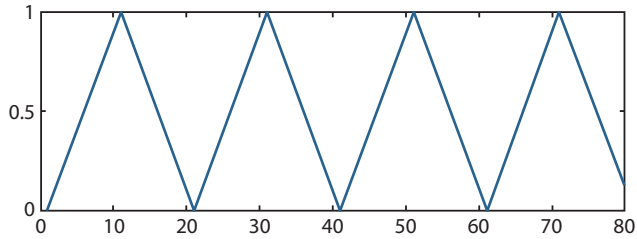


Fig. 3.14 Illustration of the `repmat` function used to create a triangle-like signal.

Let us conclude this section with three more functions, which are shown in Table 3.8.

Here is how they can be used:

```
>> randperm(5)
ans = 2    5    4    3    1

>> randperm(10,3)
ans = 9    5    4

>> circshift([1 2 3 4],1)
ans = 4    3    1    2
```

```
>> perms([1 3 5])
ans = 5    3    1
      5    1    3
      3    5    1
      3    1    5
      1    5    3
      1    3    5
```

3.2.4 Reshaping, Editing, and Displaying Arrays

Once you have an array, you may reshape it to another size or other dimensions using the `reshape(A,m,n,p,...)` function. Here `A` is the name of array, and `m`, `n`, `p`, . . . are the number of elements along each dimension in a proposed new (multidimensional) array. No need to mention that the number of elements in the new array, `m*n*p*` . . ., should be the same as the number of elements in `A`.

Consider an example. Suppose you have a 3-by-4 matrix you want to reshape. To better understand the underlying concept, you have to recall that arrays are stored in computer's memory as a 1-D sequence, column vector, which is illustrated in Fig. 3.15a. Therefore, the `reshape(A,m,n)`

Table 3.8 Permutation Functions

Function	Brief Description
`randperm(n,k)`	Returns a row vector containing `k` unique integers selected randomly from 1 to `n` inclusive (`k` is an optional parameter)
`perms(v)`	Returns a matrix containing all permutations of elements of vector `v` in reverse lexicographic order
`circshift(A,k)`	Circularly shifts the elements in array `A` by `k` positions

command applied to matrix A will return an m-by-n matrix, whose elements are taken one by one from that column vector shown on the right-hand side of Fig. 3.15a. Of course, you can do this only if m*n yields 12; otherwise an error would occur indicating that A does not have m*n elements. Some possibilities for reshaping our specific array A are presented in Fig. 3.15b. On three occasions, we only changed the size of the original matrix (to that of 4-by-3, 6-by-2, and 2-by-6), and once we reshaped our matrix to a 3-D array (2-by-3-by-2).

One more option to change the size (and content) of an existing array is to use the empty matrix formalism introduced in Fig. 3.1. For instance, creating matrix Ah as

```
>> Ah = [0.7680   0.4387   0.3200   0.7446   0.6833
         0.9708   0.4983   0.9601   0.2679   0.2126
         0.9901   0.2140   0.7266   0.4399   0.8392
         0.7889   0.6435   0.4120   0.9334   0.6288];
```

and issuing the following commands

```
>> Ah(:,[2,4])=[]; Ah(3,:)=[]
```

eliminates the second and fourth columns as well as the third row from the matrix Ah and returns

```
Ah = 0.7680   0.3200   0.6833
     0.9708   0.9601   0.2126
     0.7889   0.4120   0.6288
```

So far, we have discussed changing arrays programmatically, but of course you can always employ the **Variables Editor** of the MATLAB development environment. Double-clicking a variable in the Workspace browser, or using

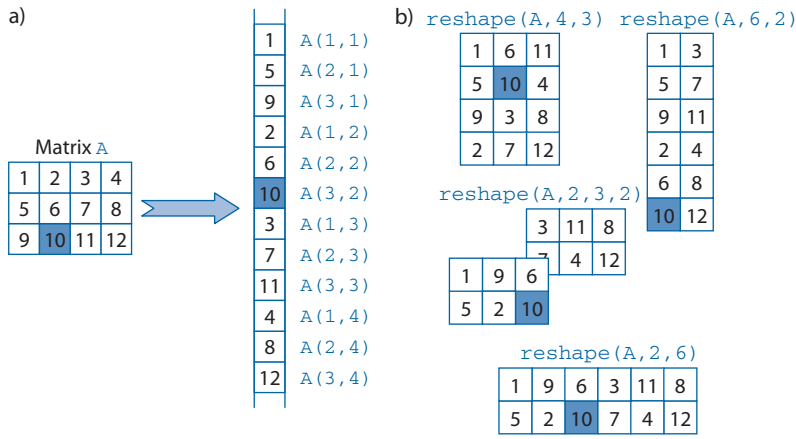


Fig. 3.15 Example of a) storing a 3-by-4 matrix in computer's memory and b) some of reshaping options.

`openvar variablename` command, brings up the **Variables Editor** window as shown in Fig. 3.16. You may use the **Variables Editor** to view and edit any variable (array) available in the workspace as well as load data from somewhere else. It also allows you to visualize the content of numeric arrays using a variety of appropriate plotting functions enabled in the Plots tab in the MATLAB Toolstrip. The Plots tab shows a gallery of supported plot types based on the variable(s) you select from your workspace. For additional plot types, you should click the upside-down triangle at the end of the gallery.

Right-clicking the name of the variable in the Workspace window opens a drop-down menu shown in Fig. 3.17a. This list also features the list of the appropriate plotting functions to visualize the selected variable. Choosing the last option from this menu, **Plot Catalog** . . ., brings up another window that can be used to explore all plotting options (Fig. 3.17b).

Figure 3.18 shows a couple of options when choosing the `plot` and `stem` functions from the Plots tab gallery to visualize the third row of the `H` matrix shown in Fig. 3.16 (Figs. 3.18a and 3.18b, respectively). It should be noted that the plotting functions in this case are used in their simplest syntax. As a result, you might not get what you expected to see. For example, the `plot(H(3,:))` command actually produces a linear interpolation between the data points, so that the points themselves are not shown explicitly. Chapter 6 will show how to use this specific function, `plot`, slightly differently to achieve the desired result. For example, the `plot(H(3,:),'o')` command might be more appropriate because it explicitly shows the data points using the marker specified by the second input argument (Fig. 3.18c).

MATLAB also has other means for viewing matrices. One of them, the `spy` function, is especially useful for visualizing sparse matrices that have a

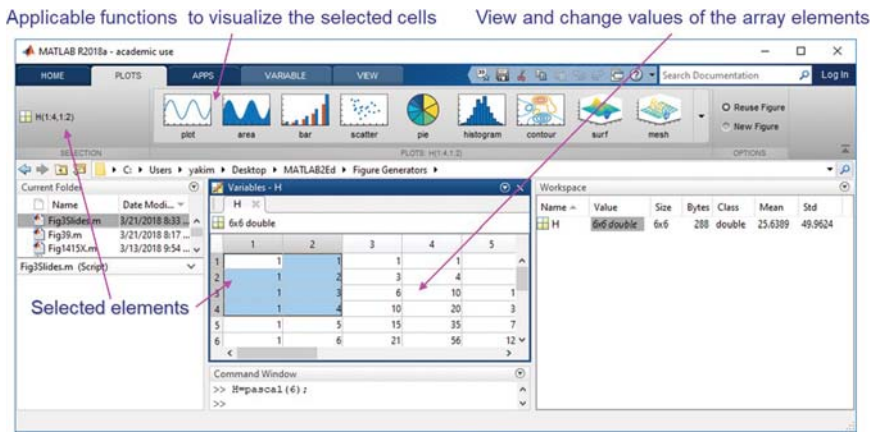


Fig. 3.16 Variables Editor window and Plots tab gallery.

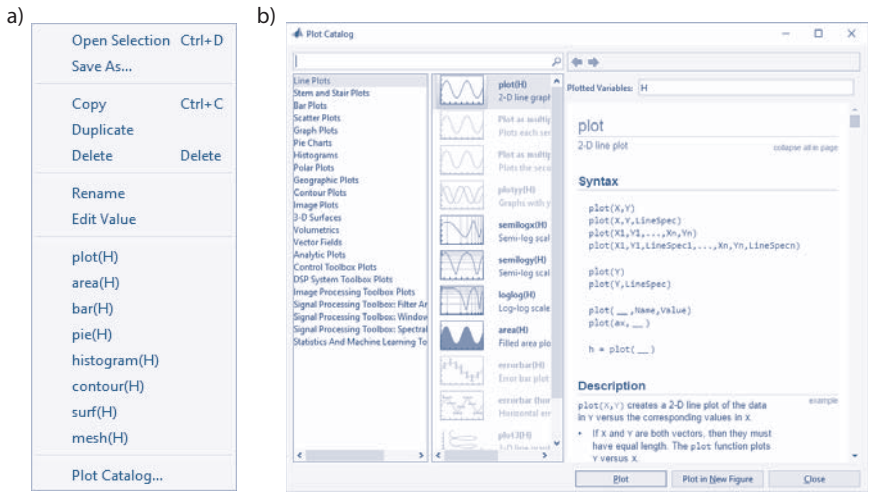


Fig. 3.17 a) Variable context menu, and b) **Plot Catalog** window.

lot of zero elements. Consider the following set of commands to create a sparse matrix `z`:

```
>> z=ones(100,200);
>> z(10:50,150:170)=0;
>> z(70:80,20:35)=0;
>> z(4:5:100,50:130)=0;
>> z(3:5:100,50:130)=0;
>> z(2:5:100,50:130)=0;
>> z=z-eye(100,200);
```

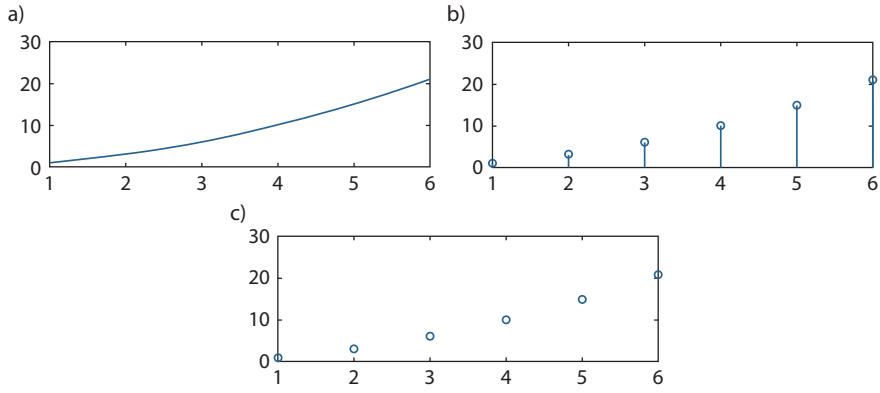


Fig. 3.18 Visualization of the data points using a) the `plot` function, b) the `stem` function, and c) the `plot` function specifying a marker symbol.

Then, the command

```
>> spy(z)
```

brings up the window shown in Fig. 3.19. As seen, this command produces a template view of the sparse structure, where each point on the graph represents the location of nonzero element. Below the graph, the number of nonzero elements (nz) is shown explicitly. [This number can be obtained independently by issuing the `nnz(z)` command.]

You may also use another MATLAB function, `imagesc`, which finds the spread of elements in a matrix and matches it to the full range of the current `colormap` (to be considered in Sec. 6.9), presenting the matrix as a color picture where each color corresponds to a certain magnitude. For example, Fig. 3.20a shows how the matrix `z`, developed earlier, would look like after executing the following two commands:

```
>> imagesc(z)
>> colorbar
```

(the `colorbar` function resizes the current axes to display the current `colormap` to the right of the plot).

Figure 3.20b shows another example, visualizing the matrix `Ah` introduced earlier, using another colormap and slightly modified syntax of the `imagesc` function:

```
>> imagesc(Ah,[0.2 1])
>> colormap gray
>> colorbar
```

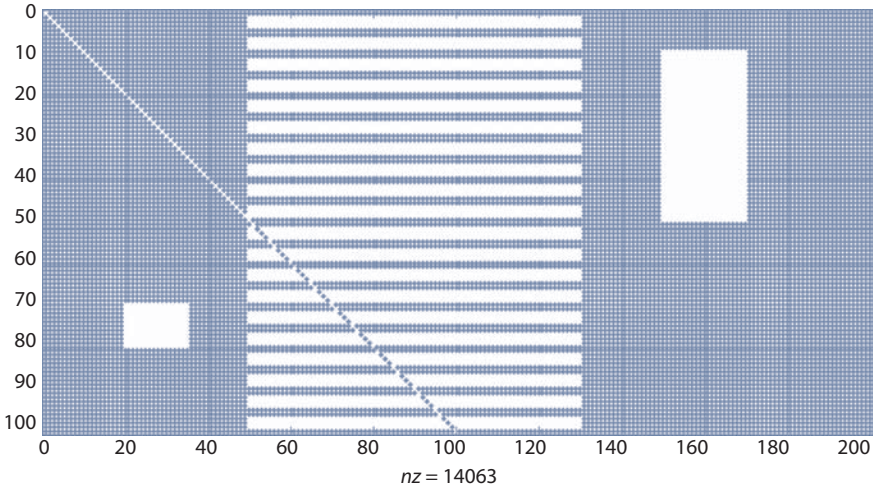


Fig. 3.19 Visualization of a matrix sparsity structure using the `spy` function.

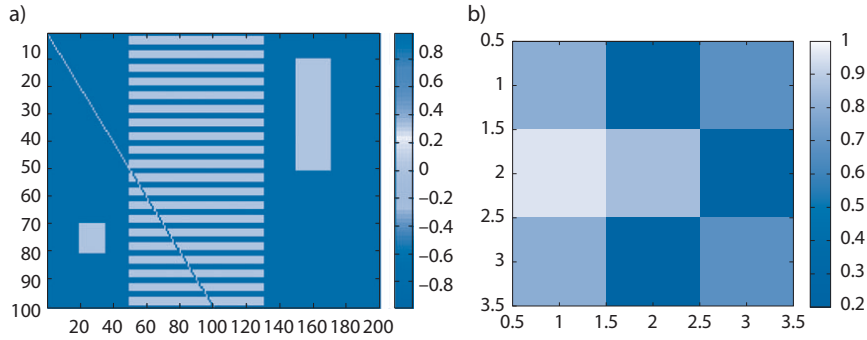


Fig. 3.20 Visualization of matrices a) `z` and b) `Ah` using the `imagesc` function.

where the vector `[0.2 1]` specifies the range of the color bar explicitly. As will be discussed in Chapter 6, some other functions, typically used for 3-D plotting, can be utilized to visualize arrays as well.

To visualize the arrays composed of the random numbers you can use the `histogram` or `hist` function. For example, the command

```
>> histogram(rand(100,1))
```

generates a histogram bar chart of elements in a vector composed of `100` uniformly distributed pseudorandom numbers (Fig. 3.21a). Similarly,

```
>> hist(randn(1000,10),20)
```

shows the histogram bar chart for `10` realizations with `1000` normally distributed pseudorandom numbers (Fig. 3.21b). The second argument, `20`, sorts the input vector into the specified number of bins as opposed to some default number, as was the case in Fig. 3.21a.

To finalize this section, we should warn you that when dealing with arrays and using the simple nonunique names for them, you have to be careful with what you have in the workspace already. For example, suppose you have

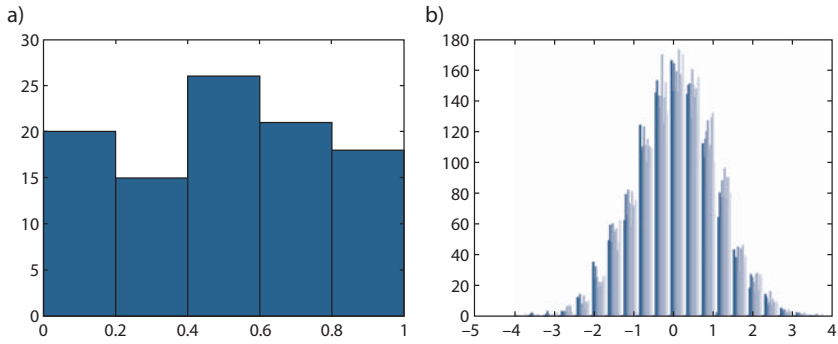


Fig. 3.21 Visualization of random numbers using a) `histogram` and b) `hist` functions.

some 4-by-7 array `x` [say you created it by issuing the `x=zeros(4,7);` command] and later on you decided to use the same name `x` to create another array, completely forgetting that you had another one already. If your new array has the same size, that is, 4-by-7, and you will be defining all elements of this array, you will have no trouble because the new values will override the old ones. However, if your new `x` has a smaller size you may run into a problem. For instance, typing

```
>> x(1:3,1:3)=ones(3)
```

"unexpectedly" returns

```
x = 1   1   1   0   0   0   0
    1   1   1   0   0   0   0
    1   1   1   0   0   0   0
    0   0   0   0   0   0   0
```

To avoid possible problems, either use the unique names or clear variables (arrays) you do not need anymore. In the latter case use

```
>> clear variablename
```

to delete just one variable `variablename` or

```
>> clear all
```

to clear the entire workspace. The wildcard character "*" can also be used to clear variables that match a pattern. For example,

```
>> clear x*
```

clears all variables (arrays) starting with `x`. (Now you understand why in the beginning of Sec. 2.2.3 we created a shortcut button **CrearStart**, which includes the `clear all` command (Fig. 2.9b)—it is not a bad idea to push it once in a while to start with an empty base workspace.)

3.3 Array Operations

Applied to arrays, MATLAB allows you to use arithmetic operators +, −, *, /, and ^ in two different ways. The following considers both of them, starting from the conventional way of treating arrays resulting in standard matrix operations and then proceeding with nonstandard operations.

3.3.1 Standard Array Operations

The three basic operations that can be applied to modify matrices are *matrix addition*, *scalar multiplication*, and *transposition*.

Let us start from transposition, which we have used throughout this chapter already. The transposition

```
>> D=A';
```

interchanges rows and columns of matrix A, so that

$$D = [d_{ij}] = [a_{ji}] \tag{3.4}$$

In fact, in MATLAB this operation is called a *complex conjugate transpose* because if matrix A has complex elements, this operation not only interchanges rows and columns but also changes any complex number to its complex conjugate; for example, if $A = [a_{ij} + jb_{ij}]$, then $D = [a_{ji} - jb_{ji}]$. MATLAB also has an alternative that does not involve conjugation. The way to apply this type of transposition is to use the following syntax:

```
>> D= A.';
```

(It is the period character (.) that distinguishes between two transposition operations.) In this case, $D = [a_{ij} + jb_{ji}]$.

Given the same-size matrices A and B, matrix addition uses syntax as simple as

```
>> C = A + B;
```

The resulting, same-size matrix C is composed of the sum of the corresponding elements

$$\begin{aligned} C = [c_{ij}] &= A + B = [a_{ij} + b_{ij}] \\ &= \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{21} & \dots & a_{1n} + b_{1n} \\ a_{21} + b_{21} & a_{22} + b_{22} & \dots & a_{2n} + b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} + b_{m1} & a_{m2} + b_{m2} & \dots & a_{mn} + b_{mn} \end{bmatrix} \end{aligned} \tag{3.5}$$

The same-size requirement is very important. For instance, you cannot add the row and column vectors together. Scalar addition is the only exception to the size rule. In this case, a scalar is added to each element of array. For example, assuming B is a scalar, Eq. (3.5) transforms to

$$\begin{aligned} C = [c_{ij}] &= A + B = [a_{ij} + B] \\ &= \begin{bmatrix} a_{11} + B & a_{12} + B & \dots & a_{1n} + B \\ a_{21} + B & a_{22} + B & \dots & a_{2n} + B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} + B & a_{m2} + B & \dots & a_{mn} + B \end{bmatrix} \end{aligned} \tag{3.6}$$

MATLAB also assures the *commutative* property, that is,

A+B returns exactly the same result as B+A

However, the *associative* property

(A+B)+C should return the same result as A+(B+C)

generally does not hold. Chapter 8 discusses how this is related to the way the numbers are represented in computer's memory.

All of the above holds for the *matrix subtraction* as well (you just need to replace an addition sign with a subtraction sign).

Scalar multiplication of matrix A and scalar k is given by multiplying every entry of A by k:

$$k * A = [ka_{ij}] = \begin{bmatrix} ka_{11} & ka_{12} & \dots & ka_{1n} \\ ka_{21} & ka_{22} & \dots & ka_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ ka_{m1} & ka_{m2} & \dots & ka_{mn} \end{bmatrix} \tag{3.7}$$

The commutative property holds here as well, that is, k*A returns the same result as A*k. The same applies to *scalar division*. In this case, A/k returns the same result as k\A (note that k/A and A\k will not work).

All arithmetic operations have their function syntax as shown in Table 3.9. For example, the command

```
>> 3^4
```

is equivalent to

```
>> mpower(3,4)
```

Both transpose operations are compatible with addition and scalar multiplication, that is, (k*A)' returns the same result as k*A' and (A+B)' is equivalent to A'+B' (the same holds for .').

Two of the three aforementioned basic operations, addition (subtraction) and scalar multiplication (division), are also applicable to multidimensional arrays. An attempt to apply transposition, however, returns an error. For example,

```
>> ones(2,2,2)'
```

results in

```
??? Error using ==> ctranspose
Transpose on ND array is not defined.
```

As opposed to scalar multiplication, matrix multiplication of two matrices A and B

```
>> E=A*B;
```

can only occur when their "inner" dimensions agree in size (that is, they are conformable). If A is an *m*-by-*n* matrix and B is an *n*-by-*p* matrix, then the product A*B exists because of the same number of columns in A and

Table 3.9 Functions for Standard Arithmetic Operations

Function	Description	Character(s)
`plus`	Plus	+
`uplus`	Unary plus	+
`mtimes`	Matrix multiply	*
`mpower`	Matrix power	^
`minus`	Minus	−
`uminus`	Unary minus	−
`mrdivide`	Slash or right matrix divide	/
`mldivide`	Backslash or left matrix divide	\
`transpose`	Nonconjugate transpose	.'
`ctranspose`	Complex conjugate transpose	'

rows in `B` (m-by-$n \times n$-by-p), returning an m-by-p matrix.

$$\mathtt{E} = [e_{ij}] = \mathtt{A} * \mathtt{B} = \left[\sum_{k=1}^{n} a_{ik} b_{kj}\right] \tag{3.8}$$

By definition, the ijth element of `E` is computed as a sum of the products of elements of the ith row of `A` and jth column of `B`.

Using the same matrices when $m \neq p$, the product `B * A` does not exist (the inner dimension of `B` is p and the inner dimension of `A` is m). If $m = p$, `B*A` returns a square n-by-n matrix. Therefore, even if both `A` and `B` are square matrices to begin with, so that both products `A*B` and `B*A` exist, `A*B` is not the same as `B*A`, that is, the commutative property does not hold. There are two exceptions:

1. If one of the matrices is a null matrix **0**, then `A*`**0** yields the same result as **0**`*A`.
2. If one of the matrices is an identity matrix **I**, then `A*`**I** provides the same result as **I**`*A`.

You may expect that properties, associative and distributive, hold, as is the case in symbolic computations. However in general, due to round-off errors (to be discussed in Chapter 8), neither of them holds, that is, `A*(B+C)` is not exactly the same as `A*B+A*C`, and `(A*B)*C` may return a slightly different result compared with that of `A*(B*C)`. Of course, we are talking about negligibly small errors, but they are present and may have significant impact on the outcome of your numerical computations. Chapter 8 discusses practical ways of mitigating these types of errors.

In the case when `A` is a row vector and `B` is a column vector with the same number of elements, the result of `F=A*B` is a scalar [Eq. (3.8)]. This specific case is referred to as a *scalar* or *dot product* and has its own function syntax, `dot`, which can be used along with `mtimes`. For example,

```
>> F=dot([1 2 3 4],[4 3 2 1])
F = 20
```

Now, if A is a column vector and B is a row vector, and they are composed of three elements each, then according to Eq. (3.8) their *matrix product* returns a 3-by-3 matrix as follows:

$$\mathrm{F} = [f_{ij}] = \mathrm{A} * \mathrm{B} = \begin{bmatrix} a_1b_1 & a_1b_2 & a_1b_3 \\ a_2b_1 & a_2b_2 & a_2b_3 \\ a_3b_1 & a_3b_2 & a_3b_3 \end{bmatrix} \tag{3.9}$$

However, there is another standard mathematical operation we may have between two three-element vectors, which is known as a *cross product*. In such a case

$$\mathrm{K} = \mathrm{A} \times \mathrm{B} = [\, a_2b_3 - a_3b_2 \quad a_3b_1 - a_1b_3 \quad a_1b_2 - a_2b_1 \,] \tag{3.10}$$

(here, $\times$ denotes a cross product operation). To compute a `cross` product, MATLAB has the `cross` function. The following set of commands produces the plot presented in Fig. 3.22a:

```
A=[1.5,0,0]; B=[0,0.8,0];
K=cross(A,B);
        quiver3(0,0,0,A(1),A(2),A(3),'b','Linewidth',3), hold
        quiver3(0,0,0,B(1),B(2),B(3),'g--','Linewidth',3)
        quiver3(0,0,0,K(1),K(2),K(3),'r--','Linewidth',3), axis equal
xlabel('Axis x'), ylabel('Axis y'), zlabel('Axis z'), view([-140 20])
legend('Vector A','Vector B','cross(A,B)','location','nw')
```

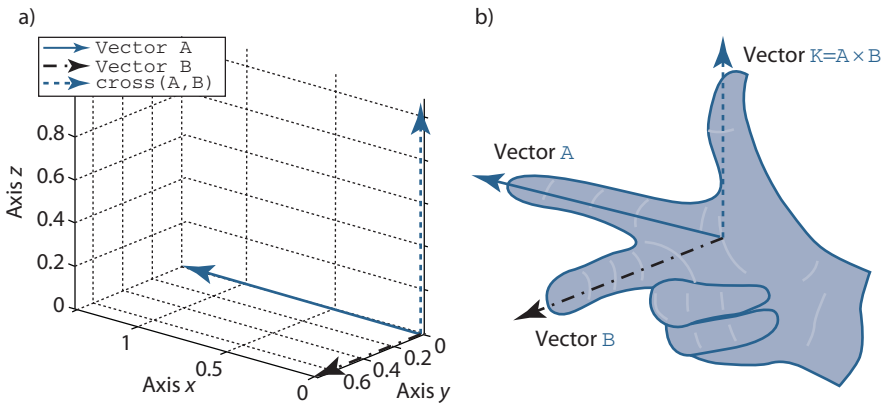


Fig. 3.22 Illustration of the `cross` function output producing a right-hand system (a) according to the right-hand rule (b).

Specifically, the first line introduces two noncollinear vectors, and the second line utilizes the `cross` function, creating the vector, which is perpendicular to those two. The remaining lines utilize the plotting functions to be discussed in Chapter 6.

Using this example, let us introduce the remaining matrices presented in Fig. 3.1. An *orthogonal matrix* is a square matrix that has *orthogonal columns*. To define orthogonal columns, we consider each column as a separate column vector. Mathematically, if a matrix $\mathbf{V}$ is composed of n column vectors $[\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n]$, such that $\mathbf{v}_i^T \mathbf{v}_j = 0$ for all $i \neq j$, then $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n$ are said to be the *orthogonal set of vectors*.

In our case, vectors `A`, `B`, and `K` form an orthogonal set of vectors because the vectors `A` and `B` were chosen to be orthogonal. Formally, it can easily be proven by checking their pair-wise orthogonality:

```
>> [A*B', A*K', B*K']
ans = 0    0    0
```

Therefore, the matrix

```
>> OrM = [A' B' K']
OrM = 1.5000          0          0
           0     0.8000          0
           0          0     1.2000
```

(we are using transposition because our vectors were defined as the row vectors) represents an example of an orthogonal matrix. Moreover, the `cross` function complements its two input arguments, the 3-D vectors, in such a way that geometrically the three vectors `A`, `B`, and `K` form a right-hand system according to the right-hand rule (Fig. 3.22b).

In addition, if these orthogonal vectors are all unit vectors, they form an *orthonormal matrix*. A unique property of orthonormal matrices is that, $\mathbf{Q}^T\mathbf{Q} = \mathbf{Q}\mathbf{Q}^T = \mathbf{I}$ (and $\mathbf{Q}^T = \mathbf{Q}^{-1}$). Let us now check whether our orthogonal matrix is orthonormal:

```
>> OrM*OrM'-eye(3)
ans = 1.2500          0          0
           0    -0.3600          0
           0          0     0.4400
```

Because the return is not a null matrix, the `OrM` matrix is not orthonormal (column vectors are not the unit vectors). Can we make it orthonormal? Yes. All we have to do is normalize our column vectors. To do this, the `norm` function calculates several different types of matrix norms including the Euclidean norm of a vector

$$\|\mathrm{X}\| = \sqrt{\sum_{k=1}^{n} x_k^2} \tag{3.11}$$

For example,

```
>> a = [1,2,3,4]; norm(a)
```

returns

```
ans = 5.4772
```

Normalization implies dividing a vector by its norm, so that a normalized vector's norm is 1:

```
>> a_norm = a/norm(a)
a_norm = 0.1826    0.3651    0.5477    0.7303
>> norm(a_norm)
ans = 1.0000
```

Equipped with this knowledge, we may now construct an example of orthonormal matrix as follows:

```
>> OnM = [A'/norm(A) B'/norm(B) K'/norm(K)]
OnM = 1    0    0
      0    1    0
      0    0    1
```

Before we proceed with other operators of Table 3.9, let us finish with the `cross` function. This function is also applicable to matrices or even (multidimensional) arrays, which have at least one dimension of length 3. In such a case, the `cross` returns the cross product of `A` and `B` along the first dimension of length 3.

The *matrix power* operation (`mpower` or `^`) is applicable only to *square* matrices, which have the same number of rows and columns, in two ways. First, you may use it with the matrix base and scalar exponent, `A^n`. If `n` is a positive integer, the power is computed by repeated squaring. For example,

```
>> magic(3)^3;
```

produces the same result as

```
>> magic(3)*magic(3)*magic(3)
ans = 1197    1029    1149
      1077    1125    1173
      1101    1221    1053
```

(If `n` is a negative integer, `A` is inverted first.) For other values of `n`, the calculation involves eigenvalues and eigenvectors, to be discussed in Chapter 9.

Second, you may have a scalar base with a matrix exponent `n^A`. In this case, the resultant `n`-by-`n` matrix is evaluated as the power series

$$\mathtt{n}^{\mathtt{A}} = \sum_{k=0}^{\infty} \frac{\ln(\mathtt{n})^k \mathtt{A}^k}{k!} \tag{3.12}$$

An expression involving a matrix base with a matrix exponent is not valid and returns an error:

```
??? Error using ==> mpower
At least one operand must be scalar.
```

To perform a matrix division, `A/B` or `A\B`, using either one of the two functions of Table 3.9 (`mrdivide` and `mldivide`), both matrices `A` and `B` must have the same number of rows. The only exception is when one of two matrices is, in fact, a scalar; specifically, when `B` happens to be a scalar in `A/B` or `A` is a scalar in `A\B`, then the problem reduces to a scalar division (all elements of the resultant matrix are divided by a scalar). In general, this operation has to do with solving systems of linear algebraic equations and will be covered in detail in Chapter 8.

3.3.2 Nonstandard Array Operations

Along with standard array (matrix) operations discussed in the previous section, MATLAB allows you to treat arrays as groups of numbers, not common-sense arrays, and apply nonstandard operations to them. The list of these nonstandard operations can be found in Table 3.10 and differs from that shown in Table 3.9 by a period character (.), as was the case with the nonconjugate transpose operation. The intention of these nonstandard operations is to utilize element-by-element operations as opposed to matrix-by-matrix operations.

Element-by-element multiplication is only applicable to the same-size arrays and results in multiplying the corresponding elements in two matrices by each other, as shown for the 2-D case

$$\mathtt{C} = [c_{ij}] = \mathtt{A.*B} = [a_{ij}b_{ij}] = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{21} & \dots & a_{1n}b_{1n} \\ a_{21}b_{21} & a_{22}b_{22} & \dots & a_{2n}b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{m1} & a_{m2}b_{m2} & \dots & a_{mn}b_{mn} \end{bmatrix} \quad (3.13)$$

Table 3.10 Function for Nonstandard Arithmetic Operations

Function	Description	Corresponding Character(s)
`times`	Array multiply	`.*`
`power`	Array power	`.^`
`rdivide`	Right array divide	`./`
`ldivide`	Left array divide	`.\`

For example,

```
>> A = [1 2 3 4; 4 3 2 1], B = [2 2 1 1; 1 2 1 2]
A = 1   2   3   4
    4   3   2   1
B = 2   2   1   1
    1   2   1   2
>> C = A.*B
C = 2   4   3   4
    4   6   2   2
```

Similarly, *element-by-element division* on the same-size arrays results in

$$\mathtt{C} = [c_{ij}] = \mathtt{A./B} = \left[\frac{a_{ij}}{b_{ij}}\right] = \begin{bmatrix} \frac{a_{11}}{b_{11}} & \frac{a_{12}}{b_{21}} & \cdots & \frac{a_{1n}}{b_{1n}} \\ \frac{a_{21}}{b_{21}} & \frac{a_{22}}{b_{22}} & \cdots & \frac{a_{2n}}{b_{2n}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{a_{m1}}{b_{m1}} & \frac{a_{m2}}{b_{m2}} & \cdots & \frac{a_{mn}}{b_{mn}} \end{bmatrix} \tag{3.14}$$

$$\mathtt{C} = [c_{ij}] = \mathtt{A.\backslash B} = \left[\frac{b_{ij}}{a_{ij}}\right] = \begin{bmatrix} \frac{b_{11}}{a_{11}} & \frac{b_{12}}{a_{21}} & \cdots & \frac{b_{1n}}{a_{1n}} \\ \frac{b_{21}}{a_{21}} & \frac{b_{22}}{a_{22}} & \cdots & \frac{b_{2n}}{a_{2n}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{b_{m1}}{a_{m1}} & \frac{b_{m2}}{a_{m2}} & \cdots & \frac{b_{mn}}{a_{mn}} \end{bmatrix} \tag{3.15}$$

(both operations are also applicable to multidimensional arrays). Using the same matrices `A` and `B`, we obtain

```
>> C = A./B
C = 0.5000    1.0000    3.0000    4.0000
    4.0000    1.5000    2.0000    0.5000
>> C = A.\B
C = 2.0000    1.0000    0.3333    0.2500
    0.2500    0.6667    0.5000    2.0000
```

Note, element-by-element multiplication and division will also work if one of the operands is a scalar. In this case, even `A./B` and `A.\B`, where `A` is a scalar, will work properly (which was not the case in the previous section without using the period character). For example,

```
>> C = 2./B
C =       1     1     2     2
          2     1     2     1
```

Following the same paradigm, the *element-wise array power* operation, `A.^B`, applied to same-size arrays returns the same-size array composed of the elements of array `A` in the individual powers presented by the elements of array `B`. For example,

$$\mathtt{C} = [c_{ij}] = \mathtt{A.\hat{}B} = [a_{ij}^{\,b_{ij}}] = \begin{bmatrix} a_{11}^{\,b_{11}} & a_{12}^{\,b_{21}} & \dots & a_{1n}^{\,b_{1n}} \\ a_{21}^{\,b_{21}} & a_{22}^{\,b_{22}} & \dots & a_{2n}^{\,b_{2n}} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}^{\,b_{m1}} & a_{m2}^{\,b_{m2}} & \dots & a_{mn}^{\,b_{mn}} \end{bmatrix} \tag{3.16}$$

Using the same matrices `A` and `B`, we get

```
>> C = A.^B
ans = 1     4     3     4
      4     9     2     1
```

If `B` is a scalar, then `A.^B` raises each element of the array `A` to the power of scalar `B`. For example,

```
>> C = A.^3
```

returns

```
C =  1     8    27    64
    64    27     8     1
```

If `A` is a scalar, then `A.^B` returns a new array of the same size as `B` with the scalar `A` raised to the power of the elements of array `B`. Hence,

```
>> C = 5.^A
```

returns

```
C =   5    25   125   625
    625   125    25     5
```

3.4 Array Functions

To start with, it should be noted that the elementary functions introduced in Sec. 2.4.1 are applicable not only to scalars but also to arrays. Any of those functions applied to an array results in an element-wise operation, that is, this function will be applied to each element of this array, returning the same-size array composed of the results of individual operations. For instance, the command

```
>> sin([0 pi/4 pi/2 3*pi/4; pi -3*pi/4 -pi/2 -pi/4])
```

returns

```
ans =      0     0.7071     1.0000     0.7071
      0.0000    -0.7071    -1.0000    -0.7071
```

Similarly,

```
>> floor(randn(3,4,2))
```

returns

```
ans(:,:,1) =  -1    0    -1    -2
               2    1    -1     0
              -1    0     0     1
ans(:,:,2) =  -1   -2    -1     0
               0   -2     0     1
               1    0     0     0
```

showing the result of applying the `floor` function to each element of the 3-D array created using the `randn` function.

In addition to those elementary functions, MATLAB offers a variety of other array-handling functions applicable to both matrices (including vectors and scalars) and multidimensional arrays (including matrices and scalars). Depending on the number of dimensions used, syntax for these functions may be slightly different, but the general idea is that these functions have the same formalism for the scalar and multidimensional array. Tables 3.11–3.15 present the general overview of the major functions notationally grouped into five categories (of course, you may learn more about any specific function of your interest by checking the MATLAB Help system).

The functions presented in Table 3.11, allow analyzing the array. For example, assume we have a 3-D array created using the following command:

```
>> A = rand(2,4,5);
```

Then, here is what we may learn about it by applying the functions of Table 3.11:

```
>> ndims(A)
ans = 3
```

```
>> numel(A)
ans = 40
```

```
>> length(A)
ans = 5
```

```
>> size(A)
ans = 2   4   5
```

Table 3.11 Functions to Assess Array Properties

Function	Brief Description
`ndims(A)`	Returns the number of dimensions in an array `A`
`size(A)`	Returns the sizes of each dimension of array `A` in a row vector with `ndims(A)` elements (if `A` is a scalar, which MATLAB regards as a 1-by-1 array, `size(A)` returns `[1 1]`)
`[m,n]=size(A)`	Returns the number of rows in `m` and the number of columns in `n` if `A` is a matrix
`length(A)`	Returns the size of the longest dimension of `A` (the number of elements in `A` if `A` is a vector, and the largest value of either rows or columns if `A` is a matrix)
`numel(A)`	Returns the number of elements in an array or subscripted array expression

Although the `size` function permits another syntax

```
>> [l,m,n] = size(A)
l = 2
m = 4
n = 5
```

it seems to be redundant given that we got the same information using simple syntax, `size(A)`. If the number of outputs is less than the number of dimensions in an array, the very last output returns the product of the missing sizes:

```
>> [l,m] = size(A)
l = 2
m = 20
```

Additional syntax allows you to find a size along the specified dimension, for example,

```
>> size(A,2)
ans = 4
```

Also, there is an alternative way to find the number of dimensions without using the `ndims` function:

```
>> length(size(A))
ans = 3
```

The functions of Table 3.12 allow summing and multiplying elements of arrays along any dimension. For example, for the matrix

```
>> B = reshape(1:12,3,4)
B = 1   4   7   10
    2   5   8   11
    3   6   9   12
```

the returns of these functions are

```
>> sum(B)
ans = 6   15   24   33
```

```
>> prod(B)
ans = 6   120   504   1320
```

```
>> sum(B,2)
ans = 22
      26
      30
```

```
>> prod(B,2)
ans =  280
       880
      1944
```

```
>> sum(B,'all') or sum(B,[1 2]) or sum(sum(B)) or sum(B(:))
ans = 78
```

```
>> cumsum(B)
ans = 1    4    7    10
      3    9   15    21
      6   15   24    33
```

```
>> cumprod(B)
ans =  1      4      7      10
       2     20     56     110
       6    120    504    1320
```

Table 3.12 Sum and Product of Array Elements

Function	Brief Description
`sum(A)`	Sums elements of each column of `A` and returns a row vector with these sums. If `A` is a multidimensional array, `sum(A)` treats the values along the first nonsingleton dimension as the vectors, returning an array of the row vectors
`sum(A,'all')`	Computes the sum of all elements of `A` (available since R2018b)
`sum(A,dim)` `sum(A,vecdim)`	Sums along the dimension(s) of `A` specified by scalar `dim` or vector `vecdim`, respectively
`sum(...,'omitnan')`	Specifies to omit `NaN` values[a] from calculations
`cumsum(...)`	Returns the cumulative sum of elements along different dimensions of an array with syntaxes similar to those of `sum`
`prod(...)`	Returns the product of elements along different dimensions of an array with syntaxes similar to those of `sum`
`cumprod(...)`	Returns the cumulative product of elements along different dimensions of an array with syntaxes similar to those of `sum`

[a] NaN was introduced in Sec. 2.4.4 (Table 2.9).

Using the functions shown in Table 3.13, you can treat the array's elements as a sequence of numbers and perform some statistical analysis.

Syntax for these functions is the same as that of `sum` and `prod` (with the only exception being that for the `min`, `max`, and `std` functions, the dimension has now to be entered as a third rather than second input argument):

```
>> mean(B)
ans = 2   5   8   11

>> median(B)
ans = 2   5   8   11

>> std(B)
ans = 1   1   1   1

>> mean(B,2)
ans = 5.5000
      6.5000
      7.5000

>> median(B,2)
ans = 5.5000
      6.5000
      7.5000

>> std(B,[],2)
ans = 3.8730
      3.8730
      3.8730

>> min(B)
ans = 1   4   7   10

>> max(B)
ans = 3   6   9   12

>> min(B,[],2)   or   min(B')'
ans = 1
      2
      3

>> [s,l] = bounds(B,'all')
s = 1
l = 12

>> mean(B(:))
ans = 6.5000

>> std(B(:),1)
ans = 3.4521

>> max(B,[1 2])
ans = 12
```

Now that you have learned about the `max` function, you can construct an alternative to the `length` function of Table 3.11 as follows:

```
>> max(size(A))
ans = 5
```

In addition, the `min` and `max` functions assume two other syntax variations. First, along with finding the minimum and maximum values, you can also locate their indices. For example,

```
>> a = ceil(randn(3)*10)
a = -15   -3    8
    -14    7   13
      6    9    7
```

Table 3.13 Statistical Functions

Function	Brief Description

`mean(A)`
`mean(A,'all')`
`mean(A,dim)`
`mean(A,vecdim)`
`mean(...,'omitnan')`
	Returns the mean values of the elements, $\overline{x} = (1/n)\sum_{i=1}^{n} x_i$, along different dimensions of an array. Specifically, if `A` is a vector, `mean` returns the mean value of its elements; if `A` is a matrix, `mean` produces a row vector containing the mean value of each column; and if `A` is an array, `mean` treats the values along the first nonsingleton dimension as the vectors, returning an array of the mean values. (The direction can be changed by the second argument similar to that of the `sum` function of Table 3.12.) The last syntax specifies to omit `NaN` values from calculations

`median(...)`	Returns the median values of the elements along different dimensions of an array with syntaxes similar to those of `mean`

`std(A)`
`std(A,[],dim)`
`std(...,'omitnan')`
	Returns the standard deviation of elements of `A` along the optional dimension `dim`, specified as `'all'`, scalar or vector, similar to that of `mean`. The place holder `[]` may be either `0` (default value) or 1, corresponding to the square root of an unbiased estimate of the variance, $s = \sqrt{1/(n-1)\sum_{i=1}^{n}(x_i - \overline{x})^2}$, and the second moment of the set of values about their mean, $s = \sqrt{(1/n)\sum_{i=1}^{n}(x_i - \overline{x})^2}$, respectively. The last syntax specifies not to include the `NaN` values

`min(A)` `min(A,[],dim)`	Returns the smallest elements of `A` along the optional dimension `dim`, specified as `'all'`, scalar or vector, similar to that of `mean`
`[x,k] = min(...)` `[x,k] = min(...,'linear')`	Returns the minimum values in a row vector `x` along with indices in a row vector `k`. You can also request a linear index output
`min(A,B)`	Returns an array of the same size as `A` and `B` with the smallest elements taken from `A` or `B` (unless one of `A` and `B` is a scalar, their dimensions must match)
`min(...,'includenan')`	Specifies to include the `NaN` values (by default they are not included)
`max(...)`	Finds the maximum values with all syntaxes similar to those of `min`
`[s,l] = bounds(...)`	Returns the smallest element `s` [equivalent to `min(...)`] and largest element `l` [equivalent to `max(...)`] of an array with all syntaxes similar to those of `min`

```
>> [m,i] = max(a)
m = 6   9   13
i = 3   3    2
```

(note that if you try to replicate the first command that employs the random number generator `randn`, you will likely end up with a different matrix).

Second, these two functions allow you to compare two arrays, as shown in the following example:

```
>> b = ceil(randn(3)*10)
b =  12    -1   -10
   -12   -16    15
     0     3    -8
>> min(a,b)
ans = -15    -3   -10
      -14   -16    13
        0     3    -8
```

Table 3.14 presents the `find` function, which may be very useful. Let us create a matrix with some zero elements using the `rand` and `round` functions that we considered already:

```
>> c = round(rand(3,4)*2)
c = 2   0   1   1
    0   2   1   0
    1   1   0   0
```

Now, let us apply the `find` function, which finds the indices of nonzero elements:

```
>> find(c)'
ans = 1   3   5   6   7   8   10
```

(we transposed the result for compactness). Note that `find (A)` also handles logical expressions. If `A` is a *logical expression* (to be considered in Sec. 5.3), then the output contains nonzero elements of a logical array obtained by evaluating the expression `A`. We will use this feature later, in Sec. 5.2, but for consistency let us present a couple of examples here. For instance,

Table 3.14 Syntaxes for the `find` Function

Syntax	Brief Description
`find(A)`	Locates all nonzero elements of an array `A` and returns their linear indices in a column vector
`[r,c,v] = find(A)`	Returns a column vector `v` of the nonzero entries in the matrix `A`, as well as row and column indices, `r` and `c`, of these elements (to see the results in the most readable way, we recommend issuing the following instruction: `[r c v]`)
`find (A, k, ind)`	Returns at most the first (if `ind = 'first'` or omitted) or the last (`ind = 'last'`) `k` indices corresponding to the nonzero entries of an array `A`

using logical NOT, ~ (to be covered in Sec. 5.3), we may easily reverse the problem and look for zero elements rather than nonzero elements:

```
>> find(~c)'
ans = 2  4  9  11  12
```

Another example shows how to compare elements of two matrices, a and b, introduced earlier. For better understanding, let us first observe the result of comparison of two same-size matrices:

```
>> a>b
ans = 0   0   1
      0   1   0
      1   1   1
```

The resulting matrix is composed of a *true* value, 1, and a *false* value, 0. For example, the first element of matrix a is −15 and matrix b is 12, and hence an element-by-element comparison returns 0 (given that −15<12). The same happens when comparing the second elements. However, a comparison of the third elements, 6 and 0, returns 1 (given that 6>0), and so on.

Now, let us explore another syntax of the find function (only applicable to matrices), which features three output variables:

```
>> [i,j,x] = find(a>b);
>> [i  j  x]
ans = 3   1   1
      2   2   1
      3   2   1
      1   3   1
      3   3   1
```

Comparing the three output vectors, i, j, and k (we displayed them column by column in the form of a matrix) with the result of the previous command (a>b), we can clearly see that they represent row and column indices (subscripts) of nonzero elements along with the values of these nonzero elements, respectively.

Let us go a little bit further and show how to use this information. Suppose you are required to show only those elements of matrix a that satisfy an inequality a>b and zero for the rest of them. Here is how you could do this. We start by converting subscripts to a single index using the sub2ind function

```
>> t = sub2ind(size(a),i,j)'
ans = 3   5   6   7   9
```

Then, we pick the elements we need from the original matrix a:

```
>> f(t) = a(t)
f = 0   0   6   0   7   9   8   0   7
```

And finally, we reshape the resulting matrix to match the size of matrix a:

```
>> g = reshape(f,size(a))
g = 0    0    8
    0    7    0
    6    9    7
```

The last syntax of the find function in Table 3.14 allows you to find not all but a few first or last nonzero elements, which may economize computations in the case of large matrices if you only need to know whether it has nonzero elements at all.

Table 3.15 lists a few more functions you can use to manipulate single and multiple arrays. Note, whereas the unique and ismember functions perform exact comparisons, uniquetol and ismember perform comparisons using tolerance, which is very convenient when dealing with the results

Table 3.15 Array-Manipulating Functions

Function	Brief Description
Reverse the Order of Elements	
flip(A)	Returns an array of the same size as A, but with the order of elements reversed [flip(A,dim) reverses the order of elements in A along the dimension dim]
flipud(A)	Returns A with its rows flipped in the up-down direction [analogous to flip(A,1)]
fliplr(A)	Returns A with its rows flipped in the left–right direction [analogous to flip(A,2)]
rot90(A,k)	Rotates A counterclockwise by k*90 degrees, where k is a signed integer (i.e., by −90°, 90°, and 180°)
Sort Array Elements and Find Unique Elements	
issorted(A)	Determines whether elements in A are in sorted order
issortedrows(A)	Determines whether matrix A rows are sorted
sort(A)	Sorts the elements of A along the first nonsingleton dimension of an array [sort(A,dim) sorts along the dimension dim, sort(...,'descend') sorts the elements in descending order]
sortrows(A) sortrows(T,col)	Sorts the rows of matrix A based on the elements in the first or specified column col in ascending order
unique(A)	Finds the unique values in the array A
uniquetol(A,tol)	Similar to unique, but allows finding the unique values in the array A with tolerance tol
Compare Elements of Two Arrays	
intersect(A,B)	Returns data common to both A and B with no repetitions
ismember(A,B)	Returns an array containing 1 (true) where data in A is found in B [elsewhere, it returns 0 (false)]
ismembertol ... (A,B,tol)	Similar to ismember, but allows setting membership within tolerance tol
setdiff(A,B)	Returns data in A that is not in B
setxor(A,B)	Sets exclusive OR (will be explained in Sec. 5.3) of two arrays, A and B
union(A,B)	Returns combined data from A and B with no repetitions

of experiments that are only exact to a certain precision. Obviously, Table 3.15 lists only major syntax for each function, meaning that you should refer to the MATLAB Help system if you want to learn more about any specific function. The following self-explanatory examples should help in understanding the essence of these functions:

```
>> flip(magic(3),1)
ans = 4     9     2
      3     5     7
      8     1     6
```

```
>> fliplr(pascal(3))
ans = 1     1     1
      3     2     1
      6     3     1
```

```
>> rot90(pascal(3))
ans = 1     3     6
      1     2     3
      1     1     1
```

```
>> sort(B,'descend')
ans = 3     6     9     12
      2     5     8     11
      1     4     7     10
```

```
>> sortrows(ans)
ans = 1     4     7     10
      2     5     8     11
      3     6     9     12
```

```
>> sort(B,2,'descend')
ans =  10     7     4     1
       11     8     5     2
       12     9     6     3
```

```
>> unique = ([1  3   5   3   4])
ans  = 1    3    4    5
```

```
>> uniquetol([2.1 2  3  3.4 6 5.9],0.1)
ans = 2.0000    3.0000    5.9000
```

```
>> intersect([1 3 5 3 4],[3 6 1 7])
ans = 1     3
```

```
>> ismember([1  3  5  3  4],[3  6  1  7])
ans = 1     1     0     1     0
```

```
>> setdiff([1  3  5  3 4],[3 6  1 7])
ans = 4     5
```

```
>> setxor([1  3  5  3  4], [3  6  1  7])
ans = 4     5     6     7
```

```
>> union([1 3 5 3 4],[3 6 1 7])
ans = 1     3     4     5     6     7
```

```
>> issorted([1  3  5  3  4])
ans = 0
```

In addition to the functions introduced in Tables 3.11–3.15 and in general applicable to multidimensional arrays (including matrices), some other useful MATLAB functions are defined only for 2-D arrays and, more specifically, for square matrices (Table 3.16). For instance, for the g matrix calculated earlier, we may have

```
>> det(g)
ans = -336
```

```
>> inv(g)
ans = -0.1458   -0.2143   0.1667
           0    0.1429        0
      0.1250         0        0
```

```
>> eig(g)
ans =  -4.2621
       11.2621
        7.0000
```

```
>> trace(g)
ans = 14
```

Now that you know more about basic programming in MATLAB, you may check these results implicitly. For example,

```
>> 6*(-7*8)
ans = -336
```

```
>> g*inv(g)
ans = 1     0     0
      0     1     0
      0     0     1
```

```
>> sum(eig(g))
ans = 14
```

```
>> sum(diag(g))
ans = 14
```

Table 3.16 Functions for Square Matrices

Function	Brief Description
`det(A)`	Calculates the *determinant* of the square matrix `A`
`inv(A)`	Computes the *inverse* of the square matrix `A` (a warning message is printed if `A` is badly scaled or nearly singular)
`eig(A)`	Calculates eigenvalues of the square matrix `A`
`trace(A)`	Sums up the diagonal elements of `A`, which happens to be the sum of *eigenvalues* of `A`
`isymmetric(A)`	Determines if matrix `A` is *symmetric* or *skew-symmetric*

3.5 Using MATLAB Matrix Formalism To Handle Polynomials

Consider a (row) vector

```
>> c1 = [1   0   -2   5   -4]
c1 = 1   0   -2   5   4
```

Now, let us write down the following forth-order polynomial:

$$P_1(x) = x^4 - 2x^2 + 5x - 4 \tag{3.17}$$

Do you see any resemblance? Well, our vector `c1` looks like a collection of coefficients of polynomial $P_1(x)$, written in a slightly different form of

$$P_1(x) = 1x^4 + 0x^3 - 2x^2 + 5x - 4 \tag{3.18}$$

Let us introduce another vector

```
>> c2 = [1   1   0]
c2 = 1   1   0
```

We can treat this vector as a collection of coefficients of another polynomial

$$P_2(x) = x^2 + x + 0 = x^2 + x \tag{3.19}$$

In fact, to have a perfect match with the vectors of coefficients `c1` and `c2`, we can rewrite both polynomials in the following form:

$$P_1(x) = \texttt{c1}\begin{bmatrix} x^4 \\ x^3 \\ x^2 \\ x \\ 1 \end{bmatrix} \qquad P_2(x) = \texttt{c2}\begin{bmatrix} x^2 \\ x \\ 1 \end{bmatrix} \tag{3.20}$$

Now, let us use the vectors `c1` and `c2` for the following two operations:

```
>> cp = c1 + [0   0   c2]
cp = 1   0   -1   6   -4
```

and

```
>> cm = c1 - [0   0   c2]
cm = 1    0    -3    4    -4
```

(Note that we had to concatenate two leading zeros with the vector `c2` to get the same length as the vector `c1`, otherwise MATLAB would return an error). No surprise, but we can treat these two operations as if we were adding polynomials $P_1(x)$ and $P_2(x)$ together and subtracting one from another:

$$P_+(x) = P_1(x) + P_2(x), \quad P_-(x) = P_1(x) - P_2(x) \tag{3.21}$$

To automatize adding leading zeros, you can use the following combination of the functions you already know:

```
horzcat(zeros(1,length(c1)-length(c2)),c2)
```

Multiplying two polynomials and performing polynomial division cannot be expressed in one line and requires some programming. Luckily, MATLAB provides you with two useful functions shown in Table 3.17.

Let us explore these two functions:

```
>> c3 = conv(c1,c2)
c3 = 1     1     -2     3     1    -4     0
>> [c4,c5] = deconv(c1,c2)
c4 = 1    -1     -1
c5 = 0     0      0     6    -4
```

Indeed, the resulting vectors of coefficients correspond to the correct solutions:

$$P_1(x)P_2(x) = x^6 + x^5 - 2x^4 + 3x^3 + x^2 - 4x,$$
$$\frac{P_1(x)}{P_2(x)} = x^2 - x - 1 + \frac{6x - 4}{x^2 + x} \tag{3.22}$$

This is how MATLAB allows you to handle polynomials. You represent an nth-order polynomial by a vector of the length $n + 1$, arranging its

Table 3.17 Functions for Handling the Vectors Representing Polynomials

Function	Brief Description
`c=conv(a,b)`	(Abbreviated from *convolution*) Performs multiplication of two vectors, `a` and `b`, in a polynomial sense, so that the order of the vector `c` is equal to the sum of the orders of `a` and `b`
`[q,r]=deconv(a,b)`	(*Deconvolution*) Divides `a` by `b` in a polynomial sense, so that `q` is the quotient and `r` is the remainder [note that, to maintain the equality `a=conv(b,q)+r`, the vector `r` appears with the leading zeros to match the length of the vector `a`—this way it can be added to the result of `conv(b,q)` explicitly]

coefficients so that the first element represents the coefficient by the highest-order monomial, x^n, and the last coefficient—the lowest order, x^0, that is—in descending powers.

In addition to the aforementioned two functions, MATLAB provides a nice visualization tool to better understand the results of polynomial operations, the `poly2sym` function. For example,

```
>> s = poly2sym (c3)
```

yields

```
s = x^6+x^5-2*x^4+3*x^3+x^2-4*x
```

The return `s` belongs to a *symbolic* class of variables (`class (s)` returns `sym`), which will be addressed in Chapter 7, but what is important here is that for this class of variables another function, `pretty`, yields even better representation:

```
>> pretty(s)
      6     5       4       3     2
     x  + x  - 2 x  + 3 x  + x  - 4 x
```

The `sym2poly` function does the opposite. For example,

```
>> coef = sym2poly(s)
```

returns

```
coef = 1    1    -2    3    1    -4    0
```

So far, we considered three of several functions enabling polynomial handling. The complete list of these functions is presented in Table 3.18. Let us introduce a couple more functions from this list here, whereas the remaining ones will be considered in the corresponding chapters later on.

The `poly(r)` function computes coefficients of the polynomial, whose roots are given by a vector `r`. For example,

```
>> h = poly([1; -4; 5])
```

returns

```
h = 1    -2    -19    20
```

which means

$$H(x) = (x-1)(x+4)(x-5) = x^3 - 2x^2 - 19x + 20 \qquad (3.23)$$

You can check the result by applying the reciprocal function, `roots`, which computes the roots of a polynomial:

```
>> roots(h)'
ans = 5.0000    -4.0000    1.0000
```

(we used transposition for compactness).

If you apply the `poly` function to an n-by-n matrix $\mathbf{A}$, it will compute coefficients of the characteristic equation $|x\mathbf{I} - \mathbf{A}|$, where $\mathbf{I}$ is the n-by-n identity matrix.

Table 3.18 Polynomial-Handling Functions Summary

Function	Brief Description
`conv(a,b)`	Multiplies two polynomials
`deconv(a,b)`	Divides two polynomials
`poly(r)`	Creates a polynomial with the specified roots `r`
`polyder(a)`	Computes coefficients of a derivative polynomial defined by coefficients `a` (Chapter 12)
`polyint(a,c)`	Computes coefficients of a polynomial representing the integral of polynomial `a`, using a scalar constant of integration `c` (Chapter 13)
`polyeig(a1,...,ap)`	Solves the polynomial eigenvalue problem of degree *p*
`polyfit(x,y,n)`	Performs polynomial curve fitting, that is, finds the coefficients of a polynomial of degree `n` that fits (`x`;`y`) data supplied in the vectors `x` and `y` the best (Chapter 11)
`polyval(a,x)`	Evaluates a polynomial `a` at a given set of arguments specified in `x`
`polyvalm(a,M)`	Evaluates a polynomial with coefficients `a` in a matrix sense
`residue(a,b)`	Performs partial-fraction expansion (computes residues)
`roots(a)`	Finds the roots of polynomial `a` (Chapter 10)
`poly2sym(a)`	Converts a vector of coefficients `a` into symbolic polynomial representation
`sym2poly(s)`	Converts symbolic representation of a polynomial `s` to a vector of its coefficients

The following self-explanatory examples show how to use the `polyval` and `polyvalm` functions, evaluating a polynomial at a given set of arguments in a polynomial and matrix sense:

```
>> polyval([5; -4; 1],0:5)
ans = 1    2    13    34    65    106
>> polyval([5; -4; 1],eye(2))
ans = 2    1
      1    2
>> polyvalm([5; -4; 1],eye(2))
ans = 2    0
      0    2
```

By the way, note that in all three examples we defined a polynomial using a column vector, which is absolutely legitimate as well. These three commands are equivalent to

```
>> 5*[0  1  2  3  4  5].^2 -4 * [0  1  2  3  4  5]+1
>> 5*[1 0; 0 1].^2 -4 * [1 0; 0 1] + 1
>> 5*[1 0; 0 1]^2 -4 * [1 0; 0 1]^1 +[1 0; 0 1]^0
```

respectively.

Finally, the `residue` function converts a quotient of two polynomials to the pole-residue representation and back again. For example,

```
>> [r,p,k] = residue([2  -3  -9  16  -18],[1  -2  -5  6])
r = 3.0000
  -2.0000
   2.0000
p = 3.0000
  -2.0000
   1.0000
k = 2  1
```

which corresponds to

$$\frac{2x^4-3x^3-9x^2+16x-18}{x^3-2x^2-5x+6}=\frac{3}{x-3}-\frac{2}{x+2}+\frac{2}{x-1}+2x+1 \qquad (3.24)$$

You can check this result by applying this function "backwards":

```
>> [a,b] = residue([3  -2  2],[3  -2  1],[2  1])
a = 2  -3  -9  16  -18
b = 1  -2  -5   6
```

3.6 Handling Text as Character and String Arrays

MATLAB allows storing and handling sequences of characters pretty much the same way as sequences of numbers. The two options are the *character array* and *string array*.

Let us start with the character arrays. To create a character vector, you should enclose the characters in single quotes. For example,

```
>> object = 'aircraft'
object = 'aircraft'
>> number = '123.16'
number = '123.16'
```

Both created variables belong to the *char* class (type `class(object)` to get `char`). Note that the variable `number` is a character vector too, that is, it is not a decimal variable. This means that you cannot perform any arithmetic operations with it. The alternative way to create a character vector out of a decimal number is to use the `num2str` function (in the name of this and many other functions, "str" refers to the *text string*, but do not confuse it with the new class, `string`, introduced in R2016b). For example,

```
>> number = num2str(123.16)
```

produces the same result as the second command in the previous set. To convert integer numbers, there is another useful command, `int2num`. For example,

```
>> number = int2str(123.16)
```

returns

```
number = '123'
```

Let us check the size of the two character variables we just created:

```
>> size(object)
ans = 1     8
>> size(number)
ans = 1     3
```

As you can see, MATLAB does treat sequences of characters as the character arrays, dedicating one element for each character (in terms of memory, each character requires two bytes of memory). Therefore, a lot of noncomputational operations apply to character arrays as well. For example, you can concatenate them using brackets:

```
>> B = ['Mean of 1:10 is ' num2str(mean(1:10)) '.']
```

which returns

```
B =  'Mean of 1:10 is 5.5.'
```

or using a special function `append` (introduced in R2019a):

```
>> append('Mean of 1:10 is ',num2str(mean(1:10)),'.')
```

[There is another function that can be used, `strcat`, but as opposed to `append`, it removes trailing American Standard Code for Information Interchange (ASCII) whitespace characters.]

You can also address a specific character of a group of characters the same way you would for the numeric array. For example,

```
>> object(1)
ans = a
>> object(4:end)
ans = 'craft'
```

The `strfind` function may be useful for finding the locations of certain character(s):

```
>> strfind(object,'a')
ans = 1    6
```

If you want to convert a character that is an ASCII character representation of a numeric value back to its numeric representation, you need to use the `str2num` function. As a matter of fact, this function can do much more; for instance,

```
>> str2num(['123'  '+'  '17'])
```

returns the sum of two numbers

```
ans = 140
```

Applying `str2num` to the variable `object`, which is not a representation of a numeric value, results in an empty matrix, `[]`.

Now, let us talk about vertical concatenation of the character vectors. The only limitation is that they have to have the same length, that is, the same number of characters. For example, the command

```
>> carrierH = ['spacecraft' 'rotorcraft']
```

returns another sequence of characters concatenated horizontally (1-by-20 character vector):

```
carrierH = 'spacecraftrotorcraft'
```

and the command

```
>> carrierV = ['spacecraft'; 'rotorcraft']
```

results in vertical concatenation and produces a 2-by-10 character matrix as follows:

```
carrierV =
          2 × 10 char array
          'spacecraft'
          'rotorcraft'
```

(Note, MATLAB explicitly specifies the class and size of a resulting array, but for compactness, we will omit this line everywhere throughout this book.) The command

```
>> vehicle = ['spacecraft';'aircraft';'ship';'submarine']
```

however, evokes an error

```
Error using vertcat. Dimensions of arrays being concatenated are not
consistent.
```

That is why to concatenate sequences of characters vertically, you are better off using the special function `strvcat`, which appends spaces to each sequence of characters as necessary to form a valid matrix:

```
>> vehicle = strvcat('spacecraft','aircraft','ship','submarine')
vehicle = 'spacecraft'
          'aircraft  '
          'ship      '
          'submarine '
```

To assure that you do have spaces in your character array, you may run the `strfind` command looking for them. For example,

```
>> strfind(vehicle(3,:),' ')
```

returns

```
ans = 5  6  7  8  9  10
```

meaning that the length of `ship` is six characters shorter than that of the longest row, `spacecraft`, and therefore spaces were added to equalize them.

The find function of Table 3.14 happens to be very useful in handling character arrays as well. For example, the following command finds indices of the rows that start from s:

```
>> i = find(vehicle(:,1)=='s')
i = 1
    3
    4
```

followed by another command that uses these indices to retrieve the entire rows:

```
>> vehs = vehicle(i,:)
ans = 'spacecraft'
      'ship      '
      'submarine '
```

Transposition is also a legitimate command for the character arrays:

```
>> h1 = vehs'
h1 = 'sss'
     'phu'
     'aib'
     'cpm'
     'e a'
     'c r'
     'r i'
     'a n'
     'f e'
     't   '
```

Applying the rot90 function of Table 3.15

```
>> rot90(vehs,2)
```

returns

```
ans = ' enirambus'
      '      pihs'
      'tfarcecaps'
```

The char function allows you to do the same thing as the strvcat function—it automatically pads each character sequence with the blanks to form a valid matrix:

```
>> vehicle = char('spacecraft','aircraft','ship','submarine')
vehicle = 'spacecraft'
          'aircraft  '
          'ship      '
          'submarine '
```

Another syntax of the char function, char(a), converts an array a that contains nonnegative integers representing character codes into a MATLAB character array. For example, the following command prints a 2-by-80 display of the printable ASCII characters between 33 and 192:

```
>> disp(char(reshape(33:192,80,2)'))
```

```
!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnop
qrstuvwxyz{|}~ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ ¡¢£¤¥¦§¨©ª«¬-®¯°±²³´µ¶·¸¹º»¼½¾¿À
```

Compare this table with the one that shows in Microsoft Office when you are trying to insert a symbol (Fig. 3.23). The actual characters displayed depend on the character set encoding for a given font. MATLAB uses a font that Microsoft Office does not have (see Preferences as discussed in Sec. 2.7), but at least the first 127 characters are the same. In fact, MATLAB uses the integers up to 65535 corresponding to the Unicode characters; for example,

```
>> disp(char(reshape([1040:1199 8200:8359 8450:8609],80,6)'))
```

returns

Let us find the character ASCII code for the letter Z:

```
>> strfind(char(1:127),'Z')
ans = 90
```

Now we can try to apply basic operations to character arrays and understand the results. For instance,

```
>> 'Z'-'X'
```

returns

```
ans = 2
```

and simply means how far the letter Z is behind the letter X.

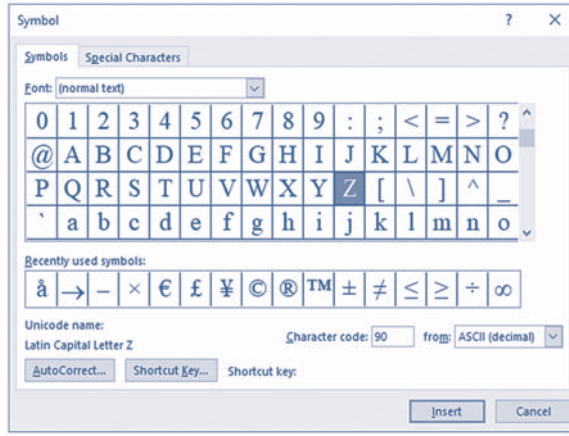


Fig. 3.23 Insert Symbol window in Microsoft Office.

Similarly, adding `'aircraft'` to `'ship'`

```
>> vehicle(2,:)+vehicle(3,:)
```

does not mean that we are creating a carrier but, rather, presents a vector, which contains a sum of character codes of the corresponding elements in both sequences of characters:

```
ans = 212   209   219   211   146   129   134   148   64   64
```

You can try to apply the `char` function to it

```
>> char(ans)
```

to see something like this

```
ans = 'ÔÑÛÓ □ □ □ □ @@'
```

Let us consider one more example of using `strfind` and some other functions introduced earlier in this chapter to provide you with a piece of code you may find useful. Suppose you want to permute letters in the word "space." First, you have to randomly pick the first letter. You can do it using the same procedure we used before, say `ceil (rand(1)*N)` generates a random number between `0` and `N`. Alternatively, you could simply use the `randi(N+1)-1` call. Then, you can throw this letter away, that is, reduce the length of the original character sequence by one and proceed with picking the next letter, and so on. Hence, running the commands

```
>> f = 'space';
>> l1 = f(ceil(rand(1)*5)); f(strfind(f,l1))=[];
>> l2 = f(ceil(rand(1)*4)); f(strfind(f,l2))=[];
>> l3 = f(ceil(rand(1)*3)); f(strfind(f,l3))=[];
>> l4 = f(ceil(rand(1)*2)); f(strfind(f,l4))=[];
>> strcat(l1,l2,l3,l4,f)
```

may result in something like

```
ans = 'asecp'
```

so that you may ask your friends to guess what meaningful word they can get out of it. We used this example to practice using several functions and show you a couple of tricks, but in practice, the same result could be achieved by issuing a single command:

```
>> f = 'space';
>> f(randperm(length(f)))
```

Now, that you know how to concatenate character vectors, let us introduce the `newline` function that you may find very useful. This function creates a newline character and is equivalent to `char(10)` or `sprintf('\n')` (the latter function is introduced in Sec. 4.9). For example, the call

```
>> L=['You were born with wings,...' newline...
      'why prefer to crawl through life?']
```

creates a 1-by-61 character vector spread over two lines

```
L = 'You were born with wings,...
    why prefer to crawl through life?'
```

Until now, we have talked about the first of two options to handle character sequences, the character array. In R2016b, another option, the string array, was introduced. Creating a string array is similar to creating a character array, with the only difference being that the text should be enclosed in double rather than single quotes. For example, repeating the two earlier examples in this section, the call

```
>> carrierHs = ["spacecraft" "rotorcraft"]
```

now returns a 1-by-2 string array (as opposed to a 1-by-20 character vector)

```
carrierHs =
  1×2 string array
      "spacecraft" "rotorcraft"
```

and the vertical concatenation command that did not work for the character vectors now returns a 4-by-1 string array

```
>> vehicles = ["spacecraft"; "aircraft"; "ship"; "submarine"]
vehicles =
  4×1 string array
      "spacecraft"
      "aircraft"
      "ship"
      "submarine"
```

As seen, each element of a string array stores a sequence of characters. These sequences can have different lengths and no padding is needed. (As mentioned above for the character arrays, MATLAB explicitly specifies the class and size of a resulting string array, but for compactness, we will omit this line everywhere throughout this book as well.)

You can also use the `string` function. For example,

```
>> string('aircraft')
```

creates a *string scalar*

```
ans = "aircraft"
```

and

```
>> z=string([10   12   13   14])
```

converts the four-element numeric vector into a 1-by-4 string array

```
z = "10"   "12"   "13"   "14"
```

The reverse operation is conducted using the `str2double` function:

```
>> str2double(z)
ans = 10    12    13    14
```

You can index into, reshape, and concatenate string arrays using standard array operations, and you can append text to strings using the `+` operator. For example, the call

```
>> 'US' + vehicles
```

returns

```
ans = "US spacecraft"
      "US aircraft"
      "US ship"
      "US submarine"
```

The `newline` function introduced earlier works with the strings as well. For example,

```
>> L="No one cares how much you know, ..." + newline +...
     "until they know how much you care"
```

displays two lines even though it is still a 1-by-1 string

```
L = "No one cares how much you know, ...
    until they know how much you care"
```

Appendix B presents a wide range of the functions, available as of R2020a, that can be used with the character and string arrays (some of these functions have already been introduced earlier in this section). At the end of Appendix C, you can also find an example of using the `wordcloud` function to create a word cloud chart from text data. You are welcome to explore these functions on your own as needed.

To conclude this section, let us introduce one more function, `eval`, to show how concatenation of the character vectors allows you to create names of variables programmatically (you may want to use this feature when you become a proficient MATLAB user). [The call `eval(S)` allows evaluating a MATLAB expression in the text string `S`.]

Assume that you are computing some quantity that differs from one set of input data to another; say you are varying the number of data points. Suppose you have some variable `Speed` that was computed using 10 date points and equals `150`. Using `eval` enables reassigning data from `Speed` to a programmatically created `SpeedFor10Points` that directly indicates what dataset was used:

```
>> v = ['SpeedFor' num2str(10) 'Points'];
>> eval([v ' = ' 'Speed' ';']);
```

Upon execution, two variables, `v ='SpeedFor10Points'` and `SpeedFor10Points=150`, are created. When creating a variable name (in the

first line) you might want to check the validity of the name; in this case you could use one of the two functions introduced in R2014a:

```
>> v = matlab.lang.makeValidName(['SpeedFor' num2str(10) 'Points']);
```

or

```
>> v = matlab.lang.makeUniqueStrings(['SpeedFor' num2str(10) 'Points']);
```

Problems

3.1 Use the vector notation and `linspace` function to create five vectors with constant spacing and concatenate them vertically to form a matrix

$$\texttt{B1} = \begin{pmatrix} 1 & 4 & 7 & 10 & 13 & 16 & 19 \\ 72 & 66 & 60 & 54 & 48 & 42 & 36 \\ 0 & 0.125 & 0.25 & 0.375 & 0.5 & 0.625 & 0.75 \\ 0.3 & 0.4 & 0.5 & 0.6 & 1.2 & 1.4 & 1.6 \\ 9 & 8 & 7 & 6 & 5 & 4 & 3 \end{pmatrix}$$

Then, address the corresponding elements and replace them with the `zeros(1,4)`, `ones(1,4)`, `zeros(2,3)`, and `magic(3)` matrices to obtain

$$\texttt{B2} = \begin{pmatrix} 1 & 4 & 7 & 10 & 8 & 1 & 6 \\ 0 & 66 & 0 & 54 & 0 & 42 & 0 \\ 1 & 1 & 1 & 1 & 3 & 5 & 7 \\ 0 & 0 & 0 & 0.6 & 4 & 9 & 2 \\ 0 & 0 & 0 & 6 & 5 & 4 & 3 \end{pmatrix}$$

Employ the `spy` function to visualize nonzero elements of matrix `B2` and the `find` function to find the row and column indices of nonzero elements.

3.2 An aerial delivery system release altitude is $h_0 = 1000$ m. However, something went wrong and the system falls ballistically. Using the formula $h = h_0 - 0.5gt^2$, where t is the time in seconds and $g = 9.8$ m/s^2 is the acceleration due to gravity, determine system's altitude at $t = 0, 1, \ldots, 10$ s. (Create a vector t and determine h using element-by-element calculation.)

3.3 Use arrays to handle polynomials and character sequences. For both cases considered, employ the `randi([0 M],1,N)` command to generate a row vector with `N` random integers within the range `[0;M]`. Do the following:

a) Create the seven-element and three-element vectors with `M` = 6 and `M` = 10, respectively. Treating these vectors as coefficients of polynomials, use the `deconv` function to find a quotient. Display a symbolic polynomial representation of this quotient (using the `poly2sym` and `pretty` functions).

b) Use `M` = 26 and generate a sequence of `N` = 26 random integers. Using the `char` function, convert this sequence to the text string containing letters A – Z (add 64 to the integers sequence, because the character code for letters starts from 65). Check whether your text string contains a letter A (use the `strfind` function). Reshape your text vector to a 2-by-13 matrix. Check whether this matrix has columns containing the same letter (use the `find` function applied to the difference between the two rows).

Chapter 4 Mixed-Type Data "Containers" and Input/Output Management

- Cell and Structure Arrays, Tables
- Importing and Exporting Data
- Displaying Formatted Data
- Interactive Input and Output

`cell`, `struct`, `table`, `[]`, `num2str`, `save`, `load`, `textread`, `textscan`, `xlsread`, `importdata`, `dlmread`, `readmatrix`, `readcell`, `imread`, `VideoReader`, `fprintf`, `sprintf`, `input`, `menu`, `inputdlg`, `uigetfile`, `pause`, and others

4.1 Introduction

Before proceeding with writing sophisticated programs, this chapter concludes the introduction to MATLAB as an effective technical environment and engineering tool. It also addresses some additional topics allowing you to become a proficient MATLAB user. It begins with a review of MATLAB data types followed by a detailed description of the cell arrays, structure arrays, and tables, enabling storing and handling mixed-type data. Next, the basic types of MATLAB files are formally introduced. This is followed by presenting a few more MATLAB functions that allow a user to manage workspace variables (`save` and `load`) and store all commands issued and all responses received during the current session (`diary`). This chapter further proceeds with unformatted import/export of data from the different data files (`dlmread`, `xlsread`, `textscan`, `readmatrix`, and other functions) and writing formatted data to a file/text string or displaying it on a screen (`fprintf` and `sprintf`). The chapter ends with presenting several user-friendly functions enabling interactive input of data (including the `input` and `menu` functions) and managing a program flow using the predefined (readily available) dialog boxes (`inputdlg`, `uigetfile`, `waitbar`, and other functions).

4.2 Data Types

MATLAB supports several fundamental and a variety of additional (including derivative) data types (classes), as shown in Fig. 4.1. The function handles are always scalars, that is, 1-by-1 matrices (see Fig. 3.1). Other fundamental data types can be in the form of a matrix or array. Such a matrix or array is a minimum of 0-by-0 in size (empty array) and can grow up to an *n*-dimensional array of any size. Table 4.1 provides some simple examples of creating variables of the different fundamental data types. Figure 4.2 shows the content of MATLAB workspace after creating all variables of Table 4.1, clearly indicating their class. You can observe the names of these data types in the Class column of the Workspace browser (see Figs. 2.15a and 3.8) or explicitly request the class of variable using the `class` function.

The numeric data types enable numeric computations. The major difference between them is a memory allocation to store one element of such a data type. By default, MATLAB assigns a double-precision type, `double`, to all numeric data, meaning that it allocates eight bytes of memory to store one numeric value. In Chapter 3, you have already performed some mathematical operations with this type of data. A `single` precision scalar requires four bytes of memory. Signed and unsigned integer scalars may occupy one to eight bytes of memory. All numeric types support basic mathematical operations. Chapter 7 will provide you with more details on the ways numeric data are represented and stored in computer's memory.

In Chapter 3, you had a chance to familiarize yourself with `char` and `string` data types assigned to data, which are represented by characters and text strings. The `logical` and `function_handles` data types will be explained in Chapter 5. Another class, symbolic variables, `sym` (not shown in Fig. 4.1 because it belongs to the Symbolic Math Toolbox), will be discussed in detail in Chapter 7.

Matrices and arrays can be composed of elements of most any data type as long as all elements in the matrix are of the same type. If you mix class type when constructing an array, MATLAB converts some elements so that all

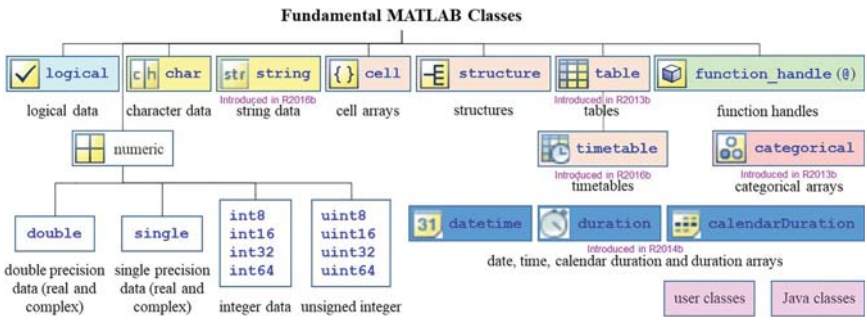


Fig. 4.1 MATLAB data types.

Table 4.1 Fundamental Data Types MATLAB

Data Type	Examples	Description

double	`>> w = 2.5*10^300, h = 2-i`
	`w = 2.5000e+300`
	`h = 2.0000 - 1.0000i`
		The default numeric type in MATLAB

single	`>> single(3*10^300)` `ans = Inf`	Requires less storage space than double but has less precision and a smaller range

int8, int16, int32, int64
uint8, uint16, uint32, uint64
	`>> a = int8(500)`
	`a = 127`
	`>> b = uint8(500)`
	`b = 255`
		Enables more efficient use of memory for signed and unsigned integers

logical	`>> f = rand(2) > 0.5`
	`f = 1 1`
	`0 1`
		Is used to represent the logical values of 1's and 0's that represent the true and false values, respectively

char	`>> c = 'Summer'` `c = 'Summer'`	Is used to store a sequence of characters
string	`>> s = ["WA" "OR" "CA"]` `s = "WA" "OR" "CA"`	Is used to store sequences of characters of different lengths

cell	`>> d{1,1} = 12;`
	`>> d{1,2} = 'Red';`
	`>> d{1,3} = magic(4)`
	`d = {[12]} {'Red'} {4 × 4 double}`
		Is used to store mixed-type data in a manner similar (in terms of indexing) to numeric or logical arrays

structure	`>> u.day = 12;`
	`>> u.color = 'Red';`
	`>> u.mat = magic(3)`
	`u = day: 12`
	`color: 'Red'`
	`mat: [3 × 3 double]`
		Is used to store mixed-type data in a C-like structure composed of the named fields

table	`>> T = table([1934;1923],...`
	`{'R';'A'},'VariableNames',...`
	`{'Year' 'State'},...`
	`'RowNames',{'Yuri';'Alan'})`
	`T =`
	`Year State`
	`Yuri 1934 {'R'}`
	`Alan 1923 {'A'}`
		Is used to store mixed-type data in a rectangular column-oriented container

function_handle	`>> p = @sin` `p = @sin`	A pointer to a function (handle) that can be saved or passed as an input argument to another function

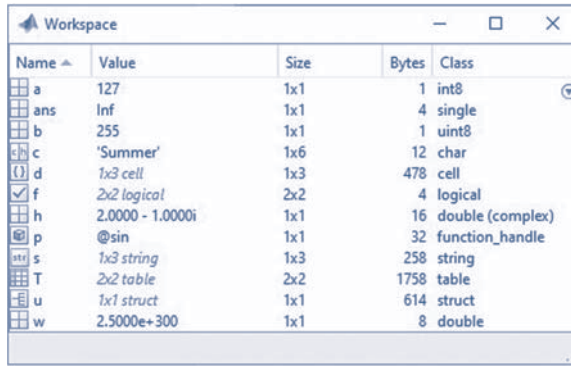


Fig. 4.2 MATLAB workspace with variables defined in Table 4.1.

elements of the resulting matrix are of the same type. Table 4.2 shows the six classes you can concatenate with an unlike type without generating an error (with the exception of character and logical).

If you want to preserve the unlike types of data within the same variable, you have to use one of the three remaining fundamental classes of data—a cell array, structure array, or table. Before dealing with them, let us briefly mention other (nonfundamental) classes shown in Fig. 4.1.

The `categorical` data type was introduced together with the `table` class in R2013b to define the groups of rows by their values from a finite set of discrete categories (e.g., good/bad, A/A-/B+/B/B-/C/..., or small/medium/large). A categorical array provides efficient storage and convenient manipulation of nonnumeric data while also maintaining the meaningful names for the values. The `categorical` function allows creating an ordinal categorical array from a cell array of text strings. The `discretize` function can be used to group even numeric data into categories. One simple example of creating and using an ordinal categorical array will be given in Sec. 4.5, but for more details we refer you to Appendix C.

The date and time data types, `datetime`, `duration`, and `calendarDuration`, were introduced in R2014b to support efficient computations, comparisons, and formatted display of dates and times. Again, one

Table 4.2 Valid Combinations of Unlike Classes

Type	character	string	integer	single	double	logical
character	character	string	character	character	character	invalid
string	string	string	string	string	string	string
integer	character	string	integer	integer	integer	integer
single	character	string	integer	single	single	single
double	character	string	integer	single	double	double
logical	invalid	string	integer	single	double	logical

example of using the `datetime` data class will be presented in Sec. 4.5, but for more details we refer you to Appendix C, where all three date and time data handling classes are discussed in conjunction with a special type of table, timetable, introduced in R2016b and also shown in Fig. 4.1.

All MATLAB data types are implemented as object-oriented classes. Hence, you can add data types of your own to your MATLAB environment by creating additional classes. These user-defined classes (Fig. 4.1) define the structure of your new data type, and the *m*-file functions or methods. The latter defines the behavior of that data type. A Java class is another MATLAB data type providing an interface to the Java programming language, enabling creation of the objects from Java classes and calling Java methods on these objects.

Now it is time to get back to those three classes we skipped earlier. The `cell` and `structure` fundamental data types allow storing heterogeneous (mixed-type) data all together in one variable. The `table` data type may be quite useful in storing flat (matrix-type) tables containing heterogeneous data as well. Hence, these three data types require special attention and are explained in the following three sections. Before proceeding with this discussion, Table 4.3 shows a variety of functions you can use to convert from one data type to another (if needed).

4.3 Cell Arrays

In Sec. 3.6, we faced the problem of not being able to effectively store the different-length character sequences in a single character array. A remedy introduced there was to use the string arrays instead. Now, let us revisit the example of Sec. 3.6 and accomplish the mission using the *cell array*.

The following command utilizing the curly brackets (braces) as opposed to the square brackets

```
>> vehicle={'spacecraft';'aircraft';'ship';'submarine'}
```

solves the problem and creates a valid 4-by-1 cell array:

```
vehicle = {'spacecraft'}
          {'aircraft'}
          {'ship'}
          {'submarine'}
```

(As it was the case for the character and string arrays (see. Sec. 3.6), MATLAB explicitly specifies the class and size of a resulting array, but for compactness, this line is omitted).

Each cell by itself requires 112 bytes, so the total number of bytes required to store this particular array is 510 (112×4 bytes to maintain the 4-by-1 cell array plus 2 bytes for each of 31 characters).

You can refer to either the individual cell of a cell array (which is called *cell indexing*) or the cell's content (*content indexing*). For instance,

Table 4.3 Type Conversion Functions

Type	Numeric	Math Expression	Character	String	Symbolic Expression	Cell	Structure	Table	Function Handle	Inline Function
Numeric			`int2str`, `num2str`, `mat2str`	`siring`	`sym`	`num2cell`, `mat2cell`		`array2table`		
Math Expression			'' (single quotes)	"" (double quotes)						
Character	`eval`, `str2num`, `str2double`	`eval`, `str2num`		`string`	`str2sym`				`str2func`	`inline`
String	`double`, `str2num`, `str2double`	`eval`, `str2num`	`char`, `str2mat`		`str2sym`				`str2func`	`inline`
Symbolic Expression	`double`, `vpa`		`char`						`matlabFunction`	
Cell	`cell2mat`		`cell2str`		`cell2sym`		`cell2struct`	`cell2table`		
Structure						`struct2cell`		`struct2table`		
Table	`table2array`					`tabler2cell`	`table2struct`			
Function Handle			`func2str`							
Inline Function			`char`							

```
>> a = vehicle(3)
```

returns

```
a = {'ship'}
```

with `a` being a 1-by-1 cell, and

```
>> b = vehicle{3}
```

yields

```
b = 'ship'
```

that is, the content of the third cell, which means you can use `b` in a variety of operations applicable to the variables of the `char` type, such as

```
>> sort(b)
ans = 'hips'
```

You can also mix both types of indexing. For example,

```
>> vehicle{2}(4:end)
```

where we use the braces to access the content of a specific cell and parentheses to address a certain part of this content, returns

```
ans = 'craft'
```

Note that in the latter case, you can only address the content of a single cell, that is, the command

```
>> vehicle{2:3}(1:3)
```

returns an error

```
Expected one output from a curly brace or dot indexing expression, but there
were two results.
```

Of course, a cell array can hold not only character sequences but anything you want: a scalar, a numeric array, a text string, another cell, etc. Hence, a cell array allows you to effectively handle mixed-type data. For instance, consider a 2-by-2 cell array `c` containing the two text strings, matrix, and vector (its graphical interpretation is given in Fig. 4.3)

```
c{1,1} = "2-by-2";
c{1,2} = 'eigenvalues of eye(2)';
```

Cell 1,1 "2-by-2"	Cell 1,2 'eigenvalues of eye(2)'
Cell 2,1 $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	Cell 2,2 $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$

Fig. 4.3 Graphical interpretation of the cell array `c`.

```
c{2,1} = eye(2);
c{2,2} = eig(eye(2));
```

MATLAB has a handy function, `cellplot`, to visualize a structure and distinguish a content of the cells. For instance, typing

```
>> cellplot(vehicle)
```

and

```
>> cellplot(c)
```

for the two aforementioned examples produces a graphical display of the cell array's contents shown in Fig. 4.4a and 4.4b, respectively.

Let us consider one more example. The following set of commands creates a cell array that contains some generalized data on five generations of jet fighter aircraft:

```
Gen{1,1,1}='Ramjet';          % type of engine
Gen{2,1,1}=[0.3 1; 0.8 6];    % thrust to weight ratio, CL_max, M_max, nz_max
Gen{1,2,1}=[12000 900];       % ceiling (m), range (km)
Gen{2,2,1}=[0.1; 4000];       % price (million dollars), number manufactured
Gen(1,1,2)={'Turbojet'};
Gen(2,1,2)={[0.4 0.6; 0.9 7]};
Gen(1,2,2)={[14000 1100]};
Gen(2,2,2)={[0.2; 34000]};
Gen(1,1,3:5)={'Turbojet w/AB' 'Turbojet w/AB' 'LBRTurbofan'};
Gen(2,1,3:5)={[0.6 0.8; 2.2 8] [0.73 1.6; 2 9] [0.6 1.1; 1.6 7]};
Gen(1,2,3:5)={[16000 1700]; [17000 2200]; [16000 2000]};
Gen(2,2,3:5)={[1.4; 43000], [30; 1000], [150; 1000]};
```

(The text appearing after the symbol % is a comment, which is ignored when you execute a command.) This set of commands features some alternatives you may want to exploit when assigning values to a cell array. The first

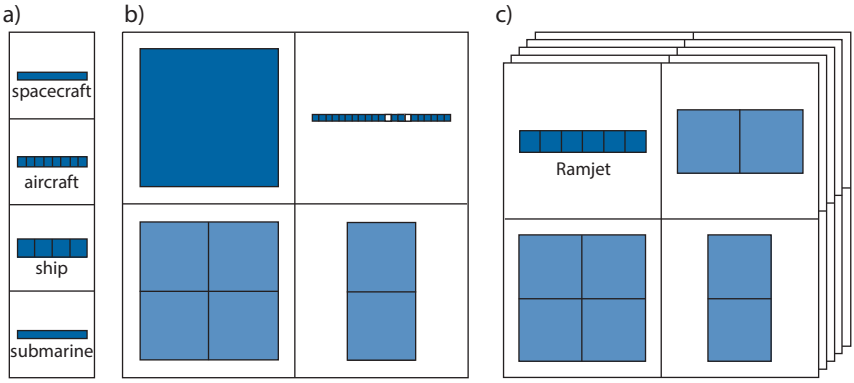


Fig. 4.4 Content of the cell array a) `vehicle`, b) `c`, and c) `Gen`.

four lines define data for the first generation of aircraft (the first page or layer in Fig. 4.4c). The next four lines define the second generation. Here, we present another option of using the braces, {}, on the right-hand side, that is, converting different objects to cell elements while performing an operation on the right as opposed to doing it when assigning the result of this operation to the cell element on the left as earlier. Finally, the last four lines use the bulk assignment (in this case, the braces may be used only on the right-hand side of expressions).

Note, the `cellplot(Gen)` command used to generate Fig. 4.4c only shows a content of the very first page, that is, you cannot see what lies behind it. In a general case, if you want to display a content of some multidimensional cell array, you have to do it layer by layer. For example, if you want to see the third page, you should type `cellplot(Gen(:,:,3))`.

As mentioned earlier, you may address any specific portion of a single cell's content by mixing the two types of indexing, like `Gen{2,1,2}(1,2)`. However, you cannot mix indexing to access several cells simultaneously. Is there any way to overcome this inconvenience? Well, the following presents an example of how you could do this:

```
>> format bank
>> Data=[Gen{2,1,:}]
>> [a,b]=max([Data(3:4:end)])
>> format
```

The trick is to assign the content of the multiple cells to some variable, in this case `Data`, and then treat this new variable in a usual way. The preceding commands yield the following output:

```
Data = Columns 1 through 5
    0.30   1.00   0.40   0.60   0.60
    0.80   6.00   0.90   7.00   2.20
      Columns 6 through 10
    0.80   0.73   1.60   0.60   1.10
    8.00   2.00   9.00   1.60   7.00
a = 1.60
b = 4.00
```

This means that the fourth-generation aircraft has the largest lift coefficient $C_{L\max}$ of 1.6. (Note that we used the first `format` command (introduced in Sec. 2.6) to make the output more compact and then returned it to default by issuing another `format` command.)

To conclude this section, Table 4.4 formally lists all MATLAB functions you can use in conjunction with the cell arrays.

Also, you may find it useful to use the nested cell arrays that have cells containing cell arrays within cells that contain cell arrays, and so on. To create the nested cell arrays, use the nested braces {{}}.

Table 4.4 Cell Array-Handling Functions

Function	Brief Description
Create a cell array	
`cell(n,m,...)`	Creates an `n`-by-`m`-by-`...` cell array of empty matrices
`cellstr(C)`	Converts the character array `C` to a cell array of strings (also used in conjunction with a datetime, duration, or calendar-duration array)
`num2cell(B)`	Creates a cell array by placing each element of the numeric array `B` into a separate cell
`mat2cell(A,dimn,dimm,...)`	Divides the numeric array `A` into smaller arrays and returns them in a cell array with potentially different sized cells [vectors `dimn`, `dimm`, ... specify how to divide the rows, columns, and (when applicable) higher dimensions of `A`]
`struct2cell(S)`	Converts the *m*-by-*n* structure `S` (with *p* fields) into a *p*-by-*m*-by-*n* cell array
`table2cell(T)`	Converts the table `T` to a cell array
Reveal a cell array content	
`celldisp(C)`	Displays the contents of the cell array `C`
`cellplot(C)`	Displays a graphical depiction of the cell array `C`
`iscell(C)`	Determines whether the input `C` is a cell array
`iscellstr(C)`	Determines whether the input `C` is cell array of strings
Convert cell arrays to different data types	
`[o1,o2,...]=deal(C{:})`	Distributes the inputs of the cell array `C` to the requested to outputs `o1,o2,...`
`cell2mat(C)`	Converts the cell array `C` to a an ordinary array of the underlying data type (inverse to `mat2cell`)
`cell2struct(C,f,dim)`	Converts the cell array `C` to a structure array (the fields argument `f` specifies the field names for the structure array)
`cell2table(C)`	Converts the cell array `C` to a table
Cell operations	
`cellfun(fun,C)`	Applies the function `fun` to the contents of each cell of a cell array `C`, one cell at a time

4.4 Structure Arrays

Similar to the cell arrays, the *structure arrays* may be composed of dissimilar ordinary arrays. The main difference is that the structure arrays are accessed by the *named fields*, not by standard indexing. In some sense, such indexing is similar to spreadsheet/column/row indexing in a Microsoft Excel spreadsheet. You may assign the values directly to the structure array using the following syntax:

```
variable.field_name=value
```

For example, the set of instructions

```
>> vehicle.type ='spacecraft';
>> vehicle(2).type ='aircraft';
>> vehicle(3).type ='ship';
>> vehicle(4).type ='submarine'
```

creates a 1-by-4 structure as follows:

```
vehicle = 1×4 struct array with fields:
    type
```

(for compactness, this line explicitly specifying the class and size of a resulting array will henceforth be omitted).

You may add the additional fields by just typing the name of a new field after the period, such as

```
>> vehicle(2).domain ='atmosphere';
```

In this case, this new field is added to every element of structure. Of course, they hold an empty matrix until alternative values are assigned to them (Fig. 4.5). As was the case with the cell arrays, the structure array also requires some overhead memory. In the preceding example, storing 41 characters would normally require only 82 bytes, but maintaining the 1-by-4 structure with two fields requires an additional 712 bytes (794 bytes in total).

Let us consider a more sophisticated (practical) example. Suppose we want to use a structure to store the heterogeneous data shown in Table 4.5 (the figures inside parentheses indicate the total number of missions flown by each astronaut).

The following set of commands does the job (Fig. 4.6):

```
Lander.Name                = 'Eagle/Apollo 11';
```

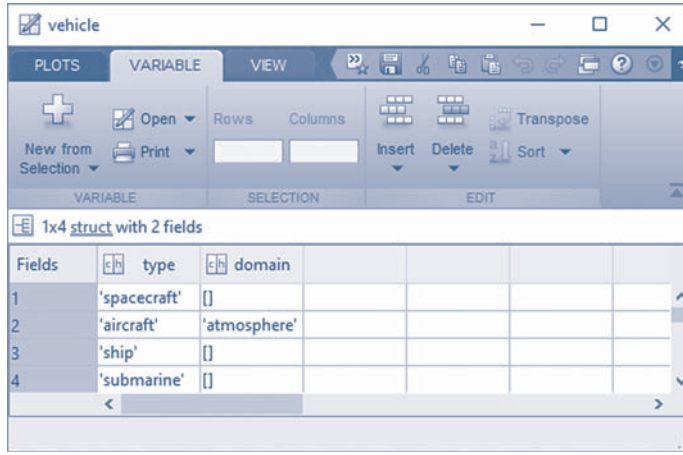


Fig. 4.5 Content of the `vehicle` structure.

```
Lander.LandingDate       = '20-Jul-69';
Lander.TimeonSurface     = '21:31';
Lander.Crew              = {'Neil Armstrong';'Edwin Aldrin'};
Lander.EVATime           = '02:31';
Lander.Flights           = [2  2];
Lander(2).Name           = 'Intrepid/Apollo 12';
Lander(2).LandingDate    = '19-Nov-69';
Lander(2).TimeonSurface  = '31:31';
Lander(2).Crew           = {'Charles Conrad';'Alan Bean'};
Lander(2).EVATime        = '07:45';
Lander(2).Flights        = [4  2];
Lander(3)                = struct('Name','Antares/Apollo 14',...
                            'LandingDate','05-Feb-71',...
'TimeonSurface','33:30','Crew',{{'Alan Shepard';'Edgar Mitchell'}},...
'EVATime','09:21','Flights',[2  1]);
Lander(4:6)= struct('Name',{'Falcon/Apollo 15','Orion/Apollo 16',...
'Challenger/Apollo 17'},'LandingDate',{'30-Jul-71','21-Apr-72',...
'11-Dec-72'},'TimeonSurface',{'66:55','71:02','74:59'},'Crew',...
{{'David Scott';'James Irwin'},{'John Young';'Charles Duke'},...
{'Eugene Cernan';'Harrison Schmitt'}},'EVATime',...
{'18:33','20:14','22:04'},'Flights',{[3  1],[6  1],[3  1]});
```

Specifically, the first six commands take care of the first line in Table 4.5 (note how a cell array is used to keep multiple names in the field `Crew`). Similarly, the next six commands fill data for the second element of the structure `Lander`. To fill the remaining lines of Table 4.5, the bulk form is used. The third element is created using the MATLAB function `struct`, which creates a structure array with the specified fields and values. The remaining elements are created using the same function `struct` but employing cell

Table 4.5 Manned Moon Landing Missions

Lander/Mission	Landing Date	Time on Surface	Crew	EVA[a] Time
Eagle/Apollo 11	20 Jul. 1969	21:31	Neil Armstrong (2), Edwin Aldrin (2)	2:31
Intrepid/Apollo 12	19 Nov. 1969	31:31	Charles Conrad (4), Alan Bean (2)	7:45
Antares/Apollo 14	5 Feb. 1971	33:30	Alan Shepard (2), Edgar Mitchell (1)	9:21
Falcon/Apollo 15	30 Jul. 1971	66:55	David Scott (3), James Irwin (1)	18:33
Orion/Apollo 16	21 Apr. 1972	71:02	John Young (6), Charles Duke (1)	20:14
Challenger/Apollo 17	11 Dec. 1972	74:59	Eugene Cernan (3), Harrison Schmitt (1)	22:04

[a]EVA stands for extravehicular activity.

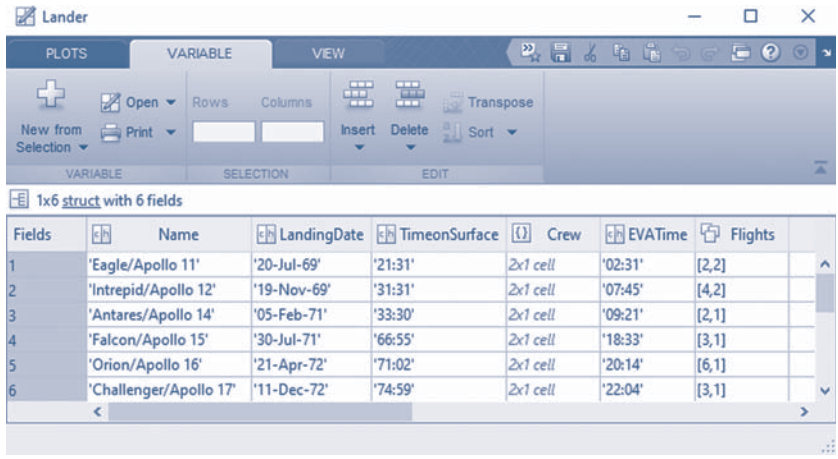


Fig. 4.6 Content of the `Lander` structure.

arrays to assign multiple values to the same field of different elements. Note that in the two latter cases (when using the `struct` function), the field name should always be followed by its value; otherwise it will not work. If for any reason you do not want to assign the value right away but rather want to skip it and assign later, you must use a placeholder, an empty matrix `[]`.

Now, accessing the elements of structure arrays is as easy as follows. To access the entire element of a structure, type structure's name and element number. For instance,

```
>> disp(Lander(3))
```

returns

```
         Name: 'Antares/Apollo 14'
  LandingDate: '05-Feb-71'
TimeonSurface: '33:30'
         Crew: {2×1 cell}
      EVATime: '09:21'
      Flights: [2  1]
```

To access a specific field, type a period after the structure array name followed by the field name. For example,

```
>> Lander(3).EVATime
```

returns a text string

```
ans = '09:21'
```

To access elements within the field, add their index in the parentheses. For example,

```
>> Lander(5).EVATime(1:2)
```

yields

```
ans = '20'
```

As in the case with cell arrays, the best thing about keeping heterogeneous data together is that you can easily manipulate it using the common MATLAB operators and functions. However, be aware that you cannot address the same field of several structure elements directly. For instance, an attempt

```
>> Lander(:).LandingDate
```

returns

```
ans = '20-Jul-69'
ans = '19-Nov-69'
ans = '05-Feb-71'
ans = '30-Jul-71'
ans = '21-Apr-72'
ans = '11-Dec-72'
```

Hence, we need to use a couple of tricks. Here is one example of how you can still operate on the same field of multiple structure elements. The following call

```
>> Dates=reshape([Lander(:).LandingDate]',9,6)'
```

collects all multiple returns into one vector. By reshaping it, the call converts all data of the field `LandingDate` into a single array `Dates` that you can use for further operations:

```
Dates = '20-Jul-69'
        '19-Nov-69'
        '05-Feb-71'
        '30-Jul-71'
        '21-Apr-72'
        '11-Dec-72'
```

Similarly, the following commands

```
>> y=[Lander(:).EVATime];
>> y=reshape(y,5,6)';
>> EVAMinutes = str2num(y(:,1:2))*60 + str2num(y(:,4:5))
```

first collect `EVATime` data to a single 30-by-1 character array, then reshape it into a 6-by-5 character array (containing the hh:mm text strings), and finally operate on each line of it to convert the text strings representing time in hours and minutes to a numeric array showing the same value in minutes:

```
EVAMinutes = 151
              465
              561
             1113
             1214
             1324
```

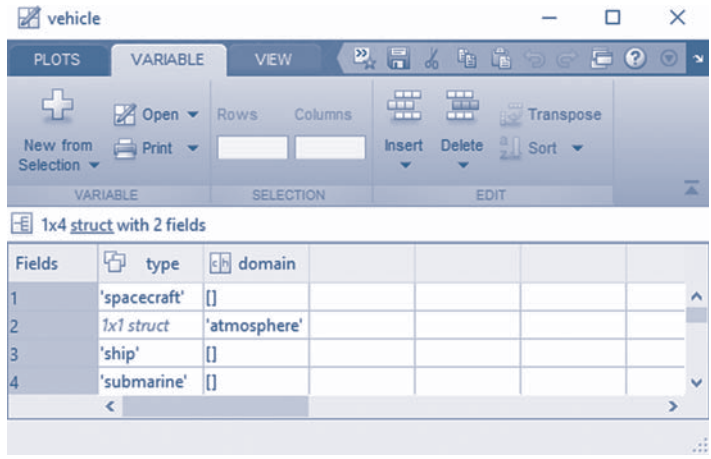


Fig. 4.7 Content of the modified structure `vehicle`.

Of course, if the field contains a numeric value to begin with, the code can be more compact. For example, the following command

```
>> mean([Lander.Flights])
```

finds an average number of missions executed by 12 astronauts who landed on the Moon:

```
ans=2.3333
```

Similar to the nested cell arrays, a structure element may be a structure by itself. For example, you may further extend the structure `vehicles`, introduced in the beginning of this section, to include more data (Fig. 4.7):

```
>> vehicle(2).type = struct('group','Fighter');
>> vehicle(2).type.group = struct('name','F-22','built',145);
```

Table 4.6 lists all the functions relative to structures. We will start using the structures in Sec. 4.8.3, and even more examples of using the cell arrays and structures will be presented in Chapter 6.

Next, let us proceed with the table class.

4.5 Tables

A *table* is a MATLAB container for storing heterogeneous column-oriented variables that have the same number of rows. Tables can be created from the delimited text files, from existing variables, or from spreadsheets. For example, consider a table shown in Fig. 4.8 and saved as a Microsoft Excel *DropData.xlsx* file. A simple command

```
>> T1=readtable('DropData.xlsx')
```

Table 4.6 Structure Array-Handling Functions

Function	Brief Description
Create a structure array	
`struct('f',val)`	Creates a structure array with the specified field and value
`cell2struct (C,'f',dim)`	Converts the cell array `C` to a structure array (the fields argument `f` specifies the field names)
`table2struct(T)`	Converts the table `T` to a structure array
Manipulate the structure fields	
`fieldnames(S)`	Returns a cell array of strings containing the names of the fields in structure `S`
`getfield(S,'f')`	Returns the contents of the specified field (equivalent to `S.f`)
`setfield(S,'f',val)`	Sets the contents of the specified field (equivalent to `S.f=value`)
`rmfield(S,f)`	Removes the specified field(s) `f` from the structure array `S`
`orderfields(S)`	Reorders the fields in a structure array `S` so that an output structure array has its field names in ASCII dictionary order
Check a structure array content	
`isstruct(S)`	Returns `true` for the structures
`isfield(S,'f')`	Returns `true` if the structure `S` includes the field `f`
Convert a structure to two other data types	
`struct2cell(S)`	Converts the *m*-by-*n* structure `S` (with *p* fields) into a *p*-by-*m*-by-*n* cell array
`struct2table(S)`	Converts the structure array `S` to a table
Cell operations	
`structfun(fun,S)`	Applies the function `fun` to each field of a scalar structure `S`, one field at a time

creates a 3-by-7 table object

```
T1 =
```

Var1	Date	ID	Weight_lbs	Time	Miss_ft	Bearing__
'SNCA'	10/23/2007	'AD-09'	467	0.61956	568	84
'CADS'	10/24/2007	'AD-10e'	350	0.63051	329	181
'MMIST'	10/25/2007	'AD-11'	420	0.63638	423	145

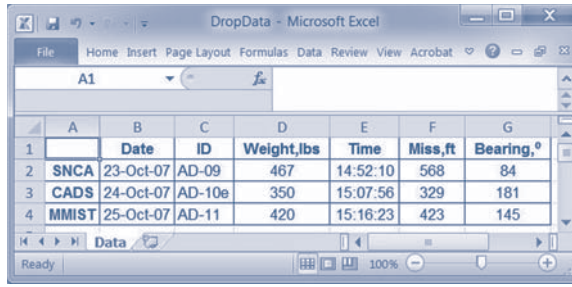


Fig. 4.8 Example of a table given as an Excel spreadsheet.

(for compactness, the line explicitly specifying the class and size of a resulting table object hereafter is omitted). As seen, the row names are stored together with mixed-type table data as a single variable `T1`. The column names were modified to constitute valid variable names, and the default column name `Var1` was added to the first column that had no name.

There are several ways to index into a table. Using parentheses allows you to select a subset of data in a table while preserving the table class of a result. For example,

```
>> a1=T1(2,5)
```

returns

```
a1 =
    Time
  ________
  0.63051
```

Similar to the cell arrays and structures, the curly braces and dot indexing allow you to extract data from a table rather than return a subset of a table. For example,

```
>> a2=T1{2,5:end}
```

returns

```
a2 = 0.6305  329.0000  181.0000
```

and

```
>> a3= T1.Miss_ft(2)
```

yields

```
a3 = 329
```

Being able to extract data from a table, you can carry out computations with the elements of table columns in a usual manner. For example,

```
>> a4=mean(T1{:,6})
```

computes an average value of the Miss_ft column

```
a4 = 440
```

and

```
>> a5=T1{:,6}.*cosd(T1{:,7})
```

computes the downrange component of the miss error for each row:

```
a5 =  59.3722
    -328.9499
    -346.5013
```

You might notice that when creating the `T1` table MATLAB converted the Time column strings to the numeric values. Now that we know how to index table elements, we might want to bring the data stamps back to the

original format. For example, you can use the `datestr` function, specifying the output format you want:

```
>> a6=datestr(T1{:,5},'HH:MM:SS')
a6 ='14:52:10'
    '15:07:56'
    '15:16:23'
```

Similarly, you can modify the Date column (if needed). You can use either the same `datestr` function or `datetime` function. For example,

```
>> a7a=datestr(T1{1,2},'dddd, mmmm dd')
```

returns a `char` variable

```
'Tuesday, October 23'
```

and

```
>> a7b=datetime(T1{1,2},'Format','eeee, MMMM d')
```

returns a `datetime` class variable

```
Tuesday, October 23
```

More examples on using the `datetime` data type are given in Appendix C.

Now, let us create another table but this time using variables available in our workspace after playing with the table `T1`. The general syntax for doing this is

```
T=table(var1,...,varN,name,value)
```

which creates a table from the input variables, `var1`, . . ., `varN`. Variables can be of different sizes and data types, but all variables must have the same number of rows. Optional input arguments include one or more `name`–`value` pair arguments. The command

```
>> T2=table(T1{:,1},datestr(T1{:,2},'dddd'),a5)
```

creates a new 3-by-3 table:

```
T2 =
    Var1          Var2          Var3
    _______    _________    ________

    'SNCA'     Tuesday         59.372
    'CADS'     Wednesday      -328.95
    'MMIST'    Thursday        -346.5
```

Note, because we did not define the names of the columns, they have the default names. That is where those optional `name`–`value` pair arguments could help. Slightly modifying the previous command

```
>> T2=table(T1{:,1},datestr(T1{:,2},'dddd'),a5,'VariableNames', ...
              {'System', 'Day' ,'Overshoot'})
```

results in a 3-by-3 table:

```
T2 =
   System        Day          Overshoot
   _______    _________      _________

   'SNCA'     Tuesday          59.372
   'CADS'     Wednesday       -328.95
   'MMIST'    Thursday         -346.5
```

You can observe the table's properties by referring to its `Properties` field. For example,

```
>> p=T2.Properties
```

returns

```
p =            Description: ''
                  UserData: []
           DimensionNames: {'Row' 'Variables'}
            VariableNames: {'System' 'Day' 'Overshoot'}
     VariableDescriptions: {}
            VariableUnits: {}
       VariableContinuity: []
                  RowNames: {}
         CustomProperties: No custom properties are set.
       Use addprop and rmprop to modify CustomProperties.
```

Another useful `name–value` pair argument allows creating the row names (as shown in the preceding example, as of now, the `RowNames` field is empty). For instance,

```
>> T3=table(datestr(T1{:,2},'dddd'),a5,'RowNames',T1{:,1})
```

returns a 3-by-2 table:

```
T3 =
             Var1          a5
           _________     _______

SNCA       Tuesday        59.372
CADS       Wednesday     -328.95
MMIST      Thursday       -346.5
```

Alternatively, the very same table could have been created using the following two commands:

```
>> T3=table(datestr(T1{:,2},'dddd'),a5); T3.Properties.RowNames=T1{:,1};
```

The advantage of having the row names is that they can be used for indexing into a table as well. For example, either

```
>> T3{'SNCA','a5'}
```

or

```
>> T3.a5('SNCA')
```

returns

```
ans =
  59.3722
```

[the call `T3('SNCA','a5')` returns a `table` class variable with the aforementioned element in it].

When creating the table `T1` in the beginning of this section, we could have treated variables in the first column of the spreadsheet in Fig. 4.8 as the row names from the start. All we had to do was slightly modify the `readtable` call to include another name–value pair argument:

```
>> T1=readtable('DropData.xlsx','ReadRowNames',true)
```

Figure 4.9 shows the workspace after performing all the aforementioned operations of this section. Note that the table objects require more overhead memory compared with other data types for storing heterogeneous data.

When using the name–value pair arguments, make sure that the variable names and row names defined in the cell arrays are composed of nonempty, distinct strings with the number of strings equal to the number of variables and rows, respectively (of course, these strings must be the valid MATLAB variable names). As mentioned at the end of Chapter 3, the `matlab.lang.makeValidName(S)` call could help you construct a valid MATLAB variable name from an input string `S`.

Section 4.1 mentioned another class introduced in R2013b along with the `table` class, `categorical`. Let us show one example on how this data type could be useful. To begin with, let us convert data in the `Var1` column of table `T3` into a *categorical array*. While doing this, we are also specifying that the categories have a mathematical ordering (the first category specified is the smallest and the last category is the largest):

```
>> T3.Var1 = categorical(cellstr(T3.Var1),'Ordinal',true);
```

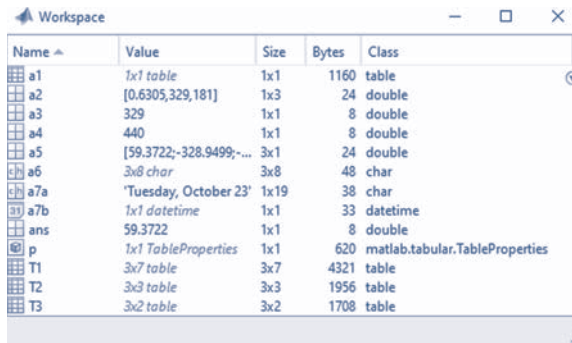


Fig. 4.9 Workspace snapshot after creating tables `T1`, `T2`, and `T3`.

The output of the summary function confirms that the Var1 column is now an ordinal categorical array (1's by the days of week indicate the number of rows with this value):

```
>> summary(T3)
Variables:
  Var1: 3×1 ordinal categorical
      Values:
          Tuesday     1
          Wednesday   1
          Thursday    1
    a5: 3×1 double
      Values:
          min          -346.5
          median      -328.95
          max          59.372
```

Now you can use relational operators, such as less than and greater than (to be discussed in Chapter 5) to display only specific rows of data. For example, if you are only interested in data collected no later than Wednesday, you can use

```
>> T3(T3.Var1<='Wednesday',:)
```

which returns

```
ans =
                Var1          a5
              _______      _______
  SNCA        Tuesday       59.372
  CADS        Wednesday    -328.95
```

As usual, the same result could be achieved using different techniques/tools, but the aforementioned example should help you to start using categorical data in case you are involved in spreadsheet data analysis. More examples on using the categorical data type are given in Appendix C.

To complete this section, Tables 4.7 and 4.8 list a handful of functions available to support the table data type. Many array-handling functions listed in Tables 3.9–3.12 work with the tables as well. [You are welcome to explore other recently introduced functions—findgroups, splitapply, inner2outer (R2018a), groupsummary (R2018a), grouptransform (R2018b), groupcounts (R2019a), and groupfilter (R2019b)—on you own.]

4.6 Types of MATLAB Files

Before we proceed any further, let us formally introduce two basic types of files that MATLAB relies upon. They are

1. ***MAT*-files** (with the default extension *.mat*), which are double-precision binary MATLAB-format files created by the save command and readable by the load command (described in the next section). These files can be

Table 4.7 MATLAB Functions to Create and Analyze the Tables

Function	Brief Description
Create a table	
`readtable(filename)`	Creates a table by reading column-oriented data from a file
`table(var1,...,varN)`	Creates a table from workspace variables `var1, ..., varN`
`array2table(A)`	Converts the homogeneous array `A` to a table
`cell2table(C)`	Converts the cell array `C` to a table
`struct2table(S)`	Converts the structure `S` array to a table
Convert a table to another data type	
`table2array(T)`	Converts the table `T` to a homogeneous array
`table2cell(T)`	Converts the table `T` to a cell array
`table2struct(T)`	Converts the table `T` to a structure array
`writetable(T)`	Writes the table `T` to a comma-delimited text file
Analyze a table	
`istable(T)`	Returns logical `1` (true) if `T` is a table [otherwise, returns logical `0` (false)]
`height(T)`	Returns the number of rows in the table `T`
`width(T)`	Returns the number of variables in the table `T`
`summary(T)`	Prints a summary of table or categorical array `T`
`head(T)`	Gets the first eight (by default) rows of table `T`
`tail(T)`	Gets the last eight (by default) rows of table `T`

created on one computer and later read using MATLAB on another computer with a different floating-point format, retaining as much accuracy and range as the disparate formats allow. They can also be manipulated by other programs, external to MATLAB.

2. *M*-files (with the default extension *.m*), which are files that contain the multiple lines of code in the MATLAB language. You can create *m*-files using any text editor (including MATLAB's *m*-file Editor, introduced in Sec. 2.2.3 and described in more detail in Secs. 5.10.2–5.10.4), and use them as you would use any other MATLAB function or command. The two kinds of *m*-files are
   - *scripts*, which do not accept input arguments or return output arguments (they usually operate on data in the base workspace) and
   - *functions*, which can accept multiple input arguments and return multiple output arguments.

Note that, if you duplicate the *m*-file names, MATLAB executes the one that occurs first in the search path (see Fig. 2.22 and Sec. 5.8.6).

Many of the functions provided with MATLAB are implemented as *m*-files just like the *m*-files you will be creating yourself soon. Other MATLAB functions are precompiled executable programs, called *built-ins*, that run much more efficiently. Unlike the *m*-file functions, you cannot see the source code for built-ins. Although most built-in functions do have an

Table 4.8 MATLAB Functions to Manipulate with the Table Class Data

Function	Brief Description
`join(T1,T2)`	Merges tables `T1` and `T2` by matching up rows using key variables[a]
`innerjoin(T1,T2)`	Creates a table that is the inner join between the tables `T1` and `T2` (the inner join retains only the rows that match between `T1` and `T2` with respect to the key variables)[a]
`outerjoin(T1,T2)`	Creates a table that is the outer join between the tables `T1` and `T2` (the outer join includes the rows that match between `T1` and `T2` and also unmatched rows from either `T1` or `T2`, all with respect to the key variables)[a]
`addvars(T,var,...,rel,col)`	Adds variable `var` relative (`rel`) to column `col` (`rel` is either `'Before'` or `'After'`) [R2018a]
`movevars(T,vars,rel,col)`	Moves variables `vars` with respect to column `col` (`rel` is either `'Before'` or `'After'`) [R2018a]
`removevars(T,vars)`	Deletes the table `T` variables specified by `vars` (R2018a)
`mergevars(T,vars)`	Combines the table `T` variables specified by `vars` to create one multicolumn variable (R2018a)
`splitvars(T)`	Splits all multicolumn variables in `T` so that they are single-column variables (R2018a)
`hasvars(T,vars)`	Returns `1` (true) if table `T` contains any of the variables specified by `vars` (otherwise, it returns `0` (false)) (R2020a)
`renamevars(T,vars,newNam)`	Renames variables specified by `vars` in table T using the names specified by `newNam` (R2020a)
`rows2vars(T)`	Reorients a table `T` so that rows become variables
`stack(W,vars)`	Stacks data from multiple variables `vars` in the table `W` into a single variable[a]
`unstack(T,vars,ivars)`	Unstacks data from a single variable in `T` determined by `vars` into multiple variables specified by `ivars`[a]
`ismissing(T)`	Finds elements of table `T` with missing values (returns a logical array that indicates which elements in the table `T` contain a missing value)[b]
`standardizeMissing(T,id)`	Replaces all instances of the values specified in `id` occurring within the table `T` with the standard missing value indicators
`varfun(func,T)`	Applies the function `func` separately to each variable of the table `T`
`rowfun(func,T)`	Applies the function `func` to each row of table `T`

[a]This function can be used interactively in one of the Tables and Timetables **Task** interfaces of Live Editor (Fig. 2.13a).
[b]This function can be used interactively in the Clean Missing Data **Task** interface of Live Editor (Fig. 2.13a).

m-file associated with them, these files are mainly there to supply the help documentation for the function. Examples of built-in functions are `pi`, `sin`, `find`, `format`, `fprintf`, and `reshape`.

Both types of files can be transported directly between computers: *m*-files because they are platform independent and *mat*-files because they contain a computer signature in the file header. MATLAB checks the signature when

it loads a file. If a signature indicates that a file is foreign, then MATLAB performs a necessary conversion.

The MATLAB environment allows using data (matrices, text, etc.) stored by some other applications. MATLAB easily "understands" the ASCII format as well as some other formats that will be presented later in Sec. 4.8. In the latter case, while loading data, MATLAB converts it into the proper format.

As already mentioned in Sec. 2.2.3, Live Script files are saved as ***mlx***-files, containing code, formatted content, and output. One more commonly used type of file is a ***fig***-file (with the default extension *.fig*) that contains figures and GUIs created in MATLAB. Applications created using the App Designer tool (discussed in Appendix F) are saved as ***mlapp***-files (MATLAB Application Packages) containing descriptions of GUI component properties and callback codes. The default file format for the Simulink models since R2012b is **SLX** (with the default file extension *.slx*). Prior to that, Simulink models were saved as ***mdl***-files (having the *.mdl* extension). The new format, *slx*, constitutes a compressed package that conforms to the Open Packaging Conventions (OPC) interoperability standard.

You may also encounter ***mex***-files (MATLAB executables), built by using the `mex` function of MATLAB. These files are computer-dependent shared libraries with the compiled and linked C, C++, or FORTRAN source files, which you can run in MATLAB exactly like the MATLAB *m*-files and built-in functions (see Appendix F). The MATLAB command `pcode` creates ***p***-files (MATLAB Compiled Programs), content-obscured executable files. Once created, a *p*-file takes precedence over the corresponding *m*-file for execution, even if the *m*-file is subsequently changed (Sec. 5.8.6). **DLL**-files are dynamic-link libraries. Different MATLAB toolboxes may exploit the files with other extensions.

4.7 Recording MATLAB Sessions and Storing Workspace Variables

Although the MATLAB Command History window presents a log of statements most recently run in the Command Window, there is the `diary` function that allows you to keep track of what commands were issued during the current session as well as what responses were obtained (Table 4.9).

Table 4.9 Syntaxes for the `diary` Function

Syntaxes	Brief Description
`diary`	Toggles the switch to record the session on or off
`diary on`	Turns the diary switch on
`diary off`	Turns the diary switch off
`diary filename`	Records the session in the file with the specified name `filename`

Table 4.10 Syntaxes for the `save` Function

Syntaxes	Brief Description
`save`	Saves all workspace variables in a default binary file called *matlab.mat*
`save filename`	Saves all workspace variables in a binary file called *filename.mat*
`save filename var1 var2`	Saves only the variables `var1` and `var2`
`save filename -ascii`	Saves a supported[a] data type in the eight-digit ASCII format
`save filename -double`	Saves the supported[a] data types in the ASCII double-precision format (16 digits)
`save filename -tabs`	Saves a supported[a] data type in the tab-delimited format

[a]Numeric data and text strings (cell arrays, structures, handles, etc. can be saved in a binary format only).

If the diary name (`filename`) is not specified, MATLAB creates a file named *diary* in the current directory.

What is shown in Table 4.9 is referred to as the *command syntax* of a function. It is only a very small fraction of MATLAB functions that allow it. The `diary` function can also use the full-fledged *function syntax*. For example,

```
diary('on')
```

constitutes a valid call as well. (This equivalence is referred to as *command-function duality*.)

The diary file features the following:

1. The diary file is created when the session is turned on, but it is not available for editing until the session is turned off.
2. The diary stores not only the commands themselves (as the Command History window does), but it also stores the response of the system, and so it can be really useful in analyzing the session post facto.
3. The output of a diary is an ASCII file, suitable for printing or for inclusion in reports and other documents.
4. It can be edited using any word processor.
5. If the diary file already exists, the output is appended (!) to the end of the file.

You may check the diary recording status (`on`/`off`) by typing `get(0,'Diary')` or `get(0,'DiaryFile')` (be sure to use a zero `0`, not a capital "Oh," `O`).

Along with recording the current session, you may also be interested in saving (and further retrieving) some specific workspace variables. That is where the `save` and `load` functions become handy. The `save` function may use one of the most common command syntaxes, shown in Table 4.10. Each of the calls in Table 4.10 has its function syntax. For example,

```
save(filename,'var1','var2',...)
```

Table 4.11 Syntaxes for the `load` Function

Syntaxes	Brief Description
`load`	Retrieves variables in the file *`matlab.mat`* and loads them into workspace
`load filename`	Retrieves variables in the file specified by *`filename`* and loads them into workspace
`load filename.ext`	Loads the ASCII file *`filename.ext`* into the matrix `filename` (the ASCII file must be space delimited)
`load filename ... var1 var2`	Retrieves two specified variables, `var1` and `var2`, from the *mat*-file *`filename`*
`load -ascii filename`	Retrieves and loads variables from a space-delimited ASCII file
`S=load('filename')`	Returns the content of *`filename`* in a variable `S`. If *`filename`* is a *mat*-file, `S` is a structure containing the fields matching retrieved variables. If *`filename`* is an ASCII file, `S` is a double-precision array

Once you stored your data using the `save` function, you can always load it back to the MATLAB development environment (workspace) using the `load` function (Table 4.11). Note that the function syntax may be the only option if `filename` is stored in a text string, when the result is saved in a specified variable (as is the case in the last example), or if `filename` contains spaces.

4.8 Importing and Exporting Data

MATLAB allows you to work with a wide variety of data. That includes delimited or formatted text files, spreadsheets, documents written in XML, scientific data in a specified format, images, graphics files, audio and video files, and data over Transmission Control Protocol/Internet Protocol (TCP/IP) interface. Section 4.8.1 addresses the issue of importing/exporting numeric and textual data into/from the MATLAB workspace programmatically, followed by Sec. 4.8.2 showing how to utilize the Import Tool. Section 4.8.3 provides the basics of importing media data.

4.8.1 Importing and Exporting Numeric and Textual Data Programmatically

When importing numeric and textual data programmatically, you must know some a priori information about the data files you are dealing with specifically

1. How many data items are in each row?
2. Are these items numeric, text strings, or both?

3. Does each row or column have a descriptive header?
4. What character is used as a delimiter, that is, the character used to separate the data items from each other, or the columns (including columns in a table)

Depending on this information, you have a choice of using several MATLAB functions to import data into the MATLAB development environment most effectively. These functions are listed in Table 4.12.

In what follows, we will only discuss a few of these functions but if needed, you are welcome to use the MATLAB Help system to learn about the rest. We start with reading numeric (textual) data, remembering that normally it is formatted in one of the following categories:

1. Space-delimited
2. Tab-delimited
3. Comma- or semicolon-delimited
4. Mixed text and numeric
5. Text headers only

The most common function to import numeric data, `dlmread`, has the following format:

```
M=dlmread(filename,delimiter)
```

This function allows you to read an ASCII-delimited file `filename` (defined as a character vector or string scalar) into a matrix `M` using the specified `delimiter`. You may use `'\t'` to specify tab-delimited data and `';'` to specify semicolon-delimited data. If `delimiter` is not specified, a comma `','` is assumed to be a default delimiter.

Table 4.12 Functions to Import/Export Numeric and Textual Data

File Format	File Content	Extension	Functions
MATLAB formatted	Saved MATLAB workspace	*.mat*	`load, save`
Text/data[a]	Text	any	`textread, textscan fscanf, sscanf`
	Delimited numbers/text	any	`dlmread, dlmwrite`
	Comma-separated numbers/text	*.csv*	`csvread, csvwrite`
Spreadsheet data[a]	Excel worksheet	*.xls, .xlsx*	`xlsread, xlswrite`
	Lotus 123 worksheet	*.wk1*	`wklread, wklwrite`
Webpage data	HTML content	URL	`webread, webwrite (urlread, urlwrite)`
Extensible Markup Language	XML-formatted text	*.xml*	`xmlread, xmlwrite`
Scientific data	Data in Common Data Format	*.cdf*	`cdfread, cdfwrite`
	Flexible Image Transport System data	*.fits*	`fitsread`
	Data in Hierarchical Data Format	*.hdf*	`hdfread, h5read`

[a]Several newer functions, introduced in R2019a, will be presented in Table 4.15.

The dlmread function also allows you to read only a portion of data. The two corresponding syntaxes are

```
M=dlmread(filename,delimiter,R,C)
```

and

```
M=dlmread(filename,delimiter,range)
```

(Note that in this case, you must specify a delimiter even if it is ','.)

In the first syntax, the values R and C specify the upper-left corner (row R and column C) of data that needs to be read (zero values, R=0 and C=0, specify the first value in the file, which is the upper-left corner). The second syntax allows you to read a range specified by a four-element vector range=[R1 C1 R2 C2], where (R1,C1) define the upper-left corner of data to be read and (R2,C2) define its lower-right corner. The input argument range can also be specified using a spreadsheet notation as in range='A1...B7'. For instance, given the file *data.txt* that contains the comma-separated values

```
82,  10,  16,  15,  66,  76
13,  55,   ,  92,  85,  40
92,  96,  49,  80,  94,  66
64,  97,  81,  96,  68,  18
```

the following two commands produce the corresponding results shown beneath them:

```
>> dlmread('data.txt',',',2,4)
ans = 94 66
      68 18
>> dlmread('data.txt',',',[1,1,2,3])
ans = 55   0  92
      96  49  80
```

Note that the dlmread function fills the empty delimited fields with zeros.

In a similar manner, an array M can be exported to the ASCII data files by using the dlmwrite function

```
dlmwrite(filename,M,[delimiter,R,C])
```

This call writes M into an ASCII-format file filename using the default delimiter (,) to separate the matrix elements. Data are written to the upper-left-most cell of the spreadsheet. Optionally (that is what [] mean), you can change delimiter and also specify the row and column offsets, R and C, respectively (again, R=0 and C=0 correspond to no offset, that is, point to the upper-left corner cell).

Along with the more general dlmread and dlmwrite functions, MATLAB has another pair of functions allowing you to read and write comma-separated numeric data. They are csvread and csvwrite,

respectively. Their syntax can be easily understood by exploring the following commands:

```
M=csvread(filename,[R,C,range])
csvwrite(filename,M, [R,C])
```

(In the case of the most comprehensive syntax, including all three parameters enclosed in square brackets, `R` and `C` should match `range(1:2)`.) For instance, for the same file *data.txt* the command

```
>> B=csvread('data.txt',2,0,[2,0,3,3])
```

returns

```
B = 92   96   49   80
    64   97   81   96
```

If your file contains mixed numeric–text data, you may only rely on the `textread` or `textscan` function. Let us start with the first one:

```
[A,B,C,...]=textread(filename,formSp,N)
```

This call reads data from the file `filename` into variables `A`, `B`, `C`, etc., using the specified format defined in the `formSp` character vector or string scalar, `N` times (if `N`<0 or is not present at all, the entire file is read).

Although the function `textread` is well suited for reading mixed numeric–text data with a fixed or free format, you still have to know this format, that is, the organization of the file. The `textread` function matches and converts groups of characters from the input. Each input field is defined as a sequence of nonwhitespace characters (by default, a white-space character is one of the following: `' '` – space, `'\b'` – backspace, or `'\t'` – horizontal tab) that extends to the next whitespace or delimiter character or to the maximum field width. For example, if the first line of the *mydata.dat* file is

```
John  LT  3.34  25  13
```

then, reading this line using the specified format (to be discussed in more detail in the next section),

```
>> [name,rank,GPA,age,degree]=textread('mydata.dat','%s %s %f %d %s',1)
```

returns

```
name = {'John'}
rank = {'LT'}
GPA = 3.34000000000000
age = 25
degree = {'13'}
```

[Note that, while the (repeated) whitespace characters in the format string are ignored, the repeated nonwhitespace delimiter characters are significant.] The number of format specifiers (each starting with the `%` symbol) in the

formSp input determines the number and types of return arguments. Available format specifiers for reading data into the workspace are listed in Table 4.13. (The formSp input supports a subset of the conversion specifiers and conventions of the C language's fscanf routine.)

More complex syntax of the textread function, utilizing one or more name–value pair arguments,

```
[...] = textread(...,name,value,...)
```

provides even more capabilities that may be very useful for reading data from files created by other programs and containing a lot of redundant information. For instance, parameter headerlines allows you to skip the specified number of lines at the beginning of the file.

Consider the following practical example. Suppose the *Data.log* file contains some useful data (100 lines starting with symbol c) advanced by 10 lines of secondary information as follows:

```
v ATLAS
# Created by ATLAS on Fri Nov 14 09:48:20 2008.
#SSF 2.0 generic
```

Table 4.13 Format Specifiers for Reading Data into Workspace

Format Code	Action	Output
Literals (ordinary characters)	Ignores the matching characters. For example, in a file that has MAE followed by a number (for course number), to skip the MAE and read only the number, use 'MAE' in the formSp string	No output
%d	Reads a signed integer value	Double array
%u	Reads an integer value	Double array
%f	Reads a floating-point value	Double array
%s	Reads a whitespace or delimiter-separated text string	Cell array of strings
%q	Reads a text string, which could be in double quotes	Cell array of strings (does not include the double quotes)
%c	Reads characters, including any whitespace character	Character array
%[]	Reads the longest text string containing characters specified in the brackets	Cell array of strings
%[^]	Reads the longest nonempty text string containing characters that are not specified in the brackets	Cell array of strings
%* (instead of %)	Ignores the matching characters specified by one of the conversion specifiers following * (for example, %*f ignores a floating-point value)	No output
%w... (instead of %)	Reads field width specified by w. The %f format supports %w.pf, where w is the field width and p is the precision	Double array

```
########################################################################
#       Copyright © 1984-2008          #
#       Silvaco Data Systems, Inc. All rights reserved     #
# KEY=SVC86hn7562asdx          PROD=345  #
########################################################################
j  4  1  2  94  95
k  2  2  0.342109921  0.670000017
c  1  -250  -0.87 0
c  2  -250  -0.866666667 0
c  3  -250  -0.863333333 0
...
```

Then, the command

```
>> [a,b,c,d]=textread('Data.log','%*c %f %f %f %f',100,'headerlines',10)
```

skips the first 10 lines and reads 100 elements of data (as specified in `formSp`) into the four column vectors `a`, `b`, `c`, and `d`.

In this specific case, to read the *Data.log* file you could also have used the `dlmread` function specifying the delimiter and the upper-left corner of data to read:

```
>> D=dlmread('Data.log',' ',10,1)
```

This command reads tabular data into the `D` matrix. (In this particular case this command happens to be a little bit more compact, compared with using `textread`.)

If you are using the `textread` function and do not necessarily intend to create all output variables, you can avoid creating them by utilizing a tilde symbol, `~`. For example, if in the previous case you do not care about the first and last columns you could use the following syntax:

```
>> [~,b,c,~]=textread('Data.log','%*c %f %f %f %f',100,'headerlines',10)
```

Consider another example. Suppose the file *winds.txt* is organized as follows:

```
37 2
2008-12-16_00Z,DZID00 i,j=( 15.9, 15.6),lat,long=(54.832,83.104)
SFALT, SFPRES
381 968.3
LINE AGL(m) T(C) RH(%) WSPD(m/s) WDD P(mb)
1   2.0   11.76  50.93   9.33   182.93   968.04
2  15.2  11.64  51.16  10.97   182.91   966.53
3  56.2  11.26  52.30  13.47   182.37   961.80
...
```

The goal is to read not only numeric data starting from line 6—and we do not know how many lines of data are in there—but also some data from the header lines as well. The following set of commands does the job:

```
% Read header information
[Y,M,D,H,f1,f2,f3,lat,lon]=textread('winds.txt',...
'%4c-%2c-%2c_%2c %s i,j=( %f , %f ),lat,long=( %f , %f',1,'headerlines',1);
clear f1 f2 f3
% Find the length of the file
file=textread('winds.txt','%s','delimiter','\n'); N=length(file);
% Read numeric data
[q,w,e,r,t,y,u]=textread('winds.txt','%f %f %f %f %f %f %f',...
                                        N-5,'headerlines',5);
```

This piece of code first reads the year, month, day, hour, latitude, and longitude information from the second line, skipping all the text (and clearing auxiliary variables `f1` , . . ., `f3`). Then, it reads all lines of the file (using the end-of-line delimiter `'\n'`) to find its length (number of lines) `N`. Finally, it reads `N-5` lines of data into the seven column vectors `q`, `w`, `e`, `r`, `t`, `y`, and `u`.

When reading large text files, reading from a specific point in a file, or reading file data into a single cell array rather than multiple vectors, you might prefer using the `textscan` function:

```
C=textscan(fid,formSp,[N,name,value])
```

For instance, in the latter example instead of the last line of code, you could have used the following:

```
F = fopen('winds.txt');
C = textscan(F,'%f %f %f %f %f %f %f',N-5,'headerlines',5);
fclose(F);
```

All mixed-type data would then be read from the opened file *winds.txt* into a single cell array `C`.

Having a file opened allows reading different pieces of data sequentially, without starting from the beginning of the file, as you would need to do when using the `textread` function. For example, you may read the date stamp appearing in the beginning of the second line of the *winds.txt* file and numeric data employing two auxiliary cell arrays you do not actually need:

```
F = fopen('winds.txt');
aux1 = textscan(F,'%s',2);
Date = textscan(F,'%s',1,'delimiter','_');
aux2 = textscan(F,'%s',4,'delimiter','\n');
Data = textscan(F,'%d %f %f %f %f %f %f',5);
fclose(F);
```

Figure 4.10 shows the content of all four cell arrays produced by the following four commands:

```
subplot(2,2,1), cellplot(aux1)
subplot(2,2,2), cellplot(Date)
subplot(2,2,3), cellplot(aux2)
subplot(2,2,4), cellplot(Data)
```

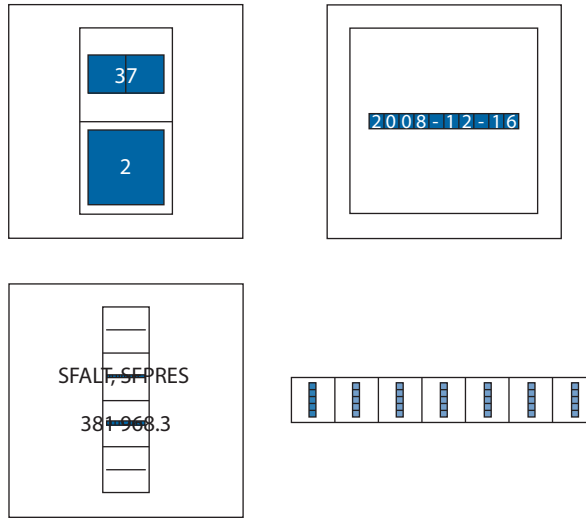


Fig. 4.10 The content of the cell arrays created using the `textscan` function.

The `textscan` function also allows you to read from a text string `str` (without opening a file):

```
C = textscan(str,...)
```

It also allows returning the file or string position at the end of the scan as the second output argument:

```
[C, position] = textscan(...)
```

For a string, `position` indicates how many characters `textscan` read. For an opened file, `position` is a zero-based integer that indicates the number of bytes from the beginning of the file.

Another function that may be quite useful is the `xlsread` function, enabling reading numeric data and text directly from Microsoft Excel spreadsheet files (with an extension *.xls* or *.xlsx*):

```
A=xlsread(filename)
[A,B]=xlsread(filename,[sheetname])
```

The first format returns only numeric data in array `A` from the first sheet in the Microsoft Excel spreadsheet file named *filename*. With this syntax, the `xlsread` function automatically ignores leading rows or columns of text. If any cell (located not in a leading row or column) is empty or contains text instead of numeric data, `xlsread` puts `NaN` in its place. The second syntax of the `xlsread` function returns numeric data in array `A` and text data in cell array `B`. If the spreadsheet contains leading rows or columns of text, `xlsread` returns only those cells in `B`. If the spreadsheet contains text that is not in a row or column header, `xlsread` returns a cell array of the same size as the original spreadsheet with the text strings in the

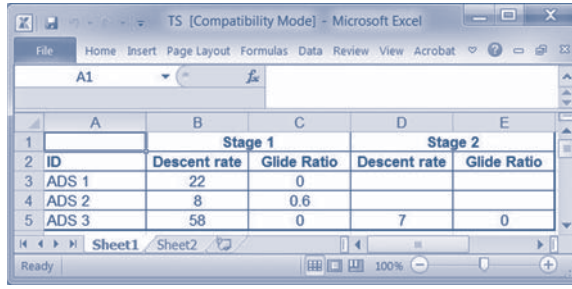


Fig. 4.11 Content of the *TS.xls* file to be read into MATLAB workspace.

cells that correspond to text in the original spreadsheet. All the cells that correspond to numeric data are empty. Optionally, you can point at a specific `sheetname`.

For example, for the Microsoft Excel file shown in Fig. 4.11, the command

```
>> [numTS,txtTS]=xlsread('TS.xls')
```

returns

```
numTS = 22.0000        0         NaN        NaN
      8.0000      0.6000        NaN        NaN
     58.0000        0         7.0000        0
txtTS = Columns 1 through 5
    ''    'Stage 1'       ''          'Stage 2'       ''
    'ID'          'Descent rate'    'Glide Ratio'     'Descent rate'
'Glide Ratio'
'ADS 1'      ''           ''            ''            ''
'ADS 2'      ''           ''            ''            ''
'ADS 3'      ''           ''            ''            ''
```

As always, more syntaxes are available (see the Help system of MATLAB).

Another function to explore is `importdata` (Table 4.14). This somewhat universal function works with the text files, spreadsheets, and even data from clipboard. This function looks at the file extension to determine which data-importing function to use. If it can recognize the file extension, `importdata` calls the appropriate function, specifying the maximum number of output arguments. If it cannot recognize the file extension, `importdata` calls `finfo` to determine which function to use. If no data-importing function is identified for this particular file extension, `importdata` treats the file as delimited text. For instance, for the aforementioned file *Data.log,* the command

```
>> R=importdata('Data.log','\t')
```

returns a cell array `R` putting each line of text into an individual cell.

Table 4.14 Syntaxes for the `importdata` Function

Syntaxes	Brief Description
`A=importdata(filename)`	Loads data into array `A`
`A=importdata(...,delimiter)`	Interprets `delimiter` as the column separator
`A=importdata(...,delimiter,header)`	Loads data reading numeric data starting from line `header + 1`
`[A,delOut,headOut] = importdata(...)`	Additionally returns a detected delimiter character for the input ASCII file in `delOut` and the detected number of header lines in `headOut`

In R2019a, several new functions that allow reading different types of data from delimited text files (*.txt*, *.dat*, or *.csv*) and spreadsheet files *(.xls*, *.xlsb*, *.xlsm*, *.xlsx*, *.xltm*, *.xltx*, or *.ods*) were introduced. They are shown in Table 4.15 and complement the corresponding functions of Table 4.12. The reciprocal writing functions are `writematrix`, `writecell`, `writetable`, and `writetimetable`.

Compared with the existing functions considered earlier in this section, the `readmatrix` and `readcell` functions feature automatic detection of data format and types, the ability to use import options to control the data import process (including the handling of errors and missing data), and better cross-platform support and performance.

As far as reading tabular data, the `detectImportOptions` function, introduced in R2016b, may help locate a table in a file and returns the import options for importing the table:

```
opts=detectImportOptions(filename,[Name,Value])
```

Table 4.15 The R2019a Functions to Read Data from Delimited Text Files and Spreadsheets

Function	Brief Description
`A=readmatrix(filename,... [opt,name,value])`	Reads homogeneous numeric or text data from `filename` into a matrix `A`. The file format is determined from the file extension. Import options can be specified in `opts` object and by one or more `name-value` pair arguments
`C=readcell(filename,... [opt,name,value])`	Reads mixed numeric and text data from `filename` into a cell array `C`
`T=readtable(filename,... [opt,name,value])`	Reads column-oriented data from `filename` into a table `T` (introduced earlier, in R2013b)
`TT=readtimetable(filename,... [opt,name,value])`	Reads column-oriented data from `filename` into a timetable `TT`
`[v1,...,vn]=readvars(filename,... [opt,name,value])`	Reads column-oriented data from `filename` into variables `v1,...,vn`

Alternatively, the import options object `opts` for the tables and timetables can be created manually, using `DelimitedTextImportOptions`, `FixedWidthImportOptions`, and `SpreadsheetImportOptions`. The correctness of defining tabular data can be checked using the `preview` function

```
T=preview(filename,[opts])
```

returning a table containing the first eight rows of data in `filename`, using the import options `opts`.

It should be noted that there are no plans to remove the corresponding Table 4.12 functions because they offer very useful specific-format-tailored reading options. However, depending on your file structure and content, you might want to try using `readmatrix` instead of `dlmread`, `csvread`, and `xlsread` (in case you intend to read homogeneous numeric or text data) and `readcell` instead of `xlsread` (in case you are reading mixed numeric and text data).

Finally, MATLAB offers a couple of functions to deal with webpages (see Table 4.12). The `web(url)` call opens the `url` webpage [specified by the Uniform Resource Locator (URL) path] using your system browser by default (prior to R2019b—in the MATLAB Web browser). Right-clicking and choosing the **Page Source** option from the pop-up menu opens another window showing an HTML source code. Then, the `webread` (or `urlread`) function allows importing this HTML code to a MATLAB character vector. To extract necessary data from this character vector, you would need to write a script employing the `textread` or `textscan` function as explained in the earlier examples in this section. [While the `webread` (`urlread`) function saves web content to a character vector, the `webwrite`

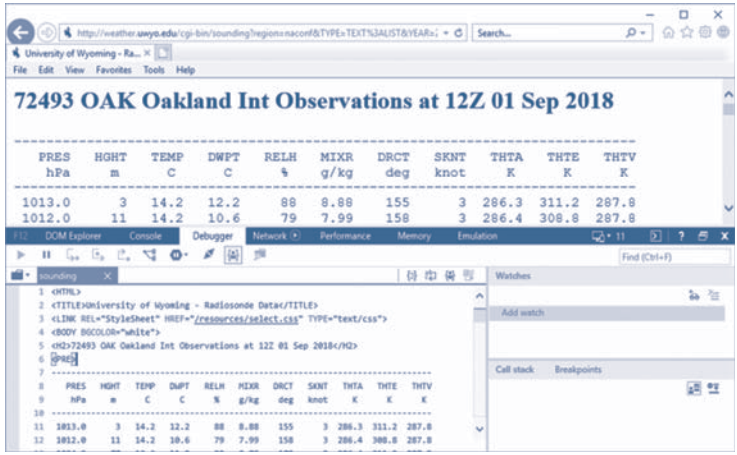


Fig. 4.12 Example of web-based weather data.

(or urlwrite) function can download a URL's content and save it to a specified file.]

Let us consider one example. Suppose you want to read in sounding rocket data available on the University of Wyoming website, as shown in Fig. 4.12. Choosing the **Source** option from the Internet Explorer's View tab drop-down menu splits the window and shows HTML source code in the lower part. Because we only need to import the numeric data, the trick here is to find where it starts and where it ends.

In this particular case, numeric data ends with the HTML</PRE> tag, indicating the end of preformatted text. We could have used the HTML <PRE> tag for pointing at the beginning of the table, but in this case, we would need to count all characters in the next four lines (to navigate to the very first number, 1013.0). Instead, the following code tries to find where the dashed separating line ends:

```
sta_num='72493';
Y='2018'; M='09'; D='1';      % Read data for 9/1/2018
%% Create the URL address
url=['http://weather.uwyo.edu/cgi-bin/sounding?region=naconf&TYPE=TEXT',...
  '%3ALIST&YEAR=',Y,'&MONTH=',M,'&FROM=',D,'12&TO=0212&STNM=',sta_num];
%% Import source (HTML) code
SourceFile=webread(url);
%dlmwrite('AtmData.txt',SourceFile,'delimiter','');
%% Find the beginning and end of numeric data
Pattern='-----'; K=strfind(SourceFile,Pattern);
[~,ind]=find(diff(K)>1); start=K(ind(2))+length(Pattern);
pos=strfind(SourceFile,'</PRE>'); ending=pos(1)-1;
%% Read numeric data in the source code
C=textscan(SourceFile(start:ending),'%f %f %f %f %f %f %f %f %f %f %f');
%% Plot data
subplot(1,2,1)
plot(C{:,3},C{:,2}/1000,'.--'), grid, xlabel('T, ^oC'), ylabel('H, km')
subplot(1,2,2)
plot(C{:,1},C{:,2}/1000,'.--'), grid, xlabel('P, hPa'), ylabel('H, km')
```

Figure 4.13 proves that weather data was indeed stored in the C cell array. (The commented dlmwrite command shows an alternative way of storing HTML source code in an ASCII text file if needed.)

It should be noted that for the sake of robustness, source code processing as shown above probably needs some additions (like filling nonexisting table entries with NaN, etc.). Also, quite often, numeric data do not appear as a preformatted text (one chunk of data) but rather shows up split into individual cells. In this case, instead of using the <PRE> tag, you might need to look for the <tr> tag, representing a row of cells in an HTML table; the <td> tag, representing an individual data cell (or table data); or the <tbody> tag, used for grouping table rows. That is why it is important to carefully analyze source code and understand its structure.

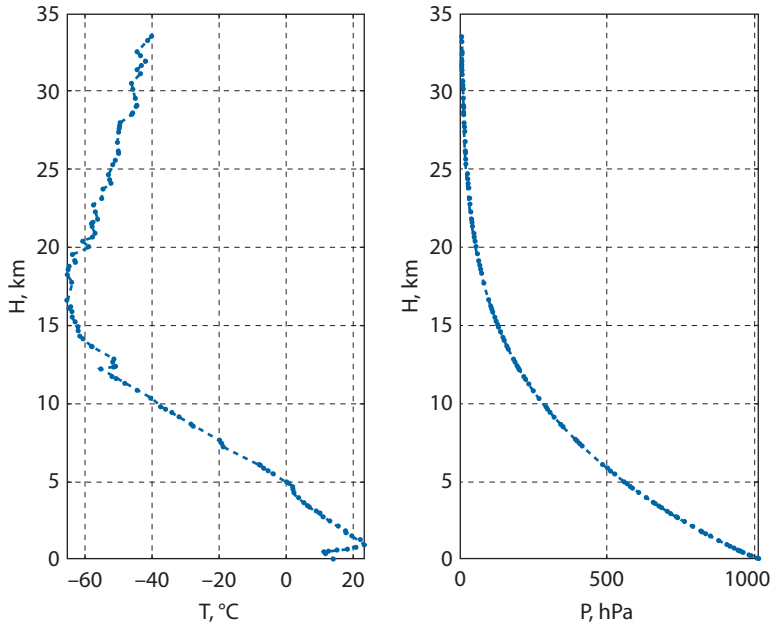


Fig. 4.13 Plotting data imported from a webpage.

4.8.2 Importing Data Using the Import Tool

Along with importing data into MATLAB programmatically as discussed previously, you can always do it using a special tool—the MATLAB Import Tool (previously, Import Wizard). To activate the MATLAB Import Tool, you may choose the file you want to import to the workspace in the Current Folder window and right-click it. From the different options that will show up in the pop-up window, select **Import Data**... (this option will only show up for recognized data files). Alternatively, you may click the **Import Data** icon on the MATLAB Toolstrip in the Home tab (which will prompt you to choose the file to open) or use the `uiimport` command (in this case, a Select Source menu will pop-up suggesting that you import data either from a file or clipboard).

To use this tool, you do not even need to know the format of data. You simply specify the file that contains data and the Import Tool processes the file's content automatically. Figures 4.14 and 4.15 provide examples of importing data from an ASCII text file and from a Microsoft Excel spreadsheet.

As seen in Fig. 4.14b, the Import Tool automatically recognizes what type of data you are trying to import (Fig. 4.14b). You can change the column delimiter for data that should be read when the Import Tool does not recognize it correctly, specify a data range you are interested in, and even adjust the

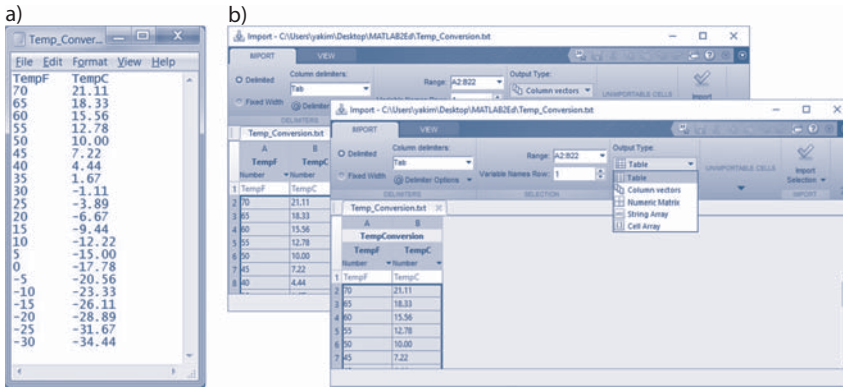


Fig. 4.14 Importing data from a) an ASCII text file using b) the Import Tool.

imported data format. To this end, for numeric and textual data, the Import Tool has five choices: read in each column of digital data into a separate column vector, read all digital data in a numeric matrix, or read in both numeric and textual data into a string array, a cell array, or a table (Fig. 4.14b). When choosing between these five options the Import Tool surrounds the part of the file that can be read by a thick line. If required to read in heterogeneous data, it distinguishes them by color (yellow corresponds to the textual data) (Fig. 4.15). Within this tool you can also specify what to do with unimportable cells (by default, they are replaced with `NaN`). Finally, you have the choice of either importing data or generating a script or function to import it [these options are available when clicking the **Import Selection** icon at the far right of the Import Tool toolstrip (Figs. 4.14b, 4.10)]. In the latter two cases, you will end up with a full-fledged MATLAB code you could reuse to read another piece of data that has the same structure.

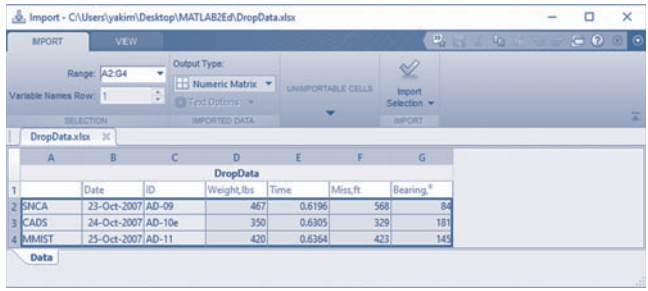


Fig. 4.15 Importing data from the Microsoft Excel spreadsheet shown in Fig. 4.8 using the Import Tool.

4.8.3 Working with Media Data

Along with numeric and textual data, MATLAB allows dealing with media data as well. The list of the corresponding functions to read audio, video, and graphic files is given in Table 4.16 (note, as of R2012b, the older functions such as `auread`, `wavread`, and `aviread` have been removed).

Let us briefly show how to load different media files into the MATLAB workspace. To begin with, the `imread` function allows importing different graphic files. Importing different-type files involves some specifics, but as an example, the following two commands show how simple it is to import a JPEG image and then display it (Fig. 4.16a):

```
>> A=imread('CriticalApha.jpg');
>> image(A)
```

In this particular case, `imread` returns a 283-by-367-by-3 array. The three layers (slices, pages) of this array represent the red, green, and blue (RGB) components of each pixel (color representation will be addressed in Sec. 6.9). (Note that pixels are counted down and right from the upper-left

Table 4.16 Functions to Import/Export Media Data

Type of Media File	Supported Formats	Extension	MATLAB Functions
Graphics	BMP (Windows Bitmap) CUR (Cursor File) GIF (Graphics Interchange Format) HDF4 (Hierarchical Data Format) ICO (Icon File) JPEG (Joint Photographic Experts Group) and JPEG 2000 PBM (Portable Bitmap) PCX (Windows Paintbrush) PGM (Portable Graymap) PNG (Portable Network Graphics) PPM (Portable Pixmap) RAS (Sun Raster) TIFF (Tagged Image File Format) XWD (X Window Dump)	*.bmp* *.cur* *.gif* *.hdf4* *.ico* *.jpeg, .jp2* *.pbm* *.pcx* *.pgm* *.png* *.ppm* *.ras* *.tiff* *.xwd*	`imread` `imwrite`
Video	Motion JPEG and Motion JPEG 2000 encoded video MPEG-1 MPEG-4, including H.264 encoded video Windows Media video Apple QuickTime Movie video Any format supported by Microsoft Media Foundation and Microsoft DirectShow	*.avi, .mj2* *.mpg* *.wmv, .asf, .asx* *.mp4, .m4v* *.mov*	`VideoReader` `VideoWriter`
Audio	WAVE, OGG, FLAC, AU, MP3, MPEG-4, AAC files	*wav, .ogg, .flac, .au, .mp3, .m4a, .mp4*	`audioread` `audiowrite`

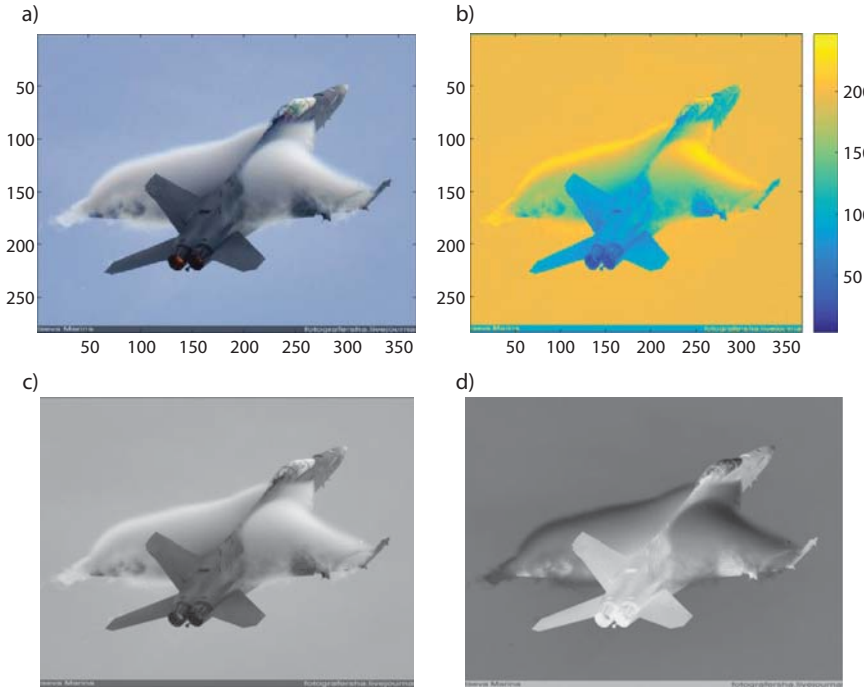


Fig. 4.16 Reading and manipulating a graphic file.

corner). For example, Fig. 4.16b shows the content of the third (blue-color intensity) layer graphically employing two simple commands

```
>> imagesc(A(:,:,3)), colorbar
```

As seen, the values spread between 0 and 255 (this is the full range for the `uint8` data class, as will be addressed in Sec. 8.3.2).

The Image Processing and Computer Vision Toolboxes of MATLAB offer a variety of functions to process the graphic files, including their analysis and enhancements, feature/object detection and extraction, image registration, and geometric transformations, so if you are involved with image processing we refer you to those two toolboxes. To conclude this short still-image-importing discussion, let us show the results of the following commands, manipulating with the original array `A`:

```
>> B=rgb2gray(A);
>> imshow(B)
>> imshow(255-B)
```

The first two commands convert the original 3-D array to a 2-D matrix (degrading the true-color RGB image to the grayscale intensity image) and then display it using the `imshow` function (Fig. 4.16c). As opposed to the `image` function, visualizing an image as a matrix of numerals, `imshow`

treats the input matrix as an image, which involves optimizing figure, axes, and image object properties for better representation (including removing the axis tick marks and maintaining an aspect ratio). The last command shows the results of grayscale image inversion (Fig. 4.16d).

Modified image data `A` can be stored to the file specified by `filename` (inferring the file format from its extension) using the `imwrite` function

```
imwrite(A,filename)
```

Handling video data is similar to handling a single graphic image. The first step is to create a VideoReader object. For example,

```
>> Vobj = VideoReader('MVI_2674.mov')
```

creates

```
Vobj =
   VideoReader with properties:
    General Properties:
              Name: 'MVI_2652.MOV'
              Path: 'C:\Users\Oleg\Desktop\TestVideo'
          Duration: 3.2115
       CurrentTime: 0
               Tag: ''
          UserData: []
  Video Properties:
             Width: 1920
            Height: 1080
         FrameRate: 23.9760
      BitsPerPixel: 24
       VideoFormat: 'RGB24'
```

(as a source file you can use any video clip taken by your smart phone or camera).

Then, you can view (work on) each frame individually, running a loop (to be covered in Sec. 5.5). For example, running the `while` loop may look like the following:

```
while hasFrame(Vobj)
   video = readFrame(Vobj);
   image(video)
   pause(1/Vobj.FrameRate);
end
```

Here, the `hasFrame` function determines whether a frame is available to read, `readFrame` reads the next available video frame from the file associated with `Vobj`, and `pause` pauses execution for `1/Vobj.FrameRate` seconds before continuing (otherwise, the video might be played very

fast—so fast that you will not even notice you are looking at the very last frame only). Running the `for` loop calls for a computation of the number of frames first:

```
nFrames = fix(Vobj.Duration*Vobj.FrameRate);
for k = 1 : nFrames
currentframe=read(Vobj,k);
imshow(currentframe)
end
```

(As you probably noticed, the output of the `VideoReader` function has a structure-like format that enables referring to its fields as described in Sec. 4.4.)

To write video data to an *avi*-file, you should first create a VideoWriter object using the `VideoWriter` function. An example of creating a video file will be given in Sec. 6.12.

Finally, let us show how to deal with audio files. Surely, you have some audio files stored on your computer. You may now navigate to one of them and play it from within MATLAB using the following commands:

```
>> File='Magic_Fly.mp3';
>> audioinfo(File)
>> [y,Fs]=audioread(File);
>> sound(y,Fs)
```

The first two commands define the name of the audio file and provide information about it. For example,

```
ans =
            Filename: 'C:\Users\Oleg\Desktop\Music\Magic_...'
   CompressionMethod: 'MP3'
         NumChannels: 2.00
          SampleRate: 44100.00
        TotalSamples: 11333700.00
            Duration: 257.00
               Title: 'Magic Fly'
             Comment: []
              Artist: 'Space'
             BitRate: 128.00
```

The `audioread` function reads data from the file named `File`, and returns sampled data, `y`, and a sample rate for that data, `Fs`. Single-channel (mono) audio data are stored in an *m*-by-1 column vector and stereo data in an *m*-by-2 matrix. For stereo data, the first column contains the left channel, and the second column contains the right channel. In either case, *m* is the number of samples. The `sound` function sends the audio signal `y` to a speaker at the specified sample rate (most sound cards support sample rates between 5 Hz and 48 kHz). Decreasing and increasing the sample rate allows playing audio slower or faster.

For more flexibility during playback, including the ability to pause, resume, or define callbacks, you may use the `audioplayer` function. With signal `y` and rate `Fs`, you can create an AudioPlayer object, and then play it using the `play` function

```
>> playerobj=audioplayer(y,Fs);
>> play(playerobj)
```

Similarly, to record data from an audio input device (microphone) you would need to create an AudioRecorder object (using the `audiorecorder` function) and then call the `record` or `recordblocking` function to execute the recording. For example, the following set of commands allows you to create a 10,000-Hz, 16-bit, two-channel AudioRecorder object and then record your voice for 5 s:

```
>> recObj = audiorecorder(10000,16,2)
>> disp('Start speaking now')
>> recordblocking(recObj,5);
>> disp('End of recording');
```

You may now listen to your recording by using the `play` function

```
>> play(recObj);
```

To extract audio data into a double-precision array `y`, you would need to use the following call:

```
>> y = getaudiodata(recObj);
```

You may write your recording to a variety of supported formats (Table 4.16) using the `audiowrite` function with the following self-explanatory syntax:

```
audiowrite(filename,y,Fs,name,value)
```

To conclude this section, it should be noted that the Import Tool considered in Sec. 4.8.2 may be used for importing media files as well. Of course, it employs the same MATLAB functions considered previously. (Although we have shown some simple syntax for these functions, you are welcome to explore these functions further by using the MATLAB Help system.)

4.9 Writing and Displaying Formatted Data

So far, we have primarily dealt with displaying unformatted data, as stored in computer's memory, with quite limited control over how we want it to be presented or saved. To be more specific, in Sec. 2.6 the `format` function was introduced to control the output format of numeric values displayed in the Command Window. In Sec. 4.8.1 (Table 4.13), specifiers were introduced to distinguish between the different types of data when reading from data files. This section elaborates on the specifiers concept and shows how to apply a specific format to each variable to be displayed on the screen or

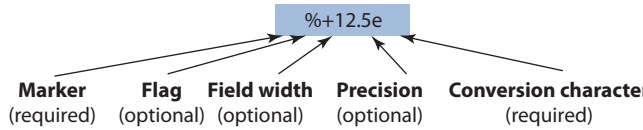


Fig. 4.17 Example of the format specifier.

written to the file. Specifically, it is the `fprintf` function that allows you to do so. Its general format is

```
fprintf(fid,formSp,A1,A2,...)
```

This function formats data in (the real part of) array `A1`, `A2`, etc. under control of the `formSp` character vector (in single quotes) or string scalar (in double quotes) containing format specifiers (formatting operators) and writes formatted data to the file associated with a file identifier `fid`. The argument `fid` is an integer file identifier obtained from the function `fopen` (an example will be provided later in this section). This argument may also assume the value of `1` for standard output, 2 for standard error, or be omitted—in all three cases, `fprintf` will displays the results on the screen (if you opened a file with `fopen`, you should close it later with the reciprocal function `fclose`).

As mentioned in Sec. 4.8.1, the `formSp` character vector or string scalar (as of R2016b) is composed of format specifiers that have the following extended pattern (Fig. 4.17) as opposed to that of Table 4.13:

`%[-][number1][.number2]Y`

([] denotes the optional fields). In this pattern

`%`	Starts the conversion specification (ending with the conversion character `Y`)
`-`	Specifies the alignment
`number1`	Specifies the field width (minimum number of characters to display)
`number2`	Specifies precision - the number of digits to the right of the decimal point (for %f, %e, or %E) or the number of significant digits (for %g or %G)
`Y`	Specifies the notation of the output (format)

Table 4.17 Formatted Output Alignment Flags

Flag	Action
`-`	Left-justifies the converted argument in its field
`+`	Shows a sign character (+ or −)
`0`	Pads with zeros rather than spaces
`empty space`	Inserts a space before the value

Three main alignment codes (flags) are shown in Table 4.17. The list of *conversion characters* (including those mentioned in Table 4.13 for the `textread` function) is given in Table 4.18.

With `*` as the field width operator, you can print different values with different widths (widths and values can be pairs of arguments or pairs), e.g. `fprintf(formSp,fw,a,...)`. With `*` as the precision operator, you can print different values to different precisions (precisions and values can be pairs of arguments or pairs), e.g. `fprintf(formSp,p,a,...)`. When you specify `*.*` as field width and precision operators, you must specify field widths, precisions, and values as triplets, e.g. `fprintf(formSp,fw1,p1,a,fw2,p2,b,...)`.

In between format specifiers (before or after) the `formSp` input argument can also include additional text. The text can be ordinary text you want to display/write or special nonprinted characters that you cannot enter as ordinary text. Table 4.19 shows how to represent special and nonprinted characters in `formSp` using the `\` marker.

Consider two examples allowing you to better understand how the `fprintf` function actually works. The first example accommodates displaying a simple scalar on the screen:

```
>> speed = 62.3;
>> fprintf('The speed is: %3.1f fps \n', speed)
The speed is: 62.3 fps
```

Table 4.18 Conversion Characters for the Formatted Output

Value Type	Conversion Character	Details
Integer, signed	`%d` or `%i`	Base-10 signed integer
Integer, unsigned	`%u`	Base-10 unsigned integer
	`%o`	Base-8 (octal) unsigned integer
	`%x`	Base-16 (hexadecimal) unsigned integer (using lowercase letters a–f)
	`%X`	Base-16 (hexadecimal) unsigned integer (using uppercase letters A–F)
Floating-point number	`%f`	Fixed-point notation (as in `3.14159`)
	`%e`	Exponential (scientific) notation with lowercase e (as in `3.14159e+00`)
	`%E`	Same as `%e`, but using an uppercase E
	`%g`	`%e` or `%f`, whichever is shorter (insignificant zeros are not displayed/written)
	`%G`	Same as `%g`, but using an uppercase G
Characters or strings	`%c`	Single character
	`%s`	Character vector or string array (the type of the output text is the same as the type of `formSp` argument)

Table 4.19 Specifying Special and Nonprinted Characters in the Formatted Output

Special Character	Action
'' (two single quotes)	Apostrophe
\\	Backslash
%%	Percent character
\xN	Character whose Unicode numeric value can be represented by the hexadecimal number, N
\N	Character whose Unicode numeric value can be represented by the octal number, N
\n	Start a new line
\r	Carriage return
\t	Horizontal tab

The second example allows you to write a matrix into the *expon.txt* file:

```
>> x = 0:.1:1;
>> y = [x; exp(x)];
>> fid = fopen('expon.txt','w');
>> fprintf(fid,'%6.2f %12.8f \n',y);
>> fclose(fid)
```

(the `fopen` function defines where to write data to, and `'w'` specifies the permission to open a file, or create a new file for writing, and discard existing contents if any). The newly created file *expon.txt* now contains a short table of the exponential function:

```
0.00   1.00000000
0.10   1.10517092
...     ...
1.00   2.71828183
```

Note that, whereas the first two commands create a 2-by-11 vector `y`, which shows up when using the output display format `format bank` as

```
0      0.10   0.20   0.30   0.40   0.50   0.60   0.70   0.80   0.90   1.00
1.00   1.11   1.22   1.35   1.49   1.65   1.82   2.01   2.23   2.46   2.72
```

the `fprintf` function saves its transpose, that is, an 11-by-2 vector, meaning that the format specification is applied to all elements of array `y` in a column order (in which arrays are stored in computer's memory; see Fig. 3.15a). This is important and should not be overlooked.

If needed, data from this file can be read back by using the reciprocal function `fscanf` as

```
>> fid = fopen('expon.txt');
>> a = fscanf(fid,'%g %g',[2 inf]) % It has two rows now
>> a = a';
>> fclose(fid)
```

Here, the default permission in the `fopen` function (omitted second argument `'r'`) allows you to open the file *expon.txt* for reading alone.

Let us conclude this section with introducing two more functions, `sprintf` and `compose` (introduced in R2016b), which you can use to produce a formatted output as well. The call

```
str = sprintf(format,A1,A2,...)
```

is about the same as the corresponding `fprintf` function call, with the only difference being that it writes formatted data to the character vector or string scalar `str` (the type of the output is the same as the type of `format`). The call

```
str = compose(format,A1,A2,...)
```

converts data (defined in `A1`, `A2`, . . .) into a cell array of character vectors or string array (again, defined by the type of `format`). In both cases, the `sscanf` function can be used to read formatted data back from a character vector or string scalar.

4.10 Interactive Input and Output

To conclude the introductory portion of describing MATLAB, let us present several functions that enable interactive input and output, therefore making the programming process more entertaining.

Let us start with the two simplest options for friendly interactive input of data while running the script. First, you may enjoy using a simple straightforward function `input`. The command

```
>> x = input('text')
```

displays a prompt in the `text` character vector and then waits for an input from a keyboard. The input can be any MATLAB expression, which is evaluated using the variables in the current workspace and then returned in `x`. For instance, the command

```
>> Area = pi*input('Enter the radius: ')^2
```

prompts and expects you to enter a radius, after which it computes and returns the area

```
Enter the radius: 5
Area = 78.5398
```

To enter text, without evaluating the input as an expression, a slightly modified format should be used:

```
>> x=input('text','s')
```

This command displays the prompt and waits for a simple text response. The typed input is not evaluated, so the characters are simply returned in `x`:

```
>> Name = input('Enter the name: ','s')
Enter the name: Yuri Gagarin
Name = 'Yuri Gagarin'
```

The prompt may contain one or more `'\n'`, allowing you to skip to the beginning of the next line. This enables the prompt string spanning several lines, as in the following example:

```
>> N = input('Pick a number between 1 and 10\nYour choice: ')
Pick a number between 1 and 10
Your choice: 5
N = 5
```

(In lieu of `'\n'`, you could have two pieces of text concatenated using the `char(10)` or `newline` call, as explained in Sec. 3.6.)

A more enhanced input capability is provided using the `menu` function. The command

```
k = menu('title','option1','option2',...)
```

creates a multiple-choice dialog box composed of `title` followed by the menu items `option1`, `option2`, . . . It returns the number of the selected menu items as a scalar `k` (if `option1` is chosen, `k=1`; if `option2`, `k=2` etc.). For example, the command

```
>> k=menu('Choose a data marker','o','*','x');
```

can be used to interactively choose the type of the marker. It results in the menu shown in Fig. 4.18a, and you can make your choice by pushing any of the three buttons.

To complete this simple example, let us show how you can use the choice you made to plot, say, a piec3e of parabola. If marker `'o'` is chosen, then the set of commands

```
>> x=1:10; y=x.^2;
>> type = ['o', '*', 'x'];
>> plot(x,y,type(k))
```

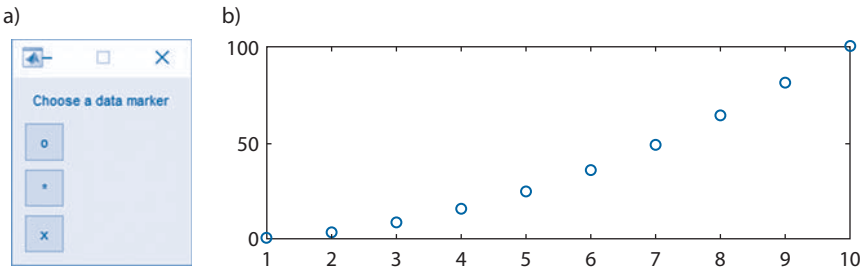


Fig. 4.18 a) Example of user's menu and b) parabola with an interactively chosen marker.

will generate the plot presented in Fig. 4.18b. Note that `k=menu('title',itemlist)`, where `itemlist` is a cell array containing a cell array of character vectors or string array represents a valid syntax. For instance, in the aforementioned example, a cell array `itemlist={'o' '*' 'x'}` might be used.

In addition to the two aforementioned simplest interactive input options, MATLAB provides you with a variety of readily-available professional-looking dialog boxes you may use. Table 4.20 presents a complete list of these predefined dialog boxes (later on, you will be able to develop your own menus using the GUIDE and App Designer tools, as discussed in Appendix F). Let us briefly introduce several of them (you are welcome to use the Help system to get more details and to explore the others).

Let us start from the `questdlg` function. Its usage is as simple as in the following example. The command

```
>> k=questdlg('You want to continue?','Premature Exit','Yes','No','Yes');
```

opens the dialog box shown in Fig. 4.19a. Depending on the button you push, the variable `k` can become either `'Yes'` or `'No'` (`char`-type variable). The last `'Yes'` (must be `'Yes'`, `'No'`) specifies which push button is the default in the event that the <Esc> key is pressed.

Table 4.20 Predefined Dialog Boxes in MATLAB

Function	Description
`dialog`	Creates a dialog box
`errordlg`	Creates an error dialog box
`exportsetupdlg`	Displays the figure export settings dialog box
`export2wsdlg`	Creates a dialog for exporting variables to the workspace
`helpdlg`	Displays a help dialog box
`inputdlg`	Creates an input dialog box
`listdlg`	Creates a list selection dialog box
`msgbox`	Creates a message dialog box
`printpreview`	Displays a dialog box showing the figure as it will print
`printdlg`	Displays a print dialog box
`questdlg`	Creates question dialog box
`uigetdir`	Displays a dialog box to retrieve the name of a directory
`uigetfile`	Displays a dialog box to retrieve the name of a file for reading
`uigetpref`	Conditionally opens a dialog box according to user preference
`uiopen`	Opens a dialog box for selecting a file to load into the workspace
`uiputfile`	Displays a dialog box to retrieve the name of a file for writing
`uisave`	Opens a dialog box for saving variables to *mat*-file
`uisetcolor`	Opens a color selection dialog box
`uisetfont`	Opens a font selection dialog box
`waitbar`	Displays a wait bar
`warndlg`	Creates a warning dialog box

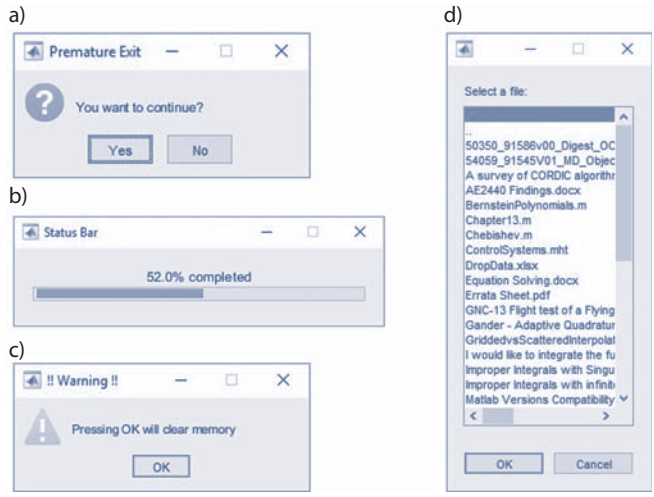


Fig. 4.19 Examples of predefined dialog boxes in MATLAB.

The `waitbar` function is typically used inside a `for` loop (to be discussed in Sec. 5.1) that performs a lengthy computation. For example, the set of commands

```
>> h=waitbar(0,'Please wait...','Name','Status Bar');
>> Npoints=100;
>> for i=1:Npoints, % computation starts here
>> waitbar(i/Npoints,h,[num2str(i/Npoints*100,'%3.1f') '% completed'])
>> pause(0.1)
>> end
>> close(h)
```

shows what percentage of calculation is complete. Throughout the computation's progress, the horizontal bar in Fig. 4.19b fills up to the right, and the number above it gradually increases from 0 to 100% (note how we used the formatted output `'%3.1f'` for this). Because there are no sophisticated computations involved in this example, the `pause(0.1)` line is added to slow it down; otherwise, it is not needed.

The statement

```
>> warndlg('Pressing OK will clear memory','!! Warning !!')
```

uses the `warndlg` function and evokes the dialog box shown in Fig. 4.19c.

The next example presents a dialog box (Fig. 4.19d) that enables a user to select a file from the current directory:

```
>> d = dir;
>> str = {d.name}
>> [s,v] = listdlg('PromptString','Select a file:','SelectionMode',...
                                        'single','ListString',str)
```

The `listdlg` function returns two values, where `s` is the index to the selected file, and `v` is either `1`, indicating that the selection has been made, or `0`, if no file was selected. For example, if the file *Chapter13.m* was chosen, then the returns are as follows:

```
str = {'.'} {'..'} ... {'Chapter13.m'} {'Chebishev.m'} ...
s = 8
v = 1
```

so that `str(8) = {'Chapter13.m'}`.

In this regard, two other functions, `uigetfile` and `uiputfile`, could be more convenient in defining the name of the input and output file. For instance,

```
>> [FileName,PathName] = uigetfile('*.m','Select the M-file');
```

allows you to bring up the window shown in Fig. 4.20a, navigate to the directory of your choice displaying only *m*-files (`'*.m'` serves as a filer employing the `*` wildcard), and return the name and path of a chosen file in `char` variables `FileName` and `PathName`, respectively. Hence, the next command loading the selected file into the current workspace may look like:

```
>> open([PathName FileName]);
```

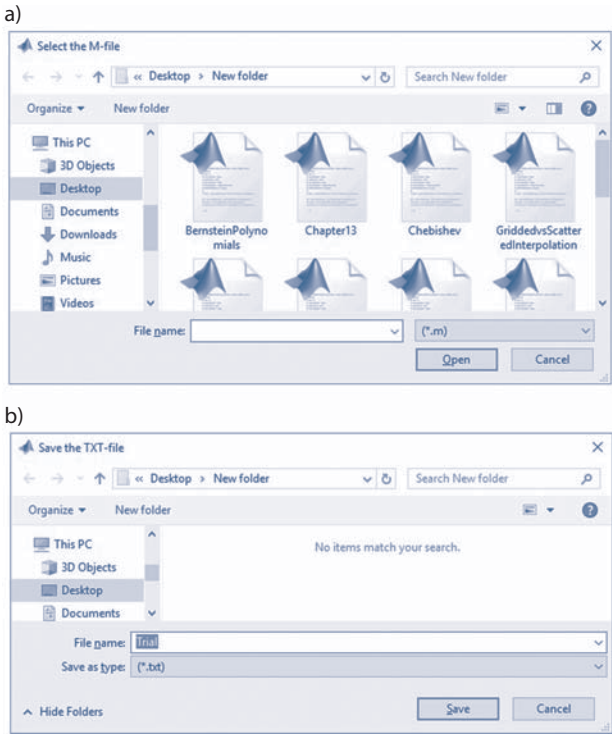


Fig. 4.20 Dialog boxes for a) `uigetfile` and b) `uiputfile` functions.

Similarly, the `uiputfile` displays a modal dialog box used to select or specify a file for saving. For example, the command

```
>> [FileName,PathName]=uiputfile('*.txt','Save
   the TXT-file','Trial');
```

brings up the window shown in Fig. 4.20b. (The optional default filename `Trial` appears in the **File name** field.) Now you can use output variables to save your workspace to the ASCII file:

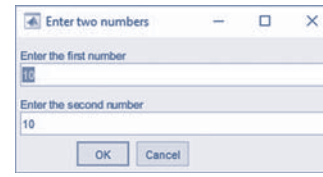


Fig. 4.21 Input dialog window created using the `inputdlg` function.

```
>> save([PathName FileName],'-ascii');
```

One more function you might find very useful for entering the numeric data is `inputdlg`. The following fragment brings a scalable menu, as shown in Fig. 4.21:

```
>> prompt = {'Enter the first number','Enter the second number'};
>> dlg_title = 'Enter two numbers'
>> num_lines = 1;
>> def = {'10','10'};
>> answer = inputdlg(prompt,dlg_title,num_lines,def,'on');
>> a=str2num(answer{1});
>> b=str2num(answer{2});
```

(the optional `'on'` in the `inputdlg` call enables resizing the menu window). Note that the default values showing up in the corresponding fields are stored in a cell array `def` as `char`-type variables. Similarly, the output `answer` is also a cell array, and that is why we need the last two lines of code, accessing the content of the cells and converting it into the numeric format. Hence, after entering two numbers and hitting the **OK** button, these numbers will appear in the current workspace as decimal `a` and `b`, ready to be used in numeric computations.

Once again, you are welcome to learn more about the aforementioned and other dialog boxes presented in Table 4.20 via the MATLAB Help system. As mentioned previously, in addition to these dialog boxes, MATLAB provides tools that enable development of your own more sophisticated GUIs. These tools, GUIDE and App Designer, presented in Appendix F, allow you to use panels, buttons, text fields, sliders, menus, and so on.

Problems

4.1 The following table presents some of the speed records established by turbojet aircraft (it shows only those entries when the record went from one country to another).

Date	Country	Vehicle	Speed (mph)
7 Nov. 45	Great Britain	Gloster Meteor F Mk4	606
19 Jun. 47	USA	Lockheed XP-80R	624
7 Sep. 53	Great Britain	Hawker Hunter F Mk3	728
3 Oct. 53	USA	Douglas XF4D-1	753
10 Mar. 56	Great Britain	Fairey Delta Two	1132
12 Dec. 57	USA	McDonnell F-101A	1208
31 Oct. 59	USSR	Mikoyan E-66	1484
15 Dec. 59	USA	Convair F-106A	1526
7 Jul. 62	USSR	Mikoyan E-166	1666

Create and fill out a structure `Record`, composed of the following fields: `Date`, `Country`, `Vehicle`, and `Speed`. Manipulate the corresponding fields of this structure to a) convert the records to m/s (1 mph = 0.44704 m/s) and b) find the countries that held their speed records for the shortest and the longest times. Hint: use the `datenum` function, for example, `datenum('7-Nov-45', 'dd-mmm-yyyy')`, to convert data to a serial date number.

4.2 Develop (and display) a 2-by-2-by-9 cell array containing the same information as in Problem 4.1. Manipulate the corresponding elements of this cell array to a) convert the records to km/h (1 mph = 1.61 km/h) and b) compute the number of days each record was held before being beaten by another country.

4.3 Issue the following set of commands that creates an ASCII file containing temperatures in degrees of Fahrenheit and Celsius:

```
clear all, clc
T(:,1)=70:-10:-70;
T(:,2)=(T(:,1)-32)/1.8;
save('Temp.txt','-ascii')
clear
```

Now, write a script allowing you to upload data from the *Temp.txt* file into the MATLAB environment (using the `textread` or `dlmread` command), and then display it using formatted output in the following two-column left-adjusted format with a vertical separator (|):

```
|   Temp, F   |   Temp, C  |
|   70        |   21.1     |
|   60        |   15.6     |
|   50        |   10.0     |
|   40        |   4.4      |
|   ...       |   ...      |
```

Chapter 5 Programming in MATLAB

- Relational and Logical Operators
- Conditional and Looping Constructs
- Professional-Looking Functions
- Editing, Debugging, and Profiling

`lt`, `le`, `gt`, `ge`, `eq`, `ne`, `and`, `or`, `not`, `if-else`, `for`, `while`, `switch`, `continue`, `break`, `inline`, `function`, `which`, `isinf`, `iscell`, `isvector`, `ismatrix`, `all`, `any`, `global`, `nargin`, `nargout`, `varargin`, `varargout`, `arguments`, `fcnchk`, `feval`, `pause`, `echo`, `tic-toc`, and others

5.1 Introduction

This chapter is devoted to writing efficient MATLAB programs—scripts and functions. It first addresses the relational and logical operators and functions, followed by several code constructs to alter the straightforward stream of the program (`if` and `switch` constructs, `for` and `while` loops, `continue` and `break` statements). It is followed by creating the simplest scripts. The chapter continues with presenting different types of functions that allow you to perform the same (group of) operations repeatedly. It also addresses the issue of sharing variables between the base and local workspaces used by the functions. It proceeds with showing you how to benefit from the open-source paradigm, which MATLAB supports, to learn about the tricks you can use to develop professional-looking user-defined functions (including how to employ the `nargin`, `nargout`, `varargin`, `varargout`, `fcnchk`, and `feval` functions). Some useful MATLAB functions for fast plotting and zero and minimum finding are introduced in conjunction with exploring available capabilities for passing a function on to another function. The chapter ends with a discussion on using programmatic and MATLAB Editor-enabled tools to debug your code and ensure its superb performance.

5.2 Relational Operators

As in any other programming language, MATLAB has six traditional *relational operators* useful for making comparisons between arrays. They are shown in Table 5.1.

When applying the relational operators, the following holds:

1. The same-size arrays are compared on an element-by-element basis (an exception is when comparison is made to a scalar, and then every element is compared with the scalar).
2. The test compares only the real parts of numeric arrays.
3. Relational operators return a logical array containing elements set to
   - `1` if comparison is `true` and
   - `0` if comparison is `false`.
4. The precedence rules establish that
   - arithmetic operators have precedence over relational operators and
   - relational operators have equal precedence among themselves—they are evaluated in their order from left to right.

Here are some self-explanatory examples of using these relational operators:

```
>> 5 > 2
ans = 1
```

```
>> le(0.1,-4)
ans = 0
```

```
>> 2 ~= 6-4
ans = 0
```

```
>> [0 4] == 0
ans = 1 0
```

Now that you know about relational operators, let us discuss how useful they may be in addressing certain elements of arrays or, to be more specific, the elements that obey certain conditions. One way of doing it is addressing these elements directly via placing the condition in parentheses right after the expression. For example, two commands

```
>> x = 10:-1:0;
>> z = x (x<3)
```

return

```
z = 2  1  0
```

Table 5.1 Relational Operators

Operator	Description
`<`	Shortcut for the `lt` (less than) function [e.g., `A<B`, which is equivalent to `lt(A,B)`]
`<=`	Shortcut for the `le` (less than or equal to) function [e.g., `A<=B` or `le(A,B)`]
`>`	Shortcut for the `gt` (greater than) function [e.g., `A>B` or `gt(A,B)`]
`>=`	Shortcut for the `ge` (greater than or equal to) function [e.g., `A>=B` or `ge(A,B)`]
`==`	Shortcut for the `eq` (equal to) function [e.g., `A=B` or `eq(A,B)`]
`~=`	Shortcut for the `ne` (not equal to) function [e.g., `A~=B` or `ne(A,B)`]

A more sophisticated way of enabling some additional options is to address certain elements indirectly via the `find` function, introduced in Table 3.14. Recall that `find(a)` returns indices of nonzero elements of array `a`. Hence, if the input array `a` is a logical vector, `find(a)` returns indices of `true` elements. For example, for the vector `x` created earlier, the command

```
>> ind = find(x<3)
```

yields the indices of the elements that are less than 3

```
ind = 9  10  11
```

To refer to elements with these indices, we could use

```
>> z = x(find(x<3))
z = 2  1  0
```

Alternately, the same result could be achieved even without using a relational operator. For example, you can manipulate with the original vector using the `min` function as follows:

```
>> z = x(find(min(x-3,0)))
z = 2  1  0
```

5.3 Logical (Boolean) Operators and Functions

MATLAB offers several logical operators and functions. To begin with, it provides you with three standard (Boolean) element-wise logical operators:

`~` (Boolean NOT), so that `~A` or `not(A)` returns an array of the same dimension as `A`, which has `1`'s where `A` is `0` and `0`'s where `A` is nonzero

`&` (Boolean AND), so that `A&B` or `and(A,B)` returns a logical array of the same dimension as `A` and `B`, which has `1`'s where both `A` and `B` have nonzero elements and `0`'s where either `A` or `B` has `0`

`|` (Boolean OR), so that `A|B` or `or(A,B)` returns an array of the same dimension as `A` and `B`, which has `1`'s where at least one element in `A` or `B` is nonzero and `0`'s where `A` and `B` are both `0`

One more logical operator, namely the `xor(A,B)` function (exclusive OR), is also available. This function is defined in terms of AND, OR, and NOT operators as

```
function X = xor(S,T)
X = (S|T) & ~(S&T);
```

By design, it returns `0`'s where `A` and `B` are either both nonzero or both zero and `1`'s where `A` or `B` is nonzero, but not both.

To better understand all four operators, the following set of commands produces the so-called *truth table*, which defines the outcomes of applying

standard logical operators and the `xor` function (remember, `1` stands for `true` and `0` for `false`)

```
>> x = [1;1;0;0];
>> y = [1;0;1;0];
>> disp('  x   y   ~x   x&y   x|y   xor(x,y)')
>> disp([x,y,~x,x|y,x&y,xor(x,y)])
     x   y   ~x   x&y   x|y   xor(x,y)
     1   1    0    1    1     0
     1   0    0    0    1     1
     0   1    1    0    1     1
     0   0    1    0    0     0
```

Now, have another look at the truth table. Note that for the `x&y` column, if `x` is `0` (`false`), it does not really matter what the value of `y` is. Similarly, if `x` is `1` (`true`), the result of `x|y` is immediately defined as well. This logical short-circuiting behavior is employed in two more logical operators, `&&` and `||`, intended to be used for comparing scalar logical expressions (as opposed to arrays). Specifically, when using `&&` and `||`, the second operand is evaluated only if the result is not fully defined by the first operand.

For example, it does not make sense to evaluate `1/b` if `b` is undefined. Hence, before evaluating the second operand in the following statement: `exist('b')&&(1/b>2)`, the first operand checks the existence of variable `b` in the workspace (see Table 2.12). If the result of `exist('b')` is logical `0`, the second operand is not even evaluated and the short-circuit AND returns `0`.

Table 5.2 Operator Precedence

Order	Operator	Description
1	`()`	Parentheses
2	`.', .^, ', ^`	Transpose, power, complex conjugate transpose, and matrix power
3	`+, -, ~`	Unary plus, unary minus, and logical negation
4	`.*, ./, .\, *, /, \`	Multiplication, right division, left division, matrix multiplication, matrix right division, and matrix left division
5	`+, -`	Addition and subtraction
6	`:`	Colon operator
7	`<, <=, >, >=, ==, ~=`	Less than, less than or equal to, greater than, greater than or equal to, equal to, and not equal to
8	`&`	Element-wise AND
9	`|`	Element-wise OR
10	`&&`	Short-circuit AND
11	`||`	Short-circuit OR

Be aware that when you use the element-wise `&` and `|` operators in the context of the `if` or `while` loop constructs (to be considered in Sec. 5.4), they use short circuiting to evaluate expressions implicitly. In all other cases, you must specify `&&` or `||` to opt in to short-circuiting behavior explicitly.

As to operator precedence, logical operations have lower precedence than arithmetic and relational operators with the exception of the `not` operator (`~`). For instance, the command `0|~0` needs no parentheses [i.e., it is executed as `0|(~0)`]. The `and` operator has higher precedence than `or`. For example, the result of `1|0&0` is `1`, not `0` [because it is executed as `1|(0&0)`].

Now that we have considered all types of operations (algebraic, relational, and logical), the complete table of precedence looks as shown in Table 5.2.

MATLAB code robustness is enabled by a variety of the logical functions. To this end, Table 5.3 lists most of the functions available to check the type or content of an array. Table 5.4 lists data type testing functions.

Table 5.3 Array Type/Content Testing Functions

Function	Brief Description
`all(x)` and `all(A)`	Returns a `1` if all the elements in the vector `x` or columns of matrix `A` are nonzero and a `0` otherwise
`any(A)`	Returns a row vector with same number of columns of `A` containing ones and zeros depending on whether or not the corresponding columns of `A` contain any nonzero elements
`any(x)`	Returns a scalar, that is, `1` if any of the elements in the vector `x` is nonzero and `0` otherwise
`isfinite(A)`	Returns an array of the same dimension as `A` with `1`'s where the elements of `A` are finite
`iscolumn(A)`	Determines if input `A` is a column vector
`isempty(A)`	Returns a `1` if `A` is the empty matrix and `0` otherwise (checks whether `A==[]`)
`isfloat(A)`	Determines whether input `A` is a floating-point array
`isinf(A)`	Returns an array of the same dimension as `A` with `1`'s where `A` has `inf` and `0`'s elsewhere
`isinteger(A)`	Determines whether input `A` is an integer array
`islogical(A)`	Determines whether input `A` is a logical array
`ismatrix(A)`	Determines whether input `A` is a matrix
`isnan(A)`	Returns an array of the same dimension as `A` with `1`'s where `A` has `NaN` and `0`'s elsewhere
`isnumeric(A)`	Determines whether input `A` is a numeric array
`isreal(A)`	Returns a `1` if `A` has no elements with imaginary parts and `0` otherwise
`isrow(A)`	Determines whether input `A` is a row vector
`isscalar(A)`	Determines whether input `A` is a scalar
`issorted(A)`	Determines whether a set of elements `A` is in sorted order
`isvector(A)`	Determines whether input `A` is a vector

Table 5.4 Data Type Testing Functions

Function	Brief Description
`iscell(A)`	Determines whether input `A` is a cell array
`iscellstr(A)`	Determines whether input `A` is a cell array of strings
`ischar(A)`	Determines whether input `A` is a character array
`isfile(A)`	Determines whether input `A` is a file
`isfolder(A)`	Determines whether input `A` is a folder
`iskeyword('A')`	Determines whether input `A` is a MATLAB keyword
`isletter(A)`	Determines which characters in a character array or string scalar `A` are letters
`isspace(A)`	Determines which characters in a character array or string scalar `A` are whitespace characters
`isstring(A)`	Determines whether input `A` is a string array
`isStringScalar(A)`	Determines whether input `A` is a string array with one element
`isstrprop(A,cat)`	Determines whether string `A` is of a specified category `cat` (`'alpha'`, `'alphanum'`, `'cntrl'`, `'digit'`, `'graphic'`, `'lower'`, `'print'`, `'punct'`, `'wspace'`, `'upper'`, `'xdigit'`)
`isstruct(A)`	Determines whether input `A` is a structure array
`istable(A)`	Determines whether input `A` is a table
`isvarname(A)`	Determines whether input `A` is a valid variable name

5.4 Conditional Statements

The statements inside your source files are generally executed from top to bottom, in the order that they appear. MATLAB provides a standard programming means to construct the so-called *control flow statements*, breaking up the flow of execution by employing decision making, looping, and branching, which enables your code to conditionally execute particular blocks of code.

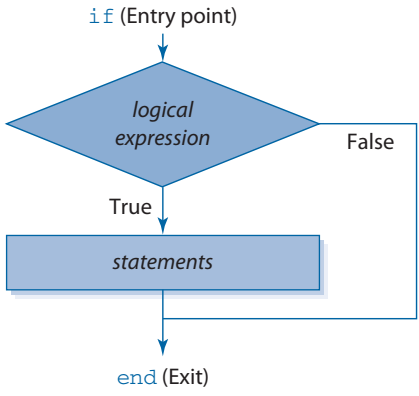


Fig. 5.1 Flowchart representing the `if` construct.

Specifically, logical expressions and *conditional constructs*, containing `if`, `elseif`, and `else` statements, allow executing different computations or actions depending on whether a programmer-specified *Boolean condition* (queries that compare two values with each other) evaluates to `true` or `false`.

The `if` statement has the following basic form (Fig. 5.1):

```
if logical expression
   statements
end
```

where the first line establishes some decision-making condition and the `end` statement denotes the end of a conditional construct, which is required to close the programming logic and cause the statements to be executed when the condition is `true`.

When coding the `if` constructs

1. A space is required between the `if` and *logical expression*.
2. The *logical expression* may be a compound expression (multiple arithmetic, relational, and logical operators).
3. If the *logical expression* is performed on an array, then the result of a conditional test is `true` only if all elements of the *logical expression* are `ture` (nonzero).
4. The *statements*, representing a series of MATLAB commands, are only executed if the *logical expression* is `ture`.
5. For compactness, the `if` structure with a short statement may be written on a single line: `if` *logical expression, statements,* `end`.

For example, the construct

```
U=rand(1);
if U > 0.5
   disp('U > 0.5')
end
```

provides a return only if a random scalar `U` happens to be greater than 0.5.

The `if` statements may be nested (Fig. 5.2). Note that in this case, each `if` statement has its corresponding `end`

```
if logical expression 1
      statements 1
   if logical expression 2
         statements 2
```

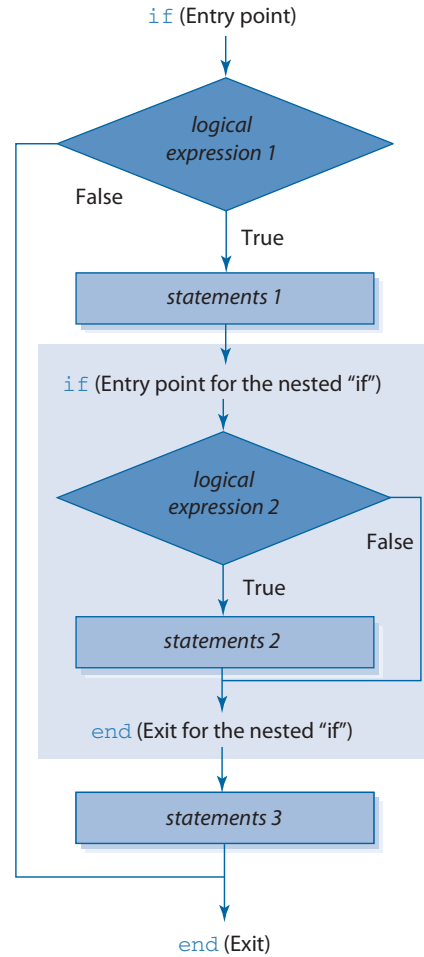


Fig. 5.2 The nested `if` construct.

```
   end
     statements 3
end
```

If more than one action can occur, the `else` and/or `elseif` statements should be used (depending on the number of actions). The basic form for the `if-else` construct is represented as Fig. 5.3

```
if logical expression
     statements 1
else
     statements 2
end
```

For example, the construct

```
U=rand;
if U > 0.5
   disp('U > 0.5')
else
   disp('U <= 0.5')
end
```

provides two different returns based on whether a random scalar `U` happens to be greater than 0.5 or not.

Note, the logical expression may be based off comparing text data as well. For example, the construct

```
Choices=["Heads" "Tail"]'; U=randi(2);
if Choices(U)=="Heads",    disp('Heads')
else,                      disp('Tails'), end
```

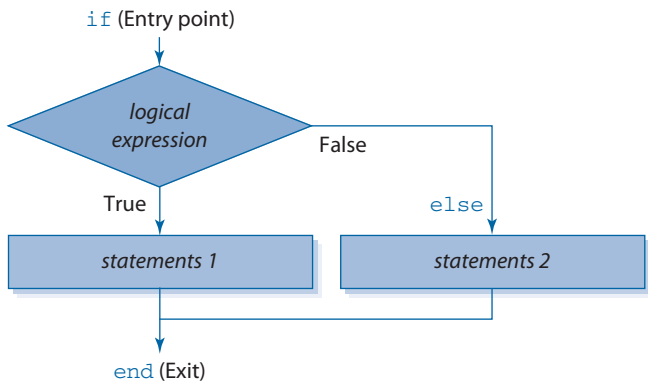


Fig. 5.3 Flowchart of the `if-else` construct.

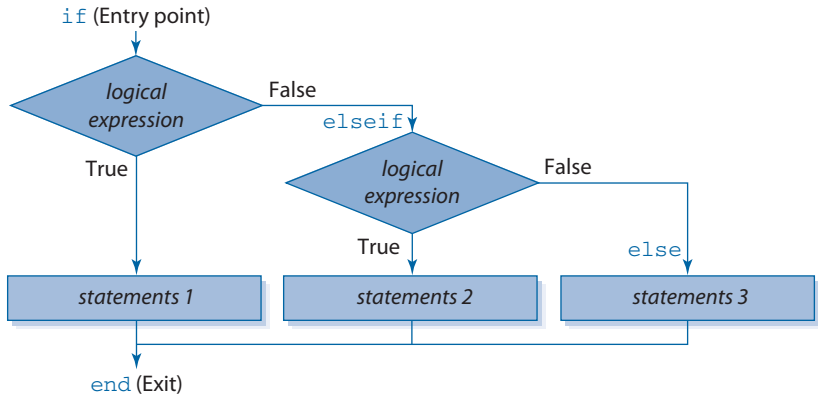


Fig. 5.4 Flowchart for the `if-elseif` construct.

emulates tossing a coin. Be aware, that when comparing character vectors, cell arrays of character vectors, or string arrays, they are equal *if and only if* every character is the same including a letter case and number of blank spaces (if any). To compare two textual inputs, `s1` and `s2`, you may also consider using the `strcmp(s1,s2)` function (see Table B.6 of Appendix B).

The basic form for the `if-elseif` construct may be represented as follows (Fig. 5.4):

```
if logical expression 1
      statements 1
elseif  logical expression 2
      statements 2
else
     statements 3
end
```

For example, the construct

```
U=rand(1,3);
if U(1)>0.5
   disp('Heads')
elseif U(2)>0.5
   disp('Two heads!')
elseif U(3)>0.5
   disp('No way!!!')
end
```

emulates tossing a coin three times. The general rule is that the `if` construct may have multiple `elseif` statements, but only one `else` statement. (Of course, all rules for the `if` portion should be upheld as well.)

5.5 Loops

A *loop* is a construct for repeating a calculation for a number of times. We call each repetition of the loop a *pass*. MATLAB supports two types of loops: *implicit loops* and *explicit loops*. The implicit loops have already been considered on several occasions: first, when we studied the usage of a colon (for example, in `x=0:5:100`); second, when array operations were introduced [for example, `y=cos(x)` or `z=length(x)`, where `x` is a vector]; and finally, when matrix operations were considered (for example, the multiplication of two matrices implies using the loops to sum the results of pair element multiplication). The following provides some details about using the two types of explicit loops, `for` loops and `while` loops.

5.5.1 The `for` Loops

The `for` loop has the following basic form (Fig. 5.5):

```
for loop_variable = m:s:n
      statements
end
```

subject to a set of regulations:

1. The expression `m:s:n` assigns the loop variable an initial value `m` and increments it by the value `s` until the result is less than or equal to `n`. Looping breaks when the `loop_variable` exceeds the terminating value `n`.

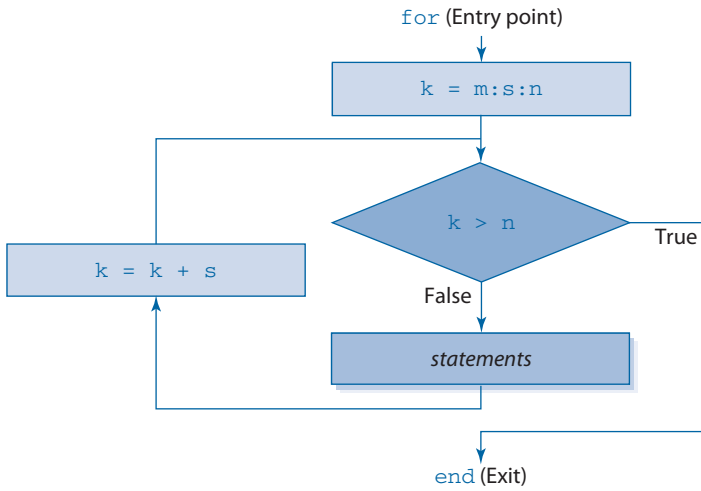


Fig. 5.5 Flowchart for the `for` loop (`k` is a `loop_variable`).

2. Parameters `m`, `s`, and `n` may assume any real value or be real variables (no complex part) or expressions. Additionally
   - If the increment `s` is omitted, the default value is `1`
   - If `m=n`, the loop will be executed only once
   - If the increment `s` is not an integer, round-off errors may cause the loop to execute a different number of passes than expected
3. The statements are executed once during each pass using the current value of the `loop_variable`.
4. An attempt to alter the `loop_variable` inside the loop will fail because MATLAB will not recognize this change (this is a sort of self-protection).
5. The `end` statement closes the programming logic and is mandatory.
6. Indenting is typically used to improve the readability; however, the `for` loop may be written on a single line:

   `for loop_variable=m:s:n,` *statements*, `end`

7. Other loops and conditional statements may be *nested* within the `for` loop.

For example, the `for` loop

```
R=0;
for i=0:4
 R=R+10;
 disp(['i=' int2str(i) '; R=' int2str(R)])
end
```

executes four times returning

```
i=0; R=10
i=1; R=20
i=2; R=30
i=3; R=40
i=4; R=50
```

5.5.2 The `while` Loops

The `while` loops are used to create a loop that terminates when a certain condition is satisfied. They have the following basic form (Fig. 5.6):

```
while logical expression
      statements
end
```

subject to regulations:

1. The *logical expression* should contain a *loop variable* that can be changed to make the expression `false` and end the loop.

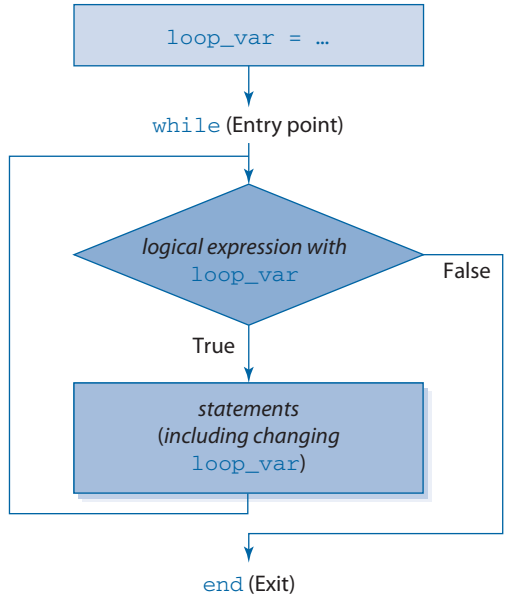


Fig. 5.6 Flowchart for the `while` loop (`loop_var` is a loop variable).

2. The *loop variable* must have a value before the `while` statement is executed.
3. The *loop variable* must be changed somehow by the *statements*; otherwise, you may end up with an *infinite loop* (infinite loops is loop that never ends, that is, the *logical expression* never becomes `false`).
4. *Statements* are executed once during each pass, unless there are nested loops.
5. The mandatory `end` statement should close the programming logic.

Be cautious when using equality (`==`) or nonequality (`~=`) in the *logical expression*. As opposed to the `for` loop, where you define the *loop variable* explicitly, in the `while` loop this variable changes as a result of some computations. If the compared quantities are not integers, the round-off error may cause the loop to continue when you would have expected it to end.

Similar to the example of Sec. 5.5.1, the `while` loop

```
R=0; i=0;
while i<=4;
R=R+10;
  disp(['i=' int2str(i) '; R=' int2str(R)])
  i=i+1;
end
```

yields exactly the same return.

5.5.3 `continue` and `break` Statements

There are two statements that can be used in conjunction with explicit loops to debug and/or improve the efficiency of the loops. They are `continue` and `break`.

The `continue` statement passes control to the next pass of the `for` or `while` loop in which it appears, skipping any remaining statements in the body of the loop. For example,

```
s=0; f=0;
for i=1:10
  s=s+1;
  continue
  f=f+1;
end
[s, f]
```

returns

```
ans = 10  0
```

because the statement `f=f+1` was never executed. In the case of a nested loop, `continue` passes control to the next pass of the `for` or `while` loop enclosing it.

The `break` statement terminates the execution of a `for` or `while` loop. Statements in the loop that appear after the `break` statement are not executed. For example,

```
s=0; f=0;
while s<=10
    s=s+1;
    if s==5
        break
    end
    f=f+1;
end
[s, f]
```

results in

```
ans = 5  4
```

In nested loops, `break` exits only from the loop in which it occurs. Control passes to the statement that follows the `end` of that loop.

Usually, the use of the `break` statement in the final (debugged) program is discouraged. Instead, a properly constructed `while` loop should be used.

5.6 The `switch` Construct

The `switch` construct provides an alternative to using the `if` construct and for some applications is more readable. Its basic form is (Fig. 5.7)

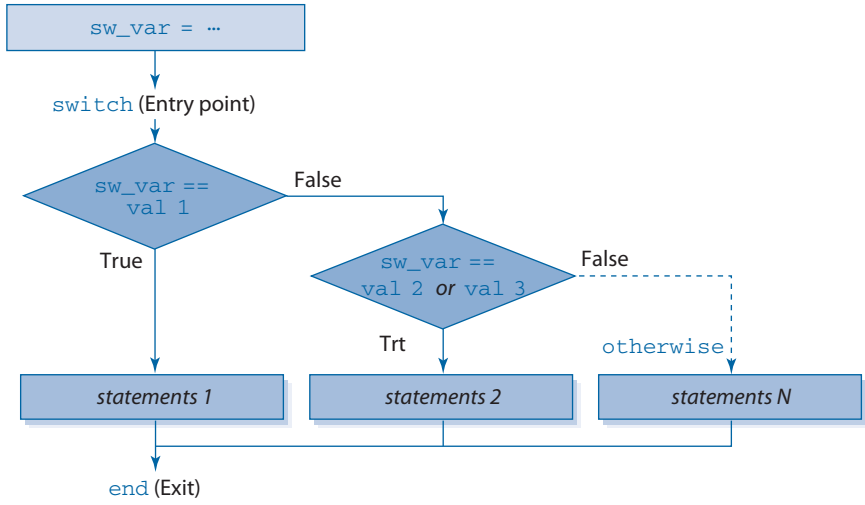


Fig. 5.7 Flowchart for the two-case `switch` construct with the `otherwise` option.

```
switch switch_variable
  case value 1
          statements 1
  case {value 2, value 3}
          statements 2
...
  otherwise
          statements N
end
```

When executing the `switch` construct, the *switch_variable* is compared to each *value*. If they match, the statements following that `case` statement are executed. The following rules apply:

1. Each `case` *value* statement must appear on a separate line.
2. The `switch` construct can handle multiple conditions in a single `case` statement by enclosing several *values* into a cell array.
3. Only the first matching `case` is executed.

The `otherwise` statement is optional. If it is not present and no `case` match occurs, then the next statement following the `end` statement is executed.

For example, consider creating a list of the fighter jets using the `menu` function

```
k=menu('Choose an aircraft','F-4','F-6','F-15','F-16',...
       'F-18','F-22','F-35','F-80','F-86','F-104','F-105');
```

Then, after choosing one of menu options, the two following constructs (one using `if` and one using `switch`) produce exactly the same result:

```
if k==8
 disp('1st gen. jet fighter')
elseif k==9
 disp('2nd gen. jet a/c')
elseif k==1 || k==2 || ...
       k==10 || k==11
 disp('3rd gen. jet a/c')
elseif k==3 || k==4
 disp('4th gen. jet a/c')
elseif k==6 || k==7
 disp('5th gen. jet a/c')
else
 disp('4+ gen. jet a/c')
end
```

```
switch k
 case 8
 disp('1st gen. jet a/c')
 case 9
 disp('2nd gen. jet a/c')
 case {1,2,10,11}
 disp('3rd gen. jet a/c')
 case {3,4}
 disp('4th gen. jet a/c')
 case {6,7}
 disp('5th gen. jet a/c')
 otherwise
 disp('4+ gen. jet a/c')
end
```

5.7 The Script Files

A *script file* is a file that contains any sequence of MATLAB statements (by default, the scripts have a filename extension *.m* and are therefore referred to as *m*-files). They require no declarations or begin/end delimiters. By typing the filename in the Command Window, these statements are executed and the corresponding output is produced.

The scripts represent the simplest kind of *m*-files. They are useful for automating blocks of MATLAB commands, such as computations you have to perform repeatedly from the command line. Scripts can operate on existing data in the workspace, or they can create new data on which to operate. Although the scripts do not return output arguments, any variables they create remain in the workspace, so you can use them in further computations. In addition, the scripts can produce graphical output using a variety of the 2-D and 2-D plotting functions like `plot` and `surf`.

Like any *m*-file, scripts can contain *comments* (nonexecutable sequences of characters). Any text following a percent sign, `%`, on a given line is comment text. To distinguish it from the rest of the text, MATLAB makes its color green (the default color for comments, as defined in the preferences shown in Fig. 2.30b). Comments can appear on lines by themselves, or you can append them to the end of any executable line. For instance,

```
% Earth's Parameters
e = 298.257223563;  % WGS-84 Earth's ellipticity
f = 1 / e;          % Earth's flattening
eps2= 1-(1-f)^2;    % Earth's eccentricity (squared)
```

Another use for the comments is when you are testing your files or looking for errors and would like to temporarily turn some lines of code into comments to see how the *m*-file runs without those lines.

You may also comment a contiguous group of lines without commenting each of them individually (this is useful when you want to identify the section of a file that is not working as expected). Simply type `%{` before the first line and `%}` after the last line you want to comment, as in the following example (for better understanding the commented block is indented):

```
g=9.81;
 %{
 g=-3.986004418e14/rp^3*(eye(3)+...
 1.5*1.0826267e-3*(re/rp)^2*(egm-5*sin(mu)*eye(3)))*p;
 %}
display(g)
```

This is referred to as a *block comment*. Note that, the lines that contain `%{` and `%}` cannot contain any other text. After typing the opening block comment symbols, `%{`, all subsequent lines to the very end of the file assume the syntax highlighting color for comments until you type the closing block comment symbols, `%}`. At that point, only the lines between the block comment symbols show up in green. You can easily extend a block comment without losing the original block comment, that is, create a nested block comment, as shown in the following example:

```
%{
g=9.81;
  %{
  g=-3.986004418e14/rp^3*(eye(3)+...
  1.5*1.0826267e-3*(re/rp)^2*(egm-5*sin(mu)*eye (3)))*p;
  %}
display(g)
%}
```

You can also comment out a text within a multiline statement using the ellipsis (`...`). MATLAB ignores any text appearing after the `...` on a line and continues to the next line. This effectively makes a comment out of anything on the current line that follows the `...`. For example, in

```
d=-omegab+R_becef'*...
    ... omegae*
    R_becef
```

the term appearing on the second line is ignored.

Comments also determine what text will be displayed when you run the `help` function for your specific *m*-mile. This option will be addressed in more detail in Secs. 5.8.2 and 5.9.2. Overall, using the extensive comments

in an *m*-file to describe the goal of code, its flow, and input and output arguments or to make special remarks is a good habit and is highly encouraged, especially if you intend to share this code with others or reuse it later yourself. In the latter case, it will help you to recall the details of your own code when you return to it after a while.

One more special character you might find useful is the exclamation point (`!`). The exclamation point precedes an operating system command that you want to execute from within a MATLAB script. For example,

```
! unzip Garmin-1370-QWX77
```

unzips the *Garmin-1370-QWX77.zip* file (residing in the current directory).

To display a content of the specified file `filename` in the MATLAB Command Window, you can use a command-syntax call `type filename` or a function-syntax call `type('filename')`, where `filename` is a character vector (or string scalar). If you do not specify a `filename` extension and there is no `filename` file without an extension, the `type` function assumes that the extension is *.mlx* (for the live scripts), *.mlapp* (for the files created using the App Designer tool), or *.m* (for the *m*-files).

It may be useful to use the `type` command following the `more on` instruction to see the listing on one screen at a time (by default, `more` is disabled). For example, try

```
>> more on
>> type fzero
```

When enabled, `more` defaults to displaying 23 lines per page [`more(n)` displays `n` lines per page]. You can then use your keyboard to perform the following operations:

<Return>	Advances to the next line of output
<Space>	Advances to the next page of output
<Q>	Terminates display of the text

The `more off` instruction disables paging of the output.

5.8 Functions

With two exceptions, a *function* is another type of *m*-file that contains several lines of code. Basically, it only differs from a usual script by its very first line. This line, the function declaration statement, declares the function's name, and defines the list of input parameters the function works with and the list of output parameters it can produce. But before we consider this type of function, let us introduce those two exceptions that do not need a full-scale *m*-file but can rather be represented by a single line of code.

5.8.1 Inline and Anonymous Functions

As of R2020a, the two options for defining a simple single-statement function that is not stored in an *m*-file are using either the inline or anonymous function. Even though the *inline function* is expected to be removed in a future release, let us briefly describe it. The command

```
inline(expr)
```

constructs an inline function object from the MATLAB expression contained in a character vector or string scalar `expr`. The input argument(s) for the `inline` function are determined automatically by accessing `expr`. For example,

```
>> f1 = inline('ia*sqrt(xot)+X*i')
```

returns

```
f1 =
     Inline function:
     f1(X,ia,xot) = ia*sqrt(xot)+X^2*i
```

You can now call this function by passing on three input arguments. For example,

```
>> f1(5,7,2)
```

yields

```
ans = 9.8995 + 25.0000i
```

Table 5.5 presents the three auxiliary functions, allowing you to examine an inline function object and determine how it was created. Also, it shows the function helping to vectorize a mathematical expression automatically.

If applied to the inline function `f1` defined earlier, the first two functions of Table 5.5 return

```
'ia*sqrt(xot)+X*i'
```

and the third function yields

Table 5.5 Functions for Manipulating with an Inline Function Object

Function	Brief Description
`formula(fun)`	Returns the formula for the inline object `fun` (this function can also be used with a symbolic `fun`)
`char(fun)`	Converts the inline function `fun` into a character vector [for the inline function, the return is identical to `formula(fun)`]
`argnames(fun)`	Returns the names of the input arguments of the inline object `fun` as a cell array of the character vectors
`vectorize(fun)`	Inserts a `.` before any `^`, `*`, or `/` in the formula for `fun`, resulting in a vectorized version of the inline function enabling the array inputs

```
3×1 cell array
   {'X' }
   {'ia' }
   {'xot'}
```

The fourth function

```
>> f2=vectorize (f1)
```

results in

```
f2 =
     Inline function:
     f2(X,ia,xot) = ia.*sqrt(xot)+X.^2.*i
```

so that you may now use it with the array inputs

```
>> f2(2:4,[1;-3;2],[0;1;1])
```

returns

```
ans = 0.0000 + 4.0000i   0.0000 + 9.0000i   0.0000 +16.0000i
     -3.0000 + 4.0000i  -3.0000 + 9.0000i  -3.0000 +16.0000i
      2.0000 + 4.0000i   2.0000 + 9.0000i   2.0000 +16.0000i
```

(passing the array inputs into the original inline function `f1` would result in an error).

Consider another example. The following call defines the function `f3` to be dependent on three variables, `C`, `alpha`, and `x`, in the implicitly defined order

```
>> f3 = inline('alpha*cos(C*x)')
f3(C,alpha,x) = alpha*cos(C*x)
```

A more complete syntax, `inline(expr,arg1,arg2,...)`, allows you to change the order. Continuing the previous example, we could do it as follows:

```
>> f4 = inline('alpha*cos(C*x)','x','alpha','C')
f4(x,alpha,C) = alpha*cos(C*x)
```

Be aware that one way or another, all variables entering the `expr` should be declared as the arguments, meaning that you cannot declare something like

```
>> f5 = inline('alpha*cos(C*x)','x')
```

returning

```
f5(x) = alpha*cos(C*x)
```

In this latter case, you would not be able to use this function because you would not be able to specify `C` and `alpha` afterward.

An *anonymous function* is another simple form of the MATLAB function that does not require an *m*-file. The syntax for creating an anonymous function from a mathematical expression is

```
fhandle = @(arglist) expression
```

where `expression` consists of any single, valid MATLAB expression, and `arglist` is a comma-separated list of the input arguments used by `expression` and to be passed to the function (these two components are similar to the body and argument list components of any *m*-file function, as addressed in the following section).

Leading off the entire right-hand side of this statement is an `@` sign. The `@` sign is the MATLAB operator that constructs a *function handle* (of the `function_handle` data type; see Fig. 4.1), a MATLAB value that provides a means of calling a function indirectly. The aforementioned statement constructs the anonymous function, returns a handle to this function, and stores the value of the handle in a variable `fhandle`. For instance, the following statement creates an anonymous function that finds the cubic root of a number:

```
>> cubroot = @(x) power(x,1/3);
cubroot =
  function_handle with value:
    @(x)power(x,1/3)
```

To execute the `cubroot` function, you simply type

```
>> cubroot(27)
ans = 3
```

For the functions that do not take any input arguments, you can construct the anonymous function with an empty argument list, for example

```
>> t = @() datestr(now);
```

When calling such a function, you should use an empty argument list enclosed in parentheses

```
>> t()
ans = 22-Jan-2019 21:39:38
```

Another example shows how to create and use a handle to the function of two variables:

```
>> A = 3; B = 2;
>> sumAxBy = @(x,y) (A*x + B*y);
>> sumAxBy(5,6)
ans = 27
```

(Note that, similar to `inline`, you have to define all parameters in the expression beyond its arguments beforehand.)

One more example demonstrates that the input argument list may contain arrays as well (do not forget to use a vectorized version of mathematical expression in this case). For example,

```
>> myfun = @(x) sin(x(1)-2).*cos(x(2))
>> [x,y]=fminsearch(myfun,[2;-2])
```

Here `fminsearch` is the MATLAB function that accepts the mathematical expression and vector of initial guesses on varied parameters (x_1 and x_2 in this case). Although the `fminsearch` function is to be formally introduced in Chapter 10, the important point to make here is that the function handle can be passed on to another function as an argument. These two lines of code define $y = f(\mathbf{x})$, where $\mathbf{x} = [x_1, x_2]^T$ and then solve for its minimum

$$\mathbf{x}_* = \arg\left(\min_{\mathbf{x}} f(\mathbf{x})\right) \tag{5.1}$$

returning $\mathbf{x}_*$ in `x`, and $f(\mathbf{x}_*)$ in `y`.

```
x =  3.5708
    -3.1416
y = -1.0000
```

Speaking of handles, it should be noted that you can create a function handle to any named function, including the MATLAB functions. For example,

```
>> f_sin = @sin;
```

creates a handle to the built-in MATLAB function `sin`. You can now use this handle as if you were using the function itself:

```
>> f_sin(1:3)
ans = 0.8415  0.9093  0.1411
```

The good thing about function handles is that you can store them in the data structures for later use. For instance, the following fragment creates a handle to an anonymous function, stores it along with other handles in a cell array, and then uses it as an element of this cell array to produce a plot:

```
>> sincos = @ (x) sin(x).*cos(x);
>> trigFun = {@sin, @cos, sincos, @tan};
>> plot(trigFun{3}(-pi:0.01:pi))
```

The `plot` function (to be discussed in Sec. 6.2) creates a plot shown in Fig. 5.8a.

Even though you are not explicitly defining the output arguments when creating an anonymous function, you can request them if the expression in the function assumes multiple outputs. For example, the `ndgrid` function can return as many outputs as the number of input vectors. As such, after executing the instructions

```
>> c = 15;
>> mygrid=@(x,y) ndgrid((-x:x/c:x),(-y:y/c:y));
```

it is appropriate to request the multiple (up to two) outputs (enclosed in square brackets):

```
[X,Y]=mygrid(pi,2*pi);
```

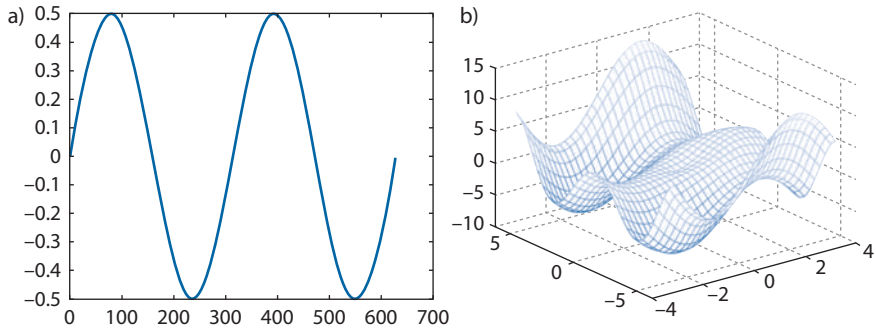


Fig. 5.8 Examples of using a function handle as an input argument to another function to produce a 2-D (a) or 3-D (b) plot.

You can use the output from `mygrid` to create a mesh or surface plot (to be discussed in detail in Sec. 6.8). For example, the two lines

```
>> Z=(sin(X)+cos(Y)).*sqrt(X.^2+Z.^2);
>> mesh(X,Y,Z)
```

return the plot shown in Fig. 5.8b.

You can also create and use a *vector-valued anonymous function*. For example,

```
>> vvf = @(x)cos((1:5)*x);
```

creates a vector-valued function returning five values [for $\cos(x), \ldots, \cos(5x)$]. Not many functions accept a vector-valued function as an input argument, but the `quadv` and `integral` functions (presented Tables 13.4 and 13.6, respectively) do. For example, the call

```
>> q = integral(vvf,-1,1,'ArrayValued',true)
```

integrates `vvf` from $x = -1$ to $x = 1$ returning

```
q =   1.6829   0.9093   0.0941  -0.3784  -0.3836
```

It is the array-valued function flag defined using the name–value pair argument in this call that makes it possible (by default the flag value is `false`).

Table 5.6 presents several MATLAB functions that are specifically dedicated to work with the function handles.

Table 5.6 Functions for Manipulating with a Function Handle

Function	Brief Description
`functions`	Returns a structure describing a function handle
`func2str`	Constructs a character vector from a function handle
`str2func`	Constructs a function handle from a function name, character vector, or string scalar

For example, let us create two function handles using the following statements:

```
>> fh1 = str2func('rand');
>> fh2 = str2func(['@(t)' '3*2*t^2']);
```

Then,

```
>> functions(fh1)
```

yields

```
ans =
  struct with fields:
          function: 'rand'
              type: 'simple'
              file: 'MATLAB built-in function'
```

and

```
>> func2str(fh2)
```

returns

```
ans =  '@(t)3*2*t^2'
```

Be aware that MATLAB maps the handle to the function you specify and saves this mapping information in the handle. A function handle retains that same mapping even if its corresponding function goes out of scope. For example, if, after creating the `fh1` handle, you change the MATLAB path so that a different function with the same name (`rand`) now takes precedence, invoking the function handle still executes code to which the handle was originally mapped.

Converting in between the two types of functions described in this section can be done using the functions of Tables 5.6 and 5.4. For example, given the inline and anonymous functions as

```
>> f1_inl = inline('x^4-x*3+1e3*sin(x)')
>> f2_ano = @(x) cos(2*x)-sin(1.5*x^2)
```

you can convert them to an alternative type as follows:

```
>> f1_ano = str2func(['@(x)' char(f1_inl)])
>> f = func2str(f2_ano); ind = find(f==')'); f2_inl = inline(f(ind(1)+1:end))
```

5.8.2 *M*-File Functions

As explained in the previous section, only very simple expressions can be coded as inline and anonymous functions. More complex algorithms are programmed as the *m*-file functions. The first line of an *m*-file function, function header, defines its syntax, that is, the way this function can be called. Let us proceed with introducing four frequently used MATLAB functions, `flot`, `fzero`, `fminbnd`, `fminsearch`, with the

goal of understanding how to call them and pass in necessary parameters. Then, we will formally introduce the anatomy of the function and, in the following sections, address several issues associated with developing, storing, and using the user-defined functions.

The MATLAB `fplot` function plots the specified function of interest within the certain limits. One of its simplest syntaxes is `fplot(fun,limits)`, where `fun` is a function handle, function name, or inline expression (expected to be removed in a future release), and `limits` is a two-element vector specifying the x-axis interval. For example,

```
>> fplot('sin(x)*cos(x)', [0,pi])
```

plots the mathematical expression $\sin(x)\cos(x)$ within the $[0; \pi]$ interval.

Another example is the MATLAB `fzero` function, which finds a zero of a single-variable function. Its syntax is

```
y = fzero(fun,x0)
```

If `x0` is a scalar, it represents the user-supplied initial guess of a zero (typically the `fzero` function finds the closest zero to the initial guess). If `x0` is a two-element vector, it serves as the interval [where the sign of the value of the function at `x0(1)` differs from the value of the function at `x0(2)`]. For example,

```
>> z = fzero("cos",0)
```

finds a zero of the cosine function closest to `x`=`0`, whereas

```
>> z = fzero(@(x)cos(x)^2-0.5,[2 3])
```

returns the zero of $\cos^2(x) - 0.5$ found within the range $x = [2;3]$. (Note that, it is always a good idea to plot the function first using `fplot` to ensure the function values at the interval endpoints differ in sign.)

The MATLAB `fminbnd` function finds a minimum of a single-variable function

$$x_* = \arg\left(\min_{x\in[x_1;x_2]} f(x)\right) \tag{5.2}$$

Its syntax is

```
[x,y]=fminbnd(fun,x1,x2)
```

returning x_* in `x`, and $f(x_*)$ in `y`. Values `x1` and `x2` are the lower and upper limits of the interval in which to search for the minimum. (Because there is no `fmaxbnd` function to find the maximum of a function, you would have to redefine its output to be the negative of the desired function and use the `fminbnd` function.)

Finally, the MATLAB `fminsearch` function used in Sec. 5.8.1 finds the minimum of a function of more than one variable [Eq. (5.1)]. Its general syntax is

```
[x,fval]=fminsearch(multi_var_fun,x0)
```

with x0 representing the user-supplied input vector containing the initial guesses for all variables. This function returns x, a vector with the values of variables minimizing the multi_var_fun function, and fval, a scalar that is the minimum value of the function.

The theory (algorithms) behind the fzero, fminbnd, and fminsearch functions will be considered in Chapter 10, but here we would like to make several important points. First, each function has its own workspace where variables used by the function are stored. Generally speaking, you cannot access this workspace from outside and vice versa. Hence, the list of input and output arguments is the only way of exchanging data between the workspaces (with a few exceptions presented in the Secs. 5.8.4 and 5.8.5).

Second, the professionally written functions allow some flexibility in defining input arguments. This was demonstrated earlier in this section by the fplot and fzero functions accepting a different type of input as a fun argument (to be discussed further in Sec. 5.9.4). Other input arguments may assume a single scalar value or vector, so that somewhere inside the function there is a logic distinguishing between different cases based on the type of the input argument and its dimension.

Third, the *m*-file function may have different syntaxes accepting a different number of arguments in the input. In fact, all of the aforementioned functions accept at least one extra argument defining some internal parameters used by these functions. The fact that these functions work fine even without this extra input argument means that these internal parameters assume some default values inside the functions and use them in the case when the expected input argument is not present, rather than returning an error saying that there are not enough input arguments. The same applies to the number of output arguments—the very same function may return a different number of arguments, depending on how we call it.

The aforementioned features distinguish the professionally written function from the beginner-written functions. The former are flexible and at the same time foolproof. We will use the open-source paradigm of MATLAB to look inside some of these functions in Sec. 5.9.1, but now let us proceed with writing your own, very simple function.

Although there is some flexibility, it is a good habit to have the name of a user-defined function *m*-file coincide with the name of the function itself, as established in its header (with the *.m* extension added). To this end, assume that we have the following code saved as the *stat.m* file in the current directory (or any directory on the search path, as addressed in Fig. 2.22)

```
function [mean,stdev] = stat(x)
% This function computes the mean value and standard deviation of a vector x
n = length(x);
mean = sum(x)/n;
stdev = sqrt(sum((x-mean).^2/n));
```

(Note that, there is no need to end the body of the function with any ending statement like `end` or `return`.) This function is supposed to accept a single argument, a numeric vector, and calculate its mean value and standard deviation. You may address this function in the way you would address any MATLAB function. For instance, two instructions

```
>> x = 0:100;
>> [mx,sx] = stat(x)
```

return

```
mx = 50
sx = 29.1548
```

You can also call it asking for just a single output argument

```
>> a = stat(0:10)
```

which returns only the mean value

```
a = 5
```

Using this simple example of a complete *m*-file function, let us formally introduce the function definition line, header

`function [`*output variables*`]=function_name(`*input variables*`)`

This header defines everything, namely

`function`	Indicates that this is an *m*-file function as opposed to an *m*-file script
[*output variables*]	The list of (optional) output arguments, which may contain
A single argument	In this case, you may omit square brackets
Multiple arguments	The list of comma-separated variables (must be enclosed in square brackets)
No output	Neither the brackets nor assignment operator is required
`function_name`	Constitutes a valid function name (which begins with an alphabetic character and can contain letters, numbers, or underscores)
(*input variables*)	Refers to the (optional) list of comma-separated variables that are needed by the function to work properly (there maybe no input arguments in which case the parentheses are not required)

With respect to the help comment lines, they may be placed anywhere within the function, but any comments placed immediately after the function header will be displayed when the `help` command is used. For example, in our case

```
>> help stat
```

returns

```
This function computes the mean value and standard deviation of a vector x
```

Sometimes your algorithm contains a portion that can be organized as a function, but to keep it short you prefer breaking it into several subfunctions, some of which may be very short and only used by another function, not by the

main code. In this case, saving these pieces as separate *m*-files would create a mess in your current directory. The question is whether it is possible to save several functions in the same *m*-file. The following section addresses that.

5.8.3 The Main and Local Functions

Each *m*-file may contain code for more than one function. The function that appears first in the file is referred to as a *main* or *primary function*. Any function that follows the main function in the same *m*-file is referred to as a *local function* or *subfunction*. For instance, if the *m*-file *newst.m* contains the following:

```
function [av,sc] = newst(u)    % the main function
% NEWST finds the mean value of the input vector and also scales it from 0 to 1
av = mean(u);
sc = scale(u);

function a = mean(v)           % a local function
% Calculate average
a = sum(v)/length(v);

function m = scale(v)          % a local function
% Scale from 0 to 1
m = (v-min(v))/(max(v)-min(v));
```

then the function `newst` represents a *main function*, whereas `mean` and `scale` are two local functions (Fig. 5.9). Note that there is no need to separate the functions by any statement because the command `function` itself serves as an indicator that a new function has started and, therefore, the previous one has ended.

The main functions have a wider scope than the local functions. That is, the main functions can be invoked from outside of their *m*-files (from the MATLAB command line or from functions in other *m*-files), whereas the local functions cannot be invoked from outside. By design, the local functions are visible only to the main function and other local functions within their

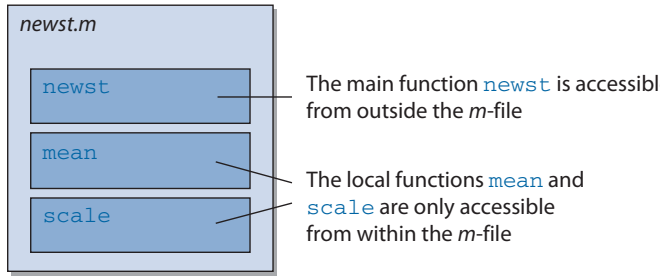


Fig. 5.9 Example of a main function followed by two local functions.

own *m*-files. When you call a function from within an *m*-file, MATLAB first checks the file to see if this function is a local function. Because of that, you can override existing *m*-files using the local functions with the same name as other MATLAB functions (as is done for the `mean` subfunction in Fig. 5.9).

If for any reason you want to organize several functions that you want to have an external access to within one *m*-file, you should use a specially organized main function returning a handle structure as in the following example:

```
function fh = ellipseVals
fh.focus = @eFocus;
fh.area = @eArea;
  function f = eFocus(a,b)
  f = sqrt(a^2-b^2);
function ae = eArea(a,b)
% eArea computes ellipse area
ae = pi*a*b;
```

Running this code

```
>> h = ellipseVals
```

results in a two-field handle structure

```
 h =
  struct with fields:
            focus: @eFocus
      eccentricity: @eArea
```

that you can use. For example,

```
>> h.area(3,1)
```

yields

```
ans = 9.4248
```

You can write the help comments for the local functions using the same rules that apply to the main functions. To display the help section for a local function, precede its name with the name of the *m*-file that contains it (minus file extension) and a greater than symbol, >. For example, to get help on the `eArea` local function, you would need to type

```
help ellipseVals>eArea
```

As of R2016b, the local functions are also supported in the scripts (they should appear at the end of a script). For example, you can save the call to the `stat` function (from the previous section) and the function itself in the *allinone.m* file:

```
a = stat([1,2,3,4])
   function [mean,stdev] = stat(x)
   % This function computes the mean value and
   % standard deviation of a vector x
```

```
    n = length(x);
    mean = sum(x)/n;
    stdev = sqrt(sum((x-mean).^2/n));
```

and run it by issuing the command

```
>> allinone
```

(another example will be given at the end of Sec. 10.9.1). Note that passing variables between the functions in the *newst.m* file (between their own workspaces), which this section started with, was accomplished using the lists of input and output arguments. Next, we show an alternative way of doing it.

5.8.4 Nested Functions

The latest versions of MATLAB allow you to define one or more functions within another function as opposed to sequentially. Such an inner function is said to be the *nested function* within the function that contains it. You can also nest functions within other nested functions (like the Russian nesting doll, Matryoshka). To write a nested function, simply define one function within the body of another function in an *m*-file, for example,

```
function x = A(p1, p2)
...
  function y = B(p3)
  ...
  end
...
end
```

Note that, while the main function and local functions do not normally require a terminating `end` statement, this rule does not hold when you nest functions (Fig. 5.10). If an *m*-file contains one or more nested functions, you *must* terminate all functions (including the local functions) in the *m*-file with `end`, whether they contain the nested functions or not.

Like other functions, a nested function has its own workspace where variables used by this function are stored. However, as opposed to the case when the functions are stored in one *m*-file as the main and local functions, the nested functions have access to the workspaces of all functions in which they are nested. For example, a variable that has a value assigned to it by the main function can be read or overwritten by a function nested at any level within the primary one. Similarly, a variable that is assigned in a nested function can be read or overwritten by any of the parent functions containing that function.

Consider a fragment of *m*-file in Fig. 5.10 and its graphical interpretation in Fig. 5.11 (which also contains a local function `F`).

```
function A(x,y)          % Main function
B(x,y);
D(y);
    function B(a,b)      % Function nested in A
    C(a);
    D(b);
        function C(k)    % Function nested in B
        D(k);
        end
    end
  ...
    function D(m)        % Function nested in A
    E(m);
        function E(p)    % Function nested in D
         ...
        end
    end
end
```

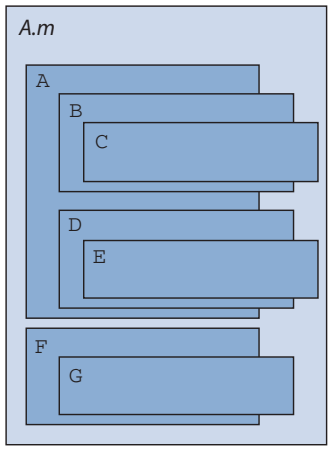
Fig. 5.10 Example of the nested functions.

In this example, you can call a nested function

1. From the level immediately above it (function A can call B or D, but not C or E)
2. From a function nested at the same level within the same parent function (function B can call D, and D can call B)
3. From a function at any lower level (function C can call B or D, but not E)

You can also call a local function from any nested function in the same *m*-file, e.g. C can call F, but not (Fig. 5.11).

The following section continues the issue of "sharing" workspaces.



Function A can call the local function F and nested functions B and D. It cannot, however, call the next-level nested functions C and E, as well as the nested function G

Function B can call the parent function A, same-level nested function D, nested function C, and local function F

Function C can call the parent function B, next-level host function A, and local function F

Variables defined in the parent function A are visible inside the nested functions B, C, D, and E

Fig. 5.11 Graphical illustration of the nested functions.

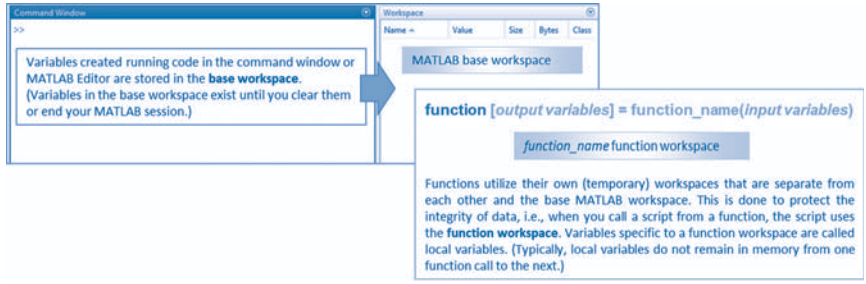


Fig. 5.12 The base and local workspaces.

5.8.5 Sharing Data between the Base and Function Workspaces

As you already know, MATLAB stores variables in a part of memory called a workspace. The *base workspace* holds variables created during your interactive MATLAB session, including those created as a result of running the *m*-file scripts. Hence, given that they belong to the same base workspace, these variables can also be used by other scripts.

Functions, however, do not use the base workspace. Every function has its own *function workspace* (Fig. 5.12). Even local functions that are defined in the same *m*-file have their separate function workspaces, that is, all variables within the body of the function are *local variables*, meaning that their values are only available within each function itself (Sec. 5.8.3). In other words, each function workspace is kept separate from the base workspace and all other workspaces to protect the integrity of the data used by that function, which means that

1. If a variable by the same name as one of your workspace variables is used by a function, any change made to that variable by the function is not passed back to the current workspace unless it is an output variable
2. All local variables are erased after the function finishes executing
3. When calling the function, you do not have to use the same name for input variables as those the function header calls the input variables. This makes functions "portable" to other applications, so that they do not need to be rewritten for fear of using the same variable names in other calculations

We have discussed several techniques for sharing variables between base and local workspaces already, but let us formally list all possible options here. You can

1. Pass arguments via the lists of input and output variables defined by the function header (Secs. 5.8.2 – 5.8.3)
2. Use the nested functions (Sec. 5.8.4)

3. Use the external file(s), for example, binary files, ASCII files, and Excel files. (e.g., you can use the `save` and `load` functions inside the user-defined functions, i.e., organizing data exchange via a binary file; Sec. 4.8)
4. Declare variables to be `global`
5. Explicitly evaluate in another workspace (the `evalin` and `assignin` functions)
6. Store in / retrieve data from a graphics object using the `setappdata` and `getappdata` functions
7. Employ `persistent` variables

With the first two options covered already, let us talk about the remaining ones. To minimize the number of variables needed to be passed through the list of input variables, you may want to declare certain variables to be global (Option 4). *Global variables* form their own workspace, which is separate from the base and local workspaces and visible from inside/outside a function. This is done by using the `global` function in the main *m*-script and within every function that needs access to the global workspace. The syntax for declaring variables global is

```
global var1 var2 var3
```

For example, defining variables `b` and `d` to be global in the main script

```
global b d
b=8;
a=product(20);
d;
```

as well as in the function

```
function a = product(c)
global b d
a=b*c;
d=b-c;
```

allows them to be shared outside of the lists of input and output arguments. (Any value changes to these variables, in any function, are visible to all the

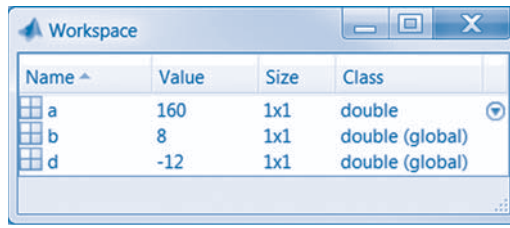


Fig. 5.13 Designation of global variables in the base workspace.

```
function TryMe
Trial                   % call the local function
B*2                     % B was created by Trial in the TryMe workspace
G=getappdata(0,'C')     % G was set using the setappdata function in Trial
load TR                 % load Tr.mat created by Trial in the current folder
h                       % h is a variable that was stored in TR.mat

function Trial
h=reshape(magic(4),2,8);  % create an array h
assignin('base','A',h);   % make it available in the base workspace
assignin('caller','B',h); % make it available in the calling function
setappdata(0,'C',h);      % make it available using the gettappdata function
save TR;                  % save the function workspace to TR.mat

ans =
    32  18   4  14   6  12  26  24
    10   8  22  28 20  30  16   2
G =
    16   9   2   7   3   6  13  12
     5   4  11  14 10  15   8   1
h =
    16   9   2   7   3   6  13  12
     5   4  11  14 10  15   8   1
```

Fig. 5.14 Illustration of the different options for sharing data between the function workspaces.

functions that declare them as global.) The base workspace after executing the aforementioned code looks like that shown in Fig. 5.13. To simplify handling global variables in MATLAB Editor, starting with R2011a, they are highlighted with a different color, thus warning you that changing their values may affect other related code execution.

While global variables seem to represent a good way to extend the variable scope, there is a pitfall. If you do declare some variables global, you need to make sure that no function with access to the variable overwrites its value unintentionally. Also, using global variables may restrict the universality of your functions because you need to explain which variables have to be global and which should be passed through the list of input arguments. Although beginners might enjoy using the global variables (which also saves the computational time), professionally developed programs utilize sharing data via the lists of input and output arguments. To make this list shorter, they employ data structures (which are capable of storing heterogeneous data in one variable).

Next, Fig. 5.14 shows a published printout of running the *TryMe.m* file that contains the main function and one local function. It provides you with examples on how to share data between two functions using some

other options from the aforementioned list (Options 3, 5, and 6). The only variable created in the base workspace when running this file is a 2-by-8 matrix `A` (created by the local function `Trial`).

The last option in the list (Option 7) is declaring some function variables as *persistent variables* to retain their values from one function call to the next, which eliminates the necessity of passing this variable out and in every time you call the function (because only the base workspace remains active between the function calls).

Suppose the *myFun.m* file contains the following:

```
function myFun()
persistent n
    if isempty(n)
        n = 0;
    end
n = n+1;
disp(n)
```

Given that the persistent variable `n` does not exist the first time you call this function (`n=[]`), you need to initialize it (that is what the `if` construct, setting `n=0`, is used for). Calling `myFun` several times returns

```
>> myFun
     1
>> myFun
     2
>> myFun
     3
```

Persistent variables are similar to global variables because MATLAB creates permanent storage for both (they are even colored similarly in the Editor). They differ from global variables in that persistent variables are known only to the function in which they are declared. This prevents persistent variables from being changed by other functions or from the MATLAB command line.

Whenever you clear or modify a function that is in memory, MATLAB also clears all persistent variables declared by that function, that is,

```
>> clear myFun
>> myFun
```

yields

```
  1
```

You can keep a function in memory until MATLAB quits, by using the `mlock` function. For example, modifying the first three lines of *myFun.m* to

```
function myFun()
mlock
persistent n
```

results in

```
>> myFun
     1
>> myFun
     2
>> clear myFun
>> myFun
     3
```

To unlock persistent variables, use the `unlock` function

```
>> unlock('myFun')
>> myFun
     1
```

Note, it would be an error to declare a variable persistent if a variable with the same name exists in the current workspace or to declare any of a function's input or output arguments persistent within that same function. For example, the following persistent declaration is invalid:

```
function [a,b]=myFun1(x,y,z)
persistent a y
```

5.8.6 Search Order and Private Functions

As mentioned throughout the text several times already, intentionally or unintentionally you may end up having variables in your base workspace that have the same name as your *m*-file function, or your created *m*-file function has the name of one of the MATLAB functions. The question is how MATLAB manages to distinguish between them, if at all. The following explains it.

Suppose you typed the following in the Command Window:

```
>> test
```

In this case, the search order is as follows. First, MATLAB checks to see if `test` is a variable in your base workspace, and if so, its value is displayed in the Command Window. If there is no such variable, MATLAB looks in the current directory for a file named *test.m* and executes it if it is found. If not, MATLAB checks to see if `test` is one of its own built-in commands or functions and executes it if it is. If not, MATLAB then searches the directories in its search path in the defined order and if the file *test.m* is found, it executes it. Finally, if nothing works out, the message

```
Undefined function or variable 'test'.
```

appears in the Command Window.

The `exist` function may help you sort everything out even without browsing the content of the current workspace and directory. The call

```
exist name
```

Table 5.7 Order of Search and Values Returned by the `exist` Function

Order of Search	Return Value	Type of Entity
1	1	`name` is a variable in the workspace
2	5	`name` is a built-in MATLAB function
3	7	`name` is a directory
4	3	`name` is a *mex*- or *dll*-file on your MATLAB search path (Sec. 4.6)
5	4	`name` is an *mdl*-file on your MATLAB search path
6	6	`name` is a *p*-file on your MATLAB search path
7	2	`name` is an *m*-file on your MATLAB search path. It also returns 2 when `name` is the full pathname to a file or the name of an ordinary file on your MATLAB search path
8	8	`name` is a Java class (`exist` returns `0` if you start MATLAB with the `-nojvm` option)
	0	`name` does not exist

checks for the existence of variable, function, or directory *name* and returns one of the values shown in Table 5.7. It can also be used as

```
exist name kind
```

that is, specifying the `kind` of the entity: `builtin` (for built-in functions), `class` (for Java classes), `dir` (for directories), `file` (for files or directories), and `var` (for variables). You can also use a function syntax, `exist('name','kind')`. An argument `name` may include an extension [that is, `exist('file.ext')`] to preclude conflicting with other similar filenames or a partial pathname [`exist('matlab/general/dir.m')`].

For example,

```
>> exist('pi')
```

returns

```
ans = 5
```

because `pi` is a built-in function, but

```
>> exist('pi.m')
```

returns

```
ans = 2
```

which is farther down in the order of the search because the *m*-file *pi.m* also exists (Sec. 2.2.5). The call

```
>> pi=5; exist pi
```

returns

```
ans = 1
```

because you overrode whatever exists on the path with the variable in the current workspace.

One more place to look at can be defined using the *private functions*. Private functions are functions that reside in subdirectories with the special name "*private.*" These functions are called private because they are visible only to the *m*-file functions and *m*-file scripts that meet the following two rules:

1. A function that calls a private function must be defined in an *m*-file that resides in the directory immediately above that private subdirectory.
2. A script that calls a private function must itself be called from an *m*-file function that has access to the private function according to rule number one.

For example, assume that you added your directory "mydir" to the search path, so that whatever is inside this directory is visible to the MATLAB calls and functions. Nevertheless, if inside this directory you create a directory with the name *private*, the functions that are in this directory can only be seen (called) from the "mydir" directory.

Because the private functions are invisible outside the parent directory, they can use the same names as functions in other directories. This is useful if you want to create your own version of a particular function while retaining the original one in another directory.

To conclude, let us slightly correct the search sequence introduced in the beginning of this section. Before looking for the standard *m*-file functions, MATLAB checks the private functions (if any).

5.9 Developing Professional-Looking Functions

This section shows that progressing from a beginner programmer to an intermediate-level or professional programmer of MATLAB may not take too much of an effort compared with other languages. Primarily, this is due to the open-source paradigm allowing you to look at and understand the professionally developed code and employ useful code constructs in your own functions. The latest addition to the Live Editor, the Task apps (Fig. 2.13a), serves this purpose as well.

5.9.1 Open-Source Paradigm

As mentioned already, the distinctive feature of MATLAB is that it supports an open-source paradigm. Not only does it allow users to share their source code, for example, via MATLAB Central (Fig. 1.4), but it also enables you to learn about numerical algorithms realized in the MATLAB functions and to teach yourself a trick or two, used by Mathworks' code developers, by just looking at the source code.

For example, a portion of the `ode23tb` *m*-file function shown in Fig. 5.15 specifies some parameters of the algorithm described later in Sec. 14.6.2.

```
% Initialize method parameters
pow = 1/3;
alpha = 2 - sqrt(2);
d = alpha/2;
gg = sqrt(2)/4;
% Coefficients of the error estimate
c1 = (alpha - 1)/3;
c2 = 1/3;
c3 = -alpha/3;
% Coefficients for the predictors
p31 = 1.5 + sqrt(2);
p32 = 2.5 + 2*sqrt(2);
p33 = - (6 + 4.5*sqrt(2));
```

Fig. 5.15 Excerpt from the *ode23tb.m* file.

To locate a source code for any MATLAB function, use the `which` function (see Table 2.12). For example, the command

```
>> which fzero
```

returns a location of the `fzero` function

```
C:\...\MATLAB\R2020a\toolbox\matlab\funfun\fzero.m
```

By the way, many of the MATLAB functions are *overloaded*, meaning that they have multiple implementation to handle different data types of input arguments appropriately. When you use the `which` command with the `-all` option, MATLAB returns all occurrences of the file you are looking for. For example, the command

```
>> which -all sin
```

returns something like (depending on the installed toolboxes)

```
built-in (..\toolbox\matlab\elfun\@double\sin)   % double method
built-in (..\toolbox\matlab\elfun\@single\sin)   % single method
..\toolbox\symbolic\@sym\sin.m                   % sym method
..\toolbox\distcomp\gpu\@gpuArray\sin.m          % gpuArray method
```

To understand the key elements of professional-looking code, let us open the `fzero` *m*-file function by using the following call:

```
>> open(which('fzero'))
```

Several excerpts from the `fzero` function are presented in Fig. 5.16. We will use these excerpts in the following two sections to learn about a couple of useful code constructs.

```
function [b,fval,exitflag,output] = fzero(FunFcnIn,x,options,varargin)
%FZERO  Single-variable nonlinear zero finding.
%  X = FZERO(FUN,X0) tries to find a zero of the function FUN near X0,
%  if X0 is a scalar.
%  ......
%  X = FZERO(PROBLEM) finds the zero of a function defined in PROBLEM.
%  PROBLEM is a structure with the function FUN in PROBLEM.objective,
%  the start point in PROBLEM.x0, the options structure in PROBLEM.options,
%  and solver name 'fzero' in PROBLEM.solver.
      .........
% Detect problem structure input
if nargin == 1
    if isa(FunFcnIn,'struct')
        [FunFcnIn,x,options] = separateOptimStruct(FunFcnIn);
    else % Single input and non-structure.
        error('MATLAB:fzero:InputArg',...
            getString(message ('MATLAB:optimfun:fzero:InputArg')));
    end
end
      .........
% Check for non-double inputs
if ~isa(x,'double')
 error('MATLAB:fzero:NonDoubleInpu  ,...
    getString (message ('MATLAB:optimfun:fzero:NonDoubleInput')));
end
      .........
% Convert to function handle as needed
[FunFcn,errStruct] = fcnchk(FunFcnIn,length(varargin));
if ~isempty(errStruct)
    error(message(errStruct.identifier));
end
% We know fcnchk succeeded if we got to here
if isa(FunFcn,'inline')
    if isa(FunFcnln,'inline')
        Ffcnstr = inputname(1); % name of inline object (e.g. f=inline('x*2'))
        if isempty(Ffcnstr)  % inline('sin(x)')
            Ffcnstr = formula(FunFcn); % grab formula, no argument name
        end
        Ftype = 'inline object';
    else % not an inline originally (character array expression)
        Ffcnstr = char(FunFcnIn); % get the character array expression
        Ftype = 'expression';
    end
elseif isa(FunFcn,'function_handle') % function handle
    Ffcnstr = func2str(FunFcn); % get the name passed in
    Ftype = 'function_handle';
else  % not converted, must be m-file or built-in
    Ffcnstr = char(FunFcnIn);  % get the name passed in
    Ftype = 'function';
end
       .........
```

Fig. 5.16 Excerpts from the *fzero.m* file.

5.9.2 The Code Designation Comments and Alarming Behavior Feedback

The first thing to note when looking at any MATLAB function source code, including that presented in Fig. 5.16, is that developers do a very good job providing an extensive comments section explaining code designation and possible syntaxes. Again, the lines immediately following the function header in the *fun_name.m* file constitute the `help fun_name` call response.

Also, a well-written MATLAB code has many foolproof checks preventing it from malfunctioning if wrong/not enough arguments are passed on. In some cases, missing arguments can be/are replaced with some default values, and in other cases, the function produces an error or warning message (using the `error` and `warning` functions, respectively).

Three error messages shown in Fig. 5.16 display descriptive messages telling exactly what went wrong using an `error(msgid,errmsg)` syntax, where `msgid` is an optional message identifier and `errmsg` is a character vector (the built-in `message` function utilizes the MathWorks message catalogs to display an appropriate message).

You are welcome to use the `error` and `warning` functions to display the corresponding messages in your code as well. For example,

```
>> error('Not enough arguments')
>> warning('Algorithm converges too slow')
```

produce the corresponding color messages (`error` also aborts code execution). By the way, assuming the value of `2` for the `fid` input argument of the `fprintf` function introduced in Sec. 4.9, also produces a red message, but without breaking code.

5.9.3 Enabling Variable-Length List of Input and Output Arguments

Continuing exploring code shown in Fig. 5.16 and looking at the first few lines following the comment block, it becomes clear that although the header shows four input arguments, the `fzero` function may work fine with a shorter argument list as well. This option is enabled by the `nargin` function, returning the number of arguments that were actually passed in. In this particular case, `fzero` accepts a single input argument (`nargin==1`) assuming it is a three-field structure (the `isa` function checks whether the object is of a given class). If this is the case, the `separateOptimStruct` function breaks this structure into separate elements, one of which, `x`, represents the vector if initial guesses, which may alternatively be passed directly as a second input argument to the `fzero` function. This is followed by checking whether each input belongs to the class it is supposed to belong to (in our case, all elements of the vector `x` should be the double-precision values).

In general, the `nargin` function allows your code to be flexible, so that it could be used with a different number of input parameters without returning an error message. Based on the return of the `nargin` function you either execute a portion of code that does not rely on parameters that were not provided or use some default values for these missing parameters (defined in the beginning of your function).

Consider the function `testarg`, which may accept one or two input arguments featuring different computations in each case,

```
function [maxc] = testarg(z,a)
  if (nargin == 1)
  c = z.^2;
  elseif (nargin == 2)
  c = z.^a;
  end
plot(z,c)
  if nargout == 1
  maxc = max(c);
  end
```

As seen, the `if-else` construct takes care of two possible situations and uses only those input arguments that are available. This code also features another, reciprocal function, `nargout`. Similar to `nargin`, `nargout` returns the number of the function's output arguments specified in the call. In this specific case, the `testarg` function can be called with no output argument, producing a plot $y = x^2$ or $y = x^a$ (depending on the number of input arguments), or one output argument, which produces the same plot but also returns a corresponding value of $\max(x^2)$ or $\max(x^a)$.

If being called outside the function, just at the command line, like `nargin(fun)` and `nargout(fun)`, these functions return the number of declared inputs and outputs for the *m*-file function `fun`, respectively.

Another relevant function, `return` (not shown in Fig. 5.16), causes a normal return to the invoking function or to the `keyboard`. It also terminates the `keyboard` mode (discussed later, in Sec. 5.10.1). For instance, suppose you want to develop a function that computes a determinant of square matrix `A`. For some reason, if matrix `A` happens to be empty, we want the output of our function to assume a value of 1 and quit executing the remaining code. The following piece of code shows an example of how to accomplish this using a `return` statement:

```
function d = det(A)
%DET det(A) is the determinant of A.
if isempty(A)
    d = 1;
    return
else
 ...
end
```

Consider another typical example combining the output of the `nargin` function, `isempty` function, and `return` command. The MATLAB function `median` allows two syntaxes: `median(A)` and `median(A,dim)` (Table 3.13). What you may find after the header of this function and the Help comments

```
function y = median(x,dim)
%MEDIAN Median value.
%...
```

is the following block of commands:

```
if nargin == 1
   dim = min(find(size(x)~=1));
   if isempty(dim), dim = 1; end
end
if isempty(x), y = []; return, end
```

If only one variable is supplied (no `dim` defined), these commands determine the default value for `dim` by itself. If this function detects an empty input array, it immediately returns to the calling program with an empty output array `y`.

Finally, looking at the function header in Fig. 5.16, you may notice the last input argument after three explicitly declared inputs, `varargin`. When a function executes, `varargin` becomes a 1-by-*n* cell array, where *n* is the number of inputs that the function receives after the explicitly declared inputs (if the function receives no inputs after the explicitly declared inputs, then `varargin` is an empty cell array).

Many MATLAB functions use the most concise syntax, utilizing implicitly declared inputs only. For instance, if you open the *fplot.m* file, you will see that its header defines nether the list of input or output arguments explicitly at all:

```
function varargout = fplot(varargin)
```

Here `varargout` is an output variable enabling the function to return any number of output arguments, similar to `varargin`. When function code executes, its output is combined into the 1-by-*m* cell array `varargout`, where *m* is the number of outputs requested after the explicitly declared outputs.

To better understand how it works, let us consider the following function saved as a *VarArgTest.m* file:

```
function varargout=VarArgTest(varargin)
% Analyze a function definition statement

disp(['Number of input arguments:  ' int2str(nargin)])
disp("Number of output arguments: " + nargout)
varargin

% Use the varargin list
```

```
    hObj=fplot(varargin{:},'r-.o');
% Use the varargout list

if nargout==1
  varargout={hObj(:)};
elseif nargout==2
  varargout={hObj.XData, hObj.YData};
end
```

Basically, it is a so-called *function wrapper* for the MATLAB `fplot` function, producing the same output but changing the default line's style, color, and marker (that is what `'r-.o'` triplet does—see Sec. 6.3). If you run this function using the call

```
>> VarArgPlay(@sin,[-3,5])
```

you get

```
Number of input arguments:  2
Number of output arguments: 0
varargin = 1×2 cell array
                {@sin}   {1×2 double}
```

and a sinusoid plotted within the [−3;5] interval. Two input arguments accepted as the variable-length `varargin` list are then passed on to the `fplot` function. [By the way, note an alternative (to what was shown at the end of Secs. 5.5.1 and 5.5.2) way to concatenate variables in the `disp` function using the string scalar and `+`.]

Even though no outputs were requested, all the plot data are saved in the `hObj` variable, representing a graphics handle (to be discussed in Sec. 6.4). By construction, the `VarArgTest` function also allows you to have one or two output variables, so that the `h=VarArgPlay (@sin,[-3,5]);` call returns the graphics handle (making it possible to change any parameter of the plot later on programmatically) and the `[x,y]=VarArgPlay (@sin,[-3,5]);` call returns the sinusoid's x and y data.

5.9.4 Passing a Function as an Argument

Studying the last code excerpt presented in Fig. 5.16 reveals why the `fzero` function accepts multiple classes for the first input argument (defining a mathematical function to find a zero of). Specifically, it is the `fcnchk` function that makes it possible (the expectation is that it will be replaced with another function in a future release). This function returns an inline object if the input is a character vector or string scalar containing parentheses, variables, and math operators. If the input argument is a function handle, `fcnchk` simply returns it as is; if it is a function name (e.g., `'sin'`)—`fcnchk` returns a function handle to that function.

Let us formalize what input the MATLAB functions can possibly accept as a function input in general. In the function call

```
function(funIn,p1,p2,...)
```

where `function` is the name of an appropriate MATLAB function (`fzero`, `fminsearch`, etc.) and `p1`, `p2`, ... are corresponding parameters (if at all), the argument `funIn` can be one of the following (as of R2020a):

1. The name of an *m*-file function (as a character vector or string scalar), for example,

```
>> z=fzero('cos',0)
```

2. A function handle to an *m*-file function, for example,

```
>> fplot(@tan)
```

3. A function handle to an anonymous function, for example,

```
>> fun = @(x,c) 1./(x.^3-2*x-c);
>> q = integral(@(x)fun(x,5),0,2)
```

4. A character vector or string scalar with variable *x*, for example,

```
>> ezplot('sin(x)*x^2', [0,pi])
```

5. An inline function defined beforehand, for example,

```
>> myfun = inline('(x(1)-3)^2+(x(2)+2)^2');
>> x = fminsearch(myfun,[1,1])
```

6. A symbolic expression (to be discussed in Ch. 7), for example,

```
>> syms y(x); G=dsolve(diff(y)==y+1);
>> fmplicit(G)
```

Not all MATLAB functions necessarily accept all of the aforementioned options. It all depends on how the input argument defining the function is processed inside the calling function. In this regard, as of R2020a the functions belonging to the Optimization Toolbox (`fzero`, `fminsearch`, etc.) seem to be the most universal, and the reason for that is that they utilize the `fcnchk` as presented in Fig. 5.16. In a future release, Options 4 and 5 are expected to be eliminated, leaving Options 1 – 3, 6 as the only alternatives.

If you want your code to accept all Options, 1 – 5, you may use the `fcnchk` in it, following the example shown in Fig. 5.16. The simplest syntax is `fun=fcnchk(fun)`. Another syntax, `fun=fcnchk(fun, 'vectorized')` allows you to process a character vector or string scalar representing a mathematical formula to produce a vectorized inline function `fun`, inserting a `.` before any `^`, `*`, or `/` (similar to the `vectorize` function introduced in Sec. 5.8.1).

After `fun` is preprocessed with `fcnchk`, it can be evaluated using another useful function, `feval`. Formally, the command

```
feval(fun,x1,...,xn)
```

evaluates `fun` with the given set of arguments, `x1`, ... , `xn`.

The following example demonstrates all approaches you may use to pass the function/expression along with other parameters into another function. Suppose the actual function you are using to compute some expression is

```
function y = PassFun2(x,z)
global  a  b
y = a * sin(x) + b * sqrt(x) * cos(z);
```

(Note that some of the parameters are passed as global variables, whereas others are passed through the list of input arguments.) Then, you use another function, `PassFun1`, to pass some function and two arguments through:

```
function x = PassFun1(fun,h,k)
fun = fcnchk(fun);
x = feval(fun, h, k);
```

(that is, where you use `fcnchk` and `feval` functions).

Finally, your main program may address your function `PassFun1`, passing its first argument in three ways, using the handle to the *m*-file function `PassFun2`, the name of the *m*-file function `PassFun2` itself, and the mathematical expression invoking the *m*-file function `PassFun2`):

```
global a b
a = 1; b = -2;
g = PassFun1(@PassFun2,1,4)*PassFun1('PassFun2',5,7)+...
   PassFun1('3+sin(d)*PassFun2(d,r)',1,4)
```

(Any other function using two input variables, could be used here instead of function `PassFun2`.) The preceding code produces a valid result:

```
g = -4.4970
```

5.9.5 Function Argument Validation

As you observed in code presented in Fig. 5.16, any MATLAB function starts from multiple tests to validate class, size, and other aspects of the function input values. In the case of violating a constraint, the function would return a warning or error message (Sec. 5.9.2).

In R2020a the function argument validation construct was introduced to perform these tests without writing a cumbersome input-argument-checking code, which improves readability, robustness, and maintainability of your code.

This optional construct is a way to declare specific restrictions on function input arguments, and as such, should appear before the first executable line of the function. Function argument validation constructs are delineated by the keywords `arguments` and `end`

```
function myFun(inputArg)
   arguments
   inputArg (dim1,dim2,...) ClassName   {fcn1,fcn2,...}   =   default value
                      Size          Class        Validation functions          Default value
   end
% function code
...
end
```

As presented, the function argument declaration may include any of the following four elements:

- Size – the length of each dimension enclosed in parentheses (e.g., `(2,2)` or `(:,5)`)
- Class – the name of a single MATLAB class (e.g., `char`, `double`, `cell`, `table`)
- Validation function(s) `fcn1,fcn2,...` – a comma-separated list of validation functions, enclosed in braces (during the validation process, MATLAB passes the argument value to each validation function listed for that argument and throws the first error encountered)
- Default value for the input argument (obviously, the default value must satisfy the declared restrictions for that argument).

Consider a self-explanatory example employing some restrictions on three input arguments of a function:

```
function out = myFunction(A, B, method)
   arguments
   A (1,1) string
   B {mustBeNumeric,mustBeReal}
   method (1,:) char {mustBeMember(method,{'linear','spline'})} = 'linear'
   end
% Function code
...
end
```

This example uses three of the eighteen validation functions introduced in R2017a:

`mustBeNumeric`	`mustBeNonempty`	`mustBeNegative`
`mustBeInteger`	`mustBeNonsparse`	`mustBeNonnegative`
`mustBeReal`	`mustBeMember`	`mustBeGreaterThan`
`mustBeNumericOrLogical`	`mustBeNonzero`	`mustBeGreaterThanOrEqual`
`mustBeFinite`	`mustBePositive`	`mustBeLessThan`
`mustBeNonNan`	`mustBeNonpositive`	`mustBeLessThanOrEqual`

These functions accept the potential property value as an input argument, employ the functions of Table 5.3, and throw corresponding errors if the validation fails (no return otherwise).

Let us conclude this section with one more simple example. Suppose your function is

```
function y = MyFun(A)
   arguments
      A (2,2) double {mustBeNonnegative} = ones(2)
   end
y=sqrt(A);
```

Then, the call

```
>> MyFun(rand(3))
```

results in

```
Invalid input argument at position 1. Value must be 2×2 matrix.
```

the call

```
>> MyFun(-rand(2))
```

yields

```
Invalid input argument at position 1. Value must be nonnegative.
```

and the no-argument call

```
>> MyFun
```

returns

```
ans = 1   1
      1   1
```

5.10 Editing and Debugging

When you write a piece of code and attempt to run it, most likely you will encounter some errors that need to be fixed. Hence, you will spend some time correcting your code. MATLAB detects the more obvious syntax errors for you automatically, displaying the corresponding error and warning messages and specifying error locations (which is done via the `error` and `warning` functions employed by MATLAB functions, as shown in the example presented in Fig. 5.16). You may also encounter another type of error, *runtime errors,* or errors due to an incorrect mathematical procedure, such as division by zero. This section presents several means that are intended to simplify the process of code debugging and making it more efficient.

5.10.1 Programmatic Tools

By default, the lines of code themselves are not displayed (echoed) in the Command Window while code is running (i.e., only the code execution results show up). Although this is the exact behavior we expect from final

code, allowing the commands to be viewed as they execute, that is, command echoing, provides a very simple and yet effective tool for code debugging or algorithm explanation. It is the `echo` function that allows you to control this process.

The `echo` command behaves slightly differently for the script and function files. For the script files, the use of `echo` is simple—echoing can be either turned `on` or `off`, in which case any script used is affected:

`echo on`	Turns on the echoing of commands in all script files
`echo off`	Turns off the echoing of commands in all script files
`echo`	Toggles the echo state

With the function files, the use of `echo` is more complicated. You may `echo` some specific function only or all functions:

`echo fcnname on`	Turns on echoing of the named function file
`echo fcnname off`	Turns off echoing of the named function file
`echo fcnname`	Toggles the echo state of the named function file
`echo on all`	Sets echoing on for all function files
`echo off all`	Sets echoing off for all function files

Note that if `echo` is enabled, the function files are interpreted rather than compiled. Each input line is displayed as it is executed, which may substantially slow down your computations.

Another command useful for debugging purposes is the `pause` command. It waits for the user's response as follows:

`pause`	Causes a procedure to stop and wait for the user to strike any key before continuing
`pause(n)`	Pauses for `n` seconds before continuing, where `n` can also be a fraction (the resolution of the clock is platform specific, but fractional pauses of 0.01 s should be supported on most platforms)
`pause off`	Indicates that any subsequent `pause` or `pause(n)` commands should not actually pause (this allows the normally interactive scripts to run unattended)
`pause on`	Indicates that subsequent `pause` commands should pause

Finally, there is one more programmatic tool to use for debugging your *m*-files. It is the `keyboard` statement. When placed in an *m*-file, it stops execution of the file and gives control to the keyboard, that is, it enters the so-called debug mode. The special status is indicated by a `K` appearing before the prompt

```
K >>
```

You can examine variables in the workspace created by this point and, if necessary, change them to see how your code executes the remaining

computations (with these changes). To terminate debug mode and continue execution, use the `dbcont` command. To terminate the debug mode and exit the file without completing execution, use the `dbquit` command.

5.10.2 Viewing Options of MATLAB Editor and Its Message Bar

The MATLAB Editor featuring three Toolstrip tabs (see Fig. 2.11) has already been introduced (in Sec. 2.2.3). Figure 5.17 shows an example of *m*-file as it appears in the (undocked) Editor window. Figure 5.17 also demonstrates the capability of splitting the Editor window into two (top/bottom) panels, which is a very simple and yet very effective tool for debugging. You can split the window into two panels by double-clicking the dark gap right above the vertical scroll bar (Fig. 5.18). More options are available within the View tab (see Figs. 2.11c and 5.19). You can click on and move the dividing horizontal bar between two panels up and down, and if you want to combine two panels back into a single one, you can simply double-click this bar.

Note that this option divides the Editor window into two panels to display the same file. If you have several *m*-files open, you can change the way they are displayed by using the Tiles group of buttons of the View tab. For example, Fig. 5.19 features four different files opened in a 2-by-2 panel grid. If needed, you can further split any of these panels vertically or horizontally (note that, in addition to the top/bottom split gap shown in Fig. 5.18, Fig. 5.19 features the similar left/right split gaps to the left of the horizontal scroll bars.)

To the right of the vertical scroll bar (Figs. 5.18 and 5.19), the Editor window features the *warning and error messages bar*, where the MATLAB Code Analyzer automatically and continuously checks your code for coding problems (recall a sample Code Analyzer Report shown in Fig. 2.20b). When

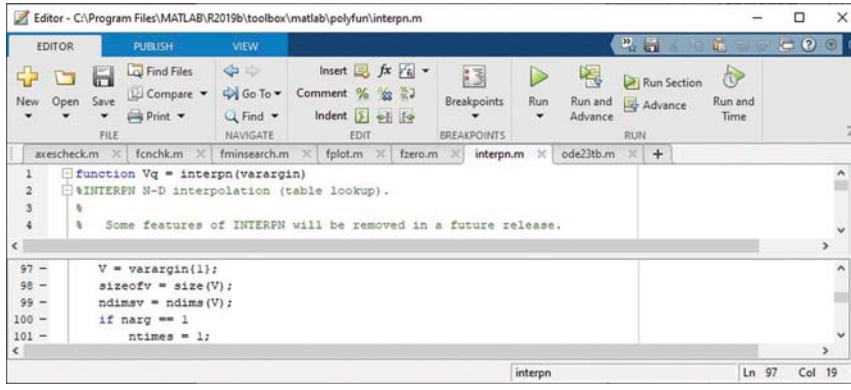


Fig. 5.17 A top/bottom split of the MATLAB Editor window.

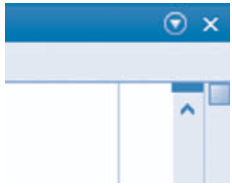


Fig. 5.18 The MATLAB Editor window split gap.

no errors, warnings, or opportunities for code improvement are detected, the square box at the top, the *message indicator*, becomes green. If it is orange, warnings or opportunities for improvement, but no errors, are detected. Red indicates that fatal syntax errors were detected. In the latter two cases, the specific locations of warnings and/or errors are indicated by the corresponding-color horizontal bars (dashes). You may click the message indicator to go to the next code fragment containing a message or click the corresponding dash explicitly, which invokes a message like the one shown in Fig. 5.19 for the *fcnchk.m* function. The code fragment corresponding to a message is underlined, and so you can also view the message by moving the mouse pointer within the underlined code fragment. The message opens in a tooltip and may contain the **Details** and/or **Fix** buttons providing access to additional information (if any) and allowing you to automatically fix the problem, if appropriate (the message shown in Fig. 5.19 does not have additional information or an automatic fix, so neither of these buttons shows up). Note that multiple messages can represent a single problem or multiple problems. Addressing one might address all of them, or after addressing one, the other messages might change or what you need to do might become clearer.

Although most of the Editor tab buttons are self-explanatory and have been introduced in Sec. 2.2.3 already, let us describe several buttons of the Edit group in more detail. With a cursor anywhere on a specific line or with several lines selected, three bottom buttons of the Edit group (see Fig. 5.17) enable smart or manual indentation of a selected piece of code. Three center buttons allow you to convert your selection into comments [by inserting a percent symbol `%` in front of the line(s)], uncomment it

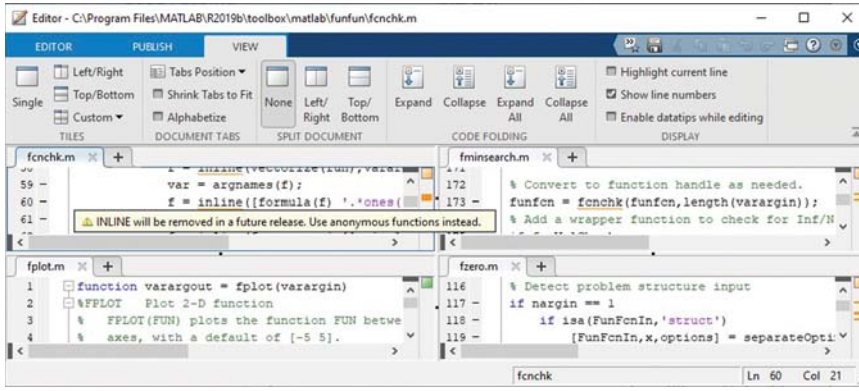


Fig. 5.19 A multitile Editor window showing four different files.

back to executable code (by removing %), or wrap lengthy comments. Alternatively, you can do this by right-clicking the selection of lines and selecting the **Wrap Comments**, **Comment**, or **Uncomment** options from the pop-up context menu (see Fig. 2.10b).

By default, to ensure that the file prints (publishes) without text cropping, the maximum text width is chosen to be 75 characters (see Fig. 2.31a). As you reach the 75th column when typing a comment line, the comment automatically continues on the next line (a one-pixel-wide gray vertical line across the Editor window seen in Fig. 5.17 defines the location of the last symbol for the comments). However, if you happen to have the comment lines exceeding the established (75-column) format, wrapping allows you to automatically reformat your selection. As a result, the lengthy comment lines will become shorter, and the lines that were shorter than the specified maximum will be merged to make longer lines (if they were at the same level of indentation).

Finally, the first three buttons of the Edit group allow introducing new sections by inserting a double percent symbol, %%, in a new line (to be covered in the very next section), looking up a function using the Function Browser introduced in Fig. 2.27b, and inserting different constructors (a special type of subroutines called to create an object for class-based object-oriented programming).

5.10.3 Separating MATLAB Code into Sections

When developing *m*-files, you often experiment with your code—modifying, testing, and updating it—until you have an *m*-file that does exactly what you want. That may include a simple adjustment of the numerical values of constants, but most of the time you are trying to refine the small pieces of your code without changing the rest of it. As such, it seems to be practical to break your code into multiple sections you could execute (debug) individually as opposed to running the entire script altogether. That is how the concept of *m*-file *sections* (originally referred to as *code cells*) was born.

As mentioned at the end of Sec. 5.10.2, you define the beginning of a section by entering two percent signs (%%) at the start of the line. The (optional) text that follows a blank space after %% constitutes a section's title. Although the entire line starting with %% is still considered to be a comment [which is explicitly defined by its (green) color, like the color of any regular comment], the Editor emphasizes the special meaning of the section title by using a boldface font (Fig. 5.20). Although creating the section titles is optional, they definitely improve readability of the file and are especially useful for publishing (to be described later, in Sec. 5.12). Once you define the sections, you may navigate quickly from one section to another, evaluating code in each section (if needed), one at a time, and reviewing the results in the Command Window or base workspace.

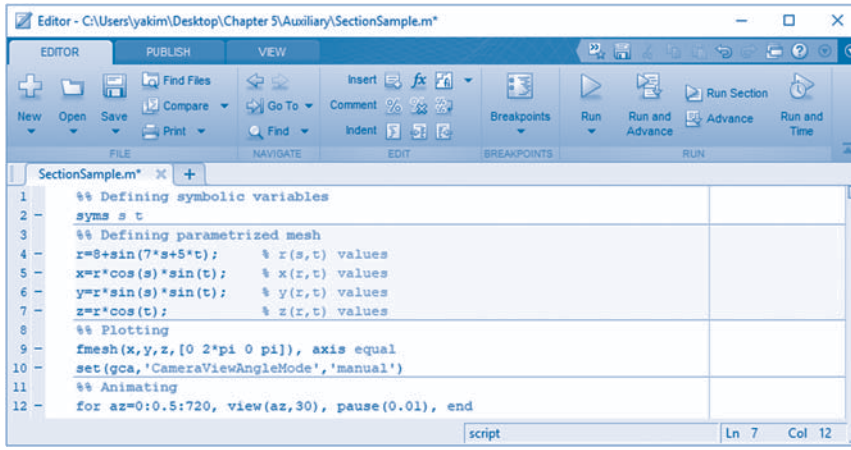


Fig. 5.20 Separating code into sections.

You can run the entire script altogether (by clicking the **Run** button of the Editor toolstrip Run group or pushing <F5>), or you can execute each section separately. When the cursor is placed in any line within a code section, the Editor highlights the entire section with a light yellow background (like the second section in Fig. 5.20). This identifies it as the current section. The highlighted section is the section evaluated when clicking the **Run Section button**. You can also evaluate the current section by clicking the **Run and Advance** (to the next section) button. To move to the next section without executing it, select the **Advance** button.

Note that the drop-down menu of the **Run** button also allows pausing code execution on errors or warnings or when `NaN` or `inf` values are returned. The **Run and Time** button starts the Profiler (to be discussed in Sec. 5.11).

Although code evaluation as a whole requires the file to be saved first (note that code shown in Fig. 5.20 has some unsaved changes because the **Save** button is not faded), the section evaluation runs the current section even if the file contains unsaved changes. Moreover, the file does not even have to be on the search path. The only requirement is that the values the current section requires must already exist in the workspace.

If needed, you can easily degrade section header to a simple comment line. All you have to do is to replace a double percent sign with the single one. This changes the line from a section to a standard comment and merges the section with the preceding section. You can also delete the entire line that contains the `%%`, which will have the same effect.

To execute a single instruction or block of commands, you have to select it (them) and push <F9> (or choose the **Evaluate Selection** option in the drop-down menu when you highlight and right-click this instruction or

block). You may apply this procedure even for the entire script without first saving your file (you may use <Ctrl>+<A> to select entire code).

5.10.4 Debug Mode with Breakpoints

Among other buttons, Figs. 5.17 and 5.20 also feature the **Breakpoints** button, which allows you to debug your code using *breakpoints*. Breakpoints are the points in file where execution stops temporarily, so that you can examine the values of variables up to that point and debug your program.

Clicking the **Breakpoints** button brings up a drop-down menu as shown in Fig. 5.21. Choosing the **Set/Clear** option allows setting up the breakpoint on a current line (where the pointer currently is). The red icon (circle) shows up by the line number, indicating that a valid breakpoint is set at this line (the file should be saved first). Alternatively, you can simply click almost any dash to the right of the line number to set it up (a consecutive click removes it). Figure 5.21 features a breakpoint (red circle) set up on line 70. Right-clicking it brings up a context pop-up menu, also shown in Fig. 5.21.

There are two types of breakpoints you can set in *m*-files:

1. A *standard breakpoint*, which stops at a specified line in an *m*-file (as shown in Fig. 5.21). The green arrow to the right of the breakpoint shows up after you run the file
2. A *conditional breakpoint*, which stops at a specified line in your *m*-file only under specified conditions. To specify the condition, you may choose the **Set/Modify Condition** option of the context menu or **Set Condition** option of the **Breakpoints** button drop-down menu (Fig. 5.21). In any case, the conditional breakpoint GUI shows up (Fig. 5.22a). The conditional breakpoint is denoted by the yellow icon (circle)

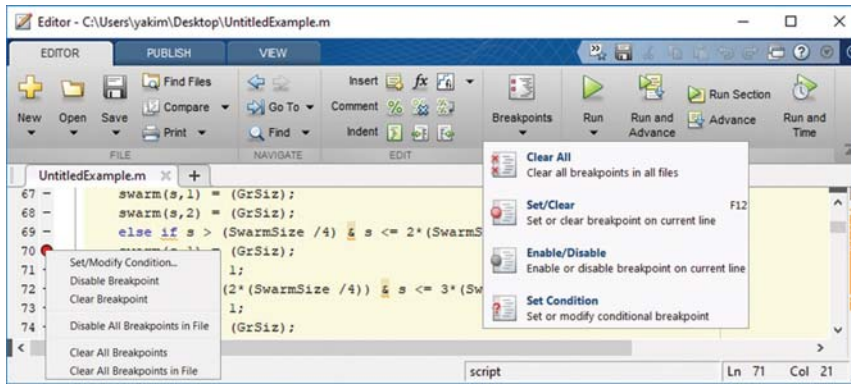


Fig. 5.21 Manipulating with breakpoints.

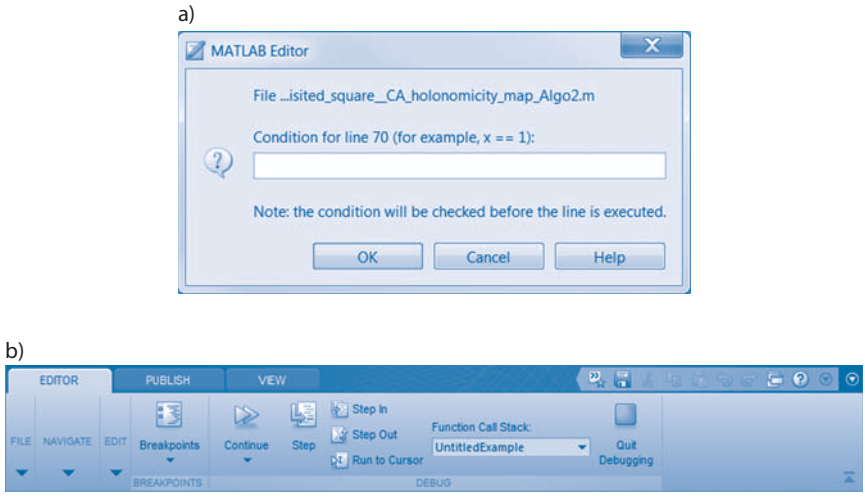


Fig. 5.22 a) Conditional breakpoint menu and b) Debug button group of the Editor tab toolstrip in the debug mode.

When you set a breakpoint it may turn gray, indicating that it is not valid. This may occur in two cases—either the file was not saved since the changes were made (check whether the **Save** button is not faded as in Fig. 5.21) or there is a syntax error somewhere in the file. In the latter case when you set a breakpoint, an error message appears, indicating where the syntax error is.

When you run a file that has breakpoints set up, you enter the debug mode, which was introduced at the end of Sec. 5.10.1. In this mode, the Run group of the Editor tab is replaced with the Debug group of buttons (compare Fig. 5.22b with Figs. 5.17, 5.20, and 5.21). Code runs up to the first breakpoint with a green arrow appearing to the right of it, pointing to the line that has not yet been executed.

Using the Debug group buttons you can step through your *m*-file, pausing at breakpoints where you want to examine the variable values. Specifically, the **Continue** button executes the *m*-file until completion or until another breakpoint is encountered; the **Step** button executes the current line of *m*-file; the **Step In** button executes the current line of *m*-file and, if this line is a call to another function, steps into that function; the **Step Out** button runs the rest of the called function or subfunction after stepping into it, exits it, and pauses; and the **Run to Cursor** button allows running a piece of code up to a line containing a cursor. If there are no more breakpoints, the debug mode is abandoned (you can also exit it anytime by clicking the **Quit Debugging** button).

The **Function Call Stack** field in Fig. 5.22b displays the file names of the function calls that led to the current pause condition, listed in the order in

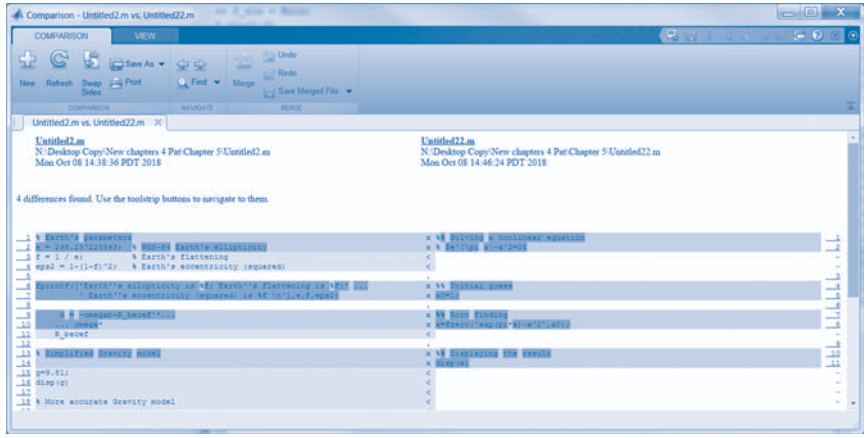


Fig. 5.23 Comparison tool of the Editor.

which they execute. The display starts with the currently executing functions (as shown in Fig. 5.22b) and continues until it reaches the topmost function (in this particular case, "Base," which would show up in the expanded field).

Using either menu of Fig. 5.21, you can disable standard and conditional breakpoints so that MATLAB temporarily ignores them (in this case the breakpoint icon appears with a cross over it), or clear (i.e., remove) them. Breakpoints are not maintained after you exit the MATLAB session.

Let us make two final comments about the debug mode. First, you can use the aforementioned tools to work with both *m*-file scripts and functions. The only difference is that when debugging a functions, all variables it refers to should be available in the workspace. Second, running the *m*-file from the MATLAB Editor in the debug mode, as opposed to from the Command Window by calling the name of the *m*-file, results in changing the prompt in the Command Window to `K>>`.

5.10.5 File Comparison

The files and folders Comparison tool might be useful for code debugging as well. This tool can be called by clicking the **Compare** button of the Editor tab File button group (Fig. 5.21). An example of comparing two files is shown in Fig. 5.23. The two files appear aligned in a window with highlights and indicators for any lines that differ.

5.11 Code Analyzer and Profiler

From Sec. 5.10.2 you learned that the Code Analyzer (M-Lint) is embedded into the Editor/Debugger ensuring the best performance in creating syntax-error-free code. You can use the Code Analyzer Report and other

associated tools in separate windows if you choose to (Fig. 2.20b). You can invoke this tool by selecting the first option from the Editor action menu (shown in Fig. 2.10a), or you can do it programmatically. To this end, a simple command

```
>> mlintrpt
```

scans all files with an *.m* file extension in the current folder and reports the results in a MATLAB web browser. Other syntaxes include scanning a specific file or directory:

```
mlintrpt('filename','file')
mlintrpt('dirname','dir')
```

You can also apply the Code Analyzer preference settings to enable or suppress messages as indicated in the specified *settings.txt* file:

```
mlintrpt('filename','file','settings.txt')
mlintrpt('dirname','dir','settings.txt')
```

These settings can be accessed by clicking the **Preferences** button of the Home tab Environment group and choosing **Code Analyzer** on the left (Fig. 5.24).

Another function, `checkcode`, allows you to look for syntax errors in your file(s) even without opening the Code Analyzer browser. The results

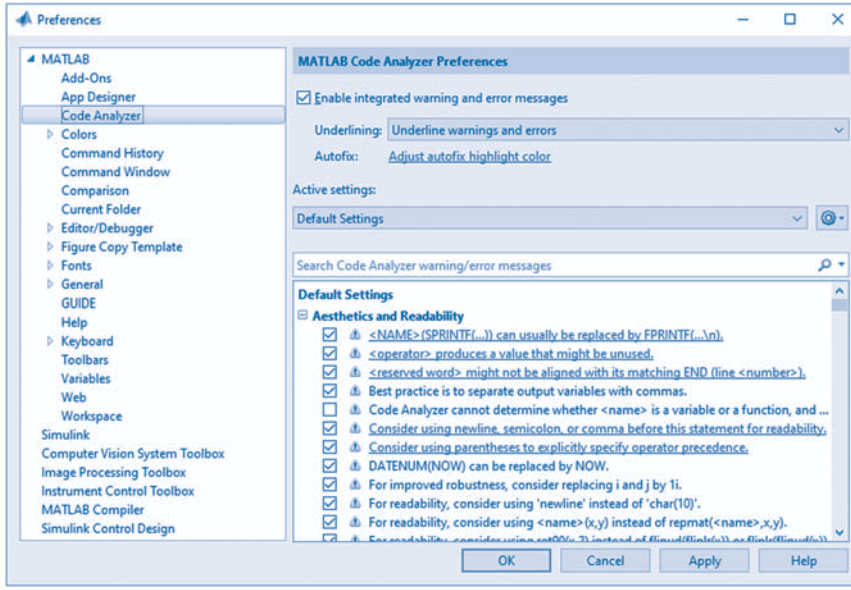


Fig. 5.24 Code Analyzer preferences window.

of either of the following two commands will be displayed directly in the Command Window:

```
checkcode('filename')
checkcode('filename','-config=settings.txt')
```

The typical output contains the Code Analyzer messages that indicate the lines and columns with potential problems. For example,

```
L 2 (C 4-9): The value assigned here to variable 'Speed' might never be used.
L 3 (C 5): Terminate statement with semicolon to suppress output (in scripts).
L 7 (C 8): Use || instead of | as the OR operator in (scalar) conditional
           statements.
L 15 (C 13): There may be a parenthesis imbalance around here.
L 21 (C 15-17): 'dim' might be growing inside the loop. Consider preallocating
              for speed.
L 31 (C 20): Use of brackets [] is unnecessary. Use parenthesis to group, if
           needed.
```

Clicking the line number opens the Editor and navigates directly to the candidate error, suggesting appropriate actions to fix it. If you wish not to show some specific warning, say, about line termination with a semicolon, you may always uncheck the corresponding radio button in the Code Analyzer preferences browser (Fig. 5.24).

Once you have fixed all syntax errors and debugged your code using any of the tools discussed in Sec. 5.10, the last thing you may want to consider (especially if you are dealing with sophisticated code that requires a lot of computational resources) is how to possibly improve its effectiveness, that is, reduce the central processing unit (CPU) time required to get the results.

The way to measure the total time required to run your code (or a particular portion of it) is to wrap it with the two commands, `tic` and `toc`. This pair measures performance using a stopwatch timer. Specifically, `tic` starts a stopwatch timer, and `toc` displays the elapsed time since `tic` was used. You may assign this time to some variable for further use if needed.

Consider the following self-explanatory example:

```
A=zeros(500,500); B=zeros(500,500); C=zeros(500,500);
for n = 1:500
  A = rand(n,n); B = rand(n,n);
    tic
    C = A.^B;
    C=sqrt(C);
    t(n) = toc;
end
plot(t*10^3,'.'), grid
xlabel('Size of the problem'), ylabel('CPU time, ms')
```

The result is shown in Fig. 5.25. As expected, the required computational resources (i.e., CPU time) grow with the size of the problem (not necessarily monotonically, but definitely showing a general tendency).

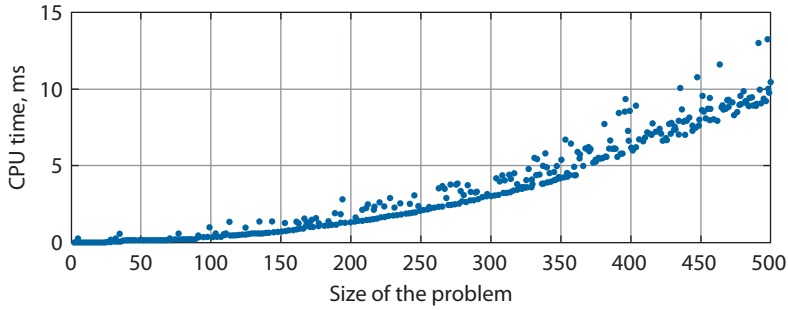


Fig. 5.25 Measuring required computational resources with the `tic-toc` pair.

Note that, in principle, the first line in the aforementioned piece of code is not required. However, repeatedly expanding the size of arrays `A`, `B`, and `C` in the `for` loop over time can adversely affect the performance of your program because

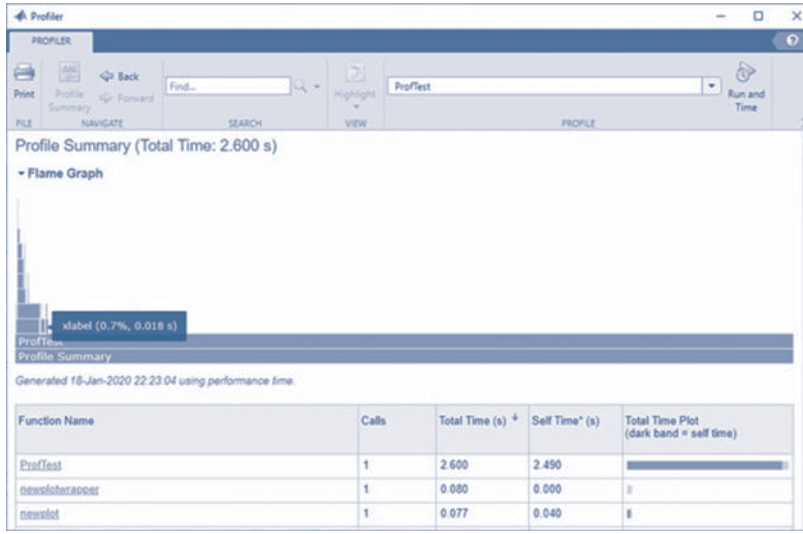
1. MATLAB has to spend time allocating more memory each time you add more elements to arrays
2. The newly allocated memory is likely to be noncontiguous, thus slowing down any operations that MATLAB needs to perform on your arrays

Hence, the first line of the aforementioned code allows *preallocating memory* (or *preallocating arrays*), that is, you preallocate the maximum possible amount of memory for all three arrays even before they were used for the first time. This way, the program performs one memory allocation that reserves one contiguous block. Of course, in our example the advantages of memory preallocation may be negligible, but this is definitely one of the ways of improving performance of your code if it involves the loops with large variable arrays.

MATLAB features another more sophisticated tool that allows you to perform a detailed code profiling to find where a program spends most of the time (bottlenecks). That is where you should look to optimize your code. As mentioned previously, the Profiler is invoked by clicking the last button of the Editor tab, **Run and Time**. The results for the piece of code we just considered are shown in Fig. 5.26a (if you are using any release prior to R2020a, the Profiler Summary window looks slightly different). Another alternative to running the Profiler is to use the `profile` function. For example, the commands

```
profile on
ProfTest
profile viewer
```

a)



b)

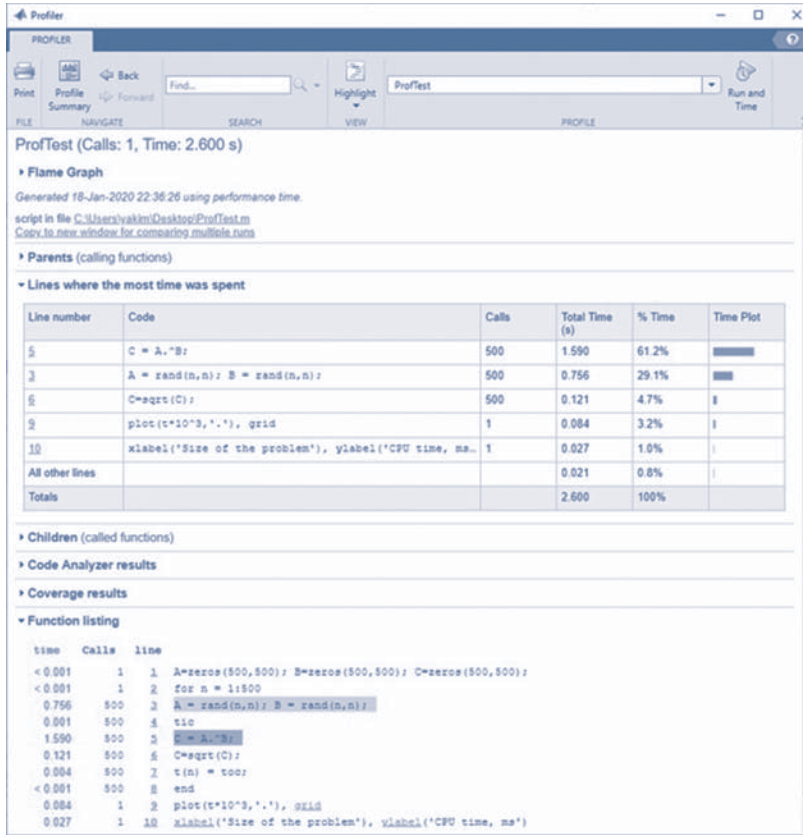


Fig. 5.26 a) Profiler Summary and b) code lines where most of the time was spent.

would profile the aforementioned code (stored as *ProfTest.m*), producing the same result.

The Profile Summary provides detailed information for each function called in the form of an interactive *flame graph* and a table. This information includes.

Function Name	A function or subfunction name called by the profiled function
Calls	A number of times the function was called while profiling was `on`
Total Time	Time spent in a function, including all child functions called, in seconds (the total time for the files, whose running time was inconsequential, shows as zero). The same result is repeated in the fifth column, Time Plot, graphically using the two-color horizontal bars
Self-Time	Time spent in the function, not including time for any child functions called (if it can be determined), in seconds

Clicking the name of any function (including the main script) in the Profile Summary brings up another window (Fig. 5.26b), which provides specific details on each line of code. You can also access and review other information within this window by expanding the corresponding section.

A general procedure for analyzing the results is as follows. Once you identify which functions are consuming most of the time, you can determine what they specifically do and whether there are ways to minimize their use and thus improve computational performance. Because scripts and functions may have several layers (one function calls another), your code might not explicitly call the most time-consuming functions. In this case, it is important to determine which of your functions are responsible for such the calls. Possible solutions include

1. Avoiding unnecessary computations (if possible)
2. Changing an algorithm to avoid costly computations (or functions)
3. Avoiding recomputation by storing intermediate results in variables and using them instead

5.12 Formatting MATLAB Comments for Publishing

Sometimes, rather than trying to fix bugs by just reading code from the screen, it is better to print your work and have a closer look at it away from the computer. In this case, it would be nice to see the *m*-file code itself, all comments, and execution results. Maybe you would also want to include your debugged code in your report or even publish it on the web later on.

To meet this demand, MATLAB offers a nice capability for publishing your work as an HTML document by simply clicking, **Publish**, the rightmost icon of the Publish tab of the Editor (Fig. 5.27a). If you click the

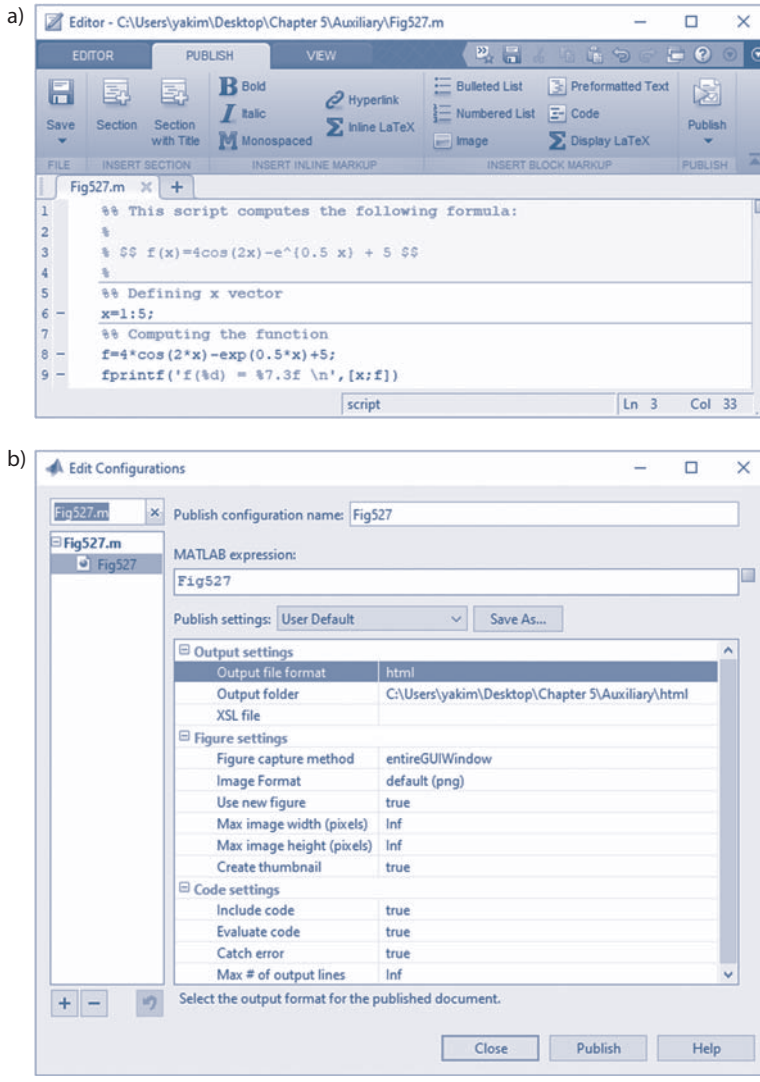


Fig. 5.27 a) Example of a multisection script with LaTeX equation markup and b) the Edit Configurations window.

down-arrow portion below this icon, and choose **Edit Publishing Options**. . . from the drop-down menu, the Edit Configurations window will pop up, as shown in Fig. 5.27b. Clicking "html" to the right of the Output file format setting brings up a menu showing that HTML is not the only format available for publishing. Other formats are *xml* for Extensible Markup Language, *latex* for LaTeX, *doc* for Microsoft Word, *ppt* for Microsoft PowerPoint, and *pdf*

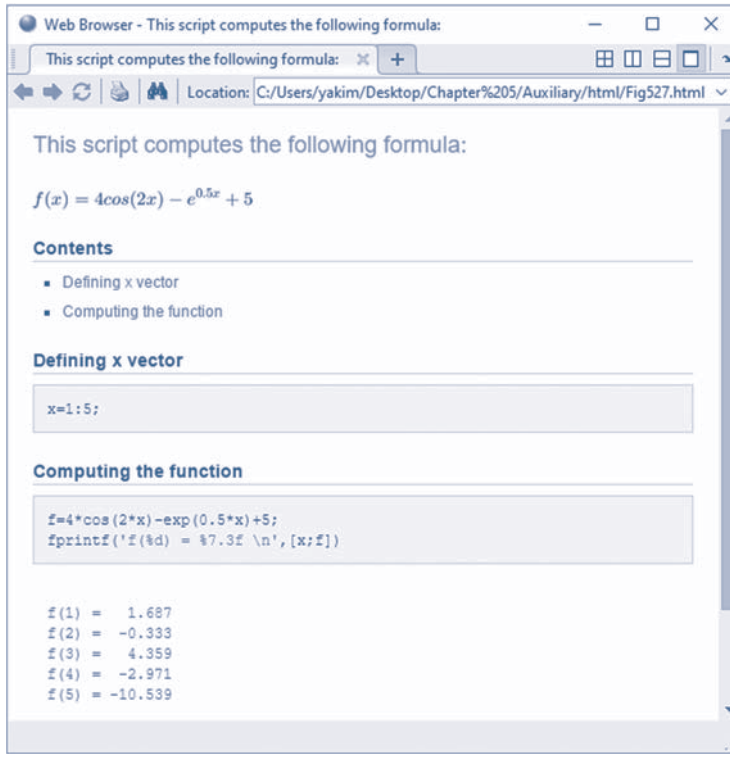


Fig. 5.28 Example of a published script (shown in Fig. 5.27a).

for Portable Document Format. Within the Publish Settings panel, you may also change the figure capture method (using the `getframe` or `print` functions to get an entire GUI or just figure window) and image format (*png*, *jpeg*, *bmp*, or *tiff*) as well as tweak some other publishing settings. (You can save any changes you make to a publish settings file by clicking the **Save As**... button.)

For example, organizing code shown in Fig. 5.27a as a sequence of sections and adding (optional) text markup (in the first section), results in a published document that looks like that presented in Fig. 5.28 (by default, it is an HTML document opened in the MATLAB Web Browser).

As you can see, the code sections introduced in Sec. 5.10.3 happen to be very useful in defining how a printout looks. The section titles show up in a boldface font, the Contents section is added automatically, the results follow the commands, and, in addition to that, the document features a nice symbolic representation of the equation used in computations. As seen from Fig. 5.27a, this latter option was enabled by placing the line with the equation (that uses a LaTeX language syntax) in the comments section after the section header (the line starting with `%%`). The equation is embraced

by $$ from both sides and has one empty line (starting with %) before and possibly after it (if your comments continue). Section 6.5 and Appendix A provide you with more details on the LaTeX syntax, but for now, let us mention other available publishing markup options beyond the section titles and equations.

Tables 5.8 and 5.9 summarize all available options that are provided by pushing the buttons of the Publish tab of the Editor (Fig. 5.27a) or typing in LATEX constructs manually, allowing you to create a good-looking document. Figures 5.29 and 5.30 show examples of using these options to publish a specially formatted script to an HTML document.

Let us mention a couple of useful tricks. First, to produce Fig. 5.30, we reduced the max image width in the options box of Fig. 5.27b to 200 pixels. Second, an image file you want to be inserted (*AIAALogo.jpg* in Fig. 5.30) should reside in the same directory where the HTML file is stored, that is, in the *html* subfolder. Note that none of markups disturb the flow of the program—they appear as comments. If you end up having significant portions of a formatted text in your code, you may save them as a separate *m*-file and then call it from the main file, where appropriate.

By the way, you do not need to insert markups manually or switch to the Publish tab to do so. You can also conveniently do this by selecting the corresponding option from the pop-down list after right-clicking anywhere within the Editor window and choosing **Insert Text**

Table 5.8 Summary of Inline Markup (Not Requiring a Formatted Block)

Markup	Result in Published Document
`%% DOCUMENT TITLE` `% INTRODUCTORY TEXT`	Document title and introduction (the very first markup appears different from a regular section title)
`%% SECTION TITLE` `% DESCRIPTIVE TEXT`	Section title and description
`%%% SECTION TITLE` `% DESCRIPTIVE TEXT`	Section title without a section break
`%%`	Section break without a title or description
`% *BOLD TEXT*`	Bold text
`% _ITALIC TEXT_`	Italic text
`% |MONOSPACED TEXT|`	Monospaced text
`% <my.nps.edu/web/ads>`	Hyperlinked text
`% TEXT(TM)`	Trademark symbol
`% TEXT(R)`	Registered trademark symbol
`snapnow;`	Code to force a snapshot of the current output
`% $ e^{\beta x} cos(5x) = 0 $`	Inline LaTeX equation

Table 5.9 Summary of Block Markup

Markup	Result in Published Document
`% <<FILENAME.JPG>>`	Image
`% * ITEM 1` `% * ITEM 2`	Bulleted list
`% # ITEM 1` `% # ITEM 2`	Numbered list

`%  for i = 1:10`
`%      disp x`
`%  end`
	MATLAB code

`% <include>filename.m</include>`	External file content
`% $$ e^{\beta x} cos(5x) = 0 $$`	Display LaTeX equation

`% <html>`
`% <table border=1><tr>`
`% <td>alpha</td>`
`% <td>beta</td></tr></table>`
`% </html>`
	HTML markup

`% <latex>`
`% \begin{tabular}{|c|c|} \hline`
`% $n$ & $n!$ \\ \hline`
`% 1 & 1 \\`
`% 2 & 2 \\ \hline`
`% \end{tabular}`
`% </latex>`
	LaTeX markup [only works if you intend to publish it with the specified output file format LaTeX (Fig. 5.28); if it is anything else, including HTML, then LaTeX markup will be excluded]

Markup (Fig. 2.10b), which brings up the drop-down menu shown in Fig. 5.31.

As is always the case, you can publish a MATLAB file programmatically, without opening the Editor, by using the `publish` function:

```
publish('file')
publish('file','format')
publish('file',name,value)
publish('file',options)
mydoc=publish('file',...)
```

The first syntax publishes *file.m* by running it in the base workspace, one section at a time. It saves the results to an HTML output file, *file.html*, stored in an *html* subfolder of the folder containing *file.m.* Other syntax allows changing an output file format and any of other options shown in Fig. 5.27b and also saving the path of the published HTML file to the variable `mydoc`.

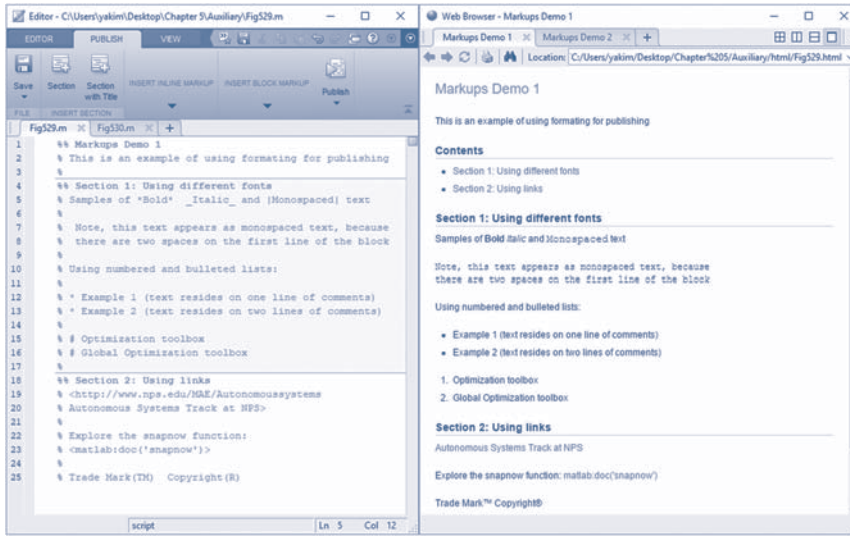


Fig. 5.29 Using markups to produce a readable document.

When publishing an *m*-file function, you would not have much beyond a nicely formatted text. If you really want to see the results of this function execution, you might want to comment the function declaration statement and make sure that all the input values you would normally pass

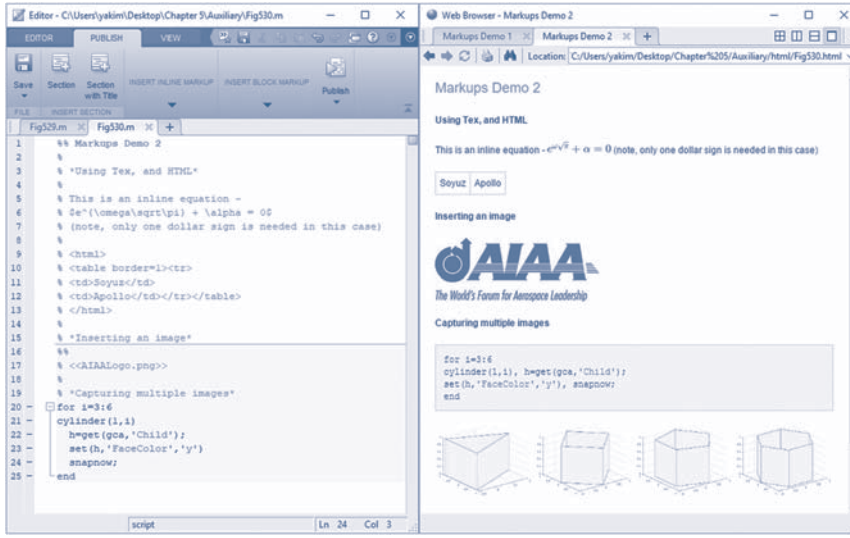


Fig. 5.30 Using markups to insert TeX and HTML code along with (multiple) images.

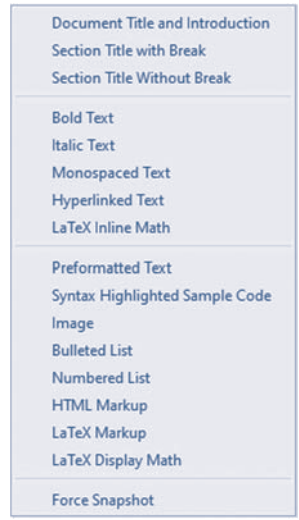


Fig. 5.31 Publishing markup options available in MATLAB Editor.

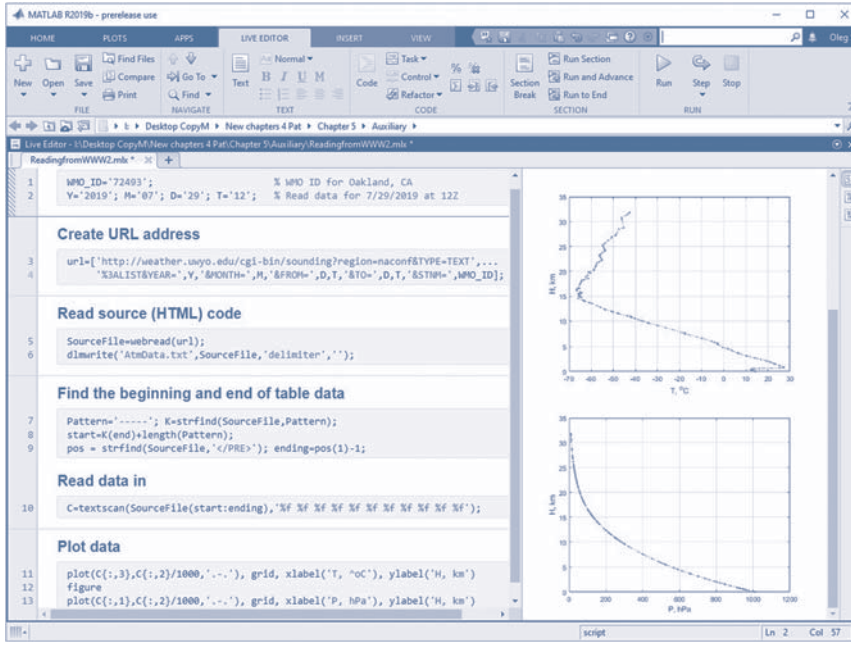


Fig. 5.32 Live Script saving options.

over to this function via a list of input arguments are available in the base workspace.

If you recall live scripts, introduced in Sec. 2.2.3, you realize that for better readability, they take advantage of similar markup options (see Fig. 2.13a) even without formal publishing, that is, they appear as webpages from the beginning (recall the example of a live script in Fig. 2.14). Figure 5.32 shows another example of a live script. As mentioned in Sec. 2.2.3, you can save (publish) a live script as a PDF, Word, HTML, or LaTeX document converting all formatted content to the publish markup of Tables 5.8 and 5.9.

Let us conclude this section by introducing another MATLAB function, `grabcode`, that you might want to use to do exactly the opposite of what we have previously described, that is, extracting MATLAB code from a file published to HTML. Specifically,

`mycode=grabcode('name.html')`	Copies MATLAB code from the file *name.html* and pastes it into a document *mycode.m* in the Editor
`grabcode url`	Copies MATLAB code from the page or file specified by `url` and pastes it into an untitled document in the Editor
`code = grabcode(filename)`	Returns the MATLAB code from `filename` to a character array `code`

Problems

5.1 The average daily temperature (in °F; rounded to the nearest integer) recorded by the Cape Canaveral Air Force Station in Florida and Edwards Air Force Base (AFB) in California during the month of January 2010 is given in the following vectors (to get data for any other period of time or place, go to the National Climatic Data Center website, *https://www.ncdc.noaa.gov/cdo-web/datatools/findstation*):

```
TCan = [ 62 50 46 44 43 41 43 45 41 36 41 42 48 51 60 68 71 61 ...
         56 55 66 72 65 70 69 59 56 54 59 67 61]
TEdw = [ 39 40 37 37 37 40 42 47 43 45 44 45 55 46 41 44 48 49 ...
         45 40 38 41 41 39 42 42 47 46 43 43 41]
```

Write a program (a script file) that utilizes the `find` function to answer the following questions:

- What was the average temperature for the month at each place?
- How many days was the temperature below its average at each location?

- How many days and on which dates in the month was the temperature at Edwards AFB higher than the temperature at Cape Canaveral?
- How many days and on which dates in the month was the difference between the temperatures at both places within 3 degrees?
- Among those days found in the previous question, how many days was the temperature at Edwards AFB higher than that of Cape Canaveral?

5.2 Write a function (anonymous or inline) computing the following expression $f(x) = \cos(x)\sin(10x)$. Use this function in conjunction with the MATLAB functions introduced in Sec. 5.8.2 to

- Plot the graph of this function within the range $x \in [0; 0.5]$
- Find a zero of this function within the range $x \in [0; \pi]$
- Find a minimum of this function within the range $x \in [0; \pi]$
- Find a maximum of this function within the range $x \in [0.5; 1.0]$

5.3 Write a function that returns parameters of the standard atmosphere at a certain altitude. You may use tabular data for certain altitudes, bottom edges of isothermal or nonisothermal layers, or any approximations you can find. For examples, see *www.grc.nasa.gov/WWW/K-12/airplane/atmos.html*.

Chapter 6 2-D and 3-D Plotting and Animation

- Anatomy of the Plot and Graphics Objects
- Creating Self-Contained Plots
- 2-D and 3-D Plotting Capabilities
- Interactive Plotting and Animation

`plot`, `plot3`, `line`, `title`, `text`, `datatip`, `axes`, `fplot`, `fimplicit`, `figure`, `subplot`, `linkaxes`, `legend`, `hold`, `grid`, `polarplot`, `stem`, `hist`, `mesh`, `surf`, `bar`, `bar3`, `stackedplot`, `geoplot`,`cylinder`, `patch`, `gca`, `gcf`, `get`, `set`, `shading`, `gtext`, `ginput`, `colormap`, `animatedline`, `VideoWriter`, `getframe`, `writeVideo`, `movie`, `uicontrol`, and many others

6.1 Introduction

In engineering, the different kinds of plots are used to present and explain tabulated data produced by employing analytical formulas and/or a numerical experiment. This chapter addresses MATLAB's plotting capabilities that are the most advanced among all programming languages. As of R2014b, the graphics system relies on improved graphics-object-based infrastructure, new colormap, anti-aliased fonts and lines, and many other new features for improved charts and customization. The variety of the 2-D and 3-D plotting functions extends far beyond the simplest `plot` and `plot3` functions and include `line`, `fplot`, `fimplicit`, `loglog`, `semilogx`, `semilogy`, `polarplot`, `area`, `pie`, `bar`, `contour`, `contourf`, `fcontour`, `comet`, `compass`, `feather`, `quiver`, `stem`, `stairs`, `hist`, `stackedplot`, `geoplot`, `fill`, `triplot`, `scatter`, `errorbar`, `fplot3`, `mesh`, `meshc`, `meshz`, `fmesh`, `surf`, `surfc`, `fsurf`, `trisurf`, `bar3`, `pie3`, `contour3`, `quiver3`, `stem3`, `ribbon`, `waterfall`, `slice`, `comet3`, `cylinder`, `ellipsoid`, `sphere`, `fill3`, and `scatter3`. There are also many auxiliary functions such as `figure`, `subplot`, `tiledlayout`, `title`, `axis`, `xlim`, `grid`, `xlabel`, `hold`, `legend`, `datatip`, `text`, `annotation`, `view`, `set`, `get`, `meshgrid`, `trimesh`, `colormap`, and `shading`. Some other functions enable interactive input (`gtext` and `ginput`) and animations creation (`getframe`, `frame2im`, `im2frame`, `movie`, `VideoWriter`, and `addframe`). This chapter starts with the simplest 2-D plotting functions, addressing all

environmental variables of a single plot, followed by a discussion of line properties. MATLAB's capabilities for enhancing plot's readability by formatting the text strings and including special symbols in them, and the ways to access and modify the properties of the graphics objects, are considered next. The discussion proceeds with having more than just one axis within one Figure window and/or more than just one line per set of axes. Special types of the 2-D plots and different types of the 3-D plots are introduced after this, followed by a brief discussion of how to manage a color palette and use the function-plotting functions. The plot-editing options of MATLAB readily available within the Figure window are addressed next. For advanced users, this chapter also presents capabilities for interactive plotting and creating MATLAB movies and standard video format files. The chapter concludes with a list of common-sense requirements that any engineering plot should satisfy.

6.2 Anatomy of the 2-D (*x*–*y*) Plots in MATLAB

To start, let us consider an anatomy of a 2-D plot. The typical plot consists of the following (Fig. 6.1):

1. The horizontal axis, *abscissa*, and vertical axis, *ordinate*, both defined by the *scale*—the range and spacing of the numbers (*linear* or *logarithmic*) and *tick marks* locations
2. The *curve* itself, represented by a sequence of (connected) *data points* (*x* and *y* data). It can also include the *marker* used to identify data points
3. The *text inserts* to show

Title	Identifying what the plot is about
Axis labels	Showing the name and units of the quantity plotted along each axis
Tick mark labels	Specifying numbers or letters that correspond to tick marks, and possibly
Legend	Differentiating between multiple sets of data on one plot

4. The *grid*

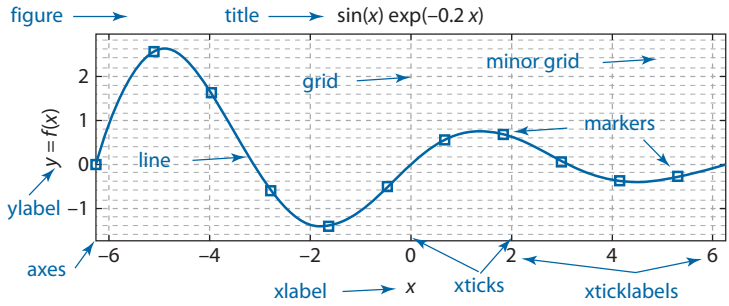


Fig. 6.1 Anatomy of a typical 2-D plot.

Table 6.1 Basic 2-D Plotting Functions

Function	Brief Description
`plot(y)`	Generates a plot of the array `y` vs its indices if `y` is a real vector, or plots the imaginary parts of `y` vs the real parts of `y` if `y` is complex vector. This function also creates the figure, axes, and line objects
`plot(x,y)`	Generates a plot of array `y` vs array `x`
`plot(x,y,LineSpec,...)`	Plots array `y` vs array `x` using `LineSpec` line specification that determines the line style, color, and marker symbol (to be considered in Sec. 6.3)
`line(x,y)`	Adds the line(s) defined in the arrays `x` and `y` to the current[a] axes
`fplot(fun,[limits],...)`	Generates a plot of the function $f(x)$ (specified as a function handle `fun`[b] to a named or anonymous function) within the specified `limits` (by default $x \in [-5; 5]$)
`fplot(funx,funy,...)`	Plots the curve defined parametrically, as $x = f_x(t)$ and $y = f_y(t)$
`fimplicit(fun,...)`	Plots the implicit function $f(x,y) = 0$ defined by a function handle `fun` (`fimplicit` was introduced in R2016b)
`ezplot(fun,...)`	Plots the function $f(x)$ or $f(x,y) = 0$ described by the function handle `fun` within the default domain $-2\pi \le x \le 2\pi$
`ezplot(funx,funy,...)`	Plots the planar curve defined parametrically, $x = f_x(t)$ and $y = f_y(t)$

[a] "Current" means the last one created or clicked with the mouse.
[b] As of R2020a, it supports the character vector inputs as well, but it is expected to be removed in a future release.

The basic *line-* and *function*-plotting functions shown in Table 6.1 allow creating some of the aforementioned elements for a typical 2-D plot all at once (defining some of them explicitly and using some default values for the remaining ones).

Examples of using these functions are presented in Figs. 6.2 and 6.3. Specifically, the command

```
>> plot(sqrt([20:40]))
```

creates a plot of $\sqrt{x}$ vs the indices of the input vector, defining its axes and tick marks automatically (Fig. 6.2a). The line style and color for the curve itself are set to some default values. No other parts of the plot are defined.

In contrast, the command

```
>> plot(20:40,sqrt([20:40]),'or--')
```

(Fig. 6.2b) plots the same plot vs its argument, explicitly defining the data point marker (circle), line style (dashed line), and color (red).

Employing the function-plotting function, `fplot`,

```
>> fplot(@cos,[0 pi])
```

allows creating axes using the explicitly defined range for the abscissa and calculating the corresponding range for the ordinate based on the data

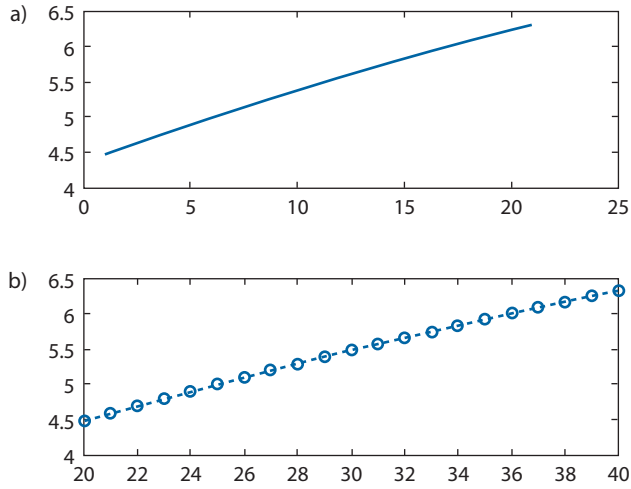


Fig. 6.2 Examples of using the `plot` function.

points computed internally utilizing the function handle to the *cos.m* file (Fig. 6.3a). The tick mark locations and labels are determined and added automatically. The command

```
>> ezplot('sqrt(x)')
```

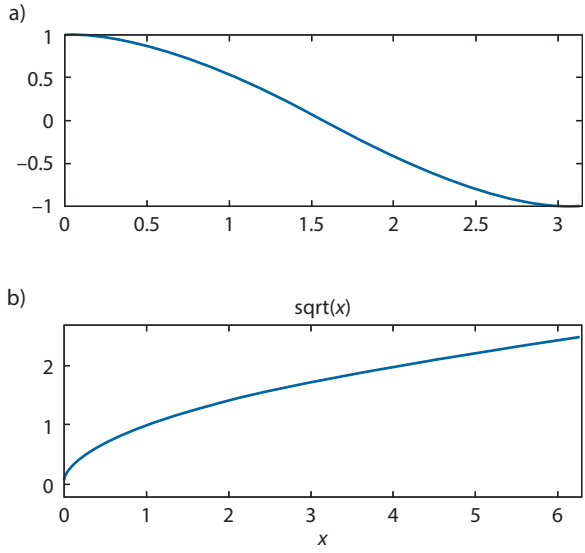


Fig. 6.3 Examples of using the a) `fplot` and b) `ezplot` functions.

(Fig. 6.3b) creates a plot of the $\sqrt{x}$ function over the default interval $0 \le x \le 2\pi$ (the function is not defined for $x<0$), adding the x-axis label (x) and title of the plot [sqrt(x)] automatically.

Figure 6.4 shows how the functions of two variables can be visualized using the `fplot`, `fimplicit`, and `ezplot` functions. These plots were produced using the following sets of commands:

```
>> fplot(@(t)sin(2*t),@(t)sin(2.5*t)), grid, axis equal
>> fimplicit(@(x,y)sin(x)+sin(y)-sin(x.*y),2*pi*[-1 1 -1 1]),...
                                        grid, axis equal
>> ezplot(@(x,y)sin(x)+sin(y)-sin(x.*y)), grid, axis equal
```

Unlike the two other functions, the `ezplot` function in the last call also produces the x-axis label and title (similar to Fig. 6.3c). This title, however, was modified along with x-axis tick locations and labels using the following commands:

```
>> ax=gca;
>> S=sym(ax.XLim(1):pi/2:ax.XLim(2));
>> ax.XTick=double(S);
>> ax.XTickLabel=arrayfun(@texlabel,S,'UniformOutput',false);
```

Specifically, the first command retrieves a structure defining the Axes *graphics object* (to be discussed in Sec. 6.4), and the last two commands change some of its properties using a standard structure field indexing syntax. (If you use the `whos ax` call or simply look in the Workspace panel, you will see that the full class name of the object `ax` is `matlab.graphics.axis.Axes`, but for simplicity herinafter we refer to it as an Axes object.) This particular example takes advantage of using a symbolic variable `S` defined by the second command (this data type will be addressed in Chapter 7).

All three plots in Fig. 6.4 feature a grid created by the `grid` function and equal tick mark spacing along the x- and y-axes defined by the `axis` function. Table 6.2 shows more syntaxes for these two functions, allowing manipulation with the first group of the plot content (axes).

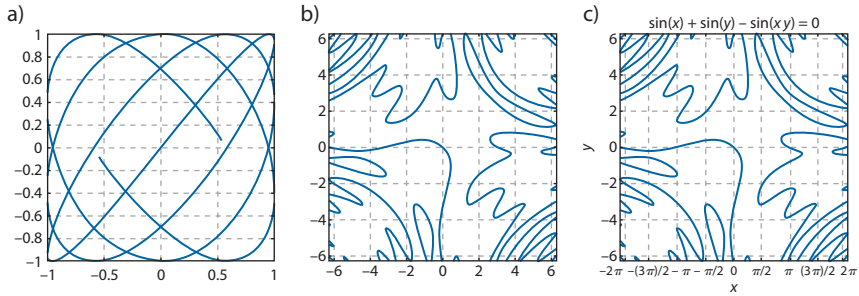


Fig. 6.4 Examples of using the a) `fplot`, b) `fimplicit`, and c) `ezplot` functions to visualize the functions of two variables.

Table 6.2 The `axis` and `grid` Functions

Function	Brief Description
`axis([xmin xmax... ymin ymax])`	Sets the minimum and maximum limits of the *x*- and *y*-axes. If you want to specify just one limit but want MATLAB to autoscale the other, use `Inf` or `-Inf` for autoscaled limits or use `xlim` and `ylim` functions
`axis square`	Selects the axes' limits so that the plot will be square
`axis equal`	Assures that the *x*- and *y*-axes have the same tick mark spacing
`axis tight`	Sets the limits to the range of the data
`axis vis3d`	Freezes aspect ratio properties to enable rotation of 3-D objects and overrides stretch-to-fill
`axis manual`	Freezes the scaling at the current limits so that if `hold` is turned on (Sec. 6.6), subsequent plots will use the same limits
`axis auto`	Returns the axis scaling to its default autoscaling mode (other available modes include `'auto x'` and `'auto y'`)
`axis ij`	Reverses a direction of the *y*-axis with the values increasing from top to bottom (as opposed to the default one, `xy`)
`v=axis;`	Returns the current axis scaling in the vector `v` [so that you may use the same scaling for another plot by calling `axis(v)`]
`axis off`	Turns visibility of axes lines and background off
`grid`	Displays gridlines at the tick marks corresponding to the tick labels. You may use `grid on` to turn gridlines on and `grid off` to turn them off. When used by itself, `grid` toggles the grid switch `on` or `off`
`grid minor`	Toggles the visibility of the minor grid lines (note that not all types of charts support the minor grid lines)

For example, the following set of commands

```
>> t=linspace(0,2*pi);
>> line(cos(t),sin(t)), grid
```

results in the plot shown in Fig. 6.5a. By default, axes are scaled automatically (in this particular case, the `axis tight` command would result in the same plot), and the `grid` function adds gridlines. However, if you substitute the second line with

```
>> line(cos(t),sin(t)), axis equal
```

you will have the same scale for both axes, and therefore a unit circle will appear as it is supposed to (Fig. 6.5b).

Most of the third group of the plot content (text inserts) can be manipulated using the functions of Table 6.3. For example, the set of commands

```
>> title('My Plot'), xlabel('x-axis'), ylabel('y-axis')
>> text(0.5,0.5,'Text')
```

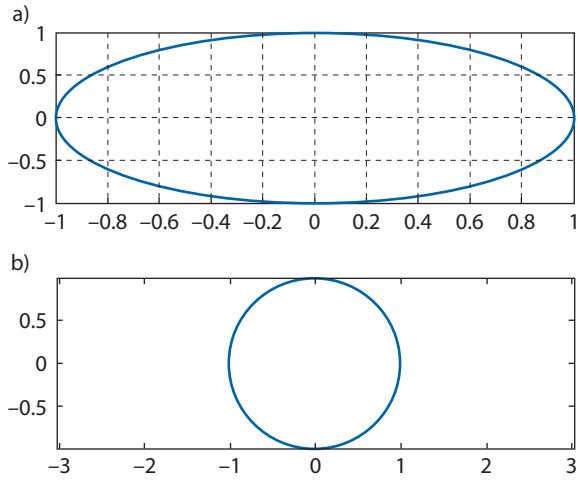


Fig. 6.5 Examples of changing axis properties.

results in axes shown in Fig. 6.6a. The commands

```
>> annotation('arrow',[0.5 0.7],[0.7 0.8])
>> gtext('Chosen Location')
```

add an arrow, specifying the *x*- and *y*-coordinates of its origin and end points. As you move the pointer into a Figure window, the pointer becomes crosshairs (as shown in Fig. 6.6b) to indicate that `gtext` is waiting for you to select a location at which to add the character vector. Once you click a mouse button, the text `Chosen Location` will be added to your figure.

Table 6.3 Text and Annotation Functions

Function	Brief Description
`title('text')`	Places `text` in a title at the top of a plot. The command `title({'First line';'Second line'})` creates a two-line title
`sgtitle('text')`	Adds the `text` title above the grid of subplots in the current figure (introduced in R2018b)
`xlabel('text')`	Places the `text` label on the *x*-axis (abscissa)
`ylabel('text')`	Places the `text` label on the *y*-axis (ordinate)
`text(x,y,'text')`	Adds `text` to a location `(x,y)` on the current axes, where `(x,y)` are expressed in the current plot units
`gtext('text')`	Places `text` in the Figure window at a point specified by the mouse click
`annotation(type,...` `Name,Value)`	Creates the specified annotation `type` (line-type: `'line'`, `'arrow'`, `'doublearrow'`, `'textarrow'`, or shape-type: `'rectangle'`, `'ellipse'`, `'textbox'`) and defines its properties as `Name-Value` pair arguments (specific for each annotation `type`)

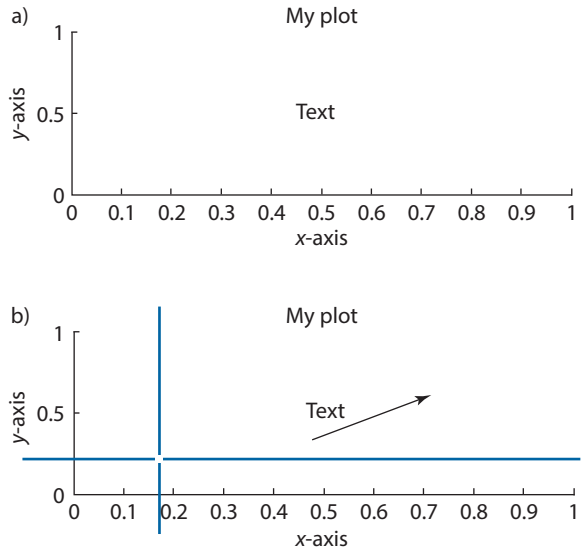


Fig. 6.6 Examples of adding the a) text and b) annotations to the plots.

Note that, by default, the annotation objects use the *normalized coordinates* to specify locations within the figure. In normalized coordinates, the point (0, 0) is the lower-left corner and the point (1, 1) is the upper-right corner of the Figure window, regardless of the figure size and proportions. It is not convenient, so if you intend to use the `annotation` function, you might consider writing your own function to convert your axis coordinates into the normalized ones (see Problem 6.1).

A few more functions, introduced in R2016b and shown in Table 6.4, allow you to modify the *x*-axis tick label parameters (similar functions are also available for the *y*- and *z*-axis).

Table 6.4 Modifying Axis Tick Label Parameters

Function	Brief Description
`xticks(ticks)`	Sets the *x*-axis tick values for the current axis as a vector of increasing numeric values, `ticks`, specifying locations along the *x*-axis where the tick marks appear
`xticklabels(labels)`	Sets the *x*-axis tick labels for the current axes as a string array or a cell array of character vectors `labels`
`xtickangle(angle)`	Rotates the *x*-axis tick labels for the current axes by the specified `angle` in degrees (a positive value for counterclockwise rotation and a negative value for clockwise rotation)
`xtickformat(fmt)`	Changes the format for the numeric *x*-axis tick labels using the format specifier `fmt` as defined in Fig. 4.17 (the default format is `'%g'`). Other predefined formats include `'usd'`, `'degrees'`, `'percentage'`. `fmt` can also specify the label's format for date and time data (see Table C.5 of Appendix C)

More options for changing the plot content and format will be discussed in Sec. 6.4. Specifically, it will be done by manipulating properties of the graphics objects. Section 6.11 will present an alternative method, utilizing the figure's plotting tools. Section 6.5 will show how to utilize different fonts and use TeX and LaTeX document markup languages, to represent symbolic expressions. Before we proceed further, let us briefly address how to change the properties of the line itself (corresponding to the second group of the plot content, curve).

6.3 Line Specifications

As shown in Table 6.1 for the `plot` function, a line specification, `LineSpec`, allows you to determine such properties as the line style, color, and marker symbol (many other plotting functions accept a `LineSpec` argument as well). To this end, Table 6.5 presents all options MATLAB has to offer (note that the circle and all subsequent data markers are fillable markers, meaning that you can fill them with color, not necessarily coinciding with the color of the edges).

You may specify the `LineSpec` components as a character vector or string scalar in any order (as seen from Table 6.5, they are all unique) or even omit some of them. For example,

```
>> plot([0:10].^2,'g>:')
```

plots a parabola using a dotted line (`:`), places right triangle markers (`>`) at the data points, and colors both the line and markers green (`g`) (Fig. 6.7a). Another command

```
>> fplot(@tanh,[-pi, pi]/2,"md")
```

Table 6.5 `LineSpec` Options

Line Types		Data Markers		Colors	
Solid line	-	Dot (.)	.	Black	k
Dashed line	--	Asterisk (*)	*	Blue	b
Dash-dotted line	-.	Cross (x)	x	Cyan	c
Dotted line	:	Plus sign (+)	+	Green	g
		Circle (o)	o	Magenta	m
		Square (□)	s	Red	r
		Diamond (◇)	d	White	w
		Pentagram (five-point star)	p	Yellow	y
		Hexagram (six-point star)	h		
		Left triangle (◀)	<		
		Up triangle (▲)	^		
		Right triangle (▶)	>		
		Down triangle (▼)	v		

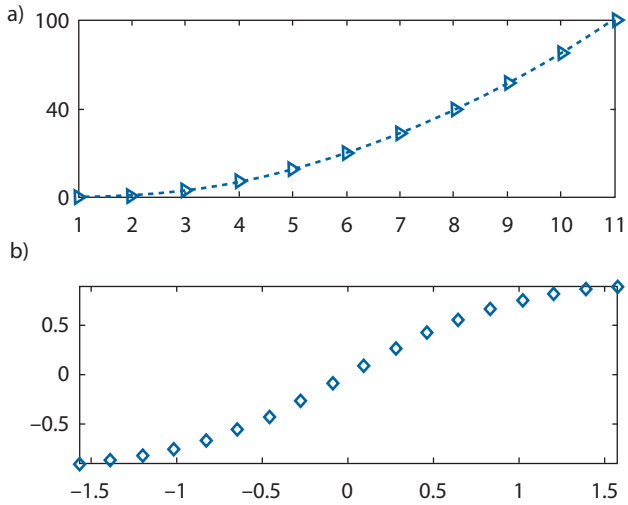


Fig. 6.7 Examples of using the `LineSpec` options.

produces a plot of a hyperbolic tangent within the $-\frac{1}{2}\pi \le x \le \frac{1}{2}\pi$ interval with the data points marked by magenta diamond markers (Fig. 6.7b).

When using the `plot` function (as well as the `fplot` and some other functions), you may also specify other characteristics of lines using their graphics properties as additional (optional) name–value pair arguments. Main properties, beyond those addressed in Table 6.5 already, are listed in Table 6.6. (The list of all Line object properties will be discussed in the next section.) For example,

```
>> x = -pi:pi; y = sin(x);
>> plot(x,y,'Color','r','Marker','p','LineWidth',3,'MarkerSize',7)
```

plots `y` vs `x` using a three-point-wide red-color line with a 7-point-size pentagram marker as shown in Fig. 6.8a. Note that `Color` and `Marker`

Table 6.6 Line Graphics Properties

Function	Brief Description
`LineWidth`	Specifies the line's width (by default, it uses *points*, where 1 point is equal to 1/72 of an inch)
`MarkerEdgeColor`	Specifies the color of markers or the edge color for filled markers (circle, square, diamond, pentagram, hexagram, and the four triangles)
`MarkerFaceColor`	Specifies the color of a face of filled markers
`MarkerSize`	Specifies the size of marker in points

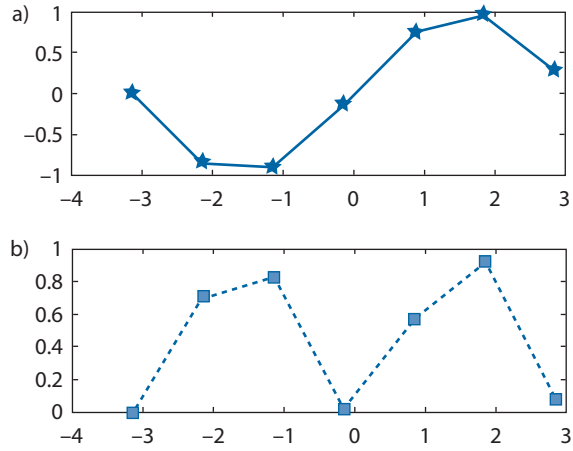


Fig. 6.8 Examples of varying a line specification.

properties were specified as name–value pair arguments rather than using the `LineSpec` argument as before. On the contrary, the command

```
>> plot(x,y.^2,'--s','MarkerSize',10,'MarkerFaceColor','r')
```

blends both ways of specifying line properties together—via `LineSpec`, defining the dashed line and square marker, and via changing some of the other properties explicitly (Fig. 6.8b).

The vector `x` in the previous example was created using a small number of elements on purpose. Obviously, this is one reason the curves in Fig. 6.8 look ugly. The logical move is to increase the number of points. Using 100 data points usually yields a good result. For example,

```
>> x=linspace(0,pi); plot(x,cos(6*x).*exp(-x),'--^')
```

produces the plot shown in Fig. 6.9a. However, marking all 100 points might not be a good idea.

There may be different reasons for adding markers (for example, distinguishing between the multiple curves), but usually you need just a few of them. The following two commands show how to achieve this using two separate sets of the data points (Fig. 6.9b):

```
>> x=linspace(0,pi); xm=linspace(0,pi,20);
>> p=plot(x,cos(6*x).*exp(-x),xm,cos(6*xm).*exp(-xm),'^');
```

Note that the `plot` function in the latter example uses a syntax that allows plotting multiple curves with just one command (to be discussed in detail in Sec. 6.6).

The data tip at the 5th index in the plotted data set in Fig. 6.9b was added to the second plot (showing markers only) using one additional command

```
>> datatip(p(2),'DataIndex',5,'Location','southeast');
```

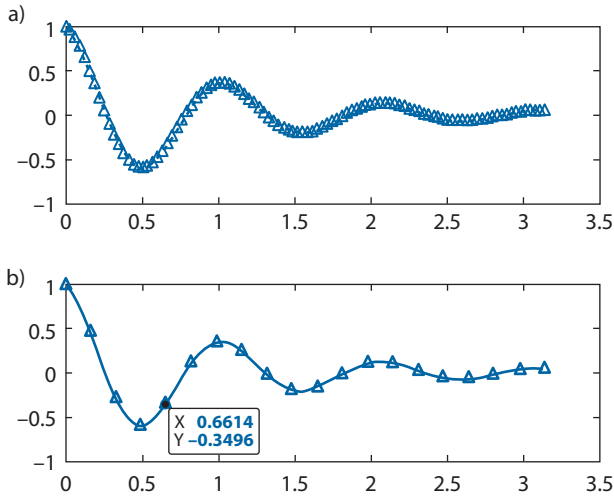


Fig. 6.9 a) Marking all data points and b) a preferred way of marking some data points.

(The `datatip` function was introduced in R2019b.) In this command, `p(2)` specifies the second graphics object (a line) the data tip is added to, and that is why the previous command had a `p=plot(...)` syntax. The second (optional) name-value pair argument specifies a relative position of the data tip with respect to the data point. The graphics objects are considered in the very next section.

6.4 Accessing Plot Properties via the `get` and `set` Functions

In the previous section, it was shown how some of the default values of the line properties can be accessed and changed. Promoting the open-source paradigm, MATLAB allows you to change any other property of any graphics object, including the Figure and its components, even after it was created. To this end, Fig. 6.10 presents all possible graphics objects your figure may be composed of, also showing their hierarchy.

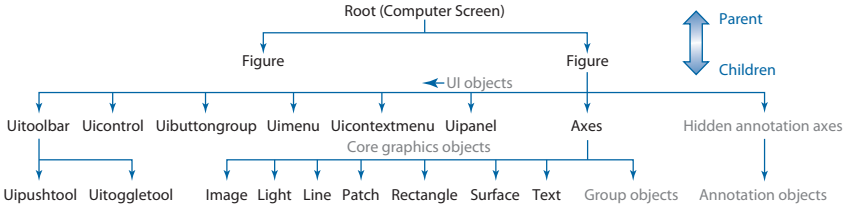


Fig. 6.10 Graphics objects hierarchy.

There are three key things about the graphics objects you should know to be able to proficiently manipulate with them programmatically (another way is to use interactive plotting tools, as discussed in Sec. 6.11):

1. Graphics objects are stored as a tree structure with *parents* and *children*. The special object at the top of the tree is called the *graphics root*.
2. Graphics objects may have multiple children but only one parent. To this end, the Figure object is the parent for the Axes object, whereas the Line object is one of the Axes object's children.
3. You can retrieve (change) properties of any graphics object by using either the dot notation that has the form `object.Property` (similar to when you refer to the field of a structure) or the `get` and `set` functions.

The simplest way to retrieve the Line object is to use the output argument when calling the `plot` (or `line`) command as we did it in the last example of the previous section. For example, the call

```
>> h_line=plot(10:-1:1);
```

does exactly what the `plot(10:-1:1)` call would do, that is, it creates a new Figure window with a single Axes object and plots the line, but it also creates a Line object. If the output has not been suppressed, you would see the most common properties displayed in the Command Window with a hyperlink you could click to see all properties. Alternatively, you can see all properties of the selected Line object issuing the `get(h_line)` command (Table 6.7).

It should be noted that prior to R2014 the aforementioned call would return a special type of number called a *graphics handle*. This handle would then be used to retrieve or change the properties of the graphics object using the `get` and `set` functions.

Before we proceed with using the dot notation to access properties of the graphics object `h_line`, let us show how to use the aforementioned two functions. Suppose you are interested only in some specific property of the object – you may recover it explicitly using `get(object,'PropertyName')` syntax of the `get` function. For example, typing

```
>> get(h_line,'Color')
```

yields

```
ans = 0 0.4470 0.7410
```

which defines the RGB components of color (to be explained in Sec. 6.9).

You can change any property in a similar manner, that is, by addressing it directly using the `set` function with a general syntax `set(object,'PropertyName',PropertyValue)`. For example,

```
>> set(h_line,'color','r','LineWidth',2.5)
```

changes the line color to red and increases the line width to 2.5 points. To see all available colors or other properties, issue the `set` command without specifying a value. For example,

```
>> set(h_line,'LineStyle')
```

Table 6.7 Properties of the Line Object with their Values

```
>> get(h_line)
```

```
AlignVertexCenters: 'off'
        Annotation: [1×1 matlab.graphics.eventdata.Annotation]
      BeingDeleted: 'off'
        BusyAction: 'queue'
     ButtonDownFcn: ''
          Children: [0×0 GraphicsPlaceholder]
          Clipping: 'on'
             Color: [0 0.4470 0.7410]
         CreateFcn: ''
   DataTipTemplate: [1×1 ... DataTipTemplate]
         DeleteFcn: ''
       DisplayName: ''
  HandleVisibility: 'on'
           HitTest: 'on'
     Interruptible: 'on'
          LineJoin: 'round'
         LineStyle: '-'
         LineWidth: 0.5000
            Marker: 'none'
   MarkerEdgeColor: 'auto'
   MarkerFaceColor: 'none'
     MarkerIndices: [1 2 3 4 5 6 7 8 9 10]
        MarkerSize: 6
            Parent: [1×1 Axes]
     PickableParts: 'visible'
          Selected: 'off'
SelectionHighlight: 'on'
               Tag: ''
              Type: 'line'
     UIContextMenu: [0×0 GraphicsPlaceholder]
          UserData: []
           Visible: 'on'
             XData: [1 2 3 4 5 6 7 8 9 10]
        XDataMode: 'auto'
      XDataSource: ''
             YData: [10 9 8 7 6 5 4 3 2 1]
      YDataSource: ''
             ZData: [1×0 double]
      ZDataSource: ''
```

returns a 5-by-1 cell array

```
{'-'   }
{'--'  }
{':'   }
{'-.'  }
{'none'}
```

(see Table 6.5).

As mentioned already, starting from R2014b, you can use the dot notation to query and set properties as well. For example,

```
>> h_line.Marker='d'
```

adds a diamond marker at all data points. Note that the property names are not case sensitive when you are using the `get` and `set` functions (in fact, you can even abbreviate them), but they are case sensitive when you use the dot notation.

Typing "`h_line.`" and pushing the <Tab> key brings up a Tab completion pop-up menu, similar to that of Fig. 2.27a, with all available properties.

Can you access the properties of the Axes object? Definitely. You have to recall that the Axes object is the parent to the Line object and, therefore, the command

```
>> h_axes=get(h_line,'parent');
```

retrieves the Axes object for you. Now, all Axes object properties can be seen by issuing the `get(h_axes)` command, the result of which is shown in Table 6.8. Obviously, the command

```
>> h_figure=get(h_axes,'parent');
```

retrieves the Figure object. For your reference, the result of the `get(h_figure)` call is presented in Table 6.9.

In fact, the Root, Figure and Axes objects are created automatically. To retrieve them, you may use `groot` (get root object), `gcf` (get current figure) and `gca` (get current axes) functions, respectively (Table 6.10). As a result, you do not even need to use an output argument in your `plot` or `line` command to begin with because you can simply retrieve the Line object from the Axes object

```
>> h_line=get(gca,'children');
```

Be cautious, however, because according to Table 6.10, requesting the `Children` property of an object composed of multiple (n) graphics objects results in an n-by-1 graphics array. For example, the commands

```
>> ezplot(@sin), hold, ezplot(@cos)
>> legend('sin(x)','cos(x)')
>> Hf=get(gcf,'Children')
```

Table 6.8 Properties of the Axes Object with their Values

```
>> get(h_axes)
                    ALim: [0 1]
                  ALimMode: 'auto'
  ActivePositionProperty: 'outerposition'
              AlphaScale: 'linear'
                 Alphamap: [1×64 double]
      AmbientLightColor: [1 1 1]
           BeingDeleted: 'off'
                     Box: 'on'
                BoxStyle: 'back'
              BusyAction: 'queue'
           ButtonDownFcn: ''
                    CLim: [0 1]
                 CLimMode: 'auto'
          CameraPosition: [5.5000 5.5000 17.3205]
      CameraPositionMode: 'auto'
           CameraTarget: [5.5000 5.5000 0]
        CameraTargetMode: 'auto'
          CameraUpVector: [0 1 0]
       CameraUpVectorMode: 'auto'
         CameraViewAngle: 6.6086
     CameraViewAngleMode: 'auto'
                Children: [1×1 Line]
                Clipping: 'on'
           ClippingStyle: '3dbox'
                   Color: [1 1 1]
              ColorOrder: [7×3 double]
          LineStyleOrder: '-'
     LineStyleOrderIndex: 1
               LineWidth: 0.5000
          MinorGridAlpha: 0.2500
      MinorGridAlphaMode: 'auto'
          MinorGridColor: [0.1000 0.1000 0.1000]
      MinorGridColorMode: 'auto'
      MinorGridLineStyle: ':'
                NextPlot: 'replace'
           OuterPosition: [0 0 1 1]
                  Parent: [1×1 Figure]
           PickableParts: 'visible'
      PlotBoxAspectRatio: [1 0.7903 0.7903]
  PlotBoxAspectRatioMode: 'auto'
                Position: [1×4 double]
              Projection: 'orthographic'
                Selected: 'off'
     SelectionHighlight: 'on'
              SortMethod: 'childorder'
                     Tag: ''
                  TickDir: 'in'
             TickDirMode: 'auto'
    TickLabelInterpreter: 'tex'
              TickLength: [0.0100 0.0250]
              TightInset: [1×4 double]
                   Title: [1×1 Text]
```

```
         ColorOrderIndex: 2
              ColorScale: 'linear'
                Colormap: [64×3 double]
              CreateFcn: ''
           CurrentPoint: [2×3 double]
        DataAspectRatio: [4.5000 4.5000 1]
    DataAspectRatioMode: 'auto'
              DeleteFcn: ''
              FontAngle: 'normal'
               FontName: 'Helvetica'
               FontSize: 10
           FontSizeMode: 'auto'
         FontSmoothing: 'on'
              FontUnits: 'points'
             FontWeight: 'normal'
              GridAlpha: 0.1500
          GridAlphaMode: 'auto'
              GridColor: [0.1500 0.1500 0.1500]
          GridColorMode: 'auto'
          GridLineStyle: '-'
       HandleVisibility: 'on'
                HitTest: 'on'
           Interactions: [1×1 ...DefaultAxesInteractionSet]
          Interruptible: 'on'
LabelFontSizeMultiplier: 1.1000
                  Layer: 'bottom'
                 Legend: [0 × 0 GraphicsPlaceholder]
```

```
TitleFontSizeMultiplier: 1.1000
        TitleFontWeight: 'bold'
                Toolbar: [1×1 AxesToolbar]
                   Type: 'axes'
          UIContextMenu: [0×0 GraphicsPlaceholder]
                  Units: 'normalized'
               UserData: []
                   View: [0 90]
                Visible: 'on'
                  XAxis: [1×1 NumericRuler]
          XAxisLocation: 'bottom'
                 XColor: [0.1500 0.1500 0.1500]
             XColorMode: 'auto'
                   XDir: 'normal'
                  XGrid: 'off'
                 XLabel: [1×1 Text]
                   XLim: [1 10]
              XLimMode: 'auto'
             XMinorGrid: 'off'
             XMinorTick: 'off'
                 XScale: 'linear'
                  XTick: [1 2 3 4 5 6 7 8 9 10]
             XTickLabel: {10×1 cell}
         XTickLabelMode: 'auto'
     XTickLabelRotation: 0
              XTickMode: 'auto'
```

The last 17 lines are repeated for Y and Z axes

Table 6.9 Properties of the Figure Object with their Values

```
>> get(h_figure)
          Alphamap: [1×64 double]
       BeingDeleted: 'off'
         BusyAction: 'queue'
      ButtonDownFcn: ''
           Children: [1×1 Axes]
           Clipping: 'on'
    CloseRequestFcn: 'closereq'
              Color: [0.9400 0.9400 0.9400]
           Colormap: [64×3 double]
          CreateFcn: ''
        CurrentAxes: [1×1 Axes]
   CurrentCharacter: ''
     CurrentObject: [0×0 GraphicsPlaceholder]
      CurrentPoint: [0 0]
         DeleteFcn: ''
        DockControls: 'on'
            FileName: ''
  GraphicsSmoothing: 'on'
   HandleVisibility: 'on'
     PaperOrientation: 'portrait'
        PaperPosition: [1.3333 3.3125 5.8333 4.3750]
    PaperPositionMode: 'auto'
           PaperSize: [8.5000 11]
           PaperType: 'usletter'
          PaperUnits: 'inches'
              Parent: [1×1 Root]
             Pointer: 'arrow'
    PointerShapeCData: [16×16 double]
PointerShapeHotSpot: [1 1]
            Position: [1440 918 560 420]
            Renderer: 'opengl'
        RendererMode: 'auto'
              Resize: 'on'
          Scrollable: 'off'
       SelectionType: 'normal'
      SizeChangedFcn: ''
                 Tag: ''
             ToolBar: 'auto'
```

```
      InnerPosition: [1440 918 560 420]
      IntegerHandle: 'on'
      Interruptible: 'on'
    InvertHardcopy: 'on'
       KeyPressFcn: ''
     KeyReleaseFcn: ''
           MenuBar: 'figure'
              Name: ''
          NextPlot: 'add'
            Number: 1
       NumberTitle: 'on'
     OuterPosition: [1432 910 576 513]
```

```
                Type: 'figure'
     UIContextMenu: [0×0 GraphicsPlaceholder]
             Units: 'pixels'
          UserData: []
           Visible: 'on'
  WindowButtonDownFcn: ''
WindowButtonMotionFcn: ''
    WindowButtonUpFcn: ''
    WindowKeyPressFcn: ''
  WindowKeyReleaseFcn: ''
 WindowScrollWheelFcn: ''
       WindowState: 'normal'
       WindowStyle: 'normal'
```

Table 6.10 Alternative Ways of Retrieving Graphics Objects

Command	Brief Description

`h=gcf`
`h=get(groot,'CurrentFigure')`
`h=get(gca,'Parent')`
`h=get(groot,'Children')`
	Returns the current Figure object with properties as defined in Fig. 6.9. In the case when *n* Figure windows are opened, the last syntax returns an *n*-by-1 Figure object array. You can use Figure object to access/change any figure property, e.g., `h(1).Color = 'w'` or `set(h(1),'Color','w')` changes Figure 1 color

`h=gca`
`h=get(gcf,'CurrentAxes')`
`h=get(gcf,'Children')`
`h=get(get(groot,'Children'),...`
`'Children')`
	Returns the current Axes object with properties as defined in Fig. 6.8. When the current figure has *m* children objects, such as multiple Axes, UI and Annotation objects (e.g., the Legend and ColorBar objects) (Fig. 6.10), the third syntax returns an *m*-by-1 graphics array. Assuming `h(2)` is a legend, `set(h(2),'Color','y')` or `h(2).Color='y'` changes its color. When *n* Figure windows are opened with multiple children objects in each of them, the last call returns an *n*-by-1 cell array. The property-changing syntax becomes a little bit more complicated, e.g., `set(h{1}(2),'Color','y')` or `h{1}(2).Color='y'`

`h=get(gca,'Children')`	Returns a graphics array that enables accessing/changing (properties of) all children objects of the current Axes object, including the Line(s), Text(s), etc. (Fig. 6.10)

return a 2-by-1 graphics array

```
Hf =
 2×1 graphics array:
 Legend (sin(x), cos(x))
 Axes   ({cos})
```

and the plot shown in Fig. 6.11a (the `hold` and `legend` functions will be discussed in Sec. 6.6).

You can retrieve the `Children` property of the Axes object as well. Either

```
>> Ha1=get(gca,'Children')
```

or

```
>> Ha1=allchild(gca)
```

returns a 2-by-1 Line array

```
Ha1 =
  2×1 Line array:
  Line (cos(x))
  Line (sin(x))
```

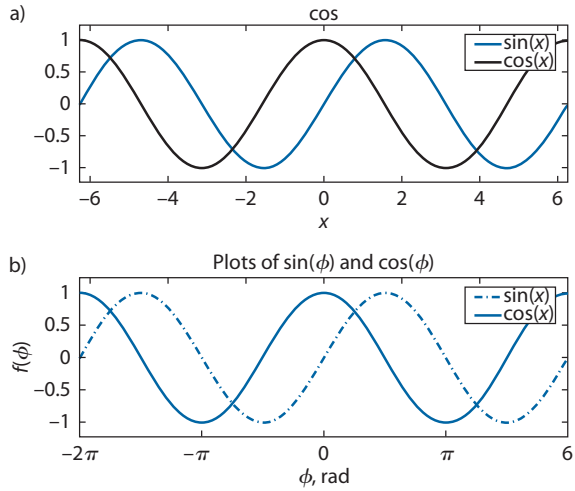


Fig. 6.11 Example of changing properties of already existing graphics objects.

As opposed to the `allchild`, the `findall` function returns even the hidden objects in the hierarchy under the specified object

```
>> Ha2=findall(gca)
Ha2 =
   5×1 graphics array:
   Axes ({cos})
   Line (cos(x))
   Line (sin(x))
   Text ({cos})
   Text ({x})
```

You can limit the `findall` function output by specifying the object's type. For example,

```
>> Ha3=findall(gca,'type','text')
```

yields

```
Ha3 =
   2 × 1 Text array:
   Text  ({cos})
   Text  ({x})
```

Let us show you a few more examples of how to modify certain properties of the graphics objects retrieved using the aforementioned commands. The calls

```
>> set(Ha1,'Color','b');
>> Ha1(2).LineStyle='--';
>> Ha3(1).String='Plots of sin(\phi) and cos(\phi)';
>> ax=gca;
>> ax.XTick=[-2*pi:pi:2*pi];
>> ax.XTickLabel={'-2\pi';'-\pi';'0';'\pi';'2\pi'};
>> xlabel('\phi, rad'), hyl=ylabel('f(\phi)');
```

modify the plot shown in Fig. 6.11a to that in Fig. 6.11b. Note how the `set` function in the first command was useful to change the `Color` property for both lines. You could not do it using the dot notation. On the contrary, the dot notation could be useful if you want to set properties of two objects to be the same; for example, you could have used

```
>> Ha1(1).Color=Ha1(2).Color;
```

Also note how we created a Text object, `hyl`, in the last command. We did it with the sole purpose of showing the `Interpreter` property of the first object

```
>> disp(hyl.Interpreter)
tex
```

As seen from Table 6.8, the `TickLabelInterpreter` property of the Axes object has the same value as well. That is what enabled using the TeX language to reproduce the Greek alphabet letters π and ϕ in Fig. 6.11b.

6.5 Text Strings

As shown in the previous section, when the text `Interpreter` property is set to `Tex` (the default) or `Latex`, you can use a subset of TeX (LaTeX) commands embedded in the `String` property of any Text object to produce special characters such as the Greek alphabet letters and mathematical symbols as shown in Table 6.11. All the functions adding textual descriptions to your plot, like `text`, `title`, `sgtitle`, `xlabel`, and `xticklabels`, can benefit from this feature. When `Interpreter` is set to `none`, no characters in the `String` are interpreted, and everything is displayed as typed. You can also use text stream modifiers controlling text font as shown in Table 6.12.

The first four modifiers of Table 6.12 are mutually exclusive. However, you can use `\fontname` in combination with one of the other modifiers. Text stream modifiers remain in effect until the end of the string, until the next font modifier, or only within the context defined by braces `{}`.

The underscore character "`_`" and the superscript character "`^`" modify the character or substring defined in braces immediately following it. To display the special characters used to define the Tex strings, when `Interpreter` is `Tex`, prefix them with the backslash character, "`\`", for instance: `\\`, `\{`, `\}`, `\_`, `\^`.

All aforementioned options are summarized in the following illustrative example. The script

```
fplot(@(t)0.25*exp(-0.006*t).*sin(0.2*t),[0 900])
a='\fontsize{12}{\itAe}^{-\alpha\itt}sin\beta{\itt}';
b='   \\ \alpha<<\beta';
title(strcat(a,b))
```

Table 6.11 Special Symbols

Character Sequence	Symbol	Character Sequence	Symbol	Character Sequence	Symbol
\alpha	α	\upsilon	υ	\sim	$\sim$
\beta	β	\phi	ϕ	\leq	$\leq$
\gamma	γ	\chi	χ	\infty	∞
\delta	δ	\psi	ψ	\clubsuit	$\clubsuit$
\epsilon	ϵ	\omega	ω	\diamondsuit	$\diamondsuit$
\zeta	ζ	\Gamma	Γ	\heartsuit	$\heartsuit$
\eta	η	\Delta	Δ	\spadesuit	$\spadesuit$
\theta	θ	\Theta	Θ	\leftrightarrow	$\leftrightarrow$
\vartheta	ϑ	\Lambda	Λ	\leftarrow	$\leftarrow$
\iota	ι	\Xi	Ξ	\uparrow	$\uparrow$
\kappa	κ	\Pi	Π	\rightarrow	$\rightarrow$
\lambda	λ	\Sigma	Σ	\downarrow	$\downarrow$
\mu	μ	\Upsilon	Υ	\circ	$\circ$
\nu	ν	\Phi	Φ	\pm	$\pm$
\xi	ξ	\Psi	Ψ	\geq	$\geq$
\pi	π	\Omega	Ω	\propto	$\propto$
\rho	ρ	\forall	$\forall$	\partial	∂
\sigma	σ	\exists	$\exists$	\bullet	$\bullet$
\varsigma	ς	\ni	$\ni$	\div	$\div$
\tau	τ	\cong	$\cong$	\neq	$\neq$
\equiv	$\equiv$	\approx	$\approx$	\aleph	$\aleph$
\Im	$\Im$	\Re	$\Re$	\wp	$\wp$
\otimes	$\otimes$	\oplus	$\oplus$	\oslash	$\oslash$
\cap	$\cap$	\cup	$\cup$	\supseteq	$\supseteq$
\supset	$\supset$	\subseteq	$\subseteq$	\subset	$\subset$
\int	$\int$	\in	$\in$	\o	o
\rfloor	$\rfloor$	\lceil	$\lceil$	\nabla	∇
\lfloor	$\lfloor$	\cdot	$\cdot$	\ldots	...
\perp	$\perp$	\neg	$\neg$	\prime	$'$
\wedge	$\wedge$	\times	$\times$	\0	$\emptyset$
\rceil	$\rceil$	\surd	$\surd$	\mid	$\mid$
\vee	$\vee$	\varpi	ϖ	\copyright	©
\langle	$\langle$	\rangle	$\rangle$		

```
xlabel('\bf\itTime\rm, \musec')
ylabel('\fontname{Times}\fontsize{12}Amplitude')
```

produces the plot shown in Fig. 6.12a (the concatenation of two strings, a and b, was used with the only goal of avoiding a long line).

Table 6.12 Text Stream Modifiers

Modifier	Brief Description
`\bf`	Forces using bold font
`\it`	Forces using italics font
`\sl`	Forces using oblique font (if available)
`\rm`	Forces using normal font
`\fontname{fontname}`	Specifies the name of the font family to use
`\fontsize{fontsize}`	Specifies the font size (in font units)

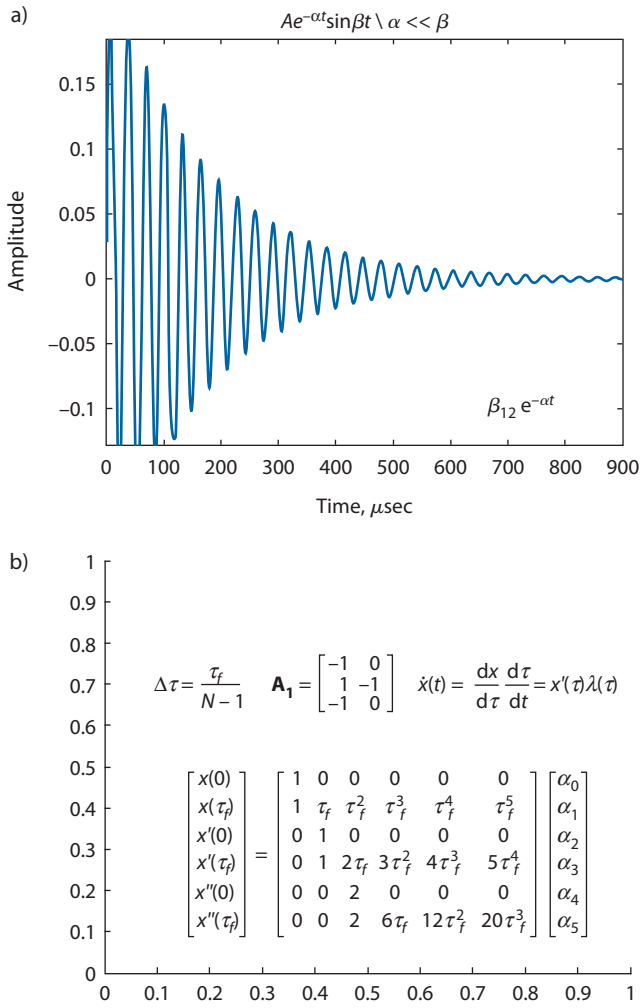


Fig. 6.12 Example of the enhanced text using a) TeX and b) LaTeX language.

You may also use the `texlabel` function that automatically converts a character vector or string scalar involving Greek variable names (lambda, delta, and so on) into the TeX equivalent to be used in the annotation functions of Table 6.3. For example,

```
>> text(750,-0.1,texlabel('beta12*e^{-alpha*t}'))
```

is essentially equivalent to

```
>> text(750,-0.1,'{\beta_{12}} {e}^{-{\alpha} {t}}')
```

and results in an additional annotation in the lower-right corner of Fig. 6.12a.

Even more options are available using the LaTeX language (see Appendix A). For instance, you can visualize fractions, matrices, and derivatives as shown in Fig. 6.12b. The LaTeX sequence should be embraced with a double dollar sign, `$$`, and the LaTeX interpreter specified explicitly. The following commands used to produce the first line of expressions in Fig. 6.12b provide some self-explanatory examples (again, concatenation is used to avoid long character vectors):

```
t1='\Delta\tau={\tau_f\over N-1}';
text(0.1,0.7,['$$' t1 '$$'],'Interpreter','latex')
t2='{\bf A_1}=\left[\matrix{-1&0\cr 1&-1\cr -1&0\cr}\right]';
text(0.35,0.7,['$$' t2 '$$'],'Interpreter','latex')
t3='\dot x(t)={dx\over d\tau}{d\tau\over dt}=x''(\tau)\lambda(\tau)';
text(0.65,0.7,['$$' t3 '$$'],'Interpreter','latex')
```

By the way, rather than using the `axis off` call (Table 6.2) to make the Axes object in Fig. 6.12b invisible, you may now change its corresponding property (see Table 6.8)

```
>> set(gca,'Visible','off')
```

As mentioned in Sec. 5.12, LaTeX expressions can also be used in the comment sections of your *m*-scripts to improve readability when publishing them. In Chapter 15, you will see that TeX commands can be used in Simulink models as well (for exactly the same reason, that is, improving readability). Since using symbolic expressions does serve a good purpose, MATLAB even offers the `latex` function that converts a symbolic expression to the LaTeX representation, to be further used to annotate (live) scripts and plots (to be covered in Sec. 7.9.1).

6.6 Overlays, Legends, Subplots, and Multiple Figures

So far (excluding Figs. 6.9b and 6.11), we have only had a single Line object per figure. Of course, MATLAB allows you to do more, having several Line objects within one Axes object, creating several Axes objects within one Figure window and spreading your graphical data between multiple Figure windows.

Table 6.13 Syntaxes of the `plot` Function Enabling Overlays

Modifier	Brief Description
`plot(A)`	Plots the columns of the *m*-by-*n* matrix `A` vs their indices generating *n* curves, one for each column
`plot(x,A)` or `plot(A,x)`	Generates *m* or *n* plots depending on the length of vector `x` (`A` is the *m*-by-*n* matrix and `x` is the vector of length *m* or *n*)
`plot(A,B)`	Plots the columns of the matrix `B` vs the columns of matrix `A` (`A` and `B` are both *m*-by-*n* matrices)
`plot(x1,y1,x2,y2,... LineSpec,x3,y3,...)`	Plots vector `y1` vs vector `x1`, `y2` vs `x2`, `y3` vs `x3`, etc., in one call (you can mix `xn-yn-LineSpec` triplets with `xn-yn` pairs relying on default specifications)

Let us start with the *overlay* plots. An overlay plot is the one in which multiple lines or sets of data are shown together. The overlay plots can be created using several variants of employing the basic `plot` function as shown in Table 6.13.

As you may recall, we have used the last syntax of Table 6.13 to create a plot shown in Fig. 6.9b already. Let us explore other syntaxes. The commands

```
>> a=1:10;
>> b=rand(10,3);
>> plot(a,b)
```

produce a plot shown in Fig. 6.13a. Note that in this case to distinguish between multiple lines, the `plot` function uses different colors, which is good but may not be enough. If you want to change a line style or add a

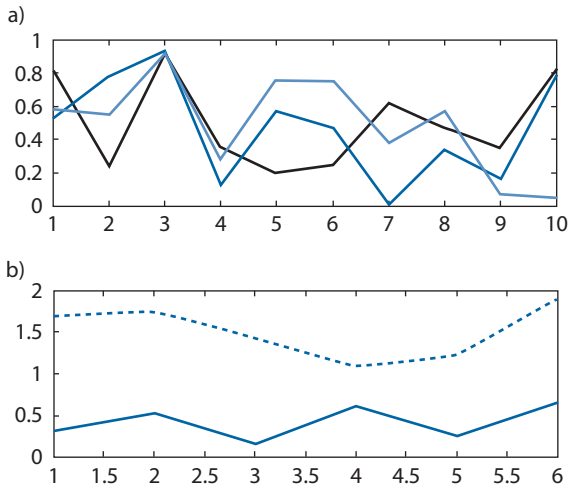


Fig. 6.13 Examples of the overlay plots.

marker, the only option you have is to access and change these line's properties using the Line objects. In this particular case, the commands

```
>> hl=get(gca,'Children');
>> set(hl(2),'LineStyle','--')
```

retrieve a 3-by-1 Line object array `hl` containing the properties of all three lines, and change the `LineStyle` property of the second Line object.

The overlay plots can also be created using the `line` function following the `plot` command (the `line` command does not create a new figure or set of axes, it just adds a line to the current axes). For example, the following two commands

```
>> plot(rand(1,6))
>> line(1:6,rand(1,6)+1,'LineStyle','--')
```

result in overlay plots shown in Fig. 6.13b. Similar to the `plot` function, if `x` and `y` are matrices of the same size, the call `line(x,y)` adds multiple lines, one line per column. Let us make two comments here. First, note that the `line` function does not support `x-y-LineSpec` triplets, as the `plot` function does, so you have to specify line properties explicitly by using one or more name–value pair arguments. Second, both `x` and `y` values have to be specified explicitly, and that constitutes another difference with the `plot` function, where you can plot a vector vs its indices.

As an alternative to using the `line` function, you may add multiple lines or sets of data to the current plot by issuing multiple `plot` commands. However, in this case, you must hold the originally drawn axes by using the `hold` function. Specifically, use `hold on` to turn the hold feature on and `hold off` to turn it off. When used by itself, `hold` toggles the hold switch `on` or `off`. When `hold` feature is `on`, any `plot` command will add the line(s) to the current axes. In this regard, the result of the two commands

```
>> plot(rand(1,6)), hold on
>> plot(rand(1,6)+1,'--')
```

is identical to that of the previous two commands that employed the `line` function. Note that, you are only required to issue the `hold` command once, after plotting the very first line. For example, in the latter case you may continue adding more lines using the `plot` function without issuing additional `hold` commands.

There is one more function allowing you to not only plot two curves within one axis but also to use a different *y*-axis for each of them. The `yyaxis` function introduced in R2016a activates either a left or right *y*-axis (prior to R2014a you would use the `plotyy` function). For example, the script

```
x=linspace(0,15);
y1=10*exp(-0.36*x).*sin(2.7*x);
y2=200*exp(-0.05*x).*sin(0.8*x);
```

```
yyaxis left, plot(x,y1), ylabel('Short Period motion (\alpha, ^o)')
yyaxis right, plot(x,y2,'--'), ylabel('Phugoid motion (h, ft)')
xlabel('Time, s'), title('T-37 Longitudinal Dynamics')
```

produces the plot shown in Fig. 6.14a.

The Statistics and Machine Learning Toolbox features a few useful functions that allow you to add the reference lines to an existing plot. They are the `refline` and `refcurve` (adding a straight line or polynomial in the current axes), `lsline` (superimposing a least-squares line on each scatter plot), and `gline` (drawing a line segment by clicking the pointer at the two end points). Although you may not have access to these functions, let us show how easy it is to create your own function that does something similar. To this end, create and save the following code as the *horline.m* script:

```
function y=horline(Y,sp)
def= 'k--';                    % default settings
if nargin==0; Y=0;       end % no input arguments
if nargin<=1; sp=def; end     % less than two input arguments
   linecolors='rgbcmykw';    % define line colors
   [c,n]=intersect(sp,linecolors);          % find color specification
if isempty(n), c=def(1); else, sp(n)=[]; end % no color
if isempty(sp), sp=def(2:3); end             % no line style
   yy=xlim;                    % find x-limits
   line(yy,Y* [1 1],'Color',c,'LineStyle',sp)
```

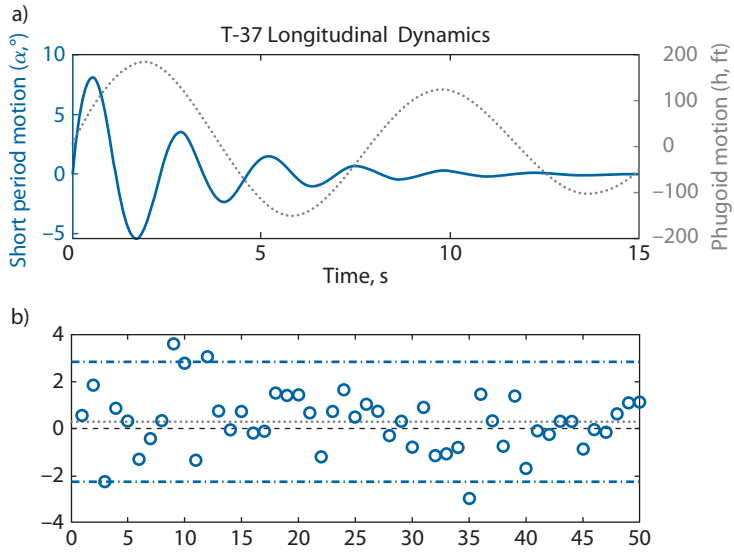


Fig. 6.14 a) Example of using the `yyaxis` function and b) adding the reference lines to the existing plot.

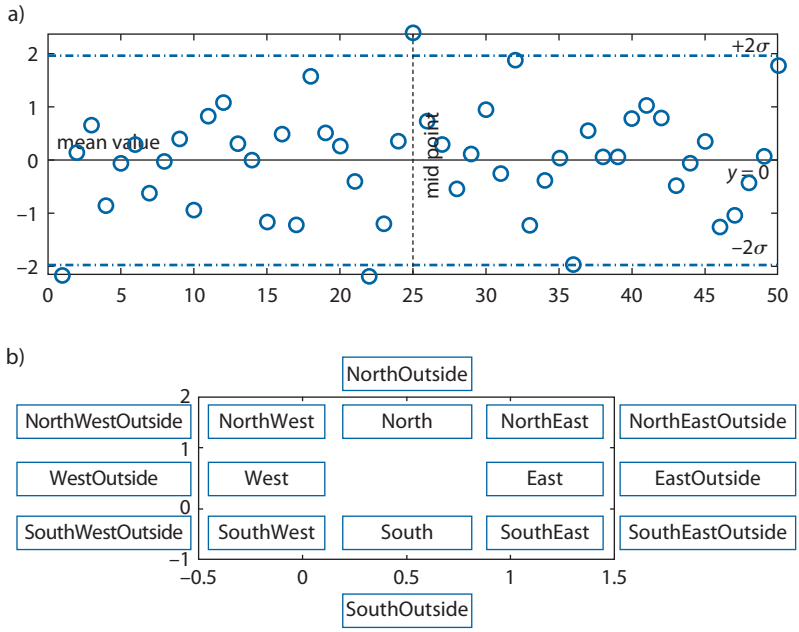


Fig. 6.15 a) Illustration of employing the `xline` and `yline` functions; and b) visualization of legend locations.

Now, you can use it to produce a horizontal line and specify its color and line style if necessary. For example, the following commands result in the plot shown in Fig. 6.14b:

```
a=randn(50,1); plot(a,'o')
horline(mean(a),'--c')
horline(mean(a)+2* std(a),'--b-.')
horline(mean(a)-2* std(a),'--b')
horline
```

In fact, in R2018b, two functions, `xline` and `yline`, were added to basic MATLAB to do exactly that. The following self-explanatory example mimics the previous code and demonstrates the capability to add and modify labels (Fig. 6.15a):

```
a=randn(50,1); plot(a,'o')
  h1=yline(mean(a),':r','mean value');
  h1.LabelHorizontalAlignment= 'left';
yline(mean(a)+2* std(a),'b--','+2\sigma');
yline(mean(a)-2* std(a),'b--','-2\sigma');
  h2=xline(25,'k--','midpoint');
  h2.LabelVerticalAlignment= 'middle';
yline(0,'Label','y=0','LabelVerticalAlignment','bottom');
```

When having multiple lines within one axes, you would definitely want to add a legend. The `legend` function allows you to do this using several syntaxes, as shown in Table 6.14. Along with the legend locations shown in Fig. 6.15b and specified as character vectors or strings, two additional locations, `'best'` and `'bestoutside'`, allow you to place the legend inside axes, where the least conflict occurs with plot data, and outside the upper-right corner of the axes, respectively. The `Lcn` character sequence is not case sensitive and can be abbreviated by sentinel letter (for example, `'S'`, `'se'`, and `'NwO'`).

As of R2018a, other name–value pair arguments are available for all three syntaxes of Table 6.14 as well. Using them, you can change text color (`'TextColor'`), background color (`'Color'`), number of columns (`'NumColumns'`), orientation (`'Orientation'`), font size (`'FontSize'`), etc. For example, `legend(...,'Orientation','horizontal')` forces the legend items to be displayed side by side (as opposed to the default `'vertical'` orientation), and `legend(...,'NumColumns',3)` creates a three-column legend item list. Surely, having too many lines (graphic objects) would result in the legend box occupying most of the plot space, so as a precaution, `legend` used to only depict the first 20 objects in the graph (this constraint has been eliminated in R2019b).

Starting from R2017a, `legend` automatically updates when you add or remove data series from the axes. If you do not want `legend` to automatically update, set the `'AutoUpdate'` property of a legend to `'off'`. If you did not request the output argument when creating a legend, that is, you did not create a Legend object, you can still retrieve the latter as one of the Figure object's children. As an example, the piece of code

```
h1=fplot(@sin,'--'); hold
fplot(@(x)sin(x).*cos(x),'--');
h2=fplot(@cos,'--');
legend([h1 h2],{"sin(x)" "cos(x)"})
hlgd=findall(gcf,'type','legend');
hlgd.AutoUpdate='off';
fplot(@(x)sin(x)+cos(x))
```

Table 6.14 Common Syntaxes of the `legend` Function

Modifier	Brief Description
`legend(lbl1,lbl2,...,...` `'Location',lcn)`	Creates a legend with descriptive labels `lbl1`, `lbl2`,... for each plotted data series at a specified location `lcn` (Fig. 6.15b). The labels are specified as character vectors or strings, e.g., `'South'` or `'West'`
`legend(labels,...)`	Specifies the labels as a single cell array of character vectors, a string array, or a character array `labels`
`legend(subset,labels,...)`	Only includes items in the legend for the data series listed in a vector of graphics objects `subset`

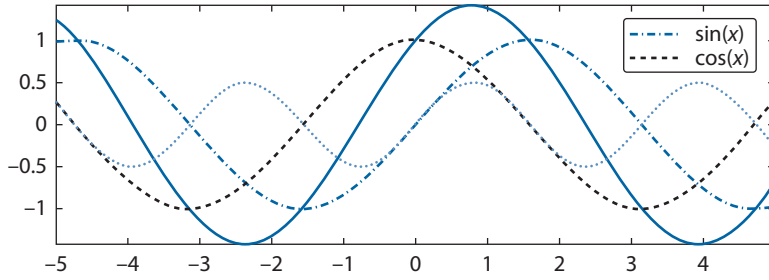


Fig. 6.16 Creating a legend for the specified two (of three) data sets.

produces the plot, shown in Fig. 6.16. It uses the third syntax of Table 6.14 specifying two of three data sets to show the legend for explicitly, and it also turns the `AutoUpdate` property of the legend retrieved in `hlgd` off. Without it, the fourth data set added to the plot after creating a legend would automatically update the legend to include one more item. [By the way, if needed, you can create a Group object, which includes multiple graphics objects, as a child of the current axes (Fig. 6.10) using the `hggroup` function, but this falls beyond the scope of this textbook.]

MATLAB also allows you to divide the Figure window into an array of rectangular panes and use each one (or the rectangular group of them) to produce the separate axes. The `subplot` function, which makes this possible, has the following syntaxes:

`subplot(m,n,p)` or `subplot(mnp)` or `subplot mnp`

where `m` is the number of panes rows, `n` the number of columns of panes, and `p` the pane number counting sequentially across a top row, then to the next row, and so on.

The subplot function allows you to produce asymmetrical arrangements of subplots by combining two and more rather than just one pane together. For example, the command

```
subplot(2,2,[2 4])
```

combines the second and fourth panes, that is, allows you to use the right column of panes together. Of course, you can always specify a location of axes explicitly:

```
subplot('Position',[x y width height])
```

Here, a four-element position vector provides the location (x- and y-coordinates) of a lower-left corner of the Axes object, along with its width and height. As usual, by default these values are given in normalized coordinates in the range from 0 to 1. The following example explains the aforementioned options all together and produces a Figure window with multiple panes as shown in Fig. 6.17a:

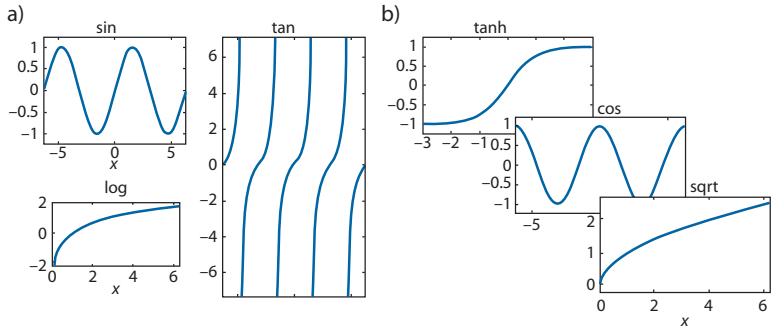


Fig. 6.17 Examples of using a) `subplot` and b) `axes` functions.

```
>> subplot(221), ezplot(@sin)
>> subplot(2,2,[2,4]), ezplot(@tan)
>> subplot('Position',[0.15,0.25,0.3,0.2]), ezplot(@log)
```

By the way, now we can reveal a small secret—in all preceding figures of this chapter, except Figs. 6.4 and 6.12, we used the `subplot(211)` command to halve the height of a plot.

Be aware that when you use an explicit four-element position vector and the location of a new subplot interferes with what you have plotted already, your new subplot eliminates all underlying subplots. For instance, if in the preceding case you use `[0.35,0.45,0.3,0.2]`, that is, move your last (log) plot toward an upper-right corner, it will eliminate both other plots.

In R2019b, the two new functions to create a tiled layout for displaying multiple plots and navigate through it, `tiledlayout` and `nexttile`, were introduced. For example, the commands

```
x=linspace(0,2*pi);
tiledlayout(2,2)
nexttile,       plot(x,sin(x))
nexttile([2 1]), plot(x,tan(x))
nexttile(3),    plot(x,log(x))
```

produce a grid of subplots similar to that of Fig. 6.17a. In this example, the `tiledlayout` command creates a tiled 2-by-2 layout for displaying multiple plots in a figure, the simplest syntax, `nexttile`, prepares an axes object in the next empty tile of a tiled layout, `nexttile(span)` syntax creates an axes object that spans multiple tiles (the vector [*r c*] defines the tile spanning *r* rows and *c* columns of tiles in the layout), and `nexttile(tilenumber)` syntax specifies the tile number to place the axes into. The good thing about these two new functions is that the `tiledlayout` function also enables two name-value pair arguments, `'TileSpacing'`-`'compact'`

and `'Padding'-'compact'`, minimizing spacing between the tiles and around the perimeter of the layout, respectively.

If you want to produce the overlapping plots like the ones shown in Fig. 6.17b, you should use the `axes` command:

```
>> axes('Position',[0.1 0.6 0.4 0.3]), ezplot(@tanh)
>> axes('Position',[0.3 0.35 0.4 0.3]), ezplot(@cos)
>> axes('Position',[0.5 0.1 0.4 0.3]), ezplot(@sqrt)
```

Figure 6.18 shows another example featuring the inserted (nested) Axes objects also created using the `axes` function. When creating them, the syntax `ax=axes(...)` was used so that Axes objects `ax` could be used explicitly to specify where the plot needs to be created (in this case, all syntaxes of Table 6.1 would include `ax` as an additional first input argument that otherwise is ommited.)

One more useful function when having multiple axes is `linkaxes`, which allows synchronizing the limits of specified 2-D axes. Specifically, the call `linkaxes(ax, 'xy')`, where multiple Axes objects are specified in the vector `ax`, links both *x*- and *y*-axis (other available options are `'x'`, `'y'`, and the default `'off'`). Another function, `linkprop`, allows linking (synchronizing) other properties of the different graphics objects.

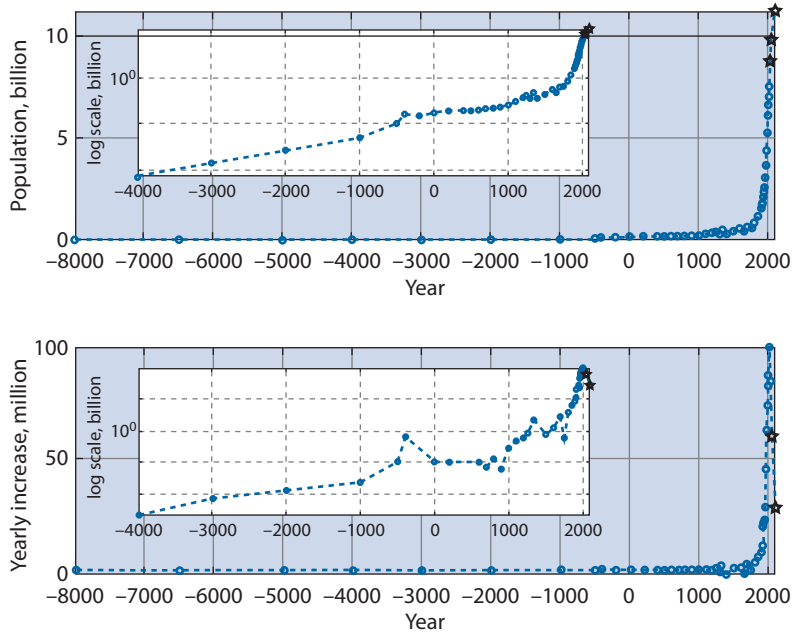


Fig. 6.18 Examples of the nested Axes objects.

Finally, if you have too much data to fit on a single figure, you can dedicate several figures to display them. The `figure` function creates a new Figure object. The simplest syntax,

```
>> figure
```

creates a new Figure window using the default property values (each Figure window will automatically be numbered in numerical order with the default title **Figure X**, where X is its number). If needed, you may specify the Figure number explicitly, for example,

```
figure(num)
```

where `num` should be an integer. If the figure with this number exists already, it becomes the current figure, so that the subsequent plots are drawn in it by default, that is, you may switch back and forth between multiple opened Figure windows. By the way, prior to R2014b, the `num` designator happened to be a handle of the Figure, so that the following commands were (and as of R2020a still are) valid:

```
>> h=4; figure(h); get(h); set(h,'Color','y')
```

Of course, you can specify any property of the Figure object when creating it explicitly

```
figure('PropertyName',PropertyValue,...)
```

All other properties, not defined explicitly, maintain their default values. For example, the command

```
>> figure('NumberTitle','off','Name','New Window','Color','w')
```

suppresses numbering, creates a figure name, and forces white background (as opposed to the default gray) (Fig. 6.19). This latter feature may be especially useful if you want to print your plots afterward (which helps in saving toner).

Recall, we tried to enforce a white background already by changing Figure Copy Options preferences (see Fig. 2.32a). However, there is a big difference between these two approaches. If you want to copy your Figure object (not Figure window) and paste it into a Microsoft Word document or

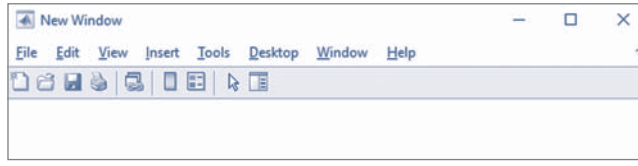


Fig. 6.19 Example of manipulating Figure's properties.

Microsoft PowerPoint slide, you will need to choose the **Edit** drop-down menu from the Figure window toolbar and click **Copy Figure**. This copies the content of the figure with its background. So if you do not force it to be white, it will appear gray. That is where you may choose to change Figure Copy Options preferences (just once). Nevertheless, even if you changed this option as explained in Fig. 2.32a and try to publish your *m*-script that has the `figure` commands in it, you will end up having a gray background. The preceding alternative of changing a background color while creating a figure, `figure('Color','w')`, saves you from worrying about it once and forever.

The `close` function (Table 2.12) deletes the current figure, the specified figure (`close(h)`), or all figures (`close all`). The `clf` function (abbreviated from clear current figure) clears the current Figure window (deletes all graphics objects) and also resets all of its properties such as `hold`, `axis`, etc. The `cla` function (clear axes) deletes all the lines and text from the current axes (without resetting the `axes` properties).

The last function to be introduced in this section is `findobj`. The call

```
H=findobj('PropertyName',PropertyValue,...)
```

returns all graphics objects having the property `PropertyName` set to the value `PropertyValue`. You can specify more than one name–value pair argument, in which case, `findobj` returns only those objects having all specified property values. For example, the set of commands

```
>> x=linspace(-pi,pi,50);
>> hp=plot(x,sin(x),x,cos(x),'rp',x,sin(x).*cos(x),'m+:');
>> legend([hp(1) hp(3)],'f_1=sin(x)','f_2=sin(x)*cos(x)',...
                                        'location','ne')
```

produces the three curves shown in Fig. 6.20a (by the way, note how a vector of graphics objects was used to reduce the number of the descriptive labels for three plotted data series in the legend down to two). Now, let us find and delete one of them:

```
>> h1=findobj('Marker','p');
>> delete(h1)
```

In addition to that, let us change the background color of the legend

```
>> h2=findobj('Tag','legend');
>> h2.Color='y';
```

The result is shown in Fig. 6.20b.

Remember that `findobj(...)` searches through the Root object and all its descendants (Fig. 6.10), whereas `findobj(objhandles,...)` restricts the search to objects listed in `objhandles` and their descendants.

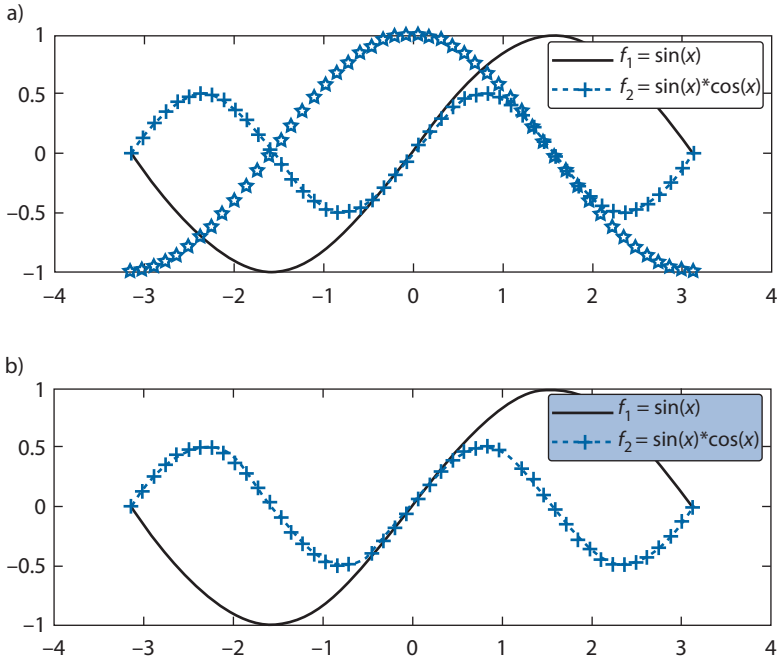


Fig. 6.20 a) Specifying a subset of plotted data series in a legend, and b) manipulating with the graphics objects by using the `findobj` function to find specific graphics objects to delete them or change their properties.

6.7 Other Types of the 2-D Plots

By now, we have covered all the fundamentals of 2-D plotting, but we have introduced only five basic plotting functions (Table 6.1). MATLAB has much more to offer. Tables 6.15–6.18 list most of the available 2-D plotting functions and Figs. 6.21–6.27a provide some illustrative examples of using them.

To begin with, Table 6.15 shows basic syntaxes of the functions that enable you to plot in the logarithmic (base-10) and polar coordinates, complemented by Figs. 6.21 and 6.22, which demonstrate some examples (for more syntaxes, refer to the MATLAB Help system).

Note that the three key polar plot functions shown in Table 6.15 (`polarplot`, `polarscatter`, and `polarhistogram`) were introduced relatively recently and represent a certain improvement over the functions used in the past (`polar` and `rose`). This includes a capability to access and change any of their properties using either the name–value pair argument or corresponding graphics object. To simplify handling polar plot r-ticks and θ-ticks (Fig. 6.23a) even further, R2016b introduced several standalone functions similar to those of Table 6.4 for the Cartesian coordinates

Table 6.15 Logarithmic and Polar Plot Functions

Function	Brief Description
`loglog(x,y)`	Plots data with the logarithmic scales for both axes
`semilogx(x,y)`	Plots data with logarithmic *x*-axis and linear *y*-axis
`semilogy(x,y)`	Plots data with linear *x*-axis and logarithmic *y*-axis
`polarplot(th,ro)`	Generates a plot in polar coordinates (θ,*r*), where `th` is the angular coordinate θ in radians and `ro` is the radial coordinate *r* (introduced in R2016a)
`polarscatter(th,ro)`	Plots `th` vs `ro`, displaying a circle at each data point (introduced in R2016b)
`polarhistogram(th)`	Creates a histogram plot in polar coordinates by sorting the values in `th` into equally spaced bins (introduced in R2016b)
`compass(u,v)`	Displays a compass graph plotted on a circular grid with arrows originating at the origin and tips defined by `u`, `v` in Cartesian coordinates. `compass(z)`, where `z` is complex, is equivalent to `compass(real(Z),imag(Z))`
`ezpolar(fun)`	Plots the polar curve $r = \text{fun}(\theta)$ over the default interval $0 \leq \theta \leq 2\pi$ (`fun` can be a function handle, a character vector, or a string)
`polaraxes`	Creates the default polar axes in the current figure. `polaraxes(Name,Value,...)` specifies properties for the PolarAxes object using one or more name–value pair arguments

Table 6.16 Stem, Stairs and Errorbar Plotting Functions

Function	Brief Description
`stem(x,y)`	Generates a stem plot of `y` vs `x`
`stairs(x,y)`	Generates a stairs plot of `y` vs `x`
`errorbar(x,y,e)`	Plots `y` vs `x` with the symmetric ($\pm$) errorbars `2*e` long (arrays `x`, `y`, and `e` must have the same size)

(`rlim`, `rticks`, `rticklabels`, `rtickformat`, `rtickangle`, `thetalim`, `thetaticks`, `thetaticklabels`, and `thetatickformat`).

The last function of Table 6.15 allows you to create the PolarAxes object, which has an extended set of properties compared with that of other polar plotting functions. Specifically, it enables changing the location

Table 6.17 Contour Plot Functions

Function	Brief Description
`contour(z)`	Displays isolines of matrix `z`. You may label the contour lines using the `clabel` function
`contourf(z)`	Displays isolines calculated from matrix `z` and fills the areas between the isolines using the constant colors (the color of the filled areas depends on the current Figure's colormap)
`fcontour(f)`	Plots the contour lines of *f*(*x*,*y*), where `f` is a function handle that represents a mathematical function of two variables, *x* and *y* (as of R2020a, you can also use `ezcontour` and `ezcontourf`)

Table 6.18 Functions for Plotting Areas, Histograms, and Vector-Type Data

Function	Brief Description
`area(x)` `area(x,y,bv)`	Plots the values in each column of matrix `x` as a separate curve and fills the area between the curve and the *x*-axis. Plots `y` vs `x` and fills the area between `bv` and `y` (the default `bv` is 0)
`pie(x)`	Displays the percentage that each element in a vector or matrix `x` contributes to the sum of all elements (when the sum of elements in the first input argument is equal to or greater than 1, `pie` normalizes the values, and when the sum of the elements in the first input argument is less than 1, `pie` draws a partial pie)
`bar(x)`	Displays the values in a vector or matrix `x` as the horizontal or vertical bars; `bar(...,width)` sets the relative bar width (0.8 is the default value) and controls the separation of bars within a group; `bar(...,'style')` specifies the style of the bars (`'style'` can be either `'grouped'`, which is a default mode, or `'stacked'`)
`barh(...)`	Creates the horizontal bars
`histogram(x,n)`	Creates a histogram of `x` using `n` bins (introduced in R2014b)
`histogram2(x,y)`	Creates a bivariate histogram of `x` and `y` (introduced in R2015a)
`boxchart(y)`	Creates a box plot for each column of `y` (introduced in R2020a)
`feather(u,v)`	Displays the vectors specified by `u` and `v`, containing the *x* and *y* components as the relative coordinates
`quiver(x,y,u,v)`	Displays the velocity vectors with components (`u`,`v`) as arrows originating at the points (`x`,`y`)

of a zero reference axis and the direction of the angles increase. This latter capability enables charts to be used in a variety of aviation applications that require the reference coordinate frame to be fixed to true north. Figure 6.23b shows an example of such a plot created using the following script:

```
theta=0:0.01:2*pi;
rho=100*sin(.5*theta).*cos(.5*theta+pi/2);
rho1=rho+18*sin(0.5*19*theta);
pax=polaraxes;
polarplot(theta,rho,'-','LineWidth',2); hold
polarplot(theta,rho1,'--','LineWidth',2);
    pax.ThetaDir='clockwise';
    pax.ThetaZeroLocation='top';
    pax.FontSize=12;
    pax.ThetaColor='r';
  rticks(10*[2 5 9])
  rticklabels({'r=20nmi','r=50nmi','r=90nmi'})
thetaticks(0:90:270)
thetaticklabels({'N','E','S','W'})
legend('cardioid radiation pattern cardioid ', ...
       'pattern with superimposed ripples','location','s')
```

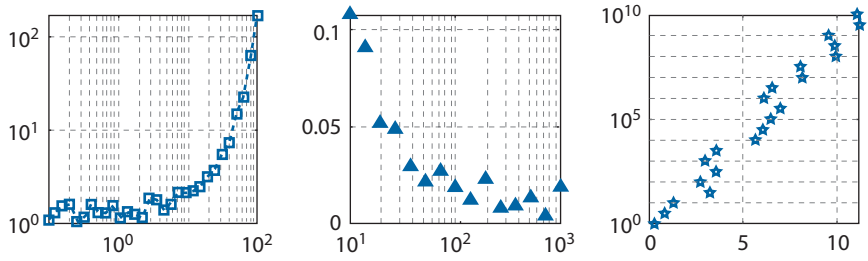


Fig. 6.21 Examples of the `loglog`, `semilogx`, and `semilogy` plots.

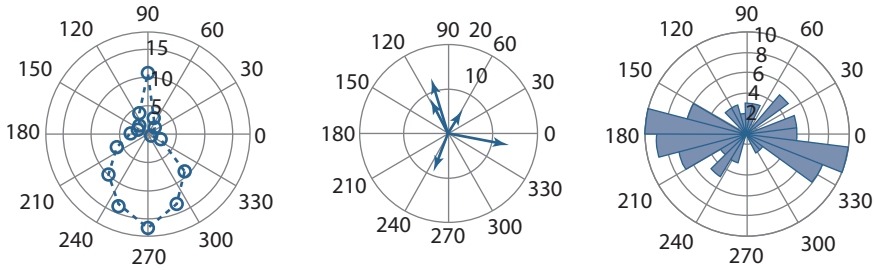


Fig. 6.22 Examples of the `polarplot`, `compass`, and `polarhistogram` charts.

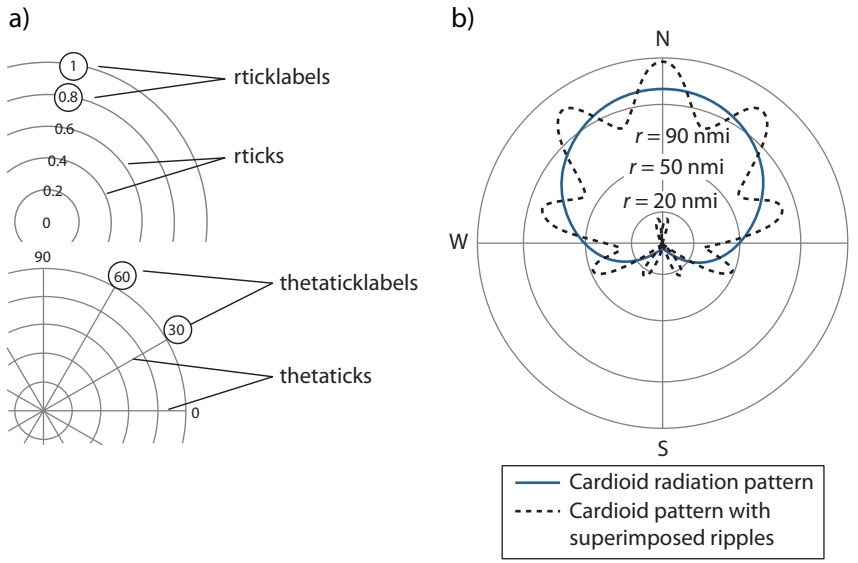
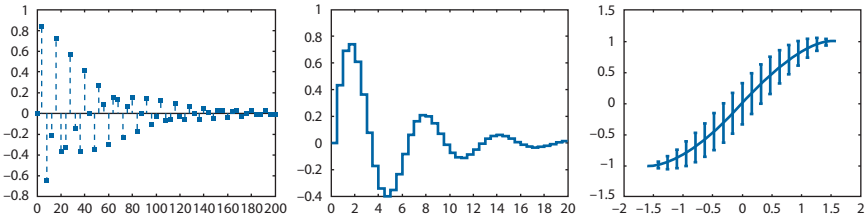


Fig. 6.23 a) Polar plot tics and Labels and b) example of a PolarAxes object.



```
al=0.02; beta=0.5;
t=0:4:200;
y=exp(-al*t).*sin(beta*t);
stem(t,y,'--sr','fill')
```

```
x=0:.5:20;
stairs(x,sin(x).* ...
       exp(-0.2*x),'c')
```

```
x=-pi/2:pi/20:pi/2;
y=sin(x);
e=cos(x)/3;
errorbar(x,y,e,'m')
```

Fig. 6.24 Examples of the discrete data and errorbar plots.

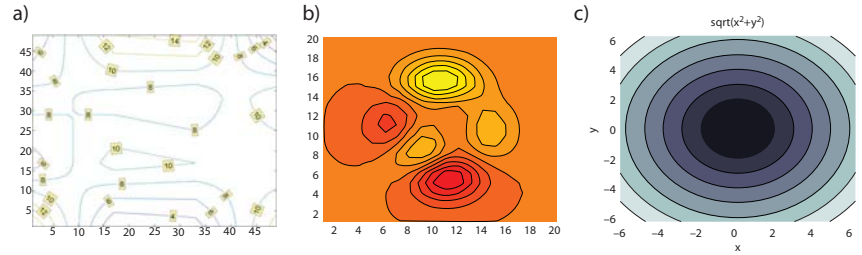
(Compare Fig. 6.23b with the first plot of Fig. 6.22, which attempts to visualize distribution of daily wind directions at the Monterey Regional Airport during the daytime but does not make sense because of the wrong axes orientation.)

Table 6.16 shows the functions that are convenient for visualizing discrete data and errorbar plots, with some examples provided in Fig. 6.24 along with the corresponding sets of commands (hereinafter in this chapter, the newly introduced functions appear in boldface).

Table 6.17 presents a variety of the contour plotting enabling functions, with examples shown in Fig. 6.25. Note that the `colormap` function used in code for Fig. 6.25b,c changes a color matrix of the Figure's Colormap property (Table 6.9); this will be discussed in detail in Sec. 6.9.

Figures 6.26 and 6.27a,b illustrate MATLAB capabilities to produce the area plots, histograms, direction-type plots, and box plots using the functions of Table 6.18.

To partition the input value(s) into the bins and return the count in each bin as well as the bin edges (without plotting a histogram), you can use




```
Z=magic(4); cc=0.7*[1 1 1];
[C,h]=contour(interp2(Z,4));
clabel(C,h,...
'BackgroundColor',...
    [1 1 .6],'Edgecolor',cc)
```

```
[C,h]=contourf(peaks(20),10);
colormap autumn
```

```
f=@(x,y)sqrt(x.^2+y.^2);
fcontour(f,'Fill','on')
colormap bone
```

Fig. 6.25 Examples of the contour plots.

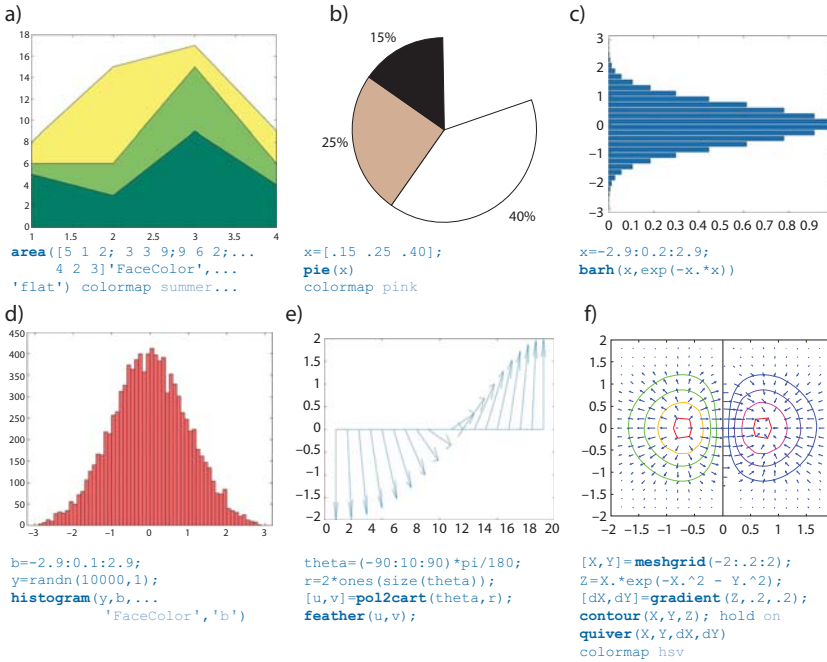


Fig. 6.26 Examples of the area plots (a,b), histograms (c,d), and direction-type plots (e,f).

the `histcount` and `histcount2` functions, introduced in R2015a. One more function, `discretize`, returns indices of bins that contain the elements of the input array (the behavior of `discretize` is similar to that of the `histcounts` function). Two additional functions `morebins(ah,...)` and `fewerbins(ah,...)`, both introduced in R2014b, can be used with both Histogram and Histogram2 objects to quickly increase/decrease the number of bins in the histogram `ah` by 10%.

In case you need to combine different types of plots within the same axes and, say, overlay data on a bar graph, the following example shows how to have an independent *y*-axis for the overlaid dataset (in contrast to the `hold` statement that enables adding data to the same axes), resulting in Fig. 6.27c:

```
generation=1:5;
number=[4.2 33.6 42.7 9.5 1];
price=[0.1 0.2 1 30 150];
k=bar(generation,number,'y'); axis tight, grid
text(k.XEndPoints'-0.12,k.YEndPoints'+3,num2str(number'))
xlabel('Jet fighter generation')
ylabel('A/c produced, thousands')
h1=gca;
```

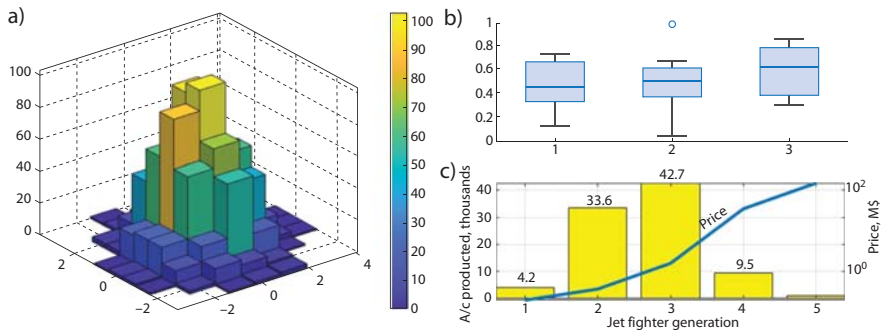


Fig. 6.27 Examples of the a) bivariate histogram and b) box plot; c) blending two different types of plots.

```
% Set the new axes atop the first ones
h2=axes('Position',get(h1,'Position' ));
% Add the second plot to the new axes
semilogy(generation,price,'LineWidth',3)
% Modify second axes settings
set(h2,'Color','none','YAxisLocation','right','XLim',get(h1,'XLim'),...
       'XTickLabel',[],'TickLength',[0 0])
text(3.5,10, 'Price','Rotation',38), ylabel('Price, M$')
```

As seen, using the second Axes object, this script modifies some of its properties. Most importantly, it makes it transparent and defines the location for the *y*-axis. Other modifications include setting the *x*-axis range to that of the first set of axes and eliminating *x*-axis labeling (because we already have one) and axes ticks (otherwise, the log-scale ticks for the second set of axes would be superimposed on those of the first set).

Figure 6.28a shows another example, produced by the following code:

```
C=nan(length(A),1); C(B~="Fair")=3;
stime=datenum('02-01-2018 00:54');
etime=datenum('02-03-2018 23:54');
t=linspace(stime,etime,3*24);
plot(t,A,'.--',t,A(:,1)-A(:,2),'m.--'), grid, hold
bar(t,C,'y')
datetick('x','HH:MM PM')
xlabel('Time'), ylabel('Temperature, ^oF')
legend('Air','Dew point','\DeltaT','Overcast','Location','E')
```

This code also features a time argument vector created using the `datenum` function, allowing you to convert a text representing date and time (a point in time) to a serial date number, that is, the number of days from January 0, 0000 (see Sec. C.1 of Appendix C). In this example, the three-day hourly air and dew point temperatures at Monterey Regional Airport are stored as an

n-by-2 array `A`, and the hourly sky condition is provided by the Meteorological Aerodrome Report (METAR) in the string vector `B`. By the way, the `stackedplot` function, introduced in R2018b, allows you to produce a stacked plot of several variables with a common *x*-axis. Figure 6.28b shows an example of plotting the same original variables as in Fig. 6.28a using the following code:

```
deg=char(176); del=char(916);
Ylabels={['T, ' deg],[ del 'R, ' deg],[del 'T, ' deg]};
s=stackedplot(datetime(datestr(t)),...
      [A A(:,1)-A(:,2)],'DisplayLabels',Ylabels);
s.LineProperties(2).LineStyle= '--';
s.LineProperties(3).PlotType = 'stairs';
```

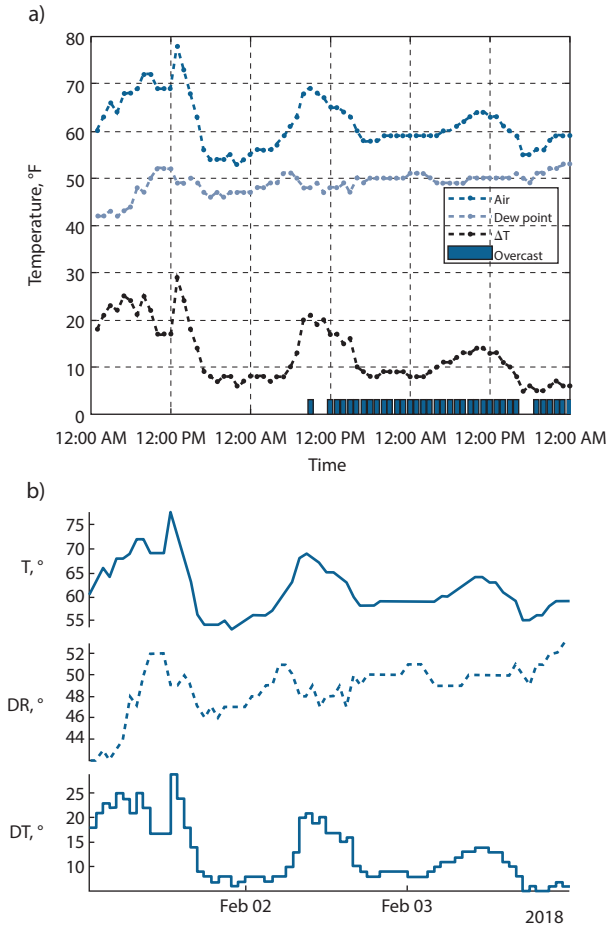


Fig. 6.28 Examples of plotting time-dependent data.

Note that the `stackedplot` function was primarily designed to quickly preview the content of timetables (see Fig. C.3a of Appendix C) and, as such, accessing/changing its properties is kind of tricky. The preceding script also utilizes the `datetime` and `datestr` functions to convert serial date numbers to datetime arrays (so that they show up nicely, as shown in Fig. 6.28b). For more details, we refer you to Sec. C.1 of Appendix C.

Proceeding with the common plotting functions, Table 6.19 lists the available functions for creating polygon, triangular, and scatter plots. Figure 6.29 provides a few examples (an example of using the `rectangle` function will be shown in Sec. 6.12).

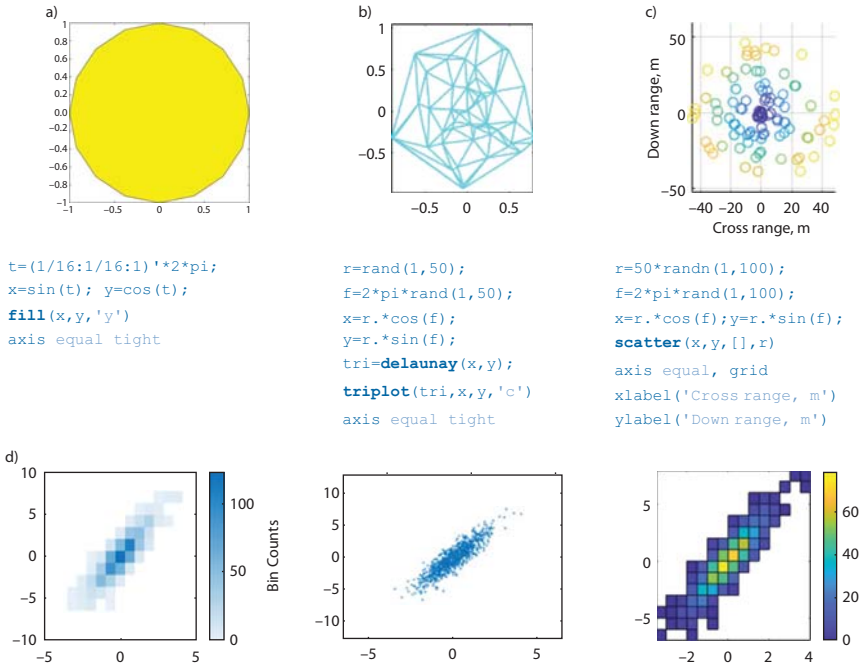
For completeness, Table 6.20 introduces a few specific functions that allow you to visualize tabular and text data. Although an example of using the `stackedplot` function has already been provided in Fig. 6.28, examples of employing the remaining functions are given in Appendix C.

Finally, Table 6.21 shows basic syntaxes for a few functions added to basic MATLAB in R2018b for plotting geographic data. Figure 6.30a

Table 6.19 Functions for Creating 2-D Polygons, Triangular Plots, and Scatter Charts

Function	Brief Description
`rectangle('Position',...pos,'Curvature',cur,...)`	Creates a rectangle in 2-D coordinates by a specified four-element vector `pos` of the form [*x y width height*] in data units [note, this is different from the `annotation('rectangle',[x y w h])` call (Table 6.3) where the position cannot possibly be defined in data units]. A two-element vector `cur` of the form [*horizontal vertical*] defines curvatures in two directions as the fractions of *width* and *height* (`[0 0]` corresponds to no curvature, `[1 1]` to an ellipse or to a circle)
`fill(x,y,c)`	Creates filled polygons from data in `x` and `y` with a vertex color specified by `c` (`c` is a vector or matrix used as an index into the `colormap` function)
`triplot(tri,x,y,c)`	Creates a 2-D triangular plot, that is, the triangles defined in the *n*-by-3 matrix `tri` [a row of `tri` created with the `tri=delaunay(x,y)` command contains indices into the vectors `x` and `y` that define a single triangle—to be explained in Fig. 6.37[a]], with a color specified by `c`
`scatter(x,y,s,c)`	Displays the colored circles at the locations specified by the same-size vectors `x` and `y` (`s` determines the area of each marker; `c` determines the colors of each marker)
`binscatter(x,y)`	Displays a binned scatter plot for vectors `x` and `y` (introduced in R2017b)
`plotmatrix(x,y)`	Creates a matrix of subaxes containing scatter plots of the columns of `x` against the columns of `y`. If `x` is *p*-by-*n* and `y` is *p*-by-*m*, then `plotmatrix` produces an *n*-by-*m* matrix of subaxes

[a]Also see the `delaunayTriangulation` and `triangulation` functions introduced in R2013a.




```
t=(1/16:1/16:1)'*2*pi;
x=sin(t); y=cos(t);
fill(x,y,'y')
axis equal tight
```

```
r=rand(1,50);
f=2*pi*rand(1,50);
x=r.*cos(f);
y=r.*sin(f);
tri=delaunay(x,y);
triplot(tri,x,y,'c')
axis equal tight
```

```
r=50*randn(1,100);
f=2*pi*rand(1,100);
x=r.*cos(f);y=r.*sin(f);
scatter(x,y,[],r)
axis equal, grid
xlabel('Cross range, m')
ylabel('Down range, m')
```

```
x=randn(1e3,1); y=2*x+randn(1e3,1);
subplot 131, binscatter(x,y), subplot 132, plotmatrix(x,y)
subplot 133, histogram2(x,y,'FaceColor','flat'), colorbar, view(2),grid off
```

Fig. 6.29 Examples of the polygon, triangular, and scatter plots.

shows an example of using these functions to visualize a great-circle trajectory (shortest distance between two points on the globe) to navigate an airplane from San Francisco to Frankfurt (note, the `track2` function belongs to the Mapping Toolbox), and Fig. 6.30b shows Census 2010 data for Monterey County (the input data resides in `lat`, `lon`, `pop`, and `MunType` vectors). Of course, more functions for analyzing

Table 6.20 Functions for Visualizing Table Content

Function	Brief Description
`heatmap(tbl)`	Creates a heatmap of a table or numeric matrix data (introduced in R2017a)
`stackedplot(tbl)`	Creates a stacked plot of several variables with a common *x*-axis (introduced in R2018b)
`parallelplot(tbl)`	Creates a parallel coordinates plot from a table or numeric matrix data (introduced in R2019a)
`wordcloud(tbl)`	Creates a word cloud chart from a table, text string vector, or categorical array (introduced in R2017b)

Table 6.21 Geographic Data Plotting Functions

Function	Brief Description
`geoplot(lat,lon)`	Plots a line in geographic axes with vertices at the latitude and longitude locations specified (in degrees) by the vectors `lat` and `lon`
`geoscatter(lat,lon)`	Displays the colored circles in geographic axes at the `lat` and `lon` locations
`geobubble(lat,lon,size)`	Creates a geographic bubble chart scaling the areas of the bubbles according to the numeric values in `size`
`geodensityplot (lat,lon,w)`	Creates a density plot in geographic axes using locations specified in `lat` and `lon` and weights for the data points in `w`
`geoaxes`	Creates geographic axes with a live map that allows you to pan and to zoom in and out on the map to view regions in more detail. The `geolimits` function allows adjusting the geographic limits
`geobasemap(map)`	Sets the basemap of the current geographic axes to the value specified by map [`'darkwater'` (default), `'grayland'`, `'grayterrain'`, `'bluegreen'`, `'landcover'`, and `'colorterrain'`]

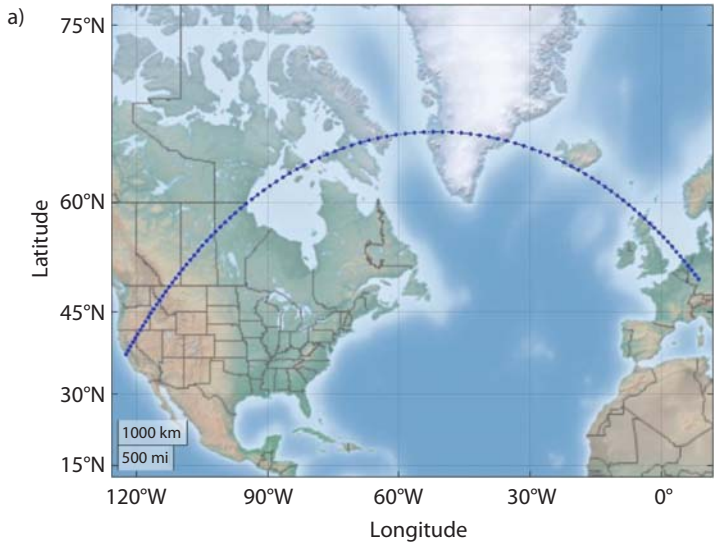
geographic data and creating map displays are available in the Mapping Toolbox.

6.8 The 3-D Plots

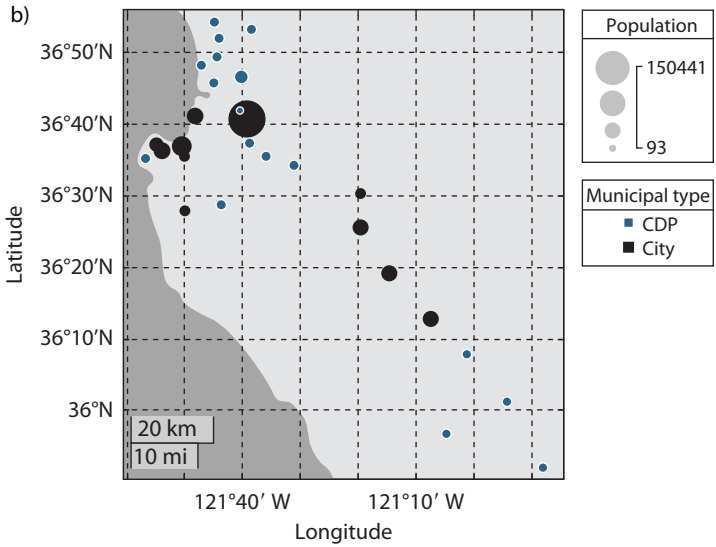
The basic MATLAB function to produce a 3-D plot is `plot3(x,y,z)`. It displays a 3-D plot of a data point set defined by three coordinates, *x*, *y*, and *z*, stored in the same-size vectors `x`, `y`, and `z`. All that we discussed about the 2-D plots applies here as well (except that the third coordinate, *z*, now needs to be added when applicable). For example, to place a label for the *z*-axis, the `zlabel` function should be used in the very same way the `xlabel` and `ylabel` functions were used for the 2-D plots. To this end, a 3-D helix, shown in Fig. 6.31a, can be produced using the following script:

```
t=0:pi/50:10*pi;
plot3(sin(t),cos(t),t)
grid on, axis square
xlabel('x-axis'), ylabel('y-axis'), zlabel('z-axis')
```

The *viewpoint* can be specified using the `view` function in terms of a point in 3-D space or more conveniently in terms of *azimuth* and *elevation* [`view(az,el)` or `view([az,el])`], defined as shown in Fig. 6.31b. The azimuth, `az`, is the horizontal rotation about the *z*-axis as measured




```
geoaxes;
[latgc,longc]=track2('gc',37.62,-122.38,50.03,8.57);
geoplot(latgc,longc,'b.-.'), geobasemap('colorterrain')
```




```
gb=geobubble(lat,lon,pop,MunType)
gb.SizeLegendTitle='Population';
gb.ColorLegendTitle='Municipal Type';
gb.BubbleColorList(2,:)=[0 1 0];
```

Fig. 6.30 Examples of plotting georeferenced data.

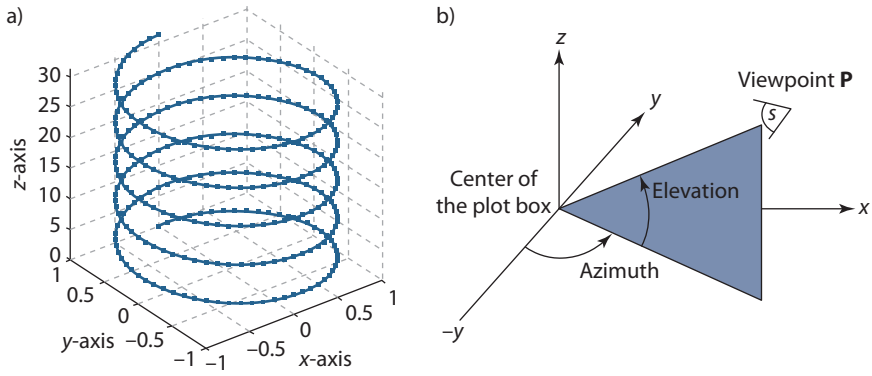


Fig. 6.31 a) Helix produced with the `plot3` function and b) viewpoint definition.

in degrees from the negative *y*-axis (positive values indicate counterclockwise rotation of the viewpoint). The elevation, `el`, is the vertical elevation of the viewpoint in degrees (positive values of elevation correspond to moving above the object). By default, MATLAB uses `view(-37.5,30)`. For comparison, Fig. 6.32a shows another view of the helix depicted in Fig. 6.31a using `view(20,15)`.

If the viewpoint is set along the ray originating at the origin of the Cartesian system of coordinates and point *P(x,y,z)* {`view([x,y,z])`}, it is actually equivalent to `view(az,el)` with `az` and `el` computed as

```
el=180*atan2(z,sqrt(x^2+y^2))/pi; az=180*atan2(x,-y)/pi;
```

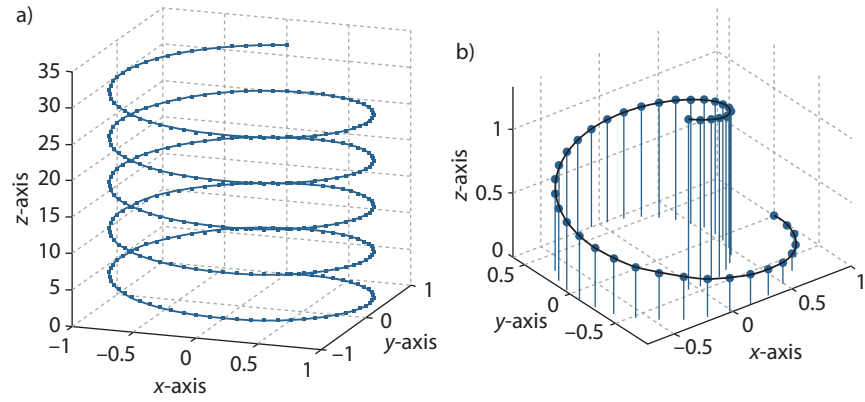


Fig. 6.32 a) Using the `view` function to change the viewpoint (compare with Fig. 6.31a) and b) using the `stem3` function.

You can always return the viewpoint to the default values by issuing the `view(3)` command. The `view(2)` command provides a bird-eye (2-D) view with `az=0` and `el=90`. (The `view(1)` syntax does not exist.)

Figure 6.32b shows an example of using another function, `stem3`, which can be convenient for visualizing, say, a 3-D trajectory of an aerial vehicle. Code producing this plot is as follows:

```
t=0:0.2:8; y=exp(-(.1+i)*t);
stem3(real(y),imag(y),t/6,'filled')
hold, plot3(real(y),imag(y),t/6,'r'), axis equal
xlabel('x-axis'), ylabel('y-axis'), zlabel('z-axis')
```

The `plot3(x,y,z)` function accepts not only the same-size vectors but also the same-size matrix arguments. In this case, instead of plotting a 3-D curve (spatial trajectory), it produces a surface with `x` and `y` matrices representing a rectangular *mesh grid* and matrix `z` elevation (function values) at the nodes of this grid. Hence, plotting the surface is a three-step process. First, you need to produce an $X-Y$ grid (Fig. 6.33). Next, you compute the function values at the nodes of this grid, $Z = f(X, Y)$. Finally, you pass three same-size matrices to the plotting function to plot the surface.

The proper way to create a rectangular grid is to use the `meshgrid` function. Suppose the length of the vectors `x` and `y` (containing grid coordinates for each dimension) is n and m, respectively (Fig. 6.33). Then, the command

```
[X,Y]=meshgrid(x,y)
```

creates two n-by-m matrices `X` and `Y` (`[X,Y]=meshgrid(x)` is the same as `[X,Y]=meshgrid(x,x)`). For example,

```
>> [X,Y]=meshgrid(1:10,1:3)
```

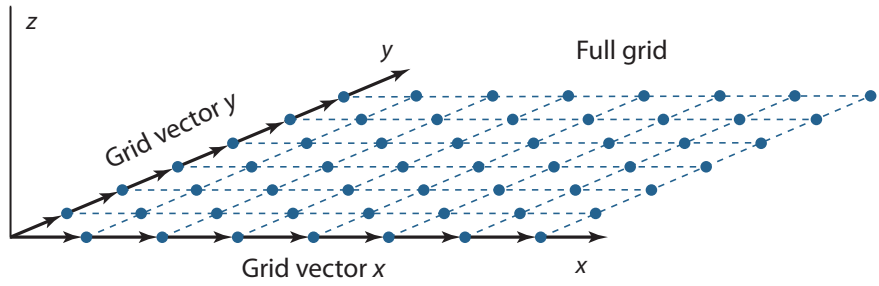


Fig. 6.33 A rectangular $x-y$ mesh grid.

yields

```
X = 1   2   3   4   5   6   7   8   9   10
    1   2   3   4   5   6   7   8   9   10
    1   2   3   4   5   6   7   8   9   10
Y = 1   1   1   1   1   1   1   1   1    1
    2   2   2   2   2   2   2   2   2    2
    3   3   3   3   3   3   3   3   3    3
```

This function simply concatenates m rows of the vector `x` vertically for `X` and n columns of the vector `y` horizontally for `Y`. Hence, the x-coordinate changes in the matrix `X` across the columns, whereas the y-coordinate changes in the matrix `Y` across the rows.

The next step is to compute the z values on this grid, for instance,

```
>> Z=sin(0.1*Y).*cos(0.1*X);
```

(do not forget to use element-wise operators). Finally, you may use the `plot3` function

```
>> plot3(X,Y,Z)
```

which produces a surface as shown in Fig. 6.34a. As seen, it simply plots n columns of matrix `Z`, shifting each new line along the x-axis. If you use another grid, say, `[X,Y]=meshgrid(1:40)` (and repeat the last two commands), the result changes to that of Fig. 6.34b.

You may argue that it does not look like a surface. Well, MATLAB provides you with a variety of other functions producing a better result, but their syntax is essentially the same as that of the `plot3` function. These functions are listed in Table 6.22. For example, using the `mesh` function instead of `plot3` in the latter example produces the result shown in Fig. 6.35a (the properties of the Surface object are shown in Table 6.23). As for Fig. 6.35b, instead of a simple call, `surf(X,Y,Z)`,

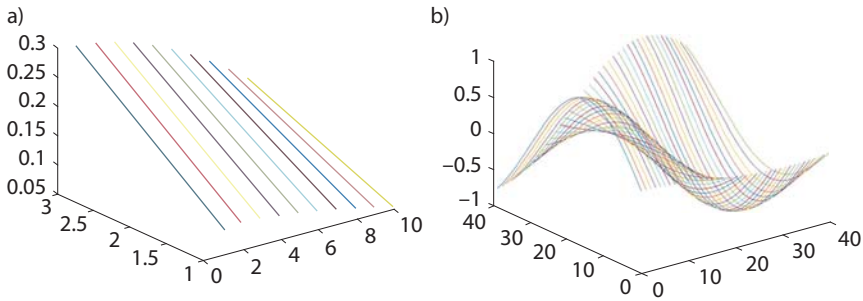


Fig. 6.34 Examples of using the `plot3` function to create a surface.

Table 6.22 Surface and Mesh Plotting Functions

Function	Brief Description
`mesh(x,y,z)`	Creates a 3-D mesh-surface plot with `x` and `y` being matrices defining the mesh grid (intersections of the wireframe grid lines) and the same-size matrix `z` representing surface height (by default color is proportional to the height). `x` and `y` may be given as the grid vectors as well (Fig. 6.33)
`meshc(x,y,z)`	Creates a 3-D mesh-surface plot with a contour plot underneath the surface
`meshz(x,y,z)`	Creates a 3-D mesh-surface plot with a series of vertical reference lines under the surface
`surf(x,y,z)`	Creates a shaded 3-D mesh-surface plot
`surfc(x,y,z)`	Creates a shaded 3-D mesh-surface plot with a contour plot underneath it
`surfl(x,y,z,'light')`	Creates a colored, lighted surface
`surface(x,y,z)`	Adds a Surface object to the current axes (it is the low-level function for creating Surface graphics objects as opposed to `surf`, similar to the `line` to `plot` relationship)
`waterfall(x,y,z)`	Draws a mesh similar to the `meshz` function but does not generate lines from the columns of matrices (this produces a "waterfall" effect). For column-oriented waterfall, use `waterfall(X',Y',Z')`
`ribbon(x,y)`	Plots `x` vs the columns of `y` as the 3-D strips (`x` and `y` are the same-size vectors or matrices, or alternatively, `x` can be a row or column vector, and `y` – a matrix with `length(x)` rows)

we first loaded the picture from Fig. 4.16a and then used it as a texture for the surface:

```
F=flipud(importdata('CriticalApha.jpg'));
surf(X,Y,Z,F,'FaceColor','texturemap','EdgeColor','none')
view(56,28)
```

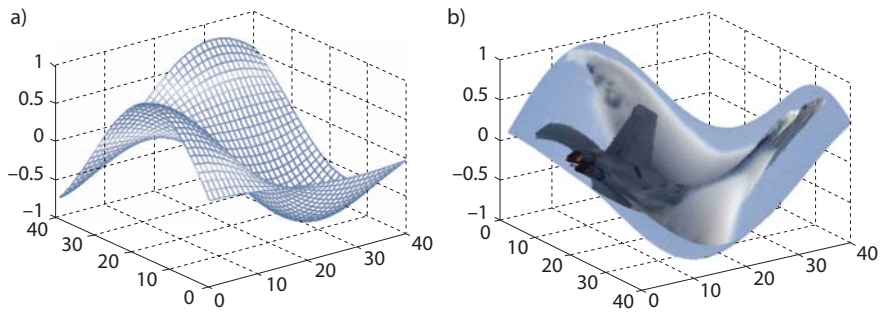


Fig. 6.35 Example of using a) `mesh` and b) `surf` functions to create a surface.

Table 6.23 Properties of the Surface Object with their Values

```
>> get(gca,'Children')
```

```
AlignVertexCenters: 'off'
         AlphaData: 1
   AlphaDataMapping: 'scaled'
    AmbientStrength: 0.3000
         Annotation: [1×1 matlab.graphics.eventdata.Annotation]
   BackFaceLighting: 'reverselit'
       BeingDeleted: 'off'
         BusyAction: 'queue'
      ButtonDownFcn: ''
              CData: [49×49 double]
      CDataMapping: 'scaled'
         CDataMode: 'auto'
       CDataSource: ''
          Children: [0×0 GraphicsPlaceholder]
          Clipping: 'on'
         CreateFcn: ''
         DeleteFcn: ''
    DiffuseStrength: 0.6000
        DisplayName: ''
          EdgeAlpha: 1
          EdgeColor: 'flat'
      EdgeLighting: 'flat'
          FaceAlpha: 1
          FaceColor: [1 1 1]
      FaceLighting: 'none'
       FaceNormals: [48×48×3 double]
    FaceNormalsMode: 'auto'
   HandleVisibility: 'on'
           HitTest: 'on'
     Interruptible: 'on'
         LineStyle: '-'
         LineWidth: 0.5000
            Marker: 'none'
     MarkerEdgeColor: 'auto'
     MarkerFaceColor: 'none'
          MarkerSize: 6
         MeshStyle: 'both'
            Parent: [1×1 Axes]
      PickableParts: 'visible'
          Selected: 'off'
  SelectionHighlight: 'on'
SpecularColorReflectance: 1
   SpecularExponent: 10
   SpecularStrength: 0.9000
                Tag: ''
               Type: 'surface'
        UIContextMenu: [0×0 GraphicsPlaceholder]
           UserData: []
      VertexNormals: [49×49×3 double]
  VertexNormalsMode: 'auto'
            Visible: 'on'
              XData: [49×49 double]
           XDataMode: 'manual'
         XDataSource: ''
              YData: [49×49 double]
           YDataMode: 'manual'
         YDataSource: ''
              ZData: [49×49 double]
         ZDataSource: ''
```

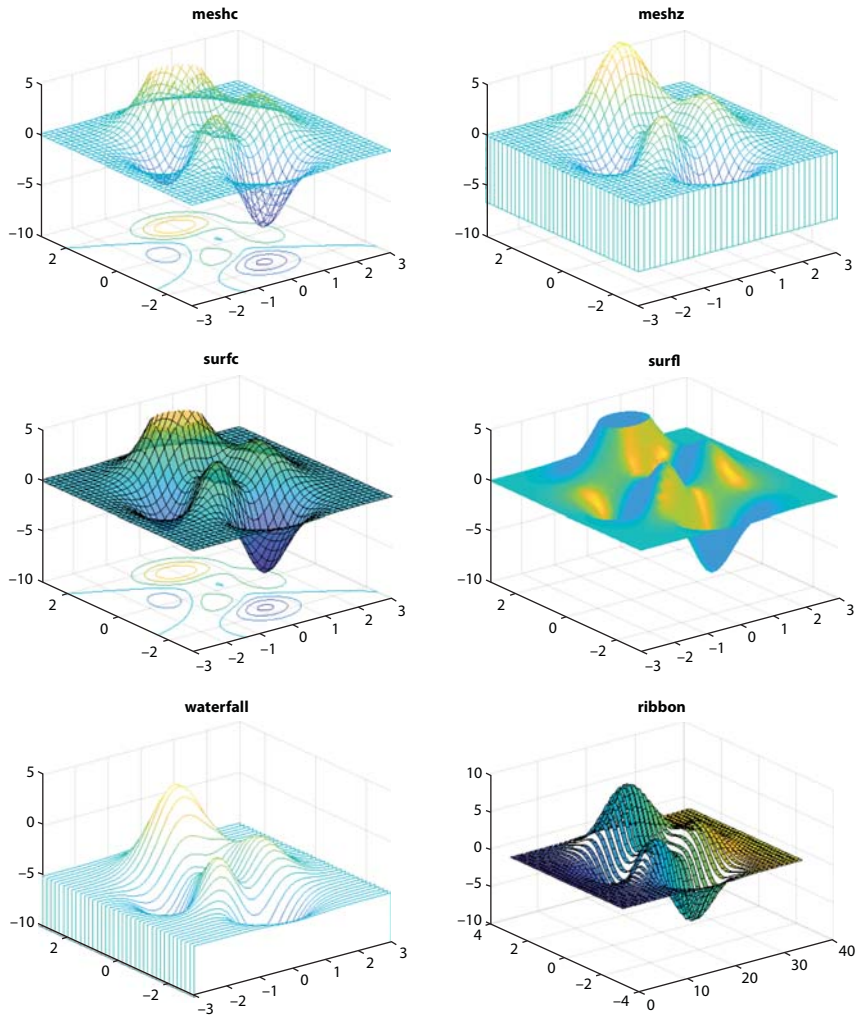


Fig. 6.36 Illustration of a variety of the surface-plotting functions.

Figure 6.36 provides examples of employing other functions of Table 6.22 to visualize the MATLAB's function of two variables coded in *peaks.m*. These plots were produced using the following script:

```
[x,y]=meshgrid(-3:1/8:3); z=peaks(x,y);
tiledlayout(2,3)
nexttile, meshc(x,y,z),              title('meshc')
          axis([-3 3 -3 3 -10 5]); v=axis; hidden off
nexttile, meshz(x,y,z), axis(v);     title('meshz')
          ax=gca; ax.Clipping='off';
nexttile, surfc(x,y,z,'FaceAlpha',0.7)
          axis(v),                   title('surfc')
```

```
nexttile, surfl(x,y,z), axis(v)
          shading interp,        title('surfl')
nexttile, waterfall(x,y,z-5), axis(v), title('waterfall')
nexttile, ribbon(y,z),           title('ribbon')
```

Rather than using the simplest syntaxes, we used some tricks, just to make a few points. First, note how we captured the axes settings for the first plot and then used them for the following four plots. This might be quite useful to prevent autoscaling when using different plotting functions. Second, the *z*-axis limits were intentionally set in such a way that the *z* values greater than 5 are not supposed to be visualized. However, the Clipping property of the current axes (Table 6.8) can be turned off so that all data points are still visualized even if they are beyond the preset axis limits (see the second plot). (By the way, along with the `'3dbox'` value, the ClippingStyle property for the Axes object may accept the `'rectangle'` value.) The first plot also demonstrates that, if needed, the `hidden` function can be used to prevent the lines in the back of a mesh from being hidden by those in front. Similarly, you can vary the FaceAlpha property (third plot) to vary face transparency (from 0 to 1) (Table 6.23). The very same result can be achieved by issuing the `alpha 0.7` command rather than using the name–value pair argument. (Note, the `alpha` function also controls `AlphaData` and `AlphaDataMapping` values of all objects supporting these properties.) Finally, the `shading` function allows you to control the color shading of the surface, with three options available. The `flat` option forces each mesh line segment and face to have a constant color, the default `faceted` option superimposes black mesh lines, and the `interp` option allows the color to vary across the line or face.

MATLAB also allows you to use the triangular rather than the rectangular 2-D grid. Such a grid can be a result of the *Delaunay triangulation* connecting each point to its natural neighbors, as was shown in Fig. 6.29b. To clarify, consider an example where the first three lines of code visualize six arbitrarily chosen points (Fig. 6.37a), followed by conducting the 2-D Delaunay triangulation and displaying the triangles defined in the three-column *face matrix* `tri`, each row of which represents a triad of indices of the neighboring points (Fig. 6.37b):

```
>> N=6; x=rand(N,1); y=rand(N,1); plot(x,y,'pr') , hold
>> xticks([]), yticks([]), xlabel('x axis'), ylabel('y axis')
>> text(x+0.01,y,cellstr(num2str([1:N]')))
>> tri=delaunay(x,y), triplot(tri,x,y)
tri =   1   3   4
        3   2   4
        1   5   3
        6   1   4
        6   5   1
        2   6   4
```

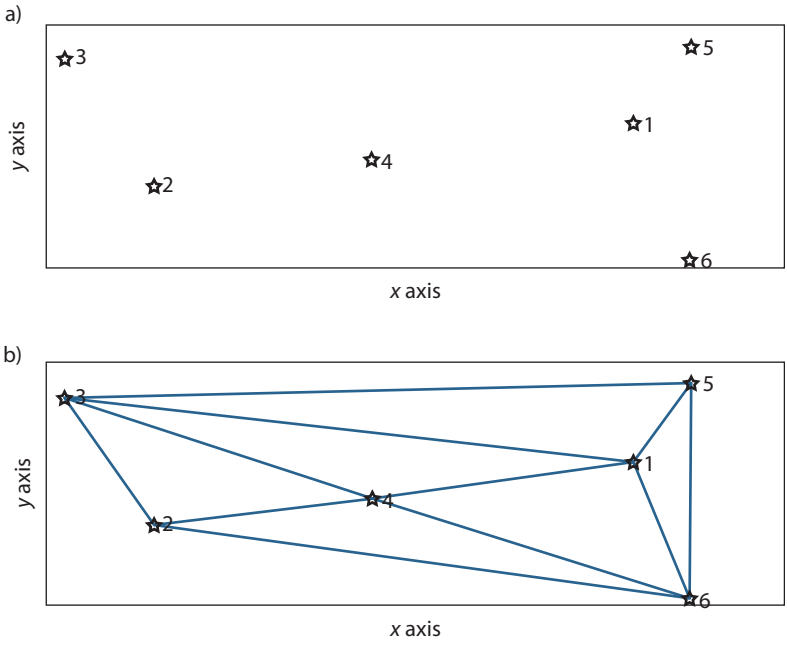


Fig. 6.37 Illustration of the 2-D Delaunay triangulation of six points.

(For the record, the `delaunay`, `delaunayTriangulation`, and `triangulation` functions allow triangulation in 2-D or 3-D, and the `delaunayn` function—in *n*-D.)

The first two functions listed in Table 6.24 allow you to use the results of the 2-D triangulation to produce a triangular-face mesh and surface. For example, the following set of commands creates 500 data points (*vertex vectors* `x`, `y`, and `z`), then computes the face matrix `tri`, and finally uses it as a triangular grid to produce a mesh plot, as shown in Fig. 6.38a:

```
>> x=rand(1,500); y=rand(1,500); z=peaks(6*x-3,6*y-3);
>> tri=delaunay(x,y); trimesh(tri,x,y,z)
```

Table 6.24 Functions for Visualizing the Triangular Surfaces and Shapes

Function	Brief Description
`trimesh(tri,x,y,z)`	Displays triangles defined in the *m*-by-3 triangulation matrix `tri` as a mesh (each row of `tri` defines a single triangular face by indexing into the vectors or matrices that contain the `x`, `y`, and `z` vertices
`trisurf(tri,x,y,z)`	Displays triangles defined in the *m*-by-3 face matrix `tri` as a surface
`tetramesh(T)`	Displays tetrahedrons defined in the *m*-by-4 matrix `T` as mesh (`T` is usually the output of a Delaunay triangulation of a 3-D set of points). This function is a 3-D analog of the 2-D `triplot` function of Table 6.19

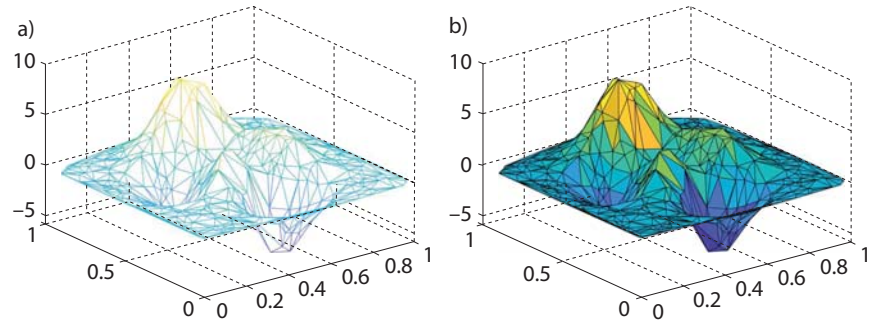
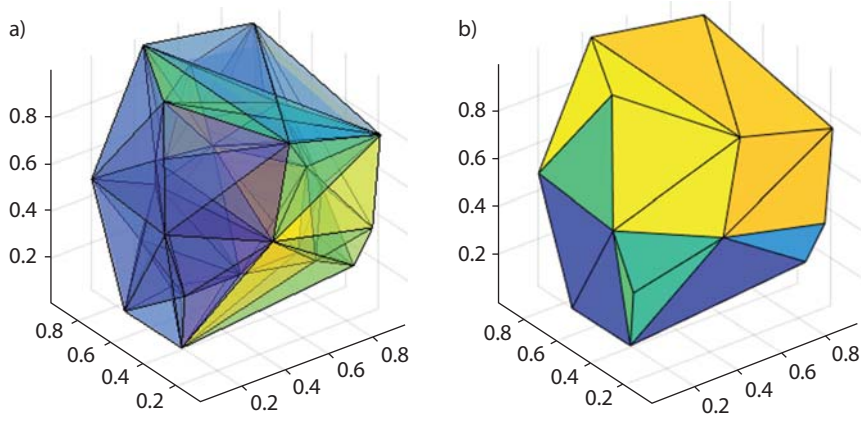


Fig. 6.38 Examples of the triangular mesh (a) and surface (b) plots.

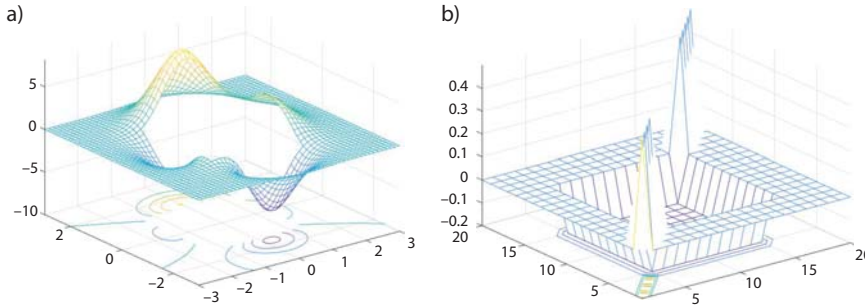
Utilizing the `trisurf` function in the last command produces the triangular surface shown in Fig. 6.38b.

Figure 6.39a provides an example of employing the last function of Table 6.24 to visualize the results of the 3-D Delaunay triangulation, and Fig. 6.39b provides an example of using the `convexHull` function (available




```
figure
x=gallery('uniformdata',[30 1],0); % random-number vector with seed 0
y=gallery('uniformdata',[30 1],1); % random-number vector with seed 1
z=gallery('uniformdata',[30 1],2); % random-number vector with seed 2
    DT=delaunayTriangulation(x,y,z);
    tetramesh(DT,'FaceAlpha',0.3), grid, axis equal
figure
    K=convexHull(DT); % compute the convex hull of 3-D triangulation
    trisurf(K,DT.Points(:,1),DT.Points(:,2),DT.Points(:,3)),axis equal
```

Fig. 6.39 Visualizing the result of a) 3-D Delaunay triangulation and b) its convex hull.




```
[x,y]=meshgrid(-3:.125:3);
z=peaks(x,y);
z(15:35,15:35)=NaN; meshc(x,y,z)
```

```
w=0.5*eye(20);
w(5:15,5:15)=-0.2*ones(11);
meshc(w)
```

Fig. 6.40 Examples of a) cropping the surface and b) using `meshc` to visualize a matrix.

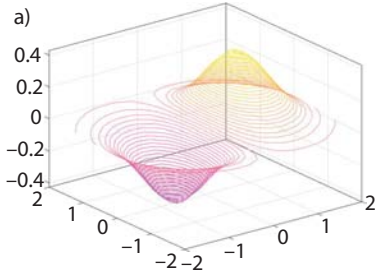
since R2013a) to compute its convex hull followed by the `trisurf` function to display the result.

Before we proceed with other 3-D plotting functions, let us mention two kinds of unusual applications of the 3-D mesh plots. The first one, which actually applies to the 2-D plots as well, is when for some reason you want to cut away a portion of a surface, so that it does not show up on the plot. This can be easily accomplished by assigning the `NaN` values to elements of the z matrix at the corresponding grid nodes as shown in Fig. 6.40a.

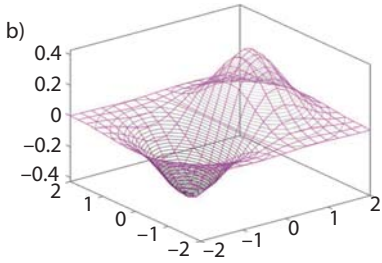
The second unusual usage of the surface plots is when you use them to "visualize" matrices as shown in Fig. 6.40b. A glimpse of such a plot may be much more effective compared with staring blankly at the bare numbers.

Table 6.25 3-D Contour and Vector-Type Data Functions

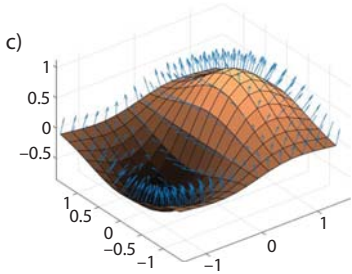
Function	Brief Description
`contour3(z)`	Draws a contour plot of matrix `z` in a 3-D view (`z` is interpreted as heights with respect to the x–y plane). The number and values of the contour levels and the values of contour levels are chosen automatically. The ranges of the x- and y-axes are `[1:n]` and `[1:m]`, where `[m,n]=size(z)`
`quiver3(x,y,z,u,v,w)`	Plots the vectors with components (`u, v, w`) originated at the points (`x, y, z`). Matrices `x, y, z, u, v,` and `w` must all be of the same size
`surfnorm(x,y,z)`	Computes and displays the unnormalized surface normals to the surface defined by the same-size matrices `x, y,` and `z`




```
[X,Y]=meshgrid([-2:.25:2]);
Z=X.*exp(-X.^2-Y.^2);
contour3(X,Y,Z,50)
colormap spring
```




```
[X,Y]=meshgrid([-2:.25:2]);
Z=X.*exp(-X.^2-Y.^2);
contour3(X,Y,Z,50)
surface(X,Y,Z,'EdgeColor','m',...
                'FaceColor','none')
grid off, colormap gray
```




```
[X,Y]=meshgrid(-1.4:.2:1.4); Z=2*X.*exp(-X.^2-Y.^2);
[U,V,W] = surfnorm(X,Y,Z); quiver3(X,Y,Z,U,V,W,1)
hold, surf(X,Y,Z), colormap copper, axis equal
```

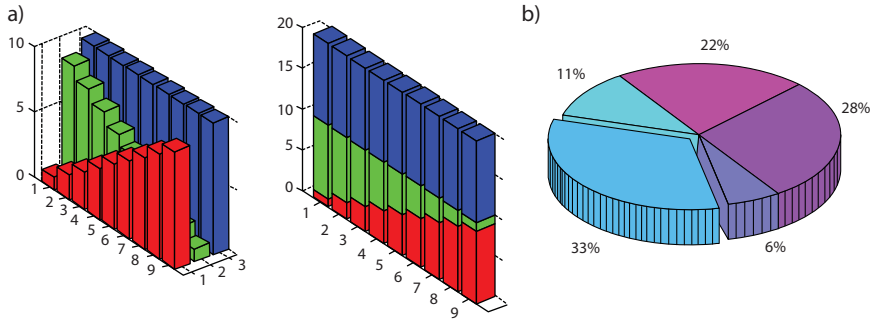
Fig. 6.41 Examples of the 3-D contour plots (a,b) and surface with normals (c).

Table 6.25 provides the 3-D analogues of the `contour` and `quiver` functions introduced in Tables 6.17 and 6.18, respectively. Figure 6.41 presents examples of using the `contour3` function, overlaying a 3-D contour plot with the surface itself, and using the `surfnorm` function to compute surface normals, which are then visualized using the `quiver3` function.

Table 6.26 and Fig. 6.42 present available functions for 3-D bar and pie plots, similar to their 2-D analogs shown in Table 6.18.

Table 6.26 3-D Bar and 3-D Pie Functions

Function	Brief Description
`bar3(x)` `bar3h(x)`	Draws the vertical and horizontal 3-D bar charts, where each element in `x` corresponds to one bar
`pie3(x,y)`	Draws a 3-D pie chart using data in `x`, with each element in `x` represented as a slice (optional same-size vector `y` containing zeros and ones defines whether the element is detached from the pie)




```
x=[1:9;9:-1:1]'; x(:,3)=x(:,1)+x(:,2);
subplot(1,2,1), bar3(x,'detached')
subplot(1,2,2), bar3(x,'stacked'), colormap(eye(3))
```

```
x=[1 3 0.5 2.5 2];
det=[0 1 0 0 0];
pie3(x,det), colormap cool
```

Fig. 6.42 Examples of the 3-D bar (a) and pie (b) graphs.

Table 6.27 lists the functions for visualizing different 3-D shapes followed by Fig. 6.43 providing some examples and also showing how to use the graphics objects to manipulate some of the shape's properties. Figures 6.43b and 6.43f also feature the `rotate` function that you can use to rotate a graphics object (which must be a child of the same axes) in 3-D space around the axis of rotation according to the right-hand rule (Fig. 6.44). Similar to Fig. 6.31b, the axis of rotation is defined by either the origin and Cartesian coordinates of some point $\boldsymbol{P}$ or the spherical coordinates (ϑ, φ).

Because all surfaces in Fig. 6.43 are composed of individual planes or faces, MATLAB offers two special functions that allow you to create and color individual polygons and patches from scratch (Table 6.28). Several examples are shown in Fig. 6.45.

Table 6.27 The 3-D Shape Functions

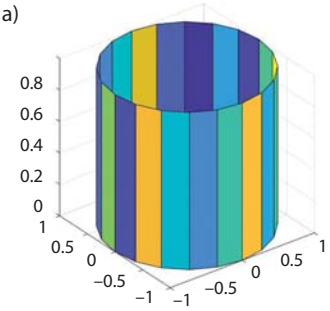
Function	Brief Description

`cylinder`
`cylinder(r,n)`
`[x,y,z]=cylinder(r,n)`
	Generates a unit-height cylinder using (optional) `r` to define a profile curve and (optional) `n` to specify the number of equally spaced points around its circumference (`n` = 20 by default). If requested, returns `x`, `y`, `z` coordinates in the 2-by-(`n` + 1) matrices (to then be used with `surf` or `mesh`).

`sphere`
`sphere(n)`
`[x,y,z]=sphere(n)`
	Generates a unit-radius sphere with the default 20-by-20 (or `n`-by-`n`) faces. If requested, returns the `x`, `y`, `z` coordinates in three (`n` + 1)-by-(`n` + 1) matrices (to then be used with `surf` or `mesh`).

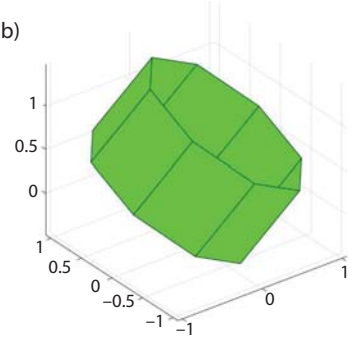
`ellipsoid(xc,yc,zc,xr,yr,zr)`
`ellipsoid(xc,yc,zc,xr,yr,zr,n)`
`[x,y,z]=ellipsoid(...)`
	Generates an ellipsoid with the center at `xc`, `yc`, `zc` and radii `xr`, `yr`, and `zr` as a surface

a)



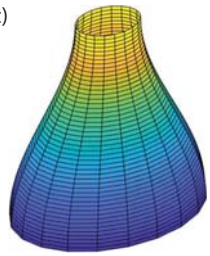
```
cylinder
h=findobj('Type','surface');
C=rand(size(get(h,'CData')));
set(h,'CData',C)
axis square
```

b)



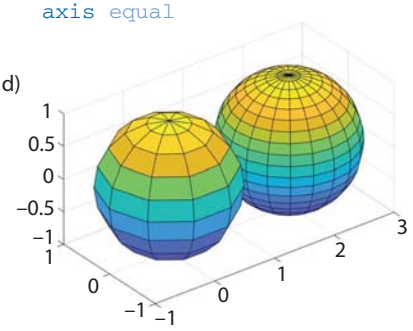
```
cylinder(1,8)
hc=get(gca,'Children');
rotate(hc,[1 1 1],45)
set(hc,'FaceColor','g')
axis equal
```

c)



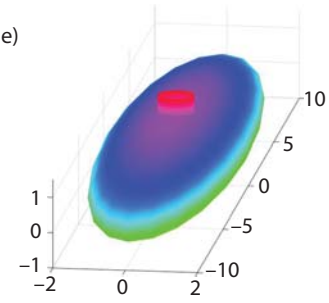
```
t=linspace(0,pi,50);
[X,Y,Z]=cylinder(2+cos(t));
surf(X,Y,Z)
axis square
axis off
```

d)



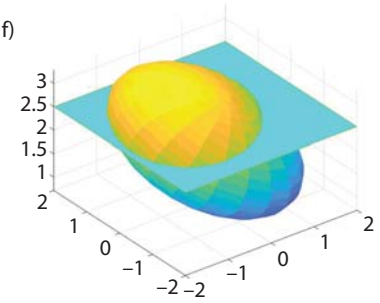
```
sphere(10)
hold on
[x,y,z]=sphere;
surf(x+2,y,z)
axis equal
```

e)



```
ellipsoid(0,0,0,2,10,1)
hold, axis equal
[x,y,z]=cylinder(0.5,0);
surf(x,y,z+0.5)
shading interp
view([8,14])
colormap hsv
```

f)



```
ellipsoid(0,0,0,2,2,1), hold
h=get(gca,'Children');
rotate(h,[3 3 3],34)
z=get(h,'Zdata'); shading flat
set(h,'Zdata',z+2), axis equal
hs=mesh([-2 2; -2 2],...
   [-2 -2; 2 2],2.5*ones(2));
set(hs,'FaceColor','c')
```

Fig. 6.43 Examples of visualizing the 3-D shapes.

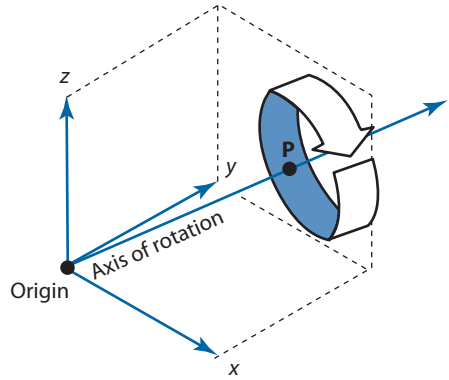


Fig. 6.44 The essence of the `rotate` function.

The `fill3` function in Fig. 6.45a produces two polygons defined by 3-by-2 matrices `x`, `y`, `z`—the slant one and the one parallel to the *x*-*y* plane. It colors them defining the vertices with the hottest (`1`) and coldest (`0`) color, specified by a matrix `c`. In our case, the second (horizontal) polygon, defined by three vertices (points), (1, 0, 1), (0, 1, 1), and (1, 1, 1), is colored so that the first vertex is the coldest one (0, 1, 1). (The coldest and hottest colors depend on the current colormap—to be discussed in Sec. 6.9.) To specify colors in Fig. 6.45c, the `-z` matrix was used. This results in the hottest color for each patch appearing at the bottom.

Color `c` can also be defined as vector. If it is a row vector, `length(c)` must equal `size(x,2)` and `size(y,2)`; if it is a column vector, `length(c)` must equal `size(x,1)` and `size(y,1)`. Finally, all polygons can be monotone. In this case, the color is defined by a single 3-by-1 RGB vector or by one of the characters corresponding to the predefined color (as shown in the last column of Table 6.5 and discussed in Sec. 6.9).

Using patches, you can create any complex graphics object for further use (for example, animation). For instance, the following script produces a 3-D model of aircraft, as shown in Fig. 6.46a:

Table 6.28 The 3-D Polygon and Patch Plotting Functions

Function	Brief Description
`fill3(x,y,z,c)`	Fills *n* 3-D triangular polygons with the vertices defined by triplets `x`, `y`, `z` (3-by-*n* matrices), and `c` specifying their color
`patch(x,y,z,c)`	Creates a 3-D patch with the elements of `x`, `y`, and `z` specifying the vertices of a polygon and `c` specifying the color

```
fx=[6.6 5.3 2.3 1.5 -2 -2.5 -10 -10 0 5.3 5.3]';              % fuselage
fy=zeros(11,1);
fz=[1.2 1.7 1.9 1.2 1.2 2.6 1.1 0.7 -0.2 0.3 0.8]';
    cx=[2.3 0 -2.5 -2 1.5]';            % cockpit canopy
    cy=zeros(5,1);
    cz=[1.9 3.1 2.6 1.2 1.2]';
wingx=2.3*[1 -3 -3 -3+.6 -3+.6 -3 -3 -3+.6 -3+.6 -3 -3 1]'/4; % wing
wingy=[8.5 8.5 7.8 7.8 2.2 2.2 -2.2 -2.2 -7.8 -7.8 -8.5 -8.5]';
wingz=zeros(12,1);
    ax=2.3*[-3+.6 -3 -3 -3+.6]'/4;      % right aileron
    ay=[7.8 7.8 2.2 2.2]';
    az=zeros(4,1);
hsx=1.3*[0 -1 -1 0]'-8.1;               % horizontal stabilizer
hsy=5.1*[1 1 -1 -1]'/2;
hsz=0.7*ones(4,1);
    ex=0.6*[0 -1 -1 0]'-9.4;            % elevator
    ey=5.1*[1 1 -1 -1]'/2;
    ez=0.7*ones(4,1);
vsx=[0 -1.7 -2.7 -1.8]'-7.3;            % vertical stabilizer
vsy=zeros(4,1);
vsz=[1.65 4.5 4.5 1.25]';
    rx=[-2.7 -3.3 -3.1 -1.8]'-7.3;      % rudder
    ry=zeros(4,1);
    rz=[4.5 4.5 1.5 1.25]';
pf=patch(fx,fy,fz,'c'); pw=patch(wingx,wingy,wingz,'c');
pc=patch(cx,cy,cz,'b'); ps=patch([hsx vsx],[hsy vsy],[hsz vsz],'c');
pu=patch([ax ax ex rx],[ay -ay ey ry],[az az ez rz],'m');
axis equal, axis off, view(135,20)
```

You can gather all individual patches together and have a single Patch object representing the entire aircraft:

```
>> haircraft=[pf pc pw ps pu];
```

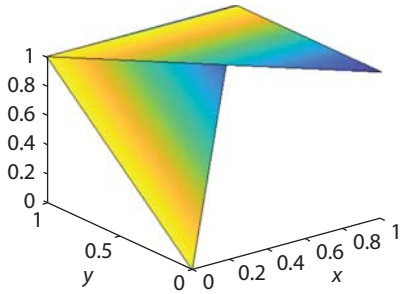
You may now use this object in different graphics manipulations; for example, you can rotate it using the command

```
>> rotate(haircraft,[25,35],-50)
```

resulting in what is shown in Fig. 6.46b.

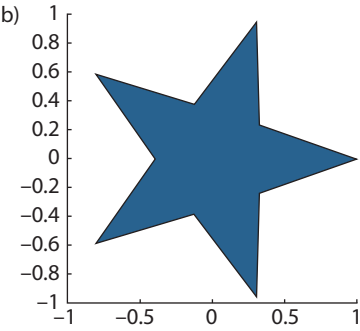
If needed, the command `g=surf2patch(X,Y,Z)` allows you to convert the geometry (and color data) from the Surface object defined by matrices `X`, `Y`, and `Z` into the patch format. It returns the face, vertex, and color data in the structure `g`. You can then pass this structure directly to the patch function as `p=patch(g)`, possibly followed by the `reducepatch(p,r)` call to reduce the number of patch faces (`r` is a fraction of the original number of faces).

a)



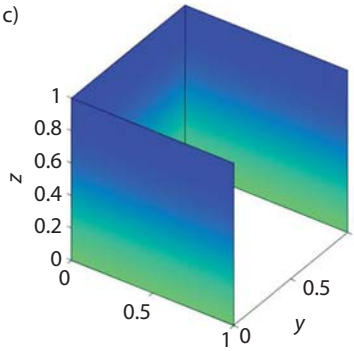

```
x = [0 1; 1 0; 0 1];
y = [1 0; 1 1; 0 1];
z = [1 1; 1 1; 0 1];
c = [1 0; 0 1; 1 1];
fill3(x,y,z,c)
xlabel('x'), ylabel('y')
```

b)




```
t=linspace(0,2*pi,11);
x=cos(t); y=sin(t);
x(2:2:10)=.4*cos(t(2:2:10));
y(2:2:10)=.4*sin(t(2:2:10));
patch(x,y,'b')
axis square
```

c)




```
x=[0 0 0 0; 0 1 1 0;0 1 1 0]';
y=[0 1 1 0; 0 0 0 0;1 1 1 1]';
z=[0 0 1 1; 0 0 1 1;0 0 1 1]';
patch(x,y,z,-z), colormap winter
axis square, view([35,35])
zlabel('z'), ylabel('y')
```

Fig. 6.45 Examples of the 3-D polygon and patch graphics objects.

Three functions, shown in Table 6.29 and Fig. 6.47, enable you to create the scatter plots (similar to their 2-D analogs shown in Fig. 6.29c) and to visualize volumetric data, slice by slice.

More vector field and volume visualization functions are also available (`coneplot`, `streamline`, `streamslice`, `streamparticles`, `streamtube`, and `streamribbon`). For more details, please refer to the MATLAB Help system.

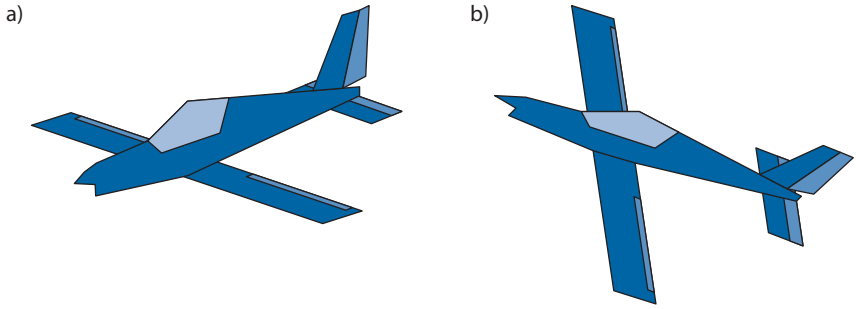


Fig. 6.46 Example of a) creating and b) rotating a complex 3-D object using patch graphics.

To conclude this section, let us mention that MATLAB also allows you to add a light object and to specify the light-reflecting characteristics of the surfaces and patches so that they look like the real solid objects. Figure 6.48 provides a couple of examples, and Table 6.30 lists the corresponding functions you can use.

In addition, Table 6.31 lists the functions that query and vary the **Camera**... properties of the Axes object (Table 6.8). These properties define the 2-D projection of the plot box onto the screen (Fig. 6.49) (see the "View Control with the Camera Toolbar" Help topic for more detail). These functions can be very useful in adding interactive behavior to your plots and creating animations as discussed in Sec. 6.12.

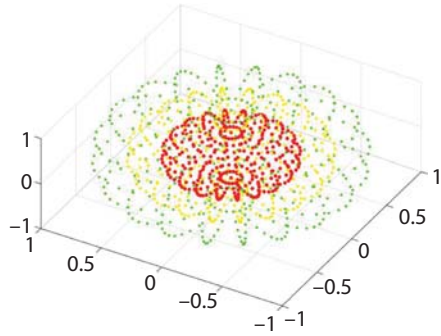
6.9 Changing the Color Palette

Table 6.5 presented eight preset colors you may use for lines, background, and text strings. For these applications, this is probably more than enough. However, when it comes to visualizing the surfaces and shapes, more colors might be needed. If you click any color-changing menu, for instance in the MATLAB Colors Preferences window shown in Fig. 2.30b, you will see that in fact there are millions and millions of colors to choose from

Table 6.29 The 3-D Scatter Plots and Volumetric Data Visualization Functions

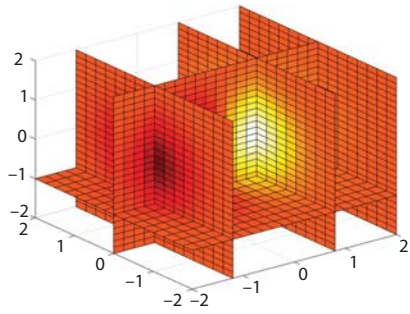
Function	Brief Description
`scatter3(x,y,z,s,c)`	Displays the colored circles of the size `s` at the locations specified by the same-size vectors `x`, `y`, and `z`, with `c` determining the colors of each marker
`slice(v,xs,ys,zs)`	Displays the orthogonal slice planes defined in `xs`, `ys`, and `zs` through volumetric data `v`
`contourslice(v,xs,ys,zs)`	Draws the contours in slices for the volumetric data `v` at slice locations `xs`, `ys`, and `zs`

a)



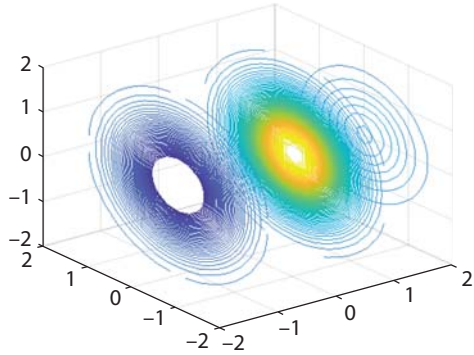
```
[x,y,z]=sphere;
X=[x(:)*0.5 x(:)*0.75 x(:)];
Y=[y(:)*0.5 y(:)*0.75 y(:)];
Z=[z(:)*0.5 z(:)*0.75 z(:)];
S=repmat([1 .75 .5]*10,prod(size(x)),1);
C=repmat([1 2 3],prod(size(x)),1);
scatter3(X(:),Y(:),Z(:),S(:),C(:),'filled')
view(-60,60)colormap prism
```

b)



```
[x,y,z]=meshgrid(-2:.2:2,-2:.25:2,-2:.16:2);
v=x.*exp(-x.^2-y.^2-z.^2);
figure, xsl=[-1.2,.8,2]; zsl=-1;
slice(x,y,z,v,xsl,0,zsl), colormap hot
figure, xslice=[-1.2,0.8,2]; lvls=-0.2:0.005:0.4;
```

c)



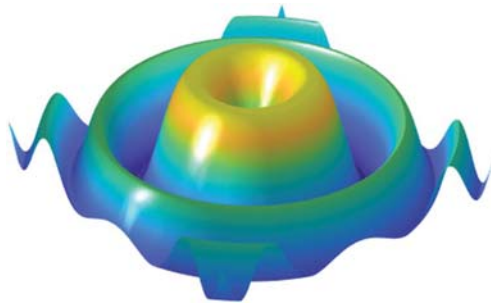
```
contourslice(x,y,z,v,xslice,[],[],lvls)
view(3),grid on
```

Fig. 6.47 Examples of the 3-D scatter plot and volume visualization.

(256^3 to be more precise). Each color is defined by three numbers: either the RGB or hue–saturation–value (HSV) triplet. Sometimes, the latter is also referred to as the hue–saturation–brightness (HSB) model.

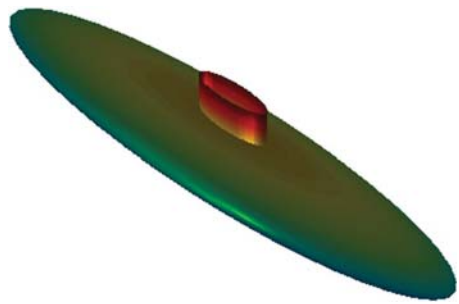
Figure 6.50a shows the concept of the HSV model, and Fig. 6.50b presents the options available when you fix one of the HSV values and try to vary other two. In this model, the first number, hue, ranges from 0 to 360 deg, with each degree representing a distinct color starting from red (0 deg) up to violet (360 deg) (yellow is 120 deg, green −180 deg, and blue −240 deg). The second number, saturation, represents the amount of color or, more exactly, its percentage, with its value ranging from 0 to 100,

a)



```
F=@(x,y)sin(x.^2+y.^2)./(x.^2+y.^2).^0.5;
h=fsurf(F,pi*[-1 1 -1 1],'EdgeColor','none');
shading interp
light('Color','w',...
      'Position',[-4 0 0],'Style','local')
material shiny,  h.AmbientStrength=0.7;
set(gca,'Visible','off'), view(-38,58)
```

b)



```
ellipsoid(0,0,0,2,10,1,40), hold
[x,y,z]=cylinder(0.5,40); axis equal
surf(0.8*x,3*y,1.5*z+0.5),shading interp,
light('Co','y','Po',[-4 0 -2],'St','lo')
material metal
set(gca,'Visible','off'), colormap jet
```

Fig. 6.48 Adding a light source and adjusting light-reflecting characteristics.

Table 6.30 The Lighting Functions

Function	Brief Description
`light(name,value,...)`	Creates a light in the current axes (lights affect only patch and surface objects), specifying its color, position, and style properties. The `'style'` property is either the default `'local'`, meaning that the light is a point source that radiates from the location in all directions, or `'infinite'`, corresponding to the light shining in parallel rays

`camlight('headlight')`
 	
`camlight(az,el)`
`camlight(...,style)`
	Creates a light at the camera position. Other options are `'right'` (default) and `'left'` creating a light right/left and up from the camera, respectively
	Creates a light at the specified azimuth `az` and elevation `el` with respect to the camera position. You can always change the position later using the `lightangle(az,el)` call, similar to that of the `view` function. You can also define the optional style argument to be `'local'` (default) or `'infinite'` (see the `light` function)

`lighting flat`	Produces uniform lighting across each of the faces of the faceted object. Another option, `gouraud`, calculates the vertex normals and interpolates lighting linearly across the curved faces

`material shiny`
 	
`material dull`
 	
`material metal`
	Sets the reflectance properties so that the object
	• has a high specular reflectance relative to the diffuse and ambient light, and the color of the specular light depends only on the color of the light source (see the AmbientStrength, DiffuseStrength, SpecularStrength, SpecularExponent, SpecularColorReflectance, and VertexNormals properties of the surfaces and patches);
	• reflects more diffuse light and has no specular highlights, but the color of the reflected light depends only on the light source; or
	• has a very high specular reflectance and very low ambient and diffuse reflectance, and the color of the reflected light depends on both the color of the light source and the color of the object

where 0 represents no color and 100 represents the full color. Finally, the third number, value or brightness, allows enhancing the color brightness by adding the white color or reducing it by adding the black color. In this case, 0 represents the white color and 100 represents the black color. (Note, there is yet another way to represent the RGB color model, hue-saturation-lightness (HSL); however it uses a slightly different color cylinder compared with that of Fig. 6.50a.)

The computer, however, "understands" RGB colors rather than HSV colors. Each pixel in your screen can be thought of as having three little bulbs that have RGB shades you can vary between 0 (bulb is off) and 255 (bulb is on). For more convenience, these numbers are scaled down to the [0;1] range (i.e., multiplied by 255^{-1}) so that the eight basic colors introduced in Table 6.5 have their RGB representation as shown in Table 6.32.

Table 6.31 Camera Graphics Functions

Function	Brief Description
`campos` `campos([x y z])`	Queries or sets the position of the camera as a three-element vector containing the *x*, *y*, and *z* coordinates in the data units (see CameraPosition and CameraPositionMode axes properties of the Axes object in Table 6.8)

`camva`
`camva(view_angle)`
`camva(mode)`
	Queries or sets the values of CameraViewAngle and CameraViewAngleMode properties (Table 6.8) (see example in Fig 6.63). The `view_angle` is defined in degrees, and `mode` is either the default `'auto'` (automatically selects the field of view as the minimum angle that captures the entire scene) or `'manual'`

`camzoom(factor)`	Zooms in (`factor`>1) or out (0<`factor`<1) on the scene (also changes CameraViewAngle and CameraViewAngleMode axes properties)
`camtarget` `camtarget([x y z])`	Queries or sets the camera target position (the camera points at) in the data units (CameraTarget and CameraTargetMode axes properties)

`camup`
`camup([x y z])`
`camup(mode)`
	Queries or sets the camera up vector mode in the current axes to the specified value (Camera UpVector and CameraUpVectorMode axes properties). In the `'auto'` `mode`, `[0 1 0]` is the up vector for 2-D views (meaning that the *y*-axis points up). For 3-D views, the up vector is [0 0 1], meaning the *z*-axis points up

`camdolly(dx,dy,dz)`	Moves the camera and camera target positions by the specified amounts `dx`, `dy`, and `dz`
`camlookat(h)`	Views the objects identified in `h` (in the case axes have multiple objects, like different shapes)
`camorbit(dpsi,dtheta,...)`	Rotates the camera position around the camera target by the amounts specified in `dpsi` (horizontal) and `dtheta` (vertical), both in degrees. You can also vary the center of rotation
`campan(dpsi,dtheta,...)`	Rotates the camera target around the camera position by the amounts specified in `dpsi` and `dtheta`
`camproj(projection)`	Sets the projection type to either `'orthographic'` (default) or `'perspective'`
`camroll(dphi)`	Rotates the camera around the camera viewing axis by the amount specified in `dphi` (in degrees)

Wherever you used the short names of these colors, they could be substituted with their full names or the corresponding RGB vector. Hence, to represent any other of 16 million-plus colors, you may now use an RGB triplet. In fact, it has been done once already—see code corresponding to Fig. 6.25a.

MATLAB offers about a dozen functions for managing colors, that is, performing color operations. Some of them are listed in Table 6.33.

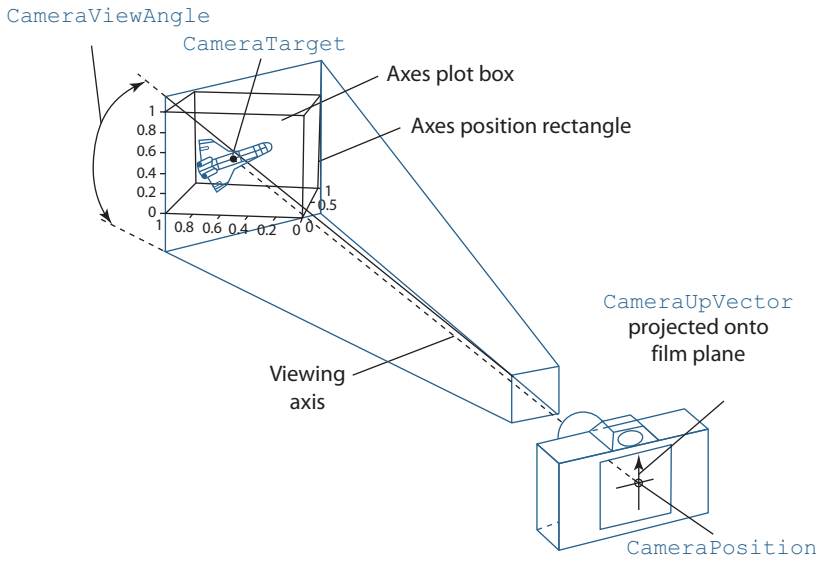


Fig. 6.49 Camera definition in terms of properties of the Axes object.

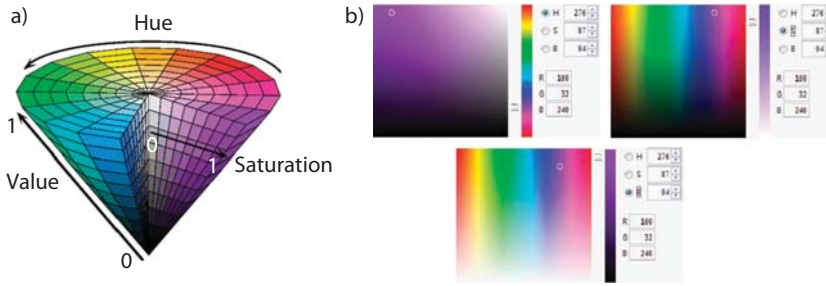


Fig. 6.50 The HSV color model.

Table 6.32 The RGB Values of Preset Colors

Short Name	Full Name	RGB Vector
'k'	'black'	[0 0 0]
'b'	'blue'	[0 0 1]
'c'	'cyan'	[0 1 1]
'g'	'green'	[0 1 0]
'm'	'magenta'	[1 0 1]
'r'	'red'	[1 0 0]
'w'	'white'	[1 1 1]
'y'	'yellow'	[1 1 0]

Table 6.33 Color-Varying Functions

Function	Brief Description
`colorbar`	Turns on a color bar by the plot showing the color scale (Figs. 3.20 and 6.27a)
`colormap`	Allows you to set and get the current colormap
`colormapeditor`	Starts the Colormap Editor, allowing you to modify the existing and create the new colormaps
`colororder(H)`	Sets the color order for the current figure with `H` being a three-column matrix of RGB triplets (introduced in R2019b)
`hsv2rgb`	Converts an HSV colormap to an RGB colormap
`rgb2hsv`	Converts an RGB colormap to an HSV colormap
`rgbplot`	Plots a colormap
`shading`	Sets color shading properties
`whitebg`	Changes the axes background color

Next, let us discuss the colormaps used throughout this chapter. Let us use the `rgbplot` function of Table 6.33 and visualize a few of them (Fig. 6.51)

```
tiledlayout(4,3,'TileSpacing','compact','Padding','compact')
nexttile, rgbplot(spring),  axis tight, text(150,0.15,'spring')
nexttile, rgbplot(summer),  axis tight, text(150,0.15,'summer')
nexttile, rgbplot(autumn),  axis tight, text(150,0.15,'autumn')
nexttile, rgbplot(winter),  axis tight, text(150,0.15,'winter')
nexttile, rgbplot(bone),    axis tight, text(150,0.15,'bone')
nexttile, rgbplot(copper),  axis tight, text(150,0.15,'copper')
nexttile, rgbplot(hsv),     axis tight, text(150,0.15,'hsv')
nexttile, rgbplot(parula),  axis tight, text(150,0.15,'parula')
nexttile, rgbplot(cool),    axis tight, text(150,0.15,'cool')
```

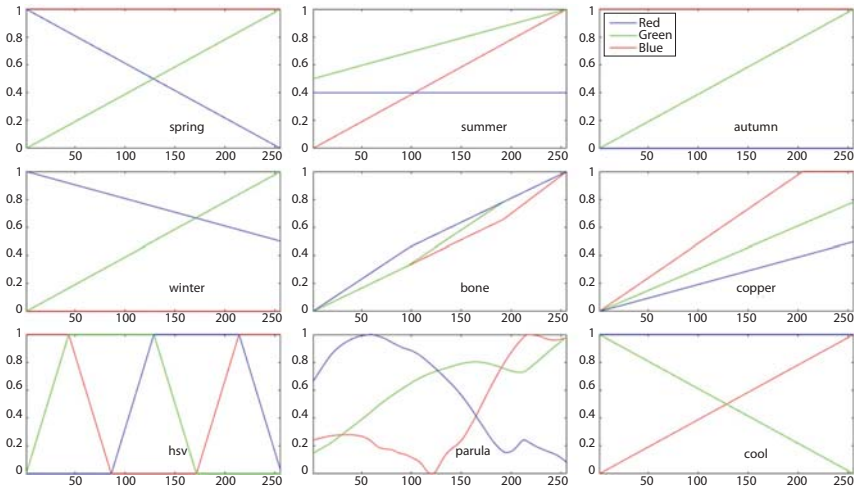


Fig. 6.51 Examples of preset colormaps.

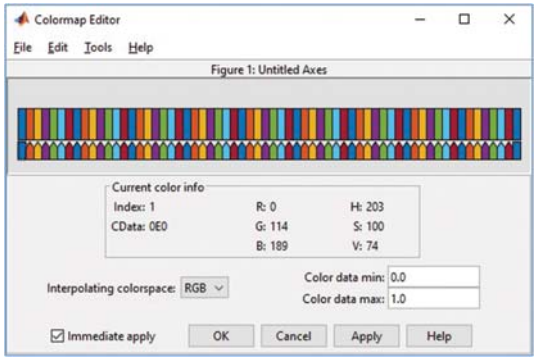
It turns out that these colormaps are simply m-by-3 matrices of real numbers between 0.0 and 1.0, so that each row represents an RGB vector defining one color. All together, they specify some particular color palette. These color matrices show how RGB colors change their "weight" across a color scale. The `colormap` function allows you to change a color palette. The preset palettes (colormaps) of MATLAB, most of which we have used in this and the previous sections already, are listed in Table 6.34.

As of R2020a, for all predefined colormaps, $m = 256$ (that is, there are 256 gradations). However, you can easily change this. For instance,

Table 6.34 Preset Colormaps

Function	Brief Description
`autumn`	Varies smoothly from red, through orange, to yellow (Fig. 6.25b)
`bone`	Presents a grayscale color palette with a higher value for the blue component and is especially useful for adding an "electronic" look to grayscale images (Fig. 6.25c)
`colorcube`	Contains as many regularly spaced colors in RGB color space as possible while attempting to provide more steps of gray, pure red, pure green, and pure blue
`cool`	Consists of the colors that vary smoothly from cyan to magenta (Fig. 6.42b)
`copper`	Varies smoothly from black to bright copper (Fig. 6.41c)
`flag`	Consists of red, white, blue, and black colors (this colormap completely changes color with each index increment)
`gray`	Returns a linear grayscale colormap (Fig. 6.41b)
`hot`	Varies smoothly from black through shades of red, orange, and yellow, to white
`hsv`	Varies a hue component of the HSV color model (Fig. 6.50), so that this colormap is particularly appropriate for displaying periodic functions (Fig. 6.26f)
`jet`	Ranges from blue to red and passes through the colors cyan, yellow, and orange (this colormap is a variation of the `hsv` map and was specifically designed for astrophysical fluid jet simulation) (Fig. 6.48b)
`lines`	Returns an m-by-3 matrix containing a "ColorOrder" colormap used when plotting lines (these colors are easy to distinguish even for people with certain types of color blindness) (introduced in R2014b and visualized in Fig. 6.52). You can change these colors as well as the number of cycling colors using the `colororder` function of Table 6.33
`parula`	Varies colors from dark to light and is perceptually uniform with smooth changes in data corresponding to smooth changes in color, whereas sharp changes in data appear as sharp changes in color (introduced in R2014b, slightly modified in R2017a)
`pink`	Contains the pastel shades of pink and provides sepia tone colorization of grayscale photographs (Fig. 6.26b)
`prism`	Repeats the six colors: red, orange, yellow, green, blue, and violet (Fig. 6.47a)
`spring`	Consists of colors that are shades of magenta and yellow (Fig. 6.41a)
`summer`	Consists of colors that are shades of green and yellow (Fig. 6.26a)
`white`	Presents an all-white monochrome colormap
`winter`	Consists of colors that are shades of blue and green (Fig. 6.45c)

a)



b)

```
>> disp(lines(8))
         0    0.4470    0.7410
    0.8500    0.3250    0.0980
    0.9290    0.6940    0.1250
    0.4940    0.1840    0.5560
    0.4660    0.6740    0.1880
    0.3010    0.7450    0.9330
    0.6350    0.0780    0.1840
         0    0.4470    0.7410
```

Fig. 6.52 a) Colormap Editor and b) seven `lines` colormap colors.

`colormap(gray(200))` creates a linear grayscale colormap with 200 colors (Have a look at just a three-color colormap, created and used in Fig. 6.42a). Of course if needed, you can use the `colormapeditor` function to bring up the *Colormap Editor* and create a brand new colormap (Fig. 6.52a).

Figure 6.52a features the `lines` colormap, which is where the colors are drawn from when producing the multiline graphs. Figure 6.52b shows the RGB triplets for the seven colors this colormap cycles (the command `colororder` would have the same return). These are the values of the Colormap and ColorOrder properties of the Axes object (Table 6.8).

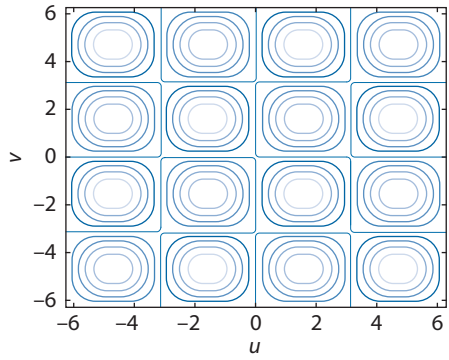
6.10 Function-Plotting Functions

MATLAB provides a collection of plotting functions that allow you to visualize mathematical expressions and functions, without the burden of computing the arrays of points, by simply specifying the function handle(s) to a named or anonymous function(s). (Of course, the data points are still being computed, but internally.) Some of these *function-plotting functions* have already been introduced in Table 6.1 (`fplot` and `fimplicit`), but the full list is shown in Table 6.35.

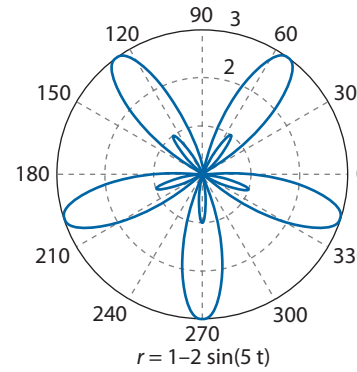
The `ezplot` function, also introduced in Table 6.1 and used throughout this chapter, belongs to another family of the plotting functions, *easy-to-plot functions*, which MATLAB relied on before R2016a–R2016b. This family was composed of the `ezcontour`, `ezcontourf`, `ezmesh`, `ezmeshc`, `ezplot`, `ezplot3`, `ezpolar`, `ezsurf`, and `ezsurfc` functions. As mentioned already, these functions are expected to be removed in a future release, although as of R2020a they were still functional. Moreover, as seen from Table 6.35, the `ezpolar` function does not yet have an analog in the function-plotting family of functions.

Table 6.35 Function-Plotting Functions

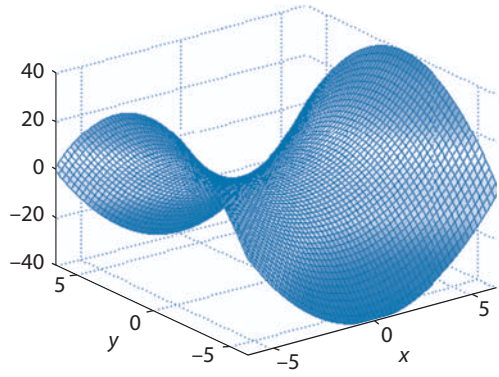
Function	Brief Description
`fcontour(f)`	Plots the contour lines of the function of two variables $f(x,y)$ defined by `f`
`fimplicit(f)`	Plots an implicit function defined by $f(x,y) = 0$
`fimplicit3(f)`	Plots an implicit function defined by $f(x,y,z) = 0$
`fmesh(f)`	Creates a mesh plot of $f(x,y)$
`fplot(f)`, `fplot(fx,fy)`	Plots a function of one variable $f(x)$ or a parametric curve $x(t) - y(t)$ defined by `fx` and `fy`, respectively
`fplot3(fx,fy,fz)`	Plots a parametric curve $x(t) - y(t) - z(t)$ defined by `fx`, `fy`, and `fz`, respectively
`fsurf(f)`	Creates a surface plot of $f(x,y)$
`ezpolar(f)`	Plots a polar curve $\rho = f(\theta)$




```
fcontour(@(u,v)sin(u).*sin(v),...
         2*pi*[-1,1])
```




```
ezpolar('1-2*sin(5*t)')
```




```
fsurf(@(x,y)x.^2-y.^2)
```

Fig. 6.53 Examples of the plots produced with the function-plotting and easy-to-plot functions.

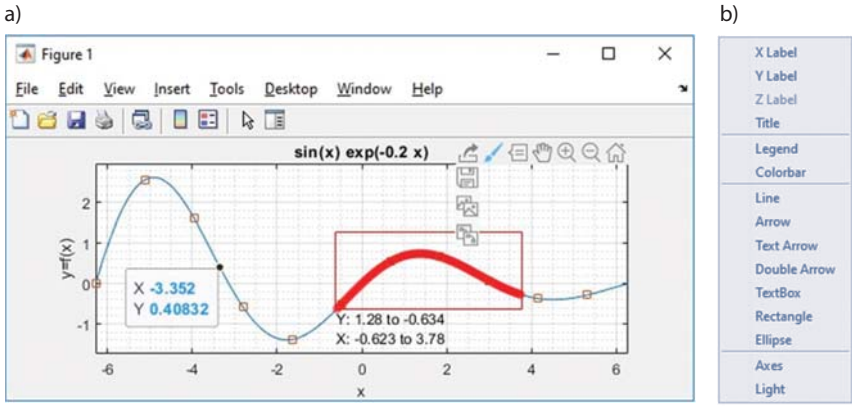


Fig. 6.54 a) Figure window and b) **File** menu.

Figure 6.4 provided some examples of using `fplot` and `fimplicit` functions, and some more examples are shown in Fig. 6.53. It should be mentioned that all the functions of Table 6.35 allow you to have more input arguments. That includes redefining the default range of $[-5;5]$ ($[0;2\pi]$ for `ezpolar`), setting the basic line specifications (style, marker, color), and other properties using one or more name–value pair arguments.

Last, it should be mentioned that as an input argument, the functions of Table 6.35 accept not only the function handle but also a symbolic expression or function (easy-to-plot functions also accept character vectors or strings). Examples will be provided in Sec. 7.9.2.

6.11 Plot Editing

Graphics shown in all figures so far were copied from the Figure window and pasted using the **Copy Figure** option of the Figure window's **Edit** menu (as mentioned at the end of Sec. 2.7). Let us discuss now what manual editing options the Figure window has to offer. To this end, Fig. 6.54a shows the very first plot of this chapter shown in Fig. 6.1 (with a couple of features that will be discussed later), but this time it was copied with <Alt>+<PrtScn>, that is, taking the screenshot of the active window. The Figure window features the menu bar, the toolbar below it, and the axes toolbar in the upper-right corner of the Axes object (if it is not visible, just hover over the axes and it will show up).

Five of eight drop-down menus in the menu bar are shown in Fig. 6.55a–e, with brief explanations of their options provided in Table 6.36. Figure 6.56 shows the same Figure window but with almost all toolbars and additional panels (shown in Fig. 6.55b) opened. Note, the Property Inspector window on the right visualizes all properties of the selected graphics object grouped into several larger categories for your convenience. For the Line,

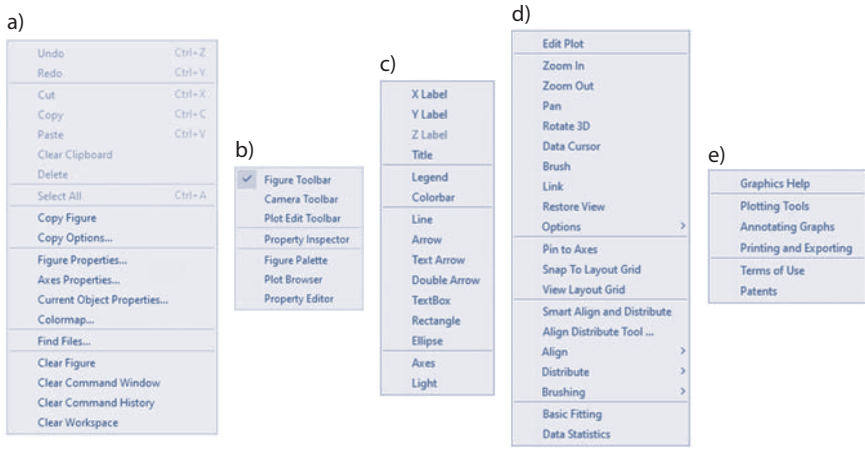


Fig. 6.55 a) **Edit**, b) **View**, c) **Insert**, d) **Tools**, and e) **Help** menus.

Axes, Figure, and Surface objects, these properties are exactly the same as those of Tables 6.7–6.9 and 6.23).

Figure 6.56 also features the Camera toolbar (which can also be added programmatically using the `cameratoolbar` command). The first seven buttons enable camera motion controls; the next four allow you to select a

Table 6.36 Figure Window Menus

Menu	Brief Description
File	Assists in opening, saving, exporting, and printing the figure. It also allows importing data, saving the workspace and figure setup, and even generating a separate all-sufficient *m*-script for the current Figure window
Edit	Helps with cutting, copying, and pasting the entire figure as well as separate items such as legend or text titles; allows exploring graphics object properties in the Property Inspector window (Fig. 6.56) and managing other MATLAB windows
View	Allows bringing up additional toolbars and panels to edit the figure and its children objects (Fig. 6.56)
Insert	Allows inserting text and annotations (see Table 6.3), legend and colorbar, additional Axes objects and light
Tools	Enables a variety of capabilities including editing, zooming, panning, 3-D rotating, aligning, distributing, basic fitting, and data statistics (Fig. 6.57b–d)
Debug	Allows debugging the script the figure was created in
Desktop	Allows docking the Figure window with the MATLAB Desktop (sometimes, it is more convenient because this way you do not have to search for the opened Figure window among multiple other windows that might also be open)
Window	Allows arranging multiple figure windows on the screen and also switching between the opened Figure windows and MATLAB windows (Command Window, Command History, Current Directory, Workspace, etc.)
Help	Provides assistance with the different plotting-related issues

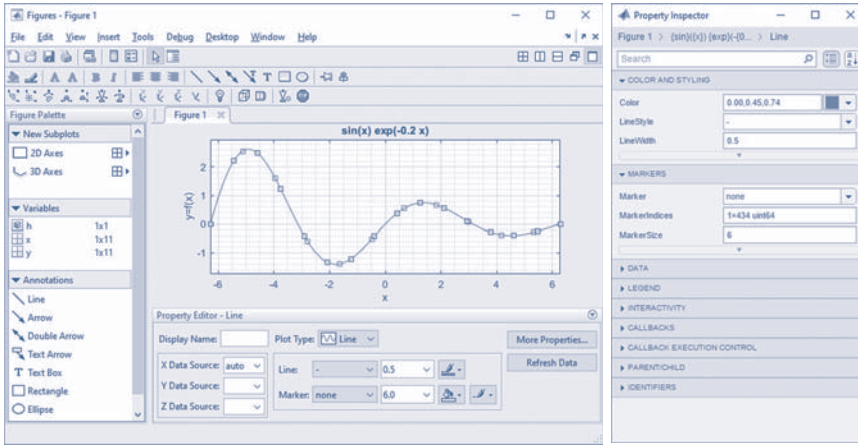


Fig. 6.56 Figure window with all but Plot Browser tools of the **View** menu opened.

principal axis (x, y, z, or none); and the remaining buttons allow you to add light, choose between orthographic and perspective projections, reset the camera and scene light, and stop camera/light motion (Fig. 6.49). The enabling functions have been presented in Table 6.31 already).

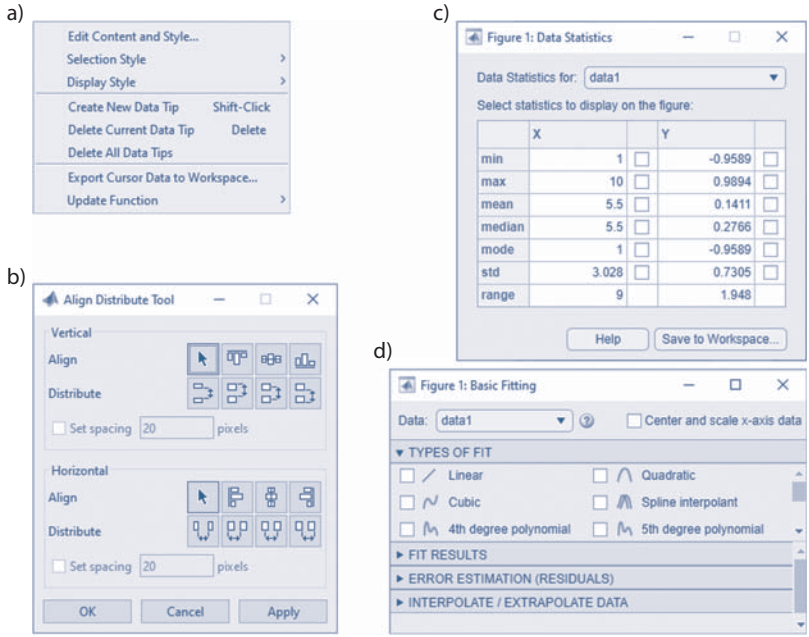


Fig. 6.57 a) Line context menu (with the **Data Cursor** icon activated), b) Align Distribute Tool, c) Data Statistics, and d) Basic Fitting tools of the **Tools** menu.

The first four icons of the Figure toolbar (Fig. 6.54a or Fig. 6.56) allow you to create a new figure, open an existing figure, save, and print the opened figure (with the default *fig* extension). The next icon turns on data linking for the current figure, resulting in automatic updating of the graphs when variables in the base or function's workspace change (corresponds to the `linkdata on` command). The next two icons add the colorbar and legend, respectively. The **Edit Plot** button (with an arrow icon on it) allows you to select a specific graphics object. The last icon brings up the Property Inspector for the currently selected graphics object (or for the figure object itself if no selection was made).

The Axes toolbar introduced in R2018b contains several data exploration buttons for quick access to some options of the **Tools** menu in Fig. 6.55d. Specifically, the Axes toolbar of Fig. 6.54a features the **Export**, **Brush**, **Data Cursor**, **Pan**, **Zoom In**, **Zoom Out**, and **Restore View** icons (for the 3-D plots, the **Rotate** icon would also show up before **Pan**).

The first option of the **Export** icon drop-down menu, added in R2019a, allows you to save the axes as a tightly cropped image or PDF file, which was a long-overdue feature to simplify preparation of graphical results for journal publication. Starting with R2019b, you can also copy the contents of a set of axes to a clipboard as a PNG image or vector graphics for PDFs (the second and third options of the **Export** drop-down menu, and , respectively, as shown in Fig. 6.54a).

The second and third icons of the Axes toolbar are used to interactively mark a portion of the plot (Fig. 6.54a shows the process of dragging a selection rectangle around some data points of interest) and place a data tip in a small text box that shows up when you position the cursor over a data point and hover there. The brushing mode (which can also be activated programmatically using the `brush` function) allows you to delete, modify (replacing with `NaN` or constant values), and save selected observations. With the **Data Cursor** icon of the Axes toolbar activated (alternatively you can do it programmatically using the `datacursormode` function), a right-click the line brings up the menu shown in Fig. 6.57a. This menu offers some additional options, including displaying the data values in a stationary cursor window inside the Figure object that you can position anywhere (as opposed to just moving a data tip to one of four quadrants around the point as shown in Fig. 6.54a), creating multiple data tips, and exporting data values to the workspace.

If required, the `axtoolbar` function allows replacing the default toolbar with a toolbar that contains only the specified buttons. For example, the

```
tb=axtoolbar({ 'pan','restoreview' });
```

command reduces the number of icons to just a specified two and creates an AxesToolbar object, `tb`, that you can use to add other icons using the `axtoolbarbtn` function, like

```
axtoolbarbtn(tb,'push','Icon', 'rotate');
```

The buttons appear in a standard order on the toolbar regardless of the order in which you specify them (each button can appear only once). It should be noted that, prior to R2018b, the data exploration buttons were part of the Figure toolbar. The two commands `addToolbarExplorationButtons(gcf)` and `removeToolbarExplorationButtons(gcf)` allow you to display these buttons between the **Link Plot** and **Insert Colorbar** buttons of the Figure toolbar and remove them, respectively.

The pop-up windows for three options of the **Tools** menu of Fig. 6.55d are shown in Fig. 6.57c–d. The Basic Fitting tool will be considered in Sec. 11.6.2 (Fig. 11.15).

The plotting tools shown in Fig. 6.56—Figure Palette, Plot Browser, and Property Editor (see the lower portion of the View menu of Fig. 6.55b)—can also be brought up programmatically by issuing the `plottools` call in the Command Window. If you want to bring up any of them separately, you should use `figurepalette`, `plotbrowser`, and `propertyeditor` functions, respectively. The first time you open the plotting tools, all three of them appear grouped around the current figure. If you close, move, or undock any of the tools, MATLAB remembers the configuration you left them in and restores it when you invoke the tools for subsequent figures. The **Property Inspector** can also be opened programmatically using the `inspect` command. The content of the Property Editor and Property Inspector will change depending on what graphics object is selected.

All these tools allow editing of the figure and all other graphics objects in it without going back and changing your code, that is, after the Figure window was created. Moreover, the plotting tools allow you to create plots from scratch, without even knowing how to do it programmatically! To do this, you should load data into the current folder, say, from Excel using the Import Wizard, then open the plotting tools and develop the layout you want (as many different plots as you want, any notations and shapes you would like to have, etc.). You may also align and distribute the objects within the figure space (using the **Tools** menu).

Once you are done editing your figure content, you go to the **File** menu (Fig. 6.54b) and choose the **Generate Code** option. Believe it or not, MATLAB will generate an *m*-script, describing the complete layout for the figure you just created interactively, automatically! You may save this file and call it up later to visualize another, new set of data using the same layout. If you are interested in seeing code for some particular graphics object rather than for the entire figure, right-click this object and choose the **Show Code** option from the context menu.

Starting from R2020a, those three options of the **Export** icon drop-down menu (the first icon of the **Axes** toolbar), , , and (Fig. 6.54a), have

their programmatic representation via the `exportgraphics` (which is especially convenient for batch processing) and `copygraphics` functions. Specifically, the call

```
exportgraphics(obj,filename,name,value,...)
```

saves the contents of the graphics object `obj` to a file `filename` with optional comma-separated `name-value` pair arguments specifying additional options. The graphics object can be any type of axes, a figure, a chart that can be a child of the figure, or a tiled chart layout. The content is tightly cropped to a thin margin surrounding `obj`. For example,

```
fimplicit(@(x,y) y.*sin(x)+5*x.*cos(y)-1)
exportgraphics(gca,'myplot.jpg','Resolution',300)
```

saves the contents of the current axes as a 300-DPI (dots per inch) image file *myplot.jpg*. (The default resolution is 150 DPI.)

You can export to JPEG, PNG, TIFF, PDF, EMF, or EPS file format (defined by the file extension). When saving as a PDF, EMF, or EPS file, you may also specify another `name-value` pair argument—the `'ContentType'`, which may assume one of three options:

`'auto'` Stores either vector graphics or image data based on a heuristic (the default option)
`'vector'` Stores the contents as vector graphics that can scale to any size (if you are saving a PDF file, embeddable fonts are included in the file); specifying the resolution in this case has no effect
`'image'` Rasterizes the content into one or more images within the file

You can also specify the background color, which controls the color of the margin that surrounds the axes or chart, `'BackgroundColor'`. It can be `[1 1 1]` (default), `'current'`, `'none'`, RGB triplet, or one of those eight preset color names of Table 6.5. A value of `'none'` sets the background color to transparent (if `'ContentType'` is `'vector'`) or white (when `'ContentType'` is `'image'`).

The `copygraphics` function copies the content of the graphics object specified by `obj` to the system clipboard (to be then pasted into other applications and documents). Everything discussed above for the `exportgraphics` function holds here as well (except that you do not need to specify `filename`).

Let us conclude this section with one practical tip for those using earlier versions of MATLAB. If you want to copy the entire Figure to your Microsoft Word document or any other application without saving it on a disk, you should click **Edit** and choose the **Copy Figure** option (Fig. 6.55a). Now it resides in the clipboard and is ready to be pasted. To get rid of the gray background, click **Edit**, **Copy Options** and check the **Force white background** option in the corresponding Preferences window (Fig. 2.32a) rather than

Use figure color (a default value), as shown in Fig. 2.32a. Note that you only have to do this once because MATLAB saves this change and then uses the white background every time you copy a Figure into the clipboard after this.

6.12 Interactive Plotting and Animation

MATLAB provides a variety of tools for interactive plotting, as well as creating and storing animations. This section presents several examples of how these tools can be used.

To start with, the `ginput` function enables you to use the mouse or arrow keys to select the points to plot. The `[x,y]=ginput` command returns the $x-y$ coordinates of the pointer's position when a mouse button or any key on the keyboard, except a carriage return, is pressed. It gathers an unlimited number of points until the <Return> key is pressed. The `[x,y]=ginput(n)` command gets exactly `n` points. The `[x,y,button]=ginput(n)` command along with the vectors `x` and `y` returns a third vector, `button`, that contains integers specifying which mouse button was used (1st, 2nd, 3rd from the left) or ASCII numbers if the keyboard was used. The following example illustrates the use of `ginput` with the `spline` function to create a curve by parametric 2-D interpolation (to be considered in Sec. 11.6.2):

```
axis([0 10 0 10]), hold on
xy = []; n = 0; % Initially, the list of points is empty
%% Pick up multiple points by clicking the left mouse button
% (Right mouse button means you are picking the last point)
but = 1;
   while but == 1
         [xi,yi,but] = ginput(1);
         plot(xi,yi,'ro')
         n = n+1;
         xy(:,n) = [xi;yi];
   end
%% Interpolate with a spline and use a finer spacing
t = 1:n; ts = 1:0.1:n;
xys = spline(t,xy,ts);
%% Plot the interpolated curve
plot(xys(1,:),xys(2,:),'b-'); hold off
```

First, a sequence of $x-y$ points, (`xi`, `yi`), is selected in the `while` loop and stored in the 2-by-`n` array `xy`. Then, two 1-D splines (one for x data and another one for y data stored in the first and second rows of `xy`, respectively) are passed through all data points, evaluating them with a spacing 1/10 of the original spacing. The plot in Fig. 6.58 shows a typical output of the script featuring nine original data points.

Next, let us discuss the line animation functions listed in Table 6.37.

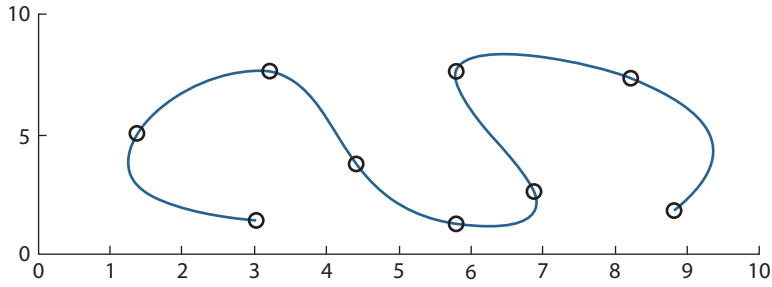


Fig. 6.58 Example of interactive plotting.

The `comet` and `comet3` functions of Table 6.37 can simply be used as a substitute for `plot` and `plot3`, respectively. Both functions produce a *comet plot*, an animated graph in which a circle (the comet head) traces the data points on the screen. The comet body is a trailing segment that follows the head. The tail is a solid line that traces the entire function.

Table 6.37 Line Animation Functions

Function	Brief Description
`an=animatedline(x,y,z)`	Creates an animated line with the initial data points defined by `x`, `y`, and `z` (in the 3-D case). As usual, one or more name–value pair arguments can be used to specify animated line properties
`addpoints(an,x,y,z)`	Adds the points defined by `x`, `y`, and `z` to the animated line specified by `an`
`[x,y,z] = getpoints(an)`	Returns the `x`, `y`, and `z` coordinates for the points that define the animated line specified by `an`
`clearpoints(an)`	Clears all the points from the animated line specified by `an`
`comet(x,y)`	Displays a comet, an animated graph in which a circle (the comet head) traces the `x`-`y` data points on the screen
`comet3(x,y,z)`	Displays a comet graph of the curve through the points `x`, `y`, and `z`
`pause`	Temporarily stops MATLAB execution and waits for the user to press any key (`pause(n)` pauses execution for `n` seconds before continuing)
`drawnow`	Updates the figure and processes any pending callbacks (see Sec. F.3 of Appendix F). Use this command if you modify graphics objects and want to see the updates on the screen immediately as they are created. `drawnow limitrate` limits the number of updates to 20 frames per second and discards the new updates if the graphics renderer is busy. Returning to the MATLAB prompt; using the `getframe`, `pause`, and `waitfor` functions; and using functions that wait for user input, such as `waitforbuttonpress`, `input`, `keyboard`, or `ginput`, are all equivalent actions
`refreshdata`	Refreshes all object data of the objects in the specified figure

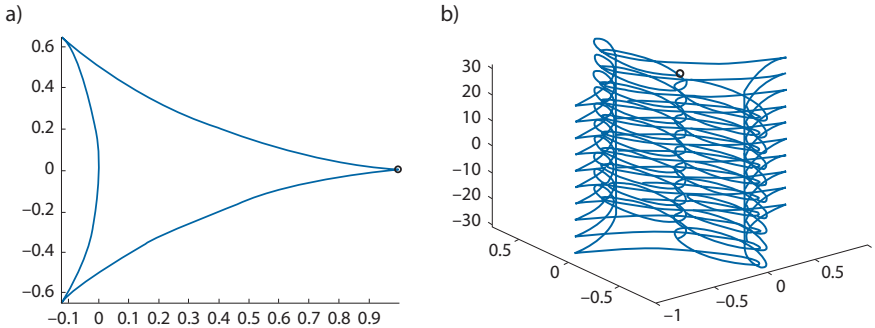


Fig. 6.59 Outputs of the a) `comet` and b) `comet3` functions.

The following script provides an example of using these functions, with the final outputs shown in Fig. 6.59:

```
figure('Name','Comet','Color','w')
t=0:.0005:pi;
x=cos(2*t).*(cos(t).^2);
y=sin(2*t).*(sin(t).^2);
comet(x,y);
pause
figure('Name','Comet3','Color','w')
t=-10*pi:pi/350:10*pi;
comet3((cos(2*t).^2).*sin(t),(sin(2*t).^2).*cos(t),t);
```

Note how we also used another function from Table 6.37, `pause`, inserted in between two plotting commands. What happens is that after producing the first comet plot (Fig. 6.59a), the execution of the script stops until any key is pressed on the keyboard, followed by another comet plot being produced (Fig. 6.59b). (Another syntax, say, `pause(3)`, would cause a 3-s pause in between the two comet plots.)

Compared with `comet` and `comet3`, the first four functions of Table 6.37, introduced in R2014b, allow some control over how the line is animated. Consider the following piece of code:

```
  x=linspace(0,4*pi,200); y=sin(x)+0.5*rand(1,length(x));
L=animatedline('Marker','p','Color','b','LineStyle','--');
  axis([min(x),max(x),min(y),max(y)]), grid
  xlabel("x-axis"), ylabel("y-axis")
% Add a status bar
  rectangle('Position',[6.5 -0.9 3 0.3]);
G=rectangle('Position',[6.5 -0.9 0 0.3],'FaceColor','g');
T=text(7.7,-0.73,' 0%');
% Animate
for k=1:length(x)
    addpoints(L,x(k),y(k));
```

```
    G.Position(3)=1.5*k/10^2;
    T.String(1:3)=pad(int2str(round(100*k/length(x))),3,'left');
    drawnow % pause(0.05) works as well
end
```

It computes all data points; creates the AnimatedLine object `L`, specifying some of its properties; and then keeps adding the data points to this line in the `for` loop referring to `L`. Also added to this script is a status bar showing how much data (in percent) have been added to the plot up to this point (compare with Fig. 4.19b). The `rectangle` function is used to add a bar frame, another `rectangle` (with the retrieved Rectangle object in `G`) is used to show the filled portion of the status bar, and the `text` function (with the retrieved Text object in `T`) is used to add a percentage. Then, in the `for` loop, only a few properties of objects `G` and `T` are being changed (the width of a filled portion of the status bar and the first three characters of the percentage). The snapshot of this animation is shown in Fig. 6.60a.

The usage of the `drawnow` function allows you to see the result of each cycle execution within the `for` loop. Without this function, the loop would be executed so fast that you could not see the animation but rather only the final result. You can use the `pause` function here as well, for example, pausing for 1/20 of a second at each cycle.

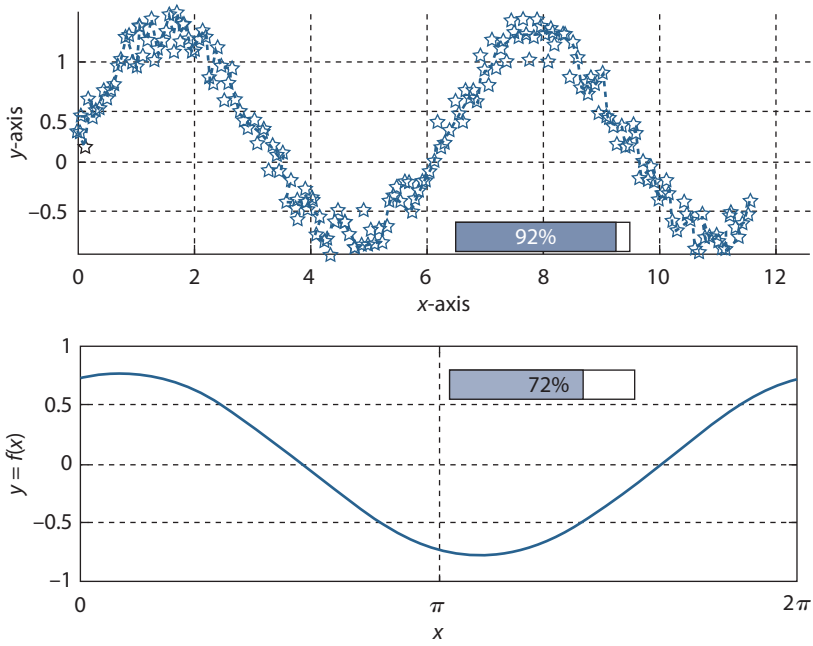


Fig. 6.60 Animations created a) using the `animatedline` and `addpoints` functions and b) varying the Line object's YData property.

Let us make an important point here. In the preceding example, you could have avoided using the `animatedline` and graphics objects altogether. Instead, inside the loop you could have used the `line(x(1:k),y(1:k),...)` call, full-fledged `annotation(...)` call, and `text(...)` call to create the brand new objects at each loop cycle. It would work just fine, except that it would definitely slow the execution speed of your graphics code. Although you might not even notice it for the simple code we considered here, performance would become a major issue when working with the large amounts of data and large number of objects. Hence, always try to update the existing objects rather than create the new ones; moreover, update only the dynamic part of the existing objects, leaving the static portion unchanged. Also, when creating many objects, keep in mind that the lower-level graphics functions (`line` vs `plot`, `surface` vs `surf`, `rectangle` vs `annotation`) perform fewer operations and can therefore be faster.

Consider another example in which the same technique (varying certain properties of the existing graphics objects) is used to slowly convert a sine wave into a cosine wave:

```
figure, subplot 211
x=linspace(0,2*pi);    % define x data
y=sin(x); z=cos(x);    % define f(x) data
plot(x,y), grid        % plot the sin(x) wave
  set(gca,'xlim',[0 2*pi],'ylim',[-1 1]);
  set(gca,'XTick',[0:pi:2*pi])
  set(gca,'XTickLabel',{'0';'\pi';'2\pi'})
  xlabel('x'), ylabel('y=f(x)')
h_line=get(gca,'children'); % retrieve the Line object
% Add a status bar
  annotation('rectangle',[0.53 0.6 0.2 0.04]);
G=annotation('rectangle',[0.53 0.6 0 0.04],...
             'FaceColor','g','FaceAlpha',0.2);
T=text(3.85,-0.788,' 0%');
% Animate
for i=1:200
    drawnow              % pause(0.0005)
    w=i/200;             % set the weighting coefficient
    d=(1-w)*y+w*z;       % blend y(x) and z(x)
set(h_line,'ydata',d); % change line's YData property
G.Position(3)=i/10^3;
T.String(1:3)=pad(int2str(round(i/2)),3,'left');
    drawnow              % pause(0.0005)
end
```

In the first four lines of this code, we compute data for two dependences, $y = \sin(x)$ and $z = \cos(x)$, and plot the first dependence. The next four lines set the x- and y-axis limits along with the x-axis ticks and axis labels.

Next, we retrieve the Line object line and then change its YData property (keeping the remaining properties untouched) in the `for` loop according to

$$\boldsymbol{y}_i = (1 - w_i)\sin(\boldsymbol{x}) + w_i \cos(\boldsymbol{x}), \text{ where } w_i = \frac{i}{200}, \quad i = 0, 1, \ldots, 200 \quad (6.1)$$

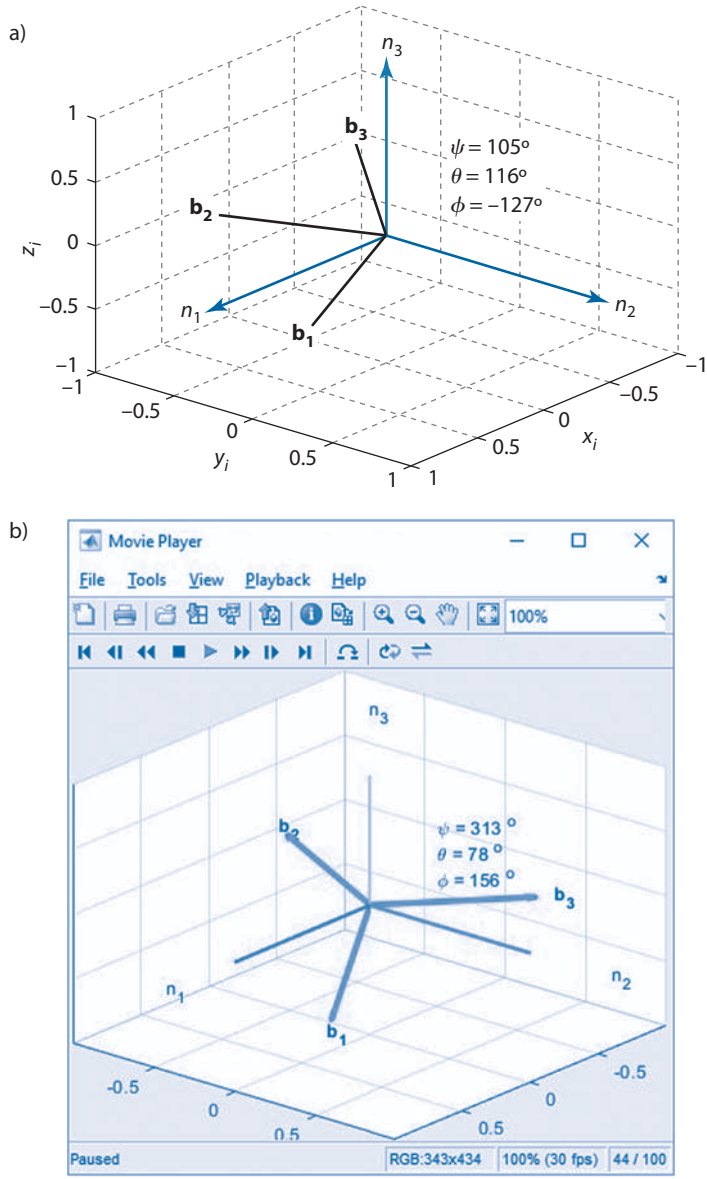


Fig. 6.61 Snapshot of a) animated axis rotation and b) the MATLAB Video Viewer app.

When $i = 0$, $w_i = 0$ and, therefore, $y_i = \sin(x)$. On the contrary, when $i \to 200$, $w_i \to 1$, and $y_i \to \cos(x)$. As a result, we see a smooth conversion of a sinusoid to a cosine wave (as grabbed in Fig. 6.60b).

By the way, note how the `annotation` function was used to create a status bar instead of the `rectangle` function in this case. Not being able to use the data units makes defining a rectangle's position tricky. Also, the FaceAlpha property should be altered to allow you to see the percentage numbers.

Consider a more sophisticated example. Suppose you need to graphically represent the rotation of the body coordinate frame (tied to a satellite) with respect to some inertial frame. Running the following code, stored in one *m*-file and composed of the main script and local function `Euler2DCM` (which computes the rotation matrix), results in animation, where one axis triad rotates with respect to another (Fig. 6.61a shows the tenth frame):

```
quiver3(0,0,0,1.5,0,0,'LineWidth',2), hold on
quiver3(0,0,0,0,1.5,0,'LineWidth',2)
quiver3(0,0,0,0,0,1.5,'LineWidth',2)
     text(1.5,0,0,'n_1'); text(0,1.5,0,'n_2'); text(0,0,1.5,'n_3')
axis ([-1 1 -1 1 -1 1]), view(130,30)
xlabel('x_i'), ylabel('y_i'), zlabel('z_i');
R=eye(3);
h(1)=quiver3(0,0,0,R(1,1),R(1,2),R(1,3),'m','Linewidth',3);
h(2)=quiver3(0,0,0,R(2,1),R(2,2),R(2,3),'m','Linewidth',3);
h(3)=quiver3(0,0,0,R(3,1),R(3,2),R(3,3),'m','Linewidth',3);
     ht(1)=text(R(1,1),R(1,2),R(1,3),'\bf{b_1}');
     ht(2)=text(R(2,1),R(2,2),R(2,3),'\bf{b_2}');
     ht(3)=text(R(3,1),R(3,2),R(3,3),'\bf{b_3}');
ha(1)=text(-0.5,0,0,'\phi = 0 ^o');
ha(2)=text(-0.5,0,0.2,'\theta = 0 ^o');
ha(3)=text(-0.5,0,0.4,'\psi = 0 ^o');
       aviobj=VideoWriter('axisrotation.avi');      % create VR object
       open(aviobj)          % open the VideoWriter (VR) object
   for i = 1:100             % animation loop
   psi=4*pi*(i-1)/99; phi=2*pi*(i-1)/99; theta=pi*(i-1)/99;
   R=Euler2DCM(psi,theta,phi);
     for j=1:3
     set(h(j),'UData',R(j,1),'VData',R(j,2),'WData',R(j,3));
     set(ht(j),'Position',[R(j,:)]);
     end
phid=mod(phi*180/pi,360);  if phid>180, phid=phid-360;  end
thetad=mod(theta*180/pi,360); if thetad>180, thetad=thetad-360; end
psid=mod(psi*180/pi,360);
     set(ha(1),'String', ['\phi = '   int2str(phid)    ' ^o']);
     set(ha(2),'String', ['\theta = ' int2str(thetad)  ' ^o']);
     set(ha(3),'String', ['\psi = '   int2str(psid)    ' ^o']);
         F=getframe;
         writeVideo(aviobj,F);      % write into the VR object
```

```
pause(0.001)
    end                                  % end animation loop
         close(aviobj);                  % close the VR object
function R=Euler2DCM(psi,theta,phi) % local function
Rpsi = [cos(psi) sin(psi) 0; -sin(psi) cos(psi) 0; 0 0 1];
Rtheta = [cos(theta) 0 -sin(theta); 0 1 0; sin(theta) 0 cos(theta)];
Rphi = [1 0 0; 0 cos(phi) sin(phi); 0 -sin(phi) cos(phi)];
R = Rphi*Rtheta*Rpsi;
end
```

Note, in this script, if you would have the Aerospace Toolbox installed, you could have used the `angle2dcm` function instead of `Euler2DCM` (see Table E.2 of Appendix E).

This code also features the `VideoWriter` function to create a VideoWriter object (defined as a standard *avi*-file in this case) that allows you to write the animation into it and to then run it outside MATLAB. The `open` and `close` functions are used to open this object for writing and close it when done. The `get` and `writeVideo` functions allow you to get each frame and add it to the opened VideoWriter object. These functions are listed in Table 6.38.

Although you can play the created *avi*-file using some external player, you can also take advantage of the *Video Viewer*, called using the `implay` function (Table 6.38). The Video Viewer allows you to play movies, videos, or image sequences, offering several tools for navigating and exploring individual frames, including performing some common image processing tasks. For example, the call `implay('axisrotation.avi')` brings up the window shown in Fig. 6.61b.

Let us consider one more example. This time, let us animate a membrane, shown in Fig. 6.48a. The following code essentially repeats the one used to create Fig. 6.48a, except that it creates an explicit meshgrid and employs the `surf` function rather than the `fsurf` function:

```
[x,y]=meshgrid(-3:1/16:3);
z=sin(x.^2+y.^2)./(x.^2+y.^2).^0.5;
h=surf(x,y,z,'EdgeColor','none');
axis tight manual, shading interp
light('Color','r','Position',[-4 0 0],'Style','local')
material shiny, h.AmbientStrength=0.7;
set(gca,'Visible','off')
set(gca,'NextPlot','replacechildren');
     for j=0:200
     h.ZData=cos(2*pi*j/50)*z;
     F(j+1)=getframe;
     end
```

The reason is that we want to vary the height of membrane, that is, we want to change its `z` matrix values by referring to the corresponding property of the Surface object as it appears in the `for` loop, and the ZData property for

Table 6.38 Line Animation Functions

Function	Brief Description
Write to/read from a standard video format file	
`v=VideoWriter(filename)`	Creates a VideoWriter object for writing video data to `filename` (an *.avi* file with Motion JPEG compression, *.mj2* Motion JPEG 2000 file, or *.mp4* or *.m4v* MPEG-4 files for Windows 7 or later or Mac OS X 10.7 and later) (see Table 4.16)
`F=getframe`	Captures the current axes as they appear on the screen as a movie frame `F` (a structure containing the image data)
`writeVideo(v,frame)`	Writes one or more movie frames `F`, typically returned by the `getframe` function, to a video file associated with `v`. You must call `open(v)` to open `v` for editing and `close(v)` when you are done editing
`v=VideoReader(filename)`	Creates a VideoReader object to read video data from the file `filename`. Use the `readFrame(v)` and `hasFrame(v)` calls to read the next available video frame from the file associated with `v` or to check if there is a video frame available to read, respectively (see Table 4.11)
`mmfileinfo(filename)`	Returns a structure with the fields containing information about the contents of the multimedia file `filename`
`implay(mov)`	Opens the Video Viewer app of the Image Processing Toolbox, introduced in R2014b, which plays MATLAB movies, videos, or image sequences in `mov`
Create/play a MATLAB movie	
`[x,cmap]=frame2im(F)`	Converts a movie frame `F` to the indexed image data `x` and associated colormap `cmap`
`F=im2frame(x,cmap)`	Converts the indexed image, `x`, and the associated colormap, `cmap`, into a movie frame `F`
`mov=immovie(x,cmap)`	Returns the movie structure array `mov` from the images in the multiframe indexed image `x` with colormap `cmap`
`movie(M,n,fps)`	Plays the recorded movie in matrix `M` (composed of the 8-bit image frames `F`) `n` times at `fps` frames per second (the default is 12 frames per second). Note, `movie` uses a default figure size of 560-by-420 and does not resize the figures to fit movies with the larger or smaller frames

`fsurf` is read only. In this code, the `manual` mode of the `axis` function prevents autoscaling of the axis limits during animation (see Table 6.2); the `'replacechildren'` value of the axes NextPlot property sets the current axes to keep their scale, that is, it removes all children objects, but it does not reset axes properties while redrawing the surface (the other options are `'add'`, `'replace'`, `'replaceall'`); and the weighting coefficient `cos(2*pi*j/50)` is used to ensure a smooth transition between 1 and −1.

As opposed to the previous examples, `F` is not a scalar structure but rather a structure vector that has all (201 in our case) frames stored in it. You can use the `movie` or `implay` function of Table 6.38 to replay your

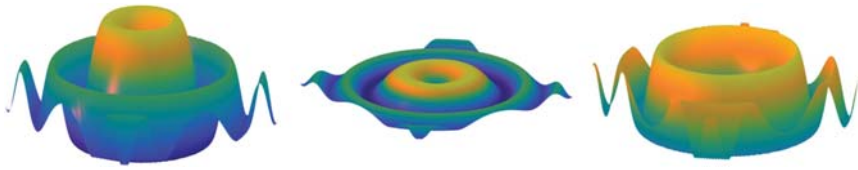


Fig. 6.62 Frames 1, 12, and 25 of animated membrane.

MATLAB movie. To visualize individual frames, you can use the `imshow(frame2im(F(k)))` call, where `k` is the frame number. That is how the images shown in Fig. 6.62 were obtained.

Let us finish this chapter with introducing the `uicontrol` function (see Fig. 6.10) that allows you to create the *push buttons* (the default user interface control) in the current figure to execute certain actions when pressed. For example, having the figure created during the last call, `imshow(frame2im(F(25)))`, and issuing the commands

```
uicontrol('Style','pushbutton','String','Zoom In',...;
  'Position',[20 20 60 20],...
  'Callback','if camva <= 1; return; else; camva(camva-1); end');
uicontrol('Style','pushbutton','String','Zoom Out',...
  'Position',[100 20 60 20],...
  'Callback','if camva >= 179; return; else; camva(camva+1); end');
```

adds the two push buttons, as shown in Fig. 6.63. Clicking them allows zooming in and out (the character vector after the Callback property represents MATLAB code that in this case uses the `camva` function of Table 6.31). Section F.2 of Appendix F has more examples, so here we will

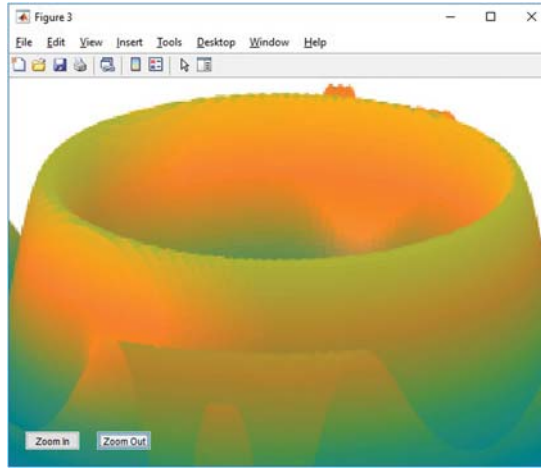


Fig. 6.63 Using the `uicontrol` function.

just mention that among other Style property values there are `'togglebutton'`, `'radiobutton'`, `'checkbox'`, `'slider'`, `'listbox'`, and `'popupmenu'`, which you can use to enable interactive behavior of your graphics. You can also use other functions of Fig. 6.10, creating *User Interface* (UI) *objects*, along with a few more, introduced recently—`uibutton` (R2016a), `uitogglebutton` (R2016a), `uitreenode` (R2017b), and `uitree` (R2017b).

6.13 Requirements to the Engineering Plots

Now that you know how to generate a variety of graphics objects, it is appropriate to introduce a set of general requirements applicable for any type of engineering graphics:

- Choose the type of plot and its scale wisely to have the best possible (that is, readable and understandable) representation of what you intend to show by this plot.
- Use a title to identify what the plot is, especially when there are multiple plots of a similar type.
- Label each axis and show the units used (place them in parentheses or after a comma right after the axis title, and be consistent for all the plots).
- Add annotations including visualizing equations, indicating specific parameters, and adding basic data statistics, as necessary.
- Ensure that each axis has regularly spaced tick marks at convenient intervals to allow quick estimates of the values.

If plotting more than one set of data or equation on the same plot:

- Distinguish each line not only by a different color (which MATLAB does for you automatically) but also by a different line style (and/or marker). Although you can see the difference between the colored lines on the screen, it may disappear when the plot is printed out on a monochrome printer or converted into grayscale for a journal publication.
- Place a legend distinguishing and explaining different curves.
- Do not overload one plot by placing too much information on it, so if appropriate, use multiple axes.

Finally, there are two more unwritten conventions:

- Measured data are usually represented by markers at each data point (different symbols can then be used for different sets of data). Connecting the data points with a solid line assumes a relationship in data that may not exist.
- On the contrary, equations are usually plotted as lines, not as individual data points.

Try to meet these requirements to produce informative, self-sufficient, and professional-looking charts.

Problems

6.1 As pointed out at the end of Sec. 6.2, the `annotation` function uses the normalized coordinates to specify locations within the figure. Write a script (function) that uses the current Axes object and allows the conversion of a pair of physical coordinates (x, y) into normalized coordinates (x_n, y_n). Use this function to draw a box with its edges represented by `arrows` (using the `annotation('arrow',...)` commands).

6.2 A defect in a crystal lattice where a row of atoms is missing is called an edge dislocation. The stress field around an edge dislocation is given by

$$\sigma_{xx} = \frac{-Gb}{2\pi(1-\nu)} \frac{y(3x^2+y^2)}{(x^2+y^2)^2}, \quad \sigma_{yy} = \frac{-Gb}{2\pi(1-\nu)} \frac{y(x^2+y^2)}{(x^2+y^2)^2},$$

$$\tau_{xy} = \frac{-Gb}{2\pi(1-\nu)} \frac{x(x^2-y^2)}{(x^2+y^2)^2}$$

where G is the shear modulus, b is the Burgers vector, and ν is the Poisson ratio. Plot the stress components (each in a separate figure) due to an edge dislocation in aluminum for which

$$G = 27.7\times10^9\text{Pa}, \quad b = 0.286\times10^{-9}\text{m}, \quad \text{and } \nu = 0.334$$

Plot the stresses in the domain $-5\times10^{-9}\text{m} < x < 5\times10^{-9}\text{m}$ and $-5\times10^{-9}\text{m} < y < 5\times10^{-9}\text{m}$. Plot the coordinates x and y in the horizontal plane and the stresses in the vertical direction. Label each axis using Greek alphabet letters.

6.3 Consider the following mode shape for a solid circular membrane clamped along its outer boundary ($r = 1$):

$$z(r, \phi) = J_1(3.8316r)\cos(\phi)$$

where $J_1(x)$ is the Bessel function of the first kind of the order of 1 (to address it properly, use the `besselj(1,x)` command), and (r, ϕ) are the polar coordinates of any point on the membrane (the value 3.8316 corresponds to one of the natural frequencies for such a membrane). Do the following:

a) Develop a script allowing you to show the membrane's shape as a 3-D surface.

b) Modify the second-to-last script of Sec. 6.12 to animate your membrane.

Chapter 7 Symbolic Math Toolbox

- Symbolic Expressions in Linear Algebra and Calculus
- Solving Algebraic and Differential Equations
- Using Symbolic Expressions in Numerical Calculations

`sym`, `syms`, `pretty`, `diff`, `limit`, `int`, `taylor`, `besselj`, `solve`, `equationsToMatrix`, `dsolve`, `collect`, `expand`, `rewrite`, `factor`, `simplify`, `subs`, `vpa`, `latex`, `fplot`, `fplot3`, `fmesh`, `matlabFunction`, `odeToVectorField`, `odeFunction`, `funtool`, `rsums`, `taylortool`, and others

7.1 Introduction

This chapter proceeds with Part II of the book (Fig. 0.1) introducing one of the most widely used MATLAB toolboxes offered with the MATLAB and Simulink Student Suite, the Symbolic Math Toolbox (SMT). This toolbox incorporates symbolic computation into the MATLAB numerical environment and can be very useful. In fact, the reason for introducing this toolbox so early in this textbook is that we will rely on the functions of this toolbox to derive a lot of formulas in the following chapters. Not that we could not have derived these formulas by hand or simply taken them from some textbook on numerical methods, but among other advantages, using SMT will allow you to get more hands-on experience in applied programming in the MATLAB development environment. As a result, you will become a proficient user not just in numerical computations, but in numerical and symbolic computations as well. The syntax for SMT is a natural extension of the normal MATLAB programming language, and therefore, there should be no difficulties following the material of this chapter after you have successfully completed the previous six. The chapter starts from the very basics—explaining how to create the symbolic objects and get help if needed. It then introduces all major functions of SMT by groups, followed by explaining how to incorporate the results of symbolic manipulations into numerical computations. Finally, three interactive GUIs that exploit the functions of SMT are presented. As of R2017a, SMT also allows using and manipulating physical units of measurement that act as symbolic expressions. This feature is briefly described as well.

7.2 Symbolic Math Toolbox Overview

Originally, the computational engine underlying SMT was the Maple kernel, developed at the University of Waterloo in Canada. Before R2007b, the *Basic* Symbolic Math Toolbox included in the Student Version was a collection of more than a hundred MATLAB functions allowing access to the functions in Maple's linear algebra package. The *Extended* Symbolic Math Toolbox augmented the basic toolbox by including access to nongraphics packages, programming features, and user-defined procedures in Maple.

As of R2007b, SMT—now a single product—uses a more powerful computational engine, MuPAD, originally developed by the MuPAD research group at the University of Paderborn, Germany. The first edition of this textbook included a brief description of the MuPAD Notebook development environment (that could be used in parallel with MATLAB). However, MuPAD Notebook is expected to be removed in a future release because the newly introduced MATLAB live scripts support most MuPAD functionality. Hence, the description of MuPAD notebooks is now omitted.

The areas supported by SMT are listed in Table 7.1. What is presented in this chapter shows the command returns as they appear in the R2020a version of MATLAB, but if you are using the earlier versions, you might see slightly different returns.

7.3 Getting Started

This section proceeds with explaining how to create symbolic objects, variables, and mathematical expressions, and how to access SMT help topics.

Table 7.1 Areas Supported by SMT

Subject	Areas Supported
Assumptions	Restrictions imposed on possible values of a symbolic object
Linear Algebra	Inverses, determinants, eigenvalues, singular value decomposition, and canonical forms of symbolic matrices
Calculus	Symbolic differentiation, integration, series operations, limits, and integral transforms
Equation Solving	Symbolic and numerical solutions to algebraic and differential equations
Formula Manipulation and Simplification	Simplification or modification of symbolic expressions, substitution of parts of expressions
Polynomials	Characteristic and minimal polynomials, coefficients of polynomials
Numbers and Precision	Variable-precision arithmetic and elementary number-theoretic operations
Number Theory	Modular arithmetic operations and sequences of numbers
Special Functions	Special functions in applied mathematics such as Airy functions, binomial coefficients, error functions, Bessel functions, Dirac delta function, Legendre polynomials, Jacobian, and Chebyshev functions
Graphics	Easy (2-D and 3-D) plotting of symbolic expressions

7.3.1 Creating Symbolic Variables

Let us start by introducing the two key functions to construct a *symbolic variable* of class "sym," which is a new class compared with the MATLAB classes presented in Fig. 4.1 (Table 7.2).

The basic syntax of the `sym` function is

```
>> sarg = sym(arg);
```

This command constructs a symbolic variable `sarg`. If the input `arg` is a character vector or string scalar (representing a valid variable name), the result is a symbolic variable. If the input argument is a numeric scalar or array, the result is a symbolic representation of the given numeric values. For instance, `x=sym('x')` creates the symbolic variable `x` and stores the result in `x`, whereas `y=sym('1/10')` (or `y=sym(1/10)`) creates a symbolic number `y` that avoids the floating-point approximations inherent in the value of 1/10 (to be discussed in Sec. 8.4.2). The instructions

```
>> A = sym('a',[m,n]);
>> B = sym('b',n);
```

allow creating the m-by-n and n-by-n symbolic matrices `A` and `B`, filled with automatically generated elements `a`i_j and `b`i_j, respectively.

Along with declaring symbolic variables, you can also assign some additional properties to them:

```
>> x = sym('x',set);
```

The second (optional) input argument `set` can be one of the following: `'real'`, `'positive'`, `'integer'`, `'rational'`, or `'clear'`. By default, all symbolic variables are assigned (assumed) to be `real`. In addition to the four aforementioned options, you may enforce additional properties by using the `assume` function (one of the Assumptions group functions of SMT shown in Table 7.3). For example,

```
>> assume(x/2,'integer');
>> assume(x>0 & x<2 * pi)
```

assumes `x` to be even and to reside between 0 and 2π, respectively. The existing assumptions on a symbolic variable `x` can be checked using

```
>> assumptions(x)
```

To clear all previous assumptions set on a symbolic variable or matrix `x`, you may use

```
>> x = sym('x','clear');
```

Table 7.2 Functions to Declare Symbolic Variables

Function	Description
sym	Creates symbolic variables, expressions, functions, and matrices using the function syntax
syms	Creates variables and functions using the command syntax

Table 7.3 SMT's Assumptions Functions

Function	Description
`assume`	Sets assumption on symbolic object
`assumeAlso`	Adds assumption on symbolic object
`assumptions`	Shows assumptions affecting symbolic variable, expression, or function
`in`	Checks a numeric type of a symbolic input (introduced in R2016b)
`piecewise`	Sets conditionally defined expression or function (introduced in R2016b)
`isAlways`	Checks whether equation or inequality holds for all values of its variables
`logical`	Checks validity of equation or inequality

or

```
>> assume('x','clear');
```

When converting a floating-point numeric scalar or array `A` to a symbolic form, you may also take advantage of an optional input argument, `flag`, specifying the technique for converting floating-point numbers:

```
>> arg = sym(A,flag);
```

Table 7.4 formally lists four available options, and Table 7.5 provides some examples.

The reverse conversion to one of the numeric types shown in Fig. 4.1 can be executed using one of the functions presented in Table 7.6.

If you have several simple variables to declare symbolic, you may use the second function of Table 7.2, the `syms` function, which allows you to use the command syntax. For example,

```
>> syms arg1  arg2 ...
```

is a short-hand notation for

```
>> arg1 = sym('arg1');
>> arg2 = sym('arg2'); ...
```

To create multiple variables in the following sections, we will primarily use this simplified approach.

Table 7.4 Flag Options

Flag	Description
`'r'`	Stands for "rational," meaning that floating-point numbers are obtained by evaluating expressions of the form `p/q`, `p*pi/q`, `sqrt(p)`, `2^q`, and `10^q` for modest-sized integers `p` and `q` (the default flag)
`'e'`	Stands for "estimate error," so that the `'r'` form is supplemented by a term involving the variable `eps` (Sec. 8.3.5), which estimates the difference between the theoretical rational expression and its actual floating-point value
`'f'`	Stands for "floating-point" (all values are represented in the form `N*2^e` or `-N*2^e`, where `N` and `e` are integers, $N \geq 0$
`'d'`	Stands for "decimal," so that the number of digits `N` is taken from the current setting defined by a preceding command `digits(N)` (fewer than 16 digits loses some accuracy, whereas more than 16 digits may not be warranted)

Table 7.5 Examples of Formatted Conversion to Symbolic Numbers

Command	Result
`sym(0.1,'r')`	`1/10`
`sym(0.1,'e')`	`eps/40+1/10`
`sym(0.1,'f')`	`3602879701896397/36028797018963968`
`sym(0.1,'d')`	`0.10000000000000000555111512312578`
`digits(4), sym(0.1,'d')`	`0.1`
`sym(3*pi/4,'r')`	`(3*pi)/4`
`sym(3*pi/4,'e')`	`3*pi/4-(103*eps)/249`
`sym(3*pi/4,'f')`	`2652839157010665/1125899906842624`
`sym(3*pi/4,'d')`	`2.356`
`digits(25), sym(pi,'d')`	`3.141592653589793115997963`

7.3.2 Constructing Symbolic Expressions and Symbolic Functions

Once you have defined symbolic variables, you can use them to construct *symbolic expressions*, for example,

```
>> syms a b c x
>> f=a*x^2+b*x+c;
```

Then, you can use these expressions for symbolic math manipulations. For example, the command

```
>> diff(f)
```

differentiates the expression `f` you created with the previous two commands with respect to an independent variable `x`:

```
ans = b + 2*a*x
```

This brings us to an important question. When manipulating the mathematical expressions involving more than one variable, the choice of independent variable is often clear from the context of the equation. However, how does MATLAB determine which variable is the independent one? Well, there is a simple rule. The independent symbolic variable in a symbolic expression is the one that starts with a letter that is closest to the lowercase `x`

Table 7.6 Functions for Converting Symbolic Numbers to Numeric Numbers

Function	Description
`double(S)`	Converts a symbolic array `S` to the numeric (double-precision) form
`single(S)`	Converts a symbolic array `S` to a single precision
`int8(S), int16(S), int32(S), int64(S)`	Converts a symbolic array `S` to signed integers
`uint8(S), uint16(S), uint32(S), uint64(S)`	Converts a symbolic array `S` to unsigned integers

alphabetically. If there are two equally close variables, the letter that is later in the alphabet is chosen. For instance, the set of commands

```
>> syms vrb zet X
>> h=vrb+zet^2+sqrt(X);
>> diff(h)
```

results in

```
ans = 2*zet
```

that is, the derivative is taken with respect to the variable `zet`. Table 7.7 provides some more examples on determining the default independent variable in symbolic mathematical expressions. If you are not sure, which variable is considered to be independent, use the `symvar` function. Specifically, `symvar(S)` returns all symbolic variables in `S` in alphabetical order, with uppercase letters preceding lowercase letters:

```
>> symvar(h)
ans = [X, vrb, zet]
```

Many MATLAB functions allow you to specify the independent variable explicitly, which can alleviate some confusion. For example, for the expression involving three variables, `vrb`, `zet`, and `X`, defined earlier, the command

```
>> diff(h,'vrb')
```

produces

```
ans = 1
```

More complex symbolic expressions can be constructed using the `compose` function (another approach, using the `subs` function, will be discussed in Sec. 7.7). This function allows you to compose a symbolic function $f(g(y))$ from two other symbolic functions, $f = f(x)$ and $g = g(y)$, as in the following example:

```
>> syms a b x y
>> f = a*x^3-sqrt(x)+b;
>> g = sin(y);
>> h = compose(f,g)
h = b + a*sin(y)^3 - sin(y)^(1/2)
```

Table 7.7 Determining the Default Independent Variable

Expression	Default Independent Variable
`x^n`	`x`
`sin(a*t+b)`	`t`
`w*y+v*z`	`y`
`exp(i*theta)`	`theta`
`sqrt(pi*alpha)`	`alpha` (`pi` has a predefined value in MATLAB)

If for any reason you want to keep `x` as an independent variable, you should use the following variant:

```
>> h1=compose(f,g,x)
h1 = b + a*sin(x)^3 - sin(x)^(1/2)
```

In addition to constructing mathematical expressions, you can also define/create symbolic functions. For example,

```
>> syms fs(x,y)
>> fs(x,y) = x/y+x^2+sin(y)
```

creates a *symbolic function* of the "symfun" class

```
fs(x,y) = x^2 + x/y + sin(y)
```

The same function can be defined in a slightly different manner using the `symfun` function

```
>> syms x y a b
>> fs = symfun(x/y+x^2+sin(y),[x y]);
```

You can now use this function. For example,

```
>> hs = diff(fs,x)
```

yields

```
hs(x,y) = 2*x + 1/y
```

(`hs` is also of the "symfun" class), and

```
>> fs(a,b)
```

returns

```
ans = a^2 + a/b + sin(b)
```

Note, passing on the numerical values

```
>> fs(4,-2)
```

still results in a symbolic expression

```
ans = 14 - sin(2)
```

so that if you want to evaluate it, you need to apply the `double` function or the `eval` function, introduced at the end of Sec. 3.6:

```
>> eval(ans)
ans=13.0907
```

The following example creates a symbolic representation of the forward–difference approximation for the first-order derivative (Sec. 12.2)

```
>> syms f(x) h
>> dfdx = (f(x+h)-f(x))/h;
```

(Note, whereas `f(x)` is a symbolic function, `dfdx` is a symbolic expression as well.)

7.3.3 Basic Operations

SMT supports all arithmetic operators (`minus`, `plus`, `times`, `ldivide`, `rdivide`, `power`, `mtimes`, `mldivide`, `mrdivide`, `mpower`, `transpose`, and `ctranspose`); equality, inequality, and other relational operations (`eq`, `ge`, `gt`, `le`, `lt`, and `ne`); and logical operations (`and`, `not`, `or`, and `xor`). Beyond this, SMT also supports many standard elementary functions introduced in Sec. 2.4.1 for numerical data. These include trigonometric functions (`sin`, `cos`, `tan`, `cot`, `sec`, `csc`, `asin`, `acos`, `atan`, `acot`, `asec`, and `acsc`), hyperbolic trigonometric functions (`sinh`, `cosh`, `tanh`, `coth`, `sech`, `csch`, `asinh`, `acosh`, `atanh`, `acoth`, `asech`, and `acsch`), exponential functions (`exp`, `log`, `log10`, `log2`, and `sqrt`), complex number functions (`abs`, `angle`, `atan2`, `imag`, `real`, `sign`, and `signIm`), and rounding and remainder functions (`ceil`, `conj`, `fix`, `floor`, `frac`, `imag`, `mod`, `quorem`, `real`, `rem`, and `round`). Similarly, SMT supports the matrix and array functions introduced in Secs. 3.3 and 3.4.

We have already talked about the key functions from the group of functions devoted to creating symbolic variables, expressions, and functions, `sym` and `syms`. One more useful function from this group is the `pretty` function. Once you have created your symbolic function, SMT allows you to display a symbolic output in a format that resembles typeset mathematics. The following example explains it:

```
>> syms x y
>> y=(x^2+1/x)/sqrt((sin(x)+cos(x))/x^3.5);
>> pretty(y)
           1    2
           - + x
           x
      ------------------
      / cos(x) + sin(x)  \
  sqrt| --------------   |
      |        7/2       |
      \       x          /
```

Being a valuable toolbox by itself, SMT also provides several functions for generating MATLAB functions and Simulink Function blocks, as well as converting symbolic expressions to other useful representations (Table 7.8).

7.3.4 Getting Help

As in the case of MATLAB itself, there are several methods to get help on the SMT functions as well. The first method is to type the following command in the MATLAB Command Window:

```
>> help sym/function
```

Table 7.8 Functions for Converting Symbolic Expressions

Function	Description
`ccode(S)`	Returns C code representation of a symbolic expression `S`
`fortran(S)`	Returns FORTRAN representation of a symbolic expression `S`
`latex(S)`	Creates LaTeX representation of a symbolic expression `S` (covered in Sec. 7.9.1)
`matlabFunction(S)`	Converts a symbolic expression `S` to a function handle or file (covered in Sec. 7.10.2)
`matlabFunctionBlock(S)`	Converts a symbolic expression `S` to a MATLAB Function block (to be considered in Sec. 15.6.4)
`simscapeEquation(S)`	Converts a symbolic expression `S` to Simscape language equation

The reason for this syntax is that there are many dual functions that use the same name but are applicable either for numeric or symbolic computations. For example, to get help for symbolic differentiation, you need to type

```
>> help sym/diff
```

which yields the following response:

```
diff   Differentiate.
    diff(S) differentiates a symbolic expression S with respect to its
    free variable as determined by SYMVAR.
    diff(S,'v') or diff(S,sym('v')) differentiates S with respect to v.
    diff(S,n), for a positive integer n, differentiates S n times.
    diff(S,'v',n) and diff(S,n,'v') are also acceptable.
    diff(S,'v1','v2',...) or diff(S,sym('v1'),sym('v2'),...) differentiates
    S with respect to v1, v2, ...
    Examples:
       syms x t
       diff(sin(x^2)) is 2*x*cos(x^2)
       diff(t^6,6) is 720.
       diff(sin(x^2+t),x,t,t) is -2*x*cos(x^2 + t)
    See also sym/int, sym/jacobian, sym/symvar.
```

However, if you type

```
>> help diff
```

you will get a different response corresponding to a completely different function:

```
DIFF Difference and approximate derivative.
   DIFF(X), for a vector X, is [X(2)-X(1) X(3)-X(2) ... X(n)-X(n-1)].
   DIFF(X), for a matrix X, is the matrix of row differences,
    [X(2:n,:) - X(1:n-1,:)].
...
```

As discussed in Sec. 5.8.2, this method (`help sym/function`) simply displays the comment lines that are located at the beginning of the *m*-function.

If you forgot the correct spelling of the function you are looking for, type several beginning letters in the Command Window and then hit the <Tab> key to get a hint in a pop-up menu, or use a Function Browser button (if activated) as discussed in Sec. 2.6 (Fig. 2.27).

Another method of getting help is to click the **Help** icon of the Home tab toolstrip and then click the subgroup "Symbolic Math Toolbox" in the Contents pane. This opens the Help window for the symbolic toolbox that allows you to find the most complete information about any symbolic function. Alternatively, you may type in the name of the function you want to learn about in the search window of the Help browser. The third method is to go to the MathWorks website and get missing information from there.

Finally, because there are multiple functions bearing the same name, how does MATLAB know the difference between symbolic and numeric use of the function? Well, it depends on the type of arguments that are passed on to it. If the type is symbolic, MATLAB "overloads" or, in other words, provides symbol-specific implementation of the function.

7.4 Linear Algebra Functions

The functions of the Linear Algebra group of SMT (Table 7.1) allow you to perform linear algebra operations on symbolic vectors and matrices, including matrix operations and transformations; dealing with linear equations, eigenvalues and eigenvectors; matrix analysis, functions and decomposition; and vector spaces and subspaces. (We have introduced many of these functions as applied to numerical computations already.) In essence, linear algebraic operations on the symbolic objects are the same as operations with numeric variables of double-precision accuracy, and some of them are considered in more detail in Chapter 9. Several simple examples, as applied to symbolic variables, are given in Table 7.9.

7.5 Calculus Functions

The Calculus group of SMT (Table 7.1) offers a variety of functions for different symbolic calculations, as shown in Table 7.10.

As an example, let us consider several frequently used functions from this group. We have introduced the `diff` function from this group in Sec. 7.3.2 already. There, we showed how to compute the first-order derivative with respect to a default or explicitly defined variable. The following commands and their returns allow revising this subject:

```
>> syms a x, F=sin(a*x^2);
>> dFdx=diff(F)
dFdx = 2*a*x*cos(a*x^2)
>> dfda=diff(F,a)
dFda = x^2*cos(a*x^2)
```

Table 7.9 Examples of Handling Symbolic Matrices

Example 1	Example 2
`>> syms a b c d e f g h i`	`>> H=hilb(3)`

	`H =`
	`1.0000 0.5000 0.3333`
	`0.5000 0.3333 0.2500`
	`0.3333 0.2500 0.2000`

`>> K = [a 0 b; 0 c d; e 0 1]`	`>> H=sym(H)`

`K =`
`[ a, 0, b]`
`[ 0, c, d]`
`[ e, 0, 1]`
	`H =`
	`[ 1, 1/2, 1/3]`
	`[ 1/2, 1/3, 1/4]`
	`[ 1/3, 1/4, 1/5]`

`>> detK=det(K)`	`>> detH=det(H)`
`detK = a*c-b*c*e`	`detH = 1/2160`
`>> K_inv=inv(K)`	`>> H_inv=inv(H)`

`K_inv =`
`[ 1/(a-b*e), 0, -b/(a-b*e)]`
`[(d*e)/(c*(a-b*e)), 1/c, -(a*d)/(c*(a-b*e))]`
`[ -e/(a-b*e), 0, a/(a-b*e)]`
	`H_inv =`
	`[ 9, -36, 30]`
	`[ -36, 192, -180]`
	`[ 30, -180, 180]`

The `diff` function allows finding the higher-order derivatives as well. To this end, the following call allows computing the second-order derivative with respect to a default independent variable (`x`)

```
>> d2Fdx2=diff(F,2)
d2Fdx2 = 2*a*cos(a*x^2) - 4*a^2*x^2*sin(a*x^2)
```

The general syntax for obtaining an `n`th-order derivative with respect to a (default) independent variable is `diff(F,n)`. You can specify a differentiation variable explicitly using a slightly modified syntax, `diff(F,var,n)`. Moreover, you can specify multiple differentiation variables for high-order derivatives using yet another syntax, `diff(F,var1,...,varN)`.

Similarly, you can apply the `diff` function to a symbolic array of functions, for example,

```
>> syms a x
>> A=[cos(a*x),sin(a*x);-sin(a*x),cos(a*x)];
>> dAdx=diff(A)
dAdx =
   [ -a*sin(a*x), a*cos(a*x)]
   [ -a*cos(a*x), -a*sin(a*x)]
```

Table 7.10 Calculus Functions of SMT

Function	Description
Differentiation	
`diff`	Computes a derivative of a symbolic expression or function
`functionalDerivative`	Returns a functional derivative of the functional
Integration	
`int`	Definite and indefinite integrals
`vpaintegral`	Numerical integration using variable precision
`changeIntegrationVariable`	Integration by substitution (R2019b)
`integrateByParts`	Integration by parts (R2019b)
`release`	Evaluate integrals (R2019b)
`rsums`	Interactive evaluation of the Riemann sums
Vector Analysis	
`curl`	Curl of the vector field
`divergence`	Divergence of the vector field
`gradient`	Gradient vector of the scalar function
`hessian`	Hessian matrix of the scalar function
`jacobian`	Jacobian matrix
`laplacian`	Laplacian of the scalar function
`potential`	Potential of the vector field
`vectorPotential`	Vector potential of the vector field
Series	
`cumprod`	Symbolic cumulative product
`cumsum`	Symbolic cumulative sum
`pade`	Padé approximant
`series`	Puiseux series
`symprod`	Product of series
`symsum`	Sum of series
`taylor`	Taylor series
`taylortool`	Taylor series calculator
Limits	
`limit`	Limit of the symbolic expression
Transforms	
`fourier, ifourier, htrans, ihtrans, laplace, ilaplace, ztrans, iztrans`	Fourier, Hilbert, Laplace and Z-transforms (discussed in detail in Table 7.12)
`sympref`	Sets symbolic preferences for `fourier, ifourier,` and `heaviside`

Symbolic integration using the `int` function (from the Integration subgroup) is more difficult than symbolic differentiation, but in many cases, this function still finds a symbolic integral. A symbolic integral of an expression `f` is found by finding another symbolic expression `F` such

that `diff(F)` yields `f`. Similar to differentiation, you can specify the variable of integration if the default symbolic variable is not the variable of integration. If the `int` function cannot find the antiderivative, then the command line is returned unevaluated (as typed). Definite integrals may also be evaluated using the same function `int`, by specifying the limits of integration. Some examples are given as follows. First, let us compute indefinite and definite integrals, $\int \sin(x)\mathrm{d}x$ and $\int_0^{\pi} \sin(x)\mathrm{d}x$, respectively:

```
>> syms x, f=sin(x);
>> F=int(f)
F = -cos(x)
>> F02pi=int(f,0,pi)
F02pi = 2
```

Next, let us find another integral, $\int \sqrt{x}\log(x)\mathrm{d}x$, also requesting to represent the result nicely using the `pretty` function; then evaluating $\int_0^1 \sqrt{x}\log(x)\mathrm{d}x$ and converting the result into the floating-point numeric format:

```
>> syms x, f=log(x)*sqrt(x);
>> pretty(int(f))
>> F01=int(f,0,1)
Graphic {BOOKS}AIAA/4439/Art/C007/ITX007x002.eps not found
F01 = -4/9
>> double(F01)
ans = -0.4444
```

The `jacobian(f,v)` function (of the Vector Analysis subgroup) computes the Jacobian of a scalar or vector `f` with respect to `v`. The (i,j)th entry of the result is a partial derivative $(\partial f_i/\partial v_j)$. When `f` is a scalar, `jacobian(f,v)` returns the gradient of `f`; when `v` is a scalar, the result is the same as `diff(f,v)`. For instance, the following computes the Jacobian matrix $\mathbf{J}$ for the transformation from the Euclidean space $\{x, y, z\}$ to spherical coordinates $\{r, \lambda, \phi\}$, where r is the distance from the origin, λ corresponds to the elevation or latitude, and ϕ denotes the azimuth or longitude:

```
>> syms r lambda phi
>> x=r*cos(lambda)*cos(phi);
>> y=r*cos(lambda)*sin(phi);
>> z=r*sin(lambda);
>> J=jacobian([x;y;z],[r,lambda,phi])
J =
  [ cos(lambda)*cos(phi), -r*cos(phi)*sin(lambda), -r*cos(lambda)*sin(phi)]
  [ cos(lambda)*sin(phi), -r*sin(lambda)*sin(phi),  r*cos(lambda)*cos(phi)]
                                                  [     sin(lambda), r*cos(lambda),0]
```

The indefinite or definite summation is done using the `symsum` function of the Series subgroup. For instance, to compute the indefinite sum

$$\sum_{i=0}^{n-1} \frac{a}{i} \tag{7.1}$$

you should issue the following two commands:

```
>> syms a n
>> symsum(a/n)
```

resulting in

```
ans = piecewise(0 < n, a*psi(n), n <= 0, a*psi(1 - n))
```

that is, $a\Psi(n)$ (the `psi` function will be introduced in Table 7.18). If only several terms are needed to be summed, say, from $i = 6$ to $i = 15$, the command

```
>> symsum(a/n,6,15)
```

would produce the desired result

```
ans = (74587*a)/72072
```

Of course, you may determine an independent symbolic variable explicitly. For example,

```
>> symsum(a/n,a)
```

will compute the sum with respect to the variable `a`:

```
ans = a^2/(2*n) - a/(2*n)
```

that is, $a(a - 1)/2/n$.

The limits may also include infinity. For example, the command

```
>> symsum(6/n^2,1,inf)
```

computes the infinite sum

```
ans = pi^2
```

and the command

```
>> symsum(a^n/sym(factorial(n)),n,0,inf)
```

returns

```
ans = exp(a)
```

The `taylor` function of the Series subgroup allows you to compute a Taylor series approximation of the function $f(x)$ about a point $x = a$:

$$f(x) = \sum_{n=0}^{\infty} (x - a)^n \frac{f^{(n)}(a)}{n!} \tag{7.2}$$

(if the series is centered at zero, that is, $a = 0$, the series is also called a Maclaurin series). Table 7.11 shows available syntaxes.

Table 7.11 Syntaxes of the Tylor function

Syntax	Result
`taylor(f)`	Computes the Maclaurin series expansion of `f` (around 0) up to the fifth order
`taylor(f,v)`	Computes the Maclaurin series expansion of `f` with respect to `v`
`taylor(f,v,a)`	Computes the Taylor series expansion of `f` with respect to `v` around the expansion point `a`
`taylor(f,[v],[a],Name,Value)`	Uses additional options specified by one or more `Name-Value` pair arguments (can be used with or without `v` and `a`)

For instance, the following finds the seventh-order Maclaurin approximation of $f(x) = \sin(x)$:

```
>> syms x, f=sin(x);
>> M=taylor(f,'Order',8);
>> pretty(M)
```

```
Graphic {BOOKS}AIAA/4439/Art/C007/ITX007x003.eps
not found
```

To find the Taylor series expansion of `sin(x)` about $\pi/2$, the following commands may be used:

```
>> syms x, f=sin(x);
>> T=taylor(f,x,'ExpansionPoint',pi/2,'Order',8);
>> pretty(T)
```

```
Graphic {BOOKS}AIAA/4439/Art/C007/ITX007x004.eps
not found
```

The `limit` function of the Calculus group allows you to compute the symbolic limits of a symbolic function. For example, let us use the fundamental theorem of calculus

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \tag{7.3}$$

If we let $f(x) = \cos(x)$, then the derivative should be $f'(x) = -\sin(x)$. The following demonstrates that rather than using the `diff` function, you could find the solution using the `limit` function as well:

```
>> syms x h, dydx=limit((cos(x+h)-cos(x))/h,h,0)
dydx = -sin(x)
```

As another example, let us determine the well-known limit

$$e^x = \lim_{n \to \infty} (1 + x/n)^n \tag{7.4}$$

The following set of commands does the job:

```
>> syms x n
```

```
>> ex=limit((1+x/n)^n,n,inf)
ex = exp(x)
```

Equation (7.3) presents an example of a "two-sided" limit, that is, it does not matter from which side you approach the limit. If the function is discontinuous (has singularities), then the direction of approach may matter. For example, let us look at the following "one-sided" limits:

$$\lim_{x\to 0^+} x^{-1} \quad \text{and} \quad \lim_{x\to 0^-} x^{-1} \tag{7.5}$$

The `limit` function allows you to specify what direction the limit is approached from, by adding the option `'right'` (to approach from the right, $x \to 0^+$) or `'left'` (to approach from the left, $x \to 0^-$) as follows:

```
>> syms x
>> lim_plus=limit(1/x,x,0,'right')
lim_plus = Inf
>> lim_minus=limit(1/x,x,0,'left')
lim_minus = -Inf
```

Note that if you use the default case, without specifying the direction of approach, you get

```
>> inv_x=limit(1/x,x,0)
inv_x = NaN
```

which means that the `limit` function could not find a consistent answer to the limit.

The key functions of the Transforms subgroup (Table 7.12) enable symbolic computation of some direct and inverse transforms (special integrals or summation). As representative examples, the following two sets of

Table 7.12 Transforms Functions of SMT

Function	Description
`fourier`	Returns the Fourier integral transform, $F(\omega) = \int_{-\infty}^{\infty} f(t)e^{-i\omega t}\mathrm{d}t$
`ifourier`	Returns the inverse Fourier integral transform, $f(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} F(\omega)e^{i\omega t}\mathrm{d}\omega$
`htrans`	Returns the Hilbert transform $H(x) = \frac{1}{\pi}\int_{-\infty}^{\infty} \frac{f(t)}{x-t}\mathrm{d}t$ (R2019a)
`ihtrans`	Returns the inverse Hilbert transform $f(t) = \frac{1}{\pi}\int_{-\infty}^{\infty} \frac{H(x)}{x-t}\mathrm{d}x$ (R2019a)
`laplace`	Returns the Laplace transform, $F(s) = \int_0^{\infty} f(t)e^{-st}\mathrm{d}t$
`ilaplace`	Returns the inverse Laplace transform, $f(t) = \int_{c-i\infty}^{c+i\infty} F(s)e^{st}\mathrm{d}s$
`ztrans`	Returns the Z-transform, $F(z) = \sum_{n=0}^{\infty} \frac{f(n)}{z^n}$
`iztrans`	Returns the inverse Z-transform, $f(n) = \frac{1}{2\pi i}\oint_{\lvert z\rvert=R} F(z)z^{n-1}\mathrm{d}z$, $n = 1, 2, \ldots$

commands compute the Laplace transform of

$$f(t) = \frac{1}{t^2} \quad \text{and} \quad f(t) = \frac{1}{e^{at}} \tag{7.6}$$

```
>> syms t
>> laplace(t^2)
ans = 2/s^3
```

```
>> syms a t x
>> L=laplace(exp(-a*t),x)
L = 1/(a + x)
```

Note how the default output variable s in the latter expression was substituted with the symbol x. The inverse Laplace transform yields

```
>> syms s
>> f=ilaplace(2/s^3)
f = t^2
```

```
>> syms a t
>> f=ilaplace(1/(t-a)^2)
f = x*exp(a*x)
```

Note that in the latter example, because we defined $F = F(t)$ as opposed to the default $F = F(s)$, `ilaplace` returns $f = f(x)$ instead of $f = f(t)$.

7.6 Solving Algebraic and Differential Equations

The Equation Solving group of SMT (Table 7.1) offers a wide variety of functions that allow you to analytically solve the *well-posed* systems of algebraic equations and ordinary differential equations (ODEs). ("Well-posed" implies the existence of a unique solution whose value changes only slightly if *initial conditions* (ICs) change.) These functions are listed in Table 7.13.

Let us consider a few frequently used functions from this group. The `finverse` function simply returns the functional inverse of the function $f(x)$. If it exists, this inverse, $g(x)$, satisfies the equation $g(f(x)) = x$. For instance,

```
>> syms x
>> f=5*x^3;
>> pretty(finverse(f))
      2/3   1/3
     5     x
     ---------
        5
```

```
>> syms x
>> f=tan(1/x);
>> pretty(finverse(f))
           1
       -------
       atan(x)
```

The two key functions, `solve` and `dsolve`, are dedicated to solving (systems of) algebraic and differential equations, respectively. To this end, one or more algebraic expressions (system of equations) can be solved using the `solve` function. For example, writing the quadratic equation and invoking the `solve` function as

```
>> syms a b c x
>> f=a*x^2+b*x+c;
>> y=solve(f)
```

yields the two well-known roots

```
y = -(b + (b^2 - 4*a*c)^(1/2))/(2*a)
```

Table 7.13 Equation Solving Functions

Function	Description
Linear and Nonlinear Equations and Systems	
`equationsToMatrix`	Converts a set of linear equations to the matrix form
`finverse`	Performs a functional inverse
`linsolve`	Solves a linear system of equations given in the matrix form
`poles`	Finds the poles of an expression or function
`solve`	Solves an equation or system of equations symbolically[a]
`vpasolve`	Solves an equation or system of equations numerically
ODEs and Systems	
`dsolve`	Solves an ODE or system of equations
`massMatrixForm`	Extracts the mass matrix and right side of a semilinear system of differential algebraic equations (DAEs)
`odeFunction`	Converts a system of symbolic algebraic expressions to a MATLAB function handle suitable for `ode45`, `ode15s`, and other ODE solvers
`odeToVectorField`	Converts higher-order differential equations to systems of the first-order differential equations
Differential Algebraic Equations	
`daeFunction`	Converts a system of DAEs to a MATLAB function handle suitable for the `ode15i` solver
`decic`	Finds consistent ICs for the first-order implicit ODE system with algebraic constraints
`findDecoupledBlocks`	Searches for decoupled blocks in systems of equations
`incidenceMatrix`	Finds an incidence matrix of a system of equations
`isLowIndexDAE`	Checks if the differential index of a system of equations is lower than 2
`massMatrixForm`	Extracts the mass matrix and right side of a semilinear system of DAEs
`reduceDAEIndex`	Converts a system of the first-order DAEs to the equivalent system of differential index 1
`reduceDAEToODE`	Converts a system of the first-order semilinear DAEs to the equivalent system of differential index 0
`reduceDifferentialOrder`	Reduces a system of higher-order differential equations to the equivalent system of the first-order differential equations
`reduceRedundancies`	Simplifies a system of the first-order DAEs by eliminating redundant equations and variables

[a]As of R2020a, this function can be used interactively in the Solve Symbolic Equation **Task** interface of Live Editor (Fig. 2.13a).

```
-(b - (b^2 - 4*a*c)^(1/2))/(2*a)
```

Note that in the earlier versions of MATLAB, the `solve` function allowed you to specify equations as the character vectors; however, as of R2014b, they should be symbolic expressions only.

By default, it is assumed that you are trying to solve for the roots of $f(x) = 0$, but you may also use inequalities and equalities. Also, as was the case of other symbolic functions, you may assign an independent variable explicitly. For example,

```
>> syms a b c x
>> f=a*x^2+b*x+c==0;
>> y=solve(f,b)
```

returns

```
y = -(a*x^2+c)/x
```

The `solve` function also allows you to solve multiple equations (system of equations). The general format is

```
solve([eq1,eq2,...,eqn],[var1,var2,...,varn])
```

A self-explanatory example of solving several algebraic equations is given as follows:

```
>> syms a b c d x y t
>> [X,Y]=solve([a*x+b*y-t,c*x+d*y])
X = (d*t)/(a*d - b*c)
Y = -(c*t)/(a*d - b*c)
```

If the second command is replaced with

```
>> g=solve([a*x+b*y==t,c*x+d*y==0])
```

assuming a single output, it returns a structure

```
g = x: [1×1 sym]
    y: [1×1 sym]
```

In this case, to access solutions, you need to refer to them by the name field; for example,

```
>> [g.x g.y]
```

returns

```
ans = [ (d*t)/(a*d - b*c), -(c*t)/(a*d - b*c)]
```

Note that both calls assumed `x` and `y` to be the default unknown variables, but you could have used the optional second input argument to specify the unknown variables explicitly. Also, whereas the first call only listed the left-hand sides of the equations, assuming the right-hand sides equal zero, in the second call both equations are "spelled out" explicitly using the `==` operator.

The `equationsToMatrix` function allows you to convert a set of linear equations to the matrix form. For example, the same two preceding equations (with two unknown variables specified explicitly as the second input argument) could first be converted to the matrix form $\mathbf{A}[x, y]^T = \mathbf{b}$ with

```
>> [A,b]=equationsToMatrix([a*x+b*y-t,c*x+d*y],[x,y])
```

returning

```
A = [ a, b]
    [ c, d]
b = t
    0
```

and then solved as

```
>> A\b
ans =  (d*t)/(a*d - b*c)
      -(c*t)/(a*d - b*c)
```

The `dsolve` function finds symbolic solutions to ODEs. For example, to solve the equation $\dot{x}^2 + x = 1$ you would need to first create a symbolic function $y(t)$ and then use the `dsolve` function as follows:

```
>> syms y(t)
>> Y=dsolve(diff(y,t)^2+y==1)
```

returns

```
x =                1
    1 - (C2 + t)^2/4
    1 - (C1 - t)^2/4
```

In this example, `diff(y,t)` represents the first-order derivative dy/dt. The second-order derivative would then be defined as `diff(y,t,2)`, and so on. Because the ICs were not specified, the solution contains two constants of integration (`C6` and `C2`).

The ICs can be specified in the second input argument of the `dsolve` function, for example,

```
>> syms y(t)
>> Dy=diff(y);
>> Y=dsolve(diff(y,2)==y+1,[y(0)==1, Dy(0)==1])
```

returning

```
Y = exp(-t)/2 + (3*exp(t))/2 - 1
```

It should be noted that in the previous releases including R2020a, the `dsolve` function allowed character vector inputs, so that instead of using the three aforementioned commands, the same result could be achieved with the single command `Y = dsolve('D2y = y + 1', 'y(0) = 1','Dy(0) = 1')`. However, this syntax is supposed to be removed in a future release.

One more self-explanatory example illustrates how to solve more than one ODE:

```
>> syms u(t) v(t)
```

```
>> [U,V]=dsolve([diff(u)==v,diff(v)==cos(t)],[u(0)==1,v(0)==0])
U = 2 - cos(t)
V = sin(t)
```

7.7 Simplification and Substitution

The Formula Manipulation and Simplification group of SMT (Table 7.1) offers several functions for simplifying a long expression to a shorter form, transforming expressions to a particular form or rewriting them in specific terms, and replacing the parts of expressions with the specified symbolic or numeric values (Table 7.14).

Most of the functions of the Formula Manipulation and Simplification group are self-explanatory. For instance, the `children` function allows extracting all the terms in a symbolic expression (subexpressions) into a symbolic vector. For example,

```
>> syms x y
```

Table 7.14 Formula Manipulation and Simplification Functions

Function	Description
Formula Rearrangement and Rewriting	
`children`	Returns a vector containing subexpressions or terms of symbolic expression
`coeffs`	Returns coefficients of polynomial
`collect`	Collects coefficients
`combine`	Combines the terms of an identical algebraic structure[a]
`compose`	Performs functional composition
`divisors`	Finds divisors of an integer or expression
`isolate`	Isolates a variable or expression in an equation
`expand`	Performs symbolic expansion of the polynomials and elementary functions[a]
`factor`	Returns all irreducible factors
`horner`	Returns the Horner nested polynomial representation
`numden`	Extracts numerator and denominator
`partfrac`	Performs partial fraction decomposition
`lhs`	Returns the left-hand side of an equation or condition
`rewrite`	Rewrites an expression in new terms[a]
`rhs`	Returns the right-hand side of an equation or condition
Simplification	
`simplify`	Preforms algebraic simplification[a]
`simplifyFraction`	Performs symbolic simplification of fractions[a]
`subexpr`	Rewrites a symbolic expression in terms of common subexpressions
Substitution	
`subs`	Performs a symbolic substitution

[a]As of R2020a, this function can be used interactively in the Simplify Symbolic Expression **Task** interface of Live Editor (Fig. 2.13a).

```
>> T=children(x^2-x*y-y^2)
```

returns

```
T = [ -x*y, x^2, -y^2]
```

The `coeffs(p,x)` call returns coefficients of polynomial `p` with respect to `x`. For instance,

```
>> syms x
>> f=x^3-2*x^2+3*x-4;
>> c=coeffs(f,x)
c =[ -4, 3, -2, 1]
```

```
>> syms a b c x
>>  y=a+b*sin(x)+c*sin(2*x);
>> c1=coeffs(y,sin(x))
>> c2=coeffs(expand(y),sin(x))
c1 = [ a+c*sin(2*x), b]
c2 = [ a, b+2*c*cos(x)]
```

(Note that the order of coefficients is reversed as compared with that described in Sec. 3.5 for numerical polynomials.)

Here, we also use the `expand` function. This function distributes the products over the sums and applies other identities involving the functions of sums. For example,

```
>> syms x y
>> g=cos(x+y);
>> g=expand(g)
g = cos(x)*cos(y)-sin(x)*sin(y)
```

The `collect` function allows you to collect all the terms of like powers of the independent variable. For example,

```
>> syms x t
>> f=(1+x)*t+x*t;
>> f=collect(f)
```

returns

```
f = 2*t*x + t
```

You can isolate a variable in a symbolic equation (to further use it with the `subs` function) by using the `isolate` function (the result is similar to solving an equation). For example,

```
syms x a b c
eqn = a*x^2 + b*x + c == 0;
xSol = isolate(eqn, x)
```

returns

```
xSol = x == -(b + (b^2 - 4*a*c)^(1/2))/(2*a)
```

If `f` is a polynomial with rational coefficients, `factor(f)` expresses `f` as a product of polynomials of the lower degree with rational coefficients (if it

cannot be done, f is returned unfactored). Two examples are presented as follows:

```
>> syms x
>> f=x^3-6*x^2+11*x-6;
>> f=factor(f)
f = [x-3, x-1, x-2]
```

```
>> syms x
>> g=x^6+1;
>> g=factor(g)
g = [x^2+1, x^4-x^2+1])
```

The horner fu produces a nested polynomial representation. For example,

```
>> syms x
>> f=x^3-6*x^2+11*x-6;
>> horner(f)
```

returns

```
c = x*(x*(x - 6) + 11) - 6
```

The numden function converts an input argument to a rational form, where the numerator and denominator are relatively prime polynomials with integer coefficients. For example,

```
>> syms x y
>> [n,d]=numden(x/y + y/x)
n = x^2+y^2
d = x*y
```

For another example, considered earlier, in Sec. 7.3.3, application of the numden function

```
>> syms x y
>> y=(x^2+1/x)/sqrt((sin(x)+cos(x))/x^3.5);
>> [n,d]=numden(y)
```

results in

```
n = x^3+1
d = x*((cos(x) + sin(x))/x^(7/2))^(1/2)
```

The lhs and rhs functions, introduced in R2017a, simply return the left-hand and right-hand side of the symbolic equation or condition, respectively. For example, a set of instructions

```
>> syms x y
>> eqn = 2*y <= x^2;
>> lhsEqn = lhs(eqn)
>> rhsEqn = rhs(eqn)
```

returns

```
lhsEqn = 2*y
rhsEqn = x^2
```

The `rewrite` function allows you to rewrite a symbolic expression in terms of another function. For example,

```
>> syms x
>> a=rewrite(cos(2*x),'sin')
>> b=rewrite(sin(x)^2,'tan')
```

returns

```
a = 1 - 2*sin(x)^2
b = (4*tan(x/2)^2)/(tan(x/2)^2 + 1)^2
```

Besides `'sin'` and `'tan'`, the second input argument may be one of the following: `'exp'`, `'log'`, `'sincos'`, `'cos'`, `'cot'`, `'sinhcosh'`, `'sinh'`, `'cosh'`, `'tanh'`, `'coth'`, `'asin'`, `'acos'`, `'atan'`, `'acot'`, `'asinh'`, `'acosh'`, `'atanh'`, `'acoth'`, `'sqrt'`, `'heaviside'`, and `'piecewise'`.

The `simplify` function of the Simplification subgroup searches for the simplest form of a symbolic expression by applying several algebraic identities involving sums, integral powers, square roots, and other fractional powers, as well as some trigonometric, logarithmic, and exponential identities. For example,

```
>> syms x
>> f=(1-x^2)/(1-x);
>> f=simplify(f)
 f = x+1
```

```
>> syms a
>> f=(1/a^3+6/a^2+12/a+8)^(1/3);
>> f=simplify(f)
 f = ((2*a + 1)^3/a^3)^(1/3)
```

(Note that, in the past, SMT used to have another, more general `simple` function. This function would return the results of a search for the simplest form of symbolic expression that had the fewest number of characters among all the aforementioned functions (`collect`, `expand`, `factor`, and `simplify`) and some other simplification functions. As of R2015a, this function has been removed.)

The last, very useful, function of the Formula Manipulation and Simplification group is `subs`. It can be invoked with one, two, or three input arguments. When invoked with only one input argument, `R=subs(S)`, it replaces all occurrences of variables in the symbolic expression `S` with the values obtained from the calling function or MATLAB workspace. For example,

```
>> syms a b c x
>> f=exp(a)*x^3+log(b)*x^2+c;
>> a=2; b=3; c=6;
>> r=subs(f)
```

returns

```
r = exp(2)*x^3+log(3)*x^2+6
```

and

```
>> x=3;
>> r=subs(f)
```

further returns

```
r = 27*exp(2) + 9*log(3) + 6
```

If needed, you can convert the last result to a double-precision scalar using either the `double` or `eval` function:

```
>> eval(r)
ans = 215.3920
```

When invoked with two input arguments, `R=subs(S,new)`, the `subs` function replaces the default symbolic variable in `S` (the closest to the lowercase `x`) with `new`:

```
>> syms a b c x, f=exp(a)*x^3+log(b)*x^2+c;
>> r=subs(f,'sigma')
r = exp(a)*(sigma)^3+log(b)*(sigma)^2+c
```

Finally, the most general form involves invoking this function with three input arguments, `R=subs(S,old,new)`, which replaces `old` with `new` in the symbolic expression `S`. The variable `old` must be of a "sym", "char," or "string" class representing a variable name, whereas the variable `new` may be symbolic or numeric. In the case that `old` and `new` represent several variables, as opposed to a single one, the list of the variables should be enclosed by braces, `{}` (constituting a cell array input) or brackets `[]`, (constituting a vector input). For example,

```
>> syms a b alpha
>> f=cos(a)+sin(b);
>> g=subs(f,{a,b},{alpha,2})
g = sin(2) + cos(alpha)
```

7.8 Other Functions

Along with the major groups considered in the previous sections, SMT contains a few more groups of functions (Table 7.1). For completeness, let us briefly discuss them as well.

7.8.1 Polynomials

The Polynomials group functions are presented in Tables 7.15 and 7.16. Two of these functions (`poly2sym` and `sym2poly`) were introduced earlier in Sec. 3.5.

Table 7.15 Polynomials Functions

Function	Description
Polynomial Operations	
coeffs	Returns coefficients of polynomial (the same as in Table 7.14)
poly2sym	Creates a symbolic polynomial from a vector of coefficients
sym2poly	Extracts a vector of all numeric coefficients, including zeros, from a symbolic polynomial
root	Finds the roots of polynomial
gbasis	Returns the reduced Gröbner basis of the vector of polynomials
polynomialDegree	Returns a degree of polynomial
polynomialReduce	Reduces polynomials by division
resultant	Computes a resultant of two polynomials
Matrices Characterization	
charpoly	Returns a vector of coefficients of the characteristic polynomial of a matrix
minpoly	Returns a vector of coefficients of the minimal polynomial of a matrix

7.8.2 Numbers and Precision

The Operations on Numbers subgroup of the Numbers and Precision group contains two functions, `imag` and `real`, that allow finding the imaginary and real parts of complex symbolic numbers.

The Precision Control subgroup of the Numbers and Precision group has two functions as well:

`digits` Sets variable-precision accuracy
`vpa` Uses variable-precision arithmetic (VPA)

The `digits` function specifies the number of significant decimal digits and is used to do VPA. The `digits(d)` command sets the current VPA accuracy to `d` digits. The default value is 32 digits. Typing `digits` without an input argument and pushing <Enter> returns the current VPA accuracy setting.

The `vpa` function uses VPA to compute the input argument to `d` decimal digits of accuracy, where `d` is the current setting of `digits`. Each element of the result is a symbolic expression. As an example, the following two calls return the symbolic representation of the *golden ratio* with the default (32 digits) and degraded accuracy of only five decimal digits:

```
>> GR1=vpa((1+sqrt(5))/2), GR2=vpa((1+sqrt(5))/2,5)
GR1 = 1.6180339887498949025257388711907
GR2 = 1.618
```

7.8.3 Number Theory

The Number Theory group functions are listed in Table 7.17.

Table 7.16 Special Polynomials

Function	Description (Recursive Definition)
`bernstein(f,n,t)`	Bernstein polynomials $B(t) = \sum_{k=0}^{n} \beta_k b_{k,n}(t)$, where β_k are Bézier coefficients and $b_{k,n}(t) = \frac{n!}{k!(n-k)!} t^k (1-t)^{n-k}$ are Bernstein basis polynomials (see an example of using these polynomials in Sec. 11.4.4)
`chebyshevT(n,x)` and `chebyshevU(n,x)`	Chebyshev polynomials of the first and second kind $T_0(x) = 1,\ T_1(x) = x,\ T_n(x) = 2xT_{n-1}(x) - T_{n-2}(x)$ $U_0(x) = 1,\ U_1(x) = 2x,\ U_n(x) = 2xU_{n-1}(x) - U_{n-2}(x)$
`gegenbauerC(n,a,x)`	Gegenbauer polynomials $C_0^{(\alpha)}(x) = 1,\ C_1^{(\alpha)}(x) = 2\alpha x,\ C_n^{(\alpha)}(x) = 1/n(2x(n+\alpha-1)C_{n-1}^{(\alpha)}(x) - (n+2\alpha-2)C_{n-2}^{(\alpha)}(x))$
`hermiteH(n,x)`	Hermite polynomials $H_0(x) = 1,\ H_1(x) = 2x,\ H_n(x) = 2xH_{n-1}(x) - 2(n-1)H_{n-2}(x)$
`jacobiP(n,a,b,x)`	Jacobi polynomials $P_0^{(\alpha,\beta)}(x) = 1,\ P_1^{(\alpha,\beta)}(x) = ((2+\alpha+\beta)x + (\alpha-\beta))/2$ $P_n^{(\alpha,\beta)}(x) = \frac{(\rho-1)((\alpha^2-\beta^2)+\rho(\rho-2)x)P_{n-1}^{(\alpha,\beta)}(x) - 2(n-1+\alpha)(n-1+\beta)\rho P_{n-2}^{(\alpha,\beta)}(x)}{2n(n+\alpha+\beta)(\rho-2)}$ where $\rho = 2n+\alpha+\beta$
`laguerreL(n,x)` and `laguerreL(n,a,x)`	Simple and generalized Laguerre polynomials $L_0(x) = 1,\ L_1(x) = 1-x,\ L_n(x) = ((2n-1-x)L_{n-1}(x) - (n-1)L_{n-2}(x))/n$ $L_0^{(\alpha)}(x) = 1,\ L_1^{(\alpha)}(x) = 1+\alpha-x,\ L_n^{(\alpha)}(x) = ((2n-1+\alpha-x)L_{n-1}^{(\alpha)}(x) - (n-1+\alpha)L_{n-2}^{(\alpha)}(x))/n$
`legendreP(n,x)`	Legendre polynomials (see an example of using these polynomials in Sec. 13.8) $P_0(x) = 1,\ P_1(x) = x,\ P_n(x) = ((2n-1)xP_{n-1}(x) - (n-1)P_{n-2}(x))/n$

Table 7.17 Numbers Theory Functions

Function	Description
Elementary Operations	
divisors	Divisors of integer or expression
gcd	Greatest common divisor
factorIntegerPower	Perfect power factoring
isPrimitiveRoot	Determine which array elements are primitive roots (R2020a)
lcm	Least common multiple
mod	Symbolic modulus after division
powermod	Modular power of the number
quorem	Quotient and remainder
rem	Remainder after division
Number Sequences	
bernoulli	Bernoulli numbers and polynomials
euler	Euler numbers and polynomials
fibonacci	Fibonacci numbers
harmonic	Harmonic function (harmonic number)
rat	Rational fraction approximation (continued fraction) (R2020a)
Prime Numbers	
nextprime	Next prime number
nthprime	*n*th prime number
prevprime	Previous prime number
Number-Theoretic Functions (introduced in R2020a)	
eulerPhi	Euler phi function
jacobiSymbol	Jacobi symbol

7.8.4 Special Functions

Table 7.1 lists most of the Special functions as available in R2020a. During symbolic computations, solutions may be expressed in terms of these functions (as was the case with the `psi` function in Sec. 7.5).

7.9 Plotting Symbolic Expressions and Using Symbolic Notations

SMT provides a set of convenient means to visualize symbolic expressions and to add symbolic notations to graphics. We will start from the latter and then proceed with plotting options.

7.9.1 LaTeX Representation of Symbolic Expressions

Let us create several symbolic expressions using the following commands:

```
>> syms x y t
>> F1 = diff(atan(x))
```

```
>> F2 = sym(hilb(3))
>> F4 =(x^2+1/x)/sqrt((sin(x)+cos(x))/x^3.5)
>> F5 = taylor(sin(x),x,'ExpansionPoint',pi/2,'Order',8)
```

They return

```
F1 = 1/(x^2 + 1)
F2 = [   1, 1/2, 1/3]
     [ 1/2, 1/3, 1/4]
     [ 1/3, 1/4, 1/5]
F4 = (1/x + x^2)/((cos(x) + sin(x))/x^(7/2))^(1/2)
F5 = (pi/2 - x)^4/24 - (pi/2 - x)^2/2 - (pi/2 - x)^6/720 + 1
```

Now, let us use the `latex` function introduced in Table 7.8

```
>> H1=latex(F1)
>> H2=latex(F2)
>> H4=latex(F4)
>> H5=latex(F5)
```

that converts the aforementioned symbolic expressions to the LaTeX format:

```
H1 = frac{1}{x^2 + 1}
H2 = \left(\begin{array}{ccc} 1 & \frac{1}{2} &
   \frac{1}{3}\\ \frac{1}{2} & \frac{1}{3} &
   \frac{1}{4}\\ \frac{1}{3} & \frac{1}{4} &
   \frac{1}{5} \end{array} \right)
H4 = \frac{\frac{1}{x} + x^2}{\sqrt{\frac{\cos\!
   \left(x\right) + \sin\!\left(x\right)}
   {x^{\frac{7}{2}}}}}
H5 = \frac{{\left(\frac{\pi}{2} - x\right)}^4}{24} - \frac{{\left(\frac{\
  pi}{2} - x\right)}^2}{2} -\frac{{\left(\frac{\pi}{2} - x\right)}^6}{720} + 1
```

Section 5.12 has already demonstrated how the LaTeX language can be used in the comment section of your script for publishing purposes (also, see Appendix A). The following shows how easy it is to add symbolic expressions to your plots. Issuing the commands

```
>> text(.2,.25,['$' H1 '$'],'interpreter','latex','fontsize',14)
>> text(.6,.2, ['$' H2 '$'],'interpreter','latex','fontsize',16)
>> text(.3,.5, ['$' H4 '$'],'interpreter','latex','fontsize',18)
>> text(.1,.8, ['$' H5 '$'],'interpreter','latex','fontsize',14)
```

results in displaying symbolic expressions (math constructions) within the default axes as shown in Fig. 7.1.

7.9.2 Plotting Symbolic Expressions

The last group of the SMT functions of Table 7.1, Graphics, allows you to create graphical representations of the results of symbolic computations. The function-plotting family of functions (`fcontour`, `fimplicit`,

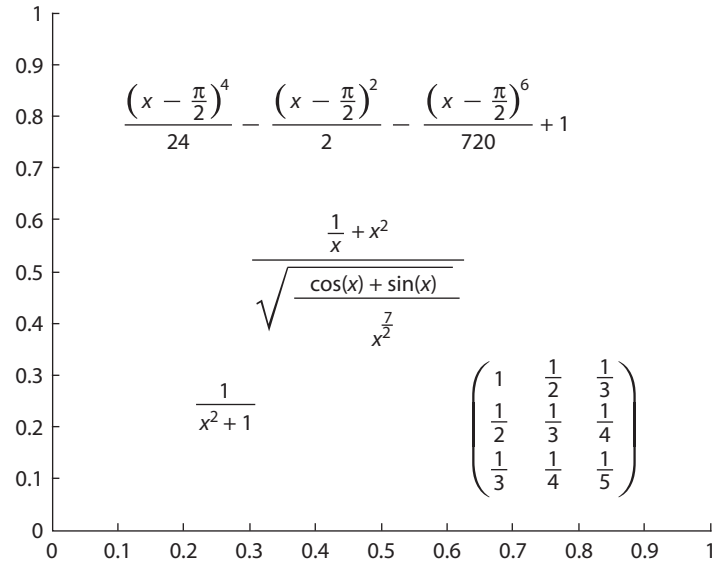


Fig. 7.1 Converting symbolic expressions to the LaTeX character vectors.

`fimplicit3`, `fmesh`, `fplot`, `fplot3`, `fsurf`, and `ezpolar`), which is also capable of working with symbolic expressions, has already been presented in Table 6.35. As mentioned in Sec. 6.10, this family is gradually replacing the easy-to-plot function family: `ezcontour`, `ezcontourf`, `ezmesh`, `ezmeshc`, `ezplot`, `ezplot3`, `ezpolar`, `ezsurf`, and `ezsurfc`.

For example, let us use the `fplot` function to plot the Bessel function (one of the functions of Table 7.18):

```
>> syms z
>> g=besselj(1,z);
>> fplot(g)
```

These commands produce the result shown in Fig. 7.2a. The default domain for `fplot` is $-5 \leq x \leq 5$, but you can alter it by including a vector with the lower and upper limits for the x-axis (and y-axis, if necessary). For example

```
>> fplot(g,50*[-1 1])
```

produces the plot shown in Fig. 7.2b. Of course, the beauty of using the aforementioned plotting functions is that there is no need to compute the vectors of the data points before calling these functions, which results in extreme compactness of code.

Next, let us derive a system of ODEs governing the oscillatory motion of a two-body mass–spring system on a horizontal frictionless surface, as shown in Fig. 7.3.

Table 7.18 Special Functions

Function	Description
Dirac and Heaviside Functions	
`dirac`, `heaviside`, `kroneckerDelta`, `rectangularPulse`, `sympref`, `triangularPulse`	Dirac delta function, Heaviside step function, Kronecker delta function, Rectangular pulse function, Set symbolic preferences, Triangular pulse function
Gamma and Beta Functions, Binomial Coefficients	
`beta`, `factorial`, `gamma`, `gammaln`, `igamma`, `nchoosek`, `pochhammer`	Beta function, Factorial function, Gamma function, Logarithmic gamma function, Incomplete gamma function, Binomial coefficient, Pochhammer symbol
Zeta Function, Dilogarithms, and Polylogarithms	
`catalan`, `dilog`, `polylog`, `psi`, `zeta`	Catalan constant, Dilogarithm function, Polylogarithm, Digamma function, Riemann zeta function
Airy and Bessel Functions	
`airy` `besseli`, `besselj`, `besselk`, `bessely`	Airy function, Modified Bessel function of the first kind, Bessel function of the first kind, Modified Bessel function of the second kind, Bessel function of the second kind
Error Functions, Trigonometric and Hyperbolic Integral Functions, Fresnel and Dawson Integrals	
`erf`, `erfc`, `erfcinv`, `erfi`, `erfinv`, `coshint`, `cosint`, `eulergamma`, `sinhint`, `sinint`, `ssinint`, `ei`, `expint`, `logint`, `dawson`, `fresnelc`, `fresnels`	Error function, Complementary error function, Inverse complementary error function, Imaginary error function, Inverse error function, Hyperbolic cosine integral function, Cosine integral function, Euler–Mascheroni constant, Hyperbolic sine integral function, Sine integral function, Shifted sine integral function, One-argument exponential integral function, Exponential integral function, Logarithmic integral function, Dawson integral, Fresnel cosine integral function, Fresnel sine integral function
Hypergeometric, Whittaker Functions	
`hypergeom`, `kummerU`, `whittakerM`, `whittakerW`	Hypergeometric function, Confluent hypergeometric Kummer U function, Whittaker M function, Whittaker W function
Elliptic Integrals	
`ellipke`, `ellipticCE`, `ellipticCK`, `ellipticCPi`, `ellipticE`, `ellipticF`, `ellipticK`, `ellipticPi`	Complete elliptic integrals of the first and second kinds, Complementary complete elliptic integral of the second kind, Complementary complete elliptic integral of the first kind, Complementary complete elliptic integral of the third kind, Complete and incomplete elliptic integrals of the second kind, Incomplete elliptic integral of the first kind, Complete elliptic integral of the first kind, Complete and incomplete elliptic integrals of the third kind
Lambert W and Wright Functions	
`lambertw`, `wrightOmega`	Lambert W function, Wright omega function

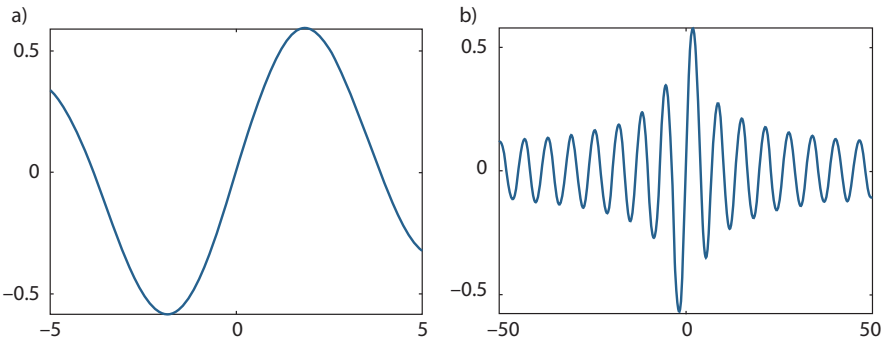


Fig. 7.2 Example of the `fplot` function output with a) default and b) specified *x* domains.

Using Newton's second law, we can write

$$\begin{aligned} m_1\ddot{y}_1 &= -k_1 y_1 + k_2(y_2 - y_1) \\ m_2\ddot{y}_2 &= -k_2(y_2 - y_1) \end{aligned} \tag{7.7}$$

where $y_1 = y_1(t)$ and $y_2 = y_2(t)$ are the displacements of the masses m_1 and m_2, such that $y_1 = 0$ and $y_2 = 0$ correspond to the position of static equilibrium, and the terms on the right-hand side describe the forces produced by the compressed (expanded) springs, with k_1 and k_2 denoting spring constants.

The following code produces a solution to Eq. (7.7) with $m_1 = m_2 = 1\,\text{kg}$, $k_1 = 3\,\text{kg/s}^2$, and $k_2 = 2\,\text{kg/s}^2$ for $t \in [0; 10]\text{s}$ with $y_1(0) = 1\,\text{m}$, $y_2(0) = 2\,\text{m}$, $\dot{y}_1(0) = -2\sqrt{6}\,\text{m/s}$, and $\dot{y}_2(0) = \sqrt{6}\,\text{m/s}$:

```
syms y1(t) y2(t)
% Define system parameters in kg and kg/s/s
m1=1; m2=1; k1=3; k2=2;
% Define equations
eqn1 = m1*diff(y1,t,2) == -k1*y1+k2*(y2-y1);
eqn2 = m2*diff(y2,t,2) == -k2*(y2-y1);
% Define initial conditions
```

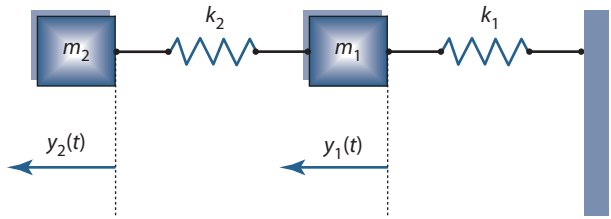


Fig. 7.3 Two-body mass–spring system.

```
Dy1 = diff(y1,t); Dy2=diff(y2,t);
cond = [y1(0)==1, Dy1(0)==-2*sqrt(6),...
        y2(0)==2, Dy2(0)==sqrt(6)];
% Find a solution
Sol = dsolve([eqn1 eqn2],cond);
% Plot the results within t=[0;10]s
subplot(221), fplot(Sol.y1,[0 10]), xlabel('t, s'), ylabel('y_1(t), m')
              title(['$' latex(Sol.y1) '$'],'interpreter','latex')
subplot(223), fplot(Sol.y2,[0 10]), xlabel('t, s'), ylabel('y_2(t), m')
              title(['$' latex(Sol.y2) '$'],'interpreter','latex')
subplot(222), fplot(diff(Sol.y1),[0 10])
           xlabel('t, s'), ylabel('$\dot{y}_1(t)$, m/s','interpreter','l')
           title(['$' latex(diff(Sol.y1)) '$'],'interpreter','latex')
subplot(224), fplot(diff(Sol.y2),[0 10])
           xlabel('t, s'), ylabel('$\dot{y}_2(t)$,m/s','interpreter','l')
           title(['$' latex(diff(Sol.y2)) '$'],'interpreter','latex')
```

The resulting plots are shown in Fig. 7.4.

Figure 7.5a shows another example, when the 3-D plotting function, `fplot3`, was used to present the results of solving the system of two differential equations as discussed in the last example of Sec. 7.6:

```
>> syms x(t) y(t)
>> [x,y]=dsolve(diff(x)==y,diff(y)==cos(t),[x(0)==1,y(0)==0]);
>> fplot3(t,x,y,[0 5*pi]), xlabel('t, s'), ylabel('x(t)'), zlabel('y(t)')
```

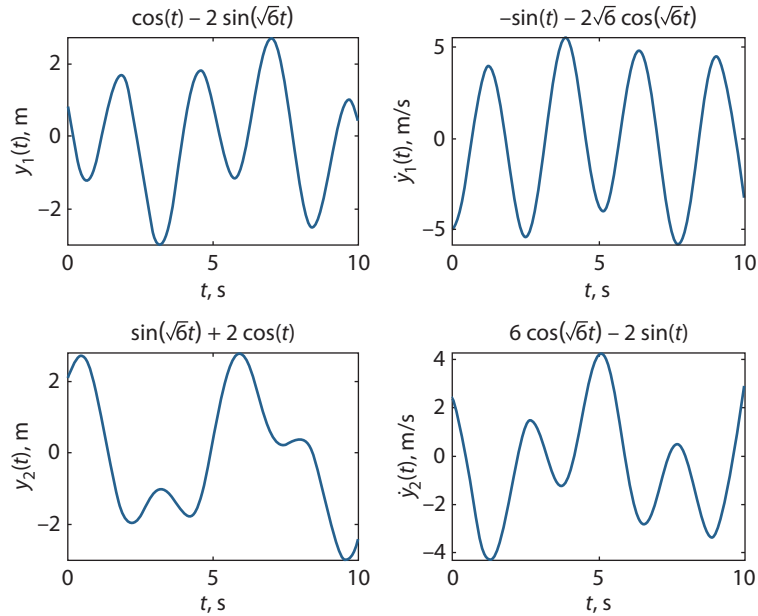


Fig. 7.4 Dynamics of the system shown in Fig. 7.3.

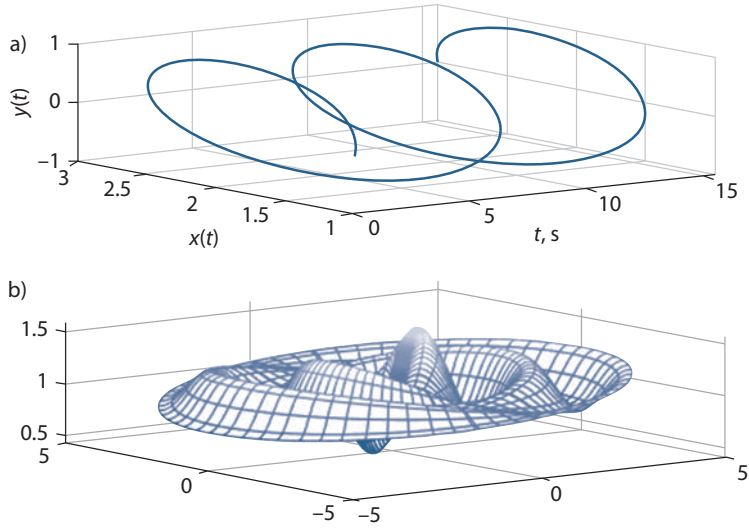


Fig. 7.5 Examples of exploiting a) `fplot3` and b) `fmesh` functions.

Finally, Fig. 7.5b shows an example of using the surface-plotting function, `fmesh`. The commands that produce this nice membrane without creating a mesh grid (as you would need to do in the case of using the `mesh` function) are as follows:

```
>> syms r phi
>> x=r*cos(phi); y=r*sin(phi);
>> z=1+besselj(1,3*r)*cos(phi);
>> fmesh(x,y,z)
```

7.10 Using Symbolic Expressions in Numerical Operations

MATLAB offers several ways of using symbolic expressions in numerical operations. In what follows, we start by showing how to use the `fcnchk` and `inline` functions introduced earlier, in Secs. 5.9.4 and 5.8.1, respectively (the `inline` function might be removed in a future release), and then proceed with the `matlabFunction` function, specifically designed for these purposes. Finally, we show how to use the SMT functions to automatize rewriting higher-order ODEs as a system of the first-order ODEs, ready to be numerically integrated with any ODE solver (considered in Chapter 14).

7.10.1 Employing the `fcnchk` and `inline` Functions

As shown in Sec. 7.9, the function-plotting functions allow using symbolic expressions as input arguments. However, other functions of MATLAB do not necessarily permit it. For example, although the call

```
>> syms x, fplot(sin(x))
```

does produce an expected sinusoid, the following commands, intended to find a minimum of a sinusoid $f(x) = \sin x$ near $x = 1$ using the MATLAB `fminsearch` function (to be addressed further in Chapter 10),

```
>> syms x, fminsearch(sin(x),1)
```

return an error

```
??? Error using ==> fcnchk at 106
If FUN is a MATLAB object, it must have an feval method.
Error in ==> fminsearch at 173
funfcn = fcnchk(funfcn,length(varargin));
```

As usual, the detailed explanation of the error provides a clue for resolving the issue. Indeed, the reason for the error is stated fairly clearly: the wrong type of an input argument provided to the `fcnchk` function, which appears in `fminsearch` to convert the first input argument into a function handle (as was discussed in Sec. 5.9.4). Hence, to use `fminsearch` and other MATLAB functions with symbolic objects, you first need to convert them into the format understood by the `fcnchk` function. Specifically, the symbolic object has to be converted to a character vector. The MATLAB function that does it is `char`. Replacing `sin(x)` with `char(sin(x))` in the aforementioned example fixes the problem.

Of course, in the latter example, we could have used a character vector as opposed to a symbolic object from the very beginning, for example,

```
>> fminsearch('sin(x)',1)
```

However, it would be a waste of time if we performed complex symbolic manipulations and were not able to use the result in numerical computations. Therefore, the recipe provided earlier is a key to the effective use of SMT.

Once your symbolic mathematical expression has been converted into a character vector, you may do some other useful things with it as well. For instance, you can convert it into an inline function. The two ways of doing this are to use either the `fcnchk` or the `inline` function. The only difference is that if you want your function to be vectorized (a `.` inserted before any `^`, `*`, or `/`), you use a slightly different format, as follows:

```
f=fcnchk(f,'vectorized') vs f=inline(vectorize(f))
```

To conclude this discussion, let us consider the following general example. Suppose that dealing with SMT, you produced some symbolic mathematical expression. You then wish to plot it (using the `fplot` or `ezplot` function, you can do it right away using just a symbolic object as an input argument). After that, you want to find one of the zeros of this mathematical expression. To do that, you will need to use the MATLAB `fzero` function, introduced in Sec. 5.8.2 (and to be considered in more detail in Chapter 10). As you know already, for this purpose, you will need to convert your symbolic mathematical expression into a function handle to a scalar-valued anonymous function or inline function or character

vector/string scalar. Finally, you would like to add your initial guess and a zero to the plot produced earlier. The following set of commands provides a complete solution to the problem:

```
syms y(t)
fsym=dsolve(diff(y)==y^2+1,y(0)==1);
ezplot(fsym), hold, grid
fcha=char(fsym);
finl=fcnchk(fcha);
xz=fzero(finl,0);
plot(0,finl(0),'gV')
plot(xz,finl(xz),'rp')
```

The resulting Workspace and Figure windows are shown in Figs. 7.6a and 7.6b, respectively.

7.10.2 Using the `matlabFunction` Function

MATLAB features another, even more powerful way to enable the use of symbolic expressions in numerical operations. Specifically, the `matlabFunction` allows you to convert the symbolic expression `f` into an anonymous function with a handle `g`. This function features several syntaxes:

```
g = matlabFunction(f)
g = matlabFunction(f1,f2,...)
g = matlabFunction(f,name,value,...)
```

You can then use the handle `g` to calculate numerical values as if you were substituting numbers for variables in a symbolic expression.

For example, the following set of commands creates a handle to the function that calculates a dynamic pressure using the static pressure `ps` and Mach number `M` inputs:

```
>> syms ps M
>> q = 0.7*ps*M^2;
>> hq = matlabFunction(q)
hq = @(M,ps)M.^2.*ps.*(7.0./1.0e1)
```

Now, the call

```
>> hq(0.5,101325)
```

returns the dynamic pressure (in Pascals) for $M = 0.5$ and a standard (static) atmospheric pressure:

```
ans = 1.7732e+004
```

Because `matlabFunction` creates a vectorized function, you can use the `hq` function with the matrix inputs as well. For example,

```
>> Mn = [0.2,0.2,0.6]; Psn = [101325, 60000, 101325];
>> hq(Mn,Psn)
```

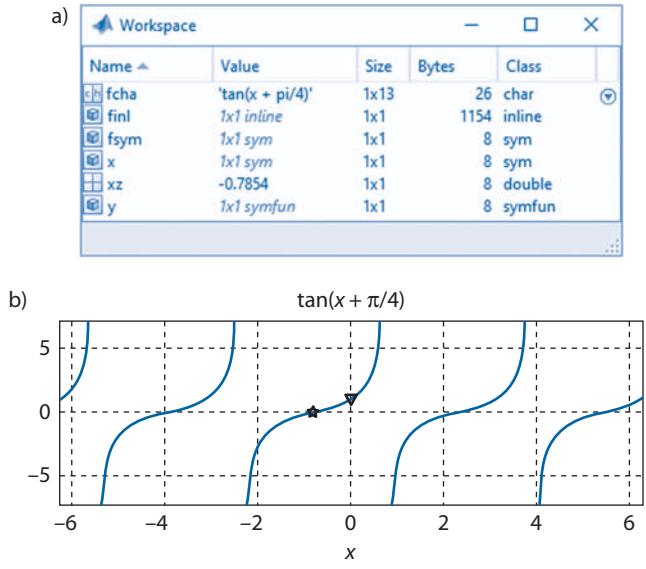


Fig. 7.6 Examples of using symbolic expressions in numerical calculations.

yields

```
ans = 1.0e+004 *
    0.2837  0.1680  2.5534
```

Note that `matlabFunction` generates the list of input arguments in alphabetical order, and that is why the function handle `hq` has `M` before `ps`. However, if needed, you can specify the order of input arguments in the function handle using the `vars` name–value pair argument, by passing a cell array of character vectors or vectors of symbolic variables. For example, the command to create an anonymous function with the reverse order of inputs can look like

```
>> hq = matlabFunction(q,'vars',{'ps','M'})
```

or

```
>> hq = matlabFunction(q,'vars',[ps M])
```

In either case, the return is

```
hq = @(ps,M)M.^2.*ps.*(7.0./1.0e1)
```

In addition, the `matlabFunction` allows you to create an *m*-function as opposed to an anonymous function. All you have to do is specify the file name using the `file` name–value pair argument (if you do not specify the path to the file, `matlabFunction` creates this file in the current directory). The following example generates an *m*-function

that calculates a rotation matrix from the Earth-Centered, Earth-Fixed coordinate frame to the East-North-Up local tangent plane using two inputs, `lat` and `lon`:

```
syms lat lon
Rlon = [cos(lon)  sin(lon)  0;  -sin(lon)  cos(lon)  0;  0  0  1];
Rlat = [cos(lat)  0  sin(lat);  0  1  0;  -sin(lat)  0  cos(lat)];
R = Rlat*Rlon;
F = matlabFunction(R,'file','RotationMatrix.m')
```

The file *RotationMatrix.m* contains the following code:

```
function R = RotationMatrix(lat,lon)
%ROTATIONMATRIX
% R = ROTATIONMATRIX(LAT,LON)
% This function was generated by the Symbolic Math Toolbox version 8.4.
% 31-Oct-2019 12:18:58
t2 = cos(lat);
t3 = sin(lon);
t4 = cos(lon);
t5 = sin(lat);
R = reshape([t2.*t4,-t3,-t4.*t5,t2.*t3,t4,-t3.*t5,t5,0.0,t2],[3, 3]);
```

As seen, `matlabFunction` generates several intermediate variables, which constitute an *optimized* code. Using intermediate variables results in more efficient code (because of reusing these intermediate expressions) and makes it easier to read (by keeping expressions short). Once again, if you do not happy with the default alphabetical order of input argumants, you may always use the `vars` name–value pair argument. Continuing the example mentioned earlier,

```
F=matlabFunction(R,'file','RotationMatrix.m','vars',[lon lat])
```

generates a file equivalent to the previous one, with a different order of inputs

```
function R = RotationMatrix (lon,lat)
...
```

The `matlabFunction` function also allows you to have multiple output variables, with customization of their names by using the `output` name–value pair argument. For instance, if in the previous example, you use a slightly modified call

```
F=matlabFunction (R,Rlat,Rlon,'file',...
                  'RotationMatrix.m','outputs',{'R','Rlat','Rlon'})
```

it will result in generating the following function:

```
function [R,Rlat,Rlon] = RotationMatrix(lat,lon)
%ROTATIONMATRIX
% [R,RLAT,RLON] = ROTATIONMATRIX(LAT,LON)
```

```
% This function was generated by the Symbolic Math Toolbox version 8.1.
% 03-Sep-2019 12:18:59
t2 = cos(lat);
t3 = sin(lon);
t4 = cos(lon);
t5 = sin(lat);
R = reshape([t2.*t4,-t3,-t4.*t5,t2.*t3,t4,-t3.*t5,t5,0.0,t2],[3, 3]);
if nargout > 1
   Rlat = reshape([t2,0.0,-t5,0.0,1.0,0.0,t5,0.0,t2],[3, 3]);
end
if nargout > 2
   Rlon = reshape([t4,-t3,0.0,t3,t4,0.0,0.0,0.0,1.0],[3, 3]);
end
```

As seen, this latter function automatically incorporates possibilities to be called using one, two, or three output arguments (and uses the `nargout` function discussed in Sec. 5.9.3).

7.10.3 Rewriting Higher-Order ODEs into State Space

Let us conclude this section with two examples showing how to prepare a higher-order ODE to be solved using one of the numerical ODE solvers of MATLAB suited to handle only the first-order ODEs (to be discussed in Chapter 14). Suppose you have the following fourth-order differential equation:

$$y^{(iv)} + 2\ddot{y} - 10y\dot{y} = 0 \tag{7.8}$$

Issuing the following set of commands allows the conversion of this differential equation to the anonymous function in the correct form, called *state–space representation*, as required by ODE solvers:

```
>> syms y(t)
>> Eqn=diff(y,4)+2*diff(y,2)-10*y*diff(y);
>> [V,S]=odeToVectorField(Eqn)
>> odefun=matlabFunction(V,'vars',{'t','Y'})
V =
                  Y[2]
                  Y[3]
                  Y[4]
   10*Y[1]*Y[2] - 2*Y[3]
 S =
       y
      Dy
     D2y
     D3y
 odefun =
  function_handle with value:
    @(t,Y) [Y(2);Y(3);Y(4);Y(3).*-2.0+Y(1).*Y(2).*1.0e+1]
```

(odeToVectorField is one of the Equation Solving group functions of Table 7.13).

Essentially, Eq. (7.8) was converted to four first-order equations as follows:

$$\begin{aligned} \dot{Y}_1 &= Y_2 && (= \dot{y}) \\ \dot{Y}_2 &= Y_3 && (= \ddot{y}) \\ \dot{Y}_3 &= Y_4 && (= \dddot{y}) \\ \dot{Y}_4 &= -2Y_3 + 10Y_1Y_2 && (= -2\ddot{y} + 10y\dot{y}) \end{aligned} \tag{7.9}$$

The anonymous function odefun can now be used to produce a numerical solution (to solve the so-called *initial-value problem*). For example,

```
initConditions=[0,-3,5,-3];          % set initial conditions
ode23(odefun,[0,2],initConditions) % use ODE23 solver
grid, xlabel('Time, s'), ylabel('Solution')
legend('$x(t)$','$\dot{x}(t)$','$\ddot{x}(t)$','$x^{iii}(t)$',...
       'location','sw','interpreter','latex','fontsize',13,...
       'Numcolumns',2)
```

integrates Eq. (7.9) within $t \in [0; 2]s$ with $y(0) = 0$, $\dot{y}(0) = -3$, $\ddot{y}(0) = 5$, and $\dddot{y}(0) = -3$ and produces the plot shown in Fig. 7.7.

The same result could be achieved using a combination of several other functions from the Equation Solving group of Table 7.13. Specifically, the same odefun function can be created using the following code:

```
%% Define the 4th-order ODE
syms y(t)
eqn=diff(y,4)+2*diff(y,2)-10*y*diff(y);
%% Rewrite ODE as a system of 1st-order ODEs
[eqs,vars]=reduceDifferentialOrder(eqn,y)
%% Find the mass matrix M and the right-hand sides of equations
% $M(t,x(t),\dot{x}(t)=F(t,x(t))$
[M,F]=massMatrixForm(eqs,vars)
%% Rewrite the system in the standard form
```

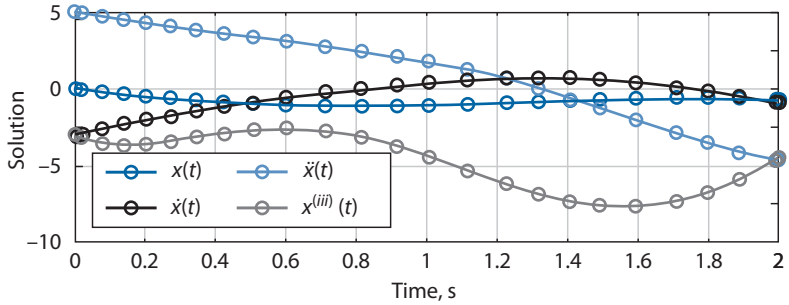


Fig. 7.7 Numerical solution of the fourth-order ODE given as a symbolic equation.

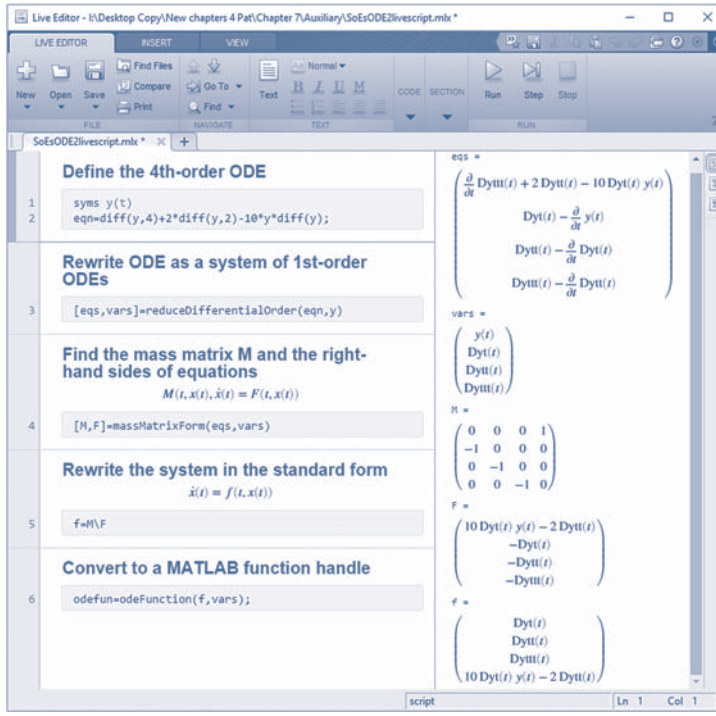


Fig. 7.8 Example of using `reduceDifferentialOrder`, `massMatrixForm`, and `odeFunction` functions.

```
% $\dot{x}(t)=f(t,x(t))$
f=M\F
%% Convert to a MATLAB function handle
odefun=odeFunction(f,vars);
```

For better visualization, let us take advantage of opening this script as a Live Script and running it in Live Editor. The outputs are shown in Fig. 7.8.

7.11 Symbolic Calculators

As for any other MATLAB toolbox or Simulink blockset, SMT has several demos providing hints and examples on using a variety of functions. Among these demos, there are three interactive GUIs, shown in Table 7.19,

Table 7.19 Functions Invoking GUIs for Symbolic Computations

GUI	Description
`funtool`	Function calculator
`rsums`	Interactive evaluation of the Riemann sums (also shown in Table 7.10)
`taylortool`	Taylor series calculator (also shown in Table 7.10)

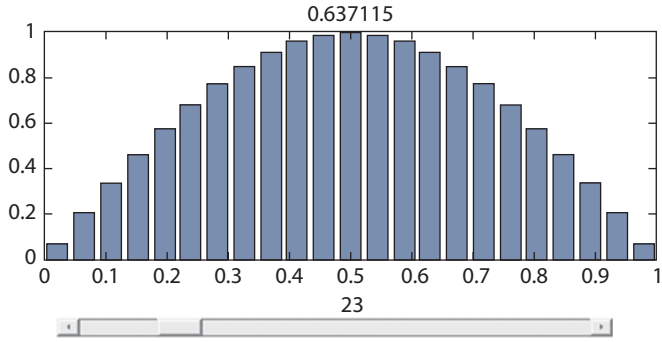


Fig. 7.9 Example of using the interactive Riemann sums calculator of SMT.

that exploit different functions of this toolbox and present the results graphically.

For educational purposes, let us briefly mention these GUIs here because once you learn how to use the MathWorks GUI design environment, GUIDE, presented in Appendix F, you will be able to create such GUIs yourself.

We start with the `rsums` function, which can be used in one of the following formats: `rsums(f)`, `rsums(f,a,b)`, or `rsums(f,[a,b])`,

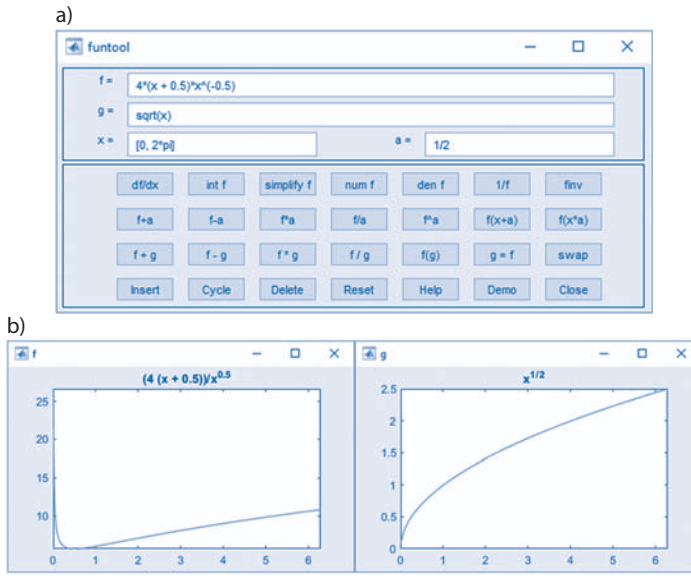


Fig. 7.10 Examples of using the function calculator of SMT.

where `f` is the function (a symbolic expression or character vector), and `a` and `b` are the limits (the default values are `0` and `1`, respectively). A Riemann sum is a method for numerical evaluation of a definite integral (approximating the total area underneath a curve on a graph). In fact, there are several approaches including Left Rectangles sum, Right Rectangles sum, and Trapezoidal rule (to be considered in Chapter 13 in detail). For example, the call

```
>> syms x, rsums(sin(pi*x))
```

displays a graph of $f(x) = \sin(\pi x)$ within $x \in [0;1]$ and allows you to adjust the number of terms taken in the Riemann sum by using a slider below the graph (Fig. 7.9 shows 23 terms). The number of terms available ranges from 2 to 128.

The `funtool` call invokes three windows of a visual function calculator (Fig. 7.10) that manipulates and displays two functions of one variable, $f(x)$ and $g(x)$. At the click of a button, this function calculator draws a graph

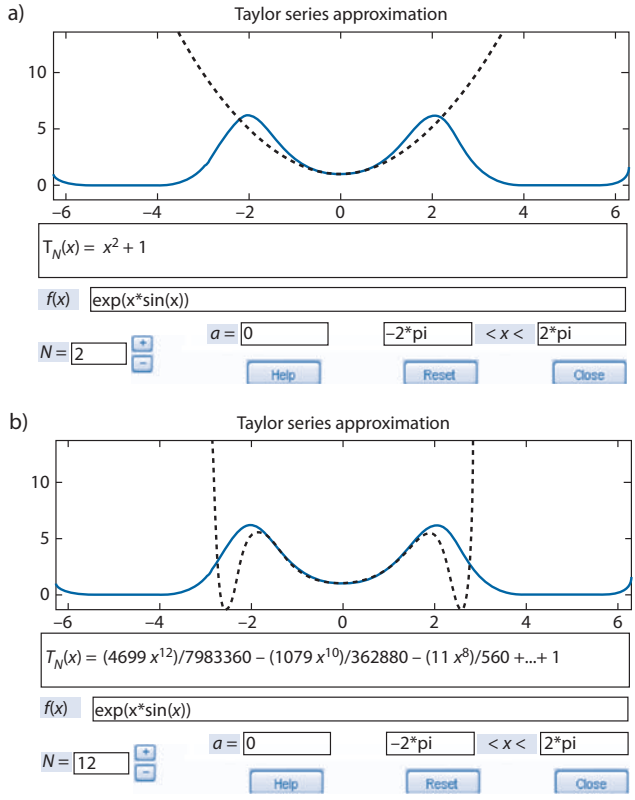


Fig. 7.11 Examples of using the Taylor tool of SMT for $f(x) = e^{x\sin(x)}$ with a) 2 and b) 12 terms.

representing the sum, product, difference, or ratio of two functions that you specify (the result shows up in the $f(x)$ function window).

Finally, the Taylor series calculator, which can be invoked by typing `taylortool` or `taylortool(f)` in the Command Window, brings up a display (Fig. 7.11) allowing you to graphically explore the quality of the Taylor series of a function of interest `f` with a specified number of terms [see Eq. (7.2)]. As shown in Fig. 7.11, increasing the number of terms in the Taylor series expansion allows you to have a better approximation of the function.

7.12 Units of Measurement

In R2017a, a new SMT feature allowing you to convert between units of measurement was introduced. For completeness, several self-explanatory examples are shown in a live script in Fig. 7.12.

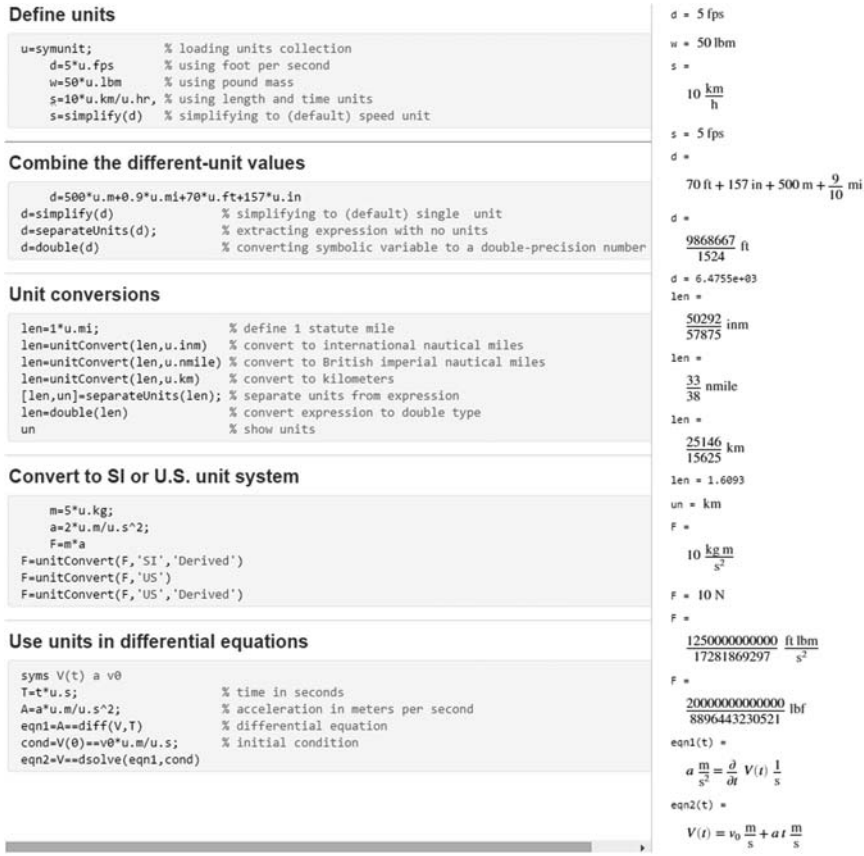


Fig. 7.12 Example of using and manipulating with the units of measurement in SMT.

Problems

7.1 Use the `limit` function to compute $\lim_{x\to 0^+}(1-x)^{x^{-2}}$. Show that it is different from $\lim_{x\to 0^-}(1-x)^{x^{-2}}$. Prove your findings graphically (using the `fplot` function).

7.2 The equation for a curve called the "lemniscate" (of Bernoulli) in polar coordinates (r,θ) is $r^2 = a^2\cos(2\theta)$. Use the `subs` function to find the equation for this curve in Cartesian coordinates (x,y), where $x = r\cos\theta$ and $y = r\sin\theta$. Use the `fplot` function to plot several lemniscates computed for different values of coefficient a in the same figure. Add symbolic notations.

7.3 Use the `dsolve` function to solve two differential equations: $\dot{x} - y = 0$ and $\dot{y} + 2x + 5 = 0$ with $x(0) = 0$ and $y(0) = 1$. Plot the results in the range $t = [0;5]$s using the `fplot` function. Use the `fzero` function to compute the location of the first three zeros for both $x(t)$ and $y(t)$. Add them to the plot. Add the axis labels, and using the `text` function, show the mathematical expression of each solution by its curve.

Chapter 8 Accuracy of Digital Computations

- Understanding Computer Number Representation
- Estimating Round-Off and Truncation Errors
- Numerical Error Control

`bin2dec`, `dec2bin`, `dec2hex`, `hex2dec`, `num2hex`, `hex2num`, `base2dec`, `dec2base`, `uint8` ... `uint64`, `int8` ... `int64`, `cast`, `typecast`, `intmax`, `intmin`, `eps`, `realmax`, `realmin`, `single`, `double`, `vpa`

8.1 Introduction

Before proceeding with any numerical method, it is important to understand what to expect in terms of accuracy and precision when programming mathematical formulas on a computer. The primary objective of this chapter is to acquaint you with the major sources, quantification, and minimization of errors involved in numerical computations. The first source of errors is the computer itself because it is limited in its ability to represent the magnitudes and precision of numbers. Second, this imperfection is aggravated further by inappropriate handling of these numbers. Last, the mathematical approximation or numerical method may introduce some additional errors. Specific objectives and topics covered in this chapter are divided into four sections. The chapter starts by introducing concepts of accuracy and precision of calculations and measurements. Next, it discusses the alternative ways of representing numbers in computer's memory, followed by showing how a limited ability to represent the numbers results in round-off errors. Another source of numerical errors, truncation errors, associated with the fact that mathematical expressions, even those as simple as π, are represented by approximations rather than exact formulas, is presented later. The chapter ends with some practical advices on how to control (mitigate) these errors.

8.2 Accuracy and Precision of Calculations and Measurements

The errors associated with both calculations and measurements can be characterized with the regard to their *accuracy* and *precision*. Accuracy

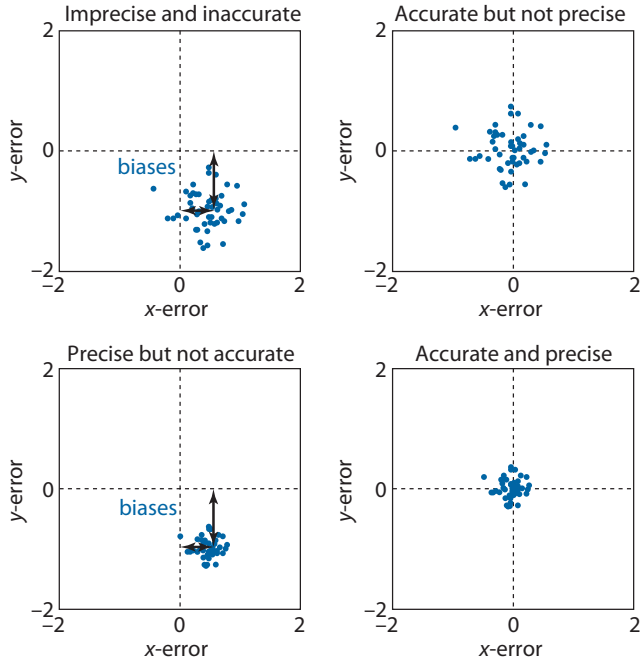


Fig. 8.1 Illustration of accuracy and precision concepts.

refers to how closely a computed or measured value agrees with the true value. Precision refers to how closely individual computed or measured values agree with each other. *Inaccuracy* (also called *bias*) is defined as systematic deviation from the truth, whereas imprecision (also called *uncertainty*) refers to the magnitude of the scatter. These concepts are illustrated in Fig. 8.1, where you can assume that the true value for some 2-D quantity is located at the (0,0) point.

The general requirement for any numerical method to be discussed in the following chapters (Chapters 9–14) is that it should be sufficiently accurate or unbiased to meet the requirements of a particular problem and also precise enough so that you do not get different results every time you get a numerical solution. We will use the collective term *error* to represent both inaccuracy and imprecision of our estimates. For each numerical method, we explicitly address the issue of the upper bound of the error this method guarantees, thus helping you understand the method's limitations.

8.3 Computer Number Representation

Recall that by default, MATLAB allocates 8 bytes per each numeric variable, which means it deals with the *double-precision* numbers. However,

MATLAB also allows you to represent numbers using other formats or *data types*. The term data type refers to the way in which a computer represents the numbers in its memory. A data type determines the amount of storage allocated to a number, method used to encode the number's value as a pattern of binary digits, and operations available for manipulating with the specific data type. Most computers provide a choice of data types for representing numbers, each with specific advantages in the areas of precision, dynamic range, performance, and memory usage. To enable you to take advantage of different ways to represent numerical data to optimize (increase) the performance of your programs (and decrease the size of code generated from the MATLAB and Simulink models), MATLAB allows you to vary the data type of variables.

Numerical round-off errors are directly related to the manner in which the numbers are stored in computer's memory. The fundamental unit whereby information is represented is called a *word*. This is an entity that consists of a string of *binary* digits, or *bits*. Numbers are typically stored in one or more words. To understand how this is accomplished, we must first review different numerical systems and the way they represent the numbers.

8.3.1 Numerical Systems

A number system is merely a convention for representing quantities. Because we have 10 fingers (and 10 toes), the number system that we are most familiar with is the *decimal*, or *base-10*, number system. A base is the number used as the reference for constructing the system. The base-10 system uses 10 digits—0, 1, 2, 3, 4, 5, 6, 7, 8, and 9, traditionally called Arabic—to represent the numbers.

In fact, real Arabic digits, Hindi digits, represented by the values U + 0660 through U + 0669 in Unicode as shown in the following table, were adopted from writing in India and look quite different from Western or European digits as Unicode Character Standard refers to them

European Digits	0	1	2	3	4	5	6	7	8	9
Arabic Digits	٠	١	٢	٣	٤	٥	٦	٧	٨	٩

(By the way, although Arabic text is written right to left, numbers are written the same way as in left-to-right languages, with the most significant digit on the left.)

By themselves, these digits are satisfactory for counting from 0 to 9. For larger quantities, combinations of these basic digits are used, with the position or *place value* specifying the magnitude. The rightmost digit in a whole number represents a number from 0 to 9 or a multiple of 10 in the zeroth power. The second digit from the right represents a multiple of 10 (in the first power). The third digit from the right represents a multiple of 100 (10 in the second power), and so on. If we have a fractional

number, the same paradigm applies—the first number to the right of a *decimal point* (aka *radix point*) is a multiple of 0.1 (10 in the minus first power), etc. For example, if we have the number 3451.67, then we have three groups of 1000, four groups of 100, five groups of 10, one group of 1, six groups of 0.1, and seven groups of 0.01, or

$$\mathbf{3} \times 10^3 + \mathbf{4} \times 10^2 + \mathbf{5} \times 10^1 + \mathbf{1} \times 10^0 + \mathbf{6} \times 10^{-1} + \mathbf{7} \times 10^{-2} = \mathbf{3451.67}$$

This type of representation is called *positional notation*.

Now, because the decimal system is so familiar, it is not commonly realized that there are alternatives. Ancient peoples knew the *base-12* or *duodecimal* system (which originated in ancient Mesopotamia and is still used today to count time and as the English system of units), the *base-20* or *vigesimal* system (used by the ancient Mayans and traced in many European languages), the *base-60* or *sexagesimal* system (also used in ancient Mesopotamia by Sumerians and Babylonians), and others. Some systems, like the *quinary* or *base-5* system (derived from having five fingers on either hand) are used subconsciously as a sub-base system. The main usage of a quinary system, for example, is as a *biquinary* system, that is, the decimal system using 5 as a sub-base. Another example of a sub-base system is the base-60 system mentioned earlier, which uses 10 as a sub-base.

If human beings happened to have three fingers (like the Asgard race in the *Stargate* TV serial), we would probably use a *ternary* or *base-3* representation. Such a number system would use only three digits, 0, 1, and 2, but could still use positional notation. In this case, the ternary number 103.22_3 (we use a subscript to denote the base of numeration, *radix*, of a specific non-decimal numerical system) would be converted to the decimal system as follows:

$$\begin{aligned}\mathbf{110.22}_3 &= \mathbf{1} \times 3^2 + \mathbf{1} \times 3^1 + \mathbf{0} \times 3^0 + \mathbf{2} \times 3^{-1} + \mathbf{2} \times 3^{-2} \\ &= 9 + 3 + 0 + 2/3 + 2/9 = 12 + 8/9 = 12.88(8)\end{aligned}$$

In fact, related to computers, it has been proven (in the numbers theory) that it is the ternary system that provides the most effective (including compact) computations. That is why in the beginning of the computer era there were several known attempts to create a computer that would use trinary arithmetic. However, these attempts failed because it is quite difficult to support this system electronically. On the contrary, having two levels of a signal—on/off electronic components—happened to be the easiest solution for digital computers, and that is why all modern computers use a *binary* or *base-2* numerical systems and its derivatives, *octal*, or *base-8*, and *hexadecimal*, or *base-16*, systems. (For the record, the modern binary number system was first fully documented by Gottfried Leibniz in the 17th century in his article "Explication de l'Arithmétique Binaire".) Just as with the decimal and ternary systems, numeric quantities in these three numerical systems can be represented using positional notation, as shown in the

following examples:

$$\begin{aligned}\mathbf{101.11}_2 &= \mathbf{1}\times 2^2 + \mathbf{0}\times 2^1 + \mathbf{1}\times 2^0 + \mathbf{1}\times 2^{-1} + \mathbf{1}\times 2^{-2}\\ &= 4+0+1+0.5+0.25 = 5.75\\ \mathbf{268.42}_8 &= \mathbf{2}\times 8^2 + \mathbf{6}\times 8^1 + 7\times 8^0 + \mathbf{4}\times 8^{-1} + \mathbf{2}\times 8^{-2}\\ &= 128+48+7+0.5+1/32 = 183.53125\\ \mathbf{1F0.B}_{16} &= \mathbf{1}\times 16^2 + \mathbf{F}\times 16^1 + \mathbf{0}\times 16^0 + \mathbf{B}\times 16^{-1} + \mathbf{2}\times 16^{-2}\\ &= 256+240+0+11/16 = 496.6875\end{aligned}$$

Note that, because we have only 10 digits—0, 1, 2, 3, 4, 5, 6, 7, 8, 9—and for the base-16 system we need six more, the letters **A** through **F** are used to represent them as follows: **A** – 10, **B** – 11, **C** – 12, **D** – 13, **E** – 14, **F** – 15.

By the way, to play with these numerical systems you are welcome to use the Calculator app for Windows. You are probably familiar with the standard (simplified) format already. You can change it to the advanced format developed to do operations with bases and conversions between them by choosing the **Programmer** option from the **View** menu (Fig. 8.2a). As seen from Fig. 8.2b, by choosing the corresponding option from the four-item list located by the left edge of the calculator window, you may switch to and conduct operations in the hexadecimal, decimal, octal, or binary system and conveniently convert the numbers between them.

MATLAB offers several functions to convert between these representations – `bin2dec`, `dec2bin`, `dec2hex`, `hex2dec`, `num2hex`, `hex2num`, `base2dec`, and `dec2base`. (Use the Help system to learn more about these functions [Figs. 2.24, 2.25]). For example,

```
disp(bin2dec('11111111'))   returns  255
disp(dec2bin(22))           returns  10110
disp(base2dec('203',4))     returns  35
disp(dec2base(203,3))       returns  21112
```

Figure 8.3a presents general ideas of how to convert numbers from one numerical system to another, whereas Fig. 8.3b provides the basics of binary arithmetic.

Finally, the following example demonstrates that the octal and hexadecimal systems are indeed the derivatives from the binary system because you can easily convert any number represented in the binary system to the other two by appropriate grouping of digits in a binary number (from the radix point to the left for the integer part and the to right for the decimal part, adding zeros if necessary) and computing decimal values for these

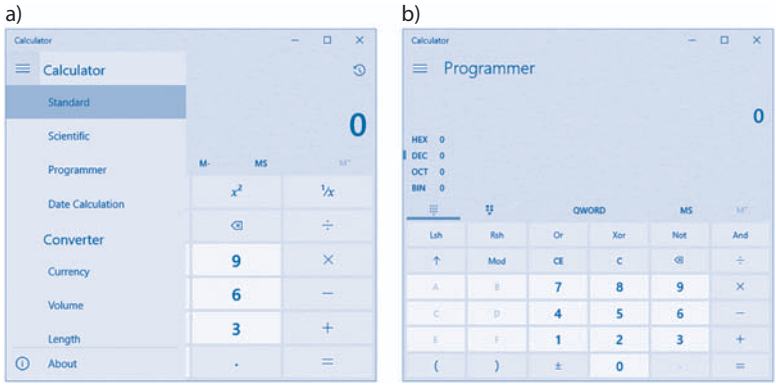


Fig. 8.2 Changing Microsoft Calculator from a) Standard to b) Programmer format.

a)

Decimal to binary

136_{10}

Operation	Remainder
136 ÷ 2 = 68	0
68 ÷ 2 = 34	0
34 ÷ 2 = 17	0
17 ÷ 2 = 8	1
8 ÷ 2 = 4	0
4 ÷ 2 = 2	0
2 ÷ 2 = 1	0
1 ÷ 2 = 0	1

Decimal to ternary

136_{10}

Operation	Remainder
136 ÷ 3 = 45	1
45 ÷ 3 = 15	0
15 ÷ 3 = 5	0
5 ÷ 3 = 1	2
1 ÷ 3 = 0	1

Decimal to octal

136_{10}

Operation	Remainder
136 ÷ 8 = 17	0
17 ÷ 8 = 2	1
2 ÷ 8 = 0	2

b)

10001000_2 12001_3 210_8

Addition

0 + 0 → 0
0 + 1 → 1
1 + 0 → 1
1 + 1 → 0, **carry 1**
(since 1 + 1 = 0 + 1 × 10 inbinary)

```
  1 1 1 1 1   (carried digits)
    0 1 1 0 1
+   1 0 1 1 1
-------------
= 1 0 0 1 0 0
```

Subtraction

0 − 0 → 0
0 − 1 → 1, **borrow 1**
1 − 0 → 1
1 − 1 → 0

```
    * * * *   (columns borrowed from)
  1 1 0 1 1 1 0
−   1 0 1 1 1
-------------
= 1 0 1 0 1 1 1
```

Multiplication

```
        1 0 1 1
    ×   1 0 1 0
    -----------
        0 0 0 0
+     1 0 1 1
+   0 0 0 0
+ 1 0 1 1
---------------
= 1 1 0 1 1 1 0
```

Division

```
1 1 0 1 1 1 | 1 0 1
-------------------
−1 0 1        1 0 1 (quotient)
-------
  0 1 1
− 0 0 0
-------
    1 1 1
  − 1 0 1
  -------
      1 0 (remainder)
```

Fig. 8.3 a) Illustration of an algorithm of converting from the decimal to other numerical systems and b) the basics of binary arithmetic.

three- and four-digit groups, respectively:

$$11001110001111.1_2 = 11\ 001\ 110\ 001\ 111.100_2 = 31617.4_8$$
$$11001110001111.1_2 = 11\ 0011\ 1000\ 1111.1000_2 = 338F.8_{16}$$

8.3.2 Handling Integer Numbers

As mentioned earlier, the binary system matches organization of computer's memory the best. Each bit can be either 1 or 0. A sequence of 8 bits, a *byte* (the alteration and blend of "bit" and "bite"), is processed as a single unit of information. In digital hardware, numbers are stored in binary words [a fixed-length sequence of binary digits (ones and zeros)]. One computer word is usually made of 32 bits or 4 bytes (alternatively, it may be as small as 1 byte and all way up to 8 and more bytes). The way in which hardware components or software functions interpret this sequence of ones and zeros) is described by a *data type*.

Consider a simple example when we want to use just 1 byte (8 bits) to store an integer number. We start filling this byte with ones from the rightmost bit b_0, called the *least significant bit* (LSB), all the way to the leftmost bit b_7, called the *most significant bit* (MSB), as shown in Fig. 8.4a. As seen, 8 bits may accommodate $2^8 = 256$ positive integer numbers from 0 to 255.

Formally, an N-bit binary word is decoded (to the decimal system) as

$$\text{integer} = \sum_{i=0}^{N} b_i 2^i \tag{8.1}$$

where b_i, $i = 0,2, \ldots, N$ assume a value 0 or 1.

Now, what if we want to represent negative integers too? In mathematics, negative numbers in any base are represented by prefixing them with a minus sign. However, in a computer, there is no single way of representing a number's sign. The straightforward approach would be simply allocating the MSB to be a *sign bit* designated to store, say, 0 for positive numbers and 1 for negative numbers (Fig. 8.4b). For example, decimal -47 encoded in an 8-bit word this way is 10101111_2 (the first "1" stands for a "$-$" sign). This approach, known as *sign-and-magnitude*, is directly comparable to the common way of showing a sign (placing a "$+$" or "$-$" next to the number's magnitude). However, it results in having two representations of zero, plus zero and minus zero, as shown in Fig. 8.5a.

A better way to introduce negative numbers would be having an agreement to subtract some specified quantity, bias, from whatever number we get from a 1-byte representation of Fig. 8.4a. To make it more or less symmetric with respect to zero when we are dealing with an N-bit word, we may use

$$bias_N = 2^{N-1} \tag{8.2}$$

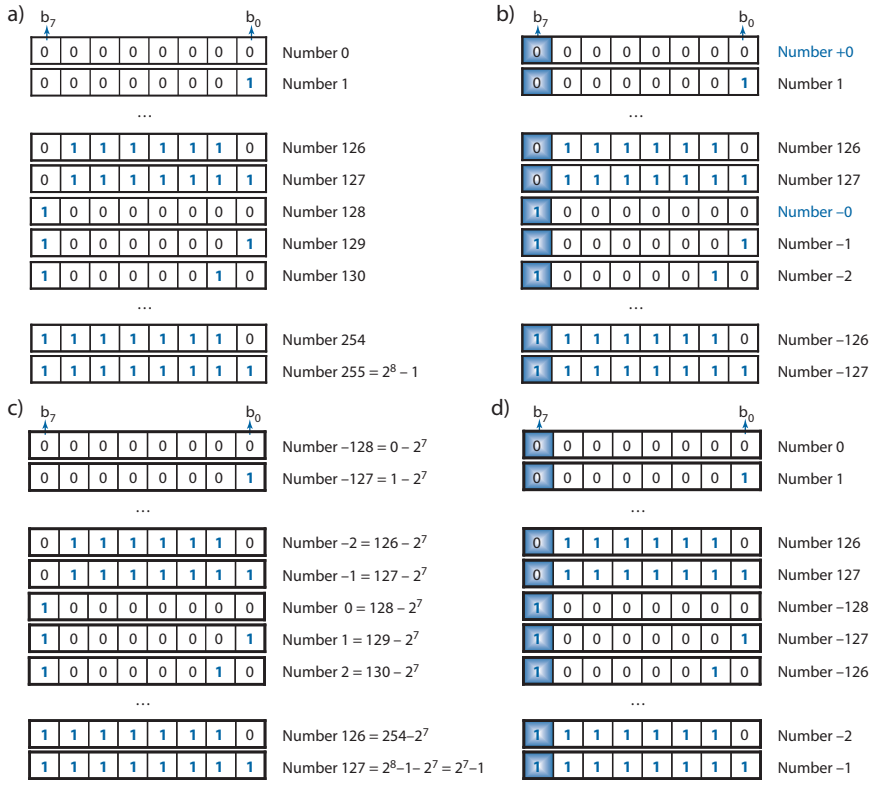


Fig. 8.4 Eight-bit word format for a) unsigned, b) sign-and-magnitude, c) excess-M, and d) two's complement integers.

For example, if $N = 8$ (1-byte word), the bias computed according to Eq. (8.2) becomes $2^7 = 128$. This bias results in having different numbers for exactly the same binary words (compare Fig. 8.4c with Fig. 8.4a). This method of representing signed numbers is known as the *excess-M* method, where M is a biasing value, $M = bias_N$ (Fig. 8.6). Hence, what is shown in Fig. 8.4c can be referred to as an excess-128 method. As seen from Fig. 8.6, after biasing, zero will be defined by a binary representation of the number M (10000000_2 for the excess-128 method applied to the 8-bit word) and $-M$

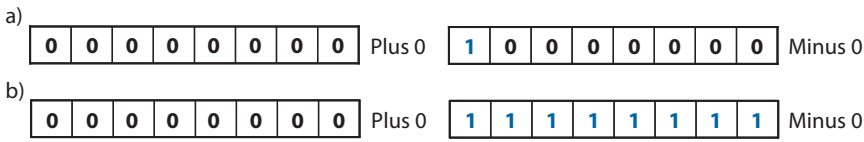


Fig. 8.5 Two-zero problem when using a) sign-and-magnitude and b) one's complement method to represent negative numbers.

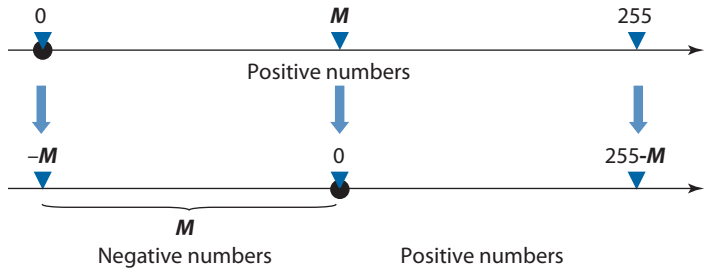


Fig. 8.6 Introducing a bias for representing negative numbers.

will be represented by the all-zeros bit pattern (Fig. 8.4c). The excess-*M* method is primarily used within the floating-point number format to be discussed further in Sec. 8.3.4.

Two more alternatives to represent negative numbers are the *one's* and *two's* complements. The one's complement form of a binary number is the bitwise `not` (`~`) applied to it (that is every *bit inversion*)—the complement of its positive counterpart. As an example, the one's complement form of 00101111_2 (47) becomes 11010000_2 (-47). However, similar to the sign-and-magnitude representation, the one's complement still suffers from having two representations for zero: 00000000_2 ($+0$) and 11111111_2 (-0) (Fig. 8.5b).

The problem of multiple representations of 0 is circumvented by a system called two's complement. In two's complement, negative numbers are represented by the bit pattern that is one greater (in an unsigned sense) than the one's complement of the positive value. Negating a number (converting positive to negative numbers or vice versa) is done by inverting all the bits and then adding 1 to that result. The MSB of a signed two's complement binary numeral still indicates a sign. If the leftmost bit is 0, the number is interpreted as a nonnegative binary number. If the MSB (leftmost) is 1, the remaining bits contain a negative number in the two's complement form.

For example, how do you find a two's complement of an 8-bit binary representation of a decimal value -5? The first step would be to invert all the bits of an 8-bit binary representation of 5, 00000101_2, yielding 11111010_2 (at this point, the numeral become the one's complement of the decimal value 5). The second step is to add 1 producing 11111011_2, a signed binary numeral representing the decimal value -5 in the two's complement form. The initial bit is 1, and so the numeral is interpreted as a negative value. This procedure works conversely too. For example, inverting the bits of -5 gives 00000100_2, and adding one gives the final value 00000101_2, that is, a positive decimal 5 (see Fig. 8.4d for the rest of the numbers in the 8-bit binary representation). Note that the two's complement of zero, 00000000_2, is zero itself: inverting gives all ones and adding one changes the ones back to zeros (the overflow is ignored). The two's complement of

Table 8.1 Functions for Converting Arrays to Integer Data Types

Function	Description
`int8(X)`, `int16(X)`, `int32(X)`, `int64(X)`	Converts the values in `X` to a signed integer data type that requires 8, 16, 32, and 64 bits per number, respectively
`uint8(X)`, `uint16(X)`, `uint32(X)`, `uint64(X)`	Converts the values in `X` to an unsigned integer data type that requires 8, 16, 32, and 64 bits per number, respectively

the most negative number, 10000000_2, sometimes called the *weird number*, also ends up as itself.

The decimal value of a two's complement representation of a negative binary number can be easily obtained in the usual manner by not accounting the MSB and adding -2^{N-1} to the number. For instance, the decimal value for a signed binary numeral in the two's complement form 11111011_2 can be obtained as $-2^7+2^6+2^5+2^4+2^3+2^1+2^0=-5$. In this sense, it is sort of similar to biasing binary numbers starting from 1 in the MSB, with the bias defined by Eq. (8.2), while disregarding this leftmost bit.

What is very important here is that when you apply any of the aforementioned methods to represent negative numbers you cannot discern whether the resulting binary numbers are signed or unsigned data types and what method was used merely by inspection. This information is not explicitly encoded within the word. Everything depends on the a priori agreement.

As shown in Fig. 4.1, MATLAB supports several classes of signed and unsigned integers, depending on how many bits are used to store a single number. The functions converting the input argument to one of these classes were introduced in conjunction with symbolic numbers in Table 7.6. Table 8.1 essentially repeats part of this table specifying how many bits are used in operations with these classes.

Tables 8.2 and 8.3 define the range of integers that can be represented using the aforementioned types of unsigned and signed integer numbers (to find the range limits, the corresponding commands, such as `intmin('int8')` or `intmax('int64')`, can be used).

Now, we are ready to introduce an error that can occur using these types of data. For instance, the command

```
>> y=int8(300)
```

Table 8.2 Smallest and Largest Unsigned Integer Numbers

	uint8	uint16	uint32	uint64
Largest integer (`intmax`)	255 (2^8-1)	65535 ($2^{16}-1$)	4294967295 ($2^{32}-1$)	18446744073709551615 ($2^{64}-1$)
Smallest integer (`intmin`)	0	0	0	0

Table 8.3 Smallest and Largest Signed Integer Numbers

	int8	int16	int32	int64
Largest integer (intmax)	127 (2^7-1)	32767 ($2^{15}-1$)	2147483647 ($2^{31}-1$)	9223372036854775807 ($2^{63}-1$)
Smallest integer (intmin)	-128 (-2^7)	-32768 (-2^{15})	-2147483648 (-2^{31})	-9223372036854775808 (-2^{63})

returns an erroneous response

```
y = 127
```

because 300 exceeds the maximum value that can be represented by the `int8` data type. Although this example is obvious, the following example might be easily overlooked even by the experienced programmer:

```
>> 3*int8(100)
y = 127
```

Although `int8(100)` returns the correct result, it also defines the output data type. Hence, we run into the same problem as in the previous example.

Two MATLAB functions, `cast` and `typecast`, allow converting variables to a different data type (class). The simplest syntax for these functions is `cast(A,newclass)` and `typecast(A,newclass)`, where `A` is a variable to convert and `newclass` is the name of a built-in data type compatible with `A` (including `int8`, . . . , `int64` and `uint8`, . . . , `uint64`). Whereas the `cast` function truncates any values in `A` that are too large to map into `newclass`, the `typecast` function converts data types without changing underlying data. For example,

```
>> cast(uint8(130),'int8')
```

returns

```
y = 127
```

(which is the largest possible number in the `int8` class) and

```
>> typecast(uint8(130),'int8')
```

returns

```
y = -126
```

(which has the same sequence of ones and zeros in `int8` class (Fig. 8.4d) as the number 130 has in the `uint8` class (Fig. 8.4a).

Now that you know how integer numbers are stored in a computer and what integer data types are available in MATLAB, let us explore what difference it makes to representing noninteger, that is, real numbers.

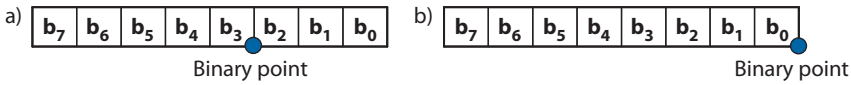


Fig. 8.7 a) Introducing a binary point and b) expanding it to the case of integer numbers.

8.3.3 Fixed-Point Format

One way to represent noninteger numbers is to follow the same routine we just considered, but this time to additionally introduce a virtual binary point in the binary word, as shown in Fig. 8.7a. As a matter of fact, we can assume that, while considering the integer numbers, this point was already present and located to the right of the LSB (Fig. 8.7b).

This type of representation is called a *fixed-point* data type, and the binary point is the means by which the fixed-point values are scaled. Depending on where we think this binary point is located, we may have different numbers represented by the same sequence of ones and zeros (Fig. 8.8).

Essentially, we can cast the placement of the binary point as a scaling problem. In the example of Fig. 8.8, moving the binary point two places to the right means multiplying by 2^2. Compared with Fig. 8.7b, moving the binary point w places to the left means multiplying whatever integer number we had by 2^{-w}. Therefore, the main difference between different fixed-point data types is the location of their binary point. Combining Eqs. (8.1) and (8.2), the fixed-point real numbers employing the excess-M method can be decoded as

$$\text{number} = 2^{-w}(\text{integer} - bias_N) = 2^{-w}\sum_{i=0}^{N-1} b_i 2^i - 2^{-w}2^{N-1} \tag{8.3}$$

that is, they are defined by the word length in bits N, the fraction length w (slope 2^{-w}), and whether the biasing is on (defining whether they are signed or unsigned). Again, whether a fixed-point value is signed or unsigned is usually not encoded within the binary word explicitly (that is, there is no sign bit). Instead, the sign information is implicitly defined within computer architecture.

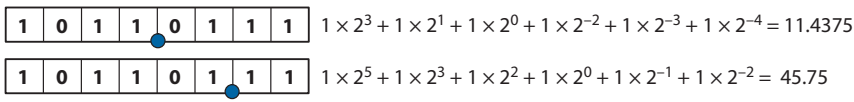


Fig. 8.8 Examples of representing real numbers using a fixed-point data type.

The limits of representation [Eq. (8.3)] (its range) with the bias defined by Eq. (8.2) are

$$\text{range} = 2^{-w}[-2^{N-1}; 2^{N-1} - 1] \tag{8.4}$$

Obviously, by letting $w = 0$, Eqs. (8.3) and (8.4) naturally extend to include the excess-$bias_N$ representation of integer numbers.

Along with the error associated with the limited range [Eq. (8.4)] we may observe the error associated with the data type's *precision*, which is characterized by the distance between successive numbers in the representation. Consider the same example, where we use just 1 byte to represent the real numbers. Regardless of the scaling and biasing, this 1 byte can still accommodate only 256 different numbers. And the distance between any two successive numbers is

$$\text{precision} = 2^{-w} \tag{8.5}$$

meaning that we cannot possibly have a more precise solution while operating with the fixed-point data type numbers than the one prescribed by Eq. (8.5). For instance, if $w = 0$, the distance between two successive numbers is 1 (i.e., there is no error for integer numbers), for $w = 1$ it is 0.5, for $w = 2$ it is 0.25, etc. If $w = N$, then we would have the best precision (2^{-8} in our case), and . . . the worst (the smallest) range. The range–precision tradeoff, by the way, is always the case (Fig. 8.9).

It should also be noted that the relative precision for the fixed-point data type is independent of the fraction length w and degrades with the decreasing number magnitude:

$$\frac{\text{precision}}{\text{number}} = \left| \sum_{i=0}^{N-1} b_i 2^i - bias_N \right|^{-1} \tag{8.6}$$

Although the Simulink fixed-point blockset (specifically designed to handle fixed-point numbers and fixed-point arithmetic) does support the

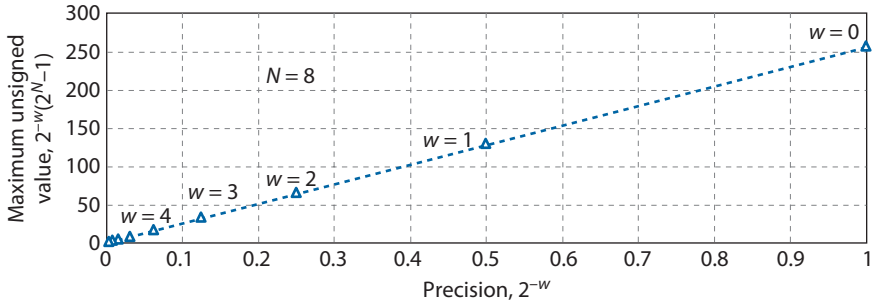


Fig. 8.9 Maximum unsigned value vs precision for the 8-bit fixed-point numbers.

fixed-point format with the word sizes of up to 128 bits, MATLAB itself uses another form of representing the real numbers, as discussed next.

8.3.4 Floating-Point Format

An alternative way to represent real numbers is to use the *floating-point* data types. In general, a floating-point number x in the base-b system can be represented by two numbers m and e, such that

$$x = mb^e \tag{8.7}$$

where e is called the *exponent* and m is the *significand* or, informally, *mantissa*. The significand can be thought of as a p-digit integer, fraction, or some other fixed-point form by choosing an appropriate exponent offset. For example, it can be represented as $\pm d.ddd\ldots ddd$, where each digit d is an integer between 0 and $b - 1$ inclusive. If the leading digit of m is nonzero, then the number is said to be normal or *normalized*.

Specifically, a general representation of a normalized nonzero base-2 floating-point number

$$x = (-1)^s(1+f)2^e \tag{8.8}$$

includes the *fraction* f, which satisfies an inequality $0 \le f < 1$, and a separate sign bit s to represent positive and negative numbers ($s = 0$ and $s = 1$, respectively). Thus, a normalized binary number has a of the form $1.f$, where f has a fixed size for a given data type. Because the leftmost significand bit is always a 1, it is unnecessary to store this bit, and therefore it is hidden (implicit). Thus, an N-bit fraction stores an $(N+1)$-bit number.

Following this setup for the 8-bit word, we need to allocate the MSB to store the sign s, then devote some number of bits to represent exponent e, and the rest of them will characterize the fraction f as shown, for instance, in Fig. 8.10.

To decode the number shown in Fig. 8.10, we may use the following formula:

$$x = (-1)^{MSB}\left(1 + \sum_{i=1}^{F} \frac{b_{F-i}}{2^i}\right) 2^{\sum_{i=0}^{N-F-2} b_{F+i}2^i - bias^{**}} \tag{8.9}$$

where F is the number of bits representing the fraction, and the $bias^{**}$ is introduced into the exponent to be able to represent both negative and positive numbers. Therefore, one of the advantages of the floating-point data type is that it allows a large range of magnitudes to be represented within a given number of bits, which is not possible with fixed-point notation. To have a more or less symmetric

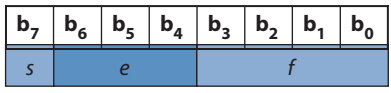


Fig. 8.10 Introducing a floating-point format.

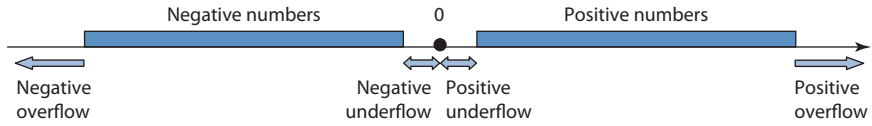


Fig. 8.11 Range of floating-point numbers.

range of the exponent, the bias can be computed using Eq. (8.2) as $bias^{**} = bias_{N-F-1} = 2^{N-F-2}$ (to exceed the number of negative exponents by one) or

$$bias^{**} = 2^{N-F-2} - 1 \tag{8.10}$$

(to have an opposite). With Eq. (8.10), the range of the exponent is $[-2^{N-F-2} + 1; 2^{N-F-2}]$. In practice, however, some values of the exponent are reserved for flagging `Inf` (infinity), `NaN` (not-a-number), and denormalized numbers (see further), and so the true range of values is as follows:

$$[e_{\min}; e_{\max}] = [-2^{N-F-2} + 2; 2^{N-F-2} - 1] \tag{8.11}$$

The absolute value of the minimum number accounting for the exponent's range [Eq. (8.11)] is given by

$$V_{\min} = 2^{-2^{N-F-2}+2} \tag{8.12}$$

and the absolute value of the maximum number is given by

$$V_{\max} = (2 - 2^{-F})2^{2^{N-F-2}-1} \tag{8.13}$$

Figure 8.11 provides a graphical illustration of the range of representable numbers using Eq. (8.9).

As seen from Fig. 8.11, there is always a gap around zero in floating-point arithmetic. To fill this gap and therefore provide a gradual underflow allowing the calculation to lose precision slowly, rather than all at once, *denormal* or *subnormal numbers* are used. These numbers, denormalizable to the form of Eq. (8.8), use significand of the form 0. f rather than 1. f. Denormal numbers are encoded with a biased exponent $e = 0$, but are interpreted with the value of the smallest allowed exponent. Zero has a special bit pattern, where $e = 0$ and $f = 0$ (due to the sign-and-magnitude representation of floating-point numbers, there are two representations of zero, one positive and one negative).

To summarize, positive numbers greater than $V_{\max}$ and negative numbers less than $-V_{\max}$ are overflows, so that they are mapped to `+Inf` and `-Inf`, respectively. Positive numbers less than $V_{\min}$ and negative numbers greater than $-V_{\max}$ are either underflows or denormalized numbers, including zero.

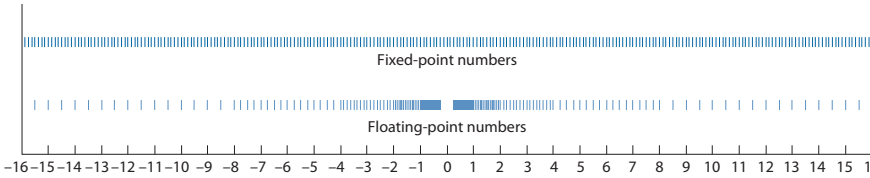


Fig. 8.12 Example of signed fixed- and floating-point numbers stored in the 8-bit word.

Figure 8.12 provides an example of the signed real numbers that can be stored in 8 bits ($N = 8$) using the fixed-point and floating-point formats. For the fixed-point format, given by Eq. (8.3), we assume the binary point to reside between the third and fourth bit (counting from the left), so that $w = 3$ (Fig. 8.7a). To have about the same range for the floating-point format, given by Eq. (8.9) with the bias of Eq. (8.10), we let $F = 4$ (Fig. 8.10).

According to Eq. (8.12), the underflow around zero, which is clearly seen in Fig. 8.12, ranges from -0.25 to 0.25; the range of the floating-point representation, determined by Eq. (8.13), in this case is from -15.5 to 15.5. Note that compared with that of the fixed-point numbers [Eq. (8.4)], the range for the floating-point numbers is fully symmetric. Also note that because we reserved two exponents to keep `Inf`, `NaN`, and denormalized numbers, the 8 bits for the floating-point numbers in Fig. 8.12 now represent only 192 rather than 256 different numbers (2 signs $\times$ 6 different exponents $\times$ 16 different fractions).

As seen from Fig. 8.12, the floating-point numbers exhibit one more very important feature. As opposed to the fixed numbers, they are not distributed evenly, that is, precision of the floating-point numbers varies and goes from as small as

$$\text{precision}_{\min} = 2^{e_{\min} - F} \tag{8.14}$$

for small numbers, to as large as

$$\text{precision}_{\max} = 2^{e_{\max} - F} \tag{8.15}$$

for large numbers. For $N = 8$ and $F = 4$ (Fig. 8.10), this corresponds to 0.0156 and 0.5, respectively.

To have a single unified merit, the distance from 1.0 to the next largest floating-point number, referred to as a *round-off constant,* is used. It corresponds to exponent equal to zero and therefore is defined as

$$\varepsilon = 2^{-F} \tag{8.16}$$

For the aforementioned example of $F = 4$, it yields $\varepsilon = 0.0625$. This constant also happens to be the upper bound on the relative precision. As opposed to that of the fixed-point numbers [Eq. (8.6)], the relative precision of floating-

point numbers does not depend on the magnitude of a real number and stays almost the same within all range of numbers. Increasing the exponent field (decreasing $e_{\min}$ and increasing $e_{\max}$) improves precision for the small numbers [Eq. (8.14)] but simultaneously leads to worsening precision for the large numbers [Eq. (8.15)]. Increasing the fraction field leads to better precision overall [Eqs. (8.14)–(8.16)].

In the previous discussion, we made several assumptions on what part of the N-bit word should be dedicated to keep the fraction f and how to introduce the biased exponent e. By choosing different values of N and F, we obviously obtain different precisions and ranges of the representable floating-point numbers. To be able to operate with these numbers, we have to know their format up front; otherwise, if some other computer uses a different representation, your code will produce erroneous results (if any).

Back in the 1960s to 1980s, the situation was even more complicated. Not only did each computer have its own floating-point number system, but some of them were binary and some were decimal. Among the binary computers, some used 2 as the base; others used 8 or 16. Each of them had different precision. In 1985, the American National Standards Institute (ANSI) and the Institute of Electrical and Electronics Engineers (IEEE) Standards Board adopted the ANSI/IEEE Standard 754-1985 for binary floating-point arithmetic. This was the culmination of almost a decade of work by almost a 100-person working group of mathematicians, computer scientists, and engineers from universities, computer manufacturers, and microprocessor companies.

Since 1985, all computers designed use ANSI/IEEE floating-point arithmetic. This does not mean that they all provide exactly the same results because there is still some flexibility within the standard. However, it does mean that we can rely on a *machine-independent* model of how floating-point arithmetic behaves. Among other things, the ANSI/IEEE standard (renewed as the IEEE 754-2019) specifies four floating-point number formats of which single-precision format (using a 32-bit word) and double-precision format (using a 64-bit word) are the most widely used.

8.3.5 Standard Single- and Double-Precision Formats

As discussed earlier to enable representing both small and large numbers, the exponent in the floating-point representations is biased. The ANSI/IEEE floating-point standard defined the exponent field of a single-precision number as an 8-bit excess-127 field [according to Eq. (8.10), $bias^{**} = 2^{8-1} - 1 = 127$]. The double-precision exponent field uses an 11-bit excess-1023 field ($bias^{**} = 2^{11-1} - 1 = 1023$).

Therefore, the *single-precision floating-point format* is a 32-bit word divided into the 1-bit sign indicator s, 8-bit biased exponent e, and 23-bit fraction f, as shown in Fig. 8.13. Formally, the relationship between this

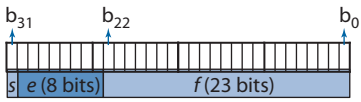


Fig. 8.13 Single-precision (32-bit word) format.

format and its representation of real numbers is given by

$$x = (-1)^s \left(1 + \sum_{i=1}^{23} \frac{b_i}{2^i}\right) 2^{e-127}, \quad 0 < e < 255 \tag{8.17}$$

The *double-precision floating-point format* is based on a 64-bit word divided into the 1-bit sign indicator s, 11-bit biased exponent e, and 52-bit fraction f, as shown in Fig. 8.14. The formal relationship between this format and its representation of real numbers is given by

$$x = (-1)^s \left(1 + \sum_{i=1}^{52} \frac{b_i}{2^i}\right) 2^{e-1023}, \quad 0 < e < 2047 \tag{8.18}$$

Because some values of the exponent are reserved for flagging different numbers, as discussed in the previous section, the true exponent values for single precision range from −126 to 127 (a pure excess-127 would provide −127 to 128 in range) and for double precision range from −1022 to 1023 (a pure excess-1023 would provide −1023 to 1024 in range) [compare Eq. (8.11) vs Eq. (8.10)].

MATLAB has traditionally used the IEEE double-precision format (which is the default format for all mathematical operations). As shown in Fig. 4.1, MATLAB also supports single-precision arithmetic. Although a single-precision format saves space, it is not necessarily much faster. You may convert any array `X` to the single-precision format and back to the double-precision format by using the functions of Table 8.4 (which have already been introduced in conjunction with the symbolic numbers in Table 7.6).

Based on Eqs. (8.17) and (8.18) with account of a reduced exponent range, the round-off constant (relative precision) for singles and doubles, as well as the min/max numbers they can represent, are estimated as presented in

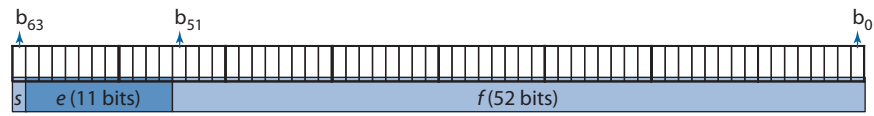


Fig. 8.14 Double-precision (64-bit word) format.

Table 8.4 Functions for Converting Arrays to Single and Double Floating-Point Values

Function	Description
`single(X)`	Converts the values in `X` to the single-precision format (defined as the `single` class)
`double(X)`	Converts the values in `X` to the double-precision format (defined as the `double` class)

Table 8.5 Floating-Point Relative Precision, the Smallest and Largest Floating-Point Numbers

	Single-Precision Format	Double-Precision Format
Round-off constant	`eps('single')` $= 2^{-23}$	`eps` $= 2^{-52}$
Underflow	`realmin('single')` $= 2^{-126}$	`realmin` $= 2^{-1022}$
Overflow	`realmax('single')` $= (2 - \text{eps}('\text{single}'))\, 2^{127}$	`realmax` $= (2 - \text{eps})\, 2^{1023}$

Table 8.6 Decimal Representations of the Values of Table 8.5

	Single-Precision Format	Double-Precision Format
Round-off constant	`1.1921e-007` $= 1.1921 \times 10^{-7}$	`2.2204e-016` $= 2.2204 \times 10^{-16}$
Underflow	`1.1755e-038` $= 1.1755 \times 10^{-38}$	`2.2251e-308` $= 2.2251 \times 10^{-308}$
Overflow	`3.4028e+038` $= 3.4028 \times 10^{+38}$	`1.7977e+308` $= 1.7977 \times 10^{+308}$

Tables 8.5 and 8.6. Table 8.5 employs the `eps` function that returns the distance from 1.0 to the next representable number [Eq. (8.16)], `realmin` – returning the smallest positive normalized floating-point number on your computer [Eq. (8.12)] (anything smaller underflows or is denormal), and `realmax` – returning the largest floating-point number representable on your computer [Eq. (8.13)] (anything larger overflows).

8.4 Round-Off Errors

As we have learned already, due to the fact that digital computers have range and precision limits on their ability to represent numbers, they cannot represent some quantities exactly, causing *round-off errors*. These errors are important in engineering and scientific problem solving because they can make calculations unstable or lead to erroneous results. The latter can be either obvious or, if they lead to subtler discrepancies, quite difficult to detect.

It turns out that the range and precision limits are not the only source for round-off errors involved in numerical calculations. Certain numerical manipulations can aggravate precision due to round-off errors even further. This can result from both mathematical considerations as well as from the way in which computers perform arithmetic operations. Before we proceed with considering these cases, let us review the fixed-point and floating-point formats from the standpoint of round-off errors.

8.4.1 Fixed-Point vs Floating-Point Numbers

As long as a numeric value uses only the number of digits specified after the radix (in this case - binary) point, the fixed-point values can represent all values up to its maximum value (determined by the number of bits in its representation) exactly. This is in contrast to floating-point representations,

which include an automatically managed exponent but, given the same number of bits in its representation, cannot represent as many digits accurately.

That is why a common use for fixed-point numbers is for storing monetary values, where the inexact values of floating-point numbers are often a liability. Fixed-point representations are also sometimes used if either the executing processor does not have any floating-point unit (FPU) or if fixed-point arithmetic provides an improved performance necessary for an application. For instance, some audio codecs use fixed-point arithmetic because, to save money, many audio decoding hardware devices do not have an FPU, and audio decoding requires enough performance that a software implementation of floating-point on low-speed devices would not produce a real-time output.

Very few computer languages, however, include built-in support for fixed-point values because, for most applications, floating-point representations are fast enough and accurate enough. Floating-point representations are more flexible than fixed-point representations because they can handle a wider dynamic range. Floating-point representations are also slightly easier to use because they do not require programmers to specify the number of digits after the radix point.

Historically, fixed-point representations were the norm for decimal data types (for example, in PL/1 or Cobol). The Ada programming language includes built-in support for both fixed-point and floating-point. However, if they are needed, fixed-point numbers can be implemented even in programming languages like C and C++ that do not include such support built in. MATLAB also relies on floating-point arithmetic, although it supports fixed-point numbers (in Simulink).

Floating-point numbers usually behave very similarly to the real numbers they are used to approximate. However, this can easily lead programmers into overconfidently ignoring the need for numerical analysis. There are many cases where floating-point numbers do not model real numbers well. For example, the decimal fraction 0.1 or 0.01 cannot be exactly represented in any binary floating-point format (to be discussed later).

In general, when fixed precision is required, fixed-point arithmetic is usually a better choice. However, a floating-point representation is more likely to be appropriate when relative (proportional) precision over a range of scales is needed. The common errors in floating-point computation are to be discussed next.

8.4.2 Range and Precision

Formally, the errors associated with range and precision are

1. Overflow (which for the floating-point numbers yields an infinity)
2. Underflow (often defined as an inexact tiny result outside the range of the normal numbers for a format), which yields zero, a subnormal number, or the smallest normal number
3. Rounding and nonrepresentable numbers

As follows from Secs. 8.3.3–8.3.5, all types of data in MATLAB have their own limits. For integer and floating-point numbers, these limits are given in Tables 8.2, 8.3, 8.5, and 8.6. If you try to assign a number that exceeds these limits, you will get an erroneous (undetermined) result. For integers, it will be the boundary number for a specific range, and for floating-point numbers, it is infinity.

Another source of errors for floating-point numbers is their limited precision. To start with, some numbers cannot be represented exactly. For example, irrational numbers such as π, e, or $\sqrt{5}$ cannot be expressed by a finite number of significand figures. Therefore, they cannot be represented exactly by a computer. In general, for computer tools that use a 32-bit word (single precision), the significand can be expressed to about eight base-10 digits of precision. Thus, π is represented as

```
>> format long, single(pi)
ans = 3.1415927
```

Performing arithmetic operations, for example, 1/3 or 6/7, will also be affected by floating-point numbers precision. For example, for 1/3, the single-precision format produces 0.6666667 instead of 0.(6) and, for 6/7, yields 0.8571429 instead of 0.(857142).

For tools using a 64-bit word, precision increases to about 15 base-10 digits. For example, the number π is now represented as

```
>> format long, double(pi)
ans = 3.14159265358979
```

In addition, because computers use a binary, or base-2, representation, they cannot possibly precisely represent certain exact base-10 numbers. For example, the exact base-10 quantity 0.1 cannot be represented exactly in a base-2 system. It may be hard to see with MATLAB's default 64-bit floating-point format, but if we intentionally degrade precision down to the `single` format, you will be able to see that there is a difference between the two representations, indicating that 1/10 is not represented precisely:

```
>> 0.1-single(0.1)
ans = -1.4901161e-009
```

(in the case that the precise representation exists, the return should be zero).

If you think of 0.1 as being some kind of an exceptional number, run the following script:

```
i=[0:0.1:1];
er=i-single(i);
stem(i,er), xlabel('Decimal number'), ylabel('Error')
```

The result it produces is shown in Fig. 8.15.

As seen, among those 11 fractions, only three—0, 0.5=2^{-1}, and $1 = 2^0$—have exact representations. The remaining ones do not. MATLAB's 64-bit

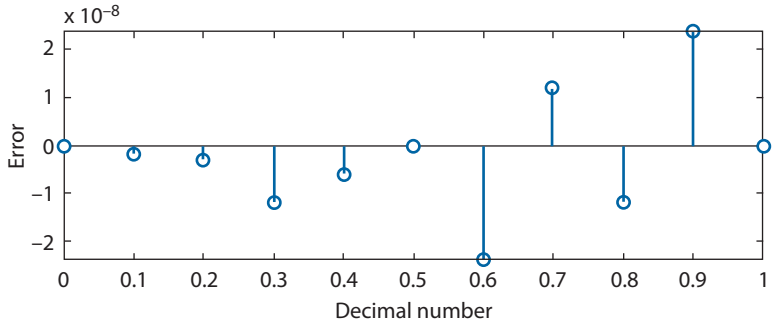


Fig. 8.15 Error in representing the floating-point numbers.

word format does not make these errors disappear, but simply makes them much smaller.

Certain numerical methods require extremely large numbers of arithmetic manipulations to arrive at their final results. In addition, these computations are often interdependent. That is, the later calculations are dependent on the results of the earlier ones. Consequently, even though an individual round-off error could be small, the cumulative effect over the course of a large volume of computations can be significant.

Another implication is that floating-point arithmetic is neither *associative* nor *distributive*. This means that in general for floating-point numbers x, y, and z

$$(x+y)+z \neq x+(y+z), \quad (xy)z \neq x(yz), \quad x(y+z) \neq xy+xz \tag{8.19}$$

For instance, while

```
>> (1e100 - 1e100) + 1000
```

produces

```
ans = 1000
```

a slightly different command

```
>> 1e100 - (1e100 - 1000)
```

yields an erroneous result

```
ans = 0
```

That is because the smaller number in the second example was "absorbed" by the much larger number. Specifically, the command

```
> eps(1e100)
```

shows that the very next representable number is greater than `1e100` by

```
ans = 1.9427e+084
```

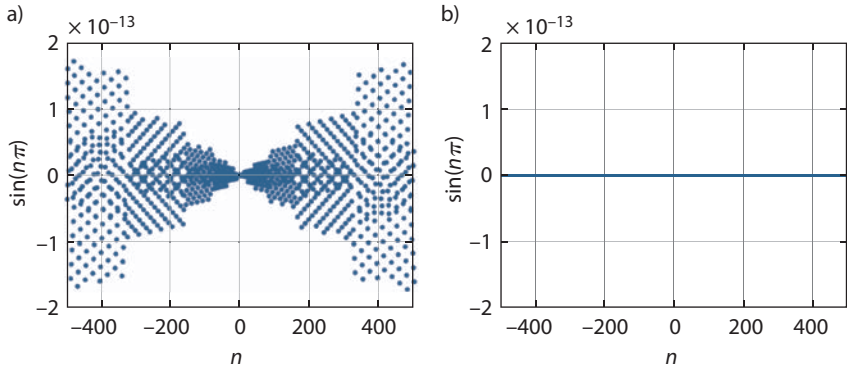


Fig. 8.16 Error in computing $\sin(n\pi)$ using the a) `sin(n*pi)` and b) `sinpi(n)` calls.

As was demonstrated in Fig. 8.12, precision for large numbers is worse than that for small numbers, and so it is no wonder that `1e3` was absorbed. Even if you used, say, `1e83` instead of `1e3`, it would be absorbed anyway.

To conclude this section, let us show one more example. Usually, computation of trigonometric functions is based on a power series, involving quite a few mathematical operations. Even though the number of terms in the power series to achieve the desired accuracy is set to match precision of the 64-bit word format, it is easy to show the effect of accumulation of the round-off error (absorption). For example, the call

```
>> n = -500:500; plot(n,sin(n*pi),'.'), xlabel('n'), ylabel('sin(n\pi)')
```

results in a growing error in computing the values of $\sin(n\pi)$ while increasing a magnitude of n from 0 to 500 as shown in Fig. 8.16a. You will receive a similar same plot for any other angle and any other trigonometric function.

However, the `sinpi` and `cospi` functions (see the footnote to Table 2.4), introduced in R2018b, allow computing `cos(n*pi)` without explicitly computing `n*pi`, so that the results contain only small numerical errors due to the fact that `pi` is a floating-point approximation of the true value of π. The call

```
>> n = -500:500; plot(n,sinpi(n),'.'), xlabel('n'), ylabel('sin(n\pi)')
```

results in Fig. 8.16b.

8.5 Truncation Errors

The results of numerical computations can also be affected by another type of errors caused by using approximations in place of exact mathematical expressions. This type of errors is known as *truncation errors*. The following formally introduces several definitions associated with truncation errors and proceeds with a simple example.

8.5.1 Error Definitions

Let us start from the basic relation establishing the relationship between the exact, or true, expression ζ and its approximation ζ^*

$$\zeta = \zeta^* + \delta_\zeta^t \tag{8.20}$$

where δ_ζ^t is used to designate the exact value of error (the superscript t is included to designate that this is the "true" error). By rearranging terms in Eq. (8.20), we find the *true error* δ_ζ^t as

$$\delta_\zeta^t = \zeta - \zeta^* \tag{8.21}$$

A shortcoming of this definition is that it takes no account of the order of magnitude of the value under examination. For example, an error of a centimeter is much more significant if we are measuring a wing thickness as opposed to its span. One way to account for magnitudes of quantities being evaluated is to normalize the error to the true value

$$\overline{\delta}_\zeta^t = \frac{\delta_\zeta^t}{\zeta} \tag{8.22}$$

The sign of expressions in Eqs. (8.21) and (8.22) can be either positive or negative. However, when performing computations, we may be less concerned with the sign of error compared to its magnitude. Thus, in practice we usually deal with the nonnegative *absolute error*, $|\delta_\zeta^t|$, and *relative error*

$$\varepsilon^t = \left|\overline{\delta}_\zeta^t\right| \tag{8.23}$$

Note that the relative error is a unitless number expressed as a fraction. However, it can also be expressed as a *percent error*

$$\varepsilon^t[\%] = \varepsilon^t \times 100\% \tag{8.24}$$

For example, a relative error of 10^{-2} corresponds to a 1% percent error.

Equations (8.20)–(8.23) imply that we know the true value ζ. However, such information is rarely available. Hence, we need to use the estimates of these parameters, that is, the relative error of Eq. (8.23) becomes

$$\varepsilon^a = \left|\frac{\hat{\zeta} - \zeta^*}{\hat{\zeta}}\right| \quad (\varepsilon^a[\%] = \varepsilon^a \times 100\%) \tag{8.25}$$

Here the superscript a signifies that the error is normalized to an approximate value of ζ.

Computing ε^a in the absence of knowledge regarding the true value of ζ constitutes one of the challenges of numerical methods. Different numerical methods use the different approaches to address it. For instance, certain numerical methods rely on *iterations*. In this case, a previous value of ζ, ζ_{i-1} is used to produce the next iteration ζ_i ($i = 1,2,\ldots$). This process is performed repeatedly, or *iteratively*, to successively compute (hopefully) better

and better approximations. For such cases, the error is often estimated based on the difference between previous and present approximations, so that Eq. (8.25) becomes

$$\varepsilon_i^a = \left| \frac{\zeta_i - \zeta_{i-1}}{\zeta_i} \right| \tag{8.26}$$

This error is estimated at each iteration and is compared with some prespecified *tolerance* ε^s. If the iterative procedure converges, ε_i^a becomes smaller and smaller and at some point satisfies an inequality

$$\varepsilon_i^a \leq \varepsilon^s \tag{8.27}$$

This relationship is referred to as a *stopping criterion*. If it is satisfied, the result is assumed to be within the prespecified acceptable level ε^s.

In practice, the true value ζ can be a very small number or zero. To avoid singularity in Eq. (8.26), the stopping criterion assumes a more robust form

$$\frac{|\zeta_i - \zeta_{i-1}|}{\max\left(|\zeta_i|, \dfrac{\varepsilon^{abs}}{\varepsilon^{rel}}\right)} \leq \varepsilon^{rel} \tag{8.28}$$

where ε^{abs} and ε^{rel} are absolute and relative tolerances. In MATLAB, the typical default value of ε^{abs} is 10^{-6}, and ε^{rel} - of the order of 10^{-3}, which corresponds to 0.1% accuracy. When $|\zeta_i| \to 0$, Eq. (8.28) reduces to

$$|\zeta_i - \zeta_{i-1}| \leq \varepsilon^{abs} \tag{8.29}$$

Because the ratio of tolerances is small (of the order of 10^{-3}), some algorithms use the sum of the two terms in the denominator of Eq. (8.28) instead of finding a maximum value.

It is also convenient to relate the aforementioned errors to the number of correct significand figures in the approximation. It can be shown that the result, which is correct to at least N significand figures ($N = 1,2,\ldots$), ensures the upper bound on the percent error

$$\varepsilon^{ub} = \frac{10^{2-N}}{2}\% \tag{8.30}$$

This equation can be resolved to yield

$$N = \left[2 - \log_{10}(2\varepsilon)\right] \tag{8.31}$$

Here [] denotes the integer part and ε's superscript is dropped. This dependence is presented graphically in Fig. 8.17. Given the number N, we may find the upper bound on the percent error, and vice versa; given the estimate of the percent error, this dependence yields the number of correct significand figures.

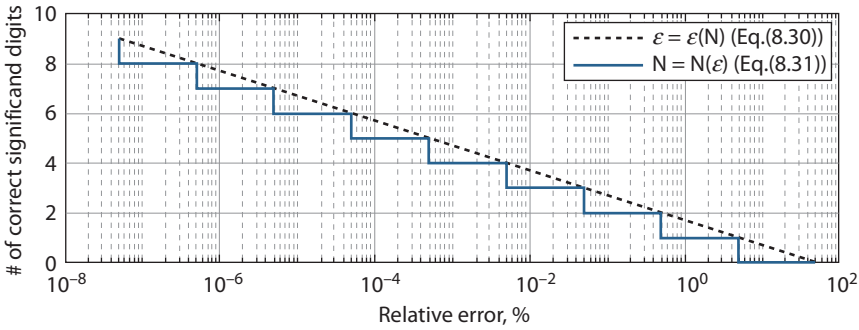


Fig. 8.17 Number of the correct significand figures vs percent error.

Consider a simple example. As is well known, any function $f(x)$ can be represented as a *Taylor series expansion*, an infinite sum of terms calculated using the values of function's derivatives estimated at some nominal point $x = a$:

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \cdots$$
$$= \sum_{i=0}^{\infty} \frac{f^{(i)}(a)}{i!}(x-a)^i \tag{8.32}$$

which for $a = 0$ is also known as a *Maclaurin series expansion:*

$$f(x) = f(0) + f'(a)x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 + \cdots = \sum_{i=0}^{\infty} \frac{f^{(i)}(0)}{i!}x^i \tag{8.33}$$

Let us use Eq. (8.33) for the exponential function e^x

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = \sum_{i=0}^{\infty} \frac{x^i}{i!} \tag{8.34}$$

to approximate its value at $x = 1$. The (Euler's) number e (do not confuse it with the e notation used in Secs. 8.3.4, 8.3.5) is an irregular number, and so the more terms we use in Eq. (8.34), the better accuracy we get. In this example, we will consider, say, a 16-digit representation

$$e = 2.718281828459046\ldots \tag{8.35}$$

as a true value. (In MATLAB you may use either

```
>> format long, disp(exp(1))
```

or

```
>> disp(vpa(exp(1),16))
```

to obtain this value). Now, the question is how the number of elements in the series [Eq. (8.34)] affects approximation's accuracy.

According to Eq. (8.26), the estimate of a percent error for approximation of Eq. (8.34) can be computed as

$$\varepsilon_n^a = \frac{\dfrac{x^n}{n!}}{\displaystyle\sum_{i=0}^{n} \frac{x^i}{i!}} 100\% \tag{8.36}$$

In this particular case, because the true value is known [Eq. (8.35)], we can also compute a true value of the percent error as

$$\varepsilon_n^t = \frac{\left| \displaystyle\sum_{i=0}^{n} \frac{x^i}{i!} - 2.718281828459046 \right|}{2.718281828459046} 100\% \tag{8.37}$$

Finally, we can employ Eq. (8.31) to compute the number of significand figures based on approximation's accuracy. The results produced by Eqs. (8.34), (8.37), (8.36), and (8.31) (with ε equal to ε_n^t and ε_n^a, respectively), are presented in Table 8.7. They are also illustrated in Fig. 8.18. Obviously, compared with Eq. (8.37), Eq. (8.36) lags one step, but that is the best we can possibly do. Hence, approximation of the relative error given by Eq. (8.36) is always conservative.

The last two columns in Table 8.7 show the actual number of correct significand figures (for your convenience, the correct digits in approximations appear in boldface) and the upper bound on the error. For example, for the five terms the actual number of correct significand figures happens to be 2, which corresponds to the upper bound of 0.5%. The true error ε^t for this line is less than 0.5%, and the predicted $N(\varepsilon^t)$ equals 2. The estimate ε^a exceeds 0.5% tolerance, and therefore $N(\varepsilon^a)$ is less than 2. Hence, Eq. (8.26) provides quite a conservative estimate. The true error is always way less than that. Consequently, the estimate of the number of correct significand figures [Eq. (8.31)] based on ε^a is conservative too. Moreover, even the estimate of N based on the true relative error ε^t provides a worse result than in reality. That is, both Eqs. (8.26) and (8.31) ensure that the result is at least as good as they specify.

8.5.2 Application of Error Formulas to Fundamental Arithmetic Operations

Let us address one more issue here. Suppose we have some arbitrary function of several variables written in the most general form as

$$F = f(u_1, u_2, \ldots, u_m) \tag{8.38}$$

Table 8.7 Accuracy of Taylor Series Expansion to Estimate the Euler's Number

Number of Terms	Approximation	ε^t(%)	ε^a(%)	$N(\varepsilon^t)$	$N(\varepsilon^a)$	Actual N	ε^{ub}(%)
1	1	63.21		0		0	50
2	**2**	26.424	50	0	0	1	5
3	**2**.5	8.0301	20	0	0	1	5
4	**2**.67	1.89885	6.25	1	0	1	5
5	**2.7**08	0.36599	1.5385	2	1	2	0.5
6	**2.71**67	0.059418	0.30675	2	2	3	0.05
7	**2.718**06	0.008324	0.051099	3	2	4	0.005
8	**2.71825**4	0.001025	0.007299	4	3	5	0.0005
9	**2.7182**788	0.000113	0.000912	5	4	5	0.0005
10	**2.71828**152	1.11e−05	0.000101	6	5	7	0.00005

Also, assume that independent variables $u_1, u_2, \ldots, u_m$ are subject to errors $\Delta u_1, \Delta u_2, \ldots, \Delta u_m$, respectively. These errors in u's will obviously cause an error in the function F

$$F + \Delta F = f(u_1 + \Delta u_1, u_2 + \Delta u_2, \ldots, u_m + \Delta u_m) \tag{8.39}$$

The question now is how ΔF is related to errors $\Delta u_1, \Delta u_2, \ldots, \Delta u_m$.

To find the expression for ΔF, we should expand the right-hand side of Eq. (8.39) using the Taylor series expansion for a function of several variables [similar to that of Eq. (8.32)]

$$\begin{aligned} &f(u_1 + \Delta u_1, u_2 + \Delta u_2, \ldots, u_m + \Delta u_m) \\ &= f(u_1, u_2, \ldots, u_m) + \sum_{i=1}^{m} \frac{\partial f}{\partial u_i} \Delta u_i + HOT \end{aligned} \tag{8.40}$$

where *HOT* stands for the *higher-order terms* (including the squares, products, and higher powers of Δu_i). Because errors in u's are considered to be relatively small, we can neglect these HOT, and after substituting Eqs. (8.40) and (8.38) in Eq. (8.39), we obtain the general expression for computing ΔF

$$\Delta F = \sum_{i=1}^{m} \frac{\partial f}{\partial u_i} \Delta u_i \tag{8.41}$$

so that the relative error becomes

$$\frac{\Delta F}{F} = \left| \sum_{i=1}^{m} \frac{\partial f}{\partial u_i} \frac{\Delta u_i}{F} \right| \tag{8.42}$$

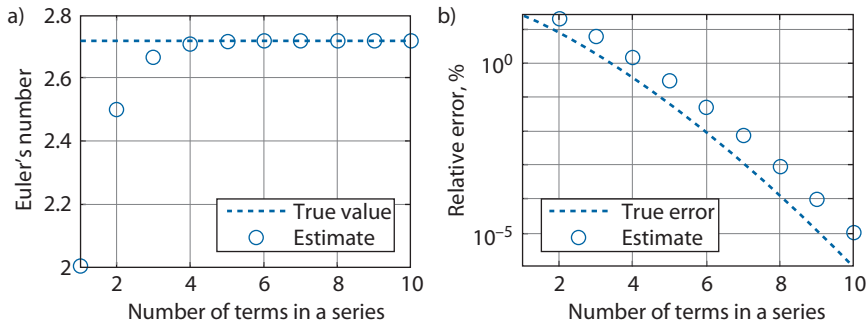


Fig. 8.18 Approximation of the Euler's number with a Taylor series.

These latter two expressions happen to be very useful. In particular, they allow you to estimate the errors while dealing with basic operations in arithmetic.

For addition and subtraction

$$F = u_1 \pm u_2 \tag{8.43}$$

and therefore

$$\Delta F = \Delta u_1 \pm \Delta u_2 \tag{8.44}$$

The errors Δu_1 and Δu_2 may be either positive or negative, and so the error cannot possibly be less than

$$|\Delta F|_{\min} = ||\Delta u_1| - |\Delta u_2|| \tag{8.45}$$

and theoretically (if no other errors are involved) should be less than

$$|\Delta F|_{\max} = |\Delta u_1| + |\Delta u_2| \tag{8.46}$$

The absolute error of the sum/difference of approximate numbers in the worse-case scenario is therefore bounded by the algebraic sum of their absolute errors.

That implies the proper way to handle addition or subtraction of two numbers of different accuracies: to round them to the worst accuracy first. Say, we want to find a sum of three numbers, 2314.2, 3.2713, and 0.56723, which are known to be correct in their last figures, but not farther. Instead of adding them directly as

$$\begin{array}{r} 2314.2 \\ + \quad 3.2713 \\ \underline{+ \quad 0.56723} \\ = 2318.03853 \end{array}$$

which implies that we actually add 2314.20000, 3.27130, 0.56723, that is, considering additional nonexisting significand figures in the first two numbers, we should first round them off to two decimals (one more than in the least accurate number), and then add them up:

$$\begin{aligned} & 2314.2 \\ + \quad & 3.27 \qquad \text{(rounding off the more accurate factor)} \\ + \quad & 0.57 \qquad \text{(rounding off the more accurate factor)} \\ = \quad & 2318.04 \\ = \quad & 2318.0 \qquad \text{(rounding off the result)} \end{aligned}$$

The last line represents rounding the result to one decimal as in the least accurate number. The reason for that is that by retaining two decimals in the more accurate numbers we eliminate the errors inherent in these numbers and thus reduce the error of the sum to that of the least accurate number [neglecting all but one term in Eq. (8.46)]. Of course, when using a computer, you would probably sum them up as it is, but it would be incorrect to present the final result as 2318.03853 (according to Eq. (8.46), the final result cannot have better accuracy than that of the least accurate number). In our case, the correct representation of the result would be 2318.0.

Similarly, when one approximate number is to be subtracted from another, they must both be rounded off to the same place before subtracting (in the case of more numbers involved in subtraction, one more decimal should be left, as in the example of summation of three numbers above). For example, subtracting 3.2713 from 2314.2, assuming that both numbers are only correct to their last figures, as

$$\begin{aligned} & 2314.2 \\ - \quad & 3.2713 \\ = \quad & 2310.9287 \end{aligned}$$

would be absurd, as opposed to

$$\begin{aligned} & 2314.2 \\ - \quad & 3.3 \qquad \text{(rounding off the more accurate factor)} \\ = \quad & 2310.9 \end{aligned}$$

Loss of significand figures by subtraction, *subtractive cancellation*, can also be cast in terms of Eq. (8.46). Suppose two numbers, 64.567 and 64.564, are each correct to five figures. Then their difference, $64.567 - 64.564 = 0.003$, is only correct to one figure.

For multiplication and division, it is convenient to consider the relative error [Eq. (8.42)] rather than the absolute error. Similar to Eqs. (8.45) and (8.46) for both multiplication and division, we may compute the lower and upper bounds of the relative error (in the case where there are no other

errors present) to be

$$\left|\frac{\Delta F}{F}\right|_{\min} = \left|\left|\frac{\Delta u_1}{u_1}\right| - \left|\frac{\Delta u_2}{u_2}\right|\right| \tag{8.47}$$

and

$$\left|\frac{\Delta F}{F}\right|_{\max} = \left|\frac{\Delta u_1}{u_1}\right| + \left|\frac{\Delta u_2}{u_2}\right| \tag{8.48}$$

Therefore, when it is required to find a product or quotient of two approximate numbers of different accuracies, to minimize the upper bound on the error [Eq. (8.48)], the more accurate number should be rounded off to contain one more significand figure than the least accurate factor. By doing so, we eliminate the error due to the more accurate factor, thus making the error of the product/quotient due solely to the error of the less accurate number [neglecting one term in Eq. (8.48)]. The final result should be given to as many significand figures as are contained in the least accurate factor.

For example, having the numbers 3.45 and 2.6781 with all figures trustworthy, the appropriate way to multiply them would be

$$\begin{aligned} &3.45 \\ &\times 2.678 \quad \text{(rounding off the more accurate factor)} \\ &= 9.23910 \\ &= 9.24 \quad \text{(rounding off the result)} \end{aligned}$$

rather than

$$\begin{aligned} &3.45 \\ &\times 2.6781 \\ &= 9.239445 \end{aligned}$$

In conclusion, let us consider one last example using division. Suppose we need to find the quotient $4.12/\sqrt{3}$ assuming that the numerator is correct to its last figure but no father. The correct way of handling it would be presenting the square root of 3 as $\sqrt{3} = 1.732$ so that

$$\begin{aligned} &41.2 \\ &/ \quad 1.732 \quad \text{(limiting } \sqrt{3} \text{ to just four significand figures)} \\ &= 23.7875288683603 \\ &= 23.8 \quad \text{(rounding off the result)} \end{aligned}$$

Let us prove it. If a number is correct to N significand figures, it is evident that its absolute error cannot be greater than half a unit in the Nth place.

Computing the right-hand side terms in Eq. (8.47) yields

$$\left|\frac{\Delta u_1}{u_1}\right| = \frac{0.05}{4.12} \approx 0.01 \quad \text{and} \quad \left|\frac{\Delta u_2}{u_2}\right| = \frac{0.005}{1.732} \approx 0.003$$

thus producing the lower bound on the possible error of the result to be ≈0.007. Now, if we leave two decimals in the result for F, we will have

$$\left|\frac{\Delta F}{F}\right| = \frac{0.05}{23.79} \approx 0.002$$

which is obviously less than it could possibly be. On the contrary, with only one decimal, the relative error will be

$$\left|\frac{\Delta F}{F}\right| = \frac{0.5}{23.8} \approx 0.02$$

which at least does not violate the lower bound. In this particular case, because of rounding off the result, we worsen the estimate of Eq. (8.48). But that is the only choice (between incorrect and less accurate).

8.6 Control of Numerical Errors

The total numerical error is a summation of truncation and round-off errors. In general, the only way to minimize round-off errors is to increase the number of significand figures of the floating-point representation. Further, we have noted that round-off error may increase due to subtractive cancellation or due to an increase in the number of computations in an analysis. Say we perform a certain number of digital computations in K points; then, if we double K, the total round-off error is likely to (at least) double too. In contrast, in many numerical methods, the truncation error is proportional to some power of $h \sim K^{-1}$ ($h << 1$) and therefore can be reduced by decreasing the step size h. Because a decrease of a step size can lead to subtractive cancellation or increase of a number of computations, the truncation errors decrease while round-off errors increase.

Therefore, we face the following dilemma. The decrease of one component of the total error leads to an increase of another component. If the total error is as shown in Fig. 8.19, the challenge is to identify the *point of diminishing returns*, where the round-off error begins to negate the benefits of step-size reduction. When using MATLAB, such situations are relatively uncommon because of its 15-digit precision. Nevertheless, they sometimes do occur and suggest a sort of "numerical uncertainty principle" that places an absolute limit on accuracy that may be obtained using certain computerized numerical methods. On top of that, we might also want to account for the required computational recourses because decreasing the step size obviously leads to increasing the computational time, t_{CPU}.

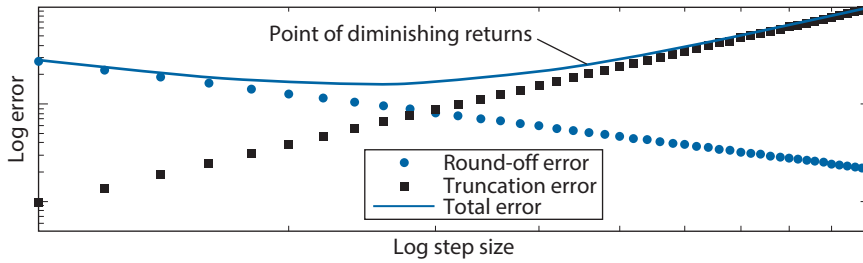


Fig. 8.19 Graphical depiction of a tradeoff between the round-off and truncation errors.

Hence, you may cast this decrease as a penalty P for having too small step size, that is, replace the round-off error in Fig. 8.19 with the required CPU time:

$$P = \varepsilon + wt_{CPU} \tag{8.49}$$

(ε accounts for the total error and w is the weighting coefficient).

In this case, even for the 15-digit precision of MATLAB, you will always end up having some step size after which a further decrease of truncation error will result in a drastic slow down of a computational process.

For most practical cases, we do not know the exact error associated with numerical methods. The exception, of course, is when we know the exact solution, which makes our numerical approximations unnecessary. Therefore, for most engineering and scientific applications, we must settle for some estimate of a computational error.

There are no systematic and general approaches to evaluating numerical errors for all problems. In many cases, the error estimates are based on the experience and judgment of engineer or scientist. Although error analysis is, to a certain extent, an art, there are several practical programming guidelines we can suggest.

First and foremost, avoid subtracting two nearly equal numbers. Loss of significance almost always occurs when this is done. Sometimes you can rearrange or reformulate the problem to avoid subtractive cancellation. Furthermore, when adding and subtracting numbers, it is best to sort the numbers and work with the smallest numbers first, which may prevent losing significance.

Second, an attempt to predict a total numerical error using theoretical formulations should be made. In the following chapters, the Taylor series expansion, like the ones shown in Eqs. (8.32), (8.33) or (8.40), will be our primary tool for doing it. Note that usually the estimates happen to be quite conservative. Sometimes there is an alternative way to estimate accuracy of your results by seeing if the results satisfy some condition or by substituting the results back into the original equation.

Finally, you should be prepared to perform numerical experiments to increase awareness of computational errors and possible ill-conditioned problems. Such experiments may involve repeating computations with a different step size or method and comparing the results. You may employ sensitivity analysis to see how your solution changes when you change the model parameters or input values. You may want to try different numerical algorithms that have different theoretical foundations, are based on different computational strategies, or have different convergence properties and stability characteristics.

Problems

8.1 Write a script, which will convert any nonnegative integer into the octal, ternary, or binary number. Use the `dec2base` function to check the correctness of your results.

8.2 Perform analysis, similar to that presented in Table 8.7 (except the last two columns) and Fig. 8.18, using a Maclaurin series expansion (at $x = 0$) of $\sin(x)$ for $x = 1$. Considering 1 to 5 terms of the expansion, compute the value of $\varepsilon^t(\%)$, $\varepsilon^a(\%)$, $N(\varepsilon^t)$, and $N(\varepsilon^a)$ and plot the results.

8.3 Try to reproduce Fig. 8.19 for the following problem. Consider evaluating the following integral numerically:

$$I = \int_0^{10} x^3 \mathrm{d}x$$

Although its exact value is known, $I^t = 0.25 \times 10^4$, for the numerical estimate, let us break the entire interval [0;10] into n equal subintervals and use the approximate formula:

$$I_n^a = \frac{10}{n^4} \sum_{i=1}^{n} \left(i - \frac{1}{2} \right)^3$$

The step size h is inversely proportional to the number of the terms in the preceding sum, $h = 10n^{-1}$. For $n = 1,2,\ldots,7$, compute the true error $\varepsilon = \left| I^t - I_n^a \right|$. Using the `tic` and `toc` functions, record the CPU time required to obtain a solution for each number of intervals. Use Eq. (8.49) with $w = 0.01\ \mathrm{s}^{-1}$ to estimate the overall penalty. Plot all three dependences, $\varepsilon(n)$, $t_{CPU}(n)$, and $P(n)$, vs the step size $h(n)$ and comment on the point of diminishing returns.

Chapter 9 Numerical Linear Algebra and Eigenvalue Problems

- Solving Systems of Linear Algebraic Equations
- Understanding Matrix Decompositions
- Using Linear Algebra To Solve Differential Equations
- Using the MATLAB Functions

`mldivide`, `mrdivide`, `rref`, `linsolve`, `lu`, `qr`, `svd`, `eig`, `det`, `rank`, `inv`, `null`

9.1 Introduction

This chapter proceeds with Part III of the book (Fig. 0.1) dealing with the basic MATLAB functions realizing different numerical methods solving applied engineering problems. Specifically, it addresses the problem of solving systems of linear algebraic equations (LAEs). MATLAB features a variety of functions to address these types of problems, `mldivide`, `mrdivide`, `rref`, and `linsolve`, each using a different method. The goal of this chapter is to help you understand the essence of these methods and the differences between them, so that you can employ them appropriately. In addition to the aforementioned functions, this chapter presents several functions dedicated to matrix decompositions, such as LU (`lu`), QR (`qr`), SVD (`svd`), and eigen (`eig`) decomposition, used to compute a determinant of a matrix (`det`), its rank (`rank`), inverse (`inv`), and null space (`null`). The latter decomposition, eigendecomposition, turns out to be very useful to solve systems of linear ODEs, and so despite the fact that this specific topic will be addressed later in Chapter 14, this chapter shows how to use this decomposition to solve linear ODEs analytically. In what follows, we first present the theoretical background behind all major methods devoted to addressing the linear algebra problems and then show examples of using the corresponding MATLAB functions.

9.2 Classification of LAEs

Consider a system of LAEs written in the matrix form

$$\mathbf{Ax} = \mathbf{b} \tag{9.1}$$

where, **A** and **b** are the m-by-n matrix and m-by-1 vector of coefficients and **x** is the n-by-1 vector of unknown variables

$$\mathbf{A} = [a_{ij}] = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \tag{9.2}$$

Depending on the properties of matrix **A** and vector **b**, this system of equations has no, one, or infinitively many solutions as shown in Fig. 9.1. No other possibilities exist. As used in this chapter, the most general case is when **A** is a square n-by-n matrix ($m = n$). As seen in Fig. 9.1, even then several possibilities exist.

The deciding point regarding the type of solution the set of LAEs yields is examining its matrix **A**. Figure 9.1 suggests either checking whether matrix **A** is invertible (in this case, another n-by-n matrix **B** exists, such that $\mathbf{AB} = \mathbf{I}$) or whether its determinant is not equal zero. In fact, some other equivalent checks may be applied:

1. **A** is invertible.
2. Determinant of **A** is zero.
3. Rank of **A** is equal to n.
4. The columns of **A** are linearly independent.

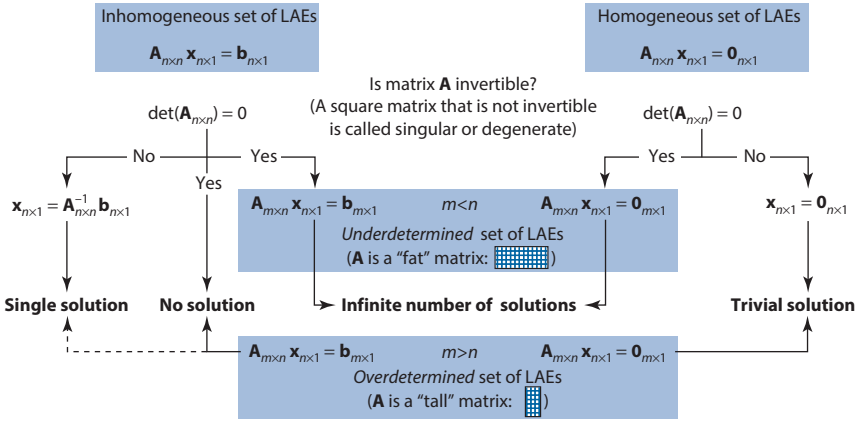


Fig. 9.1 Variety of LAEs.

5. The transpose $\mathbf{A}^T$ is an invertible matrix.
6. The matrix times its transpose, $\mathbf{A}\mathbf{A}^T$, is an invertible matrix.
7. The number 0 is not an eigenvalue of $\mathbf{A}$ ($\mathbf{A}$ is a *nonsingular* matrix).

Some of these concepts, along with all possibilities depicted in Fig. 9.1, will be addressed later in this chapter, but the major emphasis will be given to solving the sets of *inhomogeneous* equations, assuming that $\mathbf{b} \neq \mathbf{0}$ ($\mathbf{0}$ is a vector of zeros), that is, that vector $\mathbf{b}$ has at least one nonzero element [otherwise, Eq. (9.1) is referred to as a *homogeneous* set of equations].

9.3 Inhomogeneous Sets of LAEs

The case when a matrix $\mathbf{A}$ in Eq. (9.1) is square, has a full rank, and a vector $\mathbf{b}$ has at least one nonzero element, leads to a single (unique) solution as shown in Fig. 9.2. While it shows that this solution can be obtained by simply inverting the matrix $\mathbf{A}$, other, more computationally effective methods are usually employed. These methods are discussed next.

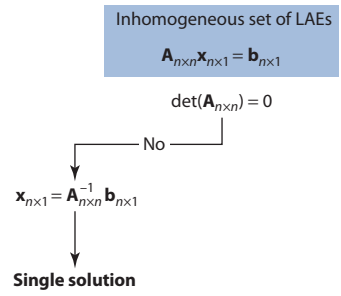


Fig. 9.2 The case of a single solution for a full-rank inhomogeneous set of LAEs.

9.3.1 Gaussian Elimination

Gauss or *Gaussian elimination* (aka row reduction) is the most widely used method for solving a set of inhomogeneous LAEs, known from ancient times and formally presented by Gauss in the early 1800s. It consists of the *forward elimination* process followed by the *backward substitution* process and makes the entire approach very suitable for computer applications. The MATLAB functions `mldivide (\)` and `mrdivide (/)` use exactly this method.

The forward elimination process starts with multiplying the first equation by a multiplier $l_{21} = a_{21}/a_{11}$ and subtracting it from the second equation which results in elimination of the first term of the second equation. In matrix notation, it means premultiplying (which means left-multiplying) both sides of Eq. (9.1) by the *elementary matrix* $\mathbf{L}^*_{21}$:

$$\mathbf{L}^*_{21}\mathbf{A}\mathbf{x} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ -l_{21} & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix} \mathbf{A}\mathbf{x} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ -l_{21} & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix} \mathbf{b} = \mathbf{L}^*_{21}\mathbf{b} \quad (9.3)$$

Likewise, multiplying the first equation of system [Eq. (9.1)] by $l_{i1} = a_{i1}/a_{11}$ and subtracting it from the ith equation eliminates the first term from the ith equation ($i = 3, 4, \ldots, n$). When applied for all but the last rows, system (9.1) transforms to

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n &= b_1 \\ a'_{22}x_2 + a'_{23}x_3 + \cdots + a'_{2n}x_n &= b'_2 \\ &\vdots \\ a'_{n2}x_2 + a'_{n3}x_3 + \cdots + a'_{nn}x_n &= b'_n \end{aligned} \tag{9.4}$$

where

$$a'_{ij} = a_{ij} - \frac{a_{i1}}{a_{11}}a_{1j} \quad \text{and} \quad b'_i = b_i - \frac{a_{i1}}{a_{11}}b_1, \quad i = 2, 3, \ldots, n, \quad j = 2, 3, \ldots, n$$

(prime does not denote a partial derivative here; it is just a notation to distinguish the new coefficients from the original ones).

In matrix notation, it means premultiplying both sides of Eq. (9.1) by corresponding elementary matrices $\mathbf{L}^*_{i1}$ one after another, so that the system (9.4) corresponds to

$$\mathbf{L}^*_{n1} \ldots \mathbf{L}^*_{31}\mathbf{L}^*_{21}\mathbf{A}\mathbf{x} = \left(\prod_{i=2}^{n} \mathbf{L}^{*T}_{i1}\right)^T \mathbf{A}\mathbf{x} = \left(\prod_{i=2}^{n} \mathbf{L}^{*T}_{i1}\right)^T \mathbf{b} \tag{9.5}$$

(The transposition of matrices is used to write them in conjunction with the product symbol, which runs from 2 up to n as opposed to from n down to 2.) Clearly (see Eq. (9.3)), all elementary matrices $\mathbf{L}^*_{ij}$ are constructed from the identity matrix with one zero element at the intersection of the ith row and jth column substituted with $-l_{ij}$. The coefficient in the denominator of multipliers l_{i1}, a_{11}, is called the *pivot*, the first pivot.

Now that we have managed to eliminate the first term from all but the first equation, it is time to eliminate the second term. For all equations starting from the third one down, we eliminate the second term by multiplying the second equation in Eq. (9.4) by $l_{i2} = a'_{i1}/a'_{22}$ and subtracting it from the ith equation ($i = 3, 4, \ldots, n$). The coefficient a'_{22} becomes the second pivot. In matrix notation, it means continuing premultiplying both sides of Eq. (9.5) by the corresponding elementary matrices $\mathbf{L}^*_{i2}$:

$$\begin{aligned} \mathbf{L}^*_{n2} \ldots \mathbf{L}^*_{42}\mathbf{L}^*_{32}\mathbf{L}^*_{n1} \ldots \mathbf{L}^*_{31}\mathbf{L}^*_{21}\mathbf{A}\mathbf{x} &= \left(\prod_{i=3}^{n} \mathbf{L}^{*T}_{i2}\right)^T \left(\prod_{i=2}^{n} \mathbf{L}^{*T}_{i1}\right)^T \mathbf{A}\mathbf{x} \\ &= \left(\prod_{i=3}^{n} \mathbf{L}^{*T}_{i2}\right)^T \left(\prod_{i=2}^{n} \mathbf{L}^{*T}_{i1}\right)^T \mathbf{b} \end{aligned} \tag{9.6}$$

This process is continued until the set of equations is converted into the *row echelon form*

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n &= b_1 \\ a'_{22}x_2 + a'_{23}x_3 + \cdots + a'_{2n}x_n &= b'_2 \\ a''_{33}x_3 + \cdots + a''_{3n}x_n &= b'' \\ &\vdots \\ a^{(n-1)}_{nn}x_n &= b^{(n-1)}_n \end{aligned} \tag{9.7}$$

with the leading terms in each equation being pivots.

In matrix notation, Eq. (9.7) is equivalent to

$$\mathbf{L}^*\mathbf{A}\mathbf{x} = \mathbf{L}^*\mathbf{b} \tag{9.8}$$

or simply

$$\mathbf{U}\mathbf{x} = \mathbf{c} \tag{9.9}$$

meaning that by construction, the matrix

$$\mathbf{U} = \mathbf{L}^*\mathbf{A} \tag{9.10}$$

is the *upper triangular* matrix, sometimes called the *right matrix* (because all entries below the diagonal are zero), and

$$\mathbf{c} = \mathbf{L}^*\mathbf{b} \tag{9.11}$$

is the vector composed of the right-hand side terms of Eq. (9.7).

Both $\mathbf{U}$ and $\mathbf{c}$ are obtained via premultiplication of the original matrix $\mathbf{A}$ and vector $\mathbf{b}$ by the compound matrix

$$\mathbf{L}^* = \mathbf{L}^*_{n,n-1}\left(\prod_{i=n-1}^{n}\mathbf{L}^{*T}_{i,n-2}\right)^T \cdots \left(\prod_{i=3}^{n}\mathbf{L}^{*T}_{i2}\right)^T\left(\prod_{i=2}^{n}\mathbf{L}^{*T}_{i1}\right)^T \tag{9.12}$$

which by construction happens to be a *lower triangular* matrix (*left matrix*). The inverse of matrix $\mathbf{L}^*$ is also a lower triangular matrix, with ones on the main diagonal and corresponding multipliers below them:

$$\mathbf{L} = (\mathbf{L}^*)^{-1} = \begin{bmatrix} 1 & 0 & \cdots & 0 & 0 \\ l_{21} & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ l_{n-1,1} & l_{n-1,2} & \cdots & 1 & 0 \\ l_{n1} & l_{n2} & \cdots & l_{n,n-1} & 1 \end{bmatrix} \tag{9.13}$$

The effectiveness of any algorithm for solving systems of LAEs is judged upon the number of operations needed to complete it. Let us ignore

operations on the right-hand side terms (which are negligibly small compared with those performed on the elements of matrix **A**) and also count each multiplication–subtraction as a single operation. At the first step [Eq. (9.4)], all n^2 coefficients of matrix **A** need to be changed except the n coefficients of the first row, making the total of n^2-n operations required. Similarly, the second step [Eq. (9.6)] requires $(n-1)^2-(n-1)$ operations, etc. Altogether, the total number of operations is

$$\sum_{i=1}^{n}\left(i^2-i\right)=\sum_{i=1}^{n}i^2-\sum_{i=1}^{n}i=\frac{n(n+1)(2n+1)}{6}-\frac{n(n+1)}{2}$$
$$=\frac{n^3-n}{3} \tag{9.14}$$

(the standard formulas for the sum of the first n numbers and sum of their squares are used here). For a large n, a good estimate for the number of operations required for the elimination process is $(n^3/3)$ (the more precise numbers are $n(n+1)/2$ divisions, $(2n^3+3n^2-5n)/6$ multiplications, and the same number of subtractions, totaling to $(2/3)n^3$).

Now that we have reduced the original system of linear equations [Eq. (9.1)] to that of system (9.9), we may proceed with the *backward substitution* process. It starts with resolving the last equation [see Eq. (9.7)], resulting in

$$x_n=\frac{b_n^{(n-1)}}{a_{nn}^{(n-1)}} \tag{9.15}$$

Substituting this value into the $(n-1)$th equation yields

$$x_{n-1}=\frac{b_{n-1}^{(n-2)}-a_{n-1,n}^{(n-1)}\,x_n}{a_{n-1,n-1}^{(n-2)}} \tag{9.16}$$

Using solutions (9.15) and (9.16) in the $(n-2)$th equation yields the solution for x_{n-2}, and so on until from the first equation of system [Eq. (9.7)] we get

$$x_1=\frac{b_1-\sum_{j=2}^{n}a_{1j}\,x_j}{a_{11}} \tag{9.17}$$

which completes the backward substitution. For your convenience, the entire process is presented graphically in Fig. 9.3.

Obviously, the backward substitution process involves fewer operations and therefore is executed faster than that of the forward elimination. Specifically, the last unknown is found in only one arithmetic operation, a division by the last pivot. The second to the last unknown requires two operations, and so on. Hence, the total number of operations required for the backward

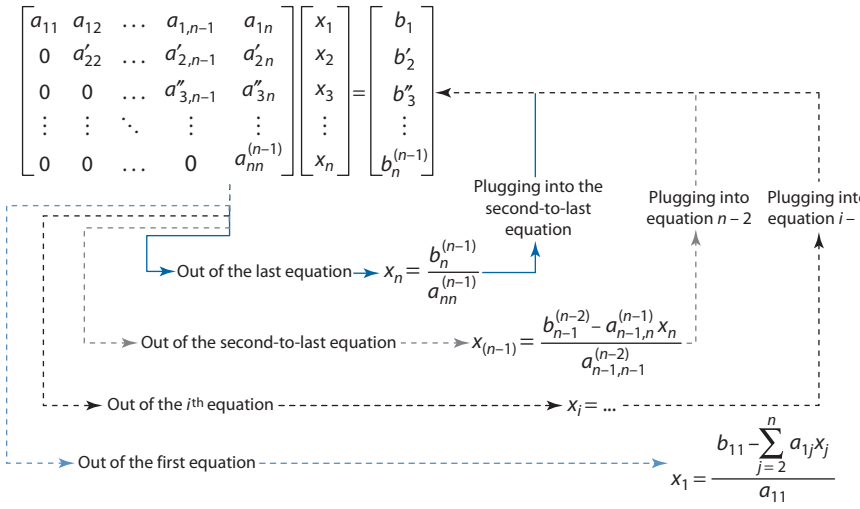


Fig. 9.3 Visualization of backward substitution.

substitution is

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \approx 0.5n^2 \tag{9.18}$$

The number of arithmetic operations required to find a solution is only one characteristic of a numerical procedure. Its accuracy is another. From this standpoint we know that for the floating-point arithmetic, the best precision is obtained in the small number areas as opposed to the large numbers (Fig. 8.12). That is why it is important what pivot values the original matrix **A** has. The extreme case is when one of the pivots happens to be a very small number close to zero. (Even if the original matrix may not have zeros, any diagonal coefficient may become zero in the process of forward elimination.) This would result in the corresponding multiplier being a huge number, so that in the consequent operations, we would be adding together very small and very large numbers. That is exactly where rightmost significand digits will be lost, as explained in the examples of Sec. 8.5.2.

Consider a classical example:

$$\begin{bmatrix} 0.00001 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \tag{9.19}$$

After forward elimination with the multiplier being 10^5, Eq. (9.19) transforms to

$$\begin{bmatrix} 0.00001 & 1 \\ 0 & -99{,}999 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ -99{,}998 \end{bmatrix} \tag{9.20}$$

with two pivots of 0.00001 and −99,999, being is absolutely out of scale. The backward substitution yields $x_2 = 0.99999$ and $x_1 = 1$. On a hypothetical computer that keeps less than five significand digits, due to the round-off error, the solution for x_2 would be $x_2 = 1$, which is wrong only in the fifth place. Substituting this "erroneous" solution back into the first equation results in $x_1 = 0$. You can see that this "solution", $x = [0,1]^T$, does not satisfy Eq. (9.19).

If we swap the equations

$$\begin{bmatrix} 1 & 1 \\ 0.00001 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \tag{9.21}$$

a forward elimination with the multiplier of 10^{-5} yields

$$\begin{bmatrix} 1 & 1 \\ 0 & 0.99999 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 0.99998 \end{bmatrix} \tag{9.22}$$

with about the same-magnitude pivots of 1 and 0.99999. The backward substitution should yield $x_2 = 0.99999$ and $x_1 = 1.00001$. However, because of the round-off error on our hypothetical computer, $x_2 = 1$. Substituting this value into the first equation results in $x_1 = 1$. Hence, swapping equations ensures that, even in the case of round-off error, we still have an accurate solution, $x = [1,1]^T$.

That is why each step of the forward elimination process of the Gauss method also involves *partial pivoting*, that is, changing the sequential order of equations, so that at each step the pivot has the largest absolute value among all other column elements. If at any step another row looks more attractive (has a larger absolute value of the leading element), it is swapped with the current pivot row. If a zero diagonal element is unavoidable even with pivoting, it indicates that the problem has no unique solution because the matrix **A** happens to be singular.

If row swapping has occurred, it means that both sides of Eq. (9.1) were premultiplied by the *permutation matrix* **P** that produced a desirable row exchange. For example, the matrix

$$\mathbf{P}_1^* = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{9.23}$$

swaps the first and third rows at the first step [the unities move from their positions in the identity matrix as shown in Eq. (9.23) with the arrows].

Overall, although pivoting adds more operations to the computational cost of algorithm, these additional operations may be necessary for the

algorithm to work at all. In fact, these additional operations are worthwhile because they add numerical stability to the final result.

As opposed to the partial pivoting, the *complete* or *maximal pivoting* considers all entries in the whole matrix, interchanging both rows and columns. Complete pivoting is rarely used because of many additional computations it introduces. Lastly, *scaled partial pivoting* is a variation of partial pivoting, so that the pivoting element is chosen as the entry that is the largest relative to remaining elements in the same row.

9.3.2 Gauss–Jordan Elimination

Gauss–Jordan elimination is a variant of Gaussian elimination in which, in place of the backward substitution, the *backward elimination* process is implemented to eliminate all coefficients except the pivots, which become unity. Starting with Eq. (9.7), which we ended up with after forward elimination, the last row is divided by a_{nn}^{n-1}, yielding

$$\left[\begin{array}{cccccc|c} a_{11} & a_{12} & \cdots & a_{1,n-2} & a_{1,n-1} & a_{1,n} & b_1 \\ 0 & a'_{22} & \cdots & a'_{2,n-2} & a'_{2,n-1} & a'_{2,n} & b'_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & a_{n-1,n-1}^{(n-2)} & a_{n-1,n}^{(n-2)} & b_{n-1}^{(n-2)} \\ 0 & 0 & \cdots & 0 & 0 & 1 & \overline{b}_n \end{array}\right] \tag{9.24}$$

where $\overline{b}_n = b_n^{(n-1)}/a_{nn}^{(n-1)}$. Note that, in Eq. (9.24), we used the *augmented matrix* form, which for compactness combines the elements of matrix **A** and vector **b** together.

Next, multiplying the nth row by $a_{in}^{(i-1)}$ and subtracting it from the ith row ($i = n-1,\ n-2,\ \ldots,\ 1$) eliminates the $a_{in}^{(i-1)}$ coefficients from the remaining $n-1$ rows upward:

$$\left[\begin{array}{cccccc|c} a_{11} & a_{12} & \cdots & a_{1,n-2} & a_{1,n-1} & 0 & \overline{b}_1 \\ 0 & a'_{22} & \cdots & a'_{2,n-2} & a'_{2,n-1} & 0 & \overline{b}_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & a_{n-1,n-1}^{(n-2)} & 0 & \overline{b}_{n-1} \\ 0 & 0 & \cdots & 0 & 0 & 1 & \overline{b}_n \end{array}\right] \tag{9.25}$$

Here, $\overline{b}_i = b_i^{(i-1)} - \overline{b}_n a_{in}^{(i-1)}$ and $i = 1, 2, \ldots, n-1$. Continuing this process for normalizing the $(n-1)$th row and eliminating $a_{i,n-1}^{(i-1)}$ ($i = n-2,\ n-1,\ \ldots,1$),

coefficients from the remaining $n-2$ rows upward yields

$$\left[\begin{array}{cccccc|c} a_{11} & a_{12} & \cdots & a_{1,n-2} & 0 & 0 & \overline{b}_1 \\ 0 & a'_{22} & \cdots & a'_{2,n-2} & 0 & 0 & \overline{b}'_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & a^{(n-3)}_{n-2,n-2} & 0 & 0 & \overline{b}'_{n-2} \\ 0 & 0 & \cdots & 0 & 1 & 0 & \overline{b}'_{n-1} \\ 0 & 0 & \cdots & 0 & 0 & 1 & \overline{b}_n \end{array}\right] \tag{9.26}$$

$$\text{with } \overline{b}'_{n-1} = \frac{\overline{b}_{n-1}}{a^{(n-2)}_{n-1,n-1}} \quad \text{and} \quad \overline{b}'_i = \overline{b}_i - \overline{b}'_{n-1} a^{(i-1)}_{i,n-1},\ i = 1, 2, \ldots, n-2.$$

Repeating the elimination process for the remaining rows results in the *reduced row echelon form* (RREF)

$$\left[\begin{array}{ccccc|c} 1 & 0 & \cdots & 0 & 0 & \overline{b}^{(n-1)}_1 \\ 0 & 1 & \cdots & 0 & 0 & \overline{b}^{(n-2)}_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 & \overline{b}'_{n-1} \\ 0 & 0 & \cdots & 0 & 1 & \overline{b}_n \end{array}\right] \tag{9.27}$$

which ends the backward elimination process. The solution becomes

$$x_i = \overline{b}^{(n-i)}_i, \quad i = 1, 2, \ldots, n \tag{9.28}$$

that is, the rightmost column of RREF matrix happens to be a column vector containing the final solution. This algorithm is implemented in the MATLAB `rref` function.

9.3.3 LU Decomposition

As shown in Sec. 9.3.2, the forward elimination process of the Gauss elimination method resulted in converting the original matrix **A** in Eq. (9.1) into the upper triangular matrix **U** in Eq. (9.9). Premultiplying both sides of Eq. (9.10) by the inverse of matrix $\mathbf{L}^*$ and accounting for Eq. (9.13) results in

$$\mathbf{A} = \mathbf{LU} \tag{9.29}$$

which is referred to as the lower-upper (*LU*) *decomposition* or *factorization* of matrix **A**.

Formally, the LU decomposition scheme is a transformation of a nonsingular (with a nonzero determinant) matrix **A** to a product of two matrices, one being a lower triangular matrix and another—an upper triangular

matrix. For example, for a 3-by-3 matrix, the LU decomposition yields

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix} \tag{9.30}$$

Following the forward elimination routine resulting in Eq. (9.7), a general scheme to perform the LU decomposition of a square n-by-n matrix is as follows:

The first row of **U** is obtained as	$u_{1j} = a_{1j}$	for $j = 1, 2, \ldots, n$
The first column of **L** is obtained as	$l_{i1} = \dfrac{a_{i1}}{u_{11}}$	for $i = 2, 3, \ldots, n$
The second row of **U** is obtained by	$u_{2j} = a_{2j} - l_{21} u_{1j}$	for $i = 2, 3, \ldots, n$
The second column of **L** is obtained by	$l_{i2} = \dfrac{a_{i2} - l_{i1} u_{12}}{u_{22}}$	for $i = 3, 4, \ldots, n$
...		
The sth row of **U** is obtained by	$u_{sj} = a_{sj} - \sum_{k=1}^{s-1} l_{sk} u_{kj}$	for $j = s, s+1, \ldots, n$
The sth column of **L** is obtained by	$l_{is} = \dfrac{a_{is} - \sum_{k=1}^{s-1} l_{ik} u_{ks}}{u_{ss}}$	for $i = s+1, s+2, \ldots, n$
...		

Note that in this process, the diagonal elements of **L** are not calculated because they are unity. Although the LU decomposition follows exactly the same steps as a regular forward elimination procedure, defining matrices **L** and **U** explicitly has its computational advantage, especially in the case when there is a need to solve multiple sets of LAEs featuring the same matrix **A**

$$\mathbf{A}\mathbf{x}_1 = \mathbf{b}_1, \ldots, \mathbf{A}\mathbf{x}_p = \mathbf{b}_p \tag{9.31}$$

Recall that forward elimination (the LU decomposition of matrix **A**) requires $\sim n^3$ operations, whereas backward substitution only requires $\sim n^2$ operations. If we were to solve all p systems of Eq. (9.31) "from scratch," it would require $\sim p(n^3 + n^2)$ operations. Instead, all we have to do is to perform the LU decomposition of matrix **A** just once and then use its results to resolve all p systems of Eq. (9.31).

Using Eq. (9.29), we may rewrite Eq. (9.1) as

$$\mathbf{A}\mathbf{x} = \mathbf{L}\mathbf{U}\mathbf{x} = \mathbf{b} \tag{9.32}$$

or as a combination of two sets (also see Eqs. (9.11) and (9.9))

$$\mathbf{Lc} = \mathbf{b} \quad \text{and} \quad \mathbf{Ux} = \mathbf{c} \tag{9.33}$$

for each $\mathbf{b}_k$, $k = 1, 2, \ldots, p$. (For simplicity, we are not showing the permutation matrix $\mathbf{P}$, assuming that it is incorporated in $\mathbf{A}$ and $\mathbf{b}$.) Because $\mathbf{L} = [l_{ij}]$ is a lower triangular matrix, we can easily solve the first equation in Eq. (9.33) for each $\mathbf{c}_k = [c_1, c_2, \ldots, c_n]^T$ (for different $\mathbf{b}_k = [b_1, b_2, \ldots, b_n]^T$) using *forward substitution:*

$$c_1 = b_1, \quad c_i = b_i - \sum_{j=1}^{i-1} l_{ij}c_j, \quad \text{for } i = 2, 3, \ldots, n \tag{9.34}$$

requiring altogether $\sim pn^2$ operations. We then can use the solution for $\mathbf{c}_k$ [Eq. (9.34)] and employ the upper triangular matrix $\mathbf{U} = [u_{ij}]$ to solve for the corresponding $\mathbf{x}_k = [x_1, x_2, \ldots, x_n]^T$ using *backward substitution:*

$$x_n = \frac{c_n}{u_{nn}}, \quad x_i = c_i - \sum_{j=i+1}^{n} u_{ij}x_j, \quad \text{for } i = n-1, n-2, \ldots, 1 \tag{9.35}$$

also requiring $\sim pn^2$ operations altogether. Therefore, instead of $\sim p(n^3 + n^2)$ operations, we only have $\sim n^3 + 2pn^2$, which is of the order of p times less.

Additionally, because the upper part of $\mathbf{L}$ and the lower part of $\mathbf{U}$ contain only zeros, we can store both matrices in the same n-by-n matrix (to save memory) as

$$\begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1,n-1} & u_{1n} \\ l_{21} & u_{22} & \cdots & u_{2,n-1} & u_{2n} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ l_{n-1,1} & l_{n-1,2} & \cdots & u_{n-1,n-1} & u_{n-1,n} \\ l_{n1} & l_{n2} & \cdots & l_{n,n-1} & u_{nn} \end{bmatrix} \tag{9.36}$$

The memory usage can be reduced even further by noticing that each element in $\mathbf{A}$ is used for calculation of l_{ij} or u_{ij} during the factorization process only once. Thus, once l_{ij} or u_{ij} is computed, it can be overwritten into a_{ij} making it unnecessary to create a separate matrix.

The `lu` function expresses a matrix $\mathbf{A}$ as the product of two essentially triangular matrices [Eq. (9.29)], with the only difference that a lower triangular matrix is returned permutated. As explained later in Sec. 9.7.1, however, there is a syntax allowing you to obtain a lower triangular matrix $\mathbf{L}$ with a unit diagonal and a permutation matrix $\mathbf{P}$ separately, so that

$$\mathbf{PA} = \mathbf{LU} \tag{9.37}$$

Another function to solve the linear system [Eq. (9.1)], `linsolve`, employs the LU factorization when $\mathbf{A}$ is a square matrix as well. Examples will be provided in Sec. 9.7.1.

9.3.4 Inverse and Determinant of a Square Matrix

By definition, the *inverse of a square matrix*, $\mathbf{A}^{-1}$ satisfies the relationship

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{I} \tag{9.38}$$

You may treat Eq. (9.38) as the set of matrix equations

$$\mathbf{A}\mathbf{x}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \mathbf{A}\mathbf{x}_2 = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \ldots, \quad \mathbf{A}\mathbf{x}_n = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix} \tag{9.39}$$

where solutions $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n$ constitute the columns of coefficients of the inverse matrix $\mathbf{A}^{-1} = [\mathbf{x}_1 \ \mathbf{x}_2 \cdots \mathbf{x}_n]$. This is exactly what the LU decomposition is good for. You may find the matrix inverse by performing the LU decomposition of matrix $\mathbf{A}$, followed by solving Eq. (9.33) for each set of Eq. (9.39).

In fact, in this specific case, you can solve all n systems of Eq. (9.39) simultaneously. If you recall, in Sec. 9.3.2, we introduced the augmented matrix, which was used for forward and backward elimination to produce RREF of Eq. (9.27). We can do essentially the same here, augmenting the matrix $\mathbf{A}$ with the same-size identity matrix $\mathbf{I}$ to begin with:

$$\left[\begin{array}{cccc|cccc} a_{11} & a_{12} & \cdots & a_{1n} & 1 & 0 & \cdots & 0 \\ a_{21} & a_{22} & \cdots & a_{2n} & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & 0 & 0 & \cdots & 1 \end{array}\right] \quad \Leftrightarrow \quad [\mathbf{A} \mid \mathbf{I}] \tag{9.40}$$

Utilizing the Gauss–Jordan elimination procedure, laid out in Sec. 9.3.2, solves Eq. (9.40) by bringing it to the form

$$\left[\begin{array}{cccc|cccc} 1 & 0 & \cdots & 0 & \overline{a}_{11}^{(n-1)} & \overline{a}_{12}^{(n-1)} & \cdots & \overline{a}_{1n}^{(n-1)} \\ 0 & 1 & \cdots & 0 & \overline{a}_{21}^{(n-2)} & \overline{a}_{22}^{(n-2)} & \cdots & \overline{a}_{2n}^{(n-2)} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & \overline{a'}_{n1} & \overline{a'}_{n2} & \cdots & \overline{a'}_{nn} \end{array}\right] \quad \Leftrightarrow \quad \left[\mathbf{I} \mid \mathbf{A}^{-1}\right] \tag{9.41}$$

If the Gauss–Jordan elimination becomes unsolvable even with pivoting, meaning the matrix $\mathbf{A}$ is singular, then the inverse does not exist.

Computational effectiveness of the approach based on the LU decomposition is compared with that of the Gaussian elimination in Table 9.1. Although solving a single set of LAEs seems to favor the Gauss method

Table 9.1 Number of Numerical Operations Required to Solve n Sets of LAEs

Gauss Elimination Method	Number of Operations	LU Decomposition-Based Approach	Number of Operations
Forward elimination	$\frac{n^3-n}{3} \underset{n\to\infty}{\sim} \frac{n^3}{3}$	Matrix decomposition	$\frac{n^3-n}{3} \underset{n\to\infty}{\sim} \frac{n^3}{3}$
Backward substitution	$\frac{n^2-n}{2} \underset{n\to\infty}{\sim} \frac{n^2}{2}$	Solving two systems [Eq. (9.33)]	$n^2+n \underset{n\to\infty}{\sim} n^2$
Total for the n sets of LAEs	$n\left(\frac{n^3}{3}+\frac{n^2}{2}\right) \sim \frac{n^4}{3}$	Total for the n sets of LAEs	$\frac{n^3}{3}+n\cdot n^2 \sim \frac{4}{3}n^3$

(consider the two first lines of the table), if it comes to solving the n sets of LAEs having the same matrix $\mathbf{A}$, the LU decomposition has an obvious advantage. In the latter case (see the last row of the table), you only have to perform the LU decomposition once and then solve two auxiliary systems of LAEs n times, whereas for the Gauss method, you have to do both steps, forward elimination and backward substitution, for each set of LAEs, that is, the sum of operations in the two first rows should be multiplied by n.

The MATLAB function `inv` uses the LU decomposition approach for $n > 2$, whereas for $n = 2$, it uses the analytical formula. To obtain it, you may use

```
>> syms a b c d
>> A=[a b; c d]; inv(A)
```

which returns

```
ans = [d/(a*d-b*c), -b/(a*d-b*c)]
      [-c/(a*d-b*c), a/(a*d-b*c)]
```

One of the ways to compute the *determinant* of matrix $\mathbf{A}$, $\det(\mathbf{A})$, a special number associated with any square matrix, is to use the Laplace expansion (aka cofactor expansion) computing a weighted sum of determinants of sub-matrices along any row or column. For example, using elements of the first column yields

$$\det(\mathbf{A}) = \begin{cases} a_{11}, & n = 1 \\ \sum_{i=1}^{n} a_{i1}(-1)^{i+1}\det(\mathbf{M}_{i1}), & n > 1 \end{cases} \tag{9.42}$$

where $(-1)^{i+1}\det(\mathbf{M}_{i1})$ is called the $i,1$ *cofactor* of a_{i1}, and $\det(\mathbf{M}_{i1})$ is referred to as a $i,1$ *minor* of matrix $\mathbf{A}$. The matrix $\mathbf{M}_{ij}$ is obtained from $\mathbf{A}$ by removing the ith row and the jth column, that is, those that contain the coefficient a_{ij} (Fig. 9.4).

In turn, determinants of matrices $\mathbf{M}_{i1}$, $i = 1, 2, \ldots, n$ (which are one order less than the order of the original matrix $\mathbf{A}$) can be found using the

same Eq. (9.42), but now written for matrices $\mathbf{M}_{i1}$, and so on. The process stops when cofactors become scalars (order-one matrices).

In general, any row or column may be used to find the determinant of a matrix. For sparse matrices (matrices containing the zero blocks), the best choice would be a column or row containing a maximum number of zero elements.

Obviously, the approach based on Eq. (9.42) can only be computationally effective for relatively small matrices. Luckily, the determinant holds a very useful property

$$\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B}) \tag{9.43}$$

Using this property, you may compute the determinate using the LU decomposition in the form of Eq. (9.37):

$$\det(\mathbf{A}) = \det(\mathbf{L})\det(\mathbf{U})/\det(\mathbf{P}) = \det(\mathbf{U})/\det(\mathbf{P}) \tag{9.44}$$

By construction,

$$\det(\mathbf{L}) = 1, \quad \det(\mathbf{U}) = \prod_{i=1}^{n} u_{ii} \tag{9.45}$$

and the determinant of permutation matrix $\mathbf{P}$ is either 1 for an even number of permutations or -1 for an uneven number of permutations. The built-in `det` function of MATLAB computes the determinant using exactly this latter approach.

9.3.5 Iterative Improvement of a Solution

Solving a system of LAEs with a large n involves a large number of mathematical operations. Because of round-off errors, the accuracy of a solution may degrade fairly fast. Also, you can easily loose a couple of significand figures and make your matrix $\mathbf{A}$ pretty close to be singular, which you would not expect.

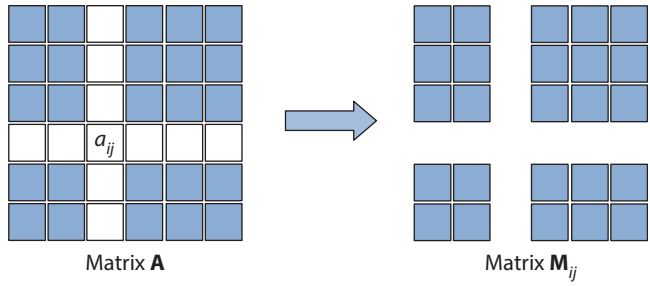


Fig. 9.4 Constructing a matrix $\mathbf{M}_{ij}$.

If it happens, there is a neat trick to restore the full precision called the *iterative improvement of a solution*. Suppose that a vector $\mathbf{x}$ is the exact solution of a linear set [Eq. (9.1)]. However, you do not know it. You only know the solution $\mathbf{x} + \delta\mathbf{x}$, which is slightly off by some unknown error $\delta\mathbf{x}$.

As a result, when multiplied by the matrix $\mathbf{A}$, this inaccurate solution yields a product, which is slightly discrepant from the desired right-hand side vector $\mathbf{b}$:

$$\mathbf{A}(\mathbf{x} + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b} \tag{9.46}$$

Subtracting Eq. (9.1) from Eq. (9.46) gives

$$\mathbf{A}\delta\mathbf{x} = \delta\mathbf{b} \tag{9.47}$$

Also, from Eq. (9.46), you may express $\delta\mathbf{b}$ as

$$\delta\mathbf{b} = \mathbf{A}(\mathbf{x} + \delta\mathbf{x}) - \mathbf{b} \tag{9.48}$$

and substitute it into Eq. (9.47), which results in

$$\mathbf{A}\delta\mathbf{x} = \mathbf{A}(\mathbf{x} + \delta\mathbf{x}) - \mathbf{b} \tag{9.49}$$

In this last equation, the right-hand side is known, because $\mathbf{x} + \delta\mathbf{x}$ is the inaccurate solution you want to improve. Hence, you may solve Eq. (9.49) for the error $\delta\mathbf{x}$ and then subtract this error from your inaccurate solution to get an improved solution.

An important extra benefit occurs if you obtained the original solution via the LU decomposition. In this case, you have the LU decomposed form of $\mathbf{A}$ already, so all you need to do is to compute the right-hand side of Eq. (9.49) and perform backward substitution.

9.4 Matrix Decompositions

The LU decomposition introduced in Sec. 9.3.3 is not the only decomposition of matrix $\mathbf{A}$. Other decompositions that may be very useful for solving the sets of LAEs are also available.

The *QR decomposition* (the `qr` function of MATLAB), which is applicable to both square and m-by-n matrices, yields

$$\mathbf{A} = \mathbf{QR} \tag{9.50}$$

where $\mathbf{Q}$ is an m-by-m orthonormal matrix (introduced and discussed in Sec. 3.3.1) and $\mathbf{R}$ is an m-by-n upper triangular matrix. The fact that $\mathbf{Q}$ is orthonormal, meaning $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$ and $\mathbf{Q}^{-1} = \mathbf{Q}^T$, allows you to rewrite Eq. (9.1) as

$$\mathbf{Rx} = \mathbf{Q}^T\mathbf{b} \tag{9.51}$$

The methods for obtaining matrices $\mathbf{Q}$ and $\mathbf{R}$ include the Gram–Schmidt process, Householder transformation, and Givens rotations. Obviously,

Eq. (9.51) is easier to solve because **R** is triangular (basically, you can proceed with backward elimination at once).

This decomposition is used in the `mldivide` (`\`) and `linsolve` functions of MATLAB for determining the effective rank of underdetermined and overdetermined cases (nonsquare matrix **A**), which will be discussed in Secs. 9.5.2 and 9.5.3. It is also used in the `polyfit` function (Sec. 11.6.1).

Eigen or *spectral decomposition* (the `eig` function of MATLAB), applicable to square matrices only, brings the original matrix **A** to

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1} \tag{9.52}$$

where $\mathbf{\Lambda}$ is a diagonal matrix

$$\mathbf{\Lambda} = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix} = \mathbf{I}[\lambda_1, \lambda_2, \ldots, \lambda_n]^T = \text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n) \tag{9.53}$$

formed by the *eigenvalues* λ_i, $i = 1, 2, \ldots, n$ of matrix **A**, and the columns $\mathbf{v}_i$, $i = 1, 2, \ldots, n$ of orthonormal matrix

$$\mathbf{V} = [\mathbf{v}_1 \quad \mathbf{v}_2 \quad \cdots \quad \mathbf{v}_n] \tag{9.54}$$

are the corresponding *eigenvectors* of **A**, satisfying the equations

$$\mathbf{A}\mathbf{v}_i = \lambda_i \mathbf{v}_i, \quad i = 1, 2, \ldots, n \tag{9.55}$$

or, in the matrix form,

$$\mathbf{AV} = \mathbf{V}\mathbf{\Lambda} \tag{9.56}$$

This decomposition is used for solving a system of linear ODEs, as will be explained in Sec. 9.6.1.

The function `eig` is also used to produce a diagonal matrix $\mathbf{\Lambda}$ and a full matrix **V**, whose columns are the corresponding eigenvectors for the generalized case

$$\mathbf{AV} = \mathbf{BV}\mathbf{\Lambda} \tag{9.57}$$

In this case, it employs the *generalized Shur decomposition* (the `qz` function), which operates on two square matrices **A** and **B** and decomposes them to

$$\mathbf{A} = \mathbf{QSZ}^* \quad \text{and} \quad \mathbf{B} = \mathbf{QTZ}^* \tag{9.58}$$

where **Q** and **Z** are orthonormal matrices (* denotes conjugate transpose), and **S** and **T** are upper triangular matrices. Then, the generalized eigenvalues

in a matrix $\mathbf{\Lambda}$ of Eq. (9.57) are the ratios of the corresponding diagonal elements of $\mathbf{S}$ to those of $\mathbf{T}$.

At last, the *singular value decomposition* (SVD), available for square and nonsquare matrices, yields

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^* \tag{9.59}$$

Here $\mathbf{\Sigma}$ is a nonnegative diagonal matrix, and $\mathbf{U}$ and $\mathbf{V}$ are orthonormal matrices. The diagonal elements of $\mathbf{\Sigma}$ are called the *singular values* of $\mathbf{A}$. Like the eigendecomposition, the SVD involves finding basis directions along which matrix multiplication is equivalent to scalar multiplication [see Eq. (9.55)]. To be more specific, SVD seeks for singular values σ, such that

$$\mathbf{A}\mathbf{v}_i = \sigma_i \mathbf{u}_i \quad \text{and} \quad \mathbf{A}^*\mathbf{u}_i = \sigma_i \mathbf{v}_i, \quad i = 1, 2, \ldots, n \tag{9.60}$$

The vectors $\mathbf{u}_i$ and $\mathbf{v}_i$, the columns of $\mathbf{U}$ and $\mathbf{V}$ in Eq. (9.59), are called the *left-singular* and *right-singular vectors* for σ_i. They are the eigenvectors of $\mathbf{A}\mathbf{A}^*$ and $\mathbf{A}^*\mathbf{A}$, respectively. The triad $\mathbf{U}$, $\mathbf{\Sigma}$, and $\mathbf{V}^*$ in Eq. (9.59) can be thought of as a set of orthonormal input basis vector directions ($\mathbf{V}$), output directions ($\mathbf{U}$), and gains multiplying each corresponding input to produce the corresponding output.

Compared with eigendecomposition, SVD has a greater generality because the matrix under consideration needs not be square. As a result, it has many applications. These include computing the pseudoinverse of a nonsquare matrix (the `pinv` function for Moore–Penrose pseudoinverse of a matrix), solving homogeneous linear equations, and evaluating range (`orth`), null space (`null`), and rank (`rank`) of matrix $\mathbf{A}$.

Specifically, the rank of matrix $\mathbf{A}$, mentioned in Sec. 9.2, defines the maximum number of linearly independent columns or rows (they are the same). The rank of an m-by-n matrix is at most $\min(m,n)$. A matrix that has a rank as large as possible is said to have a *full rank*; otherwise, the matrix is *rank-deficient*. An inhomogeneous system [Eq. (9.1)] with a square matrix yields a single solution only if the rank of matrix $\mathbf{A}$ equals n. In control theory, the rank of a matrix is used to determine whether a linear or linearized system describing vehicle or plant dynamics is controllable or observable.

The `rank` function of MATLAB computes the rank as a number of nonzero (nonsingular) elements of matrix $\mathbf{\Sigma}$. Obviously, the numerical determination requires a criterion for deciding when a value, such as a singular value from SVD, should be treated as zero. Hence, the key lines of code in the `rank` *m*-file function look as simple as

```
s = svd(A);
tol = max(size(A))*eps(max(s));
r = sum(s > tol);
```

returning the rank in `k` computed to the floating-point relative precision estimated at `max(s)`.

Most of the functions introduced in this section are the core functions of MATLAB. They are the built-in (embedded) functions relying on algorithms programmed in the LAPACK package, mentioned in Sec. 1.2. Some examples of using these functions will be provided in Sec. 9.7.1.

9.5 Special Cases of LAEs

As shown in Sec. 9.3, in the case when the determinant of matrix **A** does not equal zero (or a matrix **A** has an inverse, or it has a full rank, etc.), Eq. (9.1) with $\mathbf{b} \neq \mathbf{0}$ has a single unique solution. Examples of such a solution for the 2-D and 3-D cases are shown in Fig. 9.5.

In the 2-D case ($n = 2$), each equation describes a line, and therefore the point that belongs to both LAEs is the point where the two lines intersect (Fig. 9.5a). In the 3-D case, each equation describes a plane. Intersection of two planes produces a line, which being crossed by the third plane yields a single point belonging to all three planes (Fig. 9.5b). By the way, the script

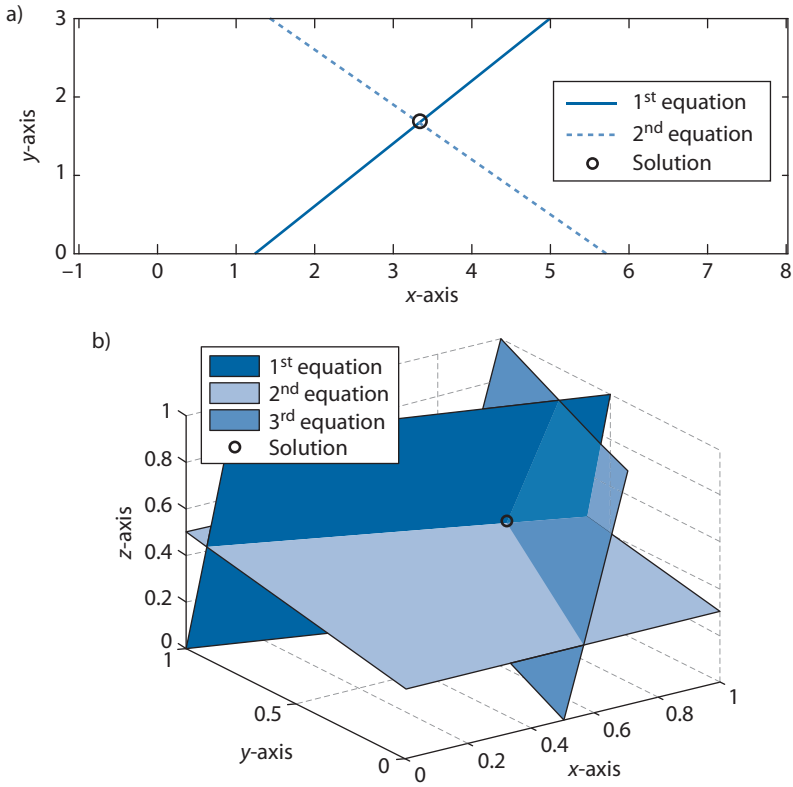


Fig. 9.5 Examples of the unique solution in the a) 2-D and b) 3-D cases.

that produced the three planes shown in Fig. 9.5b uses the `fill3` function and `FaceAlpha` property changing the transparency:

```
x=[0;1;1;0];        y=[1;0.7;0.5;0.8]; z=[0;0;1;1];
fill3(x,y,z,'b'),   hold
x=[0;1;1;0];        y=[1;1;0;0];       z=[0.5;0.5;0.3;0.3];
fill3(x,y,z,'y')
x=[0.5;0.8;1;0.7]; y=[0;1;1;0];        z=[0;0;1;1];
h=fill3(x,y,z,'g'); set(h,'FaceAlpha',0.9)
```

Even in these simple examples, you can easily think of a situation where you may have no solutions or an infinite number of solutions. On top of this, you may have a situation when you have more equations than unknowns ($m > n$) or vice versa. These cases are considered next.

9.5.1 Inconsistent Sets of LAEs

Let us change the pictorial representation of the 2-D and 3-D cases shown in Fig. 9.5 to those of Fig. 9.6. Obviously, in this case, no solution exists.

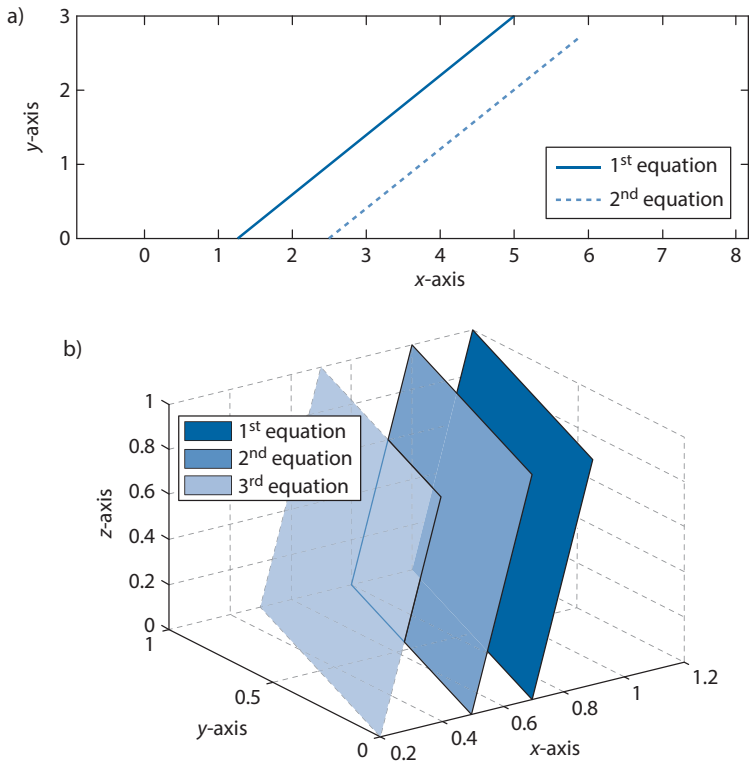


Fig. 9.6 Example of no solution in a) 2-D and b) 3-D cases.

In both cases, lines (Fig. 9.6a) or planes (Fig. 9.6b) happen to be parallel to each other.

In logic, a *consistent theory* is one that does not contain a contradiction. If we think of a solution (intersection of lines, planes, etc.) as a contradiction, then we can treat the cases shown in Fig. 9.6 as *inconsistent* sets of LAEs (the ones shown in Fig. 9.5 are *consistent* then). How does inconsistency transfer to properties of the matrix $\mathbf{A}$ and vector $\mathbf{b}$ in Eq. (9.1)?

Consider equations depicted in Fig. 9.6a:

$$\mathbf{Ax} = \begin{bmatrix} -0.8 & 1 \\ 0.2 & -0.25 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} -1 \\ -0.5 \end{bmatrix} = \mathbf{b} \tag{9.61}$$

The determinant of matrix $\mathbf{A}$ in this case equals zero (the rank equals one, not two), which indicates that two rows are linearly dependent. Indeed it is true, because the second row can be obtained by multiplying the first row by -0.25. Hence, the first indicator of possible inconsistency of a set of LAEs is a rank-deficient matrix $\mathbf{A}$.

Let us write an augmented matrix $\mathbf{A}$ for the system (9.61):

$$\begin{bmatrix} -0.8 & 1 & -1 \\ 0.2 & -0.25 & -0.5 \end{bmatrix} \tag{9.62}$$

RREF of this matrix is

$$\begin{bmatrix} 1 & -1.25 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{9.63}$$

where the last row, meaning $0x + 0y = 1$, constitutes a false statement [compare Eq. (9.63) with Eq. (9.27)]. As a result, the system (9.61) cannot possibly have a solution.

To summarize, inconsistency occurs when the rows of matrix $\mathbf{A}$ are linearly dependent but the constant terms on the right-hand side of LAEs do not satisfy the dependence relation. This case corresponds to a part of Fig. 9.1 repeated in Fig. 9.7. Examining RREF of the augmented matrix is the way to reveal this and also distinguish it from the case of underdetermined set of LAEs.

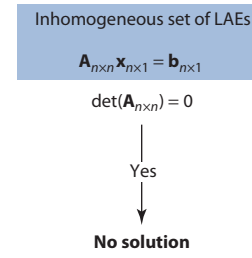


Fig. 9.7 Inconsistent system of LAEs.

9.5.2 Underdetermined Sets of LAEs

What if, in the example with the linearly dependent rows of matrix $\mathbf{A}$ considered in Sec. 9.5.1, the right-hand side of the LAEs satisfies the dependence

relation of the rows on the left-hand side as well? This case would correspond to

$$\mathbf{Ax} = \begin{bmatrix} -0.8 & 1 \\ 0.2 & -0.25 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} -1 \\ 0.25 \end{bmatrix} = \mathbf{b} \tag{9.64}$$

The determinant of matrix **A** will still be zero (the rank equals one), but RREF will look different:

$$\left[\begin{array}{cc|c} 1 & -1.25 & 1.25 \\ 0 & 0 & 0 \end{array}\right] \tag{9.65}$$

Now, there is no inconsistency in the last row (equation); and the first equation describes a line rather than a single point.

Indeed, Eq. (9.64) has an infinite number of solutions because two equations are essentially the same: the second one is the first one multiplied by −0.25. If plotted, these two equations show up as two coinciding lines (Fig. 9.8a). Hence, from the very beginning, we could have started from a

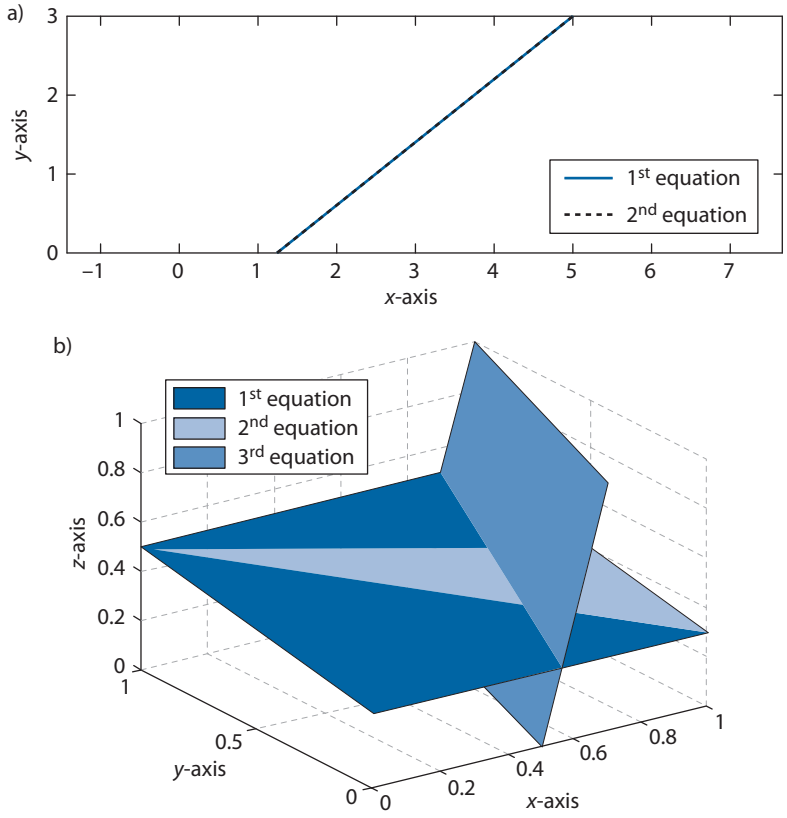


Fig. 9.8 Example of the infinite number of solutions in the a) 2-D and b) 3-D case.

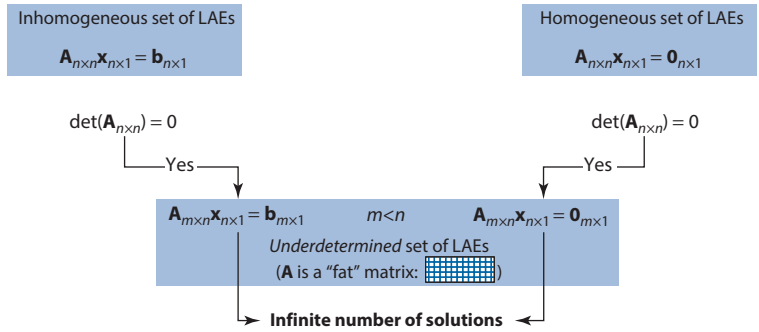


Fig. 9.9 Underdetermined systems of LAEs.

single equation with two variables

$$\mathbf{Ax} = \begin{bmatrix} -0.8 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = -1 = \mathbf{b} \tag{9.66}$$

Figure 9.8b shows another, 3-D, example when two equations (planes) are linearly dependent (coincide). The first plane may coincide with the second (as is the case in Fig. 9.8b) or with the third one; alternatively, the second plane may coincide with the third one. This results in the line of intersection of the two remaining planes to be the solution of the set of LAEs. In fact, in the 3-D case, the situation can be even worse. If all three planes coincide, all points belonging to this plane are the solutions of the degenerate system of LAEs.

The set of LAEs [Eq. (9.1)] such that $m < n$ is called *underdetermined*. You may have fewer equations than unknowns from the very beginning, or find yourself in this situation when the original square matrix $\mathbf{A}$ is degenerate (rank-deficient), as was shown for the 2-D and 3-D cases earlier. In any case, such a set of LAEs results in an infinite number of solutions. It is true for both inhomogeneous equations and homogeneous ones as shown in Fig. 9.9. The only difference is that in the latter case, all solutions will pass through the origin of the coordinate frame.

The way to write a solution for the set (9.64) [or Eq. (9.66)] is to present it parametrically. The solution space (formed by the infinite number of solutions) consists of a *particular solution* $\mathbf{x}_p$ (any point on the line) added to a solution of a homogeneous set of LAEs (obtained when $\mathbf{b} = \mathbf{0}$), $\mathbf{x}_h$ (defining a vector belonging to this line):

$$\mathbf{x} = \mathbf{x}_p + \mathbf{x}_h \tag{9.67}$$

Specifically, for Eq. (9.64), the row echelon form [Eq. (9.65)] suggests

$$x = 1.25y + 1.25 \tag{9.68}$$

If $\mathbf{b} = \mathbf{0}$, we would have

$$x = 1.25y \tag{9.69}$$

Considering y to be a *free* (independent) parameter, meaning that it can assume any value r, the solution of a homogeneous set [Eq. (9.69)] can be presented as

$$\mathbf{x}_h = r\begin{bmatrix} 1.25 \\ 1 \end{bmatrix} \tag{9.70}$$

Now, choosing, for instance, $y = 0$ and substituting it into Eq. (9.68) yields a particular solution

$$\mathbf{x}_p = \begin{bmatrix} 1.25 \\ 0 \end{bmatrix} \tag{9.71}$$

Hence, the complete solution of this underdetermined (yet consistent) set of LAEs (defining a line on the plot as shown in Fig. 9.8a) is a combination of Eqs. (9.70) and (9.71):

$$\mathbf{x} = \begin{bmatrix} 1.25 \\ 0 \end{bmatrix} + r\begin{bmatrix} 1.25 \\ 1 \end{bmatrix} \tag{9.72}$$

Consider the 3-D case, similar to that shown in Fig. 9.8b, when the set of LAEs involving three unknowns reduces to two equations

$$\mathbf{Ax} = \begin{bmatrix} 1 & 3 & 5 \\ -4 & 4 & 2 \end{bmatrix}\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 0 \\ 4 \end{bmatrix} = \mathbf{b} \tag{9.73}$$

RREF for this case is

$$\left[\begin{array}{ccc|c} 1 & 0 & 0.875 & -0.75 \\ 0 & 1 & 1.375 & 0.25 \end{array}\right] \tag{9.74}$$

Following the same routine as presented earlier and allowing z to be a free parameter, we can first write

$$\mathbf{x}_h = r\begin{bmatrix} -0.875 \\ -1.375 \\ 1 \end{bmatrix} \tag{9.75}$$

and then pick some particular solution. If we choose, say, $z = 0$, Eq. (9.74) yields $x = -0.75$ and $y = 0.25$, so that the complete parametrical solution of Eq. (9.73) can be presented as

$$\mathbf{x} = \begin{bmatrix} -0.75 \\ 0.25 \\ 0 \end{bmatrix} + r\begin{bmatrix} -0.875 \\ -1.375 \\ 1 \end{bmatrix} \tag{9.76}$$

In general, in the case when $m < n$ ($rank(\mathbf{A}) = m$), a solution of Eq. (9.1) requires $n - m$ free parameters, and so it is represented as a combination of $n - m$ vectors, which are said to be in the *null space* of matrix $\mathbf{A}$. For example, consider the case when all three planes in Fig. 9.8b coincide, so that the original set of LAEs can be reduced to a single equation, say,

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} -4 & 4 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = 4 = \mathbf{b} \tag{9.77}$$

In this case, the solution would require two free parameters because $n - m = 3 - 1 = 2$. RREF yields

$$\begin{bmatrix} 1 & -1 & -0.5 & | & -1 \end{bmatrix} \tag{9.78}$$

Choosing y and z to be two independent parameters and allowing $y = 1$, $z = 0$, then $y = 0$, $z = 1$ results in

$$\mathbf{x}_h = r \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} + s \begin{bmatrix} 0.5 \\ 0 \\ 1 \end{bmatrix} \tag{9.79}$$

The reason to choose these values for y and z is to ensure that two vectors are not linearly dependent. Picking some particular solution results in

$$\mathbf{x} = \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} + r \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} + s \begin{bmatrix} 0.5 \\ 0 \\ 1 \end{bmatrix} \tag{9.80}$$

describing the original plane [Eq. (9.77)]. Obviously, a solution representation of Eq. (9.80) is not unique and depends on the chosen basis [Eq. (9.79)]. We will elaborate on this example a little bit further, employing `svd` and `null` functions, in Sec. 9.7.1.

9.5.3 Overdetermined Sets of LAEs

What if the number of equations exceeds the number of unknowns, which corresponds to the case $m > n$ in Eq. (9.2)?

Such a system is called an *overdetermined* set of equations. Of course, we may get lucky, so that all equations still produce a single solution as shown in the examples of Fig. 9.10. In fact, this is always the case for the overdetermined homogeneous set of LAEs however, resulting in a trivial solution $\mathbf{x} = \mathbf{0}$. For overdetermined inhomogeneous systems, this would simply imply that equations are linearly dependent, so that

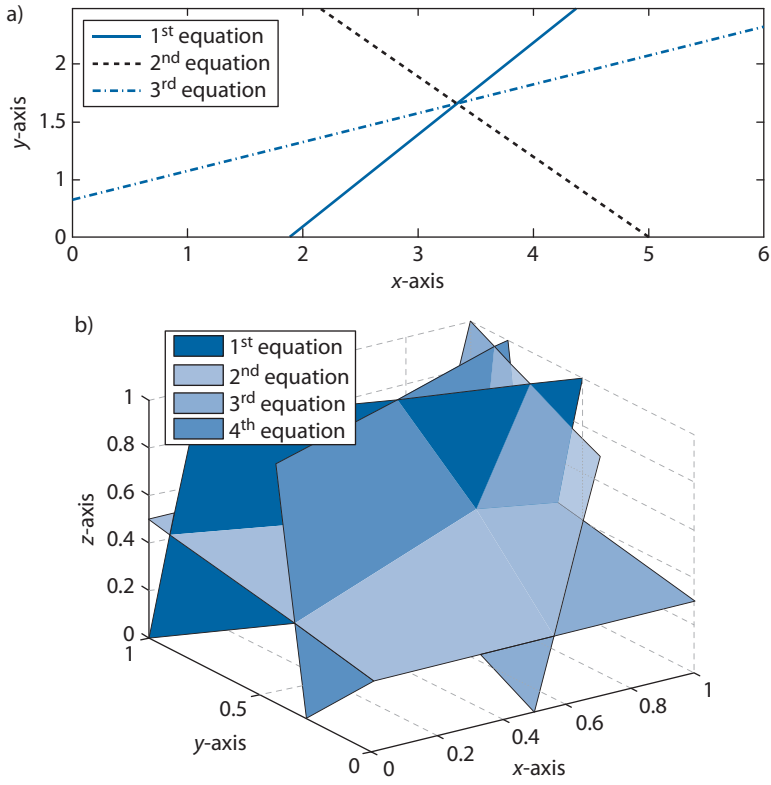


Fig. 9.10 Special case of linearly dependent equations in a) 2-D and b) 3-D.

overdetermined system can be reduced to a set of inhomogeneous LAEs with a full-rank matrix.

In the general case, though, such a system of inhomogeneous LAEs has no solution (examples are presented in Fig. 9.11), meaning that all equations do not intersect at one point. This situation is reflected in Fig. 9.12, which is the last remaining part of Fig. 9.1 that we need to discuss.

Although there is no strict solution to Eq. (9.1) in this case, Fig. 9.12 offers some kind of a compromise solution that comes closest to satisfying all equations simultaneously (depicted in Fig. 9.12 with dashed lines). This closeness can be defined in the *least-squares* sense (to be discussed in Chapter 10), meaning that we are trying to find a point that is equidistant from all hyperplanes (lines in the 2-D case and planes in the 3-D case). Mathematically, this least-squares solution $\mathbf{x}_*$ delivers a minimum value to the sum of squared differences between the left- and right-hand sides of Eq. (9.1)

$$S(\mathbf{x}) = \|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2 = (\mathbf{A}\mathbf{x} - \mathbf{b})^T(\mathbf{A}\mathbf{x} - \mathbf{b}) \tag{9.81}$$

[see definition of the norm in Eq. (3.11)], which can be expressed as

$$\mathbf{x}_* = \arg\left(\min_{\mathbf{x}} S(\mathbf{x})\right) \tag{9.82}$$

Finding a minimum means that the derivative of S with respect to $\mathbf{x}$ should equal zero. Simplifying Eq. (9.81) to

$$\begin{aligned} S(\mathbf{x}) &= (\mathbf{A}\mathbf{x}-\mathbf{b})^T(\mathbf{A}\mathbf{x}-\mathbf{b}) = (\mathbf{x}^T\mathbf{A}^T-\mathbf{b}^T)(\mathbf{A}\mathbf{x}-\mathbf{b}) \\ &= \mathbf{x}^T\mathbf{A}^T\mathbf{A}\mathbf{x}-\mathbf{b}^T\mathbf{A}\mathbf{x}-\mathbf{x}^T\mathbf{A}^T\mathbf{b}+\mathbf{b}^T\mathbf{b} \end{aligned} \tag{9.83}$$

and taking a derivative yields

$$\begin{aligned} \frac{\mathrm{d}S}{\mathrm{d}\mathbf{x}} &= \frac{\mathrm{d}}{\mathrm{d}\mathbf{x}}(\mathbf{x}^T\mathbf{A}^T\mathbf{A}\mathbf{x}-\mathbf{b}^T\mathbf{A}\mathbf{x}-\mathbf{x}^T\mathbf{A}^T\mathbf{b}+\mathbf{b}^T\mathbf{b}) \\ &= 2\mathbf{A}^T\mathbf{A}\mathbf{x}-\mathbf{A}^T\mathbf{b}-\mathbf{A}^T\mathbf{b}+\mathbf{0}=\mathbf{0} \end{aligned} \tag{9.84}$$

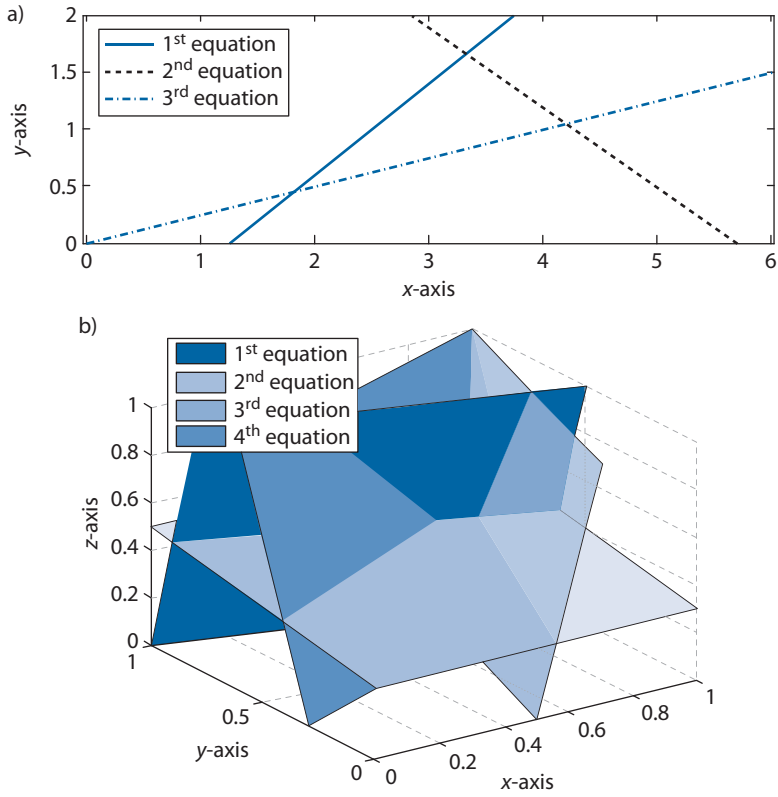


Fig. 9.11 General case of linearly independent equations in a) 2-D and b) 3-D.

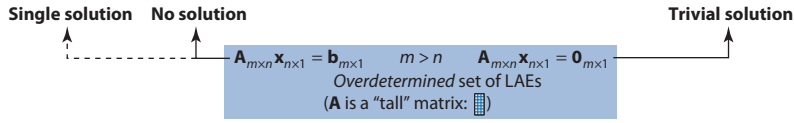


Fig. 9.12 Possible solutions for an overdetermined system of LAEs.

Equation (9.84) implies that for the column vectors, the following relations hold:

$$\frac{\mathrm{d}}{\mathrm{d}\mathbf{x}}\mathbf{A}\mathbf{x} = \mathbf{A}^T, \quad \frac{\mathrm{d}}{\mathrm{d}\mathbf{x}}\mathbf{x}^T\mathbf{A}^T\mathbf{A}\mathbf{x} = 2\mathbf{A}^T\mathbf{A}\mathbf{x}$$

Simplifying and rearranging the terms in Eq. (9.84) results in

$$(\mathbf{A}^T\mathbf{A})\mathbf{x} = \mathbf{A}^T\mathbf{b} \tag{9.85}$$

The MATLAB functions `mldivide` (`\`) and `mrdivide` (`/`) do exactly this—they reduce the set (9.1) with a nonsquare m-by-n matrix $\mathbf{A}$ to the standard set of n equations with n unknowns by premultiplying both sides with the transpose of matrix $\mathbf{A}$, $\mathbf{A}^T$. Solving Eq. (9.85) determines a least-squares solution [Eq. (9.82)].

As an example, Fig. 9.13a presents the least-squares solution (marked with a circle) for the situation of Fig. 9.11a. It also shows the distances from this solution to all three lines, the sum of squares of which [Eq. (9.81)] was minimized. Figure 9.13b shows the surface $S(\mathbf{x})$ (with the contour plot and whatever was shown in Fig. 9.13a beneath it) clearly indicating that the solution we found delivers a minimum of $S(\mathbf{x})$ [Eq. (9.82)].

9.5.4 Homogeneous Sets of LAEs and Matrix Eigenvalue Problems

In Sec. 9.3, we dealt with a set of inhomogeneous equations [Eq. (9.1)] with a square matrix $\mathbf{A}$, which has a single unique solution when $\det(\mathbf{A}) \neq 0$. Similarly, when $\det(\mathbf{A}) \neq 0$ a set of homogeneous LAEs with a square matrix $\mathbf{A}$,

$$\mathbf{A}\mathbf{x} = \mathbf{0} \tag{9.86}$$

has a single unique solution too:

$$\mathbf{x} = \mathbf{0} \tag{9.87}$$

This solution is referred to as a *trivial solution*. You may consider the 2-D and 3-D examples of Fig. 9.5 describing this trivial solution, with the only difference being that the intersection of the lines (Fig. 9.5a) or planes (Fig. 9.5b) occurs at the origin of the coordinate frame.

When equations happen to be linearly dependent (homogeneous equations are always consistent), that is, when $\det(\mathbf{A}) = 0$, an infinite number of solutions exist, similar to the cases shown in the examples of Fig. 9.8. Again, the only difference is that the intersection of hyperplanes (lines in the 2-D case and planes in the 3-D case) always involves the $\mathbf{x} = \mathbf{0}$ point. The solution then can be represented parametrically, similar to those of Eqs. (9.72), (9.76), and (9.80), without a particular solution (which in the case of homogeneous LAEs is simply $\mathbf{x}_p = \mathbf{0}$).

Let us introduce the *eigenvalue equation*

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x} \tag{9.88}$$

where $\lambda = \text{const}$. This equation has a lot of applications. For example, if a matrix $\mathbf{A}$ represents a linear transformation, such as rotation, reflection, and stretching, Eq. (9.88) seeks for such a nonzero vector $\mathbf{x}$, called an *eigenvector*, that its direction is not changed by the transformation $\mathbf{A}$. The only effect this transformation has on the eigenvector is that it is scaled by a

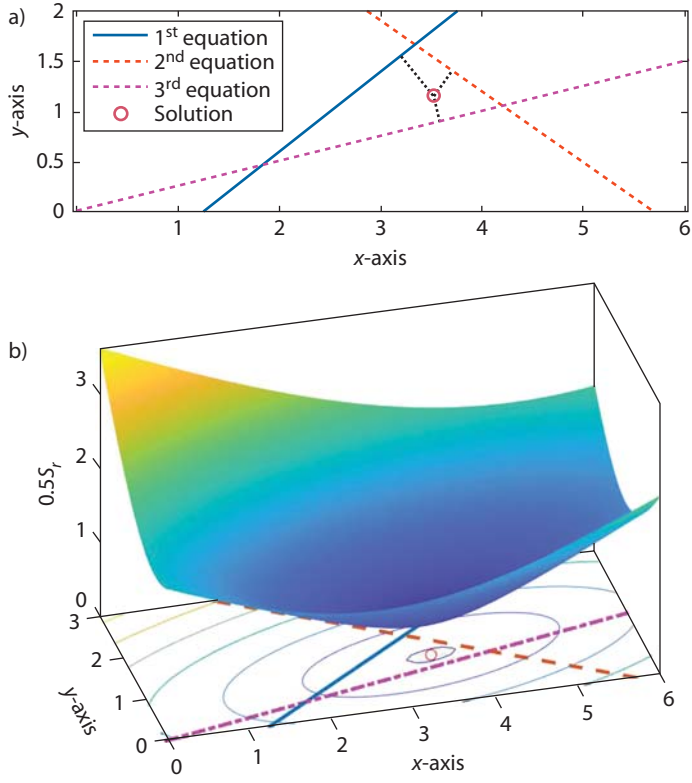


Fig. 9.13 a) Least-squares solution and b) $S(\mathbf{x})$ surface.

factor of λ, called the *eigenvalue* of $\mathbf{A}$ (corresponding to the eigenvector $\mathbf{x}$, because such a solution is not unique). We can rewrite Eq. (9.88) in the form of a standard set of homogeneous LAEs

$$\mathbf{Ax} - \lambda\mathbf{Ix} = (\mathbf{A} - \lambda\mathbf{I})\mathbf{x} = \mathbf{0} \tag{9.89}$$

where the identity matrix $\mathbf{I}$ is used to be able to combine λ with the matrix $\mathbf{A}$. In the scalar form, Eq. (9.89) looks as follows:

$$\begin{aligned}
(a_{11} - \lambda)x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= 0 \\
a_{21}x_1 + (a_{22} - \lambda)x_2 + \cdots + a_{2n}x_n &= 0 \\
&\vdots \\
a_{n1}x_1 + a_{n2}x_2 + \cdots + (a_{nn} - \lambda)x_n &= 0
\end{aligned} \tag{9.90}$$

By now, we know that Eq. (9.90) has a nontrivial solution only if the determinant of its matrix is zero:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = \begin{vmatrix} a_{11} - \lambda & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} - \lambda & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} - \lambda \end{vmatrix} = 0 \tag{9.91}$$

This leads to the *characteristic equation*

$$f(\lambda) \equiv \det(\mathbf{A} - \lambda\mathbf{I}) = 0 \tag{9.92}$$

which is a polynomial of order n. The eigenvalues are the roots of Eq. (9.92). Once these eigenvalues are determined, the nontrivial solution to the set of homogeneous equations [Eq. (9.89)] is found for each eigenvalue, as was shown earlier in Eq. (9.55). Typically, all eigenvectors are written as one matrix $\mathbf{V}$ [Eq. (9.54)], called the *eigenvector matrix* or *modal matrix*. The collection of all eigenvectors of matrix $\mathbf{A}$ forms a *fundamental set of solutions*.

A unique feature of an eigenvalue problem is the *similarity transformation*, which follows from Eq. (9.56) and ties the original matrix $\mathbf{A}$, modal matrix $\mathbf{V}$, and eigenvalues matrix $\mathbf{\Lambda}$ [Eq. (9.54)] together, so that

$$\mathbf{V}^{-1}\mathbf{AV} = \mathbf{\Lambda} \tag{9.93}$$

(by construction, eigenvectors $\mathbf{v}_i$, $i = 1, 2, \ldots, n$ are linearly independent, and therefore the inverse of modal matrix $\mathbf{V}$ exists).

9.6 Applying Linear Algebra To Solve Systems of ODEs

Many principles of physics, engineering, and other sciences are described in terms of ODEs, that is, equations involving variables and their derivatives. Different methods for solving ODEs numerically along with the

corresponding MATLAB functions will be introduced in Chapter 14. The goal of this section is to show how the solution developed for the problem of Sec. 9.5.4 can be applied to solve linear ODEs analytically. It is the *Laplace transform* that makes it possible to handle linear ODEs as LAEs. The following section shows a similarity between solving a single linear ODE and a set of the first-order linear ODEs, and then extends it to the case of multiple ODEs taking advantage of the state–space representation.

9.6.1 Treating a System of the First-Order ODEs as a Single Equation

Consider a single homogeneous first-order ODE

$$\dot{x} = ax \tag{9.94}$$

where a is a constant and $x = x(t)$ is an unknown function to be determined. Applied to ODEs, "homogeneous" means that there are no terms except those involving the function and its derivative. Equation (9.94) is ordinary because it involves only one independent variable and is referred to as being of the first order because of the highest order of a derivative.

As is well known, the solution of Eq. (9.94) can be expressed in the following form:

$$x(t) = e^{at}c \tag{9.95}$$

where c is a constant. This constant is defined by an *initial condition* (IC)

$$x(t_0) = x_0 \tag{9.96}$$

so that

$$c = e^{-at_0}x_0 \tag{9.97}$$

If $t_0 = 0$, Eq. (9.97) reduces to

$$c = x_0 \tag{9.98}$$

Because every solution of Eq. (9.94) must be of the form of Eq. (9.95), the latter one is called the *general solution*.

We can also obtain the solution (9.95) by applying the Laplace transform to Eq. (9.94), yielding

$$sX(s) = aX(s) \quad \text{or} \quad (a - s)X(s) = 0 \tag{9.99}$$

(At this point, simply assume that this transform replaces the function $x(t)$ with some function $X(s)$ and the derivative $x(t)$ with $X(s)$ multiplied by a constant s.)

Equation (9.99) resembles Eq. (9.89), so that you may follow the routine of Sec. 9.5.4 and find the (only) eigenvalue $s = a$ and corresponding eigenvector $v = 1$ (or any constant). Therefore, we can express the general solution

[Eq. (9.95)] using the eigenvalue–eigenvector pair as

$$x(t) = ve^{st}c \tag{9.100}$$

The constant c is defined by the IC at $t_0 = 0$ as

$$c = v^{-1}x_0 \tag{9.101}$$

so that the complete solution becomes

$$x(t) = ve^{st}v^{-1}x_0 \tag{9.102}$$

In the case of n first-order linear homogeneous ODEs with constant coefficients instead of Eqs. (9.94) and (9.96), we have

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} \tag{9.103}$$

and

$$\mathbf{x}(t_0) = \mathbf{x}_0 \tag{9.104}$$

where $\mathbf{A}$ is a square matrix of coefficients (let us call it a *state matrix*) and $x_i,\ i = 1, 2, \ldots, n$ are the elements of an unknown vector to be determined. Applying the Laplace transform to Eq. (9.103) yields

$$s\mathbf{X}(s) = \mathbf{A}\mathbf{X}(s) \quad \text{or} \quad (\mathbf{A} - s\mathbf{I})\mathbf{X}(s) = \mathbf{0} \tag{9.105}$$

which is a vector analog of Eq. (9.99). Because of that, the general solution of Eqs. (9.103) and (9.104) can be written using exactly the same pattern as in Eq. (9.100), using the corresponding matrices and vectors instead of scalars:

$$\mathbf{x}(t) = \mathbf{V}e^{\mathbf{\Lambda}t}\mathbf{c} = \mathbf{v}_1 e^{\lambda_1 t}c_1 + \mathbf{v}_2 e^{\lambda_2 t}c_2 + \cdots + \mathbf{v}_n e^{\lambda_n t}c_n \tag{9.106}$$

In this equation,

$$e^{\mathbf{\Lambda}t} = \mathbf{I}\left[e^{\lambda_1 t}, e^{\lambda_2 t}, \ldots, e^{\lambda_n t}\right]^T = \text{diag}(e^{\lambda_1 t}, e^{\lambda_2 t}, \ldots, e^{\lambda_n t})$$

(compare with Eq. (9.53)) where $\lambda_1, \lambda_2, \ldots, \lambda_n$ are eigenvalues of matrix $\mathbf{A}$ (we denote them as λ's instead of s's), and $\mathbf{c} = [c_1, c_2, \ldots, c_n]^T$ is a vector of constants determined by n ICs, similar to that of Eq. (9.101):

$$\mathbf{c} = \mathbf{V}^{-1}\mathbf{x}_0 \tag{9.107}$$

Hence, the complete solution can be written in the form similar to that of Eq. (9.102)

$$\mathbf{x}(t) = \mathbf{V}e^{\mathbf{\Lambda}t}\mathbf{V}^{-1}\mathbf{x}_0 \tag{9.108}$$

To summarize, knowing eigenvalues and eigenvectors of the state matrix $\mathbf{A}$ defining the set of the first-order linear ODEs [Eq. (9.103)] allows you to find its general solution [Eq. (9.106)]. Given ICs

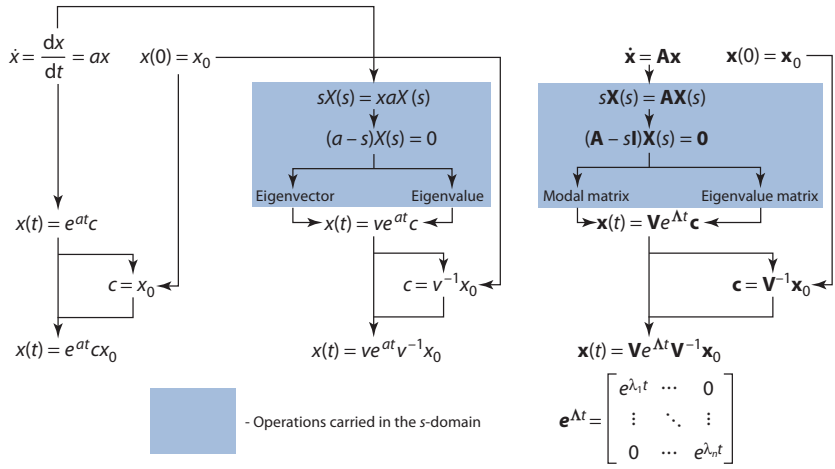


Fig. 9.14 Similarity between solving a single differential equation and a set of first-order differential equations.

[Eq. (9.104)], the unknown coefficients in Eq. (9.106) can be determined using Eq. (9.107). Figure 9.14 shows the logic of this solution side by side with a solution of a single ODE. Section 9.7.2 will show a couple of practical examples of employing the MATLAB functions to solve a system of linear ODEs numerically, but before we proceed, let us show how higher-order differential equations can be brought to the form of Eq. (9.103).

9.6.2 State–Space Representation of ODEs

In practice, when deriving equations driving dynamics of some mechanical system, you usually end up with a set of the higher-order differential equations (involving higher-order derivatives) as opposed to a single first-order ODE or several ODEs written in the form of Eq. (9.103). The question then is how to reduce these higher-order ODEs to the compact form of Eq. (9.103), called the *state–space form* representation of ODEs?

Recall the two-body mass–spring system shown in Fig. 7.3 and the system of differential equations governing its dynamics in Eq. (7.7). These equations can be rewritten as

$$\begin{aligned} \ddot{y}_1 &= -\frac{k_1 + k_2}{m_1} y_1 + \frac{k_2}{m_1} y_2 \\ \ddot{y}_2 &= \frac{k_2}{m_2} y_1 - \frac{k_2}{m_2} y_2 \end{aligned} \tag{9.109}$$

constituting a system of two second-order equations, which we would like to bring to the form of Eq. (9.103).

Let us introduce the *state vector* **x**:

$$\mathbf{x} = [x_1, x_2, x_3, x_4]^T = [y_1, \dot{y}_1, y_2, \dot{y}_2]^T \tag{9.110}$$

The number of states is defined by the total order of a system, which in the case of Eq. (9.109) totals to $2 \times 2 = 4$. Taking the derivative of the state vector with account of Eq. (9.109) yields

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \\ \dot{x}_4 \end{bmatrix} = \begin{bmatrix} \dot{y}_1 \\ \ddot{y}_1 \\ \dot{y}_2 \\ \ddot{y}_2 \end{bmatrix} = \begin{bmatrix} \dot{y}_1 \\ -\dfrac{k_1+k_2}{m_1}y_1 + \dfrac{k_2}{m_1}y_2 \\ \dot{y}_2 \\ \dfrac{k_2}{m_2}y_1 - \dfrac{k_2}{m_2}y_2 \end{bmatrix} = \begin{bmatrix} x_2 \\ -\dfrac{k_1+k_2}{m_1}x_1 + \dfrac{k_2}{m_1}x_3 \\ x_4 \\ \dfrac{k_2}{m_2}x_1 - \dfrac{k_2}{m_2}x_3 \end{bmatrix} = \mathbf{A}\mathbf{x} \tag{9.111}$$

where the *state matrix* **A** is

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -\dfrac{k_1+k_2}{m_1} & 0 & \dfrac{k_2}{m_1} & 0 \\ 0 & 0 & 0 & 1 \\ \dfrac{k_2}{m_2} & 0 & -\dfrac{k_2}{m_2} & 0 \end{bmatrix} \tag{9.112}$$

Employing the findings of Sec. 9.6.1, we can write a solution of this equation at once:

$$\mathbf{x}(t) = \mathbf{V}e^{\mathbf{\Lambda}t}\mathbf{V}^{-1}[y_1(0), \dot{y}_1(0), y_2(0), \dot{y}_2(0)]^T \tag{9.113}$$

9.7 Using the MATLAB Functions

This section first discusses a variety of functions that allow you to solve systems of LAEs. It then proceeds with showing how an ability to solve LAEs helps in resolving linear ODEs.

9.7.1 Solving Systems of LAEs

MATLAB features several functions allowing you to address Eq. (9.1). These functions are listed in Table 9.2.

The primary method to solve a set of LAEs is to use the left matrix division operator or backslash, `\`. The syntax of this built-in function is as simple as

```
x=A\b
```

If `A` is a full-rank matrix (which is determined using the QR factorization) and `b` is a vector (having the same number of rows as `A`), then the `mldivide` function `(\)` returns a single solution of an inhomogeneous set of LAEs using the Gaussian elimination. The partial pivoting is used to reduce a possibility of singularities.

If `A` is an m-by-n matrix with $m \neq n$ and `b` is a column vector with m components, or a matrix with several such columns, then `x = A\b` returns a solution in the least-squares sense. It means that for an underdetermined or overdetermined system, instead of attempting to solve the original system of equations `A*x=b` with a nonsquare matrix `A`, MATLAB tries to solve the system `A'*A*x=A'*b` [Eq. (9.85)], where `A'` stands for the transpose of matrix `A`, and `A'*A` is therefore a square matrix.

The QR factorization (performing the orthogonal–triangular decomposition of matrix `A`) can be used for both square and rectangular matrices explicitly:

```
[Q,R]=qr(A); y=Q'*b; x=R\y
```

The right matrix division operator, `/`, or `mrdivide` is a very short function, which simply calls the `mldivide` function. The correct syntax is

```
x=mrdivide(b', A')'  or  x=(b'/the A')'
```

Gauss–Jordan elimination in MATLAB is accomplished with the `rref` function, for which the simplest syntax is

```
R=rref(Aa)
```

where `Aa=cat(2,A,b)` is an augmented matrix [`A`|`b`]. The aforementioned call returns RREF of `Aa` in `R`, with the last column being the solution of equation `A*x=b`. Be aware that the round-off errors may cause problems in computing RREF of a matrix. Partially, the reason for this is that the `rref` function tests for negligible column elements using some internally computed tolerance of `max(size(A))*eps*norm(A,inf)`.

Table 9.2 Functions for Solving Systems of LAEs

Function	Brief Description
`x=mldivide(A,b)` or `x=A\b`	Uses Gaussian elimination with a partial pivoting to find a solution to `A*b=b`
`y=rref([A b]); x=y(:,end)`	Employs Gauss–Jordan elimination with a partial pivoting
`x=linsolve(A,b)`	Relies on LU factorization with a partial pivoting

You may, however, use another syntax of the `rref` function and define the tolerance `tol` explicitly:

```
R=rref(Aa,tol)
```

Once RREF is found, the unique solution (in the case of nonsingular square matrix `A`) can be found as

```
x=y(:,end)
```

The `inv` function computes the inverse of a square matrix. Hence, this function may also be employed to solve Eq. (9.1) as

```
x=inv(A)*b
```

In the general case (for $n > 3$), the algorithm of `inv` relies on the LU decomposition as well as the algorithm of the `linsolve` function, which can be called as

```
x=linsolve(A,b)
```

The LU decomposition is performed with the `lu` function. The general syntax is

```
[L,U,P]=lu(A)
```

It returns the lower triangular matrix `L`, upper triangular matrix `U`, and permutation matrix `P` (an identity matrix with the rows shuffled), so that `P*A=L*U`. The alternate syntax of the `lu` function is

```
[Lp,U]=lu(A)
```

which creates an upper triangular matrix in `U` and a "pseudolower triangular matrix" in `Lp`, so that `A=Lp*U` (`Lp=inv(P)*L`). This pseudotriangular matrix `Lp` is a product of lower triangular and permutation matrices and therefore may not look like a lower triangular matrix.

Among other functions introduced in this chapter, the `det` function

```
d=det(A)
```

computes the determinant of a square matrix using the LU decomposition

```
d=sign(det(P))*prod(diag(U))
```

The `svd` function with a general syntax

```
[U,S,V] = svd(A)
```

performs SVD. The singular values of matrix **A** are returned as the diagonal elements of matrix `S`. These values can also be obtained using a simpler syntax of `svd`, `s=svd(A)`, returning them in the vector `s`.

SVD is used to determine the rank of a matrix via the `rank` function

```
r=rank(A)
```

This call simply returns the number of singular values of `A` that are larger than some internally defined tolerance (based on the maximum singular

value of matrix A). The SVD algorithm is the most time consuming, but also the most reliable. The rank function also allows you to specify the tolerance explicitly:

```
r=rank(A,tol)
```

Finally, the null function

```
Z=null(A)
```

returns an orthonormal basis for the null space of A obtained from SVD as well.

Consider several examples. Let us define the matrix A and vector b

```
>> A=[2 1 1; 4 -6 0; -2 7 2]; b=[5; -2; 9]; n=length(b)
```

To start with, the following script shows what a piece of code that realizes forward elimination of the Gauss method may look like:

```
Ab=[A b]                              % display the original augm. matrix
i=1; j=1;
while i<=n                            % loop over the entire matrix
% Execute partial pivoting (row swapping)
 for k=i+1:n
 l=A(k,j)/A(i,j);                     % compute multiplier
 A(k,j:n)=A(k,j:n) -l*A(i,j:n);       % subtract pivot row
 b(k)=b(k)-l*b(i);
 Ab=[A b]                             % display augm. matrix at every step
 end
i=i+1; j=j+1;
end
```

This code is pretty straightforward, following derivations of Sec. 9.3.1, and for your convenience displays every step as an augmented matrix [A b], so that the sequential elimination of the three leading terms in the matrix A along with the changes in the vector b show up as follows:

$$\left[\begin{array}{cccc} 2 & 1 & 1 & 5 \\ 4 & -6 & 0 & -2 \\ -2 & 7 & 2 & 9 \end{array}\right] \left[\begin{array}{cccc} 2 & 1 & 1 & 5 \\ 0 & -8 & -2 & -12 \\ -2 & 7 & 2 & 9 \end{array}\right] \left[\begin{array}{cccc} 2 & 1 & 1 & 5 \\ 0 & -8 & -2 & -12 \\ 0 & 8 & 3 & 14 \end{array}\right] \left[\begin{array}{cccc} 2 & 1 & 1 & 5 \\ 0 & -8 & -2 & -12 \\ 0 & 0 & 1 & 2 \end{array}\right]$$

By the way, the determinant of the original matrix and the upper triangular matrix happens to be the same:

```
>> d=det(A)
d = -16
```

The aforementioned code does not involve pivoting, but you can easily add it by replacing the comment

```
% Execute partial pivoting (row swapping)
```

with the following:

```
% Find the value and index of the largest element in column j
[p,m]=max(abs(A(i:n,j))); m=m+i-1;
A([i m],j:n)=A([m i],j:n);b([i m])=b([m i]); % swap the i-th and m-th rows
```

In this case, at the very first step, the first line will be swapped with the second one, so that the elimination process develops as follows:

$$\left[\begin{array}{cccc} 2 & 1 & 1 & 5 \\ 4 & -6 & 0 & -2 \\ -2 & 7 & 2 & 9 \end{array}\right] \left[\begin{array}{cccc} 4 & -6 & 0 & -2 \\ 0 & 4 & 1 & 6 \\ -2 & 7 & 2 & 9 \end{array}\right] \left[\begin{array}{cccc} 4 & -6 & 0 & -2 \\ 0 & 4 & 1 & 6 \\ 0 & 4 & 2 & 8 \end{array}\right] \left[\begin{array}{cccc} 4 & -6 & 0 & -2 \\ 0 & 4 & 1 & 6 \\ 0 & 0 & 1 & 2 \end{array}\right]$$

Because of this swapping, the determinant of a final matrix changes its sign:

```
>> d=det(A)
d = 16
```

The backward substitution process realizing Eqs. (9.15)–(9.17) can then be coded as

```
x(n)=b(n)/A(n,n);                    % compute x(n)
 for k=n-1:-1:1
 b(1:k)=b(1:k)-A(1:k,k+1)*x(k+1); % substitute x(k+1) to all rows above k
 x(k)=b(k)/A(k,k);                   % compute x(k)
 end
x'
```

Continuing the aforementioned example (that is, applying this latter code to the lower triangular matrix that we ended up with after the forward elimination process), we get

```
x = 1
    1
    2
```

Let us now define the augmented matrix to be used to demonstrate the essence of the Gauss–Jordan elimination process described in Sec. 9.3.2. For instance, we are interested in finding the inverse of the `magic(3)` matrix. Then, we define

```
>> A=[magic(3) eye(3)]; [m,n]=size(A);
```

The piece of code realizing the Gauss–Jordan elimination (for the general case of a nonsquare matrix) is essentially a combination of the aforementioned two pieces of the Gaussian elimination:

```
A                                % display the original augmented matrix
i=1; j=1;
while i<=m && j<=n               % loop over the entire matrix
% Find the value and index of the largest element in column j
[p,k]=max(abs(A(i:m,j))); k=k+i-1;
% if p<=tol
```

```
% A(i:m,j)=zeros(m-i+1,1);   % zero out the negligible column
% j=j+1;
% else
  A([i k],j:n) = A([k i],j:n); % swap the i-th and m-th rows
  A(i,j:n) = A(i,j:n)/A(i,j); % divide the pivot row by the pivot element
for k=[1:i-1 i+1:m]
A(k,j:n)=A(k,j:n)-A(k,j)*A(i,j:n); % subtract pivot row from all other rows
end
A            % display the results after finishing each column
 i= i+1; j=j+1;
% end
end
```

That is exactly what appears in the MALTAB `rref` function. The commented lines use internally defined or supplied tolerance `tol` to zero the negligibly small elements. Also note how the `k=i` line is skipped in the `for` loop. Applied to our matrix `A`, this code returns

```
A = 8   1   6   1   0   0
    3   5   7   0   1   0
    4   9   2   0   0   1
A = 1.0000 0.1250  0.7500  0.1250       0         0
         0 4.6250  4.7500 −0.3750  1.0000         0
         0 8.5000 −1.0000 −0.5000       0    1.0000
A = 1.0000       0  0.7647  0.1324       0 −0.0147
         0 1.0000 −0.1176 −0.0588       0  0.1176
         0       0  5.2941 −0.1029  1.0000 −0.5441
A = 1.0000       0       0  0.1472 −0.1444  0.0639
         0 1.0000       0 −0.0611  0.0222  0.1056
         0       0 1.0000 −0.0194  0.1889 −0.1028
```

The last three columns of the final return constitute `inv(magic(3))`.

Next, let us show an example of employing the `lu` function. Consider some random matrix

```
>> A=randi(9,4)
A = 8   1   8   1
    9   2   2   2
    5   8   9   6
    1   2   3   4
```

In this case, the full syntax of the `lu` function

```
>> [L,U,P]=lu(A)
```

returns all three matrices as explained in Sec. 9.3.3

```
L = 1.0000         0       0       0
    0.5556    1.0000       0       0
    0.8889   −0.1129  1.0000       0
```

```
    0.1111    0.2581    0.1043    1.0000
U = 9.0000    2.0000    2.0000    2.0000
         0    6.8889    7.8889    4.8889
         0         0    7.1129   -0.2258
         0         0         0    2.5397
P =   0    1    0    0
      0    0    1    0
      1    0    0    0
      0    0    0    1
```

You can visualize the results using the `spy` and `imagesc` functions as discussed in Sec. 3.2.4. The following script produces the 3-by-2 grid shown in Fig. 9.15:

```
subplot(321), spy(L)
subplot(323), spy(U)
subplot(325), spy(P)
subplot(322), imagesc(L), colorbar
subplot(324), imagesc(U), colorbar
subplot(326), imagesc(P), colorbar
```

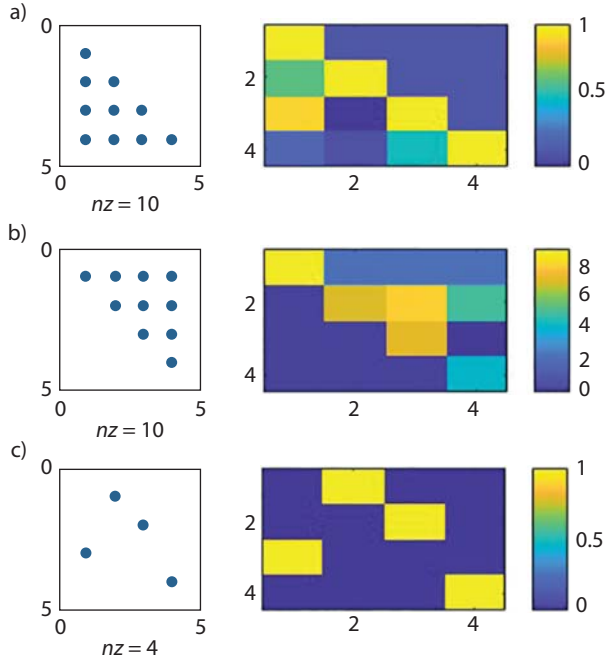


Fig. 9.15 Visualization of the a) **L**, b) **U** and c) **P** matrices after LU decomposition of a 4-by-4 matrix.

You can also request to see the result of partial pivoting in the "vector" form, that is, explicitly showing the order of equations

```
>> [~,~,P] = lu(A,'vector')
P =   2   3   1   4
```

which in this particular case means that for better accuracy the second and third rows were moved up.

Another syntax

```
>> [Lp,U]=lu(A)
```

yields

```
Lp = 0.8889   -0.1129   1.0000         0
     1.0000         0        0         0
     0.5556    1.0000        0         0
     0.1111    0.2581   0.1043    1.0000
U =  9.0000    2.0000   2.0000    2.0000
          0    6.8889   7.8889    4.8889
          0         0   7.1129   -0.2258
          0         0        0    2.5397
```

with the `Lp` matrix being a combination of `L` and `P` matrices, so that `Lp = inv(P)*L`.

Finally, the simplest syntax

```
>> lu(A)
```

returns matrices `L` and `U` in a single matrix stored according to Eq. (9.36):

```
ans =
    9.0000    2.0000    2.0000    2.0000
    0.5556    6.8889    7.8889    4.8889
    0.8889   -0.1129    7.1129   -0.2258
    0.1111    0.2581    0.1043    2.5397
```

Let us now compare the performance of different functions allowing you to solve the set of LAEs. We define some random matrix `A` and vector `b`

```
>> A=rand(1000); b=rand(1000,1);
```

and apply the set of commands examining different options:

```
tic,    x1=mldivide(A,b);                          toc
tic,    x2=mrdivide(b',A')';                       toc
tic,    y=rref([A b]); x3=y(:,end);                toc
tic,    [l,u]=lu(A); c=l\b; x4=u\c;                toc
tic,    [q,r]=qr(A); y=q'*b; x5=r\y;               toc
tic,    x6=linsolve(A,b);                          toc
tic,    x7=inv(A)*b;                               toc
[norm(A*x1-b); norm(A*x2-b); norm(A*x3-b); norm(A*x4-b); ...
              norm(A*x5-b); norm(A*x6-b); norm(A*x7-b)]
```

Table 9.3 Comparison of Performance of Different MATLAB Functions to Solve the System of LAEs

Method	Elapsed Time, s	Accuracy ($\times 10^{-10}$)
/	0.035 (1.1)	0.048 (7.9)
\	0.061 (2.0)	0.025 (4.2)
rref	7.605 (250.4)	0.027 (4.4)
lu	0.030 (1.0)	0.053 (8.9)
qr	0.078 (2.6)	0.006 (1.0)
linsolve	0.034 (1.1)	0.048 (7.9)
inv	0.054 (1.8)	0.082 (13.7)

Specifically, we are interested in the CPU time required to solve the problem and the attained accuracy of solution. A typical output is shown in Table 9.3. Numbers in the parentheses show a relative increase compared with the smallest value in each column.

As you can see, the QR decomposition features the best accuracy, and that is why it is used to estimate the rank of a matrix for the Gaussian elimination. It takes more time, though, to compute the solution. On the contrary, the `inv` function takes about the same CPU time, but features the worst accuracy. The `rref` function seems to be the worst from the standpoint of required CPU resources. On the positive side, this function provides more information about the solution, especially in the cases of inconsistent or underdetermined system. Let us demonstrate it.

Consider an inconsistent set of LAEs

```
>> A=[1 1; -1 -1]; b=[2; 3];
```

Obviously, the rows of matrix `A` are linearly dependent. We can prove it, for example, by employing the `det` and `rank` commands:

```
>> d=det(A), r=rank(A)
d = 0
r = 1
```

Any of the following calls

```
x=A\b
x=(b'/A')'
x=linsolve(A,b)
x=inv(A)*b
```

returns

```
Warning: Matrix is singular to working precision.
x = -Inf
     Inf
```

The `rref` call

```
>> x=rref([A b])
```

allows you to really understand what is going on:

```
x = 1  1  0
    0  0  1
```

Specifically, the last row suggests that the system is inconsistent (Sec. 9.5.1).

Now, consider an underdetermined system of LAEs

```
>> A=[1 1]; b=[0];
```

Any of the calls

```
x=A\b
x=(b'/A')'
x=linsolve(A,b)
```

returns a particular solution

```
x = 0
    0
```

whereas the call

```
>> s=rref([A b])
```

reveals

```
s = 1  1  0
```

allowing you to define a general solution parametrically as, say, $\mathbf{x} = r[-1,\ 1]^T$ (see Sec. 9.5.2).

Let us modify the aforementioned example to the inhomogeneous system:

```
>> A=[1 1]; b=[2];
```

Any of the calls

```
x=A\b
x=(b'/A')'
x=linsolve(A,b)
```

returns a particular solution

```
x = 2
    0
```

whereas the call

```
>> s=rref([A b])
```

returns

```
s = 1  1  2
```

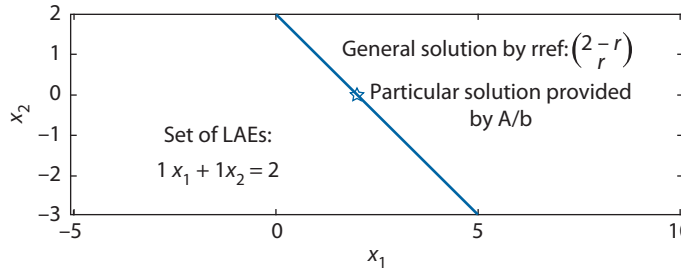


Fig. 9.16 Example of an undetermined system of LAEs.

which may be used to establish a general solution as $\mathbf{x} = [2, 0]^T + r[-1, 1]^T$. The following script reproduces this solution as shown in Fig. 9.16:

```
A=[1 1]; b=[2];
x1=0:5; x2=(b-A(1)*x1)/A(2);      % define the equation
plot(x1,x2), hold
xs=A\b;                           % define a particular solution
plot(xs(1),xs(2),'rp')
xlabel('x_1'), ylabel('x_2'), axis equal
s=rref([A b]); xh(2)=1; xh(1)=-s(2)*xh(2); xp(2)=0; xp(1)=s(3);
% Display the set of LAEs, general and particular solutions
text(-2.8,-1,'Set of LAEs:')
f='%+2.0f'; t1=' x_1 '; t2=' x_2 = ';
text(-3,-2,[num2str(A(1)) t1 num2str(A(2),f) t2 num2str(b)],...
     'fontsize',10)
text(-xh(1)+0.3,xh(2),'General solution by \it rref\rm :')
syms r, x=xp'+r*xh'; H=latex(x);
text(6.7,1,['$$' H '$$'],'interpreter','latex','fontsize',14)
text(xs(1)+0.3,xs(2),'Particular solution provided by \bf A\rm\\\bf b')
```

Obviously, for nonsquare matrices, the `inv` function fails to produce a solution:

```
x=inv(A)*b
Error using inv
Matrix must be square.
```

Consider another undetermined system

```
>> A=[1  2  3; 2  5  1]; b=[1; 2];
```

Any of the calls

```
x=A\b
x=(b'/A')'
x=linsolve(A,b)
```

returns a particular solution

```
x = 0
    0.3846
    0.0769
```

Employing

```
>> s=rref([A b])
s =  1   0    13   1
     0   1   −5   0
```

allows you to find a general solution

```
>> xh(3)=1; xh(1:2)=-s(1:2,3)*xh(3), xp(3)=0; xp(1:2)=s(1:2,4)
xh = −13   5    1
xp =   1   0    0
```

that is, $\mathbf{x} = [1, 0, 0]^T + r[-13, 5, 1]^T$.

Let us normalize the latter solution of homogeneous system

```
>> xh=xh/norm(xh)
xh = −0.9309
      0.3581
      0.0716
```

and employ SVD (Sec. 9.4):

```
>> [U,S,V]=svd(A)
U = −0.5145   −0.8575
    −0.8575    0.5145
S =  6.2450         0         0
          0    2.2361         0
V = −0.3570    0.0767   −0.9309
    −0.8513    0.3835    0.3581
    −0.3845   −0.9204    0.0716
```

The results are visualized in Fig. 9.17.

As seen, the last column of matrix `V` corresponding to the third column of matrix `S` (with two zero elements in it) contains the solution of the homogeneous system that matches that of the normalized vector `xh`. To retrieve just this vector, we can also employ the `null` function

```
>> null(A)
```

resulting in

```
ans = −0.9309
       0.3581
       0.0716
```

Let us see how this works for another undetermined system, which has three linearly dependent rows. Consider the set described by Eq. (9.77):

```
>> A = [-4 4 2]; b=[4];
```

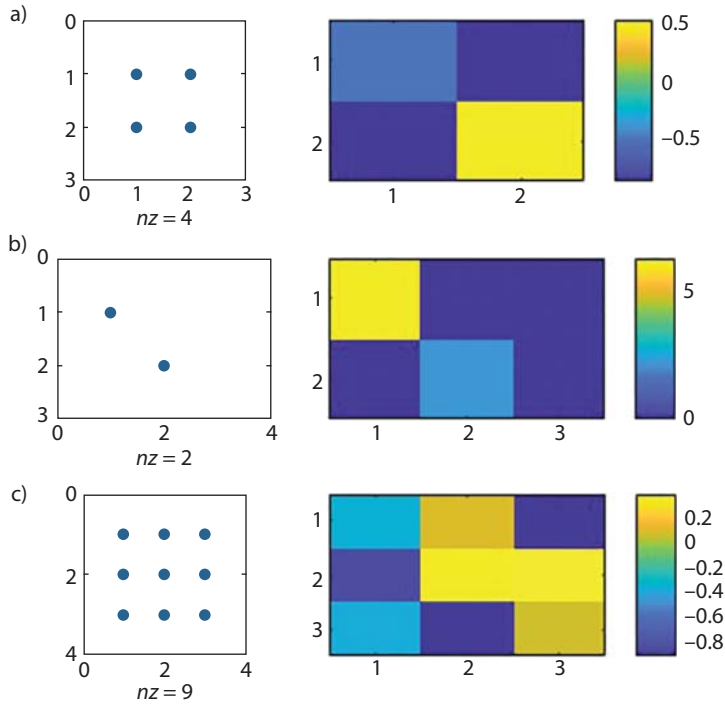


Fig. 9.17 Visualization of the a) **U**, b) $\boldsymbol{\sigma}$ and c) **V** matrices after SVD of a 2-by-3 matrix.

The rank of matrix `A` in this case equals one:

```
>> r = rank(A)
r = 1
```

and the `rref` function returns what was shown in Eq. (9.78):

```
>> rref([A b])
ans = 1.0000   −1.0000   −0.5000   −1.0000
```

from which we derived the solution shown in Eq. (9.80). The solution of the homogeneous set of LAEs, which in this case represents a plane, can be described by any two noncollinear vectors belonging to this plane. We choose these two vectors to be [1;1;0] and [05;0;1] [Eq. (9.79)]. Chosen quite arbitrarily, these vectors are not orthogonal:

```
>> [1 1 0]*[0.5;0; 1]
ans =  0.5
```

Now let us examine the corresponding returns of the `svd` and `null` functions:

```
>> [U,S,V]=svd(A)
U = 1
S = 6  0  0
```

```
V = -0.6667    0.6667    0.3333
     0.6667    0.7333   -0.1333
     0.3333   -0.1333    0.9333
>> xh=null(A)
xh =  0.6667     0.3333
      0.7333    -0.1333
     -0.1333     0.9333
```

The two basis vectors suggested by these two functions belong to the same plane as those of Eq. (9.79), but on top of that happen to be orthonormal:

```
>> xh(:,1)'*xh(:,2)
ans = 0
```

Being complemented with a particular solution

```
>> xp=A\b
xp = -1
      0
      0
```

they constitute a general solution in the following form:

$$\mathbf{x} = \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} + r \begin{bmatrix} \frac{2}{3} \\ \frac{11}{15} \\ \frac{-2}{15} \end{bmatrix} + s \begin{bmatrix} \frac{1}{3} \\ \frac{-2}{15} \\ \frac{14}{15} \end{bmatrix} \tag{9.114}$$

(we used the `format rat` call to come up with these ratios). The bottom line is that you can use the `rref` function to examine the underdetermined system and then conveniently obtain a general solution using the backslash operator `\` (for a particular solution) and `null` function (for the solution of the homogeneous set of LAEs).

In the case of the overdetermined system (Sec. 9.5.3), for instance,

```
>> A=[2  1; 2  5; 4  4]; b=[1; 2; 3];
```

the `rref` function returns no result

```
>> x=rref([A b])
x = 1  0  0
    0  1  0
    0  0  1
```

whereas any of other functions

```
x=A\b
x=(b'/A')'
x=linsolve(A,b)
[l,u]=lu(A); c=l\b; x4=u\c
[q,r]=qr(A); y=q'*b; x5=r\y
x=inv(A'*A)*A'*b
```

Table 9.4 Examples of Possible Responses of LAE-Solving Functions in Different Situations

	Single solution	Inconsistency	Infinite number of solutions		Least-squares solution
Example	`A=[ 1 3; -3 2]; b=[11;0];`	`A=[ 1 1; -1 -1]; b=[2;3];`	`A=[ 1 1; -1 -1]; b=[2;-2];`	`A=[ 1 1 2; -1 -1 -2]; b=[2;-2];`	`A=[2 1; 2 5; 4 4]; b=[1;2;2];`

`mldivide(A,b)` or `A\b`
`mrdivide(b',A')'` or `(b'/A')'`
`linsolve(A,b)`
	`ans = 2 3`	**Matrix is singular**
		`ans = -Inf Inf`
			Matrix is singular
			`ans = NaN NaN`
				Rank deficient
				`ans = 0 0 1`
					`ans = 0.2500 0.2857`

`rref([A b])`	`ans = 1 0 2 0 1 3`	**Matrix is singular** `ans = 1 1 0 0 0 1`	**Matrix is singular** `ans = 1 1 2 0 0 0`	`ans = 1 1 2 2 0 0 0 0`	`ans = 1 0 0 0 1 0 0 0 1`
`inv(A)*b`	`ans = 2 3`	**Matrix is singular** `ans = -Inf Inf`	**Matrix is singular** `ans = NaN NaN`	**Error using inv** **Matrix must be square**	

produces the least-squares solution

```
x = 0.2500
    0.2857
```

To summarize, when using any of the LAE-solving functions presented in this chapter, it is important to understand the output you are getting. You might think that you are getting a single unique solution, whereas in reality, you are getting a particular solution (one of the infinite number of solutions) or a least-squares solution. To this end, Table 9.4 is intended to help you understand where you are in terms of the tree of possible situations depicted in Fig. 9.1 based on MATLAB's response. You should create a habit of checking properties of the state matrix **A**, trying a couple of different functions, and plugging your solution back to the equation to estimate the residual errors.

For the sake of building up your programming skills, let us conclude this section with an example of code employing SVD for image compression. This code first reads the RGB image shown in the upper portion of Fig. 9.18a, degrades it to grayscale, and then executes SVD. The S matrix is composed of 1045 singular values (SVs) shown in Fig. 9.18b. Then, all but the very first (largest) 21 singular values are zeroed, and a new image, shown in the bottom portion of Fig. 9.18a, is constructed:

```
imIn=imread('Cirrus SR22.jpg');
imGr=rgb2gray(imIn);
imDbl=double(imGr);
%% Decompose the image using singular value decomposition
[U,S,V]=svd(imDbl);
Ns=min(size(S));            % Total number of the singular values
figure
semilogy(diag(S)), grid, axis tight, hold
xlabel('SV sequence number'), ylabel('SV magnitude')
```

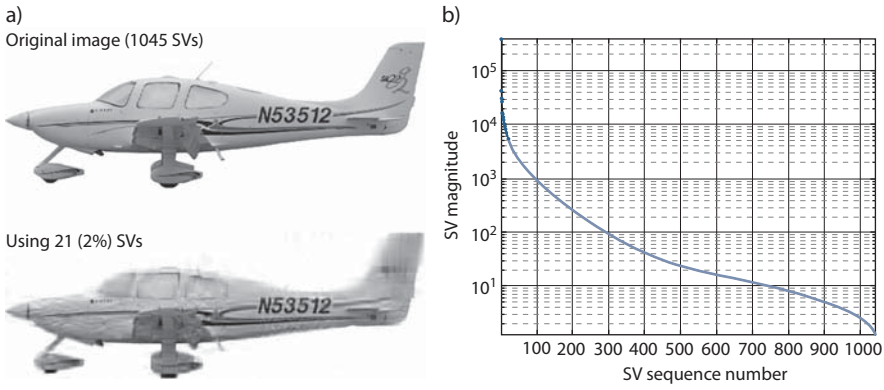


Fig. 9.18 Demonstration of information carried by the top 2% of singular values.

```
%% Use subset of singular values (diagonal of S) to compress the image
N=21;              % Consider only the first N singular values,
Np=N/Ns*100;       % which is only Np percent of all N singular values
semilogy(diag(S(1:N,1:N)),'r.')% Show N singular values only
C=S;               % Store the singular values in a temporary variable
C(N+1:end,:)=0; % Discard the diagonal values not required for compression
D=U*C*V';          % Construct an image using the selected singular values
%% Plot the results
figure
subplot(211), imshow(imGr)
text(80,100,['Original image (' int2str(Ns) 'SVs)'],'color','b')
subplot(212), imshow(uint8(D))
text(80,100,['Using ' int2str(N) '(' num2str(Np,2) '%) SVs'],'color','b')
```

9.7.2 Solving Eigenvalue Problems and Linear Differential Equations

The major function of MATLAB specifically dedicated to address the eigenvalue problem is `eig`. The simplest call

```
lambda=eig(A)
```

returns the vector `lambda` containing eigenvalues of matrix `A`. For example,

```
>> A=magic(3); eig(magic(3))
```

yields

```
ans =  15.0000
        4.8990
       -4.8990
```

An alternate syntax for the `eig` function is

```
[V,D]=eig(A)
```

which returns eigenvectors of matrix `A` in the modal matrix `V` and eigenvalues in the diagonal matrix `D`, for example,

```
>> [V,D]=eig(A)
V = -0.5774   -0.8131   -0.3416
    -0.5774    0.4714   -0.4714
    -0.5774    0.3416    0.8131
D = 15.0000         0         0
         0    4.8990         0
         0         0   -4.8990
```

Eigenvectors are normalized, but not necessarily orthonormal:

```
>> [norm(V(:,1)) norm(V(:,2)) norm(V(:,3))]
ans = 1  1  1
>> [V(:,1)'*V(:,2) V(:,2)'*V(:,3) V(:,3)'*V(:,1)]
ans = -0.0000  0.3333    0
```

The similarity transformation of Eq. (9.93) does hold, but you should be careful examining it, because the straight comparison

```
>> A*V==V*D
```

yields a confusing result

```
ans = 0   0   0
      0   0   0
      0   0   0
```

The more robust approach

```
>> A*V-V*D
ans = 1.0e-014 *
       0.1776  0.4885  0.0222
       0.3553  0.0444  0.0444
      -0.1776  0.1332  0.1776
```

explains the results obtained with the straightforward approach—the equality holds to a degree of the round-off errors.

In principle, you may obtain these results using other functions of MATLAB. For educational purposes, let us use an alternative (maybe clearer) approach and then compare it with that of the `eig` function. Let us start by employing the `poly` function to obtain coefficients of the characteristic polynomial $|\mathbf{A}-\lambda\mathbf{I}|$:

```
>> p = poly(A)
p = 1.0000  -15.0000  -24.0000  360.0000
```

Then, you can find its roots (eigenvalues) using the `roots` function (to be discussed in Chapter 10):

```
>> lambda=roots(p)
lambda =   15.0000
           -4.8990
            4.8990
```

These eigenvalues happen to be the same as mentioned earlier, but in a slightly different order. Next, you may want to find eigenvectors by solving the corresponding system of LAEs. The best way would be employing the `rref` function

```
>> rr=rref([A-lambda(1)*eye(3)])
```

which returns RREF of matrix $\mathbf{A}-\lambda\mathbf{I}$

```
rr = 1   0   -1
     0   1   -1
     0   0    0
```

Following the routine outlined Sec. 9.5.2, you may choose the third element to be a free parameter, which results in an eigenvector

```
>> v=-rr(:,3); v(3)=1
v = 1
    1
    1
```

If normalized

```
>> v=v/norm(v)
```

it coincides with that obtained using the aforementioned `eig` function (it points to the opposite direction, which is all right):

```
v = −0.5774
    −0.5774
    −0.5774
```

Doing the same operations for all three eigenvalues

```
for i=1:3
rr=rref([A-lambda(i)*eye(3)]);
v=-rr(:,3); v(3)=1;
V(:,i)=v/norm(v);
end
```

returns

```
V = 0.5774  −0.3416 −0.8131
    0.5774  −0.4714   0.4714
    0.5774   0.8131   0.3416
```

which is the same as the result returned by the `eig` function, but corresponding to a different order of eigenvalues. However, although the results are similar, the specially developed `eig` function does computations thousands of times faster than an alternative algorithm. To this end, the following script employs the `tic–toc` pair introduced in Sec. 5.11 to compare both approaches:

```
A=magic(5); n=length(A);
tic, [V,D]=eig(A); toc     % performance of the eig function
tic
p=poly(A); lambda=roots(p);
for i=1:n
rr=-rref([A-lambda(i)*eye(n)]);
v=rr(:,n); v(n)=1; V(:,i)=v/norm(v); D=diag(lambda);
end
toc                        % performance of the alternative approach
```

This script returns something like

```
Elapsed time is 0.000079 seconds.
Elapsed time is 0.033833 seconds.
```

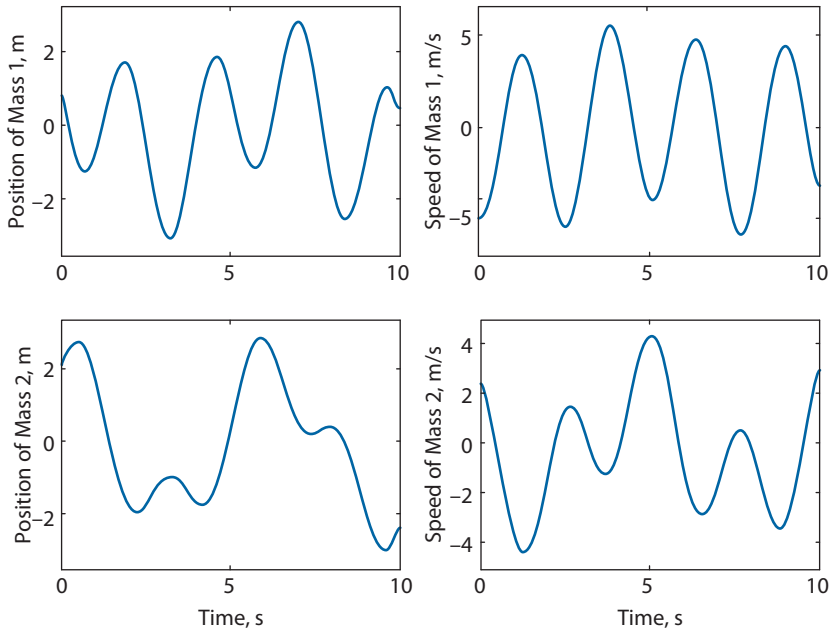


Fig. 9.19 Dynamics of the two-body mass–spring system of Fig. 7.3.

You may see some different results on your computer, but on average, the `eig` function performs much faster.

Finally, let us use the `eig` function to solve the system of linear ODEs introduced in Eqs. (9.111) and (9.112), representing the spring–mass system of Fig. 7.3, using the same numerical values for its parameters and ICs as we used to produce Fig. 7.4. The following script performs eigendecomposition of the state matrix in full accordance with Eq. (9.113) and shows the results in Fig. 9.19

```
% Define system parameters  in kg and kg/s/s
m1=1; m2=1; k1=3; k2=2;
% Define the initial conditions in m and m/s
y0=[1, 2]'; ydot0=[-2*sqrt(6), sqrt(6)]';
% Form the state matrix
A=[0 1 0 0; -(k1+k2)/m1 0 k2/m1 0; 0 0 0 1; k2/m2 0 -k2/m2 0];
% Solve linear ODEs using eigendecomposition of matrix A
[V,D]=eig(A);
c=inv(V)*[y0(1) ydot0(1) y0(2) ydot0(2)]';
% Develop a symbolic (analytical) representation of solution
syms t
x=V*diag(diag(exp(D*t)))*c;
subplot(2,2,1), fplot(x(1),[0 10]), ylabel('Position of Mass 1, m')
subplot(2,2,2), fplot(x(2),[0 10]), ylabel('Speed of Mass 1, m/s')
```

```
subplot(2,2,3), fplot(x(3),[0 10]), ylabel('Position of Mass 2, m')
xlabel('Time, s')
subplot(2,2,4), fplot(x(4),[0 10]), ylabel('Speed of Mass 2, m/s')
xlabel('Time, s')
```

Obviously, the results match those of Fig. 7.4. The only difference is that in this case the MATLAB Symbolic Math Toolbox has difficulty simplifying symbolic expressions for components of the state vector **x** to those shown as the plot titles in Fig. 7.4.

Problems

9.1 Consider the following set of LAEs:

$$\begin{aligned} 3x_3 + 5x_4 &= b_1 \\ 3x_1 + 7x_2 - 4x_3 + 5x_4 &= b_2 \\ 5x_1 - 6x_2 + 7x_3 + 8x_4 &= b_3 \\ 4x_2 + 5x_4 &= b_4 \end{aligned}$$

where b_i, $i = 1,2,3,4$ are the uniformly distributed pseudorandom numbers (use the `b=rand(4,1)` command). Solve it employing all appropriate functions introduced in this chapter, that is, `\`, `inv`, `rref`, `linsolve`, `lu`, and `qr`. Compare the results and check one of solutions (any of them) by substituting it back into the system of LAEs. Comment on the results.

9.2 One of the methods to solve an inhomogeneous system of LAEs that we did not consider in this chapter is Cramer's rule. The solution is expressed explicitly in terms of determinants of the n-by-n (square) coefficient matrix `A` and matrices obtained from `A` by replacing one column by the vector of the right-hand-side parts of the equations, `b`:

$$x_i = \frac{\det(\tilde{\mathbf{A}}_i)}{\det(\mathbf{A})}, \quad i = 1, 2, \ldots, n$$

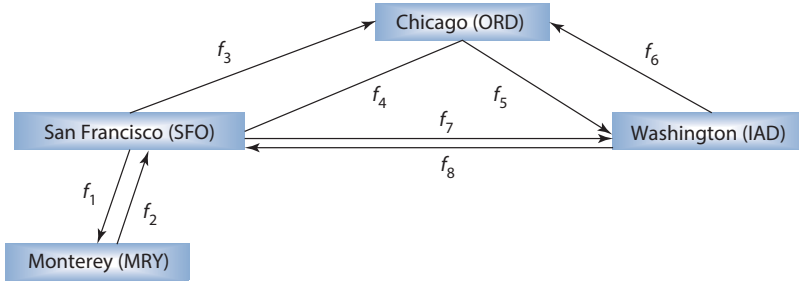
where

$$\tilde{\mathbf{A}}_i = \begin{bmatrix} a_{11} & \cdots & a_{1,i-1} & b_1 & a_{1,i+1} & \cdots & a_{1n} \\ a_{21} & \cdots & a_{2,i-1} & b_2 & a_{2,i+1} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ a_{n1} & \cdots & a_{n,i-1} & b_n & a_{n,i+1} & \cdots & a_{nn} \end{bmatrix}$$

Although Cramer's rule is important theoretically, for large matrices, it has little practical value because the computation of large determinants is somewhat cumbersome and not precise. Develop the m-file function

realizing this algorithm and compare its performance with that of the Gauss elimination method.

9.3 Consider the following simple scheme of transporting cargo between four airports:



Suppose you know that for some specific day, the total cargo handled in Monterey (incoming and outgoing) was 100 conditional units; in San Francisco—600; in Chicago—800; and in Washington, DC—600. In addition to this, you know that $f_1 = f_2$, whereas the difference between incoming and outgoing cargo in Washington was 100 units. Finally, you know that 200 units of cargo were transported from Washington to Chicago. Using eight unknown variables, f_1 through f_8, develop equations describing this traffic. Determine whether you have enough independent equations to find the unique solution. If not, add additional reasonable observation(s) to come up with the feasible results (so that $f_i > 0$, $i = 1, 2, \ldots, 8$). Vary your observation to see how sensitive to these changes your solution is.

Chapter 10 Root Finding and Introduction to Optimization

- Utilizing Bracketing and Open Methods for Root Finding
- Searching Extremum of a Single-Objective Function
- Using the MATLAB Functions

`fzero`, `roots`, `fminbnd`, `fminsearch`, `fminunc`, `fmincon`, `optimset`, `optimoptions`

10.1 Introduction

Suppose you need to find a root (a zero) of a scalar function of a single variable,

$$f(x) = 0 \tag{10.1}$$

For some cases, the roots of Eq. (10.1) can be obtained analytically (for instance, one can easily solve a quadratic equation); for others—an approximate solution technique is needed (for example, for complex algebraic functions, not analytically defined functions, transcendental or nonalgebraic equations). MATLAB offers two basic functions to address these types of problems. The `fzero` function is intended to find a single root of a continuous function of one variable, and the `roots` function finds all roots of a polynomial. Specifically, the `fzero` function uses the so-called *zeroin algorithm*, exploiting several numerical techniques based on two different approaches. That is why this chapter starts with introducing them. The first approach, the so-called *bracketing methods*, requires two initial guesses obeying certain properties, and the second one, the *open methods*, requires only one initial guess. The chapter proceeds with explaining the zeroin algorithm and addressing several root-finding-related problems, followed by introducing an algorithm behind the `roots` function. In the case of a vector equation, when both x and f in Eq. (10.1) are vectors rather than scalars, MATLAB offers the `fsolve` function.

It turns out that the function minimization (maximization) problem

$$x_* = \arg\left(\min_x g(x)\right) \tag{10.2}$$

is closely related to the root-finding problem of Eq. (10.1). Indeed, solving Eq. (10.2) implies finding the solution of Eq. (10.1) with $f(x) = g'(x)$, where $g'(x) = \mathrm{d}g/\mathrm{d}x$ is the derivative of function $g(x)$. Therefore in general, you can still use the `fzero` function to address Eq. (10.2). However, in addition to this, MATLAB offers a variety of other tools to deal with Eq. (10.2)-type problems, that is, tools to solve standard and large-scale optimization problems. These tools are combined into the Optimization Toolbox and the even more advanced (specialized) Global Optimization Toolbox (Table 1.2). Several basic functions from the first toolbox are available to users of the basic MATLAB core and, therefore, are also introduced in this chapter. They are `fminbnd` to minimize a function of one variable on a fixed interval, `fminsearch` to minimize a function of several unconstrained variables, `fminunc` to find a minimum of an unconstrained multivariable function, and `fmincon` to find a minimum of a constrained multivariable function. These functions developed to address the problem of Eq. (10.2) can be used to solve Eq. (10.1) as well because the latter one is, in general, an equivalent to

$$x_* = \arg\left(\min_x f^2(x)\right) \tag{10.3}$$

For the sake of completeness, this chapter presents a variety of methods for solving Eq. (10.1), followed by the discussion of several basic techniques to address Eq. (10.2).

10.2 Bracketing Methods

To find a solution of Eq. (10.1), two groups of methods can be employed. This and the following chapters introduce them one after another. The *bracketing methods*, introduced in this chapter, exploit the fact that the function $f(x)$ typically changes a sign in the vicinity of its root x_* (see the example in Fig. 10.1). Bracketing means that two initial guesses are required. These guesses must "bracket" or be on either side of the root (for example, $x = a$ and $x = c$ in Fig. 10.1 constitute a valid bracket). With each iteration, bracketing methods systematically reduce the width of the bracket and, hence, home in on the correct answer. Therefore, these methods guarantee unconditional convergence of numerical algorithm. This is the main advantage of bracketing methods. Moreover, given a required tolerance ε, the number of iterations necessary to find an approximate solution can easily be estimated. Methods considered in this chapter are the bisection, false position, and golden section.

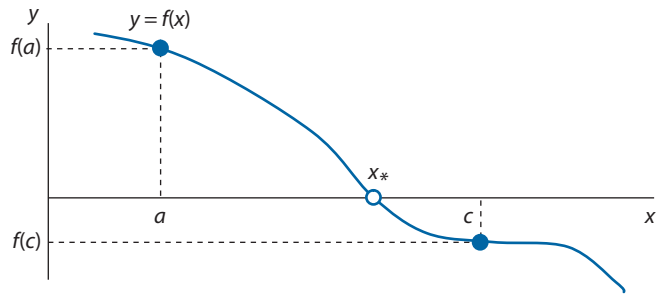


Fig. 10.1 Typical geometry in the close vicinity of a single-variable function root.

10.2.1 Bisection Method

The *bisection method* is the simplest and most robust method for root finding (if the root is localized within the initially assumed interval). The bisection method relies on the principle that if an interval $[a;c]$ has one root, then the sign of $f(x)$ at $x = a$ and $x = b$ are opposite to each other, or either $f(a)$ or $f(c)$ equals zero. Mathematically, this can be represented as

$$f(a)f(c) \leq 0 \tag{10.4}$$

The numerical algorithm looks as follows (for better understanding, several consecutive steps are shown graphically in Fig. 10.2):

1. Given $f(x)$, find an initial interval enclosing the root (like $[a;c]$ in Fig. 10.2a) so that the inequality of Eq. (10.4) holds.
2. Find a trial point (Figs. 10.2b, 10.2d, and 10.2f) as

$$x = b = \frac{1}{2}(a + c) = \frac{1}{2}a + \frac{1}{2}c \tag{10.5}$$

3. If the size of the new interval $[a;b]$ is less than or equal to the predefined tolerance ε

$$|a - b| \leq \varepsilon \tag{10.6}$$

   stop iterations, assuming the solution at $x_* = b$.
4. If Eq. (10.6) does not hold, continue iterations by choosing the correct interval between the two available ones ($[a;b]$ or $[b;c]$)
   - If inequality $f(a)\,f(b) \leq 0$ holds, then assign $c = b$ (Fig. 10.2e).
   - Otherwise, assign $a = b$ (Fig. 10.2c)
5. Go to Step 2 with a new value of c or a and produce the next iteration with a halved interval.

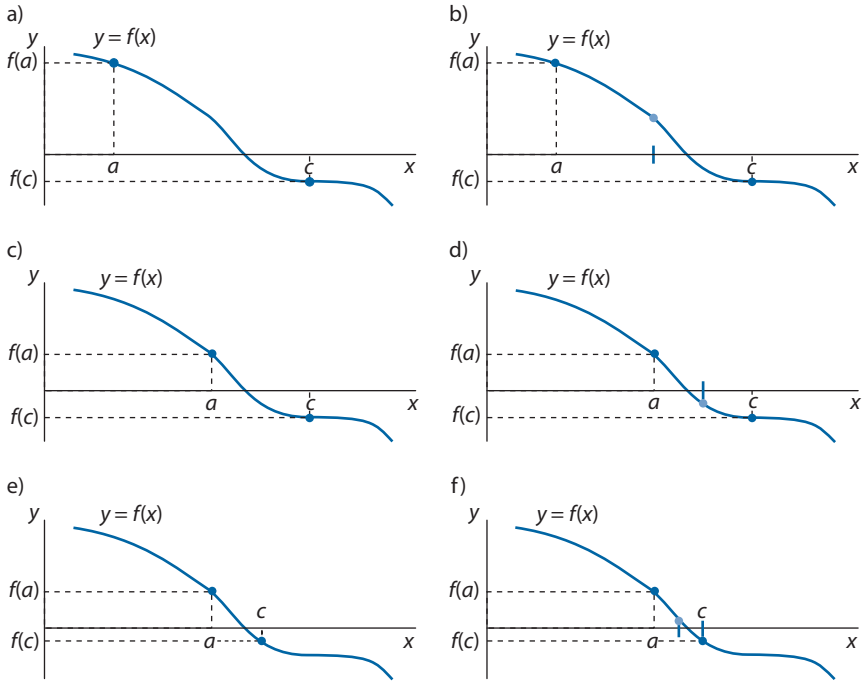


Fig. 10.2 Illustration of several consecutive steps using the bisection method.

Because you are halving the original interval $[a_0;c_0]$ at each iteration, exiting the algorithm after n iterations means

$$\frac{|c_0 - a_0|}{2^n} \leq \varepsilon \tag{10.7}$$

This obvious relation allows you to have an exact number of iterations needed to satisfy the tolerance ε

$$n \geq \frac{\ln \frac{|c_0 - a_0|}{\varepsilon}}{\ln(2)} \tag{10.8}$$

[The greater or equal sign in Eq. (10.8) means that to produce an integer number, the right-hand-side expression should be rounded toward infinity.]

Given the bounds of the initial root bracketing interval, x_0 and x_1, function $f(x)$ developed as an anonymous/inline function an m-file, and relative tolerance ε^{rel} [see Eq. (8.28)], the bisection method can be programmed as

```
a=x0; c=x1;
if f(a)*f(c)<=0
  while abs(c-a)>EPS*max(abs(c),1e-3) % meet the relative tolerance
  b=(a+c)/2;                          % halve the interval
```

```
if f(a)*f(b)<=0
  c=b;
else
  a=b;
  end
end
disp([b,f(b)])                          % display the results
else
warndlg('The interval does not enclose the root','!! Warning !!')
end
```

Once again, the first step is very important. It helps establish the correct initial interval and bracket (isolate) the correct root you are trying to find. If you are not careful in choosing the initial interval, you may enclose a singularity instead of (or in addition to) the root, as in the example shown in Fig. 10.3. That is why plotting your function, even with the easiest `fplot` or `ezplot` function, should be your first check.

Suppose you overlooked a singularity and picked an interval like the one shown in Fig. 10.3 that still satisfies Eq. (10.4). The bisection method will still work and produce the result, in our case converging to a singularity. How can you possibly recognize this? Well, it is a good idea to do a second check examining whether

$$f(b) \to 0 \tag{10.9}$$

That will tell if the method converged to a singularity rather than a root.

Obviously, any bracketing method relying on Eq. (10.4) cannot find a root if the function $f(x)$ only touches the abscissa axis, not crossing it (an example is shown in Fig. 10.4). In this case, the root-finding problem can be recast as an extremum-finding problem [Eq. (10.2)] (even without squaring the function) and solved using the minimum-finding algorithms (Sec. 10.7 discusses this issue further).

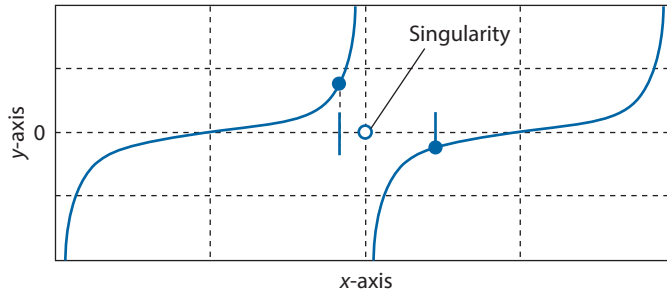


Fig. 10.3 Example of singularity.

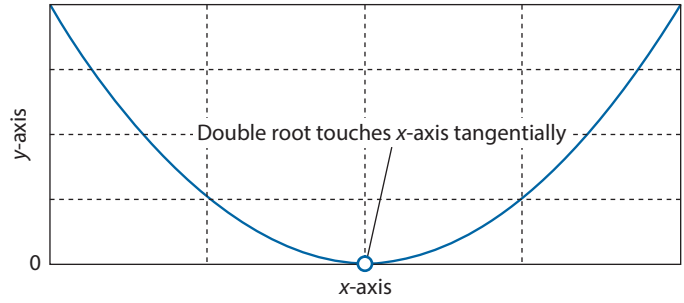


Fig. 10.4 Example of an abscissa-touching function.

Finally, bracketing methods may not be applied to finding the complex roots because of the obvious reason that the criterion for defining a bracket (that is, a sign change) does not translate to the complex domain.

10.2.2 False-Position Method

Although the bisection method is a perfectly valid technique for determining a root, its "brute-force" approach is relatively inefficient. A simple modification of Eq. (10.5) allows you to reduce the number of iterations necessary to find the root.

A major shortcoming of the bisection method is that, in dividing the current interval into equal halves, no account is taken of magnitudes of $f(a)$ and $f(c)$. The easiest way to take a magnitude into account, and therefore improve convergence performance, is to join $f(a)$ and $f(c)$ by a straight line and consider the intersection of this line with the x-axis as the next trial point, as shown in Fig. 10.5. Then, the new trial point b can be determined as

$$b = c - \frac{f(c)(a-c)}{f(a)-f(c)} = \frac{-f(c)}{f(a)-f(c)}a + \frac{f(a)}{f(a)-f(c)}c = w_a f(a) + w_c f(c) \tag{10.10}$$

now featuring adaptive unequal coefficients as opposed to two fixed equal coefficients of Eq. (10.5). Equation (10.10) constitutes the *false-position formula*. According to the numerical routine outlined in the previous section, the new point b replaces either of the two initial guesses, a or c, to satisfy Eq. (10.4). As expected, weighting coefficients w_a and w_c ($w_a + w_c = 1$) are now proportional to the closeness of $f(c)$ and $f(a)$ to the x-axis, respectively (if $|f(a)| < |f(c)|$, then $w_a > w_c$ and otherwise).

Given the bounds of the initial root-bracketing interval, x_0 and x_1, function $f(x)$ developed as an anonymous/inline function or m-file, and relative tolerance ε^{rel}, the false-position method could be programmed as

```
a=x0; c=x1;
if f(a)*f(c)<=0
while abs(c-a)>EPS*max(abs(c),1e-3) % meet the relative tolerance
wa=-f(c)/(f(a)-f(c)); wc=1-wa;      % compute weighting coefficients
b=wa*a+wc*c;                        % produce the next iteration
  if f(a)*f(b)<=0
   c=b;
  else
   a=b;
  end
 end
disp([b,f(b)])                      % display the results
else
warndlg('The interval does not enclose the root','!! Warning !!')
end
```

Although the false-position method would seem to always be the bracketing method of preference, there are cases when it performs poorly. It happens, for instance, when the function $f(x)$ at one of the two initial guesses, a or c, is flat (that is, its derivative is close to zero) and is much smaller than at the other end of the interval. This means degradation of one of weighting coefficients so that information about the corresponding endpoint in Eq. (10.10) will be neglected, resulting in the trial point being very close to another endpoint, as shown in Fig. 10.6a.

Because of this "one-sidedness," the pure false-position algorithm may result in a relatively slow convergence. The remedy is to periodically check

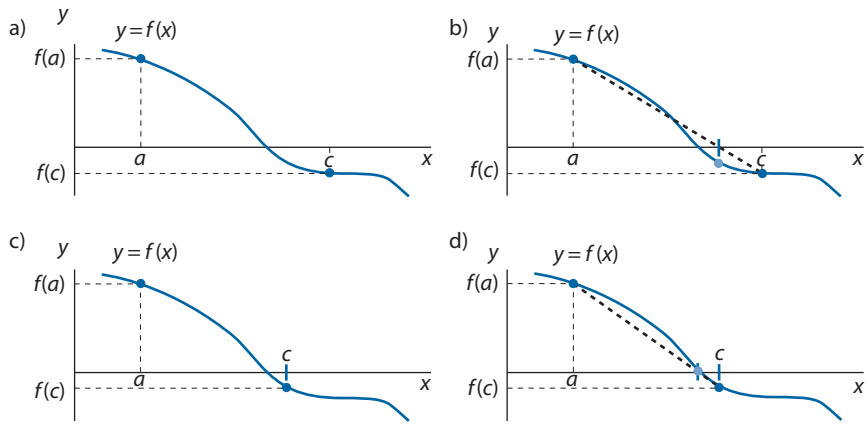


Fig. 10.5 Illustration of several consecutive steps using the false-position algorithm.

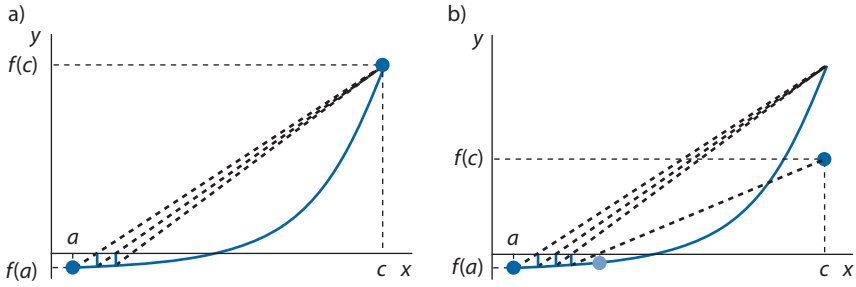


Fig. 10.6 Illustration of a) potential pitfall of false-position algorithm and b) possible remedy.

whether one of the bounds is stuck and, if this occurs, either apply one step of the bisection method or halve the function value at the bound opposite to the stagnant one (artificially increasing its weight), as shown in Fig. 10.6b. The latter procedure is known as the *modified false-position method*.

10.2.3 Golden-Section Method

As opposed to varying weighting coefficients depending on geometry as was the case for the false-position method, the *golden-section method*, applied to root-finding problems, basically uses another pair of constant coefficients, different from the 0.5 and 0.5 of the bisection method [Eq. (10.5)]. It implies $w_a \neq w_c$. It is similar in spirit to other methods, but to make it work (to ensure a fair choice between the two same-size intervals), two rather than one intermediate points are required at the very first iteration (after bracketing) (Fig. 10.7).

Following the notations of Fig. 10.7, we want to start from the interval of size $(l_1 + l_2)$ ($[a;c]$), then reduce it to l_1 ($[b_1;c]$), then to l_2 ($[b_2;c]$), etc. For consistency, the following condition should hold:

$$\frac{l_1 + l_2}{l_1} = \frac{l_1}{l_2} \tag{10.11}$$

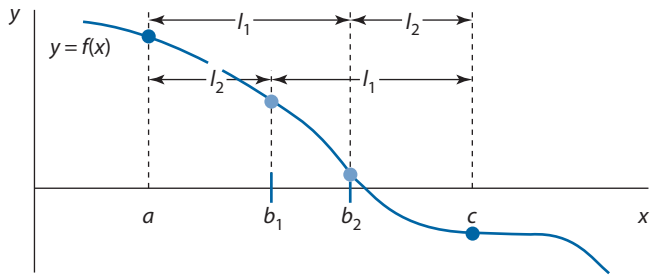


Fig. 10.7 Finding two intermediate points for the golden-section algorithm.

Introducing a notation $R = l_2/l_1$ yields

$$1 + R = \frac{1}{R} \tag{10.12}$$

meaning

$$R^2 + R - 1 = 0 \tag{10.13}$$

Resolving for the positive root of Eq. (10.13) gives

$$R^{\otimes} = \frac{\sqrt{5} - 1}{2} \approx 0.61803 \tag{10.14}$$

This value of $R^{\otimes}$ has been known since antiquity and is called the *golden ratio*. It is related to an important mathematical series known as the *Fibonacci numbers*, which are 0, 1, 1, 2, 3, 5, 8, 13, ... The Fibonacci numbers start from $n_1^F = 0$ and $n_2^F = 1$, and then the next number n_i^F is computed as the sum of the previous two:

$$n_i^F = n_{i-1}^F + n_{i-2}^F, \quad i = 3, 4, \ldots, \infty \tag{10.15}$$

It happens that

$$\lim_{i \to \infty} \frac{n_i^F}{n_{i+1}^F} = R^{\otimes} \tag{10.16}$$

meaning that the ratio of the consecutive Fibonacci numbers approaches the golden ratio $R^{\otimes}$ (and very quickly, we might add: $0/1 = 0$, $1/1 = 1$, $1/2 = 0.5$, $2/3 = 0.667$, $3/5 = 0.6$, $5/8 = 0.625$, $8/13 = 0.615$, and so on).

The numerical algorithm for root finding looks similar to that of bisection and false-position methods:

1. Given $f(x)$, determine the initial interval $[a;c]$, so that the inequality of Eq. (10.4) holds.
2. Find two trial points as

$$b_1 = c - R^{\otimes}(c - a) \quad \text{and} \quad b_2 = a + R^{\otimes}(c - a) \tag{10.17}$$

3. If the size of the new interval $[a; b_1]$ is less than or equal to the predefined tolerance ε

$$|a - b_1| \leq \varepsilon \tag{10.18}$$

   stop iterations assuming the solution at $x_* = b_1$.
4. If Eq. (10.18) does not hold, continue iterations by choosing the correct interval among the two possible ones ($[a;b_2]$ and $[b_1;c]$)
   - If inequality $f(a)f(b_2) \leq 0$ holds, then assign $c = b_2$, $b_2 = b_1$, and find $b_2 = c - R^{\otimes}(c - a)$
   - Otherwise assign $a = b_1$, $b_1 = b_2$, and find $b_2 = a + R^{\otimes}(c - a)$
5. Go to Step 2 with a new interval.

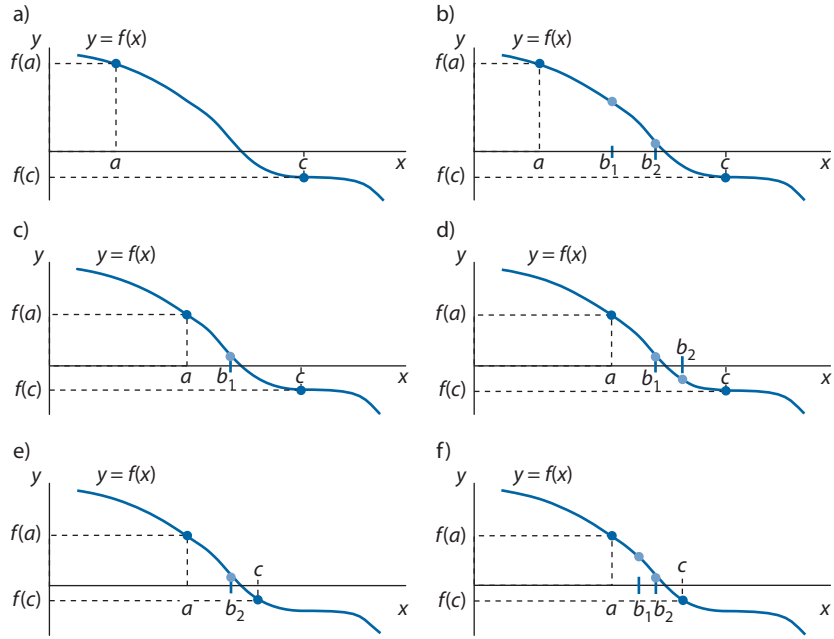


Fig. 10.8 Illustration of several consecutive steps using the golden-section algorithm.

Compared to Eq. (10.10), weighting coefficients for a and c in Eq. (10.17) are $R^{\otimes}$ and $1 - R^{\otimes}$ (or vice versa, depending on what trial point was chosen for further processing).

Given the bounds of the initial root-bracketing interval, x_0 and x_1, function $f(x)$ developed as an anonymous/inline function or m-file, and relative tolerance ε^{rel}, the golden-section method might be programmed as

```
a=x0; c=x1; R=(sqrt(5)-1)/2;
b1=c-R*(c-a); b2=a+R*(c-a);
if f(a)*f(c)<=0
  while abs(a-b1)>EPS*max(abs(b1),1e-3) % meet the relative tolerance
   if f(a)*f(b2)<=0
     c=b2;
     b2=b1;
     b1=c-R*(c-a);
else
     a=b1;
     b1=b2;
     b2=a+R*(c-a);
    end
  end
disp([b1,f(b1)])                        % display the results
```

```
else
warndlg('The interval does not enclose the root','!! Warning !!')
end
```

The few first iterations of this algorithm, as applied to the function shown in Fig. 10.8a, are shown in Figs. 10.8b–f. As seen, starting from the second iteration, we only have to find a single trial point at each step because, by construction, we have another one from the previous iteration already. Therefore, except the very first iteration, the number of computations remains the same as in any other bracketing method. However, compared with the bisection method with the termination condition defined by Eq. (10.6) resulting in Eqs. (10.7) and (10.8), in the golden-section method, the size of interval $|a-c|$ shrinks more slowly (by the factor of $R^{\otimes} > 0.5$). For example, after, say, 10 iterations, the interval shrinks to 0.618^{10} or about 0.8% of its original length, whereas for the bisection algorithm, it reduces to 0.5^{10} or about 0.1%.

Because at each step there is an extra point to play with, the golden-section method belongs to the *ternary search algorithms*, a technique usually used for finding a minimum or maximum of a function (see Sec. 10.7.1), and in this capacity, it can easily be modified to handle situations like the one presented in Fig. 10.4.

10.3 Open Methods

As opposed to bracketing methods, *open methods* require only a single value of x to start iterations [some open methods may still require two or even more values, but not necessarily bracketing the root as required by Eq. (10.4)]. As such, they generally do not guarantee convergence as computations progress unless the initial guess x_0 is chosen wisely, that is, very close to a root x_* (as shown in Fig. 10.9). However, even then, the numerical algorithm may still diverge. Nevertheless, the advantage of open methods is that if they converge, they demonstrate a much better convergence rate compared with that of the bracketing methods. The methods belonging to this groups and to be considered in this section are the *fixed-point iteration method, Newton's method*, *secant method* (requiring two points to start the iterative

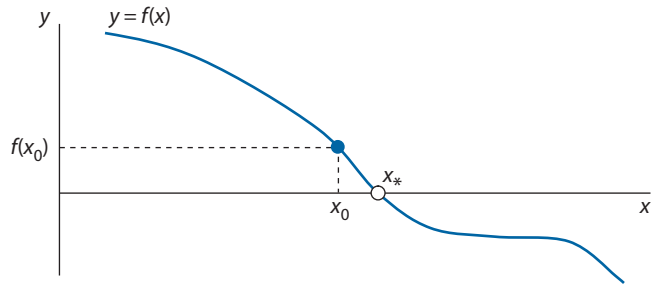


Fig. 10.9 Choosing the initial guess for an open method.

procedure), and *inverse quadratic interpolation* (IQI) *method* (requiring three initial points).

10.3.1 Simple Fixed-Point Iteration

Let us start from the simplest method, which is known as the *fixed-point iteration*, *one-point iteration*, or *successive substitution method*. Before continuing, we need to slightly modify the original problem. Let us add x to both sides of Eq. (10.1)

$$x + f(x) = x \tag{10.19}$$

and rewrite the result as

$$x = g(x) \tag{10.20}$$

(here, $g(x) \equiv x + f(x)$). Solving Eq. (10.1) automatically solves Eq. (10.19) and vice versa—if we manage to solve Eq. (10.20) for some x_*, the very same x_* is also the solution of Eq. (10.1).

Note that to obtain the equation that looks like Eq. (10.20), we could have added, say, Kx to both sides of Eq. (10.1) as well:

$$Kx + f(x) = Kx \tag{10.21}$$

This equation can be brought to the form of Eq. (10.20) with

$$g(x) \equiv x + K^{-1} f(x) \tag{10.22}$$

Hence, the reduction of the original equation of the form (10.1) to that of Eq. (10.20) can be accomplished in different ways. Sometimes, you can even "extract" x from $f(x)$ analytically. Say, if $f(x) = x^3 + ax^2 + bx + c$, then you can simply rewrite it as Eq. (10.20) with $g(x) = -b^{-1}(x^3 + ax^2 + c)$.

The utility of Eq. (10.20) is that it explicitly provides a formula to predict a new value of x as a function of an old value:

$$x_{i+1} = g(x_i), \quad i = 0, 1, \ldots \tag{10.23}$$

As a matter of fact, instead of finding where the function $f(x)$ crosses the x-axis, we are now trying to find an intersection of two curves, $y = x$ and $y = g(x)$ (Fig. 10.10a).

Let us address the convergence issue. Figures 10.10b and 10.11a, b complement Fig. 10.10a and show three other possible situations of a relative position of curve $y = g(x)$ with respect to $y = x$. As seen, among those three, only the case of Fig. 10.10b exhibits convergence, whereas the two other cases (depicted in Figs. 10.11a, b) diverge. What causes divergence?

To answer this question, we should look at the value of derivative $g'(x) = dg(x)/dx$ in the vicinity of the root. For example, in the case of Figs. 10.10a, b, $0 < g'(x_*) < 1$ and $-1 < g'(x_*) < 0$, respectively. In the case of

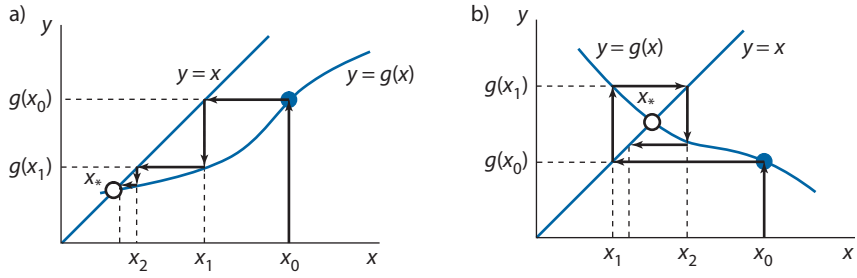


Fig. 10.10 Illustration of converging iterations in the simple fixed-point method.

Fig. 10.11a, $g'(x_*) > 1$, and in the case of Fig. 10.11b, $g'(x_*) < -1$. Therefore, convergence is guaranteed only when in the vicinity of the root $|g'(x)| < 1$. In addition to that, if the derivative $g'(x)$ is positive (Fig. 10.10a), the error exhibits a monotonic behavior (not changing its sign from one iteration to another), whereas if $g'(x)$ is negative (Fig. 10.10b), the error oscillates (changes its sign).

That is where having the capability to have a different function $g(x)$, when converting Eq. (10.1) to the form of Eq. (10.20), may really help. For instance, suppose you need to find the root of Eq. (10.1) with $f(x) = e^{x-1} - 2$ (in this particular case, the root can be found analytically as $x_* = \ln(2) + 1 \approx 1.693$). If you convert this equation to the form of Eq. (10.20) by adding x to both sides as in Eq. (10.19), you get $g(x) = x + e^{x-1} - 2$. The problem with this is that $g'(x)|_{x \sim x_*} \sim 3 \geq 1$, that is, the successive substitution method would diverge. However, if we add $-4x$ to both sides of the original equation, we get $g(x) = x - 0.25\,(e^{x-1} - 2)$, with $g'(x)|_{x \sim x_*} \sim 0.5$, and so the successive substitution method would produce a solution.

Consider Eq. (10.22). The derivative of $g(x)$ in this general case is

$$g'(x) = 1 + K^{-1} f'(x) \tag{10.24}$$

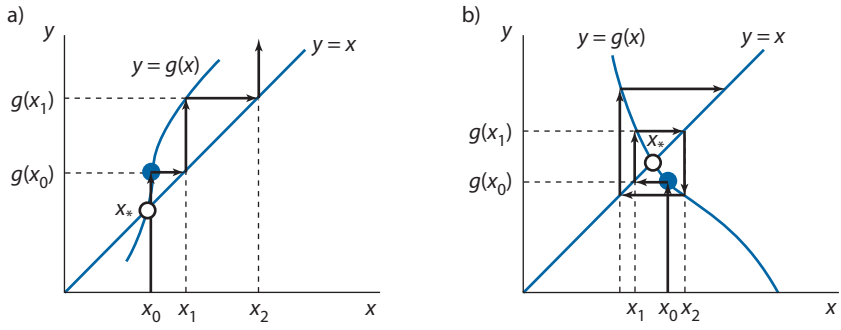


Fig. 10.11 Examples of successive substitution algorithm divergence.

For convergence, it is required to have

$$-1 < 1 + K^{-1} f'(x) < 1 \tag{10.25}$$

or

$$-2 < K^{-1} f'(x) < 0 \tag{10.26}$$

Hence, assuming that the initial guess is fairly close to the solution, we can always choose K as

$$\begin{array}{ll} K < -0.5 f'(x_0), & \text{if } f'(x_0) > 0 \\ K > -0.5 f'(x_0), & \text{if } f'(x_0) < 0 \end{array} \tag{10.27}$$

In the case of $f(x) = e^{x-1} - 2$ considered earlier, Eq. (10.27) estimated at $x = x_*$ yields $K < -1$, but the actual value of K that guarantees convergence depends on the initial guess x_0 and function behavior in the vicinity of solution. Hence, despite all precautions, any open method must have a safeguard—a firm limit on the number of iterations.

Given the initial guess, function $f(x)$, and relative tolerance, the successive substitution algorithm may be realized as follows:

```
x=x0;
h=0.01*x; if h==0, h=0.01; end
K=-4*(f(x+h)-f(x))/h;      % choose the gain K
for k=1:500                % limit the number of iterations
a=x;                       % keep the current iteration
x=x+f(x)/K;                % find the next iteration
if abs(x-a)<=EPS*max(abs(x),1e-3), break, end % check the accuracy
end
disp([x,f(x),k])           % display the results
```

(the second and third lines of this script are dedicated to estimating the first-order derivative numerically, which will be discussed in more detail in the following section). As seen, we actually combined Eq. (10.22) with an estimate of the gain [Eq. (10.27)], so that Eq. (10.23) takes the form

$$x_{i+1} = x_i - \frac{f(x_i)}{4f'(x_0)}, \quad i = 0, 1, 2, \ldots \tag{10.28}$$

[Note that in this realization we chose the coefficient 4 in the denominator quite arbitrarily with the only goal of satisfying Eq. (10.27) for sure, at least at the initial point.]

Let us use the previous code and find the root of $f(x) = e^{x-1} - 2$ by issuing the following commands:

```
f=@(x) exp(x-1)-2;
x0=0.5; EPS=1e-6;
...    % here comes the successive substitution algorithm code shown above
fplot(f,[0 2.5]), grid, hold % the ezplot function can be used as well
plot(x0,f(x0),'go','MarkerSize',10,'MarkerFaceColor','g')
```

```
plot(x,f(x),'rp','MarkerSize',12,'MarkerFaceColor','r')
xlabel('x'), ylabel('f(x)=e^{x-1}-2')
legend('Function f(x)','Initial guess','Solution','location','nw')
text(0.95,1,'The method converged')
text(1.1,0.5,['in' int2str(k)'iterations'])
```

The results are presented graphically in Fig. 10.12a. With the given initial guess and tolerance, it took 11 iterations to converge. How good is this?

In dealing with a sequence of successive approximations for any iterative method, the *rate of convergence* $\mu \in [0;1]$ can be formally defined as

$$\mu = \lim_{i \to \infty} \frac{|x_{i+1} - x_*|}{|x_i - x_*|^q} \tag{10.29}$$

If for $q = 1$ the limit [Eq. (10.29)] converges to some number $\mu \in (0;1)$, the sequence x_i, $i = 0,1, \ldots, \infty$ is said to converge *linearly* to x_*. If Eq. (10.29) holds with $\mu = 1$, then the sequence is said to converge *sublinearly*. One says that the sequence converges *superlinearly* if it converges to $\mu = 0$. In the latter case, you may consider the higher values of q. Convergence to $\mu \in (0;1)$ with $q = 2$ is called *quadratic convergence*, convergence with $q = 3$ is called *cubic*, etc.

The ratio $|x_{i+1} - x_*|/|x_i - x_*|$ for bracketing methods does not necessarily converge, but most of the time stays within $\mu \in (0;1)$; therefore, these methods ensure a linear convergence. The behavior of this ratio in the

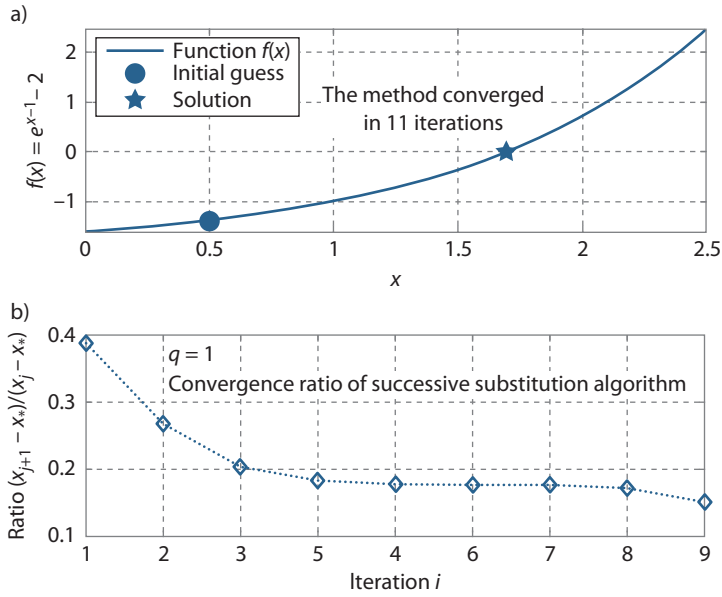


Fig. 10.12 a) Example of a solution using the successive substitution algorithm and b) a proof of its linear convergence.

example we just considered is presented in Fig. 10.12b. As seen, it converges to some number $\mu \in (0;1)$ (with $q = 1$) and therefore proves the linear convergence property of the simple fixed-point iterations method.

10.3.2 Newton's Method

Of all the root-finding methods, perhaps the most widely used is *Newton's method* (also known as the *Newton–Raphson method*). Newton's method is derived from a Taylor series expansion about the point x_i [see Eqs. (7.2) and (8.32)]

$$f(x_{i+1}) = f(x_i) + (x_{i+1} - x_i)f'(x_i) + HOT \tag{10.30}$$

Neglecting the higher-order terms, the tangential line passing through the point $(x_i, f(x_i))$ is given by

$$f(x_i) + f'(x_i)(x_{i+1} - x_i) = 0 \tag{10.31}$$

Solving for x_{i+1} yields the formula for Newton's method successive approximations:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \tag{10.32}$$

Figures 10.13a, b provide a graphical interpretation of this formula showing two consecutive iterations.

To enhance the robustness of Eq. (10.32), Raphson suggested using the following modification with $\alpha < 1$:

$$x_{i+1} = x_i - \alpha\frac{f(x_i)}{f'(x_i)} \tag{10.33}$$

As seen, Eq. (10.33) has exactly the same form as Eq. (10.28). The only difference is that the first-order derivative in the denominator is now updated at every iteration, ensuring a better robustness. To be more precise, Newton's method guarantees quadratic convergence, meaning that the number of significant figures of accuracy approximately doubles with each iteration [according to Eq. (10.29), the true percent relative error is roughly

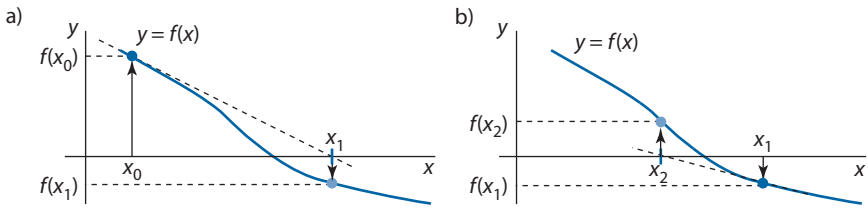


Fig. 10.13 Graphical interpretation of Newton's method.

proportional to the square of the previous iteration error]. This can easily be proved using the same example we used in the previous section. Newton's method can be programmed as follows:

```
syms x
fs=exp(x-1)-2;  f=matlabFunction(fs);          % f(x) handle
fdots=diff(fs); fdot=matlabFunction(fdots);    % f_dot(x) handle
x=x0;
for k=1:500                                  % limit the number of iterations
a=x;                                           % keep the current iteration
x=x-f(x)/fdot(x);                              % find the next iteration
  if abs(x-a)<=EPS* max(abs(x),1e-3), break, end % exit the condition
end
disp([x,f(x),k])                               % display the results
```

Note how we used the `matlabFunction` function of the Symbolic Math Toolbox (presented in Sec. 7.10.2) in the second and third lines of code to create two function handles to the function itself and its analytically-derived derivative. If you use this code for $f(x) = e^{x-1} - 2$ with the same settings as in the previous section, you will arrive at the solution much faster compared with the simple fixed-point iteration algorithm (compare Fig. 10.14a with Fig. 10.12a). As seen from the top plot of Fig. 10.14b, the rate of convergence μ [Eq. (10.29)] with $q = 2$ equals 0.5. Moreover, although Eq. (10.32), adaptive to the first-order derivative, ensures convergence in a fairly decent range of initial guesses, Eq. (10.28) diverges if we start from, say, $x_0 = 0$ (because the derivative estimated at x_0 is quite different from that estimated at x_*).

Now, let us address the issue of computing the first-order derivative. In the particular example considered earlier with $f(x)$ defined analytically, we know that $f'(x) = e^{x-1}$, and so we took advantage of it supplying (computing symbolically) an analytical formula for the derivative. In practice, however, the function $f(x)$ may not be defined analytically, meaning that its first-order derivative has to be estimated numerically. Although this issue is covered in detail in Chapter 12, the easiest way to do this is to use a forward difference approximation

$$f'(x_i) \cong \frac{f(x_i + h) - f(x_i)}{h} \tag{10.34}$$

where h has to be sufficiently small to ensure a certain accuracy. We have used this approximation in the script following Eq. (10.27) already (in there, we chose h to be 1% of x_0 or 0.01 if x_0 is zero). In that script, it caused no problems because the derivative was computed only once and then multiplied by some arbitrary number (4). However, if we use this approximation instead of an exact analytical formula in Newton's method, its convergence rate would degrade to superlinear (shown in the lower plot of Fig. 10.14b).

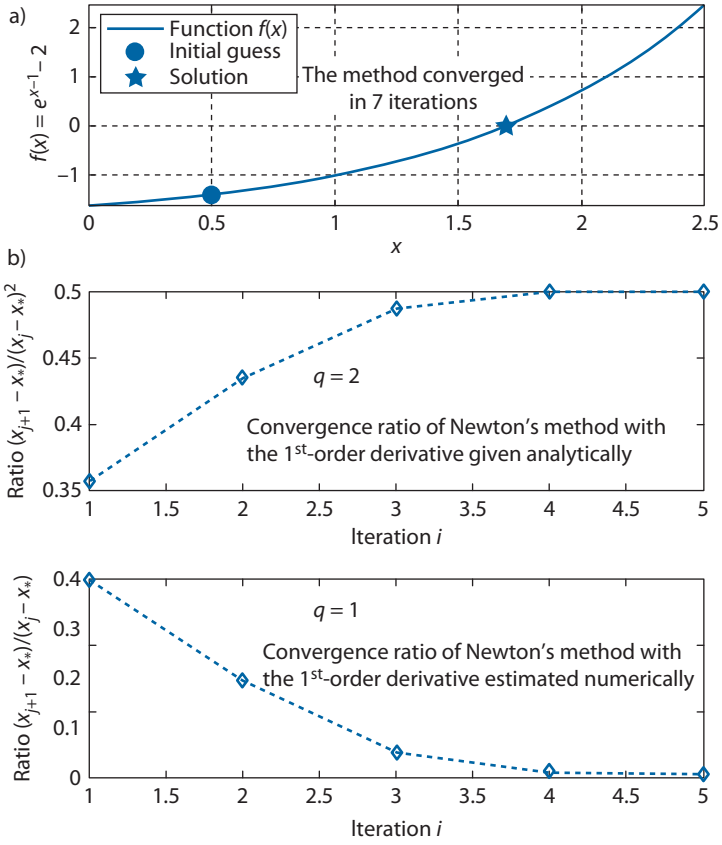


Fig. 10.14 a) Example of solution using Newton's method and b) its rate of convergence.

Although the Newton–Raphson method is often very efficient, there are situations where it performs poorly. For instance, it cannot handle multiple roots. However, even when dealing with simple roots, difficulties with convergence can also arise. A few cases are presented in Fig. 10.15.

Specifically, Fig. 10.15a depicts the case where an *inflection point* (that is, $f''(x) = 0$) occurs in the vicinity of the root. Notice how iterations progressively diverge from the root. A typical example of such a case is finding the root of the function $f(x) = \text{sign}(x-a)\sqrt{|x-a|}$. Newton's method will iterate forever. Figure 10.15b illustrates the tendency of the Newton–Raphson technique to oscillate around a local maximum or minimum. Moreover, as shown in Fig. 10.15b, if a near-zero slope is reached, the solution may be sent far from the area of interest. [By the way, that is one of the cases where a step-limiting gain α in Eq. (10.33) may help.] In the particular situation of Fig. 10.15b, one may argue that the problem was caused by unfortunate

choice of the initial guess, so that $f'(x_*)\,f'(x_0) < 0$, but even if we start from x_1, we would run into the same problem. Figure 10.15c shows how an initial guess that is close to one root can jump to another location several roots away. The tendency to move away from the area of interest is caused by the varying first-order derivative sign ($\text{sign}(f'(x)\big|_{x\sim x_*}) = var$), so that the near-zero slopes are encountered. Obviously, a zero slope ($f'(x)|_{x\sim x*} \sim 0$) is always a disaster because it causes division by zero in both Eq. (10.32) and Eq. (10.33). Graphically, it means that the solution shoots off horizontally and never hits the x-axis (Fig. 10.15d).

Thus, no general convergence criterion for the Newton–Raphson method exists. Its convergence depends on the nature of the function and on the accuracy of the initial guess. The only remedy is to have an initial guess that is "sufficiently" close to the root. For some functions, no guess will work at all! Good guesses are usually predicated on knowledge of the physical problem setting or on visualization of the function that provides insight into its behavior in the vicinity of root. The lack of a general convergence criterion also suggests a script realizing this algorithm (in fact, any open-method algorithm) be designed to recognize slow convergence or even divergence, making the corresponding adjustment iterations or simply quitting if nothing helps.

To summarize, a good programming paradigm as applied to the realization of the Newton–Raphson method (and any other open method) implies

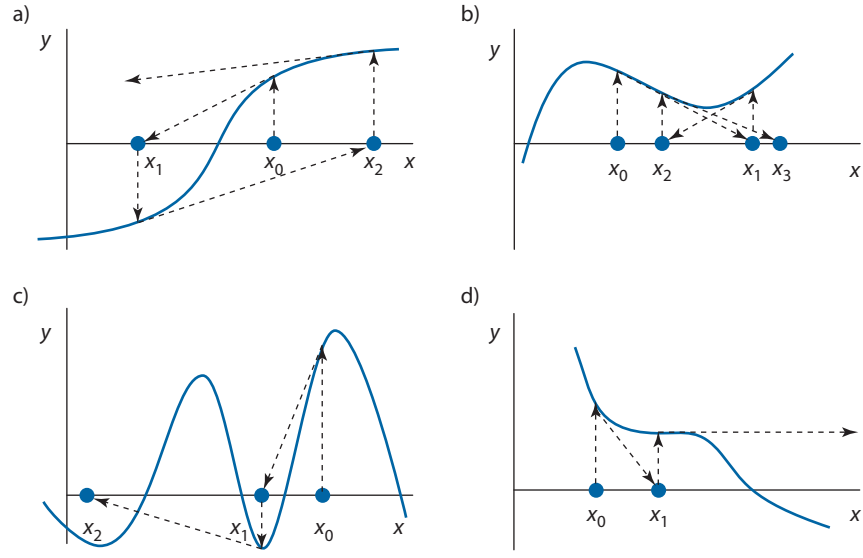


Fig. 10.15 Pitfalls of Newton's method.

- Plotting the function to choose a good initial guess
- Alerting a user and taking into account a possibility that $f'(x)$ might become zero at any time during the computation
- Limiting the number of iterations to guard against oscillating, slowly convergent or divergent solutions that could persist interminably
- Always checking the solution (substituting it into the original function to compute whether the result is close to zero)

The last check partially guards against those cases where slow or oscillating convergence may lead to a small value of ε_i^a [Eq. (8.26)], although the solution is still far from a root.

10.3.3 Secant Method

As discussed in the previous section, evaluation of the first-order derivative (in the case of a not analytically given function) causes a certain problem. As suggested by Eq. (10.34), the derivative can be approximated by a forward difference with as small h as possible to make the estimate more accurate. As a result, instead of just one evaluation of the function at each iteration with the analytically given derivative, we end up with two numerical evaluations, which obviously degrades the algorithm's performance.

An attractive alternative for estimating the derivative, or at least the tendency of a function change, is to use the previous iteration as follows:

$$f'(x_i) \approx \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}} \tag{10.35}$$

Indeed, this constitutes a very rough estimate of derivative using a backward difference approximation (to be discussed in Chapter 12). Substituting Eq. (10.35) into Eq. (10.32) yields the formula known as the *secant method:*

$$x_{i+1} = x_i - \frac{f(x_i)(x_i - x_{i-1})}{f(x_i) - f(x_{i-1})} \tag{10.36}$$

To initialize iterations, two points are required. As shown in Fig. 10.16, you start from x_0 and x_1 to produce x_2, then use x_1 and x_2 to produce x_3, and so on.

The code for this algorithm may look like the following:

```
a=x0;
h=0.01*a; if h==0, h=0.01; end
x1=a+h;
for k=1:500                      % limit the number of iterations
  fdot=(f(x1)-f(a))/(x1-a);      % estimate the derivative
  a=x1;
  x=x1-f(x1)/fdot;               % find the next iteration
  x1=x;
   if abs(x-a)<=EPS*abs(x), break, end   % check the accuracy
end
disp([x,f(x),k])                 % display the results
```

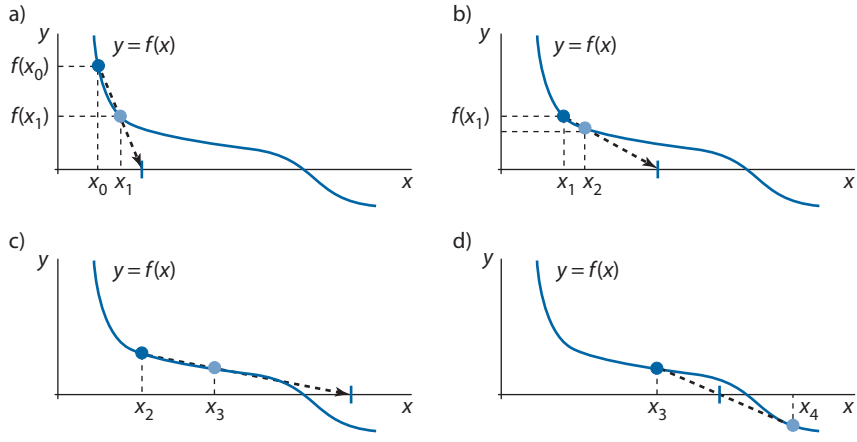


Fig. 10.16 Illustration of the secant algorithm.

Obviously, there is a similarity between the secant method and the false-position method. Both use two initial estimates to compute an approximation of a slope of the function that is used to produce a new estimate of the root. However, a critical difference between these methods is how one of the initial values is replaced by a new estimate. In the false-position method, the latest estimate of the root replaces either of the original values yielding a function value with the same sign as the replaced one, therefore always bracketing the root. Consequently, the method always converges. In contrast, the secant method replaces the values in a strict sequence, with the new value replacing the previous one. As a result, the two consecutive values can reside on the same side of the root (Fig. 10.16a–c), as opposed to being on different sides of the root (Fig. 10.16d). As discussed earlier, this may lead to divergence.

10.3.4 Inverse Quadratic Interpolation Method

Basically, what we did in the previous section is that we used two trial points to obtain a local linear approximation of the function, and then, by finding the intersection of this approximation with the x-axis, we found the next trial point. This allowed us to obtain an algorithm ensuring close to quadratic (superlinear) convergence. As it turns out, to increase the order of convergence, the same idea can be explored further to involve three trial points for a local approximation of the function.

If three trial points are used (note, they may not necessarily appear to the left or to the right of the root, as shown in Fig. 10.17a, though), a quadratic approximation of the function $f(x)$, $f^*(x)$, may be used (Fig. 10.17b). This

approximation based on three points $(x_{i-2}, f(x_{i-2}))$, $(x_{i-1}, f(x_{i-1}))$, and $(x_i, f(x_i))$ is given by the equation

$$f^*(x) = A(x - x_i)^2 + B(x - x_i) + C \tag{10.37}$$

where coefficients A, B, and C are computed as

$$A = \frac{f(x_{i-2})\Delta_{i,i-1} - f(x_{i-1})\Delta_{i,i-2} + f(x_i)\Delta_{i-1,i-2}}{\Delta_{i,i-1}\Delta_{i,i-2}\Delta_{i-1,i-2}}$$

$$B = \frac{f(x_{i-2})\Delta_{i,i-1}^2 - f(x_{i-1})\Delta_{i,i-2}^2 + f(x_i)\Delta_{i-1,i-2}(\Delta_{i,i-1} + \Delta_{i,i-2})}{\Delta_{i,i-1}\Delta_{i,i-2}\Delta_{i-1,i-2}} \tag{10.38}$$

$$C = f(x_i)$$

$(\Delta_{i,j} = x_i - x_j)$.

To start this algorithm, you have to have two approximations, x_0 and x_1, to be obtained somehow. Among the two roots of $f^*(x) = 0$ defined by the well-known formula

$$x = x_i + \frac{-B \pm \sqrt{B^2 - 4AC}}{2A} \tag{10.39}$$

the nearest to the point x_i may be chosen to constitute the next guess x_{i+1} (Fig. 10.17c). Then, the entire procedure is repeated for the new triplet and so on.

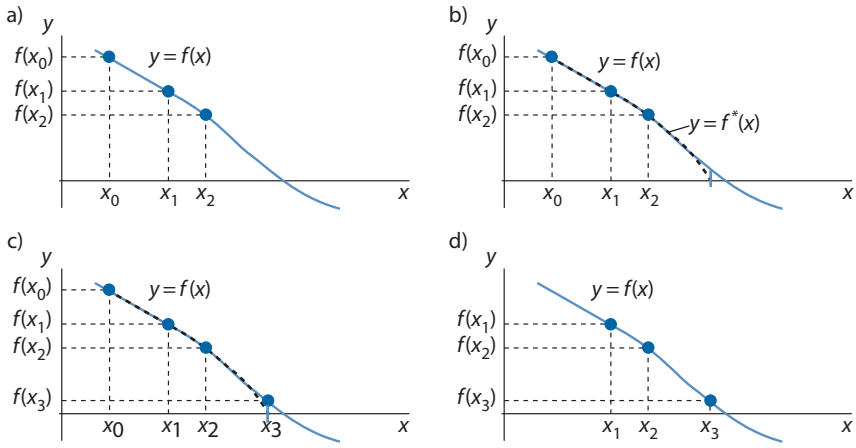


Fig. 10.17 Illustration of the quadratic interpolation algorithm.

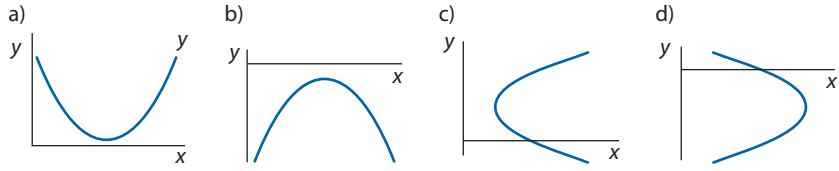


Fig. 10.18 a, b) Quadratic vs c, d) inverse quadratic interpolation.

The problem with this approach, though, is that for a relatively flat function $f(x)$, the parabola may not intersect the x-axis (Figs. 10.18a and 10.18b), so that a quadratic three-point approximation $f^*(x)$ does not necessarily have the real roots (formally, it happens when $B^2 - 4AC < 0$). Sometimes, this can be regarded as an advantage (for example, some methods benefit from using the complex roots to find the complex zeros of a polynomial). However, for this particular application, we want to avoid complex arithmetic and rather have a single real root guaranteed at each iteration. Well, the trick is to rotate the parabola (Figs. 10.18c and 10.18d).

Hence, instead of using three points, $(x_{i-2}, f(x_{i-2}))$, $(x_{i-1}, f(x_{i-1}))$, and $(x_i, f(x_i))$, to develop an approximation in x, we should be looking at an approximation in $f(x)$

$$x = P(f(x)) = \tilde{A}f(x)^2 + \tilde{B}f(x) + \tilde{C} \tag{10.40}$$

where coefficients $\tilde{A}$, $\tilde{B}$ and $\tilde{C}$ of a sideways (horizontal) parabola, $P(f(x))$, are determined by the interpolation conditions $x_{i-2} = P(f(x_{i-2}))$, $x_{i-1} = P(f(x_{i-1}))$, and $x_i = P(f(x_i))$. By construction, Eq. (10.40) always intersects the x-axis ($f(x) = 0$) at $x = P(0) = \tilde{C}$, which becomes the next, $(i+1)$th, iteration. Given the three points and function $f(x)$, the IQI procedure can be programmed as follows:

```
a=x0; b=x1; c=x2; k=0;
while abs(c-b)>EPS*abs(c)
  coef=polyfit([f(a),f(b),f(c)],[a,b,c],2); % find IQI coefficients
  a=b;
  b=c;
  c=coef(3);                    % the next candidate point
  k=k+1;
  if k>100, break, end          % limit the number of iterations
end
disp([c,f(c)])                  % display the results
```

Note that, to perform a polynomial curve fitting, we used the `polyfit` function, which was introduced in Table 3.18 of Sec. 3.5 (and will be

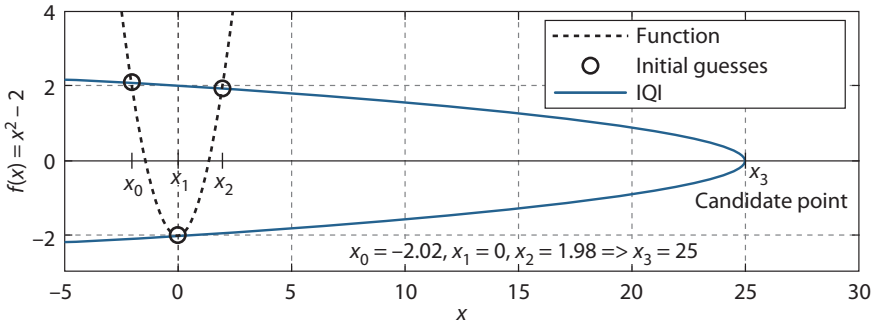


Fig. 10.19 Example of an unsuccessful IQI step.

discussed in detail in Chapter 11). This function resolves for coefficients of IQI [Eq. (10.40)] as follows:

```
A_IQI = [f(a)^2 f(a) 1;
         f(b)^2 f(b) 1;
         f(c)^2 f(c) 1];
B_IQI = [a;b;c];
coef = A_IQI\B_IQI;
```

If it converges, the IQI method provides superquadratic convergence. The trouble with the "pure" IQI algorithm, however, is that the polynomial interpolation requires the values of $f(x_{i-2})$, $f(x_{i-1})$, and $f(x_i)$ to be distinct. Yet, generally, this is not guaranteed. For example, if we try to compute $\sqrt{2}$ using $f(x) = x^2 - 2$ and start with $x_0 = -2$, $x_1 = 0$, and $x_2 = 2$, we would end up with $f(x_0) = f(x_2)$, meaning that the first step is undefined. If we start somewhere nearby this singular situation, say, with $x_0 = -2.02$, $x_1 = 0$, and $x_2 = 1.98$, the very first iteration will bring us to $x_3 = 25$ (Fig. 10.19). Hence, the IQI method needs to be supervised to decide whether the candidate point it provides is of any use.

10.4 Zeroin Algorithm

Figure 10.20 summarizes the findings of the two previous sections and sorts all methods by their convergence properties. As seen, although the bracketing methods guarantee convergence, the speed of convergence is slower than that of the open methods. The latter ones, however, do not guarantee convergence.

Therefore, the idea is to combine the best properties of both approaches, and that is what is being realized in the zeroin algorithm (the `fzero` function). This algorithm, originated by van Wijngaarden, Dekker, and others at the Mathematical Center in Amsterdam in the 1960s, combines the

reliability of the bisection method with the convergence speed of the secant and IQI methods. It is implemented in MATLAB as Richard Brent's version, which looks as follows:

1. Start with a and b so that $f(a)f(b) < 0$.
2. Use a secant step to produce c between a and b.
3. Check the exit conditions, which are $|b - a| < \varepsilon|b|$ or $f(b) = 0$.
4. Arrange a, b, and c so that
   - $f(a)$ and $f(b)$ have opposite signs, meaning that a root is bracketed
   - $|f(b)| \leq |f(a)|$, meaning that b is closer to a root then a
   - c represents the previous value of b
5. If $c \neq a$, meaning that a, b, and c are distinct points, perform an IQI step.
6. If $c = a$, perform a secant step.
7. If the IQI or secant step produces a "reasonable" iteration (the definition of "reasonable" is rather technical, but essentially it means that the candidate point resides inside the current interval $[a;b]$ and not too close to the end points), accept it.
8. If the last step produces a candidate point that resides beyond the interval $[a;b]$, use a bisection iteration and go back to Step 3.

By construction, the aforementioned iterative procedure guarantees the decrease of interval length at each iteration (due to root bracketing) and the fastest convergence when the function $f(x)$ is well behaved. As an example, Fig. 10.21 represents two sets of three sequential steps (shown in the upper and lower rows). These snapshots were obtained using the `fzerogui` function written by Cleve Moler. The function itself is shown as a dashed line, whereas a short vertical line in each plot represents a bisection point (halving the current interval $[a;b]$ or $[b;a]$).

The first set of sequential steps (the upper row of Fig. 10.21) shows that in the beginning (the 1st plot in the upper row), the interval $[b;a]$ encloses a

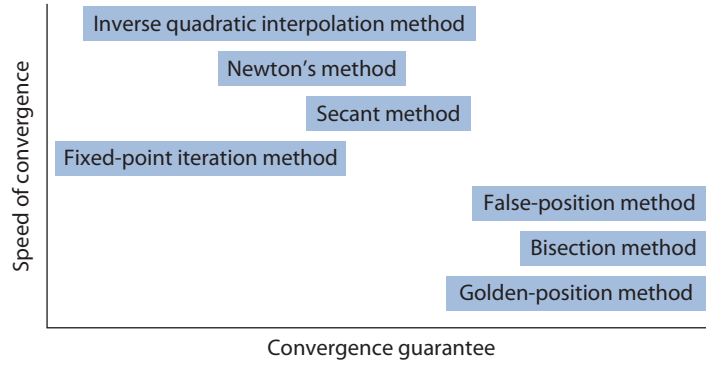


Fig. 10.20 Bracketing vs open methods.

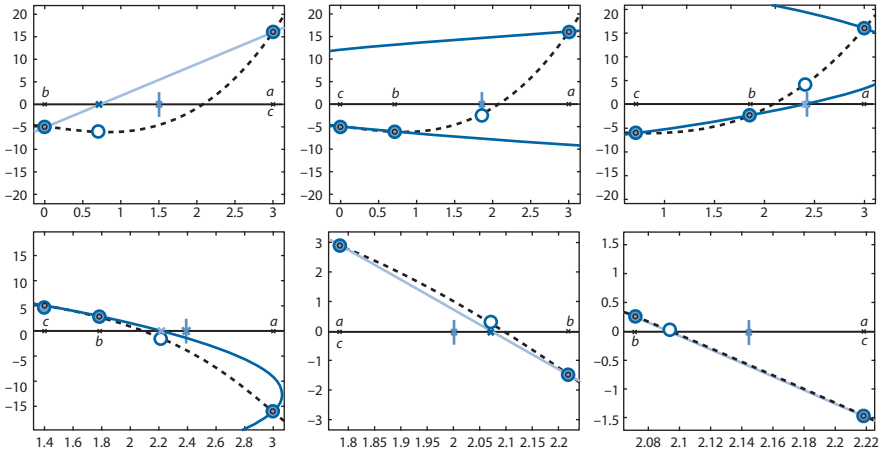


Fig. 10.21 Illustration of the zeroin algorithm (two sets of three sequential steps).

root, b and a are arranged so that $|f(b)| \leq |f(a)|$ and point c coincides with a. Because there are only two distinct points available (depicted as filled circles), a secant step (corresponding to Step 6 of the iterative procedure) is made to produce the candidate third point (depicted with a white-face-color circle). The point happens to be reasonable (see Step 7), and as a result, the old point b is moved to a new position, and point c is moved to the old b's position (Step 4) which is shown in the 2nd plot of the upper-row sequence. Because three distinct points are now available, an IQI step is made to produce the next candidate iteration (Step 5). Unfortunately, it turns out that this candidate point does not belong to the interval $[b;a]$ (the intersection of the inverse parabola shown in the same plot with the x-axis occurs far to the left). Because of this, the candidate point is rejected, and the bisection point (depicted with a white-face-color circle) is taken as the next iteration instead. This point becomes point b and its old value is assigned to c (as shown in the 3rd plot in the upper row). This time, the IQI step produces a reasonable solution (depicted as a white-face-color circle slightly to the left of a bisection point). (Note, all plots in Fig. 10.21 utilize the autoscaling mode (Table 6.2), so that the x-axis for the 3rd plot is scaled differently compared to that of the 1st and 2nd plots.)

The second sequence in Fig. 10.21 starts with a situation (the 1st plot in the lower row) that is very similar to that which we ended up with in the previous sequence. Having three distinct points, an IQI step is made. Because the new candidate point happens to be much closer to the root it becomes a new point b, causing point a to accept an old value of b (the 2nd plot in the lower row). By definition, c also accepts the old value of b (Step 4 of the iterative sequence). Hence, now that $c = a$, we have no other choice but to produce a secant step (Step 6). As seen from the 3rd lower-row

plot, the situation with the point b jumping over the root repeats, and we are left with only two points to produce a secant iteration yet again (Step 6).

A major advantage of the zeroin algorithm is that it is foolproof. It never loses track of the root trapped in a shrinking interval. On average, for the well-behaved functions, this combination of root-finding methods saves about half the function evaluations per root (compared with the other methods), exploiting a rapidly converging method only when it is reliable. It uses slower but reliable methods otherwise.

10.5 Multiple Roots and Value Finding

This section addresses several specific topics related to a root or specified value finding. It starts with an issue of automating bracketing the multiple roots of a function. It then proceeds with handling root multiplicity. Lastly, the value-finding problem is addressed.

10.5.1 Multiple Roots Bracketing

As shown in Secs. 10.2 and 10.4, bracketing methods rely on root localization. If the original interval is found, any of these methods ensures convergence. The question then is how to automatize finding such an interval, especially when the function has multiple roots (meaning that you need to find multiple intervals).

In what follows, we present a self-explanatory user-written function `lroot` (root localizing a root) that does the job (this function includes several professional elements, such as the `nargin`, `isempty`, `fcnchk`, and `feval` functions that were discussed in Sec. 5.9):

```
function Br=lroot(fun,xmin,xmax,nx,flag)
if nargin<5, flag=0; end                % deny the graphical output
if nargin<4 | isempty(nx), nx=20; end % the default number of brackets
% Create vectors of potential brackets and corresponding f(x) values
x=linspace(xmin-0.1,xmax+0.1,nx);
f=fcnchk(fun); f=feval(f,x);
% Search for the brackets
nb=0; Br=[];                            % Br is null unless brackets are found
for k=1:length(f)-1
  if sign(f(k))*sign(f(k+1))<=0         % true if sign(f(x)) changes
  nb=nb+1;
  Br(nb,1)=x(k); Br(nb,2)=x(k+1);       % save the bracket bounds
  end
end
% Plot function f(x) with the brackets
if flag~=0
% Prepare data to draw boxes that indicate brackets
```

```
ytop=max(f); ybot=min(f);          % y-coordinates of the box
ybox=0.05*[ybot ytop ytop ybot ybot]; % around a bracket
c=[0.7 0.6 0.7];                   % RGB color for the boxes
hold on
for i=1:nb
  fill([Br(i,1) Br(i,1) Br(i,2) Br(i,2) Br(i,1)],ybox,c); % add box
end
xp=linspace(xmin-0.1,xmax+0.1);  % vector of arguments for plotting
fp=fcnchk(fun); fp=feval(fp,xp); % vector of f(x) values brackets
plot(xp,fp,'Linewidth',2), plot(get(gca,'xlim'),[0 0],'--k');
xlabel('x'); ylabel('f(x) with localized roots');
    if nb==0
       text(xmin,0.65*ytop,'No roots found','background','w')
    elseif nb==1
       text(xmin,0.65*ytop,'One root found','background','w')
    else
       text(xmin,0.65*ytop,[num2str(nb) ' roots found'],'background','w')
    end
    grid
    plot([xmin,xmax],zeros(2,1),'r--')
    hold off
    if isempty(Br)                 % warning message
     warning ('No brackets found. Check [xmin, xmax] or increase nx');
    end
end
```

(Note, this function is not automatically available in MATLAB. In order to use it, you need to create your own *m*-file or download it from the textbook supporting website.)

The inputs to the `lroot` function are `fun`, the name of *m*-file function evaluating $f(x)$; `xmin` and `xmax`, the end points of a larger interval the individual roots will be bracketed within; `nx`, the number of samples along the x-axis to be used to test for bracket end points; and `flag`, indicating whether to display the results graphically. The latter two inputs are optional. The only constraint is that `nx` should be large enough (greater than a number of potential roots within the defined interval [`xmin`; `xmax`]) to localize each root individually. The default value for `nx`, established in the third line of the script, is 20.

This function finds subintervals of a function argument x that contain the sign changes of $f(x)$, so that each root may have its own brackets. The output of this function is a two-column matrix of bracket limits `Br`, so that `Br(k,1)` is the left endpoint and `Br(k,2)` is the right endpoint for the `k`th root. If no brackets are found, `Br` returns an empty matrix. In addition to this, the results can be presented graphically, featuring the plot of $f(x)$ with the determined bracketing intervals overlaid over it.

You may call this function using either syntax

```
lroot(fun,xmin,xmax)
lroot(fun,xmin,xmax,nx)
lroot(fun,xmin,xmax,nx,flag)
lroot(fun,xmin,xmax,[],flag)
```

For example, the call

```
>> lroot(@cos,-2*pi,2*pi,[],1);
```

returns four intervals shown in Fig. 10.22a, and the call

```
>> b=lroot('4*cos(2*x)-exp(0.5*x)+4',-3,4,6,1)
```

returns five intervals

```
b = -3.1000  -1.6600
   -1.6600  -0.2200
   -0.2200   1.2200
    1.2200   2.6600
    2.6600   4.1000
```

shown in Fig. 10.22b. (In the latter case, we intentionally decreased the value of `nx` down to the smallest experimentally-determined number that still guarantees localizing all five roots.)

The brackets found using `lroot` can be further passed over to the `fzero` function for root finding, as shown later in Sec. 10.9.1.

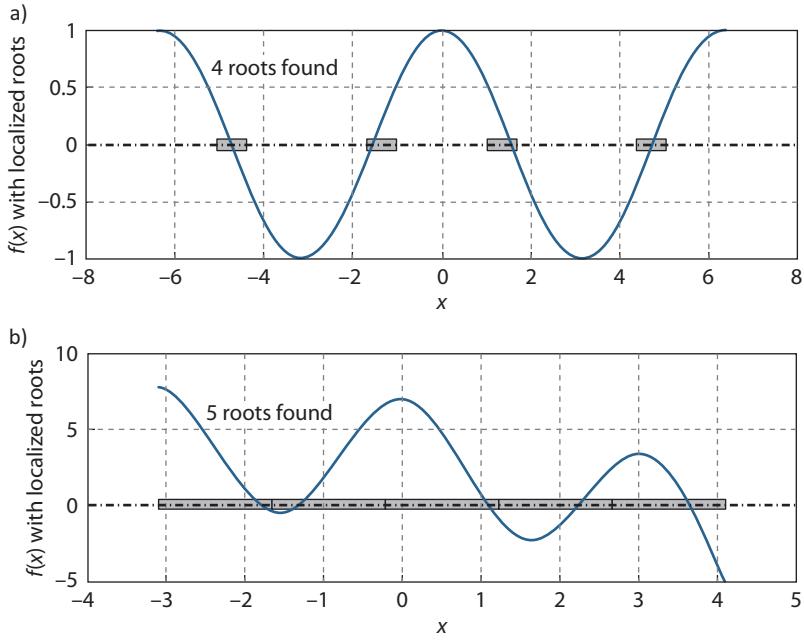


Fig. 10.22 Automatic individual bracketing of multiple roots.

10.5.2 Handling Root Multiplicity

The root x_* is called a zero of *multiplicity* k of the function $f(x)$ if there exists a real number $l \neq 0$ such that

$$l = \lim_{x \to x_*} \frac{f(x)}{|x - x_*|^k} \tag{10.41}$$

Geometrically, the roots of nonunity multiplicity show up as a point where a function is tangent to the x-axis, as shown in two examples of Figs. 10.23a, b.

As mentioned already, neither the bracketing methods nor open methods can explicitly handle multiple roots. The fact that the function does not change a sign at the even multiple roots (*double*, *quadruple*, etc.) precludes the use of bracketing methods. The open methods may diverge.

The latter is especially true for the Newton–Raphson and secant methods, both of which contain the current derivative (or its estimate) in the denominator [Eqs. (10.33) and (10.36), respectively]. Because at the multiple roots not only $f(x)$, but also $f'(x)$, goes to zero, both recurrent formulas encounter division by a very small number as they approach the root. That is why it is important to have a safeguard in root-finding software, as suggested at the end of Sec. 10.3.2, warning about $f'(x)$ approaching zero.

Even if the Newton–Raphson or secant method happens to converge, in the case of a nonunity multiplicity root, the convergence rate degrades

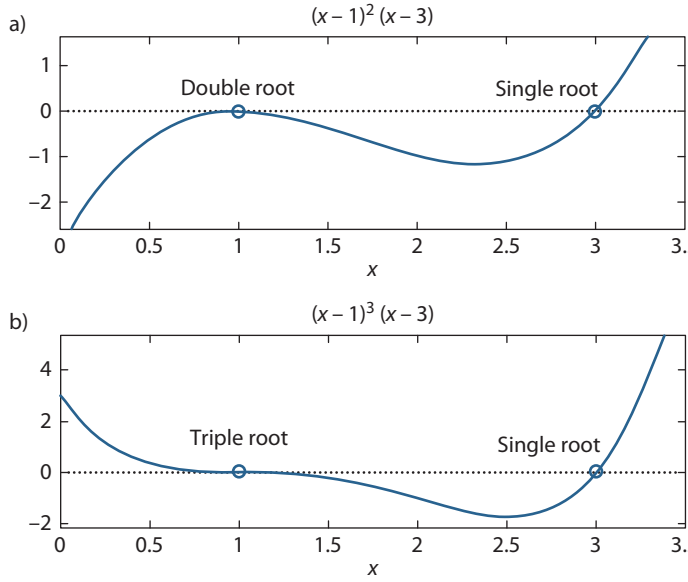


Fig. 10.23 Examples of a) double and b) triple root multiplicity as compared with a single-multiplicity root.

to linear as opposed to quadratic (Fig. 10.20). To restore a superb (quadratic) convergence, you may use a slightly modified version of Newton's equation (10.32)

$$x_{i+1} = x_i - m\frac{f(x_i)}{f'(x_i)} \tag{10.42}$$

where m is a multiplicity of the root. However, it implies foreknowledge of the root's multiplicity, which is not always available.

Another alternative, suggested by Ralston and Rabinowitz, is to replace $f(x)$ with a new function

$$u(x) = \frac{f(x)}{f'(x)} \tag{10.43}$$

which by construction has the same roots. Then, Eq. (10.32) for the nonunity multiplicity roots becomes

$$x_{i+1} = x_i - \frac{f(x_i)f'(x_i)}{(f'(x_i))^2 - f(x_i)f''(x_i)} \tag{10.44}$$

As seen, having $f'(x) \to 0$ no longer poses a problem. Obviously, implementation of this formula requires more computational resources (to compute or estimate $f''(x)$ along with $f'(x)$ and $f(x)$) and therefore should be used with caution. To conserve computational resources when handling the nonunity multiplicity roots, you may consider employing Eq. (10.43) for the secant method (Eq. (10.36)) as well

$$x_{i+1} = x_i - \frac{u(x_i)(x_i - x_{i-1})}{u(x_i) - u(x_{i-1})} \tag{10.45}$$

If neither of the aforementioned two options helps, you can always reformulate a nonunity multiplicity root-finding problem to that of finding a function's minimum and solve it using the optimization tools of MATLAB (discussed in Sec. 10.7).

10.5.3 Value-Finding Problem

Up to this point, we have been dealing with a root-finding problem. However, algorithms introduced in the previous sections of this chapter are suitable for a value-finding problem as well. This problem can be formulated as follows. Given a function $F(x)$ and a value η, find ξ so that $F(\xi) = \eta$.

Obviously, to handle this problem, we can use any root-finding approach on a translated function

$$f(x) = F(x) - \eta \tag{10.46}$$

This gives you the desired ξ so that $f(\xi) = 0$, and hence $F(\xi) = \eta$.

10.6 Roots of Polynomials

In principle, any of the approaches discussed in Secs. 10.2 and 10.3 may be applied to find the roots of a polynomial equation

$$f_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 \tag{10.47}$$

where n is the polynomial's order and a_i, $i = 0, 1, \ldots, n$ are its real constant coefficients. To be more accurate, the applicability and efficacy of these approaches depend on whether the roots of Eq. (10.47) are complex. If only real roots exist, then any of the previously described methods can be employed. Yet, the problem of finding good initial guesses would complicate both bracketing and open methods, whereas as usual, open methods can be susceptible to divergence.

Once solved for one root, the root location procedure may be continued to find the next root. However, if the initial guess is not accurate, you may end up finding the same root again and again. Therefore, it makes sense to remove the found root before attempting to find another one. The removal process is referred to as *polynomial deflation*. Several computer algorithms are available for performing this operation. For instance, the algorithm

```
n=length(a)
r=a(1);
a(1)=0;
for i=2:n
     s=a(i);
     a(i)=r;
     r=s+r*t;
end
```

uses a simple method, the so-called *synthetic vision approach*, dividing a polynomial of Eq. (10.47), defined via the vector of its coefficients `a`, by a monomial $x - t$. If `t` is a root of the polynomial, the reminder `r` will be zero, and coefficients of the quotient will be stored in the same vector `a` (with `a(1)`, corresponding to x^n, equal zero).

Because each calculated root is known only approximately, the deflation is sensitive to round-off errors. In some cases, they can grow to the point that the results can become meaningless. One way to reduce the round-off errors is to consider each successive root estimate obtained during deflation not as a final solution but rather a hypothesis. These hypotheses can then be used as the initial guesses (one by one) to run the root-finding algorithm using the original nondeflated polynomial yet again. This procedure is referred to as *root polishing*.

For finding complex roots, only open methods can be employed. Among them, the conventional Newton–Raphson method provides a viable approach. In this case, the deflation algorithm should be capable of dividing the polynomial by a binomial rather than just a monomial, as was the case for

the real roots. As you might expect, the procedure is susceptible to convergence problems.

For this reason, special methods to find the real and complex roots of a polynomial (still related to the open method approach) have been developed. MATLAB utilizes an algorithm involving computing eigenvalues of the *companion matrix*

$$A = \begin{bmatrix} -\frac{a_{n-1}}{a_n} & -\frac{a_{n-2}}{a_n} & \cdots & -\frac{a_1}{a_n} & -\frac{a_0}{a_n} \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \tag{10.48}$$

Because the characteristic polynomial of this matrix yields the nth-order polynomial [Eq. (10.47)], the eigenvalues of matrix A represent the roots of this polynomial. It is as simple as that! (This example again demonstrates the beauty of dealing with matrices rather than with algebraic equations.)

The MATLAB function `roots` can essentially be reduced to the three lines of code that construct the companion matrix and compute its eigenvalues:

```
A = diag(ones(n-1,1),-1);
A(1,:) = -a(2:n+1)./a(1);
x=eig(A)
```

Moreover, there is a special function to compute a companion matrix, `compan`, that can simplify code even further, to just a single line:

```
x=eig(compan(a))
```

Some examples of using the `roots` function will be provided in Sec. 10.9.

10.7 From Root Finding to 1-D Unconstrained Optimization

Root finding and optimization are related in the sense that both involve guessing and searching for an argument where a function $f(x)$ satisfies a certain condition. The only difference between the two types of problems is that root finding involves searching for function's zeros [Eq. (10.1)], whereas optimization involves searching for an extremum (minimum or maximum) value [Eq. (10.2)].

It should be pointed out that if you have an algorithm that allows you to find a minimum value of the function $f(x)$, you can always use it to find a maximum as well, because

$$\max_x (f(x)) = \min_x (-f(x)) \tag{10.49}$$

As a result, MATLAB features several functions allowing finding a minimum (not a maximum per se). These functions can be used for root-finding as well

[Eq. (10.3)]. Similarly, the root-finding functions may be applied to find an extremum, because the first-order derivative, $f'(x)$, becomes zero there. Such a point, x_*, may be a local maximum, if $f''(x_*) < 0$ or a local minimum, if $f''(x_*) > 0$. If $f''(x_*) = 0$, a *higher-order derivative test* up to the first nonzero derivative, $f^{(n+1)}(x_*)$, determines four possibilities (local maximum or minimum for odd n and the same signs as above, or strictly decreasing/increasing *point of inflection* for even n).

In what follows, just for educational purposes, we will consider three simple algorithms derived directly from the root-finding techniques, presented in Secs. 10.2 and 10.3. These algorithms are used in the `fminbnd` and `fminunc` functions of MATLAB. The `fminunc` function is then addressed in more detail in Sec. 10.8, which also formally introduces two more functions, `fminsearch` and `fmincon`.

10.7.1 Golden-Section Method

The ternary search golden-section method presented in Sec. 10.2.3 can easily be modified to search for a minimum value of a 1-D *unimodal* function. (A function $f(x)$ between two bounds is called unimodal if for some value m (local minimum) it is monotonically decreasing for $x < m$ and monotonically increasing for $x > m$.) This algorithm, complemented with an illustration of a few consecutive steps in Fig. 10.24, is as follows:

1. Given $f(x)$, determine the initial interval $[a;c]$ containing a single minimum.
2. Find two trial points, b_1 and b_2, as prescribed by Eq. (10.17) (Fig. 10.24a).
3. If the size of a new interval $[a;b_1]$ is less than or equal to the predefined tolerance in Eq. (10.18), stop iterations, assuming the solution at $x_* = b_1$.
4. If Eq. (10.18) does not hold, continue iterations by choosing the correct interval among the two possible ones ($[a;b_2]$ and $[b_1;c]$) (Fig. 10.24b).
   - If inequality $f(b_1) < f(b_2)$ holds, then assign $c = b_2$, $b_2 = b_1$ and find $b_1 = c - R^{\otimes}(c - a)$ (transition from Fig. 10.24c to Fig. 10.24d)
   - Otherwise assign $a = b_1$, $b_1 = b_2$ and find $b_2 = a + R^{\otimes}(c - a)$ (transition from Fig. 10.24b to Fig. 10.24c)
5. Go to Step 3 with a new interval.

This is a bracketing method, and therefore if the initial interval is chosen correctly (that is, if it contains a single minimum), the algorithm guarantees convergence. This algorithm is implemented in the `fminbnd` function of MATLAB, which solves for $\min f(x)$ subject to $x_{\min} \le x \le x_{\max}$.

As you already know, this algorithm does not provide the fastest convergence, and that is why `fminbnd` is supplemented with another derivative-free algorithm, which is considered next.

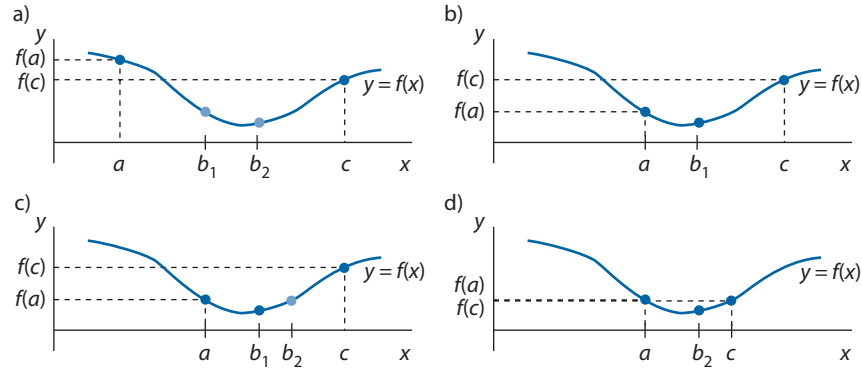


Fig. 10.24 Illustration of using the golden-section algorithm to find a minimum of a function.

10.7.2 Quadratic Interpolation Method

Quadratic interpolation takes advantage of the fact that a second-order polynomial often provides a good approximation to the shape of a function $f(x)$ near an extremum, as shown in Fig. 10.25a.

As in the IQI method of Sec. 10.3.4, having three iterations, $(x_{i-2}, f(x_{i-2}))$, $(x_{i-1}, f(x_{i-1}))$, and $(x_i, f(x_i))$, we can first construct a quadratic function [Eq. (10.37)] with coefficients [Eq. (10.38)] that fits a parabola to these points (Fig. 10.25b). Then, we define the extremum value by differentiating this approximation and equating it to zero, which results in the next candidate iteration being

$$x_{i+1} = x_i - \frac{B}{2A} \tag{10.50}$$

or

$$x_{i+1} = \frac{1}{2}\frac{f(x_{i-2})\Lambda_{i,i-1} - f(x_{i-1})\Lambda_{i,i-2} + f(x_i)\Lambda_{i-1,i-2}}{f(x_{i-2})\Delta_{i,i-1} - f(x_{i-1})\Delta_{i,i-2} + f(x_i)\Delta_{i-1,i-2}} \tag{10.51}$$

$(\Lambda_{i,j} = x_i^2 - x_j^2,\ \Delta_{i,j} = x_i - x_j)$.

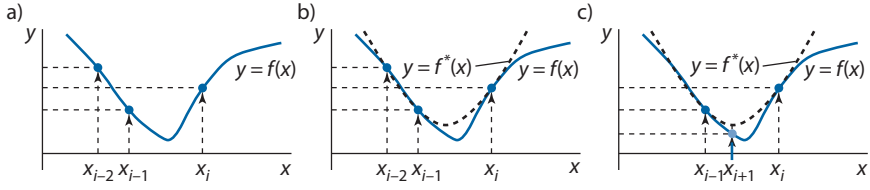


Fig. 10.25 Quadratic interpolation in the vicinity of function's extremum.

If this candidate point is reasonable (falls within $[x_{i-2}; x_i]$), it is accepted (Fig. 10.25c) and a new quadratic approximation is made using a new triple, $(x_{i-1}, f(x_{i-1}))$, $(x_i, f(x_i))$, and $(x_{i+1}, f(x_{i+1}))$. The MATLAB script that computes Eq. (10.51) is given as

```
%% Define symbolic variables
syms x a b c fa fb fc xim2 xim1 xi fim2 fim1 fi
%% Put together and solve a system of linear algebraic equations
A = [(a-c)^2 (a-c) 1;
     (b-c)^2 (b-c) 1;
     (c-c)^2 (c-c) 1];
B = [fa; fb; fc];
coef=A\B;
%% Find an argument corresponding to parabola's maximum
m=c-coef(2)/coef(1)/2;
%% Make substitutions
m=subs(m,{a,b,c,fa,fb,fc},{xim2,xim1,xi,fim2,fim1,fi});
%% Reorganize the terms and display the result
m=collect(m,[fim2 fim1 fi]); pretty(m)
```

It returns

```
        2    2           2      2             2      2
  (ximl - xi )  fim2 + (xi - xim2 )  fim1 + (xim2 - xim1 ) fi
-----------------------------------------------------------
(2 xim1 - 2 xi) fim2 + (2 xi - 2 xim2) fim1 + (2 xim2 - 2 xim1) fi
```

10.7.3 Newton's Method

While the `fminbnd` function uses a combination of two derivative-free methods, the `fininunc` function relies on Newton's method, which is adapted from Eq. (10.32) for the minimum-finding problems as

$$x_{i+1} = x_i - \frac{f'(x_i)}{f''(x_i)} \tag{10.52}$$

This equation uses the fact that at extremum, the first-order derivative equals zero. Being an open method, this algorithm shares all advantages and disadvantages of the original Newton's method [Eq. (10.32)] developed for root finding. If the first- and second-order derivatives are not readily available, a secant-like version of Newton's method employing appropriate finite-difference approximations like the one in Eq. (10.35) can be used.

10.8 Introduction to Multidimensional Optimization

Among the four major MATLAB optimization functions, `fminbnd`, `fminsearch`, `fminunc`, and `fmincon` (the complete list of functions will be presented in Sec. 10.9.2), only the first one handles a function of a

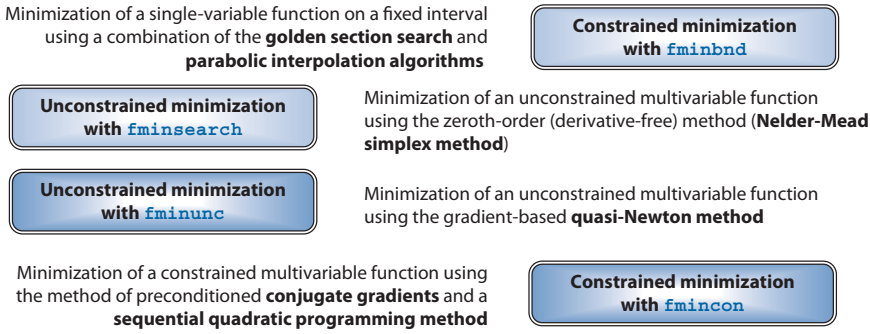


Fig. 10.26 Major optimization functions of MATLAB.

single variable, $f(x)$. Other functions deal with a more general case when the function depends on several variables $f(\mathbf{x}) = f(x_1, x_2, \ldots, x_n)$ (Fig. 10.26).

Among the remaining three, both `fminunc` and `fmincon` one way or another use an analog of Eq. (10.52). In the case of a multivariable function, this equation becomes

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \frac{\nabla f(\mathbf{x}_i)}{|\mathbf{H}(\mathbf{x}_i)|} \tag{10.53}$$

where instead of the first- and second-order derivatives the *gradient* of f

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n}\right]^T = [f_{x_1}, f_{x_2}, \ldots, f_{x_n}]^T \tag{10.54}$$

and the *Hessian* (matrix)

$$\mathbf{H}(\mathbf{x}) = \begin{bmatrix} \dfrac{\partial^2 f}{\partial x_1^2} & \dfrac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \dfrac{\partial^2 f}{\partial x_1 \partial x_n} \\ \dfrac{\partial^2 f}{\partial x_2 \partial x_1} & \dfrac{\partial^2 f}{\partial x_2^2} & \cdots & \dfrac{\partial^2 f}{\partial x_2 \partial x_n} \\ \cdots & \cdots & \cdots & \cdots \\ \dfrac{\partial^2 f}{\partial x_n \partial x_1} & \dfrac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \dfrac{\partial^2 f}{\partial x_n^2} \end{bmatrix} = \begin{bmatrix} f_{x_1x_1} & f_{x_1x_2} & \cdots & f_{x_1x_n} \\ f_{x_1x_2} & f_{x_2x_2} & \cdots & f_{x_2x_n} \\ \cdots & \cdots & \cdots & \cdots \\ f_{x_1x_n} & f_{x_2x_n} & \cdots & f_{x_nx_n} \end{bmatrix} \tag{10.55}$$

are used. That is why these methods are called *gradient-based* methods (as opposed to *nongradient* methods that do not evaluate a gradient). As you can imagine, a numerical estimation of the *Hessian* matrix, especially for a large n, causes a major problem.

One way to avoid computing a Hessian is to use the *gradient descent* method. To find a local minimum of $f(\mathbf{x})$, this method takes steps that are proportional to the negative of the gradient (or approximate gradient) of

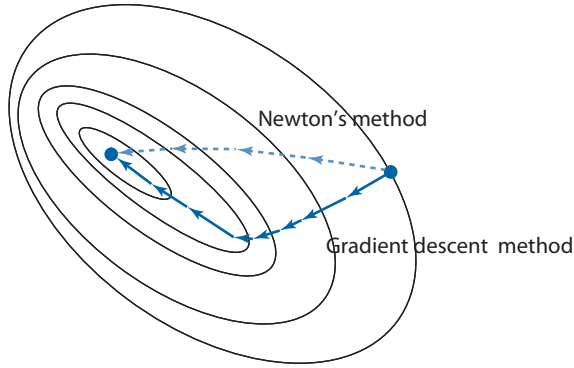


Fig. 10.27 Newton's method vs gradient descent method.

the function at the current point:

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \alpha_i \nabla f(\mathbf{x}_i) \tag{10.56}$$

The value of the step size $0 < \alpha_i << 1$ can be constant or variable (changing at every iteration). Geometrically, these steps are orthogonal to the multidimensional surface (in the 2-D case, they are orthogonal to the contour line going through that point).

Figure 10.27 presents a qualitative comparison of the gradient descent and Newton's methods for minimizing a function of two variables with a small step size. The geometric interpretation of Newton's method in this case is that at each iteration i, Eq. (10.53) approximates $f(\mathbf{x})$ by a quadratic function around $\mathbf{x}_i$ and then takes a step toward the minimum of that quadratic function. As seen, because Newton's method uses curvature information (residing in a Hessian), it results in a more direct route to the minimum; therefore, the capability to estimate a Hessian matrix is essential for the convergence performance.

In *quasi-Newton methods*, the Hessian matrix is approximated at each (or every other) step using the updates specified by gradient evaluations (or approximate gradient evaluations). Quasi-Newton methods are a generalization of the secant method to find the root of the first-order derivative for multidimensional problems. Since in this case, the secant equation does not specify a unique solution, quasi-Newton methods differ in how they do that. The Broyden–Fletcher–Goldfarb–Shanno (BFGS) method, implemented in MATLAB's `fminunc` function, is one of the most popular members of this class. For completeness, its numerical procedure is provided as follows:

1. Given an initial guess $\mathbf{x}_0$ and approximate Hessian matrix $\hat{\mathbf{H}}_0$, which can be as simple as an identity matrix $\mathbf{I}$ meaning the gradient descent step [Eq. (10.56)], repeat the following steps until

$$\|\mathbf{x}_{i+1} - \mathbf{x}_i\| \leq \varepsilon \tag{10.57}$$

2. Obtain a direction $\mathbf{p}_i$ by solving

$$\hat{\mathbf{H}}_i \mathbf{p}_i = -\nabla f(\mathbf{x}_i) \tag{10.58}$$

3. Perform a line search to find an acceptable step size α_i and compute the next iteration as

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \alpha_i \mathbf{p}_i \tag{10.59}$$

4. Set $\mathbf{s}_i = \alpha_i \mathbf{p}_i$, evaluate $\mathbf{y}_i = \nabla f(\mathbf{x}_{i+1}) - \nabla f(\mathbf{x}_i)$, and update an estimate of the Hessian as

$$\hat{\mathbf{H}}_{i+1} = \hat{\mathbf{H}}_i + \frac{\mathbf{y}_i \mathbf{y}_i^T}{\mathbf{y}_i^T \mathbf{s}_i} - \frac{\hat{\mathbf{H}}_i \mathbf{s}_i (\hat{\mathbf{H}}_i \mathbf{s}_i)^T}{\mathbf{s}_i^T \hat{\mathbf{H}}_i \mathbf{s}_i} \tag{10.60}$$

The *line search* mentioned in Step 3 searches along the search direction $\mathbf{p}_i$, attempting to decrease the objective function $f(\mathbf{x})$. There may be different variants, but one of the basic ones is to start with the value of one and continually halve the step length (bisection) until a reduction is achieved. Then, a quadratic approximation (using the current and two other points along the direction of search) is used to determine the best step size.

The `fminsearch` function of MATLAB uses the nongradient Nelder–Mead simplex method, which is referred to as one of the most effective *direct search* methods. The term direct (or pattern) search was introduced by Hooke and Jeeves, the authors of another gradient-free method. The Hooke–Jeeves algorithm (realized in a user-written `fminhj` function provided in Appendix D) simply samples the function around the current point, one argument at a time, by the steps of the same magnitude, makes one or several steps in the direction of the largest decrease, and then repeats the pattern search again (halving the step size when no further decrease is observed). The Nelder–Mead method bases its pattern on a *simplex*, which in the n-dimensional space is characterized by the $n + 1$ distinct vectors, simplex vertices. At each step of the search, a new point in or near the current simplex is generated. The function value at the new point is compared with the function's values at the vertices of the simplex, and, usually, one of the vertices is then replaced by the new point, giving a new simplex. This step is repeated until the diameter of the simplex is less than the specified tolerance. Both the Hooke–Jeeves and Nelder–Mead methods are also referred to as the *random-walk*, *univariate* (changing one variable at a time), or *cyclic coordinate descent* methods.

Finally, the `fmincon` function deals with the constrained optimization. As opposed to addressing a simple problem

$$\mathbf{x}_* = \arg\left(\min_{\mathbf{x}} f(\mathbf{x})\right) \tag{10.61}$$

this function tries to find the solution to Eq. (10.61) subject to a variety of constraints. These constraints represent lower- and upper-bound constraints

$$\mathbf{x}_{lb} \leq \mathbf{x} \leq \mathbf{x}_{ub} \tag{10.62}$$

linear inequality and equality constraints

$$\mathbf{A}(\mathbf{x}) \leq \mathbf{b}, \quad \mathbf{A}^{eq}(\mathbf{x}) = \mathbf{b}^{eq} \tag{10.63}$$

and nonlinear inequality and equality constraints

$$\mathbf{c}(\mathbf{x}) \leq \mathbf{0}, \quad \mathbf{c}^{eq}(\mathbf{x}) = \mathbf{0} \tag{10.64}$$

This is generally referred to as a *constrained nonlinear optimization* or *nonlinear programming* problem. Depending on the scale of the problem, the `fmincon` function employs *sequential quadratic programming*, *trust-region*, or the *preconditioned conjugate gradient method*. For the medium-scale algorithm, the function computes a quasi-Newton approximation to the Hessian of the *Lagrangian* that combines the objective function [Eq. (10.61)] together with constraints [Eqs. (10.62)–(10.64)] using Lagrange multipliers:

$$\begin{aligned} L(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\mu}^T [\mathbf{x}_{lb} - \mathbf{x}; \mathbf{x} - \mathbf{x}_{ub}; \mathbf{A}(\mathbf{x}) - \mathbf{b}; \mathbf{c}(\mathbf{x})] \\ + \boldsymbol{\lambda}^T [\mathbf{A}^{eq}(\mathbf{x}) - \mathbf{b}^{eq}; \mathbf{c}^{eq}(\mathbf{x})] \end{aligned} \tag{10.65}$$

An estimate of the Hessian is updated at each iteration using the BFGS formula [Eq. (10.60)].

Examples of using the `fminbnd`, `fminsearch`, `fminunc`, and `fmincon` functions are provided in Sec. 10.9.2.

10.9 Using the MATLAB Functions

In what follows, we first discuss the two root-finding functions, followed by introduction of a variety of functions enabling unconstrained and constrained optimization of the functions of one or more variables.

10.9.1 Root-Finding Functions

As mentioned previously, MATLAB features only two functions for root finding. They are `fzero`, which is applicable for any function of one variable, and `roots`, which is used to find all roots of a polynomial.

The `fzero` function using the zeroin algorithm discussed in Sec. 10.4 has the following general syntax:

```
x = fzero(fun,x0)
```

This function tries to find a root of `fun` having `x0` as an initial guess. If `x0` is a scalar, the zero-finding routine proceeds with an open-method step in an attempt to find an initial bracketing interval. If `x0` is a vector of length two, `fzero` continues with a bracketing iteration, assuming `x0` to be an interval bracketing a root.

Alternatively, you may use one of the following syntaxes, enabling more options:

```
x = fzero(fun,x0,options)
[x,fval] = fzero(...)
[x,fval,exitflag] = fzero(...)
[x,fval,exitflag,output] = fzero(...)
```

In addition to the candidate solution, `x`, `fval` returns the value of an objective function `fun` at the solution `x` (a safety check to ensure it is zero), and `output` returns a structure that contains information about the iterative process:

- Algorithm used
- Number of function evaluations
- Number of iterations taken to find an initial bracketing interval (in the case the search started from a single guess)
- Number of zero-finding iterations

The `exitflag` returns the exit condition of `fzero`, as MATLAB sees it (Table 10.1).

The optional structure `options`, which can be modified using the `optimset` function, allows changing some parameters of the algorithm to meet your preferences (Table 10.2). The default values of these parameters can be found by issuing the following command:

```
>> optimset fzero
```

which returns a long list of parameters (because the `optimset` structure is also used by other functions of the Optimization Toolbox of MATLAB) with mostly empty fields, but among those, three with the nonempty fields are

```
     Display: 'notify'
        TolX: 2.2204e-016
FunValCheck: 'off'
```

Table 10.1 Exit Conditions for the `fzero` Function

exitflag Value	Description
1	Algorithm converged to a solution successfully based on termination tolerance options.TolX
-1	Algorithm was terminated by the output function (provided by a user)
-3	NaN or Inf function value was encountered during the search within an interval containing a sign change
-4	A complex function value was encountered during the search within an interval containing a sign change
-5	There is a suspicion that fzero might have converged to a singular point
-6	Algorithm could not detect a change in sign of the function to establish the initial bracketing interval

Table 10.2 Modifiable Optimization Parameters for the `fzero` Function

Parameter	Description
`Display`	Changes the way the results are displayed: `'off'` displays no output; `'iter'` displays output at each iteration; `'final'` displays just the final output; and `'notify'` (default) displays output only if the function does not converge
`FunValCheck`	Checks whether the objective function values are valid: `'on'` displays an error when the objective function returns a value that is complex or `NaN` and `'off'` (default) displays no error
`OutputFcn`	Specifies one (or more) user-defined function, called at each iteration, that can interrupt the iteration procedure
`PlotFcns`	Plots various measures of progress while the algorithm executes. You may write your own progress plotting function or select one of two predefined ones: `@optimplotx` plots the current point `@optimplotfval` plots the function value The default is none `([])`
`TolX`	Defines termination tolerance on `x` (the default is `eps`, 2.2204e−16)

To change any of these values, the field name followed by its new value should appear in the `optimset` function, for example,

```
>> options=optimset('Display','iter');
```

The simplest `fzero` call may look like

```
>> fzero(@sin,3)
```

which returns the root of $f(x) = \sin(x)$ closest to $x_0 = 3$:

```
ans = 3.1416
```

The `fzero` function also accepts an inline function. For example,

```
>> x=fzero(inline('sin(5*x)+cos(x)'),1)
```

finds the root of $f(x) = \sin(5x) + \cos(x)$ closest to $x_0 = 1$:

```
x = 0.7854
```

A more sophisticated call

```
>> Opt=optimset('display','iter');
>> [X,fval,exitflag]=fzero('cos(x)+x',[-2 2],Opt)
```

allows obtaining more detailed information:

```
Func-count     x              f(x)             Procedure
2                  2            1.58385      initial
3            0.416147             1.3308      interpolation
4           -0.791927          -0.0894511      bisection
5           -0.715839           0.038704       interpolation
6           -0.738818        0.000446888       interpolation
7           -0.739085       -8.91982e-008      interpolation
8           -0.739085        5.26001e-012      interpolation
9           -0.739085                   0      interpolation
```

```
Zero found in the interval [-2, 2]
X        =  -0.7391
fval     =  0
exitflag =  1
```

As mentioned throughout this chapter, another way to check the validity of the obtained solution is to present it graphically. For example, the commands

```
>> ff='cos(x)+x';
>> [X,Y]=fzero(ff,1,optimset('display','iter','TolX',10^-2))
>> ezplot(ff), hold, plot(X,Y,'sm')
>> grid
```

display the results of finding a bracketing interval (where `fzero` spends most of the time) and several IQI iterations, along with the plot shown in Fig. 10.28a.

```
Search for an interval around 1 containing a sign change:
Func-count      a             f(a)         b      f(b)        Procedure
   1                1          1.5403       1     1.5403   initial interval
  ...
  21             0.36          1.2959    1.64    1.57085    search
  ...
  26         -0.810193       -0.120835    2.28    1.62877    search
```

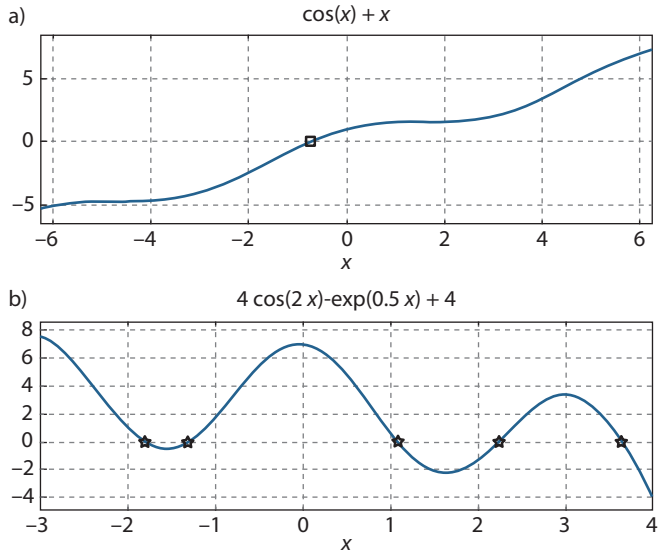


Fig. 10.28 a) Checking the results of root-finding graphically and b) finding multiple roots by employing the user-written `lroots` function.

```
Search for a zero in the interval [-0.810193, 2.28]:
Func-count     x          f(x)          Procedure
  26       -0.810193    -0.120835      initial
  27       -0.810193    -0.120835      interpolation
  28       -0.736766   0.00387855      interpolation
  29       -0.736766   0.00387855      interpolation
Zero found in the interval [-0.810193, 2.28]
X = -0.7368
```

Figure 10.28b presents the results of finding multiple roots using the `lroot` function introduced in Sec. 10.5.1. We use brackets found in Fig. 10.22b as the intervals supplied to the `fzero` function:

```
f=inline('4*cos(2*x)-exp(0.5*x)+4');
b=lroot(f,-3,4);
n=length(b); x=zeros(n);
ezplot(f,[-3 4]), hold, grid % the fplot function can be used as well
for i=1:n
    x(i)=fzero(f,b(i,:));
    plot(x(i),f(x(i)),'rp')
end
```

The `PlotFcns` field of the `optimset` structure (see Table 10.2) can be very useful in visualizing convergence of the objective function at each iteration as well. For example, the commands

```
>> Opt=optimset('TolX',10^-8,'PlotFcns',{@optimplotx,@optimplotfval});
>> X=fzero('cos(x)+x',1,Opt);
```

result in an animated plot as shown in Fig. 10.29. It should be noted that this field works the same way for all other functions discussed in the remainder of this chapter.

In the situation when the function only touches the x-axis but does not cross it, `fzero` returns no valid roots and executes until `Inf`, `NaN`, or a complex value is detected:

```
>> fzero('x^2',0.5)
Exiting fzero: aborting search for an interval containing a sign change
because NaN or Inf function value encountered during search.
(Function value at -1.7162e+154 is Inf.)
Check function or try again with a different starting value.
ans = NaN
```

You may request a more detailed report using

```
>>[x,fval,exitflag,output]=fzero('x^2',0.5)
```

which yields

```
Exiting fzero: aborting search for an interval containing a sign change
because NaN or Inf function value encountered during search.
(Function value at -1.7162e+154 is Inf.)
```

```
Check function or try again with a different starting value.
x = NaN
fval = NaN
exitflag = -3
output = intervaliter: 1038
          iterations: 0
           funcCount: 2076
           algorithm: 'bisection, interpolation'
             message: 'Exiting fzero: aborting search for an interval
                       containing a ...'
```

As seen, after 1038 open-method iterations, this call fails to find an initial bracketing interval. The `message` field of the `output` structure contains detailed information on the obtained result.

Likewise, for the function

$$f(x) = (x-1)^4(x-3) \tag{10.66}$$

similar to that shown in Fig. 10.23a, although initiated very close to the quadruple root, the `fzero` function converges to the second, single root

```
>> fzero('(x-1)^4*(x-3)',1.000001)
ans = 3
```

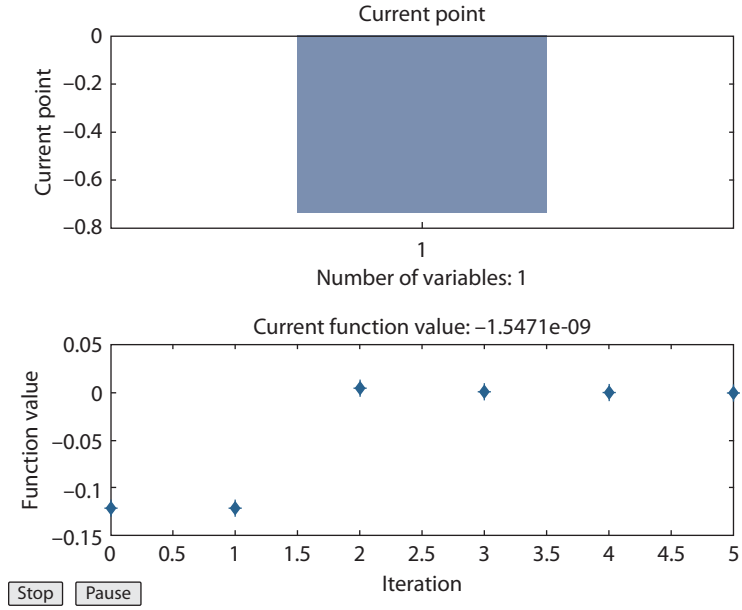


Fig. 10.29 Visualization of an objective function convergence using the `optimplotx` (upper plot) and `optimplotfval` (lower plot) functions, defined by the `optimset` structure.

Introducing a new function as suggested by Eq. (10.43)

$$u(x) = \frac{f(x)}{f'(x)} = \frac{(x-1)^4(x-3)}{4(x-1)^3(x-3)+(x-1)^4} = \frac{(x-1)(x-3)}{4(x-3)+(x-1)} \tag{10.67}$$

allows resolving the issue. The following script is intended to demonstrate this:

```
syms x
f=(x-1)^2*(x-3);
f_prime=diff(f);
u=f/f_prime;
[x1,y1]=fzero(char(u),-2); [x2,y2]=fzero(char(u),4);
ezplot(char(f)), hold    % ezplot(f) or fplot(f) would work as well
h=ezplot(char(u));       % ezplot(u) or fplot(u) would work as well
set(h,'Color',[0.8 0.1 0],'LineStyle','--')
plot(x1,y1,'cs'), plot(x2,y2,'mo'), grid
L=legend('original function f(x)','new function u(x)',...
         'quadruple root','single root','location','NW'), L.Box='off';
```

The results are shown in Fig. 10.30. Being initiated at $x_0 = -2$ and $x_0 = 4$, `fzero` successfully converges to $x_* = 1$ and $x_* = 3$, respectively.

The syntax for the `roots` function is

```
r=roots(p)
```

Here, `p` is a vector containing $n+1$ coefficients of the nth-order polynomial shown in Eq. (10.47) ($\mathbf{p} = [a_n, a_{n-1}, \ldots, a_0]^T$). For example, the command

```
>> r=roots([1 -10  35  -50  24])
```

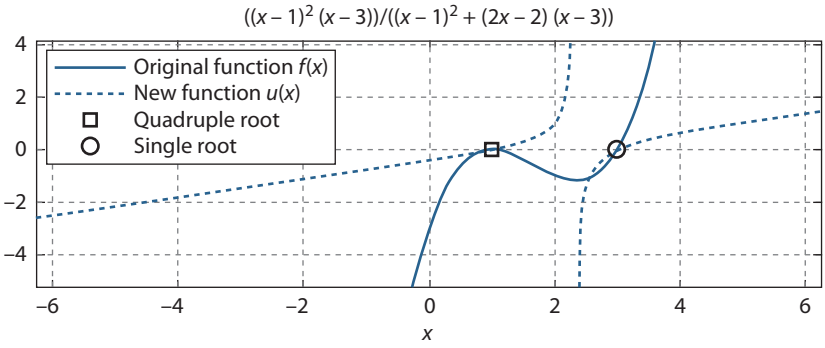


Fig. 10.30 Example of finding a quadruple root.

which means that you are trying to find the roots of the fourth-order polynomial (see Sec. 3.5)

$$f_4(x) = x^4 - 10x^3 + 35x^2 - 50x + 24 \tag{10.68}$$

returns

```
r = 4.0000
    3.0000
    2.0000
    1.0000
```

(in descending order) that is,

$$f_4(x) = (x-4)(x-3)(x-2)(x-1) \tag{10.69}$$

As mentioned, the `roots` function simply employs two other functions, `compan` and `eig`. These functions can be employed explicitly

```
>> A = compan([1  -10  35  -50  24])
>> r = eig(A)
```

to yield the companion matrix

```
A = 10    -35     50    -24
     1      0      0      0
     0      1      0      0
     0      0      1      0
```

and its eigenvalues (in descending order)

```
r = 4.0000
    3.0000
    2.0000
    1.0000
```

which are the roots of the polynomial.

Let us illustrate that the location of the roots can be very sensitive to perturbations in coefficients of the polynomial. The following script produces the so-called Wilkinson's polynomial

$$f_{20}(x) = (x-1)(x-2)\ldots(x-20) \tag{10.70}$$

then, slightly disturbs one of its coefficients and attempts to find the roots of this disturbed polynomial (Fig. 10.31a):

```
% Compose a 20th-order polynomial with the roots at x=1,2,...,20
p = poly(1:20);
or = roots(p);
% Plot the original roots (marked by the red crosses)
plot(1:20,or,'xr')
xlabel('Root number'), ylabel('Root value'), hold
```

```
delta = 2^(-10);
p(3) = p(3)-delta;  % disturb one coefficient
z=roots(p);         % find the roots of disturbed polynomial
% Add the 'disturbed' roots marked by blue circles
plot(1:20,z,'o')
legend('Original roots','Roots of disturbed polynomial')
```

Specifically, the coefficient $p_3 = 20{,}615$ (corresponding to a_{n-2} in Eq. (10.47)) is replaced with $p_3 = 20{,}614.999$, which may be seen as a negligible disturbance. Yet, some of the roots of the disturbed polynomial happen to be greatly displaced. In fact, many of the roots become complex (Fig. 10.31a shows only the real parts).

The problem is that, despite the fact that the original roots $r_i = i$, $i = 1,2,\ldots,20$ seem to be widely spaced (and therefore disturbing one of coefficients should not do much damage), it is the ratio $r_i/(r_i - r_j)$ that defines root's "closeness" of other roots in terms of stability. You may run the script

```
r=20:-1:1;    % define the roots in descending order
d1=r(3)-r; d1=d1 (d1~=0); s1=r(3)^19/abs(prod(d1))
d2=r(end)-r; d2=d2 (d2~=0); s2=r(end)^19/abs(prod(d2))
```

to find out that for the third largest root ($r_3 = 18$), the product

$$\prod_{\substack{j=1 \\ j\neq 3}}^{20} \frac{|r_3|}{|r_3 - r_j|} \tag{10.71}$$

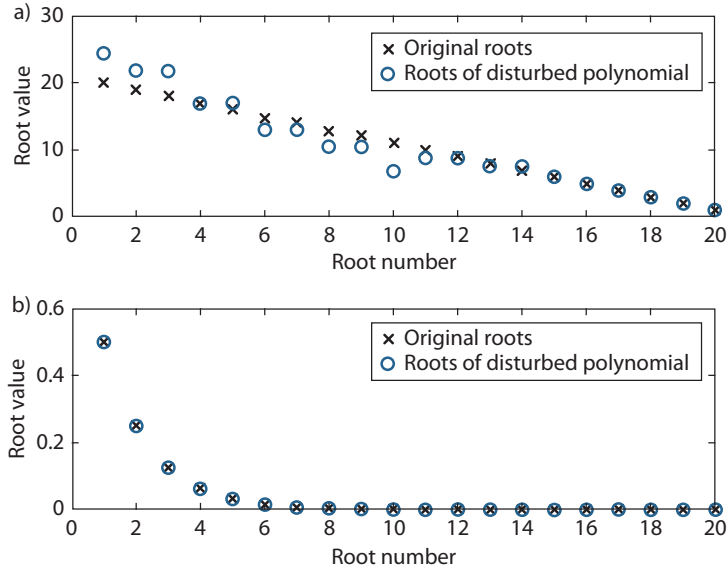


Fig. 10.31 a) Sensitive and b) nonsensitive Wikinson's polynomials.

is of the order $\sim 10^8$, whereas for the smallest root ($r_{20} = 1$), it is $\sim 10^{-18}$. It is no wonder the larger roots are so sensitive to even tiny disturbances—they have many close roots around in the sense that the distance $|r_i - r_j|$ between them is smaller than $|r_i|$.

Figure 10.31b shows similar results for another Wilkinson's polynomial

$$f_{20}(x) = (x - 2^{-1})(x - 2^{-2}) \dots (x - 2^{-20}) \tag{10.72}$$

produced using the same script as earlier, with only two lines changed:

```
a = poly(2.^-[1:20]);          % compose a new 20th-order polynomial
...
delta = a(3)*2^(-10)/20615; % same relative error as in the previous case
...
```

Computing the sensitivity coefficient [Eq. (10.71)] in this case with

```
r=2.^-[20:-1:1];
d1=r(3)-r; d1=d1 (d1~=0); s1=r(3)^19/abs(prod(d1))
d2=r(20)-r; d2=d2 (d2~=0); s2=r(20)^19/abs(prod(d2))
```

yields the orders of ~ 1 and 10^{-46}, respectively.

To conclude this section, let us also briefly mention one more function you might want to explore (on your own) and use in your applications, `fsolve`. As opposed to solving a scalar equation [Eq. (10.1)], this function solves a nonlinear system of equations

$$\mathbf{F}(\mathbf{x}) = \mathbf{0} \tag{10.73}$$

for $\mathbf{x}$, where $\mathbf{x}$ is a vector or matrix and $\mathbf{F}$ returns a vector.

Syntaxes for this function are pretty much the same as for the `fzero` function. In fact, for all examples of this section, `fsolve` could be used instead of `fzero` with no problem. This function uses a nonlinear least-squares algorithm, and as such, if the system (10.73) does not have a solution, it still returns a point where the residual is the smallest. The following self-explanatory script dealing with a 2-D nonlinear system of equations, saved as single file, provides you with an example of using this function:

```
%% Visualize equations
F1=@(x,y) x.^2+y.^2-26;
F2=@(x,y) 3*x.^2+25*y.^2-100;
fimplicit(F1,6*[-1 1],'--'), hold,
    fimplicit(F2)
xlabel('x'), ylabel('y'), axis equal, grid
%% Find a solution
x0=[-3,-1];
[x,fval,exitflag]=fsolve(@TryFsolve,x0); plot(x(1),x(2),'rp')
legend(char(F1),char(F2),'Solution','location','s'), Y=axis;
text(0.6*Y(2),0.8*Y(4),['Exit flag:' int2str(exitflag)])
%% Define a system of nonlinear equations
   function f=TryFsolve(x)
```

```
    f(1)=x(1).^2+x(2).^2-26;
    f(2)=3*x(1).^2+25*x(2).^2-100;
end
```

In this script, you first visualize your equations using the `fimplicit` function (that accepts implicit equations defined as $f(x,y) = 0$) and then assign the initial conditions and apply the `fsolve` function (which requires nonlinear equations to solve to be specified as a single-input function handle or function name). This example also illustrates that as of R2016b, the local function can be saved along with the main script, at the end of it (Sec. 5.8.3). This script results in the plot shown in Fig. 10.32a. Similar to `fzero`, when successful, `fsolve` only gives one root. Figure 10.32b shows another example of using the preceding script for a different 2-D nonlinear system.

10.9.2 Basic Optimization Functions

Formally, all optimization functions of MATLAB belong to the Optimization Toolbox (the majority of them are still available in the basic MATLAB). As of R2015a, the functions of this toolbox are divided into six groups, as shown in Table 10.3. (Note that the Systems of Nonlinear Equations group contains two functions that we dealt with in the previous section.)

The Optimization Toolbox GUI, mentioned at the end of Table 10.3 (Fig. 10.33), is also accessible by pushing **Optimization** icon of the MATLAB Toolstrip Apps tab (Fig. 2.6b). Even though this app (still available in R2020a) affords convenient ways to set options and run problems, it is said to be removed in a future release. Instead, MathWorks suggests using the `optimoptions` function to define a problem, which automatically shows appropriate suggestions for `optimoptions` names and values (as opposed to manually typing them in, when using the `optimset` function).

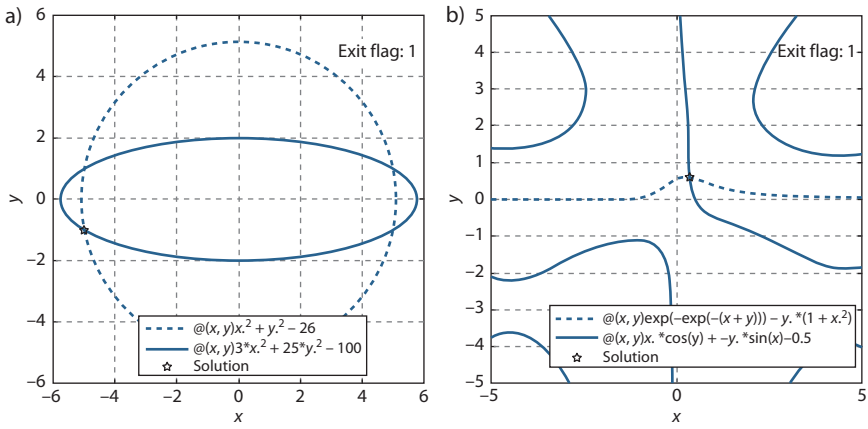


Fig. 10.32 Visualization of solving two 2-D nonlinear systems using the `fsolve` function.

Table 10.3 Optimization Toolbox Functions

Function	Description
The Nonlinear Optimization group	
`fminsearch`	Finds a minimum of an unconstrained multivariable function using a derivative-free method ($\min_{\mathbf{x}} f(\mathbf{x})$)
`fminunc`	Finds a minimum of an unconstrained multivariable function ($\min_{\mathbf{x}} f(\mathbf{x})$)
`fminbnd`	Finds a minimum of a single-variable function on a fixed interval ($\min_{a \le x \le b} f(x)$)
`fmincon`	Finds a minimum of a constrained nonlinear multivariable function ($\min_{\mathbf{x}} f(\mathbf{x})$), subject to a variety of constraints on $\mathbf{x}$)
`fseminf`	Finds a minimum of a semi-infinitely constrained multivariable nonlinear function ($\min_{\mathbf{x}} f(\mathbf{x})$), subject to a variety of constraints on $\mathbf{x}$)
`fgoalattain`	Solves a multiobjective goal attainment problem ($\min_{\mathbf{x}} \max_i (f_i(\mathbf{x}) - \eta_i) w_i^{-1}$), subject to a variety of constraints on $\mathbf{x}$ [w_i is a vector of relative importance factors of goals η_i; see Eq. (10.46)]
`fminimax`	Solves a minimax constraint problem $\min_{\mathbf{x}} \max_i f_i(\mathbf{x})$), subject to a variety of constraints on $\mathbf{x}$. This problem is a special case of the unscaled goal attainment problem with $w_i = 1$ and $\eta_i = 0$
The Linear Programming and Mixed-Integer Linear Programming group	
`intlinprog`	Addresses a mixed-integer linear programming (MILP) problem ($\min_{\mathbf{x}} \mathbf{f}^T\mathbf{x}$), subject to a set of linear and integer constraints on $\mathbf{x}$
`linprog`	Solves a linear programming problem ($\min_{\mathbf{x}} \mathbf{f}^T\mathbf{x}$), subject to a set of linear constraints on $\mathbf{x}$
The Quadratic Programming group	
`quadprog`	Addresses a quadratic programming problem ($\min_{\mathbf{x}} (\frac{1}{2}\mathbf{x}^T\mathbf{H}\mathbf{x} + \mathbf{f}^T\mathbf{x})$), subject to a set of linear constraints on $\mathbf{x}$
The Least-Squares group	
`lsqlin`	Solves a constrained linear least-squares problem ($\min_{\mathbf{x}} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2$), subject to a set of linear constraints on $\mathbf{x}$
`lsqnonneg`	Solves a nonnegative least-squares constraint problem ($\min_{\mathbf{x} \ge \mathbf{0}} \|\mathbf{C}\mathbf{x} - \mathbf{d}\|_2^2$)
`mldivide, \`	Solves a system of linear equations ($\mathbf{A}\mathbf{x} = \mathbf{b}$)
`lsqcurvefit`	Solves a nonlinear curve fitting (data-fitting) problem in least-squares sense ($\min_{\mathbf{x}} \|\mathbf{f}(\mathbf{x}, \mathbf{x}_d) - \mathbf{y}_d\|_2^2$)
`lsqnonlin`	Solves a nonlinear least-squares (nonlinear data-fitting) problem ($\min_{\mathbf{x}} \|\mathbf{f}(\mathbf{x})\|_2^2$)
The Systems of Nonlinear Equations group	
`fsolve`	Solves a system of nonlinear equations ($\mathbf{F}(\mathbf{x}) = \mathbf{0}$)
`fzero`	Finds a root of a nonlinear function ($f(x) = 0$)
The Optimization Problem Setup group	
`optimset`	Creates or edits an optimization options structure (common for all functions of the Optimization Toolbox)

(Continued)

Table 10.3 Optimization Toolbox Functions *(Continued)*

Function	Description
`optimoptions`	Creates or edits an optimization options structure (`optimoptions` is recommended instead of `optimset` for all solvers except `fzero`, `fminbnd`, `fminsearch`, and `lsqnonneg`)
`optimtool`	Brings up the Optimization Toolbox GUI (Fig. 10.33), allowing selecting a solver, varying optimization options, running problems using a friendly environment as opposed to doing it programmatically

Let us present several examples of using the four major minimization functions of the Nonlinear Optimization group, also shown in Fig. 10.26. The algorithm implemented by the `fminbnd` function is based on the golden-section search and parabolic interpolation. The simplest call is

```
x = fminbnd(fun,x1,x2)
```

It returns a value `x` that is a local minimum of the function `fun` residing in the interval `x1` $\leq$ `x` $\leq$ `x2`.

Similar to the `fzero` function (see Sec. 10.9.1), you can use an additional (optional) input `options` and up to four outputs, so that the most comprehensive call is

```
[x,fval,exitflag,output] = fzero(fun,x1,x2,options)
```

The `exitflag` values for `fminbnd` are given in Table 10.4; the list of modifiable `options` is pretty much the same as for the `fzero` function, and so Table 10.5 presents only two additional parameters beyond those given in Table 10.2.

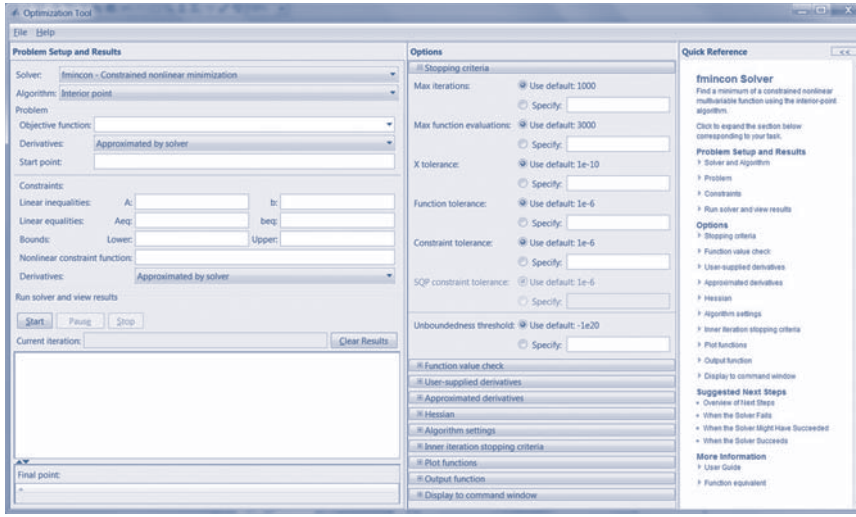


Fig. 10.33 Optimization Toolbox GUI.

Table 10.4 Exit Conditions for the `fminbnd` Function

`exitflag` Value	Description
1	Algorithm converged to a solution based on `options.TolX`
0	Maximum number of function evaluations or iterations was reached
-1	Algorithm was terminated by the output function
-2	Bounds are inconsistent (`x1 > x2`)

The default values of modifiable parameters can be obtained by calling

```
>> optimset fminbnd
```

which, among others, returns

```
      Display: 'notify'
  MaxFunEvals: 500
      MaxIter: 500
         TolX: 1.0000e-004
  FunValCheck: 'off'
```

As in the case of the `fzero` function, `fval` returns the value of an objective function `fun` calculated at the solution `x`, and `output` returns a structure with the following fields:

```
output.algorithm    Defines algorithm used
output.funcCount    Shows the number of function evaluations
output.iterations   Contains the number of iterations
output.message      Keeps an exit message
```

Suppose you are tasked to find a minimum of a cubic polynomial $f(x) = x^3 + x^2 - x + 1$ for $x \in [-1; 1]$. The following set of commands accomplishes this:

```
>> fun          = @(x) x^3+x^2-x+1;
>> options = optimset('TolX', 1e-2);
>> [x, fval, exitflag, output] = fminbnd (fun, -1, 1, options)
x =                0.3341
fval =             0.8148
exitflag = 1
output=
      struct with fields: iterations: 6
                 funcCount: 7
                 algorithm: 'golden section search, parabolic interpolation'
                   message: details on termination conditions
```

Table 10.5 Additional Modifiable Parameters for the `fminbnd` Function

Parameter	Description
`MaxFunEvals`	Maximum number of function evaluations allowed
`MaxIter`	Maximum number of iterations allowed

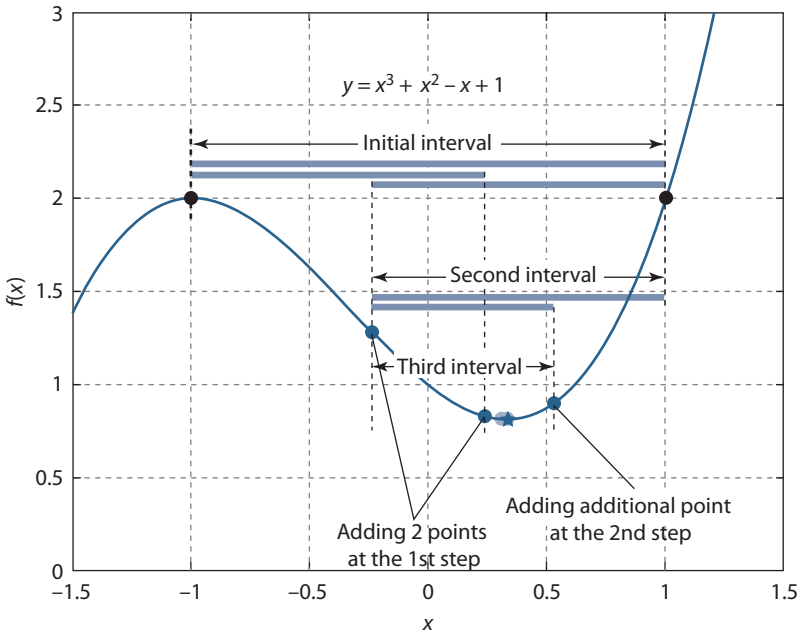


Fig. 10.34 Visualization of the `fminbnd` search routine.

A closer look (using the `'disp'` – `'iter'` name-value pair argument in the `optimset` structure) reveals that before `TolX` was reached, there were two golden-ratio steps followed by four parabolic-interpolation steps, as shown in Fig. 10.34.

The simplest call of the `fminsearch` function looks like that of `fminbnd`:

```
x=fminsearch(fun,x0)
```

Iterations start at `x0`, and a value of a local minimum is returned in `x`. The most comprehensive call is

```
[x,fval,exitflag,output] = fminsearch(fun,x0,options)
```

The default values of nonempty fields of the `options` structure can be obtained as usual via

```
>> optimset fminsearch
```

which returns

```
        Display: 'notify'
    MaxFunEvals: '200*numberofvariables'
        MaxIter: '200*numberofvariables'
         TolFun: 1.0000e-004
           TolX: 1.0000e-004
    FunValCheck: 'off'
```

As seen, the only new field (in addition to those of `fminbnd`) is `TolFun`, establishing termination tolerance on the function value. The `exitflag` values are the first three of Table 10.4, with the only difference for `flag=1`. In addition to satisfying the termination criteria `options.TolX` on $\mathbf{x}$, $f(\mathbf{x})$ should satisfy the convergence criteria `options.TolFun` as well. The `output` has the same fields as that of the `fminbnd` function.

The simplest call for the `fminunc` function is similar to those of `fminbnd` and `fminsearch`:

```
x=fminunc(fun,x0)
```

The most comprehensive call features two more output parameters along with the value of an objective function at the solution `x` (`fval`):

```
[x,fval,exitflag,output,grad,hessian]=fminunc(fun,x0,options)
```

Because it is based on quasi-Newton method, it also returns the values of the gradient (in `grad`) and Hessian (in `hessian`). The call

```
>> optimset fminunc
```

returns a lot of nonempty fields, but among those are

```
         Display: 'final'
     MaxFunEvals: '100*numberofvariables'
         MaxIter: 400
          TolFun: 1.0000e-006
            TolX: 1.0000e-006
     FunValCheck: 'off'
         Hessian: 'off'
      HessUpdate: 'bfgs'
  ObjectiveLimit: -1.0000e+20
```

(note, you could have used the `optimoptions fminunc` call as well).

If the Hessian field is `'on'`, `fminunc` uses a user-defined Hessian; if it is `'off'`, `fminunc` approximates the Hessian using finite differences. As for the `HessUpdate` option, `fminunc` allows you to choose either Eq. (10.60) (`'bfgs'`) or Eq. (10.56) (`'steepdesc'`). The `exitflag` values are presented in Table 10.6.

Finally, the simplest call for the `fmincon` function is

```
x=fmincon(fun,x0,A,b)
```

The most comprehensive ones include up to 10 inputs and 7 outputs:

```
x=fmincon(fun,x0,A,b,Aeq,beq,lb,ub,nonlcon,options)
[x,fval,exitflag,output,lambda,grad,hessian]=fmincon(...)
```

Additional inputs are to provide different type of constraints [Eqs. (10.62)–(10.64)], and one extra output, `lambda`, returns a structure whose fields contain the Lagrange multipliers at the solution `x` [vectors $\boldsymbol{\mu}$ and $\boldsymbol{\lambda}$ defined in Eq. (10.65)]. The order of the input arguments is important, and

Table 10.6 Exit Conditions for the `fminunc` Function

exitflag Value	Description
1	Magnitude of gradient became smaller than the specified tolerance `options.TolFun`
2	Change in `x` was smaller than the specified tolerance `options.TolX`
3	Change in the objective function value was less than the specified tolerance `options.TolFun`
5	Predicted decrease in the objective function was less than the `options.TolFun` tolerance
0	Number of iterations exceeded `options.MaxIter` or number of function evaluations exceeded `options.MaxFunEvals`
-1	Algorithm was terminated by the output function
-3	The objective function at current iteration went below `options.ObjectiveLimit`

so if some of the constraints are not present, an empty array, `[]`, should be used. Among others, the call

```
>> optimset fmincon
```

returns

```
         Display: 'final'
     MaxFunEvals: []
         MaxIter: []
          TolFun: 1.0000e-006
            TolX: []
     FunValCheck: 'off'
       Algorithm: 'interior-point'
         Hessian: []
  ObjectiveLimit: -1.0000e+20
          TolCon: 1.0000e-006
```

(try `optimoptions fmincon` as well). The last field defines the termination tolerance on the constraint violation. The `fmincon` function combines several algorithms so that the `Algorithm` option can be set as one of the following: `'interior-point'` (default), `'active-set'`, `'trust-region-reflective'`, `'sqp'`, or `'sqp-legacy'`. The `exitflag` values are given in Table 10.7.

Next, let us apply all aforementioned functions to solve the same problem of minimizing the function of two variables

$$f(\mathbf{x}) = e^{x_1}(4x_1^2 + 2x_2^2 + 4x_1x_2 + 2x_2 + 1) \tag{10.74}$$

First, we program Eq. (10.74) as an anonymous function

```
>> fun = @(x) exp(x(1))*(4*x(1)^2+2*x(2)^2+4*x(1)*x(2)+2*x(2)+1);
```

and define the initial guess

```
>> x0 = [0.95;-0.55];
```

Also, let us degrade the default termination tolerance to obtain a solution faster:

```
>> options = optimset('TolFun',1e-3,'TolX',1e-3);
```

Now, you may employ the `fminsearch` function as

```
>> [x,fval,exitflag,output]=fminsearch(fun,x0,options)
```

The output of this set of commands appears as follows:

```
x = 0.5003
  -1.0002
fval = 2.5164e-007
exitflag = 1
output = iterations: 32
         funcCount: 59
         algorithm: 'Nelder-Mead simplex direct search'
           message: 'Optimization terminated: the current x satisfies the
              termination criteria using OPTIONS.TolX of 1.000000...'
```

Table 10.7 Exit Conditions for the `fmincon` Function

exitflag Value	Description
1	Magnitude of gradient at the solution became less than `options.TolFun` and maximum constraint violation was less than `options.TolCon`
2	Change in `x` was smaller than the specified tolerance `options.TolX` and maximum constraint violation was less than `options.TolCon` (for all but the `active-set` algorithm)
3	Change in the objective function value was less than the specified tolerance `options.TolFun` and maximum constraint violation was less than `options.TolCon` (for the `trust-region-reflective` algorithm only)
4	Magnitude of the search direction was less than `2*options.TolX` and constraint violation was less than `options.TolCon` (for the `active-set` algorithm only)
5	Magnitude of directional derivative in search direction was less than `2*options.TolFun` and maximum constraint violation was less than `options.TolCon` (for the `active-set` algorithm only)
0	Number of iterations exceeded `options.MaxIter` or number of function evaluations exceeded `options.MaxFunEvals`
-1	Algorithm was terminated by the output function
-2	No feasible point was found
-3	The objective function at current iteration went below `options.ObjectiveLimit` and maximum constraint violation was less than `options.TolCon` (for `interior-point` and `sqp` algorithms)

Because we know the exact solution ($\mathbf{x}_* = [0.5, -1]^T$), we can also estimate the accuracy of the obtained solution

```
>> dx = norm([x(1)-0.5;x(2)+1])
```

which returns

```
dx = 3.3557e-004
```

It took 32 iterations (plus the additional 27 function evaluations needed to create a simplex where appropriate) to get from the initial ("wild") guess to the solution. The required tolerance of `1e-3` was met with the errors for $\mathbf{x}_*$ and $f(\mathbf{x}_*)$ of the order of `1e-4` and `1e-7`, respectively. The results of iterations are shown in Fig. 10.35. Figure 10.35a features a 3-D plot, also showing the objective function's topology in the vicinity of the minimum value, and in Fig. 10.35b, iterations overlie over a contour plot. It takes 18 iterations to arrive at the vicinity of the solution and then the additional 14 iterations (which are happening in the close vicinity of the solution) to satisfy the tolerance on $\mathbf{x}_*$.

The `PlotFcns` field of the `optimset` structure introduced in Table 10.2 can help visualizing the objective function convergence, similar to how it was done in Fig. 10.29. All you need to do is name the corresponding plotting functions

```
>> x=fminsearch(fun,x0,optimset('TolFun',1e-3,'TolX',1e-3,'PlotFcns',...
                {@optimplotx,@optimplotfval}));
```

The plots this command produces are shown in Fig. 10.36.

The advantage of using the `fminsearch` function is that, because it uses a gradient-free algorithm, it allows you to solve even nondifferentiable problems and can often handle discontinuities, particularly if they do not occur near the solution. For the same reason, the convergence of `fminsearch` is the slowest.

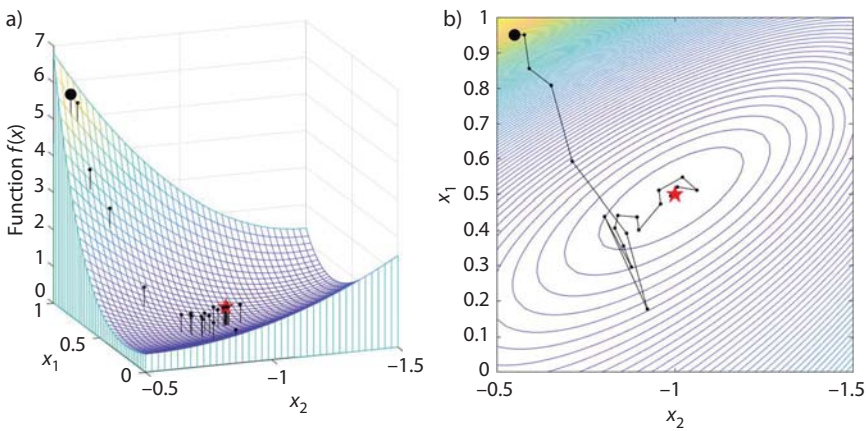


Fig. 10.35 Example of `fminsearch` performance.

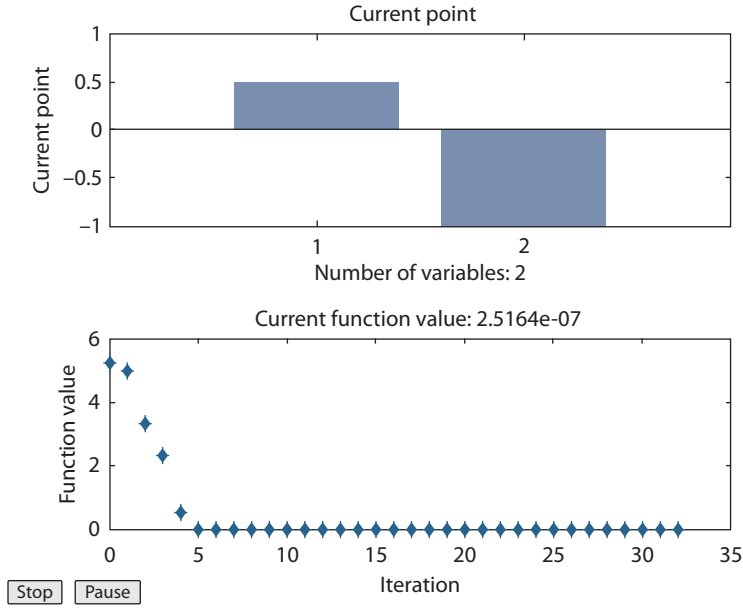


Fig. 10.36 Using the `optimplotx` (upper plot) and `optimplotfval` (lower plot) functions, defined by the `optimset` structure, to visualize convergence of an objective function.

Let us now minimize the same function [Eq. (10.74)] using the `fminunc` function. The command

```
>> [x,fval,exitflag,output]=fminunc(fun,x0,options)
```

returns

```
x = 0.5000
  -0.9997
fval = 1.9208e-007
exitflag = 1
output = iterations: 7
         funcCount: 27
          stepsize: 1
      firstorderopt: 0.0016
         algorithm: 'quasi-newton'
           message: 'Local minimum found. Optimization completed because the
                     size of the gradient is less than the selected ...'
```

The command

```
>> dx = norm([x(1)-0.5;x(2)+1])
```

returns

```
dx = 2.6151e-004
```

As opposed to the gradient-free algorithm, it only took seven iterations (plus the additional 20 function evaluations needed to estimate a direction and step size at each iteration) to get from the initial guess to the solution with an accuracy of `2.6e-4` for $\mathbf{x}_*$ and `1.9e-7` for $f(\mathbf{x}_*)$ (Fig. 10.37a). Obviously, `fminunc` executes faster than `fminsearch`.

However, although the `fminunc` function allows you to solve differentiable (continuous) problems like the one we just considered very quickly, it is not applicable for solving nondifferentiable problems and cannot handle discontinuity as `fminsearch` does. Also, for solving problems that are the sums of squares, another gradient-based function, `lsqnonlin`, might be even a better choice.

For the sake of comparison, Fig. 10.36b also presents the results of using a user-written `fminhj` function realizing another gradient-free method (mentioned in Sec. 10.8). To match the tolerances achieved for `fminsearch` and `fminunc` solutions, this function was initialized with

```
>> options = optimset('TolFun',1e-3,'TolX',1e-3);
```

and returned the error of `1.4e-4` for $\mathbf{x}_*$ and `1.7e-7` for $f(\mathbf{x}_*)$ (that is, better than those for `fminsearch` and `fninunc`). In general, the performance of `fminhj` is somewhat similar to that of `fminunc`, as opposed to `fminsearch`. In this particular case, the Hooke–Jeeves algorithm (`fminhj`) converged even faster than the gradient-based `fminunc` (six against seven iterations) and employed about the same number of function evaluations (26 vs 27), as shown in Fig. 10.38.

To be able to demonstrate the use of the constrained optimization function `fmincon`, we need to slightly modify the problem. Let us impose the

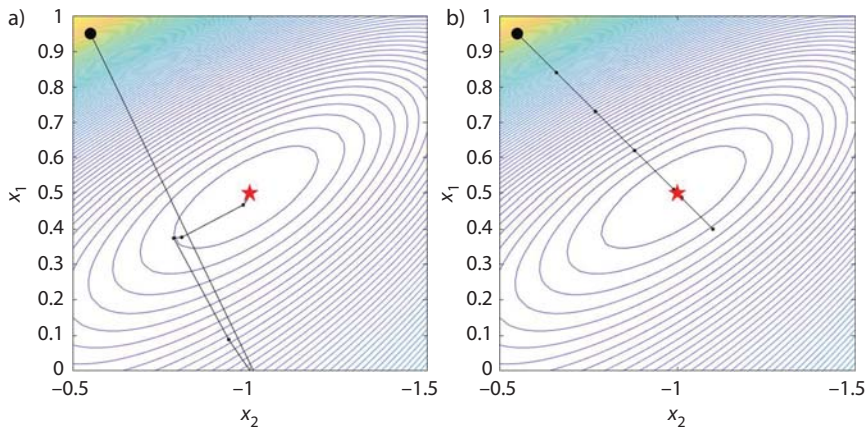


Fig. 10.37 Examples of performance of a) `fminunc` and b) Hooke-Jeeves algorithm.

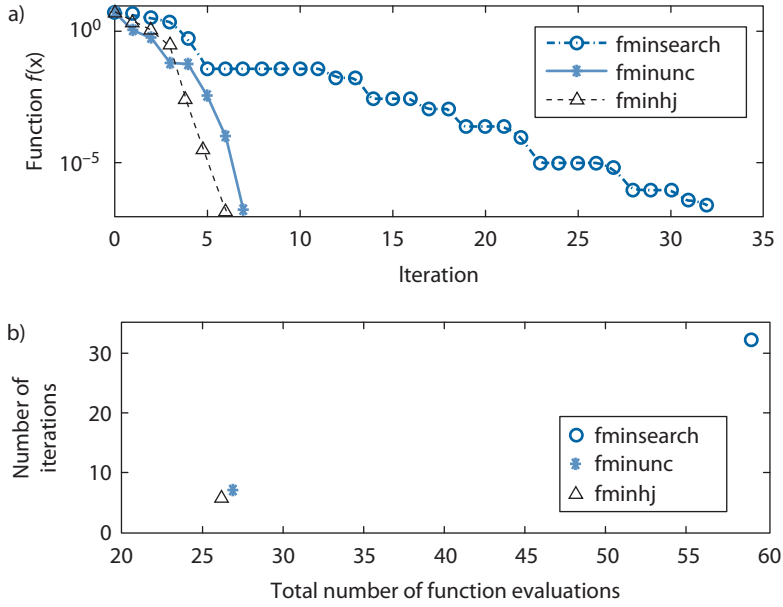


Fig. 10.38 Comparison of a) convergence history of various methods and b) their overall effectiveness.

following set of constraints:

$$\begin{aligned} c_1(\mathbf{x}) &= 0.2^2 - (x_1 - 0.6)^2 - (x_2 + 0.9)^2 \leq 0 \\ c_2(\mathbf{x}) &= -x_1 - 0.1(x_2 + 0.4)^{-1} \leq 0 \\ -1.15 &\leq x_2 \end{aligned} \tag{10.75}$$

Graphically, this means that the solution cannot be in any of the shaded areas shown on the 2-D projection in Fig. 10.39a. Now, we need to program these constraints. First, we create and save a `confun` function *m*-file that represents nonlinear constraints as follows:

```
function [c,ceq] = confun(x)
% relative inequality constraints
c(1) = 0.2^2-(x(1)-0.6)^2-(x(2)+0.9)^2;
c(2) = -x(1)-0.1./(x(2)+0.4);
ceq = []; % nonlinear equality constraints
```

(the last line must appear in this function even though we have no nonlinear constraints). Then, we invoke minimization by using the following set of commands (note that an empty array, `[]`, appears in the input arguments list instead of nonexisting constraints):

```
>> options=optimset('TolFun',1e-4,'TolX',1e-4,'TolCon',1e-4, ...
                    'Algorithm','active-set');
```

```
>> x0=[0.95;-0.55];
>> [x,fval,exitflag,output]=fmincon(fun,x0,[],[],[],...
                                    [],[0;-1.15],[],@confun,options)
```

The output appears as

```
x =          0.4000
            -0.9007
fval =       0.0298
exitflag =   5
  output = iterations: 10
           funcCount: 30
        lssteplength: 1
            stepsize: 1.8870e-04
           algorithm: 'active-set'
       firstorderopt: 0.0022
    constrviolation: -8.8144e-07
             message: 'Local minimum possible. Constraints satisfied. fmincon
                       stopped because the size of the current...'
```

The `fmincon` solution follows one of the constraints and rapidly (in 10 iterations and a total of 30 function evaluations) reaches the closest minimum located on another constraint (Fig. 10.39b).

Similar to `fminunc`, the gradient-based `fmincon` function allows solving feasible nonlinear constrained minimization problems, where the objective and constraint functions are both continuous and have continuous first-order derivatives, fairly fast. Similar to `fminsearch` the `fmincon` function is not applicable for solving nondifferentiable problems and cannot handle discontinuity though. Also, as well as all other functions, `fmincon` can only find a local minimum. In fact, the solution shown in

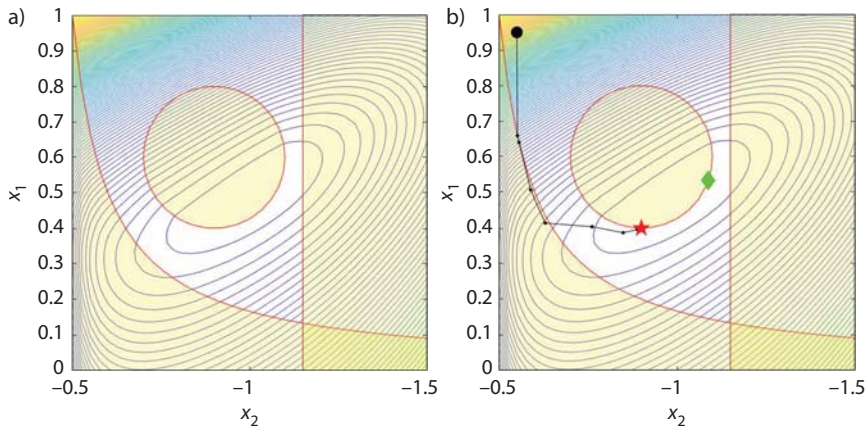


Fig. 10.39 a) Setting the constraints (prohibited areas) and b) example of `fmincon` performance.

Fig. 10.39b is a local minimum, whereas the global minimum is located farther to the right (depicted with the rhomb).

Can we compare the performance of the unconstrained optimization functions with that of the constrained optimization functions available in MATLAB? In other words, is it possible to employ unconstrained minimization algorithms (`fminsearch` and `fminunc`) to solve constrained minimization problems? The answer is yes. You can do it by augmenting the old objective function with weighted (quadratic) penalties for constraint violation. Specifically, for the problem posed as Eqs. (10.61) – (10.64), such augmented function becomes

$$f^*(\mathbf{x}) = f(\mathbf{x}) + w_{eq} \| \mathbf{p}_{eq} \|^2 + \mathbf{p}^T \text{diag}(\mathbf{w}) \mathbf{p} \tag{10.76}$$

where

$$\mathbf{p}_{eq} = \begin{bmatrix} \mathbf{c}_{eq}(\mathbf{x}) \\ \mathbf{A}_{eq}\mathbf{x} - \mathbf{b}_{eq} \end{bmatrix} \quad \text{and} \quad \mathbf{p} = \max \left(\mathbf{0}, \begin{bmatrix} \mathbf{c}(\mathbf{x}) \\ \mathbf{A}\mathbf{x} - \mathbf{b} \\ \mathbf{x}_{lb} - \mathbf{x} \\ \mathbf{x} - \mathbf{x}_{ub} \end{bmatrix} \right) \tag{10.77}$$

(note, the max operator returns a vector). Having this augmented function, we convert the problem to that of unconstrained minimization, because achieving $\min_{\mathbf{x}} f^*(\mathbf{x})$ implies $\min_{\mathbf{x}} f(\mathbf{x})$ and satisfying all constraints.

As an example, let us develop a new augmented function for Eq. (10.74) with constraints [Eq. (10.75)]:

$$\begin{aligned} f^*(\mathbf{x}) = f(\mathbf{x}) &+ \max([0, \ -1.15 - x_2])^2 + \max([0, c_2(\mathbf{x})])^2 \\ &+ w \max([0, c_1(\mathbf{x})]) \end{aligned} \tag{10.78}$$

For the sake of simplicity, we use a single weighting coefficient $w \geq 1$ to weight the most significant (active) inequality constraint. Augmenting the objective function simply means that we are reshaping the problem topology. Figure 10.40 shows an exaggerated landscape with $w = 20$. In practice though, there is no need to have such a high weighting coefficient because it disturbs the original topology too much. Having $w = 1$ or $w = 2$ may introduce enough information to account for the constraint.

An anonymous function describing the new, augmented objective function $f^*(\mathbf{x})$ can be programmed as

```
>> funA = @(x,w) exp(x(1))*(4*x(1)^2+2*x(2)^2+4*x(1)*x(2)+2*x(2)+1)...
          + max([0,-1.15-x(2)])^2+max([0,-x(1)-0.1/(x(2)+0.4)])^2 ...
          + w*max([0,0.2^2-(x(1)-0.6)^2-(x(2)+0.9)^2])
```

Now, we can define the initial guess and weighting coefficient

```
>> x0 = [0.95;-0.55]; w = 1;
```

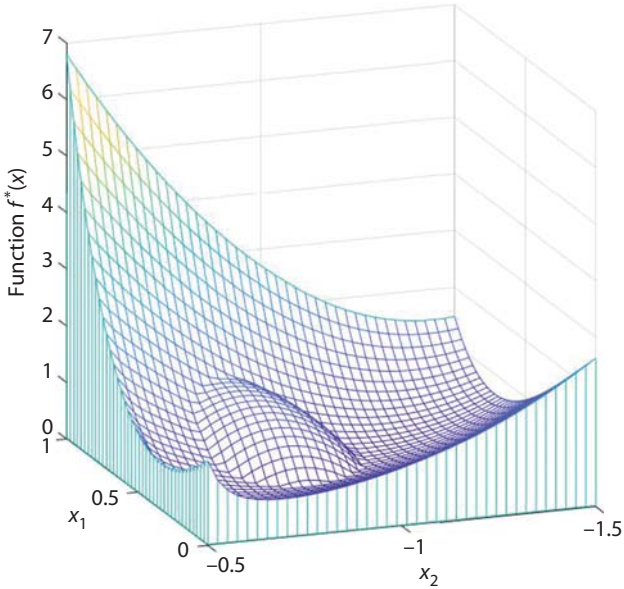


Fig. 10.40 Changing topology of the objective function.

and call either the `fminsearch` function

```
>> x = fminsearch(@(x)funA(x,w),x0)
```

or the `fminunc` function

```
>> x = fminunc(@(x)funA(x,w),x0)
```

The results for `w=1` and `w=2` are presented in Figs. 10.41 (for `fminsearch`) and 10.42 (for `fminunc`). As seen, when using either function

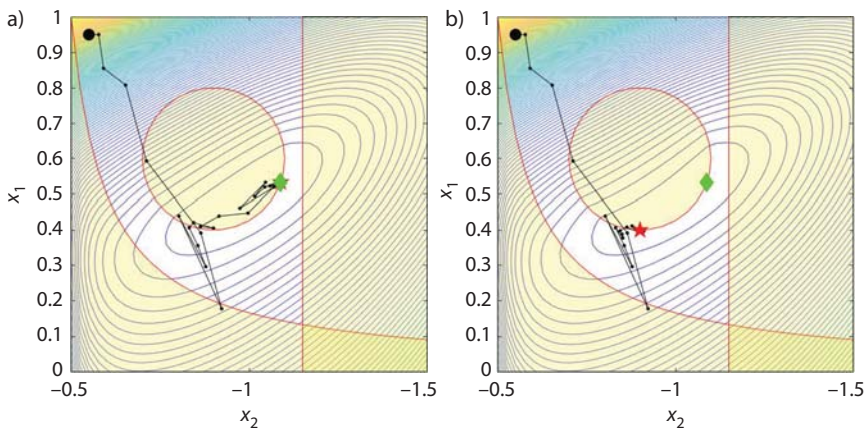


Fig. 10.41 Convergence of the `fminsearch` function with a) `w=1` and b) `w=2`.

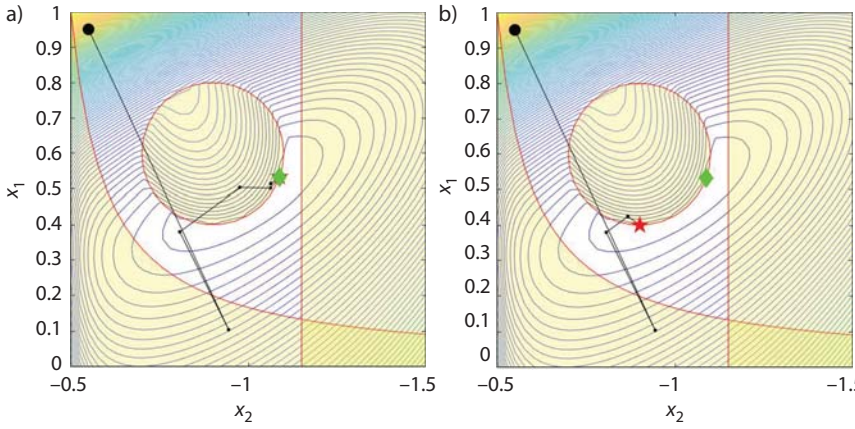


Fig. 10.42 Convergence of the `fminunc` function with a) `w=1` and b) `w=2`.

with `w=1`, the algorithm converges to the correct solution, but when increasing the weighting coefficient to `w=2`, it gets stuck at the local minimum. In terms of performance, it takes 48 iterations (a total of 91 function evaluations) and 43 iterations (a total of 83 function calls) for the `fminsearch` function and 15 iterations (a total of 183 function calls) and 7 iterations (a total of 111 function calls) for the `fminunc` function to arrive at the solution.

For the sake of comparison, the results of using the `fminhj` function are shown in Fig. 10.43. This algorithm converges to the right solution with `w=1` and `w=2` (Fig. 10.43a) but gets stuck at the local minimum in the case of

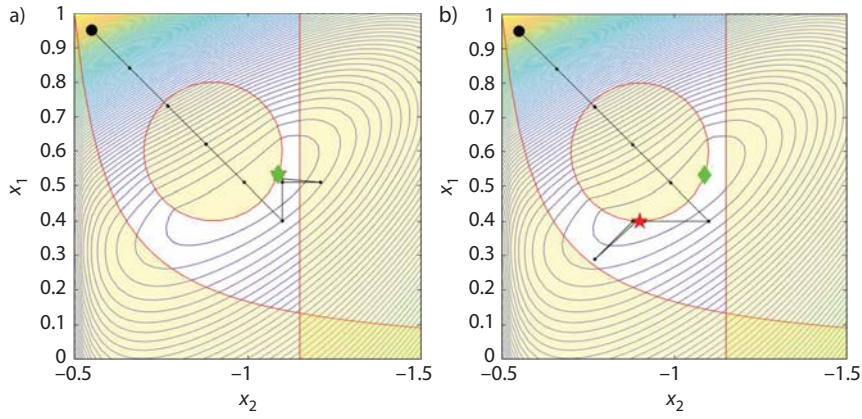


Fig. 10.43 Convergence of the Hooke–Jeeves algorithm with a) `w=2` and b) `w=3`.

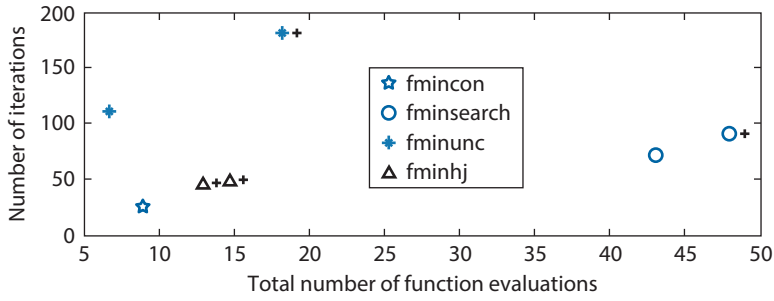


Fig. 10.44 Comparison of solving the constrained optimization problem using different algorithms (a plus sign designates the global as opposed to a local solution).

$w \geq 3$ (Fig. 10.43b). For the cases shown in Fig. 10.43, it takes 15 iterations (a total of 52 function calls) and 13 iterations (a total of 49 function evaluations), respectively.

Figure 10.44 compares all four minimization algorithms discussed earlier. Compared with `fmincon`, it takes many more function evaluations for `fminsearch` and `fminunc` to find the direction of the next iteration (the first one needs to create a simplex and the second one needs to estimate a gradient). However, `fminunc` performs more efficiently in terms of using less iterations. The performance of the Hooke–Jeeves algorithm happens to be somewhere between that of `fmincon` and `fminunc`. Using the same initial conditions for all four algorithms, `fmincon` can only find a local minimum, although by varying the weighting coefficient in the augmented objective function [Eq. (10.78)] used by the other three methods, we were able to find a global minimum (in Fig. 10.44, data corresponding to the global minimum solution are marked with a plus sign).

Finding a global minimum is always a difficult problem, and that is why, in addition to the Optimization Toolbox, MathWorks also offers the Global Optimization Toolbox dedicated to a search of the global minimum. To do that, it enables multiple starting point search and features several additional minimum-finding functions realizing

- Another direct search algorithm (the `patternsearch` function)
- A genetic algorithm, heuristic search algorithm that mimics the process of natural evolution (`ga`)
- Particle swarm algorithm for derivative-free unconstrained optimization or optimization with bounds (`particleswarm`, introduced in R2014b)
- Algorithm for surrogate optimization of time-consuming objective functions with bounds and optional integer constraints (`surrogateopt`, introduced in R2018b)
- Simulated annealing algorithm for derivative-free unconstrained optimization or optimization with bounds (`simulannealbnd`)

The first two functions are applicable for problems with all types of constraints [Eqs. (10.62) – (10.64)].

The Global Optimization Toolbox also offers two solvers for multi-objective optimization: `gamultiobj`—to find a Pareto front (frontier) of multiple objective (aka fitness) functions using a `ga` algorithm, and `paretosearch`—to find nondominated points in the Pareto set (introduced in R2018b).

10.9.3 Visualizing the Optimization Results

It is worth emphasizing one more time that visualizing the results of your computations is a very important step in solving engineering problems and, among other things, provides you with an extra opportunity to verify that you have found the correct solution. Using the graphical tools of MATLAB, this is very easy to do, and that is why many examples of 2-D and 3-D plots are provided throughout this textbook.

To conclude this chapter, we would like to give you one more example of finding an extremum of the function of two variables

$$z(x, y) = -(4x^2 + y^2 - 32x + 4y) \tag{10.79}$$

Consider the following code:

```
% Define two functions
Fxy = @(x,y) -(4*x.^2+y.^2-32*x+4*y);    % to be used with ezmesh
FX = @(x) -Fxy(x(1),x(2));               % to be used with fminsearch
% Visualize the surface Fxy
ezmesh(Fxy,[3 5 -3 -1]), hold
% Find the minimum value using FX
[h,f]=fminsearch(FX,[0,0]);
% Show the peak point
plot3(h(1),h(2),-f,'^r','MarkerFaceColor','r','MarkerSize',12)
% Redefine the axes limits
axis tight
```

It produces the plot shown in Fig. 10.45.

This example uses a few tricks to draw your attention to. First, any 3-D plotting function, as presented in Sec. 6.8, assumes a certain format for an input function, that is, $f(x, y)$. The function handle `Fxy`, defined in Line 2, and to be used by the `ezmesh` function to visualize Eq. (10.79) in Line 5, follows exactly this format, i.e. takes advantage of two clearly defined scalar arguments. However, any optimization function, considered in Sec. 10.9.2, assumes a slightly different format, $f(\mathbf{x})$, where two components of vector $\mathbf{x}$ are $x_1 = x$ and $x_2 = y$. The easiest way to program the original function in this format is shown in Line 3. The function handle `Fx` is then used with `fminsearch` function in Line 7.

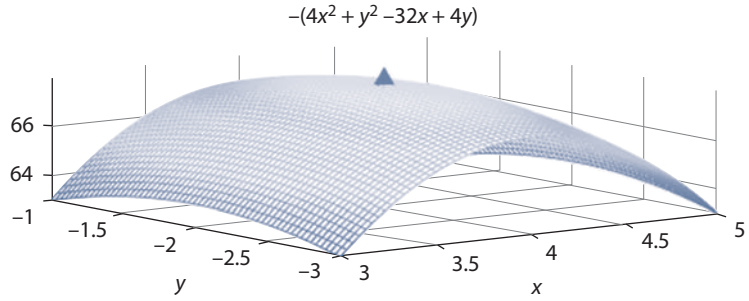


Fig. 10.45 Finding the peak of a surface.

The second trick is that so far we have always tried to find a minimum value because all functions of MATLAB considered in Sec. 10.9.2 can only solve minimization problems. Adding a minus sign in the definition of the `Fx` function in Line 3 obviously inverts the problem, so that the `fminsearch` function is now employed to find a maximum value of the original function. Once this value, `f`, is found for the inverted function, it is visualized using its noninverted value, `-f`, in Line 9.

Finally, we should draw your attention to the very last command, which rescales the vertical axis of the plot. Otherwise, the added marker will not show up.

Problems

10.1 Try to apply all the functions introduced in this chapter (`roots`, `fzero`, `fminbnd`, `fminsearch`, and `fminunc`) to find a value of the cubic root of 200. Use the `tic-toc` pair to record CPU time and present the results of your findings graphically. When appropriate (for minimization functions), determine how many function evaluations were required.

10.2 Determine the first three (closest to $x = 0$) positive roots of the equation

$$4\cos(2x) - e^{0.5x} + 5 = 0$$

Use the `ezplot` or `fplot` function to find out where to look for these roots. Then, use the `lroot` function of Sec. 10.5.1 to bracket the corresponding roots. Finally, use these brackets to initialize the search procedure employing the `fzero` function.

10.3 Using the corresponding scripts of Secs. 10.2.2 and 10.3.3, develop two MATLAB functions with the header lines

```
function x=falseposition(fun,a,b,EPS)
```

and

```
function x = secant(fun,a,b,EPS)
```

to find a solution to the equation $\cos(x) = 2x^3$. Use the interval $[-4;2]$ to apply the first function and points -4 and -3.8 to use the second. For the same accuracy of `EPS=1e-5`, estimate the number of iterations needed to achieve the goal. When developing these two functions, use the `fcnchk` and `feval` functions to enable accepting function handles, function name and inline objects in `fun`.

Chapter 11 Curve Fitting to Measured Data

- Producing Least-Squares Regressions
- Utilizing Piecewise Approximations
- Understanding Fourier Approximation
- Using the MATLAB Functions

`polyfit`, `spline`, `pchip`, `interp1`, `interp2`, `interp3`, `interpn`, `griddata`, `griddatan`, `ndgrid`, `griddedInterpolant`, `scatteredInterpolant`, `fft`, `interpft`, `fillmissing`, `filloutliers`, `detrend`

11.1 Introduction

Fitting experimental data with analytical dependences is one of the most common problems. For instance, you are getting discrete experimental data from the wind tunnel or flight tests and you are required to estimate parameters at the points between these discrete values (interpolation). You may have some complex model you need to reduce to a simpler model, and one way to do this is to compute its outputs at a number of discrete settings within the range of interest and then fit these values to some model of lesser complexity (*model reduction*). Another case is that you know the analytical dependence for your data already, and all you want to do is determine some varied parameters (*system identification*). You may want to use data to predict behavior of your system in the future (extrapolation). In all the aforementioned cases, you may have to deal with the linear or nonlinear functions of one or several variables. Yet another application is that you are dealing with systems that oscillate or vibrate, and you want to analyze the spectral content of the output to find natural frequencies and other harmonics (spectrum analysis).

All these applications are known as curve fitting. There are two general approaches for curve fitting that are distinguished from each other on the basis of the amount of errors associated with data. First, if data exhibit a significant degree of error or "noise," the strategy is to derive a single curve that represents a general trend in data. Because any individual data point may be

incorrect, we make no effort to intersect every point. One approach of this nature is known as a least-squares regression, and MATLAB features two functions that allow you to do this: `polyfit` for polynomial curve fitting (regression) and `fft` for discrete Fourier transform (DFT) or fitting with the trigonometric functions. Second, if data are known to be very precise, the basic approach is to fit a curve or a series of curves that pass directly through each of the points. When an analytical function is established by one way or another, this function can be readily used to obtain estimates of the value without additional (quite often very costly) experiments. The two major functions of MATLAB to address this type of problem are `spline` and `pchip`, which produce piecewise cubic interpolations. In addition to these two functions, MATLAB offers a few more functions for piecewise interpolation of a function of one or more variables, `interp1`, `interp2`, `interp3`, and `interpn`. For the periodic functions, one may choose to use the `interpft` function relying on DFT. For scattered 2-D, 3-D, and *n*-D data, you can use the `griddata` and `gridatan` functions. If you use the same data to perform interpolation at different query points, you might want to use the `scatteredInterpolant` and `griddedInterpolant` functions, creating the data interpolant objects that can be called like a regular function. If you are dealing with the large chunks of data, you might benefit from using the `fillmissing` and `filloutliers` functions for data preconditioning. The `detrend` functions allows to remove a polynomial trend from your data. All the aforementioned functions are considered in this chapter following a detailed explanation of the least-squares and full-degree polynomial regressions, along with piecewise interpolation algorithms behind them. For the sake of completeness, this chapter briefly introduces a special case of parametrically defined splines, Bézier curves, as well. Of course, the Curve Fitting Toolbox of MATLAB offers many more functions for fitting curves and surfaces to data using regression, interpolation, and smoothing, but these functions are beyond the scope of this chapter.

11.2 Least-Squares Regression

Measured data are typically compared with some "theoretical" value derived from an empirical or physical relationship between two quantities. A function may then be assumed, which correlates to this theoretical relationship, and deviations from the function may be determined. *Regression analysis* is the method used to develop a function that minimizes this deviation. In what follows, we will start with a simple linear regression, progressing to the higher-order regression and multiple linear regressions, followed by the analysis of regression errors and attempt to employ least-squares regression formulas to a nonlinear relationship (in the case you do not want to deal with optimization). This will allow you to better understand what is programmed in the MATLAB function `polyfit`.

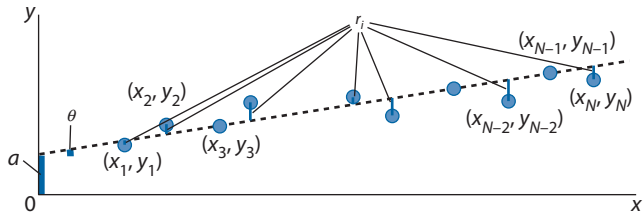


Fig. 11.1 Residuals for the linear regression.

11.2.1 Linear Regression

Suppose we have a set of data (x_i, y_i), $i = 1,2, \ldots, N$ (denoted with circles in Fig. 11.1), to which we postulate a linear relationship (dashed lines):

$$y = f(x) = a + bx \tag{11.1}$$

Two unknown coefficients in Eq. (11.1) are a and $b = \tan(\theta)$, where θ is the slope of regression. We can quantify the deviation of this function from the actual data points as *residuals*

$$r_i = y_i - f(x_i), \quad i = 1, 2, \ldots, N \tag{11.2}$$

representing distances between the data points and suggested regression curve along the y-axis. The sum of squared residuals

$$S_r = \sum_{i=1}^{N} r_i^2 \tag{11.3}$$

for the linear function [Eq. (11.1)] is then given by

$$S_r(a, b) = \sum_{i=1}^{N} (y_i - a - bx_i)^2 \tag{11.4}$$

The goal is to minimize the value of S_r by varying coefficients a and b.

Minimizing S_r implies finding values of a and b such that all partial derivatives become zero:

$$\begin{aligned} \frac{\partial S_r}{\partial a} &= -2\sum_{i=1}^{N} (y_i - a - bx_i) = 0 \\ \frac{\partial S_r}{\partial b} &= -2\sum_{i=1}^{N} x_i(y_i - a - bx_i) = 0 \end{aligned} \tag{11.5}$$

Expanding Eq. (11.5) and dropping the "−2" yields

$$\begin{aligned}\sum_{i=1}^{N} y_i - \sum_{i=1}^{N} a - \sum_{i=1}^{N} bx_i = 0 \\ \sum_{i=1}^{N} x_i y_i - \sum_{i=1}^{N} ax_i - \sum_{i=1}^{N} bx_i^2 = 0\end{aligned} \quad \text{or} \quad \begin{aligned}Na + b\sum_{i=1}^{N} x_i = \sum_{i=1}^{N} y_i \\ a\sum_{i=1}^{N} x_i + b\sum_{i=1}^{N} x_i^2 = \sum_{i=1}^{N} x_i y_i\end{aligned} \tag{11.6}$$

Rewriting Eq. (11.6) in the matrix form gives

$$\begin{bmatrix} N & \sum_{i=1}^{N} x_i \\ \sum_{i=1}^{N} x_i & \sum_{i=1}^{N} x_i^2 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^{N} y_i \\ \sum_{i=1}^{N} x_i y_i \end{bmatrix} \tag{11.7}$$

Solving for a and b reduces to

$$\begin{aligned}\begin{bmatrix} \hat{a} \\ \hat{b} \end{bmatrix} &= \begin{bmatrix} N & \sum_{i=1}^{N} x_i \\ \sum_{i=1}^{N} x_i & \sum_{i=1}^{N} x_i^2 \end{bmatrix}^{-1} \begin{bmatrix} \sum_{i=1}^{N} y_i \\ \sum_{i=1}^{N} x_i y_i \end{bmatrix} \\ &= \frac{1}{N\sum_{i=1}^{N} x_i^2 - \left(\sum_{i=1}^{N} x_i\right)^2} \begin{bmatrix} \sum_{i=1}^{N} x_i^2 & -\sum_{i=1}^{N} x_i \\ -\sum_{i=1}^{N} x_i & N \end{bmatrix} \begin{bmatrix} \sum_{i=1}^{N} y_i \\ \sum_{i=1}^{N} x_i y_i \end{bmatrix}\end{aligned} \tag{11.8}$$

or, in the scalar form,

$$\hat{a} = \frac{\sum_{i=1}^{N} x_i^2 \sum_{i=1}^{N} y_i - \sum_{i=1}^{N} x_i \sum_{i=1}^{N} x_i y_i}{N\sum_{i=1}^{N} x_i^2 - \left(\sum_{i=1}^{N} x_i\right)^2} \quad \text{and} \quad \hat{b} = \frac{N\sum_{i=1}^{N} x_i y_i - \sum_{i=1}^{N} x_i \sum_{i=1}^{N} y_i}{N\sum_{i=1}^{N} x_i^2 - \left(\sum_{i=1}^{N} x_i\right)^2} \tag{11.9}$$

(the hat over symbols is commonly used in system identification to emphasize that they represent the estimated values in the presence of measurement noise). Coefficients $\hat{a}$ and $\hat{b}$ ensure the "best" fit for the given data points, where "best" means the minimum sum of squared residuals [Eq. (11.4)], and that is why Eq. (11.9) is referred to as a *least-squares solution*.

After a functional relation describing data is determined, you can use this function to predict other values for the data points that lie within the x-range

of original data. This process is called *interpolation*. Also, you might need to use this functional relation for *extrapolation*, which is the process of estimating a value of $f(x)$ for x residing outside the x-range of the known data points. Because regression analysis minimizes the sum of squared residuals (errors), you can be confident that your prediction is reasonably accurate as long as your original assumptions of the functional relation are valid.

11.2.2 Regression Analysis with Higher-Order Polynomials

If you do not know for sure that there is a linear relationship between the points in the dataset (x_i,y_i), $i = 1,2, \ldots, N$, you may want to try fitting these data with a second- (Fig. 11.2) or even higher-order polynomial. In general, for a Pth-order polynomial ($P < N$), we may write

$$y = f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_Px^P = \sum_{j=0}^{P} a_jx^j \tag{11.10}$$

where a_j, $j = 0,1, \ldots, P$ are the constants that need to be determined.

Following the least-squares regression approach of the previous section, we start by composing the sum of squared residuals [Eq. (11.3)] for Eq. (11.10):

$$S_r(a_0, a_1, \ldots, a_P) = \sum_{i=1}^{N} \left(y_i - \sum_{j=0}^{P} a_jx_i^j \right)^2 \tag{11.11}$$

Next, in order to minimize S_r, we need to ensure that all partial derivatives of S_r with respect to undetermined coefficients a_j, $j = 0, 1, \ldots, P$ are zero, that is, we have to resolve a system of linear (with respect to these coefficients) algebraic equations

$$\frac{\partial S_r}{\partial a_j} = -2 \sum_{i=1}^{N} x_i^j \left(y_i - \sum_{j=0}^{P} a_jx_i^j \right) = 0 \tag{11.12}$$

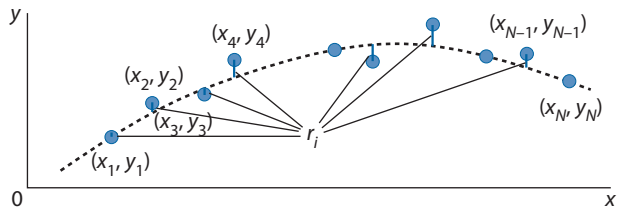


Fig. 11.2 Residuals for the second-order polynomial regression.

Being written in the matrix form, this set of equations becomes

$$\begin{bmatrix} N & \sum_{i=1}^{N} x_i & \sum_{i=1}^{N} x_i^2 & \cdots & \sum_{i=1}^{N} x_i^P \\ \sum_{i=1}^{N} x_i & \sum_{i=1}^{N} x_i^2 & \sum_{i=1}^{N} x_i^3 & \cdots & \sum_{i=1}^{N} x_i^{P+1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^{N} x_i^P & \sum_{i=1}^{N} x_i^{P+1} & \sum_{i=1}^{N} x_i^{P+2} & \cdots & \sum_{i=1}^{N} x_i^{2P} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_P \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^{N} y_i \\ \sum_{i=1}^{N} x_i y_i \\ \vdots \\ \sum_{i=1}^{N} x_i^P y_i \end{bmatrix} \tag{11.13}$$

This system of LAEs can be solved using any method described in Sec. 9.3.

11.2.3 Multiple Linear Regression

The results of Sec. 11.2.1 can be easily extended to the case where we want to develop a multiple linear regression for the data set that describes a function depending on not just one, but multiple independent variables. For a function of L independent variables, such linear regression takes the form

$$y = a_0 + \sum_{j=1}^{L} a_j x_j \tag{11.14}$$

(for $L = 1$ this equation represents a line [Eq. (11.1)], for $L = 2$ – a plane).

Employing the standard least-squares routine for N data points ($L < N$), we form the sum of squared residuals

$$S_r(a_0, a_1, ..., a_L) = \sum_{i=1}^{N} r_i^2 = \sum_{i=1}^{N} \left(y_i - a_0 - \sum_{j=1}^{L} a_j x_{ji} \right)^2 \tag{11.15}$$

followed by equating all its partial derivatives to zero:

$$\begin{aligned} \frac{\partial S_r}{\partial a_0} &= -2 \sum_{i=1}^{N} \left(y_i - a_0 - \sum_{j=1}^{L} a_j x_{ji} \right) = 0, \\ \frac{\partial S_r}{\partial a_j} &= -2 \sum_{i=1}^{N} x_{ji} \left(y_i - a_0 - \sum_{j=1}^{L} a_j x_{ji} \right) = 0, \quad j = 1, 2, ..., L \end{aligned} \tag{11.16}$$

This results in the linear algebraic system

$$\begin{bmatrix} N & \sum_{i=1}^{N} x_{1i} & \sum_{i=1}^{N} x_{2i} & \cdots & \sum_{i=1}^{N} x_{Li} \\ \sum_{i=1}^{N} x_{1i} & \sum_{i=1}^{N} x_{1i}^2 & \sum_{i=1}^{N} x_{1i}x_{2i} & \cdots & \sum_{i=1}^{N} x_{1i}x_{Li} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^{N} x_{Li} & \sum_{i=1}^{N} x_{Li}x_{1i} & \sum_{i=1}^{N} x_{Li}x_{2i} & \cdots & \sum_{i=1}^{N} x_{Li}^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_L \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^{N} y_i \\ \sum_{i=1}^{N} x_{1i}y_i \\ \vdots \\ \sum_{i=1}^{N} x_{Li}y_i \end{bmatrix} \tag{11.17}$$

to solve for the vector of unknown coefficients a_j, $j = 0,1, \ldots, L$.

11.2.4 Relation of Linear Regression Error to That of Statistics

By construction, regressions obtained in Secs. 11.2.1 and 11.2.2 ensure the smallest value of the sum of squared residuals S_r [computed according to Eqs. (11.4), (11.11), and (11.15)] among all other curves of the same class (same-order polynomials). If the distribution of individual residuals happens to be normal, and the spread of the points around the fit curve is of a similar magnitude along the entire range of data, then these regressions are the best not only in the sense of delivering a minimum of S_r, but also in the sense of the *maximum likelihood* principle. The latter means that the most likely value of the error variance is the variance of residuals. In such a case, a "standard deviation," or *standard error of the estimate* for the regression curve, can be determined as

$$s_{y/x} = \sqrt{\frac{S_r}{N - (P+1)}} \tag{11.18}$$

where N is the number of data points and P the regression polynomial's order (for multiple linear regression $s_{y/x} = \sqrt{S_r/(N - (L+1))}$). The subscript notation "y/x" designates that the error is estimated for a predicted value of y corresponding to a particular value of x.

Note that the denominator of Eq. (11.8) has the $N - (P+1)$ term as opposed to just N for the *standard deviation of the sample* or $N - 1$ for the *sample standard deviation*. The reason is that $P+1$ data-derived estimates (polynomial coefficients) were used to compute S_r, reducing the number of degrees of freedom by exactly this number, $P+1$. Another reasoning is that if the number of the data points N equals $P+1$, then the Pth-order polynomial passes through all the points with no error (see Sec. 11.3). According to Eq. (11.8), this case corresponds to a distinguishable zero-by-zero division (resulting in `NaN` in MATLAB).

Just as is the case with the sample standard deviation, the standard error of the estimate [Eq. (11.18)] quantifies the spread of data. The difference, though, is that $S_{y/x}$ quantifies the spread around the regression curve, in contrast to the standard deviation S_y that quantifies the spread around the mean value

$$\overline{y} = \frac{1}{N}\sum_{i=1}^{N} y_i \tag{11.19}$$

so that

$$s_y = \sqrt{\frac{S_t}{N-1}}, \quad S_t = \sum_{i=1}^{N} (y_i - \overline{y})^2 \tag{11.20}$$

As a result, the aforementioned concepts can be used to quantify the "goodness" of the fit. This is particularly useful for comparison of several regressions. The difference between the two quantities, $S_t - S_r$, quantifies the error reduction due to characterization of data in terms of a regression curve rather than an average value. Because the magnitude of this quantity is scale dependent, the difference is normalized to S_t to yield

$$R^2 = \frac{S_t - S_r}{S_t} \tag{11.21}$$

where R^2 is called the *coefficient of determination* and R, the *correlation coefficient*. For a perfect fit, $S_r = 0$ and $R = R^2 = 1$, signifying that the regression curve explains 100% of data variability. If $S_r = S_t$, then $R = R^2 = 0$, meaning that the regression is not any better than a simple approximation of data with

$$y = f(x) = \overline{y} \tag{11.22}$$

where the mean value $\overline{y}$ is determined by Eq. (11.19).

11.2.5 Application of Linear Regression to a Nonlinear Relationship

So far, we have only dealt with linear regressions, that is, regressions where unknown parameters entered the approximating equation linearly, as in Eq. (11.14). Generally speaking, if we know that data should be represented by some nonlinear dependence

$$y = f(x, \boldsymbol{\theta}) \tag{11.23}$$

where $\boldsymbol{\theta}$ represents the vector of unknown parameters, then the least-squares regression should be cast as a minimization problem:

$$\boldsymbol{\theta} = \arg\left(\min_{\boldsymbol{\theta}} S_r(\boldsymbol{\theta})\right) = \arg\left(\min_{\boldsymbol{\theta}} \sum_{i=1}^{N} [y_i - f(x_i, \boldsymbol{\theta})]^2\right) \tag{11.24}$$

The reason we cannot directly use the same approach we used before is that taking partial derivatives in this case results in nonlinear equations. As is known from Chapter 10, resolving Eq. (11.24) may not be so straightforward, especially in the case of highly nonlinear functions $f(x, \mathbf{\theta})$ and a large number of coefficients (the solution may diverge). Besides, there may be multiple minima of S_r, and none of minimization methods considered in Chapter 10 guarantees convergence to the global minimum.

Luckily, some nonlinear regression problems can be moved to a linear domain by a suitable transformation of the model formulation and, therefore, still be addressed using methods of linear algebra considered in the previous sections of this chapter. For example, consider a *power law* relationship

$$y(x) = ax^b \tag{11.25}$$

with unknown coefficients a and b. By taking the base-10 logarithm of both sides, this equation becomes

$$\log(y) = \log(a) + b\log(x) \tag{11.26}$$

which produces a linear dependence of the form of Eq. (11.1) in the log–log axes with a slope of $\tan^{-1}(b)$ and a y-intercept of $\log(a)$. Hence, we reduced the original problem [Eq. (11.25)] to

$$y^* = a^* + bx^* \tag{11.27}$$

where $y^* = \log(y)$, $a^* = \log(a)$, and $x^* = \log(x)$. Once resolved, meaning that coefficients $\hat{a}^*$ and $\hat{b}$ of Eq. (11.27) are found via Eq. (11.9), the parameter a of Eq. (11.25) can be computed as

$$\hat{a} = 10^{\hat{a}^*} \tag{11.28}$$

Strictly speaking, by using the preceding approach, instead of Eq. (11.3), we minimized the following function:

$$\begin{aligned} S_r^*(a,b) &= \sum_{i=1}^{N} [\log(y_i) - \log(y^*(x_i))]^2 \\ &= \sum_{i=1}^{N} \left(\log\left(\frac{y_i^* + r_i}{y_i^*} \right) \right)^2 = \sum_{i=1}^{N} \left(\log\left(1 + \frac{r_i}{y_i^*} \right) \right)^2 \sim \sum_{i=1}^{N} \left(\frac{r_i}{y_i^*} \right)^2 \end{aligned} \tag{11.29}$$

This function represents the sum of weighted squared residuals, paying more attention to the data points with the smaller values of y_i^* [compare Eq. (11.29) with Eq. (11.3)]. As will be shown in an example in Sec. 11.6.1, the results of minimizing Eq. (11.29) as opposed to Eq. (11.3) do not look too bad and quite often can be used as is. However, if needed, you may use the estimates $\hat{a}$ and $\hat{b}$

obtained via linear least-squares regression in the modified domain [minimization of Eq. (11.29)] as a good initial guess for addressing the minimization problem of Eq. (11.24). This applies to all other examples as follows.

An *exponential law* relationship

$$y(x) = ac^{bx} \tag{11.30}$$

can be handled similarly to that of a power law. Specifically, if we apply the base-c logarithm to both sides of Eq. (11.30), we obtain

$$\log_c(y) = \log_c(a) + bx \tag{11.31}$$

or

$$y^* = a^* + bx \tag{11.32}$$

where $y^* = \log_c(y)$ and $a^* = \log_c(a)$. Upon solving it for $\hat{a}^*$ and $\hat{b}$, we get back to $\hat{a}$ using

$$\hat{a} = c^{\hat{a}^*} \tag{11.33}$$

The only problem with the realization of Eqs. (11.31) and (11.32) in MATLAB is that it does not have an arbitrary-base logarithm function. However, using a well-known formula

$$\log_a(b) = \frac{\log_c(b)}{\log_c(a)} \tag{11.34}$$

we can always bring it down to one of the three available logarithm functions, base-2 (`log2`), base-10 (`log10`), or natural logarithm (`log`) (that is, $c = \{2, 10, e\}$).

Another example is a multiple linear regression of the general form

$$y = a_0 \prod_{j=1}^{L} x_j^{a_j} \tag{11.35}$$

Taking the logarithm of both sides yields

$$\log(y) = a_0 + \sum_{j=1}^{L} a_j \log(x_j) \tag{11.36}$$

which now has the form of Eq. (11.14) and can be handled appropriately with $y^* = \log(y)$ and $x_j^* = \log(x_j)$.

Next, consider the not-so-obvious example of nonlinear equation (describing the Michaelis–Menten model for enzyme kinetics)

$$y = \frac{ax}{b + x} \tag{11.37}$$

In this case, you can bring it to the linear form by simply inverting both sides of equation, which yields

$$\frac{1}{y} = \frac{1}{a} + \frac{b}{a}\frac{1}{x} \tag{11.38}$$

You can further rewrite it in the form of Eq. (11.1) as

$$y^* = a^* + b^* x^* \tag{11.39}$$

with $y^* = y^{-1}$, $a^* = a^{-1}$, $b^* = ba^{-1}$, and $x^* = x^{-1}$. Once solved for $\hat{a}^*$ and $\hat{b}^*$, we get back to $\hat{a}$ and $\hat{b}$ using

$$\hat{a} = \hat{a}^{*-1} \quad \text{and} \quad \hat{b} = \hat{b}^* \hat{a}^{*-1} \tag{11.40}$$

In the latter case, the actual function being minimized through the linear least-squares regression in the modified domain is

$$S_r^*(a,b) = \sum_{i=1}^{N} \left(\frac{1}{y_i} - \frac{1}{y_i^*}\right)^2 = \sum_{i=1}^{N} \left(\frac{r_i}{(y_i^* + r_i)y_i^*}\right)^2 \approx \sum_{i=1}^{N} \left(\frac{r_i}{y_i^{*2}}\right)^2 \tag{11.41}$$

Similar to Eq. (11.29), the regression produced in this case pays even more attention to the data points with the smaller values of y_i^*.

Finally, let us slightly change Eq. (11.37) to

$$y = \frac{a\sqrt[3]{x^2}}{b + x} \tag{11.42}$$

This equation resembles the Sutherland formula for dynamic viscosity of gases vs absolute temperature. Suppose you performed some measurements and want to find coefficients $\hat{a}$ and $\hat{b}$, so that your data best fit this formula. By inverting it and multiplying by $\sqrt[3]{x^2}$, you may bring it to the form

$$\frac{\sqrt[3]{x^2}}{y} = \frac{b}{a} + \frac{1}{a}x \tag{11.43}$$

Using substitutions $y^* = \sqrt[3]{x^2}/y$, $a^* = ba^{-1}$ and $b^* = a^{-1}$, you can further reduce it to the linear form

$$y^* = a^* + b^* x \tag{11.44}$$

Once solved for $\hat{a}^*$ and $\hat{b}^*$, the original coefficients $\hat{a}$ and $\hat{b}$ can be calculated as

$$\hat{a} = \hat{b}^{*-1} \quad \text{and} \quad \hat{b} = \hat{a}^* \hat{b}^{*-1} \tag{11.45}$$

Compared with other cases considered in this section, the new variable y^* now contains the independent variable x, which results in the least-squares

regression applied to Eq. (11.44), minimizing

$$S_r^*(a,b)=\sum_{i=1}^{N}\left(\frac{\sqrt[3]{x_i^2}}{y_i}-\frac{\sqrt[3]{x_i^2}}{y_i^*}\right)^2=\sum_{i=1}^{N}\left(\sqrt[3]{x_i^2}\frac{r_i}{(y_i^*+r_i)y_i^*}\right)^2\approx\sum_{i=1}^{N}\left(\frac{\sqrt[3]{x_i^2}}{y_i^{*2}}r_i\right)^2 \tag{11.46}$$

with an even more complex scaling coefficient than those in Eqs. (11.29) and (11.41). As mentioned earlier, Sec. 11.6.1 will present a couple of nonlinear function approximations produced using the least-squares linear regression formula and function minimization.

For completeness, let us finish this section with presenting the *Gauss–Newton iterative method*, which can be employed to find an exact solution to Eq. (11.24) with the good guess obtained as explained in the aforementioned examples.

Suppose, we have a good initial guess for the values of unknown parameters $\boldsymbol{\theta}_0$ and we want to find a shift vector $\Delta\boldsymbol{\theta}$, which would bring it to the exact solution of Eq. (11.24) $\boldsymbol{\theta}^*$

$$\boldsymbol{\theta}_*=\boldsymbol{\theta}_0+\Delta\boldsymbol{\theta} \tag{11.47}$$

We can use the Taylor series expansion, similar to that of Eq. (10.30), to linearize the function [Eq. (11.23)] about the approximate solution as

$$f(x,\boldsymbol{\theta})\approx f(x,\boldsymbol{\theta}_0)+\nabla f_{\boldsymbol{\theta}}(x,\boldsymbol{\theta}_0)^T\Delta\boldsymbol{\theta} \tag{11.48}$$

where $\nabla f_\theta\,(x,\boldsymbol{\theta}_0)$ denotes the gradient of $f\,(x,\boldsymbol{\theta})$ [see Eq. (10.54)]. Hence, the sum of squared residuals can be represented as

$$\begin{aligned}S_r(\boldsymbol{\theta})&=\sum_{i=1}^{N}r_i^2=\sum_{i=1}^{N}[y_i-f(x_i,\boldsymbol{\theta})]^2\\&=\sum_{i=1}^{N}\left[y_i-f(x_i,\boldsymbol{\theta}_0)-\nabla f_{\boldsymbol{\theta}}(x_i,\boldsymbol{\theta}_0)^T\Delta\boldsymbol{\theta}\right]^2\end{aligned} \tag{11.49}$$

Introducing two notations

$$\Delta\mathbf{y}=\begin{bmatrix}y_1-f(x_1,\boldsymbol{\theta}_0)\\y_2-f(x_2,\boldsymbol{\theta}_0)\\\vdots\\y_N-f(x_N,\boldsymbol{\theta}_0)\end{bmatrix}\quad\text{and}\quad\mathbf{J}(\mathbf{x},\boldsymbol{\theta}_0)=\begin{bmatrix}\nabla f_{\boldsymbol{\theta}}(x_1,\boldsymbol{\theta}_0)^T\\\nabla f_{\boldsymbol{\theta}}(x_2,\boldsymbol{\theta}_0)^T\\\vdots\\\nabla f_{\boldsymbol{\theta}}(x_N,\boldsymbol{\theta}_0)^T\end{bmatrix} \tag{11.50}$$

Eq. (11.49) can be rewritten in the matrix form as

$$S_r(\boldsymbol{\theta})=(\Delta\mathbf{y}-\mathbf{J}(\mathbf{x},\boldsymbol{\theta}_0)\Delta\boldsymbol{\theta})^T(\Delta\mathbf{y}-\mathbf{J}(\mathbf{x},\boldsymbol{\theta}_0)\Delta\boldsymbol{\theta}) \tag{11.51}$$

The goal is to find $\Delta\boldsymbol{\theta}$ that minimizes $S_r\,(\boldsymbol{\theta})$. In fact, we have dealt with this type of problem already [Eq. (9.81)], so that we can write a solution in the

form of a set of LAEs right away [compare with Eq. (9.85)]:

$$(\mathbf{J}(\mathbf{x}, \boldsymbol{\theta}_0)^T \mathbf{J}(\mathbf{x}, \boldsymbol{\theta}_0))\Delta\boldsymbol{\theta} = \mathbf{J}(\mathbf{x}, \boldsymbol{\theta}_0)^T \Delta\mathbf{y} \tag{11.52}$$

This equation can be applied iteratively to find a solution to Eq. (11.24) to meet a desired tolerance.

11.3 Full-Degree Polynomial Approximation

Let us a consider a different type of a problem when we do not care about producing a known function fitting to noisy data, but rather have a requirement to develop an approximation that passes through each of the data points precisely. Obviously, in this case, the least-squares regression method cannot be applied (S_r equals zero by construction).

Having $N+1$ data points (x_i, y_i), $i = 1,2, \ldots, N+1$ (no two x_i are the same), the most straightforward approach we can use is to apply the *full-degree polynomial interpolation* determined by the Nth-order polynomial

$$f_N(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_N x^N = \sum_{i=0}^{N} a_i x^i \tag{11.53}$$

This form of the polynomial representation is referred to as the *power* or *monomial-basis form* (for illustration, Fig. 11.3 depicts a few monomials τ^i, $i = 0, 1, \ldots, 10$ within the range $\tau \in [-1; 1]$). Coefficients of Eq. (11.53) ensuring it passes through all the data points can be found by solving the following system of LAEs:

$$\begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^N \\ 1 & x_2 & x_2^2 & \cdots & x_2^N \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N+1} & x_{N+1}^2 & \cdots & x_{N+1}^N \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_N \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{N+1} \end{bmatrix} \tag{11.54}$$

The matrix on the left-hand side of Eq. (11.54) is called the *Vandermonde* matrix.

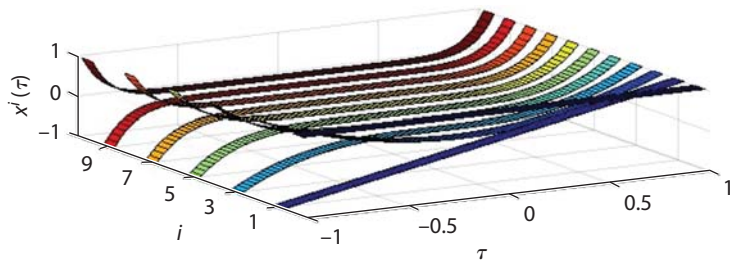


Fig. 11.3 Monomials τ^i, $i = 0, 1, \ldots, 10$, $\tau \in [-1; 1]$.

For large N, the Vandermonde matrix may be badly conditioned (for $\mathbf{Ax} = \mathbf{b}$, the condition number bounds the ratio of the relative error in $\mathbf{x}$ to the relative error in $\mathbf{b}$). Hence, application of the power form of Eq. (11.53) is usually limited to problems involving a few well-spaced and well-scaled data points. Fortunately, there are a variety of other mathematical formats besides that of Eq. (11.53) in which a polynomial can be expressed (constructed). Indeed, because of uniqueness of polynomial that passes through all the data points, all these forms eventually produce exactly the same polynomial.

The most compact of all the forms is the form known as the *Lagrange interpolating polynomial*

$$f_N(x) = \sum_{i=1}^{N+1} L_i(x) y_i \tag{11.55}$$

In this form, the *Lagrange basis polynomials*

$$L_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{N+1} \frac{x - x_j}{x_i - x_j} \tag{11.56}$$

assume the values of either 1 ($L_i(x_i) = 1$) or 0 ($L_i(x_j) = 0, j \neq i$). For example, the Lagrange form of the linear equation ($N = 1$) is

$$f_1(x) = \frac{x - x_2}{x_1 - x_2} y_1 + \frac{x - x_1}{x_2 - x_1} y_2 \tag{11.57}$$

and the second-order polynomial ($N = 2$) can be represented as

$$f_2(x) = \frac{x - x_2}{x_1 - x_2} \frac{x - x_3}{x_1 - x_3} y_1 + \frac{x - x_1}{x_2 - x_1} \frac{x - x_3}{x_2 - x_3} y_2 + \frac{x - x_1}{x_3 - x_1} \frac{x - x_2}{x_3 - x_2} y_3 \tag{11.58}$$

For computational purposes, Eq. (11.55) can be modified as

$$f_N(x) = \prod_{j=1}^{N+1} (x - x_j) \sum_{i=1}^{N+1} \frac{w_i}{x - x_i} y_i \tag{11.59}$$

where the weights w_i can be computed a priori as

$$w_i = \prod_{\substack{j=1 \\ j \neq i}}^{N+1} \frac{1}{x_i - x_j} \tag{11.60}$$

This form is known as the first form of the *barycentric interpolation formula*.

Equations (11.57) and (11.58) can also be expressed as

$$f_1(x) = y_1 + \frac{y_2 - y_1}{x_2 - x_1} (x - x_1) \tag{11.61}$$

and

$$f_2(x) = y_1 + \frac{y_2 - y_1}{x_2 - x_1}(x - x_1) + \left[\frac{(y_3 - y_2/x_3 - x_2) - (y_2 - y_1/x_2 - x_1)}{x_3 - x_1}\right](x - x_1)(x - x_2) \tag{11.62}$$

which constitutes another polynomial form referred to as the *Newton's interpolating polynomial.* Although the second terms in both formulas represent the slope of the line connecting the points x_1 and x_2, the last term in Eq. (11.62) introduces the curvature, proportional to the second-order derivative. In general,

$$f_N(x) = \sum_{i=1}^{N+1} a_i N_i(x) \tag{11.63}$$

with the *Newton's basis polynomials*

$$N_i(x) = \prod_{j=1}^{i-1} (x - x_j) \tag{11.64}$$

and coefficients a_i defined as fractional expressions like those in Eq. (11.62) representing the *finite divided difference* approximations of the corresponding derivatives (Chapter 12). In this sense, (if all x_j coincide), the Newton's interpolating polynomial is the straightforward divided-difference-based version of the Taylor polynomial [compare with Eqs. (7.2) and (8.32) that use instantaneous derivatives].

In practice, Eq. (11.63) can be solved for coefficients a_i using the following set of LAEs:

$$\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 1 & x_2 - x_1 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N+1} - x_1 & (x_{N+1} - x_1)(x_{N+1} - x_2) & \cdots & \prod_{j=1}^{N} (x_{N+1} - x_j) \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_{N+1} \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{N+1} \end{bmatrix} \tag{11.65}$$

Different forms of an interpolation polynomial feature different computational advantages and disadvantages. For example, compared with the Vandermonde matrix of Eq. (11.54), Eq. (11.65) employs a much simpler lower triangular matrix, so that this set of LAEs can be solved faster. Another

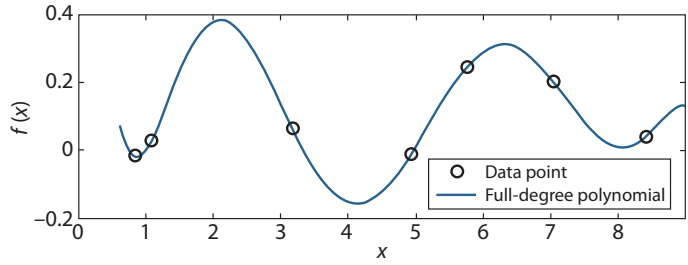


Fig. 11.4 Full-degree polynomial wiggle and extrapolation errors.

advantage is that adding more data points (to the right) does not involve recomputation of the entire matrix. Some regard the Lagrange interpolating polynomial (specifically, its barycentric form) as the general formula of choice. However, using a full-degree polynomial approximation shares the same major pitfalls among all approaches.

First, increasing the number of data points leads to increasing an oscillatory behavior of approximation (Fig. 11.4). This phenomenon is called *polynomial wiggle*. This is due to the fact that in general the Nth-order polynomial has $N-1$ extremums. Such excessive variations in $f(x)$ between the data points (overshooting its values at the neighboring data points) are especially typical for the first and last subintervals. The situation aggravates when the data points are not equally spaced along the x-axis. As a result, a full-degree polynomial approximation can definitely not be used for extrapolation, when $f(x)$ can easily diverge from the prediction with very large errors.

Second, the higher-order polynomials tend to be ill conditioned, that is, they tend to be highly sensitive to round-off error (which is also true for the higher-order polynomial regression). This results in coefficients being highly inaccurate, so that at some point a round-off error interferes with the ability to interpolate.

Finally, in general, the change of the value of a single point (either x_i or y_i) or adding a few extra points to the data set leads to the necessity of recomputing all coefficients.

As a result of these disadvantages, the full-degree polynomial interpolation is hardly ever used for data and curve fitting. Its primary application is in the derivation of other numerical methods (as will be shown in Chapters 12 and 13).

The MATLAB `polyfit` function exploiting a monomial form of the interpolating polynomial of the order `n` is concise and straightforward. It first forms the Vandermonde matrix **V**

```
V(:,1) = ones(length(x),1);
for j = 2:n+1
   V(:,j) = x.*V(:,j-1);
end
```

and then uses the backslash operator, \, to solve Eq. (11.54). If necessary, you may modify this function to use other forms of the full-degree polynomial interpolation, but the beauty of this approach is that it also allows you to solve the least-squares problems using the very same *m*-file function.

Suppose, you have $N > 2$ data points but want to use a linear interpolating polynomial. In this case, Eq. (11.7) becomes

$$\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \tag{11.66}$$

Since the Vandermonde matrix **V** is not square, the MATLAB backslash operator premultiplies both sides of this equation by the transpose of **V** [Eq. (9.85)], which yields

$$\begin{bmatrix} 1 & 1 & \cdots & 1 \\ x_1 & x_2 & \cdots & x_2 \end{bmatrix} \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ x_1 & x_2 & \cdots & x_2 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \tag{11.67}$$

That transforms Eq. (11.66) to the form of Eq. (11.7)! This procedure (premultiplication by the transpose matrix) works for the higher-order polynomials as well, converting overdetermined sets of LAEs to the form of Eq. (11.13).

11.4 Piecewise Approximation

As seen from the preceding discussion, despite the fact that the full-degree polynomial captures all meanderings suggested by the data points, quite often such high-order functions lead to erroneous results because of round-off errors and overshoot between the data points. An alternative approach is to apply lower-order polynomials to subsets of the data points and then "seamlessly" join these approximations together. Such connecting polynomials are called the *spline functions*.

11.4.1 Linear and Quadratic Splines

To introduce some basic concepts and problems associated with the spline interpolation, first consider a piecewise linear interpolation in which a linear function is assumed to exist between each pair of adjacent data points, as shown in Fig. 11.5.

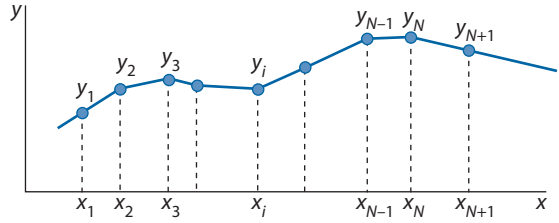


Fig. 11.5 Piecewise linear interpolation.

The *linear* (first-order) *splines* for a group of the ordered data points (x_i, y_i), $i = 1,2, \ldots, N+1$ can be defined as a set of the linear functions

$$f(x_i) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i), \quad x_i \le x < x_{i+1}, \quad i = 1, 2, \ldots, N \quad (11.68)$$

This method of interpolation is fairly accurate when the data points are fairly closely spaced, and any nonlinearities in the function between the data points can be approximated by a linear function. However, even then it has an obvious disadvantage—the first-order piecewise approximation is not smooth. At the point where two lines meet (referred to as a *node*, *knot*, *breakpoint*, or *break*), the slope changes abruptly, that is, the first-order derivative is discontinuous. Let us try to overcome this deficiency by using a higher-order polynomial spline.

For each of N subintervals, the *quadratic* (second-order) *spline* uses a second-order polynomial

$$f_i(x) = a_i(x - x_i)^2 + b_i(x - x_i) + y_i, \quad x_i \le x \le x_{i+1}, \quad i = 1, 2, ..., N \quad (11.69)$$

featuring two unknown coefficients (by construction, an approximation for each subinterval passes through the left knot). Consequently for N subintervals, we have $2N$ unknown constants (a_i, b_i, $i = 1,2, \ldots, N$). To find their values, the $2N$ conditions should be imposed. They are as follows:

1. For each of N subintervals, the approximation must pass through the right knot, which will ensure the continuity of a function.
2. At $N - 1$ interior nodes, the first-order derivative of two adjacent polynomials must be equal to each other, which ensures a continuity of the first-order derivative.

These conditions result in the equations shown in Table 11.1 (using the notation $h_i = x_{i+1} - x_i$ for the length of the ith subinterval). This table also shows the budget of the number of equations.

Table 11.1 Stitching Conditions for Quadratic Spline

Condition No.	Equation	No. of Equations
1	$a_i\, h_i^2 + b_i\, h_i + y_i = y_{i+1}$ for $i = 1, 2, \ldots, N$	N
2	$2a_{i-1}\, h_{i-1} + b_{i-1} = b_i$ for $i = 2, 3, \ldots, N$	$N - 1$

As seen, the sum of the number of equations in Table 11.1 equals $2N - 1$, meaning that we have the freedom of enforcing one more condition. For example, we may require that the second-order derivative at either endpoint is zero, resulting in either $a_1 = 0$ or $a_{N+1} = 0$. A visual interpretation of such a condition is that the first (or the last) two points will be connected by a straight line.

Having $2N$ conditions allows you to find $2N$ coefficients a_i, b_i, $i = 1, 2, \ldots, N$ of Eq. (11.69) from the system of LAEs

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 \\ h_1^2 & h_1 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 \\ 2h_1 & 1 & 0 & -1 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 \\ 0 & 0 & h_2^2 & h_2 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 \\ 0 & 0 & 2h_2 & 1 & 0 & -1 & \cdots & 0 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & h_{N-1}^2 & h_{N-1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 2h_{N-1} & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & h_N^2 & h_N \end{bmatrix} \begin{bmatrix} a_1 \\ b_1 \\ a_2 \\ b_2 \\ a_3 \\ b_3 \\ \vdots \\ a_N \\ b_N \end{bmatrix}$$

$$= \begin{bmatrix} 0 \\ y_2 - y_1 \\ 0 \\ y_3 - y_2 \\ 0 \\ \vdots \\ y_N - y_{N-1} \\ 0 \\ y_{N+1} - y_N \end{bmatrix} \tag{11.70}$$

($a_1 = 0$ was assumed in the first equation). Compared with the matrix of Eq. (11.54) or even Eq. (11.65), the sparse matrix of Eq. (11.70) is much easier to handle. In fact, you may start with the very first equation, solving

it for a_1, then find b_1 from the second equation, a_2 from the third equation, b_2 from the fourth equation, and so on.

The two shortcomings of the quadratic splines fit are

- Asymmetry (we have only one extra condition to be imposed on either end)
- Discontinuity of the second-order derivative at the interior nodes, which may lead to undesirable high swings between the points

On the positive side, analyzing the linear and quadratic splines leads to the conclusion that, to ensure the continuity of the kth-order derivative, a spline of at least the $(k+1)$th-order must be used. Hence, we should proceed with a cubic spline.

11.4.2 Cubic Splines

The *cubic spline* relies on a third-order polynomial approximation between each pair of knots ($i = 1, 2, \ldots, N$)

$$f_i(x) = a_i(x - x_i)^3 + b_i(x - x_i)^2 + c_i(x - x_i) + y_i, \quad x_i \le x \le x_{i+1} \tag{11.71}$$

For the same $N+1$ data points (x_i, y_i) $i = 1, 2, \ldots, N+1$, and N subintervals between them, we now have $3N$ unknown coefficients to evaluate. Hence, in addition to those two types of conditions shown for the quadratic spline, we may now add one more:

1. For each of N subintervals the approximation must pass through the right knot, which will ensure the continuity of a function.
2. At $N-1$ interior nodes, the first-order derivative of two adjacent polynomials must be equal to each other, which ensures a continuity of the first-order derivative.
3. At $N-1$ interior nodes, the second-order derivative of two adjacent polynomials must be equal to each other, which ensures a continuity of the second-order derivative.

Similar to Table 11.1, Table 11.2 shows the corresponding equations and counts their number in the rightmost column.

Table 11.2 Stitching Conditions for Cubic Spline

Condition No.	Equation	No. of Equations
1	$a_i h_i^3 + b_i h_i^2 + c_i h_i + y_i = y_{i+1}$ for $i = 1, 2, \ldots, N$	N
2	$3a_{i-1} h_{i-1}^2 + 2b_{i-1} h_{i-1} + c_{i+1} = c_i$ for $i = 2, 3, \ldots, N$	$N-1$
3	$6a_{i-1} h_{i-1} + 2b_{i-1} = 2b_i$ for $i = 2, 3, \ldots, N$	$N-1$

Table 11.3 Endpoint Conditions for Cubic Spline

	Condition	Name of Spline
A	The second-order derivative at endpoints is zero	*Natural* or *relaxed* spline (the function becomes a straight line at endpoints)
B	The second-order derivative at endpoints is *extrapolated* from the adjacent interior knots (which ultimately leads to a continuity of the third-order derivative at the second and last but one nodes)	*Not-a-knot* or *extrapolated* spline
C	The first-order derivative at endpoints is specified	*Clamped, fixed-slope,* or *complete* spline
D	The second-order derivative at endpoints is specified	*Endpoint curvature-adjusted* spline
E	The third-order derivative at endpoints is forced to be zero	*Parabolically terminated* spline (the first and the last subintervals feature quadratic rather than cubic splines)

At this point, we are two conditions short. Therefore, this time, we have the luxury of enforcing two additional conditions, one at each endpoint. These additional conditions may vary as shown in Table 11.3.

Before computers, different drafting tools were used by illustrators and designers to build their drawings by hand. Among them, draftsmen often used long, thin, flexible strips of wood, plastic, or metal called splines, and that is where the term spline originates. The splines were held in place with weights or nails as shown in Fig. 11.6. The elasticity of the spline material would cause the strip to take the smoothest possible shape that minimizes the potential energy required for bending it between the knots. Specifically, Fig. 11.6 shows the natural spline with no bending at endpoints.

Among the five types of splines shown in Table 11.3, the MATLAB `spline` function allows you to create two types. They are the not-a-knot endpoint condition spline (by construction requiring no additional inputs

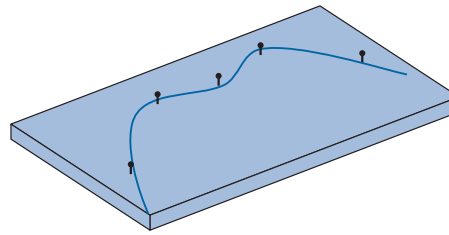


Fig. 11.6 Interpretation of the natural spline.

from the user beyond a set of data points) and the fixed-slope spline (for which the endpoint slopes should be defined explicitly).

Having the three conditions described in Table 11.2 complemented with two more for a variety of splines presented in Table 11.3 allows you to compute $3N$ coefficients in Eq. (11.71). We could follow the routine laid out for the quadratic spline and develop a system like Eq. (11.70) here as well, but from a practical standpoint, it is better to do it slightly differently, starting with explicit notations for the second-order derivative at each knot as m_i. For each subinterval of a cubic spline ($i = 1, 2, \ldots, N$), the second-order derivative is linear. Hence, it can be represented in the Lagrange form [Eq. (11.57)] as

$$\begin{aligned} f_i''(x) &= \frac{x - x_{i+1}}{x_i - x_{i+1}} m_i + \frac{x - x_i}{x_{i+1} - x_i} m_{i+1} \\ &= \frac{1}{h_i}[(x_{i+1} - x)m_i + (x - x_i)m_{i+1}] \end{aligned} \tag{11.72}$$

Integrating Eq. (11.72) twice yields

$$f_i(x) = \frac{(x_{i+1} - x)^3 m_i + (x - x_i)^3 m_{i+1}}{6h_i} + p_i(x_{i+1} - x) + q_i(x - x_i) \tag{11.73}$$

where the last two terms represent two constants of integration (we choose to write them in this form to simplify further analysis). To solve for p_i and q_i, we should rewrite Eq. (11.73) for $x = x_i$ and $x = x_{i+1}$, which yields

$$y_i = \frac{h_i^2 m_i}{6} + p_i h_i \quad \text{and} \quad y_{i+1} = \frac{h_i^2 m_{i+1}}{6} + q_i h_i \tag{11.74}$$

Hence,

$$p_i = \frac{y_i}{h_i} - \frac{h_i^2 m_i}{6h_i} \quad \text{and} \quad q_i = \frac{y_{i+1}}{h_i} - \frac{h_i^2 m_{i+1}}{6} \tag{11.75}$$

Substituting these values back into Eq. (11.73) results in an alternative representation for a cubic spline, which instead of $3N$ coefficients a_i, b_i,

c_i, $i = 1, 2, \ldots, N$ features $N + 1$ unknown second-order derivatives m_i:

$$f_i(x) = \frac{(x_{i+1} - x)^3 m_i + (x - x_i)^3 m_{i+1}}{6h_i} + \left(\frac{y_i}{h_i} - \frac{h_i m_i}{6}\right)(x_{i+1} - x) + \left(\frac{y_{i+1}}{h_i} - \frac{h_i m_{i+1}}{6}\right)(x - x_i) \tag{11.76}$$

Note that for Eqs. (11.72) and (11.73), we have already used two of the spline-defining conditions (the continuity of the second-order derivative and the function itself, respectively). However, we still have $N - 1$ conditions ensuring the continuity of the first-order derivative at the interior nodes plus any two from Table 11.3. These $N + 1$ conditions will allow us to find the unknown value of m_i at each knot. The only thing is that we will need to express these $N + 1$ conditions so that they become dependent on the second-order derivative.

Let us differentiate Eq. (11.76):

$$f_i'(x) = \frac{-(x_{i+1} - x)^2 m_i + (x - x_i)^2 m_{i+1}}{2h_i} - \left(\frac{y_i}{h_i} - \frac{h_i m_i}{6}\right) + \left(\frac{y_{i+1}}{h_i} - \frac{h_i m_{i+1}}{6}\right) \tag{11.77}$$

This formula defines the first-order derivative within the ith subinterval. Evaluating it at the left node, that is, at $x = x_i$, yields

$$f_i'(x_i) = \frac{-h_i m_i}{2} - \left(\frac{y_i}{h_i} - \frac{h_i m_i}{6}\right) + \left(\frac{y_{i+1}}{h_i} - \frac{h_i m_{i+1}}{6}\right) \tag{11.78}$$

By using the notation

$$\delta_i = \frac{y_{i+1} - y_i}{h_i} \tag{11.79}$$

for the first divided difference, we can further reduce Eq. (11.78) to

$$f_i'(x_i) = -\frac{h_i m_i}{3} - \frac{h_i m_{i+1}}{6} + \delta_i \tag{11.80}$$

Similarly, we can write an equation like Eq. (11.77) for the previous subinterval (by simply replacing i with $i - 1$, and $i + 1$ with i) and estimate it at the right node, that is, at $x = x_i$ again, which results in

$$f_{i-1}'(x_i) = \frac{h_{i-1} m_i}{3} + \frac{h_{i-1} m_{i-1}}{6} + \delta_{i-1} \tag{11.81}$$

Equating Eqs. (11.80) and (11.81), we get $N-1$ relations involving $N+1$ unknowns

$$h_{i-1}m_{i-1} + 2(h_{i-1}+h_i)m_i + h_i m_{i+1} = 6(\delta_i - \delta_{i-1}),$$
$$i = 2, 3, \ldots, N \tag{11.82}$$

The relations of Table 11.4 supply the expressions for m_1 and m_{N+1} for all cases listed in Table 11.3, allowing you to exclude them from the first and the last equations of Eq. (11.82), respectively. Therefore, we can finally write a tridiagonal system of $N-1$ linear equations for determining the remaining $N-1$ unknown second-order derivatives m_i as

$$\begin{bmatrix} \alpha_{1;1} & \alpha_{1;2} & 0 & \cdots & 0 & 0 & 0 \\ h_2 & 2(h_2+h_3) & h_3 & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & h_{N-2} & 2(h_{N-2}+h_{N-1}) & h_{N-1} \\ 0 & 0 & 0 & \cdots & 0 & \alpha_{N;N-1} & \alpha_{N;N} \end{bmatrix} \begin{bmatrix} m_2 \\ m_3 \\ \vdots \\ m_{N-1} \\ m_N \end{bmatrix}$$
$$= \begin{bmatrix} 6(\delta_2 - \delta_1) - \Delta_2 \\ 6(\delta_3 - \delta_2) \\ \vdots \\ 6(\delta_{N-1} - \delta_{N-2}) \\ 6(\delta_N - \delta_{N-1}) - \Delta_N \end{bmatrix} \tag{11.83}$$

where six variables for the first and last equations are defined for each type of spline in Table 11.5. After all unknowns m_i, $i = 1, 2, \ldots, N+1$ are

Table 11.4 Endpoint Conditions for the Second-Order Derivative of Cubic Spline

	Type of Spline	Expression for the Second-Order Derivative
A	Natural spline	$m_1 = 0, \quad m_{N+1} = 0$
B	Not-a-knot spline	$m_1 = m_2 - \frac{h_1}{h_2}(m_3 - m_2), \quad m_{N+1} = m_N + \frac{h_N}{h_{N-1}}(m_N - m_{N-1})$
C	Clamped spline	$m_1 = \frac{3}{h_1}(\delta_1 - y'_1) - \frac{m_2}{2}, \quad m_{N+1} = \frac{3}{h_N}(y'_{N+1} - \delta_N) - \frac{m_N}{2}$
D	Endpoint curvature-adjusted spline	$m_1 = y''_1, \quad m_{N+1} = y''_{N+1}$
E	Parabolically terminated spline	$m_1 = m_2, \quad m_{N+1} = m_N$

Table 11.5 Coefficients of the First and Last Equations for the Tridiagonal System [Eq. (11.83)]

Number	$\alpha_{1;1}$	$\alpha_{1;2}$	$\alpha_{N;N-1}$	$\alpha_{N;N}$	Δ_2	Δ_N
A	$2(h_1 + h_2)$	h_2	h_{N-1}	$2(h_{N-1} + h_N)$	0	0
B	$3h_1 + 2h_2 + \frac{h_1^2}{h_2}$	$h_2 - \frac{h_1^2}{h_2}$	$h_{N-1} - \frac{h_N^2}{h_{N-1}}$	$2h_{N-1} + 3h_N + \frac{h_N^2}{h_{N-1}}$	0	0
C	$\frac{3}{2}h_1 + 2h_2$	h_2	h_{N-1}	$2h_{N-1} + \frac{3}{2}h_N$	$3(\delta_1 - y'_1)$	$3(y'_{N+1} - \delta_N)$
D	$2(h_1 + h_2)$	h_2	h_{N-1}	$2(h_{N-1} + h_N)$	$h'_1\ y''_1$	$h_N\ y''_{N+1}$
E	$3h_1 + 2h_2$	h_2	h_{N-1}	$2(h_{N-1} + 3h_N)$	0	0

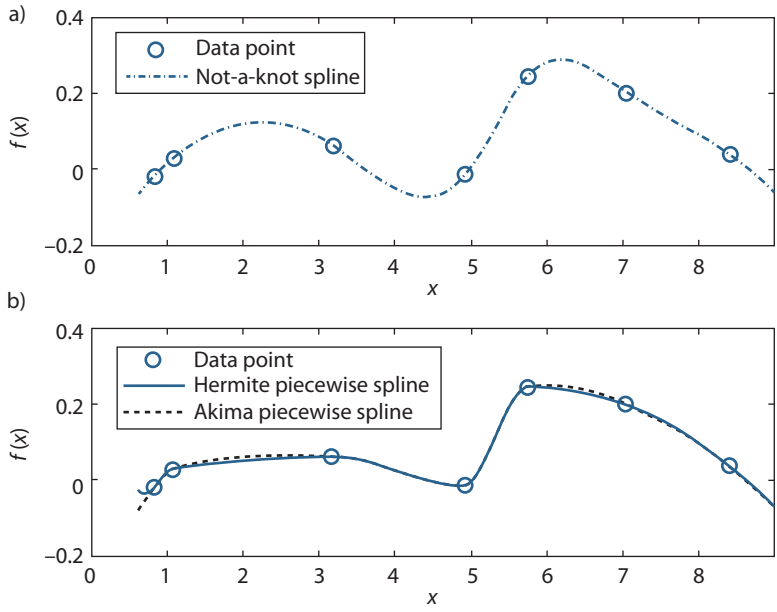


Fig. 11.7 a) Not-a-knot spline and b) piecewise cubic interpolations.

determined, coefficients of the original spline [Eq. (11.71)] can be related to them as

$$a_i = \frac{m_{i+1} - m_i}{6h_i}, \quad b_i = \frac{m_i}{2}, \quad c_i = \delta_i - \frac{h_i(2m_i + m_{i+1})}{6}, \quad i = 1, 2, \ldots, N \tag{11.84}$$

An example of not-a-knot spline passing through the same data points as the full-degree polynomial approximation of Fig. 11.4 is shown in Fig. 11.7a. As seen, it exhibits much less overshooting compared with that of Fig. 11.4, but it can still return the questionable interpolation values.

11.4.3 Cubic Splines Defined Locally

As shown in the previous section, cubic splines ensure the continuity of the first- and second-order derivatives and therefore are frequently used in practice. The discontinuity of the third-order derivative poses no problem and in fact cannot be detected visually anyway. On the contrary, sometimes it is worth sacrificing continuity of even the second-order derivative to preserve a general shape of approximation.

Suppose, we know the data points (x_i, y_i), $i = 1, 2, \ldots, N+1$, but we have to satisfy the first-order derivatives d_i at all $N+1$ nodes as well. In such a case, using notations of the previous section (the length of subinterval h_i

and the first divided difference δ_i), the following interpolating expression for each of N subintervals can be derived as

$$f_i(x) = \frac{3h_i s_i^2 - 2s_i^3}{h_i^3} y_{i+1} + \frac{h_i^3 - 3h_i s_i^2 + 2s_i^3}{h_i^3} y_i + \frac{s_i^2(s_i - h_i)}{h_i^2} d_{i+1} + \frac{s_i(s_i - h_i)^2}{h_i^2} d_i, \quad x_i \le x \le x_{i+1} \tag{11.85}$$

The *Hermite* or *osculatory* interpolant is one of the widely used approaches that allows satisfying conditions on derivatives. ("Osculari" means "to kiss" in Latin, which as used in this chapter to imply passing through the knots at a certain angle, that is, touching or "kissing" the derivative constraints at the knots.) The Hermite interpolation [Eq. (11.85)] assumes that the slopes d_i are given, but if not, we need to define them somehow. The approach used in the MATLAB `pchip` and `makima` functions produces a *shape-preserving* "visually pleasing" interpolation by defining the first-order derivatives at the knots using a set of empirical rules.

The key idea in determining the slopes d_i for the Hermite interpolant is that the function values in between the nodes do not overshoot those at endpoints of the current subinterval, as was the case in Fig. 11.4 for the full-degree polynomial approximation or in Fig. 11.7a for a regular cubic spline. In doing this, the following intuitive rules apply for all interior knots:

1. If δ_i and δ_{i-1} have opposite signs, or if either of them is zero (that is, $\delta_i\ \delta_{i-1} \le 0$), then x_i is considered to be a local minimum/maximum, so that

$$d_i = 0 \tag{11.86}$$

2. If δ_i and δ_{i-1} have the same signs, then d_i is computed as a weighted harmonic mean, with the weights determined by the lengths of two adjacent subintervals

$$\frac{w_{1i} + w_{2i}}{d_i} = \frac{w_{1i}}{\delta_{i-1}} + \frac{w_{2i}}{\delta_i} \tag{11.87}$$

where $w_{1i} = 2h_i + h_{i-1}$ and $w_{2i} = h_i + 2h_{i-1}$. For same-length subintervals, this gives a harmonic mean of the two slopes

$$\frac{1}{d_i} = \frac{1}{2}\left(\frac{1}{\delta_{i-1}} + \frac{1}{\delta_i}\right) \tag{11.88}$$

(that is, the reciprocal slope of the Hermite interpolant is the average of the reciprocal slopes of the piecewise linear interpolant on either side).

The slopes d_1 and d_{N+1} at either endpoint are determined by a slightly different, one-sided (noncentered), three-point formula as

$$d_1 = \frac{(2h_1 + h_2)\delta_1 - h_1\delta_2}{h_1 + h_2} \quad \text{and}$$
$$d_{N+1} = \frac{(2h_N + h_{N-1})\delta_N - h_N\delta_{N-1}}{h_N + h_{N-1}} \tag{11.89}$$

respectively. If the sign of d_1 differs from the sign of d_2, then it is set to be 0 (the same correction applies for the pair d_{N+1} and d_N). The `pchip` function also limits the magnitude of the slope d_1 (d_{N+1}) to that of $3\delta_1$ ($3d_N$) in the case the knot 2 (N) was assumed to be a local extremum at the previous steps (while computing the slopes at interior knots).

An example of a Hermite spline is shown in Fig. 11.7b. It passes through the same data points as the full-degree polynomial approximation of Fig. 11.4 and not-a-knot spline approximation of Fig. 11.7a. Compared with those two, there is no overshooting; however, we had to pay with approximation smoothness.

Not only does this approach produce a shape-preserving approximation, but it also has the very nice property of being *locally determined*. This means that if one data point was changed or additional point introduced, you only have to recompute coefficients for two adjacent cubic Hermite interpolation splines, not for all of them as would be the case with the regular splines, which are *determined globally* [via Eq. (11.70) or (11.83)]. In other words, the perturbance introduced at some point for the locally determined splines does not propagate beyond two adjacent subintervals.

Figure 11.7b also features another locally-determined spline, the modified Akima piecewise cubic Hermite interpolation. While using the same idea presented in Eq. (11.85), it employs a slightly different set of rules compared with those in Eqs. (11.86)–(11.89). Specifically, there is no forced flattening established by Eq. (11.86). Instead, the first-order derivatives at the interior nodes are computed as a weighted sum

$$d_j = \frac{w_{j-1}\delta_{j-1} + w_j\delta_j}{w_{j-1} + w_j} \tag{11.90}$$

where the two weights are

$$w_{j-1} = |\delta_{j+1} - \delta_j| \quad \text{and} \quad w_j = |\delta_{j-1} - \delta_{j-2}| \tag{11.91}$$

To be able to apply the same weighted sum formula (11.90) for estimating derivatives at endpoints [instead of using Eq. (11.89), the fictitious trends beyond the data range are determined as

$$\delta_{-1} = 2\delta_0 - \delta_1, \quad \delta_0 = 2\delta_1 - \delta_2, \quad \delta_{N+1} = 2\delta_N - \delta_{N-1}, \quad \text{and}$$
$$\delta_{N+2} = 2\delta_{N+1} - \delta_N \tag{11.92}$$

As a result, the order of interpolation for the first and the last subintervals is 2.

The piecewise cubic Akima interpolant (realized in `makima` function and as one of the options in the `interp1`, ..., `interpn` family of functions) differs from that of Hermite in that it does not flatten as aggressively and can be constructed with as few as only two points (a single interval).

Let us make some concluding remarks. There is always a tradeoff between smoothness and the somewhat subjective property of shape preservation or *local monotonicity*. The piecewise linear interpolant [Eq. (11.68)] is at one extreme. It has hardly any smoothness. It is continuous, but there are jumps in its first derivative. On the other hand, it preserves the local monotonicity of data. It never overshoots and is increasing, decreasing, or remaining constant within each subinterval. The full-degree polynomial interpolant is at the other extreme. It is infinitely differentiable. However, it fails to preserve a shape, particularly near the endpoints.

The cubic Hermite and Akima shape-preserving interpolants along with the regular spline are in between these two extremes. The regular spline is smoother than the shape-preserving spline. The regular spline has two continuous derivatives, whereas a shape-preserving spline has only one. A discontinuous second-order derivative implies discontinuous curvature. The human eye can detect large jumps in curvature in graphs and/or in mechanical parts made by numerically controlled machine tools. However, the cubic Hermite and Akima interpolants are guaranteed to preserve a shape, whereas the regular spline might not.

11.4.4 Interpolation with the Bézier Curves

Just to broaden your horizons, it is worth mentioning that the monomials are not the only basis for approximating data. Other basis, for instance Jacobi, Chebyshev, Laguerre, and other polynomials or trigonometric functions, might also be used. To conclude the discussion on the different types of approximation, let us introduce an example of one more, parametrically defined, polynomial where additional flexibility is required.

The parametrically defined Bézier curves, developed in the late 1950s by Paul de Casteljau at Citroën and publicized in the early 1960s by Pierre Bézier at Renault (who used them to design automobile bodies), belong to a class of approximating splines. This type of spline forms the basis of the entire Adobe drawing software family and continues to be the primary method of representing curves and surfaces in computer graphics, computer-aided design, and computer-aided manufacturing.

Although the data points we have dealt with up to this point are treated differently (being referred to as the *control points*), the general idea still

applies. Given a set of points (x_i,y_i) $i = 0,1, \ldots ,N$, the 2-D Bézier curve $y(x)$ is defined parametrically using a dimensionless parameter $\tau \in [0;1]$ as

$$x(\tau) = \sum_{i=0}^{N} x_i B_{i;N}(\tau) \quad \text{and} \quad y(\tau) = \sum_{i=0}^{N} y_i B_{i;N}(\tau) \tag{11.93}$$

with the basis functions $B_{i;N}(\tau)$ being the *Bernstein polynomials* (Table 7.16)

$$B_{i;N}(\tau) = \frac{N!}{i!(N-i)!}\tau^i(1-\tau)^{N-i} \tag{11.94}$$

Figures 11.8a and 11.8b show examples of the basis functions $B_{i;N}(\tau)$ for $N = 3$ and $N = 15$, respectively.

These basis functions have certain useful properties to make things work. To illustrate some of these properties, consider a cubic Bézier curve defined by four points. In this case, $N = 3$ and Eq. (11.93) becomes

$$\begin{aligned} x(\tau) &= x_0B_{0;3}(\tau) + x_1B_{1;3}(\tau) + x_2B_{2;3}(\tau) + x_3B_{3;3}(\tau) \\ y(\tau) &= y_0B_{0;3}(\tau) + y_1B_{1;3}(\tau) + y_2B_{2;3}(\tau) + y_3B_{3;3}(\tau) \end{aligned} \tag{11.95}$$

According to Eq. (11.94), the Bernstein polynomials of the third-order are

$$\begin{aligned} &B_{0;3}(\tau) = (1-\tau)^3, \quad B_{1;3}(\tau) = 3\tau(1-\tau)^2, \\ &B_{2;3}(\tau) = 3\tau^2(1-\tau), \quad B_{3;3}(\tau) = \tau^3 \end{aligned} \tag{11.96}$$

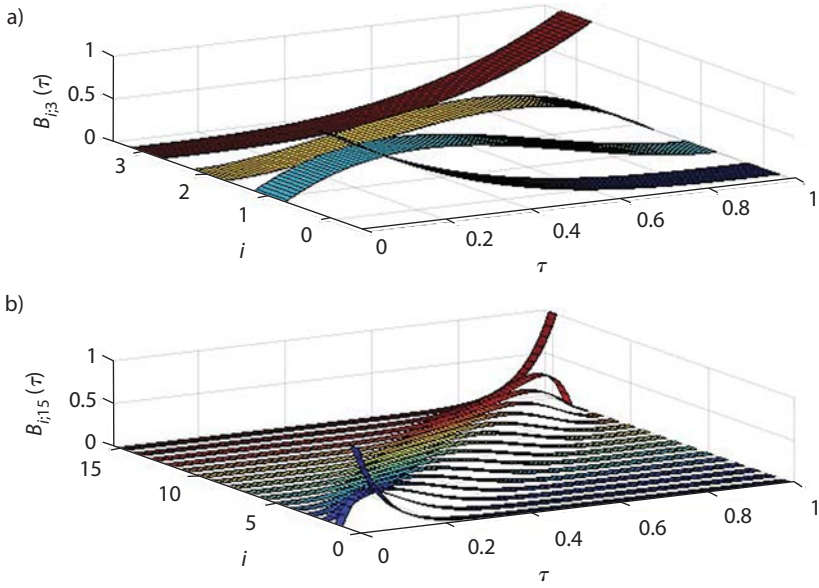


Fig. 11.8 Bernstein polynomials with a) $N = 3$ and b) $N = 15$.

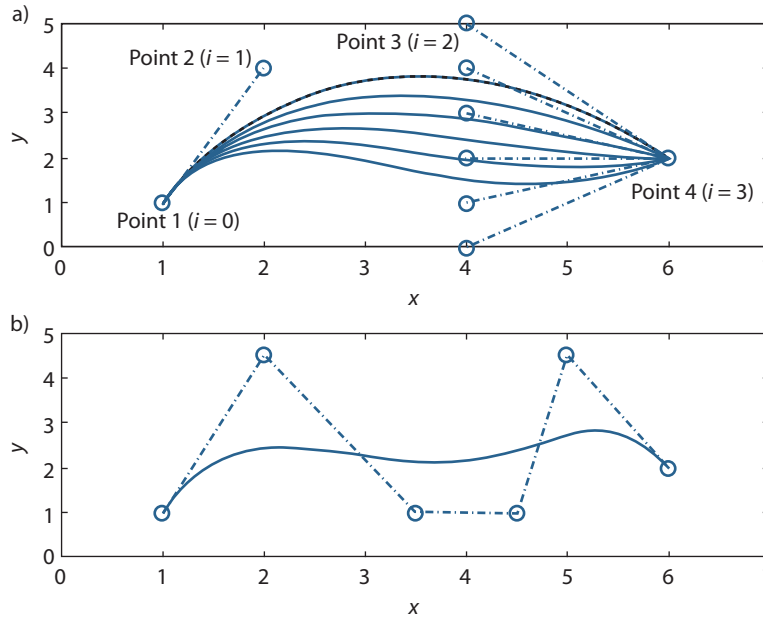


Fig. 11.9 Bézier curve with a) four and b) six control points.

(depicted in Fig. 11.8a). Substituting them into Eq. (11.95) yields

$$\begin{aligned} x(\tau) &= x_0(1-\tau)^3 + x_1 3\tau(1-\tau)^2 + x_2 3\tau^2(1-\tau) + x_3\tau^3 \\ y(\tau) &= y_0(1-\tau)^3 + y_1 3\tau(1-\tau)^2 + y_2 3\tau^2(1-\tau) + y_3\tau^3 \end{aligned} \tag{11.97}$$

Simplifying Eq. (11.97) results in

$$\begin{aligned} x(\tau) &= (x_3 - 3x_2 - 3x_1 - x_0)\tau^3 + (3x_2 - 6x_1)\tau^2 + 3x_1\tau + x_0 \\ y(\tau) &= (y_3 - 3y_2 - 3y_1 - y_0)\tau^3 + (3y_2 - 6y_1)\tau^2 + 3y_1\tau + y_0 \end{aligned} \tag{11.98}$$

By construction, a Bézier curve always starts at the first data point and ends at the last one. Other points are used to provide guidance on interpolant curvature, so that the curve never passes through them. That is why all together they are called the control points. An example of a four-point Bézier curve $y(x)$ is shown in Fig. 11.9a. This figure also shows how the curve changes its shape when the third point moves up and down. Similarly, you can change the slope at the first point by moving around the second one. Figure 11.9b presents another example with six points (four of which are used to control the curvature of approximation passing through the first and last points).

In the case of Fig. 11.9b, the Bézier curve is defined globally using all the data points as the control points, and therefore the approximation does not pass through them (except endpoints). Alternatively, for the curve fitting

problem, we could use a sequence of the low-order Bézier curves like the ones shown in Fig. 11.9a and stich them at the nodes the approximation should go through (*composite Bézier curves*). For each subinterval, we would add one or two control points to vary the shape in between the nodes to ensure smoothness of the overall approximation. In fact, the idea is quite similar to that used to go from the higher-order full-degree polynomial approximation down to piecewise lower-order (cubic) splines.

11.5 Fourier Approximation

Up to this point, all approximations used either monomials, $1, x, x^2, \ldots, x^p$, (Secs. 11.2 and 11.3) or Bernstein polynomials (Sec. 11.4) as the basis functions. Let us consider another important class of the basis functions, trigonometric functions: $1, \cos(x), \cos(2x), \ldots, \cos(Px), \sin(x), \sin(2x), \ldots, \sin(Px)$ (Fig. 11.10). These functions play a fundamental role in signal processing and modeling of oscillating or vibrating systems. The *Fourier analysis* represents a systematic framework for using trigonometric series for this purpose. In this section, we will use a *Fourier series* to decompose any periodic function or signal into a sum of finite sets of simple oscillating functions, sines and cosines (or complex exponentials), casting the problem as the least-squares regression.

Consider the well-known sinusoid (the term *sinusoid* represents any waveform that can be described as a sine or cosine), which can, in general, be expressed in the amplitude-phase form as

$$f(t) = a_0 + c_1 \cos(\omega_0 t + \theta) \tag{11.99}$$

Four parameters, the *mean value* a_0, the *amplitude* c_1, the *angular frequency* ω_0, and the *phase shift* θ, serve to characterize the sinusoid. Another way of representing the very same sinusoid is the sine-cosine form

$$f(t) = a_0 + a_1 \cos(\omega_0 t) + b_1 \sin(\omega_0 t) \tag{11.100}$$

This representation is more convenient for our purpose because it is cast in the format of a linear (with respect to a_0, a_1, and b_1) model with two new

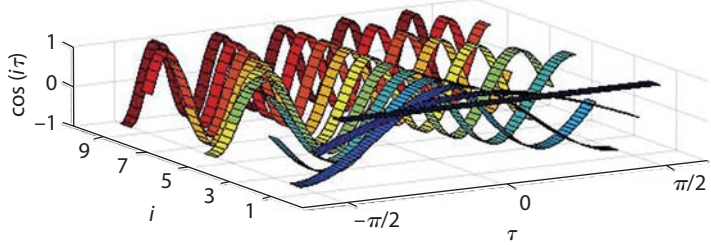


Fig. 11.10 Example of a trigonometric basis functions: $\cos(i\tau)$, $i = 0, 1, \ldots, 10$, $\tau \in [-\pi/2; \pi/2]$.

parameters, a_1 and b_1, which can be related to those of Eq. (11.99) as

$$a_1 = c_1 \cos\theta \quad \text{and} \quad b_1 = -c_1 \sin\theta \tag{11.101}$$

Having a set of data points, (t_k, y_k), $k = 0, 2, \ldots, N-1$, we can proceed with the standard routine established in Sec. 11.2.1, that is, define the sum of squared residuals for trigonometric approximation [Eq. (11.100)]

$$S_r = \sum_{k=0}^{N-1} r_k^2 = \sum_{k=0}^{N-1} [y_k - a_0 - a_1 \cos(\omega_0 t_k) - b_1 \sin(\omega_0 t_k)]^2 \tag{11.102}$$

differentiate it with respect to three unknown coefficients, and equate these partial derivatives to zero, which results in the following system of LAEs:

$$\begin{bmatrix} N & \sum_{k=0}^{N-1} \cos(\omega_0 t_k) & \sum_{k=0}^{N-1} \sin(\omega_0 t_k) \\ \sum_{k=0}^{N-1} \cos(\omega_0 t_k) & \sum_{k=0}^{N-1} \cos^2(\omega_0 t_k) & \sum_{k=0}^{N-1} \cos(\omega_0 t_k)\sin(\omega_0 t_k) \\ \sum_{k=0}^{N-1} \sin(\omega_0 t_k) & \sum_{k=0}^{N-1} \cos(\omega_0 t_k)\sin(\omega_0 t_k) & \sum_{k=0}^{N-1} \sin^2(\omega_0 t_k) \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ b_1 \end{bmatrix}$$

$$= \begin{bmatrix} \sum_{k=0}^{N-1} y_k \\ \sum_{k=0}^{N-1} y_k \cos(\omega_0 t_k) \\ \sum_{k=0}^{N-1} y_k \sin(\omega_0 t_k) \end{bmatrix} \tag{11.103}$$

However, rather than doing this, let us examine a special case in which N observations are spaced evenly over the range $t \in [t_0;\ t_{N-1}]$ with $\Delta t = T(N-1)^{-1}$, where $T = t_{N-1} - t_0$. For large N, it is safe to assume the following average values:

$$\sum_{k=0}^{N-1} \cos(\omega_0 t_k) = \sum_{k=0}^{N-1} \sin(\omega_0 t_k) = \sum_{k=0}^{N-1} \cos(\omega_0 t_k)\sin(\omega_0 t_k) = 0,$$

$$\frac{1}{N}\sum_{k=0}^{N-1} \cos^2(\omega_0 t_k) = \frac{1}{N}\sum_{k=0}^{N-1} \sin^2(\omega_0 t_k) = \frac{1}{2} \tag{11.104}$$

Thus, for the evenly spaced data points, Eq. (11.103) may be reduced to

$$\begin{bmatrix} N & 0 & 0 \\ 0 & 0.5N & 0 \\ 0 & 0 & 0.5N \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ b_1 \end{bmatrix} = \begin{bmatrix} \sum_{k=0}^{N-1} y_k \\ \sum_{k=0}^{N-1} y_k \cos(\omega_0 t_k) \\ \sum_{k=0}^{N-1} y_k \sin(\omega_0 t_k) \end{bmatrix} \tag{11.105}$$

The inverse of a diagonal matrix is merely another diagonal matrix whose elements are the reciprocals of the original one. Therefore, unknown coefficients can be determined as

$$\begin{aligned} a_0 &= \frac{1}{N}\sum_{k=0}^{N-1} y_k, \quad a_1 = \frac{2}{N}\sum_{k=0}^{N-1} y_k \cos(\omega_0 t_k), \\ b_1 &= \frac{2}{N}\sum_{k=0}^{N-1} y_k \sin(\omega_0 t_k) \end{aligned} \tag{11.106}$$

So far, our approximation has only three terms, relying on a single angular frequency ω_0, but we can easily add more terms with multiple frequencies so that Eq. (11.100) becomes

$$f(t) = a_0 + \sum_{j=1}^{P} \left[a_j \cos(j\omega_0 t) + b_j \sin(j\omega_0 t)\right] \tag{11.107}$$

In the case of evenly spaced data, the linear least-squares regression then yields

$$\begin{aligned} a_0 &= \frac{1}{N}\sum_{k=0}^{N-1} y_k, \quad a_j = \frac{2}{N}\sum_{k=0}^{N-1} y_k \cos(j\omega_0 t_k), \\ b_j &= \frac{2}{N}\sum_{k=0}^{N-1} y_k \sin(j\omega_0 t_k), \quad j = 1, 2, \ldots, P \end{aligned} \tag{11.108}$$

In the case of $N > 2P + 1$, Eqs. (11.107) and (11.108) can be used to fit data in the least-squares regression sense. If $N = 2P + 1$, Eq. (11.107) becomes an interpolant passing through all data points (similar to the full-degree polynomial case). Increasing N to infinity leads to the continuous time Fourier series (CTFS)

$$f(t) = a_0 + \sum_{j=1}^{\infty} \left[a_j \cos(j\omega_0 t) + b_j \sin(j\omega_0 t)\right] \tag{11.109}$$

with

$$a_0 = \frac{1}{T}\int_0^T f(t)\mathrm{d}t, \quad a_j = \frac{2}{T}\int_0^T f(t)\cos(j\omega_0 t)\mathrm{d}t,$$
$$b_j = \frac{2}{T}\int_0^T f(t)\sin(j\omega_0 t)\mathrm{d}t, \quad j = 1, 2, \ldots, \infty \tag{11.110}$$

This basically shows that any arbitrary periodic function $f(t)$ with a period T can be represented by an infinite series of sinusoids of harmonically related frequencies [Eq. (11.109)], where $\omega_0 = 2\pi T^{-1}$ is called the *fundamental frequency* and its constant multiples, $2\omega_0$, $3\omega_0$, etc., are called *harmonics*. The relation

$$P \leq \frac{N-1}{2} \tag{11.111}$$

is basically equivalent to the *Nyquist–Shannon sampling theorem*, stating that perfect reconstruction of a signal is possible only when the sampling frequency (number of points N) is greater than twice the maximum frequency of the signal being sampled (P). If the lower sampling rates are used (not enough data points N), the original signal's information may not be completely recoverable from the sampled signal.

DFT coefficients [Eq. (11.108)] can be combined into a single complex quantity (the exponential form)

$$c_j = a_j + ib_j = \frac{2}{N}\sum_{k=0}^{N-1} y_k[\cos(j\omega_0 t_k) + i\sin(j\omega_0 t_k)]$$
$$= \frac{2}{N}\sum_{k=0}^{N-1} y_k e^{ij\omega_0 t_k}, \quad j = 0, 2, \ldots, P \tag{11.112}$$

($i = \sqrt{-1}$), which may be computed more efficiently. Generally speaking, in the context of this chapter, the MATLAB `fft` function computes the sums in Eq. (11.112) (although with a negative sign of the exponent argument) using the *fast Fourier transform* (FFT) algorithm. Once c_j are computed, the original coefficients can be reconstructed as

$$a_j = \mathrm{Re}(c_j) \quad (a_0 = \mathrm{Re}(c_0)/2) \quad \text{and} \quad b_j = \mathrm{Im}(c_j) \tag{11.113}$$

11.6 Using the MATLAB Functions

In this section, we start with presenting the functions enabling polynomial-based regression and interpolation, followed by the functions

used for piecewise polynomial interpolation. Then, the trigonometric series regression functions are considered. The section ends with a brief discussion of a couple of the recently introduced functions for data preconditioning.

11.6.1 Polynomial Approximations

As mentioned in Sec. 11.3, MATLAB features one major function, `polyfit`, which allows you to perform both least-squares regression and full-degree polynomial interpolation. Its simplest syntax is

```
p=polyfit(x,y,n)
```

where `x` and `y` are the vectors defining the data set, and `n` is the power of polynomial chosen to fit data (`n` ≤ `length(x)-1`). This simplest call returns a row vector `p` of length `n+1` containing the fitting polynomial coefficients p_i starting with the coefficient of x^n, i.e. in the order adopted by all MATLAB functions (see Sec. 3.5)

$$[\,p_1 \quad p_2 \quad \cdots \quad p_{n+1}\,][\,x^n \quad x^{n-1} \quad \cdots \quad 1\,]^T = p_1 x^n + p_2 x^{n-1} + \cdots + p_{n+1} \tag{11.114}$$

For example, a set of commands

```
>> x1=linspace(1,10,500)'; y1=rand(1,500)';
>> p=polyfit(x1,y1,1)
```

generates two vectors, the evenly spaced vector `x1` and vector `y1` with 500 uniformly distributed pseudorandom numbers. Then, these data are fit with the first-order polynomial. The result is quite obvious

```
p = -0.0017 0.5021
```

that is, $y = -0.0017x + 0.5021$, basically meaning that the regression is almost horizontal (has a very shallow negative slope) and the y-intercept is very close to the mean value

```
>> mean(y)
ans = 0.4926
```

The `polyfit` function employs the QR factorization [see Eq. (9.51) of Sec. 9.4] of a matrix defined by Eq. (11.13) [or Eq. (11.54)] (returning the vector of coefficients in the reversed order). Hence, the aforementioned linear regression could be obtained using the `qr` function explicitly (note how we reversed the order of equations compared to that of Eq. (11.13)):

```
>> [Q,R] = qr([sum(x1.^2) sum(x1); sum(x1) 500],0);
>> p=R\(Q'*[sum(x1.*y1); sum(y1)])
```

which returns

```
p = -0.0017
     0.501
```

Another syntax of `polyfit` features two output parameters

```
[p,s]=polyfit(x,y,n)
```

In this case, along with a vector of polynomial coefficients, it returns a structure `s` that contains three fields providing additional information about the solution. Specifically, these fields are: the upper triangular matrix `R`, received after the *Cholesky decomposition* (of a symmetric, positive-definite matrix of Eq. (11.13) into the product of a lower triangular matrix and its conjugate transpose), the number of degrees of freedom `df`, and the sum of squared residuals `normr`. You may assess any field of the structure `s` by using a standard dot notation, e.g. `s.normr`. For example, the commands

```
>> x2=[1:5]'; y2=[2 5 8 20 26]';
>> [p,s]=polyfit(x2,y2,2)
```

return

```
p = 1.0714   -0.1286   0.8000
s =      R: [3×3 double]
        df: 2
    normr: 3.4393
```

In this return, the `normr` is computed as `norm(y2-polyval(p,x2))`, `df` of `2` suggests that we still have two free degrees of freedom allowing increasing the order of a fitting polynomial up to four (which is true because the order of a full-degree polynomial for five points is four), and the matrix `R`, which can be displayed by typing

```
>> s.R
ans = -31.2890   -7.1910   -1.7578
            0   -1.8136   -1.3010
            0         0    0.4663
```

is `-chol(V)`, where `V` is formed as

```
N=3; % the number of coefficients to be determined
for i=1:N
    for j=1:N
    V(i,j)=sum(x.^(2*N-i-j));
    end
end
```

Using the output parameter `s.normr` and employing the MATLAB function `std` to compute a standard deviation, we may compute the coefficient of determination [Eq. (11.21)]. For example, for the 10 data points generated as

```
>> N=10; xd=linspace(0,10,N); yd=xd.^2+5*randn(1,N);
```

and the sum of squared residuals around the mean value S_t [Eq. (10.20)]

```
>> St=std(yd)^2*(N-1);
```

we can explore the goodness of different fitting polynomials as

```
>> [p1,S1]=polyfit(xd,yd,1);   Rsq1=(St-S1.normr^2)/St
>> [p2,S2]=polyfit(xd,yd,2);   Rsq2=(St-S2.normr^2)/St
>> [p9,S9]=polyfit(xd,yd,N-1); Rsq9=(St-S9.normr^2)/St
```

which returns the values of R^2

```
Rsq1 =   0.9202
Rsq2 =   0.9874
Rsq9 =   1
```

Obviously, in terms of R^2, increasing the order of a polynomial regression makes the fit "better" (R^2 gets closer to 1). However, you should be careful not to ascribe more meaning to it than is warranted because all it means is that the sum of the squared residuals gets smaller. The following script visualizes the results of regression we just performed (Fig. 11.11):

```
plot(xd,yd,'ro'), hold
xr=linspace(xd(1),xd(end)); % create a finer grid
yr1=polyval(p1,xr);
yr2=polyval(p2,xr);
yr9=polyval(p9,xr);
plot(xr,yr1,'k-.',xr,yr2,'m--',xr,yr9)
legend('Data points','Linear regression','Quadratic regression',...
       'Full-degree polynomial fit','location','nw')
xlabel('x-axis'), ylabel('y-axis')
```

Obviously, despite the fact that for the full-degree polynomial we have $R^2 = 1$, the approximation looks weird.

The most comprehensive syntax of `polyfit` features three output parameters

```
[p,s,mu]=polyfit(x,y,n)
```

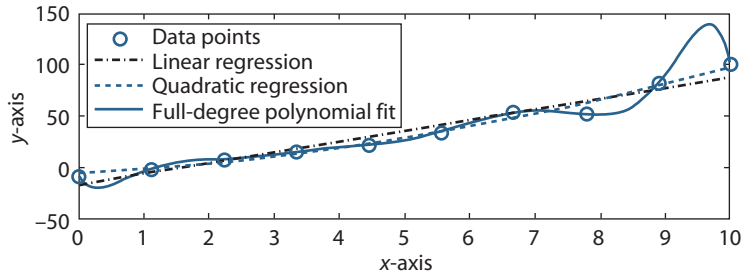


Fig. 11.11 Examples of polynomial regressions.

where mu is a two-element vector. This call finds coefficients of polynomial in a specially constructed domain with a new argument

$$\tilde{x} = \frac{x - \mu_1}{\mu_2} \tag{11.115}$$

with

$$\mu_1 = \frac{1}{N}\sum_{i=1}^{N} x_i \quad \text{and} \quad \mu_2 = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(x_i - \mu_1)^2} \tag{11.116}$$

denoting the mean value (mu(1)=mean(x)) and standard deviation (mu(2)=std(x)) of the x-data vector. This centering and scaling transformation improves numerical properties of both the polynomial and fitting algorithm. However, you should be careful because this syntax obviously returns different values of the fitting polynomial. Continuing the aforementioned example with the data vectors x2 and y2, the command

```
>> [pc,s,mu]=polyfit(x2,y2,2)
```

returns

```
pc =       2.6786  9.9612  11.0571
s =        R: [3×3 double]
          df: 2
       normr: 3.4393
mu =    3.0000
        1.5811
```

Hence, instead of $y = 1.0714x^2 - 0.1286x + 0.8$, we now get

$$y = 2.6786\left(\frac{x - \mu_1}{\mu_2}\right)^2 + 9.9612\frac{x - \mu_1}{\mu_2} + 10.0571 \tag{11.117}$$

To check whether it matches the original polynomial in x, you may use the following commands (involving the Symbolic Math Toolbox functions syms, simplify, expand, subs, and poly2sym):

```
>> syms x
>> ex = simplify(expand(subs(poly2sym(pc),x,(x-mu(1))/mu(2))))
```

to obtain

```
ex = (15*x^2)/14-(9*x)/70+4/5
```

To avoid the vulgar fractions, you may further employ the vpa function; for example, the command

```
>> vpa(ex,3)
```

transfers the previous result to

```
ans = 1.07*x^2-0.129*x+0.8
```

Now, let us try to use a linear regression to fit nonlinear data. As discussed in Sec. 11.2.5, we may try employing the following call to fit the power law data [Eq. (11.25)]:

```
coeff=polyfit(log10(x),log10(y),1)
```

Regression coefficients of the original power law relationship can then be obtained as `a=10^coeff(2)` and `b=coeff(1)`. For the exponential law relationship [Eq. (11.30)], the `polyfit` function can be used as

```
coeff=polyfit(x,log10(y),1)
```

for powers of base 10 and

```
coeff=polyfit(x,log(y),1)
```

for a natural base exponential function. Coefficients a and b of Eq. (11.30) can then be estimated as `b=coeff(1)` and `a=10^coeff(2)` (`a=exp(coeff(2))`).

As mentioned in Sec. 11.2.5, these pseudolinear regressions minimize the sum of squared weighted residuals [like those shown in Eqs. (11.29), (11.41), and (11.46)] and are therefore less accurate than those obtained via minimization [Eq. (11.24)]. As an example, the following script explores the two types of regressions (obtained via least-squares linear regression equation and nonlinear minimization) for a nonlinear dependence of Eq. (11.37):

```
%% Define the function
f=@(a,b,x) a*x./(b+x); at=3; bt=2.55;
%% Produce "experimental" data
x=[1:2:20]'; n=length(x); xp=linspace(x(1),x(end))';
y=f(at,bt,x)+0.2*randn(n,1);
subplot(211)
plot(x,y,'ok'), hold, xlabel('x-axis'), ylabel('y-axis')
%% Linear regression in the modified domain
subplot(212)
c=polyfit(1./x,1./y,1); % pay attention to the input arguments !!!
yp=polyval(c,1./xp);
plot(1./x,1./y,'ok'), hold, plot(1./xp,yp,'b--')
xlabel('1/x'), ylabel('1/y')
ar=1/c(2); br=c(1)/c(2);
legend('Data points','Algebraic solution','Location','SE')
% Convert the results of linear LS regression back to the original domain
subplot(211)
plot(xp,f(ar,br,xp),'b--')
Srr=norm(y-f(ar,br,x)); % sum of squared residuals for regression
text(6,1.0,['S_r^{reg}=' num2str(Srr,3)])
%% Minimization in the original domain
fn=@(a) norm(y-f(a(1),a(2),x));
[c,Sropt]=fminsearch(fn,[ar,br]);
```

```
plot(xp,f(c(1),c(2),xp),'b')
text(6,0.4,['S_r^{opt}=' num2str(Sropt,3)])
legend('Data points','Algebraic solution','Minimization','Location','SE')
%% Compare individual errors
figure
title('Residuals for the pseudolinear regression and minimization')
subplot(211), bar(y-f(ar,br,x))
text(5,.25,'... obtained via algebraic solution')
xlabel('Data point'), ylabel('Residual'), ylim(0.3*[-1 1])
subplot(212), bar(y-f(c(1),c(2),x))
text(5,.25,'... obtained via minimization')
xlabel('Data point'), ylabel('Residual'), ylim(0.3*[-1 1])
```

Specifically, this script generates some data points and then follows a routine outlined in Sec. 11.2.5 to convert the physical domain to some other domain where the linear algebraic regression analysis can be carried out (using the `polyfit` function). The bottom portion of Fig. 11.12a shows the linear regression produced in the modified $(x^{-1} - y^{-1})$ domain. Regression coefficients of this linear approximation are then converted back to the original domain and used to initialize the optimization routine (using the `fminsearch` function). The top portion of Fig. 11.12a features both regressions. As seen, they are slightly different, with the one obtained via minimization exhibiting a lesser value of the sum of squared residuals. The individual residuals for both approaches are shown in Fig. 11.12b. As expected, because of inevitable scaling [Eq. (11.41)], the linear regression pays more attention to residuals with the smaller values of y, allowing residuals for the last data points to be larger.

11.6.2 Piecewise Polynomial Interpolations

Now, let us introduce three functions for approximating the `x−y` data points with piecewise cubic splines. The following script presents a self-explanatory example of using the `spline`, `pchip` and `makima` functions of Table 11.6, resulting in the graphical output shown in Fig. 11.13a

```
x=-3:3; y=[1 -1 -1 -2 3 1 1];
t=linspace(x(1),x(end));
s=spline(x,y,t);          % not-a-knot spline
h=pchip(x,y,t);           % Hermite spline
a=makima(x,y,t);          % Akima spline
plot(x,y,'bo'), hold
plot(t,s,'m-.',t,h,'b-',t,a,'k--')
legend('Data points','Not-a-knot spline',...
       'Hermite spline','Akima spline','location','nw')
xlabel('x'), ylabel('f(x)'), axis tight
```

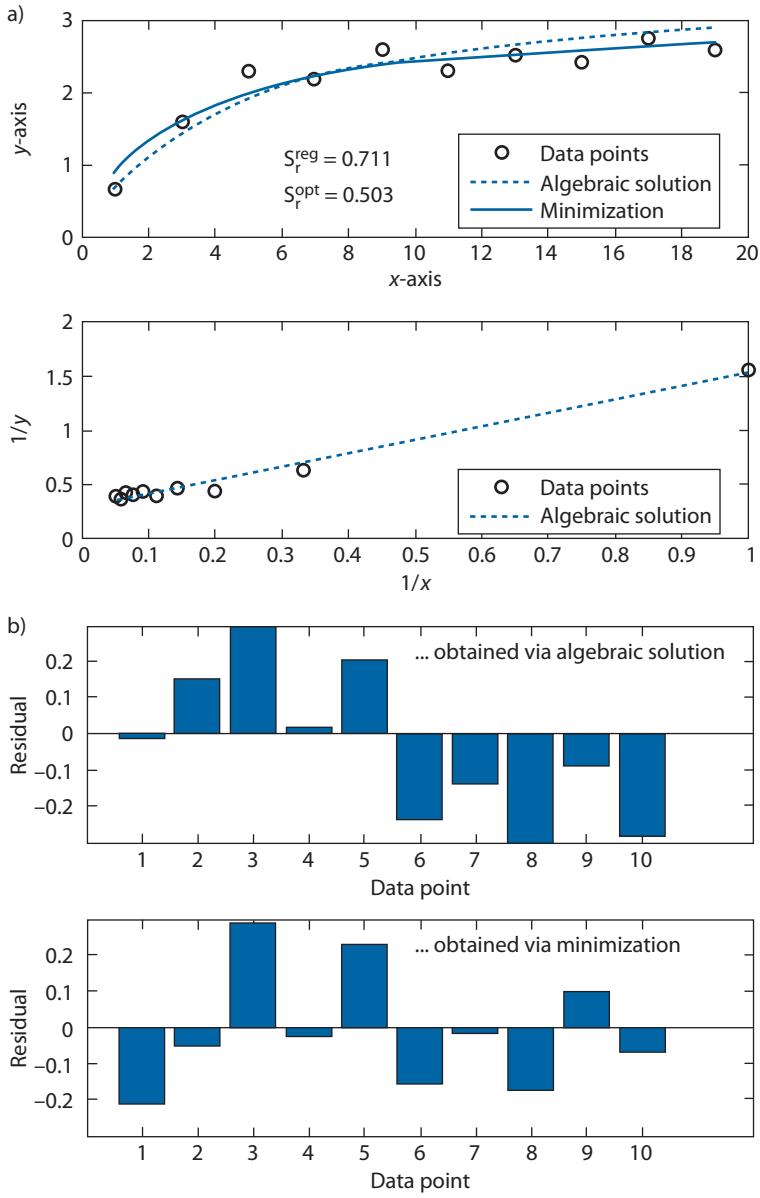


Fig. 11.12 Examples of a) nonlinear regression and b) individual residuals.

Figure 11.13b features another example where, for a specific data set, the not-a-knot, Hermite, and Akima spline interpolations are very close to each other and practically coincide with a full-degree polynomial interpolation (the key is in a special arrangement of x-data points). Of course, extrapolation varies greatly.

Table 11.6 Functions for Computing Piecewise Cubic Splines

Function	Brief Description
`y_int=spline(x,y,x_int)`	Computes a not-a-knot or clamped spline interpolation (between the data points defined in the `x` and `y` vectors) at the set of arguments `x_int`. If `x` and `y` have the same length, the output is a not-a-knot spline. If `y` has two more elements than `x`, the very first and very last elements of the vector `y` are considered to be the end slopes, and the output is a clamped spline
`y_int=pchip(x,y,x_int)`	Computes a shape-preserving Hermite interpolating polynomial (between the data points defined in the `x` and `y` vectors) at the set of arguments `x_int`
`y_int=makima(x,y,x_int)`	Computes a shape-preserving Akima's modification of the Hermite interpolating polynomial between the data points defined in the `x` and `y` vectors at the set of arguments `x_int`

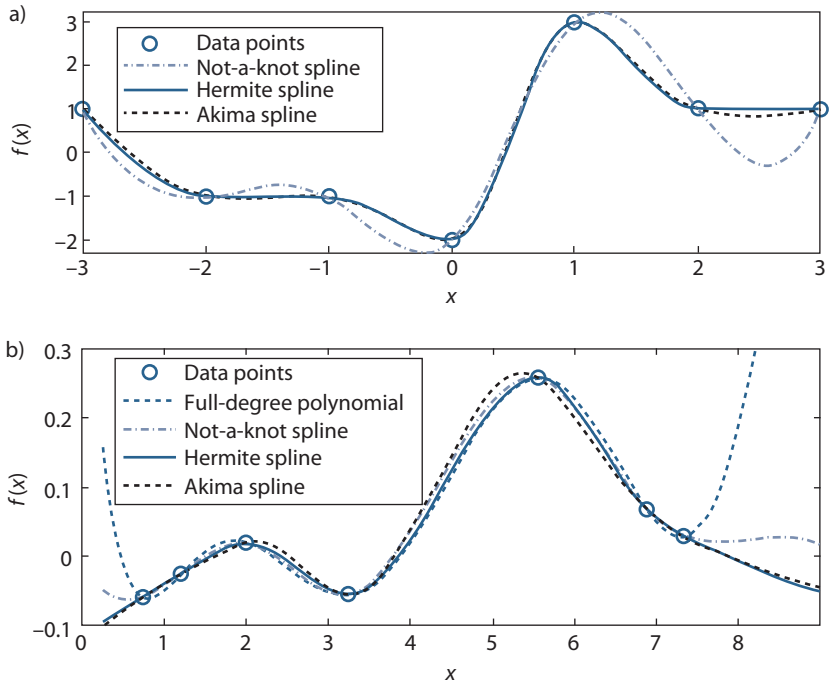


Fig. 11.13 a) Examples of different splines and b) comparison of different types of approximations.

The simpler syntax for the piecewise cubic spline functions is

`pp = spline(x,y)`, `pp = pchip(x,y)` and `pp = makima(x,y)`

These calls do not do interpolation for the set of x-data provided in the third input argument, as it was the case in the script that produced Fig. 11.12a. Instead, they create a structure that contains all information pertaining to piecewise approximations defined by the vectors `x` and `y`. This structure can be used later by the functions listed in Table 11.7.

For example, a self-explanatory set of commands

```
x=1:5; y=[2 5 6 3 4];
pp = spline(x,y),          [kn,cf]=unmkpp(pp)
xx=linspace(x(1),x(end)); yy=ppval(pp,xx);
plot(x,y,'ks',xx,yy,'b.'), xlabel('x'), ylabel('f(x)'), hold
plot(xx,spline(x,[0 y 0],xx),'k--'))
legend('Data points','Not-a-knot spline',...
       'Clamped spline with f''(x)=0 at endpoints','Location','best')
```

returns

```
pp =    form: 'pp'
      breaks: [1 2 3 4 5]
       coefs: [4×4 double]
      pieces: 4
       order: 4
         dim: 1
kn = 1      2      3      4      5
cf =   -0.7500    1.2500    2.5000    2.0000
       -0.7500   -1.0000    2.7500    5.0000
        1.7500   -3.2500   -1.5000    6.0000
        1.7500    2.0000   -2.7500    3.0000
```

and produces the plot shown in Fig. 11.14a. The `pp` structure returned by the `spline` call in the second line features six fields. The content of these fields can be accessed by using the dot indexing, like `pp.breaks` or `pp.coefs`. Alternatively, the very same data can be recovered by splitting the `pp` structure using the `unmkpp` function, returning the breaks in `kn` and the rows of

Table 11.7 Utility Functions for Piecewise Cubic Approximations

Function	Brief Description
`yy=ppval(pp,xx)`	Returns the y-values of the piecewise polynomial structure `pp` evaluated at the x-points in `xx`
`[kn,cf]=unmkpp(pp)`	Splits the piecewise polynomial structure `pp` into a set of knots in `kn`, and set of four polynomial coefficients per each of `length(pp)-1` subintervals in the rows of matrix `cf` (unmkpp is the abbreviation for unmake piecewise polynomial)
`pp=mkpp(kn,cf)`	Builds a piecewise polynomial structure `pp` from the knots in `kn` and rows of coefficients in matrix `cf` (mkpp is the abbreviation for make piecewise polynomial)

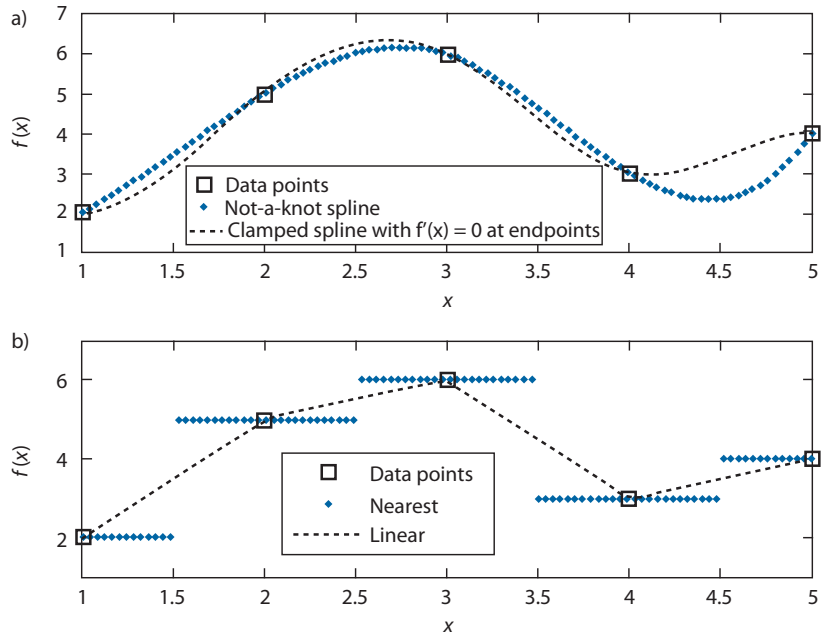


Fig. 11.14 Examples of a) spline produced using the `unmkpp` utility and b) approximations produced using the `interp1` function.

the third-order polynomial coefficients for each subinterval in `cf`. The `pp` structure has some extra fields defining the number of subintervals (`pieces`) and the number of coefficients per each subinterval (`order`). The last field (`dim`), defines the number of dimensions of a problem based on the size of array `y` in the `spline(x,y)` call (in the multidimensional case, interpolation is performed for each set of data in `y` against the same vector `x`).

Figure 11.14a also features an example of a clamped spline (produced by the fifth line of code), showing a capability to vary the endpoint slopes (in this particular case - both set to zero).

MATLAB features one more function that allows interpolating a function of one variable, `interp1`, as presented in Table 11.7, with Table 11.8 listing a variety of available interpolation methods. Note that by default, the `interp1` function does not do extrapolation outside the interval spanned by `x` for the `'linear'` and `'nearest'` methods returning `NaN`. If you need it, you may request it directly, for example,

```
yy=interp1(x,y,xx,'nearest','extrap')
```

When the Figure window **Tools** menu was introduced in Sec. 6.11 (see Fig. 6.55d), one of the options was **Basic Fitting**, which allows you to conveniently explore different interpolants. Suppose you created a plot using the call

```
>> xx=linspace(0,2*pi,17); plot(xx,sin(xx),'.'), axis tight
```

Table 11.8 `interp1 ... interpn` Functions

Function	Brief Description
`yy=interp1(x,y,xx,method);`	Interpolates a function of one variable $y = f(x)$, returning a vector `yy` at the specified values `xx` using data stored in `x` and `y` to produce a piecewise approximation. An optional argument `method` defines the interpolation method (see table 11.9)
`zz=interp2(x,y,w,xx,yy,... method);`	Interpolates a function of two variables, $w = f(x,y)$, returning a vector `zz` at the specified values `xx` and `yy` based on data stored in `x`, `y`, and `w`, with `method` specified in the same manner as for the `interp1` function
`ww=interp3(x,y,z,w,xx,... yy,zz,method);`	Interpolates a function of three variables $w = f(x,y,z)$, returning a vector `zz` at the specified values `xx`, `yy`, and `zz` based on data stored in `x`, `y`, `z`, and `w` and using interpolation method `method`
`vv=interpn(x1,x2,...,xn,w,... xx1,xx2,...,xxn,method);`	Interpolates a function of n variables based on data stored in the multidimensional lookup table $w = f(x_1, x_2, \ldots, x_n)$

Choosing the **Basic Fitting** option from the **Tools** menu brings up the Basic Fitting tool window shown in Fig. 11.15a (compare with that of Fig. 6.57d). Picking a couple of interpolants to compare and checking off several radio buttons results in the two-strip plot shown in Fig. 11.15b. Now that we have managed to create all these plots programmatically in the previous examples of this section, you should totally understand all the options for curve fitting that the Basic Fitting tool has to offer and be able to explain the results. Note that neither spline or shape-preserving spline interpolant (the latter option resides at the bottom of the Types of Fit list) generates an equation to be shown, as is the case for the first- through tenth-order interpolants.

The remaining functions of Table 11.8 allow you to perform multidimensional piecewise interpolation (note that in this case, you may only specify a scalar value that will be assigned to all queries that lie outside the domain of the sample points.) For example, the following set of commands interpolates the MATLAB `peaks` function over a finer grid and generates a plot shown in Fig. 11.16:

```
[x,y]=meshgrid(-3:0.25:3);
z=peaks(x,y);
[xi,yi]=meshgrid(-3:0.125:3);
zi=interp2(x,y,z,xi,yi);
mesh(x,y,z), hold, mesh(xi,yi,zi+15)
axis([-3 3 -3 3 -5 20])
xlabel('x'), ylabel('y'), zlabel('z=f(x,y)'), colormap jet
```

The 2-D interpolation is commonly used for coarse image refining. Consider the following script producing the two images shown in Fig. 11.17:

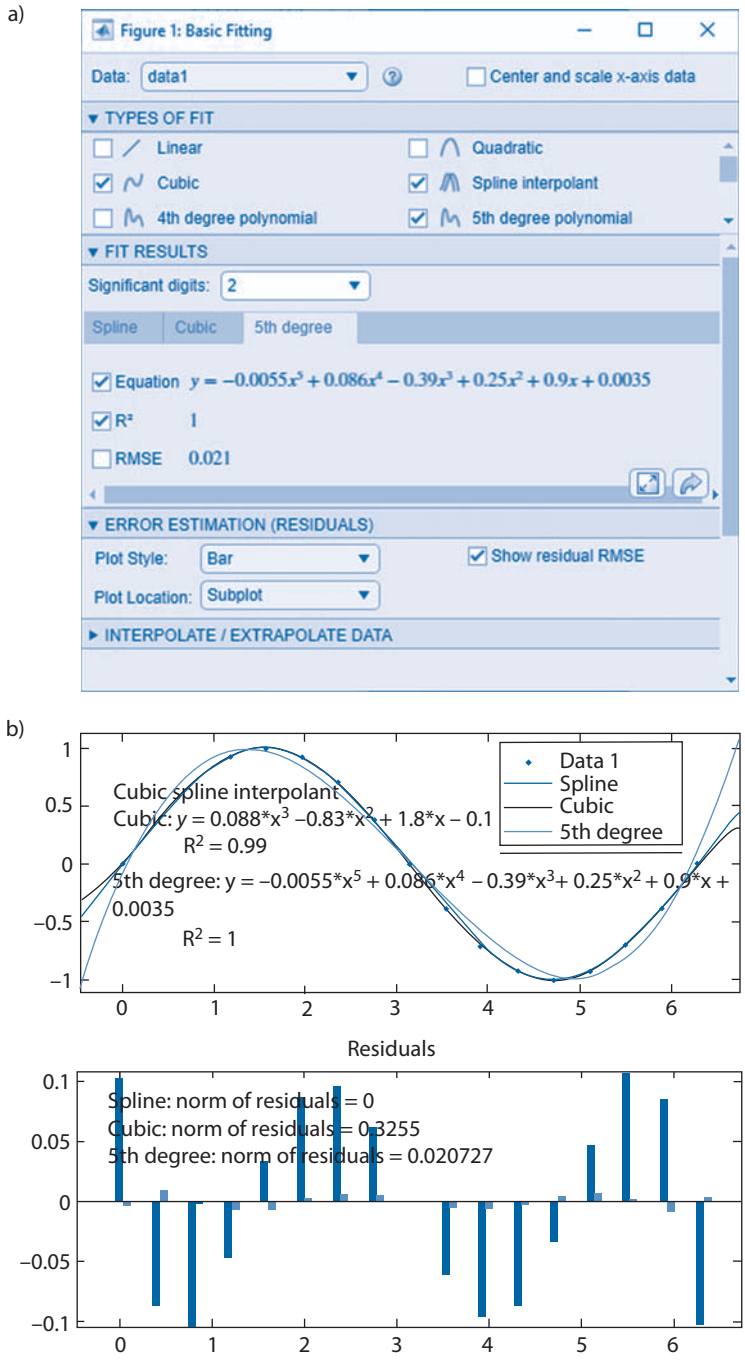


Fig. 11.15 a) Using the Basic Fitting Tool to b) fit a sinusoid with the spline and fourth-order polynomial.

```
load clown
% Cast a portion of the image to single
V = single(X(1:124,1:300));
subplot(211)
imagesc(V);
axis image, axis off, colormap gray
title('Original image');
subplot(212)
% Create a refined grid (dividing subintervals 2 times in each dimension)
Vq = interp2(V,2);
imagesc(Vq);
axis image, axis off, colormap gray
title('Linear interpolation')
```

Note the syntax of the `interp2` function, where the refinement factor $k = 2$ specifies the number of times to repeatedly divide subintervals of the refined grid in each dimension, which results in $2^k - 1$ interpolated points between sample values $(N - 1)(2^k - 1) + N = (N - 1)2^k + 1$ total points. In this particular case, the 124-by-300 pixel photo becomes a 493-by-1197 pixel photo.

The next script employs a 3-D interpolation to compute a take-off distance of a Cessna-172 aircraft to clear-off a 50-ft obstacle (Fig. 11.18), the task all pilots do to better understand operating limitations. This distance is computed based on the table data for a variety of ground winds, airport elevations, and aircraft weights:

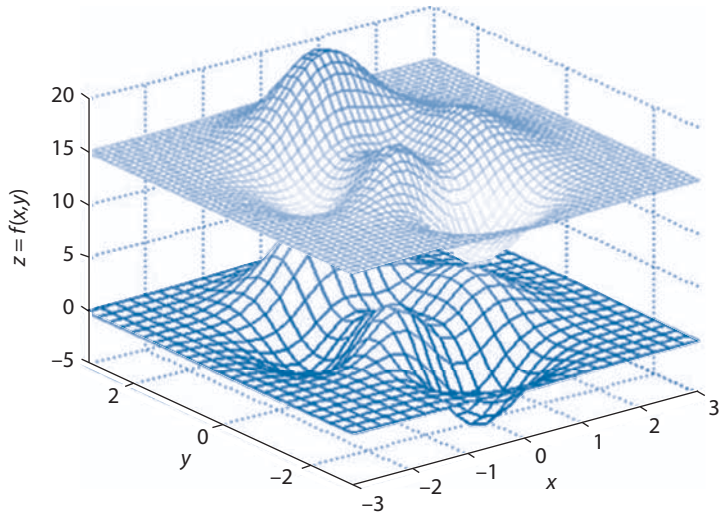


Fig. 11.16 Example of 2-D interpolation featuring a finer mesh grid.

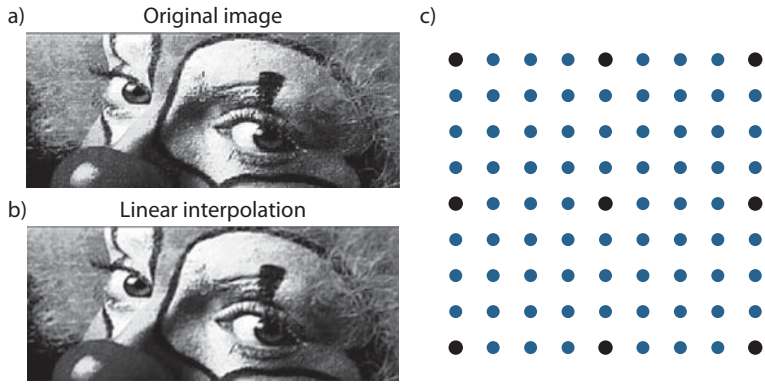


Fig. 11.17 a) Original and b) refined images; c) a fragment of a refined grid.

```
wind=[0 10 20];
elevation=[0 2500 5000 7500];
weight=[1700 2000 2300];
tkof(:,:,1)=[780 920 1095 1370; 570 680 820 1040; 385 470 575 745];
tkof(:,:,2)=[1095 1325 1625 2155; 820 1007 1250 1685; 580 720 910 1255];
tkof(:,:,3)=[1525 1910 2480 3855; 1170 1485 1955 3110; 850 1100 1480 2425];
We=2100; El=254; Wi=0:20;
w=interp3(elevation,wind,weight,tkof,El,Wi,We,'spline');
plot(Wi,w), grid
text(1,900,'Aircraft: Cessna-172 Skyhawk','BackgroundColor','w')
text(1,800,[' Gross weight: ' num2str(We) ' lb'],'BackgroundColor','w')
text(1,700,['Elevation: ' num2str(El),' ft MSL'],'BackgroundColor','w')
xlabel('Head wind, kt'), ylabel('Take-off distance, ft')
title('Take-off distance to clear-off a 50-ft obstacle')
```

The following script features another example of interpolation for 3-D fluid flow data as provided by the MATLAB function flow:

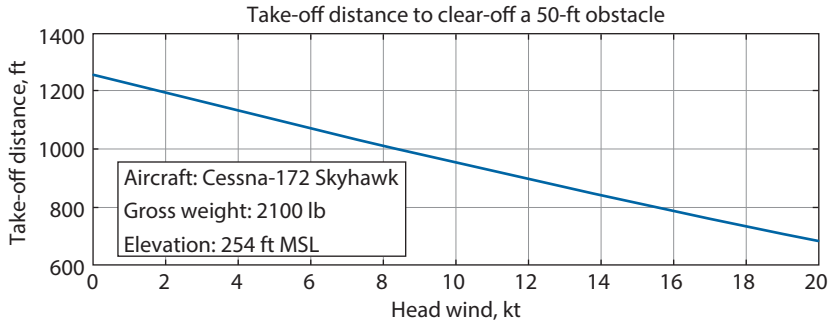


Fig. 11.18 Example of a 3-D interpolation using the lookup tables.

```
[x,y,z,v]=flow(10);              % 10-by-20-by-10 arrays
[xi,yi,zi]=meshgrid(.1:.25:10,-3:.25:3,-3:.25:3);
vi=interp3(x,y,z,v,xi,yi,zi); % 25-by-40-by-25 arrays
xsl=[2.5 6 9.5]; ysl=2; zsl=[-2.5 0];
slice(x,y,z,v,xsl,ysl,zsl), slice(xi,yi,zi,vi,xsl,ysl,zsl)
shading interp, colormap jet
```

In this script, the `[x,y,z,v]=flow(n)` call returns the `x`, `y`, and `z` coordinates as well as volume `v` of the fluid flow in `n`-by-`2*n`-by-`n` arrays. Then, the `interp3` function is employed to interpolate over a finer mesh. The `slice` function is used to visualize both the coarse (Fig. 11.19a) and refined (Fig. 11.19b) flows. For better visual effect, the `shading` function varies the color in each face by interpolating the colormap index value across the face (Fig. 11.19c), and the `colormap` call changes the default colormap setting (`parula`) to the brightest one (Fig. 11.19d).

Note that, to produce a rectangular grid in 2-D and 3-D space in the preceding examples, we used the `meshgrid` function. This function, however, is limited to only 2-D and 3-D grids, so that if you intend to use the `interpn` function, you would need to switch to the `ndgrid` function, which supports 1-D to *n*-D grids. The only caveat is that although the coordinates produced by each function (`meshgrid` and `ndgrid`) are the same, the shape of the output arrays in the first two dimensions is different. For example, if the grid vectors `x1` and `x2` are of lengths *m* and *n*, respectively,

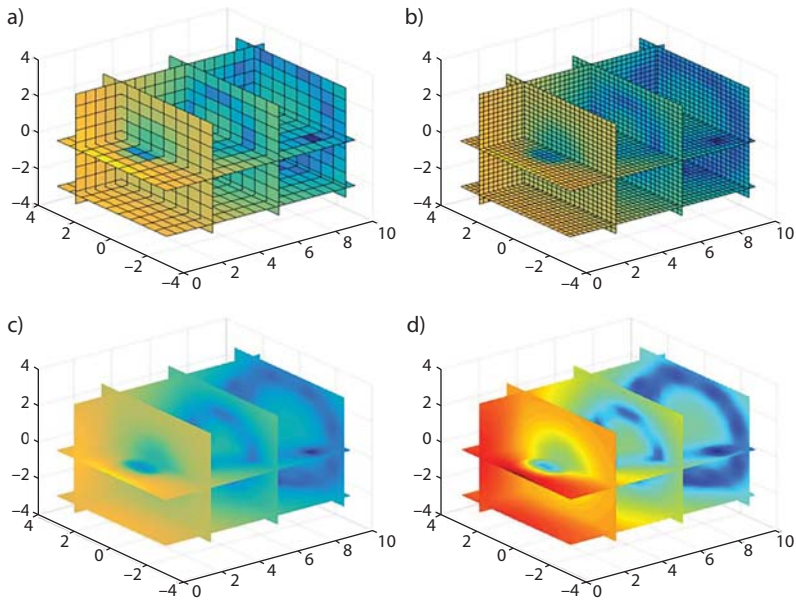


Fig. 11.19 Example of 3-D interpolation converting a) coarse mesh data to b) finer mesh data, followed by color shading (c) and choosing a brighter color map (d).

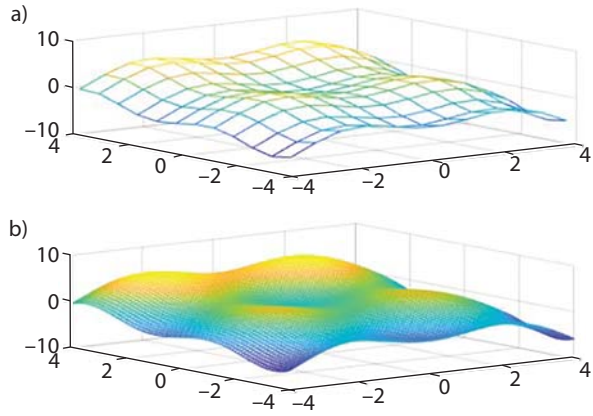


Fig. 11.20 a) Coarse and b) finer-grid surfaces.

`ndgrid(x1,x2)` outputs an *m*-by-*n* array, whereas `meshgrid(x1,x2)`-*n*-by-*m*.

Beyond `interp1`, . . . , `intern`, MATLAB features another function that allows creating and manipulating with the interpolant objects (or, in other words, it is an interpolating function that you can evaluate at query points). The `griddedInterpolant`, introduced in R2011b, allows you to perform interpolation on a 1-D, 2-D, 3-D, or *n*-D gridded data set. For example, in the 2-D case, `F=griddedInterpolant (X,Y,Z)` returns the interpolant `F` for the given data set in the full grid form (`X`, `Y`, `Z` are matrices). Then, you can evaluate `F` at a new set of query points in the full grid form, for example (`xq`, `yq`), to produce the interpolated `z` values—`z=F(xq,yq)`. This approach is more efficient in cases where you need to evaluate a `griddedInterpolant` object `F` at many different sets of query points (compared with using `interp1`, . . . , `interpn` several times).

Consider the following script that takes advantage of using the interpolant object:

```
[X,Y]=ndgrid(-4:0.7:4);   Z=X.*sin(X)-Y.*cos(Y);
subplot(211), mesh(X,Y,Z)
F=griddedInterpolant(X,Y,Z,'spline'); % define F using the full grid form
[Xq,Yq]=ndgrid(-4:0.1:4); Vq=F(Xq,Yq);
subplot(212), mesh(Xq,Yq,Vq)
```

(note how we used the `ndgrid` function, but in this particular case, we could have used `meshgrid` as well). The original and finer-grid surfaces are shown in Figs. 11.20a and 11.20b, respectively. Of course, the same result could have been achieved using the grid vectors (as opposed to full grids)

```
x=-4:0.7:4; y=x'; Z=x.*sin(x)-y.*cos(y);
F=griddedInterpolant({x,y},Z,'spline'); % define F using the grid vectors
xq=-4:0.1:4; yq=xq'; Vq=F({xq,yq}); mesh(xq,yq,Vq)
```

and the `interp2` function

```
[X,Y]=meshgrid(-4:0.7:4); Z=X.*sin(X)-Y.*cos(Y);
[Xq,Yq]=meshgrid(-4:0.1:4); Vq=interp2(X,Y,Z,Xq,Yq,'spline')
mesh(Xq,Yq,Vq)
```

So far, we have always used gridded data, that is, a set of points that are axis-aligned and ordered. What if you have to deal with scattered data (a set of points that have no structure among their relative locations)? In this case, MATLAB offers two functions, `griddata` (to interpolate 2-D or 3-D scattered data) and `griddatan` (to interpolate *n*-D scattered data). The following script provides an example of using these functions:

```
x=2*rand(1000,1)-1; % scattered x data points on an interval [-1;1]
y=2*rand(1000,1)-1; % scattered y data points on an interval [-1;1]
v=x.^3+y.^3;        % compute the z values at scattered (x,y) pairs
     d=linspace(-1,1,40);           % define a grid vector
     [xq,yq]=meshgrid(d);           % create a full grid
vq=griddata(x,y,v,xq,yq,'nearest'); % interpolate scattered data on the grid
plot3(x,y,v,'r.'), hold, grid, surfc(xq,yq,vq), colormap bone
xlabel('x'), ylabel('y'), zlabel('v')
```

Figure 11.21a shows the result. Other interpolation methods supported by `griddata` and `griddatan` are shown in Table 11.10 (compare to

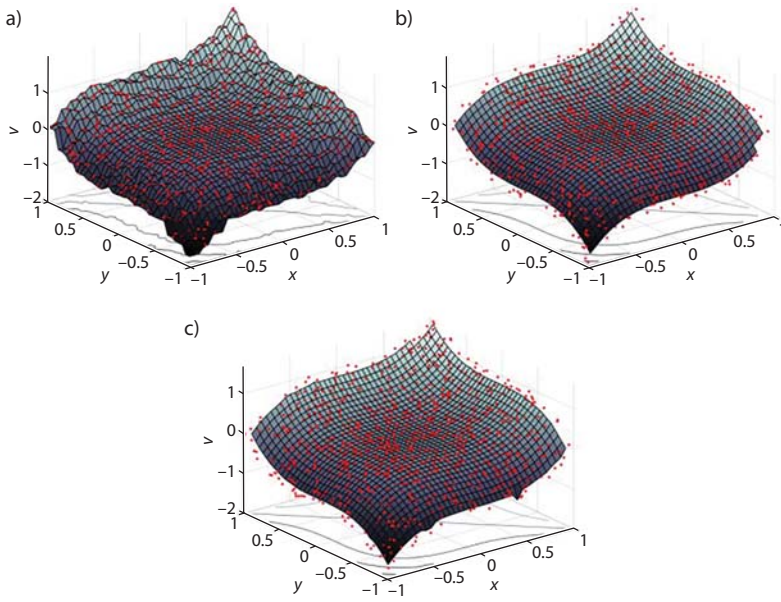


Fig. 11.21 Interpolating scattered data using the a) nearest point, b) linear, and c) cubic interpolation.

Table 11.9 Interpolation Methods

method	Description
'linear'	Produces a piecewise linear interpolation (the default method)
'nearest'	Produces a ladder-type nearest-neighbor interpolation (see Fig. 11.14b)
'next'	Creates a ladder-type next-neighbor interpolation[a]
'previous'	Generates a ladder-type previous-neighbor interpolation[a]
'spline'	Yields a cubic spline interpolation with not-a-knot end conditions
'pchip'	Employs a shape-preserving cubic Hermite interpolation[a]
'makima'	Uses a modified Akima cubic Hermite interpolation (does not flatten as aggressively as 'pchip')
'cubic'	Returns an interpolation based on a cubic convolution (applicable for uniform spacing in each dimension)

[a]Applicable for interp1 (1-D) only.

Table 11.9). Figures 11.21b and 11.21c present the use of linear and cubic interpolation for the same data set.

In addition to griddata and griddatan, R2013a introduced the scatteredInterpolant function for performing interpolation on a 2-D or 3-D scattered data set. This function can be used in a manner similar to griddedInterpolant. For example, the script

```
% Create interpolant from scattered (nongridded) data points
Fs=scatteredInterpolant(x,y,V);
% Evaluate the interpolant at query locations
[xr,yr]=meshgrid(linspace(min(x),max(x),60),...
                 linspace(min(y),max(y),60));
vr=Fs(xr,yr); surf(xr,yr,vr), hold
plot3(x,y,V,'bo','MarkerFaceColor','b')
xlabel('x, in'), ylabel('y, in'), zlabel('\DeltaC_p')
colormap(jet), colorbar, view([140,30]), axis tight
```

returns the plot shown in Fig. 11.22. Of course, the same result could have been achieved using the griddata function.

Table 11.10 Interpolation Methods to Create a Grid from the Scattered Data Points

Method	2-D	3-D	*n*-D	Description
'linear'	+	+	+	Triangulation-based linear interpolation (the default method)
'nearest'	+	+	+	Triangulation-based nearest-neighbor interpolation
'natural'	+	+		Triangulation-based natural-neighbor interpolation (this method is an efficient tradeoff between linear and cubic)
'cubic'	+			Triangulation-based cubic interpolation
'v4'	+			Biharmonic spline interpolation (unlike the other methods, this interpolation is not based on a triangulation)

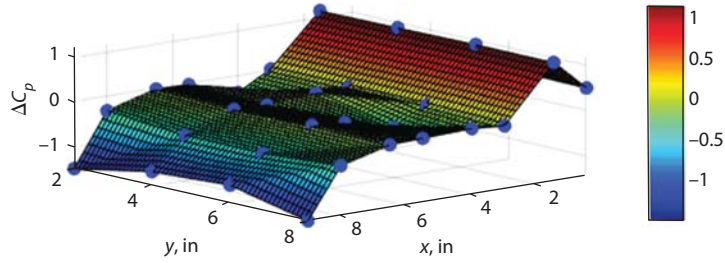


Fig. 11.22 Example of using a scattered interpolant.

11.6.3 Trigonometric Series Approximations

Let us show an example of least-squares regression applied to a sinusoid as discussed in Sec. 11.5. The following script generates the data points based on a sinusoid corrupted with some noise (Fig. 11.23a) and then applies Eqs. (11.100) and (11.106) to fit these data with a least-squares regression:

```
%% Generate data points
a0=1; a1=0.5; b1=3; w=1; N=100;
td=linspace(0,4*pi,N);
yd=a0+a1*cos(w*td)+b1*sin(w*td)+0.3*randn(1,N);
subplot(2,1,1)
plot(td,yd,'r.'), hold
%% Perform a least-squares regression
a0est=sum(yd)/N;
```

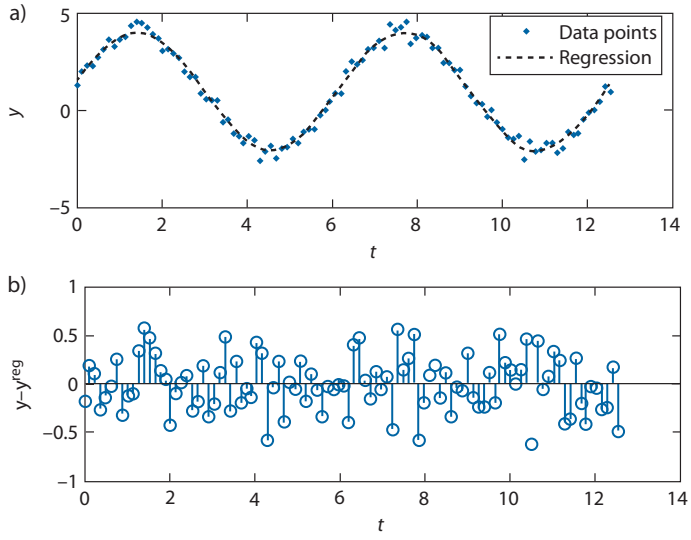


Fig. 11.23 a) Least-squares regression for the noise-corrupted sinusoid and b) corresponding individual residuals.

```
a1est=2*sum(yd.*cos(w*td))/N;
b1est=2*sum(yd.*sin(w*td))/N;
t=linspace(0,4*pi,2*N);
y=a0est+a1est*cos(w*t)+b1est*sin(w*t);
plot(t,y,'--'), xlabel('t'), ylabel('y')
legend('Data points','Regression')
%% Show individual residuals
subplot(2,1,2)
stem(td,yd-a0est-a1est*cos(w*td)-b1est*sin(w*td))
xlabel('t'), ylabel('y-y^{reg}')
```

Figure 11.23b shows individual residuals.

In the example of Fig. 11.23a, we knew what frequency to use for a regression, but in the general case, this information is not available, and that is when Eq. (11.107) becomes useful. Before we show a practical example, consider a general case where you would like to use CTFS [Eq. (11.109)] to approximate a rectangular wave function within one period, that is, within $t \in [0;T]$

$$f(t) = \begin{cases} -1 & 0 < t \le \frac{1}{4}T \\ 1 & \frac{1}{4}T < t \le \frac{3}{4}T \\ -1 & \frac{3}{4}T < t \le T \end{cases} \tag{11.118}$$

Applying Eq. (11.110) for a_0 yields

$$a_0 = \frac{1}{T}\int_0^T f(t)\mathrm{d}t = \frac{1}{T}\left(-\int_0^{T/4} \mathrm{d}t + \int_{T/4}^{3T/4} \mathrm{d}t - \int_{3T/4}^{T} \mathrm{d}t \right) = 0 \tag{11.119}$$

Similarly, you can compute

$$b_j = 0, \quad j = 1, 2, ..., \infty \quad \text{and} \quad a_j = \begin{cases} -4/(j\pi) & \text{for } j = 1, 5, 9, ... \\ 4/(j\pi) & \text{for } j = 3, 7, 11, ... \\ 0 & \text{for } j = 2, 4, 6, 8, ... \end{cases} \tag{11.120}$$

The following script proves these findings via employing the Symbolic Math Toolbox function `int`:

```
syms T t
   for j=0:7
   f1=cos(j*2*pi/T*t); f2=sin(j*2*pi/T*t);
   a(j+1)=2*(-int(f1,-T/2,-T/4)+int(f1,-T/4,T/4)-int(f1,T/4,T/2))/T;
   b(j+1)=2*(-int(f2,-T/2,-T/4)+int(f2,-T/4,T/4)-int(f2,T/4,T/2))/T;
   end
[0:7; a; b]
```

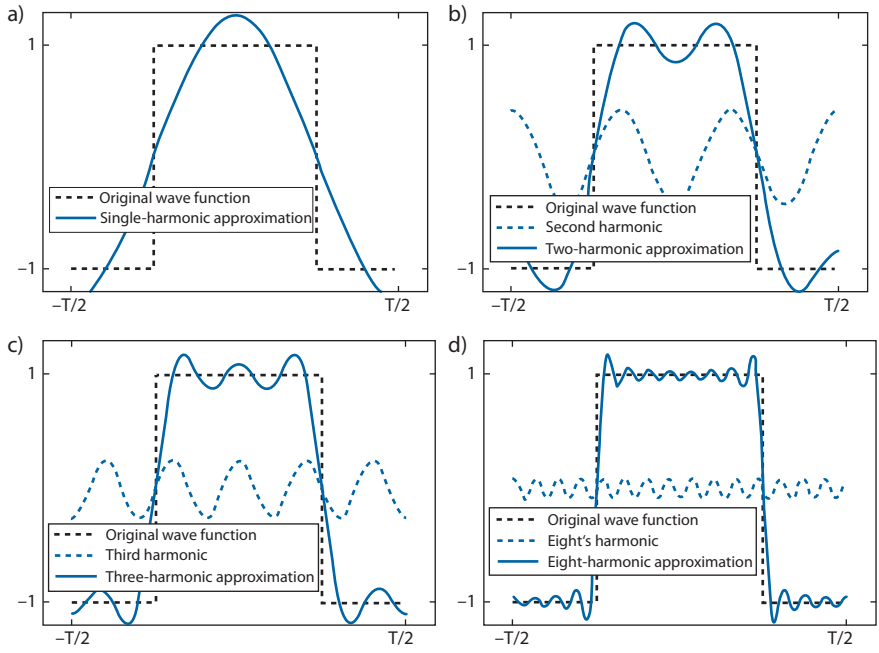


Fig. 11.24 First four terms of the Fourier series to approximate the square of a rectangular wave.

returning

```
ans = [ 0,      1,   2,           3,   4,         5,   6,          7]
      [ 0, -4/pi,   0,   4/(3*pi),   0, -4/(5*pi),   0,  4/(7*pi)]
      [ 0,      0,   0,           0,   0,         0,   0,          0]
```

Therefore, the Fourier series for Eq. (11.118) is

$$f(t) = -\frac{4}{\pi}\cos(\omega_0 t) + \frac{4}{3\pi}\cos(3\omega_0 t) - \frac{4}{5\pi}\cos(5\omega_0 t) + \frac{4}{7\pi}\cos(7\omega_0 t) - \cdots \tag{11.121}$$

where $\omega_0 = 2\pi T^{-1}$. Figure 11.24 shows the results of adding up to eight terms, whereas Fig. 11.25 features an alternative way of presenting information provided by Eq. (11.121) via the *amplitude spectrum*. The following script shows how to employ the `fft` function to produce the same amplitude spectrum as shown numerically in Fig. 11.25a:

```
%% Generate the data points
T=2*pi; N=1000; quat=N/4;
t=linspace(0,T,N);
y=-ones(1,N); y(quat:3*quat)=1;
%% Apply the fft function
d=fft(y);                          % perform discrete Fourier transform
a=2*real(d)/N; b=-2*imag(d)/N;     % evaluate coefficients a and b
```

```
c=2*sqrt(d.*conj(d))/N;          % compute the magnitude of coefficient c
%% Plot amplitude spectrum
subplot(2,1,1)
f=2*pi/T*[0:8]; stem(f,c(1:length(f)))
text(f(2:2:8)'+0.2,c(2:2:8)',num2str(c(2:2:8)','%-5.3f'))
xlabel('Harmonic'), ylabel('Amplitude')
%% Plot the DFT interpolant
subplot(2,1,2)
NUP=ceil((N+1)/2); yf=a(1)/2;
for i=1:NUP
   yf=yf+a(i+1)*cos(2*pi/T*i*t)+b(i+1)*sin(2*pi/T*i*t);
end
plot(t,yf), xlabel('Time, s'), ylabel('Function')
```

Specifically, the command `d=fft(y)` returns complex quantities of DFT of vector `y` (sums in Eq. (11.112)). The amplitude of DFT multiplied by $2N^{-1}$ yields coefficients c_j [Eq. (11.112)]. Figure 11.25b shows the absolute values of these coefficients matching those of Fig. 11.25a. Note that we could also

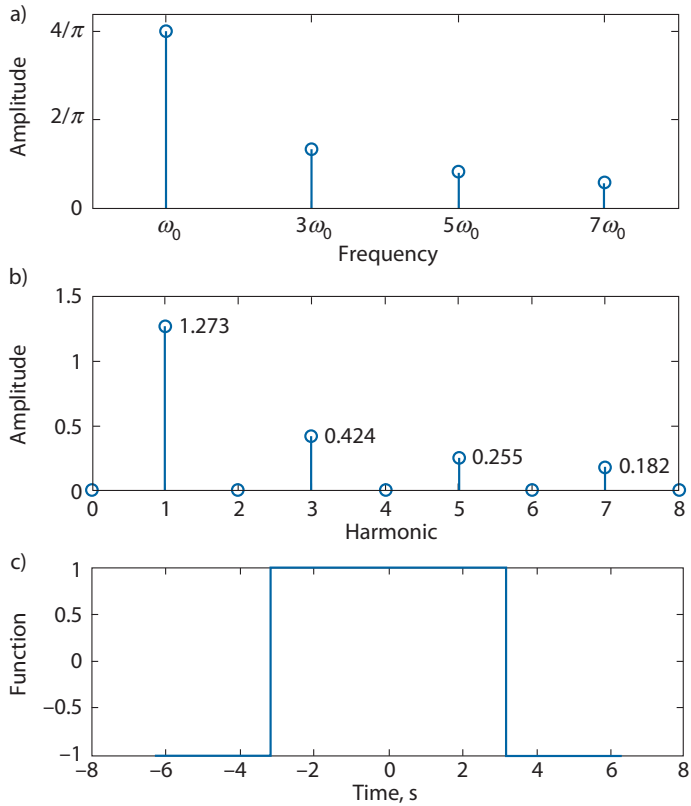


Fig. 11.25 a, b) Amplitude spectrum of the Fourier series (for the first four terms) and c) the resulting DFT approximation of a rectangular wave function.

plot the values of coefficients a_j explicitly, because the values of b_j are negligibly small (because of the round-off errors, they are not exactly zeros, though). The `fft` call returns the same amount of data as vector `y` has, but in fact only half of these data happen to be unique; the second half is just a reflection of the first half. Hence, the line `NUP=ceil((N+1)/2)` evaluates the number of unique frequencies, followed by computation of the DFT interpolant, shown in Fig. 11.25c. This interpolant passes through all $N = 1000$ data points.

Let us consider another example where, similar to Fig. 11.23a, we want to find a regression to better fit some experimental data. As opposed to the previous formulation, however, we do not know any frequency-related information up front. On the contrary, it is required to find the fundamental frequency of the signal, ω_0, and major harmonics.

MATLAB provides several examples showing how to generate data formed by two sinusoids corrupted with some noise and then apply the `fft` function to extract two major harmonics. The following script goes a little bit farther and allows you to change data and also reconstruct the original signal by automatically finding two harmonics on the power spectrum plot:

```
%% Define the signal
Harm1=50;                    % the first harmonic (Hz)
Harm2=220;                   % the second harmonic (Hz)
AmpRat=2;                        % ratio of harmonics amplitudes
NoiseLevel=2.5;
%% Create a noisy signal from two harmonics
t=0:0.001:0.6;                   % time stamps sampled at 1000Hz
ti=0:0.0005:0.6;
x=sin(2*pi*Harm1*t)+AmpRat*sin(2*pi*Harm2*t); % blend 2 sinusoids
y=x+NoiseLevel*randn(size(t)); % add signal noise
subplot(211)                     % plot original and corrupted signals
plot(1000*ti(1:100),interp1(t,x,ti(1:100),'spline'),'--g'), hold
plot(1000*t(1:50),y(1:50),'.b'), xlabel('Time, ms'), ylim([-5 15])
%% Extract signal power spectrum using FFT
Y = fft(y,512);            % the 512-point fast Fourier transform
Pyy = 2*Y.* conj(Y) / 512; % compute the power at various frequencies
f = 1000*(0:256)/512;      % define the frequency range (half of DFT return)
subplot(212)
plot(f,Pyy(1:257)), hold % plot power spectrum
xlabel('Frequency, Hz')
%% Reconstruct the signal from two major harmonics
[peak1,ifreq1]=max(Pyy(1:257)); % find the first harmonic
Pyy(ifreq1-10:ifreq1+10)=0;      % exclude the first harmonic
[peak2,ifreq2]=max (Pyy(1:257));% find the second harmonic
freq1=(ifreq1-1)*1000/512; freq2=(ifreq2-1)*1000/512;
if freq1 > freq2                 % rearrange harmonics in the ascending order
```

```
fr=freq1; freq1=freq2; freq2=fr;
pe=peak1; peak1=peak2; peak2=pe;
end
plot([freq1 freq1],[0 peak1],'--r')        % add both harmonics
plot([freq2 freq2],[0 peak2],'--m')        % to the power spectrum plot
legend('Data power spectrum','1^{st} harmonic','2^{nd} harmonic')
z = sin(2*pi*freq1*ti)+...
  sqrt(peak2/peak1)*sin(2*pi*freq2*ti);  % compute reconstructed signal
subplot(211)
plot(1000*ti(1:100),z(1:100),'--r')       % plot reconstructed signal
legend('Original uncorrupted signal',...
       'Corrupted signal (data points)','DFT approximation')
```

The consequent results of this self-explanatory script accompanied by the extended comments are shown in Fig. 11.26 (eventually, this code produces Fig. 11.26c as subplot 1 and 11.26b as subplot 2).

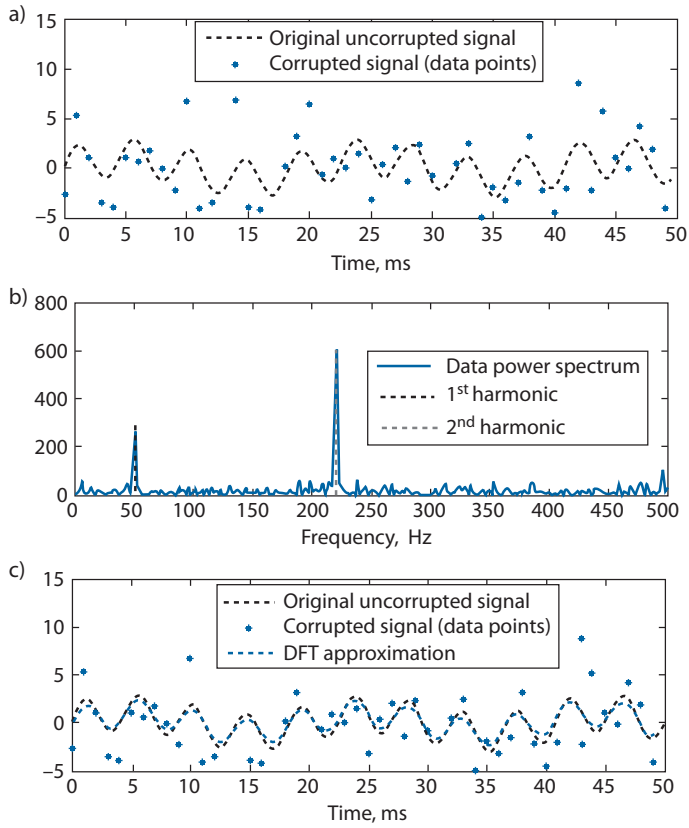


Fig. 11.26 Example of a) a noisy signal, b) frequency content of this signal, and c) DFT reconstruction using two harmonics.

DFT can also be used to interpolate 1-D data for a periodic function. Specifically, the syntax

```
Y=interpft(X,M)
```

returns the vector `Y` of length `M` that contains the value of the periodic function `X` (sampled at n equally spaced points) resampled to `M` equally spaced points ($n \leq$ `M`). The `interpft` function does this by first taking the standard n-point DFT of `X` using the `fft` function, then constructing the `M`-point DFT assuming that vector `Y` has the same period as `X`, and finally applying the `M`-point inverse FFT (using the `ifft` function). If n is an integer multiple of `M`, then the `Y` vector passes through the n original points defined by the `X` vector.

The following script produces interpolations for the sinusoidal and triangle-like signals by specifying the number of subintervals the original intervals have to be broken into:

```
%% Interpolation for a sinusoidal signal
subplot(2,1,1)
fplot(@sin,[0,2*pi],'LineStyle','-.'), hold
  x=linspace(0,2*pi,10);    % generate x for sample points
  y=sin(x);                 % generate y for sample points
N=length(x);    % number of data points
L=3;            % resample factor (dividing each subinterval by L)
M=N*L;          % number of interpolation points for interpft
xi=0:M-1; xi=max(x)*xi/(M-L);
yi=interpft(y,M);
plot(x,y,'gd','MarkerFaceColor','g'), plot(xi,yi,'m+')
xlim([0,max(x)]), xlabel('x-axis'), ylabel('y-axis')
legend('Original function','Data points','Interpolated data')
text(0.1,-0.5,['Number of data points ' int2str(N)])
text(0.1,-0.76,['Number of subintervals ' int2str(L)])
%% Interpolation for a triangle-like signal
clear all
subplot(2,1,2)
  x=linspace(0,7,9);     % generate x for sample points
  y=[0 1 2 3 4 3 2 1 0];     % generate y for sample points
N=length(x);    % number of data points
L=5;            % resample factor (dividing each subinterval by L)
M=N*L;          % number of interpolation points for interpft
xi=0:M-1; xi=max(x)*xi/(M-L);
yi=interpft(y,M);
plot(x,y,'gd','MarkerFaceColor','g'), hold, plot(x,y,'-.',xi,yi,'m+')
legend('Data points','Original function','Interpolated data',...
       'location','S')
xlim([0,max(x)]), xlabel('x-axis'), ylabel('y-axis')
text(0.1,3.6,['Number of data points ' int2str(N)])
text(0.1,3.1,['Number of subintervals ' int2str(L)])
```

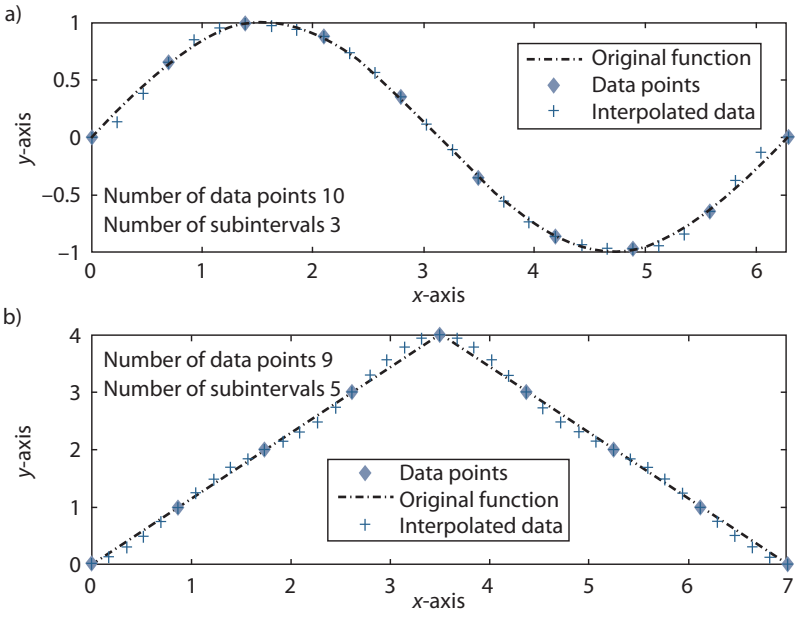


Fig. 11.27 Interpolation of a) sinusoidal and b) triangle-like signals using DFT.

These interpolations are shown in Figs. 11.27a and 11.27b, respectively. Note that the `interpft` function generates the values for `M=N*L` data points, which includes `L-1` points after the last original point. That is why we limited the *x*-axis to the original range (`xlim([0,max(x)])`), so these extra points do not show up.

11.6.4 Data Preconditioning

Let us conclude this chapter by introducing a few functions that might be very useful when you work with large arrays of data that may be corrupt. (This is what the Data Preprocessing **Task** tools of Live Editor shown in Fig. 2.13a can help you with. Also, see more details as applied to timetables in Appendix C.)

Say you are getting data from some noisy sensors (or downloading data from the Internet) and then importing it into the workspace using the Import Data tool of MATLAB. While importing, MATLAB recognizes the unimportable cells and replaces them with `NaN` (see Secs. 4.8.1–4.8.2). Before producing any fit, you might want to eliminate these data points, replace them with some specific value, or simply do a local approximation. The `fillmissing` function, introduced in R2016b, does exactly that (Table 11.11).

As an example, consider fixing the three `NaN` values in the row vector using one of syntaxes of Table 11.11:

```
A = [0.84 NaN -0.16 -0.97 NaN 0.4 0.1 NaN -0.7];
subplot(211), plot(A,'o-.r'), hold
xlabel('Data point'), ylabel('Value')
ind=find(TF1);                    % show locations of the missing points
  for i=1:length(ind), plot(ind(i)*[1 1],ylim(gca),'k--'), end
A1=fillmissing(A,'constant',0);  % fill with 0
subplot(212), h1=plot(A1,'^-.'); hold
xlabel( 'Datapoint'), ylabel('Value')
A2=fillmissing(A,'nearest');      % fill with the nearest nonmissing value
A3=fillmissing(A,'movmean',3);    % fill using a moving window mean of 3pts
h2=plot(A2,'s-.'); h3=plot(A3,'d--');
  for i=1:length(ind), plot(ind(i)*[1 1],ylim(gca),'k--'), end
legend([h1,h2,h3],'Filled with 0','Nearest','Movmean-3','location','n')
```

Figure 11.28 visualizes the results, showing the location of the missing points with the vertical dashed lines. While these three points were missing in the original data (Fig. 11.28a), they were filled with some automatically generated values in Fig. 11.28b.

If you are only interested in finding the missing points, you may use the `ismissing` function. For example, the call

```
>> TF1 = ismissing(A)            % the same as TF1=isnan(A)
```

applied to the vector `A` of the previous example returns the indices of the three missing points (in the 1-by-9 logical array)

```
TF1 =   0   1   0   0   1   0   0   1   0
```

and the call

```
>> TF2 = ismissing(A,-0.97)      % the same as TF2= A == -0.97
```

results in finding the indices of the data points that have a certain value (−0.97) (as seen, it works even with the `fillmissing(A,v)` syntax):

```
TF2 = 0    0    0   1    0    0    0    0    0
```

Table 11.11 Most Common Syntaxes of the `fillmissing` Function

Syntax	Brief Description
`fillmissing...(A, 'constant',v)`	Fills missing entries of an array or table `A` with the constant value `v`
`fillmissing(A,method)`	Fills missing entries using one of the methods discussed in conjunction with the `interp1,...,interpn` family of functions in Table 11.9 (i.e., `method` assumes `'linear'`, `'nearest'`, `'next'`, `'previous'`, `'spline'`, or `'pchip'`)
`fillmissing...(A, movmethod,window)`	Fills missing entries using a moving window mean (when `movmethod` is `'movmean'`) or median (when `movmethod` is `'movmedian'`) with a window length `window`

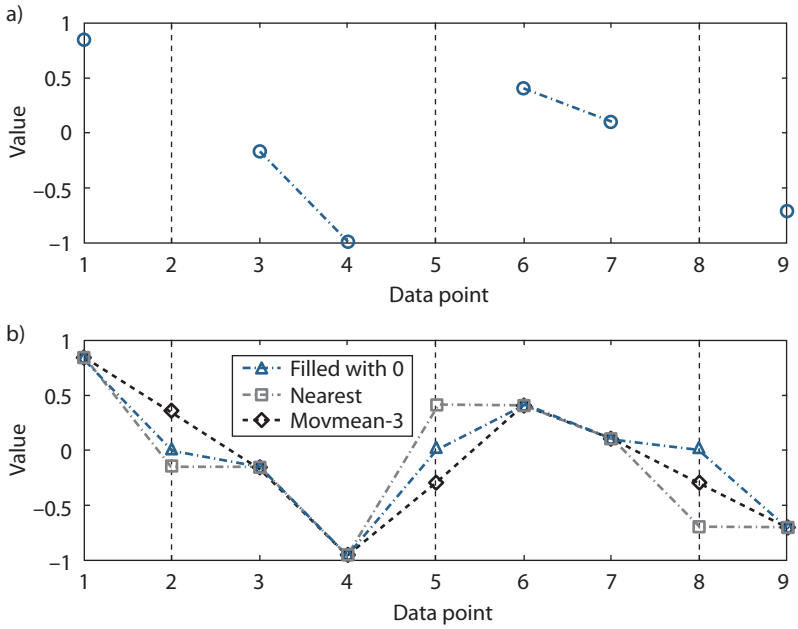


Fig. 11.28 a) Original data set and b) filling missing data points.

For removing the missing points there is the `rmmissing` function, so that the call `rmmissing(A)` would remove all missing points from `A`.

Along with `fillmissing`, there is another useful function, `filloutliers`, introduced in R2017a. Two syntaxes of this function are provided in Table 11.12, whereas Tables 11.13 and 11.14 list available methods to find and replace outliers.

Be aware that in some cases removing outliers is a relatively simple task not requiring any supervision. However, trying to improve data in the not-so-obvious situations may lead to an error. Consider an example featuring two outliers (shown in bold):

```
Theta=[0 100 19 31 42 48 55 62 69 72 76 0 85 87 88 89 90 90 90 90 90 90 90 90];
N=length(Theta);
subplot(211)
B=filloutliers(Theta,'linear','median');
plot(1:N,Theta,'*r--',1:N,B,'o-.')
legend('Original Data','Interpolated Data','location','se')
xlabel('Data point'), ylabel('Value')
subplot(212)
B=filloutliers(Theta,'spline','movmedian',5);
plot(1:N,Theta,'*r--',1:N,B,'o-.')
legend('Original Data','Interpolated Data','location','se')
```

Table 11.12 Two Common Syntaxes of the `filloutliers` Function

Syntax	Brief Description
`filloutliers(A,fillmethod,... movemethod,window)`	Specifies a moving method for determining local outliers (`movmethod` is either `'movmean'` or `'movmedian'`) according to a window length defined by `window` and replaces them according to `fillmethod` (see Table 11.13)
`filloutliers...(A,fillmethod, [findmethod])`	Finds outliers in `A` according to (optional) `findmethod` (see Table 11.14) and replaces them according to one of the `fillmethods` (Table 11.13)

Table 11.13 Replace Outliers Methods for the `filloutliers` Function

fillmethod	Brief Description
numeric scalar	Fills with a specified scalar value
`'linear','nearest','next', 'previous','spline', 'pchip','makima'`	See Table 11.9
`'center'`	Uses a center value determined by `findmethod`
`'clip'`	Uses the lower/upper threshold values for elements smaller/larger than thresholds determined by `findmethod`

Table 11.14 Find Outliers Methods for the `filloutliers` Function

findmethod	Brief Description
`'median'` (the default method)	Outliers are defined as elements more than three scaled median absolute deviations from the median, defined as `c*median(abs(A-median(A)))`, where `c=-1/(sqrt(2)*erfcinv(3/2))`
`'mean'`	Outliers are defined as elements more than three standard deviations from the mean
`'quartiles'`	Outliers are defined as elements more than 1.5 interquartile ranges above the upper quartile (75%) or below the lower quartile (25%) (this method is useful when data in `A` is not normally distributed)
`'grubbs'`	Applies Grubbs's test for outliers, which iteratively removes one outlier per iteration based on hypothesis testing (this method assumes that data in `A` is normally distributed)
`'gesd'`	Applies the generalized extreme Studentized deviate test for outliers (similar to `'grubbs'`, but performs better when there are multiple outliers masking each other)

As shown in Fig. 11.29a, an attempt to use the default median (`'median'`) method to find outliers results in fixing the second outlier but totally misses the leftmost 10 points because the first outlier is identified as a correct point,

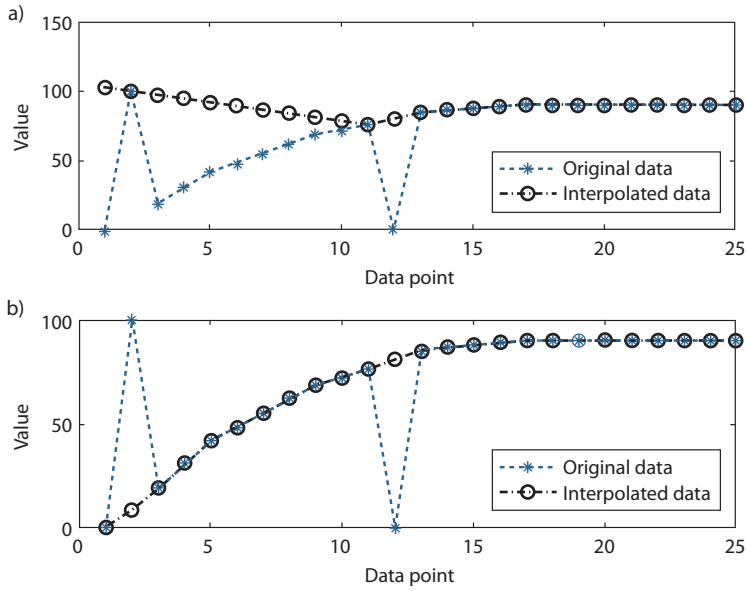


Fig. 11.29 Attempting to identify and remove outliers.

whereas the surrounding points are deemed to be erroneous. The attempt to use the moving median (`'movemedian'`) method with a five-point window (Fig. 11.29b) results in correctly identifying and fixing both outliers. However, if the size of the window is reduced to just four points, the first outlier would not be identified yet again.

Similar to `ismissing`, the `isoutlier` function allows for detecting outliers without fixing them.

One more function that you might find useful is `detrend`, which enables you to focus your analysis on the fluctuations in data about the trend. Specifically, `detrend(x)` and `detrend(x,n)` subtract the mean or a best-fit *n*th-degree polynomial from your data (applying the `polyfit` function) to find either a continuous or piecewise trend with multiple segments (which in this case are defined by the break points specified in the third input argument). Whether it makes sense to remove the polynomial trend from data depends on objectives of your analysis.

Problems

11.1 For gases, the variation of dynamic viscosity with temperature can be modeled by Sutherland's formula, which has the form [Eq. (11.42)]

$$\mu = \frac{C\sqrt[3]{T^2}}{T+S}$$

Here, μ is the absolute (dynamic) viscosity, T is the absolute temperature, and C and S are empirical constants. The following table gives the absolute viscosity of air at different temperatures:

T (°C)	−20	0	40	100	200	300	400	500	1000
μ (N s/m^2) ($\times 10^{-5}$)	1.63	1.71	1.87	2.17	2.53	2.98	3.32	3.64	5.04

Using linear least-squares regression, determine the constants C and S and present your results as a three-strip plot (`subplot(3,1,...)`). The first plot should present the dependence of viscosity from the temperature (use the `semilogy` function to plot it). Show the values of C and S along with the coefficient of determination R^2 (use the `num2str` function). The second plot should represent the same data but in coordinates you introduced to perform a linear regression. The third plot should present residuals for your linear regression.

11.2 Compare a locally defined spline with "not-a-knot" and "clamped" globally defined splines applied to approximate the following two "experimental" sequences of points:

a)
```
x = [0 2 4 5 7.5 10];
y = exp(-x/6).*cos(x);
```

b)
```
x = linspace(0,1.5,10);
y = humps(x);
```

For the "clamped" spline, try to adjust the slope at endpoints to better fit your data. Obviously, to compute spline coefficients, you have to use only "experimental" points, but once you have your splines developed, do not be stingy; increase the number of `x` points to show how smooth your approximations are.

11.3 Repeat the example of retrieving the useful signals from noise-corrupted data using the Fourier transform (follow the example in Sec. 11.6.3). Assume true data containing four sinusoids of the same amplitude of 1 with the frequencies of 40, 70, 170, and 210 Hz sampled at 1000 Hz. Corrupt it with a double-amplitude normally distributed noise. Plot the signal in the time domain and its retrieved components in the frequency domain, as in Fig. 11.26.

Chapter 12 Numerical Differentiation

- Utilizing Two- and Multipoint Approximation Schemes
- Estimating Approximation Errors
- Handling Partial Derivatives
- Using the MATLAB Functions

`diff`, `gradient`, `polyder`

12.1 Introduction

Quite often in engineering practice, there is a need to numerically estimate the first- or even higher-order derivative of a function (a signal) with respect to some argument at the trial points. For instance, you have measurements of a vehicle position along some predetermined direction $x(t)$ and need to assess its speed along this direction, that is, you need to estimate the first-order derivative of the coordinate x with respect to time. Another example is that you have the 2-D (planar) coordinates of a vehicle $(x(t), y(t))$ and need to know its heading $\Psi(t)$, that is, you need to estimate the first-order derivative of the coordinate y with respect to the coordinate x. The estimate of the second-order derivative in the latter case might also be needed because it is related to a torque control required to follow a path described by $y(x)$. Of course, we are not talking here about measurement errors or signal noise, but rather we assume that data are somehow filtered already, so that the aforementioned derivatives make sense. The general problem considered in this chapter is to estimate the first- and higher-order derivatives, $f'(x)$, $f''(x)$, etc., at n nodes using discrete data (measurements) $(x_i, f(x_i))$, $i = 1,2,\ldots, n$. Depending on a specific application, these estimates may use either data from several previous points (in this case, the derivative can be evaluated on-line) or data on multiple points before and after the current one (in this case, the estimates can be computed off-line only, when all data become available). The chapter starts with the simplest case, where only the knowledge about two data points is used to evaluate the first-order derivative at these points, followed by estimation of the local truncation errors associated with this case using the Taylor series expansions. It then proceeds with the

development of the more accurate estimates involving several, as opposed to just two, data points around each trial point, followed by expanding this approach to develop formulas for higher-order derivatives. An alternative way of obtaining the same formulas via using fitting polynomials, followed by assessing the values of partial derivatives, is discussed next. Finally, this chapter presents examples of employing the `diff` function to numerically estimate the first- and higher-order derivatives, along with introducing two more functions: `gradient`, to compute a gradient of a multivariable function and `polyder`, to differentiate a polynomial.

12.2 Two-Point Difference Approximations of the First-Order Derivative

Consider a function $y = f(x)$ shown in Fig. 12.1 (depicted with the dashed line). Of course, this function is not known; instead, the multiple data points depicted with circles are available. For the evenly spaced argument, we may present these data points as $x_{i\pm v} = x_i \pm vh$ $(v = 1, 2,\ldots)$ and $y_{i\pm v} = f(x_{i\pm v}) = f(x_i \pm vh)$. The objective is to estimate the first-order derivative at point x_i.

Let us start with the easiest possible estimations of the first-order derivative. These estimations are based on a linear piecewise interpolation, shown in Fig. 12.1 with the solid line. By exploiting the geometry of an appropriate triangle, we receive either a *forward* or *backward difference approximation* as follows:

$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_i)}{h} \tag{12.1}$$

$$f'(x_i) \approx \frac{f(x_i) - f(x_{i-1})}{h} \tag{12.2}$$

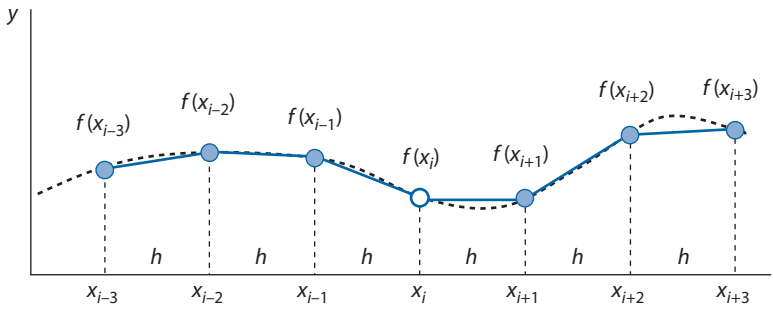


Fig. 12.1 Two-point difference approximation based on linear piecewise interpolation.

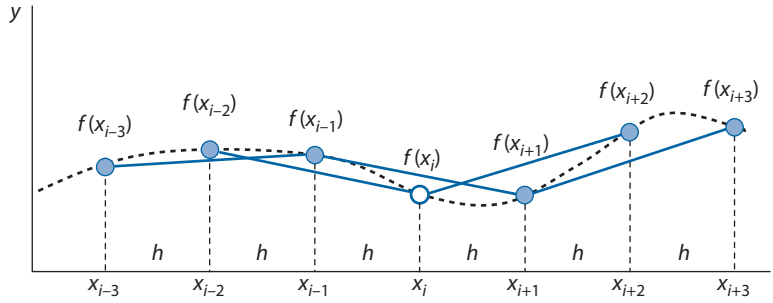


Fig. 12.2 Centered difference approximation of the first-order derivative.

For the particular situation depicted in Fig. 12.1, the forward difference approximation seems to give a better result for the points x_{i-2} and x_{i-1}, whereas the backward difference approximation is better for all other points. However, in practice, you cannot possibly know which approximation would work better, and that is why it is probably safer to use their combination, that is, Eqs. (12.1) and (12.2) blended together with the weighting coefficients of 0.5, which results in the *centered difference approximation* formula

$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_{i-1})}{2h} \tag{12.3}$$

Geometrically, it means estimating a slope between two points, nearest to the trial point, as shown in Fig. 12.2.

As seen from Figs. 12.1 and 12.2, not only do the three developed approximations not coincide, but all of them produce somewhat erroneous estimates anyway. It is only in the case of a linear behavior of a function $f(x)$ that all three formulas (12.1)–(12.3) would yield the same exact result. Otherwise, intuitively, the overall error of approximation (12.3) should be smaller than that of approximations (12.1) and (12.2). None of the discussed formulas, however, provides us with the estimates of approximation error. To get an estimate, we have to use a formal approach exploiting the Taylor series expansion of the function $f(x)$ [refer to Eq. (8.32)].

12.3 Error Estimates from the Taylor Series Expansion

Rather than using a geometric approach or linear interpolation to obtain the formulas of the previous section, let us use a Taylor series expansion, assuming that the grid spacing (spacing between any two nodes) h is small enough ($h << 1$). The Taylor series expansions of $f(x_{i+1})$ about the trial

point x_i yields [Eqs. (7.2) and (8.32)]

$$f(x_{i+1}) = f(x_i) + hf'(x_i) + \frac{h^2}{2!}f''(x_i) + \frac{h^3}{3!}f'''(x_i) + \frac{h^4}{4!}f^{(iv)}(x_i) + \cdots \quad (12.4)$$

Solving for $f'(x_i)$ results in

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h} - \frac{h}{2}f''(x_i) + HOT \approx \frac{f(x_{i+1}) - f(x_i)}{h} + O(h) \quad (12.5)$$

While the first term in Eq. (12.5) is the same as Eq. (12.1), the error of the forward difference approximation, denoted as $O(h)$, is proportional to h (and the second-order derivative of the function $f(x)$ at x_i):

$$O(h) \cong -\frac{h}{2}f''(x_i) \quad (12.6)$$

Similarly, for the backward difference approximation (12.2), we start with the Taylor series expansion for $f(x_{i-1})$ about the data point x_i

$$f(x_{i-1}) = f(x_i) - hf'(x_i) + \frac{h^2}{2!}f''(x_i) - \frac{h^3}{3!}f'''(x_i) + \frac{h^4}{4!}f^{(iv)}(x_i) - \cdots \quad (12.7)$$

which yields

$$f'(x_i) = \frac{f(x_i) - f(x_{i-1})}{h} + \frac{h}{2}f''(x_i) + HOT \approx \frac{f(x_i) - f(x_{i-1})}{h} + O(h) \quad (12.8)$$

Thus, for the backward difference approximation, the error is defined by

$$O(h) \cong \frac{h}{2}f''(x_i) \quad (12.9)$$

The centered difference approximation (12.3) can be obtained by subtracting the Taylor series approximation for $f(x_{i-1})$ [Eq. (12.7)] from that of $f(x_{i-1})$ [Eq. (12.4)], yielding

$$\begin{aligned} f(x_{i+1}) - f(x_{i-1}) = &\left(f(x_i) + hf'(x_i) + \frac{h^2}{2!}f''(x_i) + \frac{h^3}{3!}f'''(x_i) + \frac{h^4}{4!}f^{(iv)}(x_i) + \cdots\right) \\ &- \left(f(x_i) - hf'(x_i) + \frac{h^2}{2!}f''(x_i) - \frac{h^3}{3!}f'''(x_i) + \frac{h^4}{4!}f^{(iv)}(x_i) - \cdots\right) \end{aligned}$$

$$f(x_{i+1}) - f(x_{i-1}) = 2hf'(x_i) + \frac{2h^3}{6}f'''(x_i) + \frac{2h^5}{120}f^{(v)}(x_i) + \cdots \quad (12.10)$$

Solving for $f'(x_i)$ in this case yields

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1})}{2h} - \frac{h^2}{6} f'''(x_i) + HOT \tag{12.11}$$

Hence, the complete formula for the centered difference approximation [compare Eq. (12.11) with Eq. (12.3)] should be written as

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1})}{2h} + O(h^2) \tag{12.12}$$

with the error defined by

$$O(h^2) \cong -\frac{h^2}{6} f'''(x_i) \tag{12.13}$$

that is, being proportional to h^2 (and the third-order derivative) [because of cancellation of the second-order derivative terms in Eq. (12.10)].

To summarize, the *two-point forward and backward difference approximations* introduced in Eqs. (12.1) and (12.2), respectively, turned out to be the first-order approximations, meaning that the error in both cases is proportional to the grid spacing, h [Eqs. (12.6) and (12.9)]. These formulas produce an exact solution if the original function $f(x)$ is represented by a first-order polynomial (because in this case, $f''(x) \equiv 0$). As expected, the centered difference approximation (12.3) turned out to be more accurate, namely the second order, because the error is proportional to h^2 [Eq. (12.13)]. Hence, it produces an exact solution if the original function $f(x)$ is represented by up to a second-order polynomial (because in this case, $f'''(x) \equiv 0$). Other than that, the only way to have smaller errors is to decrease the grid spacing h (sampling time, in case x is time), meaning that measurements need to be taken more often (at a higher update rate). However, we may not have such luxury.

Before we start developing more accurate formulas in the following section, let us practice with the MATLAB Symbolic Math Toolbox in order to obtain Eqs. (12.5), (12.8), and (12.12) (it may seem too easy here, but it will pay off later). For instance, the self-explanatory script

```
%% Define symbolic variables
syms h f_i f_iplus1 f_prime f_2prime
%% Introduce the Taylor series expansion
Eq = -f_iplus1 + f_i + h*f_prime + h^2*f_2prime/factorial(2);
%% Solve the equation for f_prime
a = solve(Eq,f_prime);
%% Collect coefficients and display the result for f_prime
r=collect(a,f_2prime); pretty(r)
```

results in the following output, matching Eq. (12.5):

```
/   h  \             f_i - f_iplus1
| - -   | f_2prime - -------------
\   2  /                   h
```

Note that, if you are using earlier versions of MATLAB, your output may look slightly different. Back then, the `simple` function would be used to modify the output to what you would like it to be (as of R2015a, this function has been removed). To separate the error term, the preceding script uses the `collect` function. In this particular case, the call `collect(a,f_2prime)` produces the best result, but sometimes you need to play with the second argument to represent the result the way you want it. For instance, the call

```
r=collect(a,h^2); pretty(r)
```

would yield

```
/   f_2prime \   2  f_i - f_iplus1
| - -------- | h - -------------
\      2 h   /            h
```

If we substitute the second and fourth line of the previous script with

```
syms h f_i f_iminus1 f_prime f_2prime
```

and

```
Eq = -f_iminus1 + f_i - h*f_prime + h^2*f_2prime/factorial(2);
```

we get the backward difference approximation formula (12.8)

```
h            f_i - f_iminus1
- f_2prime + --------------
2                    h
```

Finally, to get the centered difference approximation of Eq. (12.12), we employ the following script:

```
%% Define symbolic variables
syms h f_i f_iplus1 f_iminus1 f_prime f_2prime f_3prime
%% Introduce two equations
Eq1 = -f_iplus1 +f_i+h*f_prime+h^2*f_2prime/2+h^3*f_3prime/6;
Eq2 = -f_iminus1 +f_i-h*f_prime+h^2*f_2prime/2-h^3*f_3prime/6;
%% Solve two equations for f_prime and f_2prime
a = solve(Eq1,Eq2,f_prime,f_2prime);
%% Collect coefficients and display the result for f_prime
r=collect(a.f_prime,f_3prime); pretty(r)
```

yielding

```
/    2  \
|   h   |           3 f_iplus1 - 3 f_iminus1
| - --  | f_3prime + ----------------------
\   6   /                    6 h
```

(Again, in the absence of the `simple` function, the Symbolic Math Toolbox does not seem to do a good job simplifying expressions anymore.) Note that in the latter case, we needed to keep more terms in both Taylor series expansions. Also, because we used two equations, they had to be resolved with respect to two unknowns rather than a single one as in the previous example. We will use the same script later in this chapter to get the solution for the second-order derivative, `a.f_2prime`. In the meantime, we will proceed with the development of more accurate approximations for the first-order derivative.

12.4 More Accurate Multiple-Point Approximations

So far, we have used only two points to get an estimate of the first-order derivative of the function. We learned that asymmetric (forward and backward) two-point difference approximations provide the first-order estimates [Eqs. (12.5) and (12.8)] while the three-point* centered difference approximation [Eq. (12.11)] happens to be of the second order. But why not use more points, if available, to increase the accuracy of approximation? In what follows, we will show that the results of the previous section may be generalized further, so that the following statements hold:

1. The first-order derivative approximation involving k points produces an error of the order of h^{k-1} at the worst.
2. Centered approximations are one order more accurate than asymmetric ones, that is, of the order of h^k (be careful though because in this case k refers to the actual number of terms in the approximation's numerator rather than approximation's name).

As an example, consider an approximation of the first-order derivative using three points instead of two. Involving Taylor series expansions for $f(x_{i+1})$ and $f(x_{i+2})$

$$f(x_{i+1}) = f(x_i) + hf'(x_i) + \frac{h^2}{2!}f''(x_i) + \frac{h^3}{3!}f'''(x_i) + \frac{h^4}{4!}f^{(iv)}(x_i) + \cdots$$

$$f(x_{i+2}) = f(x_i) + 2hf'(x_i) + \frac{(2h)^2}{2!}f''(x_i) + \frac{(2h)^3}{3!}f'''(x_i) + \frac{(2h)^4}{4!}f^{(iv)}(x_i) + \cdots \tag{12.14}$$

*Even though a centered difference approximation (12.11) utilizes only two points, it is actually referred to as a three-point centered difference approximation; it is just that $f(x_i)$ does not show up in it as it was the case for approximations (12.5) and (12.8).

and eliminating the second-order derivative terms by subtracting $f(x_{i+2})$ from $4f(x_{i+1})$ yields

$$4f(x_{i+1}) - f(x_{i+2}) = 4\left(f(x_i) + hf'(x_i) + \frac{h^2}{2!}f''(x_i) + \frac{h^3}{3!}f'''(x_i) + \frac{h^4}{4!}f^{(iv)}(x_i) + \cdots\right)$$
$$- \left(f(x_i) + 2hf'(x_i) + \frac{(2h)^2}{2!}f''(x_i) + \frac{(2h)^3}{3!}f'''(x_i) + \frac{(2h)^4}{4!}f^{(iv)}(x_i) + \cdots\right)$$
$$4f(x_{i+1}) - f(x_{i+2}) = 3f(x_i) + 2hf'(x_i) - \frac{4h^3}{6}f'''(x_i) - \frac{12h^4}{24}f^{(iv)}(x_i) - \cdots \tag{12.15}$$

Solving for $f'(x_i)$ produces the *three-point forward difference approximation*

$$f'(x_i) = \frac{-f(x_{i+2}) + 4f(x_{i+1}) - 3f(x_i)}{2h} + \frac{h^2}{3}f'''(x_i) + HOT \tag{12.16}$$

or

$$f'(x_i) = \frac{-f(x_{i+2}) + 4f(x_{i+1}) - 3f(x_i)}{2h} + O(h^2) \tag{12.17}$$

with

$$O(h^2) \cong \frac{h^2}{3}f'''(x_i) \tag{12.18}$$

Hence, the error is proportional to h^2 (and the third-order derivative), just as in the three-point centered difference approximation (12.13). The Symbolic Math Toolbox can help to produce this result as follows:

```
%% Define symbolic variables
syms h f_i f_iplus1 f_iplus2 f_prime f_2prime f_3prime
%% Introduce two equations
Eq1 = -f_iplus1+f_i+h*f_prime+h^2*f_2prime/2+h^3*f_3prime/6;
Eq2 = -f_iplus2+f_i+2*h*f_prime+(2*h)^2*f_2prime/2+(2*h)^3*f_3prime/6;
%% Solve two equations for f_prime and f_2prime
a = solve(Eq1,Eq2,f_prime,f_2prime);
%% Collect coefficients and display the result for f_prime
r=collect(a.f_prime,f_3prime); pretty(r)
```

yielding

```
 2
h             9 f_i - 12 f_iplus1 + 3 f_iplus2
-- f_3prime - ------------------------------
 3                          6 h
```

Similarly, the *three-point backward difference approximation* can be developed using another three points, namely $f(x_i)$, $f(x_{i-1})$, and $f(x_{i-2})$, yielding

$$f'(x_i) = \frac{3f(x_i) - 4f(x_{i-1}) + f(x_{i-2})}{2h} + O(h^2) \tag{12.19}$$

with

$$O(h^2) \cong \frac{h^2}{3} f'''(x_i) \tag{12.20}$$

If four points were considered for the forward and backward difference approximations, then, following the same routine and having three Taylor series expansions, we would eliminate the second-order derivative terms from the first two series expansions and use the third one to eliminate the third-order derivative term. Thus, we would end up with the third-order approximations, i.e. with the error proportional to h^3 (and the fourth-order derivative). For instance, for the third-order backward difference approximation of the first-order derivative, the script

```
%% Define symbolic variables
syms h f_i f_im1 f_im2 f_im3 f_pr f_2pr f_3pr f_4pr
%% Introduce three equations
Eq1 = -f_im1+f_i-h*f_pr+h^2*f_2pr/2-h^3*f_3pr/6+h^4*f_4pr/24;
Eq2 = -f_im2+f_i-2*h*f_pr+(2*h)^2*f_2pr/2-(2*h)^3*f_3pr/6 ...
      +(2*h)^4*f_4pr/24;
Eq3 = -f_im3+f_i-3*h*f_pr+(3*h)^2*f_2pr/2-(3*h)^3*f_3pr/6 ...
      +(3*h)^4*f_4pr/24;
%% Solve three equations for f_prime, f_2prime and f_3prime
a = solve(Eq1,Eq2,Eq3,f_pr,f_2pr,f_3pr);
%% Collect coefficients and display the result for f_prime
r=collect(a.f_pr,f_4pr); pretty(r)
```

produces

```
 3
h          22 f_i - 36 f_im1 + 18 f_im2 - 4 f_im3
-- f_4pr + -------------------------------------
 4                          12 h
```

In the full correspondence with the second statement formulated in the beginning of this section, if four points, $f(x_{i-2})$, $f(x_{i-1})$, $f(x_{i+1})$, and $f(x_{i+2})$, were used for the *five-point centered difference approximation*, an even more accurate, fourth-order accuracy approximation would be derived as follows:

$$f'(x_i) = \frac{-f(x_{i+2}) + 8f(x_{i+1}) - 8f(x_{i-1}) + f(x_{i-2})}{12h} + O(h^4) \tag{12.21}$$

where

$$O(h^4) \cong \frac{h^4}{30} f^{(v)}(x_i) \tag{12.22}$$

The MATLAB script employing the Symbolic Math Toolbox may look like

```
%% Define symbolic variables
syms h f_i f_ip1 f_ip2 f_im1 f_im2 f_p f_2p f_3p f_4p f_5p
%% Introduce equations for four points
Eqp1 =-f_ip1+f_i+h*f_p+h^2*f_2p/2+h^3*f_3p/6+h^4*f_4p/24+h^5*f_5p/120;
Eqp2 =-f_ip2+f_i+2*h*f_p+4*h^2*f_2p/2+8*h^3*f_3p/6+16*h^4*f_4p/24 ...
      +32*h^5*f_5p/120;
Eqm1 =-f_im1+f_i-h*f_p+h^2*f_2p/2-h^3*f_3p/6+h^4*f_4p/24-h^5*f_5p/120;
Eqm2 =-f_im2+f_i-2*h*f_p+4*h^2*f_2p/2-8*h^3*f_3p/6+16*h^4*f_4p/24 ...
      -32*h^5*f_5p/120;
%% Resolve four equations for f_p, f_2p, f_3p, and f_4p
sol=solve(Eqp1,Eqp2,Eqm1,Eqm2,f_p,f_2p,f_3p,f_4p);
%% Collect coefficients and display the result for f_p
r=collect(sol.f_p,f_5p); pretty(r)
```

resulting in

```
 4
h        40 f_im1 - 5 f_im2 - 40 f_ip1 + 5 f_ip2
-- f_5p - ----------------------------------------
30                          60  h
```

To summarize, Table 12.1 presents different formulas for the first-order derivatives, which can be easily programmed and used in off-line and on-line (backward difference) applications. If needed, the higher-accuracy formulas employing more adjacent points can be obtained using the MATLAB scripts similar to the ones shown in this and previous sections.

It should be noted that, although the backward difference approximation formulas seem to be less accurate than that of the centered difference, they represent a very important class of formulas that can be used on the go or on-line, if you wish, because they rely on the past points only.

Let us introduce the notation

$$\Delta f_i = f_i - f_{i-1} \tag{12.23}$$

denoting a simple backward difference of f. Applying the difference operator Δ twice results in

$$\Delta(\Delta f_i) \equiv \Delta^2 f_i = f_i - f_{i-1} - (f_{i-1} - f_{i-2}) = f_i - 2f_{i-1} + f_{i-2} \tag{12.24}$$

Table 12.1 Approximations of the First-Order Derivative, $f'(x_i)$

Name	Formula	Error Estimate
Two-point forward difference	$\frac{f(x_{i+1}) - f(x_i)}{h}$	$-\frac{1}{2}hf''(x_i)$
Two-point backward difference	$\frac{f(x_i) - f(x_{i-1})}{h}$	$\frac{1}{2}hf''(x_i)$
Three-point centered difference	$\frac{f(x_{i+1}) - f(x_{i-1})}{2h}$	$-\frac{1}{6}h^2f'''(x_i)$
Three-point forward difference	$\frac{-f(x_{i+2}) + 4f(x_{i+1}) - 3f(x_i)}{2h}$	$\frac{1}{3}h^2f'''(x_i)$
Three-point backward difference	$\frac{3f(x_i) - 4f(x_{i-1}) + f(x_{i-2})}{2h}$	$\frac{1}{3}h^2f'''(x_i)$
Four-point forward difference	$\frac{2f(x_{i+3}) - 9f(x_{i+2}) + 18f(x_{i+1}) - 11f(x_i)}{6h}$	$-\frac{1}{4}h^3f^{(iv)}(x_i)$
Four-point backward difference	$\frac{11f(x_i) - 18f(x_{i-1}) + 9f(x_{i-2}) - 2f(x_{i-3})}{6h}$	$\frac{1}{4}h^3f^{(iv)}(x_i)$
Five-point centered difference	$\frac{-f(x_{i+2}) + 8f(x_{i+1}) - 8f(x_{i-1}) + f(x_{i-2})}{12h}$	$\frac{1}{30}h^4f^{(v)}(x_i)$

and so on. Using these notations, the k-point backward difference approximation of the first-order derivative at the point x_i can be calculated as

$$f'(x_i) = \frac{1}{h}\sum_{m=1}^{k}\frac{1}{m}\Delta^m f(x_i) + O(h^{k-1}) \tag{12.25}$$

where the estimate of the local truncation error for approximation is

$$O(h^{k-1}) = \frac{1}{k}h^{k-1}f^{(k)}(x_i) \tag{12.26}$$

12.5 Second- and Higher-Order Derivatives

From the previous analysis, it is intuitively clear that for a derivative of order k, the smallest number of data points necessary to derive a difference approximation is $k + 1$. For example, we saw that a difference approximation for the first-order derivative of the function needs at least two data points. Likewise, the second-order derivative needs no fewer than three data points.

To derive the difference approximation for the second-order derivative, the basic principle is to eliminate the first-order derivative from the Taylor series expansion and as many higher-order terms (in this case, those greater than two) as possible. For instance, the *three-point forward difference approximation for the second-order derivative* $f''(x_i)$ can be found by considering the same Taylor series expansions as in Eq. (12.14) but treating them slightly differently. Specifically, subtracting $f(x_{i+2})$ from $2f(x_{i+1})$

eliminates the first-order derivative:

$$\begin{aligned}2f(x_{i+1})-f(x_{i+2}) &= 2\left(f(x_i)+hf'(x_i)+\frac{h^2}{2!}f''(x_i)+\frac{h^3}{3!}f'''(x_i)+\frac{h^4}{4!}f^{(iv)}(x_i)+\cdots\right)\\ &\quad-\left(f(x_i)+2hf'(x_i)+\frac{(2h)^2}{2!}f''(x_i)+\frac{(2h)^3}{3!}f'''(x_i)\right.\\ &\quad\left.+\frac{(2h)^4}{4!}f^{(iv)}(x_i)+\cdots\right)\\ 2f(x_{i+1})-f(x_{i+2}) &= f(x_i)-\frac{2h^2}{2}f''(x_i)-\frac{6h^3}{6}f'''(x_i)-\frac{14h^4}{24}f^{(iv)}(x_i)-\cdots\end{aligned} \tag{12.27}$$

Now, we can solve Eq. (12.27) for $f''(x_i)$, which yields

$$f''(x_i)=\frac{f(x_{i+2})-2f(x_{i+1})+f(x_i)}{h^2}+O(h) \tag{12.28}$$

where the error for the second-order derivative estimate is proportional to h (and the third-order derivative):

$$O(h)\cong -hf'''(x_i) \tag{12.29}$$

In fact, we can employ the very same script found after Eq. (12.18) in the previous section but with a new last line:

```
r=collect(a.f_2prime,f_3prime); pretty(r)
```

which will yield

```
                f_i - 2 f_iplus1 + f_iplus2
(-h) f_3prime + ---------------------------
                             2
                            h
```

Similarly, the *three-point backward difference approximation for the second-order derivative* can be found using $f(x_i)$, $f(x_{i-1})$, and $f(x_{i-2})$ as

$$f''(x_i)=\frac{f(x_i)-2f(x_{i-1})+f(x_{i-2})}{h^2}+O(h) \tag{12.30}$$

where

$$O(h)\cong hf'''(x_i) \tag{12.31}$$

The *three-point centered difference approximation for the second-order derivative* is found by taking a Taylor series expansion for $f(x_{i-1})$

[Eq. (12.7)] and $f(x_{i+1})$ [Eq. (12.4)] to yield

$$\begin{aligned} f(x_{i+1}) &= f(x_i) + hf'(x_i) + \frac{h^2}{2!}f''(x_i) + \frac{h^3}{3!}f'''(x_i) + \frac{h^4}{4!}f^{(iv)}(x_i) + \cdots \\ f(x_{i-1}) &= f(x_i) - hf'(x_i) + \frac{h^2}{2!}f''(x_i) - \frac{h^3}{3!}f'''(x_i) + \frac{h^4}{4!}f^{(iv)}(x_i) - \cdots \end{aligned} \tag{12.32}$$

Adding them together [rather than subtracting as in Eq. (12.10)] allows you to get rid of the first-order derivative

$$\begin{aligned} f(x_{i+1}) + f(x_{i-1}) &= \left(f(x_i) + hf'(x_i) + \frac{h^2}{2!}f''(x_i) + \frac{h^3}{3!}f'''(x_i) + \frac{h^4}{4!}f^{(iv)}(x_i) + \cdots \right) \\ &\quad + \left(f(x_i) - hf'(x_i) + \frac{h^2}{2!}f''(x_i) - \frac{h^3}{3!}f'''(x_i) + \frac{h^4}{4!}f^{(iv)}(x_i) - \cdots \right) \\ f(x_{i+1}) + f(x_{i-1}) &= 2f(x_i) + h^2 f''(x_i) + \frac{h^4}{6}f^{(iv)}(x_i) + \cdots \end{aligned} \tag{12.33}$$

Solving Eq. (12.33) for $f''(x_i)$ yields

$$f''(x_i) = \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1})}{h^2} + O(h^2) \tag{12.34}$$

where the error is proportional to h^2 (and the fourth-order derivative of the function)

$$O(h^2) \cong -\frac{h^2}{12} f^{(iv)}(x_i) \tag{12.35}$$

(Note that unlike approximation (12.11), the $f(x_i)$ term is now present in (12.34), so there is no confusion about the the approximation's name and the number of terms in the numerator.)

Again, we have developed the MATLAB script that computes this approximation at the end of Sec. 12.3 already. We only need to add one more term to each Taylor series expansion (otherwise, we will not get an error estimate) and modify the last line to output the second-order rather than the first-order derivative:

```
%% Define symbolic variables
syms h f_i f_iplus1 f_iminus1 f_prime f_2prime f_3prime f_4prime
%% Introduce two equations
Eq1 = -f_iplus1+f_i+h*f_prime+h^2*f_2prime/2+h^3*f_3prime/6 ...
      +h^4*f_4prime/24;
Eq2 = -f_iminus1+f_i-h*f_prime+h^2*f_2prime/2-h^3*f_3prime/6 ...
      +h^4*f_4prime/24;
%% Solve two equations for f_prime and f_2prime
a = solve(Eq1,Eq2,f_prime,f_2prime);
%% Collect coefficients and display the result for f_2prime
r=collect(a.f_2prime,f_4prime); pretty(r)
```

Running this script results in the following:

```
/    2  \
|   h    |           12 f_iplus1 - 24 f_i + 12 f_iminus1
| - --   | f_4prime + ----------------------------------
\   12   /                              2
                                   12 h
```

Now, we can summarize that obtaining approximations of higher-order derivatives unfolds as follows:

1. The pth-order derivative, $f_i^{(p)}$, needs at least $p+1$ data points.
2. The difference approximation is derived from the Taylor series expansions about x_i by combining them to eliminate the derivatives of the order of less than p.
3. The error estimate is the lowest term truncated.

Let us have another look at the MATLAB script appearing after Eq. (12.20). It exploits three Taylor series expansions (utilizing four points) and therefore can be used to obtain an estimate of up to the third-order derivative. In fact, it does this already—look at the second-to-last command, which solves three equations for three derivatives. Hence, all we have to do is append two more lines to this script:

```
r=collect(a.f_2pr,f_4pr); pretty(r)
r=collect(a.f_3pr,f_4pr); pretty(r)
```

to receive the four-point backward difference approximation for $f''(x_i)$

```
    2
11 h          24 f_i - 60 f_im1 + 48 f_im2 - 12 f_im3
----- f_4pr + ---------------------------------------
  12                                 2
                                12 h
```

and $f'''(x_i)$

```
3 h        2 f_i - 6 f_im1 + 6 f_im2 - 2 f_im3
--- f_4pr + -----------------------------------
 2                              3
                              2 h
```

Similarly, the MATLAB script shown after Eq. (12.22) can be used to obtain approximations of up to the fourth-order derivative. For instance, if we append one more line to this script

```
r=collect(sol.f_3p,f_5p); pretty(r)
```

we will get the *five-point centered difference approximation for the third-order derivative*

```
/    2 \
|   h  |       4 f_im1 - 2 f_im2 - 4 f_ip1 + 2 f_ip2
| - --  | f_5p + ------------------------------------
\   4  /                              3
                                   4 h
```

To get five-point centered difference approximations for the second- and fourth-order derivatives together with the error terms, we would need to add one more term to the Taylor series expansion as follows:

```
%% Define symbolic variables
syms h f_i f_ip1 f_ip2 f_im1 f_im2 f_p f_2p f_3p f_4p f_5p f_6p
%% Introduce equations for four points
Eqp2=-f_ip2+f_i+2*h*f_p+4*h^2*f_2p/2+8*h^3*f_3p/6+16*h^4*f_4p/24 ...
     +32*h^5*f_5p/120+64*h^6*f_6p/720;
Eqp1=-f_ip1+f_i+h*f_p+h^2*f_2p/2+h^3*f_3p/6+h^4*f_4p/24+h^5*f_5p/120 ...
     +h^6*f_6p/720;
Eqm1=-f_im1+f_i-h*f_p+h^2*f_2p/2-h^3*f_3p/6+h^4*f_4p/24-h^5*f_5p/120 ...
     +h^6*f_6p/720;
Eqm2=-f_im2+f_i-2*h*f_p+4*h^2*f_2p/2-8*h^3*f_3p/6+16*h^4*f_4p/24 ...
     -32*h^5*f_5p/120+64*h^6*f_6p/720;
%% Resolve four equations for f_p, f_2p, f_3p, and f_4p
sol=solve(Eqp1,Eqp2,Eqm1,Eqm2,f_p,f_2p,f_3p,f_4p);
%% Collect coefficients and display the result for f_3p and f_4p
r=collect(sol.f_3p,f_5p); pretty(r)
r=collect(sol.f_4p,f_6p); pretty(r)
```

Running this script yields

```
/    2 \
|   h  |       4 f_im1 - 2 f_im2 - 4 f_ip1 + 2 f_ip2
| - --  | f_5p + ------------------------------------
\   4  /                              3
                                   4 h
```

for $f'''(x_i)$ and

```
/    2 \
|   h  |      36 f_i - 24 f_im1 + 6 f_im2 - 24 f_ip1 + 6 f_ip2
| - - -  | f_6p + -------------------------------------------------
\   6  /                                  4
                                       6 h
```

for $f^{(iv)}(x_i)$.

Tables 12.2–12.4 summarize the findings of Sec. 12.5. Analyzing these tables, the observation made in the beginning of Sec. 12.4 can be extended to the case of higher-order derivatives:

1. A pth-order derivative approximation involving k points ($k > p$) produces an error of the order of h^{k-p} at the worst.
2. Centered approximations are one order more accurate than asymmetric ones, that is, of the order of h^{k-p+1} (again, k refers to the actual number of

Table 12.2 Approximations of the Second-Order Derivative, $f''(x_i)$

Name	Formula	Error Estimate
Three-point forward difference	$\frac{f(x_{i+2}) - 2f(x_{i+1}) + f(x_i)}{h^2}$	$-hf'''(x_i)$
Three-point backward difference	$\frac{f(x_i) - 2f(x_{i-1}) + f(x_{i-2})}{h^2}$	$hf'''(x_i)$
Three-point centered difference	$\frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1})}{h^2}$	$-\frac{1}{12}h^2 f^{(iv)}(x_i)$
Four-point forward difference	$\frac{-f(x_{i+3}) + 4f(x_{i+2}) - 5f(x_{i+1}) + 2f(x_i)}{h^2}$	$\frac{11}{12}h^2 f^{(iv)}(x_i)$
Four-point backward difference	$\frac{2f(x_i) - 5f(x_{i-1}) + 4f(x_{i-2}) - f(x_{i-3})}{h^2}$	$\frac{11}{12}h^2 f^{(iv)}(x_i)$
Five-point centered difference	$\frac{-f(x_{i+2}) + 16f(x_{i+1}) - 30f(x_i) + 16f(x_{i-1}) - f(x_{i-2})}{12h^2}$	$\frac{1}{90}h^4 f^{(vi)}(x_i)$

terms in the approximation's numerator, which for the second-order derivative is aligned with the name of approximation).

Obviously, if there is no control over grid spacing h but a higher accuracy is required, the only way to achieve this goal is to increase the number of points k used to produce an approximation formula.

Finally, if we use notations of Eqs. (12.23) and (12.24), for the backward difference approximations of the higher-order derivatives, we can write

$$f^{(k)}(x_i) \approx \frac{1}{h^k}\Delta^k f(x_i) \tag{12.36}$$

Table 12.3 Approximations of the Third-Order Derivative, $f'''(x_i)$

Name	Formula	Error Estimate
Four-point forward difference	$\frac{f(x_{i+3}) - 3f(x_{i+2}) + 3f(x_{i+1}) - f(x_i)}{h^3}$	$-\frac{3}{2}hf^{(iv)}(x_i)$
Four-point backward difference	$\frac{f(x_i) - 3f(x_{i-1}) + 3f(x_{i-2}) - f(x_{i-3})}{h^3}$	$\frac{3}{2}hf^{(iv)}(x_i)$
Five-point centered difference	$\frac{f(x_{i+2}) - 2f(x_{i+1}) + 2f(x_{i-1}) - f(x_{i-2})}{2h^3}$	$-\frac{1}{4}h^2 f^{(v)}(x_i)$

Table 12.4 Approximation of the Fourth-Order Derivative, $f^{(iv)}(x_i)$

Name	Formula	Error Estimate
Five-point centered difference	$\frac{f(x_{i+2}) - 4f(x_{i+1}) + 6f(x_i) - 4f(x_{i-1}) + f(x_{i-2})}{h^4}$	$-\frac{1}{6}h^2 f^{(vi)}(x_i)$

12.6 Alternative Ways of Obtaining Approximation Formulas

The way of obtaining approximation formulas presented in Secs. 12.3–12.5 is the best because it also allows estimating the error of approximation. However, there are some alternative ways of obtaining exactly the same approximation formulas. One of them is the *multiple-application scheme*.

As shown in the previous section, any formula provides an estimate of derivative in the following form:

$$D \approx \hat{D}(h) + \hat{e}(h) \tag{12.37}$$

Here, the derivative approximation $\hat{D}(h)$ and the error approximation $\hat{e}(h)$ both depend on the grid spacing h. The error approximation also depends on a higher-order derivative at x_i, which is unknown (see the last column in Tables 12.1–12.4). The idea to improve accuracy of approximation is to somehow estimate $\hat{e}(h)$ without estimating this higher-order derivative at x_i.

To do so, we apply the same approximation formula twice, with a single step, h, and a double-step, $2h$, which yields

$$D \approx \hat{D}(h) + \hat{e}(h) \approx \hat{D}(2h) + \hat{e}(2h) = \hat{D}(2h) + 2^m \hat{e}(h) \tag{12.38}$$

where m is the order of approximation error computed as $m = k - p$ for noncentered and $m = k - p + 1$ for centered approximations (k refers to the number of terms appearing in approximation's numerator and p is the derivative's order). From this equation, it immediately follows that

$$\hat{e}(h) \approx \frac{\hat{D}(h) - \hat{D}(2h)}{2^m - 1} \tag{12.39}$$

Substituting Eq. (12.39) back into Eq. (12.38) yields

$$D \approx \hat{D}(h) + \frac{\hat{D}(h) - \hat{D}(2h)}{2^m - 1} = \frac{2^m \hat{D}(h) - \hat{D}(2h)}{2^m - 1} \tag{12.40}$$

The error of this later approximation is of the order of the first term in the *HOT* remainder of the exact approximation formula. Specifically, for noncentered approximations, it is just the next order, that is, $m + 1$; for centered approximations, because of cancellations, it may be even the next nearest order, that is, $m + 2$ (see the rightmost columns in Tables 12.1–12.4). In numerical analysis, this approach is known as *Richardson extrapolation* and is widely used to improve the rate of convergence of numerical algorithms.

For instance, if we apply the suggested double-step scheme to the three-point centered difference approximation for $f''(x_i)$ with $m = 2$ (Table 12.2), we will end up with the following more accurate approximation:

$$D \approx \frac{4}{3}\hat{D}(h) - \frac{1}{3}\hat{D}(2h) \tag{12.41}$$

Substituting

$$\hat{D}(h) \approx (f(x_{i+1}) - 2f(x_i) + f(x_{i-1}))/h^2$$

and

$$\hat{D}(2h) \approx (f(x_{i+2}) - 2f(x_i) + f(x_{i-2}))/4h^2 \tag{12.42}$$

results in

$$D \approx \frac{-f(x_{i+2}) + 16f(x_{i+1}) - 30f(x_i) + 16f(x_{i-1}) - f(x_{i-2})}{12h^2} \tag{12.43}$$

which happens to be the five-point centered difference shown in the last row of Table 12.2.

Another way of obtaining exactly the same difference approximation formulas is to use k data points to produce a $(k-1)$th-order fitting polynomial. Then, derivatives can be estimated by differentiating this polynomial and estimating the result at x_i. For instance, the following MATLAB script:

```
%% Define symbolic variables
syms x_i h f_i f_ip1 f_ip2 f_ip3 x
%% Solve for coefficients of a cubic interpolation
A = [     x_i^3            x_i^2         x_i   1;
     (x_i+h)^3        (x_i+h)^2       x_i+h   1;
   (x_i+2*h)^3      (x_i+2*h)^2     x_i+2*h   1;
   (x_i+3*h)^3      (x_i+3*h)^2    x_i+3*h    1];
b= [f_i;f_ip1;f_ip2;f_ip3];
coef = A\b;
%% Form the polynomial out of vector of its coefficients
Cpol=[x^3 x^2 x 1]*coef;
%% Compute the first-order derivative at x=x_i
D1 = diff(Cpol,1); D1=subs(D1,x,x_i); pretty(simplify(D1))
%% Compute the second-order derivative at x=x_i
D2 = diff(Cpol,2); D2=subs(D2,x,x_i); pretty(simplify(D2))
%% Compute the third-order derivative at x=x_i
D3 = diff(Cpol,3); D3=subs(D3,x,x_i); pretty(simplify(D3))
```

employs the Symbolic Math Toolbox to

1. Compute coefficients of the third-order polynomial passing through four data points $(x_{i+\nu}, y_{i+\nu})$, $\nu = 0, 1, 2, 3$
2. Form a symbolic representation of this polynomial
3. Find expressions for the first- through third-order derivatives and estimate them at x_i

Running this script yields three expressions

```
  11 f_i - 18 f_ip1 + 9 f_ip2 - 2 f_ip3
- ---------------------------------------
                    6 h
2 f_i - 5 f_ip1 + 4 f_ip2 - f_ip3
---------------------------------
                   2
                  h
  f_i - 3 f_ip1 + 3 f_ip2 - f_ip3
- -------------------------------
                    3
                   h
```

which are the four-point forward difference approximations of the first-, second-, and third-order derivatives (compare them with those of Tables 12.1 – 12.3).

12.7 Partial Derivatives

The formulas obtained in the previous sections of this chapter can also be used to estimate partial derivatives in the case of a multivariable function. Consider, for example, the two-variable function $z = f(x, y)$. The difference approximation for the partial derivative

$$f_x = \frac{\partial}{\partial x} f(x, y) \quad \text{at } x = x_i \text{ and } y = y_i \tag{12.44}$$

can be derived by fixing y at $y = y_i$ and finding the derivative of f with respect to x. Therefore, the different schemes derived earlier can be used to approximate the partial derivatives as well. For instance, with h_x being the grid spacing along x, the following formulas hold:

$$f_x(x_i, y_i) \approx \frac{f(x_i + h_x, y_i) - f(x_i, y_i)}{h_x} \tag{12.45}$$

$$f_x(x_i, y_i) \approx \frac{f(x_i, y_i) - f(x_i - h_x, y_i)}{h_x} \tag{12.46}$$

$$f_x(x_i, y_i) \approx \frac{f(x_i + h_x, y_i) - f(x_i - h_x, y_i)}{2h_x} \tag{12.47}$$

They represent forward, backward, and centered difference approximations of the first-order partial derivative, respectively.

The difference approximations for the second-order partial derivatives can be obtained in a similar manner. For instance, the *centered difference approximation* for *the second-order partial derivatives* of $f(x, y)$ at x_i and

y_i, with h_y being the grid spacing along y, are

$$f_{xx} = \frac{\partial^2}{\partial x^2} f(x, y) \approx \frac{f(x_i + h_x, y_i) - 2f(x_i, y_i) + f(x_i - h_x, y_i)}{h_x^2}$$

$$f_{yy} = \frac{\partial^2}{\partial y^2} f(x, y) \approx \frac{f(x_i, y_i + h_y) - 2f(x_i, y_i) + f(x_i, y_i - h_y)}{h_y^2}$$

$$f_{xy} = \frac{\partial^2}{\partial x \partial y} f(x, y) \approx \frac{f(x_i + h_x, y_i + h_y) - f(x_i - h_x, y_i + h_y)}{4h_x h_y} + \frac{f(x_i + h_x, y_i - h_y) - f(x_i - h_x, y_i - h_y)}{4h_x h_y} \quad (12.48)$$

12.8 Using the MATLAB Functions

Unfortunately, MATLAB does not have a single function realizing different-order approximations discussed in Secs. 12.2–12.5. The only function that can somehow be used to calculate derivatives (besides the `gradient` and `polyder` functions to be discussed later in this section) is the `diff` function.

Of course, you are familiar with a symbolic implementation of the `diff` function already (Sec. 7.3.2, Table 7.10). For a symbolic input this function does derive a symbolic derivative, which you can then use to compute an exact numeric value of derivative at any given function argument. However, in its numerical implementation, this function has little to do with computing derivatives. Three basic syntaxes for this function are

```
d=diff(x)
d=diff(x,N)
d=diff(x,N,dim)
```

where `x` is either a vector of length n or a matrix with m rows and n columns. If `x` is a vector, then `diff(x)` calculates differences between adjacent elements of `x`, so that the result `d` is a vector of length $n-1$. If `x` is a matrix, then `d` is a matrix with $m-1$ rows and n columns containing the differences between adjacent rows in each column. For example,

```
>> x=[1,3,5,2,4,6];
>> d=diff(x)
```

returns

```
d = 2  2  -3  2  2
```

and

```
>> X=[1,2,3,4,5;5,4,3,2,1;1,3,5,2,4]
>> d=diff(X)
```

yields

```
X =  1   2   3   4   5
     5   4   3   2   1
     1   3   5   2   4
d =  4   2   0  -2  -4
    -4  -1   2   0   3
```

The following call

```
>> diff(x,2)
```

which is equivalent to

```
>> diff(diff(x))
```

returns

```
ans = 0  -5  5  0
```

that is, it applies the `diff` function recursively `N` times (in our case twice because `N = 2`), resulting in the *N*th difference (Fig. 12.3).

The dimension of calculating the differences in the case where `x` is a matrix can be changed using a scalar input argument `dim`. For instance, for the above matrix `X`, the call

```
>> d=diff(X,[],2)
```

produces

```
d =  1   1   1   1
    -1  -1  -1  -1
     2   2  -3   2
```

(note that you have to use a placeholder, `[]`, because the order of arguments matters).

How can we use the `diff` function to calculate derivatives? Assume we have two *n*-element vectors, `x` and `y`, containing the arguments and function values. Then, for the first $n - 1$ data points we can compute the two-point forward difference approximation of the first-order derivative dy/dx by issuing the following command:

```
diff(y)./diff(x);
```

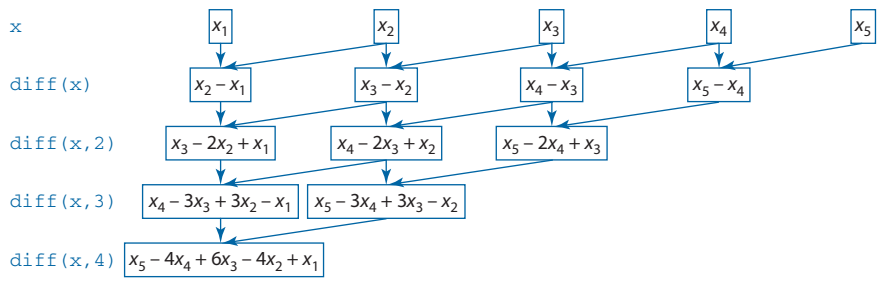


Fig. 12.3 Illustration of recursive application of the `diff` function.

The very same vector, having $n-1$ elements in it, can also be treated as the two-point backward difference approximation of the first-order derivative for the last $n-1$ data points (Table 12.1).

In the case of the evenly spaced argument with the grid spacing `h`, the second-order derivative d^2y/dx^2 can be estimated in accordance with the three-point difference approximation [Eq. (12.30)] as

```
diff(x,2)/h^2;
```

(the resulting vector has $n-2$ elements in it). The result can be treated as the three-point backward difference approximation corresponding to the data points 3 through n [Eq. (12.30)], the three-point centered difference approximation [Eq. (12.34)] corresponding to the data points 2 through $n-1$, or the three-point forward difference approximation [Eq. (12.28)] corresponding to the data points 1 through $n-2$ (Table 12.2).

Similarly, the result of

```
diff(x,3)/h^3;
```

can be treated as the four-point backward difference approximation of d^3y/dx^3 corresponding to the data points 4 through n, or the four-point forward difference approximation corresponding to the data points 1 through $n-3$ (Table 12.3). The command

```
diff(x,4)/h^4;
```

produces the five-point centered difference approximation of d^4y/dx^4 corresponding to the data points 3 through $n-2$ (Table 12.4), and so on.

The following script shows an example of using the `diff` function to compute derivatives:

```
x=linspace(0,pi,50); h=x(2)-x(1);
y=sin(2*x).*cos(x);
plot(x,y,'bo-.'), hold
plot(x(2:end),diff(y)/h,'+m')
plot(x(2:end-1),diff(y,2)/h^2,'pg'), grid
legend('Function','1^{st}-order derivative','2^{nd}-order derivative',...
       'Orientation','horizontal'), xlabel('x'), ylabel('y')
```

Running it results in the graph shown in Fig. 12.4. Note that there is no value for the first-order derivative at the very first data point and no second-order derivative estimates at the first and last data points (due to shortening the length of the vector every time you apply the `diff` function).

Besides `diff`, there is one more function that allows computing the numerical gradients (differences) of a multivariable function. For a function of N variables, $F(x, y, z,\ldots)$, the gradient is defined as [Eq. (10.54)]

$$\nabla F = \frac{\partial F}{\partial x}\mathbf{i} + \frac{\partial F}{\partial y}\mathbf{j} + \frac{\partial F}{\partial z}\mathbf{k} + \cdots \tag{12.49}$$

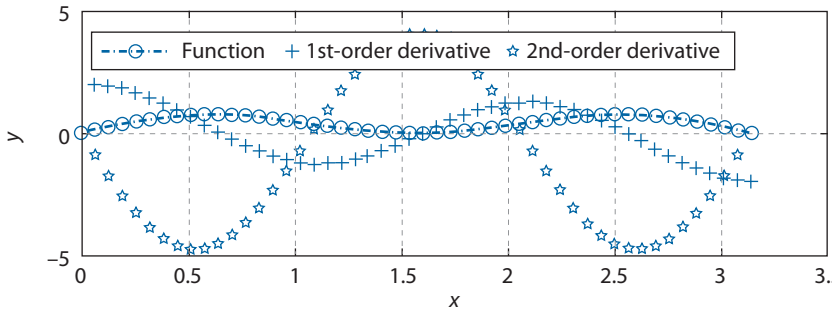


Fig. 12.4 Function $y = \sin(2x)\cos(x)$ and its numerically evaluated derivatives.

and so the MATLAB `gradient` function returns the N components of the N-dimensional numerical gradient. Depending on dimensions of the input array `F`, the syntax for the `gradient` function is one of the following:

```
Fx=gradient(F)
[Fx,Fy]=gradient(F)
[Fx,Fy,Fz,...]=gradient(F)
[...] = gradient(F,h)
[...] = gradient(F,h1,h2,...)
```

If `F` is a vector (meaning that instead of pairs $(x_i, f(x_i))$, $i = 1, 2, \ldots, n$, we are only given a set of $f(x_i)$, $i = 1, 2, \ldots, n$), then `Fx` is a vector corresponding to $\mathrm{d}F/\mathrm{d}x$, the differences in the x direction. The two-point forward difference [Eq. (12.1)] is used to estimate the derivative at the first point, the three-point backward difference [Eq. (12.2)] is used to estimate the derivative at the very last point, and the two-point centered difference [Eq. (12.3)] is used otherwise. If the argument grid spacing `h` is not defined, it is assumed to be 1. The following example is intended to clarify this (try to calculate the gradient at the first, second, and last points yourself):

```
>> t=[1  3  6  7  3  4  6]
t = 1   3   6   7   3   4   6
>> gradient(t)
ans = 2.0000  2.5000  2.0000  -1.5000  -1.5000  1.5000  2.0000
>> gradient(t,10)
ans = 0.2000  0.2500  0.2000  -0.1500  -0.1500  0.1500  0.2000
```

If `F` is a matrix of values of a two-variable function `F(x, y)`, then `Fx` returns the numerical gradient $\partial F/\partial x$, and `Fy` returns the numerical gradient $\partial F/\partial y$. Both `Fx` and `Fy` are the matrices of partial derivatives computed according to formulas similar to Eqs. (12.45)–(12.47). For the gradient of a function depending on more than two variables, that is, when `F` is an N-dimensional array, the `gradient` function returns N N-dimensional arrays.

The last two syntaxes of the `gradient` function assume either a scalar or vector as the additional input argument(s) (`h`, `h1`, `h2`,...). For instance, in the previous example, the default value of `1` and then `10` were used to represent evenly spaced argument data. You could also use a vector input as in the following two examples:

```
>> t=[1 3 6 7 3 4 6]
t = 1  3  6  7  3  4  6
>> gradient(t,t)
ans = 1  1  1  1  1  1  1
>> gradient(t,0:0.1:0.7)
ans = 20.0000  25.0000  20.0000  -15.0000  -15.0000  15.0000  20.0000
```

In this case, the length of the vector `h` should be the same as that of the vector `F`.

In the case of a multivariable function, a single spacing value `h` specifies the spacing between the data points in every direction, and N spacing values (`h1`, `h2`, ...) specify the spacing for each dimension of `F` individually. Similarly, they can assume either scalar or vector inputs.

The following script presents an example of computing and displaying the gradient of $F(x, y) = (x^2 + y^2)\cos(x)\sin(2y)$:

```
px=linspace(-pi/2,pi/2,40); hx=px(2)-px(1); % 1st argument spacing (hx)
py=linspace(-pi,-pi/4,40); hy=py(2)-py(1);  % 2nd argument spacing (hy)
[x,y] = meshgrid(px,py);                    % mesh grid of two arguments
F = (x.^2+y.^2).*cos(x).*sin(2.*y); % matrix of function values on the grid
[Fx,Fy] = gradient(F,hx,hy);                % compute the gradient
colormap('Spring')
contourf(px,py,F), hold on                  % draw a filled contour map
quiver(x,y,Fx,Fy), hold off                 % add the gradient field (the arrows)
xlabel('x'), ylabel('y')
figure                                      % show a 3-D view of the function
waterfall(x,y,F), view([-60,30]), axis tight
xlabel('x'), ylabel('y'), zlabel('z')
```

Running this script results in the two graphs shown in Fig. 12.5.

First, it creates a mesh grid of arguments and computes the values of the function in each node creating three 40-by-40 matrices. Then, it applies the `gradient` function specifically indicating grid spacings, `hx` and `hy`, to compute the 2-D gradient in this case defined by the first two terms of Eq. (10.49). This gradient can be thought of as a collection of vectors pointing in the direction of increasing values of F. The two components of these vectors, $\partial F/\partial x$ and $\partial F/\partial y$, are stored in two matrices, `Fx` and `Fy`. Next, the function itself is being drawn as a filled (colored) contour map, and finally the gradient is added to this contour map as the arrows originating at the nodes (`x`, `y`) with components (`Fx`, `Fy`) (using the `quiver` function). Lastly, for better understanding, the script also produces a 3-D plot of the function.

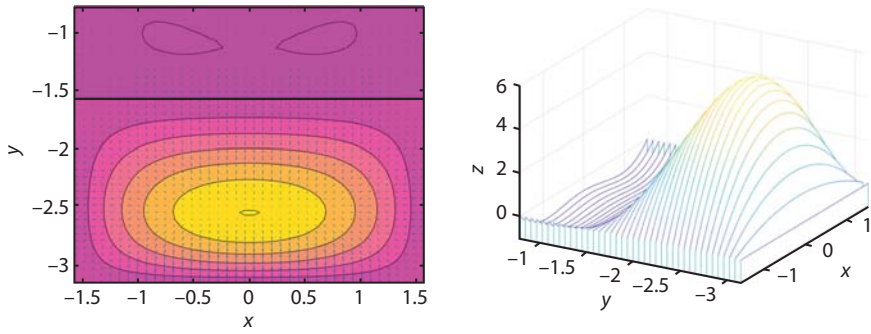


Fig. 12.5 Illustration of computing and visualizing the 2-D gradient.

Besides `gradient`, MATLAB offers one more function, `polyder`, to deal with derivatives of polynomials (defined by the vectors of their coefficients as discussed in Sec. 3.5). Its syntaxes are shown in Table 12.5.

For example, a set of commands

```
p=[1  2  3  4  5];              % define a polynomial
disp('Original polynomial')
pretty(poly2sym(p))
b=polyder(p);                   % take a derivative
disp('Derivative polynomial')
pretty(poly2sym(b))
```

results in the following output

```
Original polynomial
         4     3     2
        x + 2 x + 3 x + 4 x + 5
Derivative polynomial
           3     2
         4 x + 6 x + 6 x + 4
```

Table 12.5 Syntaxes of the `polyder` Function

Syntax	Brief Description
`b=polyder(p)`	Returns a vector containing polynomial coefficients of the derivative of the polynomial defined by the vector of coefficients (in descending powers) `p`
`b=polyder(p1,p2)`	Returns a vector containing polynomial coefficients of the derivative of the product of polynomials `p1` and `p2`
`[n,d]=polyder(p1,p2)`	Returns two vectors, `n` and `d`, containing polynomial coefficients of the numerator and denominator of the derivative of the quotient of polynomials, `p1` and `p2`

Of course, the same result can be achieved by employing a symbolic implementation of the `diff` function. The script

```
p=[1  2  3  4  5];
disp('Original polynomial')
pretty(poly2sym(p))
disp('First derivative')
d1=diff(poly2sym(p)); pretty(d1)
disp('Second derivative')
d2=diff(poly2sym(p),2); pretty(d2)
```

yields

```
Original polynomial
         4     3     2
        x + 2 x + 3 x + 4 x + 5
First derivative
           3     2
        4 x + 6 x + 6 x + 4
Second derivative
             2
        12 x + 12 x + 6
```

The following script presents another example of first computing the first-order derivative of a polynomial analytically using the `polyder` function and then using the `polyval` function to evaluate this derivative at a set of arguments:

```
r=[1,2,4,6]; a=poly(r); b=polyder(a);    % create a polynomial from its roots
x=linspace(min(r),max(r),50);            % define the x-vector for plotting
plot(x,polyval(a,x),'bo--'), hold on
plot(r,polyval(a,r),'Marker','p','MarkerFaceColor','r',...
                    'MarkerSize',15), xlabel('x'), ylabel('y')
plot(x,polyval(b,x),'+m'), hold off, grid
legend('Polynomial','Roots','Derivative','Location','North')
```

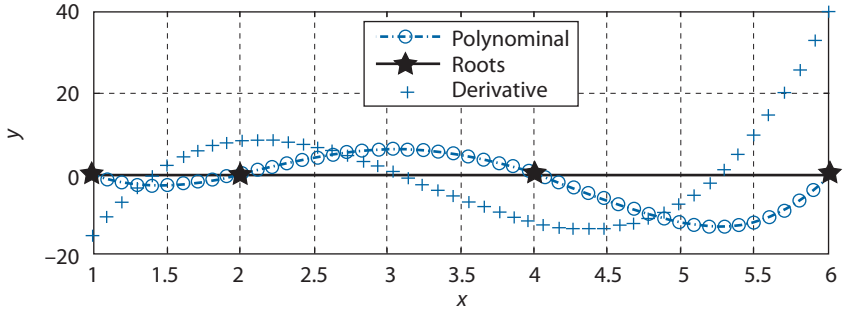


Fig. 12.6 Illustration of computing and visualizing the gradient.

Running this script results in the graph presented in Fig. 12.6. You can visually confirm the validity of the results by noting that the derivative polynomial crosses the zero line where the original polynomial has two minimums and one maximum.

Problems

12.1 Calculate the first- and second-order derivatives of $\cos(x)$ at $x = \pi/4$ with $h = \pi/100$ using the centered difference approximations [Eqs. (12.12) and (12.34)] and estimate the errors using a) Eqs. (12.13) and (12.35) with the analytically derived third and fourth derivatives in them analytical expressions and b) a multiple-application scheme [Eq. (12.39)].

12.2 Use the MATLAB Symbolic Math Toolbox to obtain the centered six-point difference approximations of the first-, second-, third-, and fourth-order derivatives. Also find the errors associated with these approximations.

12.3 Use the MATLAB `diff` function to compute the centered difference approximation for the first-order derivative of a function given by

```
x = 0:0.1:10;
f = sin(x);
```

Compare the result with the values of the analytically derived derivative ($f'(x) = \cos(x)$) and less accurate estimates based on the forward and backward difference approximations. To this end, first subtract the numerically derived values of the first-order derivative from the true values (computed using the analytical formula) and plot a `bar` graph showing the values of these discrepancies. Second, plot a graph for the approximation error $O(h^2)$ [Eq. (12.13)] based on the analytically derived expression for the third-order derivative ($f'''(x) = -\cos(x)$). In the same plot, show the error you would have if the forward or backward difference approximation [Eqs. (12.6) and (12.9)] was used instead of the centered one.

Chapter 13 Numerical Integration

- Integrating Numerical Data Using the Trapezoidal Rule
- Using the Simpson's Rules and Newton–Cotes Formulas
- Understanding Gauss Quadratures
- Using the MATLAB Functions

`trapz`, `cumtrapz`, `quad`, `quadv`, `quadl`, `quadgk`, `dblquad`, `triplequad`, `quad2d`, `integral`, `integral2`, `integral3`, `polyint`

13.1 Introduction

Suppose that you need to calculate running costs for an airplane, service, or business based on some timely spaced consumption data. Or consider an even simpler example where during an experiment (simulation) you measured (recorded) a time history of some variable, say, acceleration. Now you want to know an integral of this output, that is, a time history of velocity. In both cases, having discrete measurements of some function, $y = f(x)$, given as n pairs (x_i, y_i), $i = 1, 2, \ldots, n$ (in the latter example, x stands for time and y for acceleration), we need to numerically estimate the integral

$$I = \int_{x_1}^{x_n} f(x)\mathrm{d}x \tag{13.1}$$

A straightforward way to accomplish this in MATLAB would be to use the trapezoidal method coded in the `trapz` (`cumtrapz`) function. All you would need to do then is pass the pairs (x_i, y_i) on to this function.

What if the integrand $f(x)$ in Eq. (13.1), is given analytically? In this case, it may be possible to solve the problem analytically by using the `int` function of the Symbolic Math Toolbox as shown in a couple of examples in Sec. 7.5. However, not every analytical expression can be integrated analytically. You may still need to compute the values given by an analytical formula or a model over some interval and then use these "experimental" data values. Compared with the previous case, however, when those n

pairs were all that was available, having an analytical expression or a model allows you to compute the values of your explicit or implicit function at any argument, which enables the use of a variety of methods for numerical integration way beyond the simple trapezoidal method. This chapter introduces these methods (the Newton–Cotes integration formulas, multiple-application scheme, and Gauss quadrature), showing the main ideas behind them and deriving underlying formulas using the Symbolic Math Toolbox. In MATLAB, these methods are implemented in the `quad` (`quadv`), `quadl`, and `quadgk` functions for estimation of a single integral and the `dblquad` and `triplequad` functions for estimation of double and triple integrals, respectively. The final section of this chapter describes the use of these functions along with the use of the newer functions, `integral`, `integral2`, and `integral3`, introduced in R2012a. These newer functions use vectorized adaptive quadrature and have proved to be more computationally robust, and so they will replace all other integration functions in a future release. This last section also introduces the `polyint` function for analytical integration of polynomials.

13.2 Rectangle Method

Recall that integration of a function $f(x)$ represents an area under the curve defined by $f(x)$, but we may not have the curve itself. Instead, we have some discrete points as shown in Fig. 13.1. These points can be spaced unevenly (Fig. 13.1a) or evenly (Fig. 13.1b).

The data pairs, (x_i, y_i), constitute some kind of dependence implicitly anyway, so that $y_i = f(x_i)$. Hence, the most obvious approach would be to represent a definite integral of Eq. (13.1), which represents the area under the imaginary curve passing through the data points, as a sum of $n-1$ rectangles with the widths $h_i = x_{i+1} - x_i$, $i = 1, 2, \ldots, n-1$, and heights of either $y_i = f(x_i)$ (Fig. 13.2a) or $y_{i+1} = f(x_{i+1})$ (Fig. 13.2b)

$$I^{\mathrm{LR}} = \sum_{i=1}^{n-1} f(x_i)h_i \quad \text{or} \quad I^{\mathrm{RR}} = \sum_{i=1}^{n-1} f(x_{i+1})h_i \tag{13.2}$$

These formulas are known as the *left rectangles* (LR) *formula* and the *right rectangles* (RR) *formula*, respectively. There is an obvious difference between the imaginary curve and the ladder-type approximation, resulting in an error. The obvious method to mitigate this error would be to have more points so that the difference $x_i - x_{i-1}$ is very small; however, this is a last resort. Other approaches can be used before this, and the most obvious one would be simply taking an average between I^{LR} and I^{RR}, which results in the trapezoidal rule (discussed next).

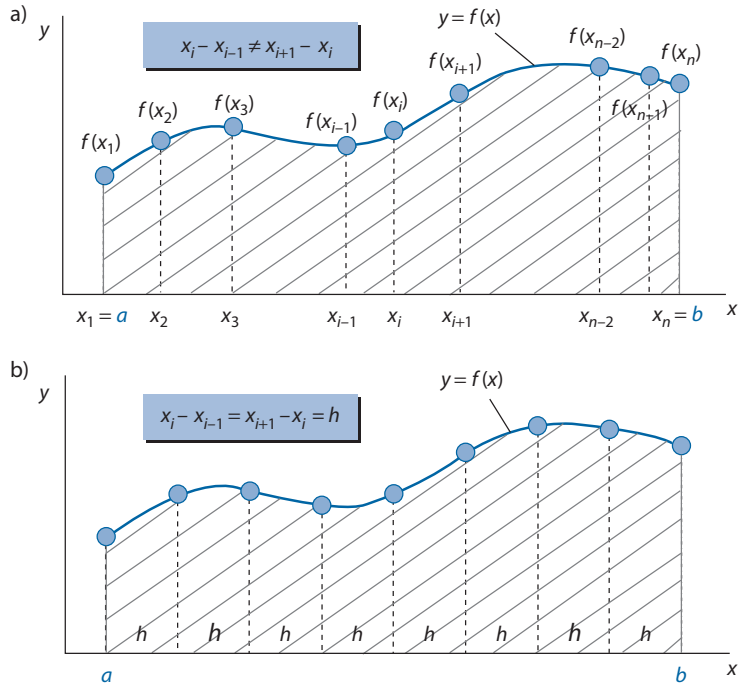


Fig. 13.1 Graphical representation of a definite integral of the function $f(x)$ with a) uneven and b) even spacing of the sample points.

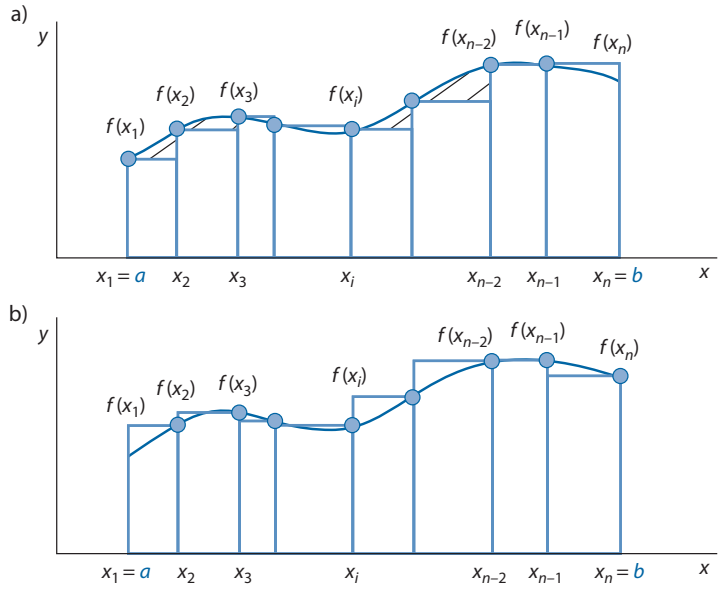


Fig. 13.2 a) Left and b) right rectangles approximations.

13.3 Trapezoidal Rule

The *trapezoidal rule* (also known as the trapezoid or trapezium rule) is a numerical integration technique derived from a linear interpolation of the function $f(x)$ (as opposed to the ladder-type interpolation in the previous section) and implying substituting an exact value of the integral with a sum of individual trapezoids' areas as shown in Fig. 13.3.

Looking at an arbitrary point on the curve x_i, $2 < i \leq n$, the integral to that point can be defined as $I(x_i)$, which is the area under the curve from x_1 to x_i. Similarly, the integral to the next arbitrary point can be defined as $I(x_{i+1})$, which is the area under the curve from x_1 to x_{i+1}. The equation for the single area approximated by a trapezoid ΔI_i^{TR} is given by

$$\Delta I_i^{\mathrm{TR}} = I(x_{i+1}) - I(x_i) = \int_{x_i}^{x_{i+1}} f^*(x)\mathrm{d}x + e_i = \frac{f(x_i) + f(x_{i+1})}{2} h_i \qquad (13.3)$$

where $f^*(x)$ is a piecewise linear interpolation of the real (unknown) function $f(x)$ and e_i is the error between the exact solution and the trapezoidal rule solution for a single trapezoid, that is, a local truncation error. As seen, Eq. (13.3) is an average of the two formulas of Eq. (13.2) for a single subinterval.

Applying the previous formula recursively to an approximation of the entire integral, from $x_1 = a$ to $x_n = b$, and assuming that for any $i = 1, 2, \ldots n-1$, $h_i = h = \text{const}$ (we will use this not-restricting-generality assumption to derive compact formulas), we get

$$I^{\mathrm{TR}}(x_n) = \int_a^b f^*(x)\mathrm{d}x = \frac{1}{2}h\left(f(a) + 2\sum_{i=1}^{n-1} f(a+ih) + f(b)\right)$$
$$+ \sum_{i=1}^{n-1} e_i = \frac{1}{2}h\left(f_1 + 2\sum_{i=2}^{n-1} f_i + f_n\right) + E \qquad (13.4)$$

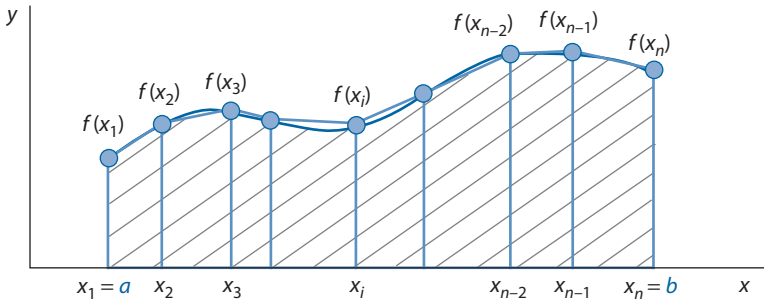


Fig. 13.3 Numerical integration using the trapezoidal rule.

Here, the error E between the exact and trapezoidal rule solutions for the entire interval $[a; b]$, the global truncation error, can be evaluated as

$$E = \sum_{i=1}^{n-1} e_i = \overline{e}(n-1) = \overline{e}\frac{b-a}{h} \tag{13.5}$$

where $\overline{e}$ is some average single-subinterval error. Note that the order of the global truncation error E (in terms of a grid spacing h) is always one unit less than that of the local truncation error e_i. Moreover, Eq. (13.5) is universal and applies to any method, not just to the trapezoidal method alone.

Now let us try to estimate these truncation errors, e_i and E. Assuming $I(x)$ is analytic, a Taylor series approximation can be made about x_i [Eq. (7.2)], yielding

$$I(x_{i+1}) = I(x_i) + hI'(x_i) + \frac{h^2}{2!}I''(x_i) + \frac{h^3}{3!}I'''(x_i) + HOT \tag{13.6}$$

By making obvious substitutions

$$I'(x_i) = f(x_i), \quad I''(x_i) = f'(x_i), \quad I'''(x_i) = f''(x_i) \tag{13.7}$$

we can rewrite Eq. (13.6) as

$$I(x_{i+1}) = I(x_i) + hf(x_i) + \frac{h^2}{2!}f'(x_i) + \frac{h^3}{3!}f''(x_i) + HOT \tag{13.8}$$

Next, using a simple two-point forward difference approximation for $f'(x_i)$ [Eq. (12.5)]

$$f'(x_i) \cong \frac{f(x_i+h) - f(x_i)}{h} - \frac{h}{2}f''(x_i) + HOT \tag{13.9}$$

and substituting it into Eq. (13.8), we obtain

$$I(x_{i+1}) = I(x_i) + hf(x_i) + \frac{h^2}{2!}\left(\frac{f(x_i+h) - f(x_i)}{h} - \frac{h}{2}f''(x_i) + HOT\right) + \frac{h^3}{3!}f''(x_i) + \cdots \tag{13.10}$$

Collecting the terms and noting that $x_{i+1} = x_i + h$ yields

$$I(x_{i+1}) = I(x_i) + \frac{h}{2}(f(x_i) + f(x_{i+1})) - \frac{h^3}{12}f''(x_i) + HOT \tag{13.11}$$

By comparing the terms in Eq. (13.11) with those of Eq. (13.3), we finally find that

$$e_i \cong -\frac{h^3}{12}f''(x_i) \tag{13.12}$$

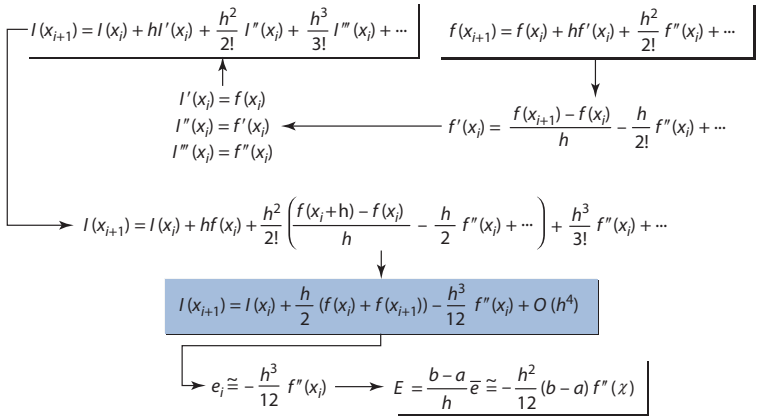


Fig. 13.4 Formal approach to estimate the truncation error for the trapezoidal rule.

Using the Mean Value Theorem, we may further write

$$\sum_{t=1}^{n-1} f''(x_i) = (n-1) f''(\chi) = \frac{b-a}{h} f''(\chi) \tag{13.13}$$

where χ is some representative point within the $[a; b]$ interval, and substitute the result into Eq. (13.12) and then into Eq. (13.5) to obtain an estimate of the error E for the entire interval $[a; b]$ as

$$E \cong \frac{h^2}{12}(b-a) f''(\chi) \tag{13.14}$$

For the sake of clarity, all transformations starting from Eq. (13.6) down to Eq. (13.14) are shown in Fig. 13.4. This helps to better understand the following script that automates the derivation of these formulas using the MATLAB Symbolic Math Toolbox:

```
%% Define symbolic variables
syms I_i I_iplus1 I_prime I_2prime I_3prime h
syms f_i f_iplus1 f_prime f_2prime
%% Introduce a Taylor series equation for the integral
Eq = -I_iplus1+I_i+h*I_prime+h^2*I_2prime/factorial(2)+ ...
                              h^3*I_3prime/factorial(3);
%% Make the first set of substitutes
Eq = subs(Eq,{I_prime I_2prime I_3prime},{f_i f_prime f_2prime});
%% Substitute function derivative with a forward difference approximation
Eq = subs(Eq,f_prime,(f_iplus1-f_i)/h-h*f_2prime/2);
%% Compute a single area under the curve
ans = solve(Eq,I_iplus1)-I_i;
%% Collect coefficients and display the result
r=collect(ans,h); pretty(r)
```

Running this script results in

```
/  f_2prime \   3  / f_i   f_iplus1 \
| - -------- | h +|  --- + -------- | h
\     12    /     \  2       2     /
```

showing resemblance to Eq. (13.11).

To summarize our findings,

1. The trapezoidal rule is based on integration of linear interpolation of the function.
2. The global truncation error associated with the trapezoidal rule for a fixed interval $[a; b]$ is proportional to h^2 [Eq. (13.14)]. Therefore, the trapezoidal rule is referred to as a second-order method.

For the record, if in Eq. (13.8) we leave only three terms, throwing away the term proportional to the second-order derivative, we will have the formula for the left rectangles along with an estimate of a local truncation error for this method, $e_i \cong (h^2/2)\, f'(x_i)$, which makes the global error of $E \cong -(h/2)(b-a)\, f'(\chi)$.

Hence, by utilizing a linear interpolation of the function as opposed to the ladder-type interpolation for the rectangle method, we improved the accuracy of numerical estimation of a definite integral by one order of h. This accuracy still depends on h, meaning that by increasing the number of data points within the same interval (decreasing h), we can decrease the error, but even with the same h, the second-order method ensures a better result. For example, if $h = 0.1$, then the global error for the rectangle rule will be of the order of 0.1, whereas for the trapezoidal formula it will be of the order of 0.01.

Now it becomes clear how to improve the accuracy even more—we should simply use the higher-order interpolation formulas. And that is exactly what is being done in the following two sections.

13.4 Simpson's 1/3 Rule

So far, we have used a ladder-type and linear interpolation of a function between the data points. Now let us proceed with the second-order interpolation, which results in the *Simpson's 1/3 rule*. Figure 13.5 illustrates our intentions, showing that we will now use double subintervals to produce a parabolic interpolation passing through every three consecutive data points, 1-2-3, 3-4-5, 5-6-7, etc.

A quadratic (second-order) polynomial interpolation can be represented as

$$f^*(x) = Kx^2 + Lx + M \tag{13.15}$$

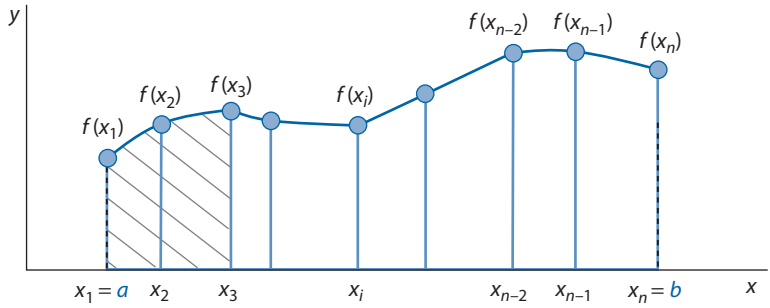


Fig. 13.5 Numerical integration using quadratic interpolation (Simpson's 1/3 rule).

Hence, for each three points (each double subinterval $[x_i;\ x_{i+2}]$, $i = 2k-1,\ k = 1,\ 2, \ldots,$ [(n-1)/2], where [] denotes the integer part) we need to determine the coefficients K, L, and M. The requirement is that at x_i, $x_{i+1} = x_i + h$, and $x_{i+2} = x_i + 2h$ the interpolation should pass through $f_i = f(x_i)$, $f_{i+1} = f(x_{i+1})$, and $f_{i+2} = f(x_{i+2})$. This results in the following three LAEs:

$$\begin{bmatrix} x_i^2 & x_i & 1 \\ (x_i+h)^2 & x_i+h & 1 \\ (x_i+2h)^2 & x_i+2h & 1 \end{bmatrix} \begin{bmatrix} K \\ L \\ M \end{bmatrix} = \begin{bmatrix} f_i \\ f_{i+1} \\ f_{i+2} \end{bmatrix} \tag{13.16}$$

Being resolved, this system yields

$$K = \frac{f_i - 2f_{i+1} + f_{i+2}}{2h^2}, \quad L = -\frac{3f_i - 4f_{i+1} + f_{i+2}}{2h} \quad \text{and} \quad M = f_i \tag{13.17}$$

Now that we have this second-order interpolation, the integral for each double subinterval may be estimated as

$$\Delta I_i^{S\frac{1}{3}} = \int_{x_i}^{x_i+2h} f^*(x)\mathrm{d}x + e_i = \frac{8}{3}h^3K + 2h^2L + 2hM + e_i \tag{13.18}$$

or substituting the coefficients of the interpolation [Eq. (13.17)] back into it as

$$\Delta I_i^{S\frac{1}{3}} = \frac{1}{3}h(f(x_i) + 4f(x_i+h) + f(x_i+2h)) + e_i \tag{13.19}$$

This numerical integration technique is credited to the mathematician Thomas Simpson and is therefore known as "*Simpson's 1/3 rule*" [because

of the 1/3 multiplier in Eq. (13.19)]. The following script employs Symbolic Math Toolbox and allows you to arrive at the same Eq. (13.19):

```
%% Define symbolic variables
syms x_i f_i f_iplus1 f_iplus2 h x
%% Solve for coefficients of a parabolic interpolation
A = [          x_i^2          x_i    1;
         (x_i+h)^2        x_i+h    1;
       (x_i+2*h)^2     x_i+2*h    1];
b= [f_i; f_iplus1; f_iplus2];
coef = A\b;
%% Compute the integral
dI = int([x^2, x, 1]*coef, x_i, x_i+2*h);
%% Display the result
pretty(dI)
```

It returns

```
h (f_i + 4 f_iplus1 + f_iplus2)
------------------------------
               3
```

To estimate local and global truncation errors, we need to use the Taylor series expansions in a manner similar to how we did in the previous section. Figure 13.6 shows how to do it step by step and utilizes the most accurate three-point centered difference approximation of the second-order derivative from the previous chapter [Eqs. (12.34) and (12.35)]. This results in

$$e_i \cong -\frac{h^5}{90} f^{(iv)}(x_i) \tag{13.20}$$

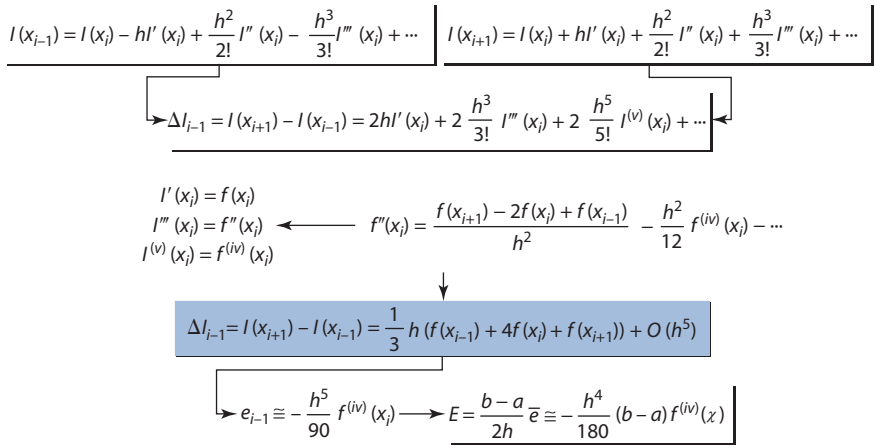


Fig. 13.6 Formal approach to estimate the truncation error for the Simpson's 1/3 rule.

and

$$E \cong -\frac{h^4}{180}(b-a)f^{(iv)}(\chi) \tag{13.21}$$

(where 180 instead of 90 in denominator occurred because of using a double subinterval (i.e. using $2h$ in denominator of Eq. (13.13))).

The following script allows you to do it automatically rather than by hand:

```
%% Define symbolic variables (assume middle point to be the i-th point)
syms I_i I_prime I_2prime I_3prime I_4prime I_5prime h
syms f_iminus1 f_i f_iplus1 f_prime f_2prime f_3prime f_4prime
%% Tylor series expansions for I_plus1 and I_minus1
Eq1=I_i+h*I_prime+h^2*I_2prime/factorial(2)+h^3*I_3prime/factorial(3)+ ...
        h^4*I_4prime/factorial(4)+h^5*I_5prime/factorial(5);
Eq2=I_i-h*I_prime+h^2*I_2prime/factorial(2)-h^3*I_3prime/factorial(3)+ ...
        h^4*I_4prime/factorial(4)-h^5*I_5prime/factorial(5);
Eq3=Eq1-Eq2; % Subtract I_minus1 from I_plus1
%% Make the first set of substitutes
Eq4=subs(Eq3,{I_prime,I_2prime,I_3prime,I_4prime,I_5prime}, ...
        {f_i,f_prime,f_2prime,f_3prime,f_4prime});
%% Substitute central diff. approximation for the 2nd-order derivative
Eq5=subs(Eq4,f_2prime,(f_iminus1-2*f_i+f_iplus1)/h^2-1/12*h^2*f_4prime);
%% Collect coefficients and display the result
r=collect(Eq5,h); pretty(r)
```

It returns

```
             5                                                  -1
(- f_4prime h  + (120 f_i + 30 f_iplus1 + 30 f_iminus1) h) 90
```

Applying Eq. (13.19) recursively to all double subintervals within $[a; b]$, we get a formula for the *composite Simpson's 1/3 rule*

$$\begin{aligned} I^{S\frac{1}{3}} &= \int_a^b f(x)\mathrm{d}x \\ &= \frac{1}{3}h\left(f(a) + 4\sum_{\substack{i=1 \\ \text{odd } i}}^{n-1} f(a+ih) + 2\sum_{\substack{i=2 \\ \text{even } i}}^{n-2} f(a+ih) + f(b)\right) + E \end{aligned} \tag{13.22}$$

or

$$I^{S\frac{1}{3}}(x_n) = \frac{1}{3}h(f_1 + 4f_2 + 2f_3 + 4f_4 + 2f_5 + \cdots + 2f_{n-2} + 4f_{n-1} + f_n) + E \tag{13.23}$$

Note that Eqs. (13.22) and (13.23) assume an odd n, that is, we have an even number of subintervals (that is why k in definition following Eq. (13.15) was limited to the integer part of $(n-1)/2$). If we happen to have one extra subinterval, that is, if we have an even number of data points n to start with, then the trapezoidal rule needs to be applied to either the

very first or the very last subinterval before proceeding with the Simpson's 1/3 rule in the regular manner.

To summarize, increasing the order of interpolation from linear to quadratic (by a unit) allows you to improve the accuracy of integration by two orders of h, bringing the global truncation error to be of the order of h^4. Hence, the Simpson's 1/3 rule is referred to as a fourth-order method.

Should we expect even better accuracy if we further increase the order of interpolation? The following section provides you with an answer to this question.

13.5 Simpson's 3/8 Rule

Following the same routine introduced in the two previous sections to increase the accuracy of numerical integration, the *Simpson's 3/8 rule* employs a third-order polynomial interpolation technique. As shown in Fig. 13.7, triple subintervals—1-2-3-4, 4-5-6-7, etc. ($[x_i;x_{i+3}]$, $i = 3k-2$, $k = 1, 2, \ldots,$ $[(n-1)/3]$) are used instead of the double subintervals of Fig. 13.5, $[(n-1)/2]$.

To save space, let us employ the following script to obtain the numerical integration formula in this case:

```
%% Define symbolic variables
syms x_i f_i f_iplus1 f_iplus2 f_iplus3 h x
%% Solve for coefficients of a cubic interpolation
A = [          x_i^3           x_i^2         x_i    1;
          (x_i+h)^3       (x_i+h)^2       x_i+h    1;
        (x_i+2*h)^3     (x_i+2*h)^2     x_i+2*h    1;
        (x_i+3*h)^3     (x_i+3*h)^2     x_i+3*h    1];
b= [f_i; f_iplus1; f_iplus2; f_iplus3];
coef = A\b;
%% Compute the integral
dI = int([x^3, x^2, x, 1]*coef, x_i, x_i+3*h);
%% Display the result
pretty(dI)
```

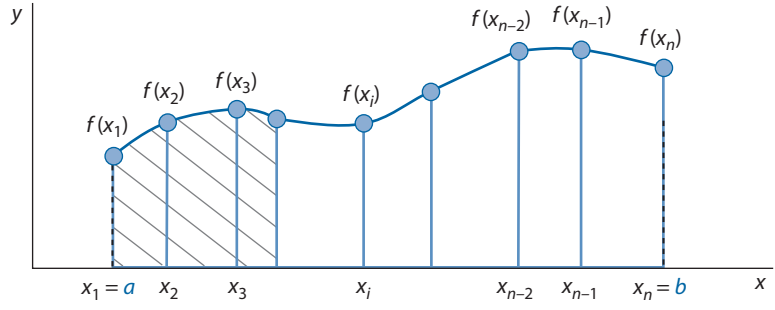


Fig. 13.7 Numerical integration using cubic interpolation (Simpson's 3/8 rule).

This script solves for the coefficients of the third-order polynomial interpolation between four points, (x_i, f_i), $(x_i + h, f_{i+1})$, $(x_i + 2h, f_{i+2})$, and $(x_i + 3h, f_{i+3})$, and then analytically integrates this interpolation between x_i and $x_i + 3h$ to yield

```
h (f_i + 3 f_iplus1 + 3 f_iplus2 + f_iplus3) 3
-----------------------------------------
                    8
```

Therefore,

$$\Delta I_i^{S\frac{3}{8}} = \int_{x_i}^{x_i+3h} f^*(x)\mathrm{d}x + e_i = \frac{3}{8}h(f(x_i) + 3f(x_i + h) + 3f(x_i + 2h) + f(x_i + 3h)) + e_i \tag{13.24}$$

To estimate a local truncation error, e_i, we need to perform Taylor series expansion computations similar to those shown in Figs. 13.4 and 13.6, which results in

$$e_i \cong -\frac{3h^5}{80}f^{(iv)}(x_i) \tag{13.25}$$

Unexpectedly, this error happens to be of the same order as that of the Simpson's 1/3 rule [Eq. (13.21)]. Hence, increasing the order of interpolation from quadratic to cubic did not lead to significant improvement of accuracy, as it took place when the order on interpolation was increased from linear to quadratic.

The composite Simpson's 3/8 rule is still a fourth-order method with

$$I^{S\frac{3}{8}}(x_n) = \frac{3}{8}h(f_1 + 3f_2 + 3f_3 + 2f_4 + 3f_5 + \cdots + 3f_{n-2} + 3f_{n-1} + f_n) + E \tag{13.26}$$

and

$$E \cong -\frac{h^4}{80}(b - a)f^{(iv)}(\chi) \tag{13.27}$$

Similar to Eq. (13.23), the number of subintervals in Eq. (13.26) is supposed to be divisible by three. If, however, the number of data points is such that there are one or two extra subintervals, the trapezoidal or Simpson's 1/3 rule should be applied to this subinterval(s), respectively.

13.6 Newton–Cotes Formulas

In the previous sections, it was demonstrated that we can increase the accuracy of numerical integration by employing the higher-order interpolation formulas. These interpolation formulas utilize m subintervals and

produce an mth-order polynomial from $m+1$ data points. Being integrated within the corresponding range (covering these $m+1$ data points), these interpolations result in the *Newton–Cotes integration closed formulas*. The Newton–Cotes formulas are represented as

$$\int_{x_i}^{x_i+mh} f^*(x)\mathrm{d}x = \alpha h(w_1 f(x_i) + w_2 f(x_{i+1}) + w_3 f(x_{i+2}) + \cdots + w_{m+1} f(x_{i+m})) \tag{13.28}$$

where α is the multiplier, and w_i, $i = 1, 2, \ldots, m+1$ are the weighting factors. Obviously, the trapezoidal rule and two Simpson's formulas are the members of the Newton–Cotes integration formulas. Table 13.1 presents them and a few more formulas in the standardized form.

Note that, as was the case with the Simpson's 1/3 and Simpson's 3/8 rules, the five- and six-point formulas have the same order error. This general characteristic holds for the higher-point formulas as well and results in the even number of subintervals (odd number of points) formulas (for example, Simpson's 1/3 rule and Boole's rule) being the methods of preference.

For the higher number of subintervals, some of coefficients w_i may become negative, degrading the accuracy due to round-off errors. Hence, in engineering practice, the higher-order formulas are rarely used, so that it is the Simpson's rules that are used in most applications. That is why MATLAB offers only two functions, `trapz` (`cumtrapz`) and `quad`, realizing the trapezoidal and Simpson's rules, respectively (to be considered in Sec. 13.9).

In addition to that, the Newton–Cotes integration formulas come with the estimates of errors, so if you have the luxury of getting as many data points as you need (for instance, when the function $f(x)$ is given analytically or $f(x_i)$ being the outputs of some script or model), you can always control (improve) the accuracy by adjusting the grid spacing h appropriately. That allows satisfying any error tolerance requirement.

Gauss quadrature (*quadrature* is an old term for the process of measuring areas) is another attractive alternative to the Newton–Cotes integration formulas and will be presented in Sec. 13.8. Before we proceed with considering

Table 13.1 Weighting Coefficients and Errors for the Newton–Cotes Closed Formulas

Number of Subintervals m	Multiplier α	Weighting Factors w_i	Order of Error E	Known as
1	1/2	1 1	h^2	Trapezoidal rule
2	1/3	1 4 1	h^4	Simpson's 1/3 rule
3	3/8	1 3 3 1	h^4	Simpson's 3/8 rule
4	2/45	7 32 12 32 7	h^6	Boole's rule
5	5/288	19 75 50 50 75 19	h^6	—

this alternative though, let us have a look at the multiple-application scheme (introduced in Sec. 12.6), which can be used for either improving the accuracy of Newton–Cotes integration formulas without introducing new data points or adjusting the current grid spacing h, based on the quick estimates of the local truncation error e_i to keep it within the given tolerance.

13.7 Multiple-Application Scheme

Consider the same rule, say, the trapezoidal or Simpson's 1/3 rule, applied to compute an integral using two different grid spacings, h_1 and h_2 ($h_1 > h_2$). Then we can write

$$I = I(h_1) + E(h_1) = I(h_2) + E(h_2) \tag{13.29}$$

Assume that $f''(\chi)$ (or $f^{(iv)}(\chi)$ for the Simpson's rule) is about the same for both grids. Then, using data from the error column of Table 13.1, we can write

$$E(h_1) \cong E(h_2)\left(\frac{h_1}{h_2}\right)^{2m} \tag{13.30}$$

(for the trapezoidal rule, $m = 1$; for the Simpson's 1/3 rule, $m = 2$; etc.).

Substituting Eq. (13.30) into Eq. (13.29) and resolving the latter one with respect to $E(h_2)$ yields

$$E(h_2) \cong \frac{I(h_1) - I(h_2)}{1 - (h_1/h_2)^{2m}} \tag{13.31}$$

[compare with Eq. (12.39)]. Therefore, without applying formulas for truncation errors, like Eqs. (13.14) or (13.21), we were able to estimate the error of the numerical integral with a thinner grid, $E(h_2)$, so that now we can enjoy a more accurate estimate

$$\begin{aligned} I = I(h_2) + E(h_2) &= I(h_2) + \frac{I(h_2) - I(h_1)}{(h_1/h_2)^{2m} - 1} \\ &= \frac{(h_1/h_2)^{2m} I(h_2) - I(h_1)}{(h_1/h_2)^{2m} - 1} \end{aligned} \tag{13.32}$$

[compare with Eq. (12.40)]. In the special case when we are using the original grid and double subintervals, that is, when $h_2 = (1/2)h_1 \equiv h$, the Richardson extrapolation, given by Eq. (13.32), yields

$$I = I(h) + \frac{I(h) - I(2h)}{4^m - 1} = \frac{4^m I(h) - I(2h)}{4^m - 1} \tag{13.33}$$

Specifically, for the trapezoidal and Simpson's 1/3 rules

$$\hat{I}^{\mathrm{TR}} = \frac{4}{3} I^{\mathrm{TR}}(h) - \frac{1}{3} I^{\mathrm{TR}}(2h) \quad \text{and} \quad \hat{I}^{S\frac{1}{3}} = \frac{16}{15} I^{S\frac{1}{3}}(h) - \frac{1}{15} I^{S\frac{1}{3}}(2h) \tag{13.34}$$

where $\hat{I}^{\mathrm{TR}}$ and $\hat{I}^{S\frac{1}{3}}$ are more accurate integrals compared with the original ones, I^{TR} and $I^{S\frac{1}{3}}$, respectively. Using a formal approach (similar to what was shown in Figs. 13.4 and 13.6, but using more terms in the Taylor series expansion), it can be shown that the error of this multiple-application formula is two orders of h higher than that of the original rule. Therefore, $\hat{I}^{\mathrm{TR}}$ features the error of the order of h^4 (compared with h^2 for I^{TR}) and $\hat{I}^{S\frac{1}{3}}$ is accurate within h^6 (compared with h^4 for $I^{S\frac{1}{3}}$).

In his online book, Cleve Moler developed an interactive GUI, `quadgui`, to visualize how the MATLAB `quad` function, exploiting the Simpson's 1/3 rule, controls the error of numerical integration. Figure 13.8 shows several snapshots of this GUI. As seen, the integration of some function starts with computing a minimum number of data points (5) to enable application of the Simpson's 1/3 rule twice, on two double subintervals as shown in Fig. 13.8a. On the other hand, the same integral can be estimated using only three points, first, third, and fifth, that is, on a rougher grid. Given that, in this specific case, $|I(h) - I(2h)|$ happens to be greater than some specified error tolerance ε, the `quad` function adds two additional data points halving the first two subintervals (Fig. 13.8b).

The entire *step-halving procedure* is then repeated again and, because the error tolerance is still not met, the `quad` function keeps introducing the new

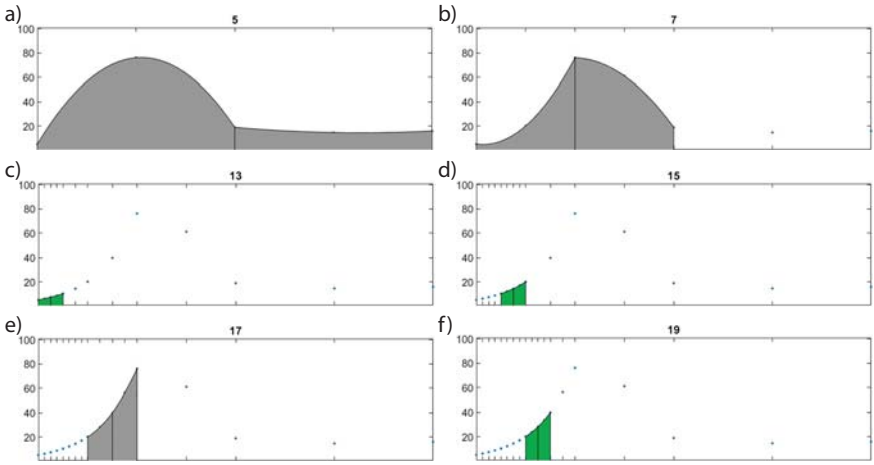


Fig. 13.8 Employing a multiple-application scheme to keep the integration error below the tolerance.

data points halving subintervals until

$$|I(h) - I(2h)| \leq \varepsilon \tag{13.35}$$

With 8 additional data points added (Fig. 13.8c), the value of the integral is now computed according to the second formula in Eq. (13.34). Next, the `quad` function adds two more data points to halve the following two subintervals and proceeds with the multiple-application scheme on them. In the case shown in Fig. 13.8, the error tolerance happens to be met again (Fig. 13.8d). At the very next step (Fig. 13.8e), the inequality of Eq. (13.35) is violated again. Hence, this latter step needs to be repeated on a tinnier grid (Fig. 13.8f), and so on.

13.8 Gauss Quadrature

The Newton–Cotes formulas considered in the previous sections were based on an evenly spaced grid. Consequently, the location of the grid nodes used in these equations was predetermined or fixed. But what if you have the luxury to compute the function values on a set of arbitrarily spaced points? Then, by positioning these points wisely, we could "balance" local negative and positive errors e_i so that the accuracy of the overall estimate could be sufficiently improved.

Gauss quadrature is the name for one class of techniques used to implement such a strategy. Let us illustrate this technique in the following example. Say we want to develop a formula to estimate an integral by only using two predetermined points, which are the endpoints of interval $[a; b]$, using the following approximation:

$$\int_a^b f(x)\mathrm{d}x \cong c_a f(a) + c_b f(b) \tag{13.36}$$

The two unknown coefficients c_a and c_b of Eq. (13.36) can be determined from the requirement that this formula must yield exact solutions for $f(x)$ being 1) a constant and 2) a straight line. These two conditions yield two LAEs

$$c_a + c_b = \int_a^b 1\mathrm{d}x = b - a \quad \text{and} \quad c_a a + c_b b = \int_a^b x\mathrm{d}x = \frac{b^2 - a^2}{2} \tag{13.37}$$

From these equations, it follows that

$$c_a = c_b = \frac{b - a}{2} \tag{13.38}$$

which gives the already known trapezoidal rule [Eq. (13.3)].

Now, let us free the location of two points in Eq. (13.36), so that

$$\int_a^b f(x)\mathrm{d}x \cong c_1 f(x_1) + c_2 f(x_2) \tag{13.39}$$

Here, in addition to coefficients c_1 and c_2, we have two more varied parameters x_1 and x_2, $a \le x_1 \le x_2 \le b$. To simplify mathematics and to make the formulation as general as possible, let us map the original domain $[a; b]$ to the new canonical domain $[-1; 1]$, that is, let us assume that $a = -1$ and $b = 1$, so that $-1 \le x_1 \le x_2 \le 1$. The objective is to find the "optimum" location of points x_1 and x_2 along with the corresponding weighting coefficients c_1 and c_2. For instance, we may request that Eq. (13.39) must yield an exact solution for two more functions, that is, 1) a constant, 2) a linear function, 3) a parabolic function, and 4) a cubic function, which results in four equations

$$\begin{aligned}
c_1 f(x_1) + c_2 f(x_2) &= c_1 + c_2 = \int_{-1}^{1} 1\mathrm{d}x = 2,\\
c_1 f(x_1) + c_2 f(x_2) &= c_1 x_1 + c_2 x_2 = \int_{-1}^{1} x\mathrm{d}x = 0,\\
c_1 f(x_1) + c_2 f(x_2) &= c_1 x_1^2 + c_2 x_2^2 = \int_{-1}^{1} x^2\mathrm{d}x = \frac{2}{3},\\
c_1 f(x_1) + c_2 f(x_2) &= c_1 x_1^3 + c_2 x_2^3 = \int_{-1}^{1} x^3\mathrm{d}x = 0
\end{aligned} \tag{13.40}$$

These equations can be conveniently resolved using the Symbolic Math Toolbox as follows:

```
%% Define symbolic variables
syms c1 c2 x1 x2
%% Define four algebraic equations
Eq1= c1+c2-2;
Eq2= c1*x1+c2*x2;
Eq3= c1*x1^2+c2*x2^2-2/3;
Eq4= c1*x1^3+c2*x2^3;
%% Solve equations
coef=solve(Eq1,Eq2,Eq3,Eq4,c1,c2,x1,x2);
%% Display the results
Weights = [coef.c1(1) coef.c2(1)];
Location = [coef.x1(1) coef.x2(1)];
```

```
pretty(Weights)
pretty(Location)
WeightsD=eval(Weights)
LocationD=eval(Location)
```

This script results in the weighting factors and locations of two points as

```
(1, 1)
/ sqrt(3)    sqrt(3) \
| -------, - ------- |
\    3          3    /
WeightsD =  1          1
LocationD = 0.5774    -0.5774
```

i.e.,

$$c_1 = c_2 = 1 \quad \text{and} \quad x_1 = -x_2 = -3^{-0.5} \approx -0.5773503 \tag{13.41}$$

which can be substituted back into Eq. (13.39) to yield the *two-point Gauss–Legendre formula*

$$\int_{-1}^{1} f(x)\mathrm{d}x \cong f\left(\frac{-1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right) \tag{13.42}$$

In the more general case, you can use as many points as we want

$$\int_{-1}^{1} f(x)\mathrm{d}x \cong \sum_{i=1}^{n} c_i f(x_i) \tag{13.43}$$

finding $2n$ varied parameters by providing the exact solution to up to the $(2n-1)$th-order polynomial. For example, with three points, you can obtain exact solutions for up to the fifth-order polynomial. The following script, which is similar to the previous one, allows you to obtain the locations of these three points and their corresponding weights:

```
%% Define symbolic variables
syms c1 c2 c3 x1 x2 x3 x
%% Define six algebraic equations
Eq1= sum([c1 c2 c3].*[1 1 1])-2;
Eq2= sum([c1 c2 c3].*[x1 x2 x3]);
Eq3= sum([c1 c2 c3].*[x1^2 x2^2 x3^2])-int(x^2,-1,1);
Eq4= sum([c1 c2 c3].*[x1^3 x2^3 x3^3]);
Eq5= sum([c1 c2 c3].*[x1^4 x2^4 x3^4])-int(x^4,-1,1);
Eq6= sum([c1 c2 c3].*[x1^5 x2^5 x3^5]);
%% Solve equations
coef=solve(Eq1,Eq2,Eq3,Eq4,Eq5,Eq6,c1,c2,c3,x1,x2,x3);
%% Display the results
Weights =  [coef.c1(1) coef.c2(1) coef.c3(1)];
```

```
Location = [coef.x1(1) coef.x2(1) coef.x3(1)];
pretty(Weights), pretty(Location)
WeightsD=eval(Weights)
LocationD=eval(Location)
```

This script returns

```
/ 5  5  8 \
| -, -, - |
\ 9  9  9 /

/ sqrt(15)    sqrt(15)   \
| --------, - --------, 0 |
\     5           5      /
WeightsD =   0.5556   0.5556   0.8889
LocationD = -0.7746   0.7746   0
```

The values for c's and x's for up to and including the seven-point formula are summarized in Table 13.2, whereas Fig. 13.9 shows a graphical representation of the location and weighting coefficients for the two-, three-, and six-point schemes. The estimate of an error of the n-point Gauss quadrature rule for the interval $[a; b]$ is:

$$E(\Delta, n) = \frac{\Delta^{2n+1}(n!)^4}{(2n+1)((2n)!)^3} f^{(2n)}(\chi) = \Delta^{2n+1} E(n), \quad a < \chi < b \qquad (13.44)$$

($\Delta = b - a$). Comparison of the Gauss quadrature rules to Newton–Cotes formulas (Table 13.1) indicates the superiority of the former ones,

Table 13.2 Arguments, Weighting Factors, and Errors for the Gauss Quadrature Formulas

Number of Points n	Function Argument x_i	Weight Factor c_i	Error $E(n)$
2	$x_1 = -x_2 = -0.577350269$	$c_1 = c_2 = 1$	$2.3 \times 10^{-4} f^{(iv)}(\overline{X})$
3	$x_1 = -x_3 = -0.774596669$ $x_2 = 0$	$c_1 = c_3 = 0.5555556$ $c_2 = 0.8888889$	$5.0 \times 10^{-7} f^{(vi)}(\overline{X})$
4	$x_1 = -x_4 = -0.861136312$ $x_2 = -x_3 = -0.339981044$	$c_1 = c_4 = 0.3478548$ $c_2 = c_3 = 0.6521452$	$5.6 \times 10^{-10} f^{(viii)}(\overline{X})$
5	$x_1 = -x_5 = -0.906179846$ $x_2 = -x_4 = -0.538469931$ $x_3 = 0$	$c_1 = c_5 = 0.2369969$ $c_2 = c_4 = 0.4786287$ $c_3 = 0.5688889$	$3.9 \times 10^{-13} f^{(x)}(\overline{X})$
6	$x_1 = -x_6 = -0.932469514$ $x_2 = -x_5 = -0.661209386$ $x_3 = -x_4 = -0.238619186$	$c_1 = c_6 = 0.1713245$ $c_2 = c_5 = 0.3607616$ $c_3 = c_4 = 0.4679139$	$1.9 \times 10^{-16} f^{(xii)}(\overline{X})$
7	$x_1 = -x_7 = -0.949107912$ $x_2 = -x_6 = -0.741531186$ $x_3 = -x_5 = -0.405845151$ $x_4 = 0$	$c_1 = c_7 = 0.1294850$ $c_2 = c_6 = 0.2797054$ $c_3 = c_5 = 0.3818301$ $c_4 = 0.4179592$	$6.5 \times 10^{-20} f^{(xiv)}(\overline{X})$

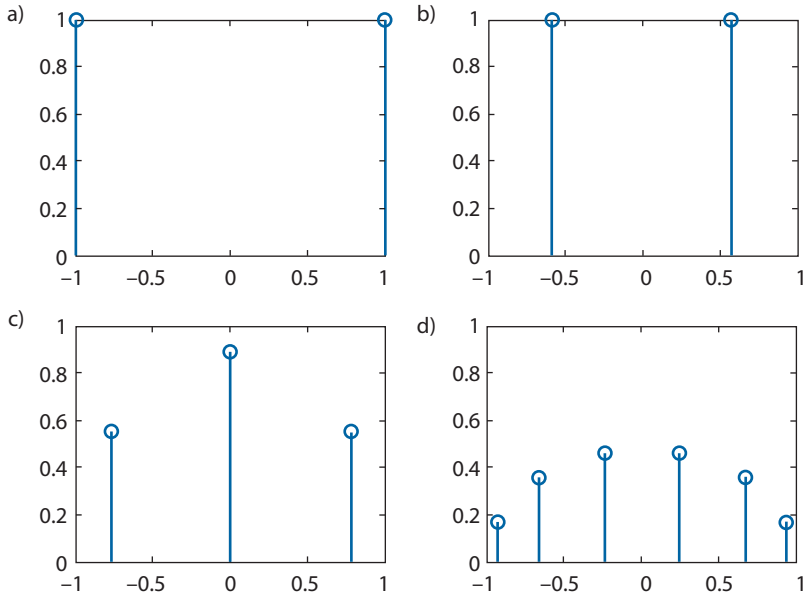


Fig. 13.9 Visualization of the location and weight factors of the sampling points for a) the trapezoidal rule and b) Gauss two-, c) three-, and d) six-point schemes.

meaning that much higher accuracy can be achieved using fewer data points (once again, in the case where we have the luxury to pick any point we want).

Now, let us go back and show how to map any arbitrary domain $[a; b]$ to the universal domain $[-1; 1]$ that the formulas of Table 13.2 were developed for. This can be accomplished by replacing the original argument x with some new argument $\eta(t)$, so that

$$\int_a^b f(x)\mathrm{d}x = \int_{\eta^{-1}(a)}^{\eta^{-1}(b)} f(\eta(t))\eta_t'\mathrm{d}t \tag{13.45}$$

This new argument $\eta(t)$ should be a monotonic function of t and satisfy boundary requirements $\eta^{-1}(a) = -1$ and $\eta^{-1}(b) = 1$, where η^{-1} denotes an inverse of η. Obviously, the simplest mapping is a linear one given by

$$x = \eta(t) = \frac{1}{2}(b-a)t + \frac{1}{2}(b+a) \tag{13.46}$$

(when $t = -1$, $x = a$, and when $t = 1$, $x = b$). In this case

$$\eta_t'(t) = \frac{1}{2}(b-a) \tag{13.47}$$

and therefore

$$\int_a^b f(x)\mathrm{d}x = \frac{1}{2}(b-a)\int_{-1}^{1} f\left(\frac{1}{2}(b-a)t + \frac{1}{2}(b+a)\right)\mathrm{d}t$$

$$\cong \frac{1}{2}(b-a)\sum_{i=1}^{n} c_i f\left(\frac{1}{2}(b-a)t_i + \frac{1}{2}(b+a)\right) \tag{13.48}$$

A numerical example will be provided in the following section.

If we attempt to apply the multiple-application scheme described in Sec. 13.7 for the Newton–Cotes formulas to the Gauss quadrature rules, it will not work. The reason is that when subdividing the original interval $[-1; 1]$, the Gauss evaluation points of the new subintervals will never coincide with the previous evaluation points (except at zero for odd numbers), and thus the integrand will need to be evaluated at every new point.

Gauss–Kronrod formulas are extensions of Gauss quadrature formulas constructed by adding $n+1$ points to an original n-point Gauss quadrature rule, so that the function values at these n points can be reused to compute the higher-order estimates. The difference between the n-point Gauss quadrature and its $(2n+1)$-point Kronrod extension $\left|I_n^G - I_{2n+1}^K\right|$ is used to estimate the local error and adjust the integration step in a manner similar to that shown in Fig. 13.8.

A popular example, realized in the MATLAB function `quadgk`, combines a seven-point Gauss rule presented in the last row of Table 13.2 (Fig. 13.10a) with a 15-point Kronrod rule. This 15-point formula utilizes seven function evaluations of the seven-point Gauss rule (with the different weights, though) and adds eight additional points as shown in Table 13.3 (the seven-point Gauss rule arguments are shown in the first column boldface). Figure 13.10b portraits the 15-point Kronrod rule overlaid over the seven-point Gauss rule.

As discussed earlier, the n-point Gauss quadrature rules are constructed to yield an exact result for up to the $(2n-1)$th-order polynomial. Say with only two points, the Gauss quadrature portrait looks as presented in Fig. 13.9b. The trapezoidal rule also uses two points, but it is designed using another paradigm, so its portrait given in Fig. 13.9a is different from that of Fig. 13.9b. Therefore, other approaches in choosing the locations x_i and weights c_i, $i = 1, 2, \ldots, n$ in Eq. (13.43) are also possible.

It turns out that the Gauss quadrature nodes are the roots of the Legendre polynomials $P_n(x)$

$$P_n(x) = \frac{1}{2^n n!}\frac{\mathrm{d}^n}{\mathrm{d}x^n}(x^2-1)^n \tag{13.49}$$

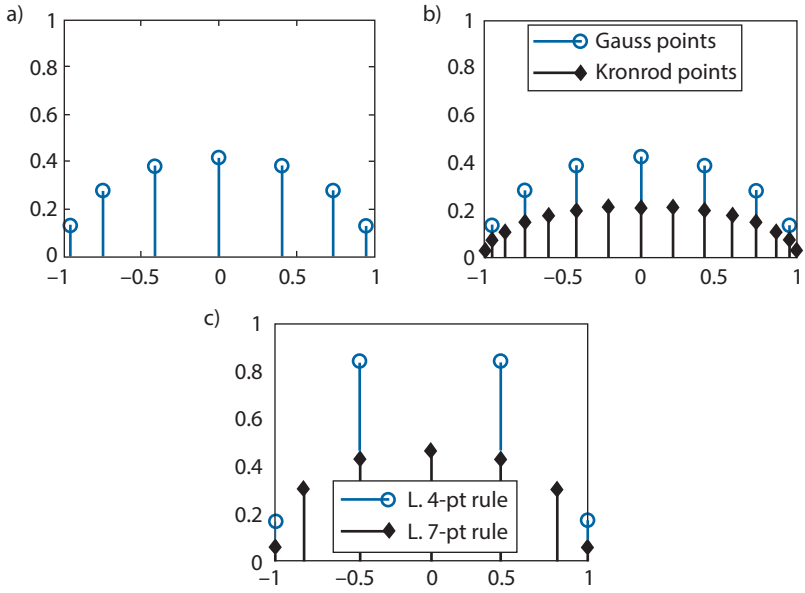


Fig. 13.10 Visualization of the argument location and weight factors for a) seven-point Gauss quadrature rule, b) Gauss–Kronrod 7–15 pair, and c) Lobatto 4–7 pair.

(note, this is an explicit formula compared with the recursive definition of Table 7.16). The Gauss quadrature weights can also be expressed via $P_n(x)$ as

$$c_i = \frac{2}{(1 - x_i^2)P_n'(x_i)^2} \tag{13.50}$$

For example,

$$P_0(x) = 1, \quad P_1(x) = x, \quad P_2(x) = \frac{1}{2}(3x^2 - 1), \quad P_3(x) = \frac{1}{2}(5x^3 - 3x), \quad \text{etc.} \tag{13.51}$$

Table 13.3 The 15-Point Kronrod Rule Points

Function Argument x_i	Weight Factor c_i
$x_1 = -x_{15} = -0.991455372$	$c_1 = c_{15} = 0.022935322$
$x_2 = -x_{14} = \mathbf{-0.949107912}$	$c_2 = c_{14} = 0.063092093$
$x_3 = -x_{13} = -0.864864423$	$c_3 = c_{13} = 0.104790010$
$x_4 = -x_{12} = \mathbf{-0.741531186}$	$c_4 = c_{12} = 0.140653260$
$x_5 = -x_{11} = -0.586087236$	$c_5 = c_{11} = 0.169004727$
$x_6 = -x_{10} = \mathbf{-0.405845151}$	$c_6 = c_{10} = 0.190350578$
$x_7 = -x_9 = -0.207784955$	$c_7 = c_9 = 0.204432940$
$x_8 = \mathbf{0}$	$c_8 = 0.209482141$

(Fig. 13.11) and, therefore, the arguments for the two-point and three-point rules (Table 13.2) are

$$x_{1;2} = \pm\frac{1}{\sqrt{3}} \quad \text{and} \quad x_{1;2;3} = \left\{0, \pm\sqrt{\frac{3}{5}}\right\} \tag{13.52}$$

respectively. According to Eq. (13.50), the weight factors for the two-point and three-point rules are

$$c_{1;2} = \frac{2}{(1-x_i^2)9x_i^2} = 1 \text{ and } c_{1;2;3} = \frac{8}{9(1-x_i^2)(5x_i^2-1)^2} = \left\{\frac{8}{9}, \frac{5}{9}, \frac{5}{9}\right\} \tag{13.53}$$

That is why the Gauss quadrature is often referred to as the *Gauss–Legengre quadrature*.

Following the same idea, the nodes x_i in Eq. (13.43) may be chosen to be the roots of other special polynomials: Jacobi, Chebyshev, Laguerre, and Hermite (see Table 7.16), and that is why *Gauss–Jacobi*, *Gauss–Chebyshev*, *Gauss–Laguerre*, and *Gauss–Hermite* quadratures also exist (for the sake of accuracy, the integrand in this case is represented as $w(x)f(x)$, with $w(x)$ being some weighting function, like $(1-x^2)^{-0.5}$, e^{-x}, or $\exp(-x^2)$, and the interval may differ from that of $[-1; 1]$). These quadratures have some special applications. For instance, in recent years, they have been widely used in the *pseudrospectral methods* for optimizing a trajectory of dynamic system, described by ordinary differential equations.

MATLAB does not have the special functions for these types of quadratures, but it features one more function realizing another adaptive quadrature (like the Gauss–Kronrod 7–15 pair). The *Gauss–Lobatto* or simply Lobatto quadrature is similar to the Gauss quadrature, except that two arguments (integration points) in Eq. (13.43) are fixed and represent the endpoints of the integration interval

$$\int_{-1}^{1} f(x)\mathrm{d}x \cong I_n^L = c_1 f(-1) + \sum_{i=2}^{n-1} c_i f(x_i) + c_n f(1) \tag{13.54}$$

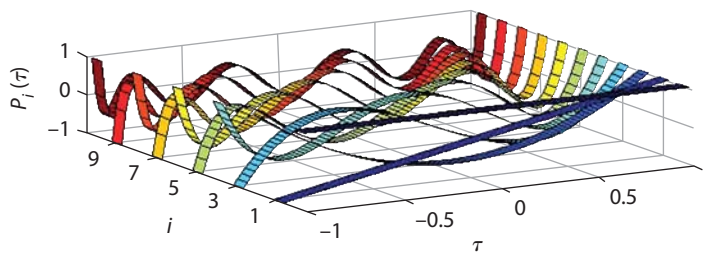


Fig. 13.11 Legendre polynomials $P_i(\tau)$, $i = 0, 1, \ldots, 10$, $\tau \in [-1; 1]$.

The remaining nodes x_i, $i = 2, 3, \ldots, n-1$ are defined as the roots of $P'_{n-1}(x)$ [Eq. (13.49)]. Finding n varied parameters, unknown weighting coefficients c_i, $i = 1, 2, \ldots, n$, is accomplished by the requirement to provide an exact solution to up to the $(n-1)$th-order polynomial, similar to that of Eq. (13.40).

For example, consider $n = 4$. In this case, $P_3(x) = (1/2)(5x^3 - 3x)$ [Eq. (13.51)], $P'_3(x) = (3/2)(5x^2 - 1)$, and therefore

$$x_{2;3} = \pm\frac{1}{\sqrt{5}} \tag{13.55}$$

Owing to symmetry, $c_4 = c_1$ and $c_3 = c_2$, so that for two unknown coefficients c_1 and c_2, we can write

$$2(c_1 f(x_1) + c_2 f(x_2)) = 2(c_1 + c_2) = \int_{-1}^{1} 1\mathrm{d}x$$

$$2(c_1 f(x_1) + c_2 f(x_2)) = 2\left(c_1 + c_2\frac{1}{5}\right) = \int_{-1}^{1} x^2\mathrm{d}x = \frac{2}{3} \tag{13.56}$$

which results in $c_1 = 1/6$ and $c_2 = 5/6$. Therefore, the four-point Lobatto quadrature rule can be written as

$$I_4^L = \frac{1}{6}(f(-1) + f(1)) + \frac{5}{6}\left(f\left(-\frac{1}{\sqrt{5}}\right) + f\left(\frac{1}{\sqrt{5}}\right)\right) \tag{13.57}$$

Similarly, for, say, a seven-point Lobatto quadrature we may obtain

$$I_7^L = \frac{11}{210}(f(-1) + f(1)) + \frac{72}{245}\left(f\left(-\sqrt{\frac{2}{3}}\right) + f\left(\sqrt{\frac{2}{3}}\right)\right) + \frac{125}{294}\left(f\left(-\frac{1}{\sqrt{5}}\right) + f\left(\frac{1}{\sqrt{5}}\right)\right) + \frac{16}{35}f(0) \tag{13.58}$$

Notably the seven-point rule uses the four function values of the four-point rule, thus increasing computational effectiveness (Fig. 13.10c). The following script utilizes the MATLAB Symbolic Math Toolbox to prove the solution:

```
%% Define symbolic variables
syms c1 c2 c3 c4 x
%% Define four algebraic equations
Eq1= sum([c1 c2 c3 c4].*[2 2 2 1])-2;
Eq2= sum([c1 c2 c3 c4].*[2 2*(2/3) 2/5 0])-int(x^2,-1,1);
Eq3= sum([c1 c2 c3 c4].*[2 2*(2/3)^2 2/5^2 0])-int(x^4,-1,1);
```

```
Eq4= sum([c1 c2 c3 c4].*[2 2*(2/3)^3 2/5^3 0])-int(x^6,-1,1);
%% Solve equations
coef=solve(Eq1,Eq2,Eq3,Eq4,c1,c2,c3,c4);
%% Display the results
Weights = [coef.c1(1) coef.c2(1) coef.c3(1) coef.c4(1)];
pretty(Weights)
```

It returns the weighting coefficients of Eq. (13.58)

```
/  11    72   125   16 \
| ---,  ---,  ---,  -- |
\ 210   245   294   35 /
```

The `quadl` function uses Eqs. (13.57) and (13.58) as a pair. It estimates I_7^L and the difference $\left|I_4^L - I_7^L\right|$ which serves as an estimate of the local error. Based on this latter estimate, the decision is made whether to proceed with computations (if the error tolerance is met) or halve the range and apply this pair to each half again (similar to the idea shown in Fig. 13.8).

13.9 Using the MATLAB Functions

As mentioned in Secs. (13.1) and (13.6), MATLAB features only one function that allows estimating a single integral given by Eq. (13.1) based on the discrete data sets `x` and `y`:

```
T=trapz(x,y)
```

It uses trapezoidal integration to compute the integral of `y` with a spacing vector (or scalar increment) `x`. Hence, `y` is an array that contains the values of the function at the points contained in `x`. If `x` is a scalar, then `trapz(x,y)` is equivalent to `x*trapz(Y)`. The call `trapz(y)` assumes `x=1`. For example,

```
>> Y = [1 4 8 10 15];
>> T=trapz(Y)
```

returns

```
T = 30
```

While `trapz` returns the integration value for the entire interval ([1;15] in our case), `T`, the `cumtrapz` function, which is closely related to `trapz` featuring the same syntax, returns intermediate values at each node. For the vector `Y` defined previously,

```
>> T=cumtrapz(Y)
```

returns

```
T = 0   2.5000   8.5000  17.5000  30.0000
```

Let us give one explicit example of using cumulative trapezoidal numerical integration. Consider Fig. 13.12a, which shows a longitudinal acceleration profile of the ascending Space Shuttle as depicted in Fig. 13.12b. It starts at lift-off, features an aerodynamic stress at the highest dynamic pressure ~40 s after lift-off, increases to about 2.5*g*, drops when solid rocket boosters are burned out and separated, and finally smoothly increases with the main engines burning to about 3*g* before their shutdown when in orbit. On

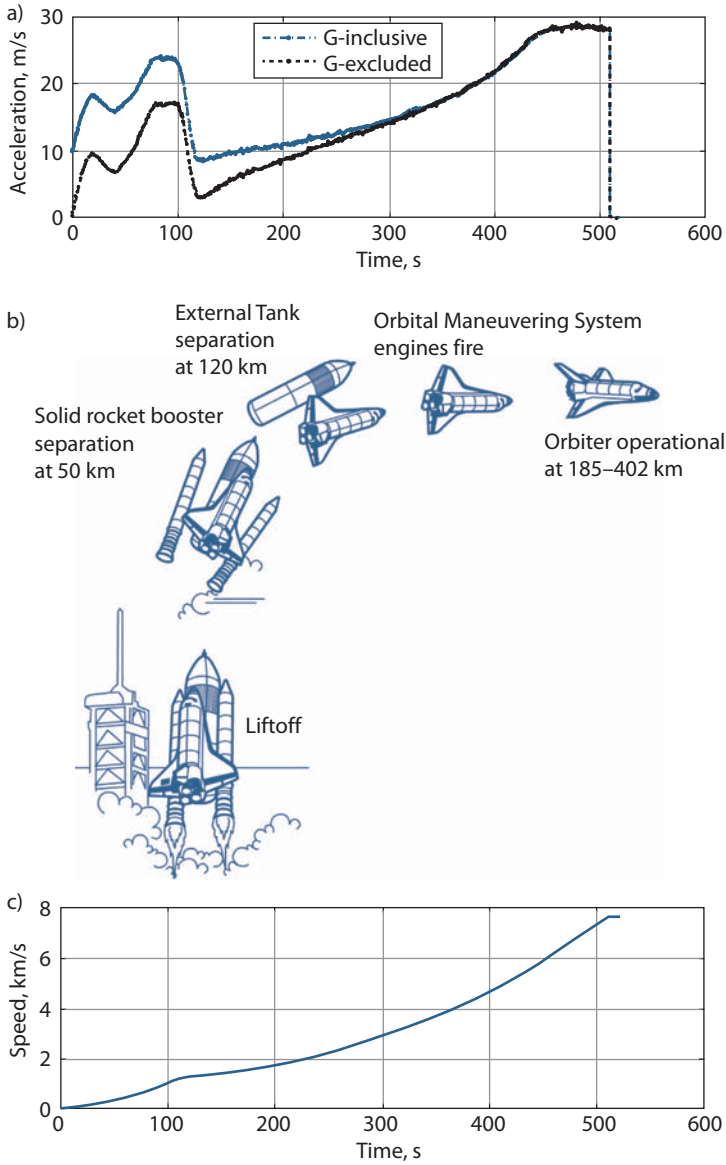


Fig. 13.12 Integrating the Space Shuttle acceleration profile.

course, the longitudinal accelerometer senses a total acceleration, including acceleration due to gravity (G). Figure 13.12a also features a G-excluded profile, when Space Shuttle attitude is taken into account. It is this latter profile that can be integrated to obtain a speed profile. If the vector `t` contains the time stamps (in seconds) and vector `A` the G-excluded acceleration data (in m/s^2), then the command

```
>> plot(t,cumtrapz(t,A)/1000)
```

produces the plot shown in Fig. 13.12c.

Let us proceed with a numerical evaluation of integrals featuring an integrand given in the form of an explicit or implicit function (an analytical expression or a model) that can be sampled at any set of arguments. To deal with this case, the MATLAB offers a wide variety of functions as presented in Tables 13.4–13.6 (as usual, more syntaxes are available via the MATLAB Help system). While Table 13.4 features the MATLAB functions for estimating a single integral, Table 13.5 lists the functions for estimating double and triple integrals:

$$\iint_S f(x, y)ds \tag{13.59}$$

$$\iiint_V f(x, y, z)dv \tag{13.60}$$

Table 13.4 Functions for Estimating a Single Integral

Function Call	Description
`Q=quad(fun,a,b,tol)`	Uses a recursive (multiple application) Simpson's 1/3 rule to compute the integral of `fun` with `a` as the lower limit and `b` as the upper limit. The first argument may be an inline expression, the name of the function, or the function handle (see Sec. 5.9.4). The optional parameter `tol` specifies the absolute error tolerance (the default tolerance is 10^{-6}). Alternative syntax, `[Q,fcnt]=quad(...)`, returns the number of function evaluations, `fcnt`, as well
`Q=quadv(fun,a,b,tol)`	Vectorized version of `quad` (for an array-valued `fun`)
`Q=quadl(fun,a,b,tol)`	Uses a recursive adaptive Lobatto integration algorithm (four- and seven-point formulas) to compute the integral from `a` to `b` with the absolute error tolerance `tol`. It may also return the number of function evaluations, if applied as `[q,fcnt]=quadl(...)`
`Q=quadgk(fun,a,b, ... param,val, ...)`	Evaluates the integral of `fun` using the Gauss–Kronrod 7- and 15-point quadrature pair using default tolerances of 10^{-10} and 10^{-6} for the absolute and relative errors, respectively (these tolerances may be changed using the `param-val` pair arguments following three obligatory arguments). Alternative syntax, `[Q,err]=quadgk(...)`, returns an approximate upper bound on the absolute error, `err`, as well

Table 13.5 Functions for Estimating Double and Triple Integrals

Function Call	Description
`Q=dblquad(fun,xmin, ... xmax,ymin,ymax, ... tol,method)`	Employs either the `quad` (if `method` is omitted) or `quadl` (`method` is specified as `@quadl` or `'quadl'`) to evaluate a double integral of `fun` with the limits of integration of `xmin` to `xmax` and `ymin` to `ymax`. Also, the optional parameter `tol` specifies the absolute error tolerance (the default tolerance is 10^{-6})
`Q=quad2d(fun,xmin, ... xmax,ymin,ymax, ... param,val, ...)`	Approximates the integral of $f(x,y)$ over the planar region `xmin` $\leq x \leq$ `xmax` and `ymin`$(x) \leq y \leq$ `ymax`(x) using 2-D quadrature. In a general case, `fun` is a function handle, and `ymin` and `ymax` may each be a scalar or a function handle. Optional comma-separated `param-val` pair arguments allow changing error tolerances (the default value of absolute tolerance is 10^{-5}), change the maximum allowed number of `fun` evaluations (the default is 2000), generate a graphical representation of the regions needing further refinement when the maximum allowed number of `fun` evaluations is reached, and employ transformations to weaken boundary singularities for better performance
`Q=triplequad(fun ... xmin,xmax,ymin, ... ymax,zmin,zmax, ... tol,method)`	Employs either `quad` (when the last input argument, `method`, is omitted) or `quadl` (`method` is specified as `@quadl` or `'quadl'`) to evaluate a triple integral of `fun` over the 3-D rectangular cuboid. Also, optional parameter `tol` specifies the absolute error tolerance (the default tolerance is 10^{-6})

Table 13.6 Functions to Replace Most of the Functions in Tables 13.4 and 13.5 in a Future Release of MATLAB

Function Call	Description
`Q=integral(fun, ... xmin,xmax,param, ... val, ...)`	Numerically integrates function `fun` from `xmin` to `xmax` using global adaptive quadrature and default tolerances of 10^{-10} and 10^{-6} for the absolute and relative errors, respectively. `fun` must be a function handle. Optional comma-separated `param-val` pair arguments allow for changing error tolerances, specifying waypoints to integrate in the complex plane, and dealing with array-valued or vector-valued function `fun`
`Q=integral2(fun, ... xmin,xmax,ymin, ... ymax,param,val, ...)`	Approximates the integral of the function $f(x,y)$ over the planar region `xmin` $\leq x \leq$ `xmax` and `ymin`$(x) \leq y \leq$ `ymax`(x). Optional comma-separated `param-val` pair arguments allow changing error tolerances and choosing between the use of 1-D and 2-D quadrature ('iterated' or 'tiled' method, respectively)
`Q=integral3(fun, ... xmin,xmax,ymin, ... ymax,zmin,zmax, ... param,val, ...)`	Approximates the integral of the function $f(x,y,z)$ over the domain `xmin` $\leq x \leq$ `xmax`, `ymin`$(x) \leq y \leq$ `ymax`(x), and `zmin`$(x,y) \leq z \leq$ `zmax`(x,y). Optional comma-separated `param-val` pair arguments allow changing error tolerances and choosing between the use of 1-D and 2-D quadrature ('iterated' or 'tiled' method, respectively)

In Eqs. (13.59) and (13.60), S is the region of integration in $\mathbf{R}^2$ (in the $x-y$ plane) for a double integral and V is the domain of integration in $\mathbf{R}^3$ (in the $x-y-z$ coordinate frame) for a triple integral. In aerospace engineering applications, integrals like (13.59) arise in computations of

- *Area:* if $f(x,y) = 1$, then the double integral gives the area of region S
- *Average:* the double integral divided by the area of the region S is the average of the function $f(x,y)$ on S
- *Volume:* the double integral is equal to volume under the surface $z = f(x,y)$ above the region S
- *Force:* if $f(x,y)$ is the force per unit area ds on the plate in the z direction, then the double integral represents the total force on the plate
- *Mass:* if S is a plate area and $f(x,y)$ is density per unit area of the plate ds, then the double integral estimates the mass of the plate

Triple integrals arise in computation of

- *Volume:* if $f(x,y,z) = 1$, then the triple integral gives the volume of V
- *Average:* the triple integral divided by the volume of the 3-D domain V is the average of the function $f(x,y,z)$ over V
- *Force:* if $f(x,y,z)$ is the force per unit volume dv, then the triple integral represents the total force on a 3-D object
- *Mass:* if V is a 3-D object volume and $f(x,y,z)$ is density per unit volume dv, then the triple integral estimates the mass of a 3-D body

Double and triple integrals are also used to compute the center of mass and moment of inertia of plane figures and volumetric bodies.

The three functions of the `integral` family shown in Table 13.6 use vectorized adaptive quadrature, deal with nonrectangular regions of integration, allow handling infinity and singularities at endpoints, and are therefore more robust than those of Tables 13.4 and 13.5.

It should be noted that as of R2020a all functions shown in Tables 13.4–13.6 were available; however, in a future release, the `integral` function will replace `quad`, `quadv`, and `quadl` functions; `integral2` will replace `quad2D` and `dblquad`; and `integral3` will replace `triplequad`.

Before we proceed with examples, it should be noted that in Tables 13.4–13.6 the integrand `fun` is supposed to be a vectorized expression capable of accepting a vector of parameters rather than a single scalar argument. For example, an attempt to evaluate the integral

$$\int_0^{\pi/2} \sin(x)\cos(x)dx \tag{13.61}$$

using the call

```
>> quadl('sin(x)*cos(x)',0,pi/2)
```

will not work and return an error. To fix the problem, you should either vectorize it explicitly

```
>> quadl('sin(x).*cos(x)',0,pi/2)
```

or implicitly, using the `vectorize` function introduced in Sec. 5.8.1:

```
>> quadl(vectorize('sin(x)*cos(x)'),0,pi/2)
```

(A graphical representation of this integral is depicted by the shaded area in Fig. 13.13.)

In the case of the `quad` function, you may also take advantage of its vectorized version, `quadv`, which `vectorizes` the input function automatically:

```
>> I=quadv('sin(x)*cos(x)',0,pi/2)
I =  0.5000
```

The `quadv` function also allows integrating the array-valued function, rather than a single function as is the case with all other integrating functions, that is, estimating an integral of different integrands within the same interval simultaneously. For example, if we define the anonymous function as a function that has four outputs in a 2-by-2 matrix form

```
>> fun = @(x) [tan(x) sin(x); sin(x)^2 cos(x)]
```

then

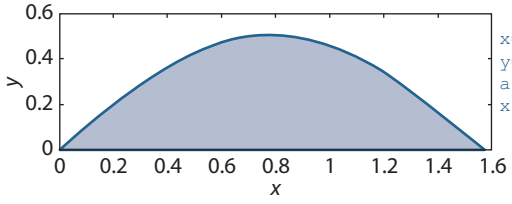
```
>> I=quadv(fun,0,pi/2)
```

returns a 2-by-2 matrix as well

```
I =  39.1458   1.0000
      0.7854   1.0000
```

Depending on your specific application, the MATLAB `quad` function is probably all you will need, especially if you are dealing with nonsmooth integrands, that is, in the case when the function values are the outputs of some script or model and generally take some time to compute. If, however, you are interested in integrating smooth analytical integrands at higher accuracies, you may want to try the `quadl` function.

The `quadgk` function is more suitable for smooth oscillatory integrands with moderate singularities at endpoints or infinite intervals. Moreover,




```
x=linspace(0,pi/2,40);
y=sin(x).*cos(x);
area(x,y,'FaceColor',[.3,.7,.9])
xlabel('x'), ylabel('y')
```

Fig. 13.13 A graphical representation of integrand in Eq. (13.61).

Table 13.7 Argument Substitutions to Handle Endpoint Singularities and Infinite Endpoints

Integral	Substitution	Weighting Factor
$\int_a^b f(x)dx = \int_{-1}^{1} f(\zeta(t))\zeta_t' dt$	$x = \zeta(t) = \frac{1}{4}(b-a)t(3-t^2) + \frac{1}{2}(b+a)$	$\zeta'(t) = \frac{3}{4}(b-a)(1-t^2)$
$\int_a^\infty f(x)dx = \int_0^1 f(\zeta(t))\zeta_t' dt$	$x = \zeta(t) = a + t^2(1-t)^{-2}$	$\zeta'(t) = 2t(1-t)^3$
$\int_{-\infty}^b f(x)dx = \int_{-1}^0 f(\zeta(t))\zeta_t' dt$	$x = \zeta(t) = b - t^2(1+t)^{-2}$	$\zeta'(t) = -2t(1+t)^3$
$\int_{-\infty}^\infty f(x)dx = \int_{-1}^1 f(\zeta(t))\zeta_t' dt$	$x = \zeta(t) = t(1-t^2)$	$\zeta'(t) = (t^2+1)(1-t^2)^{-2}$

although endpoints `a` and `b` for all other functions of Table 13.4 must be finite, `quadgk` supports infinite intervals (in the case when $f(x)$ decays fast enough as x approaches infinity) and can handle moderate singularities at endpoints. (If $f(x)$ has a singularity inside the [a;b] interval, the integral should be computed as a sum of integrals over subintervals with the singular points as endpoints.) The `quadgk` function also supports contour integration along piecewise linear paths.

To handle singularities at endpoints and infinite intervals, the `quadl` function (as well as the `integral` family functions) uses more sophisticated mappings compared with the linear one shown in Eqs. (13.46) and (13.47). These mappings are shown in Table 13.7.

Let us now consider several examples and start with a simple integral

$$\int_0^\pi \sin(x)\mathrm{d}x \tag{13.62}$$

First, we can compute the exact value of this definite integral using the Symbolic Math Toolbox. To this end, the commands

```
>> syms x
>> f=int(sin(x));
>> fun=inline(char(f));
>> I=fun(pi)-fun(0)
```

return

```
I = 2
```

(By the way, the third command can be replaced with

```
fun=matlabFunction(f);
```

which creates an anonymous rather than inline function (Sec. 7.10.2).)

To be able to use the trapezoidal rule (the `trapz` function), we need to create the data points (`x` and `y` vectors). Let us go with as few as 10 points:

```
>> x=linspace(0,pi,9); y=sin(x);
```

Now, applying the following two commands allows you to compute the integral and the relative error:

```
>> ITR=trapz(x,y)
>> fprintf('Trapezoidal rule error is %3.2f %% \n',abs(ITR-2)/2*100)
ITR = 1.9797
Trapezoidal rule error is 1.02 %
```

We can also try to use Eq. (13.14) to estimate an absolute error

```
>> Eest=abs((pi/9)^2*pi/12)
```

which returns the upper bound

```
Eest = 0.0319
```

Given that the second-order derivative of the function, $|f''(\chi)|$, is less than one, we can expect the actual error to be even smaller than that estimate. Indeed, it is true

```
>> Eact=2-ITR
Eact = 0.0203
```

Let us also try to implement any of the rectangle rules of Eq. (13.2) (due to symmetry, both of them return the same result):

```
>> ILR=sum(y(1:end-1).*(diff(x)))
>> fprintf('Rectangle rule error is %3.2f %% \n',abs(ILR-2)/2*100)
ILR = 1.9797
Rectangle rule error is 1.02 %
```

Again, due to the symmetry of the integrand within the specified interval of Eq. (13.62), in this particular case the results coincide with those of the trapezoidal formula.

We may also want to explore the `quad` function. For example,

```
>> IS13=quad('sin',0,pi,1e-1)
>> fprintf('Simpson rule error is %4.4e \n',abs(IS13-2))
```

returns

```
IS13 = 2.0000
Simpson rule error is 6.5035e-006
```

Here, the first command is intended to compute the integral within the specified limits, degrading an absolute error tolerance to as low as 0.1, whereas the second command compares the result with the exact value and shows the actual error. As seen in this particular example, the actual error happens to be much smaller than the predefined tolerance.

Next, let us explore the Gauss quadrature formulas. To use them (as explained in Sec. 13.8), we need to change the domain. Using Eq. (13.43), we can write

$$\int_0^{\pi} \sin(x)\mathrm{d}x = \frac{\pi}{2}\int_{-1}^{1} \sin\left(\frac{\pi}{2}+\frac{\pi}{2}x^*\right)\mathrm{d}x^* \tag{13.63}$$

The following commands compute this integral using the Gauss quadrature formulas for two and three points (Table 13.2), also showing the absolute error:

```
>> x=1/sqrt(3);
>> IG2=pi*(sin((pi+pi*x)/2)+sin((pi-pi*x)/2))/2;
>> x=sqrt(15)/5; c1=5/9; c2=8/9;
>> IG3=pi*(c1*sin((pi+pi*x)/2)+c1*sin((pi-pi*x)/2)+c2*sin(pi/2))/2;
>> disp([IG2, 2-IG2])
>> disp([IG3, 2-IG3])
1.9358   0.0642
2.0014  -0.0014
```

As seen, with only three wisely distributed points, we were able to outperform the trapezoidal method with 9 evenly distributed points.

As presented in the beginning of this section, the `quadl` and `quadgk` functions need no domain change and can be used similarly to the `quad` function considered earlier. For the sake of comparison, let us analyze the return of these three functions being called as follows:

```
>> [qG,nG]=quad(@sin,0,pi,1e-1);
>> [qL,nL]=quadl(@sin,0,pi,1e-1);
>> [qK,errK]=quadgk(@sin,0,pi);
>> disp([qG,nG; qL,nL; qK,errK])
2.0000   13.0000
2.0000   18.0000
2.0000    0.0000
```

Indeed, the desired tolerance of 0.1 played no effect, so that the accuracy of computations turned out to be much higher than that, even with the minimum number of function evaluations (which happens to be slightly smaller for the Simpson's 1/3 adaptive rule). However, if we request a better accuracy, say 10^{-6} (which is the default accuracy so that the fourth argument in the `quad` function call may be omitted),

```
>> [qG,nG]=quad(@sin,0,pi);
>> [qL,nL]=quadl(@sin,0,pi);
>> disp([qG,nG; qL,nL])
2.0000   57.0000
2.0000   18.0000
```

the number of function evaluations in `quadl` remains the same, whereas `quad` now requires much more resources. The effectiveness of the Gauss

quadrature rules compared with Newton–Cotes formulas becomes even clearer if you require even better accuracy:

```
>> [qG,nG]=quad(@sin,0,pi,1e-10);
>> [qL,nL]=quadl(@sin,0,pi,1e-10);
>> disp([qG,nG; qL,nL])
2.0000   225.0000
2.0000    48.0000
```

As mentioned earlier, among all the functions of Table 13.4, `quadgk` is the only function that, under certain conditions, allows handling of infinite endpoints. For example, to evaluate

$$\int_0^{\infty} e^{-0.25x}\sin(x)\mathrm{d}x \tag{13.64}$$

you can use

```
>> [q,err]=quadgk(@(x)exp(-x/4).*sin(x),0,inf,'RelTol',1e-8,'AbsTol',...
                  1e-12)
```

which returns

```
q =   0.9412
err = 1.8575e-09
```

(A graphical representation of this integral is depicted by the shaded area in Fig. 13.14.)

Compared with the `quadl` and `quadgk` functions, the `quadgk` function allows you to specify both absolute and relative tolerances so that

$$|\zeta_i - \zeta_{i-1}| \leq \max\left(\varepsilon^{abs}, \varepsilon^{rel}|\zeta_i|\right) \tag{13.65}$$

where ζ_i and ζ_{i-1} are two consecutive iterations [compare with Eq. (8.28)]. Usually, it is the relative tolerance that determines the accuracy of the integration. However, if $|\zeta_i|$ in Eq. (13.65) is sufficiently small, it is the absolute tolerance that determines the accuracy of the integration instead. Hence, a capability to define both absolute and relative tolerances provides

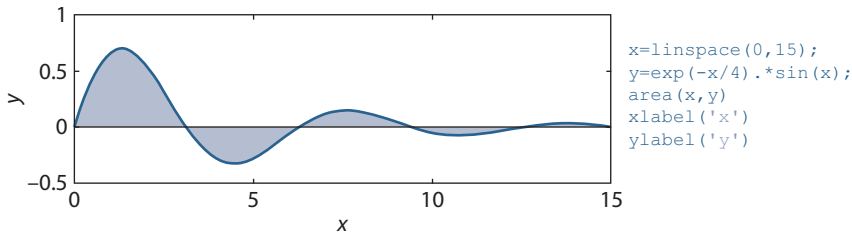


Fig. 13.14 A graphical representation of integrand in Eq. (13.64).

a way of trading off accuracy and computation time. Setting the relative tolerance to zero (`'RelTol',0`) allows you to specify the number of decimal places of accuracy explicitly (regardless how much time it might take).

While most of the functions in Table 13.4 employ adaptive quadratures sequentially, on one subinterval at a time, the `integral` function—an intended replacement for the `quad`, `quadv`, and `quadl` functions—takes advantage of vectorization, array operations, and the use of the built-in functions (of course, compiled code works much faster than interpreted code). As a result, `integral` demonstrates much better numerical efficiency compared with those functions. Similar to `quadgk`, the `integral` function also employs algebraic transformations to handle problems with infinite endpoints and moderate endpoint singularities. The `integral` function calls the compiled `integralCalc` code, which resides in the private folder. The *integralCalc.m* script can be found in the same folder, that is, the call

```
>> which('integralCalc','-all')
```

returns

```
C:\ ... \toolbox\matlab\funfun\private\integralCalc.m % Private to funfun
```

From the standpoint of syntax, a simple three-argument call looks similar to those of `quad` and `quadl`. For example,

```
>> I=integral(@(x)1./x.^2,1,inf)
```

returns

```
I =  1
```

Similar to `quadgk`, the `integral` function allows you to adjust both relative and absolute error tolerances, for example,

```
>> I=integral(@(x)exp(-x.^2).*log(x).^2,0,inf,'RelTol',1e-8,'AbsTol',1e-12)
```

returns

```
I =  1.9475
```

The parameter–value pair argument also allows you to specify waypoints to integrate in the complex plane. For example, to evaluate the contour integral

$$\oint \frac{1}{z} dz \tag{13.66}$$

over the circle $|z| = 1$, you may assign the waypoints as follows:

```
>> I=integral(@(z)1./z,1,1,'Waypoints',[i,-1,-i])
```

which yields the exact value of $2\pi i$

```
I =  0.0000 + 6.2832i
```

Finally, the `integral` function inherited the capability to integrate the array-valued function (as was the case for the `quadv` function). For example,

```
>> I=integral(@(x)cos((1:4)*x).*sin((4:-1:1)*x),0,1,'ArrayValued',true)
```

returns

```
I =   0.4033   0.3015  -0.1582  -0.2600
```

To estimate a double integral (13.59), the `dblquad` function applies a 1-D quadrature (`quad` or `quadl`), iteratively tackling one dimension first (inner integral) and then moving to the second one (outer integral). For example, to estimate an integral

$$\int_0^2 \int_1^2 (6 - x^3 + xy^3) \mathrm{d}y\mathrm{d}x \tag{13.67}$$

you can use

```
>> I=dblquad(@(x,y)6-x.^3+x.*y.^3,0,2,1,2)
```

which returns

```
I =   15.5
```

This 'iterated' approach is depicted in Fig. 13.15a, where the integral is evaluated as a sum of the volumes of 3-D slices over the rectangular 2-D domain.

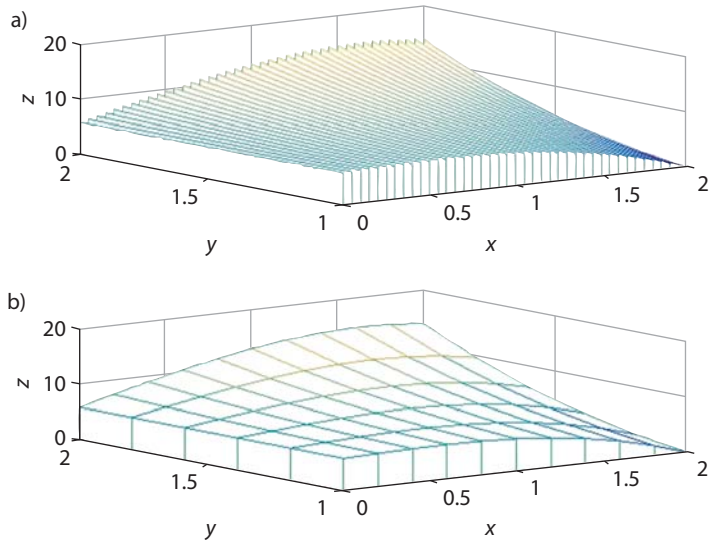


Fig. 13.15 A graphical representation of a) 'iterated' and b) 'tiled' methods to estimate double integrals.

Table 13.8 The Seven-Point Kronrod Rule Points

Function Argument x_i	Weight Factor c_i
$x_1 = -x_7 = -0.960491269$	$c_1 = c_7 = 0.104656226$
$x_2 = -x_6 = -\mathbf{0.774596669}$	$c_2 = c_6 = 0.268488090$
$x_3 = -x_5 = -0.434243749$	$c_3 = c_5 = 0.401397415$
$x_4 = \mathbf{0}$	$c_4 = 0.450916539$

On the contrary, the `quad2d` function introduced in R2009b (originally it was a part of the Curve Fitting Toolbox) utilizes a 2-D quadrature referred to as a 'tiled' method. In this case, the 2-D region of integration is divided into rectangles and a Gauss–Kronrod 3–7 quadrature is applied in 2-D to approximate the integral over each rectangle. (Arguments and weight factors for the seven-point Kronrod quadrature are shown in Table 13.8 with three-point Gauss rule arguments shown in boldface, similar to Table 13.3.) Each evaluation of integrand provides enough samples to approximate the integral over several subregions and therefore estimate the error. For the integral of Eq. (13.67), the corresponding call is

```
>> I=quad2d(@(x,y)6-x.^3+x.*y.^3,0,2,1,2)
```

and the graphical representation of the 'tiled' method is given in Fig. 13.15b.

The `quad2d` function allows limiting the number of maximum function evaluations explicitly (the default number is 2000). Also, it has the option to produce a function failure plot if this number is reached before satisfying the error tolerance (the default value of absolute tolerance is 10^{-5}). For example, consider evaluating the integral

$$\int_{-2\pi}^{2\pi}\int_{-\pi}^{\pi}(e^{-0.05(x^2+y^2)}\sin(x)\cos(2y)+0.5)\mathrm{d}y\mathrm{d}x \tag{13.68}$$

(the integrand is shown in Fig. 13.16) and limiting the maximum number of function evaluations by just 11. The call

```
>> I=quad2d(@(x,y)exp(-0.05*(x.^2+y.^2)).*sin(x).*cos(2*y)...
     +0.5,-2*pi,2*pi,-pi,pi,'MaxFunEvals',11,'FailurePlot',true)
```

results in the warning message

```
Warning: Reached the maximum number of function evaluations (11). The result
fails the global error test.
```

and the failure plot shown in Fig. 13.17a. Increasing `MaxFunEvals` to 19 does not produce a failure plot because the default absolute tolerance is satisfied.

Compared with `dblquad`, the `quad2d` function also allows for integration over nonrectangular regions. For example, for evaluation of the

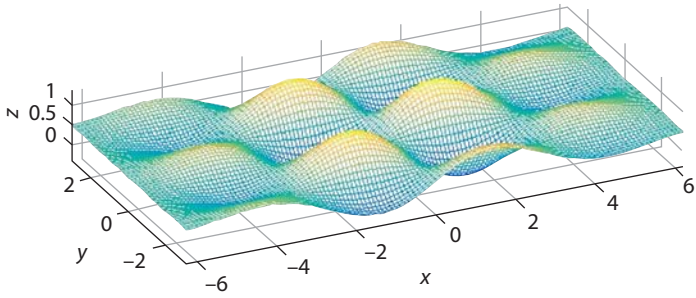


Fig. 13.16 Visualization of integrand in Eq. (13.68).

integral over the nonrectangular domain [compare with Eq. (13.68)],

$$\int_{-2\pi}^{2\pi}\int_{-|x|}^{x^2}(e^{-0.05(x^2+y^2)}\sin(x)\cos(2y)+0.5)\mathrm{d}y\mathrm{d}x \tag{13.69}$$

you can use the following call:

```
>> I=quad2d(@(x,y)exp(-0.05*(x.^2+y.^2)).*sin(x).*cos(2*y)+0.5,-2*pi,...
     2*pi,@(x)-abs(x),@(x)x.^2,'MaxFunEvals',250,'FailurePlot',true)
```

In this particular case, even though the result passes the global error test, the number of function evaluations reaches its limit (250). Hence, you will receive a warning message triggering generation of the failure plot shown in Fig. 13.17b.

To summarize, each of two methods for evaluating double integrals ('iterated' and 'tiled', implemented in `dblquad` and `quad2d`, respectively) has different strengths. Usually, the 'tiled' approach is the faster of the two, but it slows down when integrating over nonrectangular regions. Another weakness of `quad2d` is revealed when dealing with curvilinear discontinuity. With the 'iterated' method, such a discontinuity is covered by a single subinterval, whereas using 2-D quadrature leads to subdividing each tile into four pieces, increasing the number of function evaluations compared with the 1-D case.

The `integral2` function unites `dblquad` and `quad2d` functions and takes advantage of vectorization, array operations, and built-in functions, as was the case for the `integral` function. The `integral2` function calls the *integral2Calc.m* script residing in the same private folder as *integralCalc.m*. The 'tiled' or 'iterated' method is specified by the parameter–value pair argument following the five required input arguments, for example,

```
>> I=integral2(@(x,y)6-x.^3+x.*y.^3,0,2,1,2,'Method','iterated')
```

The default method is 'auto', which implies using the 'tiled' method for most cases, except when any of the integration limits are infinite (it then switches to the 'iterated' method). This function also allows defining both absolute and relative tolerances. For example,

```
>> I=integral2(@(x,y)6-x.^3+x.*y.^3,0,2,1,2,'RelTol',0,'AbsTol',1e-12)
```

specifies 12 decimal places of accuracy.

Let us proceed with evaluating a triple integral in the form of Eq. (13.60). For example, consider the triple integral over a rectangular cuboid

$$\int_0^{\pi}\int_0^{1}\int_{-1}^{1}(y\sin(x)+z\cos(x))\mathrm{d}z\mathrm{d}y\mathrm{d}x \tag{13.70}$$

The following commands solve the problem using the `triplequad` function

```
>> integrnd=@(x,y,z) y*sin(x)+z*cos(x);
>> I=triplequad(integrnd,0,pi,0,1,-1,1)
```

returning

```
I = 2.0000
```

which happens to be the exact value. This function uses the 'iterated' approach (moving from the inner integral out).

The `integral3` function, which will replace the `triplequad` function in a future release, treats the integral (13.60) as

$$\int_{x_{\min}}^{x_{\max}}\left(\iint_{S(y,z)} f(x,y,z)ds\right)\mathrm{d}x \tag{13.71}$$

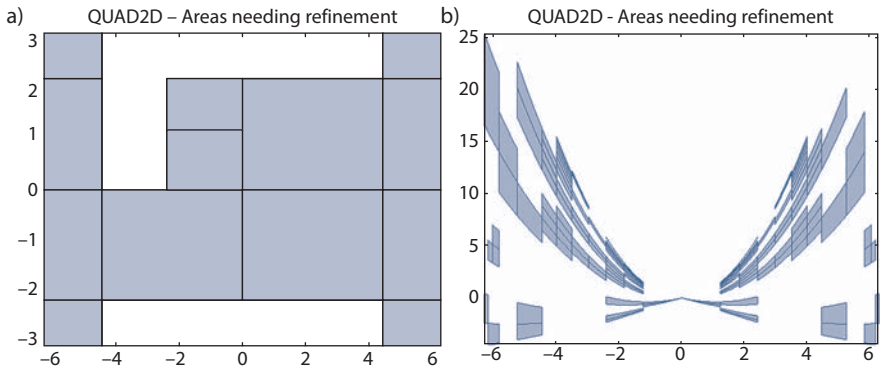


Fig. 13.17 Visualization of a failure plot caused by a bound on the number of maximum function evaluations when integrating over a) rectangular and b) nonrectangular region.

where the inner integral has the form of the double integral shown in Eq. (13.59). Consequently, the `integral3` function calls *integralCalc.m* to evaluate the outer integral and *integral2Calc.m* to evaluate the inner integral. Hence, the 'iterated' or 'tiled' option defines the way the inner integral in Eq. (13.71) is evaluated. For instance, the call

```
>> integrnd=@(x,y,z) y*sin(x)+z*cos(x);
>> I=integral3(integrnd, 0, pi, 0, 1, -1, 1,'Method','iterated')
```

changes the default 'auto' method to the 'iterated' method. Absolute and relative error tolerances can be set in a manner common for all the functions of the integral family.

Similar to the other two functions of the integral family, the `integral3` function allows handling infinity and singularities at endpoints and integrating over generalized 3-D domains (rather than over rectangular cuboids). For example, to evaluate

$$\int_0^{\pi}\int_0^{\sqrt{x}}\int_{-x^2-y^2}^{1}(y\sin(x)+z\cos(x))\mathrm{d}z\mathrm{d}y\mathrm{d}x \tag{13.72}$$

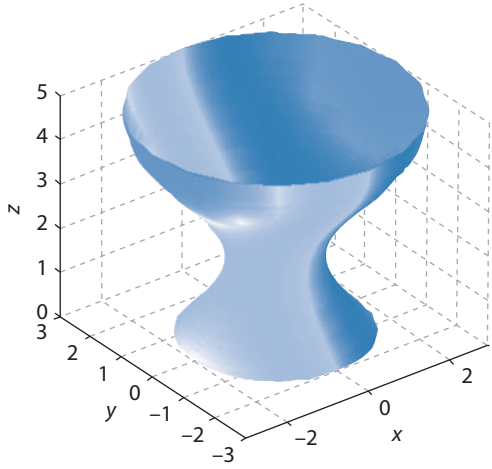
you can use

```
>> I=integral3(@(x,y,z)y.*sin(x)+z.*cos(x),0,pi,0,@(x)sqrt(x),...
              @(x,y)-x.^-y.^2,1)
```

which results in

```
I =  1.1348
```

Let us consider one more example and find the dimensionless volume of the nozzle shown in Fig. 13.18 (similar to that shown in Fig. 6.43c).




```
figure('Color','w')
z=linspace(pi/2,2*pi);
[X,Y,Z]=cylinder(2+cos(z));
surf(X,Y,5*Z,X)
axis equal
colormap hsv
shading interp
xlabel('x')
ylabel('y')
zlabel('z')
```

Fig. 13.18 Body of revolution about *z*-axis with corresponding commands.

This nozzle is obtained by a rotation of the curve given by the equation $r(z) = 2 + \cos(z)$, $0.5\pi \le z \le 2\pi$ about the z-axis, and then stretched along this axis by the factor of 5 (the `cylinder` command creates a cylinder 1 unit tall). The volume of this body is given by

$$\int_0^5 \pi r^2 dz = \int_0^5 \pi \left(2 + \cos\left(\frac{\pi}{2} + \frac{2\pi - 0.5\pi}{5} z\right)\right)^2 dz \tag{13.73}$$

The following script computes the volume of this nozzle with trapezoidal and Simpson's 1/3 and 3/8 formulas using six equal-size subintervals and compares it with a more accurate estimate obtained using the `quad` function:

```
f=@(x) pi*(2+cos(pi/2+x*(2*pi-pi/2)/5)).^2;
z=linspace(0,5,7); s=f(z);
ITR=trapz(z,s);
h=z(2)-z(1);
IS13=h*(s(1)+4*s(2)+2*s(3)+4*s(4)+2*s(5)+4*s(6)+s(7))/3;
IS38=3*h*(s(1)+3*s(2)+3*s(3)+3*s(4)+3*s(5)+3*s(6)+s(7))/8;
I=quad(f,0,5);
fprintf('Method      | Value | Rel. Error\n')
fprintf('Trapezoidal | %4.2f | %3.2f %%\n',ITR, abs(ITR-I)/I*100)
fprintf('Simpson 1/3 | %4.2f | %3.2f %%\n',IS13,abs(IS13-I)/I*100)
fprintf('Simpson 3/8 | %4.2f | %3.2f %%\n',IS38,abs(IS38-I)/I*100)
fprintf('Exact value | %4.2f |\n',I)
```

This results in the following output:

```
Method      | Value | Rel. Error
Trapezoidal | 58.05 | 1.21 %
Simpson 1/3 | 57.32 | 0.05 %
Simpson 3/8 | 58.92 | 2.73 %
Exact value | 57.35 |
```

Finally, let us introduce one more function, `polyint`, to deal with the integration of polynomial represented by the vector of its coefficients `p`. While a simple call

```
polyint(p)
```

assumes a constant of integration $C = 0$, using two input arguments

```
polyint(p,k)
```

returns a vector of coefficients of a polynomial representing the integral of polynomial `p`, using a scalar constant of integration $C =$ `k`.

For example, a set of commands

```
p=[4 6 6 4];                    % define the polynomial
disp('Original polynomial')
pretty(poly2sym(p))
```

```
b=polyint(p);                    % take an integral
disp('Integral polynomial')
pretty(poly2sym(b))
```

produces the following output:

```
Original polynomial
                   3     2
                4 x + 6 x + 6 x + 4
Integral polynomial
                4     3     2
               x + 2 x + 3 x + 4 x
```

A slightly modified call

```
pretty(poly2sym(polyint(p,5)))
```

changes the output to

```
                4     3     2
               x + 2 x + 3 x + 4 x + 5
```

Problems

13.1 Determine a mean value of the function

$$f(x) = -23 + 22.7x - 6.9x^2 + 0.855x^3 - 0.0365x^4$$

between $x = 2$ and $x = 10$. Confirm the result graphically (showing both the function and its mean value).

13.2 Use numerical integration formulas (with a function handle input) to evaluate

a) the $x-y$ coordinates of the center of gravity (CG) of a plane area defined as

$$2 \le x \le 10, \quad 0 \le y \le f(x)$$

where $f(x)$ is given in Problem 13.1. The CG coordinates can be evaluated as

$$x_{CG} = \frac{1}{A}\int_a^b xf(x)\mathrm{d}x$$

and

$$y_{CG} = \frac{1}{2A}\int_a^b f(x)^2\mathrm{d}x$$

where

$$A = \int_a^b f(x)\mathrm{d}x$$

b) the volume of a 3-D shape defined as

$$-2 \le x \le 2, \quad 0 \le y \le 4, \quad 0 \le z \le \frac{1}{16}x^2 - \frac{1}{4}y^2 + \frac{1}{32}xy^3 + 1.25$$

Visualize the area and CG location in the first case and the 3-D shape with an equivalent-volume rectangular cuboid (parallelepiped) in the second case.

13.3 Compute the surface area and volume of the Gabriel's horn (also known as Torricelli's trumpet), a shape obtained by revolving the function $f(x) = x^{-1}$ around the x-axis for $1 \le x < \infty$. The volume is given by

$$V = \int_1^\infty \pi f(x)^2 \mathrm{d}x$$

and the surface area by

$$S = \int_1^\infty 2\pi f(x)\sqrt{1 + f'(x)^2}\mathrm{d}x$$

Visualize this 3-D shape.

Chapter 14 Initial-Value Problem

- Understanding Single-Step Methods
- Understanding Multistep Methods
- Using MATLAB IVP Solvers
- Using MATLAB BVP and PDE Solvers

`ode23`, `ode45`, `ode113`, `ode15s`, `ode23s`, `ode23t`, `ode23tb`, `ode15i`, `dde23`, `ddesd`, `bvp4c`, `bvp5c`, `pdepe`, `odeexamples`

14.1 Introduction

Many engineering problems require the determination of a function satisfying an equation containing one or more derivatives of this unknown function with respect to some argument (for example, time). These types of equations are called differential equations. Differential equations can be classified into two main types: *ordinary differential equations*, where the function depends on only one variable so that only ordinary derivatives appear in the equation, and *partial differential equations* (PDEs), where the function depends on more than one variable and contains partial derivatives. Section 7.6 introduced the `dsolve` function allowing you to solve up to six relatively simple ODEs analytically. In Sec. 9.6, higher-order linear ODEs were also solved analytically, using the eigendecomposition of their state matrix. In addition to these capabilities, MATLAB offers a variety of solvers allowing you to solve any ODE or PDE numerically. The primary focus of this chapter, the first out of two chapters of Part IV of the book (Fig. 0.1) is to introduce ODE solvers that are used to address the *initial-value problem* (IVP) that is suitable for modeling system dynamics. They are `ode23`, `ode45`, `ode113`, `ode15s`, `ode23s`, `ode23t`, `ode23tb`, and `ode15i`. The Simulink modeling environment, to be introduced in Chapter 15, uses the very same variable-step solvers along with some other fixed-step solvers. Hence, to better understand the difference between these solvers and therefore use them more efficiently, this chapter provides a detailed explanation of algorithms behind them. As usual, the Symbolic Math Toolbox is used to assist in deriving some bulky expressions. The chapter starts by defining the IVP and then proceeds with single- and multiple-step algorithms, emphasizing their numerical features as implemented in the MATLAB ODE suite. It then specifically addresses handling the so-called *stiff problems* and provides a variety of examples of using

the ODE solvers. For completeness, this chapter concludes by introducing some other ODE solvers to address differential equations with delays (`dde23` and `ddesd`), *boundary-value problem* (BVP) (`bvp4c` and `bvp5c`), and PDEs (`pdepe`).

14.2 Initial-Value Problem Definition

Let us start with a few formal definitions. If $y(x)$ is an unknown function in x, then an equation of the form

$$F(x, y, y', y'', \ldots, y^{(l)}) = 0 \tag{14.1}$$

(where primes denote derivatives with respect to x) is called an ODE of order l. The *order* of differential equation is the order of the highest derivative that appears in the equation. Equation (14.1) features an *implicit* form, whereas in general, an *explicit* form is

$$F(x, y, y', y'', \ldots, y^{(l-1)}) = y^{(l)} \tag{14.2}$$

Quite often ODEs use time as an argument, so $x \triangleq t$.

A differential equation is said to be *linear* if F can be written as a linear combination of the derivatives of $y(x)$

$$a_l(x)\frac{\mathrm{d}^l y(x)}{\mathrm{d}x^l} + a_{n-1}(x)\frac{\mathrm{d}^{l-1} y(x)}{\mathrm{d}x^{l-1}} + \cdots + a_1(x)\frac{\mathrm{d}y(x)}{\mathrm{d}x} + a_0(x)y(x) = g(x) \tag{14.3}$$

Differential equations that cannot be written in the form of Eq. (14.3) are called *nonlinear* equations.

In general, the coefficients a_i, $i = 0,1, \ldots, l$ and g may depend on x, as presented in Eq. (14.3). If none of them depends on x, a differential equation is called *autonomous*. Linear differential equations follow a very important theorem called the *superposition principle*. The superposition principle states that if $y_1(x)$, $y_2(x)$, $\ldots$, $y_l(x)$ are l linearly independent solutions of an lth-order linear differential equation, then

$$y(x) = c_1 y_1(x) + c_2 y_2(x) + \cdots + c_l y_l(x) \tag{14.4}$$

is also a solution, where $c_1, c_2, \ldots, c_l$ are constant coefficients. (Nonlinear differential equations do not follow the principle of superposition.)

Any explicit differential equation of order l can be written as a system of l first-order differential equations. All you have to do is introduce a family of unknown functions instead of just one, y, as follows:

$$z_k(x) = y^{(k-1)}(x), \quad k = 1, 2, \ldots, l \tag{14.5}$$

Their derivatives can then be expressed through each other and the derivative of z_l, defined by the original equation [Eq. (14.2)], all together constituting a set of l first-order differential equations

$$\begin{aligned} z_1' &= z_2 \\ z_2' &= z_3 \\ &\vdots \\ z_l' &= F(x, z_1, z_2, z_3, \ldots, z_l) \end{aligned} \tag{14.6}$$

or concisely in vector notation

$$\mathbf{z}' = \mathbf{F}(x, \mathbf{z}) \tag{14.7}$$

where $\mathbf{z} = [z_1, z_2, \ldots, z_l]^T$ and $\mathbf{F}(x, \mathbf{z}) = [z_2, z_3, \ldots, z_l, F(x, \mathbf{z})]^T$.

Consider the following example. Suppose we have a simple mass–spring–damper system as depicted in Fig. 14.1a. The only three forces acting on the cart having a mass m are the damper force $c\dot{y}(t)$, with c being the damping coefficient, the spring force $ky(t)$, with k denoting the spring constant, and some external force $u(t)$. If the cart is moving to the left of its equilibrium state shown in Fig. 14.1a, so that y is positive and growing, then both the damper force and spring force stem this motion, that is, they are negative. Hence, Newton's second law yields

$$m\ddot{y} = \sum F_y = -c\dot{y} - ky + u \tag{14.8}$$

which can further be rewritten as

$$\ddot{y} + \frac{c}{m}\dot{y} + \frac{k}{m}y = \frac{1}{m}u \tag{14.9}$$

(dots denote derivatives with respect to time).

According to the aforementioned definitions, this equation is a second-order linear ODE and can be represented as a system of two first-order differential equations by introducing two new unknown functions, say,

$$x_1(t) = y(t), \quad x_2(t) = \dot{y}(t) \tag{14.10}$$

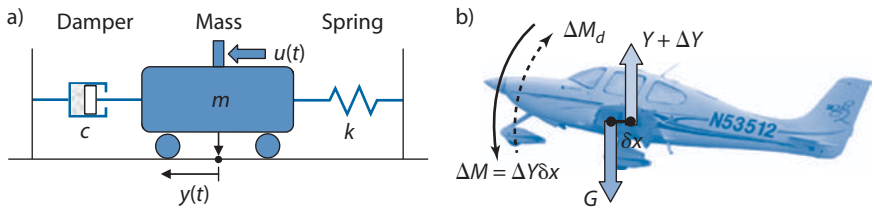


Fig. 14.1 a) Mass–spring–damper system and b) parameters defining a longitudinal motion of an aircraft.

Then, Eq. (14.9) becomes

$$\begin{aligned}\dot{x}_1 &= x_2\\ \dot{x}_2 &= \frac{-k}{m}x_1 - \frac{c}{m}x_2 + \frac{1}{m}u\end{aligned} \tag{14.11}$$

or in the matrix form

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \tag{14.12}$$

with

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix},\ \mathbf{A} = \begin{bmatrix} 0 & 1 \\ \dfrac{-k}{m} & \dfrac{-c}{m} \end{bmatrix},\ \mathbf{B} = \begin{bmatrix} 0 \\ \dfrac{1}{m} \end{bmatrix} \tag{14.13}$$

We can formally express cart's displacement, y, via $\mathbf{x}$ in the matrix form as well:

$$y = \begin{bmatrix} 1 & 0 \end{bmatrix}\mathbf{x} = \mathbf{C}\mathbf{x} \tag{14.14}$$

Compared with what we have done previously [in Eqs. (9.109)–(9.112)] for the two-body mass–spring system, we complemented the *state equation* (14.12) with the so-called *output equation* (14.14). These two constitute the state–space representation of dynamic system of Fig. 14.1a. As you know already, vector $\mathbf{x}$ is called the state vector and matrix $\mathbf{A}$ – the *state* (or *system*) *matrix*. The newly introduced matrix $\mathbf{B}$ is called the *input matrix*; y – the *output vector* (just a scalar in our case); and $\mathbf{C}$ – the *output matrix*. The canonical representation of the state-space model of linear system [with one more term in Eq. (14.14)] will be presented in Sec. 15.3.3.

The derived second-order linear ODE [Eq. (14.9)] has a fundamental importance because it is widely used to describe dynamics of mechanical, electrical, fluid, and thermal systems. For example, the model of an aircraft's landing gear, its hydrodynamic system, its accelerometer, the rotational dynamics of the entire aircraft (Fig. 14.1b), etc., can be reduced to Eq. (14.12) (with the elements of matrices $\mathbf{A}_{2\times2} = [a_{ij}]$ and $\mathbf{B}_{2\times1} = [b_{ij}]$ being some constant coefficients).

As discussed in Secs. 9.6.1 and 9.6.2, if Eq. (14.7) happens to be linear with constant coefficients, then you can solve them analytically using the `eig` function. For small l, you can also try the `dsolve` function of the Symbolic Math Toolbox (Sec. 7.6). Otherwise, the use of numerical methods discussed in this chapter is your only alternative.

A *general solution* of an lth-order ODE is a solution containing l arbitrary variables, corresponding to l constants of integration. A *particular solution* is derived from the general solution by setting the constants to particular values, chosen to satisfy ICs or boundary conditions (BCs). Consequently,

when solving ODEs numerically, we distinguish an IVP with all conditions prescribed at the initial value of argument x and a BVP with all conditions prescribed at the spatial boundaries.

Formally, the IVP can be formulated as follows. Given

$$\mathbf{y}' = \mathbf{F}(x, \mathbf{y}), \quad \mathbf{y}(x_0) = \mathbf{y}_0 \tag{14.15}$$

find the numerical values of components of the vector $\mathbf{y}(x)$—$\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_n, \ldots, \mathbf{y}_f$—corresponding to the argument values $x_1, x_2, \ldots, x_n, \ldots, x_f$, which constitute a solution over an interval $x \in [x_0; x_f]$. This problem is often referred to as a *Cauchy problem*.

In what follows, we will address problem (14.15) using different numerical methods and assuming that $x_{n+1} = x_n + h$, $n = 0, 1, 2, \ldots$, with h being the *integration step size*. For simplicity, we will use notations of the first-order ODE, that is, instead of Eq. (14.15), consider

$$y' = f(x, y), \quad y(x_0) = y_0 \tag{14.16}$$

Nevertheless, all formulas developed in this case can easily be extended to the case of Eq. (14.15).

Problem (14.16) looks somewhat similar to the numerical integration problem addressed in Chapter 13. The only difference is that there we dealt with integrating an equation $y' = f(x)$, rather than $y' = f(x, y)$, that is, function f did not depend on the unknown variable y. Therefore, neither of the methods developed in Chapter 13 can be applied to the problem (14.16) directly.

For $y' = f(x)$, given analytically, the slope y'_n can be estimated at any argument x_n. In the case of $y' = f(x, y)$, we cannot do this without estimating y_n first. Therefore, we run into the catch-22 situation. The two groups of methods to deal with it (to estimate y'_n without knowing y_n) rely on using multiple points that are already available from the previous integration steps (*multistep methods*) or the capability to estimate y'_n based on some trial points obtained within a single subinterval $x \in [x_{n-1}; x_n]$ (*single-step methods*).

14.3 Single-Step Methods

One group of methods for solving the IVP, *Runge–Kutta* (RK) *methods*, exploits the idea of representing the solution of Eq. (14.16) for each argument x_{n+1}, $n = 0, 1, 2, \ldots$, in the following form:

$$y_{n+1} = y_n + h \sum_{i=1}^{m} a_i k_i \tag{14.17}$$

Formally, a_i are weighting coefficients and k_i are gradients (slopes) evaluated at m points within the nth-subinterval, $x \in [x_n; x_{n+1}]$, as follows:

$$\begin{aligned} k_1 &= f(x_n, y_n) \\ k_2 &= f(x_n + q_1 h, y_n + p_{11} k_1 h) \\ k_3 &= f(x_n + q_2 h, y_n + p_{21} k_1 h + p_{22} k_2 h) \\ k_4 &= f(x_n + q_3 h, y_n + p_{31} k_1 h + p_{32} k_2 h + p_{33} k_3 h) \\ &\vdots \\ k_m &= f\left(x_n + q_{m-1} h, y_n + h \sum_{j=1}^{m-1} p_{m-1,j} k_j\right) \end{aligned} \tag{14.18}$$

Here q_i, $i = 1, 2, \ldots, m-1$ $(0 < q_1 \le q_2 \le \cdots \le q_{m-1} \le 1)$ and p_{ij}, $i = 1, 2, \ldots, m-1, j = 1, 2, \ldots, i$ are coefficients defined using a special procedure discussed in the following section.

To distinguish these m intermediate points from the solution points (x_{n+1}, y_{n+1}), $n = 0, 1, 2, \ldots$, Eq. (14.17) is referred to as an *m-stage* formula. Geometrically, the sum in Eq. (14.17) can be interpreted as a representative slope y' over subinterval $x \in [x_n; x_{n+1}]$, based on the slopes estimated at m intermediate points, x_n and $x_n + q_i h$, $i = 1, 2, \ldots, m-1$. Hence, weighting coefficients a_i should satisfy the obvious weighting equation

$$\sum_{i=1}^{m} a_i = 1 \tag{14.19}$$

Utilizing m points implies computing $0.5(m^2 + 3m - 2)$ unknowns, which are the coefficients a_i, $i = 1, 2, \ldots, m$; q_i, $i = 1, 2, \ldots, m-1$; and p_{ij}, $i = 1, 2, \ldots, m-1$, $j = 1, 2, \ldots, i$. Intuitively, the more intermediate points m we use, the more accurate the integration procedure is. To understand where the values of these unknown coefficients can possibly come from, let us consider the simplest cases where $m = 1$ and $m = 2$. In doing this, we will follow the steps of Runge, who developed integration formulas for $m = 2$ and $m = 3$. Then, we will proceed with the general case of Eqs. (14.17) and (14.18) as formulated by Kutta.

14.3.1 Low-Order Formulas

According to Eqs. (14.17) and (14.18), in the case of $m = 1$, we only have to find a single coefficient a_1. The only possible solution comes from Eq. (14.19), so that $a_1 = 1$. Therefore,

$$y_{n+1} = y_n + h f(x_n, y_n) \tag{14.20}$$

A geometric interpretation of this integration scheme known as the *Euler method* is shown in Fig. 14.2a. The representative slope y' over subinterval $x \in [x_n; x_{n+1}]$ is estimated as $k_1 = f(x_n, y_n)$ using a single point (x_n, y_n), the beginning of subinterval, and a full step is made along the direction defined by this representative slope.

Equation (14.20) can also be obtained using the Taylor series expansion

$$y_{n+1} = y_n + hf(x_n, y_n) + \frac{h^2}{2!} f'(x_n, y_n) + \frac{h^3}{3!} f''(x_n, y_n) + O(h^4) \qquad (14.21)$$

As seen, the first two terms of Eq. (14.21) comprise Eq. (14.20). Moreover, Eq. (14.21) provides us with an estimate of the local truncation error (the very next term), which happens to be of the order of h^2. As seen in Fig. 14.2a, we started an integration step assuming that y_n is an exact solution to Eq. (14.16). Obviously, this is only the case for the very first point given as the IC. The global error builds at each step of integration, so that for the number of steps, inversely proportional to the step size h ($(x_f - x_0)\, h^{-1}$), the global error is always one order less than a local truncation error. Hence, the Euler method [Eq. (14.20)] supplies the first-order (of h) formula.

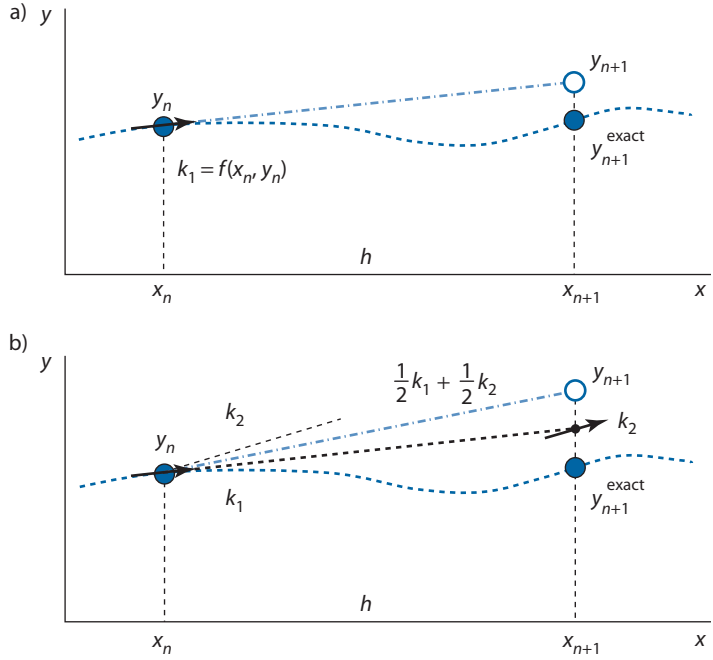


Fig. 14.2 a) Euler and b) modified Euler integration schemes.

Now, let us consider the case where $m = 2$. According to Eqs. (14.17) and (14.18), this leaves us with the following general formula:

$$y_{n+1} = y_n + h(a_1 k_1 + a_2 k_2) \tag{14.22}$$

where two slopes are defined at the origin of subinterval x_n and some intermediate point $x_n + q_1 h$:

$$\begin{aligned} k_1 &= f(x_n, y_n) \\ k_2 &= f(x_n + q_1 h, y_n + p_{11} k_1 h) \end{aligned} \tag{14.23}$$

The problem is in determining four unknown coefficients: a_1, a_2, q_1, and p_{11}. To address it, let us leave three terms in Eq. (14.21), yielding

$$y_{n+1} = y_n + hf(x_n, y_n) + \frac{h^2}{2!} f'(x_n, y_n) \tag{14.24}$$

[Intuitively, using more terms in Eq. (14.17) should lead to a higher accuracy, so leaving three terms in Eq. (14.21) instead of two makes the local truncation error to be of the order of h^3 and the global error being accrued while integrating Eq. (14.6) within $x \in [x_0; x_f]$ of the order of h^2.] Because the function f depends on two arguments, to determine f' in Eq. (14.24), we have to employ a chain-rule differentiation

$$f' = \frac{\partial f}{\partial x} + \frac{\partial f}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{\partial f}{\partial x} + \frac{\partial f}{\partial y} f \tag{14.25}$$

Substituting Eq. (14.25) into Eq. (14.24) results in

$$y_{n+1} = y_n + hf(x_n, y_n) + \frac{h^2}{2!}\left(\frac{\partial f}{\partial x} + \frac{\partial f}{\partial y} f\right) \tag{14.26}$$

Now, we have to expand the second equation in Eq. (14.23) as well:

$$\begin{aligned} k_2 &= f(x_n + q_1 h, y_n + p_{11} k_1 h) \\ &= f(x_n, y_n) + q_1 h \frac{\partial f}{\partial x} + p_{11} k_1 h \frac{\partial f}{\partial y} + O(h^2) \end{aligned} \tag{14.27}$$

This result can be substituted along with the first equation in Eq. (14.23) into Eq. (14.22) to yield

$$\begin{aligned} y_{n+1} &= y_n + h(a_2 + a_1) f(x_n, y_n) \\ &\quad + h^2 \left(a_2 q_1 \frac{\partial f}{\partial x} + a_2 p_{11} f(x_n, y_n) \frac{\partial f}{\partial y}\right) + O(h^3) \end{aligned} \tag{14.28}$$

By comparing the latter equation with Eq. (14.26), we determine that for the two equations to be equivalent, the following *order conditions*

must hold:

$$a_1 + a_2 = 1, \quad a_2 q_1 = \frac{1}{2}, \quad a_2 p_{11} = \frac{1}{2} \tag{14.29}$$

(for better understanding, the entire procedure is presented graphically in Fig. 14.3).

Employing symbolic manipulations, the script

```
%% Define symbolic variables
syms h fn a1 a2 k1 k2 q1 p11
syms epx epy dfdx dfdy
%% Make substitutions
Eq1 = h*(a1*k1+a2*k2);
sl2 = fn+epx*dfdx+epy*dfdy;
sl2 = subs(sl2,{epx,epy},{q1*h,p11*h*fn});
Eq1 = subs(Eq1,{k1,k2},{fn,sl2});
%% Define the reference equation
Eq2 = h*fn+h^2/factorial(2)*(dfdx+dfdy*fn);
%% Compare Eq1 and Eq2 by the orders of h
[Q1,R1] = quorem(Eq1,h^2,h);
[Q2,R2] = quorem(Eq2,h^2,h);
disp([Q1 Q2]);
disp([quorem(R1,h,h) quorem(R2,h,h)]);
```

produces

```
[ a2*(dfdx*q1 + dfdy*fn*p11), dfdx/2 + (dfdy*fn)/2]
[ a1*fn + a2*fn, fn]
```

allowing us to come up with Eq. (14.29). Comparing the corresponding terms in two expressions in each pair of brackets results in the same conditions as in Eq. (14.29).

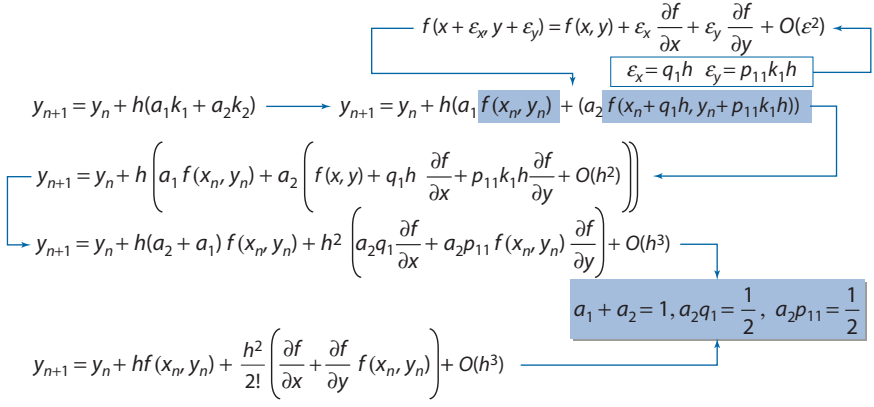


Fig. 14.3 Defining unknown coefficients for the two-stage integration scheme.

As seen, we end up having three nonlinear equations for four unknowns, which means the existence of infinite number of solutions. For example, allowing

$$a_1 = a_2 = \frac{1}{2} \tag{14.30}$$

gives

$$p_{11} = q_1 = \frac{1}{2a_2} = 1 \tag{14.31}$$

Coefficients (14.30) and (14.31) constitute the valid second-order formula known as the *Heun method* or the *modified Euler method*

$$y_{n+1} = y_n + h\left(\frac{1}{2}k_1 + \frac{1}{2}k_2\right) \tag{14.32}$$

with

$$\begin{aligned} k_1 &= f(x_n, y_n) \\ k_2 &= f(x_n + h, y_n + k_1 h) \end{aligned} \tag{14.33}$$

Geometrically, Eqs. (14.32) and (14.33) can be treated as a *prediction–correction method*, where eventually we are averaging slopes at two boundary points (Fig. 14.2b). Because we do not yet know the terminal point, we have to guess or make a prediction:

$$y^*_{n+1} = y_n + hf(x_n, y_n) \tag{14.34}$$

Then, we correct this prediction as

$$y_{n+1} = y_n + \frac{h}{2}\left(f(x_n, y_n) + f(x_n + h, y^*_{n+1})\right) \tag{14.35}$$

that is, making a full step in the direction $0.5(k_1 + k_2)$. Equations (14.34) and (14.35) are the same as Eqs. (14.32) and (14.33), but they are expressed in the predictor–corrector form. Note that, Fig. 14.2 is exaggerated in the sense that the step size h is too big, so that the slope y' varies in between x_n and x_{n+1} a lot. This is the reason that for the second-order method [Eq. (14.32); Fig. 14.2b], we ended up with the larger error compared with that of the first-order method [Eq. (14.20); Fig. 14.2a]. In fact, it may happen in practice as well, because according to Eq. (14.21), the local truncation error is not only proportional to the step size h but also to the higher-order derivative of the function f as well.

Another way to obtain Eq. (14.35) is to apply the trapezoidal rule to the corresponding integral

$$y_{n+1} = y_n + \int_{x_n}^{x_{n+1}} f(x, y)\mathrm{d}x$$

$$\cong y_n + \frac{h}{2}\left(f(x_n, y_n) + f(x_n + h, y^*_{n+1})\right) \tag{14.36}$$

Because y_{n+1} is not known (when we developed the trapezoidal rule in Sec. 13.3, we had $y' = f(x)$, not $y' = f(x,y)$), we approximate it by using the Euler method as in Eq. (14.34).

Another geometrically sound solution to Eq. (14.29) is the *midpoint method*, suggested by Runge, where we allow

$$a_1 = 0 \quad \text{and} \quad a_2 = 1 \tag{14.37}$$

Then, from Eq. (14.29),

$$p_{11} = q_1 = \frac{1}{2a_2} = \frac{1}{2} \tag{14.38}$$

Being substituted into Eqs. (14.22) and (14.23), it yields

$$y_{n+1} = y_n + hk_2 \tag{14.39}$$

with

$$\begin{aligned} k_1 &= f(x_n, y_n) \\ k_2 &= f\left(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_1 h\right) \end{aligned} \tag{14.40}$$

As seen from Fig. 14.4a, the second slope is estimated at the midpoint followed by the full step in this (k_2) direction. Equations (14.39) and (14.40) can also be cast in the predictor–corrector form:

$$y^*_{n+0.5} = y_n + \frac{h}{2}f(x_n, y_n) \tag{14.41}$$

$$y_{n+1} = y_n + hf\left(x_n + \frac{1}{2}h, y^*_{n+0.5}\right) \tag{14.42}$$

As mentioned, Eqs. (14.32) and (14.33) or Eqs. (14.39) and (14.40) are not the only two solutions of Eq. (14.29). From the practical point of view, the second point (along with x_n) can be chosen anywhere within subinterval $x \in [x_n + \frac{1}{2}h; x_{n+1}]$, resulting in $a_1 \in [0;\ 0.5]$. Presumably, among all second-order methods, obeying Eq. (14.29), the coefficients

$$a_1 = \frac{1}{3},\ a_2 = \frac{2}{3},\ p_{11} = q_1 = \frac{1}{2a_2} = \frac{3}{4} \tag{14.43}$$

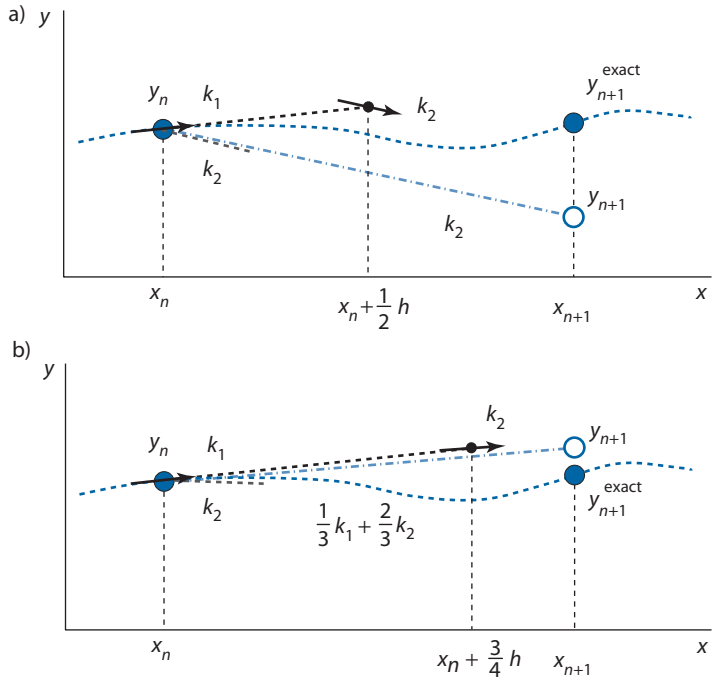


Fig. 14.4 a) Midpoint and b) Ralston integration schemes.

ensure the smallest local truncation error. Being substituted into Eqs. (14.22) and (14.23), it results in the *Ralston method*

$$y_{n+1} = y_n + h\left(\frac{1}{3}k_1 + \frac{2}{3}k_2\right) \tag{14.44}$$

with

$$\begin{aligned} k_1 &= f(x_n, y_n) \\ k_2 &= f\left(x_n + \frac{3}{4}h, y_n + \frac{3}{4}k_1 h\right) \end{aligned} \tag{14.45}$$

(Fig. 14.4b) or in the predictor–corrector form

$$y^*_{n+0.75} = y_n + \frac{3}{4}hf(x_n, y_n) \tag{14.46}$$

$$y_{n+1} = y_n + \frac{h}{3}\left(f(x_n, y_n) + 2f\left(x_n + \frac{3}{4}h, y^*_{n+0.75}\right)\right) \tag{14.47}$$

14.3.2 Higher-Order Formulas

Being illustrated for the case of $m = 1$ and $m = 2$, it is now clear how to use even more intermediate points (stages) within subinterval $x \in [x_n; x_{n-1}]$ [Eq. (14.17)]. Leaving four terms in Eq. (14.21) and following the same routine as in Fig. 14.3 leads to a system of six nonlinear algebraic equations, similar to those of Eq. (14.29), with eight unknown coefficients: a_1, a_2, a_3, q_1, q_2, p_{11}, p_{21}, and p_{22} [Eq. (14.18)]. Once again, an infinite number of solutions exists. To narrow possible variants Kutta suggested to enforce the following conditions (for all $i = 1, 2, \ldots, m$):

$$q_i = \sum_{j=1}^{i} p_{ij} \tag{14.48}$$

It does not help for the $m = 2$ case we just considered, but it allows simplifying the derivation of the order conditions [like Eq. (14.29)] for the higher-order integration schemes.

One common version of the *third-order RK method* results in

$$y_{n+1} = y_n + h\left(\frac{1}{6}k_1 + \frac{4}{6}k_2 + \frac{1}{6}k_3\right) \tag{14.49}$$

where

$$\begin{aligned} k_1 &= f(x_n, y_n) \\ k_2 &= f\left(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_1 h\right) \\ k_3 &= f(x_n + h, y_n - k_1 h + 2k_2 h) \end{aligned} \tag{14.50}$$

(that is, the slope estimates are taken at two boundary points and a midpoint). In the predictor–corrector form, it looks like

$$y^*_{n+0.5} = y_n + \frac{h}{2}f(x_n, y_n), \quad y^*_{n+1} = y_n + hf(x_n, y_n) \tag{14.51}$$

$$y_{n+1} = y_n + \frac{h}{6}\left(f(x_n, y_n) + 4f(x_n + 0.5h, y^*_{n+0.5}) + f(x_n + h, y^*_{n+1})\right) \tag{14.52}$$

Note that, if we deal with $y' = f(x)$ rather than $y' = f(x,y)$, then this third-order RK method reduces to the Simpson's 1/3 rule [Eq. (13.19)]. It uses three consecutive points and results in a local truncation error of the order of h^4. This transformation is presented in Fig. 14.5.

The most popular (and accurate) RK methods are the fourth-order methods. They are similar to the third-order schemes except that one more intermediate point is used to estimate a representative slope over subinterval. In this case, the analog of Eq. (14.29) [taking into account

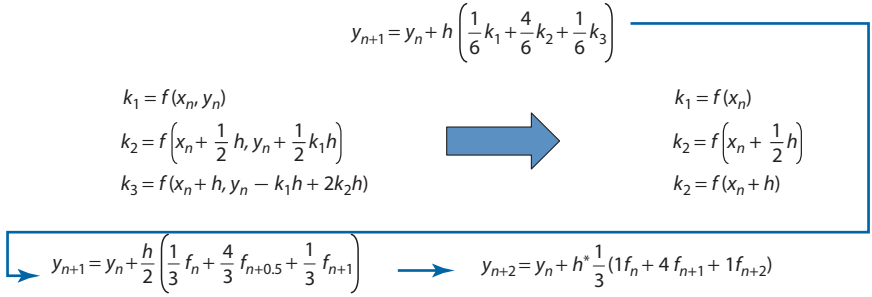


Fig. 14.5 Reduction of the third-order RK method to the Simpson's 1/3 rule for $y' = f(x)$.

Eq. (14.48)] is the following set:

$$\sum_{i=1}^{4} a_i = 1, \quad \sum_{i=2}^{4} a_i q_i = \frac{1}{2}, \quad \sum_{i=2}^{4} a_i q_i^2 = \frac{1}{3}, \quad \sum_{i=3}^{4} a_i \sum_{j=2}^{i-1} p_{ij} q_j = \frac{1}{6},$$
$$\sum_{i=2}^{m} a_i q_i^3 = \frac{1}{4}, \quad \sum_{i=3}^{4} a_i q_j \sum_{j=2}^{i-1} p_{ij} q_j = \frac{1}{8}, \quad \sum_{i=3}^{4} a_i \sum_{j=2}^{i-1} p_{ij} q_j^2 = \frac{1}{12}, \quad a_4 p_{43} p_{32} q_2 = \frac{1}{24} \tag{14.53}$$

As seen, we have eight nonlinear equations for 13 unknown coefficients, resulting in an infinite number of possibilities.

Among the variety of fourth-order methods, the *classical forth-order RK method* is used the most often. It is defined as

$$y_{n+1} = y_n + h\left(\frac{1}{6}k_1 + \frac{2}{6}k_2 + \frac{2}{6}k_3 + \frac{1}{6}k_4\right) \tag{14.54}$$

where

$$\begin{aligned} k_1 &= f(x_n, y_n) \\ k_2 &= f\left(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_1 h\right) \\ k_3 &= f\left(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_2 h\right) \\ k_4 &= f(x_n + h, y_n + k_3 h) \end{aligned} \tag{14.55}$$

or in the predictor–corrector form

$$\begin{aligned} y^*_{n+0.5} &= y_n + \frac{h}{2} f(x_n, y_n), \quad y^{**}_{n+0.5} = y_n + \frac{h}{2} f(x_n + 0.5h, y^*_{n+0.5}), \\ y^*_{n+1} &= y_n + hf(x_n + 0.5h, y^{**}_{n+0.5}) \end{aligned} \tag{14.56}$$

$$y_{n+1} = y_n + \frac{h}{6}\left(f(x_n, y_n) + 2f\left(x_n + \frac{1}{2}h, y^*_{n+0.5}\right) + 2f\left(x_n + \frac{1}{2}h, y^{**}_{n+0.5}\right) + f(x_n + h, y^*_{n+1})\right) \tag{14.57}$$

The second, also popular, version of the *fourth-order RK method* is

$$y_{n+1} = y_n + h\left(\frac{1}{8}k_1 + \frac{3}{8}k_2 + \frac{3}{8}k_3 + \frac{1}{8}k_4\right) \tag{14.58}$$

with

$$\begin{aligned} k_1 &= f(x_n, y_n) \\ k_2 &= f\left(x_n + \frac{1}{3}h, y_n + \frac{1}{3}k_1 h\right) \\ k_3 &= f\left(x_n + \frac{2}{3}h, y_n + \frac{1}{3}k_1 h + \frac{1}{3}k_2 h\right) \\ k_4 &= f(x_n + h, y_n + k_1 h - k_2 h + k_3 h) \end{aligned} \tag{14.59}$$

As with the third-order RK method [Eqs. (14.49) and (14.50)], in the case when $y' = f(x)$ rather than $y' = f(x,y)$, Eqs. (14.54) and (14.58) are reduced to the Simpson's 1/3 and 3/8 rules (Fig. 14.6), respectively.

As was the case with numerical integration (Chapter 13), although the higher-order RK formulas can be developed and are available, their stability properties worsen as m increases. Hence, instead of increasing m, the adaptive-step algorithms employing RK formulas with a moderate m were

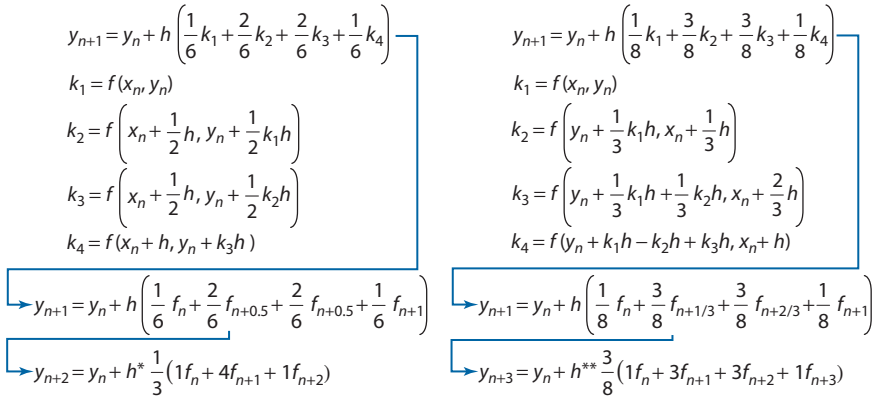


Fig. 14.6 Reduction of the fourth-order RK formulas to a) Simpson's 1/3 and b) 3/8 rules for $y' = f(x)$.

found to be more efficient. These algorithms are discussed in the following sections.

14.4 Adaptive Single-Step Algorithms

There are only two basic strategies for increasing the accuracy of numerical solution of ODE. They are

Option 1. Reducing an integration step size h
Option 2. Choosing a more accurate computational scheme

Unfortunately, as mentioned already, the dramatic success obtained by increasing the order from the Euler method to the fourth-order RK method does not continue above the fourth-order RK methods. The accuracy can be improved further, but the cost measured in increased number of function evaluations and computational instability makes these higher-order formulas less attractive. That leaves us with Option 1.

Obviously, decreasing the step size h also leads to the increased number of function evaluations. Although methods presented up to this point have used a fixed h, there is no reason why h cannot be changed from one subinterval to another. Hence, the key to further increasing the efficiency of numerical schemes is to minimize the overall number of steps necessary to achieve a solution with the predefined accuracy by adoptively varying h. The only complication is in devising a criterion for automatically choosing the value of h.

To be efficient, an algorithm for adjusting h must relate changes in h to the local accuracy of the solution. The objective is to reduce h when it is necessary to meet a user-specified error tolerance ε and to increase h when doing so will not cause the error to exceed the tolerance. For this to work, the ODE integration scheme must somehow estimate the local truncation error.

One way of making step size adjustments on the go is to apply the *step-halving procedure* introduced in Sec. 13.7. It involves computing each iteration twice, once as a full step and then independently as two half steps. The difference in the two results represents an estimate of the actual local truncation error. If there is no difference, then there is no advantage in decreasing h. If there is a significant difference, then we assume the solution obtained with the full step is inaccurate. If y_1 designates the full-step estimate and y_2 designates the estimate using the two half steps, the error Δ can be represented as

$$\Delta = y_2\left(\frac{1}{2}h\right) - y_1(h) \tag{14.60}$$

similar to Eq. (13.35). Along with providing a criterion for the step size control, Eq. (14.60) can also be used to correct the y_2 estimate [see the Richardson extrapolation formula (13.33)]. For the fourth-order RK

version, the correction is

$$y_2^{\text{MA}} = y_2 + \frac{\Delta}{15} \tag{14.61}$$

This estimate is fifth-order of h accurate.

Obviously, the step-halving procedure requires evaluation of two numerical solutions for each step, which is not efficient. Another way to estimate the local truncation error is to simultaneously advance the solution with two different-order schemes using the same value of h and the same estimates of the slope in the intermediate points. If the less accurate scheme results in $\hat{y}_{n+1}$ and the more accurate scheme – in y_{n+1}, so that

$$\Delta = \left| y_{n+1} - \hat{y}_{n+1} \right| \leq \varepsilon \tag{14.62}$$

then there is no need to decrease h. If, however, Eq. (14.62) does not hold, then h should be reduced. If Δ happens to be much smaller than ε, the step size h can probably be safely increased. MATLAB adaptive IVP solvers pursue exactly this idea.

The `ode23` solver of MATLAB uses the third-order Ralston formula to advance the solution and the second-order Bogacki–Shampine formula to monitor the accuracy. Specifically, formulas for the most and least accurate estimates are as follows:

$$y_{n+1} = y_n + h\left(\frac{2}{9}k_1 + \frac{1}{3}k_2 + \frac{4}{9}k_3\right) \tag{14.63}$$

$$\hat{y}_{n+1} = y_n + h\left(\frac{7}{24}k_1 + \frac{1}{4}k_2 + \frac{1}{3}k_3 + \frac{1}{8}k_4\right) \tag{14.64}$$

where

$$\begin{aligned} k_1 &= f(x_n, y_n) \\ k_2 &= f\left(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_1 h\right) \\ k_3 &= f\left(x_n + \frac{3}{4}h, y_n + \frac{3}{4}k_2 h\right) \\ k_4 &= f(x_n + h, y_{n+1}) \end{aligned} \tag{14.65}$$

First, three slopes, k_i, $i = 1, 2, 3$, are computed to produce y_{n+1}, then k_4 and $\hat{y}_{n+1}$ are evaluated. Note that despite the third-order result, y_{n+1} is used to compute one of the slopes for the second-order formula. The latter one's accuracy is still of the order of h^2 [having more terms in Eq. (14.64) compared with those of Eq. (14.63) does not necessarily mean a superb performance]. On the positive side, the choice of k_4 allows minimizing the number of function

evaluations because this last slope evaluation is then used at the very next subinterval $[x_{n+1};\ x_{n+2}]$ as k_1 [the so-called *first step as last step* (FSAL) paradigm].

In practical algorithms, the value of $\hat{y}_{n+1}$ is not even being computed. Instead, the local error estimate Δ [Eq. (14.62)] is computed explicitly in terms of the same four slopes as in Eq. (14.64)

$$\Delta = h\left|-\frac{5}{72}k_1 + \frac{1}{12}k_2 + \frac{1}{9}k_3 - \frac{1}{8}k_4\right| \tag{14.66}$$

This estimate Δ is used to come up with the step size h_{new} for the step

$$\frac{h_{\text{new}}}{h} \sim \left(\frac{\varepsilon}{\Delta}\right)^{1/p} \tag{14.67}$$

with p being the order of the method. For `ode23`, a constant power therefore equals $1/3$.

Similarly, the `ode45`, the default MATLAB solver, uses seven slope evaluations to calculate the fourth- and fifth-order accurate solutions using the Dormand–Prince pair

$$y_{n+1} = y_n + h\left(\frac{35}{384}k_1 + \frac{500}{1113}k_3 + \frac{125}{192}k_4 - \frac{2187}{6784}k_5 + \frac{11}{84}k_6\right) \tag{14.68}$$

$$\hat{y}_{n+1} = y_n + h\left(\frac{5179}{57600}k_1 + \frac{7571}{16695}k_3 + \frac{393}{640}k_4 - \frac{92097}{339200}k_5 + \frac{187}{2100}k_6 + \frac{1}{40}k_7\right) \tag{14.69}$$

where

$$\begin{aligned}
k_1 &= f(x_n, y_n)\\
k_2 &= f\left(x_n + \frac{1}{5}h, y_n + \frac{1}{5}k_1 h\right)\\
k_3 &= f\left(x_n + \frac{3}{10}h, y_n + \frac{3}{40}k_1 h + \frac{9}{40}k_2 h\right)\\
k_4 &= f\left(x_n + \frac{4}{5}h, y_n + \frac{44}{45}k_1 h - \frac{56}{15}k_2 h + \frac{32}{9}k_3 h\right)\\
k_5 &= f\left(x_n + \frac{8}{9}h, y_n + \frac{19372}{6561}k_1 h - \frac{25360}{2187}k_2 h + \frac{64448}{6561}k_3 h - \frac{212}{729}k_4 h\right)\\
k_6 &= f\left(x_n + h, y_n + \frac{9017}{3168}k_1 h - \frac{355}{33}k_2 h + \frac{46732}{5247}k_3 h + \frac{49}{176}k_4 h - \frac{5103}{18656}k_5 h\right)\\
k_7 &= f(x_n + h, y_{n+1})
\end{aligned} \tag{14.70}$$

As in the case of the `ode23` solver, instead of computing the value of $\hat{y}_{n+1}$, the local error estimate is computed as

$$\Delta = h\left|\frac{71}{57600}k_1 - \frac{71}{16695}k_3 + \frac{71}{1920}k_4 - \frac{17253}{339200}k_5 + \frac{22}{525}k_6 - \frac{1}{40}k_7\right| \tag{14.71}$$

The constant power in Eq. (14.67) for the `ode45` solver equals 1/5.

Many other pairs that share the slope estimates and allow for monitoring of accuracy are also known, but only the aforementioned two made it into MATLAB code.

14.5 Multistep Methods

As described in the previous Secs. 14.3 and 14.4, single-step methods use only one previous point to compute the following one, that is, the initial point (x_0, y_0) is used to compute (x_1, y_1) and only (x_n, y_n) is needed to compute (x_{n+1}, y_{n+1}). Indeed within each subinterval, $x \in [x_n; x_{n+1}]$, they do use multiple function evaluations to estimate a representative slope but they do not take advantage of employing multiple points obtained at the preceding integration steps.

On the contrary, multistep methods attempt to gain efficiency by keeping and using information about several prior steps, specifically $s+1$ values of the function $f(x_{n-i}, y_{n-i}) = f(x_n - ih, y_{n-i}) = f_{n-i}$, $i = 0,1, \ldots, s$.

14.5.1 Adams–Bashforth–Moulton Predictor–Corrector Method

Adams–Bashforth and Adams–Moulton multistep methods rely on the fundamental theorem of calculus

$$y_{n+1} = y_n + \int_{x_n}^{x_{n+1}} f^*(x)\mathrm{d}x \tag{14.72}$$

where the original function $f(x, y)$ of Eq. (14.16) is substituted with a polynomial approximation $f^*(x)$ based on the $s+1$ prior points. This allows you to take a definite integral in Eq. (14.72) analytically, so instead of Eq. (14.17), you now have

$$y_{n+1} = y_n + h\sum_{i=0}^{s} a_i f(x_{n-i}, y_{n-i}) \tag{14.73}$$

Essentially, Eq. (14.73) can be treated the same way as Eq. (14.17), so that the sum represents some derivative (slope) averaged over the interval $[x_{n-s};x_n]$.

The *Adams–Bashforth–Moulton predictor–corrector method*, incorporated in the MATLAB `ode113` solver, involves computing Eq. (14.73) twice for the original multipoint interval $[x_{n-s};x_n]$ and then for the shifted interval $[x_{n-s+1};x_{n+1}]$. That the difference between two estimates can be used to evaluate the local truncation error. This error is then used to determine whether the step size h is small enough to obtain an accurate value for y_{n+1}, yet large enough so that unnecessary and time-consuming calculations are eliminated. As will be shown next, as opposed to the single-step methods, using a combination of predictor and corrector requires only two function evaluations of $f(x, y)$ per integration step. Hence, the `ode113` solver may be more efficient than say `ode45`, especially when the function $f(x, y)$ is particularly expensive to evaluate (in terms of CPU time).

Let us consider how this works assuming $s = 3$, that is, relying on four prior points. With four points, the third-order Lagrange polynomial approximation for $f^*(x)$ in Eq. (14.72) can be written as

$$f^*(x) = Kx^3 + Lx^2 + Mx + N \tag{14.74}$$

Having four pairs, (x_{n-3}, y_{n-3}), (x_{n-2}, y_{n-2}), (x_{n-1}, y_{n-1}), and (x_n, y_n), allows you to solve for unknown coefficients K, L, M, and N by resolving the following system of LAEs:

$$\begin{bmatrix} x_n^3 & x_n^2 & x_n & 1 \\ (x_n-h)^3 & (x_n-h)^2 & x_n-h & 1 \\ (x_n-2h)^3 & (x_n-2h)^2 & x_n-2h & 1 \\ (x_n-3h)^3 & (x_n-3h)^2 & x_n-3h & 1 \end{bmatrix} \begin{bmatrix} K \\ L \\ M \\ N \end{bmatrix} = \begin{bmatrix} f_n \\ f_{n-1} \\ f_{n-2} \\ f_{n-3} \end{bmatrix} \tag{14.75}$$

The *Adams–Bashforth prediction* step then becomes

$$y_{n+1}^* = y_n + \int_{x_n}^{x_n+h} f^*(x)\mathrm{d}x = y_n + \frac{1}{4}h^4K + \frac{1}{3}h^3L + \frac{1}{2}h^2M + hN \tag{14.76}$$

As usual, we can employ the Symbolic Math Toolbox to find the solution to Eqs. (14.75) and (14.76). The script

```
%% Define symbolic variables
syms x_n h f_nm3 f_nm2 f_nm1 f_n f_np1 y_n f_np1ast x
%% Solve for coefficients of a cubic interpolation
A = [     x_n^3         x_n^2     x_n    1;
     (x_n-h)^3     (x_n-h)^2   x_n-h    1;
   (x_n-2*h)^3   (x_n-2*h)^2   x_n-2*h  1;
   (x_n-3*h)^3   (x_n-3*h)^2  x_n-3*h  1];
b= [f_n;f_nm1;f_nm2;f_nm3];
coef = A\b;
%% Produce a prediction
In = int([x^3 x^2 x 1]*coef,x_n,x_n+h);
In = simplify(In); pretty(y_n+In)
```

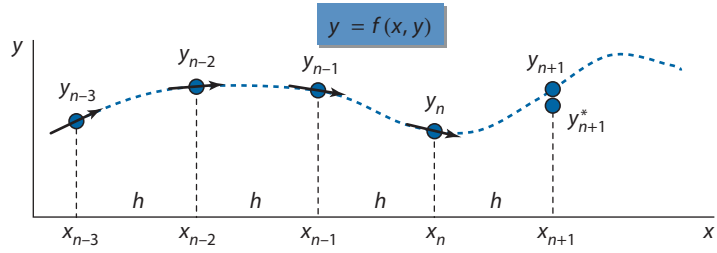


Fig. 14.7 Illustration of the four-point Adams–Bashforth prediction geometry.

results in

```
        h (55 f_n - 59 f_nm1 + 37 f_nm2 - 9 f_nm3)
y_n + ------------------------------------------
                          24
```

Therefore,

$$y^*_{n+1} = y_n + \frac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}) \tag{14.77}$$

[compare with a general solution of Eq. (14.73)].

The *Adams–Moulton corrector* is developed similarly, but this time shifting four points to the right, that is, using (x_{n-2}, y_{n-2}), (x_{n-1}, y_{n-1}), (x_n, y_n), and (x_{n+1}, y^*_{n+1}). The script

```
%% Define symbolic variables
syms x_n h f_nm2 f_nm1 f_n f_np1ast x y_n
%% Solve for coefficients of a correction cubic interpolation
A = [ (x_n+h)^3   (x_n+h)^2     x_n+h  1;
         x_n^3       x_n^2       x_n  1;
     (x_n-h)^3   (x_n-h)^2     x_n-h  1;
   (x_n-2*h)^3 (x_n-2*h)^2   x_n-2*h  1];
b= [f_np1ast; f_n; f_nm1; f_nm2];
coef = A\b;
%% Produce a correction
In = int([x^3 x^2 x 1]*coef,x_n,x_n+h);
In = simplify(In); pretty(y_n+In)
```

helps to obtain the result

```
        h (19 f_n - 5 f_nm1 + f_nm2 + 9 f_np1ast)
   y_n + ----------------------------------------
                          24
```

Hence,

$$y_{n+1} = y_n + \frac{h}{24}\left(9f^*_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}\right) \tag{14.78}$$

where f^*_{n+1} denotes $f(x_n + h, y^*_{n+1})$.

The alternative way to get Eqs. (14.77) and (14.78) is to employ the Taylor series expansion as we did in Chapter 13. The benefit is that along with prediction and correction formulas, you can also obtain the estimates of the local truncation error. The script

```
%% Define symbolic variables
syms h y_n f_n f_nm1 f_nm2 f_nm3 f_prime f_2prime f_3prime f_4prime
%% Introduce an equation for the forward Taylor series expansion around x_n
y_np1 = y_n+h*f_n+h^2*f_prime/factorial(2)+h^3*f_2prime/factorial(3)...
              +h^4*f_3prime/factorial(4)+h^5*f_4prime/factorial(5);
%% Introduce equations for f_nm3, f_nm2, and f_nm1
Eqm1 = -f_nm1+f_n-h*f_prime+h^2*f_2prime/factorial(2) ...
          -h^3*f_3prime/factorial(3)+h^4*f_4prime/factorial(4);
Eqm2 = -f_nm2+f_n-2*h*f_prime+(2*h)^2*f_2prime/factorial(2)...
          -(2*h)^3*f_3prime/factorial(3)+(2*h)^4*f_4prime/factorial(4);
Eqm3 = -f_nm3+f_n-3*h*f_prime+(3*h)^2*f_2prime/factorial(2)...
          -(3*h)^3*f_3prime/factorial(3)+(3*h)^4*f_4prime/factorial(4);
%% Resolve three equations for f_prime, f_2prime, and f_3prime
sol=solve(Eqm1,Eqm2,Eqm3,f_prime,f_2prime,f_3prime);
%% Substitute the estimates of derivatives into the y_np1 equation
y_np1=subs(y_np1,{f_prime,f_2prime,f_3prime},...
                 {sol.f_prime,sol.f_2prime,sol.f_3prime});
%% Collect coefficients and display the result
r=collect(y_np1,h); pretty(r)
```

returns

```
251 f_4prime  5    / 55 f_n     59 f_nm1    37 f_nm2     3 f_nm3  \
------------ h +  | ------  -  --------  +  --------  -  -------  |  h + y_n
      720          \  24          24          24           8     /
```

that is,

$$e_{n+1}^{*} = y_{n+1}^{\text{exact}} - y_{n+1}^{*} = \frac{251}{720} h^5 y^{(iv)}(\chi^{*}) = C_4^{*} h^5 y^{(iv)}(\chi^{*}) \tag{14.79}$$

Here y_{n+1}^{exact} denotes the exact (unknown) solution at x_{n+1} and χ^{*} is some representative point within the $[x_{n-3}; x_n]$ interval. Similarly, for the correction stage, the script

```
%% Define symbolic variables
syms h y_n f_n f_nm1 f_nm2 f_nm3 f_prime f_2prime f_3prime f_4prime f_np1
%% Introduce an equation for the forward Taylor series expansion around x_n
y_np1 = y_n+h*f_n+h^2*f_prime/factorial(2)+h^3*f_2prime/factorial(3)...
         +h^4*f_3prime/factorial(4)+h^5*f_4prime/factorial(5);
%% Introduce equations for f_nm2, f_nm1, and f_np1
Eqp1 = -f_np1+f_n+h*f_prime+h^2*f_2prime/factorial(2)...
       +h^3*f_3prime/factorial(3)+h^4*f_4prime/factorial(4);
Eqm1 = -f_nm1+f_n-h*f_prime+h^2*f_2prime/factorial(2)...
       -h^3*f_3prime/factorial(3)+h^4*f_4prime/factorial(4);
Eqm2 = -f_nm2+f_n-2*h*f_prime+(2*h)^2*f_2prime/factorial(2)...
```

```
      -(2*h)^3*f_3prime/factorial(3)+(2*h)^4*f_4prime/factorial(4);
%% Resolve three equations for f_prime, f_2prime, and f_3prime
sol=solve(Eqm1,Eqm2,Eqp1,f_prime,f_2prime,f_3prime);
%% Substitute the estimates of derivatives into the y_np1 equation
y_np1=subs(y_np1,{f_prime,f_2prime,f_3prime},...
                 {sol.f_prime,sol.f_2prime,sol.f_3prime});
%% Collect coefficients and display the result
r=collect(y_np1,h); pretty(r)
```

returns

```
/   19 f_4prime \  5   / 19 f_n    5 f_nm1    f_nm2    3 f_np1 \
| - ----------- | h  + | ------ - ------- + ----- + ------- | h + y_n
\       720     /      \   24        24        24         8    /
```

so that

$$e_{n+1} = y_{n+1}^{\text{exact}} - y_{n+1} = -\frac{19}{720}h^5 y^{(iv)}(\chi) = C_4 h^5 y^{(iv)}(\chi) \tag{14.80}$$

(χ is some representative point within the $[x_{n-2}; x_{n+1}]$ interval).

The local truncation error for both the predictor and corrector happens to be of the same order of h ($O(h^5)$). It is safe to assume that the average fifth-order derivatives $y^{(iv)}(\chi^*)$ and $y^{(iv)}(\chi)$ are close enough, so that $y^{(iv)}(\chi^*) \approx y^{(iv)}(\chi)$. This enables us to exclude them from Eqs. (14.79) and (14.80) to produce an estimate of the local truncation error based on the two computed values y_{n+1}^* and y_{n+1}:

$$\begin{aligned} e_{n+1} = y_{n+1}^{\text{exact}} - y_{n+1} &\approx \frac{C_4}{C_4^* - C_4}(y_{n+1} - y_{n+1}^*) \\ &= -\frac{19}{270}(y_{n+1} - y_{n+1}^*) \end{aligned} \tag{14.81}$$

This equation can be used to determine when to change the integration step size similar to the manner demonstrated in Fig. 13.8. Although elaborate methods are available, the following simple procedure shows how to reduce the step size to $h/2$ or increase it to $2h$ (Fig. 14.8). When the predicted and corrected values do not agree to five significant digits (Sec. 8.5.1), that is,

$$\frac{19}{270}\frac{|y_{n+1} - y_{n+1}^*|}{|y_{n+1}| + 10^{-5}} > \varepsilon^{rel} \tag{14.82}$$

where ε^{rel} is the desired relative tolerance, then the step size should be reduced to $h/2$ [compare Eq. (14.82) with Eq. (8.27)]. If they agree to seven or more significant digits, that is,

$$\frac{19}{270}\frac{|y_{n+1} - y_{n+1}^*|}{|y_{n+1}| + 10^{-5}} < \frac{\varepsilon^{rel}}{100} \tag{14.83}$$

then the step size can be safely increased to $2h$.

Reducing the step size requires four new starting values. Interpolation (14.74) is used to supply the missing values resulting in

$$f_{n-0.5} = \frac{f_{n-3} - 5f_{n-2} + 15f_{n-1} + 5f_n}{16}$$
$$f_{n-1.5} = \frac{f_{n-3} + 9f_{n-2} - 9f_{n-1} - f_n}{16} \tag{14.84}$$

Increasing the step size is an even easier task. Seven prior points are needed to double the step size. The four new points are obtained by omitting every other one (note, the point (x_{n-5},y_{n-5}) in Fig. 14.8 is not shown).

Some other multistep schemes (*Mine–Simpson method*, *Hamming method*, etc.) are available, but the self-contained MATLAB `ode113` solver utilizes the Adams–Bashforth–Moulton family of formulas of a variable order (1–13) in a manner similar to that shown for the four-point version above. The procedure starts with just one point—the IC y_0—and the Euler formula [Eq. (14.20)], which is an analog of the left rectangles formula of Eq. (13.2), is applied to produce y_1^*. Then, the corrector uses the backward Euler formula (which is the same as the right rectangles formula of Eq. (13.2)), to produce y_1 (in fact, for the first step the correction phase can be omitted). Now that we have two points to play with, the predictor uses a two-point linear approximation in Eq. (14.72) to produce y_2^*, followed by the first-order corrector to come up with y_2 using the analog of the trapezoidal rule of Eq. (13.3), etc. Table 14.1 presents formulas for the first five steps of this procedure. The last row in Table 14.1 shows how to compute coefficients a_1 in Eq. (14.73) for the remaining s, $s = 5, 6,\ldots,12$. Once the highest order of 13 is achieved, the solver proceeds with using 13 prior points until the end, adjusting the step size to meet the tolerance.

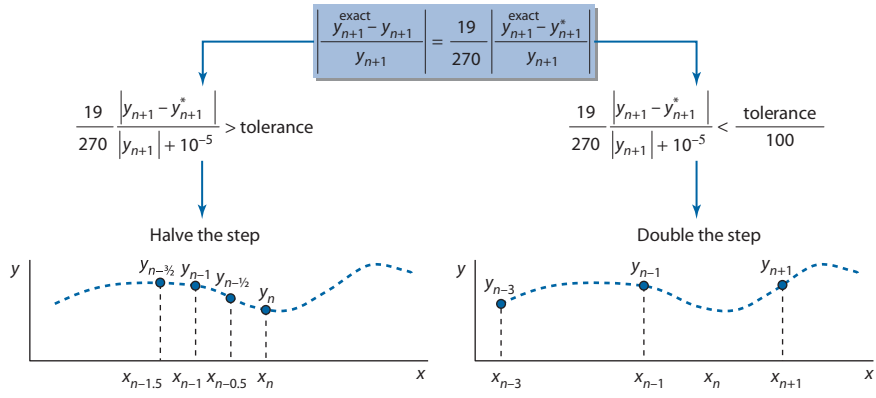


Fig. 14.8 Adaptive application of the Adams–Bashforth–Moulton four-step method.

Table 14.1 Initialization of the `ode113` Solver

Step	Prediction	Correction
0	$y_1^* = y_0 + hf_0$	$y_1 = y_0 + hf_1^*$
1	$y_2^* = y_1 + h\left(\frac{3}{2}f_1 - \frac{1}{2}f_0\right)$	$y_2 = y_1 + h\left(\frac{1}{2}f_2^* + \frac{1}{2}f_1\right)$
2	$y_3^* = y_2 + h\left(\frac{23}{12}f_2 - \frac{4}{3}f_1 + \frac{5}{12}f_0\right)$	$y_3 = y_2 + h\left(\frac{5}{12}f_3^* + \frac{2}{3}f_2 - \frac{1}{12}f_1\right)$
3	$y_4^* = y_3 + h\left(\frac{55}{24}f_3 - \frac{59}{24}f_2 + \frac{37}{24}f_1 - \frac{3}{8}f_0\right)$	$y_4 = y_3 + h\left(\frac{3}{8}f_4^* + \frac{19}{24}f_3 - \frac{5}{24}f_2 + \frac{1}{24}f_1\right)$
4	$y_5^* = y_4 + h\left(\frac{1901}{720}f_4 - \frac{1387}{360}f_3 + \frac{109}{30}f_2 - \frac{637}{360}f_1 + \frac{251}{722}f_0\right)$	$y_5 = y_4 + h\left(\frac{251}{720}f_5^* + \frac{646}{720}f_4 - \frac{264}{720}f_3 + \frac{106}{720}f_2 - \frac{19}{720}f_1\right)$
	$a_{s-k} = \frac{(-1)^k}{k!(s-k)!}\int_0^1 \prod_{\substack{i=0 \\ i\neq k}}^{s} (v+i)dv, \quad k = 0, 1, \ldots, s$	$a_{s-k+1} = \frac{(-1)^k}{k!(s-k+1)!}\int_0^1 \prod_{\substack{i=0 \\ i\neq k}}^{s+1} (v+i-1)dv, \quad k = 0, 1, \ldots, s$

Table 14.2 Coefficients for the Local Truncation Error Estimates

s	0	1	2	3	4
Predicting phase, C^*_{s+1}	1/2	5/12	3/8	251/720	95/288
Correcting phase, C_{s+1}	—	−1/12	−1/24	−19/720	−3/160

The Milne estimate for the local truncation error is described by Eq. (14.81), where the values of C^*_i and C_i for the first few steps are presented in Table 14.2.

14.5.2 Backward and Numerical Differentiation Formulas

The multistep method considered in the previous section was based on numerical integration, so that the integral in Eq. (14.72) was approximated numerically using Lagrange polynomials. Ironically, another group of multistep methods, developed even before the RK methods, relies on an exactly opposite idea and is based on numerical differentiation.

The underlying idea of the backward differentiation method is that if $s+1$ solutions $y_{n-s}, y_{n-s+1}, \ldots, y_n$ of Eq. (14.16) [Eq. (14.15)] are known, then the derivative at (x_{n+1}, y_{n+1}), $f_{n+1} = f(x_{n+1}, y_{n+1})$ can be estimated numerically using backward difference approximation formulas similar to those developed in Chapter 12 for the first-order derivative (Table 12.1). Using notations of this chapter, we can rewrite Eq. (12.25) as

$$hf_{n+1} = \sum_{m=1}^{s} \frac{1}{m} \Delta^m y_{n+1} \tag{14.85}$$

where the difference operator Δ was defined by Eqs. (12.23) and (12.24). The first- to fifth-order *backward differentiation formulas* (BDFs) are shown in Table 14.3.

Unfortunately, these formulas are implicit [f_{n+1} appears on both sides of Eq. (14.85)]. (The explicit formulas, similar to those of Eq. (14.85), but

Table 14.3 First- to Fifth-Order BDFs

s	Formula
1	$y_{n+1} - y_n = hf_{n+1}$
2	$\frac{3}{2}y_{n+1} - 2y_n + \frac{1}{2}y_{n-1} = hf_{n+1}$
3	$\frac{11}{6}y_{n+1} - 3y_n + \frac{3}{2}y_{n-1} - \frac{1}{3}y_{n-2} = hf_{n+1}$
4	$\frac{25}{12}y_{n+1} - 4y_n + 3y_{n-1} - \frac{4}{3}y_{n-2} + \frac{1}{4}y_{n-3} = hf_{n+1}$
5	$\frac{137}{60}y_{n+1} - 5y_n + 5y_{n-1} - \frac{10}{3}y_{n-2} + \frac{5}{4}y_{n-3} - \frac{1}{5}y_{n-4} = hf_{n+1}$

with f_n on the right-hand side, can also be developed, but so far they have proved to be unstable.) Therefore, the solution to Eq. (14.85) should be obtained iteratively, for example, using the simple fixed-point iteration method described in Sec. 10.3.1. The iteration process for solving Eq. (14.85) starts with the initial guess

$$y_{n+1}^{(0)} = y_n + hf_n = y_n + \sum_{m=1}^{s} \frac{1}{m} \Delta^m y_n \tag{14.86}$$

and continues until the tolerance is met

$$\left| y_{n+1}^{(r)} - y_{n+1}^{(r-1)} \right| \leq \varepsilon^{rel} \left| y_{n+1}^{(r)} \right| + \varepsilon^{abs} \tag{14.87}$$

Similar to formulas of Table 14.1, the BDF of Table 14.3 constitutes a variable-order self-starting algorithm implemented in the `ode15s` solver. The step size h is typically kept constant unless there is good reason to change it. To be more precise, the `ode15s` solver relies on a slightly modified version of the aforementioned approach (and offers BDF as an option). In this approach, to improve convergence of the iterative process of finding y_{n+1}, Eq. (14.85) is modified to add an extra term, proportional to the estimate of truncation error:

$$hf_{n+1} = \sum_{m=1}^{s} \frac{1}{m} \Delta^m y_{n+1} - \kappa (y_{n+1} - y_{n+1}^{(0)}) \sum_{m=1}^{s} \frac{1}{m} \tag{14.88}$$

Here, $y_{n+1}^{(0)}$ is defined by Eq. (14.86) and the parameter κ is chosen for each s to reduce the truncation error as much as possible while still ensuring algorithm's stability. Equation (14.88) is referred to as a *numerical differentiation formula* (NDF). For example, the first-order NDF reduces to

$$hf_{n+1} = y_{n+1} - y_n - \kappa (y_{n+1} - 2y_n + y_{n-1}) \tag{14.89}$$

For the BDFs of Table 14.3, the best value of κ used in NDFs is presented in Table 14.4. As seen, the fifth-order NDF coincides with the fifth-order BDF (because $k = 0$).

The `ode15s` solver of MATLAB, employing NDFs and BDFs, along with another variable-order solver, `ode15i`, which only employs NDFs, is recommended for the *stiff* problems that cannot be handled effectively by the implicit `ode23`, `ode45`, and `ode113` solvers. The following section provides more details on the stiffness issue and describes a few more methods to handle it.

Table 14.4 The Values of Parameter κ in the First- Through Fifth-order NDF

s	1	2	3	4	5
κ	−37/200	−1/9	−0.0823	−0.0415	0

14.6 Stiff Problems and Other Issues

For more or less homogeneous functions without singularities (let us refer to them as *nonstiff systems*), methods considered in Secs. 14.3–14.5 work fairly well, so you might never need anything else (especially, if you are a beginner). Formally, stiffness means that the solution can change on a timescale that is very short compared with the integration step, creating the need to decrease the integration step to very small values. Hence, methods that were not designed to handle stiff systems (realized in `ode23`, `ode45`, and `ode113` solvers) may work extremely slowly (if at all). For these types of problems, the MATLAB ODE suite offers some alternative solvers — `ode23s`, `ode23tb`, and `ode23t`. Although these solvers implement more sophisticated algorithms, the terminology introduced earlier in this chapter still applies. Hence, 23 in the name of these solvers reflects the fact that they use the second- and third-order pairs, similar to what `ode23` does.

Mathematically, the stiffness of the problem can be determined by looking at the ratio of maximum and minimum real parts among all eigenvalues of the Jacobian matrix **J**. That is why all solvers for the stiff problems take this matrix into account.

14.6.1 Incorporation of the Jacobian Matrix

A well-known approach for improving algorithm's stability is to use *implicit* rather than explicit formulas. In this case, the general RK solution given by Eq. (14.17) remains the same, but the slope estimates of Eq. (14.18) will have the quantity on the left-hand side appearing inside the function on the right-hand side. For example,

$$k_2 = f(x_n + q_1 h, y_n + p_{11} k_1 h + p_{12} k_2 h) \tag{14.90}$$

Instead of applying the iterative process to solve these equations for the slopes (in our specific case – to find k_2), we can try to linearize them, which results in a new class of methods. In our case, depending on what function value we linearize it about, we may obtain

$$k_2 = f(x_n + q_1 h, y_n + p_{11} k_1 h) + J p_{12} k_2 h \tag{14.91}$$

or even

$$k_2 = f(x_n, y_n) + T q_1 h + J p_{11} k_1 h + J p_{12} k_2 h \tag{14.92}$$

In these formulas, J denotes the partial derivative of the function f with respect to y, and T – the partial derivative of the function f with respect to x. [Note that, for Eqs. (14.91) and (14.92), J is estimated at two different points.] In the case of Eq. (14.15), **J** and **T** would denote the Jacobian

matrix and gradient vector, respectively

$$\mathbf{J} = \begin{bmatrix} \dfrac{\partial F_1}{\partial y_1} & \dfrac{\partial F_1}{\partial y_2} & \cdots & \dfrac{\partial F_1}{\partial y_l} \\ \dfrac{\partial F_2}{\partial y_1} & \dfrac{\partial F_2}{\partial y_2} & \cdots & \dfrac{\partial F_2}{\partial y_l} \\ \vdots & \vdots & \ddots & \vdots \\ \dfrac{\partial F_l}{\partial y_1} & \dfrac{\partial F_l}{\partial y_2} & \cdots & \dfrac{\partial F_l}{\partial y_l} \end{bmatrix} \quad \text{and} \quad \mathbf{T} = \begin{bmatrix} \dfrac{\partial F_1}{\partial x} \\ \dfrac{\partial F_2}{\partial x} \\ \vdots \\ \dfrac{\partial F_l}{\partial x} \end{bmatrix} \tag{14.93}$$

By rearranging terms, Eqs. (14.91) and (14.92) become

$$(1 - Jp_{12}h)k_2 = f(x_n + q_1 h, y_n + p_{11}k_1 h) \tag{14.94}$$

and

$$(1 - Jp_{12}h)k_2 = f(x_n, y_n) + Tq_1 h + Jp_{11}k_1 h \tag{14.95}$$

respectively. It is exactly the $1 - Jp_{12}h$ term [$\mathbf{1} - hp_{12}\mathbf{J}$, in the case of Eq. (14.15)] that makes the algorithm more robust (stable) [similar to what we had in Eq. 14.29)]. This approach, introduced by Rosenbrock, allows you to have an infinite number of unknown coefficients to choose from. Imposing additional constraints allows narrowing the number of candidate solutions and improving computational effectiveness [recall, this approach has been used before – adding Eq. (14.48) led to Eq. (14.53)]. In this particular case, two constraints can be used. The first one is that the Jacobian matrix is to be computed only once for each subinterval. The second one is that p_{12} is to be a constant value for all slope estimates.

The `ode23s` solver uses an effective FSAL pair (similar to that of Eqs. (14.63)–(14.65) for the `ode23` solver) as suggested by Shampine and Reichelt, so that the next step is computed using a third-order formula

$$y_{n+1} = y_n + hk_2 \tag{14.96}$$

where

$$\begin{aligned} k_1 &= (1 - hdJ)^{-1}(f(x_n, y_n) + hdT) \\ k_2 &= (1 - hdJ)^{-1}\left(f\left(x_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_1\right) - k_1\right) + k_1 \\ k_3 &= (1 - hdJ)^{-1}\left(f(x_{n+1}, y_{n+1}) - (6 + \sqrt{2})\left(k_2 - f\left(x_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_1\right)\right)\right. \\ &\quad \left. -2\,(k_1 - f(x_n, y_n)) + hdT\right) \end{aligned} \tag{14.97}$$

and $d = 1/(2+\sqrt{2})$. As earlier [Eq.(14.66)], instead of estimating the second-order formula, the error estimate is computed directly as

$$\Delta = \frac{h}{6}|k_1 - 2k_2 + k_3| \tag{14.98}$$

Although Eqs. (14.96)–(14.98) seem to be much more complicated compared with Eqs. (14.63)–(14.65), they preserve the same structure. As opposed to explicit formulas, Eqs. (14.96)–(14.98) feature and benefit from using additional information about the function $f(x, y)$ contained in **J** and **T**.

To be able to evaluate the solution anywhere within subinterval $x \in [x_n; x_{n+1}]$ (for example, for the purpose of rescaling), the quadratic polynomial is used:

$$y_{n+c} = y_n + h\left(\frac{c(1-c)}{1-2d}k_1 + \frac{c(c-2d)}{1-2d}k_2\right) \tag{14.99}$$

Obviously, at $c = 0$, it assumes the value of y_n, and, at $c = 1$, it becomes Eq. (14.96).

14.6.2 Using the Implicit Trapezoidal Rule

MATLAB offers two solvers, `ode23t` and `ode23tb`, that use the implicit trapezoidal rule. The `ode23tb` solver uses it as the first formula in the variable-step pair in a specific form

$$y_{n+\gamma} = y_n + \gamma h\frac{1}{2}(f_n + f_{n+\gamma}) \tag{14.100}$$

Equation (14.100) produces the estimate of solution not at x_{n+1}, but at some point $x_{n+\gamma}$, $0<\gamma \leq 1$. As the second formula, the `ode23tb` solver uses the second-order BDF (Table 14.3). In this case, the parabolic approximation should pass through three points, (x_n, y_n), $(x_{+\gamma}, y_{+\gamma})$, and (x_{n+1}, y_{n+1}). Taking its derivative and estimating it at x_{n+1} produces the estimate of the derivative. Employing the self-explained script

```
%% Define symbolic variables
syms x_n h gam y_n y_ngam y_np1 x
%% Solve for coefficients of a quadratic interpolation
A = [        x_n^2           x_n    1;
    (x_n+gam*h)^2     x_n+gam*h    1;
        (x_n+h)^2         x_n+h    1];
b= [y_n;y_ngam;y_np1];
coef = A\b;
%% Form the polynomial out of the vector of its coefficients
Cpol=[x^2 x 1]*coef;
%% Compute the first-order derivative at x=x_i
D1 = diff(Cpol,1); D1=subs(D1,x,x_n+h); pretty(simplify(D1))
```

yields

```
                                                                2           2
 y_n - y_ngam - 2 gam  y_n + 2  gam y_np1 + gam    y_n - gam   y_np1
- ------------------------------------------------------------------------
                          gam h  (gam - 1)
```

that is,

$$(2-\gamma)y_{n+1} - \frac{1}{\gamma}y_{n+\gamma} + \frac{(1-\gamma)^2}{\gamma}y_n = (1-\gamma)hf_{n+1} \tag{14.101}$$

Linearizing Eqs. (14.100) and (14.101) reveals that the corresponding multipliers, including the Jacobian matrix (see the previous section), are

$$2\mathbf{I} - \gamma h\mathbf{J} \quad \text{and} \quad (2-\gamma)\mathbf{I} - (1-\gamma)h\mathbf{J} \tag{14.102}$$

respectively. They assume the same form if

$$\frac{2}{\gamma} = \frac{2-\gamma}{1-\gamma} \tag{14.103}$$

Equation (14.103) results in $\gamma = 2 - \sqrt{2}$. Along with providing the best computational effectiveness (both implicit stages within the iterative process for each step use the same multiplier), this choice of γ ensures the least truncation error and largest stability region.

This trapezoidal-rule/BDF2 combination (TR-BDF2) can also be viewed as an implicit RK pair of the orders of 2 and 3:

$$y_{n+1} = y_n + h(wk_1 + wk_2 + dk_3) \tag{14.104}$$

where

$$\begin{aligned} k_1 &= f\,(x_n,\ y_n) \\ k_2 &= f\,(x_n + \gamma h,\ y_n + dhk_1 + dhk_2) \\ k_3 &= f\,(x_{n+1},\ y_{n+1}) \end{aligned} \tag{14.105}$$

$d = 0.5\gamma$ and $w = 0.25\sqrt{2}$. As usual, instead of estimating the second-order formula, the error estimate is computed directly as

$$\Delta = h\left|\frac{1-w}{3}k_1 + \frac{3w+1}{3}k_2 + \frac{d}{3}k_3\right| \tag{14.106}$$

Because at each step the TR-BDF2 produces three solutions and three derivative values at x_n, $x_{n+\gamma}$, and x_{n+1} to interpolate the solution anywhere within subinterval $x \in [x_n; x_{n+1}]$, it is natural to employ the piecewise cubic Hermite interpolant [Eq. (11.85)]

$$y(x) = (v_3 - 2v_2)\rho^3 + (3v_2 - v_3)\rho^2 + v_1\rho + v_0 \tag{14.107}$$

with the coefficients for $x \in [x_n;\ x_{n+\gamma}]$ and $x \in [x_{n+\gamma};\ x_{n+1}]$ given in Table 14.5.

Table 14.5 Coefficients for the `ode23tb` Cubic Hermite Interpolant

Interval	Argument $p(x)$	v_3	v_2	v_1	v_0
$x \in [x_n; x_{n+\gamma}]$	$(x-x_n)/(\gamma h)$	$\gamma h(f_{n+\gamma} - f_n)$	$y_{n+\gamma} - y_n - v_1$	$\gamma h f_n$	y_n
$x \in [x_{n+\gamma}; x_{n+1}]$	$(x-x_{n+\gamma})/((1-\gamma)h)$	$(1-\gamma)h(f_{n+1} - f_{n+\gamma})$	$y_{n+1} - y_{n+\gamma} - v_1$	$(1-\gamma) h f_{n+\gamma}$	$y_{n+\gamma}$

The iteration procedure for the first stage [Eq. (14.100)] starts with the simplest initial guess

$$f_{n+\gamma}^{(0)} = f_n \tag{14.108}$$

(the desire to maintain a single-step structure prevents using the points from the previous steps) and then proceeds as

$$y_{n+\gamma}^{(r)} = (y_n + dhf_n) + dhf_{n+\gamma}^{(r)}$$

$$D^{(r)} = (\mathbf{I} - hd\mathbf{J})^{-1} h(f(x_{n+\gamma}, y_{n+\gamma}^{(r)}) - f_{n+\gamma}^{(r)}) \quad \text{and} \quad f_{n+\gamma}^{(r+1)} = \frac{f_{n+\gamma}^{(r)}}{hD^{(r)}} \tag{14.109}$$

until the accuracy tolerance is met. Once $\hat{y}_{n+\gamma}$ is found, Eq. (14.101) is solved with the initial guess

$$f_{n+1}^{(0)} = (1.5 + \sqrt{2}) f_n + (2.5 + 2\sqrt{2}) f_{n+\gamma} - (6 + 4.5\sqrt{2})(\hat{y}_{n+\gamma} - y_n) h^{-1} \tag{14.110}$$

resulting from extrapolating the derivative of the piecewise cubic Hermite spline [Eq. (14.107)] to $x_{n+\gamma}$. The iterative process for the second stage unfolds as follows:

$$y_{n+\gamma}^{(r)} = (y_n + whf_n + whf_{n+\gamma}) + dhf_{n+1}^{(r)}$$

$$D^{(r)} = (\mathbf{I} - hd\mathbf{J})^{-1} h(f(x_{n+1}, y_{n+1}^{(r)}) - f_{n+1}^{(r)}) \quad \text{and} \quad f_{n+1}^{(r+1)} = \frac{f_{n+1}^{(r)}}{hD^{(r)}} \tag{14.111}$$

One more implicit solver of MATLAB, `ode23t`, uses an implicit trapezoidal rule in its common form

$$y_{n+1} = y_n + h\frac{1}{2}(f_n + f_{n+1}) \tag{14.112}$$

Equation (14.112) is equivalent to Eq. (14.100) with $\gamma = 1$. It relies on using a cubic interpolant to estimate the local truncation error and adjust the step size (to be discussed later).

14.6.3 Controlling the Integration Step Size

As discussed in the previous sections, the local truncation error is largely defined by the step size h. The decision on whether to keep the current integration step size or change it is based on this error estimate. Some of the methods introduced in this chapter have an embedded capability to estimate these errors (for example, the RK pairs). Let us introduce here a general step-size control strategy for other methods that may lack these embedded formulas.

Assume that the local truncation error is proportional to the second derivative, say, $e \sim (h^2/2)|y''|$, as for the Euler method or BDF1. Suppose we have computed y_{n-1}, y_n and, consequently, f_{n-1}, f_n corresponding to x_{n-1}, x_n already. Then, the value of y^n can be estimated as

$$\hat{y}'' = \frac{f_n - f_{n-1}}{x_n - x_{n-1}} \tag{14.113}$$

If this estimate stays about the same for the next subinterval, an attempt to estimate the next step size h can be based on the requirement to meet the error tolerance ε:

$$\hat{e} \sim \frac{h^2}{2}\left|\frac{f_n - f_{n-1}}{x_n - x_{n-1}}\right| \le \varepsilon \tag{14.114}$$

This leads to

$$h \le \sqrt{2\varepsilon\left|\frac{x_n - x_{n-1}}{f_n - f_{n-1}}\right|} \tag{14.115}$$

In practice, h is usually selected to be less than that dictated by Eq. (14.115), say, about 0.9 of the prescribed value. Also, we do not want the step size to be either too large, greater than some $h_{\max}$, or too low, so that the computer cannot distinguish between x_n and $x_n + h$. Therefore, the typical size selection strategy would become

$$h \le \max\left(16*\texttt{eps}*|x_n|,\ \min\left(\sqrt{2\varepsilon^{rel}\left|\frac{x_n - x_{n-1}}{f_n - f_{n-1}}\right|},\ h_{\max}\right)\right) \tag{14.116}$$

Here, ε^{rel} represents the relative accuracy and `eps`, the distance between 1 and the next floating-point number (Table 8.5).

If the local truncation error is proportional to a higher-order derivative, like $|e_n| \sim (h^2/12)y_n'''$ for the trapezoidal rule of `ode23t`, Eq. (14.113) would not apply. In this case, several prior points should be used to develop an approximating polynomial, which can be differentiated analytically and estimated at x_n or even at x_{n+1}. For example, the `ode23t` solver uses y_{n-3}, y_{n-2}, y_{n-1}, and y_n to produce a cubic approximation [Eq. (14.107)].

One more question is how to choose the initial step size when the only available piece of data is the differential equation itself and ICs. Of course,

you may start with an a priori excessively small step size, but that may waste a lot of computing time. An alternative, "smart" approach used in most of solvers is as follows.

First, we need to evaluate the function $\mathbf{F}(x,\mathbf{y})$ at the initial point. Next, we estimate the error as

$$\Delta = \frac{1}{\max\left(|x_0|,\,|x_f|\right)^{p-1}} + \left\|\mathbf{F}(x_0,\,y_0)\right\|^{p+1} \tag{14.117}$$

where p is the order of the method and $\|\cdots\|$ denotes the Euclidean norm. Then, the initial step can be chosen as

$$h = \left(\frac{\varepsilon}{\Delta}\right)^{\frac{1}{p+1}} \tag{14.118}$$

which resembles Eq. (14.67).

Very often, however, the ICs are chosen to be at an equilibrium point, resulting in $\left\|\mathbf{F}(x_0,\,y_0)\right\|$ being very close to zero (because almost all components of $\mathbf{F}(x_0,\,y_0)$ are zeros). Thus, usually, choosing the initial integration step includes two more computations. First, the Euler method is applied with h followed from Eq. (14.118) to produce x_1,$\mathbf{y_1}$, and second, the estimates (14.117) and (14.118) are repeated for this next point. Finally, the initial integration step size is chosen as the smaller of the two estimates.

14.7 Using MATLAB IVP Solvers

To begin with, it should be noted that the MATLAB ODE suite has both adaptive (variable-step) and nonadaptive (fixed-step) solvers realizing different methods as described in Secs. 14.3–14.6. The latter solvers though do not have their *m*-file representation (they are programmed in C) and are only available/used in Simulink (to be discussed in Chapter 15). Adaptive solvers available in the MATLAB development environment (and Simulink) are presented in Table 14.6.

These variable-step solvers allow handling both nonstiff and stiff problems. Moreover, some of these solvers are suitable for handling not only the explicit ODEs of the form of Eq. (14.7) but also linearly implicit ODEs of the form

$$\mathbf{M}(\mathrm{x},\,\mathbf{y})\mathbf{y}' = \mathbf{F}(\mathrm{x},\,\mathbf{y}) \tag{14.119}$$

with $\mathbf{M}(x,\mathbf{y})$ being the *mass matrix*, and even fully implicit ODEs

$$\mathbf{F}\left(x,\,\mathbf{y},\,\mathbf{y}'\right) = \mathbf{0} \tag{14.120}$$

The most common syntax for MATLAB solvers is one of the following:

```
[T,Y] = solver(odefun,tspan,y0)
[T,Y] = solver(odefun,tspan,y0,options)
[T,Y] = solver(odefun,tspan,y0,options,p1,p2,...)
```

Table 14.6 MATLAB Solvers for Explicit, Linearly Implicit, and Fully Implicit ODEs

Solver	Order of Accuracy	Description
Solvers for Nonstiff Problems		
`ode45`	Medium	Explicit single-step RK solver based on the Dormand–Price 4/5 pair [Eqs. (14.68) and (14.69)]. Suitable for nonstiff problems that require moderate accuracy. This is the first solver to try on a new problem
`ode23`	Low	Explicit single-step RK solver, based on the Bogacki–Shampine 2/3 pair [Eqs. (14.63) and (14.64)]. Suitable for problems that tolerate lower accuracy or exhibit moderate stiffness
`ode113`	Low to high	Multistep Adams–Bashforth–Moulton 1st- to 13th-order solver (Table 14.1). Suitable for nonstiff problems that require moderate to high accuracy involving problems, where $\mathbf{F}(x, y)$ is expensive to compute. Not suitable for problems where $\mathbf{F}(x, y)$ is not smooth
Solvers for Stiff Problems		
`ode15s`	Low to medium	Multistep solver of varying order (up to the default fifth), relying on NDF. Options allow integration with BDF (Table 14.3) and integration with a maximum order less than the default. It is suitable for stiff problems that require moderate accuracy. This is typically the solver to try if `ode45` fails or is too slow
`ode23s`	Low	Single-step solver based on the implicit modified second-order Rosenbrock formula [Eqs. (14.96)–(14.98)]. Suitable for stiff problems, where lower accuracy (compared with that of `ode15s`) is acceptable or where $f(x,y)$ is discontinuous
`ode23t`	Low	Implicit single-step solver relying on the trapezoidal rule [Eq. (14.112)] and a cubic interpolant [Eq. (14.107)] to estimate the local truncation error. Suitable for moderately stiff problems. Can be used to solve differential algebraic equations
`ode23tb`	Low	Implicit RK-pair solver [Eqs. (14.104)–(14.106)] combining a trapezoidal rule of Eq. (14.100) with a second-order BDF [Eq. (14.101)]. Can be more efficient than `ode15s` if using crude tolerances
Solver for Fully Implicit ODEs		
`ode15i`	Low to medium	A multistep NDF solver of the first- to fifth-order, especially suitable for solving fully implicit differential equations

where `solver` is one of if those listed in the first column of Table 14.6. In these commands, `odefun` is the function that evaluates the system of ODEs [$\mathbf{F}(x,\mathbf{y})$], `tspan` is the vector specifying the interval of integration, `y0` is the vector of ICs (the solver imposes the vector of ICs `y0` at `tspan(1)`, and integrates from `tspan(1)` to `tspan(end)`), `options` is a structure of optional parameters that changes the default integration properties, and `p1,p2,...` are additional parameters that can be passed directly to the ODE function as `odefun(t, y,p1,p2,...)` and to all functions specified in `options`. Each row in

the solution array `Y` corresponds to a time returned in the column vector `T`. Additionally,

```
sol = solver(odefun,[t0 tf],y0,...)
```

returns a structure that can be used with the function `deval` to evaluate the solution (and derivative) at any point between `t0` and `tf`.

Before we proceed with examples of using these solvers, let us try to solve an IVP using the simplest formulas developed in Sec. 14.3. Let us start with the problem given by

$$\dot{y} + y = \sin(t), \quad y(0) = -5, \quad t \in [0; 10]\text{s} \tag{14.121}$$

First, we need to rewrite Eq. (14.121) in the form of Eq. (14.16)

$$\dot{y} = f(t, y) = \sin(t) - y \tag{14.122}$$

Now we can implement, for example, the Euler formula [Eq. (14.20)], which yields

$$y_{n+1} = y_n + h(\sin(t_n) - y_n) \tag{14.123}$$

The following script implements this formula and computes two solutions for two different step sizes as shown in Fig. 14.9a:

```
h1=0.01; h2=1;
ff = @(a,b) sin(a)-b;        % define the function
y(1)=-5; z(1)=-5;
t1=0:h1:10; N1=length(t1); % define the vector of arguments for h1
t2=0:h2:10; N2=length(t2); % define the vector of arguments for h2
tic
for i=2:N1, y(i)=y(i-1)+h1*ff(t1(i-1),y(i-1)); end % solution for h1
T1=toc;
tic
for i=2:N2, z(i)=z(i-1)+h2*ff(t2(i-1),z(i-1)); end % solution for h2
T2=toc;
plot(t1,y,t2,z,'--k.')
text(4,-2.0,['T^{h=0.01s}=' num2str(T1*1000,2) 'ms'])
text(4,-2.6,['T^{h=1s} =' num2str(T2*1000,2) 'ms'])
xlabel('Time, s'), ylabel('y'), legend('h=0.01s','h=1s', 'location' , 'e' )
```

Note that we wrapped each cycle with the `tic` and `toc` commands to start a stopwatch timer and measure the elapsed time, respectively. As expected (Fig. 14.9a), the $h = 0.01$ s solution demonstrates much better accuracy than that of $h = 1$ s, but it takes more time to compute. If the size h is increased even further, the integration process becomes unstable and solution diverges. Figure 14.9b shows that the ICs in ODE do matter. Almost identical script was used in this case, with the only difference that both step sizes were set to `0.01` and solutions were initialized with different

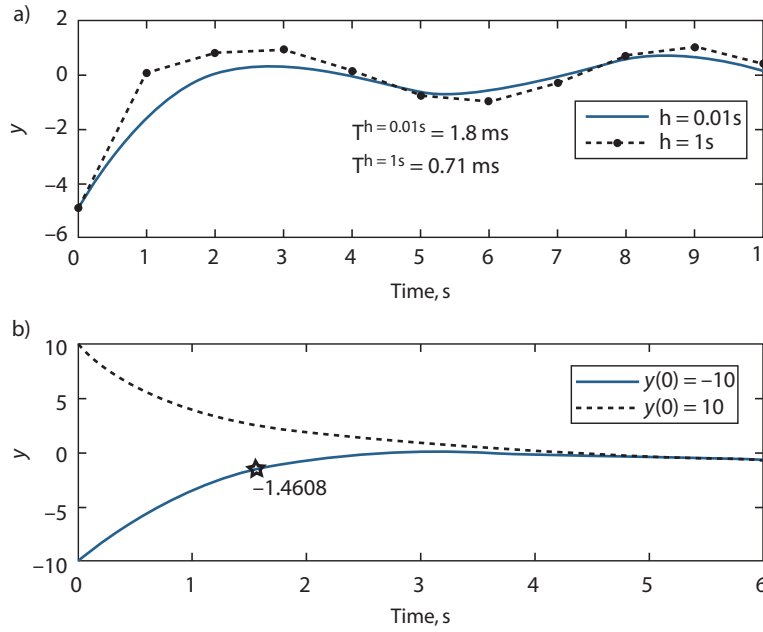


Fig. 14.9 Solving ODE with the Euler method varying a) the step size and b) IC.

values of $y(0)$. Also, as an example of how you can use the obtained solution to find the certain values, Fig. 14.9b features a star indicating a solution closest to the point $x = 0.5\pi$ for the $y(0) = -10$ case. The following piece of code shows how you can do it:

```
delta=t1-pi/2;
[C,ind]=min(abs(delta));      % find a point closest to pi/2
r=plot(t1(ind),y(ind),'rp');
text(t1(ind)+0.1,y(ind),num2str(y(ind)))
r.HandleVisibility='off';
```

(the last command removes the corresponding entry in the legend box that otherwise would show up – see Sec. 6.6).

Now, let us consider the classical second-order system introduced in the beginning of this chapter and described by Eq. (14.13). Assuming, $m = 1$ kg, $k = 1$ kg/s^2, $c = 1$ kg/s, and $u = \sin(t)$, let us rewrite it as

$$\dot{\mathbf{x}} = \begin{bmatrix} 0 & 1 \\ -1 & -1 \end{bmatrix} \mathbf{x} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} \sin(t) \tag{14.124}$$

The following script implements the Euler and Heun methods [Eq. (14.20) and (14.32), respectively] using the step size $h = 0.8$ s (Fig. 14.10a):

```
h=0.8;
ff = @(a,b) [b(2); -b(1)-b(2)+sin(a)];
```

```
y(:,1)=[-5; 0]; z(:,1)=[-5; 0];
t=0:h:15; N=length(t);
tic
for i=2:N
    y(:,i)=y(:,i-1)+h*ff(t(i-1),y(:,i-1));
end
T1=toc;
tic
for i=2:N
    k1=ff(t(i-1),z(:,i-1));
    k2=ff(t(i),z(:,i-1)+h*k1);
    z(:,i)=z(:,i-1)+h*(k1+k2)/2;
end
T2=toc;
plot(t,y(1,:),'.-.',t,z(1,:), '.-'),ylim(6*[-1 1])
text(8.2,-2.5,['t_{CPU}^{Euler}=' num2str(T1*1000,2) 'ms'])
text(8.2,-4.8,['t_{CPU}^{Heun}= ' num2str(T2*1000,2) 'ms'])
xlabel('Time, s'), ylabel('Position, m')
legend('Euler Method','Heun Method','location','ne')
```

As seen, the step size of $h = 0.8$ s for the lowest order possible (Euler method) solution not only produces a huge error as compared with that of the second-order method (Heun method) solution, but it also brings it on

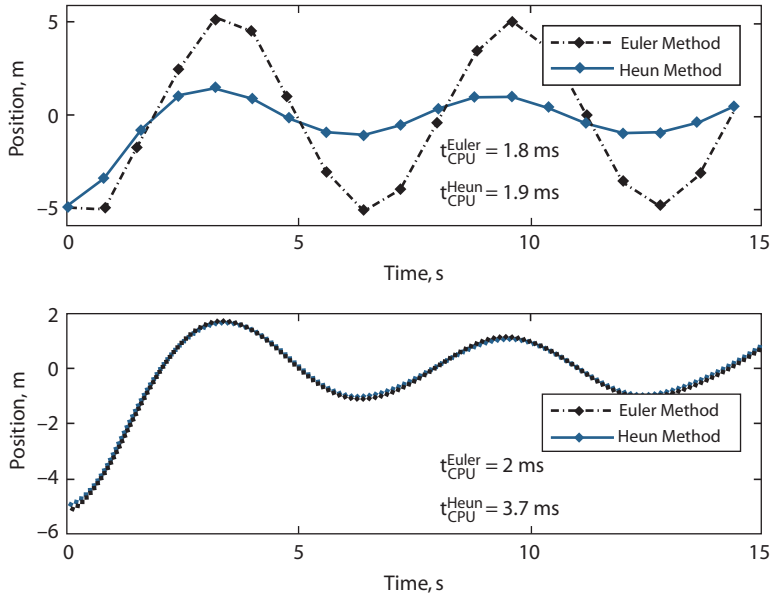


Fig. 14.10 Solving the second-order ODE with the lower-order methods using a) $h = 0.8$ s and b) $h = 0.08$ s.

the verge of stability (with $h > 0.8$ s, the Euler method solution diverges). Again, almost an identical script was used to produce another pair of solutions to the same problem, this time with $h = 0.08$ s (Fig. 14.10b). Both solutions seem to be closer to the exact solution (which in this case can be obtained analytically), with the Heun method solution being more accurate anyway. Note the difference in computational time in both cases. The Heun method employs twice as many function evaluations as the Euler method, and it clearly shows up for the $h = 0.08$ s solution of Fig. 14.10b. For the small number of nodes (Fig. 14.10a), the amount of CPU time spent on function evaluations is comparable to that of other mathematical operations, and that is why the difference in the CPU time for two solutions is smaller. (If needed, you can also estimate the effectiveness of your code using the **Profiler**, discussed in Sec. 5.11 (Fig. 5.26).)

Now let us show how to use the available MATLAB solvers (listed in Table 14.6). For example, the problem described by Eq. (14.122) can be addressed using the `ode23` solver as follows:

```
[t,y]=ode23(@(a,b)[b(2);-b(1)-b(2)+sin(a)],[0 15],[-5 0]);
plot(t,y(:,1),t,y(:,2),'-.k'), xlabel('Time, s'), ylabel('y')
legend({'Position','Speed'},'location','se')
```

This produces the result shown in Fig. 14.11a. Note that we used the same anonymous function as before, but in general the script evaluating ODEs

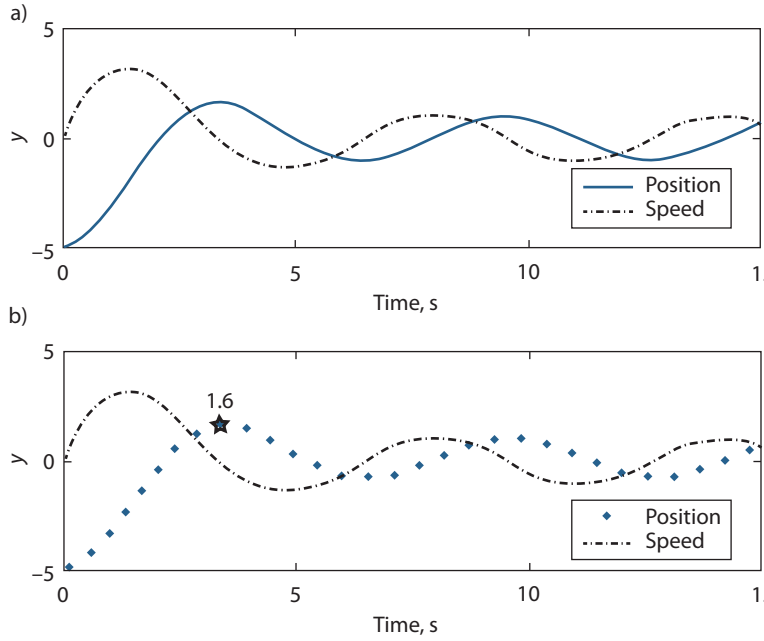


Fig. 14.11 Employing a) `ode23` solver and b) `ode113` solver.

should be stored as an *m*-file function. For example, such a function for Eq. (14.124) could look like

```
function dy = cart(t,y)
dy(1) = y(2);
dy(2) = -y(1) -y(2)+sin(t);
dy = dy'; % the output must be a column vector
```

We can also use another syntax employing a structure as an input, as shown in the following script, which produces Fig. 14.11b featuring both components of the solution of Eq. (14.124):

```
opt = odeset('RelTol',1e-2);
sol=ode113(@cart,[0 15],[-5 0],opt);
t=linspace(0,15,30); y = deval(sol,t);
h=plot(t,y(1,:),'.',t,y(2,:),'-.k'), xlabel('Time, s'), ylabel('y'), hold
legend(h,{'Position','Speed'},'location','se','autoupdate','off')
xst=pi;                yst = deval(sol,xst,1);
text(xst-0.3,yst+1,num2str(yst,2))plot(xst,yst,'pk')
```

In this case, you can use the `sol` structure with the `deval` function. The latter allows you to evaluate the ODE solution at any set of points residing within the interval for which the solution was obtained. (Another function, `odextend`, allows you to extend the solution beyond this interval, if you need to.) In our particular case, we call the `deval` function twice: first, to produce a solution on a vector `t` (note that solutions returned by the `deval` function are in the row vector format, not the column vector format); and second, to obtain just a single value of cart's position at a certain point, at $t = \pi$ s, (the third argument in this second command, `1`, indicates that we only need an estimate for the first component of solution, being a cart's position).

In the last script, we used the command that redefined some of the default `odeset` parameters (the default value of relative tolerance was set to `1e-2`, which corresponds to 1% accuracy). The `odeset` function lets you adjust a lot of other integration parameters of ODE solvers, including properties of a mass matrix $\mathbf{M}(x, \mathbf{y})$ in Eq. (14.119) and some specific parameters for `ode15s` and `ode15i` solvers, supplying an analytical Jacobian, which increases the speed and reliability of the solution for stiff problems, etc. Table 14.7 lists these and other properties, which can be obtained using the `odeget` function.

Note that at every integration step the solver estimates the local truncation error in each *i*th component of the solution. This error must be less than or equal to the acceptable error, which is a function of the specified relative tolerance, `RelTol`, and the specified absolute tolerance vector, `AbsTol`:

```
|e(i)| <= max(RelTol*abs(y(i)),AbsTol(i))
```

By default, MATLAB solvers use this more stringent component-wise error control as opposed to another error control

```
norm(e) <= max(RelTol*norm(y),AbsTol)
```

Table 14.7 `odeset` Parameters Available for Different Solvers

Parameters	ode45	ode23	ode113	ode15s	ode15i	ode23s	ode23t	ode23tb
RelTol, AbsTol, NormControl	+	+	+	+	+	+	+	+
OutputFcn, OutputSel, Refine, Stats	+	+	+	+	+	+	+	+
NonNegative	+	+	+	+/-	+/-		+/-	+/-
Events	+	+	+	+	+	+	+	+
MaxStep, InitialStep	+	+	+	+	+	+	+	+
Jacobian, JPattern, Vectorized				+	+	+	+	+
Mass, MStateDependence	+	+	+	+		+/-	+	+
MvPattern, MassSingular				+			+	+/-
InitialSlope				+			+	
MaxOrder				+	+			
BDF				+				

Table 14.8 `OutputFcn` Options

Function	Brief Description
`odeplot`	Produces argument (time) histories for all components of solution (which is a default option when you call the solver with no output arguments and you have not specified an output function)
`odephas2`	Plots a 2-D phase portrait
`plottingodephas3`	Plots a 3-D phase portrait
`odeprint`	Displays a solution as it is computed in the Command Window

which becomes available when the `NormControl` property is `on`.

Among other properties of `odeset`, you can specify an output function to provide a handle to the `OutputFcn` parameter, e.g., you can use one of the embedded functions shown in Table 14.8. For example, a simple command

```
ode113(@cart,[0 15],[-5 0],odeset('OutputFcn',@odeplot))
```

which is equivalent to

```
ode113(@cart,[0 15],[-5 0])
```

produces the plot shown in Fig. 14.12a, marking all computational nodes. Similarly, the commands

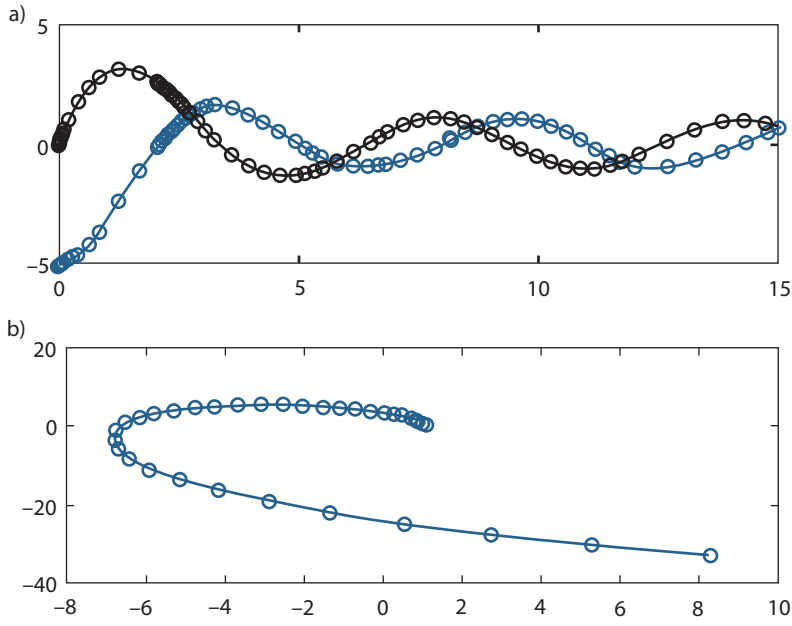


Fig. 14.12 Employing a) `odeplot` and b) `odephas2` functions.

```
opt = odeset('OutputFcn',@odephas2);
fun=@(a,b) [(b(1)-2*b(2))/3;2*b(1)+b(2)];
ode23t(fun,[-2 2],[1 1],opt)
```

produce the phase plane plot shown in Fig. 14.12b.

Figure 14.13 demonstrates a solution of the second-order equation, like the one described by Eq. (14.12), with two different state matrices **A** and $\mathbf{B} = \mathbf{0}$:

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} \tag{14.125}$$

The two plots in Fig. 14.13 feature

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \text{(Fig.14.13a)} \quad \text{and} \quad \mathbf{A} = \begin{bmatrix} -1 & 1 \\ 0 & -1 \end{bmatrix} \text{(Fig.14.13b)}$$

and are plotted as the phase space (x_2 vs x_1). The dots represent a variety of the ICs. The script to produce Fig. 14.13a is presented as follows:

```
hold
for i=-1:0.25:1
  for j=-1:0.1:1
    [t,y]=ode45(@(a,b)[b(1)+b(2);b(2)],[-2 2],[i j]);
   plot(y(1,1),y(1,2),'.r'), plot(y(:,1),y(:,2))
  end
end
xlabel('x_1'), ylabel('x_2'), axis([-1 1 -1.2 1.2]), axis equal
```

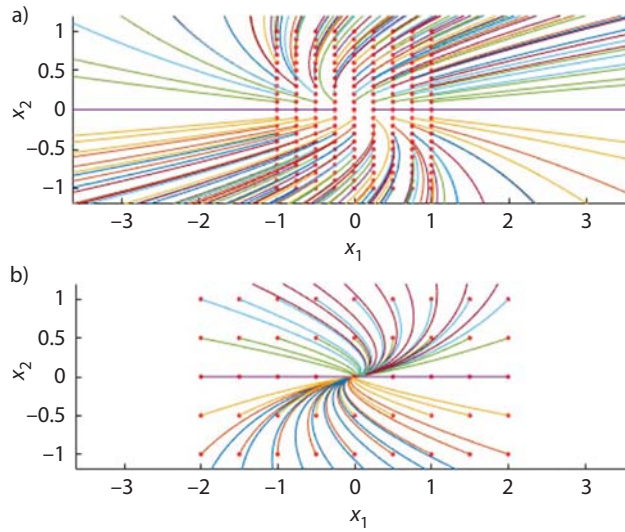


Fig. 14.13 Employing the `ode45` solver for two different state matrices in Eq. (14.125).

Some minor modification of the state matrix changing the anonymous function in the `ode45` call above produces Fig. 14.13b.

Figure 14.14a presents an example of solving a stiff problem

$$y' = \frac{1+x}{x}y, \quad x \in [-1; 1] \tag{14.126}$$

using the `ode23s` solver. As seen, this problem features a singularity at $x = 0$. Hence, none of the nonstiff problem solvers would work. The script used to produce this figure provides several ICs, so the plot presents a family of solutions with $y_0 \in [-5; 5]$

```
hold
for i=-5:0.5:5
    [x,y]=ode23s(@(x,y)(-1+x)/y,2*[-1 1],i);
    plot(x(1),y(1),'.r'), plot(x,y), xlabel('x'), ylabel('y')
end
```

Although for each solution MATLAB returns a warning saying that it was not able to meet the tolerance without reducing the step size below the smallest value allowed, it does return the solution in no time.

Using the `ode15i` solver slightly differs from all other solvers. First, the `odefun` argument has an implicit form of Eq. (14.120). Second, you must supply one extra input argument (defining the vector of ICs for a derivative), `yp0`, as in

```
[T,Y] = ode15i(odefun,tspan,y0,yp0)
```

To compute `yp0` (which may not be given upfront) MATLAB offers the `decic` function, making all ICs consistent (by ensuring that $\mathbf{F}(x_0, \mathbf{y}_0, \mathbf{y}'_0) = \mathbf{0}$)

```
[y0mod,yp0mod] = decic(odefun,t0,y0,fixed_y0,yp0,fixed_yp0)
```

Here you provide the values you do know and the guesses on parameters you do not know. Elements of the vectors `fixed_y0` and `fixed_yp0` indicate what is known for sure (`1`) and what can be varied (`0`). This function returns the modified (consistent) values of `y0` and `yp0` that can now be passed on to the `ode15i` solver.

Consider an example. Suppose we deal with the following implicit ODE:

$$\left((1 - 0.8\sin^2(x))y'\right)' - (x - \lambda)y = 0, \quad y(0) = 0, \quad y'(0) = 1, \quad x \in [0; \pi] \tag{14.127}$$

First, we need to bring this equation to two first-order equations. Introducing $z_1 = y$ and $z_2 = y'$, we obtain

$$\begin{aligned} z'_1 - z_2 &= 0 \\ -1.6\sin(x)\cos(x)z_2 + 1 - 0.8\sin^2(x)z'_2 - (x - \lambda)z_1 &= 0 \end{aligned} \tag{14.128}$$

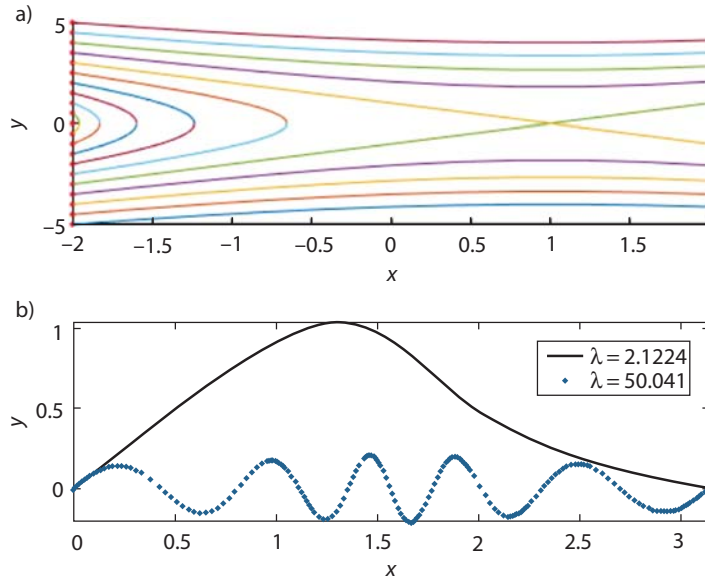


Fig. 14.14 Examples of employing a) `ode23s` solver for the stiff problem and b) `ode15i` solver for the implicit ODE.

Equation (14.128) can be programmed as an *m*-file function

```
function res = StLi(x,z,zp)
global p
res(1)=zp(1)-z(2);
res(2)=(-1.6*sin(x)*cos(x))*z(2)+(1-0.8*sin(x)^2)*zp(2)-(x-p)*z(1);
res=res';
```

(we used the `global` function to pass in the value of `lambda`).

Second, we need to establish the ICs. Although $\mathbf{z}(0) = [0,1]^T$ is given, we still need to find the values of $\mathbf{z}'(0)$. You can do it manually, resolving Eq. (14.128) at x_0, or you can simply use the `decic` function as follows:

```
[z0,zp0] = decic(@StLi,0,[0 1],[1 0],[0 0],[0 0]);
```

Here the first vector in the list of arguments is $\mathbf{z}(0)$ and the third one is our guess on $\mathbf{z}'(0)$. Although we know the values of $\mathbf{z}(0)$ for sure, we allowed one of them to be a varied parameter along with both components of $\mathbf{z}'(0)$ (look at the values of the second and forth input vectors).

Now, we can call the ODE solver. To this end, the following script computes two solutions for two values of λ, as shown in Fig. 14.14b:

```
global p
p=2.1224; [z0,zp0]=decic(@StLi,0,[0 1],[1 0],[0 0],[0 0]);
sol=ode15i(@StLi,[0 pi],z0,zp0);
```

```
plot(sol.x,sol.y(1,:)); hold
p=50.041; [z0,zp0]=decic(@StLi,0,[0 1],[1 0],[0 0],[0 0]);
sol=ode15i(@StLi,[0 pi],z0,zp0);
plot(sol.x,sol.y(1,:),'.'), axis tight, xlabel('x'), ylabel('y')
legend('\lambda=2.1224','\lambda=50.041','location','ne')
```

By the way, do not forget that you can use the `odeToVectorField` function of the Symbolic Math Toolbox to automate converting your ODEs to the state–space representation and then employ the `matlabFunction` function to create an anonymous function or *m*-file (Sec. 7.10.3).

Let us consider one more practical example showing the importance of monitoring solution accuracy. In the so-called circular restricted three-body problem, two massive bodies move in circular orbits around their common center of mass (known as the *barycenter*), with the third body of a negligibly small mass moving in between. For example, these three bodies may be Sun, Earth, and Moon, or Earth, Moon, and a spaceship. The normalized equations describing small body dynamics in a rotating reference frame associated with the first two bodies are

$$\begin{aligned} y_1'' &= y_1 + 2y_2' - (1-\mu)\frac{y_1+\mu}{d_1} - \mu\frac{y_1-\mu}{d_2} \\ y_2'' &= y_2 - 2y_1' - (1-\mu)\frac{y_2}{d_1} - \mu\frac{y_2}{d_2} \end{aligned} \tag{14.129}$$

where $0 < \mu \leq 1$ is the ratio of the two larger bodies' masses with

$$\begin{aligned} d_1(y_1,y_2) &= ((y_1+\mu)^2 + y_2^2)^{1.5} \\ d_2(y_1,y_2) &= ((y_1-(1-\mu))^2 + y_2^2)^{1.5} \end{aligned} \tag{14.130}$$

The fourth-order system of Eq. (14.129) can be represented by the following function:

```
function dx = threeBp(t,x)
global mu
dx=zeros(length(x),1); % define the output as a column vector
d1 = ((x(1)+mu)^2      + x(3)^2)^1.5;
d2 = ((x(1)-(1-mu))^2  + x(3)^2)^1.5;
dx(1) = x(2);
dx(2) = x(1)+2*x(4)-(1-mu)*(x(1)+mu)/d1-mu*(x(1)-(1-mu))/d2;
dx(3) = x(4);
dx(4) = x(3)-2*x(2)-(1-mu)*x(3)/d1 -mu*x(3)/d2;
```

Then, we can call this function using the following commands:

```
global mu
mu = 0.012277471; % Moon-to-Earth mass ratio
x0 = [ 0.994, 0, 0, -2.0015851];
```

```
tf = 17.065216;
[t,y] = ode45(@threeBp,[0 tf],x0,odeset('RelTol',1e-6));
plot(y(:,1),y(:,3)), hold
plot(y(1,1),y(1,3),'dk','MarkerSize',14,'MarkerFaceColor','k');
line(-mu,0,'Marker','o','MarkerSize',14,'MarkerFaceColor','b')
text(-mu,0.15,'Earth'), text(1-mu,0.1,'Moon'),axis equal
xlabel('Relative downrange'), ylabel('Relative crossrange')
```

These commands result in a famous periodic solution for the Earth–Moon–spaceship problem shown in Fig. 14.15a. However, if you degrade the relative

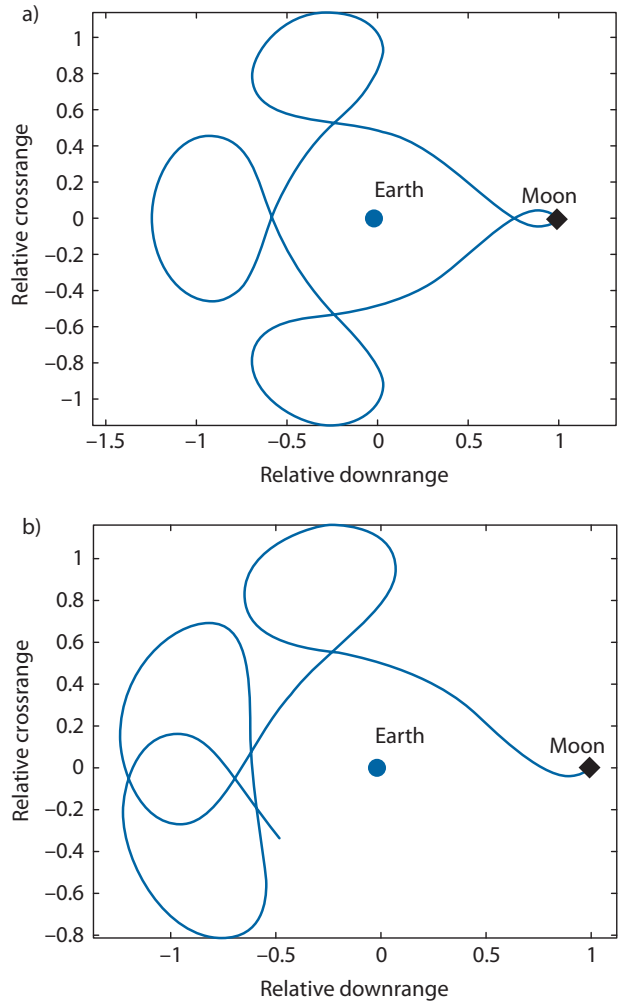


Fig. 14.15 Examples of the three-body problem trajectories.

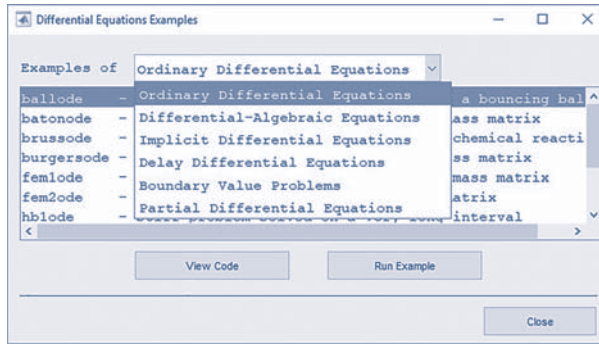


Fig. 14.16 Interactive Differential Equations GUI featuring examples of ODE solving.

tolerance of solution (its accuracy) to, say, 10^{-2} instead of 10^{-6}, you will end up with an erroneous trajectory as shown in Fig. 14.15b.

To conclude this section, let us mention that MATLAB provides an interactive GUI allowing you to see more examples of using the ODE solvers introduced in this section, as shown in Fig. 14.16. You can invoke this GUI, allowing you to run these examples and preview their source code, by typing in

```
>> odeexamples
```

14.8 Other MATLAB Solvers

For the sake of completeness, let us briefly mention other solvers available in MATLAB to address special-type IVPs, BVPs, and PDEs.

14.8.1 Solvers for IVPs with Delays

The `dde23` and `ddesd` solvers allow solving IVPs for delay differential equations (DDEs) with constant and general delays, respectively. The first ones can be described as

$$\dot{\mathbf{y}} = \mathbf{F}(t, \mathbf{y}(t), \mathbf{y}(t-\tau_1), \mathbf{y}(t-\tau_2), \ \ldots, \mathbf{y}(t-\tau_k)) \tag{14.131}$$

with $\tau_1, \tau_2, \ldots, \tau_k$ being the constant delays. The second ones—with time and state-dependent delays $d_1, d_2, \ldots, d_k$, as

$$\dot{\mathbf{y}} = \mathbf{F}(t, \mathbf{y}(t), \mathbf{y}(d_1(t, \mathbf{y})), \mathbf{y}(d_2(t, \mathbf{y})), \ \ldots, \mathbf{y}(d_k(t, \mathbf{y}))) \tag{14.132}$$

Compared with Eq. (14.131), the difference is that while looking for a solution within the interval $t \in [t_0; t_f]$, the values of $\mathbf{y}$ depend on the values of $\mathbf{y}$

at times prior to t_0. Hence, to be able to address Eq. (14.131) or (14.132), the history of the states $\mathbf{S}(t)$ prior to t_0 should be supplied.

Generally, the solution $\mathbf{y}(t)$ of IVP for a system of DDEs has a jump in its first derivative at the initial point t_0 because the first derivative of the history function does not satisfy the DDE there. This discontinuity propagates into the future. The DDEs solvers, `dde23` and `ddesd`, are designed to mitigate this phenomenon so that the solution becomes smoother as the integration proceeds. Specifically, `dde23` tracks low-order discontinuities and integrates differential equations with the explicit RK 2/3 pair and interpolant used by `ode23`. The RK formulas are implicit for step sizes longer than the delays. When the solution is smooth enough to justify large step sizes, the implicit formulas are evaluated by a predictor–corrector iteration. The `ddesd` solver integrates with the classic four-stage, fourth-order explicit RK method [Eq. (14.54)], controlling a magnitude of truncation error.

To use `dde23`, you must rewrite Eq. (14.131) as an equivalent system of the first-order differential equations, and code it in the *m*-file function.

For a example, assume that you start with the appropriately written system from the very beginning

$$\begin{aligned} \dot{y}_1 &= y_1(t-\tau_1)y_2(t) \\ \dot{y}_2 &= y_1(t-\tau_1) - y_2(t-\tau_2) \end{aligned} \tag{14.133}$$

You then code it as

```
function dydt = ddex1de(t,y,Z)
ylag1 = Z(:,1);
ylag2 = Z(:,2);
dydt = [ylag1(1)*y(2)
        ylag1(1)-ylag2(2)]; % the output must be a column vector
```

Next, you should supply the `dde23` solver with the delays organized as a vector `lags`. You should also code the history function like

```
function S = ddex1hist(t)
S = zeros(3,1);
```

Finally, you can apply the solver as

```
sol = dde23(@ddex1de,lags,@ddex1hist,range);
```

where `range` is the two-value vector defining the interval of integration. The output structure `sol` has the following main fields:

`sol.x` Contains a mesh selected by the solver to meet the tolerance
`sol.y` Represents approximation to $\mathbf{y}(t)$ at the mesh points in `sol.x`
`sol.yp` Provides approximation to $\dot{\mathbf{y}}(t)$ at the mesh points in `sol.x`

To evaluate the solution at some other mesh, you can use the `deval` function as described in the previous section.

For the state-dependent delays of Eq. (14.132), you should additionally create a function describing the state-variant delays, like

```
function lags = ddex3delay(t,y)
lags = exp(1 - y(2));
```

Then, you can use the following syntax:

```
sol = ddese(@ddex1de,@lags,@ddex1hist,range);
```

Consider a decease spreading in an imaginary space colony on Moon/Mars or a large spaceship (recall a Start Trek science fiction television series). The Kermack–McKendrick model for the number of people infected with a contagious illness in a closed population over time is given by

$$\begin{aligned} \dot{y}_1 &= -\beta y_1(t) y_2(t-\tau_1) \\ \dot{y}_2 &= \beta y_1(t) y_2(t-\tau_1) - \gamma y_2(t-\tau_2) \\ \dot{y}_3 &= \gamma y_2(t-\tau_2) \end{aligned} \tag{14.134}$$

Here t defines the normalized time; y_2 is the number of infected individuals who can pass on some virus or disease to others; y_1 is the number of susceptible individuals who have yet to contact the disease and become infectious; y_3 is the number of colony members who have been infected but cannot transmit the disease for some reason, for example, they have been isolated from the rest of the colony or developed immunity to the infection; β represents the probability of getting infected; and γ is the isolation effectiveness. This model also assumes the incubation period τ_1 and time τ_2 needed for isolation. For $t \leq 0$, consider the following values: $y_1 = 990$, $y_2 = 10$, and $y_3 = 0$. Equation (14.134) can be coded as

```
function dy = kmckd(t,y,Z)
ylag1 = Z(:,1);
ylag2 = Z(:,2);
dy = zeros(length(y),1);
dy(1) = - 0.01*y(1)*ylag1(2);
dy(2) = 0.01*y(1)*ylag1(2) - 0.2*ylag2(2);
dy(3) = 0.2*ylag2(2);
```

The call

```
>> dde23('kmckd',[.2 .7],[990; 10; 0],[0, 10]); xlabel('Days')
>> legend('Susceptible','Infected','Isolated','location','ne')
```

produces the result shown in Fig. 14.17a. (You can vary the values of parameters β and γ to match a scenario of many sci-fi movies.)

More examples of how to handle DDEs are provided by the ODE Examples GUI (Fig. 14.17b) that can be brought up by typing

```
>> odeexamples('dde')
```

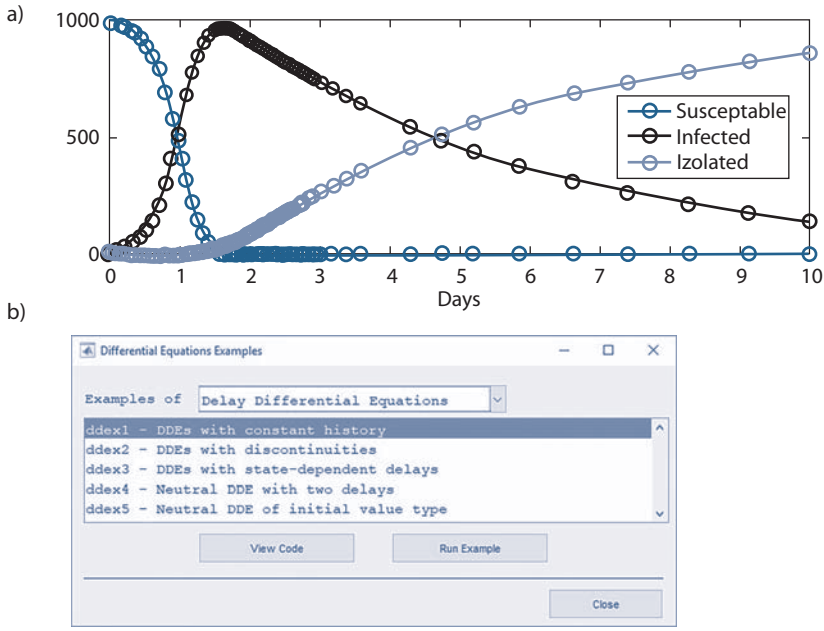


Fig. 14.17 a) Modeling a disease outbreak and b) examples of DDEs in the Differential Equations GUI.

14.8.2 BVP Solver

The `bvp4c` and `bvp5c` solvers allow you to solve BVPs, where, unlike an IVP [Eq. (14.15)], the conditions to be satisfied are established at x_0 and x_f:

$$\mathbf{y}' = \mathbf{F}(x, \mathbf{y}), \quad \mathbf{g}(\mathbf{y}(x_0), \mathbf{y}(x_f)) = \mathbf{0} \tag{14.135}$$

These solvers can also assist in finding unknown parameters for problems of the form

$$\mathbf{y}' = \mathbf{F}(x, \mathbf{y}, \mathbf{p}), \quad \mathbf{g}(\mathbf{y}(x_0), \mathbf{y}(x_f), \mathbf{p}) = \mathbf{0} \tag{14.136}$$

with $\mathbf{p}$ being the vector of parameters to be determined, so that Eq. (14.136) has a solution.

As opposed to IVPs, BVPs may have no solution, a finite number of solutions, or even an infinite number of solutions. Because of this, the programs for solving BVPs require users to provide a good guess for the solution desired. BVPs are much harder to solve than IVPs, so be prepared that the solver may not work even with the good guesses for the solution and unknown parameters.

The syntax of the `bvp4c` function is

```
sol = bvp4c(odefun,bcfun,solinit)
sol = bvp4c(odefun,bcfun,solinit,options)
```

and the initial guess is defined using the `bvpinit` function

```
solinit = bvpinit(x,yinit,params)
```

This and other input arguments are explained in more detail in Table 14.9.

The `bvp4c` solver is a finite-difference code that implements the Simpson's 1/3 rule of Eq. (13.19) (for a single interval, the multiplier becomes 1/6 rather than 1/3). To be more specific, the solution $\mathbf{S}(x)$ to Eq. (14.135) is sought as a piecewise cubic approximation satisfying the BCs

$$\mathbf{g}(\mathbf{S}(x_0), \mathbf{S}(x_f)) = \mathbf{0} \tag{14.137}$$

and satisfying differential equations (collocating) at both ends and midpoint of each subinterval $x \in [x_n; x_{n+1}]$:

$$\begin{aligned} \mathbf{S}'(x_n) &= \mathbf{F}(x_n, \mathbf{S}(x_n)) \\ \mathbf{S}'(x_{n+0.5}) &= \mathbf{F}(x_{n+0.5}, \mathbf{S}(x_{n+0.5})) \\ \mathbf{S}'(x_{n+1}) &= \mathbf{F}(x_{n+1}, \mathbf{S}(x_{n+1})) \end{aligned} \tag{14.138}$$

where $x_{n+0.5} = (x_n + x_{n+1})/2$.

These conditions result in a system of nonlinear algebraic equations for coefficients defining $\mathbf{S}(x)$, which are solved iteratively by linearization.

Table 14.9 Input Arguments for BVP Solvers

Argument	Description

`odefun`	A function handle that evaluates the right-hand side of differential equations, **F**(x,**y**), or **F**(x,**y**,**p**), whichever is appropriate
	`dydx = odefun(x,y)`
	`dydx = odefun(x,y,parameters)`
	(note that `y` and `dydx` must be the column vectors)

`bcfun`	A function handle that computes a residual in the BCs, defined by the column vectors `ya` and `yb`
	`res = bcfun(ya,yb)`
	`res = bcfun(ya,yb,parameters)`
	(`res` must be a column vector as well)

`solinit`	A structure containing the initial guess for a solution and created using the `bvpinit` function. This structure must have the following fields:
	`x` Containing the ordered nodes of initial mesh, so that the BCs are imposed at `x0=solinit.x(1)` and `xf=solinit.x(end)`
	`y` Containing the initial guess for a solution, so that `solinit.y(:,i)` corresponds to the node `solinit.x(i)`
	`parameters` Defining a vector of initial guesses for unknown parameters (if present)

`options`	An optional argument changing integration options using the `bvpset` function

Therefore, the BVP solvers rely on the linear equation solvers described in Chapter 9, rather than on IVP solvers.

The piecewise collocation polynomial provides a continuous (derivative continuous too) solution that is fourth-order accurate in the step size h. Mesh selection (the points $x_0 < x_1 < x_2 < \cdots < x_N = x_f$) and error control are based on the residuals at midpoints, computed using the three-point Lobatto quadrature (Ch. 13.8)

$$\mathbf{r} = \mathbf{S}'(x_{n+0.5}) - \mathbf{F}(x_{n+0.5}, \mathbf{S}(x_{n+0.5})) \tag{14.139}$$

The `bvp4c` solver can also solve multipoint BVPs, where the values of the function are defined at several rather than just two points b_i, $i = 0, 1, \ldots, M$ ($x_0 = b_0 < b_1 < \cdots < b_M = x_f$)). The only difference in this case is that the `bcfun` function has a slightly different format.

The `bvp5c` function is used exactly like `bvp4c`, but it is more efficient for small error tolerances. It implements the four-point Lobatto formula developed in Eq. (13.57). In this particular case, for each subinterval $x \in [x_n; x_{n+1}]$, the four points are established at $x_{n,i}^L = x_n + c_i h_n$, $i = 1, 2, 3, 4$ with $c_1 = 0$, $c_{2;3} = 0.5(1 \mp 1/\sqrt{5})$ and $c_4 = 1$. As opposed to `bvp4c`, `bvp5c` cannot solve multipoint BVPs.

Consider a simple example of a projectile motion in the vertical plan (a shot fired from a cannon). The equations

$$\begin{aligned} \dot{x} &= V\cos(\gamma) \\ \dot{y} &= V\sin(\gamma) \\ \dot{V} &= -kV^2 - g\sin(\gamma) \\ \dot{\gamma} &= -gV^{-1}\cos(\gamma) \end{aligned} \tag{14.140}$$

describe the problem with x and y denoting the horizontal and vertical coordinate; V – speed; γ – flight path angle; g – acceleration due to gravity; and k – air resistance (drag). Let us change the argument in these equations to x

$$\begin{aligned} y' &= \tan(\gamma) \\ V' &= -\frac{kV^2 + g\sin(\gamma)}{V\cos(\gamma)} \\ \gamma' &= -gV^{-2} \end{aligned} \tag{14.141}$$

and develop an m-file function assuming $g = 9.8\ \mathrm{m/s^2}$ and $k = 0.0002\ \mathrm{m^{-1}}$:

```
function dy = projectile(x,y,param)
dy = zeros(length(y),1);
dy(1) = tan(y(3));
dy(2) = -(9.8*sin(y(3))+0.0002*y(2)^2)/y(2)/cos(y(3));
dy(3) = -9.8/y(2)^2;
```

Now we can use either solver to find a solution for $y(0) = 0$ m, $V(0) = 200$ m/s, and $y(x_f) = 0$ m with a varied parameter $\gamma\ (0) = var$ (that is why the function `projectile` header lists it as a third argument even though it does not show up in Eq. (14.141) explicitly). These BCs should be saved in a separate *m*-file function, like

```
function res = BC(y0,yf,param)
res = [ y0(1)
        yf(1)
        y0(2)-200
        y0(3)-param];
```

(that is a place to utilize the varied parameter, `param`, in code).

We can call these two functions, `projectile` and `res`, in an attempt to find a solution as follows:

```
solinit=bvpinit(linspace(0,2000,5),[100 100 0.8],0.8);
sol=bvp4c(@projectile,@BC,solinit);
xbvp = linspace(0,2000); ybvp = deval(sol,xbvp);
plot(xbvp,ybvp(1,:)), hold, grid
text(850,1050,['\gamma_0=' num2str(sol.parameters*180/pi,2) '^o'])
axis equal, xlabel('x, km'), ylabel('y, km')
```

Here, the first line defines the argument range ($x \in [0; 2000]$m) and number of points to estimate the solution at (5), along with the initial guesses on $y(x)$, $V(x)$, and $\gamma(x)$ (constant values of 100 m, 100 m/s, and 0.8, respectively, for all five points) and the varied parameter $\gamma(0)$ (0.8). Note that the solution is quite sensitive to these initial guesses, so they have to be reasonable. Upon finding the solution (the second line), we use the `deval` function to evaluate the numerical solution at 100 equally spaced points (the third line) and plot the result, which is shown in Fig. 14.18a (labeled as the first solution).

Note that both `bvp4c` and `bvp5c` solvers return only one solution (if any at all). If you know that the problem might be multiple solutions, you have to play with the initial guesses hoping that the iterative process would converge to an alternative one. In our case, we know that there has to be another solution, so we could try running the same script with the decreased value of the initial guess on $\gamma(0)$. However, for educational purposes, in the following script (which continues the first portion presented earlier), we changed the initial guesses on $y(x)$, $V(x)$, and $\gamma(x)$ instead, leaving the guess on $\gamma(0)$ unchanged:

```
solinit=bvpinit(linspace(0,2000,5),[1 200 0.2],0.8);
sol=bvp4c(@projectile,@BC,solinit);
ybvp = deval(sol,xbvp);
plot(xbvp,ybvp(1,:),'.r');
legend('1^{st} solution','2^{nd} solution')
text(8500,350,['\gamma_0=' num2str(sol.parameters*180/pi,2)'^o'])
```

Luckily, it converged to the alternative solution we were looking for (denoted as the second solution in Fig. 14.19a), demonstrating the sensitivity to changes in all initial guesses. Further, if in the previous code you change

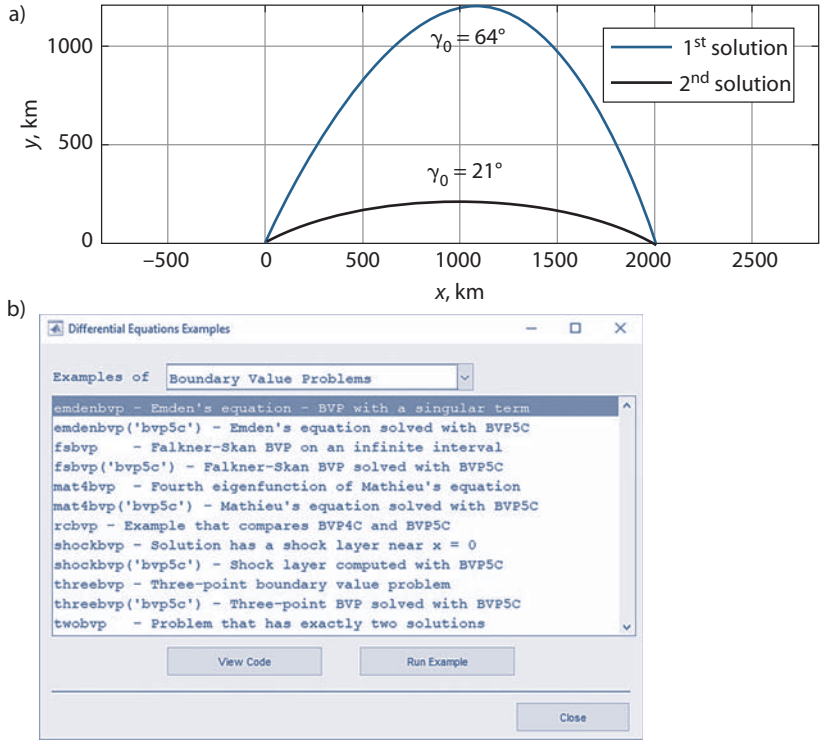


Fig. 14.18 a) Example of employing the `bvp4c` function and b) examples of BVPs in the Differential Equations GUI.

the initial guess on $V(x)$ to 50 m/s, the solver will diverge, returning the error message:

```
??? Error using ==> bvp4c at 203
Unable to solve the collocation equations -- a singular Jacobian encountered
```

Once again, the initial guesses must be reasonable.

Other examples of using BVP solvers are accessible via the MATLAB Help system or by calling the corresponding interactive GUI directly

```
>> odeexamples('bvp')
```

which brings up the window shown in Fig. 14.18b.

14.8.3 PDE Solver

The `pdepe` solver solves IVPs and BVPs for systems of parabolic and elliptic PDEs in one space variable x and time t of the form

$$\mathbf{c}\left(x, t, \mathbf{u}, \frac{\partial \mathbf{u}}{\partial x}\right)\frac{\partial \mathbf{u}}{\partial t} = x^{-m}\frac{\partial}{\partial x}\left(x^{m}\mathbf{f}\left(x, t, \mathbf{u}, \frac{\partial \mathbf{u}}{\partial x}\right)\right) + \mathbf{s}\left(x, t, \mathbf{u}, \frac{\partial \mathbf{u}}{\partial x}\right) \tag{14.142}$$

where $t \in [t_0; f_f]$, $x \in [x_0; x_f]$, m can be 0, 1, or 2, corresponding to slab, cylindrical, or spherical symmetry; $\mathbf{f}(x, t, \mathbf{u}, (\partial\mathbf{u}/\partial x))$ is the flux term; $\mathbf{s}(x, t, \mathbf{u}, (\partial\mathbf{u}/\partial x))$ is the source term; and $\mathbf{c}(x, t, \mathbf{u}, (\partial\mathbf{u}/\partial x))$ is the diagonal matrix with the nonnegative elements. If this matrix is a zero matrix, Eq. (14.142) is referred to as an *elliptic* equation; otherwise, it is referred to as a *parabolic* equation (these equations model totally different phenomena, and so the behavior of their solutions is quite different).

For $t = t_0$ and $x \in [x_0; x_f]$, the solution components satisfy the ICs

$$\mathbf{u}(x, t_0) = \mathbf{u}_0(x) \tag{14.143}$$

For any t and $x = x_0$ or $x = x_f$, the solution components satisfy a BC

$$\mathbf{p}(x, t, \mathbf{u}) + \mathbf{q}(x, t)\mathbf{f}\left(x, t, \mathbf{u}, \frac{\partial\mathbf{u}}{\partial x}\right) = \mathbf{0} \tag{14.144}$$

where the elements of $\mathbf{q}(x, t)$ are either zero or never zero.

The `pdepe` solver is called in either of two forms:

```
sol = pdepe(m,pdefun,icfun,bcfun,xmesh,tspan)
sol = pdepe(m,pdefun,icfun,bcfun,xmesh,tspan,options)
```

The `pdepe` function returns the values of the solution on a mesh provided in `xmesh`. The `pdefun`, `icfun`, and `bcfun` are the handles to the functions describing Eqs. (14.142), (14.143), and (14.144), respectively. Specifically, the `pdefun` function computes the terms of Eq. (14.142) as follows:

```
[c,f,s] = pdefun(x,t,u,dudx)
```

(`c`, `f`, and `s` should be returned as the column vectors, with `c` storing the elements of the corresponding diagonal matrix). The `icfun` function evaluates and returns the ICs in a column vector `u`

```
u = icfun(x)
```

The `bcfun` function evaluates the BCs

```
[pl,ql,pr,qr] = bcfun(xl,ul,xr,ur,t)
```

with `xl`, `xr` corresponding to endpoints x_0 and x_f, respectively; `ul` and `ur` approximating the correspondent BCs; and `pl`, `ql`, `pr`, and `qr` computing $\mathbf{p}(x, t, \mathbf{u})$ and $\mathbf{q}(x, t)$ at x_0 and x_f.

The ODEs resulting from discretization in space (Sec. 12.7) are integrated to obtain approximate solutions at the times specified in `tspan`. For example, the parabolic equation

$$\frac{\partial\mathbf{u}}{\partial t} = a^2\frac{\partial^2\mathbf{u}}{\partial x^2} \tag{14.145}$$

would become

$$\frac{\partial \mathbf{u}_i}{\partial t} = \frac{a^2}{\Delta x^2}(\mathbf{u}_{i+1} - 2\mathbf{u}_i + \mathbf{u}_{i-1}) \tag{14.146}$$

[compare with Eq. (12.48)]. The output `sol(:,:,i)` corresponds to the ith component of the solution vector. Hence, the element `sol(k,j,i)` corresponds to $u_i\,(t_k\,, x_j)$ (and also to `tspan(k)` and `xmesh(j)`). As with other solvers, you can compute the approximation and its partial derivative at an arbitrary set of points (within the original interval) using the special function, in this case the `pdeval` function.

As an example, consider the simplest parabolic equation

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} \tag{14.147}$$

with ICs and BCs given as

$$u(x, 0) = 0,\ \ u(0, t) = 0, \quad \text{and} \quad u(1, t) = \sin(3\pi t) \tag{14.148}$$

This equation represents the *heat transfer* equation in one dimension. This specific setup can be treated as heating/cooling (with a sine wave) one end of the thin, homogeneous rod of uniform thickness, which at time $t = 0$ (we are using some normalized time here) has temperature $u = 0$. For convenience, all three functions, required by the `pdepe` solver and describing Eq. (14.147) along with ICs and BCs of Eq. (14.148), can be placed in a single *m*-file together with the following calling and plotting commands:

```
function rod
x = linspace(0,1,30); t = linspace(0,1,30);
sol = pdepe(0,@heattransfer,@pdeIC,@pdeBC,x,t);
u = sol(:,:,1);
  surf(x,t,u), view([40,30])
  shading interp, light('Position',[-1 -1 0],'Style','local')
  lighting gouraud, material shiny, colormap jet
  xlabel('Distance x'), ylabel('Time t'), zlabel('Temperature T')
function [c,f,s] = heattransfer(x,t,u,DuDx)
c = 1;
f = DuDx;
s = 0;
  function u0 = pdeIC(x)
  u0 = 0;
function [pl,ql,pr,qr]=pdeBC(xl,ul,xr,ur,t)
pl = ul;
ql = 0;
pr = ur-sin(3*pi*t);
qr = a0;
```

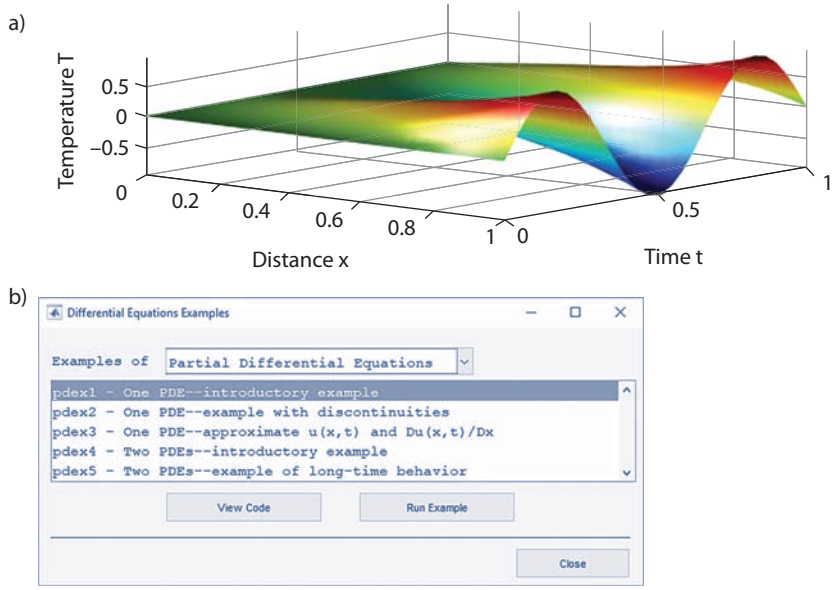


Fig. 14.19 a) Example of employing the `pdepe` function and b) examples of PDEs in the Differential Equations GUI.

Running this self-explanatory code (saved as *rod.m*) results in the plot shown in Fig. 14.19a.

MATLAB provides several more illustrative examples of how to use the `pdepe` solver, which can be accessed issuing the command

```
>> odeexamples('pde')
```

(Fig. 14.19b). On top of that, the PDEs Toolbox of MATLAB (if installed) offers many more options through the `pdetool` GUI, representing a self-contained graphical environment for solving PDEs and visualizing obtained solutions.

Problems

14.1 An airplane uses a parachute and other means of braking as it slows down on a runway after landing. Its acceleration is given by $a = (-0.0045V^2 - 3)$ m/s^2. Consider an airplane with a velocity V of 300 km/h that opens its parachute and starts decelerating at $t = 0$ s. Do the following:

a) By solving the differential equation, determine and plot the velocity as a function of time from $t = 0$ s until the airplane stops.

b) Use numerical integration to determine the distance x the airplane travels as a function of time. Make a plot of x vs time.

14.2 Consider the first-order ODE

$$y' = -2x^3 + 12x^2 - 20x + 8.5$$

Solve it for $x \in [0;4]$ with the IC at $x = 0$ being $y = 1$. Do it using the following methods:

a) Euler's first-order method with the step size of 0.5 (you have to develop your own script).

b) Euler's first-order method with the step size of 0.25.

c) Heun's second-order method (predictor–corrector approach) with the step size of 0.5 (you have to develop your own script).

d) Midpoint second-order method with the step size of 0.5 (you have to develop your own script).

e) RK low-order pair solver with the step size of 0.5 (use the `ode23` solver).

f) RK medium-order pair solver with the step size of 0.5 (use the `ode45` solver).

Present all solutions by pairs (a and b, c and d, e and f) on a three-strip plot, together with a true solution added to each pair. Do not forget to add a legend. Measure performance using the stopwatch timer (the `tic`-`toc` pair). Compute an average error for every solution and present it as an "Accuracy versus CPU time" plot.

14.3 Solve the system of ODEs

$$\dot{x} = 1 - y$$

$$\dot{y} = x^2 - y^2$$

over the interval $t \in [0; 5]$. As the ICs, pick several points within the following two segments: $y(0) = 0$, $x(0) \in [-2;2]$, and then $y(0) = 2$, $x(0) \in [-2;2]$. Present your results as a family of curves "y vs x."

Chapter 15 Simulink Basics

- Modeling Continuous Systems
- Modeling Discrete Systems
- Integration of MATLAB and Simulink
- Model Debugging and Profiling

Simulink libraries, `sim`, `simset`, `simget`, `open`, `open_system`, `new_system`, `matlabFunctionBlock`, `Simulink.sdi.view`

15.1 Introduction

As a part of the MathWorks product family, Simulink, built on top of MATLAB, provides a GUI that uses a variety of elements called blocks to create a simulation of dynamic system, that is, a system that can be modeled with differential or difference equations with one independent variable (time). These blocks can be as simple as a multiplier, sum, integrator, scope, etc. Simulink uses the same solvers of MATLAB that were considered in Chapter 14, but instead of using them programmatically, it enables a high-level interaction with the user, bringing with it the necessity to know about the syntax of different functions to a minimum. Simulating a dynamic system includes several major steps. First, you create a graphical model of the system to be simulated using the Simulink Editor. The model depicts time-dependent mathematical relationships among the system's inputs, states, and outputs. The graphical interface of Simulink enables you to choose the blocks from a variety of specialized libraries and add them to your model, and then you can specify their parameters and connect them. If needed, the groups of blocks can be combined into subsystems allowing you to create the multilayer (continuous, discrete, and multirate) models of complex real-world systems. Then, you can use Simulink to simulate system's behavior over a specified time (independent variable) span with the specified inputs and parameters. Last but not least, Simulink provides tools for debugging, profiling, and analyzing your model. Simulink represents an extensive product, especially if you consider all additional blocksets and block libraries that you can use to extend it (see Sec. 1.5.2). Although it deserves a separate textbook (and several such textbooks are available), the material of this chapter covers only the basics of Simulink. It starts with introducing the Simulink modeling environment and showing how to build and run the simplest continuous model using three different ways of representing

differential equations. Next, it shows the major difference between the two types of systems, continuous and discrete, also supplying you with some simple examples. More sophisticated examples of continuous model discretization and controller design are then followed by a demonstration of how a Simulink model can call MATLAB commands and *m*-scripts while loading and running and, vice versa, how a Simulink model can be called from a MATLAB *m*-script. It also shows a way to create the blocks from symbolic expressions developed a priori and available in the base MATLAB workspace. The chapter ends with examples of real-world systems to provide you with guidance for further exploration.

15.2 Simulink Development Environment

Clicking the **Start Simulink** icon of the Home tab of MATLAB Desktop, (see Fig. 2.1), brings up the Simulink Start Page shown in Fig. 15.1a. You may also bring up this window by typing

```
>> simulink
```

in the MATLAB Command Window. Among all available options of the New tab, this text only considers the **Blank Model** option. Examples of different (editable) Simulink models you can use when you get proficient with Simulink are available at the Examples tab (Fig. 15.1b). The Simulink Start page also provides you an easy access to the Simulink Onramp, a three-hour introductory hands-on tutorial with automated assessments and feedback (in the lower-left corner of the page).

Clicking the **Blank Model** button of the New tab (Fig. 15.1a) opens the blank Simulink Editor window shown in Fig. 15.2. As of R2019b, this window features a multitab Simulink Toolstrip with commonly used functionalities grouped together, similar to that of MATLAB. In addition, the Simulink Editor window has a vertical toolbar on the left, called a *palette* (to see/hide the names of its buttons, right-click it and make a choice). For your convenience, when the mouse pointer is hovering over any button, the corresponding prompt shows up. Figure 15.2 also features the Simulink Library Browser popping up when the Library Browser button, , is clicked.

The Simulink Library Browser presents collections of blocks, called blocksets, that are available to build up a model. Clicking any collection name in the left pane or any icon of the right pane opens a specific library. For example, Fig. 15.3 shows the content of the Sources library; Fig. 15.4 – the Sinks library; and Fig. 15.5 – the Math Operations library. Because you always need to feed your model with some input signal, perform some operations on it, and display the results, these three libraries contain the most commonly used blocks.

The Simulink Library Browser toolbar instruments allow you to go back/forward among multiple open library pages, to search for a specific

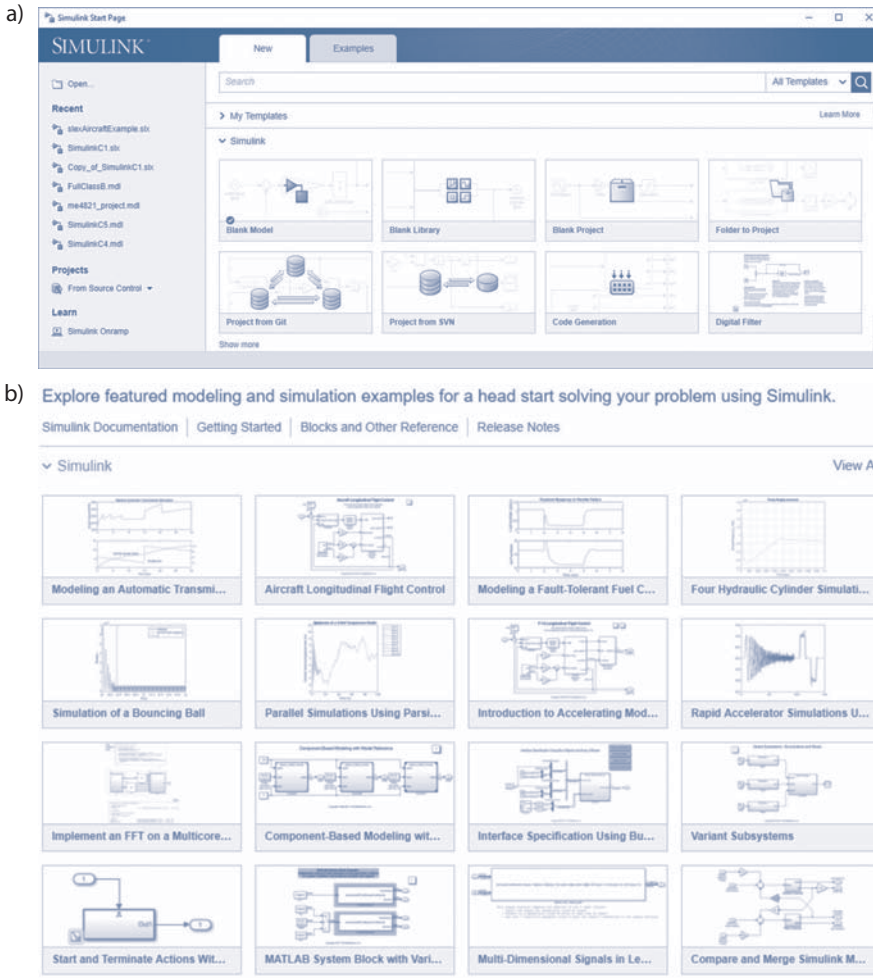


Fig. 15.1 a) Simulink Start Page and b) Simulink examples.

block in case you forgot where to find it, or to open a new or existing model. In the latter case, you may call the existing model up by simply typing its name in the MATLAB Command Window or using the `open('ModelName')` command.

Because you will use the Simulink Library Browser very often to grab different blocks from its libraries, it is not a bad idea to keep it open. For this purpose, the **Stay on top** button, , allows you to keep the Simulink Library Browser on top of all open windows all the time. The last button brings up the Help Browser opened on the "Use the Library Browser" page. As always, you can use the Help Browser to find information on any specific library or block (Fig. 15.6).

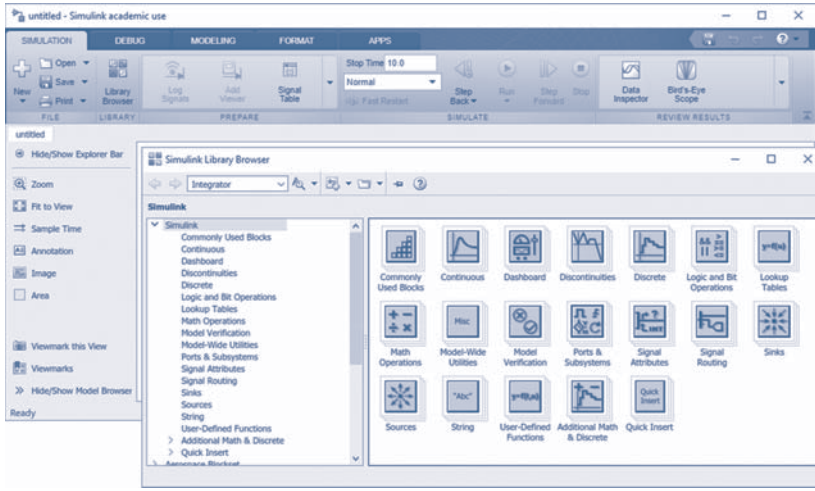


Fig. 15.2 Blank Model window and Simulink Library Browser.

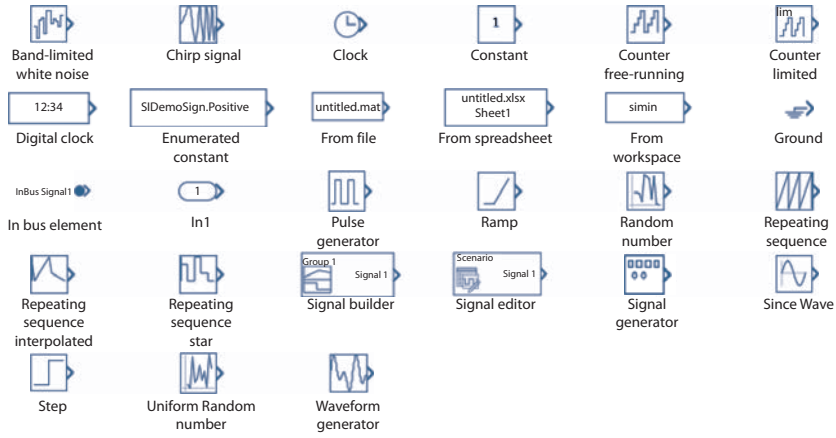


Fig. 15.3 Sources library blocks.

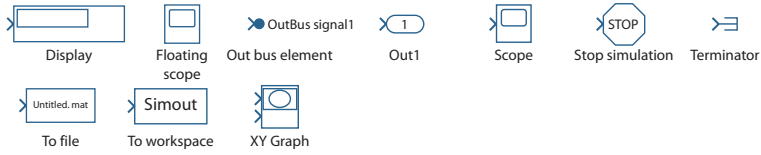


Fig. 15.4 Sinks library blocks.

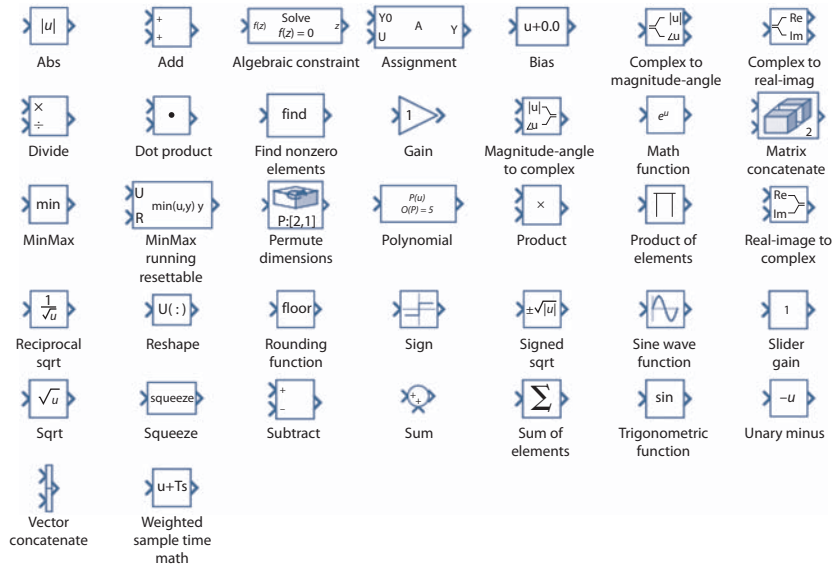


Fig. 15.5 Math Operations library blocks.

Whereas Fig. 15.2 shows the content of the Simulation tab of the Simulink Toolbox, Fig. 15.7 presents a variety of tools available within the other four tabs (prior to R2019b, Simulink Editor had a menu bar and toolbar so that most of these tools were available through them). New tabs (beyond the basic five) may appear when additional functionality is needed.

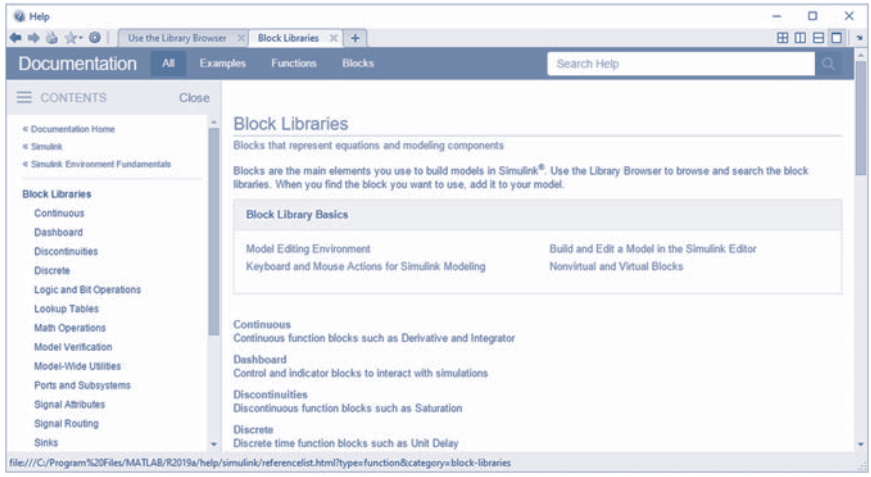


Fig. 15.6 Block Libraries help topics.

Fig. 15.7 a) Debug, b) Modeling, c) Format, and d) Apps tabs of the Simulink Toolstrip.

Some of the tool groups can be further extended by clicking the **Show more** button with the upside-down triangle on it, which opens a drop-down menu or icon panel listing additional options. For example, Fig. 15.8 features two drop-down panels of the Simulation and Modeling tabs (presented in Figs. 15.2 and 15.7b, respectively).

While describing all functionality of Simulink and all the tools accessible through the Simulink Toolstrip tabs is beyond the scope of this book, let us quickly review a few key elements of the Simulation tab (beyond the Library Browser) that we will be dealing with later in this chapter.

The Simulation tab tools (Fig. 15.2) represent the very basic instruments to build and run the model. The File group enables you to create/open a new

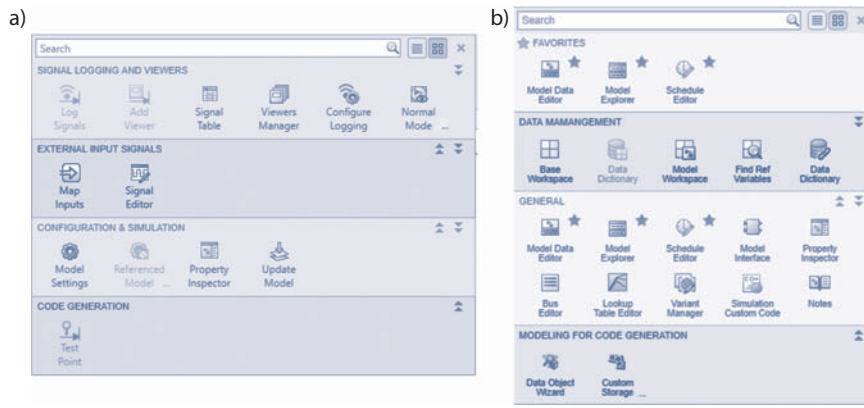


Fig. 15.8 a) Prepare and b) Design drop-down icon panels of the Simulation and Modeling tabs, respectively.

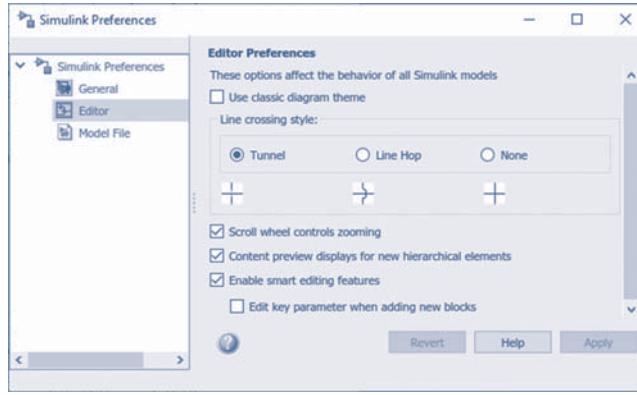


Fig. 15.9 Simulink Preferences window.

or existing model, save, and print the opened model. When you save your model, the default extension *.slx* is added automatically (this is the default file format for the Simulink models, available since R2012b, but you can choose to save it in the old MDL format, if needed). Of course, we are talking about the default Simulink settings, some of which can be changed if desired using the Simulink Preferences window (Fig. 15.9), which can be opened either from MATLAB (see Sec. 2.7) or by choosing the very last option, **Simulink Preferences**, in the drop-down menu of the **Environment** icon of the Modeling tab (Fig. 15.7b). (By the way, you may enable the Simulink Toolstrip, even if you are operating R2019a, by scrolling to the very bottom of the window shown in Fig. 15.9 and checking the corresponding box.)

The Prepare and Review Results groups of the Simulation tab (Fig. 15.2) allow you to configure your simulation parameters and inspect selected signals. Specifically, the **Model Settings** icon of the Prepare group (Fig. 15.8a), also repeated in the Modeling tab (Fig. 15.7b), , brings up the Configuration Parameters window, allowing you to change the ODE solver (to be considered in Sec. 15.3.1).

The Simulate group of the Simulation tab (Fig. 15.2), also repeated in the Debug (Fig. 15.7a) and Modeling (Fig. 15.7b) tabs, allows you to run your simulation.

Typically, you run your model using the **Run** button, , and use the **Stop** button, , if you need to stop the simulation prematurely. **Step Forward** and **Step Back** buttons allow stepping forward/backward through your simulation. The default time of simulation, t_f, defining the span of independent variables (0 to t_f), is set in the **Stop Time** field (Figs. 5.2, 15.7a, and 15.7b feature the default value of `10.0` time units). You can quickly change this parameter here or use the model Configuration Parameters window (Sec. 15.3.1).

Below the **Stop Time** field, there is a **Simulation Mode** drop-down menu, with the Normal speed mode being the default. This mode allows the greatest flexibility in making model adjustments and displaying the results when you run it, but if for any reason you want to speed it up, other modes are also available. Specifically, the Accelerator and Rapid Accelerator modes use portions of the Simulink Coder (formerly Real-Time Workshop) (Table 1.23) to replace the normal interpreted code with compiled (executable) code. Accelerator mode creates and uses the standalone executable from your model, whereas Rapid Accelerator mode creates and uses the standalone executable from your model and solver and, therefore, runs the fastest. Neither accelerator, however, supports the debugger or profiler, and they work only with those models for which C code is available for all of the blocks in the model (for instance, it does not work with any model containing an algebraic loop).

The Format tab (Fig. 15.7c) provides some options that allow you to improve the readability of your model (these options are also available through the corresponding context menus). Some selected tools from other Simulink Toolstrip tabs, specifically, the Simulation Data Inspector, (find it in Fig. 15.2), Property Inspector, (Fig. 15.8a,b), Model Discretizer, (Fig. 15.7d), Model Data Editor, , and Model Explorer, (Fig. 15.8b) will briefly be mentioned in the corresponding chapters when appropriate.

The top seven buttons of the Simulink Editor palette (Fig. 15.2) (from top to bottom) allow you to hide/show the Explorer Bar (to the right of this button), zoom in on the selected block(s) (you can continuously zoom in and out using the mouse scroll wheel as well), fit your model to the Simulink Editor window, add a Sample Time Legend window, and enhance the visual presentation of your model by adding annotations, images, and shaded areas (to visually group related model elements in an editable box). The next two buttons enable viewmarking so that you can capture graphical views of a model and navigate directly to that view from the viewmarks gallery. You can capture viewmarks for specific levels in a model hierarchy, which includes panning and zooming to capture a point of interest. The last button, at the very bottom, enables you to hide/show the Model Browser, which adds another pane at the left of Simulink Editor showing the model's hierarchy (see Sec. 15.8).

As you can see by the names of the Simulink libraries in Fig. 15.2, Simulink allows you to create two different types of models—continuous (analog) and discrete. The following two sections consider some simple examples introducing both types.

15.3 Modeling Continuous Dynamics

To begin with, let us open the Continuous library, shown in Fig. 15.10. The three major blocks of this library, Integrator, Transfer Fcn, and

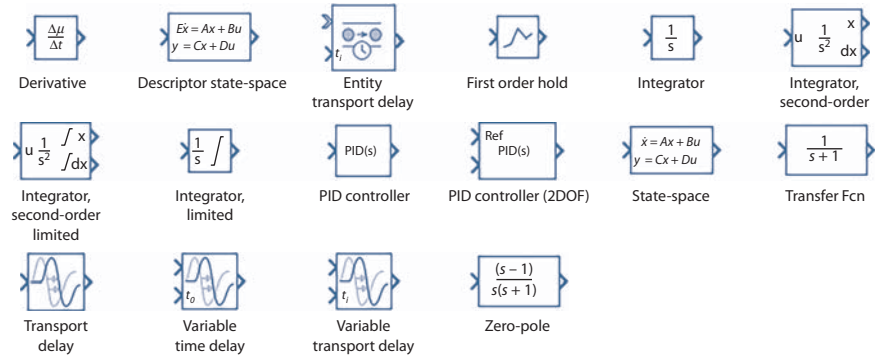


Fig. 15.10 Continuous library blocks.

State–Space, allow simulating a continuous dynamic system, described by differential equations, in three different ways:

- Using primitive linear blocks, representing each mathematical operation individually
- Taking advantage of using a transfer function (to represent a linear system as a whole)
- Using a state–space representation (of a linear system)

Depending on the problem you are dealing with, you may choose any of these ways, as discussed next.

15.3.1 Configuring an Integrator Block

As an example, consider a second-order ODE of Eq. (14.9). Using notations

$$\omega_n = \sqrt{\frac{k}{m}}, \quad \zeta = \frac{c}{2\sqrt{km}} \tag{15.1}$$

where ζ is the damping ratio and ω_n is the undamped natural frequency, we can rewrite it in a more general form

$$\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2 x = 0 \tag{15.2}$$

(note, we changed y to x because some Simulink blocks use x notation rather than y; it does not really matter). The numerical values of ζ and ω_n define behavior of dynamic system (the transient response).

For example, for an aircraft (see Fig. 14.1b), the response to disturbances in the longitudinal channel can be described by two sets of Eq. (15.2) defining *short-period motion* (with x being the angle of attack) and *long-period* or *phugoid motion* (with x representing the pitch angle). The typical values

(the order of magnitude) for the short-period motion are

$$\omega_n = 5\text{s}^{-1}, \quad \zeta = 0.5 \text{ (that is, } 2\zeta\omega_n = 5\text{s}^{-1} \text{and } \omega_n^2 = 25\text{s}^{-2}) \tag{15.3}$$

whereas for the phugoid motion,

$$\omega_n = 0.1\text{s}^{-1}, \zeta = 0.05 \text{ (that is, } 2\zeta\omega_n = 0.01\text{s}^{-1} \text{and } \omega_n^2 = 0.01\text{s}^{-2}) \tag{15.4}$$

Let us model Eq. (15.2) for the phugoid motion of an aircraft. Because Eq. (15.2) describes a second-order system, to model its behavior we will need two integrators. Hence, we start developing a new model by opening a blank (new) window of Simulink Editor and dragging two Integrator blocks from the Continuous library (Fig. 15.10), as shown in Fig. 15.11a. Similarly, you can drag the Sum block from the Math Operations library (Fig. 15.5). You can move the blocks around and align them horizontally (Simulink uses an invisible five-pixel grid to simplify this process).

Note that Simulink Editor names the blocks as you add them. For example, it names the first Integrator block that you add "Integrator"; the next –

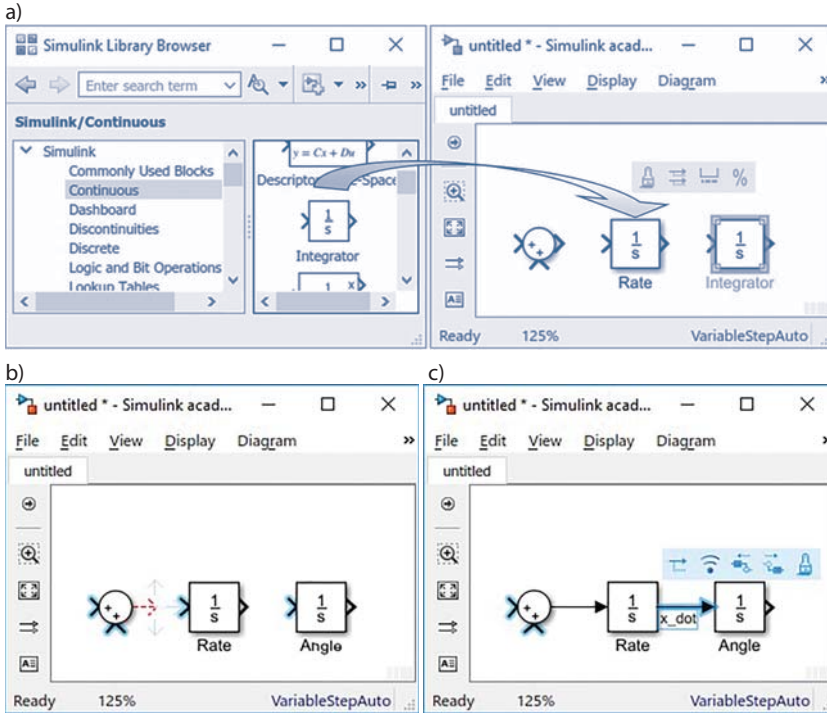


Fig. 15.11 a) Dragging an Integrator block of the Continuous library to the new Simulink model, and b) and c) connecting the blocks.

"Integrator1"; and so on (you cannot have two blocks with the same name—even if you try to intentionally, Simulink will automatically alter the name by adding 1, 2, 3,. . . at the end of the name). However, for some blocks, by default, these automatic names are hidden (as it appears for the Sum block in Fig. 15.11a). To change the default name of a block, you simply click it and type in a new name (as was done for one of the Integrators in Fig. 15.11a).

To connect two blocks, drag (left- or right-click, hold, and move the mouse out) from the output port of one block to the input port of another one (or, vice versa, from the input of one block to the output of another). Figure 15.11b shows the situation when the *signal line* was dropped (the mouse button was released) prematurely. The dashed red arrow indicates that the connection was not made, with all possible options highlighted. You can either continue where you dropped it or click this dashed arrow, delete it, and start all over again. More options to connect blocks will be considered in Sec. 15.3.2.

Double-clicking the valid signal line allows you to name it for a better readability of your model (Fig. 15.11c). To be able to use this model later, save it as *SimulinkC1.slx* (the name of the model then shows up as the name of the window).

Let us make a couple of notes here. First, the Simulink Editor window shown in Fig. 15.11a features a minimized toolstrip (enabled by right-clicking it and choosing **Minimize Toolstrip**). Second, clicking any block or signal line brings up three blue dots above the corresponding object. Clicking these dots invokes an *ellipsis menu* with additional options as shown in Figs. 15.11a,c. Specifically for the blocks, the options are (Fig. 15.11a) copying formatting, auto-routing the incoming and outgoing signal lines (you might need this after you rotate or flip a group of blocks), hiding block names, and commenting the block out (making a block not executable, like a comment in a MATLAB script). In the latter case, right-clicking the commented block (which turns gray) and choosing the **Comment Through** option from the context menu (the left menu in Fig. 15.12a), allows passing the signal through the commented block (otherwise the input and output are not connected). For the signal line, the available options are (Fig. 15.11c) auto-routing this line, enabling data logging, tracing a signal across multiple subsystem levels [by highlighting a signal from source (where it came from) to destination (where it is connected to)], and copying formatting.

Speaking of formatting (Fig. 15.12a), Simulink Editor allows you to adjust fonts, add colors to the block and background, add drop shadows, and reposition or hide block names (see the "Adjust Visual Presentation to Improve Model Readability" help topic). Alternatively, you can do these actions using the tools of the Format tab (Fig. 15.7c). Note that, having a single or multiple blocks selected, this tab changes to that of Fig. 15.12b, featuring the additional Arrange/Block Layout and Align/Distribute/Match groups of instruments, respectively. The **Auto** button of the Block Layout group (Fig. 15.12b) allows you to manage hiding/showing the block names.

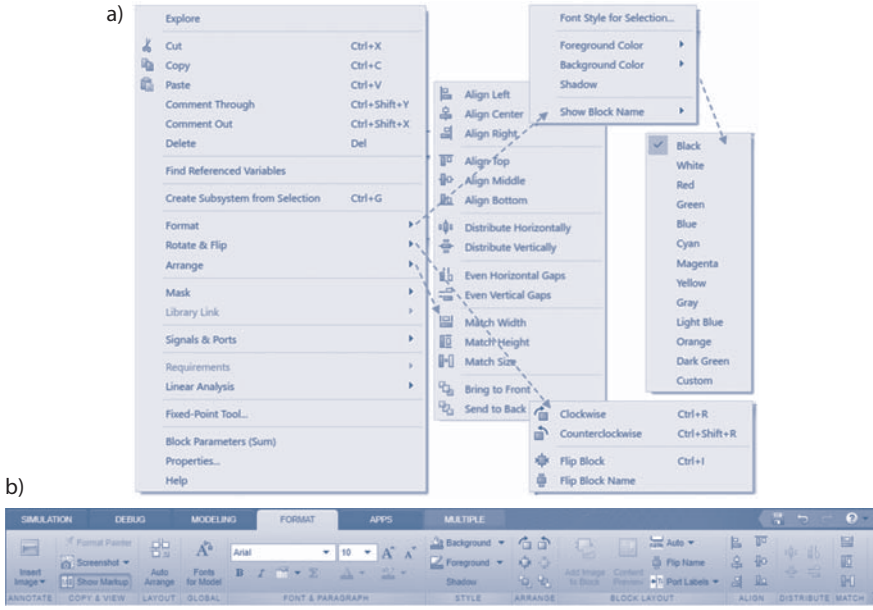


Fig. 15.12 a) The block context menu (on the left) and a variety of flyout menus for adjusting visual presentation of a model and b) additional panels of the Format tab with multiple blocks selected.

Returning to the model, the output of Rate integrator is $\dot{x}$; therefore, its input must be $\ddot{x}$. Let us rewrite Eq. (15.2) to have an explicit expression for the angular acceleration $\ddot{x}$ as the function of x and $\dot{x}$:

$$\ddot{x} = -2\zeta\omega_n\dot{x} - \omega_n^2 x \tag{15.5}$$

Substituting parameters of the phugoid motion [Eq. (15.4)] yields

$$\ddot{x} = -0.01\dot{x} - 0.01x \tag{15.6}$$

Let us also assume the ICs of

$$x(0) = 0 \quad \text{and} \quad \dot{x}(0) = 0.01\text{s}^{-1} \tag{15.7}$$

Now, you may proceed with developing the remainder of the model, with the final result as shown in Fig. 15.13. This involves three steps.

In Step 1, you need to make a couple of adjustments to the Sum and Rate blocks of Fig. 15.11. Double-clicking the Sum block brings up its Parameters dialog box, shown in Fig. 15.14a, featuring the **List of signs** sequence that defines the number and signs of the inputs. The inputs of the Sum block (in our case, intended to compute $\ddot{x}$), are drawn from 12 o'clock down to 6 o'clock counterclockwise. The default "|++" combination defines two

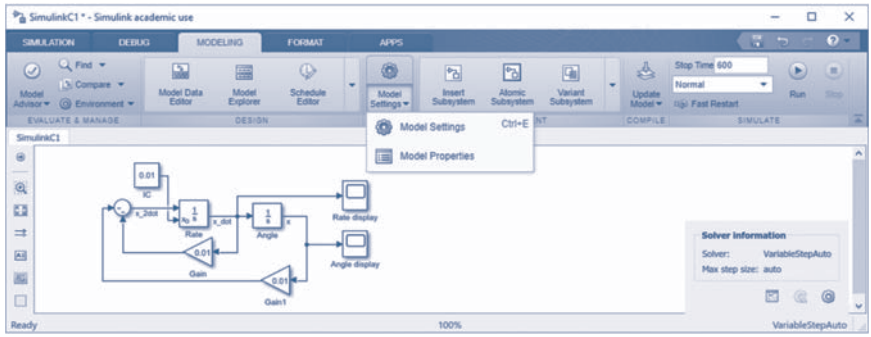


Fig. 15.13 Simulink model of the second-order system.

positive inputs and their orientation (the vertical line | serves as a spacer or virtual input, making the 12 o'clock input vacant). To match what is shown in Fig. 15.13, you need to change the **List of signs** sequence to "| −− " [because Eq. (15.5) features two negative signs]. (Figure 15.14a also shows that you have the option of changing the default round shape of the Sum block to a rectangular shape, in which case all inputs would appear on one side of the block.)

Double-clicking the Integrator block brings up its Parameters dialog box, as shown in Fig. 15.14b. Changing the **Initial condition source** field from the default *internal* to *external* adds the second input to this block (Fig. 15.13), enabling the IC to be defined outside. (Note the difference in the background color of the drop-down menus of Figs. 15.14a,b—the darker background in Fig. 15.14b warns you that you still have to apply your changes.)

Now, you may add a Constant block from the Sources library (Fig. 15.3), change its name to IC and its **Constant value** to `0.01` (Fig. 15.15a), and connect its output with the just-created second input of the Rate integrator

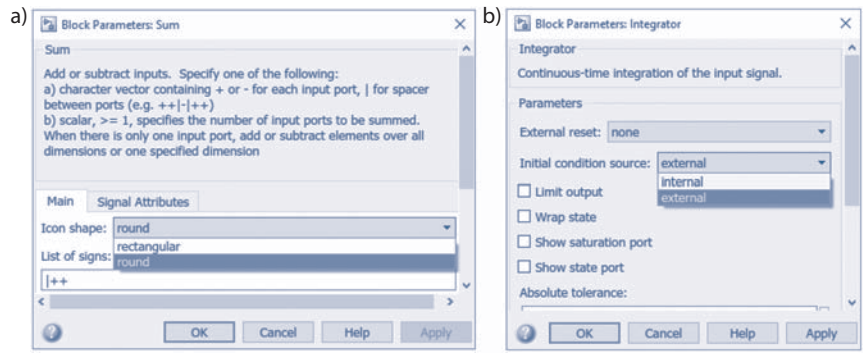


Fig. 15.14 Parameters dialog boxes for the a) Sum and b) Integrator blocks.

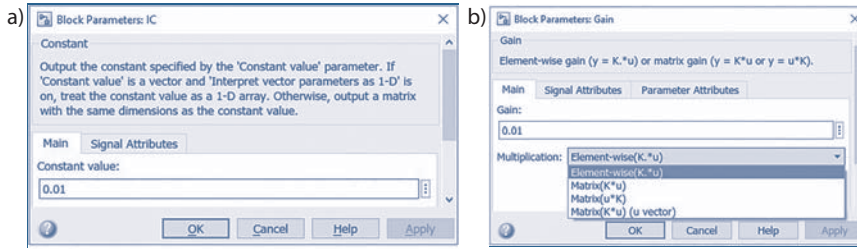


Fig. 15.15 Block Parameters dialog boxes for the a) Constant and b) Gain blocks.

block (Fig. 15.13). The internal **Initial condition source** value for the second integrator, Angle, is 0 (the default value). These are the ICs defined by Eq. (15.7). Defining the IC as an external input is a more elegant and convenient way because it eliminates the necessity of opening the Parameters dialog box of the Integrator block and introducing a new IC every time you need to change it, but in the model of Fig. 15.13, both approaches are intentionally used for you to compare.

In Step 2, you need to add two Gain blocks from the Math Operations library. For them to appear as shown in Fig. 15.13, you need to right-click each of them, choosing the **Rotate & Flip** and then **Flip Block** options from the corresponding flyout menus shown in Fig. 15.12a (which are the same for all the blocks).

Double-click each block and enter the corresponding gain values in the Parameters dialog box for the Gain block presented in the Fig. 15.15b. In this particular case, the scalar gains of Eq. (15.6) are all you have to enter. For future use, however, note that when operating with matrices, the gain K can be represented by a matrix as well, so you should be careful about how you want multiplication to be executed and choose the proper option from the **Multiplication** drop-down menu (Fig. 15.15b). Note, when the numerical value of a gain is too long to fit into the mask of the Gain block, the Gain block appears with a -K- symbol upon it. To see the value in this latter case, you have to resize (enlarge) the block by selecting it (clicking it), placing a mouse pointer over one of its corners, and stretching it out.

Now, connect these two new blocks and the summation block with integrators. Note that the Gain block in Fig. 15.13 needs to be connected to the signal line rather than an output from another block. You can accomplish this by dragging a line from the block's input port and dropping it (releasing the mouse button) on the signal line connecting two integrators, which creates a branch (pickoff) point. You can grab this branch point and move it along the signal line if needed.

In Step 3, to display angular rate and angle itself, add two Scope blocks from the Sinks library (Fig. 15.4) and rename them as shown in Fig. 15.13. Connect these two blocks with the corresponding signal lines.

After completing the three aforementioned steps, you are almost ready to run your model. Before you do so, change the default simulation time in the **Stop Time** field (Figs. 15.2, 15.7a,b) to `600` (seconds). Alternatively, you may do the same in the Configuration Parameters window (Fig. 15.16), which can be opened by clicking the **Model Settings** icon, . This icon can be found in the model (right-click) context menu (<Ctrl)+<E>), the Prepare group (Fig. 15.8a), the Modeling tab (Fig. 15.7b), or the Solver Information window that pops up when you click the solver name on the far right of the Status bar (Fig. 15.13 shows the two latter options).

To run your simulation, push the **Run** button found in any of the first three Simulink Toolstrip tabs (Figs. 15.2, 15.7a,b). Note that, as opposed to the MATLAB Editor, if you made changes to your model and attempt to run it without saving, you will not be prompted to save your model first. Double-click each Scope display to observe the results of the simulation as shown in Fig. 15.17. You may need to click the **Scale Y & X Axes Limits** button of the Scope toolbar, , to autoscale the plots (the corresponding drop-down menu is shown in Fig. 15.17a).

Other buttons of the Scope toolbar are as follows. The first button (the corresponding drop-down menu is shown in Fig. 15.17b) allows you to configure scope properties (Fig. 15.18a), define style (Fig. 15.18a) and layout (multiple plots can be presented in a grid of plots), and add a legend. The next four buttons are the same buttons available through the Simulate group in the Simulation, Debug, and Modeling tabs of the Simulink Toolstrip (they allow you to rerun a simulation, step it forward/back, and stop it). The following two buttons allow highlighting of a Simulink block/freezing a display and zooming/panning. The last two buttons enable you to set up triggers and conduct measurements (cursor measurements, signal statistics, bi-level measurements, and peak finding). As always, these Scope toolbar button options are also available via Scope's drop-down menus (Fig. 15.19).

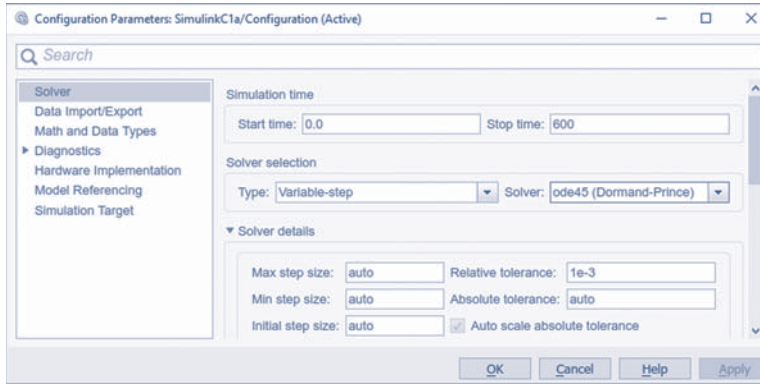


Fig. 15.16 Configuration Parameters window for variable-step solvers.

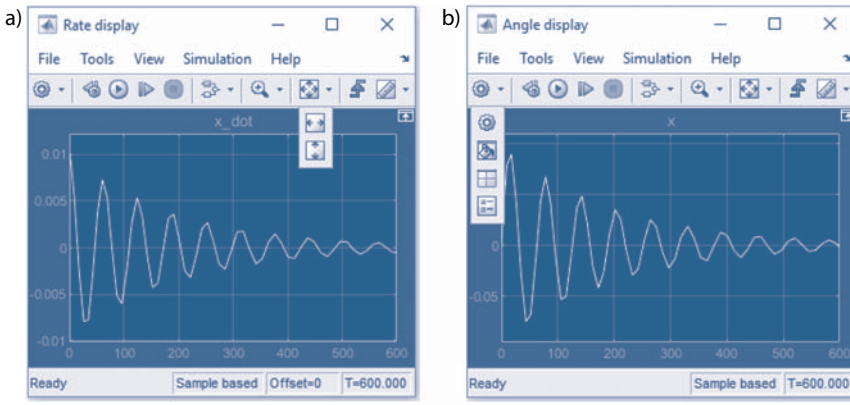


Fig. 15.17 Time histories of a) pitch rate and b) pitch angle with the default relative tolerance of `1e-3`.

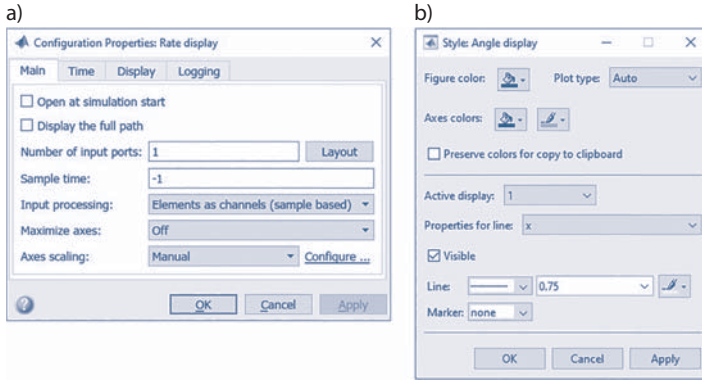


Fig. 15.18 Properties dialog boxes for the Scope's a) **Configuration** and b) **Style** buttons.

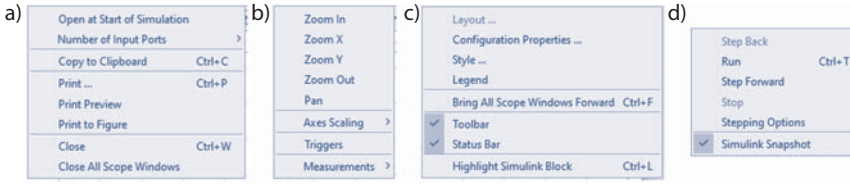


Fig. 15.19 Drop-down menus for a) File, b) Tools, c) View, and d) Simulation.

Fig. 15.20 List of variable-step solvers.

If you click the **Expand** button to the right of the solver name in Fig. 15.16, the drop-down list of all available variable-step solvers will show up as presented in Fig. 15.20. As seen, this list is identical to that given in Table 14.6, that is, Simulink uses the same solvers presented and discussed in Chapter 14. Table 15.1 shows the selection process of the auto solver. The name of the current solver shows up on the right of the Status bar at the bottom of the Simulink Editor (Figs. 15.11a and 15.13). When you run your model for the very first time with the `VariableStepAuto`, it automatically changes to say `auto(ode45)`. Clicking the second button at the bottom of the Solver Information panel of Fig. 15.13 allows you to confirm this choice, and the displayed solver information then changes to `ode45` (of course, you can choose any solver using the Fig. 15.20 menu explicitly).

Clicking the first button of the Solver Information panel of Fig. 15.13 brings up the Solver Profiler window (Fig. 15.21a). Running your simulation from this window allows you to see how the integration step size varies during simulation. The **State** button of the Solver Profiler toolbar brings up the State Explorer window, allowing you to observe the outputs from both Integrator blocks (Fig. 15.21b), meaning you do not even need to use the Scope blocks.

Along with variable-step solvers, Simulink features fixed-step solvers, and so if you change the solver **Type** in Fig. 15.16 to Fixed-step, then the Configuration Parameters window changes to that of Fig. 15.22 (also featuring the list of available solvers in this case). These simple solvers, discussed in Sec. 14.3, are programmed as C code as opposed to the *m*-functions for adaptive variable-step solvers.

Table 15.1 Auto Solver Heuristics

	The System Has . . .			
		. . . Continuous States, and Composed of . . .		
The Solver Type is	. . . Only Discrete States	. . . ODEs		. . . DAEs
Fixed-step	`FixedStepDiscrete`	`ode3`		`ode14x`
		Stiff model	Non-stiff model	
Variable-step	`VariableStepDiscrete`	`ode15s`	`ode45`	`ode23t`

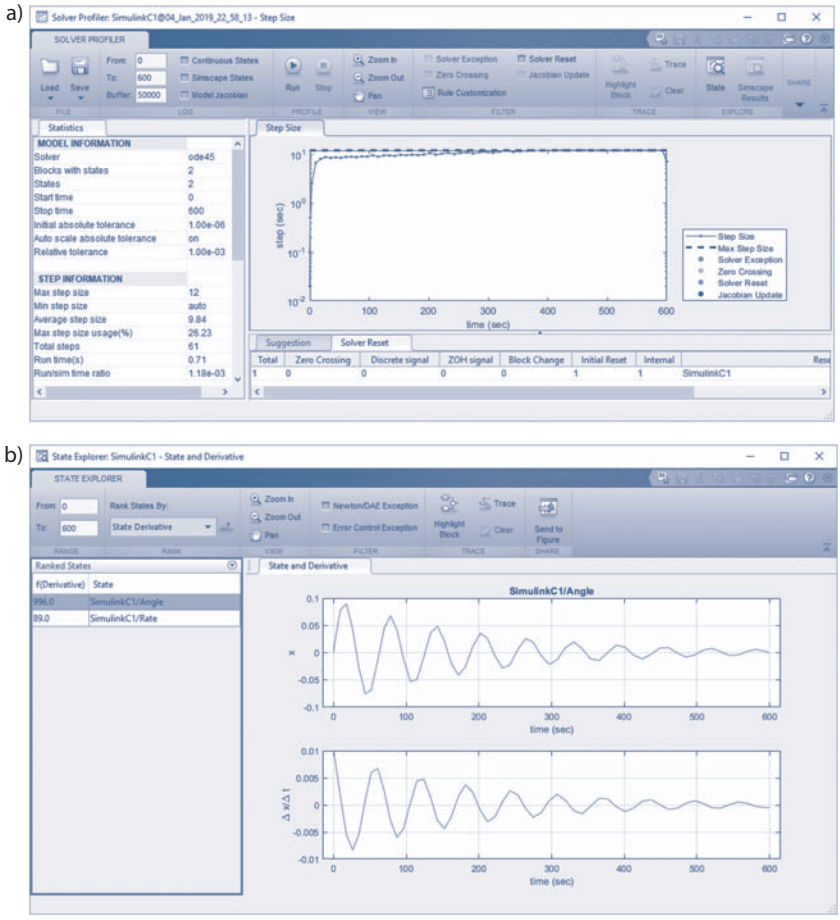


Fig. 15.21 a) Solver Profiler and b) State Explorer windows.

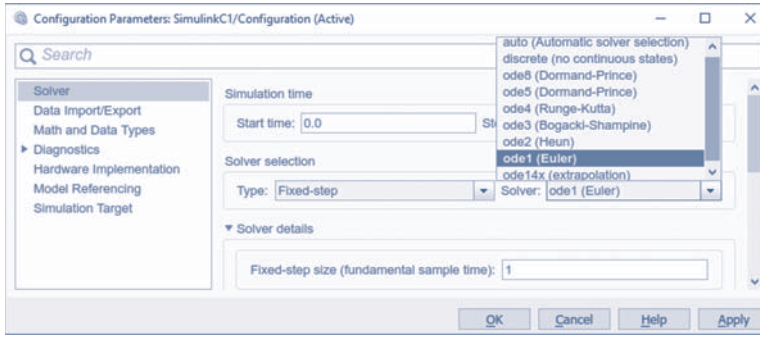


Fig. 15.22 Configuration Parameters window for fixed-step solvers (listed in a pop-up window).

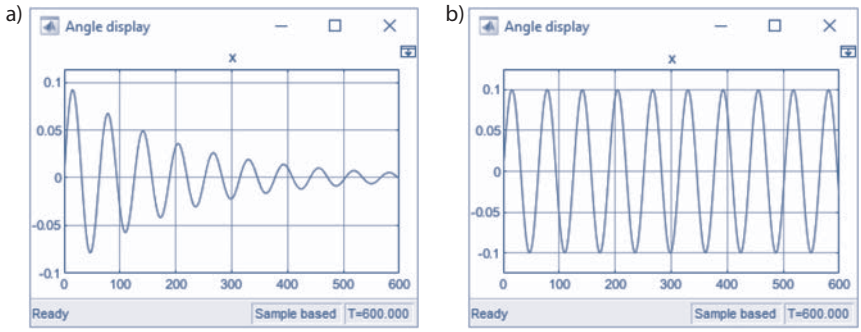


Fig. 15.23 The pitch angle time histories with the a) relative tolerance of `1e-6` and b) fixed 1-s integration step.

Before we proceed with the next model, let us make a comment. Quite often, when you run your model, you do not have enough smoothness in your results (have a closer look at both of the plots in Figs. 15.17a,b). It happens because you are using some predetermined (not small enough) integration step (see Chapter 14 for details). If this happens to be the case, to achieve a better result (at the expense of speed of integration), you may manually change certain parameters of the solver. For example, changing **Relative tolerance** in the Configuration Parameters window of Fig. 15.16 to, say, `1e-6` (as opposed to the default `1e-3`) produces a much smoother result, as shown in Fig. 15.23a (do not forget that after changing any parameter in the Configuration Parameters window, you have to rerun your model). Note that for better appearance we changed the figure and axes background color in the Scope Style window (Fig. 15.18b) to white; changed ticks, labels, and grid color to black; and hid the menus and toolbar (by clicking the small icon in the upper-right corner).

However, if your primary concern is the speed of integration, so that you are willing to degrade relative tolerance, be aware that you may end up with a completely erroneous result. In our example, changing the solver type to a fixed-step (Fig. 15.22), choosing the least accurate `ode1` (Euler method) solver, and running the simulation with a fixed step of `1` yields an erroneous result, as shown in Fig. 15.23b, indicating that the system is on the verge of stability (for the larger step sizes it becomes unstable). This result certainly differs from what we saw earlier (Figs. 15.17b and 15.23a). In fact, establishing such a big value for the step size is equivalent to introducing an artificial delay, or in other words, reformulating Eq. (15.2).

15.3.2 Accelerating Creating Models and Improving Their Readability

As a result of the most significant upgrade to Simulink introduced in R2019b, the current Simulink Editor offers a variety of tools to expedite

building readable models, which include tabbed panels, smart guides, diverse accessibility, Property Inspector, and other instruments (Figs. 15.2, 15.7, and 15.8). As you build a model, you can adjust block positions, change block and background color, add a drop shadow, adjust fonts, temporarily comment out or through parts of a design (Fig. 15.12a), organize the model architecturally and visually, and add other elements that help to improve model readability. It is always worth spending some extra time to make your model more readable and easy to use by yourself (in order not to forget what it is supposed to do) and by others when you share it (by the way, that is a reason why in the previous section we suggested naming at least the key blocks and signal lines).

When building the very first model in Sec. 15.3.1, we skipped most of the aforementioned options, but now it is time to review them and put them to good use. To start with, let us demonstrate a few smart ways to connect the blocks. One option to connect two blocks is clicking two ports to connect them. After you click the first port (or line segment), compatible ports appear highlighted (e.g., in Fig. 15.24a, the first integrator output port was clicked on). Click another (desirable) port to connect. You can still use this option even if you started dragging a line in a usual manner and then dropped it; compatible ports appear highlighted as well (Figs. 15.11b). To create a branch line, you can click the input port (of the Gain block in Fig. 15.24b) and move the cursor toward the signal line you want to branch from (the signal line connecting two integrators in Fig. 15.24b). A preview line appears, as shown in Fig. 15.24b. Click to create the branch. To construct a diagonal line from an existing signal line, press < Shift > and drag an angular point (vertex) (Fig. 15.24c).

Table 15.2 provides some more tricks of handling signal lines and their labels, including introducing multiple labels for the same signal line if needed.

Next, let us discuss several alternative ways of adding a block to your model. Section 15.3.1 used a straightforward approach of accessing the block collections using the Simulink Library Browser (Fig. 15.2). If you know the name of the block you are trying to add, there is a shortcut to do this. Double-click the model background (any empty space within your model), also referred to as the *model canvas*. You will see the annotation prompt pop-up (Fig. 15.25a). Clicking Create Annotation brings up another

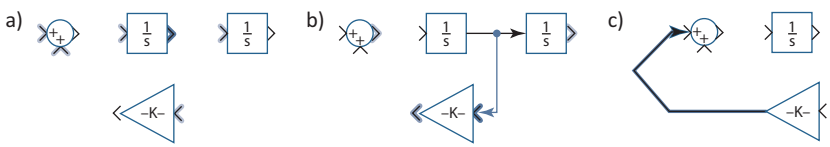


Fig. 15.24 a) Clicking block output (input), b) branch preview line, and c) creating diagonal lines.

Table 15.2 Effective Handling of the Simulink Signal Lines and Labels

Operation	Method
Connecting two blocks or group of blocks	Select the first block and < Ctrl > +click the block you want to connect it to. Selecting multiple blocks allows connecting to the block with multiple inputs (like Sum, Mux, subsystems, etc.)
Draw a signal line in segments	Drag from the output port to the first bend and release a mouse button, drag from the first bend to second bend, and so on
Branch from a signal line	Right-click the desired location of a branch point and drag with the right mouse button pushed; alternatively, do the same with the left mouse button holding down the < Ctrl > key
Make multiple, consecutive connections	After you click a port, hold <Shift > as you connect to another port. While still holding < Shift >, you can branch a new signal line and connect it to another port with one click
Copy a signal line label	Right-click the label and drag it to a desired location with the right mouse button pushed; alternatively do the same with the left mouse button holding down < Ctrl >
Delete one occurrence of a signal line label that is repeated somewhere else	Right-click the label and choose the **Delete** option from the pop-up menu
Delete all occurrences of a signal line label	Select the label and delete all characters

panel to be discussed later in this section (Fig. 15.25b), but if you start typing the name of the block you want to add, for example, `step` for the Step block of the Sources library of Fig. 15.3, you will see the drop-down menu showing the names of all blocks starting from these four symbols (Fig. 15.25c). (The list of shown suggestions is dynamically ranked based on your recent block usage history.)

Click the block name or, with the block name highlighted (as is the case in Fig. 15.25c), press <Enter>. (You can use the arrow keys to highlight the block name if it is not first in the list.) You will then see a prompt to specify one key parameter for this block, which, in the case of the Step block, is the **Step time** (Fig. 15.25d). (Note, to use this feature in R2020a, this option should be enabled in the Simulink Preferences window as shown in Fig. 15.9.) You can enter the value and push the <Enter> key or simply

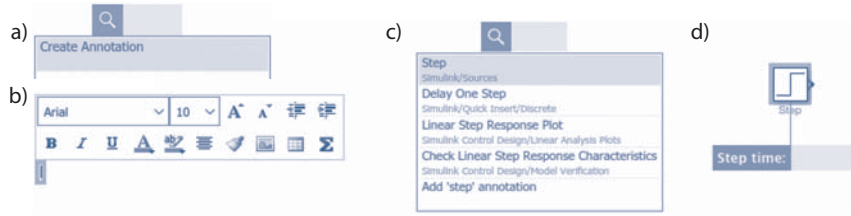


Fig. 15.25 a) Double-clicking the canvas, b) the annotation menu, c) typing in `step`, and d) Step block prompt.

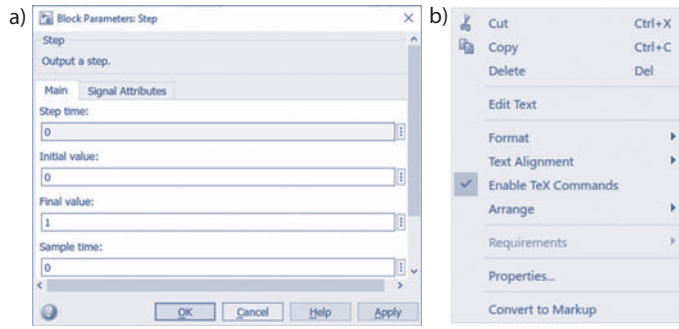


Fig. 15.26 a) Step block Parameters dialog box and b) Annotation context menu.

ignore it for now (while adding new blocks to your model), especially if you are satisfied with the default value. As usual, double-clicking the Step block opens its Parameters dialog box (Fig. 15.26a), showing that the default value of **Step time** is `0`.

If you noticed, when you were adding other blocks from Simulink libraries while creating the model shown in Fig. 15.13, you also had prompts similar to those of Fig. 15.25d. Specifically, you could have defined the **List of signs** for the Sum block (the default is `|++`), (internal) **Initial condition** for the Integrator block (`0`), **Gain** for the Gain block (`1`), **Number of input ports** for the scope block (`1`), and **Constant value** for the Constant block (`1`).

The third approach to add a block is when you already have a similar block in your model. To add a block, right-click the existing block and drag it away to create another copy. (Note, for the blocks of the Continuous and Discrete libraries, you will see a pop-up menu suggesting you confirm creation of a new copy along with two other options: **Create State Reader Block** and **Create State Writer Block**.)

Last, you can simply use the Copy and Paste commands (see the left menu of Fig. 15.12a). If you do not know the library a block is in or the full name of the block, you can search for it using the search field in the Library Browser (Fig. 15.2).

If you want to enhance your model with a textual annotation rather than adding a new block, you can type your text in the pop-up window of Fig. 15.25a. If the text you are typing happens to coincide with one of the Simulink block names, the drop-down menu will offer adding the corresponding name as text as opposed to adding a new block (see the last option in Fig. 15.25c.)

Similar to the MATLAB script comments, your textual annotations may include TeX commands. For example, double-click somewhere by the Gain block in the model shown in Fig. 15.13 and type in `2\zeta\omega_n`; then, click by the Gain1 block and type in `\omega^2_n`. These text strings

use the TeX markup language as discussed in Secs. 5.12. and 6.5. To enable the use of the TeX commands, you should select the text it applies to, right-click, and check off the **Enable TeX Commands** option of the context menu (Fig. 15.26b). Choosing the **Properties** option of this menu brings up the Annotation Properties dialog box that provides even more options (Fig. 15.27a).

Another way to add the extended semiautomatic comments to your model is to use the Model Info block (Fig. 15.27b) from the Model-Wide Utilities library (Fig. 15.27c) or to employ the DocBlock block from the same library capable of carrying an extended ASCII text.

In a broad sense, annotations are not necessarily just textual elements, but any visual elements adding descriptive notes and callouts to your model. To use the full power of Simulink annotations, you should use shortcuts on the Simulink Editor palette, the vertical strip of icons by the left edge of the Editor window (Fig. 15.2). Clicking the **Annotation** icon of this pallete and then clicking the canvas, dragging the **Annotation** icon to the canvas, or double-clicking the canvas where you want the annotation to be created and selecting **Create Annotation** from the menu opens the Annotation panel shown in Fig. 15.25b.

Along with a simple text that you can quickly change the format of using the Annotation panel, you can add images, tables, and LaTeX markups. In particular, choosing the last icon of the Annotation panel opens the Edit Equations window (Fig. 15.28a), allowing you to use the TeX, LaTeX, and MathML commands (see Sec. 6.5 and Appendix A). You can also use

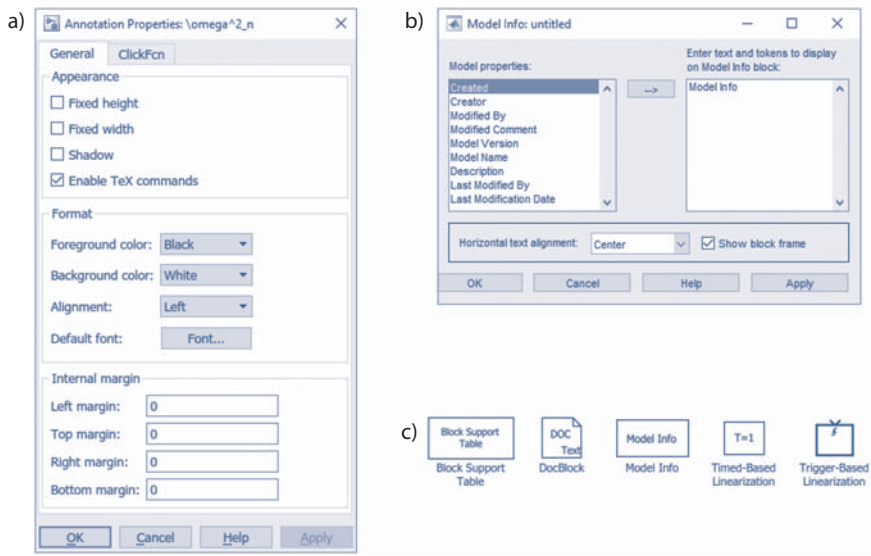


Fig. 15.27 a) Annotation Properties dialog box, b) Model Info window, and c) the Model-Wide Utilities library.

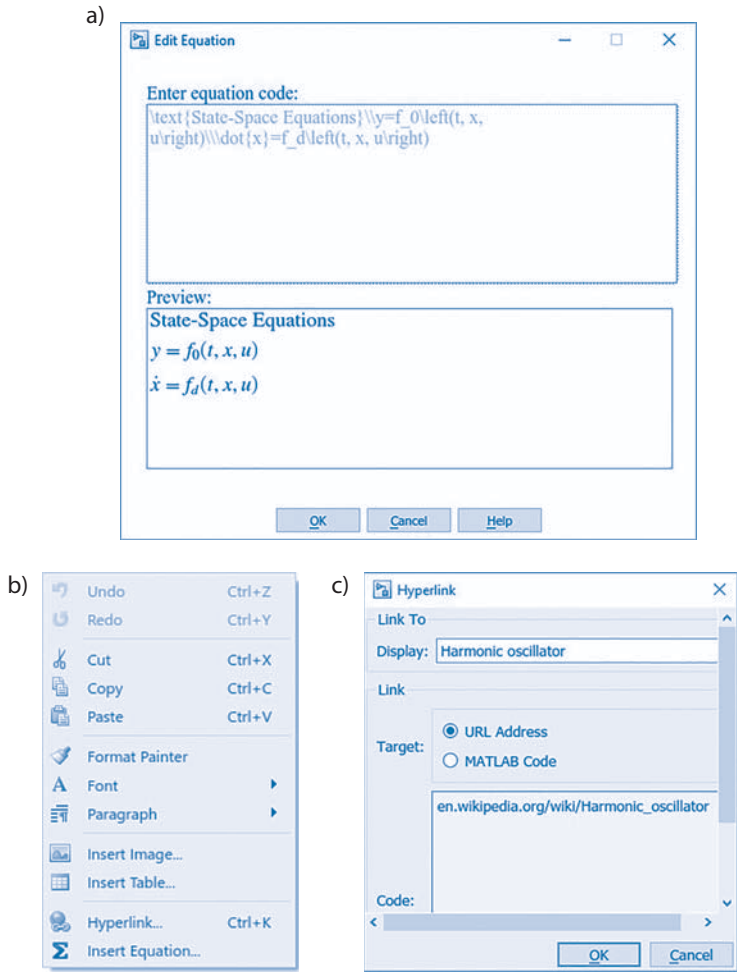


Fig. 15.28 a) Edit Equation window, b) Annotation context menu, and c) Hyperlink dialog box.

hyperlinks to open a website or make any content of a text annotation interactive (using the MATLAB commands). All you have to do to enable the latter option is to select the content that you want to be interactive, right-click, and choose **Hyperlink** from the context menu (Fig. 15.28b). In the Hyperlink dialog box (Fig. 15.28c) you can either select and add a URL address or enter a MATLAB code you want to be executed when the hyperlink is clicked on.

To visually group related model elements in a box, you can use the **Area** icon of the Simulink Editor palette. Drag this icon to your model or click the icon and then click the canvas. If needed, you can label the area (the name always appears in its upper-left corner), as well as change the area and its border color. You can move the area with the blocks it surrounds (dragging

the area near the border) or without them (holding <Alt> and dragging). You can also create an area by dragging a box around the area of interest in the model and selecting the second, **Create Area**, option from the ellipsis menu (the first option, **Create Subsystem**, will be considered later in this section).

Figure 15.29 shows an example of using some of the aforementioned readability improvements for the *SimulinkC1.slx* model of Fig. 15.13. As seen, these improvements also include adding a drop shadow and distinguishing the alike blocks by different colors (using the tools of Fig. 15.12a, 15.7c, or 15.12b).

The model shown in Fig. 15.29 also features two connector lines between annotations and the gain blocks. The connector is similar to a callout, identifying the block that an annotation applies to. If you move the annotation or block, the connector moves or resizes to keep the connection. To add a connector line, place the cursor over the annotation outline where you want the connector to start (a crosshair should appear) and drag the connector line to the block you want to connect to. To specify the color or width of connector, right-click it and choose the **Format** option.

The Property Inspector panel shown in Fig. 15.29 can be added using the corresponding tool found in any of the drop-down icon panels presented in Fig. 15.8 or by choosing the corresponding option from the model (right-click) context menu. Similar to the Property Inspector for figures (see Fig. 6.56), the Simulink Property Inspector stays open in the editor as you work on your model and allows you to check/set the values for the current selection (the Gain block in Fig. 15.29) without the necessity of opening multiple Parameters dialog boxes. You can drag the Property Inspector from the default location and dock it in another location in the editor as needed, or

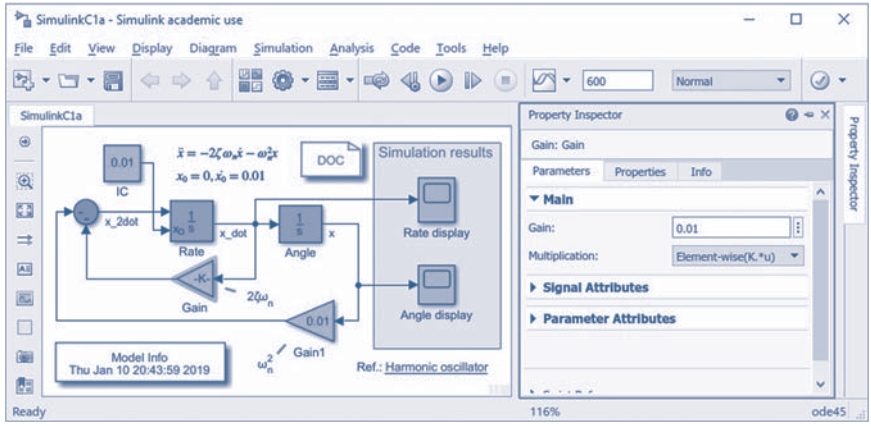


Fig. 15.29 Example of a model with enhanced readability featuring the Property Inspector panel.

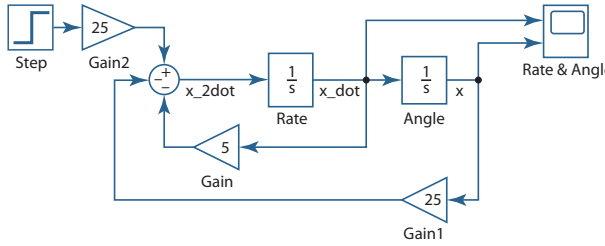


Fig. 15.30 Modified second-order system with a single Scope block (compare with Fig. 15.13).

even better, you can collapse the Property Inspector interface by clicking the pushpin icon in its upper-right corner. In this latter case, a Property Inspector tab (shown by the right edge in Fig. 15.29) appears. Clicking this tab allows bringing the Property Inspector back (as shown Fig. 15.29) or collapsing it again.

Another tool you might find useful is the Model Data Editor that can be opened using the **Model Data Editor** button, , in the Design group of the Modeling tab (Figs. 15.7b, 15.8b). Using tabular data about your model allows you to configure multiple signals, states, and algorithmic parameters at once.

Next, let us show how to use a multi-input Scope block. To this end, Fig. 15.30 represents essentially the same Simulink model as in Fig. 15.13, but with a few changes. First, the parameters of the model are changed to accommodate the short-period motion of Eq. (15.3) with $t_f = 3$.

Second, the ICs for both Integrator blocks are set to zero (internally). To perturb the system, the Step block (Fig. 15.25c) is added, and so, instead of the original Eq. (15.2), we now use

$$\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2 x = \omega_n^2 f(t) \tag{15.8}$$

where $f(t)$ represents some external force input (in our case – a *unit step function* $\mathbf{1}(t)$) and the ω_n^2 multiplier is added to normalize the output (to make the steady-state value of an angle equal to 1).

Third, compared with the model of Fig. 15.13, the model of Fig. 15.30 uses only one scope but with the two input ports. You can change the default number of the input ports while adding a new Scope block to your model using a prompt (similar to that of Fig. 15.25d) or you can change it afterwards by right-clicking the existing Scope block and choosing the **Signals & Ports** option from the context menu (Fig. 15.12a).

Running this modified model results in the scope shown in Fig. 15.31a. Note that to distinguish the multiple lines, a legend was added [you can add it using the last icon of the **Configuration Properties** drop-down menu shown in Fig. 15.17b or via **View** => **Legend** (Fig. 15.19c)]. If needed, you may change the scope's layout by clicking the second-to-last

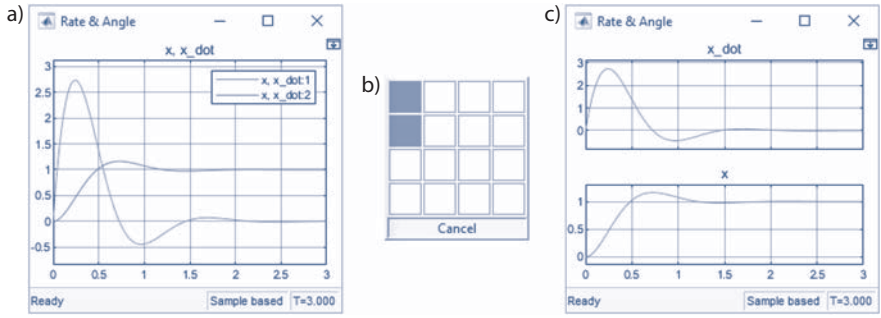


Fig. 15.31 a) Using a multi-input scope, b) changing scope's layout, and c) the 2-by-1 grid arrangement for a two-input scope.

icon of Fig. 15.17b or clicking the **Layout** button in the scope Configuration Properties window (Fig. 15.18a). Either action brings up a Layout menu, which you would use to define a 2-by-1 layout (Fig. 15.31b), resulting in the display shown in Fig. 15.31c.

You could have produced the same result shown in Fig. 15.31a by using just the default single-input Scope block. To be able to do that, you should have combined two input signals (of the same data type and numeric type) into a virtual vector. Figure 15.32a features the multiplexing, Mux, block (from the Signal Routing library shown in Fig. 15.33), blending the two model outputs together. (Note that for this model, the 2-by-1 arrangement of Fig. 15.31c does not work—both signals would appear in the upper plot).

Now that you have started developing relatively complex models with a lot of blocks, it is time to introduce Simulink *subsystems* that allow you to create a hierarchical model comprising many layers (which is somewhat similar to the functions concept of MATLAB). A subsystem is a set of blocks that you replace with a single Subsystem block and, as such, it helps in reducing the number of blocks displayed in your model window, hence also contributing to the model's readability.

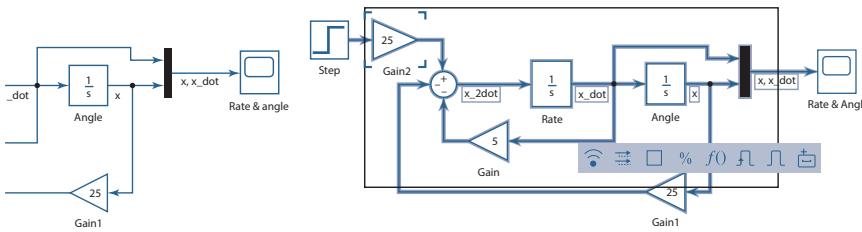


Fig. 15.32 a) Combining the signals using the Mux block and b) creating a subsystem.

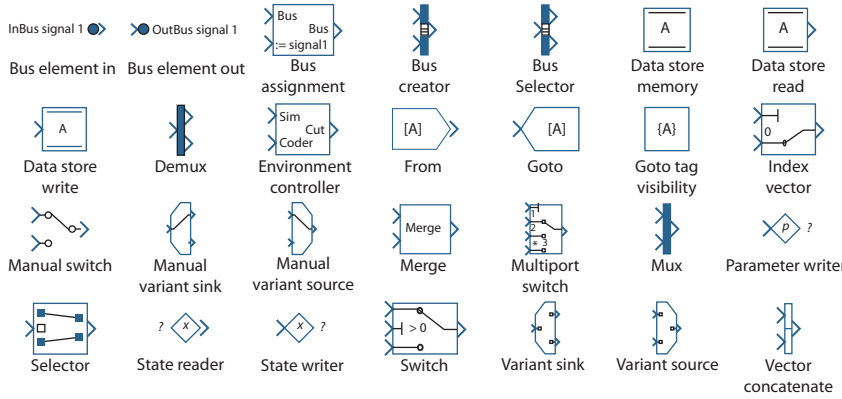


Fig. 15.33 Signal Routing library blocks.

Let us select almost all the blocks of the last model, excluding the input Step block and output Scope block, as shown in Fig. 15.32b. To select several blocks, left-click at the location of one of the corner blocks and then, while continuing to depress the mouse button, drag the bounding box to enclose the desired area. If this does not work (the blocks happen to have several unwanted blocks between them), hold down the <Shift> key and click each object you want to select/deselect. Figure 15.32b also features the ellipsis menu offering several options for the selected blocks – specifically, enabling data logging, auto-routing the lines, creating an area, commenting selected blocks out, and creating Function-call, Triggered, Enabled, and simple subsystems. Choosing the very last option creates a subsystem block as shown in Fig. 15.34a (with the default name "Subsystem" replaced with a meaningful one, "2nd-order system").

A subsystem's content is presented in Fig. 15.34b. To see it, you need to double-click the subsystem block or use the Explorer Bar (by clicking the

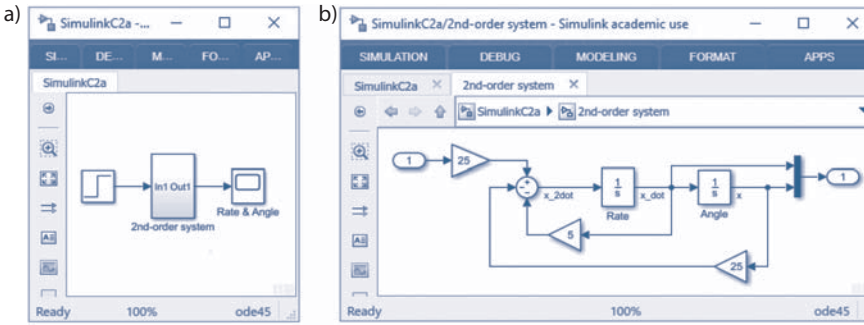


Fig. 15.34 a) Subsystem superblock and b) its content (opened in a new tab).

triangle to the right of the model's name). Alternatively, you can open it in a new tab or in a new window (these two options are added to the standard context menu of a simple block shown in Fig. 15.12a). Note that the newly created subsystem (Fig. 15.34b) features two new blocks, the Inport block from the Sources library (Fig. 15.3) and the Outport block from the Sinks library (Fig. 15.4), added automatically during subsystem creation.

Also, now that you have a multilayer model, the Explorer Bar features three additional buttons: **Navigate Back**, **Navigate Forward**, and **Navigate Up to Parent** (in our case—to the *root level*). Alternatively, you can simply click the model's name in the Explorer Bar.

To create a simple subsystem with multiple blocks selected, as in Fig. 15.32b, you could also right-click and choose the **Create Subsystem from Selection** option from the context menu, which is very similar to the upper portion of the Fig. 15.12a context menu for a single block. Alternatively, with several blocks selected, you can click the **Create Subsystem** button of the Component group of the Modeling tab (in this case, the **Insert Subsystem** button of Fig. 15.7b turns into the **Create Subsystem** button).

As seen, a subsystem is similar to an area (Fig. 15.29) in that it is a way to group related blocks. However, a subsystem creates a hierarchy, replacing multiple blocks in a model with a single block. You can initially group related blocks in an area and later decide to put those blocks in a subsystem by converting the area (to convert an area to a subsystem, right-click the area and select the **Create Subsystem from Area** option from the context menu). The resulting subsystem has the same name, blocks, description, and requirements traceability information as the area.

When you make a copy of a subsystem, the copy is independent of the source subsystem, meaning that you can reuse it across a model or models. To make your subsystem block more universal, you might consider creating a mask. This way, the content of your block will be masked, and instead of going inside it when double-clicked, its own Parameters dialog box would show up. Most of the blocksets, including the Aerospace Blockset seen in Fig. 15.2 and discussed in Appendix E, are composed of such masked superblocks.

15.3.3 Using the Transfer Function and State–Space Blocks

Consider the same second-order system of Eq. (15.8) with the normalized input, that is, having $\omega_n^2 f(t)$ on the right-hand side of equation. Assuming zero ICs (that is, $x(0) = 0$ and $\dot{x}(0) = 0$) and taking the *Laplace transform* yields

$$s^2X(s) + 2\zeta\omega_n sX(s) + \omega_n^2 X(s) = \omega_n^2 F(s) \tag{15.9}$$

The ratio of the Laplace transform of the output (angle), $X(s)$, to the Laplace transform of the input, $F(s)$, forms the standard *transfer function*

$G(s)$ of the second-order system

$$G(s) = \frac{X(s)}{F(s)} = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \tag{15.10}$$

That is the form of a dynamic system with which the Transfer Fcn block of the Continuous library (Fig. 15.10) deals.

Let us replace the Subsystem block of Fig. 15.34a with the Transfer Fcn block as shown in Fig. 15.35a. The Parameters dialog box for this block is presented in Fig. 15.35b. It shows how the numerator and denominator of the transfer function of Eq. (15.10) are introduced as a regular MATLAB-type representation of polynomials by their coefficients. The only disadvantage is that the transfer function of Eq. (15.10) links the input to the angle only. The input-to-rate transfer function needs to be developed separately. The response of the system presented in Fig. 15.35a is shown in Fig. 15.35c.

Next, let us convert Eq. (15.8) to the state–space form similar to that of Eqs. (14.12) and (14.14). Introducing two new variables, $x_1 = x$ and $x_2 = \dot{x}$, results in

$$\begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= -2\zeta\omega_n x_2 - \omega_n^2 x_1 + \omega_n^2 f(t) \end{aligned} \tag{15.11}$$

These two first-order differential equations can further be expressed using the canonical state–space representation form

$$\begin{aligned} \dot{\mathbf{x}} &= \mathbf{A}\mathbf{x} + \mathbf{B}f(t) \\ y &= \mathbf{C}\mathbf{x} + \mathbf{D}f(t) \end{aligned} \tag{15.12}$$

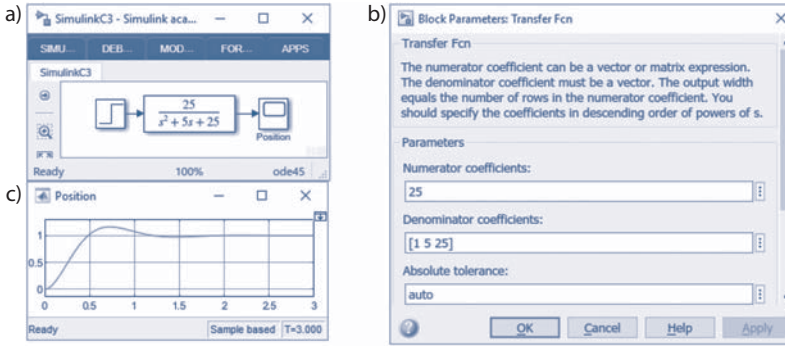


Fig. 15.35 a) Second-order system modeled with a Transfer Fcn block, b) its Parameters dialog box, and c) its response.

with the state vector, state (system) matrix and input matrix being

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \mathbf{A} = \begin{bmatrix} 0 & 1 \\ -\omega_n^2 & -2\zeta\omega_n \end{bmatrix}, \mathbf{B} = \begin{bmatrix} 0 \\ \omega_n^2 \end{bmatrix} \tag{15.13}$$

While the first equation in Eq. (15.12) represents dynamics of the system, the second equation defines the output vector $\boldsymbol{y}$. Compared with Eq. (14.14), the canonical form of the output equation is based not only on the output matrix **C** but also on an additional term involving the so-called *feedthrough* (or *feed forward*) *matrix* **D**.

In the general case of a multiple-input/multiple-output (MIMO) model, vector **x** may have n components; vector **y** – m components; and input signal $\overline{\mathbf{f}}(t)$ – r components. In this case, matrices **A**, **B**, **C**, and **D** have $n \times n$, $n \times r$, $m \times n$, and $m \times r$ elements in them. In our specific case of a single-input/single-output (SISO) model described by Eq. (15.12), $m = 1$ and $r = 1$, $\mathbf{C} = [1 \;\; 0]$ (we are only observing the time history of the state x_1), and $\mathbf{D} = [0]$ (meaning that there is no direct transmittance).

Figure 15.36 shows a block diagram of the state–space model of Eq. (15.12) implemented in Simulink explicitly as a subsystem block similar to that of Fig. 15.34b. Alternatively, you can utilize the State–Space block from the Continuous library (Fig. 15.10), as shown in Fig. 15.37.

The upper part of the model presented in Fig. 15.37a shows a Simulink model for Eq. (15.12) with the parameters of the short-period motion defined in matrices `A`, `B`, `C`, and `D` in the Parameters dialog box of the State–Space block (Fig. 15.37b). The bottom portion of this Simulink model uses one more State–Space block featuring

$$\mathbf{C} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{15.14}$$

(introduced in the State–Space block dialog box as `C=eye(2)`) and

$$\mathbf{D} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{15.15}$$

enabling observation of both the angle and rate on the same display.

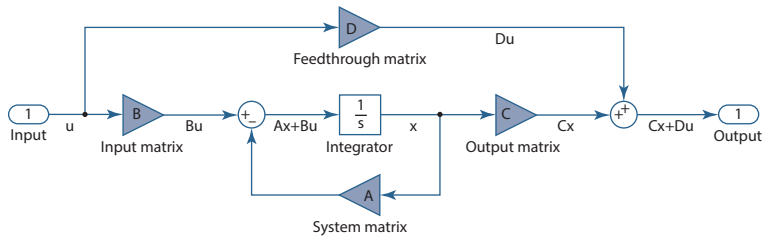


Fig. 15.36 Generic state–space model.

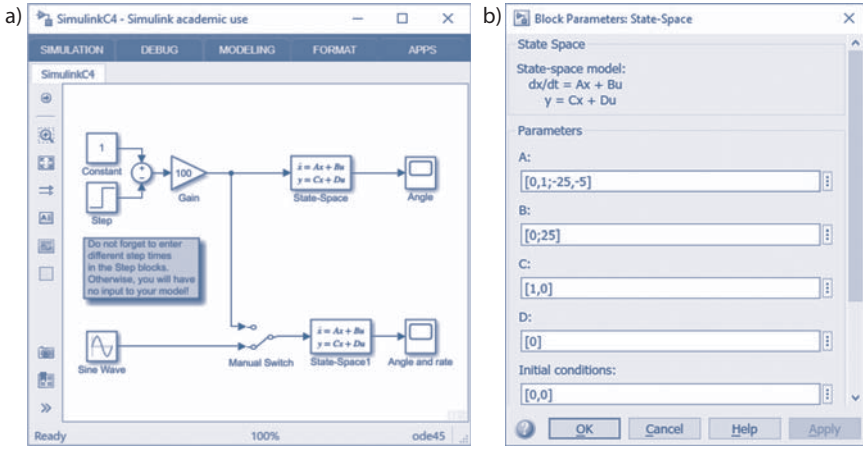


Fig. 15.37 a) State–space model of the second-order system and b) Parameters dialog box for the State–Space block.

The aperiodic input in the model of Fig. 15.37a is formed using the combination of the Constant block and Step block with the **Step Time** set to 0.01 (Fig. 15.26a) to generate an impulse input

$$f(t) = 100(\mathbf{1}(0) - \mathbf{1}(0.01)) \tag{15.16}$$

The model also includes the Sine Wave and Manual Switch blocks taken from Sources library (Fig. 15.3) and Signal Routing library (Fig. 15.33), respectively, to allow using the two different input signals.

15.4 Modeling Discrete Systems

So far, we have dealt with analog- or continuous-type systems. Simulink, however, provides full support for discrete-type systems as well. You can create discrete systems the same way that you create analog systems, but using the Discrete rather than Continuous library of the Simulink Library Browser. The content of this library is presented in Fig. 15.38, and the following two sections introduce some major blocks implemented for very simple systems to avoid getting into much detail of defining discrete systems per se.

15.4.1 Using the Discrete Library Blocks to Model Simple Systems

Figure 15.39 shows the Parameters dialog boxes of three major (most often used) blocks of the Discrete library. The Memory block (Fig. 15.39a) simply outputs its input from the previous time step. The Unit Delay block (Fig. 15.39b) delays its input by the specified sample period. The Discrete-

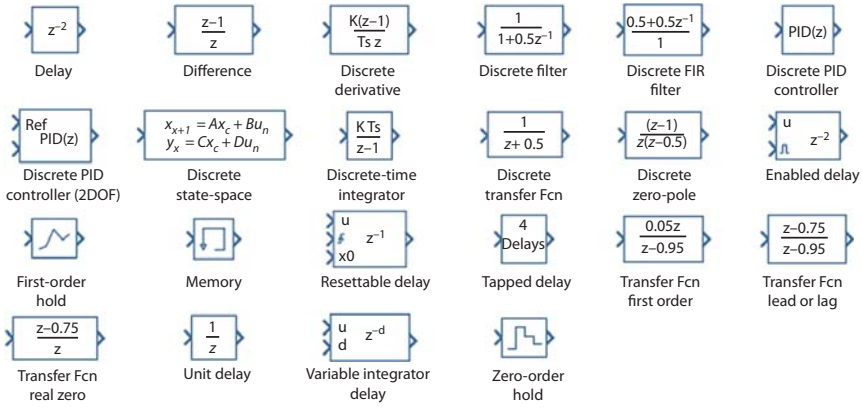


Fig. 15.38 Discrete library blocks.

Time Integrator (Fig. 15.39c) performs discrete-time integration or accumulation of an input signal. As seen, all three blocks require ICs.

In addition to these three blocks, the Zero-Order Hold block samples and holds its input for the specified sample period. It is pretty similar to the Unit Delay block but does not require the IC. The only parameter of this block is the time between samples specified with the **Sample time** parameter (similar to that appearing at the bottom of Fig. 15.39c). A setting of `-1` means the **Sample time** is inherited. (The Unit Delay block features the **Sample time** parameter as well.)

During the compilation phase of simulation, Simulink determines the sample time of the block from its **Sample time** parameter (if it has it),

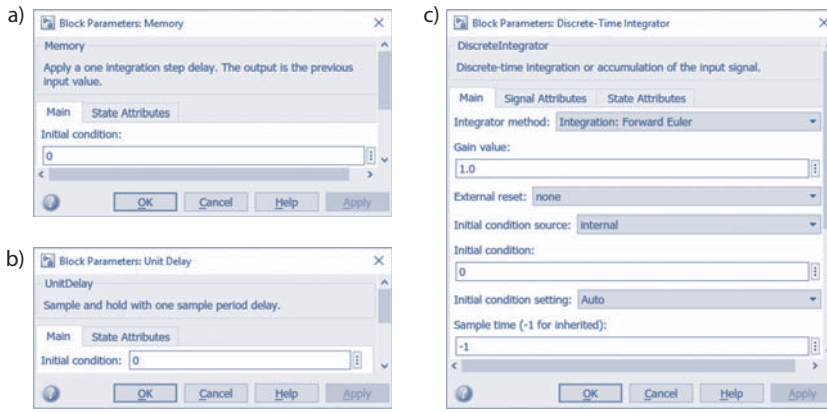


Fig. 15.39 Parameters dialog boxes for the a) Memory, b) Unit Delay, and c) Discrete-Time Integrator blocks.

sample-time inheritance, or block type (continuous blocks have a continuous sample time). It is this compiled sample time that determines the sample rate of a block during simulation.

To clarify, consider the simple model of Fig. 15.40a using the Memory, Zero-Order Hold, and Discrete-Time Integrator blocks. The **Sample time** in the two latter blocks is set to `1` (the Memory block does not have this parameter.) The Ramp block of the Sources library of Fig. 15.3 is used to provide an input signal. The Sum block is set to be rectangular (Fig. 15.14a). The **Number of input ports** in the Scope block is set to `3` (Fig. 15.18a). The model uses the default (auto) variable-step solver (with the stop time of `4` time units). When you run this model and then double-click the Scope block (and apply autoscale), the results of the simulation will show up as presented in Fig. 15.40b.

At the very first step, all signals are zero. Then, the variable-step solver starts a simulation with a small step size [corresponding to the default relative tolerance of 10^{-3} (Fig. 15.16)]. The Memory block holds the previous value, so that the upper display in Fig. 15.40b shows the step size change. The two other blocks, however, hold their ICs until the integration time reaches the sample time. At $t = 1$, the Ramp block outputs `1`, and the output of the Zero-Order Hold block jumps to `1` to be held constant for the next sampling period (defined by the sample time). According to the Euler forward integration formula of Eq. (14.20) (as defined in Fig. 15.39b), the Discrete-Time Integrator produces `0` (`0+1*0`) and holds this value for the next sampling period as well. At $t = 2$, the Ramp block outputs `2`, and so the output of the Zero-Order Hold block jumps to `2`. The Discrete-Time Integrator produces `1` (`0+1*1`). At $t = 3$, the Ramp block outputs `3`, the Zero-Order Hold block jumps to `3`, the Discrete-Time Integrator produces `3` (`1+1*2`), and so on.

Next, consider the model of Fig. 15.41a. Two Unit Delay blocks are used here to generate a Fibonacci number sequence introduced in Sec. 10.2.3

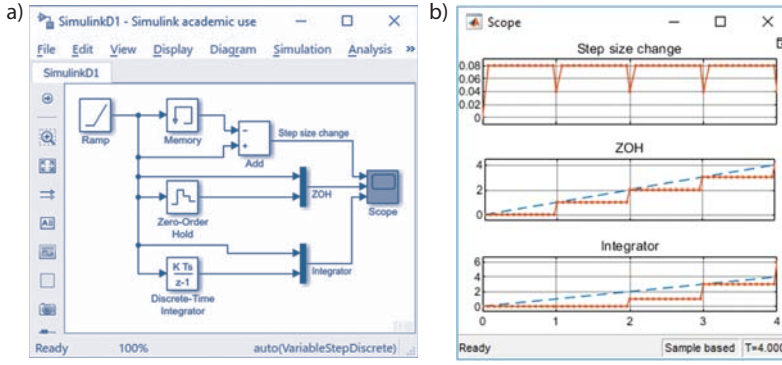


Fig. 15.40 a) Exploring Discrete library blocks while b) illustrating the effect of the **Sample time** parameter.

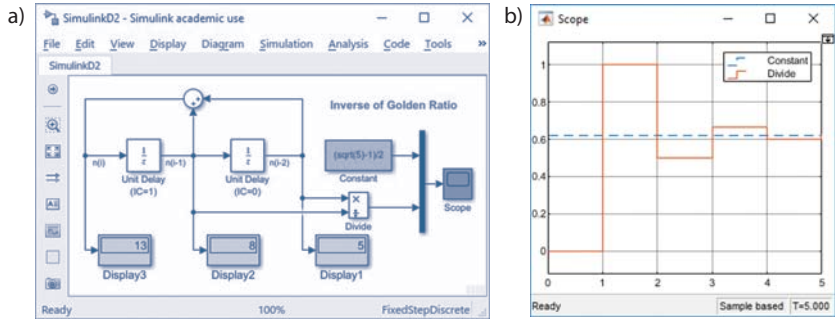


Fig. 15.41 a) Generating Fibonacci numbers and b) finding the inverse of the golden ratio.

[Eq. (10.15)]. The IC for the rightmost Unit Delay block is set to 0 (the very first number in the Fibonacci sequence), and the left block is set to 1 (the second number). The Divide block from the Math Operations library (Fig. 15.5) is used to model the inverse of Eq. (10.16), which is supposed to converge to the inverse of the golden ratio of Eq. (10.14), presented by the exact value in the Constant block. Figure 15.41a shows the result of running this model for five steps, so that the three displays at the bottom of the model (taken from the Sinks library of Fig. 15.4) show the sixth, seventh, and eighth Fibonacci numbers (from right to left), whereas the Scope in Fig. 15.41b proves that this sequence of numbers does converge to the inverse of the golden ratio.

Last, consider a loan amortization model with the loan balance b_k at the end of month k computed as the sum of the balance at the beginning of the month b_{k-1} plus the interest for the month minus the end-of-the-month payment p_k:

$$b_k = \left(1 + \frac{1}{12}\frac{r}{100}\right) b_{k-1} - p_k \tag{15.17}$$

In this equation, the annual percentage rate r is adjusted to the monthly rate.

Assume that upon completing this textbook you became a proficient programmer, allowing you to think about setting aside $3,000 a month to make payments for your own airplane. Would it be a general aviation aircraft worth $100,000 (Fig. 15.42a) or perhaps a luxury $3,000,000 turbojet (Fig. 15.42b)?

The model presented in Fig. 15.43a helps you make a choice by computing when you could pay off your loan. The Unit Delay block with the IC of 103 (thousands of dollars) to offset a $3,000 (virtual) payment occurring at month 0 is used to model the right-hand side of Eq. (15.17). The Fcn block (from the User-Defined Functions library of Fig. 15.44), displaying a mathematical expression as typed in it, computes the interest. The Discrete-Time Integrator block (with the default Euler forward method

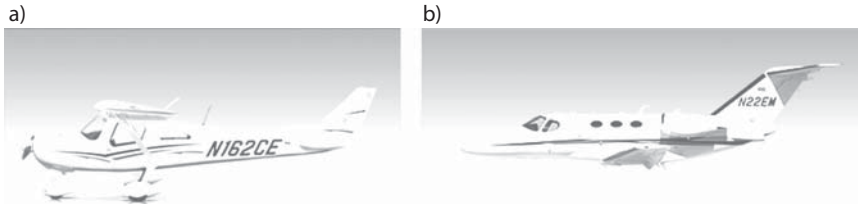


Fig. 15.42 a) Two-seat Skycatcher and b) five-seat Citation Mustang.

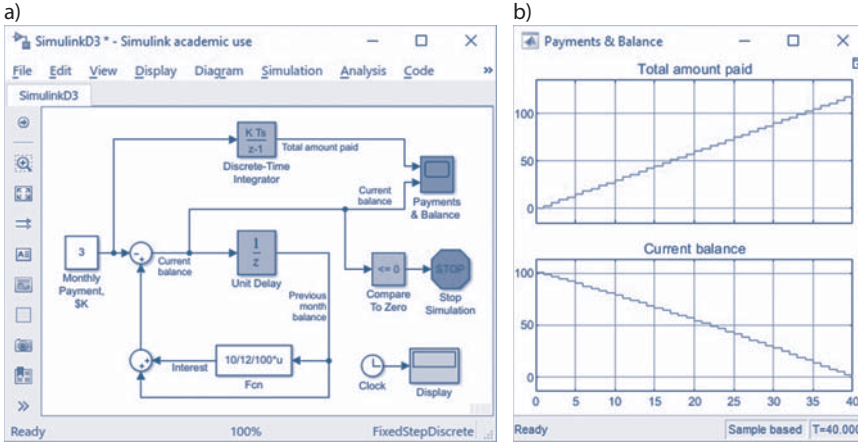


Fig. 15.43 a) Aircraft loan amortization model and b) its output.

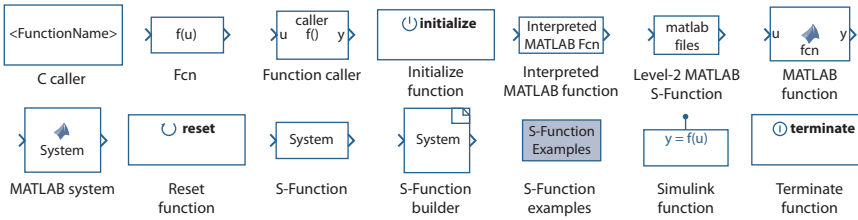


Fig. 15.44 User-Defined Functions library blocks.

of integration) is used to compute the total amount of payments. The Compare To Zero block from the Logic and Bit Operations library (Fig. 15.45) outputs 1 when the input value becomes negative (0 otherwise), and the Stop Simulation block from the Sinks library stops simulation when the input signal becomes 1. The Clock block of the Sources library outputs the current simulation time. The number of steps is set

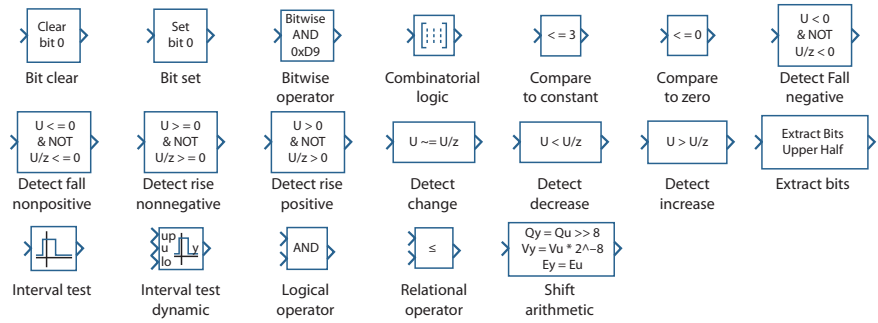


Fig. 15.45 Logic and Bit Operations library blocks.

to `100`, meaning that you are counting on paying off your loan within eight to nine years (100 months).

When you run the model of Fig. 15.43a, the balance becomes negative (meaning that you paid off your loan) sooner than 100 months. To be exact, the simulation stops at the 40th month, triggered by the Stop Simulation block (negative balance means that the actual last payment should be less than $3,000). The results of the simulation are shown in Fig. 15.43b. The bottom line is you can probably afford a $100,000 buy. What about a turbojet? Well, we leave this question for the last problem set of this textbook.

15.4.2 Continuous Model Discretization and Controller Design

The examples considered in the previous section are probably more applicable to address problems in finance, logistics, maintenance, etc. Systems dynamics and controls deal with slightly different models and require some specialized tools found in the Apps tab (Fig. 15.7d).

Open the *SimulinkC1.slx* model of Fig. 15.13 and save it under another name, say, *SimulinkC1d.slx.* Click the **Model Discretizer** app button (the second-to-last button in Fig. 15.7d). The Model Discretizer window pops up as shown in Fig. 15.46a. This tool analyzes your model, automatically identifies all continuous blocks conflicting with the discretized version (two integrators in our case), marks them with a red foreground (Fig. 15.46b), and lists them in the Contains continuous blocks pane, suggesting that you discretize them (replace them with the corresponding blocks from the Discrete library).

Pushing the fifth icon of the toolbar, **Discretize current selection**, , or a similar button at the lower right of the Simulink Model Discretizer window, converts the selected blocks (or all of them) to their discrete versions. Specifically, the *Simulink1Cd.slx* model becomes as shown in Fig. 15.47a with two converted blocks highlighted with a blue foreground (note, this particular view of the model can be obtained by clicking the lower-right corner of the canvas where the gray **Show Perspectives views** button resides).

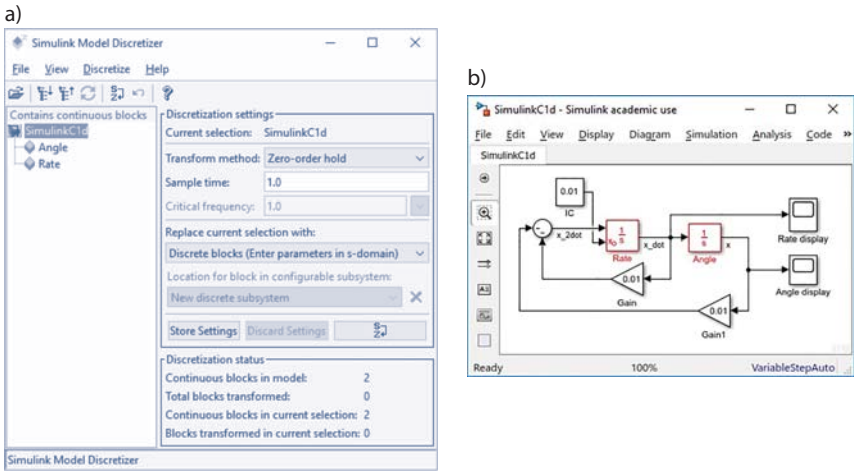


Fig. 15.46 a) Simulink Model Discretizer window and b) two blocks suggested to be discretized in the continuous model.

Running this model with $t_f = 50$ results in a discrete (ladder-type) response (compare the Angle display output of Fig. 15.47b with the smooth output of the original model presented in Fig. 15.17b). To be more precise, the output of *Simulink1Cd.slx* is a discrete version of the first 50 time units as displayed in Fig. 15.17b.

In practice, you may not need to discretize all the blocks. For example, you want to design a *controller* for some analogous (continuous) system with the goal of improving its performance or making it follow some reference signal. You first create a model of your system verifying that it matches the real one, that is, that the outputs of your model (called *plant*) are about the same as the real system. The next step is to develop a *feedback* controller that would take the outputs of your system, compare them to the

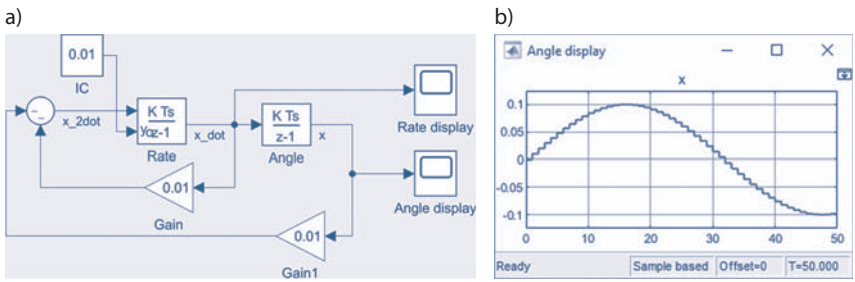


Fig. 15.47 a) Discrete version of the second-order system and b) its Angle display output.

desired outcome, and calculate the necessary corrective actions. Before you transfer the controller, developed and working properly for your continuous model, onto the real system, you would need to ensure that the real-life constraints on the controller will not degrade the system's performance. These real-life constraints may be based on the fact that the measurements of the output parameters of your system can only be taken at a certain rate, say 2 Hz or 100 Hz, and that the control actions are also discrete rather than continuous. In other words, you would want to discretize the controller portion of your model, leaving the model itself continuous. Simulink allows you to do this. It also allows you to create multirate simulations. Whereas this subject is beyond the scope of this textbook, the model of Fig. 15.48 illustrates what we are talking about.

The second-order plant of Fig. 15.48a

$$F(s) = \frac{1}{s^2 + 2s + 1} \tag{15.18}$$

is modeled with the Transfer Fcn block. The plant itself (see the upper portion of Fig. 15.48a) produces a kind of sluggish response to the step input shown in Fig. 15.48b. We would like to improve it, and for this purpose, we designed and tuned a continuous proportional-integral-derivative (PID) controller, as shown in the middle portion of the model of Fig. 15.48a. (By the way, as seen in Fig. 15.10, Simulink features two PID Controller blocks, offering a family of controllers and allowing you

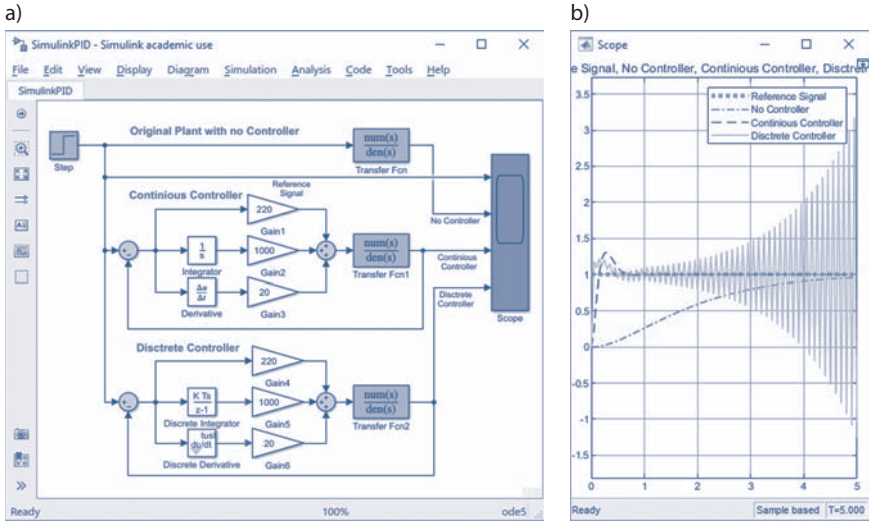


Fig. 15.48 a) Example of a PID controller design and b) illustrating divergence of a discretized controller.

to tune their parameters.) The improved response of the developed controller is illustrated in Fig. 15.48b as well. Although overshooting, it converges to the reference signal (step input) much quicker than for the original system.

Is this controller ready to be implemented on a real system? Not quite. The lower portion of the model in Fig. 15.48a represents the case where the very same controller was discretized, whereas the plant remained continuous. As seen in Fig. 15.48b, the discretized version of this controller (with the sample time of 0.05 s) results in divergence, that is, an inability to follow the reference signal. Hence, the controller must be redesigned.

15.5 Other Libraries of Simulink

For completeness and your convenience, Figs. 15.49–15.57 show the content of remaining libraries of basic Simulink (see Fig. 15.2), complementing those already shown in Figs. 15.3–15.5, 15.10, 15.27c, 15.33, 15.38, 15.44, and 15.45. A few more blocks are available in two additional libraries, Additional Math & Discrete and Quick Insert (not shown here).

15.6 Integration of MATLAB and Simulink

Now that you know both MATLAB and Simulink, you may want to use them together. This will make your models more flexible and useful. In fact, you have used MATLAB syntax to enter inputs into the Block Parameters dialog boxes already. This section provides you with more details on how you can effectively employ MATLAB code in your Simulink models and, vice versa, how to run Simulink models from your MATLAB scripts. The section ends with showing you how to create Simulink blocks from symbolic expressions.

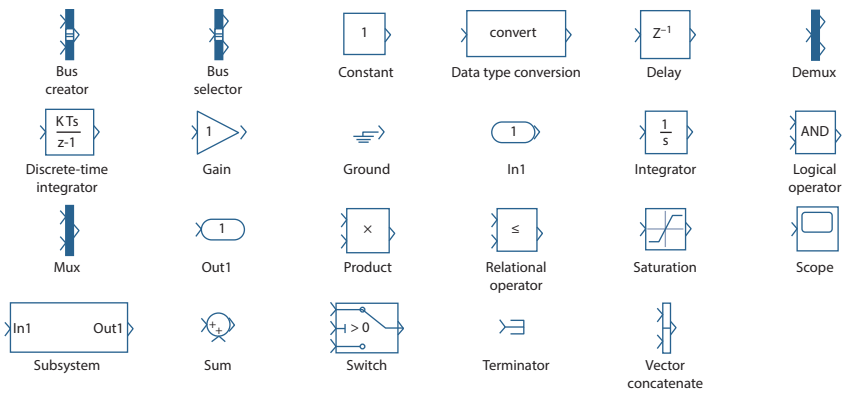


Fig. 15.49 Commonly Used library blocks.

Fig. 15.50 Dashboard library blocks.

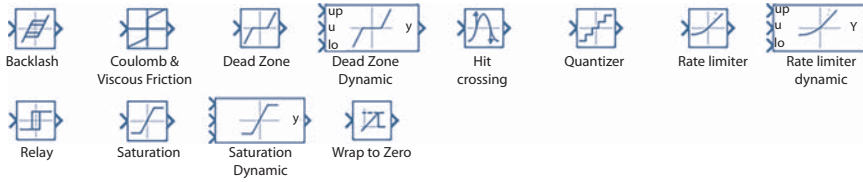


Fig. 15.51 Discontinuous library blocks.

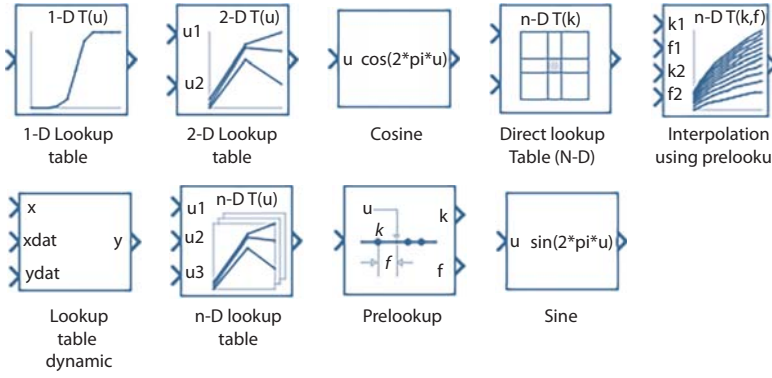


Fig. 15.52 Lookup Tables library blocks.

Fig. 15.53 Messages & Events library blocks.

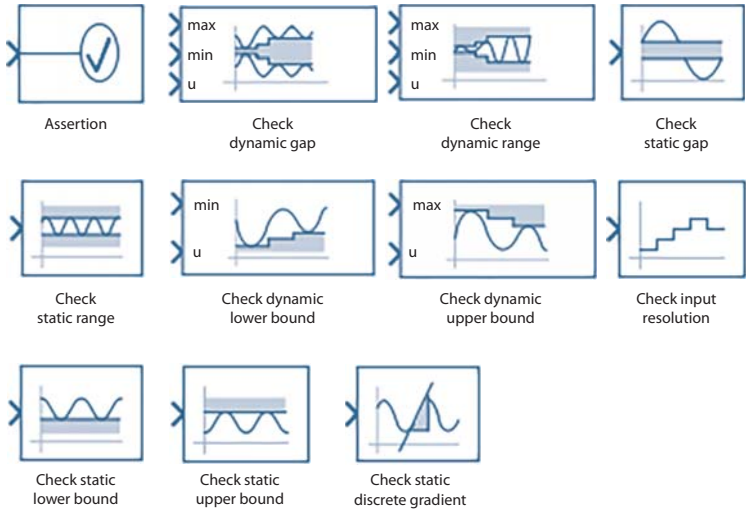


Fig. 15.54 Model Verification library blocks.

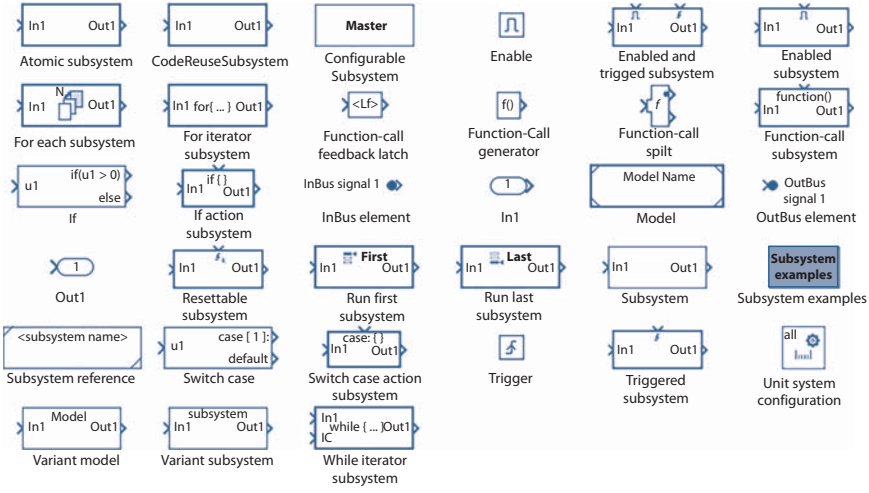


Fig. 15.55 Ports & Subsystems library blocks.

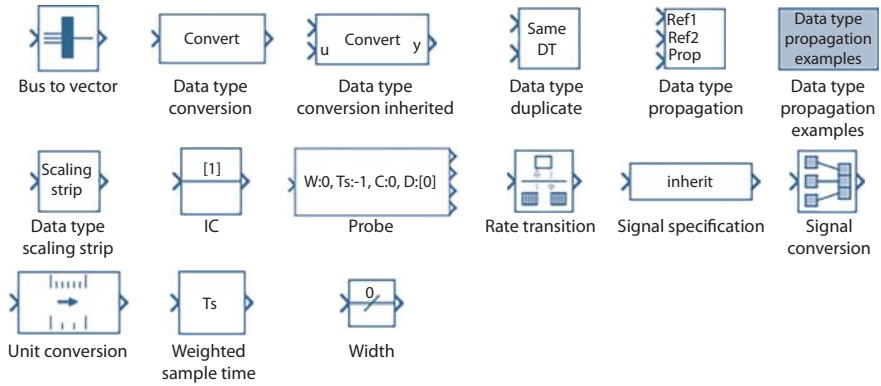


Fig. 15.56 Signal Attributes library blocks.

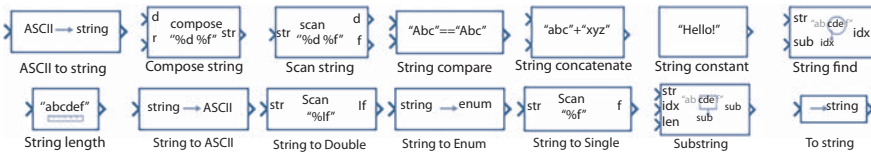


Fig. 15.57 String library blocks.

15.6.1 Calling the MATLAB Commands from Simulink

To begin with, recall the User-Defined Functions library (Fig. 15.44) featuring several blocks where you would call the MATLAB functions by their names, use mathematical expressions, or even employ *m*-file functions.

The Interpreted MATLAB Function block (Fig. 15.58a) allows you to apply a specified MATLAB function (including your own) or expression to the input signal (either scalar or vector). The Fcn block (Fig. 15.58b) applies the specified mathematical expression to its input as well (this block has been used in the model shown in Fig. 15.43a already). As seen in Fig. 15.58b, these expressions follow the MATLAB syntax (with a u notation for an input signal, either scalar or vector). Of course, an expression that appears in the Expression field by default could be modeled using simple blocks as well, but both aforementioned blocks simplify your model by allowing you to dedicate more space to more important parts of your model that cannot be modeled as simple expressions.

Compared with the Interpreted MATLAB Function block, the built-in Fcn block runs faster because it does not call the MATLAB parser during each integration step. However, it has certain deficiencies as well. Specifically, as opposed to the Interpreted MATLAB Function block, it does not support the rounding functions, and so you would need to use the Rounding

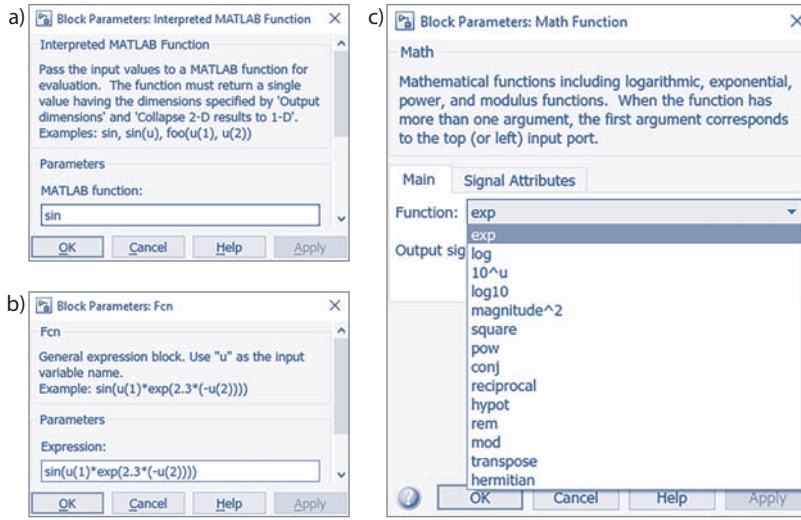


Fig. 15.58 Parameters dialog boxes for the a) Interpreted MATLAB Function, b) Fcn, and c) Math Function blocks.

Function block of the Math Operations library (Fig. 15.5) for that. Also, it uses `sgn` instead of `sign`. To utilize some standard MATLAB functions, you can also use the Math Function block of the Math Operations library (Fig. 15.58c).

The MATLAB Function block from the User-Defined Functions library allows you to open the MATLAB Editor and have the MATLAB function created while you are still working with the Simulink model (Fig. 15.59). Do not forget that you should use `u` as a single input and `y` as a single output (they both may be arrays though).

Next, you may use any variable that you have in the base workspace as a parameter in any block. For instance, you may modify the model of Fig. 15.34a so that its subsystem looks like the one shown in Fig. 15.60a. Specifically, instead of the hard-coded constant values, the Gain blocks now contain `2*z*wn` (Gain) and `wn^2` (Gain1 and Gain2). In addition to this, the IC for the Angle integrator is defined as `x0`. Now, if before

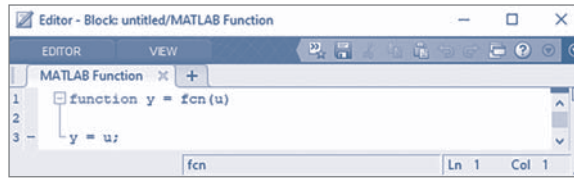


Fig. 15.59 The default Editor window of the MATLAB Function block.

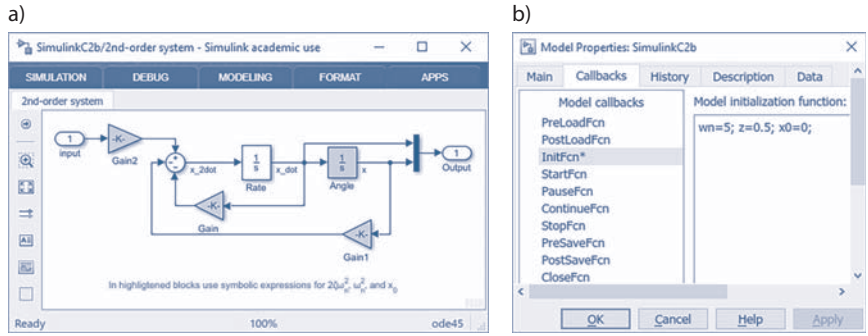


Fig. 15.60 Example of a) using variables' names rather than numerical values and b) defining these variables during model initialization using model callbacks.

running your Simulink model you issue the following commands (in the MATLAB Command Window)

```
>> wn=5; z=0.5; x0=0;
```

these variables will be defined in the base workspace and therefore visible to your Simulink model. When you run it, the values from the workspace will be used to compute the gain and integrator IC values.

Another way of defining variables, used by your Simulink model, is accessing the Callbacks dialog window (Fig. 15.60b) via the **Model Properties** option from the **Model Settings** drop-down menu (Fig. 15.13). As seen, Simulink allows multiple callback functions to be executed at different stages of working with your model. Figure 15.60b shows an example where the aforementioned MATLAB call defining three variables occurs every time before you initialize (run) your model (being executed at the model initialization stage, these variables appear in the current MATLAB workspace and thus become available in the model). The asterisk by InitFcn simply means that you have code in there, whereas other callbacks in this particular case are empty. This way, your Simulink *slx*-file constitutes a complete self-contained model that you could pass to the end user. Instead of messing with your model, the user will only need to change the values of parameters in the InitFcn callback. You may also consider creating a GUI (like the one shown in Fig. 4.21) dedicated to defining all parameters for your model interactively, which would prevent the end user from messing with the callbacks.

You can use the Description tab of the Model Properties window (Fig. 15.60b) to type in a text describing your model. This text will appear in the MATLAB Command Window in response to the help call

```
>> help SimulinkC2b
```

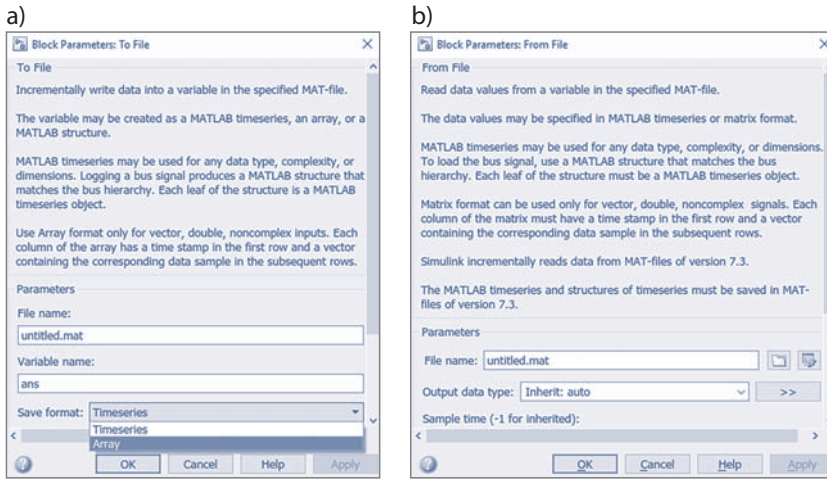


Fig. 15.61 Parameters dialog boxes for the a) To File and b) From File blocks.

Last, you may use the special blocks of the Simulink Sinks library (Fig. 15.4), To File and To Workspace, to store the results of the simulation in a file and the MATLAB workspace, respectively. To read data from these sources into your simulation, you may then use the From File and From Workspace blocks of the Sources library (Fig. 15.3). The dialog boxes for these four blocks are shown in Figs. 15.61 and 15.62. The only

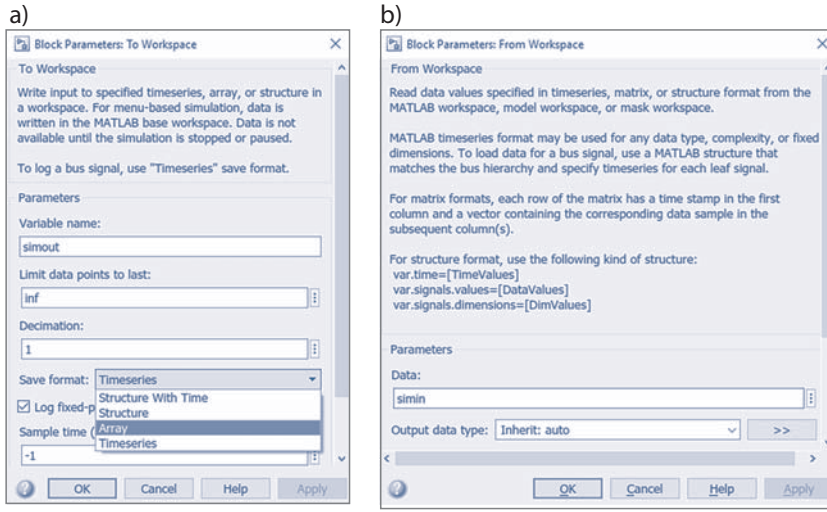


Fig. 15.62 Parameters dialog boxes for the a) To Workspace and b) From Workspace blocks.

comment is that when you store data and are thinking of looking through it/using it afterward, you (as a beginner user) may want to choose employing the **Array** format saving option, as shown in Figs. 15.61a and 15.62a, which would simplify handling it.

15.6.2 Calling the Simulink Models from MATLAB

MATLAB allows you to run Simulink models even without opening the Simulink Editor window. For example, you may run the model developed in Sec. 15.3.1 right from the MATLAB Command Window:

```
[t,x]=sim('SimulinkC1',400,simset('RelTol',1e-5));
yyaxis left, plot(t,x(:,1)*180/pi), ylabel('Pitch, ^o')
yyaxis right, plot(t,x(:,2)*180/pi,'-.'), ylabel('Pitch rate, ^o/s')
xlabel('Time, s')
```

As a result, you will see the plot shown in Fig. 15.63, which combines the two plots depicted in Fig. 15.17. [To produce a two-*y*-axis plot the `yyaxis` function is used (see the script preceding Fig. 6.14a).] It is the Simulink function `sim`, used in the very first line of the aforementioned piece of code, which allows you to run the *SimulinkC1.slx* model and return its results without opening Simulink. The `sim` function outputs a time vector `t`, as well as angle and rate data in a two-column vector `x`.

By default, the `sim` function runs a Simulink model with all the settings you used in the model when creating (saving) it. However, you may alter many of them. For instance, in the aforementioned code, the second input argument in the `sim` function header overrides the stop time for simulation. [As you may recall, in the Simulink model itself, we set it to be `600` (Fig. 15.16).] Also, the relative tolerance was modified in the first line using the `simset` function [similar to the `optimset` function of the Optimization Toolbox (Table 10.3)].

The general syntax for the `simset` function is

```
options=simset('name1',value1,'name2',value2,...)
```

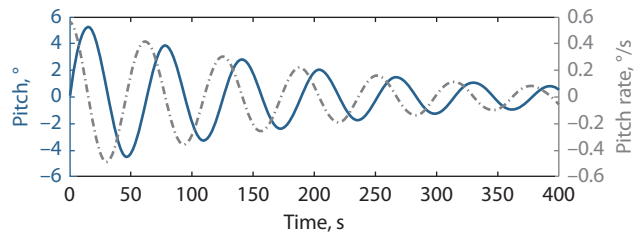


Fig. 15.63 Plotting the results of the Simulink model run in MATLAB.

which creates a Simulink structure `options` where the named properties `name1`, `name2`,... receive the specified values `value1`, `value2`,.... You can see the full list of the named properties with all possible values by issuing the command

```
>> simset
```

The list of properties set for the specific model can be obtained using the `simget` function. For example,

```
>> simget('SimulinkC1')
```

returns

```
ans =                       AbsTol: 'auto'
                    AutoScaleAbsTol: []
                              Debug: 'off'
                         Decimation: 1
                       DstWorkspace: 'current'
                     FinalStateName: ''
                          FixedStep: 1
                       InitialState: []
                        InitialStep: 'auto'
                           MaxOrder: 5
         ConsecutiveZCsStepRelTol: 2.8422e-013
                 MaxConsecutiveZCs: 800
                        SaveFormat: 'Array'
                    MaxDataPoints: 1000
                          MaxStep: 'auto'
                          MinStep: 'auto'
            MaxConsecutiveMinStep: 1
                     OutputPoints: 'all'
                  OutputVariables: 'ty'
                           Refine: 1
                           RelTol: 1.0000e-006
                           Solver: 'ode45'
                       SrcWorkspace: 'base'
                            Trace: ''
                        ZeroCross: 'on'
                    SignalLogging: 'on'
                SignalLoggingName: 'sigsOut'
               ExtrapolationOrder: 4
           NumberNewtonIterations: 1
                          TimeOut: []
ConcurrencyResolvingToFileSuffix: []
           ReturnWorkspaceOutputs: []
    RapidAcceleratorUpToDateCheck: []
    RapidAcceleratorParameterSets: []
```

Most of these properties are the default properties taken from the Configuration Parameters dialog box (Fig. 15.16).

If you need to alter some properties of a Simulink `options` structure, you pass them to the `sim` function as

```
[t,x]=sim('ModelName',timespan,options)
```

Note, if you do not need to alter the `timespan` but still want to pass a new `options` structure, you should replace the `timespan` argument with the placeholder `[]`. The `timespan` can be either a scalar, defining the **Stop time** `TFinal`; a two-element vector `[TStart TFinal]`, defining both the **Start time** and **Stop time** (see Fig. 15.22); or even a longer vector `[TStart OutputTimes TFinal]`, defining some additional time points of interest `OutputTimes` to be returned in `t`. In the latter case, `t` will contain more time points but will definitely include those defined in the sequence `OutputTimes`. You may also choose to run your model as it is, that is,

```
[t,x]=sim('ModelName')
```

15.6.3 Publishing Simulink Models

The capability to run the Simulink models from the MATLAB Command Window means that the results of simulation can be published in the same manner as *m*-scripts, as discussed in Sec. 5.12. You can publish the model itself as well as an outcome of any Scope block. For example, if the *SimulinkC1.slx* model developed in Sec. 15.3.1 was saved with both Scope blocks closed, publishing the script

```
open('SimulinkC1')                          % to publish the model
open_system('SimulinkC1/Rate display')      % to publish an unopened display
open_system('SimulinkC1/Angle display')     % to publish an unopened display
sim('SimulinkC1',[0 100])                   % to run the model
```

results in the HTML printout shown in Fig. 15.64. The first three lines of code add the model and two unopened scopes to the printout, whereas the last command runs a simulation. Note that before publishing, the Simulink Editor window must be closed; otherwise, the model will not show up in the printout.

Figure 15.65 shows another example, when the *SimulinkC1.slx* model was modified to that of Fig. 15.66a to include the InitFcn callback `a=1/80;` (similar to that of Fig. 15.60b) to provide a numerical value for the Gain block and the StopFcn callback (Fig. 15.66b)

```
plot(tout,yout,'-.^')
xlabel('Time, s'), ylabel('Output signal')
title(['d^2y/dt^2+\fontsize{14}'...
        num2str(a,4) '\fontsize{10}dy/dt+0.01y=u(t)'])
```

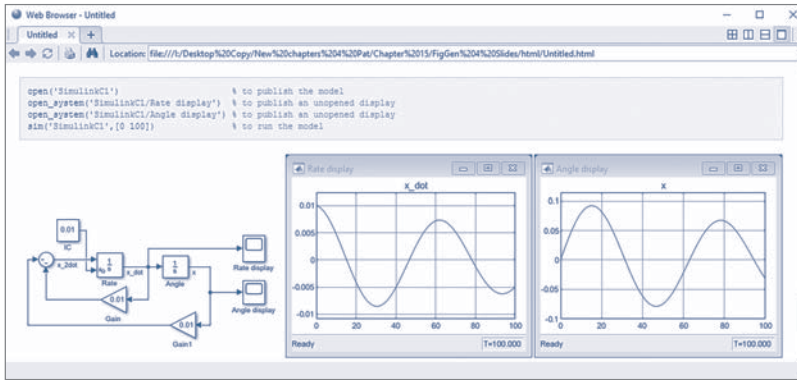


Fig. 15.64 Example of a published model with the scopes.

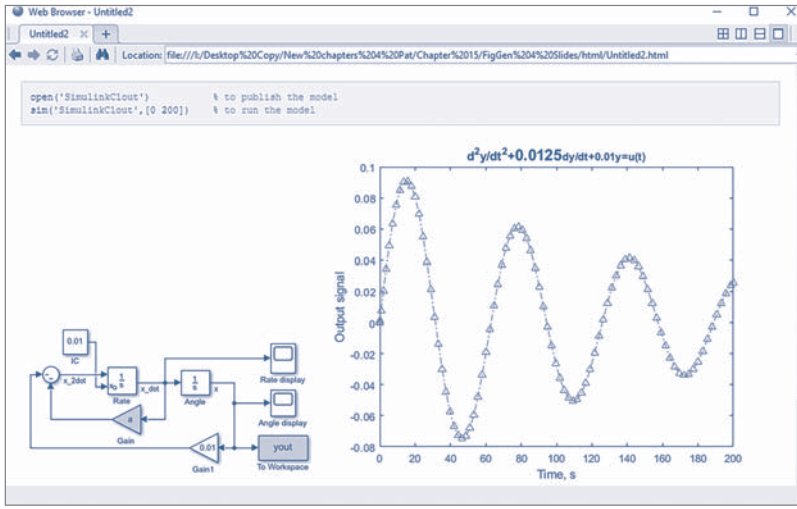


Fig. 15.65 Example of a published model with the plot created in MATLAB.

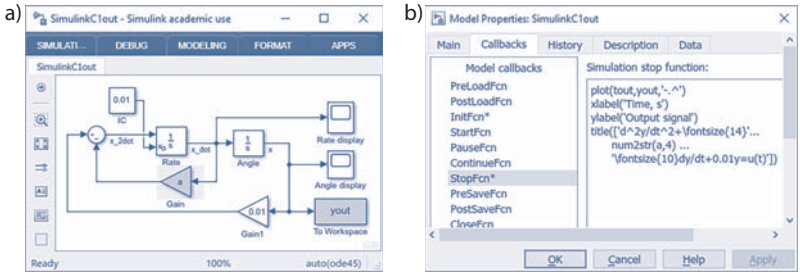


Fig. 15.66 a) Modified *SimulinkC1.slx* model with b) StopFcn callback.

intended to fully use MATLAB plotting capabilities. During simulation, the To Workspace block writes angle data to an internal buffer as the `yout` array (`simout` in Fig. 15.62a). When the simulation completes, this data is written to the base workspace (Fig. 15.67). The `tout` array containing the time stamps is (always) created automatically. Now any plotting function can be applied to these two arrays.

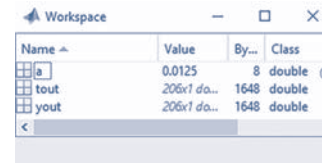


Fig. 15.67 Base workspace of MATLAB after running the model shown Fig. 15.66a.

Note, starting from R2019b the `tout` (automatically created) and `yout` (named in the To Workspace block) are stored as the fields of a `Simulink.SimulationOutput`-class structure `out` (this structure also has two other fields – `SimulationMetadata` and `ErrorMessage`). In this case, in the above code (appearing as the StopFcn callback of Fig. 15.66b), they should be replaced with `out.tout` and `out.yout`, respectively. (The base workspace shown in Fig. 15.67 would also show the `out` structure, rather than the `tout` and `yout` column vectors.)

15.6.4 Generating MATLAB Function Blocks from Symbolic Expressions

In addition to the aforementioned capabilities of MATLAB–Simulink interaction, MATLAB also allows you to create MATLAB Function blocks from symbolic expressions created using the Symbolic Math Toolbox functions. For example, the commands

```
>> new_system('SimulinkC6')
>> open_system('SimulinkC6')
```

create and open a new (empty) Simulink model. Now, you can develop symbolic expressions and convert them to the Simulink blocks using the `matlabFunctionBlock` function (which, prior to R2014b, was known as the `emlBlock` function) introduced in Table 7.8.

Consider a couple of commands creating a 10-term Taylor series expansion of $f = \sin(x)$ around $x = a$ and solving the second-order differential equation with ICs specified as $y(0) = a$ and $y'(0) = b$:

```
>> syms x a b
>> f=sin(x)
>> y=dsolve('D2y=cos(2*x)-y','y(0)=a','Dy(0)=b','x');
>> matlabFunctionBlock ('SimulinkC6/Taylor',taylor(f,x,a,'Order',10))
>> matlabFunctionBlock ('SimulinkC6/DiffEq',y)
```

The two `matlabFunctionBlock` calls create two MATLAB Function blocks, as shown in Fig. 15.68a. Figure 15.68b shows the content of the DiffEq block with vectorized optimized code in it. (When creating the

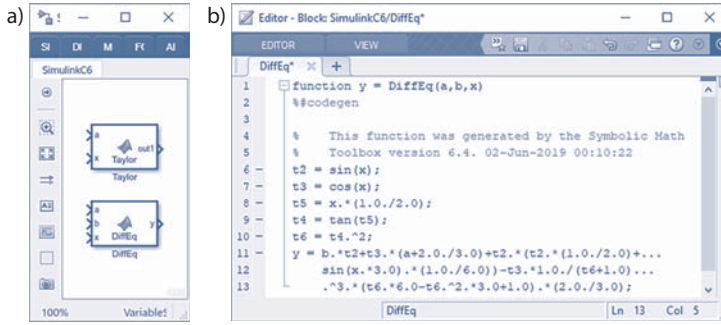


Fig. 15.68 a) Creating blocks from MATLAB and b) an example of an automatically generated function.

blocks, be careful with their names so that you do not override the definition of an existing block.)

You may add other blocks to these two to build your model in a usual fashion. For example, Fig. 15.69a features the two Constant, Clock, Display, and Scope blocks added to the *Simulink6.slx* model, which produces the output shown in Fig. 15.69b.

The Display block shows the value of the input data (the very last value in Fig. 15.69a). You can specify the format (see Table 2.10) and the frequency of display (decimation).

15.7 Practical Examples

This section presents an example of a more or less complex model, built in Simulink, with the goal of setting some trends to follow. To this end, Fig. 15.70 shows a root level of the model of the integrated inertial navigation system (INS) and Global Positioning System (GPS). As seen, the larger pieces of this model are combined into subsystems, so that the root level has only 10 superblocks (subsystems) and just a few simple blocks. Another point to make is that to reduce the number of explicit block connections, the

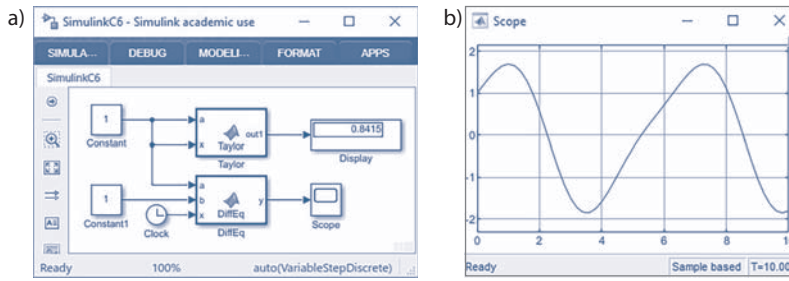


Fig. 15.69 a) Utilizing two blocks created in MATLAB and b) a Scope block output.

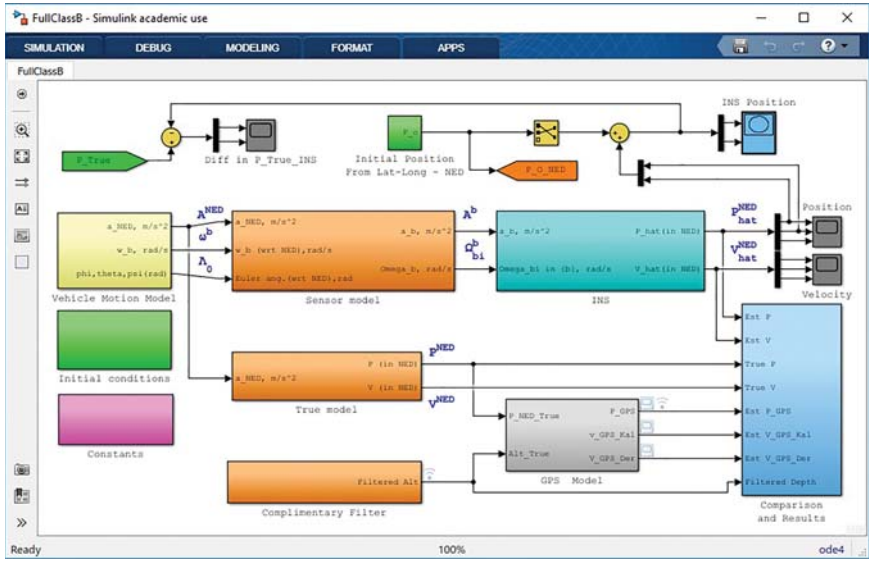


Fig. 15.70 Model of the integrated INS/GPS system.

Goto and From blocks from the Sinks (Fig. 15.4) and Sources (Fig. 15.3) libraries are employed.

For example, the Constants superblock (subsystem) in Fig. 15.70 seems to be not connected to anything. In fact, this block, featuring some constants (scalars and matrices) used in multiple blocks throughout the model, passes its values to these blocks using the Goto blocks as presented in Fig. 15.71a. When you connect the Goto blocks with other blocks, you define the tag for each specific signal (like `R_NED2ENU` in Fig. 15.71b). In another subsystem where this signal is used, for example, in the subsystem shown in

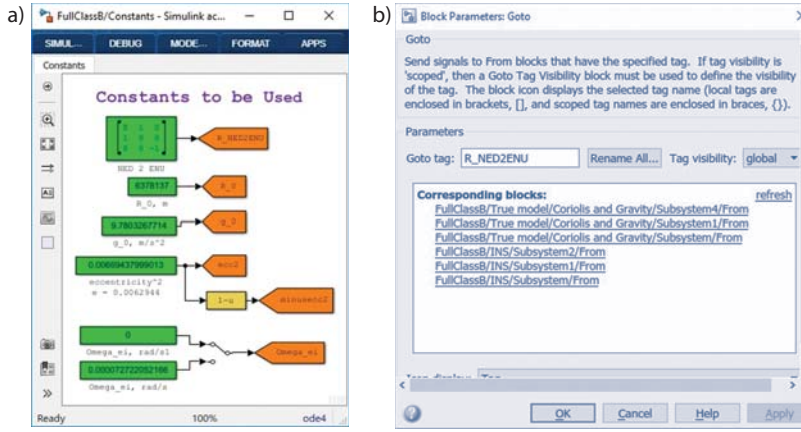


Fig. 15.71 Using a) the Goto block and b) its Parameters dialog box.

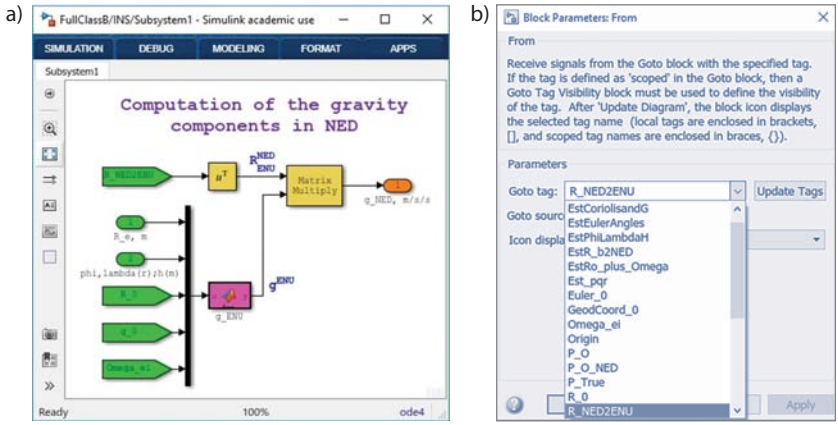


Fig. 15.72 Using a) the From block and b) its Parameters dialog box.

Fig. 15.72a, you introduce the From block and then find the tag you need from the **Goto tag** drop-down list (Fig. 15.72b). This block is then added to the Corresponding blocks list of the relevant GoTo block (Fig. 15.71b).

Returning to the Constants superblock of Fig. 15.70, in the models where you use the same constants in several places, it is worth defining them somewhere once and then passing their values to other blocks rather than creating Constant blocks with the same constants all over the place. In the case of the INS/GPS model of Fig. 15.70, where you have to use several values with a lot of significant digits—like Earth's radius, gravity, eccentricity, and rotational velocity (Fig. 15.71a)—a single typo causing a mismatch of the values in different blocks would prevent your system from working properly, and you could spend days and weeks before figuring it out (if at all).

In the model shown in Fig. 15.70, you may also have noticed three tiny Signal Viewers and two Streaming Badges above the signal lines entering and exiting the GPS Model superblock (at the lower right side of the model). These tools can be added to a selected signal line (test point) by choosing the corresponding options of the signal line context (right-click) menu, shown in Fig. 15.73a. Specifically, choosing the **Create & Connect Viewer => Simulink => Scope** adds a Signal Viewer. As opposed to the Scope block, you only need a single click to open it. Other than that, this tool looks exactly the same as a Scope block. To remove it from the signal line, right-click and choose the **Delete Viewer** option.

Choosing the **Log Selected Signals** option from the Signal line context menu (Fig. 15.73a) or left-clicking the signal line and choosing the **Enable Data Logging** option from the ellipsis menu (which is the second icon in the menu shown in Fig. 15.11c) adds the Streaming Badge, meaning that the selected signal will be sent to the base workspace (as the `Simulink.SimulationData.Dataset` class) and will be available in

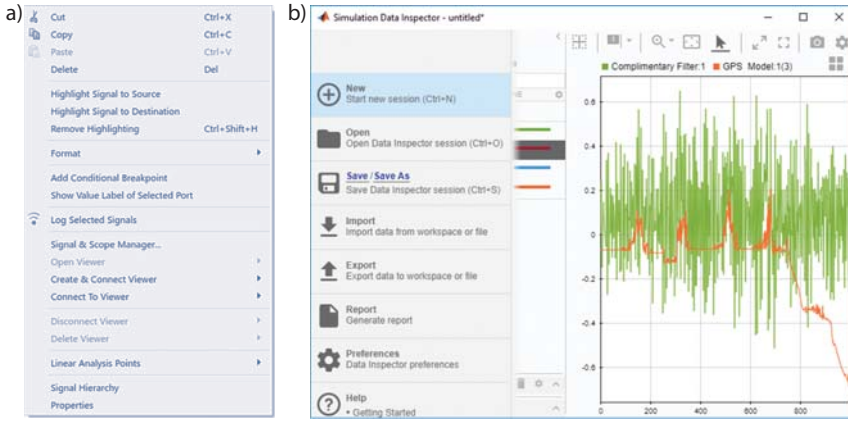


Fig. 15.73 a) Signal line context menu and b) Simulation Data Inspector window.

the Simulation Data Inspector. This latter tool (Fig. 15.73b), which can be opened by clicking any streaming badge or choosing the **Open Data Inspector** option of the Streaming Badge context menu, allows you to visualize, inspect, and compare (between the different runs) logged time series data at multiple stages of your workflow. Note that you can benefit from using this powerful tool for validation and iterating model designs in MATLAB as well. To do that, launch it from the command line

```
>> Simulink.sdi.view
```

and import the appropriate `timeseries`-type data from the base workspace or a file (like *mat-*, *xls-*, or *csv-*). To remove Streaming Badge, choose the Disable Data Logging option from the ellipsis menu.

Figure 15.74 gives one more example showing a subsystem modeling a four-GPS–satellite constellation. It demonstrates an example of how the

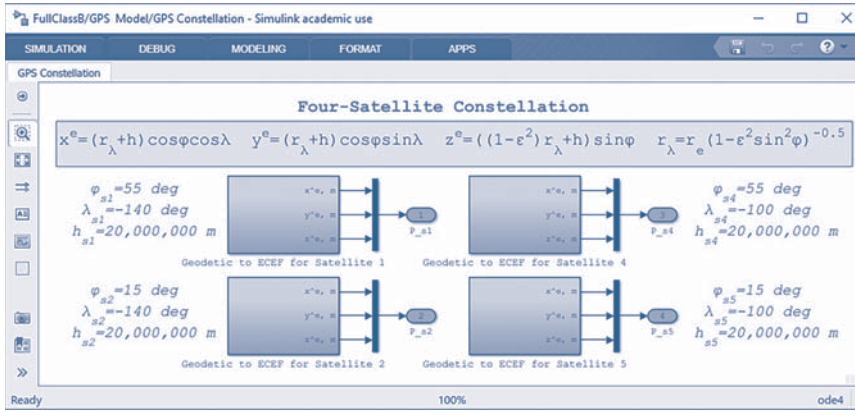


Fig. 15.74 Model of a four-GPS–satellite constellation.

very same subsystem, in this case performing some coordinate transformation, can be duplicated so that you do not need to develop this transformation for every satellite from scratch.

15.8 Model Reviewing and Profiling

As mentioned in Sec. 15.2, Simulink offers some additional tools, allowing you to review your model and analyze it. Without going deeply into it, let us briefly present a couple of these tools here.

Clicking the **Hide/Show Model Browser** button («/») at the bottom of the Simulink Editor palette (Fig. 15.2), supplements the Simulink Editor window with the Model Browser pane. For example, Fig. 15.75 shows the content of this pane for a multilayer GPS model. Browsing through the model allows you to conveniently access the content of its subsystems. Right-clicking any subsystem allows opening it in a new tab or new window.

As seen, the GPS constellation subsystem similar to that of Fig. 15.74 shows up in the Browser pane, whereas another subsystem, Disturbed measurements, is chosen to be viewed. This subsystem computes actual ranges from an observer to each of the five satellites, adding some bias and measurement noise.

It should be mentioned that using the Band-Limited White Noise block from the Sources library of Fig. 15.3 has one feature. One of the parameters that needs to be defined is **Seed** (Fig. 15.76a). The way the random numbers (noise) are generated is that they are represented by a long sequence (say 10^{13}) of randomly generated numbers, stored in computer memory. When you introduce one of the blocks relying on these numbers (Band-Limited

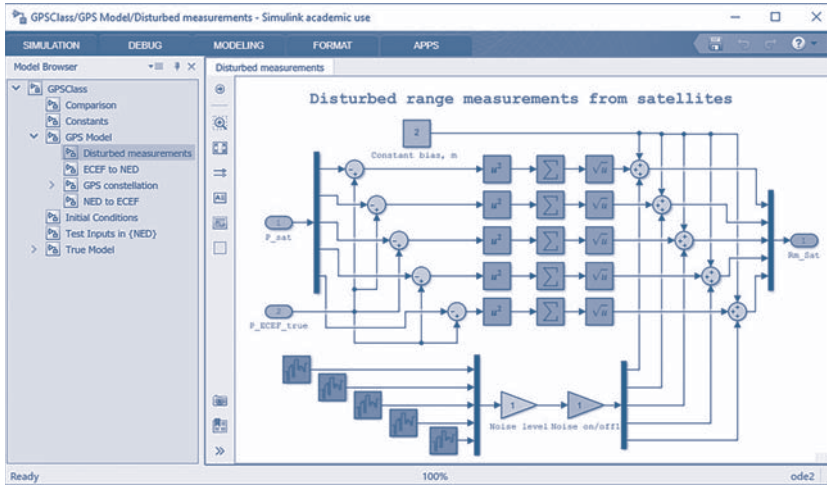


Fig. 15.75 Model Browser pane added to the Simulink Editor window.

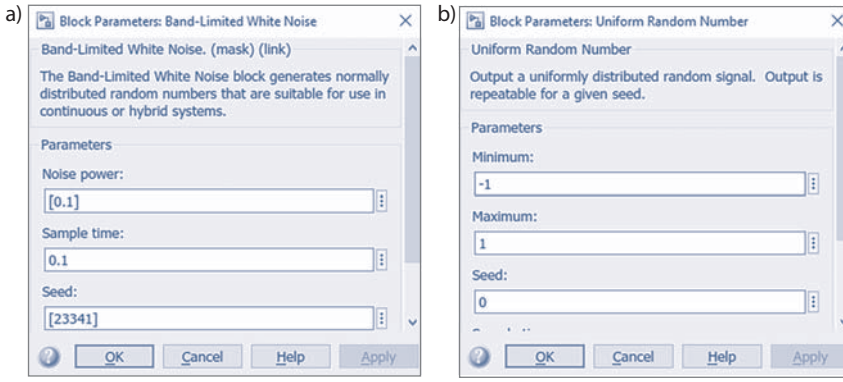


Fig. 15.76 Parameters dialog boxes for the a) Band-Limited White Noise and b) Uniform Random Number blocks.

White Noise, Random Number, and Uniform Random Number), you have to specify where to start your sequence from. Figures 15.76a and 15.76b feature the default seeds for two blocks (the Random Number Properties window is essentially the same as Uniform Random Number, along with the seed (0) defining the mean (0) and variance (1) for the Gaussian distribution).

If your model uses only one block relying on random numbers, the starting point does not matter, but when you use two or more blocks, you should define different seed values for each of them. To make the point, Fig. 15.77a presents a simple model, where the same two blocks feed the x- and y-axes of the XY Graph block of the Sinks library of Fig. 15.4. If the seeds for both the blocks are the same, they output exactly the same numbers, which results in a linear dependence between two sequences (Fig. 15.77b). Changing one of the seeds even by just one unit, changes everything, so that you seemed to have two independent sequences of random numbers (Fig. 15.77c).

The **Model Explorer** button of the Design icon group of the Modeling tab (Figs. 15.7b, 15.8b) opens the Model Explorer window (see example in

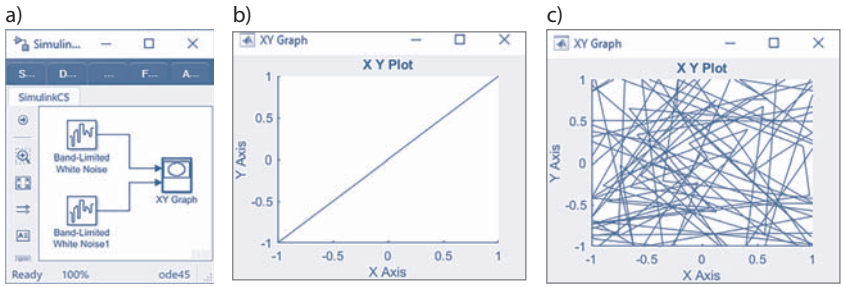


Fig. 15.77 a) Model employing two random number generators with b) the same two seeds and c) different seeds.

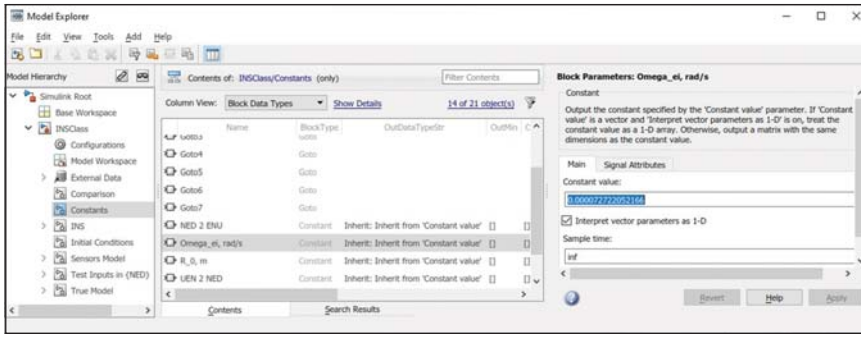


Fig. 15.78 The Model Explorer window.

Fig. 15.78). This tool shows the model hierarchy (in the left pane) as well, but it also allows you to find any specific block (in the middle pane) and access/change its parameters (in the right pane). For example, the Explorer shown in Fig. 15.78 features the Parameters dialog box of the Omega_ei Constant block of Constants subsystem, presented in Fig. 15.71a, enabling modification of its value.

Selecting the second option from the **Breakpoints List** drop-down menu of the Debug tab (Fig. 15.7a), **Debug Model,** opens the Simulink Debugger window, as shown in Fig. 15.79. Here, you can set the break/display points in your model and observe what happens when you run it. This way, you can pinpoint problems in your model to specific blocks, parameters, or interconnections. You can start the debugger from the MATLAB Command Window as well.

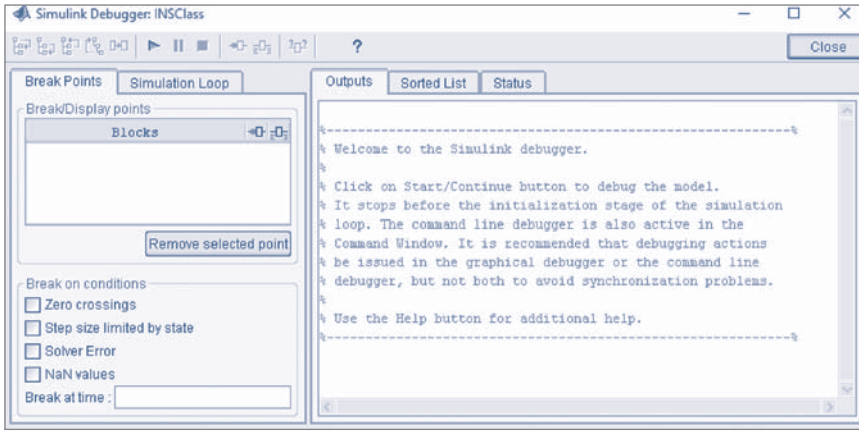


Fig. 15.79 Simulink Debugger window.

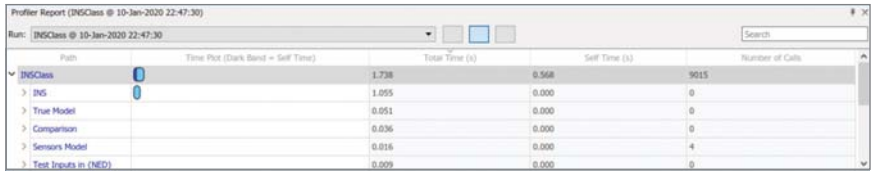


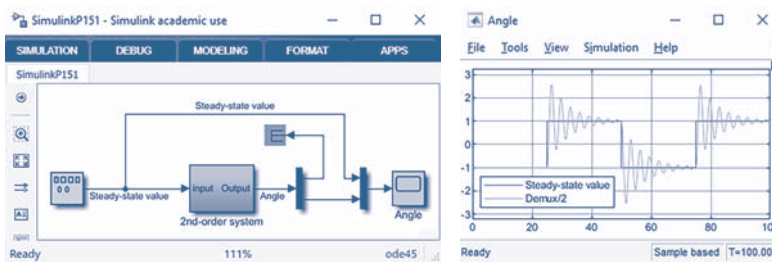
Fig. 15.80 Example of the Simulink Profiler report.

Last, Simulink allows you to capture performance data for your model by running the Solver Profiler and/or Simulink Profiler, which are the second and third options of the **Performance Advisor** drop-down menu of the Debug tab, (Fig. 15.7a). While the simulation runs, profilers are collecting data for the report. Once the simulation is complete, they create the report. An example of the Solver Profiler report was presented in Fig. 15.21a. An example of the Simulink Profiler report is shown in Fig. 15.80). (Note that the Simulink Profiler was updated in R2020a, so it looks different in the prior releases.)

Similar to the MATLAB Profiler (Fig. 5.26), the Simulink Profiler may help in identifying the parts of your model requiring the most time to simulate. You can then use this information to decide where to focus your model optimization efforts (if at all). Among other things, the report shows the total time required to simulate the model and individual superblocks and blocks (Total Time), self-execution time excluding time spent by subblocks (Self Time), and total number of invocations of block-level functions (Number of Calls). Clicking the name of the (super)block (in the leftmost column of the report) brings more information about the parent and child functions. The second column provides visualization of the third and fourth column data.

Problems

15.1 Use the *SimulinkC2a.slx* model created in Sec. 15.3.2 as a base to develop a more sophisticated model shown on the left:



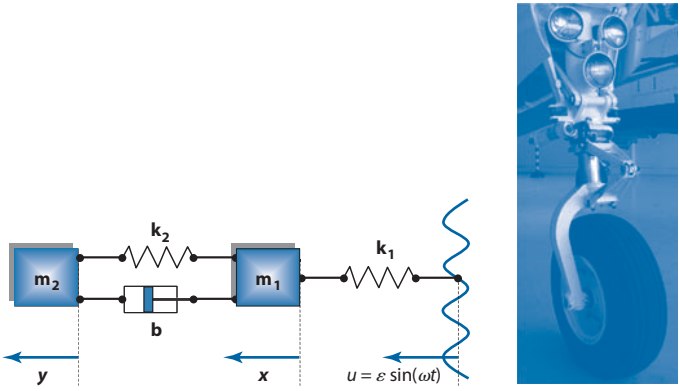
Use a Signal Generator block from the Sources library of Fig. 15.3 to feed your system with the square-form wave with an amplitude of 1 and frequency of 0.02 Hz. Choose the **Model Properties** option from the **Model Settings** drop-down menu (Fig. 15.13) followed by **Callbacks** and **InitFcn** to enter the numerical values for $\omega_n = 2$, $\zeta = 0.075$, and $x_0 = -1$ (similar to that of Fig. 15.46b). Check the correctness of your model by comparing its output with what is shown to the right of the model above.

15.2 Consider the following two-body mass–spring–damper system (which may describe, for instance, an aircraft gear), excited by some external input $u = \varepsilon \sin(\omega t)$ (uneven runway surface). Assume parameters of the system to be $m_1 = 1$ kg, $m_2 = 1000$ kg, $k_1 = 10$ kg/s^2, $k_2 = 2$ kg/s^2, and $c = 5$ kg/s. Do the following:

a) Develop a Simulink model for this mechanical system.

b) Check the output of a homogeneous part of the system for the case where $x(0) = 0$ m, $y(0) = 0$ m, $\dot{x}(0) = 1$ m/s, and $\dot{y}(0) = 0$ m/s.

c) Simulate system's behavior with zero ICs as a response to input u (pick the values of ε and ω that seem reasonable yourself).



15.3 Using the loan amortization model of Sec. 15.4.1, find how many months it would take to pay off a 3 million dollars loan with a 10% annual interest rate while making $3,000 monthly payments. Then, try to adjust the monthly payment to be able to pay it off in 40 months (as was the case in the example considered in the aforementioned section).

Appendix A LaTeX Symbols, Math Constructs, and Matrices

Section 6.5 showed how to take advantage of using the (default) TeX interpreter to convert specific character sequences prefaced by a special symbol, \, into special symbols and to vary text style (see Tables 6.11 and 6.12). As you know, MATLAB also allows you to use the LaTeX interpreter, enabling an even wider set of special symbols, math constructs, and matrices. Some examples of these constructs created using the `latex` function of the Symbolic Math Toolbox were shown in Sec. 7.9.1. For completeness, this appendix shows a complete list of the LaTeX commands available in MATLAB (this list is a subset of the LaTeX system).

A.1 Creating LaTeX Equations in Live Editor

The easiest way to visualize what additional capabilities the LaTeX interpreter of MATLAB offers is to click the **Insert equation** icon of the Live Editor's Insert tab, which opens the Equation tab shown in Fig. A.1.

Right-clicking the upside-down triangle to the right of the Symbols group brings up the menu presented in Fig. A.2. As seen, the list of symbols you can choose from in this menu is slightly broader compared with the one shown in Table 6.11 (it should be noted, however, that MATLAB does not necessarily support all of the LaTeX symbols). To discover the name of a symbol, you can hover over the corresponding icon.

Clicking the upside-down triangle to the right of the Structures group activates the menu shown in Fig. A.3. Obviously, these structures (hereinafter referred to as constructs) can be very helpful in enhancing not only your live scripts (and regular *m*-scripts), but graphics as well. Hovering over the corresponding icon shows the name of the construct.

Fig. A.1 The Equation tab of Live Editor.

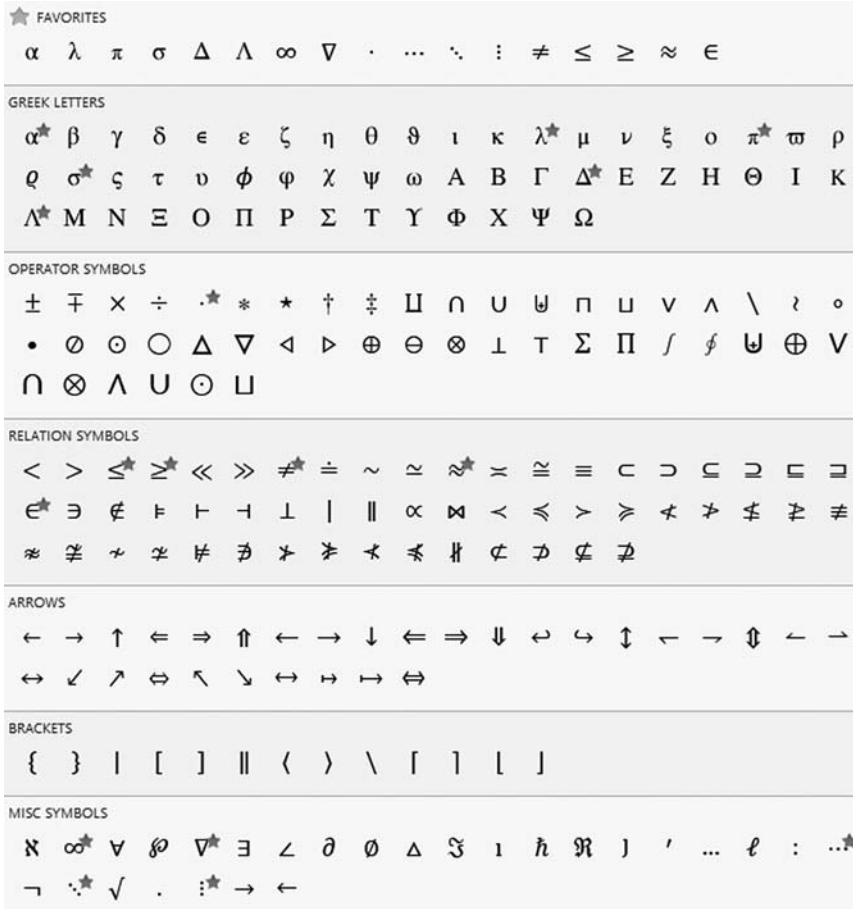


Fig. A.2 Drop-down menu for LaTeX symbols.

The **Matrices** icon of the Equation tab of Fig. A.1 allows you to create a matrix that can then be copied as a LaTeX expression and used anywhere else outside a live script (in a regular script).

Typing a backslash in the equation editor (when the **Insert equation** icon of the Live Editor's Insert tab is pushed) brings up a completion menu listing all of the supported LaTeX command names, as shown in Fig. A.4a. Choosing the **LaTeX Equation** option from the **Equation** drop-down menu of the Insert tab invokes the Edit Equation window, allowing you to create your LaTeX math construct interactively with a preview (Fig. A.4b).

The next three sections show how to use supported LaTeX symbols and constructs explicitly (by their names).

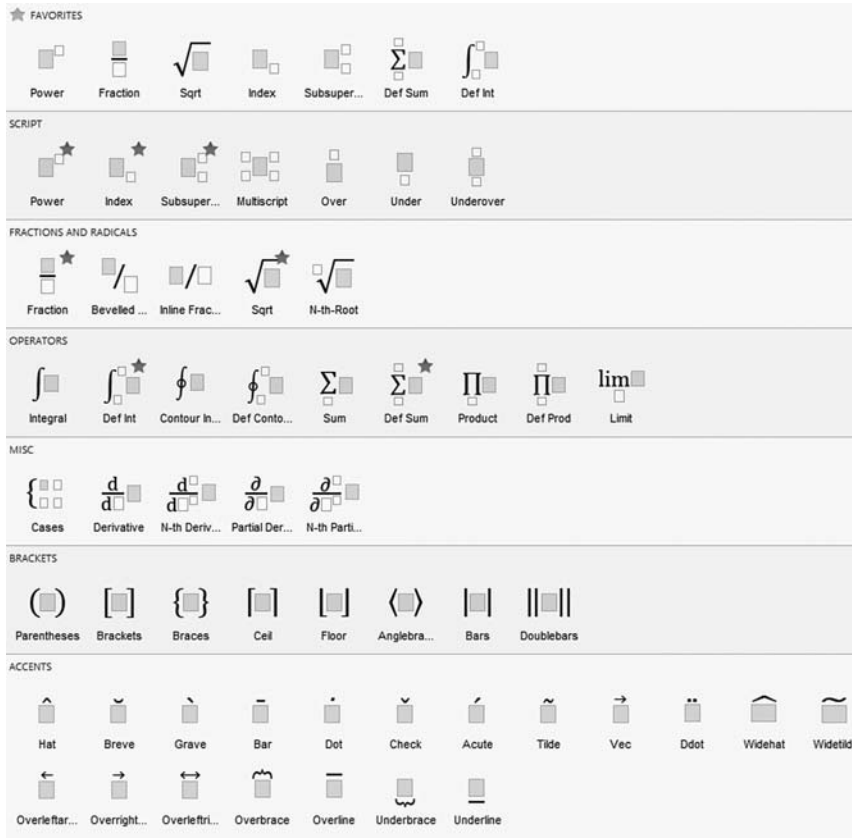



Fig. A.3 Drop-down menu for LaTeX constructs.

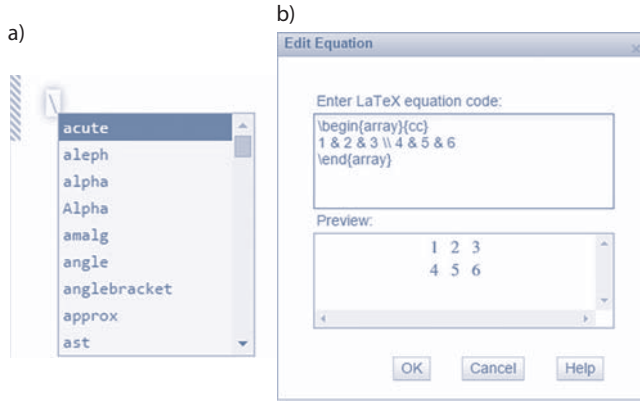


Fig. A.4 a) Completion menu and b) Edit Equation panel.

A.2 LaTeX Symbols

The LaTeX syntax for a variety of symbols available in MATLAB is shown by groups in Tables A.1–A.9.

Table A.1 Greek and Hebrew Alphabet Letters

α	\alpha	λ	\lambda	σ	\sigma	ϱ	\varrho	Φ	\Phi
β	\beta	μ	\mu	τ	\tau	ς	\varsigma	Π	\Pi
χ	\chi	ν	\nu	θ	\theta	ϑ	\vartheta	Ψ	\Psi
δ	\delta	o	\omicron	υ	\upsilon	$\aleph$	\aleph	Σ	\Sigma
ϵ	\epsilon	ω	\omega	ξ	\xi	Δ	\Delta	Θ	\Theta
η	\eta	ϕ	\phi	ζ	\zeta	Υ	\Upsilon	Ξ	\Xi
γ	\gamma	π	\pi	ε	\varepsilon	Γ	\Gamma		
ι	\iota	ψ	\psi	φ	\varphi	Λ	\Lambda		
κ	\kappa	ρ	\rho	ϖ	\varpi	Ω	\Omega		

Table A.2 Delimiters and Brackets

|	|	{	\{	$\lfloor$	\lfloor	/	/
|	\vert	}	\}	$\rfloor$	\rfloor	\	\backslash
||	\\|	$\langle$	\langle	$\lceil$	\lceil	[	[
||	\Vert	$\rangle$	\rangle	$\rceil$	\rceil	]	]

Table A.3 Variable-Sized Symbols (displayed formulas show a larger version)

$\sum$	\sum	$\int$	\int	$\uplus$	\biguplus	$\oplus$	\bigoplus	$\bigvee$	\bigvee
$\prod$	\Prod	$\oint$	\oint	$\cap$	\bigcap	$\otimes$	\bigotimes	$\bigwedge$	\bigwedge
$\coprod$	\coprod			$\cup$	\bigcup	$\odot$	\bigodot	$\sqcup$	\bigsqcup

Table A.4 Operator Symbols

*	\ast	$\pm$	\pm	$\cap$	\cap	$\curlywedge$	\curlywedge
$\star$	\star	$\mp$	\mp	$\cup$	\cup	$\curlyvee$	\curlyvee
$\cdot$	\cdot	$\amalg$	\amalg	$\uplus$	\uplus	$\bot$	\bot
$\circ$	\circ	$\odot$	\odot	$\sqcap$	\sqcap	$\top$	\top
$\bullet$	\bullet	$\ominus$	\ominus	$\sqcup$	\sqcup	$\doublebarwedge$	\doublebarwedge
$\bigcirc$	\bigcirc	$\oplus$	\oplus	$\wedge$	\wedge	$\triangleleft$	\triangleleft
$\diamond$	\diamond	$\oslash$	\oslash	$\vee$	\vee	$\triangleright$	\triangleright
$\times$	\times	$\otimes$	\otimes	$\dagger$	\dagger	$\bigtriangledown$	\bigtriangledown
$\div$	\div	$\wr$	\wr	$\ddagger$	\ddagger	$\bigtriangleup$	\bigtriangleup
$\cdot$	\centerdot			$\barwedge$	\barwedge	\	\setminus

Table A.5 Relation Symbols

$\equiv$	\equiv	$\leq$	\leq	$\succ$	\succ	|	\mid
$\cong$	\cong	$\prec$	\prec	$\succeq$	\succeq	$\|\|$	\parallel
$\neq$	\neq	$\preceq$	\preceq	$\gg$	\gg	$\bowtie$	\bowtie
$\sim$	\sim	$\ll$	\ll	$\supset$	\supset	$\bowtie$	\Join
$\simeq$	\simeq	$\subset$	\subset	$\supseteq$	\supseteq	$\ltimes$	\ltimes
$\approx$	\approx	$\subseteq$	\subseteq	$\sqsupseteq$	\sqsupseteq	$\rtimes$	\rtimes
$\asymp$	\asymp	$\sqsubseteq$	\sqsubseteq	$\vdash$	\vdash	$\smile$	\smile
$\doteq$	\doteq	$\dashv$	\dashv	$\ni$	\ni	$\frown$	\frown
$\propto$	\propto	$\in$	\in	$\ni$	\ni	$\notin$	\notin
$\models$	\models	$\geq$	\geq	$\perp$	\perp		

Table A.6 Arrow Symbols

$\leftarrow$	\leftarrow	$\longleftarrow$	\longleftarrow	$\uparrow$	\uparrow
$\Leftarrow$	\Leftarrow	$\Longleftarrow$	\Longleftarrow	$\Uparrow$	\Uparrow
$\rightarrow$	\rightarrow	$\longrightarrow$	\longrightarrow	$\downarrow$	\downarrow
$\Rightarrow$	\Rightarrow	$\Longrightarrow$	\Longrightarrow	$\Downarrow$	\Downarrow
$\leftrightarrow$	\leftrightarrow	$\longleftrightarrow$	\longleftrightarrow	$\updownarrow$	\updownarrow
$\Leftrightarrow$	\Leftrightarrow	$\Longleftrightarrow$	\Longleftrightarrow	$\Updownarrow$	\Updownarrow
$\mapsto$	\mapsto	$\longmapsto$	\longmapsto	$\nearrow$	\nearrow
$\hookleftarrow$	\hookleftarrow	$\hookrightarrow$	\hookrightarrow	$\searrow$	\searrow
$\leftharpoonup$	\leftharpoonup	$\rightharpoonup$	\rightharpoonup	$\swarrow$	\swarrow
$\leftharpoondown$	\leftharpoondown	$\rightharpoondown$	\rightharpoondown	$\nwarrow$	\nwarrow
$\rightleftharpoons$	\rightleftharpoons	$\leadsto$	\leadsto		

Table A.7 Miscellaneous Symbols

∞	\infty	$\ddots$	\ddots	ℓ	\ell
∇	\nabla	$\Im$	\Im	$\sharp$	\sharp
∂	\partial	$\Re$	\Re	$\flat$	\flat
$\clubsuit$	\clubsuit	$\forall$	\forall	$\natural$	\natural
$\diamondsuit$	\diamondsuit	$\exists$	\exists	$\hbar$	\hbar
$\heartsuit$	\heartsuit	$\nexists$	\not\exists	$'$	prime
$\spadesuit$	\spadesuit	$\emptyset$	\emptyset	$\surd$	\surd
$\cdots$	\cdots	$\varnothing$	varnothing	$\wp$	\wp
$\vdots$	\vdots	$\imath$	\imath	$\angle$	\angle
$\ldots$	\ldots	$\jmath$	\jmath	$\triangle$	\triangle

Table A.8 Accents

á	\acute{a}	ā	\bar{a}
ă	\breve{a}	ǎ	\check{a}
ä	\ddot{a}	ȧ	\dot{a}
à	\grave{a}	â	\hat{a}
ã	\tilde{a}	a⃗	\vec{a}
a̲	\b{a}	ạ	\d{a}

Table A.9 Special Symbols

ϕ	\o	Æ	\AE	Ø	\O	¶	\P	§	\S
æ	\ae	†	\dag	‡	\ddag	©	\copyright	£	\pounds
Å	\AA	ß	\ss	ı	\i	j	\j		

Table A.10 Spacing Commands

LaTeX Command	Description	Size
Direct Spacing Commands		
\quad	space equal to the current font size	(=18 mu)
\dquad	twice of \quad	(=36 mu)
\! or \negthinspace	negative space	−3/18 of a quad (=−3 mu)
\ or \thinspace	small space	3/18 of a quad (=3 mu)
\:	medium space	4/18 of a quad (=4 mu)
\;	large space	5/18 of a quad (=5 mu)
\enspace	half-em space (see Table A.11)	10/18 of a quad (=10 mu)
\mkern	manual space setting in mu units (see Table A.11)	variable (in mu units)
\kern	manual space setting in any but mu units (see Table A.11)	variable
Indirect Spacing Commands		
\mathbin	treats the argument as a binary operator (with extra spacing around it)	
\mathinner	forces the argument to appear "inside" other formulas (surrounded by additional space)	
\mathop	treats the argument as the "large operator" class (adds spacing on both sides)	
\mathord	treats the argument as the "ordinary" class for more compact representation	
\mathpunct	treats the argument as the "punctuation" class (adjusting spacing on the left and right side accordingly)	
\mathrel	treats the argument as the "binary relation" class (with extra spacing around it)	
\mathopen	treats the argument as the "opening" class	
\mathclose	treats the argument as the "closing" class	

Table A.11 LaTeX Units

Abbreviation	Value
pt	a point is approximately 1/72.27 inch, meaning about 0.0138 inch or 0.3515 mm (exactly, one point is defined as 1/864 of an American *printer's foot*, which is 249/250 of an English foot)
mm	millimeter
cm	centimeter
in	inch
ex	roughly the height of an "x" (lowercase) in the current font (it depends on the font used)
em	roughly the width of an "M" (uppercase) in the current font (it depends on the font used)
mu	math unit equal to 1/18 em, where em is taken from the math symbols family

You can change the size of brackets by utilizing the `\big`, `\Big`, `\bigg`, and `\Bigg` commands (in size-increasing order), for example, `\Big[` or `\bigg\}`. There are also `\bigl` and `\Bigl`, `\biggl` and `\Biggl`, `\bigr` and `\Bigr`, and `\biggr` and `\Biggr` commands for left and right brackets, respectively, but they produce essentially the same result (featuring a slight spacing difference only for the variable-sized symbols of Table A.3).

Note, the `\leq`, `\geq`, `\equiv`, `\approx`, `\cong`, `\sim`, `\simeq`, `\models`, `\ni`, `\succ`, `\succeq`, `\prec`, `\preceq`, `\parallel`, `\subset`, `\supset`, `\subseteq`, and `\supseteq` commands can be combined with the `\not` command to create the negated version of the symbol, for example, `\not\ni`.

MATLAB also supports a variety of LaTeX spacing commands, as shown in Table A.10. Utilizing these commands allows you to control spacing between symbols and therefore improve readability of mathematical expressions. The two major commands are `\quad` and `\qquad`. The `\kern` command is intended to define the spacing manually, using any LaTeX units shown in Table A.11. Several derivative commands also shown in Table A.10 allow you to automatically adjust the size of spacing based on content.

For example, publishing the following piece of code results in in the comment section appearing as shown in Fig. A.5 and the graphics shown in Fig. A.6:

```
%%
%% Examples of using LaTeX expressions in comments
%
% $a \negthinspace \eta \quad \| \quad a \enspace \eta
%                        \quad \| \quad a \mkern20mu \eta$
%
% $\forall x \not\in X, \quad \exists y \leq \epsilon$
```

```
%
% $$ \Bigg( \bigg( \Big( \big( ( \bigotimes ) \big) \Big) \bigg) \Bigg) $$
%
% $$ \lim_{x \to \infty} \exp(-x) = 0 $$
%
% $$\sum_{\scriptstyle 0<i<m \atop \scriptstyle 0<j<n} P(i,j)$$
%% Examples of using LaTeX expressions (spacing) in graphics
text(.05,.9,'$$ a \negthinspace \eta $$','Interpreter','latex');
text(.05,.8,'$$ a \eta $$','Interpreter','latex');
text(.05,.7,'$$ a \, \eta $$','Interpreter','latex');
text(.05,.6,'$$ a \; \eta $$','Interpreter','latex');
text(.05,.5,'$$ a \enspace \eta $$','Interpreter','latex');
text(.05,.4,'$$ a \quad \eta $$','Interpreter','latex');
text(.05,.3,'$$ a \qquad \eta $$','Interpreter','latex');
text(.05,.2,'$$ a \mkern 50mu \eta $$','Interpreter','latex');
text(.05,.1,'$$ a \kern 50pt \eta $$','Interpreter','latex');
% More examples using the math... commands
text(.37,.9,'$$ a\rm{op} b $$','Interpreter','latex');
text(.37,.8,'$$ a \mathbin{\rm{op}} b $$','Interpreter','latex');
    text(.38,.7,'$$ a=1, ..., z $$','Interpreter','latex');
    text(.38,.6,'$$ a=1, \mathinner{...}, z $$','Interpreter','latex');
    text(.38,.5,'$$ a=1, \Idots , z $$','Interpreter','latex');
text(.39,.4,'$$ \sum_{n = 1}^\infty a_n $$','Interpreter','latex');
text(.39,.28,'$$ \sum_{n\mathop = 1}^\infty a_n $$','Interpreter','latex');
    text(.4,.18,'$$ 1,234,567 $$','Interpreter','latex');
    text(.4,.1,'$$ 1\mathord{,}234\mathord{,}567 $$','Interpreter','latex');
    text(.77,.9,'$$ \dot{X}^{a^b} $$','Interpreter','latex');
    text(.77,.8,'$$ \mathord{{}\dot X^a}^b $$','Interpreter','latex');
text(.78,.7,'$$ g: X \rightarrow Y $$','Interpreter','latex');
text(.78,.6,'$$ g \mathpunct{:} X \rightarrow Y $$','Interpreter','latex');
    text(.79,.5,'$$ (u \frown v)_1 x $$','Interpreter','latex');
    text(.79,.4,'$$ \mathrel{(u \frown v)_1} x $$','Interpreter','latex');
text(.8,.3,'$$ a + < b > + c $$','Interpreter','latex');
text(.8,.2,'$$ a + \mathopen{< } b\mathclose{> } + c $$','Interpreter',...
                                                          'latex');
    text(.8,.1,'$$ sin(\phi) \tan(\Theta) $$','Interpreter','latex');
```

Note, the last line in the above script demonstrates how to use the special symbol \ to suppress the italic style in the name of the tangent function (compare appearance of the sin and tan functions in the lower-right corner of Fig. A.6).

Table A.12 shows the LaTeX commands for varying text styling. Note that using either the `\displaystyle` or `\textstyle` command produces the same-size text but results in a slightly different size of variable-sized symbols of Table A.3.

Contents

- Examples of using LaTeX expressions in comments
- Examples of using LaTeX expressions (spacings) in graphics

Examples of using LaTeX expressions in comments

$a\eta \quad \| \quad a\ \eta \quad \| \quad a \quad \eta$

$\forall x \notin X, \qquad \exists y \leq \varepsilon$

$$\lim_{x \to \infty} \exp(-x) = 0$$

$$\sum_{\substack{0<i<m \\ 0<j<n}} P(i,j)$$

Fig. A.5 Using the LaTeX commands in the comments portion of code for publishing.

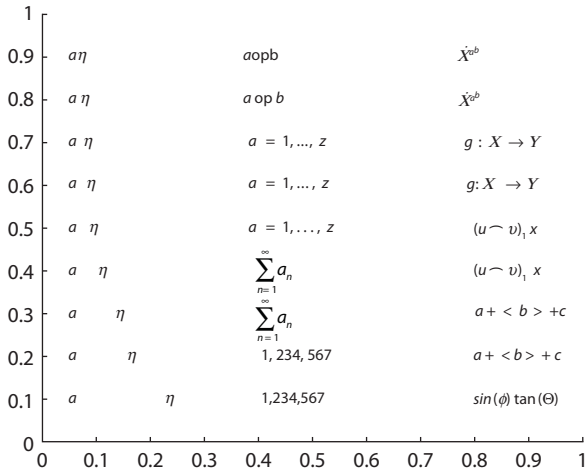


Fig. A.6 Different spacing options in graphics.

A.3 LaTeX Math Constructs

Beyond a wider choice of symbols, the LaTeX system allows you to create different math constructs, as shown in Table A.13.

Table A.12 Text Styling (font size)

$\displaystyle\int f^{-1}(x-x_a)\,dx$	`\displaystyle \int f ^{-1} (x - x_a)\, dx`
$\textstyle\int f^{-1}(x-x_a)\,dx$	`\textstyle \int f ^{-1} (x - x_a)\, dx`
$\scriptstyle\int f^{-1}(x-x_a)\,dx$	`\scriptstyle \int f ^{-1} (x - x_a)\, dx`
$\scriptscriptstyle\int f^{-1}(z-z_a)\,dz$	`\scriptscriptstyle \int f ^{-1} (x - x_a)\, dx`

Table A.13 Math Constructs

LaTeX Command	Description	LaTeX Command	Description
\sqrt{abc}	creates a square root symbol	\atop	creates fractions without horizontal fraction bar
\sqrt[n]{abc}	creates an n-th root symbol	\choose	draws notation commonly used for binomial coefficients
\overline{abc}	creates an overline symbol over the argument	\brace	creates a braced construct
\underline{abc}	creates an underline symbol over the argument	\brack	creates a bracketed construct
\overbrace{abc}	puts a (stretchy) over-brace over the argument	\frac{abc}{xyz}	draws fractions
\underbrace{abc}	puts a (stretchy) under-brace below the argument	\overwithdelims	acts as a general command for making fractions
\overrightarrow{abc}	creates an overrightarrow symbol over the argument	\buildrel ... \over ...	creates stacked binary relationship
\overleftarrow{abc}	creates an overleftarrow symbol over the argument	\stackrel	creates stacked relationship
\widehat{abc}	creates a widehat symbol	\limits	sets limits above/below any token
\widetilde{abc}	creates a widetilde symbol	\left ... \right	draws left and right symbols

Examples of these constructs produced with the following code are shown in Fig. A.7:

```
%% Examples of using LaTeX math constructs
% \sqrt, \over..., \under... and \wide... commands
text(.05,.9,'$$ \sqrt{abc} \qquad \sqrt[5]{abc} $$','Interpreter','l');
text(.05,.8,'$$ \overbrace{abc} \qquad \underbrace{abc}$$','Interpreter','l');
```

```
text(.3,.8,['$$ \underbrace{x + \cdots + x}_{n\rm\ times}^'...
                          '{\rm{(note here)}}$$'],'Interpreter','l');
text(.05,.7,'$$ \overline{abc} \qquad \underline{abc} $$','Interpreter','l');
text(.05,.6,'$$ \overrightarrow{abc} \qquad \overleftarrow{abc} $$',...
                                                    'Interpreter','l');
text(.05,.5,'$$ \widehat{abc} \qquad \widetilde{abc} $$','Interpreter','l');
% \choose, \brace and \brack commands
text(.05,.36,'$$ {a+1 \atop b+2}+c $$','Interpreter','l');
text(.23,.36,'$$ \displaystyle{n+1 \choose k+2} $$','Interpreter','l');
text(.05,.23,'$$ {a+b+c\brace d+e+f} $$','Interpreter','l');
text(.05,.1,'$$ a+{b+c\brack d+e}+f $$','Interpreter','l');
% \limits command
text(.55,.65,'$$ \int_a^b f(x)\,dx \qquad \int\limits_a^b f(x)\,dx $$',...
                                                    'Interpreter','l');
% \left ... \right commands
text(.4,.5,'$$ \left( \frac23 \right) $$','Interpreter','l');
text(.5,.5,'$$ \left( \sum_i a_i \right) \left( b+c \right) $$',...
                                        'Interpreter','l');
text(.75,.5,'$$ \Bigl( \sum_i a_i \Bigr) \Bigl( b+c \Bigr) $$',...
                                        'Interpreter','l');
% \buildrel ... \over and \stackrel commands
text(.43,.36,'$$ \buildrel \alpha\beta \over \longrightarrow $$',...
                                        'Interpreter','l');
text(.53,.36,'$$ \buildrel \rm def \over {:=} $$','Interpreter','l');
text(.6,.36,'$$ \stackrel{\alpha\beta}{\longrightarrow} $$',...
                                        'Interpreter','l');
text(.7,.36,'$$ \stackrel{\rm def}{=} $$','Interpreter','l');
text(.77,.36,'$$ [1,\stackrel{\times r}{\dots}{},n] $$','Interpreter','l');
text(.93,.36,'$$ \stackrel{... }{z}$$','Interpreter','l')
% \frac and \overwithdelims commands
text(.5,.23,'$$ \frac {s-1}{s^2+s+5} $$','Interpreter','l');
text(.8,.23,'$$ \left.\frac{x^3}{3}\right|_0^1 $$','Interpreter',...
                                                    'l'); % \left.
text(.5,.1,'$$ {s-1 \overwithdelims .| s^2+s+5}_{s=3} $$',...
                                'Interpreter','l'); % .|
text(.8,.1,'$$ {a \overwithdelims [] a^3-8} $$','Interpreter','l'); % []
text(.6,.9,['$$\sin(x)= \sum_{n=0}^{\infty}{\frac{(-1)^n x^{2n+1}}...
                                {(2n+1)'...'!}}$$'],'Interpreter','l')
```

One comment to make here is that as seen from Fig. A.7, delimiters of Table A.2 automatically change their size depending on the mathematical expression they surround (unless they are defined explicitly using the `\big...\Bigg` family of commands). When using the `\left` and `\right` commands, one of the delimiters (on either side of mathematical expression) can be made invisible by using a period, like `\left.` or `.|`.

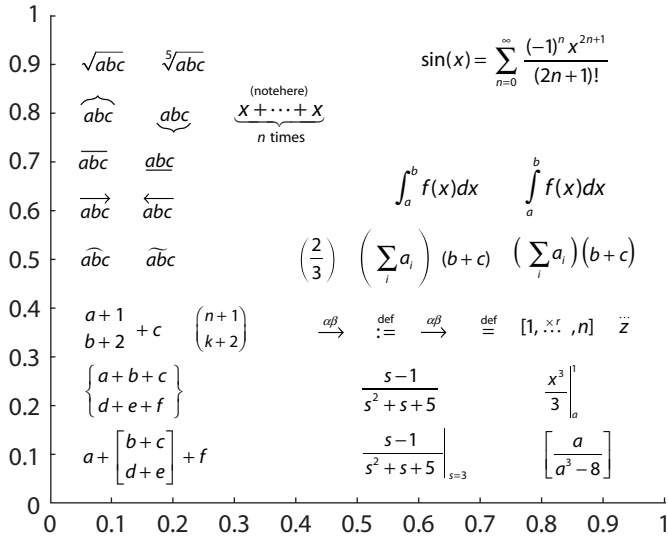


Fig. A.7 Illustration of LaTeX constructs available in MATLAB.

A.4 LaTeX Matrices and Cases

Finally, LaTeX also enables multiline constructions, other than fractions, including matrices and cases. The three available commands to construct a matrix are shown in Table A.14. In addition, the `\hline` command allows you to draw a horizontal line if needed.

The following script produces several examples shown in Fig. A.8:

```
%% Examples of using LaTeX to visualize matrices and cases
% \matrix and \pmatrix commands
text(.05,.9,'$$ \matrix{a & b \cr c & d} $$','Interpreter','l')
text(.18,.9,'$$ \pmatrix{1 & s \cr w & 8} $$','Interpreter','l')
text(.36,.9,'$$ \left[ \matrix{s^2 & 2 \cr 55 & w^2} \right] $$',...
                                        'Interpreter','l')
text(.54,.9,'$$ \left. \matrix{s^2 & 2 \cr 55 & w^2} \right| $$',...
                                        'Interpreter','l')
text(.72,.9,['$$ \left\| \matrix{\cos(\phi) & \sin(\phi) \cr' ...
```

Table 1.14 Commands for Visualizing Matrices and Cases

LaTeX Command	Sample	LaTeX Command	Syntax
\matrix	\matrix{1 & s \cr w & 8}	\begin{array}	\begin{array}{cols} row1 \\ row2 \\ . . . rowm \end{array}
\pmatrix	\pmatrix{a & b \cr c & d}		

```
                ' \sin(\phi) & \cos(\phi)} \right\| $$'], 'Interpreter','l')
text(.05,.7,['$$' ...
    'A_{m,n} = \pmatrix{' ...
    'a_{11} & a_{12} & \ldots & a_{1n} \cr' ...
    'a_{21} & a_{22} & \ldots & a_{2n} \cr' ...
    '\vdots & \vdots & \ddots & \vdots \cr'...
     'a_{m1} & a_{m2} & \ldots & a_{mn} }' ...
     '$$'],'Interpreter','latex','fontsize',12)
% \begin ... \end construction to create matrices
text(.6,.7,['$$ B = \left\{' ...
     ' \begin{array}{ccc|c}' ... % center-center-center-| -center
     '  1 & \frac12 & \frac13 & \frac13 \\' ...
     ' \frac12 & \frac13 & \frac14 & \frac14 \\' ...
     ' \hline ' ...
     ' \frac13 & \frac14 & \frac15 & \frac15' ...
     '\end{array} \right\} $$'],'Interpreter','latex','fontsize',12)
text(.15,.47,['$$ \left( \begin{array}{cc} ' ... % center-center
     '2\tau & 7\phi-\frac{5}{12} \\ ' ...
     '3\psi & \frac{\pi}8 \end{array} \right) ' ...
     '\left( \begin{array}{c} x \\ y \end{array} \right) ' ...
     '\; \rm and \; \left[ \begin{array}{c|cr} 3 & 4 & 5 \\ ' ...
     '1 & 3 & 729 \end{array} \right] $$'],'Interpreter','l','fontsize',11)
% \begin ... \end construction to create cases
text(.05,.27,['$$ f(z) = \left\{ \begin{array}{lcr}' ... % left-center-right
     ' \overline{\overline{z^2}+\cos z}, & \rm for & |z|<3 \\' ...
     ' 0, & \rm for & 3 \leq | z| \leq 5 \\' ...
     ' \sin\overline{z}, & \rm for & |z|>5 ' ...
     '\end{array} \right. $$'],'Interpreter','latex','fontsize',12)
text(.05,.1,['$$ F(x) = \left\{ \begin{array} {ll} {p''}_0\left(t \right)'...
'= -\lambda {p_0}\left(t \right)+\mu {p_1}\left(t \right) & \\ {p''}_j\'...
'left(t \right) = \lambda {p_{j-1}}\left(t \right) - \left( {\lambda + '...
'j\mu } \right){p_j}\left( t \right) + \mu \left({j+1} \right){p_{j + 1}}'...
'\left(t \right) & \end{array} \right. $$'],'Interpreter','latex',...
'fontsize',11)
```

As seen, whereas the `\pmatrix` command is used to create a matrix enclosed in parentheses, the `\matrix` command creates a matrix without parentheses (which can always be enclosed in parentheses, square and curly brackets, etc. later). The ampersand symbol, `&`, is used to separate columns, and `\cr` -to separate rows. Alignment occurs at the ampersands.

More sophisticated matrices are constructed using the `\begin {array} ... \end{array}` commands used to begin and end the specific environment (an array in our case). In this case, the rows are separated using `\\`, and each column can be aligned individually using the `{cols}` combination (see Table A.14). Specifically, `cols` includes one character (`l`, `r`, or `c` for to the left, center, or right alignment, respectively) for each column (with optional characters | inserted for vertical lines).

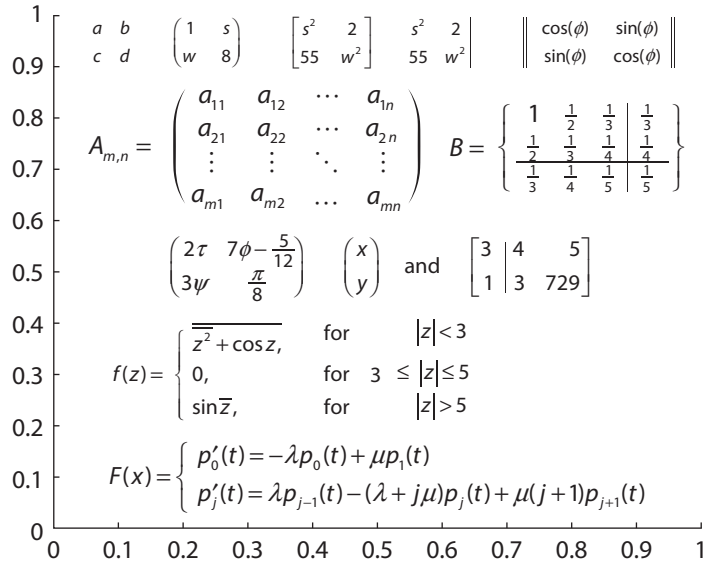


Fig. A.8 Matrices and cases.

As seen from Fig. A.8, the cases for representing piecewise functions can be created using the same `\begin{array} ... \end{array}` construction by simply suppressing the right curly bracket.

To conclude this appendix, it should be noted that not all LaTeX commands that are supported when creating LaTeX equations in Live Editor using the Edit Equation panel can be used in MATLAB scripts. That includes `\overset` and `\underset`, `\overwithdelims` and `\overeleftrightarrow`, and also the `\begin{cases} ... \end{cases}` commands.

Appendix B Functions for Handling Character and String Arrays

Tables B.1–B.6 of this appendix list the functions useful for manipulating the character and string arrays introduced in Sec. 3.6.

Table B.1 Functions for Creating and Concatenating Character and String Arrays

Function	Brief Description
Character Arrays	
`char(A)`	Converts the input array `A` into a character array
`cellstr(A)`	Converts `A` to a cell array of character vectors (cell arrays are introduced in Section 4.3)
`blanks(n)`	Creates a character array of `n` space characters (blanks)
`newline(n)`	Creates a creates a newline character
String Arrays	
`string(A)`	Converts the numeric, character, or string array `A` into a string array or cell array of character vectors
`strings`	Returns a string with no characters
`join(S)`	Combines the text in `S` by joining consecutive elements of the input array
Character or String Arrays	
`compose(fmt,A)`	Converts the numeric, character, or string array `A` into a string array according to the format specification `fmt` (see Sec. 4.9)
`sprintf(fmt,A1,...,An)`	Formats data in arrays `A1,...,An` (in the column order) into a character vector or string scalar according to the format specification `fmt`
`strcat(t1,t2,t3,...)`	Concatenates character or string arrays (or cell arrays of character vectors) `t1, t2, t3,...` horizontally, removing trailing ASCII whitespace characters
`append(t1,t2,t3,...)`	Combines the text from `t1, t2, t3,...`
`strvcat(t1,t2,t3,...)`	Concatenates character arrays vertically (ignoring the empty strings)
Convert Input Arguments	
`convertCharsToStrings(A)`	Converts a character array `A` to a string array, leaving other arrays unaltered
`convertStringsToChars(S)`	Converts a string array `S` to a cell array of character vectors, leaving other arrays unaltered

Table B.2 Determining Type and Properties of an Array

Function	Brief Description
Data Type	
`ischar(A)`	Returns logical 1 (true) if `A` is a character array and logical 0 (false) otherwise (logical values are introduced in Sec. 4.2)
`iscellstr(A)`	Returns 1 if `A` is a cell array of character vectors (or an empty cell array), and 0 otherwise
`isstring(A)`	Returns 1 if `A` is a string array, and 0 otherwise
`isStringScalar(A)`	Returns 1 if `A` is a string scalar, and 0 otherwise
Text Properties	
`strlength(S)`	Finds the number of characters in each element of the string array `S`
`isstrprop(A,ctg)`	Determines if elements of input array `A` are of the specified category `ctg`, such as letters, numbers, or whitespace
`isletter(A)`	Returns a logical array with 1's where the corresponding characters in `A` are letters, and 0's otherwise
`isspace(A)`	Returns a logical array with 1's where the corresponding characters in `A` are space characters, and 0's otherwise

Table B.3 Find and Replace Functions

Function	Brief Description
Find	
`contains(S,ptn)`	Determines if character pattern `ptn` is in string `S`
`count(S,ptn)`	Counts occurrences of character pattern `ptn` in string `S`
`endsWith(S,ptn)`	Returns logical 1 if `S` ends with the specified character pattern `ptn`, and 0 otherwise
`startsWith(S,ptn)`	Returns logical 1 if `S` starts with the specified character pattern `ptn`, and 0 otherwise
`strfind(S,ptn)`	Finds one string within another, i.e., searches `S` for occurrences of character pattern specified by `ptn`
`sscanf(S,fmt)`	Reads formatted data, as defined by `fmt`, from the string `S` (see Sec. 4.9)
Replace	
`replace(S,old,new)`	Replaces all occurrences of `old` in string array `S` with `new`
`replaceBetween(S,s0,s1,new)`	Replaces a substring in `S` identified by indicators `s0` and `s1` with `new`
`strrep(S,old,new)`	Replaces all occurrences of `old` within the string `S` with `new`

Table B.4 Join and Split Functions

Function	Brief Description
`join(S)`	Combines the text in a string array `S` by joining consecutive elements, placing a (space) character between them
`split(S)`	Divides text in a string array `S` at (whitespace) characters
`splitlines(S)`	Splits `S` at newline characters
`strjoin(C)`	Joins strings in a cell array `C` (introduced in Sec. 4.3) into a single string
`strsplit(S,dlm)`	Splits the string `S` at specified delimiter `dlm` (similar to `split`)
`strtok(S,dlm)`	Finds a part of the string `S` enclosed by the specified delimiter `dlm`

Table B.5 Editing Functions

Function	Brief Description
`erase(A,mtch)`	Deletes all occurrences of `mtch` in `A` (being a string/character vector or cell array of character vectors)
`eraseBetween(S,s0,s1)`	Deletes a substring in `S` between `s0` and `s1`
`extractAfter(S,s1)`	Extracts a substring from `S` that begins after `s1`
`extractBefore(S,s0)`	Extracts a substring from `S` that ends before `s0`
`extractBetween(S,s0,s1)`	Extracts a substring in `S` between `s0`and `s1`
`insertAfter(S,s1,nS)`	Inserts `nS` into `S` after the substring specified by `s1`
`insertBefore(S,s0,nS)`	Inserts `nS` into `S` before the substring specified by `S0`
`pad(S,lng,sd,pc)`	Adds characters `pc` to `S` up to the total length `lng` to the side of the string specified by `sd`
`strip(S)`	Removes leading and trailing (space) characters from string `S`
`lower(S)`	Converts the string `S` to lowercase
`upper(S)`	Converts the string `S` to uppercase
`reverse(S)`	Reverses the order of the characters of each element of `S`
`deblank(S)`	Removes all trailing whitespace and null characters from the end of the character string `S`
`strtrim(S)`	Removes leading and trailing whitespace from string array or character array `S`
`strjust(S)`	Returns a right-justified version of the character array `S` (for center-justified, use an additional input argument `'centered'`)

Table B.6 Compare and Match Functions

Function	Brief Description
Compare	
`matches(S,ptn)`	Returns `1` (true) if `S` equals character pattern specified by `ptn`, and returns `0` (false) otherwise (`S` can be a string array, character vector, or cell array of character vectors). Adding a `'IgnoreCase',true` name–value pair argument forces to ignore case when comparing
`strcmp(S1,S2)`	Compares two strings, `S1` and `S2`, and returns 1 (true) if the two are identical (0 otherwise)
`strcmpi(S1,S2)`	The same as `strcmp`, but case insensitive
`strncmp(S1,S2,n)`	Compares the first `n` characters of the strings `S1` and `S2`
`strncmpi(S1,S2,n)`	The same as `strncmp`, but case insensitive
Match Regular Expressions	
`regexp(S,ptn)`	Returns the starting index of each substring of `S` that matches the character pattern `ptn`
`regexpi(S,ptn)`	The same as `regexp`, but case insensitive
`regexprep(S,expr,rep)`	Replaces a text in `S` that matches `expr` with the text described by `rep`
`regexptranslate(type,S)`	Translates string `S` into a regular expression string that you can then use as input into one of the regular expression functions such as `regexp`. The `type` input can be either `'escape'` or `'wildcard'` that defines the type of translation to be performed

Appendix C Datetime and Categorical Data Types in Support of Tables and Timetables

As mentioned in Sec. 4.2, since R2013b several new data types have been introduced. That includes the `datetime` and `categorical` data types. While having value on their own, these data types have contributed substantially to simplifying handling table data. Moreover, the introduction of the `datetime` data type led to the creation of a special type of the `table` class, `timetable`. This latter class and a few functions accompanying it have proven to be very effective in handling (visualizing, analyzing, understanding, cleaning, sorting, and aggregating) time-stamped data, for example, data coming from multiple heterogeneous data sources. This appendix aims at helping you understand the potential of these new data types and shows examples of using them in conjunction with manipulating `table` and `timetable` data.

C.1 `datetime` (`duration`, `calendarDuration`) Arrays

Prior to R2014b, date and time data were stored in one of three formats:

- *Date string*, a character vector, for example, `'Thursday, December 31, 2020 11:45:44.123 PM'`
- *Date vector*, a 1-by-6 numeric vector containing the year, month, day, hour, minute, and second, for example, `[2020 12 31 23 45 44.123]`
- *Serial date number*, a single number equal to the number of days since January 0, 0000, in the proleptic Gregorian calendar (calendar produced backward to dates preceding its official introduction in 1582), for example, `7.3816e+05`

Table C.1 shows the `datestr`, `datevec`, and `datenum` functions developed to convert from one of these formats to another.

As of the R2014b release, the primary way to store date and time information is to use the `datetime` data type arrays. This data type has flexible display formats and up to nanosecond precision; can account for time zones, daylight saving time (DST), and leap seconds; and supports arithmetic, sorting, comparisons, and plotting. Many functions in MATLAB operate on datetime arrays in the same manner that they operate on other arrays.

Table C.1 Conversion between Date and Time Storage Formats

	Date String	Date Vector	Serial Date Number	Datetime Scalar
Date String		`datevec(DateStr)`	`datenum(DateStr)`	`datetime(DateStr)`
Date Vector	`datestr(DateVec)`		`datenum(DateVec)`	`datetime(DateVec)`
Serial Date Number	`datestr(DateNum)`	`datevec(DateNum)`		`datetime`[a]

Datetime Scalar	`char(DateTime)`
	`cellstr(DateTime)`
	`string(DateTime)`
		`datevec(DateTime)`	`datenum(DateTime)`	

[a] `datetime(datestr(DateNum))` or `datetime(datevec(DateNum))` or `datetime(DateNum,'ConvertFrom','datenum')`.

Table C.1 shows how to convert between the three former ways of representing points in time and the new way in the last row/column.

Table C.2 shows available syntaxes for creating datetime arrays using the `datetime` function. For example, the following command with the input arguments specifying the year, month, day, hour, minute, and second components:

```
>> t=datetime(2022,6,22,0:23,0,0);
```

creates a 1-by-24 datetime vector

```
>> disp([t(1),t(end)])
    22-Jun-2022 00:00:00    22-Jun-2022 23:00:00
```

Table C.2 Syntaxes for Creating a `datetime` Array

Syntax	Brief Description
`t=datetime` `t=datetime(relativeDay)`	Returns a scalar datetime array corresponding to the current date and time. The `relativeDay` input can be `'today'`, `'tomorrow'`, `'yesterday'`, or `'now'`
`t=datetime(DateStrings)` `t=datetime(DateStrings,'InputFormat',fmt)`	Creates an array of datetime values from the text in `DateStrings` representing points in time. Adding two additional arguments allows interpreting `DateStrings` using the format specified by `fmt`. Some other name–value pair arguments are also available
`t=datetime(DateVectors)`	Creates a column vector of datetime values from the date vectors in `DateVectors`

`t=datetime(Y,M,D)`
`t=datetime(Y,M,D,H,MI,S)`
`t=datetime(Y,M,D,H,MI,S,MS)`
	Creates an array of datetime values for corresponding elements of the `Y` (year), `M` (month), and `D` (day) (can also be specified as a date vector, `[Y M D]`) and optionally as `H` (hour), `MI` (minute), `S` (second), and even `MS` (millisecond) arrays

`t=datetime(X,'ConvertFrom',dateType))`	Converts the numeric values in `X` to a datetime array `t` specifying the type of values in `X` (Excel date number, Julian date, and others)
`t=datetime(...,name,value)`	Specifies additional options using one or more `name-value` pair arguments, in addition to any of the input arguments in the previous syntaxes. The `name` property is `'Format'`, `'Locale'`, `'Pivotyear'`, or `'Timezone'`

When creating datetime arrays you can use all the common approaches introduced in Sec. 3.2.3. For example, you can use the default step size of one calendar day to create a 1-by-31 datetime array `Ta`, spreading from June 22 through July 22:

```
>> t1=datetime(2022,6,22,0,0,0);   t2=datetime('2022-07-22');
>> Ta=t1:1:t2;   disp(Ta(1:2))
   22-Jun-2022 00:00:00   23-Jun-2022 00:00:00
```

You can also use the `linspace` function, for example,

```
>> Tb=linspace(t1,t2,60);  disp(Tb(1:2))
   22-Jun-2022 00:00:00   22-Jun-2022 12:12:12
```

Alternatively, you can specify the step size explicitly, like

```
>> Tc=t1:caldays(3):t2;  disp(Tc(1:2))
   22-Jun-2022 00:00:00   25-Jun-2022 00:00:00
```

which creates a 1-by-11 datetime array `Tc` with a three-day increment, or

```
>> Td=Tc(1):hours(4):Tc(2);  disp(Td(1:2))
   22-Jun-2022 00:00:00   22-Jun-2022 04:00:00
```

which creates a 1-by-19 datetime array `Td` with a four-hour increment.

In the two last calls, to specify an increment, we used the functions from Table C.3. These functions allow you to determine the length of time or duration using `duration` and `calendarDuration` data types (see Fig. 4.1). For example,

```
>> d1=duration(32,45,7)
```

returns a scalar duration of 32 hours, 45 minutes, and 7 seconds

```
d1 = 32:45:07
>> d2=0:seconds(15):minutes(1); disp(d2)
```

yields a 1-by-5 duration array

```
   0 sec   15 sec   30 sec   45 sec   60 sec
```

and

```
>> cd=calendarDuration(3,5,13)
```

returns a calendarDuration scalar of 3 years, 5 months, and 13 days:

```
cd = 3y 5mo 13d
```

The difference between these two data types is that whereas `duration` computes the values in *units of fixed length*, so that 1 day always equals 24 hours and 1 year always equals 365.2425 days, `calendarDuration` operates in *units of variable length*, so that 1 day might be more or less than 24 hours (because of DST changes), 1 month can be 28, 29, 30, or

Table C.3 Functions to Create `duration` and `calendarDuration` Arrays

Function	Brief Description
Create Duration Arrays	
`Z=years(X)`	Returns an array of years equivalent to the values in `X`. `Z` is a duration array in units of fixed-length years (365.2425 days) if `X` is a numeric array, and a numeric array if `X` is a duration array
`Z=days(X)`	Returns an array of fixed-length (24-hour) days equivalent to the values in `X`. Similar to `years`, works both ways (numeric array <=> duration array) as well

`Z=hours(X)`
`Z=minutes(X)`
`Z=seconds(X)`
`Z=milliseconds(X)`
	Returns an array of hours (minutes, seconds, milliseconds) equivalent to the values in `X` (given as a numeric or duration array). Similar to `years` and `days`, works both ways (numeric array <=> duration array)

`Z=duration(H,MI,S,MS)`	Creates a duration array from numeric arrays containing the number of hours, minutes, seconds, and milliseconds specified by `H`, `MI`, `S`, and `MS`, respectively (`MI`, `S`, and `MS` are optional)
Create calendarDuration Arrays	
`Z=calyears(X)`	Returns an array representing calendar years (accounting for leap days) equivalent to the values in array `X`. `Z` is a calendarDuration array if `X` is a numeric array, and a numeric array if `X` is a calendarDuration array

`Z=calquarters(X)`
`Z=calmonths(X)`
`Z=calweeks(X)`
`Z=caldays(X)`
	Returns an array representing calendar quarters (months, weeks, days) equivalent to the values in array `X`. Similar to `calyears`, works both ways (numeric array <=> duration array)

`Z=calendarDuration(Y,M,D)`	Creates an array of calendar durations from numeric arrays containing the number of years, months, and days specified by `Y`, `M`, and `D`, respectively (alternatively, you can specify the input argument as a matrix with three columns, `[Y,M,D]`)

31 days long, and 1 year can have 365 or 366 days (because of leap years). To better understand the difference between the `duration` and `calendarDuration` data types, consider a simple example (see Fig. C.1):

```
>> disp(datetime('2021-2-3 10:45 pm')+duration(24*30,0,0))
>> disp(datetime('2021-2-3 10:45 pm')+calendarDuration(0,1,0))
    05-Mar-2021 22:45:00
    03-Mar-2021 22:45:00
```

While the first call displays the specified date in February incremented by 30 days, the second call yields the same calendar date of the next month (the last call is equivalent to `disp(datetime('2021-2-3 10:45 pm')+30)`).

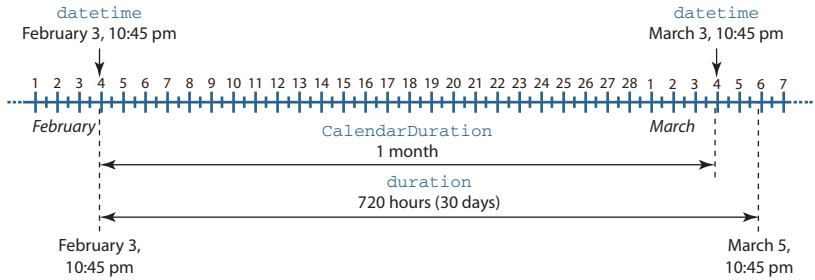


Fig. C.1 Visualization of the `datetime`, `duration`, and `calendarDuration` data types.

When manipulating with the `datetime` data type, the results of arithmetic differences are returned in duration arrays unless you use the calendar-based functions. For example, the command

```
>> dt=t(end)-t(1)
```

applied to the datetime vector `t`, created in the beginning of this section, returns a duration scalar

```
dt = 23:00:00
```

Applying the `caldiff` function, presented in Table C.4, to the elements of the `Tb` vector, created earlier,

```
>> dtc=caldiff(Tb(1:2))
```

yields a calendarDuration scalar

```
dtc = 12h 12m 12.203s
```

So far, we have used the functions of Table C.3 to create duration and calendarDuration arrays from numeric values. These functions also work the other way around. For example, the call

```
>> minutes(dt)
```

converts the duration value `dt` to a double-precision value of minutes

```
ans = 1380
```

and the call

```
>> calmonths(cd)
```

returns a numeric value of full calendar months in the calendarDuration scalar `cd`

```
ans = 41
```

Note, the `isbetween` function of Table C.4 allows you to determine whether datetime elements lie within the specified interval. For example, utilizing the `t1` and `t2` values as determined earlier, the call

```
>> isbetween(Tc(6),t1,t2)
```

yields a logical scalar corresponding to the `true` value (see Sec. 5.2):

```
ans = 1
```

The same result could also be achieved using relational and logical operators (see Secs. 5.2 and 5.3) explicitly:

```
>> Tc(6) >=t1 && Tc(6) <=t2
ans = 1
```

Table C.4 Functions for Determining `datetime` Array Properties and Computing Differences

Function	Brief Description
Calculate Differences between the Dates	
`between(t1,t2)`	Returns the calendarDuration differences between the datetime values in `t1` and `t2`
`caldiff(t)`	Calculates calendarDuration differences between adjacent datetime values in `t`
Shift Dates	

`dateshift(t,'end','unit')`
`dateshift(t,'start','unit')`
`dateshift(t,'dayofweek',...`
`dow)`
	Shifts each value in the datetime array `t` ahead to the end of the unit of time specified by `unit` (`year`, `month`, `hour`, etc.). The second argument may assume the `'start'` or `'dayofweek'` value to shift back to the beginning of the unit or the next occurrence of the specified day of the week `dow` (defined as `'Monday'`, `'thu'`, `'Sa'`, etc.), respectively

Determine Input Properties	
`isbetween(t,t1,t2)`	Determines if datetime elements in `t` lie within the closed interval `[t1, t2]`
`isdatetime(t)`	Determines if input `t` is a datetime array
`isduration(d)`	Determines if input `t` is a duration array
`iscalendarduration(c)`	Determines if input `t` is a calendarDuration array
`isweekend(t)`	Determines weekend elements in a datetime array `t`
`isnat(t)`	Determines `NaT` (Not-a-Time) elements in a datetime array `t` (`NaT` represents datetime that is undefined)
`isdst(t)`	Determines if elements in a datetime array `t` occur during the DST period
`timezones` `timezones(area)`	Displays a list of all (or a specific geographic `area` related) Internet Assigned Numbers Authority (IANA) time zones accepted by the `datetime` function. For each time zone, the list includes the offset from coordinated universal time (UTC) (in hours, where east is positive) and the DST shift (in hours). The `area` must be `'Africa'`, `'America'`, `'Antarctica'`, `'Arctic'`, `'Asia'`, `'Atlantic'`, `'Australia'`, `'Etc'`, `'Europe'`, `'Indian'`, or `'Pacific'`. (Do not confuse `timezones` with the `timezone` function from the Mapping Toolbox, which returns a complete zone description corresponding to the input longitude.)
`tzoffset(t)`	Returns an array of durations equal to the time zone offset from UTC of each datetime value in `t`

Until now, we have only used one format of datetime representation, `'YYYY-MM-DD'`. Of course, the `datetime` function of Table C.2 allows you to use many other formats, as shown in Table C.5 (see also Fig. 2.29a). Once the datetime variable is created, you can always change its display format by directly accessing its **Format** property, for example,

```
>> t.Format='''FY''''''yy, QQQQ'; disp(t(end))
    FY'22, 2nd quarter
```

(`t.Format='default'` returns to the default format, and `t.Format='defaultdate'` returns to the default format with no time components).

In a similar manner, you can request/change other properties as well. They are **Year**, **Month**, **Day**, **Hour**, **Minute**, **Second**, **TimeZone**, and **SystemTimeZone**. For example, in response to

```
>> t.SystemTimeZone
```

you may get

```
ans = 'America/Los_Angeles'
```

(which obviously depends on the time zone set up on your computer).

Unlike **SystemTimeZone**, the default **TimeZone** property of a datetime is empty (0-by-0 character array). Unless you work with datetime values from multiple time zones and/or account for DST, you do not need to care about specifying this property at all. If you do need to specify it, you can do it using the `'TimeZone'` name–value pair argument in the `datetime` function call (see Table C.2) or by referring to this property afterwards. For example, the calls

```
>> tN=datetime('Dec 17, 1998  14:33','TimeZone','UTC+7');
>> tM=datetime('Dec 16, 1998  23:33'); tM.TimeZone='local';
```

define exactly the same time but 15 time zones apart:

```
>> disp(tM-tN)
    00:00:00
```

Note that changing the month to, say, June (DST period) would result in

```
  -01:00:00
```

You can also extract the datetime array components using the functions listed in Table C.6; for example,

```
>> month(Ta(3:7:20),'name')
```

returns a 1-by-3 cell array

```
ans = {'June'}  {'July'}  {'July'}
```

and

```
>> [a,b]=split(cd,{'month','day'}); disp([a,b])
```

yields

```
  41  13
```

Table C.5 Examples of `datetime` Display Format

Value of `fmt`	Example	Notations
`'yyyy-MM-dd'`	`2019-12-18`	`y, ..., yyyy` – year `u, ..., uuuu` – ISO year[a] `M, ..., MMMM` – month `d, dd` – day of month `D, ..., DDD` – day of year `e, ..., eeee` – day of week `h, hh` – hour (12-hour notation) `H, HH` – hour (24-hour notation) `m, mm` – minute `s, ss` – second `S, ..., SSSSSSSSS` – fractional second `x, X, ..., xxxxx` – time zone offset `z, Z, ..., ZZZZ` – time zone offset `Q, ..., QQQQ` – quarter `a` – a day period `G` – Era `W` – week of the month
`'dd/MM/yy'`	`19/04/14`	
`'dd.M.yyyy'`	`19.4.2014`	
`'eeee, MMM d, uuuu'`	`Saturday, Apr 19, 2014`	
`'MMMM d, yyyy h:mm a'`	`April 19, 2014 9:41 PM`	
`'MMMM d, yyyy HH:mm z'`	`May 19, 2014 21:41 PST`	
`'yyyy-MM-dd'' T ''HH:mm:SS'`	`1962-04-14 T 13:55:00`	
`'yyyy-MM-dd HH:mm:ss.SSS'`	`2014-04-19 21:41:06.123`	
`'yyyy-MM-dd HH:mm XXX'`	`2014-04-19 21:41 -08:00`	
`'''FY''''''yy, QQQQ'`	`FY'19, 2nd quarter`	
`'yyyy G'`	`2019 CE`	

[a]The International Organization for Standardization (ISO) date format, ISO 8601, eliminates ambiguity in expressing a numeric calendar date, including BC dates.

Table C.6 Functions for Extracting Components of a `datetime` Array

Function	Brief Description
Extract datetime Array Components by Time Unit	
`year(t)`	Returns the year numbers for the datetime values in `t`
`quarter(t)`	Returns the quarter values for the datetime values in `t`
`month(t,'name')`	Returns the month number or name (if the second argument is present) for the datetime values in `t`
`week(t)`	Returns the week number for the datetime values in `t`
`day(t)`	Returns the day number or name for the datetime values in `t`
`hour(t)`	Returns the hour values for the datetime values in `t`
`minute(t)`	Returns the minute values for the datetime values in `t`
`second(t)`	Returns the second values for the datetime values in `t`
Split datetime and calendarDuration Array Values	
`[y,m,d]=ymd(t)`	Returns year, month, and day values of datetime `t` (equivalent to calling the `year`, `month`, and `day` functions on the same datetime array)
`[h,m,s]=hms(t)`	Returns hour, minute, and second values of datetime or duration `t` (equivalent to calling the `hour`, `minute`, and `second` functions)
`[x1,x2,...] = ... split(c,units)`	Splits calendarDuration values specified in `t` into numeric and duration units. The number of date and time units specified by `units` determines the number of output arguments, e.g., `{'months','days','time'}` calls for three output arguments
`time(c)`	Converts time of calendarDuration values in `t` into duration values
`timeofday(t)`	Returns an array of durations equal to the elapsed time since midnight for each of the datetime values in `t`

When creating datetime arrays from Excel date numbers (where January 1, 1900, is referred to as a serial number 1), you can use the `'ConvertFrom'` property (Table C.2), for example,

```
>> dEx=[32339,  31802;  31266,30559];
>> dM=datetime(dEx,'ConvertFrom','excel')
dM = 15-Jul-1988 00:00:00    25-Jan-1987 00:00:00
     07-Aug-1985 00:00:00    31-Aug-1983 00:00:00
```

Note, that Excel date numbers do not carry time, so if needed, the times can be added later, like

```
>> dM.Hour=[2,  10;  23,  8];
>> dM.Minute=[0,  30;  30,  25]; disp(dM)
    15-Jul-1988 02:00:00    25-Jan-1987 10:30:00
    07-Aug-1985 23:30:00    31-Aug-1983 08:25:00
```

You can also use the character vectors containing dates defined in foreign languages. All you have to do is adjust the date format (according to Table C.5) and specify the `'Locale'` property (using the lowercase ISO

639-1 two-letter code that specifies a language and uppercase ISO 3166-1 alpha-2 code that specifies a country):

```
>> dIt={'Gennaio 22, 2020','Maggio 5, 2021','Giugno 4, 2022'};
>> dM=datetime(dIt,'InputFormat','MMMM d, yyyy','Locale','it_IT')
dM = 22-Jan-2020  05-May-2021  04-Jun-2022
```

Now, let us create a 1-by-365 datetime `t` vector

```
>> t=datetime('tomorrow')+caldays(1:365);
```

(instead of `caldays(1:365)`, we could have used `calendarDuration(0,0,1:365)`). Let us also create the same-size `y` vector

```
>> y= linspace(10,40,365)+10*rand(1,365);
```

The following self-explanatory commands show examples of visualizing the `datetime` data type, specifying x-axis tick label format (note that not all combinations of Table C.5 will work though), and rescaling the datetime axis (Fig. C.2):

```
>> subplot(311), scatter(t,y), grid, Y=xlim; xtickformat('MM-dd-yy')
>> subplot(312), plot(t,y,'d--'), grid, xlim([Y(1)+150 Y(2)-200])
>> subplot(313), stem(t,y), xlim(datetime(2019,[7,8],[15,7])),...
                                        xtickformat('d MMMM')
```

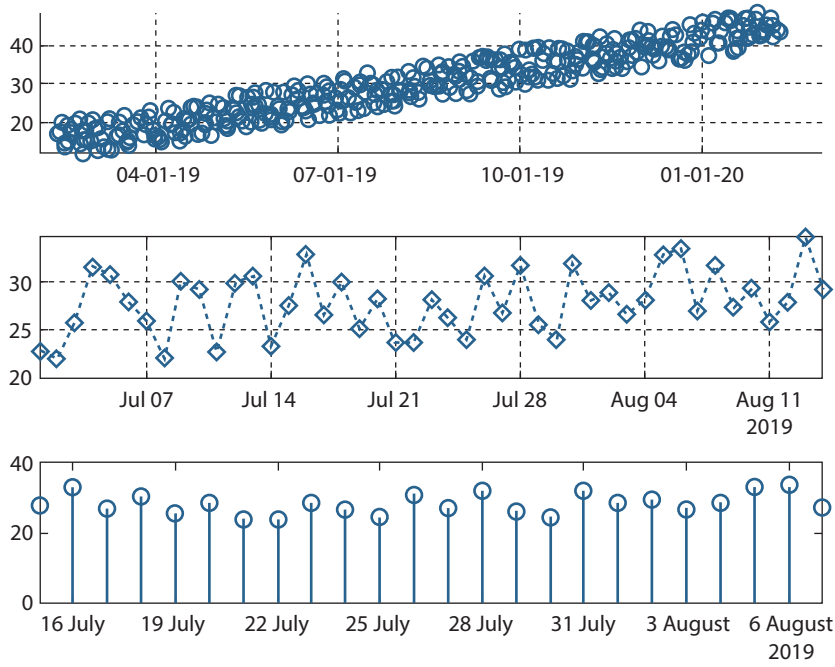


Fig. C.2 Visualization of `datetime`-based data.

Table C.7 Functions for Displaying Calendar, Current Date, and Time

Function	Brief Description
Current Date and Time as a Serial Date Number, Date Vector, and String	
`now`	Returns the current date and time as a serial date number
`clock`	Returns the current date and time as a six-element date vector
`date`	Returns the current date as a character vector
Calendar Data	

`calendar`
`calendar(date)`
`calendar(Y,M)`
	Returns a 6-by-7 matrix containing a calendar (which runs Sunday through Saturday) for the current month, the month that includes a specific `date` (specified as a serial date number or character vector), or the specified month `M` of the specified year `Y`

`eomday(Y,M)`	Returns the last day of the year and month given by corresponding elements of the numeric arrays `Y` and `M`
`[N,Name]=weekday(t)`	Returns the day of week, e.g., `[~,k]=weekday('2023-1-22')` returns `k='Sun'`

To conclude, Table C.7 presents some useful functions to display the one-month calendar, current date, and time, for example,

```
>> calendar(2023,1)
                Jan 2023
   S    M   Tu    W   Th    F    S
   1    2    3    4    5    6    7
   8    9   10   11   12   13   14
  15   16   17   18   19   20   21
  22   23   24   25   26   27   28
  29   30   31    0    0    0    0
   0    0    0    0    0    0    0
```

C.2 Timetables

`Timetable`, introduced three years after the `table` class, in R2016b, is a special type of table that associates a time with each row. Like tables, timetables can store column-oriented data variables that have different data types and sizes as long as they have the same number of rows (see Sec. 4.5).

Recall the example of Fig. 4.8 and assume we have another Excel spreadsheet with some fictitious parachute airdrop data that can be loaded using the following command:

```
>> TA=readtable('AirDropData.xlsx');
```

The `head` function (see Table 4.7) helps you visualize the content of the table by returning top rows (eight by default), so that

```
>> disp(head(TA,4))
```

produces

```
    Date          System          DropAlt_    Weight_    ExitTime_     Miss_m_
                                     ft_         lbs_       Zulu_         m_
__________    _____________    _________    _______    __________    ________

10/20/2009    '10K LCADS HALO'    17500     '9900'     '21:52:54'    '447.7'
10/20/2009    '10k Screamer'      14000     '9000'     '17:25:27'    '84.7'
10/20/2009    '10k Screamer'      14000     '8200'     '17:25:29'    '48'
10/20/2009    '2K G-12 HALO'       9999     '2170'     '18:04:15'    '364.7'
```

As mentioned in Sec. 4.5, variable names in table `TA` are modified from the titles of the columns in the Excel spreadsheet to make them valid MATLAB identifiers (in our case, by replacing the parenthesis with a legitimate underscore symbol). The original names, however, are saved in the **VariableDescriptions** property of the table. Also, you may notice that some numerical data were read in as double-precision numbers and others as character vectors. The latter is caused by the original Excel file having some non-number characters in the corresponding columns. To fix this, you may use the `str2double` function (if it cannot convert text to a number, it then returns a `NaN` value):

```
>> TA.Weight_lbs_=str2double(TA.Weight_lbs_);
>> TA.Miss_m_=str2double(TA.Miss_m_);
```

The `table2timetable` function (Table C.8) allows converting the table `TA` to a *timetable* `TT`:

```
>> TT = table2timetable(TA);
```

In doing so, the first datetime or duration variable in `TA` (in our case, `Date`) becomes the vector of *row times* of `TT`. The row times are metadata that label the rows. The remaining variables of `TA` become the variables of `TT`. Note that `TA` has six variables, whereas timetable `TT` has five:

```
>> whos TT TA
  Name      Size      Bytes     Class       Attributes
  TA        140x6     43907     table
  TT        140x5     43565     timetable
```

When displaying TT in the Command Window, the only visual difference is that the first column shows up in bold font, indicating that the first variable with times, `Date`, became the time vector of `TT`. As such, you can index into `TT` using row times from its time vector, like `TT('10/20/2009',:)`.

In this particular case, however, the aforementioned indexing would return multiple lines. This is because, in the original Excel file, the (unique) airdrop system release time was stored in a separate column,

Table C.8 Functions for Creating a `timetable` Container and Visualizing Its content

Function	Brief Description
Create a Timetable from Scratch	
`timetable(rowTimes,var1,...,varN)`	Creates a timetable array with time-stamped rows and variables of different types
`array2timetable(X,'RowTimes',rowT)`	Converts a homogeneous array `X` to a timetable using the `rowT` datetime vector
Create a Timetable from a Table or Array	
`table2timetable(T)`	Converts the table `T` to a timetable (the first datetime or duration variable in `T` becomes the vector of row times of timetable)
`table2timetable(T,'RowTimes',Var)`	Assigns the table variable `Var` as the vector of row times of an output timetable
`table2timetable(T,'RowTimes',rowT)`	Assigns the (external) vector `rowT` as the vector of row times of an output timetable
`table2timetable(T,'SampleRate',Fs)`	Uses the sample rate `Fs` to calculate regularly spaced row times [`Fs` is a positive numeric scalar that specifies the number of samples per second (Hz)]
`table2timetable(T,'TimeStep',dt)`	Uses the time step `dt` to calculate regularly spaced row times (`dt` is a duration or calendarDuration value)
`timetable2table(TT)`	Converts the *m*-by-*n* timetable `TT` to an *m*-by-($n+1$) table. The vector of row times from `TT` becomes the first variable
Visualizing Table or Timetable Content	
`stackedplot(T)`	Plots the variables of table or timetable `T` in a stacked plot with common *x*-axis (introduced in R2018b)
`parallelplot(T)`	Creates a parallel coordinates plot, where each line represents a row in table `T`, and each coordinate variable in the plot corresponds to a column in this table (introduced in R2019a)

`ExitTime_Zulu_`. To combine the `Date` and `ExitTime_Zulu_` variables into a single datetime vector, you can use the following trick:

```
>> TTtod=TT.Date+timeofday(datetime(TT.ExitTime_Zulu_))
```

Let us change the display format to make sure we achieved what we wanted:

```
>> TTtod.Format='dd.MM.uuu HH:mm:ss'; disp(TTtod(1:3))
    20.10.2009  21:52:54
    20.10.2009  17:25:27
    20.10.2009  17:25:29
```

Now, all you have to do is replace the first column and remove the now redundant `ExitTime_Zulu_` variable:

```
>> TT.Date=TTtod;  TT.ExitTime_Zulu_=[];  disp(head(TT,4));
```

Date	System	DropAlt_ft_	Weight_lbs_	Miss_m_
20.10.2009 21:52:54	'10K LCADS HALO'	17500	9900	447.7
20.10.2009 17:25:27	'10k Screamer'	14000	9000	84.7
20.10.2009 17:25:29	'10k Screamer'	14000	8200	48
20.10.2009 18:04:15	'2K G-12 HALO'	9999	2170	364.7

For the modified timetable, the call like

```
>> TT('20.10.2009 18:04',:)
```

returns the expected unique entry

Date	System	DropAlt_ft_	Weight_lbs_	Miss_m_
20.10.2009 18:04:15	'2K G-12 HALO'	9999	2170	364.7

If you need to sort the table, you can use the `sortrows` function of Table 3.15 and issue the `TT=sortrows(TT,'Date')` command.

You may have a look at table `TT` properties by issuing the command

```
>> TT.Properties
```

which returns

```
ans =
TimetableProperties with properties:
          Description: ''
             UserData: []
       DimensionNames: {'Date' 'Variables'}
        VariableNames: {'System' 'DropAlt_ft_' 'Weight_lbs_' 'Miss_m_'}
 VariableDescriptions: {1×4 cell}
        VariableUnits: {}
   VariableContinuity: []
            RowTimes: [140×1 datetime]
           StartTime: 20.10.2009 21:52:54
          SampleRate: NaN
            TimeStep: NaN
     CustomProperties: No custom properties are set.
  Use addprop and rmprop to modify CustomProperties.
```

The `DimensionNames` property shows the names of the two dimensions, with the `VariableNames` property showing the names of variables along the second dimension, that is, row-wise.

Table C.8 also shows a couple more options for creating timetables. You can create a timetable by concatenating the same-size time vector and variable vectors horizontally using the `timetable` function, or you can take advantage of the `array2timetable` function for converting a homogeneous array into a timetable; for example,

```
>> Ts=array2timetable(magic(3),'RowTimes',hours(1):1/24:hours(3))
```

returns a 3-by-3 timetable

```
Ts =
  Time    Var1    Var2    Var3
  _____   _____   _____   _____

  1 hr      8       1       6
  2 hr      3       5       7
  3 hr      4       9       2
```

(Note that in this case a duration vector was used as a row times vector.)

There are two functions have been specifically developed to assist you in visualizing the content of tables and timetables (Table C.8). The first one, `stackedplot`, allows you to plot the variables of a table or timetable in a *stacked plot* with a common *x*-axis. To this end, the call

```
>> L1=stackedplot(TT(:,2:end),'.'); grid
```

produces a three-strip chart, as shown in Fig. C.3a. It also features a movable data cursor, showing the current data values in each of the stacked plots (you just need to hover over a specific location to display *y* values for each plot). You can access/change any property of the StackedLineChart object `L1` in a usual manner (using the dot notation or `get`/`set` functions) (see Fig. 6.28b).

Another function, `parallelplot`, creates a *parallel coordinates plot* from a table or timetable and returns a ParallelCoordinatesPlot object (Fig. C.3b):

```
>> L2=parallelplot(TT);
```

Each line in the plot represents a row in the table, and each coordinate variable in the plot corresponds to a column in the table. Hovering over the values in the plot allows you to display the data tips. As illustrated in Fig. C3b, a data tip indicates the row in the table corresponding to the line in the plot.

In addition to the functions applicable to regular tables (Tables 4.7 and 4.8), timetables feature a few more functions shown in Table C.9 (as usual, these functions feature more syntaxes as opposed to the simplest ones shown here).

The subscripts allow you to select a specific group of rows/variables. For example, the call

```
>> max(TT{:,vartype('numeric')},[],'omitnan')
```

returns

```
ans=17500  31750  9249
```

(note, `max(TT,[])` would return an error).

The remaining functions allow aligning, combining, and performing group calculations with time-stamped data in one or more timetables, as discussed next.

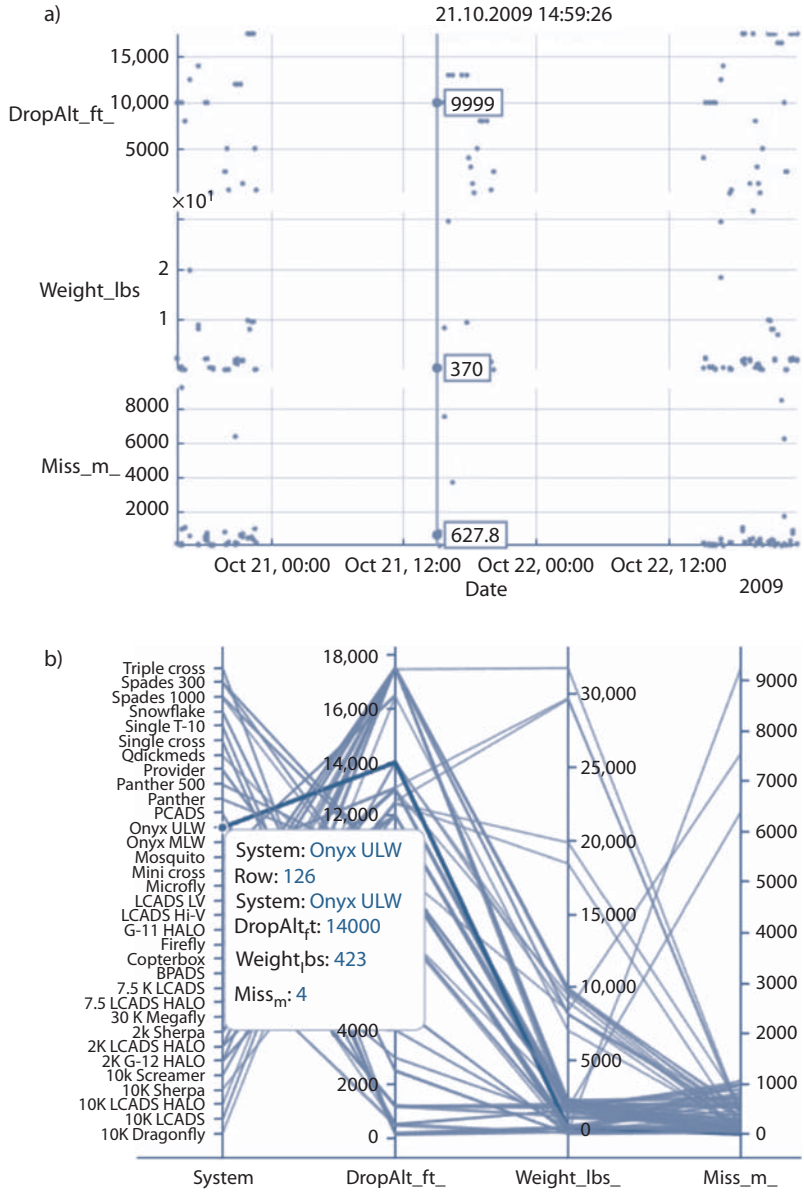


Fig. C.3 a) Stacked and b) parallel coordinates plots of timetable data.

C.3 Timetable Cleaning Workflow

Without going into too much detail, let us mention the data cleaning procedure to be followed when a large piece of time-stamped data is read in from some sensor (or other source). Such data are often messy and may contain

Table C.9 Functions for the `timetable` Data Type Handling

Function	Brief Description
Create the Subscripts for Timetables	
`S=vartype(type)`	Creates a subscript to select table variables of a specified type (`'numeric'`, `'float'`, `'integer'`, `'string'`, or `'cellstr'`), e.g., `T2=T1(:,vartype('numeric'))` returns `T2` as a table that contains only numeric variables from the table `T1`
`S=timerange(startTime,endTime)`	Creates a time range `S` for timetable row subscripting (`TT(S,:)`)
`S=withtol(rowTimes,tol)`	Creates time tolerance `tol` for a timetable row subscripting (`TT(S,:)`)
Check Properties of a Timetable	
`istimetable(TT)`	Determines if input `TT` is a timetable
`ismissing(TT)`	Returns a logical array that indicates which elements of an array or table `TT` contain missing values
`issorted(TT)`	Determines if the table `TT` is sorted
`isregular(TT)`	Determines whether times in timetable `TT` are regular (the row times are regular if they increase or decrease monotonically by a fixed time step)
Clean and Synchronize Timetable Data	

`containsrange(TT,TimeRange)`
`overlapsrange(TT,TimeRange)`
`withinrange(TT,TimeRange)`
	Returns `1` (true) if the range of the row times of timetable `TT` contains (intersects or **is entirely within**) the range of times specified by `TimeRange` (otherwise, it returns `0` (false)) (introduced in R2020a)

`retime(TT1,newTimeStep,method)`	Resamples or aggregates data in a timetable and resolves duplicate or irregular times (`method` defines filling missing data procedure, interpolation, or data aggregation technique)[a]
`synchronize(TT1,TT2)`	Synchronizes timetables `TT1` and `TT2` to a common time vector and resamples or aggregates data from input timetables[b]

`lag(TT)`
`lag(TT1,n)`
`lag(TT1,dt)`
	Shifts data in each variable in `TT` forward in time by one or `n` time step(s) or by a duration or calendarDuration time interval `dt` (the row times of `TT` must be regular). If `n` is negative, then lag shifts the data backward in time (a lead)

[a]This function can be used interactively in the Retime Timetable **Task** interface of Live Editor (Fig. 2.13a).
[b]This function can be used interactively in the Synchronize Timetables **Task** interface of Live Editor (Fig. 2.13a).

anomalies or errors. You would usually read these data in as a table and then convert them to a timetable, because timetables are well suited for handling the aforementioned issues by providing a number of different ways to resolve missing, duplicate, or nonuniform data, as shown in Fig. C.4.

The very first check you would need to do is whether sensor data misses any pieces. If you want to find and count the missing values in all timetable variables, you can issue the following two commands:

```
>> indMisData=ismissing(TT);
>> disp(sum(indMisData));
```

To display the rows with missing data, you could use

```
>> TT(any(indMisData,2),:)
```

Most important, you need to see if your data is missing a time stamp (row time). If so, during a conversion to timetable, the not-a-`datetime` values would be replaced with the `NaT` values. Assuming the time stamp is stored in the `Date` variable, the following two commands find the indices of the rows with the missing time stamps (`indTTmis` is a logical vector that contains `1` wherever `TT.Date` has a missing value) and create a new timetable `TTa` featuring no `NaT` values:

```
>> indTTmis=ismissing(TT.Date);
>> TTa=TT(~indTTmis,:);
```

If you want to remove not only the rows that have missing row times, but also the rows with the `NaN` data values, you would use the `rmmissing` function

```
>> TTa=rmmissing(TT);
```

The next step is to determine whether `TTa` is sorted and sort it:

```
>> indTTa =issorted(TTa);
>> TTb=sortrows(TTa);
```

Even if the timetable is sorted, it can still have time steps that are not uniform, that is, it will be irregular:

```
>> indTTbreg=isregular(TT);
```

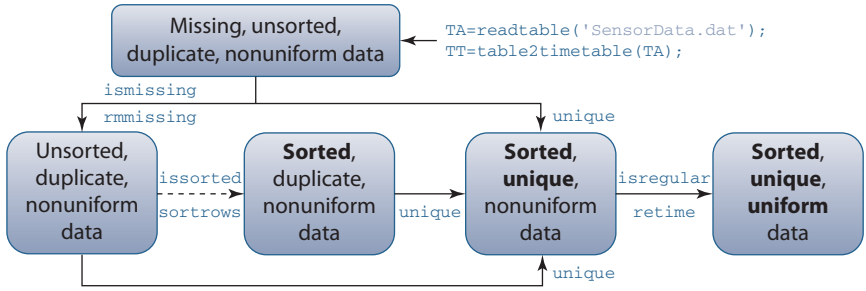


Fig. C.4 Timetable data cleaning process.

Irregularity can be caused by duplicate time stamp or duplicate rows, that is, when all data values in one row are the same as in another. To remove these latter duplicate rows, you can use the `unique` function

```
>> TTc=unique(TTb);
```

By the way, the `unique` function returns the unique rows sorted by their row times, meaning that the previous step (applying the `sortrows` function) could have been omitted. Handling the rows that have a duplicate row time but different data values requires a more delicate approach.

First, you need to sort the row times and find consecutive times that have no difference between them:

```
>> dupTimes=sort(TTb.Date);
>> indTTbdup=(diff(dupTimes)==0);
```

Then, index back into the vector of row times and return a unique set of times that identify the duplicate row times in `TTb`:

```
>> dupTimes=dupTimes(indTTbdup);
>> dupTimes=unique(dupTimes);
```

Finally, index into the timetable to display the rows with duplicate row times:

```
>> TTb(dupTimes,:)
```

You might have a closer look at the output timetable to understand why you have the same time step for several nonduplicate data rows. If it is fixable, do it manually; otherwise, you can use the `retime` function to resolve duplicate or irregular times automatically.

To resample, start from creating a vector of unique row times from `TTb`

```
>> uniqueTimes=unique(TTb.Time);
```

Then, select the first row from each set of rows that have duplicate times:

```
>> TTc=retime(TTb,uniqueTimes)
```

If instead you want to use the last rows from each set of rows that have duplicate times, use the `'previous'` argument:

```
>> TTc=retime(TTb,uniqueTimes,'previous')
```

You can also aggregate numeric data from all rows with duplicate times. For example, you can calculate the means of several measurements of the same quantity taken at the same time with

```
>> TTn=retime(TTb(:,vartype('numeric')),uniqueTimes,'mean')
```

or use alternative approaches (`'sum'`, `'min'`, `'prod'`, and others). Note, when interpolating numerical data into the `TTn` table, we ignored all but numeric data. However, your original table `TT` may have nonnumeric values as well (`logical`, `char`, `cell`, etc.). In this case, if you want to

preserve the same variables in the resampled/aggregated table, repeat resampling on that other type of data and then concatenate the results, for example,

```
>> TTnn=retime(TTb(:,vartype('char')),uniqueTimes,'firstvalue');
>> TTd=[TTn TTnn];
```

The `retime` function can also be used to make your timetable regular by interpolating numeric data onto a regular time vector. For example, the following uses hourly linear interpolation (each row time in `TTb` begins on the hour; hence, there is an one-hour interval between consecutive row times):

```
>> TTn=retime(TTb(:,vartype('numeric')),'hourly','linear')
```

Instead of using a predefined time step, you can also specify a time step of your own. For example, the following specifies a time step of 30 minutes using the `'regular'` input argument and the `'TimeStep'` name–value pair argument:

```
>> TTn=retime(TTb(:,vartype ('numeric')),'regular','pchip','TimeStep',...
                minutes(30))
```

Now that you have data that is regularly spaced in time, you may extract it for further analysis (the Simulink model of the Integrated INS/GPS system discussed in Sec. 15.7 was analyzed/tuned using such sets of data):

```
>> A=TTn.Variables
```

(this call is equivalent to `A=TTn{:,:}`).

Let us also introduce the `synchronize` function you would use to synchronize several timetables (representing, say, the numeric sensor data coming from different sources) to a common time vector and resample or aggregate data from input numeric timetables. In the rare case of the same time vectors, the command

```
>> TTs=synchronize(TT1,TT2);
```

would do the trick (in this case you can simply concatenate two timetables horizontally). However, in the general case, the time vectors may differ, so that the resulting timetable would have all the times from both timetables with `NaN` where there are no data. In this case, you would need to use a more sophisticated syntax. For example,

```
>> TTs=synchronize(TT1,TT2,'intersection');
```

constructs a common time vector from the intersection of two individual time vectors, and

```
>> TTs=synchronize(TT1,TT2,'union','linear');
```

fills missing numerical data values with linear interpolation.

To conclude this discussion, Table C.10 also mentions a few more functions introduced in the latest releases that could be very helpful to further analyze your (sensor) data (two of them, `fillmissing` and `filloutliers`,

Table C.10 Functions for Data Smoothing and Outlier Detection

Function	Brief Description
`movmean`	Computes moving averages for a numerical array (R2016a)
`fillmissing`	Fills missing entries of data with the constant value (R2016b)
`isoutlier`	Finds outliers in data (R2017a)
`filloutliers`	Detects and remove outliers in data (R2017a)
`smoothdata`	Smooths noisy data (R2017a)
`ischange`	Finds the abrupt changes in data (R2017b)
`islocalmax` or `islocalmin`	Finds local maxima or minima in data (R2017b)
`rmoutliers`	Detects and removes outliers from data (R2018b)
`detrend`	Removes a mean value or linear trend from a vector or matrix (usually for FFT processing) (R2018b)

were introduced in Sec. 11.6.4). All but the `movmean` and `detrend` functions are applicable to numerical arrays, tables, or timetables (for additional information see the "Data Smoothing and Outlier Detection" and other "Preprocessing Data" help topics). The Live Editor Tasks (Fig. 2.13a) are relying on these functions as well.

C.4 Categorical Arrays

Categorical arrays, introduced in R2013b (see Fig. 4.1), constitute a data type that provides efficient storage and allows convenient manipulation of nonnumeric data while also maintaining meaningful names for the values. This is done by assigning values to a finite set of discrete categories, like Excellent, Good, and Poor, and applying mathematical ordering to them, such as Excellent>Good>Poor, if needed (in the latter case the categorical arrays become *ordinal*). Possible syntaxes for creating a categorical array are shown in Table C.11.

A common use of categorical arrays is to specify groups of rows in a table. Let us demonstrate the benefits of using categorical arrays by continuing to manipulate with the timetable `TT` created in the previous section.

First, we convert a cell array of character vectors `TT.System` to a categorical array

```
>> TT.System=categorical(TT.System);
```

The `summary` function shows the sorted unique values (we only show the first six entries)

```
>> summary(TT.System)
    10K Dragonfly          1
    10K LCADS              2
```

```
10K LCADS HALO          1
10K Sherpa              1
10k Screamer            2
2K G-12 HALO            6
```

(the `summary` function displays the category names along with the number of elements in each category—the category counts). If you need to change the natural (alphabetical) order of the categories, specify the desired order using a second syntax from Table C.11.

You can also create a categorical array by binning numeric data utilizing the `discretize` function. For example, you can subdivide all airdrops into low-, medium-, and high-altitude airdrops by specifying edge altitudes as follows:

```
>> catNames={'LA drop' 'MA drop' 'HA drop'};
>> binDataA=discretize(TT.DropAlt_ft_,[0 5000 10000 20000],...
                      'categorical', catNames);
```

The `discretize` function returns the indices of the bins that contain the elements of `TT.DropAlt_ft_` within [0; 5000]ft, [5000; 10000]ft, and [10000; 20000]ft:

```
>> summary(binDataA)
    LA drop             33
    MA drop             49
    HA drop             68
```

Table C.11 Syntaxes for Creating a Categorical Array

Syntax	Brief Description
`B=categorical(A)`	Creates a categorical array `B` from the array `A`. The categories of `B` are the sorted unique values from `A`
`B=categorical(A,valueset)`	Creates one category for each value in `valueset` (the `categorical` function removes leading and trailing spaces from elements of `valueset` that are strings or character vectors). The categories of `B` are in the same order as the values of `valueset`. If `A` contains any values not present in `valueset`, then the corresponding elements of `B` are undefined
`B=categorical(A,valueset,catnames)`	Names the categories in `B` by matching the category values in `valueset` with the names in `catnames`
`B=categorical(A,...,name,value)`	Creates a categorical array with additional options specified by `name-value` pair arguments [the `name` is `'Ordinal'` or `'Protected'` and the values are `0` (false) or `1` (true)]

Categorical data summary can be conveniently visualized graphically as well. For example,

```
>> pie(binDataA)
```

produces the result shown in Fig. C.5.

Note that in the latter case, the created categorical array is ordinal by construction:

```
>> isordinal(binDataA)
ans = 1
```

This means that although the categorical array `TT.System` has no mathematical ordering, the `binDataA` array data are discrete, nonnumeric values that have a natural ordering, so that `LA drop<MA drop<HA drop`. For example,

```
>> sum(binDataA <= 'MA drop')
```

returns the sum of low- and medium-altitude airdrops:

```
ans = 82
```

Let us add the categorical array we created to the `TT` timetable:

```
>> TT=addvars(TT,binDataA,'before','Weight_lbs_',...
                                    'NewVariableNames','DropType');
```

Assume that the `TT` timetable had such a column from the very beginning. In this case, while categorizing it, we could also make it ordinal using the following command:

```
>> TT.DropType=categorical(TT.DropType,catNames,'ordinal',1);
```

(`1` stands for 'true').

Next, let us categorize the weight column of table `TT` as well:

```
>> catNames={'light' 'medium' 'heavy'};
>> binDataW=discretize(TT.Weight_lbs_,[0 200 2000 10000],...
                      'categorical',catNames);
>> summary(binDataW)
    light              25
    medium             59
    heavy              35
    <undefined>        31
```

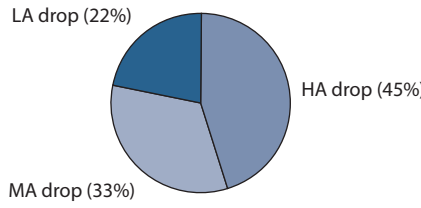


Fig. C.5 The pie chart of a categorical array.

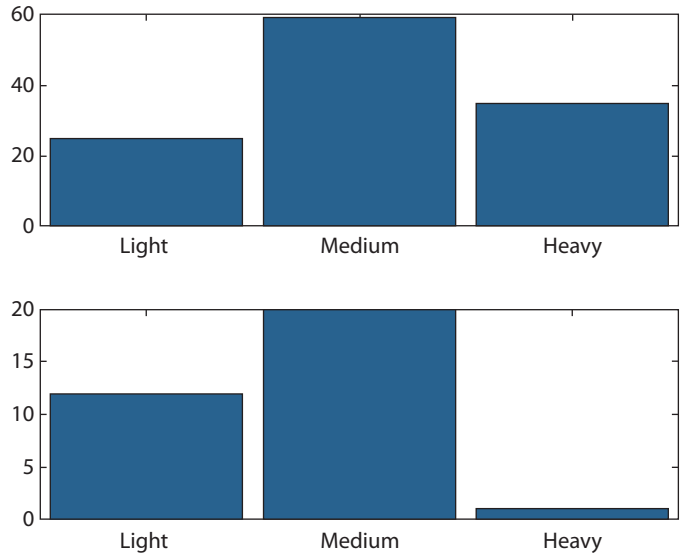


Fig. C.6 The histogram chart of a categorical array.

As seen, the `summary` function also displays the number of elements that were undefined. The following two commands plot a histogram chart of all categorical data `binDataW` as well as a part of it that corresponds to the low-altitude airdrops only (Fig. C.6):

```
>> subplot(2,1,1), histogram(binDataW)
>> subplot(2,1,2), histogram(binDataW(binDataA=='LA drop'))
```

Let us add the ordinal categorical array `binDataW` to `TT` as well:

```
>> TT=addvars(TT,binDataW,'after','Weight_lbs_',...
                                'NewVariableNames','WeightClass');
```

Now, let us use the two variables we added to the table `TT` to produce a so-called *heatmap chart*. The `heatmap` function, introduced in R2017a, allows you to create the HeatmapChart object with the two input table variables serving as the x-axis and y-axis. In addition, the `sortx` and `sorty` commands, introduced in R2017b, allow you to sort the heatmap row-wise and column-wise, respectively. For example, the two commands

```
>> h=heatmap(TT,'WeightClass','DropType');
>> sorty(h,'heavy','descend') % sort by the elements in the last column
```

produce a heatmap shown in Fig. C.7a. The default colors are based on a count aggregation, which totals the number of times each pair of x and y values appears together in the table. Alternatively, when determining the heatmap cell colors, you can use another table variable and specify the

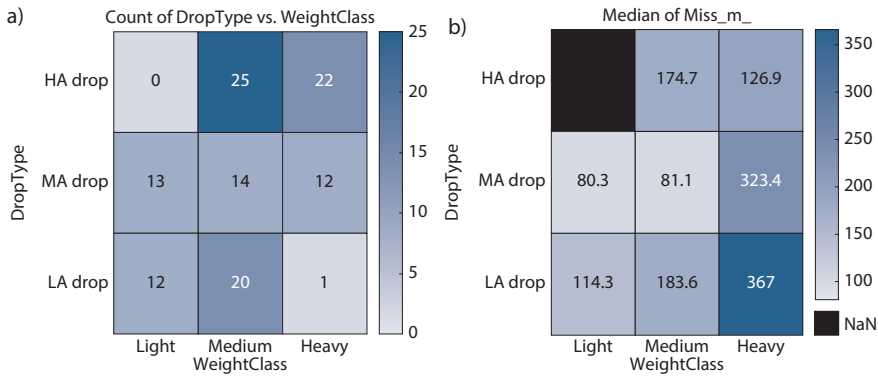


Fig. C.7 The heatmap charts of a categorical array.

calculation method to use. For example, modifying two HeatmapChart handle properties as

```
>> h.ColorVariable='Miss_m_';
>> h.ColorMethod='median';
```

results in computing the median miss distances for each combination of x and y values, as shown in Fig. C.7b.

Last, let us use the ordinal categorical variable `TT.WeightClass` to create a new table

```
>> rows=TT.WeightClass <='medium';
>> vars={'WeightClass' 'Miss_m_'};
>> TT1=TT(rows,vars), head(TT1,4)
ans =
  WeightClass    Miss_m_
  ___________    _______

    medium       147.1
    light        181.2
    medium       61
    medium       6390.3
```

and then compute the medium values for two airdrop systems categories:

```
>> MDl=median(TT1.Miss_m_(TT1.WeightClass=='light'));
>> MDm=median(TT1.Miss_m_(TT1.WeightClass=='medium'));
>> disp("Miss dist. light:   " + MDl + "ft")
>> disp("Miss dist. medium:  " + MDm + "ft")
Miss dist. light:  80.3ft
Miss dist. medium: 147.1ft
```

It should be noted that all of the preceding could have been accomplished using character arrays or cell arrays of character vectors. In this case, to compare the values, you would use the `strcmp` and/or `strfind`

function, and your code would become cumbersome. Also, with character arrays or cell arrays of character vectors, an ordering (other than alphabetical order) is not possible. With categorical arrays, we were able to use relational operators (== and <=) to test for equality and perform element-wise comparisons (the same way that you compare the numeric arrays) that have a meaningful mathematical ordering to compare elements. Another alternative to using categorical arrays would be storing categorical data using integers in numeric arrays. However in this latter case, all the useful descriptive information from the category names would be lost, and the integer values would prevent you from ordering categories in the way you want.

Table C.12 Categorical Data Manipulation Functions

Function	Brief Description
`iscategorical(C)`	Determines whether `C` is a categorical array
`categories(C)`	Returns a list of categories of a categorical array
`iscategory(C,catname)`	Tests whether `catname` is a category of `C`
`isordinal(C)`	Determines whether `C` is an ordinal categorical array
`isprotected(C)`	Determines whether categories of the categorical array `C` are protected
`addcats(C,newcats)`	Adds `newcats` categories to the end of the category list for the input categorical array `C` (additional arguments allow adding `newcats` before of after the specific category)
`mergecats(C,oldcats,newcat)`	Merges two or more categories in the categorical array `C` using the `oldcats` cell array defining categories to merge into a single new category `newcat` [if the third argument, `newcat`, is omitted, all categories listed in `oldcats` are merged into `oldcats(1)`]
`removecats(C,oldcats)`	Removes categories (defined by the cell array `oldcats`) from the categorical array `C` (if the second argument is omitted, removes unused categories from `C`)
`renamecats(C,oldnames,newnames)`	Renames all (if the second argument is omitted) or `oldnames` categories in the categorical array `C`
`reordercats(C,neworder)`	Reorders categories in the categorical array `C` to be in alphanumeric order or the order specified by `neworder` (if present)
`setcats(C,newcats)`	Sets categories in an output categorical array using categories defined by `newcats` and elements defined by `C`
`summary(C)`	Displays a summary of the categorical array `C`
`countcats(C)`	Returns the number of elements in each category of the categorical array `C`
`isundefined(C)`	Finds the undefined elements in the categorical array `C`

For completeness, Table C.12 lists all functions that could be used with the `categorical` type of data.

C.5 Word Clouds Out of Categorical Arrays

Let us finish this appendix by showing you a way to process/represent textual data in tables by converting them to a categorical array. Suppose the `Title` variable of the `Projects` table constitutes a cell array containing some paper/project titles stored as character vectors, like `'Intuitive System Training'`.

In this case, the set of commands

```
S=string(Projects.Title);      % convert to string array
S=join(S);                     % join all string elements into a 1-by-1 string
punctChar=["(" ")" "," ":" "-" "."]; % define punctuation characters
S=replace(S,punctChar," ");    % replace them with the space characters
S=split(S);                    % split on the space characters
S(strlength(S)<5)=[];          % remove words with fewer than five characters
C=categorical(S);              % convert words to a categorical array
```

breaks all titles into separate words, removes punctuation characters and short words, and eventually converts them into a categorical array.

From the previous section we know that the `summary(C)` call would produce the category names along with the category counts. Instead, let us use the `wordcloud` function, introduced in R2017b, to produce a *word*

Fig. C.8 Word cloud.

cloud, as shown in Fig. C.8 (a word cloud is a way to visualize text data, emphasizing keywords that have higher counts):

```
>> wc=wordcloud(C);
```

If necessary, you can adjust any property of the `wc` object in the usual way. For example,

```
>> wc.Color=rand(length(wc.SizeData),3);
```

sets the word colors to random values (using RGB triplets with one row for each word); the command

```
>> wc.MaxDisplayWords=50;
```

changes the number of displayed words to 50 (from the default 100), and

```
>> wc.Shape='rectangle';
```

modifies the shape of the word cloud chart from the default `'oval'` to `'rectangle'`.

Appendix D Hooke–Jeeves Minimization Function

D.1 Introduction

In this appendix, you will find a listing of the custom written `fminhj` function. It realizes the Hooke–Jeeves pattern search algorithm mentioned in Sec. 10.8. As shown in the examples of Sec. 10.9.2, this algorithm may outperform that of the `fminsearch` function of MATLAB and that is why the author has been using it in a variety of applications, including real-time optimization, for more than three decades. However, this function is listed here not to promote the method, but rather to show that after reading this book you should have all the knowledge to analyze and understand almost every line of any MATLAB function and also develop your own, professional-looking functions, like `fminhj`.

D.2 Description

The `fminhj` function is intended to work with MATLAB and that is why its inputs and outputs are similar to those used by, say, the `fminsearch` function:

- `x=fminhj(fun,x0)` Starts at `x0` and attempts to find a local minimum `x` of the function `fun`
- `x=fminhj(fun,x0,options)` Minimizes with the default optimization parameters replaced by values in a structure, `options`, created with the `optimset` function of MATLAB
- `[x,fval]=fminhj(...)` Returns the value of the objective function, described in `fun`, evaluated at `x`
- `[x,fval,exitflag]=fminhj(...)` Returns an `exitflag` that describes the exit condition of `fminhj`. Possible values of `exitflag` and the corresponding exit conditions are
  - `1` Indicates that `fminhj` converged to a solution `x`
  - `0` Notifies that the maximum number of function evaluations was reached

- `[x,fval,exitflag,output]=fminhj(...)` Returns a structure `output` with the number of iterations stored in `output.iterations`, the number of function evaluations in `output.funcCount`, the algorithm name in `output.algorithm`, and the exit message in `output.message`.

Specifically, the `fminhj` function uses the following fields of MATLAB's `optimset` structure: `Display`, `TolX`, `TolFun`, and `MaxFunEvals`. Particularly, the `Display` field may be one of the following:

`'none'` or `'off'`	To display neither termination message nor intermediate steps
`'final'`	To display the termination message
`'iter'`	To display the termination message and all intermediate steps
`'notify'`	To supplement the previous option with a graphical output

D.3 M-file Function Listing

```
function[x,fval,exitflag,output]=fminhj(fun,x,options)
%FMINHJ Multidimensional unconstrained nonlinear minimization (Hooke-Jeeves).
%  X = FMINHJ(FUN,X0) starts at X0 and attempts to find a local minimizer
%  X of the function FUN. FUN is a function handle. FUN accepts input X
%  and returns a scalar function value F evaluated at X. X0 can be a scalar
%  or vector.
%
%  X = FMINHJ(FUN,X0,OPTIONS) minimizes with the default optimization
%  parameters replaced by values in the structure OPTIONS, created
%  with the OPTIMSET function. FMINHJ uses the following options:
%  Display, TolX, TolFun and MaxFunEvals. Display is one of the following:
%
        'none','off' - no detailed output
%       'final' - display termination message
%       'iter' - display termination message and all intermediate steps
%       'notify' - show the graphical output as well
%
%  [X,FVAL] = FMINHJ(...) returns the value of the objective function,
%  described in FUN, evaluated at X.
%
%  [X,FVAL,EXITFLAG] = FMINHJ(...) returns an EXITFLAG that describes
%  the exit condition of FMINHJ. Possible values of EXITFLAG and the
%  corresponding exit conditions are
%
%  1 FMINHJ converged to a solution X
%  0 Maximum number of function evaluations is reached
```

```
%
%  [X,FVAL,EXITFLAG,OUTPUT] = FMINHJ(...) returns a structure
%  OUTPUT with the number of iterations taken in OUTPUT.iterations, the
%  number of function evaluations in OUTPUT.funcCount, the algorithm name
%  in OUTPUT.algorithm, and the exit message in OUTPUT.message.
%
%  Examples:
%    FUN can be anonymous function or specified using @:
%       fminhj('x(1)^2+3*x(2)^2',[1,5]')
%       [X,FVAL,EXITFLAG]=fminhj(@cos,3)
%
%  FMINHJ uses the Hooke-Jeeves pattern search (direct search) algorithm.
%
%  Reference: Hooke, R., and Jeeves, T.A., "'Direct Search' Solution of
%  Numerical and Statistical Problems," Journal of the Assoc. Comput. Mach.,
%  Vol.8, No.2, 1961, pp.212-229.

defaultopt = struct('Display','final','MaxFunEvals',...
   '200*numberOfVariables','TolX',1e-4,'TolFun',1e-4,'OutputFcn',[]);
% If just 'defaults' passed in, return the default options in X
if nargin==1 && nargout <= 1 && isequal(funfcn,'defaults')
   x = defaultopt;
   return
end

if nargin < 2
        error('MATLAB:fminhj:NotEnoughInputs', ...
        'FMINHJ requires at least two input arguments');
end

if nargin < 3, options=[]; end
% Check for non-double inputs
if ~isa(x,'double')
        error('MATLAB:fminhj:NonDoubleInput',...
              'FMINHJ only accepts inputsof data type double.')
end

n = numel(x); numberOfVariables=n; % number of varied parameters

printtype = optimget(options,'Display',defaultopt,'fast');
tolx = optimget(options,'TolX',defaultopt,'fast');
tolf = optimget(options,'TolFun',defaultopt,'fast');
maxfun = optimget(options,'MaxFunEvals',defaultopt,'fast');

fun = fcnchk(fun);          % convert to function handle if needed
switch printtype
    case{'none','off'}   % supress any output
        prnt = 0;
    case 'final'         % display final values
        prnt = 1;
```

```
    case 'iter'          % print out all intermediate steps
        prnt = 2;
    case 'notify'        % add a plot
        prnt = 3;
    otherwise
        prnt = 1;
end

% Define the initial step based on the largest absolute value among all
% elements of the vector of initial guesses
scale = abs(x(n)); if scale == 0, scale = n; end
k = scale/5;    % initial step size (20% of the scale)
                % (final step size is determined by x-tolerance)
%% Check the original (basic) point

indexbp = 0;            % set the basic point (BP) search index
indexps = 0;            % set the pattern search (PS) index

   y = x;               % set the latest basic point
   p = x;               % set the suggested pattern search point
   b = x;               % set the previous pattern search point

fnew = feval(fun,x);    % call minimization function
indexbp = indexbp+1;    % increment the basic-point search index

fold = fnew;
ps = 0;                 % set the pattern search flag
bp = 0;                 % set the basic-point search flag

index=indexbp+indexps;
if prnt >= 2
varpar(index)=x(1);
bpflag(index)=bp;
perindex(index)=fnew;
step(index)=k; end

  if prnt >= 2
  header='f(x) eval. x f(x) Step Procedure';
  disp(header)
end

status = 'initial function evaluation';
%% Keep looking for the minimum ...
%   ... while the step size k is greater than the x-tolerance and the value
%   of the objective function is greater than the function tolerance
   while (k > tolx) & (abs(fnew) > tolf)

index=indexbp+indexps;
   if prnt >= 2
   disp(sprintf('%6.0f   %10.3g  %10.3g    %10.3g     %-s',...
                index,x(1),fnew,k,status));
end
```

```
%% Continue the pattern search ...
%   ... if the objective function decreased compared to the previous
%     'pattern' trial, continue the pattern search, i.e. make the same
%      move in the same direction
if (fold - fnew > tolf) & (ps == 1)
status = 'pattern search';

   p = 2.*y-b;          % compute the suggested PS point as b+2*(y-b)
% !!! constrained optimization code !!!
b = y;                  % reassign the latest BP to the previous PS point
x = p;                  % assign the suggested PS point to the trial point
y = x;

z=feval(fun,x);         % check the latest trial point
indexps=indexps+1;  % increment the PS index

index=indexbp+indexps;
if prnt >= 2
varpar(index)=x(1);
bpflag(index)=bp;
perindex(index)=fnew;
step(index)=k; end

fold=fnew;
fnew=z;
%% Switch from searching around the basic point to the pattern search ...
% ... if the objective function decrease was achieved during a search
%    around the basic point
elseif (fold - fnew > tolf) & (ps == 0)
status='switching from BP to PS';

bp = 0;                  % lower the BP flag
ps = 1;                  % raise the PS flag

%% Stop PS, make one backward step & perform a new basic point search ...
% ... if the last pattern step failed
elseif (fold - fnew <= tolf) & (ps == 1)
status='stepping back to start a new BP search';

p = b;          % set everything to be equal to the previous PS point
y = b;
x = b;

fnew=feval(fun,x);
indexps=indexps+1;    % increment the PS index

index=indexbp+indexps;
if prnt >= 2
varpar(index)=x(1);
bpflag(index)=bp;
perindex(index)=fnew;
step(index)=k; end
```

```
fold=fnew;
ps=0;                    % lower the PS flag

%% Proceed with the search around the basic point
elseif (fold - fnew <= tolf) & (ps == 0)
status='BP search';

% if a search around the basic point failed, decrease the step size and
% re-examine a vicinity of the current basic point
    if bp == 1
    k=k/10;              % decrease the step size
    end

% explore the basic point by making two steps (forward and backward) in
% each direction
            for j = 1:n
            x(j) = y(j)+k;
% !!! constrained optimization code !!!
            z=feval(fun,x);
            indexbp=indexbp+1;      % increment the BP index
    index=indexbp+indexps;
    if prnt >= 2
    varpar(index)=x(1);
    bpflag(index)=1;
    perindex(index)=fnew;
    step(index)=k; end

                if z < fnew
                y(j) = x(j);
                else
                x(j) = y(j) - k;
% !!! constrained optimization code !!!
                z=feval(fun,x);
                indexbp=indexbp+1; % increment the BP index
    index=indexbp+indexps;
    if prnt >= 2
    varpar(index)=x(1);
    bpflag(index)=1;
    perindex(index)=fnew;
    step(index)=k; end
              if z < fnew
              y(j)=x(j);
              else
              x(j)=y(j);
              end
            end
    fnew=min(z,fnew);
    end
bp=1;                    % raise the BP flag
```

```
end                     % if end
if index > = maxfun     % break the loop if
break,end               % index > = maxfun
end                     % while end

%% Display the results
index=indexbp+indexps;
if prnt >= 2
    disp(sprintf('%6.0f %10.3g %10.3g %10.3g %-s',...
                        index,x(1),fnew,k,status));
end

fval=fnew;

if prnt >= 3
figure
subplot(2,1,1)
semilogy(0:index-1,perindex.*bpflag,'.k'), hold
semilogy(0:index-1,tolf*ones(length(perindex)),'--r')
legend('Search around BP','TolFun','Location','ne','AutoUpdate','off')
semilogy(0:index-1,perindex)
xlabel('Function evaluation'), ylabel('Function value')
subplot(2,1,2)
semilogy(0:index-1,step.*bpflag,'.k'), hold
semilogy(0:index-1,tolx*ones(length(step)),'--r')
legend('Search around BP','TolX','Location','sw','AutoUpdate','off')
semilogy(0:index-1,step)
xlabel('Function evaluation'), ylabel('Step size')
end

output.iterations = indexps-1; % Initial f(x) evaluation (0th iteration)
output.funcCount = index;
output.algorithm='Hooke-Jeeves pattern search';

if index >= maxfun
    msg=...
    sprintf(['Exiting:\n' '  While ' status ', the maximum' ...
    '  number\n of function evaluations has been exceeded\n' ...
    '  - increase the MaxFunEvals value.\n Current function' ...
    '  value %f'],fval);
    if prnt > 0, disp(msg), end
    exitflag=0;
else
    msg=...
    sprintf(['Optimization terminated:\n' '   While ' status ...
    ',  the current x met\n the termination criteria ' ...
    '   OPTIONS.TolX of %e and F(X)\n satisfied the convergence' ...
    '   criteria OPTIONS.TolFun of %e'], tolx, tolf);
    if prnt > 0, disp(msg), end
    exitflag=1;
```

```
end
output.message = msg;
```

Comment: This function was originally developed to handle constrained minimization problems, so that the penalty function is minimized before the search for the minimum of the objective function starts. For the purpose of this book, however, these portions of code were eliminated and substituted by the line

```
% !!! constrained optimization code !!!
```

D.4 Example

Consider a couple of examples of using the `fminhj` function. The call

```
>> [x,fval,exitflag,output]=fminhj(@cos,5,optimset('display','off'))
```

returns the minimum of the $f = \cos(x)$ function closest to $x = 5$ ($x_* = \pi$)

```
x = 3.1411
fval = -1.0000
exitflag = 1
output = iterations: 8
         funcCount: 21
         algorithm: 'Hooke-Jeeves pattern search'
           message: 1 × 197 char
```

Another call degrades default tolerance and requests for a graphical output

```
>> options=optimset('Display','notify','TolFun',1e-3,'TolX',1e-3);
>> X=fminhj('(x(1)-1.5)^2+3*(x(2)+2.22)^2',[1;5],options)
```

It returns all iterations as shown below and also presents them graphically (Fig. D.1)

f(x) eval.	x	f(x)	Step	Procedure
1	1	157	1	initial function evaluation
5	1	116	1	BP search
5	1	116	1	switching from BP to PS
6	1	82	1	pattern search
7	1	53.7	1	pattern search
8	1	31.4	1	pattern search
9	1	15	1	pattern search
10	1	4.72	1	pattern search
11	1	0.395	1	pattern search
12	1	2.08	1	pattern search

13	1	0.395	1	stepping back to start a new BP search
17	1	0.395	1	BP search
20	1.1	0.203	0.1	BP search
...	...	...	...	...
31	1.5	0.0012	0.1	switching from BP to PS
32	1.6	0.0112	0.1	pattern search
33	1.5	0.0012	0.1	stepping back to start a new BP search
37	1.5	0.0012	0.1	BP search
41	1.5	0.0003	0.01	BP search

```
Optimization terminated:
 While BP search, the current x met
 the termination criteria OPTIONS.TolX of 1.000000e-003 and F(X)
 satisfied the convergence criteria OPTIONS.TolFun of 1.000000e-003
X=1.5000
 -2.2100
```

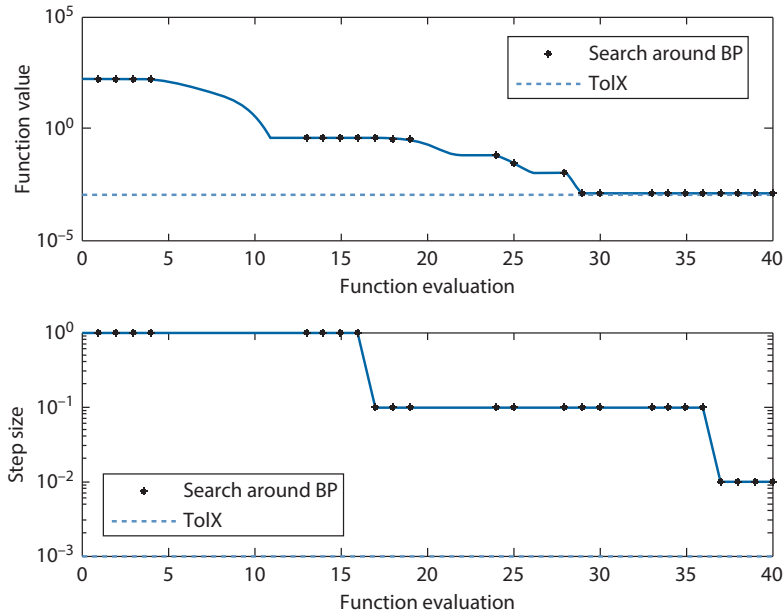


Fig. D.1 Example of the `fminhj` graphical output.

Appendix E Aerospace Systems Modeling Tools

E.1 Introduction

As mentioned in Sec. 1.5, one of MATLAB's toolboxes and one Simulink blockset are intended to enhance the main MathWorks products with scripts and blocks dedicated to modeling and simulating aerospace systems. Although these tools are not a part of the basic version of MATLAB/Simulink (that is, they need to be acquired separately), we think it is appropriate to mention these tools here and illustrate that, once you have completed this book, you are quite ready to proceed toward creating your own professional-looking functions and models. To this end, this appendix introduces one advanced topic, Simulink blocks masking, which was not covered in Chapter 15.

E.2 Aerospace Toolbox

Excluding the specific functions intended to create and manipulate graphics objects for animation within Flight Gear Simulator, the remaining functions, used to model aerospace systems in the MATLAB technical computing environment, can be subdivided into several major categories as presented in Table E.1. To be more specific, these functions are listed in Tables E.2–E.8.

Table E.1 Aerospace Toolbox Subgroups

Subgroup	Brief Description
Coordinate systems	Transforms axes of coordinate systems to different types
Environment	Simulates various aspects of aircraft environment, such as atmosphere conditions, gravity, magnetic fields, and wind
Data import	Reads standard aerodynamic file formats into the MATLAB interface
Flight parameters	Computes various flight parameters, including ideal airspeed correction, Mach number, and dynamic pressure
Gas dynamics	Provides various gas dynamics tables
Quaternion math	Ensures common mathematical and matrix operations on a quaternion
Time	Enables time calculations, including Julian dates, decimal year, and leap year
Unit conversion	Converts common measurement units from one system to another

Table E.2 Axes Transformations Functions

Function	Brief Description
angle2dcm	Creates a direction cosine matrix from rotation angles
angle2quat	Converts rotation angles to a quaternion
angle2rod	Converts rotation angles to an Euler–Rodrigues vector
dcm2alphabeta	Converts a direction cosine matrix to an angle of attack and sideslip angle
dcm2angle	Creates rotation angles from a direction cosine matrix
dcm2latlon	Converts a direction cosine matrix to geodetic latitude and longitude
dcm2quat	Converts a direction cosine matrix to a quaternion
dcm2rod	Converts a direction cosine matrix to an Euler–Rodrigues vector
dcmbody2wind	Creates a direction cosine matrix from the angle of attack and sideslip angle
dcmecef2ned	Creates a direction cosine matrix from the geodetic latitude and longitude
dcmeci2ecef	Converts Earth-centered inertial (ECI) to Earth-centered Earth-fixed (ECEF) coordinates
ecef2lla	Converts ECEF coordinates to geodetic coordinates
eci2aer	Converts ECI coordinates to azimuth, elevation, slant range (AER) coordinates
eci2lla	Converts ECI coordinates to geodetic latitude, longitude, altitude (LLA) coordinates
flat2lla	Estimates geodetic LLA from a flat Earth position
geoc2geod	Converts geocentric latitude to geodetic latitude
geod2geoc	Converts geodetic latitude to geocentric latitude
lla2ecef	Converts geodetic coordinates to ECEF coordinates
lla2eci	Converts geodetic coordinates to ECI coordinates
lla2flat	Estimates a flat Earth position from geodetic LLA
quat2angle	Converts a quaternion to rotation angles
quat2dcm	Converts a quaternion to direction cosine matrix
quat2rod	Converts a quaternion to an Euler–Rodrigues vector
rod2angle	Converts an Euler–Rodrigues vector to rotation angles
rod2dcm	Converts an Euler–Rodrigues vector to a direction cosine matrix
rod2quat	Converts an Euler–Rodrigues vector to a quaternion

Table E.3 Flight Parameters Functions

Function	Brief Description
airspeed	Computes an airspeed from velocity
alphabeta	Computes incidence and sideslip angles
correctairspeed	Converts from one airspeed to equivalent airspeed (EAS), calibrated airspeed (CAS), or true airspeed (TAS)
dpressure	Computes a dynamic pressure using velocity and density
machnumber	Computes a Mach number using velocity and speed of sound
rrdelta	Computes a relative pressure ratio
rrsigma	Computes a relative density ratio
rrtheta	Computes a relative temperature ratio

Table E.4 Environment Functions

Function	Brief Description
aeroDataPackage	Starts Add-On Explorer to download, install, or uninstall aerospace-specific data
atmoscira	Uses the COSPAR International Reference Atmosphere 1986 model
atmoscoesa	Uses the 1976 COESA model
atmoshwm	Implements the horizontal wind model
atmosisa	Uses the International Standard Atmosphere model
atmoslapse	Uses the Lapse Rate Atmosphere model
atmosnonstd	Uses climatic data from MIL-STD-210 or MIL-HDBK-310
atmosnrlmsise00	Implements a mathematical representation of the 2001 U.S. Naval Research Laboratory Mass Spectrometer and Incoherent Scatter Radar Exosphere
atmospalt	Calculates a pressure altitude based on ambient pressure
deltaCIP	Calculates the Celestial Intermediate Pole (CIP) location adjustment
earthNutation	Implements Earth nutation
geocradius	Converts from geocentric latitude to radius of ellipsoid planet
geoidegm96	Calculates the geoid height as determined from the EGM96 Geopotential model
geoidheight	Calculates the geoid height
gravitycentrifugal	Implements a centrifugal effect of planetary gravity
gravityspherical harmonic	Implements a spherical harmonic representation of planetary gravity
gravitywgs84	Implements the 1984 World Geodetic System (WGS84) representation of Earth's gravity
gravityzonal	Implements zonal harmonic representation of planetary gravity
igrfmagm	Calculates Earth's magnetic field and secular variation using the International Geomagnetic Reference Field
moonLibration	Implements Moon librations
planetEphemeris	Implements position and velocity of astronomical objects
polarMotion	Calculates polar motion of Earth
wrldmagm	Uses the World Magnetic model
datcomimport	Brings a DATCOM file into the MATLAB environment

Table E.5 Gas Dynamics Function

Function	Brief Description
flowfanno	Computes Fanno line flow relations
flowisentropic	Calculates isentropic flow ratios
flownormalshock	Produces normal shock relations
flowprandtlmeyer	Calculates the Prandtl–Meyer functions for expansion waves
flowrayleigh	Computes Rayleigh line flow relations

Table E.6 Quaternion Math Functions

Function	Brief Description
quatconj	Calculates a conjugate of a quaternion
quatdivide	Divides a quaternion by another quaternion
quatexp	Calculates an exponential of a quaternion
quatinterp	Interpolates between two quaternions
quatinv	Calculates an inverse of a quaternion
quatlog	Calculates a natural logarithm of a quaternion
quatmod	Calculates a modulus of a quaternion
quatmultiply	Calculates a product of two quaternions
quatnorm	Calculates a norm of a quaternion
quatnormalize	Normalizes a quaternion
quatpower	Calculates a power of a quaternion
quatrotate	Rotates a vector by a quaternion

Table E.7 Time Functions

Function	Brief Description
deltaUT1	Calculates a difference between Coordinated Universal Time (UTC) and Principal Universal Time (UT1)
decyear	Calculates a decimal year
juliandate	Calculates a Julian date
leapyear	Determines a leap year
mjuliandate	Calculates a modified Julian date
tdbjuliandate	Estimates a Julian date for Barycentric Dynamical Time (TDB)

Table E.8 Unit Conversion Functions

Function	Brief Description
convacc	Converts from acceleration units to the desired acceleration units
convang	Converts from angle units to the desired angle units
convangacc	Converts from angular acceleration units to the desired angular acceleration units
convangvel	Converts from angular velocity units to the desired angular velocity units
convdensity	Converts from density units to the desired density units
convforce	Converts from force units to the desired force units
convlength	Converts from length units to the desired length units
convmass	Converts from mass units to the desired mass units
convpres	Converts from pressure units to the desired pressure units
convtemp	Converts from temperature units to the desired temperature units
convvel	Converts from velocity units to the desired velocity units

For example, the `airspeed` function from the Flight Parameters group is as simple as

```
function as = airspeed(vel)
%  AIRSPEED Compute airspeed from velocity.
%   AS=AIRSPEED(V) computes M airspeeds, AS, from an M-by-3 array
%                  of velocities, V.
% ...
if ~isnumeric(vel)
    error('aero:airspeed:notnumeric',...
          'Velocity input was not a numeric value.');
end
if (size(vel,2)==3)
    as = sqrt(vel(:,1).^2+vel(:,2).^2+vel(:,3).^2);
else
    error('aero:airspeed:wrongdim','Velocity array is not M-by-3.');
end
```

It computes the norm of a three-element vector, in this case, a velocity vector. This operation could be programmed in a single line; however, this function contains several foolproof elements, which makes it more robust. It warns you when you try to pass in a non-numeric value (the `isnumeric` function was presented in Table 5.3). It also notifies you if a velocity vector has fewer or more than three elements (three columns, to be more precise). The `error` function displays a message supplemented by an optional message identifier, enabling you to better identify the source of an error, and aborts the function.

As seen, the key mathematical expression is vectorized, which allows you to process an *M*-by-3 array of velocities as opposed to just one 1-by-3 vector. Following the header line, there is an extended comment section (only part of it is shown for compactness), explaining the use of this function and showing up in response to the help call

```
>> help airspeed
```

To find the location of this Aerospace Toolbox function and open it in the Editor, you may use the following call:

```
>> open(which('airspeed'))
```

Consider one more example, the `atmoslapse` function of the Environment group, which computes parameters of atmosphere using a lapse rate for ambient temperature:

```
function [T,a,P,rho]=atmoslapse(h,g,gamma,R,L,hts,htp,rho0,P0,T0)
%  ATMOSLAPSE Use Lapse Rate Atmosphere Model.
%   [T,A,P,RHO]=ATMOSLAPSE(H,G,GAMMA,R,L,HTS,HTP,RHO0,P0,T0) implements
%   the mathematical representation of the lapse rate atmospheric
%   equations for ambient temperature, pressure, density, and speed of
```

```
%   sound for the input geopotential altitude.
%   ...
%   Limitation:
%   Below the geopotential altitude of 0 km and above the geopotential
%   altitude of the tropopause, temperature and pressure values are held.
%   Density and speed of sound are calculated using a perfect gas
%   relationship.
%   Example:
%   Calculate the atmosphere at 1000 meters with the International
%   Standard Atmosphere input values:
%      [T,a,P,rho]=atmoslapse(1000,9.80665,1.4,287.0531,0.0065,...
%                             11000,20000,1.225,101325,288.15);
%   ...
error(nargchk(10,10,nargin,'struct'));
  if~(isscalar(g) && isscalar(gamma) && isscalar(R) &&...
       isscalar(L) && isscalar(hts) && isscalar(htp) &&...
       isscalar(rho0) && isscalar(P0) && isscalar(T0))
       error( 'aero:atmoslapse:nonscalar',...
              'All inputs other than altitude must be scalars.');
  end
if~isnumeric(h)
    error('aero:atmoslapse:notnumeric',...
          'Height input was not a numeric value.');
end
  for i = length(h):-1:1
    if (h(i)>htp)
        error('aero:atmoslapse:gthtp',...
              'Height cannot exceed the tropopause altitude.');
    end
    if (h(i)<0)
        error('aero:atmoslapse:ltzero','Height cannot be below zero.');
    end
    if (h(i)>hts)
        T(i)= T0 - L*hts;
        expon(i)= exp(g/(R*T(i))*(hts - h(i)));
    else
        T(i)= T0 - L*h(i);
        expon(i)= 1.0;
    end
end
a = sqrt(T*gamma*R);
theta = T/T0;
P = P0*theta.^(g/(L*R)).*expon;
rho = rho0*theta.^((g/(L*R))-1.0).*expon;
```

(By the way, you may compare the output of this function with that of your own, developed in Problem 5.3.)

As was the case for the `airspeed` function, the `atmoslapse` function has an extended help comments section, while actual computations are carried out in the last four lines of code. The remaining lines represent the multiple guards from possible erroneous inputs. The `nargchk` function, used in the beginning of the function, has the general syntax

```
nargchk(low,high,N,'struct')
```

and returns an appropriate error message structure if `N` is not between `low` and `high`. Here, it is used with the `error` function which accepts this error message structure as an input. Note the use of short-circuit logical AND, discussed in Sec. 5.2. If any of the `isscalar` returns is `false`, there is no need to continue with the remaining ones.

From the standpoint of programming, there is no problem using `nargchk` here; however, it would be good if the developer would allow multiple syntaxes. For example, it would be natural to allow a single input argument, making other parameters assume some default values, or two input parameters, in which case the second input could be a structure having nine fields and carrying nine required scalars.

E.3 Aerospace Blockset

The Aerospace Blockset features a dozen top-level libraries, as shown in Fig. E.1. They may have as little as just one block in them (for example, the Propulsion library features only the Turbofan Engine System block). However, most top-level libraries contain either multiple blocks or even several groups of sublibraries (as shown in Fig. E.1) with multiple blocks in

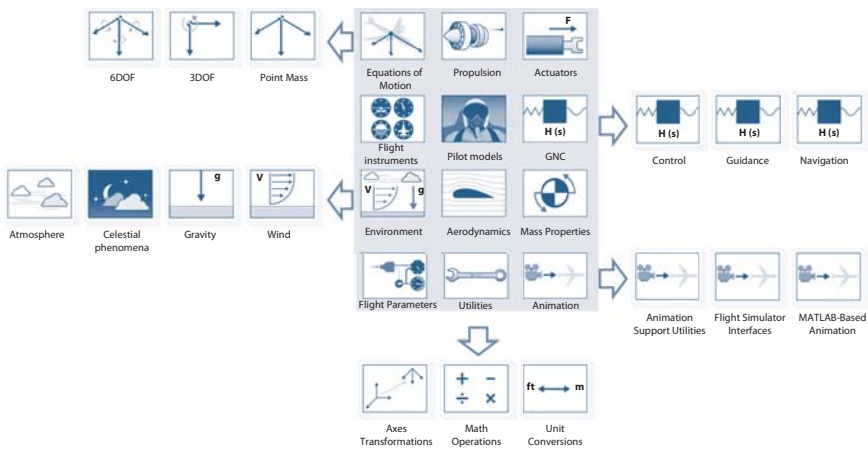


Fig. E.1 Aerospace Blockset libraries.

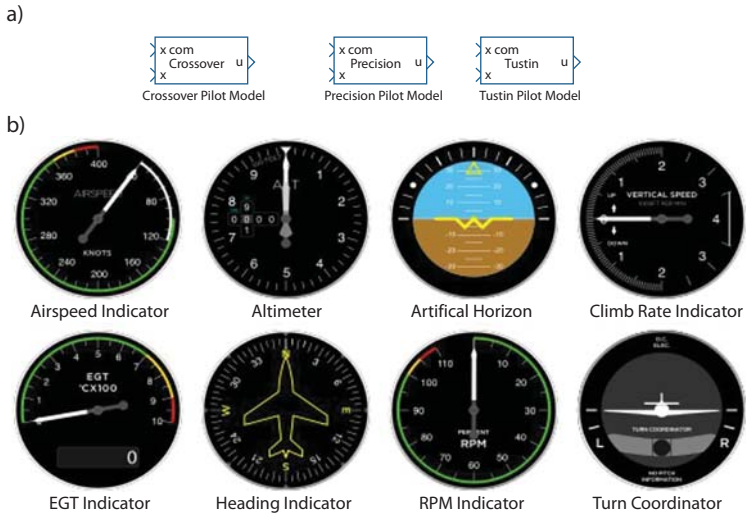


Fig. E.2 a) Pilot Models and b) Flight Instruments library blocks.

the sublibraries. The content of all top-level libraries and sublibraries is shown in Figs. E.2–E.14.

If you drag any block into your model and double-click it, the Block Parameters window will pop-up, as was the case with any basic Simulink blocks (Figs. 15.14, 15.15, 15.26a, etc.). For example, the Three-axis Accelerometer block of the Guidance, Navigation and Control (GNC) library, shown in Fig. E.6, features two tabbed panes where the user can define different parameters (Fig. E.15). (By the way, note how the Noise seeds are defined by default in the Noise pane of Fig. E.15b—they differ from one another by 1. This issue was discussed in Fig. 15.76.)

The Block Parameters window can also be opened by right-clicking the block (opening a context menu shown in Fig. E.16a), clicking **Mask**, and choosing the **Mask Parameters** option from the Mask menu (Fig. E.16b). Note that instead of "block," Simulink refers to "mask." The reason for this is that every block of the Aerospace Blockset represents a subsystem like the ones we created in Secs. 15.3.2 and 15.3.3 (Figs. 15.34b and 15.36). The only difference is that the developers made one additional step—they masked the content of the subsystem.

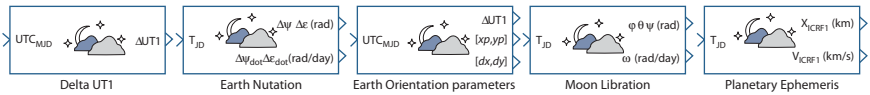


Fig. E.3 Celestial Phenomena library blocks.

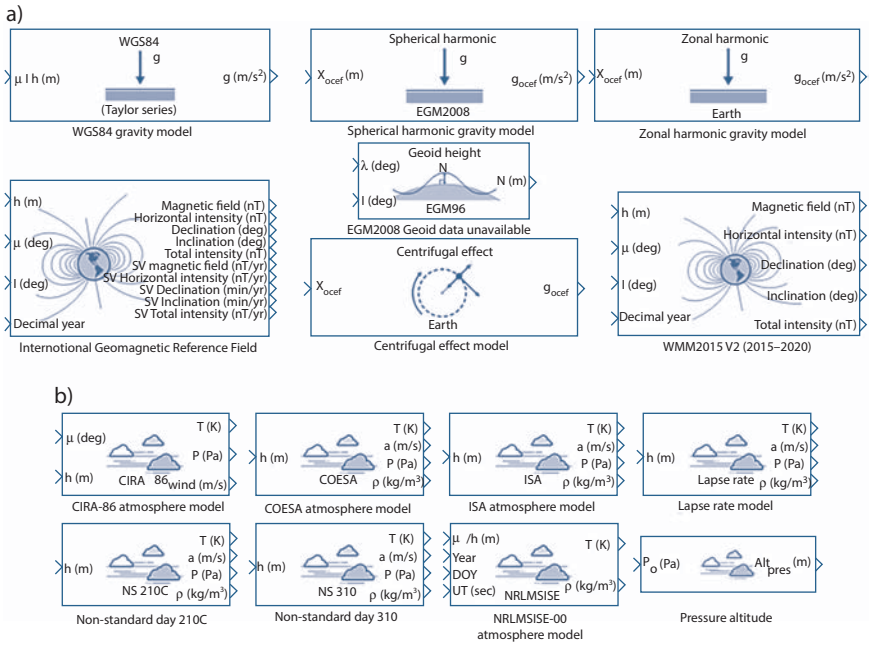


Fig. E.4 a) Gravity and b) Atmosphere library blocks.

E.4 Block Masking

There are several reasons for masking a subsystem. Masking

- Prevents unintended modification by hiding subsystem's content behind a mask
- Enables creation of a single block parameter dialog with its own description, parameter prompts, and help text
- Makes it possible to replace a subsystem's standard icon with a customized icon that depicts its purpose
- Creates a single custom (super) block that can be placed in a library

To see what resides under the mask, you should choose the **Look Under Mask** option of the Mask menu of Fig. E.16b, which in the case of the Three-axis Accelerometer block, opens the model shown in Fig. E.17.

This model uses three Constant blocks that refer to variables `acc`, `a_sf_cc`, and `a_bias`. The Dynamics subsystem block uses `w_a`, `z_a`, and `dtype_a`. The Saturation block uses the parameter `a_sat`. The noise power and seeds in the Random bias block are defined as `a_pow` and `a_seed`. These are parameters that are defined in two panes of the Block Properties window shown in Fig. E.15. To see where/how the values for these parameters are defined, you should choose the **View Base Mask** option from the drop-down menu of Fig. E.16b, which brings up the Mask

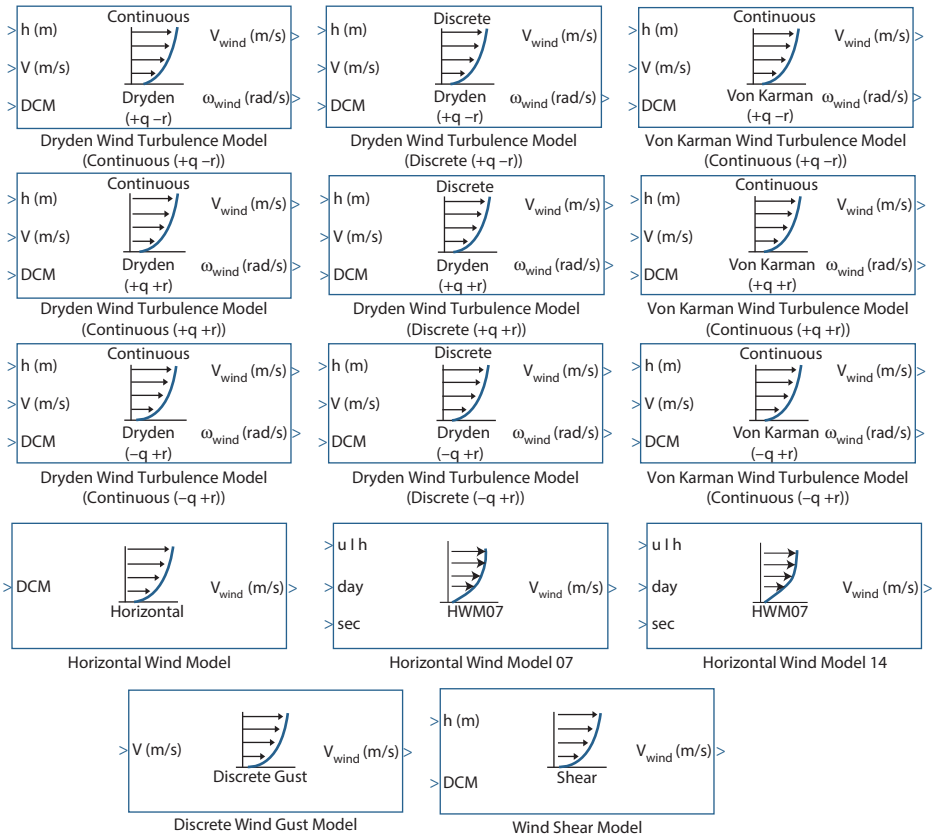


Fig. E.5 Winds library blocks.

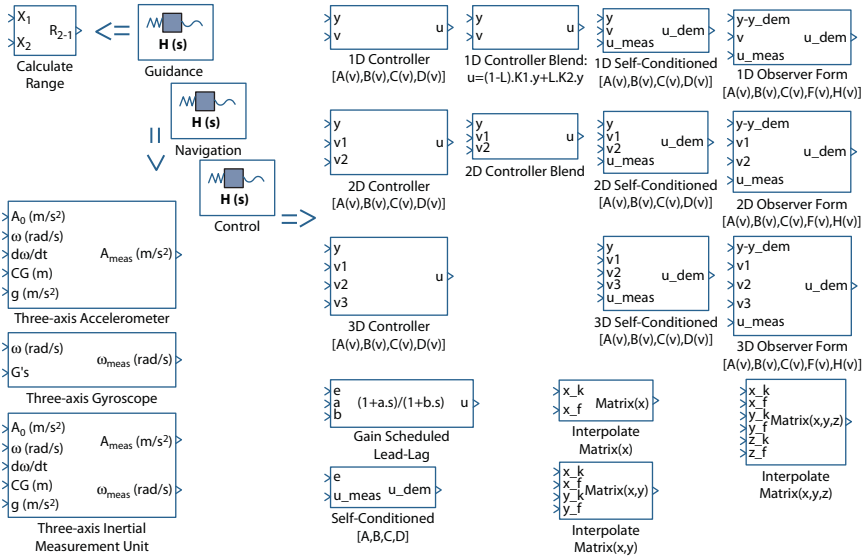


Fig. E.6 Guidance, Navigation and Control library blocks.

Editor window, shown in Fig. E.18. Choosing the second tab, Parameters & Dialog, opens another pane (Fig. E.19), which defines the names of the tabs and fields in the Block Parameters window of Fig. E.15, their types (editable window, check box, or pop-up menu), and the names of parameters. The values entered in the two panes (Fig. E.15) are stored in the mask workspace (each mask has its own workspace) from which the model reads them. A block parameter can refer to variables defined in the mask workspaces of subsystem or nested subsystems that contain the block, or base (MATLAB)

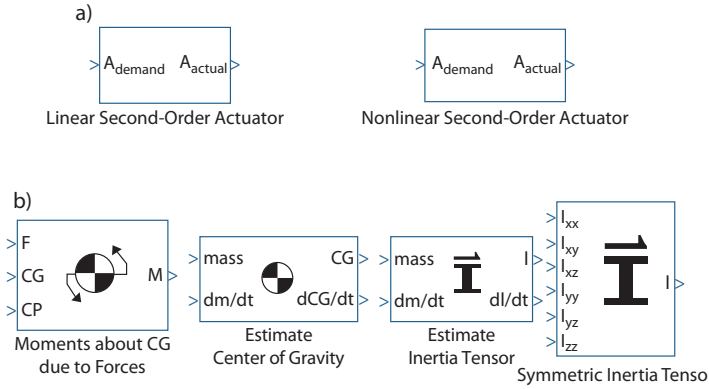


Fig. E.7 a) Actuators and b) Mass Properties library blocks.

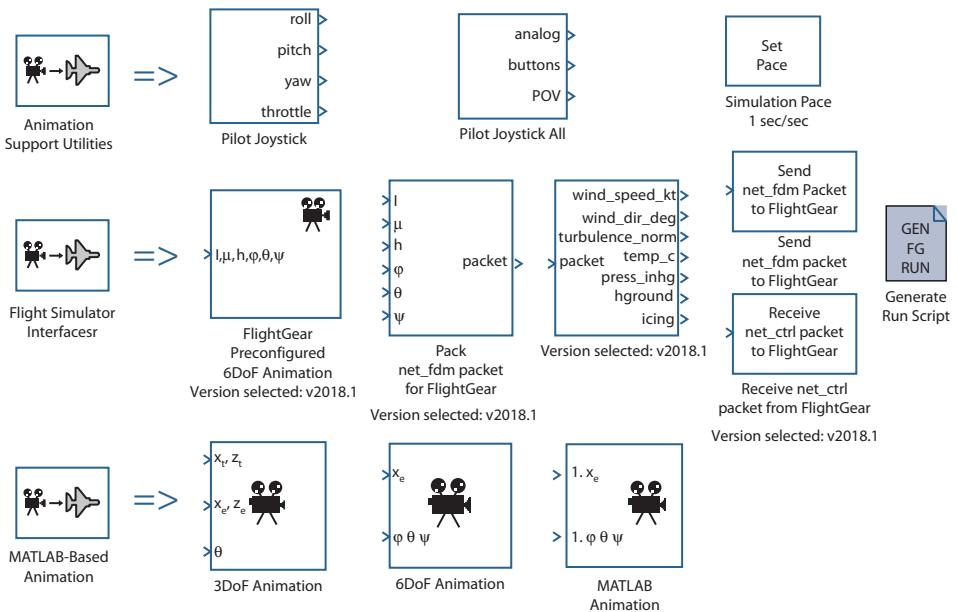


Fig. E.8 Animation library blocks.

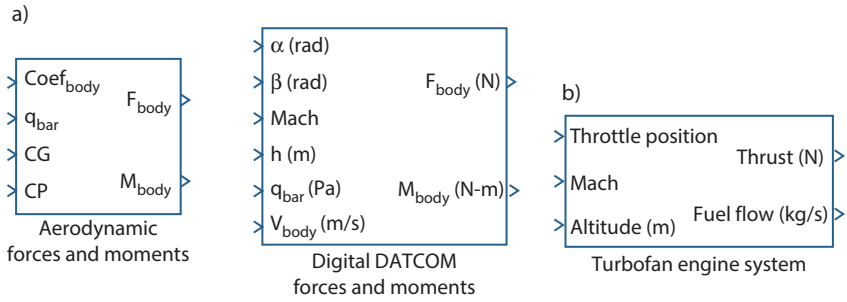


Fig. E.9 a) Aerodynamics and b) Propulsion library blocks.

workspace. If a variable is defined at more than one level in the model hierarchy, the most local definition is used.

The name of the block, its description, and the prompt, appearing when the Help button is pushed, are defined in the Documentation pane of the Mask Editor (Fig. E.20a). The Initialization pane of the Mask Editor is used to accommodate MATLAB code, initializing the model when it loads and starts. This code can refer only to variables in its local workspace. For example, the key initialization command shown in Fig. E.20b is the third one. It calls the *airoicon.m* function that defines the `ports` structure, used to represent the input and output names for a specific block. You can find this function and look at it by issuing the following instruction:

```
>> open(which('aeroicon'))
```

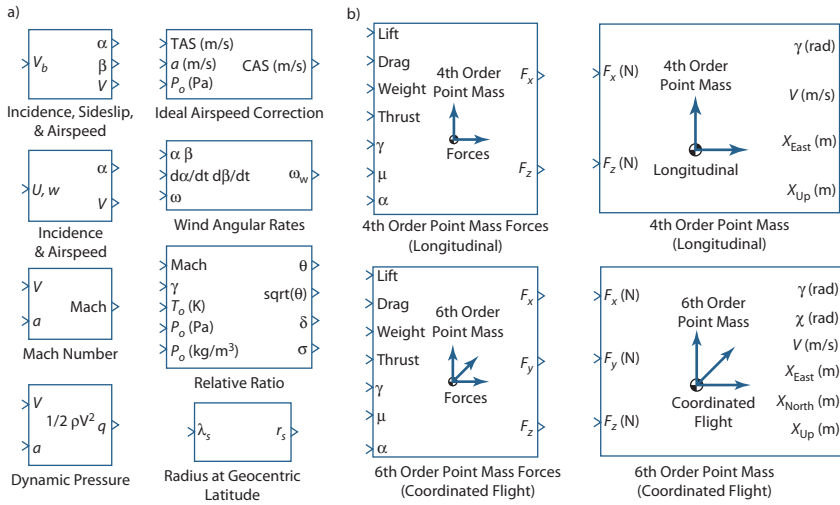


Fig. E.10 a) Flight Parameters and b) Point Mass library blocks.

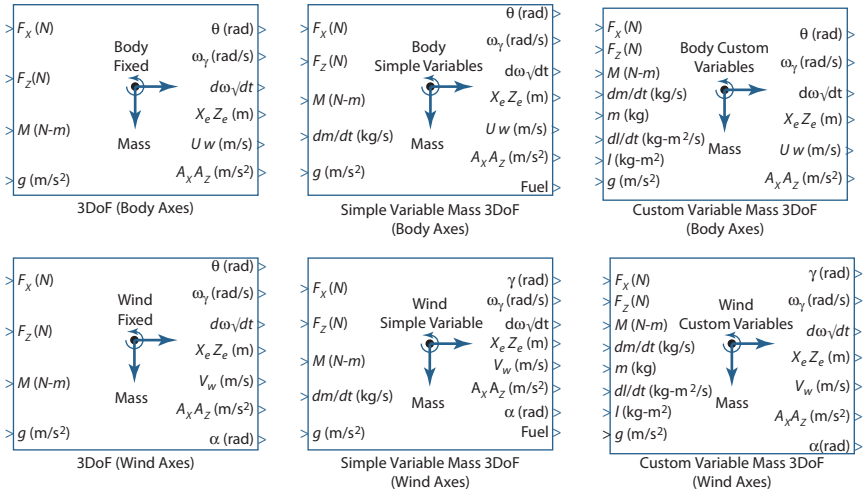


Fig. E.11 3DoF library blocks.

Specifically, for the Three-axis Accelerometer block shown in Fig. E.6, the drawing commands of Fig. E.18 rely on the `txt` field of the `port` structure to label all six ports using TeX language (to produce Greek letter ω, superscripts, and subscripts). The `port_label` is not the only drawing function you can use in the Icon & Ports pane of Fig. E.18. Other options are shown in Table E.9. The drawing functions have access to all variables in the mask workspace, so your icon can be dynamic and reflect the current values of block parameters (see an example for the `fprintf` command in Table E.9).

The Icon options—shown on the left of the Icon & Ports pane of Fig. E.18 and defined within the Parameters & Dialog pane of Fig. E.19—allow you to set up Icon properties that control its appearance. Making the block frame **Invisible** (as opposed to default **Visible**) hides the rectangle that encloses the block. Choosing **Transparent** (as opposed to default **Opaque**) allows you to see underneath the icon. The units' options are **Autoscale**, **Pixels**, and **Normalized** (default). The **Rotates** option (as opposed to default **Fixed**) of the rotation property enables icon rotation and flipping when the block is rotated or flipped.

You may access all parameters of the current block from the Command Window of MATLAB using the `get_param` function. For instance, if you have the Three-axis Accelerometer block of your model selected and issue the command

```
>> get_param(gcb,'DialogParameters')
```

you will get a structure array with the names of the dialog parameters of the specified block as the structure fields

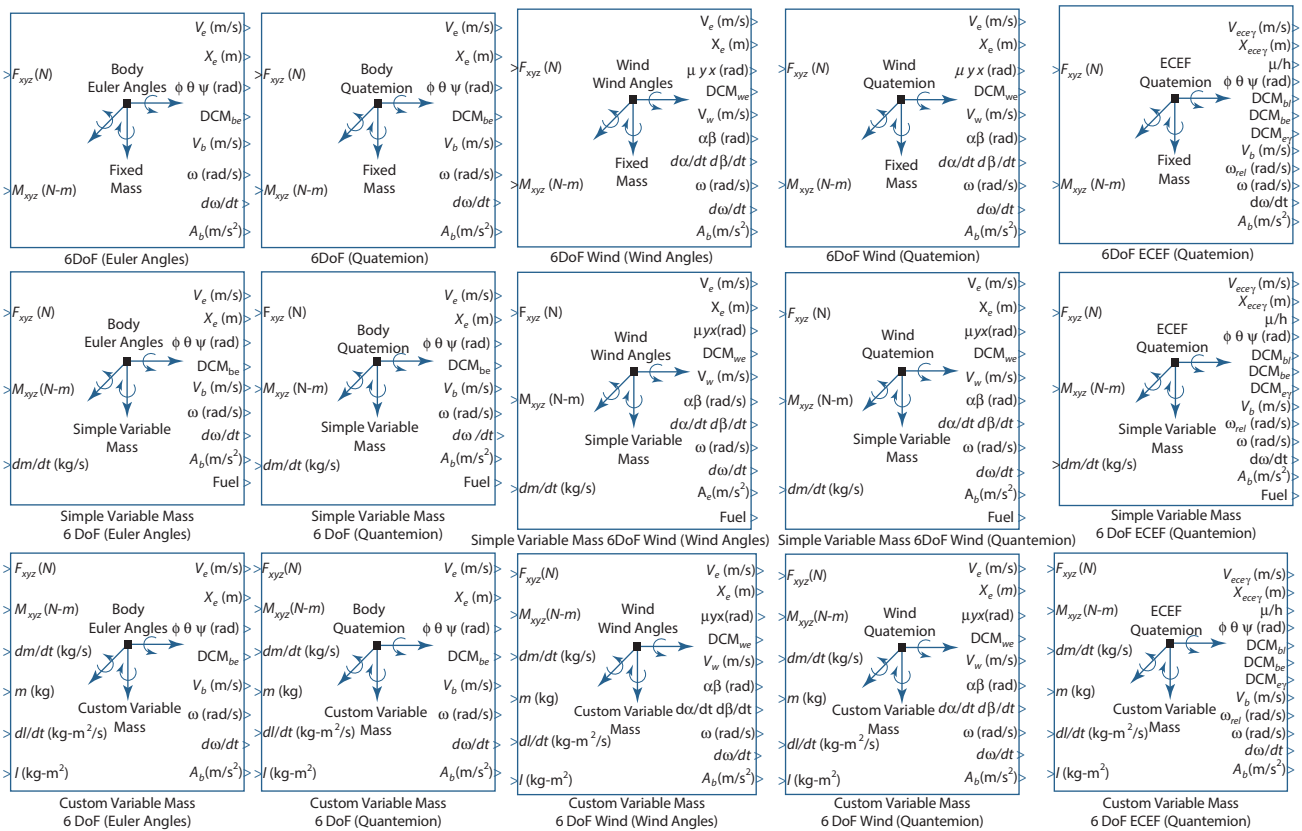


Fig. E.12 6DoF library blocks.

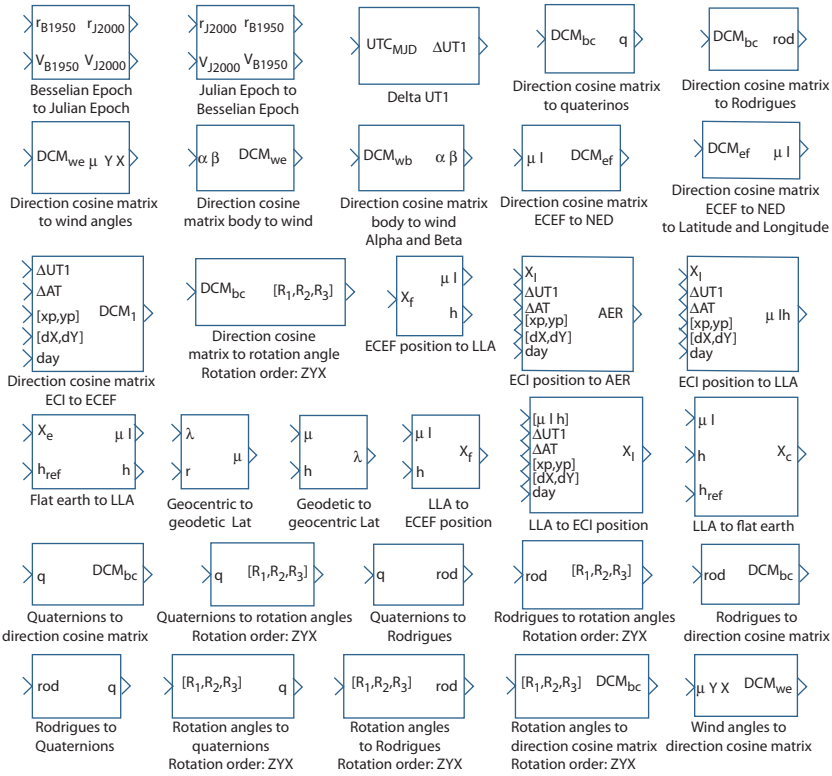


Fig. E.13 Axes Transformations library blocks.

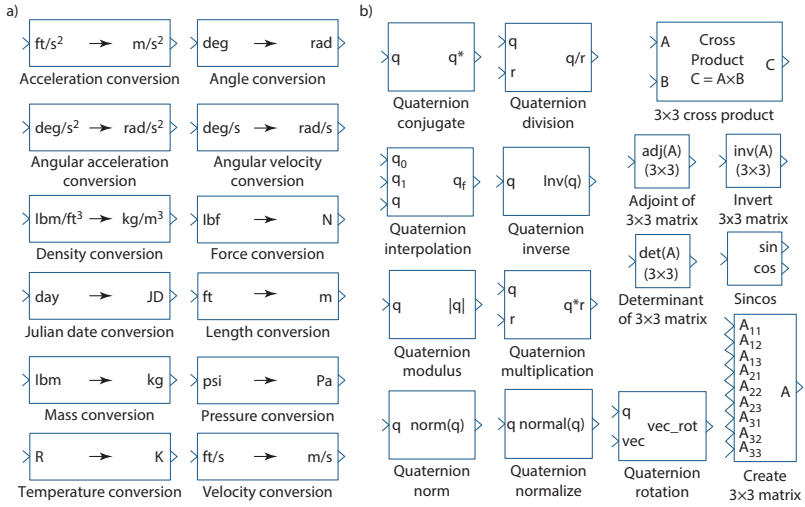
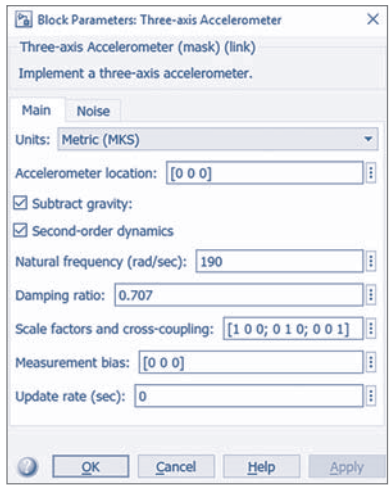


Fig. E.14 a) Unit Conversions and b) Math Operations library blocks.

a)



b)

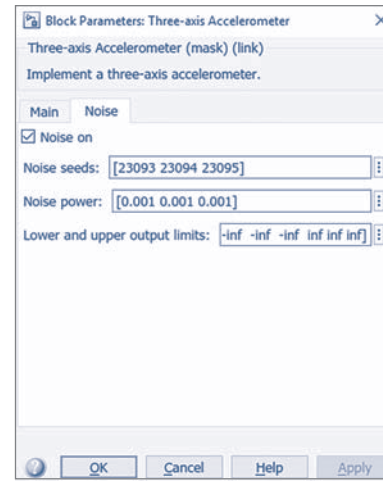
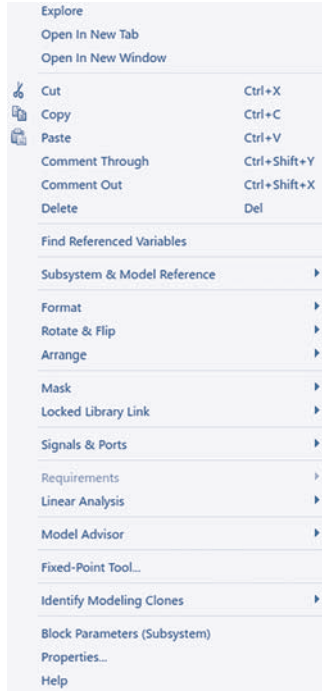


Fig. E.15 Example of the tabbed panes of the masked block.

a)



b)

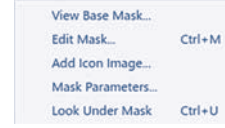


Fig. E.16 a) Right-click options menu and b) Mask submenu.

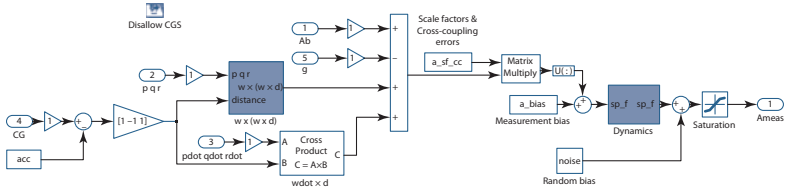


Fig. E.17 Three-axis Accelerometer.

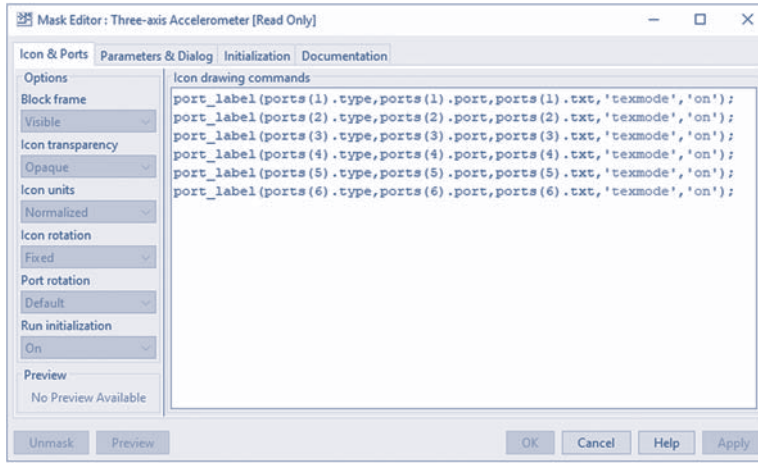


Fig. E.18 Icon & Ports pane of the Mask Editor.

```
ans =
      units: [1×1 struct]
        acc: [1×1 struct]
      gtype: [1×1 struct]
    dtype_a: [1×1 struct]
        w_a: [1×1 struct]
        z_a: [1×1 struct]
    a_sf_cc: [1×1 struct]
     a_bias: [1×1 struct]
       a_Ts: [1×1 struct]
     a_rand: [1×1 struct]
    a_seeds: [1×1 struct]
      a_pow: [1×1 struct]
      a_sat: [1×1 struct]
```

(you will not be able to access the values of variables from the base MATLAB workspace though). The `gcb` returns the full pathname of a current block in the current system (the `get(gcbh)` call will query selected block properties via its handle, `gcbh`).

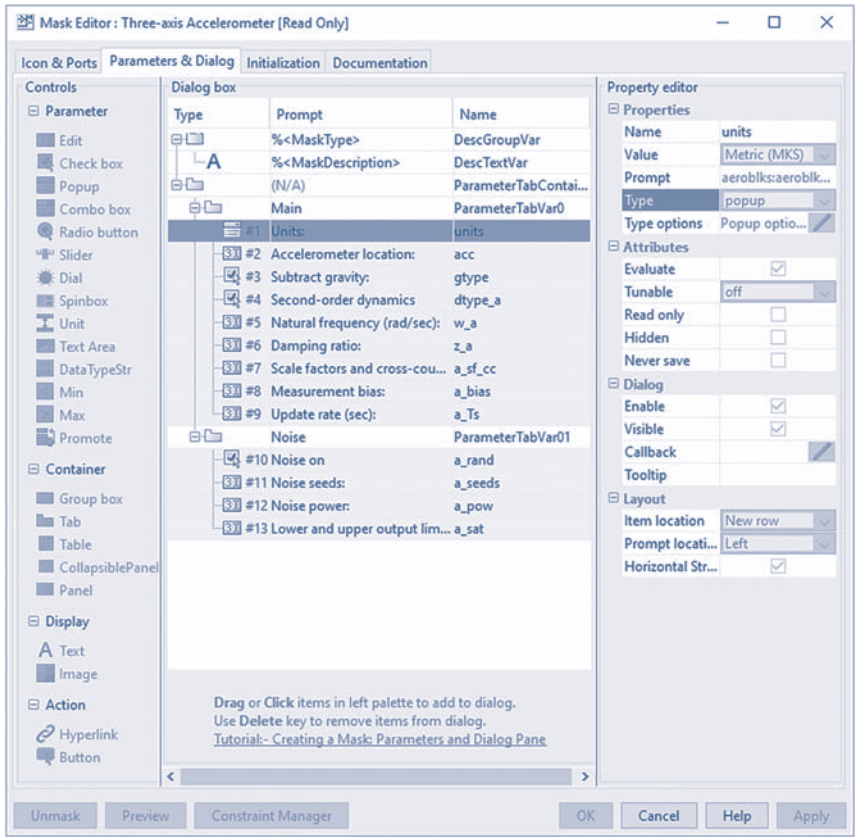


Fig. E.19 Parameters & Dialog pane of the Mask Editor.

The `get_param` function is helpful in executing the initialization commands of Fig. E.20b, and it may also be used in the Callback window of the Parameters & Dialog pane (Fig. E.19). For example, having the following set of commands entered in this window:

```
if str2num(get_param(gcb,'z_a'))<0
   error('Damping Ratio must be within [0;1]')
end
```

will bring the error message if you attempt to change the damping ratio to some negative number (that is, when you push **Apply** and **OK** buttons in the Block Parameters panes of Fig. E.15).

Now that you know how the Mask Editor works, you can go back to the subsystem created in Sec. 15.3.2, right-click it, and choose the **Mask Subsystem** option (alternatively, you can choose this option from the **Edit** menu or use <Ctrl>+<M>). This brings up an empty Mask Editor window you would use to convert the second-order system you created back there

a)

b)

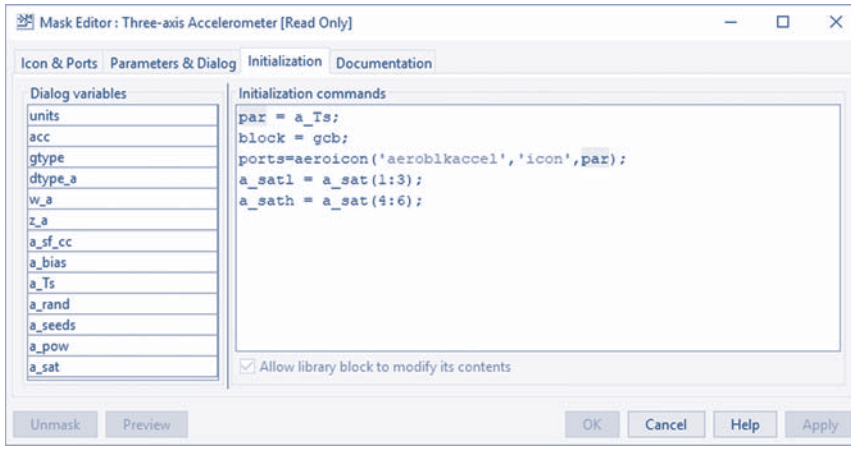


Fig. E.20 a) Documentation and b) Initialization panes of the Mask Editor.

(Fig. 15.34a) into a professional-looking masked block (you will not need to define variables in the InitFcn callback of Fig. 15.66b then).

After you created a mask, you can always turn it off by pushing the **Unmask** button of the Mask Editor (located at the lower-left corner of the window) or even from the MATLAB Command Window (having your block selected) with

```
>> set_param(gcb,'Mask','off')
```

Note, however, that you cannot unmask a Simulink library subsystem block this way (which is why the **Unmask** button of Mask Editor in Figs. E.18–E.20 is disabled). That is because when you add a masked library block or subsystem block from any Simulink blockset (including the Aerospace Blockset) to your model, a referenced instance of the library block is

Table E.9 Mask Editor Drawing Functions

Function	Use	Example
`port_label`	Labels a specific port	`port_label('output',...` `1,'Output 1')`
`disp`	Displays text centered on mask icon	`disp('MK611-8 Engine')`
`text`	Shows text at a specific location	`text(5,10,'F-22 Model')`
`fprintf`	Prints (variable) formatted text	`fprintf(['Damping '...` `'Ratio is %5.3f'],z_a)`
`plot`	Plots 2-D curves	`plot([5 0 5 10 5],...` `[0 5 2 5 0],'r')`
`patch`	Draws filled shapes (see example in Fig. 6.37b)	`patch([5 0 5 10 5],...` `[0 5 10 5 0],'y')`
`image`	Shows a picture on the block (To add a mask icon image from the user interface, you can also click **Mask**>**Add Icon Image** in the menus of Fig. E.16)	`image(imread ...` `('Cessna172.jpg'))`
`dpoly`	Displays a transfer function based on numerator and denominator polynomials (see example in the Transfer Fcn block of Fig. 15.35a)	`dpoly([0 0 1],[1 2 1])`
`droots`	Displays a transfer function based on zero-pole-gain representation (allowing you to change the default independent variable name)	`droots([],[-2 -3],5,'z')`

`color`	Changes color of the subsequent mask icon drawing command	`color('b');`
		`port_label('output',...`
		`1,'X_b')`

created. Such referenced instance of a library block is called a linked block, meaning that it contains a link or path to the parent library block (try the **Locked Library Link** option in the menu of Fig. E.16a). This allows the linked block to update when the library block is updated.

E.5 Examples of Using the Aerospace Toolbox and Blockset

As usual, the MATLAB Help system provides plenty of representative examples of how to use the Aerospace Toolbox and Aerospace Blockset. Figures E.21 and E.22 depict some developed examples showing how to conduct a variety of aerospace-related calculations, perform coordinate

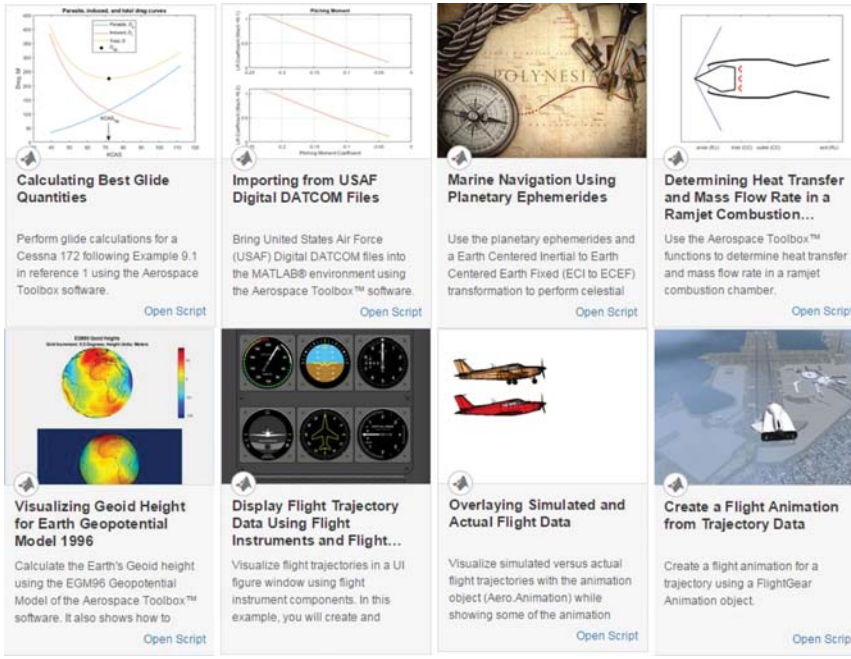


Fig. E.21 Selected Aerospace Toolbox examples.

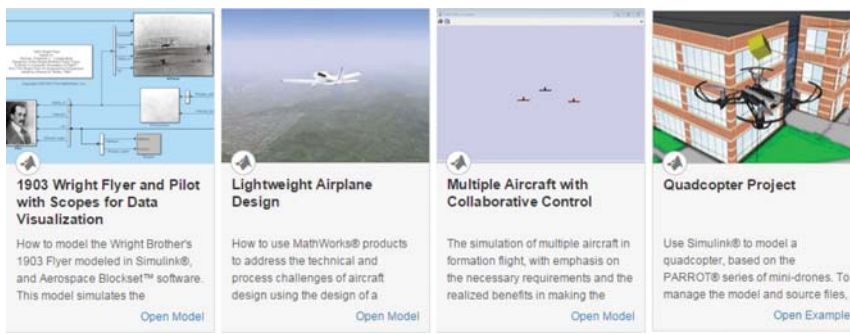


Fig. E.22 Selected Aerospace Blockset examples.

transformations, address gas dynamics problems, model planetary environment and aerospace vehicles, develop controllers, and visualize and animate flight instruments, altitude, and trajectory.

Appendix F Building and Sharing GUIs and Apps

F.1 Introduction

Once you have developed code that allows some variation, and is intended to be used repeatedly, and/or shared with your colleagues, you might consider enhancing it with a GUI (which, among other things, keeps you from having to browse through code to find parameters that need to be changed) and converting it to a sharable app. Until recently, MATLAB has offered two ways of creating GUIs, enabling *event-driven* programming. As shown in Fig. F.1, they were: a) creating GUIs programmatically, using the corresponding MATLAB functions explicitly, and b) automating the process with the help of a GUI development environment (GUIDE). These GUIs could then be converted to standalone executables. As of R2016a, an advanced GUI development environment, *App Designer*, became a primary tool for a GUI-driven app design (eventually, GUIDE will be phased out). GUIDE and App Designer are similar in the way that they provide a user interface to lay out a GUI and automate programing its components; however, App Designer ties GUI layout design and programming its components together, resulting in a more user-friendly tool. It also offers an easier interface for converting developed GUIs into the sharable products. (The darker background in Fig. F.1 encompasses a variety

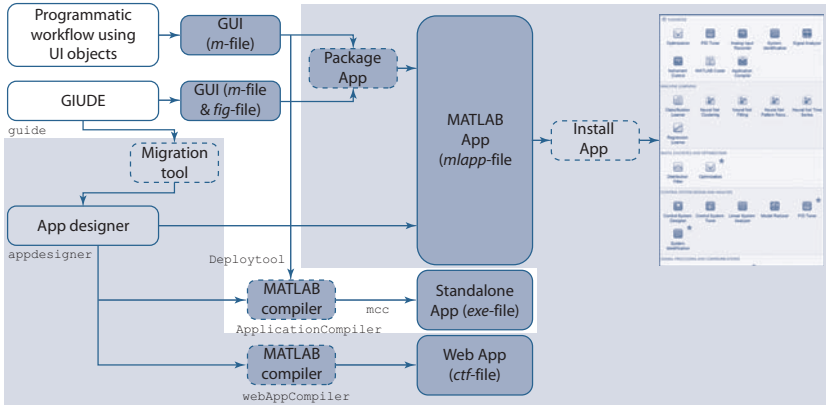


Fig. F.1 Workflow for creating GUIs and apps in MATLAB.

of recently added tools.) Each of the three aforementioned approaches offers the different workflow and slightly different set of functionalities, so the best choice for you depends on your preferences and desired level of sophistication. In what follows, we start with the first approach, which is based on using the UI graphics objects to enhance your program, allowing interaction with it by pushing buttons and using menus and sliders of your custom-designed GUI. Then, the second approach, which is based on using GUIDE, is discussed in detail. The discussion starts with an example of developing a simple GUI, followed by examining two simple GUIs of MATLAB, and then showing a couple of practical GUIs introduced as an example of a complex interface involving numerous UI objects. Next, we show how to convert your GUIDE-designed interactive GUI into a self-contained package (that can be added to the MATLAB Apps Gallery) or even to a standalone executable file (see Fig. F.1). Finally, we show how to migrate your GUIDE design to the App Designer environment (starting in R2018a).

F.2 UI Graphics Objects

Some UO graphics objects (buttons) have briefly been introduced in Sec. 6.12 (Fig. 6.63) already. Let us provide you with one more example showing a variety of these objects and the way to interact with them in your code. Suppose you need to run through a sequence of images and do some image processing. You want to be able to start processing, pause it if you find something you are interested in, go back and forth between the images, and start bulk processing again. MATLAB allows you to add such control capabilities to the figure objects quite easily. For example, Fig. F.2 features five control buttons that were created using the following commands:

```
hstart=uicontrol('Style','pushbutton','String','Start','Position',...
  [470 230 50 24],'Value',1);
hpause=uicontrol('Style','pushbutton','String','Pause','Position',...
  [470 200 50 24],'Value',1);
```

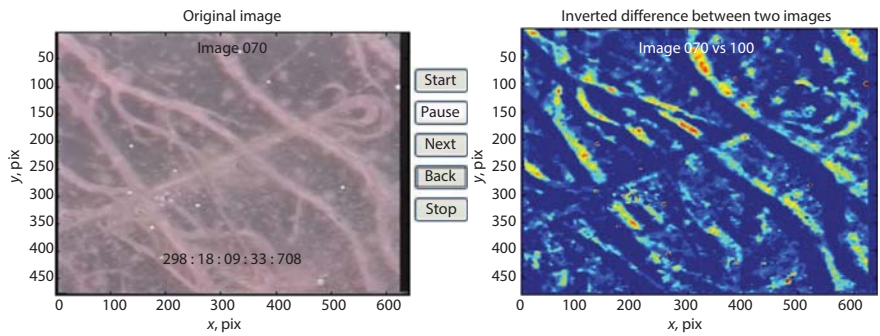


Fig. F.2 Example of adding UI control buttons to a figure object.

```
hnext=uicontrol('Style','pushbutton','String','Next','Position',...
  [470 170 50 24],'Value',1);
hback=uicontrol('Style','pushbutton','String','Back','Position',...
  [470 140 50 24],'Value',1);
hstop=uicontrol('Style','pushbutton','String','Stop','Position',...
  [470 110 50 24],'Value',1);
```

The `uicontrol` function used in these calls creates UI objects, which you use to implement GUIs. Its syntax is as simple as

```
handle=uicontrol('PropertyName',PropertyValue,...)
```

and in the aforementioned example, we defined the style, text string, $x-y$ coordinates of the lower-left corner of a button, its width and height, and finally a value (where `1` stands for a button not being pushed). By default, the four-element position vector is defined in pixels.

Beyond the push buttons, MATLAB supports other styles of UI objects (introduced in Sec. F.3) along with the `uipanel` function, assisting in grouping different components, and the `uibuttongroup` function, enabling grouping of the radio and toggle buttons with exclusive selection behavior.

As you may recall from Sec. 6.4, now that you have handles to the buttons created with the aforementioned calls, you can use them to access and change any of their properties. In the specific example of Fig. F.2, somewhere in the *m*-file code the `get` function is used to capture the state of the button. When pushed, its `'Value'` attribute changes from `1` to `0`, which signals that some predefined action should take place. The following piece of code shows an example of how the states of these five buttons are captured inside the `while` loop and what actions, in terms of the number of the image that needs to be processed, are taken:

```
%% Wait for a Start button to be pushed (for a minute)
for i=1:1000
valstart = get(hstart,'Value');
if valstart==0, break, end      % Sense the button being clicked on

pause(0.06)
end
set(hstart,'Value',1);          % Set the value back to 1

%% Process the images
while ImNum <= ImNumMax
...
%% Check the status of the push buttons
valstart = get(hstart,'Value'); valpause = get(hpause,'Value');
valnext  = get(hnext, 'Value'); valback  = get(hback, 'Value');
valstop  = get(hstop, 'Value');
  if valstop==0, break
  elseif valnext==0,
         ImNum=ImNum+1;
```

```
        set(hnext, 'Value',1); set(hpause,'Value',0);
        set(hstart,'Value',1);
  elseif valback==0
        ImNum=ImNum-1;
        set(hback, 'Value',1); set(hpause,'Value',0);
        set(hstart,'Value',1);
  elseif valpause==0 & valstart==1
        set(hnext, 'Value',1); set(hback, 'Value',1);
        set(hstart,'Value',1);
  else
        ImNum=ImNum+1;
        set(hpause,'Value',1); set(hstart,'Value',1);
  end
pause(0.05)
end
```

Using the `uicontrol` function allows you to enhance your interactive capabilities even beyond those presented in Sec. 4.10. GUIDE and App Designer elaborate on them and bring the development of interactive GUIs to the new level where all you have to do is to create a UI graphics object, whereas the code skeleton beyond it is generated automatically. After this, you simply proceed with programming different constructs like the ones considered earlier to specify what actions need to be taken when a specific UI object is activated.

For completeness, the following code shows an example of how the images shown in Fig. F.2 could be produced (this code would appear after the `while` command in the preceding piece of code instead of an ellipsis):

```
A=imread(['Frame' num2str(ImNum,'%03.0f\n')],'jpg');
B=imread(['Frame' num2str(ImNum-30,'%03.0f\n')],'jpg');
subplot(1,2,2)
image(A)
xlim([1 640]),ylim([1 480])
axis tight, axis equal
subplot(1,2,2)
image(A(:,:,2)-B(:,:,2)) % use the G channel of the RGB image
xlim([1 640]),ylim([1 480])
axis tight, axis equal
```

As mentioned in Sec. 4.8.3 (Table 4.16), the `imread` function with the general syntax

```
A=imread(filename,format)
```

reads a variety of different-type grayscale or color images into the data array `A`. This can be a M-by-N array for a grayscale image, an M-by-N-by-3 array for a true-color (256^3-color) image (as explained in Sec. 6.9), or an M-by-N-by-4 array for the *tiff* files, which use a cyan-magenta-yellow-black palette as opposed to the RGB palette. The class of `A` may also vary depending

on the bits-per-sample of the image data (for example, "logical" for a 1-bit grayscale image or "unit8" for 24-bit color). In the preceding example, the `imread` function reads 24-bit *jpeg* images into "unit8" 480-by-640-by-3 arrays.

The `image(A)` call displays matrix `A` as an image by interpreting each of *M*-by-*N* elements as an RGB value or index into the figure's color map (depending on the dimension and class of matrix `A`). In the preceding example, the first call treats a 3-D matrix `A` as a matrix of the RGB values, and the second call interprets the 2-D matrix as a matrix of indices.

F.3 GUIDE Overview

Compared with the aforementioned approach of designing GUIs programmatically, GUIDE allows you to lay out a GUI more easily, by clicking and dragging GUI components—such as panels, buttons, text fields, sliders, and menus—into the GUIDE layout area. GUIDE stores a GUI layout in a *fig*-file and automatically generates an *m*-file that controls how this GUI operates. The *m*-file initializes the GUI and contains a framework for the most commonly used callbacks for each component—the commands that execute when a user clicks a GUI component. Using the *m*-file Editor, you can then add code to callbacks to perform the actions you want.

To bring this tool up (Fig. F.3a) you should type `guide` in the Command Window. To start creating a new GUI, you should choose the **Blank GUI** option, which opens a *Layout Editor* window with a blank GUI template allowing you to work on your GUI layout (Fig. F.3b). (Note that starting R2019a, there is a prompt showing up at the bottom of a new GUI template suggesting to use the App Designer as a more advanced tool.) You create a GUI by dragging buttons from the Component Palette on the left of Layout Editor to the layout area (or clicking a button once and then clicking

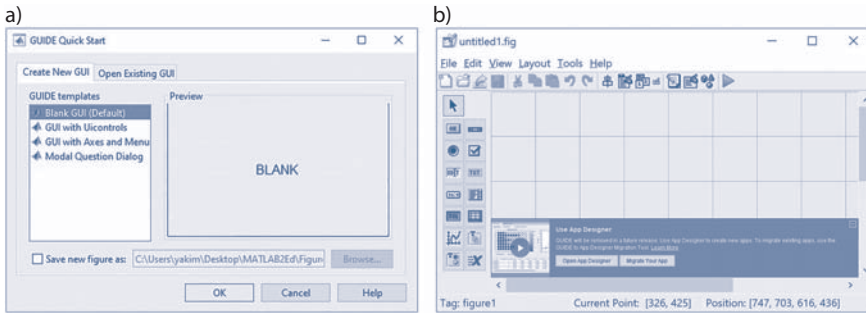


Fig. F.3 a) GUIDE Quick Start window and b) Layout Editor with a blank GUI template.

Table F.1 UI Options Available in GUIDE

Button	Description
	Push button, generating an action when pressed
	Radio button, providing a user with a number of mutually exclusive choices
	Editable text field, enabling users to enter or modify the text values
	Pop-up (drop-down) menu, displaying a list of choices from which to choose
	Toggle button, executing callbacks when clicked on and indicating their state (`on` or `off`)
	Axes, enabling your GUI to display graphics
	Button group, creating container object to exclusively manage radio buttons and toggle buttons (the `uibuttongroup` function)
	Slider bar, accepting numeric input within a specific range by enabling a user to move a sliding bar
	Check box, providing a user with a number of independent choices
	Static text, displaying lines of text
	List box, displaying a list of items and enabling users to select one or more of them
	Table, allowing you to display and work with table data within your GUI (the `uitable` function)
	Opaque panel, providing a visual enclosure for regions of a figure window with related controls (the `uipanel` function)
	ActiveX Control, enabling your GUI to work with Microsoft ActiveX controls add-ons[a]

[a]We do not recommend pushing this button if you use Microsoft Office 16 or later versions, because Microsoft ActiveX controls became not compatible with GUIDE. It is not supported by the new development environment, App Designer, either, but the latter carries on most of ActiveX controls functionality using MathWorks-developed code.

in the layout area once) and then aligning them and adjusting their properties to meet your preferences. A brief description of a variety of different UI objects and three more user interface tools—`uitable`, `uibuttongroup`, and `uipanel`—is given in Table F.1. In terms of hierarchy of the graphics objects (Fig. 6.10), a UI object is a child of a figure, uipanel, or uibuttongroup and therefore does not require axes to exist when placed in a figure window, uipanel, or uibuttongroup.

At the bottom of the Layout Editor, you may see the tag of the selected component, location of mouse pointer, and position of this selected component (the $x-y$ coordinates of a lower-left corner, width and height). Several unique buttons showing up in the Layout Editor toolbar are listed in Table F.2. These functions are also accessible via the **Tools** menu of the Layout Editor (Fig. F.4a). Specifically, the **Align Objects** option (Fig. F.4b) allows you to arrange the selected graphics objects within your GUI vertically and horizontally (compare with Fig. 6.57b), whereas the Toolbar Editor (Fig. F.4c) is used to create a toolbar for your GUI, similar to that of a

Table F.2 Selected Options of the GUIDE Toolbar

Button	Description
	Align Objects (fig. F.4b)
	Menu Editor (fig. F.16a)
	Tab Order Editor
	Toolbar Editor (fig. F.4c)
	Editor (fig. F.11)
	Property Inspector (fig. F.9)
	Object Browser (fig. F.14)

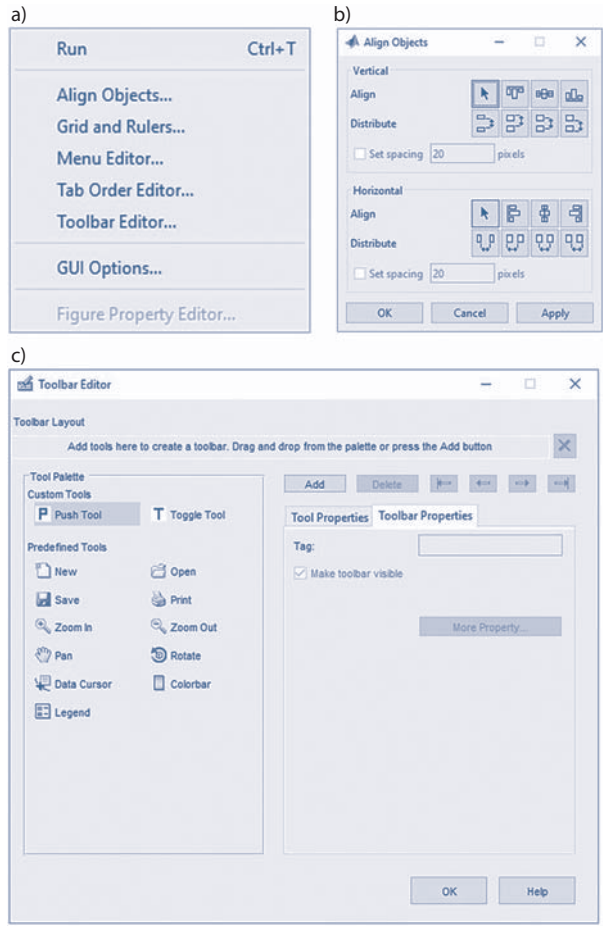


Fig. F.4 a) **Tools** menu of the Layout Editor, b) **Align Objects** icon window, and c) **Toolbar Editor** icon window.

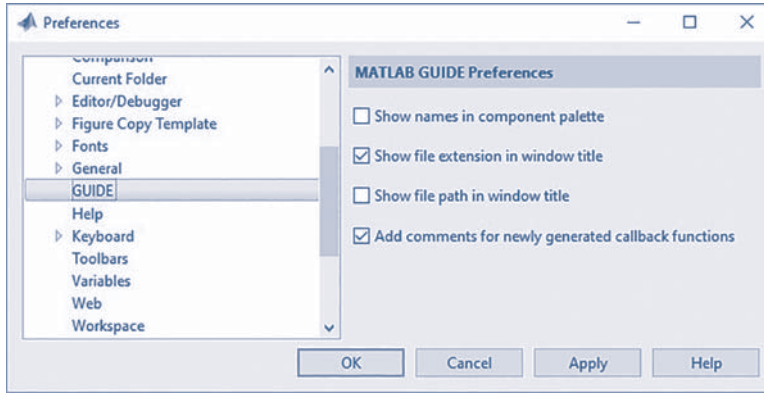


Fig. F.5 GUIDE preferences window.

standard figure window, as in Fig. 6.54a As usual, you may adjust editable properties of GUIDE and the Layout Editor via its Preferences window (Fig. F.5).

As mentioned earlier, the GUIs developed using the GUIDE tool are saved as the *fig*-files. When you save your GUI (or try to run an unsaved GUI by pushing the **Run Figure** button, ▶), the corresponding *m*-file is automatically generated. It bears the same name as the *fig*-file and contains the functions needed to open and run the GUI you have developed. Once you are done with editing your GUI, you can call it by typing its name in the Command Window of MATLAB. The following *m*-file corresponds to the blank GUI shown in Fig. F.3b and contains a primary function, `untitled`, along with two local functions, `untitled_OpeningFcn` and `untitled_OutputFcn`:

```
function varargout = untitled(varargin)
% UNTITLED M-file for untitled.fig
% UNTITLED, by itself, creates a new UNTITLED or raises the existing
% singleton*.
% H=UNTITLED returns the handle to a new UNTITLED or the handle to
% the existing singleton*.
% UNTITLED('CALLBACK',hObject,eventData,handles,...) calls the local
% function named CALLBACK in UNTITLED.M with the given input arguments.
% UNTITLED('Property','Value',...) creates a new UNTITLED or raises the
% existing singleton*. Starting from the left, property value pairs are
% applied to the GUI before untitled_OpeningFcn gets called. An
% unrecognized property name or invalid value makes property application
% stop. All inputs are passed to untitled_OpeningFcn via varargin.
% *See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one
% instance to run (singleton)".
%
```

```
% See also: GUIDE, GUIDATA, GUIHANDLES

% Edit the above text to modify the response to help untitled
% Last Modified by GUIDE v2.5 16-Oct-2019 00:31:54
% Begin initialization code - DO NOT EDIT
gui_Singleton = 1;
gui_State = struct('gui_Name',mfilename,'gui_Singleton',gui_Singleton,...
    'gui_OpeningFcn',@untitled_OpeningFcn,'gui_OutputFcn',...
    @untitled_OutputFcn,'gui_Lay outFcn',[],'gui_Callback',[]);
if nargin && ischar(varargin{1})
  gui_State.gui_Callback = str2func(varargin{1});
end

if nargout
  [varargout{1:nargout}] = gui_mainfcn(gui_State, varargin{:});
else
   gui_mainfcn(gui_State, varargin{:});
end
% End initialization code - DO NOT EDIT

% --- Executes just before untitled is made visible.
function untitled_OpeningFcn(hObject, eventdata, handles, varargin)
% This function has no output args, see OutputFcn.
% hObject handle to figure
% eventdata reserved - to be defined in a future version of MATLAB
% handles structure with handles and user data (see GUIDATA)
% varargin command line arguments to untitled (see VARARGIN)
% Choose default command line output for untitled
handles.output = hObject;
% Update handles structure
guidata(hObject, handles);
% UIWAIT makes untitled wait for user response (see UIRESUME)
% uiwait(handles.figure1);
% --- Outputs from this function are returned to the command line.
function varargout = untitled_OutputFcn(hObject, eventdata, handles)
% varargout cell array for returning output args (see VARARGOUT);
% hObject handle to figure
% eventdata reserved - to be defined in a future version of MATLAB
% handles structure with handles and user data (see GUIDATA)
% Get default command line output from handles structure
varargout{1} = handles.output;
```

These three functions are needed for opening the GUI window itself. As seen, they are accompanied by the extended comments explaining all the inputs and outputs. These three functions usually remain unchanged for any GUI (sometimes, a few lines of code are added at the end of the `..._OpeningFcn` function to initialize the GUI, that is, set its default values). Adding each new UI object that assumes some actions adds at least one

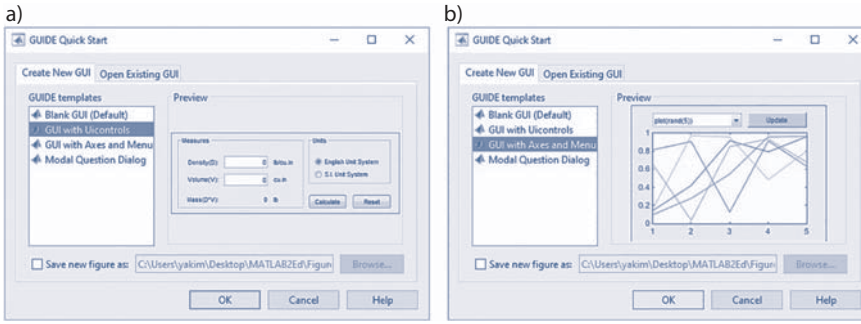


Fig. F.6 Examples of simple GUIs provided with MATLAB.

more local function to this *m*-file, as explained in Sec. F.4. Information about properties of the UI objects is passed between the functions using a single structure, `handles`, composed of the UI object handles and user data.

Figure F.6 shows examples of two simple GUIs as provided by MathWorks. The first one (Fig. F.6a) may be used to learn how to handle arithmetic calculations, and the second one (Fig. F.6b) demonstrates how to use a pop-up menu to change a graphical output. We will return to these examples in Sec. F.5, but now let us proceed with developing a simple GUI from scratch.

F.4 Laying Out and Programming a Simple GUI

Consider a simple example. You would like to supplement the very first code you developed for Problem 2.3 of Chapter 2 (day-of-the-week calculator) with a nice graphical interface. You start by opening a blank GUI as shown in Fig. F.3b and saving it as *WeekDayGUI.fig*. Immediately after pushing the **Save** button, the *WeekDayGUI.m* *m*-file is automatically generated. It looks exactly the same as presented in the previous section except that `untitled` is replaced with `WeekDayGUI` everywhere throughout your code. If you try to change the GUI name, the corresponding *m*-file will immediately be renamed too.

Let us use the buttons of the Component Palette (shown in Table F.1) and create several objects, including 10 Static texts, 3 Pop-up menus, and 2 Push buttons, as shown in Fig. F.7.

Next, let us change some properties of these graphics objects and align them using the **Align Objects** button so that your GUI looks like that shown in Fig. F.8. To adjust the properties of a graphics object, you simply double-click it, which brings up a Property Inspector window (alternatively, you can right-click and choose the **Property Inspector** option from a pop-up menu or simply click the **Property Inspector** button on the GUIDE toolbar, , having a specific UI object selected). The Property Inspector windows are

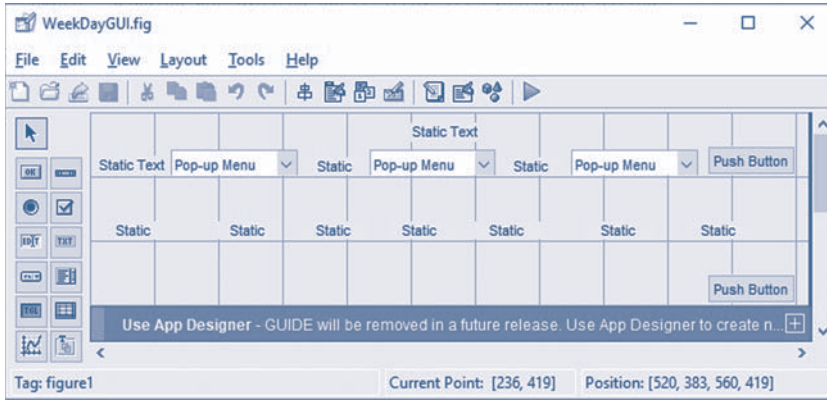


Fig. F.7 Original layout of a sample GUI.

pretty much the same for all UI objects (Fig. F.9). (In fact, among other properties, you can change the **Style** option of your UI object as well: clicking the **Expand** button at the far right of the **Style** option line brings up the menu shown in Fig. F.10a.)

Compared with Fig. F.7, the changes displayed in Fig. F.8 are that a) all objects are tagged with meaningful names (like `close`, `month`, and `day3` with their properties shown in Figs. F.9a, F.9b, and F.9c, respectively), so that you can explicitly refer to these objects in the program functions; b) the **String** attributes are changed (to `Close GUI` and `Wednesday` as shown in Figs. F.9a and F.9c, respectively); and c) the background color for weekdays is changed to white (for pop-up menus, white is a default color anyway). For three pop-up menus, instead of assigning a single **String,** we filled their menu lists with years, names of months, and days (1 through 31). You can bring the menu list up by pushing the **String text** button, .

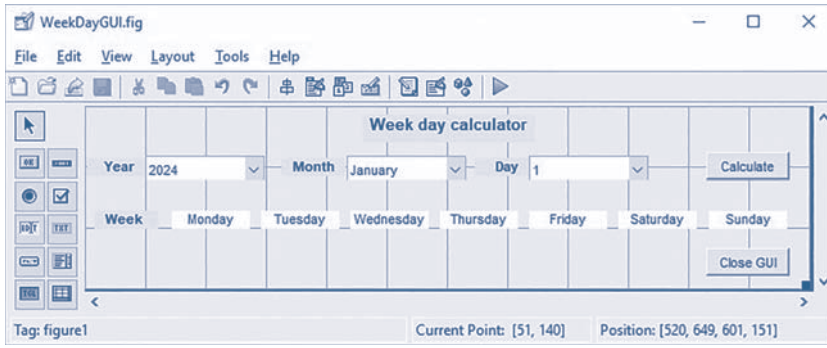


Fig. F.8 Final layout of the WeekDayGUI.

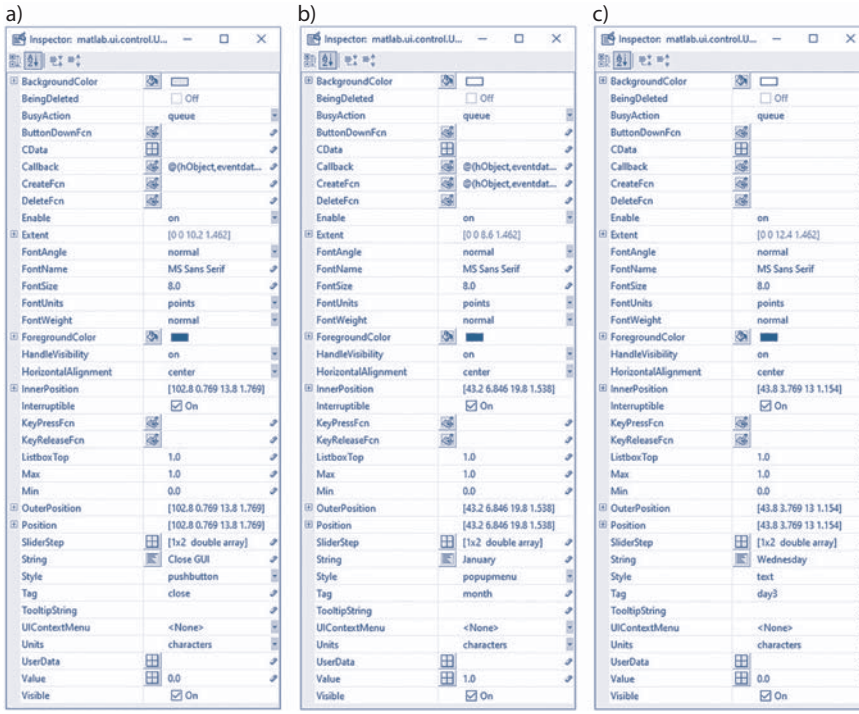


Fig. F.9 Properties of a) Close GUI push button, b) Month pop-up menu, and c) Wednesday static text.

As an example, the **String** list of Fig. F.9b is presented in Fig. F.10b (which is why January shows up as a **String** attribute).

When you save this GUI, it automatically updates the *WeekDayGUI.m* file. Now, you need to complete the programming portion of GUI

Fig. F.10 a) Pop-up menu for the Style property of a UI object, and b) example of the filled menu list of the String property (for the Month pop-up menu).

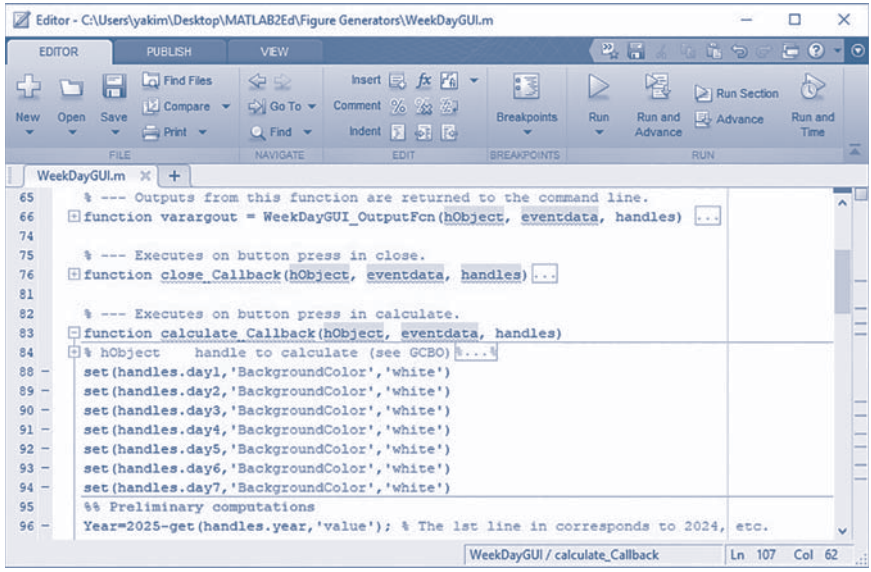


Fig. F.11 Browsing through the GUI functions in the Editor.

development. Hence, you need to open this *m*-file in the Editor (for example, by clicking the **Editor** button, , of the GUIDE Layout Editor toolbar). Figure F.11 shows the content of the *WeekDayGUI.m* file after we added and modified all the aforementioned UI objects [you can see all the function headers if you choose the **Collapse All** option of the Code Folding group of the Editor View tab (Fig. 2.11c)].

As mentioned in the previous section, each new object that assumes some actions adds new local functions to those three of the blank GUI. As seen in Fig. F.11, the Static text UI assumes no actions and, therefore, adds no functions. Each of the two push buttons adds a Callback function. Each of the three pop-up menus adds two functions, ..._CreateFcn and ..._Callback. These two functions for the Year pop-up menu show up as follows:

```
% --- Executes during object creation, after setting all properties
function year_CreateFcn(hObject, eventdata, handles)
% hObject handle to year (see GCBO)
% eventdata reserved - to be defined in a future version of MATLAB
% handles empty - handles not created until after all CreateFcns called

% Hint: popupmenu controls usually have a white background on Windows.
% See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),...
    get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
```

```
end
% --- Executes on selection change in year.
function year_Callback(hObject, eventdata, handles)
% hObject handle to year (see GCBO)
% eventdata reserved - to be defined in a future version of MATLAB
% handles structure with handles and user data (see GUIDATA)

% Hints:
% contents = get(hObject,'String') returns year contents as cell array
%          contents{get(hObject,'Value')} returns selected item from year
```

The local functions for pop-up menus are good as they are, that is, they need no additional tweaking. The idea is that you would use these three pop-up menus to choose the calendar date. This date will then be used to compute a weekday by pushing the **Calculate** push button, and this is where all programming should occur. Before we proceed, let us show the programming that needs to be done in another local function that is responsible for closing the GUI. Your **Close** button callback function may look as simple as

```
% --- Executes on button press in close.
function close_Callback(hObject, eventdata, handles)
% hObject handle to close (see GCBO)
% eventdata reserved - to be defined in a future version of MATLAB
% handles structure with handles and user data (see GUIDATA)
close
```

and the only programming we did here was adding the `close` command at the very end.

Now, let us return to the **Calculate** callback function. Suppose that to display the calculated weekday you want to simply change the background color of the corresponding Static text UI object to, say, green. You also want to be able to calculate the weekday for several different dates while the GUI is running. Hence, the first action would be bringing the background color for all weekdays back to white (you could have more sophisticated code, remembering what weekday was colored green at the previous calculation and then only reverting a background color for this specific weekday; however, for simplicity, let us apply the white color to all weekday static texts, even though six of them are white anyway). The complete `calculate_Callback` local function for Problem 2.3 of Chapter 2 may then look as follows:

```
% --- Executes on button press in calculate
function calculate_Callback(hObject, eventdata, handles)
% hObject handle to calculate (see GCBO)
% eventdata reserved - to be defined in a future version of MATLAB
% handles structure with handles and user data (see GUIDATA)
%% Change the background color to default
set(handles.day1,'BackgroundColor','white')
```

```
set(handles.day2,'BackgroundColor','white')
set(handles.day3,'BackgroundColor','white')
set(handles.day4,'BackgroundColor','white')
set(handles.day5,'BackgroundColor','white')
set(handles.day6,'BackgroundColor','white')
set(handles.day7,'BackgroundColor','white')
%% Preliminary computations
Year=2025-get(handles.year,'value'); % the 1st line corresponds to 2024, etc.
m=get(handles.month,'value')-2;
d=get(handles.day,'value');
if m <= 0              % make corrections for January and February
m=m+12;                % correct the month's number to that of in Ancient Rome
Year=Year-1;           % correct the year to that of in Ancient Rome
end
Y=mod(Year,100);       % compute the year's number in the century
c=(Year-Y)/100;        % compute the number of centuries
%% Now goes the formula
WeekDay = mod((d+fix((13*m-1)/5)+Y+fix(Y/4)+fix(c/4)-2*c),7);
%% The output
switch WeekDay
 case 1, set(handles.day1,'BackgroundColor','green')
 case 2, set(handles.day2,'BackgroundColor','green')
 case 3, set(handles.day3,'BackgroundColor','green')
 case 4, set(handles.day4,'BackgroundColor','green')
 case 5, set(handles.day5,'BackgroundColor','green')
 case 6, set(handles.day6,'BackgroundColor','green')
 case 0, set(handles.day7,'BackgroundColor','green')
end
```

Note that the only difference with code you developed in Chapter 2 is that in the beginning of this function we are using the `handles` structure to access and change the background color of UI objects tagged `day1`, `day2`, . . ., `day7` (Fig. F.9c), and at the end, when the weekday is determined, we are employing the `switch` statement to color the static text of the corresponding weekday green (using the `handles` structure again).

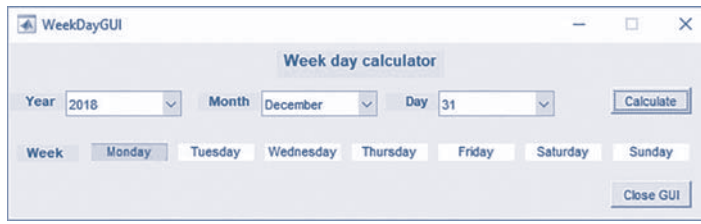


Fig. F.12 WeekDay GUI in action.

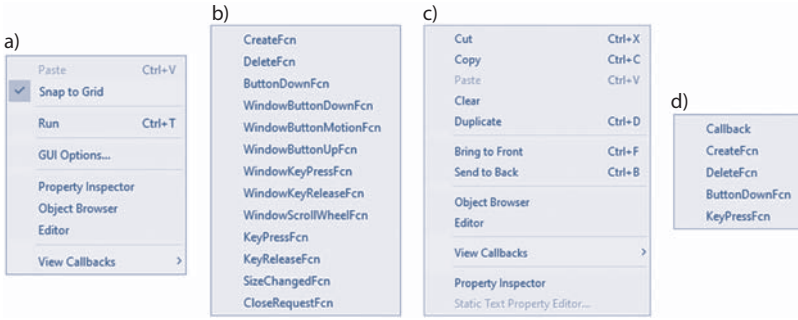


Fig. F.13 Menus in response to right-clicking a) anywhere within blank layout area, c) on any UI graphics object, and b and d) the corresponding View Callbacks options.

If you still have your Layout Editor open, you may run this GUI by pushing the **Run Figure** button. You choose the date and click the **Calculate** push button to see the result (Fig. F.12). You can also run this GUI from the Editor or from the Command Window by typing in `WeekDayGUI` and pushing <Enter>.

When you know the basics of GUIDE, you may proceed with exploring its other features on your own. For your convenience, let us make a couple of remarks. First, it should be mentioned that the GUIDE Layout Editor allows you to automatically create some other functions, beyond those two (`CreateFcn` and `Callback`) we have just considered. If you right-click anywhere in the layout area, aside from the UI objects, you will see the menu shown in Fig. F.13a. Choosing the **View Callbacks** option brings up another menu that lists other possible action functions (Fig. F.13b). Right-clicking the uibutton results in the menu of Fig. F.13c. Choosing the **View**

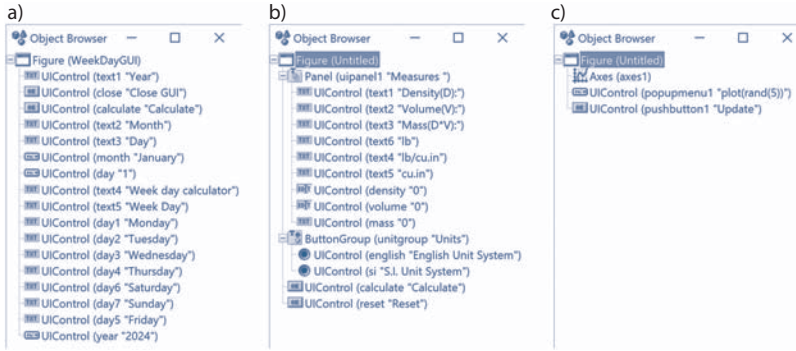


Fig. F.14 Object Browser for the a) WeekDay GUI, b) Fig. F.6a GUI, and c) Axes and Menu GUI (Fig. F.6b).

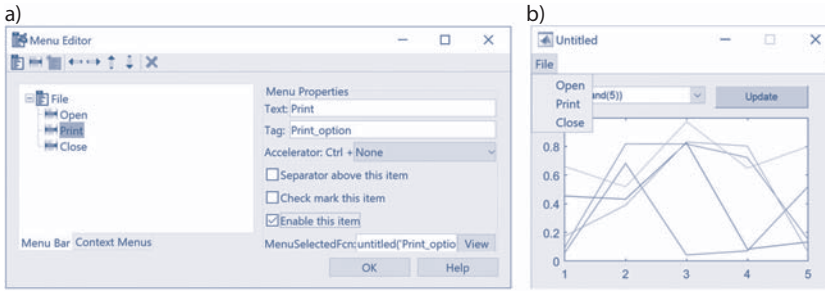


Fig. F.15 Example of using a) Menu Editor and b) the correspondingmenu items.

Callbacks option in this case brings up another menu (Fig. F.13d). Other user interfaces—`uitable`, `uibuttongroup`, and `uipanel`—have their own action function lists.

Second, the Object Browser represents a very convenient way of listing all UI graphics objects within the open GUI, providing you with their tags and text strings (names). In our case, this browser appears as shown in Fig. F.14a. Clicking any UI object brings up its Property Inspector window.

F.5 Examples of Simple and Applied GUIs

Now that you know the basics of how to lay out a GUI and what programming needs to be done behind it, let us return to the two simple GUIs shown in Fig. F.6. The Object Browser for the first one (Fig. F.6a) is shown in Fig. F.14b. If you look at this GUI code, you will find that, compared with the blank GUI code of Sec. F.3, one extra line is added at the end of the `..._OpeningFcn` function:

```
initialize_gui(hObject, handles, false);
```

This calls for a user-written local function that initializes the GUI as follows:

```
function initialize_gui(fig_handle, handles, isreset)
% If the metricdata field is present and the reset flag is false, it means
% we are just re-initializing a GUI by calling it from the cmd line
% while it is up. So, bail out as we dont want to reset the data.
if isfield(handles, 'metricdata') && ~isreset
  return;
end
handles.metricdata.density = 0;
handles.metricdata.volume = 0;
set(handles.density, 'String', handles.metricdata.density);
set(handles.volume, 'String', handles.metricdata.volume);
```

```
set(handles.mass, 'String', 0);
set(handles.unitgroup, 'SelectedObject', handles.english);
set(handles.text4, 'String', 'lb/cu.in');
set(handles.text5, 'String', 'cu.in');
set(handles.text6, 'String', 'lb');
% Update handles structure
guidata(handles.figure1, handles);
```

This function simply sets the GUI to work with the English Engineering units. If the units are changed to those of the metric system, the function behind the panel `unitgroup`, managing an exclusive choice between two systems of units, handles it as follows:

```
% --- Executes when selected object changed in unitgroup
function unitgroup_SelectionChangeFcn(hObject, eventdata, handles)
if (hObject == handles.english)
  set(handles.text4, 'String', 'lb/cu.in');
  set(handles.text5, 'String', 'cu.in');
  set(handles.text6, 'String', 'lb');
else
  set(handles.text4, 'String', 'kg/cu.m');
  set(handles.text5, 'String', 'cu.m');
  set(handles.text6, 'String', 'kg');
end
```

The callback functions behind the two input windows are made foolproof, so that if a user attempts to enter non-numerical data, he gets the warning message

```
function density_Callback(hObject, eventdata, handles)
density = str2double(get(hObject, 'String'));
if isnan(density)
  set(hObject, 'String', 0);
  errordlg('Input must be a number','Error');
end
% Save the new density value
handles.metricdata.density = density;
guidata(hObject,handles)
function volume_Callback(hObject, eventdata, handles)
volume = str2double(get(hObject, 'String'));
if isnan(volume)
  set(hObject, 'String', 0);
  errordlg('Input must be a number','Error');
end
% Save the new volume value
handles.metricdata.volume = volume;
guidata(hObject,handles)
```

Finally, the two push buttons are responsible for a simple calculation and resetting the GUI:

```
% --- Executes on button press in calculate
function calculate_Callback(hObject, eventdata, handles)
mass = handles.metricdata.density * handles.metricdata.volume;
set(handles.mass, 'String', mass);

% --- Executes on button press in reset
function reset_Callback(hObject, eventdata, handles)
initialize_gui(gcbf, handles, true);
```

The Object Browser for the second GUI (Fig. F.6b) is shown in Fig. F.14c. Compared with the GUIs considered earlier, it features a three-option **Tools** menu, created using the Menu Editor, (Fig. F.15a). In the Menu Bar tab, you may type any new items and subitems that will appear when the GUI is running, as shown in Fig. F.15b. Obviously, each new entry automatically generates the corresponding local function header. In this case, programming included writing the following code for the three subitems of the **File** menu:

```
function OpenMenuItem_Callback(hObject, eventdata, handles)
file = uigetfile('*.fig');
if ~isequal(file, 0)
  open(file);
end
function PrintMenuItem_Callback(hObject, eventdata, handles)
printdlg(handles.figure1)

function CloseMenuItem_Callback(hObject, eventdata, handles)
selection = questdlg(['Close' get(handles.figure1,'Name') '?'],...
           ['Close' get(handles.figure1,'Name') '...'],'Yes','No','Yes');
if strcmp(selection,'No')
  return;
end
delete(handles.figure1)
```

Compared with the case considered in Sec. F.4, the `popupmenu1_CreateFcn` function is used to define the list of possible options for a string attribute:

```
% --- Executes during object creation, after setting all properties
function popupmenu1_CreateFcn(hObject, eventdata, handles)
if ispc && isequal(get(hObject,'BackgroundColor'),...
                   get(0,'defaultUicontrolBackgroundColor'))
  set(hObject,'BackgroundColor','white');
end
set(hObject,'String',{'plot(rand(5))','plot(sin(1:0.01:25))',...
                      'bar(1:.5:10)','plot(membrane)','surf(peaks)'});
```

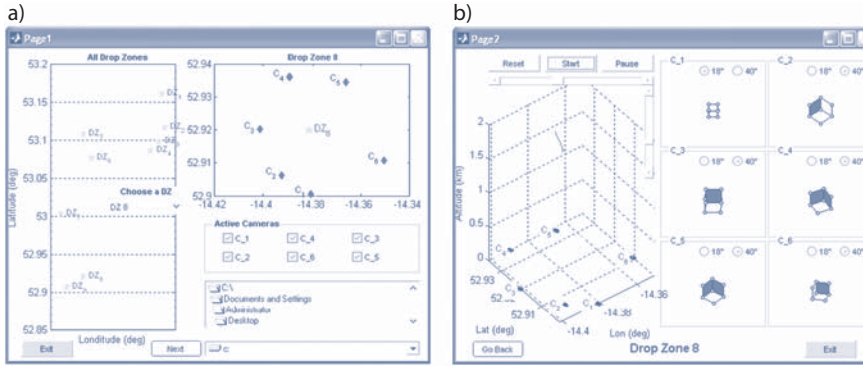


Fig. F.16 A two-page GUI to assist in solving a pose-estimation problem.

Once the choice is made, the push-button Callback function produces a plot using the switch statement:

```
% --- Executes on button press in pushbutton1
function pushbutton1_Callback(hObject, eventdata, handles)
axes(handles.axes1);
cla;
popup_sel_index=get(handles.popupmenu1,'Value');
switch popup_sel_index
   case 1, plot(rand(5));
   case 2, plot(sin(1:0.01:25.99));
   case 3, bar(1:.5:10);
   case 4, plot(membrane);
   case 5, surf(peaks);
end
```

Note that this latter function starts by taking the axes that have been created already and clearing them to draw a new plot. This very first plot is created during the `OpeningFcn` function call, which compared with the case of a blank GUI of Sec. F.3 contains the line

```
plot(rand(5));
```

It is this plot that shows up when you call this GUI for the first time (Fig. F.15b).

To encourage you to use the GUI development environment, let us show a couple of practical examples. Figure F.17 shows an example of a two-page GUI developed as an interface for addressing the so-called pose-estimation problem (when the 3-D position and attitude of an object needs to be estimated using synchronized video streams from multiple cameras). First (Fig. F.16a), a user chooses the drop zone to operate within, establishes multiple cameras around the intended point of impact, and establishes a folder containing the test data (by the way, one of the ActiveX controls,

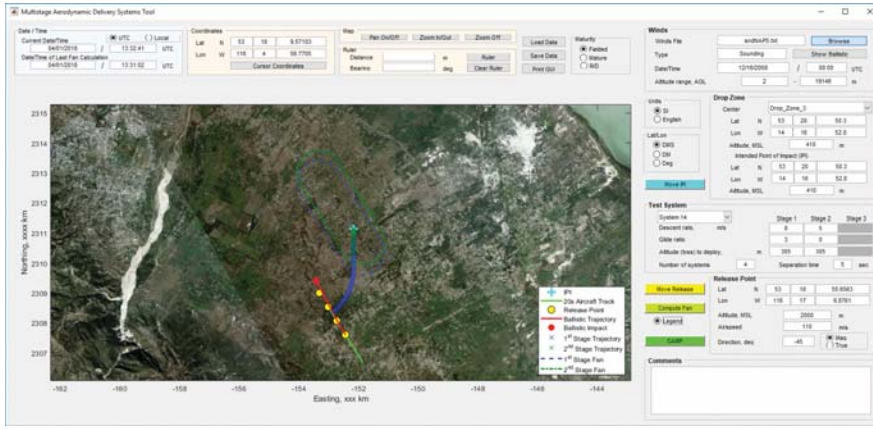


Fig. F.17 Safety Fans GUI.

`mwxpccontrolsx3x2.PathChooserCtrl`, was implemented here for this purpose). Then, the user jumps to another page and chooses the focal length of each camera to emulate what it would see while some aerospace system descends to the ground (Fig. F.16b).

Figure F.17 shows another GUI developed for test officers to ensure ground personnel and equipment safety when deploying multiple multistage

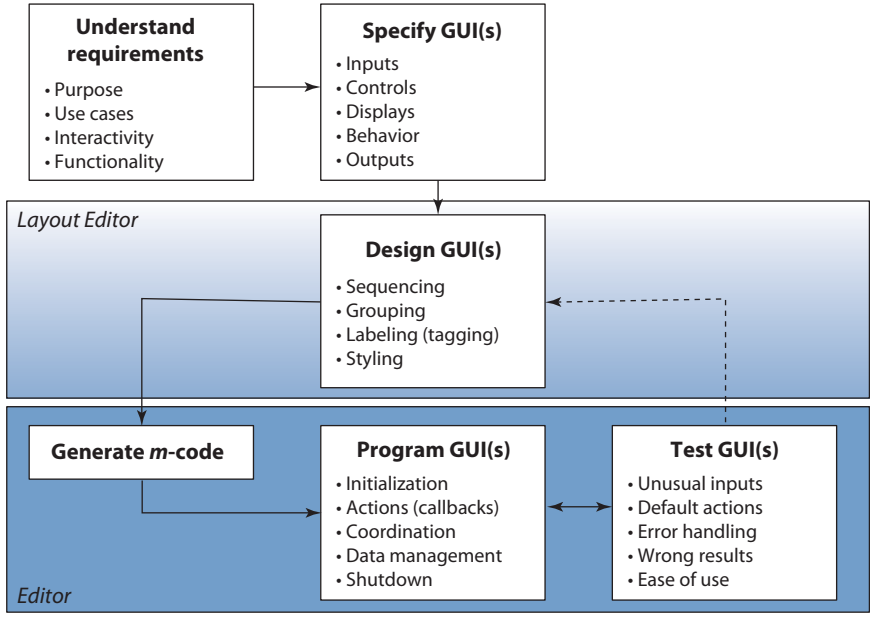


Fig. F.18 Designing professional-looking GUIs.

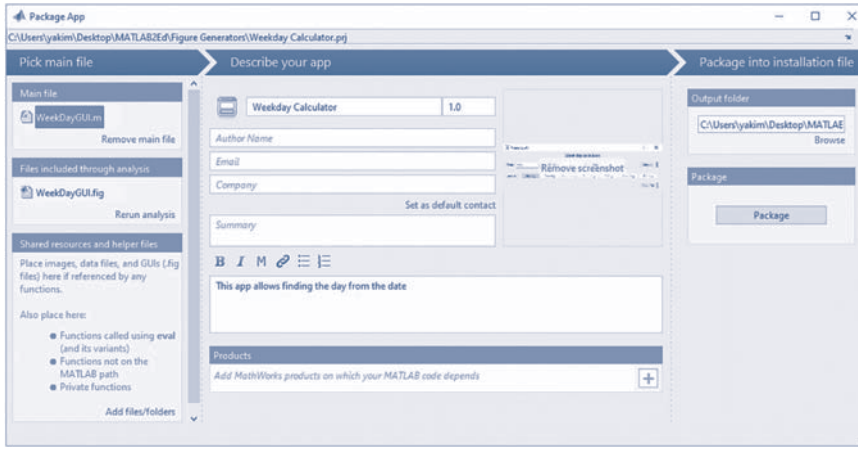


Fig. F.19 Package App window.

cargo aerial delivery systems. As seen, this GUI features a rich graphical interface that enables many interactive actions, including doing measurements on the underlying map.

In conclusion, Fig. F.18 presents a process flow for designing application-specific GUIs as suggested by MathWorks. It is natural and quite easy to follow.

F.6 Creating MATLAB Apps

Once you have used either the first (programmatic) or the second (GUIDE-assisted) approach and developed a GUI, you can easily convert it to a full-fledged MATLAB app that is sharable with your colleagues (see Fig. F.1). To do that, click the **Package App** icon of the Apps tab of the MATLAB Toolstrip (Fig. 2.6b). The Package App window pops up (Fig. F.19).

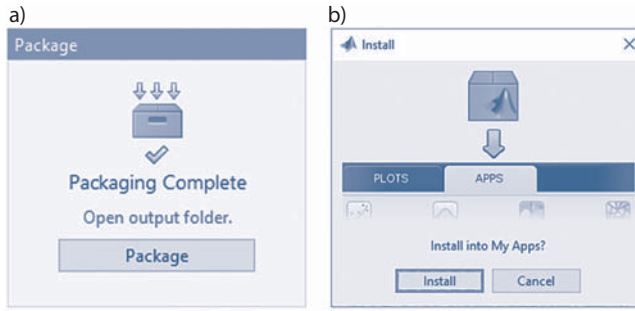


Fig. F.20 a) Packaging completion notification and b) package installation menu.

Fig. F.21 Weekday Calculator app added to the Apps tab apps gallery.

Add a main file name in the upper-left panel of this window (MATLAB may ask you to change the current folder so that all related files can be found). All related files are automatically included through analysis (*WeekDayGUI.fig* in our case). If needed, you may add additional resources in the lower-left panel. In the middle panel, enter the name of the app and any additional information you might want. You can also add a screenshot of your application.

The next step is to click the **Package** button in the lower-right panel. Upon completion, the MATLAB App Installer file (*Weekday Calulator.mlappinstall* in our case) is created in the current folder, and the package panel of Fig. F.19 changes to that of Fig. F.20a.

Now, you may click the **Install App** icon of the Apps tab of the Toolstrip (Fig. 2.6b) and navigate to the MATLAB App Installer file you just created to install the app (Fig. F.20b). During installation, you will be prompted to change the current folder (because the original *m*-file shadows your app). Successful installation results in your app being added to the apps gallery (Fig. F.21) (compare with Fig. 2.6b). If you need to remove it later on, just right-click your app icon and choose the **Uninstall** option from the drop-down menu.

F.7 Creating Standalone Executables

Apart from creating a MATLAB app, you might want to convert your GUI to a standalone executable application. This way you could share it with users who do not have MATLAB installed on their computers. The truth of the matter, though, is that to be able to run compiled MATLAB applications on computers that do not have MATLAB installed, the end users would still need to install the so-called *MATLAB (Compiler)*

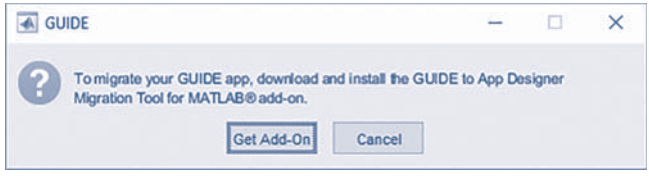


Fig. F.22 GUIDE prompt to install GUIDE to the App Designer Migration Tool.

Runtime (MCR), a collection of shared libraries and code, available as a free download from the *www.mathworks.com/products/compiler/matlab-runtime.html*. Click the version and platform corresponding to the application or component you are using (the version of MATLAB), and save the MCR installer file on the computer on which you plan to run the application or component. Double click the installer and follow the instructions in the installation wizard. Installing MCR *MATLAB_Runtime 9* enables you to run standalone executables (without having to install MATLAB). (Although MCR provides most of MATLAB's functionality, it differs from MATLAB by not using its desktop graphical interface and by encrypting MATLAB files for portability and integrity.)

It is the *MATLAB Compiler* (one of the additional toolboxes of MATLAB shown in Table 1.15) that allows building a standalone application or interface to use your code as a shared library if you need to integrate it into C or C++. If you need to integrate it into other development environments (Java, .NET, Excel), MATLAB offers separate builder products (also listed in Table 1.15) that need to be acquired and installed separately. In either case, the MATLAB Compiler builds encrypted code that the end user cannot view or modify.

The programmatic way of creating a standalone executable (see Fig. F.1) is to use the mcc function invoking the MATLAB Compiler. It has the following general syntax:

```
mcc [-options] mfile1 [mfile2 ... mfileN]
```

It prepares MATLAB file(s) for deployment outside of the MATLAB environment, generates the wrapper files in C or C++, optionally builds the standalone binary files, and writes any resulting files into the current folder (by default). Specifically,

`mcc -m test.m` generates a standalone application from the *test.m* file (this application opens the MS-DOS command window you may need if your code requires some external inputs)

`mcc -e test.m` generates a standalone application that does not open the MS-DOS command window

For example, the command line

```
>> mcc -e WeekDayGUI
```

creates a standalone executable application, the *WeekDayGUI.exe* file (and three auxiliary text log files—*readme.txt*, *mccExcludedFiles.log*, and *requiredMCRProducts.txt*—featuring a detailed checklist to successfully deploy your application on another computer, the list of various toolbox functions that were not automatically included, and the list of the additional products that might be required, respectively). You may run this executable file on any computer that has either MCR or MATLAB installed. (Note, MCR is version specific. You must run your applications with the version of MCR associated with the version of the MATLAB Compiler with which it was created.)

The alternative ways of creating standalone executables—via running `applicationCompiler` (which opens the MATLAB standalone compiler for the creation of a new compiler project) or `deploytool` (opening a list of the compiler apps to choose from)—will be discussed in Sec. F.9.

F.8 Migrating GUIDE GUI to App Designer

So far we have considered two of three approaches for building apps. Now it is time to introduce the third one, the App Designer, which can be called by either clicking the first icon of the Apps tab of the MATLAB Toolstrip (see Fig. F.21) or typing the command `appdesigner` in the Command Window. As mentioned earlier (Fig. F.1), App Designer is an advanced development environment for building MATLAB apps. Compared with GUIDE, it features

- An improved design canvas and a new generated code structure that makes it easier to share data across the app
- An expanded component set with a full set of standard user interface components shown in Table F.2; new components such as Tree for presenting lists of items in a hierarchy (`uitree`), date picker for creating a calendar (`uidatepicker`), and an enhanced Table; and components to create the control panels and human – machine interfaces (including gauges, lamps, knobs, and switches)
- The ability to package an app installer file directly from the App Designer toolstrip as well as create a standalone or web app (see Fig. F.1).

The differences between GUIDE and App Designer are outlined in Table F.3.

Indeed, compared with GUIDE, App Designer is a more advanced tool, and if you are considering developing a new app from scratch, you might want to go with App Designer (the only caveat is that it utilizes the object-oriented programming syntax, which will require some additional effort from you to learn about). If you have some legacy GUIs developed in GUIDE but want to take advantage of the new components and features offered by App Designer, MATLAB provides you with a way to migrate your GUIDE-developed GUIs to the App Designer environment. This feature, available in R2018a and subsequent releases, is depicted in Fig. F.1.

To demonstrate this capability, let us migrate our WeekDayGUI design. With the WeekDayGUI GUI opened in GUIDE, choose the **Migrate to App Designer** option of the **File** menu of GUIDE or click the **Migrate Your App** button at the bottom of the GUIDE window (see Fig. F.3b). The GUIDE prompt window shown in Fig. F.22 pops up, suggesting that you install the GUIDE to the App Designer Migration Tool.

Once installation is complete, run the **Migrate to App Designer** option again, which brings up the window shown in Fig. F.23. Click the **Migrate** button, and you will have the option of a nine-step review of the App Designer tool (you can skip it and do it later). The GUIDE app you developed

Table F.3 Differences between GUIDE and App Designer

Difference	GUIDE	App Designer
Using figures and graphics	• Creates most components with the `uicontrol` function • Calls the `figure` function to create the app window • Calls the `axes` function to create the axes for displaying plots • Supports all MATLAB graphics functions	• Calls the `uifigure` function to create the app window • Calls the `uiaxes` function to create the axes for displaying plots • Supports most 2-D and 3-D MATLAB graphics functions (that do not require polar plots, subplots, or graphics interactions, such as mouse and key-press customizations)
Using components	Available components are listed in Table F.1	• Features a new set of components that are created with separate functions • Has several components that GUIDE does not support (e.g., `uitree`, `uigauge`, `uilamp`, and `uiswitch`)
Accessing component properties	Uses the `set` and `get` function to access component properties, e.g., `name = get(HFig,'Name')`	Uses dot notation to access component properties, e.g., `name = app.UIFigure.Name`
Managing app code	Code is structured as a main function and local functions. All code is editable	The code is defined as a MATLAB class. Only callbacks, helper functions, and custom properties are editable
Writing callbacks	Uses `handles`, `hObject`, and `eventdata` input arguments	Uses `app` and `event` input arguments
Sharing data	Uses the `UserData` property, or the `guidata`, `setappdata`, `orgetappdata` functions	Uses custom properties

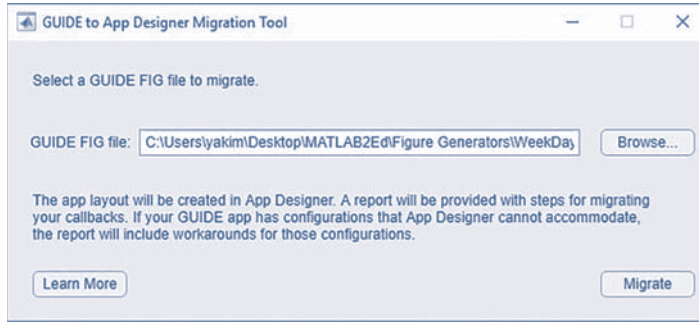


Fig. F.23 Running GUIDE to App Designer Migration Tool.

is now migrated to an App Designer file (*WeekDayGUI_App.mlapp*). The *mlapp*-file contains components with their properties configured appropriately and a copy of the GUIDE callback code. The tool may also generate a report describing the steps to follow to update your callback code for use in App Designer and any limitations or unsupported functionality the tool detected, along with workarounds if available (for example, some features of GUIDE, such as Toolbar, Context Menu, and ActiveX Control are not supported in App Designer).

Now, type `appdesigner` in the Command Window and open the *WeekDayGUI_App.mlapp* file you just created. It will look like that shown

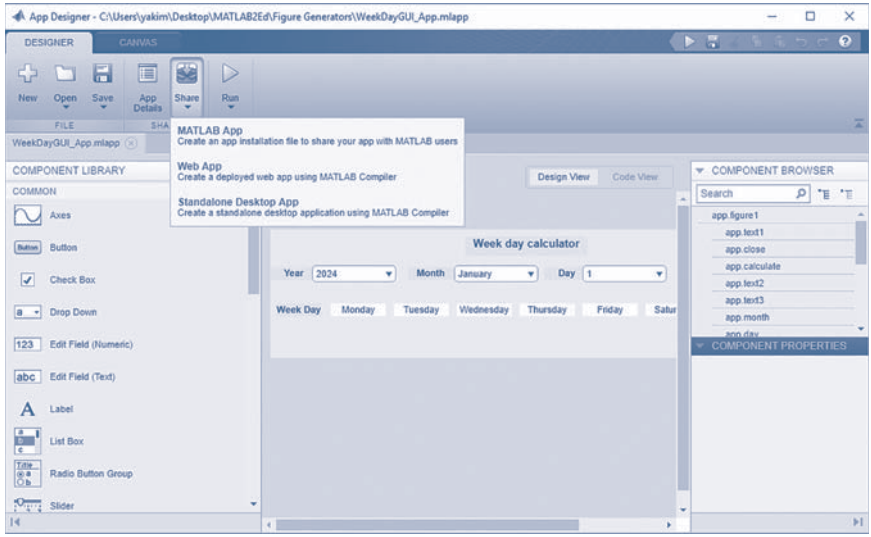


Fig. F.24 App Designer development environment with WeekDayGUI migrated into it.

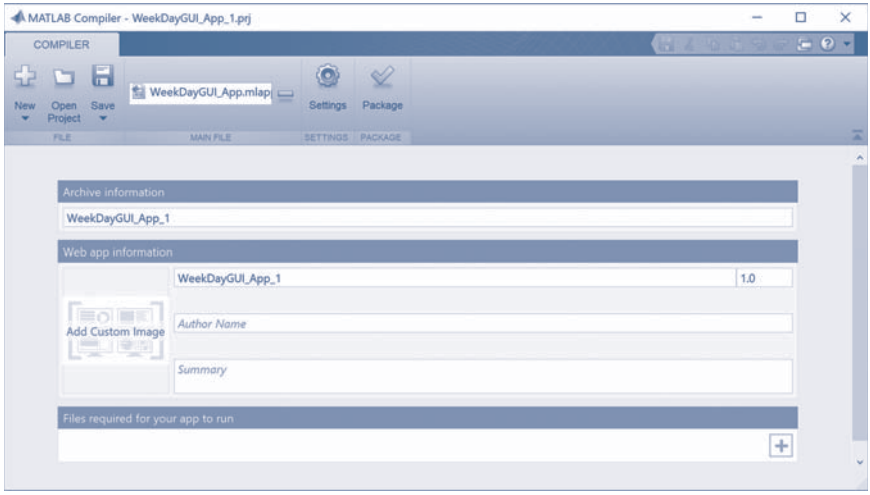


Fig. F.25 MATLAB Compiler window for web app creation.

in Fig. F.24. Clicking the **Share** icon shows what you can do with your design to share it with others.

F.9 Apps Sharing

The **MATLAB App** option of the drop-down menu of the **Share** icon (Fig. F.24) brings up the Package App window (Fig. F.19) discussed in Sec.F.6. Choosing the **Web App** option opens the MATLAB Compiler window as shown in Fig. F.25 (alternatively, you may open this window programmatically, by typing in `webAppCompiler` at the MATLAB command line or running `deploytool` and choosing the **Web App Compiler** option from the pop-up list of the compiler apps). After adding optional information and clicking the **Package** button, the packaging

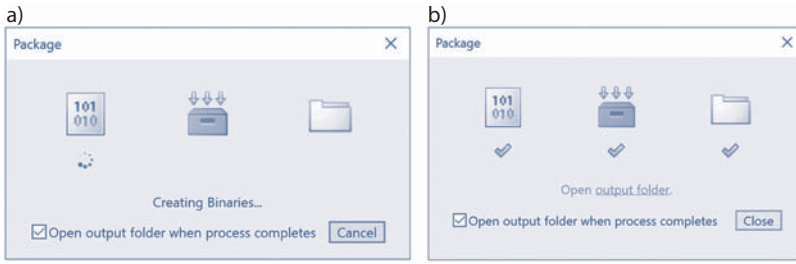


Fig. F.26 Web app packaging progress window: a) packaging has just started and b) packaging is completed.

process starts (Fig. F.26a). After its completion (Fig. F.26b), a folder with two subdirectories containing a web app archive, the *ctf*-file (ctf stands for Component Technology File), is created. This *ctf*-file, hosted by the physical or virtual machine with the *MATLAB Web App Server* installed, can then be used in an intranet environment within your organization.

The MATLAB Web App Server installation file, *MATLABWebAppServerSetup.exe*, is located at *C:\Program Files\MATLAB\R2020a\toolbox\compiler\deploy\win64*, and, when installed, allows access to the uploaded apps (*ctf*-files) in a web browser by navigating to the web apps home page, the URL of which looks like *http://webAppServer:PortNumber/webapps/home/index.html* (where "webAppServer" is your web app server URL, and "PortNumber" is the port specified when configuring the server).

Choosing the third option, **Standalone Desktop App**, from the **Share** button drop-down menu of Fig. F.24 opens the MATLAB Compiler window shown in Fig. F.27 (doing this programmatically involves either running `applicationCompiler` or running `deploytool` and choosing the **Application Compiler** option from the pop-up menu). After packaging (which invokes the same packaging progress window as in Fig. F.26), the *exe*-file is created (*WeekDayGUI_App.exe* in our case).

In fact, when the deployment process is complete, one log file and three folders will have been created—*for_redistribution*, containing the executable file that installs the application and MCR (if the corresponding radio button of the Compiler toolstrip was checked); *for_testing*, containing all the artifacts created by `mcc`; and *for_redistribution_files_only*, containing the files required for redistributing the application (assuming the end user has MATLAB or MCR installed on his or her computer already).

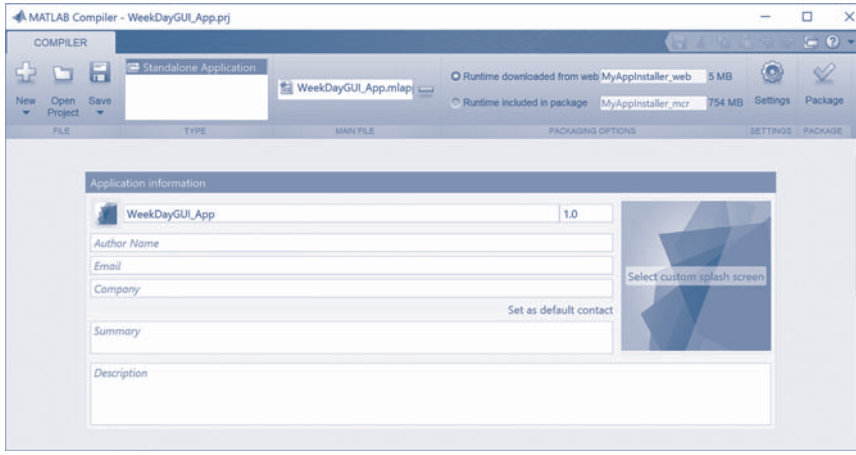


Fig. F.27 MATLAB Compiler window for standalone app creation.

Table F.4 Ways to Share Apps

Approach	End-User Requirements	Comment
Share MATLAB files directly	• Installed MATLAB and toolboxes used • Basic knowledge of MATLAB operations • Some programming skills to vary parameters	The simplest way
Package your app to appear in the Apps tab (using Package App)	• Installed MATLAB and toolboxes used • Basic knowledge of MATLAB operations	Useful for sharing apps with larger audiences
Create a deployed web app (using Web App Compiler)	• Installed web browser with intranet access	For users within the same organization
Create a standalone desktop application (using MATLAB Compiler)	• Installed MCR	The smallest set of requirements

Table F.4 concludes the discussion by summarizing different ways to share your apps with other MATLAB users, group members, and end users.

BIBLIOGRAPHY

Abramowitz, M., and Stegun, I.A., (eds.) *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*, 10th edn., Dover Publications, New York, 1972.

Ashino, R., Nagase, M., and Vaillancourt, R., "Behind and Beyond the MATLAB ODE Suite," CRM-2651 paper, *Centre de recherches mathématiques*, Université de Montréal, Québec, 2000.

Bank, R.E., Coughran, W.C., Fichtner, W.J., Grosse, E.H., Rose, D.J., and Smith, R.K., "Transient Simulation of Silicon Devices and Circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, Vol. 4, No. 4, 1985, pp. 436–451.

Bogacki, P., and Shampine, L.F., "A 3(2) Pair of Runge-Kutta Formulas," *Applied Mathematics Letters*, Vol. 2, No. 4, 1989, pp. 321–325.

Chapman, S.J., *MATLAB Programming for Engineers*, 6th edn., Cengage Learning, Boston, 2019.

Chapra, S.C., *Applied Numerical Methods with MATLAB for Engineers and Scientists*, McGraw-Hill, New York, 2005.

Chapra, S.C., and Canale, R.P., *Numerical Methods for Engineers*, 7th edn., McGraw-Hill, New York, 2015.

Colgren, R., *Basic MATLAB, Simulink and Stateflow*, American Institute of Aeronautics and Astronautics, Reston, VA, 2006.

Dabney, J.B., and Harman, T.L., *Mastering Simulink*, Prentice Hall, Upper Saddle River, NJ, 2004.

Dharmaraja, S., Wang, Y., and Strang, G., "Optimal Stability for Trapezoidal-Backward Difference Split-Steps," *IMA Journal of Numerical Analysis*, Vol. 30, No. 1, 2010, pp. 141–148.

Dormand, J.R., and Prince, P.J., "A Family of Embedded Runge-Kutta Formulae," *Journal of Computational and Applied Mathematics*, Vol. 6, No. 1, 1980, pp. 9–26.

Fausett, L.V., *Numerical Methods: Algorithms and Applications*, Prentice Hall, Upper Saddle River, NJ, 2003.

Fletcher, R., *Practical Methods of Optimization*, 2nd edn., Wiley, New York, 2000.

Forsythe, G., Malcolm, M., and Moler, C., *Computer Methods for Mathematical Computations*, Prentice-Hall, Upper Saddle River, NJ, 1977.

Fröberg, C.-E., *Numerical Mathematics: Theory and Computer Applications*, Addison Wesley, Reading, MA, 1985.

Gander, W., and Gautschi, W., "Adaptive Quadrature—Revisited," *BIT Numerical Mathematics*, Vol. 40, 2000, pp. 84–101, *www.inf.ethz.ch/personal/gander*.

Gilat, A., *MATLAB: An Introduction with Applications*, 6th edn., Wiley, New York, 2016.

Gill, P.E., Murray, W., and Wright, M.H., *Practical Optimization*, Academic Press, London, UK, 1981.

Goldberg, D.E., *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison Wesley, Upper Saddle River, NJ, 1989.

Hahn, B.D., *Essential MATLAB for Scientists and Engineers*, Elsevier, New York, 2005.

Hairer, E., and Wanner, G., *Solving Ordinary Differential Equations II. Stiff and Differential-Algebraic Problems*, 2nd edn., Springer-Verlag, Berlin, New York, 2002.

Hairer, E., Nørsett, S.P., and Wanner, G., *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd edn., Springer-Verlag, Berlin, New York, 2000.

Hanselman, D., and Littlefield, B.R., *Mastering MATLAB*, Pearson, Upper Saddle River, NJ, 2012.
Herniter, M.E., *Programming in MATLAB*, Cengage Learning, Boston, 2000.
Higham, D.J., and Higham, N.J., *MATLAB Guide*, 3rd edn., SIAM, Philadelphia, 2016.
Hooke, R., and Jeeves, T.A., "Direct Search Solution of Numerical and Statistical Problems," *Journal of the Association for Computing Machinery*, Vol. 8, No. 2, 1961, pp. 212–229.
Hosea, M.E., and Shampine, L.F., "Analysis and Implementation of TR-BDF2," *Applied Numerical Mathematics*, Vol. 20, No. 1–2, 1996, pp. 21–37.
IEEE Standard for Binary Floating-Point Arithmetic (IEEE 754-1985).
IEEE Standard for Floating-Point Arithmetic (IEEE 754-2008).
Kahaner, D., Moler, C., and Nash, S., *Numerical Methods and Software*, Prentice-Hall, Englewood, NJ, 1988.
Kierzenka, J., and Shampine, L.F., "A BVP Solver Based on Residual Control and the MATLAB PSE," *ACM Transactions on Mathematical Software*, Vol. 27, No. 3, 2001, pp. 299–316.
Kierzenka, J., and Shampine, L.F., "A BVP Solver That Controls Residual and Error," *Journal of Numerical Analysis, Industrial and Applied Mathematics*, Vol. 3, No. 1–2, 2008, pp. 27–41.
Klee, H., and Allen, R., *Simulation of Dynamic Systems with MATLAB and Simulink*, 3rd edn., Taylor & Francis, CRC Press, Boca Raton, FL, 2018.
Krylov, V.I., *Approximate Calculation of Integrals*, Macmillan, New York, 1962 (Translated from Russian).
Magrab, E.B., Azarm, S., et al. *An Engineer's Guide to MATLAB*, 3rd edn., Pearson, Upper Saddle River, NJ, 2011.
Mathews, J.H., and Fink, K.K., *Numerical Methods Using MATLAB*, 4th edn., Prentice Hall, Upper Saddle River, NJ, 2004.
Michels, H.H., "Abscissas and Weight Coefficients for Lobatto Quadrature," *Mathematics of Computation*, Vol. 17, No. 83, 1963, pp. 237–244.
Moler, C., *Experiments with MATLAB*, The MathWorks, 2009, *www.mathworks.com/moler/exm/chapters.html*.
Moler, C., *Numerical Computing with MATLAB, Revised Reprint*, SIAM, Philadelphia, 2008.
Moler, C., "The Growth of MATLAB® and The MathWorks over Two Decades," *The Mathworks News and Notes*, January 2006.
Nelder, J.A., and Mead, R., "A Simplex Method for Function Minimization," *Computer Journal*, Vol. 7, 1965, pp. 308–313.
Netlib Repository at The University of Tennessee at Knoxville and Oak Ridge National Laboratory, www.netlib.org.
Nocedal, J., and Wright, S.J., *Numerical Optimization*, Springer Series in Operations Research and Financial Engineering, Springer Verlag, Berlin, Germany, 1999.
Palm, W.J., III, *Introduction to MATLAB for Engineers*, 3rd edn., McGraw-Hill, New York, 2011.
Pratap, R., *Getting Started with MATLAB*, 7th edn., Oxford University Press, New York, 2016.
Ralston, A., *A First Course in Numerical Analysis*, McGraw-Hill, New York, 1965.
Recktenwald, G.W., *Introduction to Numerical Methods and MATLAB: Implementations and Applications*, Prentice Hall, Upper Saddle River, NJ, 2000.
Scarborough, J.B., *Numerical Mathematical Analysis*, 6th edn., Johns Hopkins University Press, Baltimore, MD, 1966.
Shampine, L.F., "MATLAB Program for Quadrature in 2D," *Applied Mathematics and Computation*, Vol. 202, No. 1, 2008, pp. 266–274.
Shampine, L.F., "Some Practical Runge-Kutta Formulas," *Mathematics of Computation*, Vol. 46, No. 173, 1986, pp. 135–150.
Shampine, L.F., "Vectorized Adaptive Quadrature in MATLAB," *Journal of Computational and Applied Mathematics*, Vol. 211, No. 2, 2008, pp. 131–140.

Shampine, L.F., "Weighted Quadrature by Change of Variable," *Neural, Parallel, and Scientific Computations*, Vol. 18, No. 2, 2010, pp. 195–206.
Shampine, L.F., and Gordon, M.K., *Computer Solution of Ordinary Differential Equations: The Initial Value Problem*, W.H. Freeman & Co. Ltd., San Francisco, 1975.
Shampine, L.F., and Hosea, M.E., "Solving ODEs and DDEs with Residual Control," *Applied Numerical Mathematics*, Vol. 52, 2005, pp. 113–127.
Shampine, L.F., and Reichelt, M.W., "The MATLAB ODE Suite," *SIAM Journal on Scientific Computing*, Vol. 18, No. 1, 1997, pp. 1–22.
Shampine, L.F., and Thompson, S., "Solving DDEs in MATLAB," *Applied Numerical Mathematics*, Vol. 37, 2001, pp. 441–458.
Shampine, L.F., Gladwell, I., and Thompson, S., *Solving ODEs with MATLAB*, Cambridge University Press, Cambridge, 2003.
Shampine, L.F., Reichelt, M.W., and Kierzenka, J., "Solving Boundary Value Problems for Ordinary Differential Equations in MATLAB with bvp4c," 2000, *www.mathworks.com/bvp_tutorial*.
TeX Tutorial, *www.tutorialspoint.com/tex_commands/index.htm*.
The MathWorks, *www.mathworks.com*.
Wikipedia, *www.wikipeida.org*.
Wilkinson, J.H., *Rounding Errors in Algebraic Processes*, Dover Publications, New York, 1994.
Wolfram MathWorld, *www.mathworld.wolfram.com*.
Ying, W., Henriquez, C.S., and Rose, D.J., "Composite Backward Differentiation Formula: An Extension of the TR-BDF2 Scheme," 2009, *www.math.sjtu.edu.cn/faculty/wying/mypapers/CBDFs.pdf*.

INDEX

Note: page numbers are followed by f or t (indicating figures or tables).

A

absolute errors, 408
abs (X) function, 51t
accuracy, 385–386, 386f
Adams–Bashforth–Moulton predictor–corrector method, 701–708, 703f, 706f, 707t, 708t
adaptive single-step algorithms, 698–701
Add-On Explorer window, 31–32, 32f
Aerospace application tools, 7–8, 861–867
  axes transformations functions, 862t
  environment functions, 863t
  examples of, 881–882, 882f
  flight parameters functions, 862t
  gas dynamics function, 863t
  quaternion math functions, 864t
  subgroups, 861t
  time functions, 864t
  unit conversion functions, 864t
Aerospace Blockset, 7, 867–869, 867f–869f
  examples of, 881–882, 882f
alarming behavior feedback, 218
algebraic equations, 355–359, 357t
AND operator, 181
angle (Z) function, 51t
animation, 326–336, 327t, 329f, 331f, 334t, 335f
annotation functions, 253–254, 253t
approximation formulas, 627–629
Apps sharing, 910–912, 910f–911f, 912t
Arabic digits, 387
area function, 284t
argnames (fun) function, 196t
array-manipulating functions, 108t
arrays, 71
  categorical, 842–848, 843t, 844f–846f, 847t
  cell, 129, 131–134
  character, 114–122, 817t
  compare functions, 820t
  creation, 78–86, 80f–82f, 81t, 83t–84t, 85f–86f
  datetime (duration, calendarDuration), 821–832, 822t, 823t, 825t, 827t, 829t–830t, 832t
  diagonal, functions for creation, 81t
  editing functions, 819t
  element(s)
    addressing, 74–78, 74t, 76f–77f, 76t
    indexing, 72–77, 77f
  find functions, 818t
  functions, 101–110
    array-manipulating, 108t
    to assess array properties, 102t
    diagonal, 81t
    editing, 819t
    find, syntaxes for, 106t
    for square matrices, 110t
    statistical, 105t
    sum and product of array elements, 104t
  to integer data types, functions for converting, 394t
  join functions, 819t
  match functions, 820t
  matrices, 73–74, 73f–74f
  multidimensional arrays, 73–74, 73f–74f
  natural-number, 83t
  operations, 92
    nonstandard, 99–101, 99t
    standard, 92–99, 95t, 96f
  replace functions, 818t
  reshaping, editing, and displaying of, 86–92, 87f–91f
  special, functions for creation, 83t
  split functions, 819t
  string, 114–122
  structure, 134–139
  type/content testing functions, 183t
  types, 72, 72f, 818t
  variables, 72
associative property, 94

audioplayer function, 168
audioread function, 167
axes object, properties of, 262t–263t
axis function, 252, 252t, 253f

B

backward differentiation method, 708–709, 708t
backward substitution process, 424–425, 425f
bar function, 284t, 304–305, 304t, 305f
barycentric interpolation formula, 558
base workspace, 209–213, 209f–211f
Bezier curves, interpolation with, 573–576, 574f–575f
biquinary system, 388
bisection method, 477–480, 478f–480f
bit, 391
block comment, 194
block masking, 869–881, 871f–880f, 881t
blocksets, Simulink, 14–18
  control systems, 15t
  event-based modeling, 15t
  physical modeling, 15t
  simulation graphics and reporting, 16t
  systems design, 16t
  systems engineering, 17t
  verification, validation, and testing, 18t
boundary-value problem (BVP), 684, 733–737, 734t, 737f
boxchart function, 284t
bracketing methods, 475, 476, 477f
  bisection method, 477–480, 478f–480f
  false-position method, 480–482, 481f–482f
  golden-section method, 482–485, 482f, 484f
break statement, 191
BVP. *See* Boundary-value problem
byte, 391

C

calculus functions, 348–355, 350t
calendarDuration arrays, 825t
camera graphics functions, 314t, 315f
categorical arrays, 842–848, 843t, 844f–846f, 847t
  data manipulation functions, 847t
  heatmap charts of, 846f
  histogram chart of, 845f
  pie chart of, 844f
  syntaxes for creation, 843t
  word clouds out of, 848–849, 848f
Cauchy problem, 687
ceil (X) function, 52t
cell arrays, 129, 131–134
  content of, 132f
  graphical interpretation, 131f
  handling functions, 134t
character arrays, 114–122, 817t
char (fun) function, 196t
children function, 359–360, 359t
circshift (A, k), 86t
Code Analyzer (M-Lint), 233–236, 234f, 236f
code designation comments, 218
coefficient of determination, 552
coeffs function, 359t, 360
collect function, 359t, 360
color-varying functions, 316t
comet function, 327–328, 328f
comet3 function, 327–328, 328f
Command History panel, 24, 24f
Command Window, MATLAB, 22–26, 24f
commutative property, 93
compare functions, 820t
complete/maximal pivoting, 427
complex conjugate transpose, 92–93
complex function, 51t
complex (X, Y) (A, B) function, 51t
compose function, 171
computer number representation, 386–387
  fixed-point format, 396–398, 396f–397f
  floating-point format, 398–401, 398f–400f
  integer numbers, 391–395, 392f–393f, 394t–395t
  numerical systems, 387–391, 390f
  single- and double-precision formats, 401–403, 402f, 402t–403t
conditional statements, 184–187, 184f–187f
conj (Z) function, 51t
consistent theory, 439
constants, MATLAB, 54–56, 55t
constrained nonlinear optimization, 514
continue statement, 191
contour functions, 303–304, 303t, 304f
contour plot functions, 283t
control flow statements, 184
conversion functions, 130t
correlation coefficient, 552
cplxpair (Z) function, 51t
cross function, 96–98, 96f
csvread function, 152–153
csvwrite function, 152–153
ctranspose function, 95t
cubic splines, 564–570, 564t–565t, 565f, 568t, 569t, 570f
  defined locally, 570–573
  functions for computation, 587t
cubroot function, 198
cumprod (. . .), 104t
cumsum (. . .), 104t
cumtrapz function, 663–665
Current Folder panel, 41–44, 42f–44f

curve fitting, 545–546
Fourier approximation, 576–579, 576f
full-degree polynomial approximation, 557–561, 560f
least-squares regression, 546
linear regression, 547–549, 547f
linear regression error, 551–552
linear regression to nonlinear relationship, 552–557
multiple linear regression, 550–551
regression analysis with higher-order polynomials, 549–550, 549f
piecewise approximation, 561
cubic splines, 564–570, 564t–565t, 565f, 568t, 569t, 570f
cubic splines defined locally, 570–573
interpolation with Bezier curves, 573–576, 574f–575f
linear splines, 561–564, 562f
quadratic splines, 561–564, 562f, 563t
using MATLAB functions, 579–580
data preconditioning, 605–609, 606t, 607f, 608t, 609f
piecewise polynomial interpolations, 585–598, 587f, 587t–588t, 589f, 590t, 591f–596f, 597t, 598f
polynomial approximations, 580–585, 582f
trigonometric series approximations, 598–605, 598f, 600f–601f, 603f, 605f

D

data
formatted, writing/displaying, 168–172, 169f, 169t–171t
import/export, 150
media data, 164–168, 164t, 165f
numeric and textual data, 150–162, 151t, 154t, 157f–158f, 159t, 162f
using import tool, 162–163, 163f
preconditioning, curve fitting, 605–609, 606t, 607f, 608t, 609f
types, 126–129, 126f, 128f, 387
conversion functions, 130t
fundamental, 127t
table, 129
testing functions, 184t
valid combinations of unlike classes, 128t
datatip function, 258–259
date function, 25
datetime (duration, calendarDuration), 821–832, 822t, 823t, 825t, 827t, 829t–830t, 832t
desktop, MATLAB, 21–22, 22f, 26–32, 26f, 28f–32f
detectImportOptions function, 159
determinant of square matrix, 432–433
detrend functions, 546
diagonal arrays, functions for creating, 81t
diary function, 148–149, 148t
differential equations, 355–359, 357t, 683
autonomous, 684
nonlinear equations, 684
superposition principle, 684
diff function, 612, 630–632, 636
digits function, 364
DLL-files, 148
dlmread function, 151–152
dlmwrite function, 152
double integral, functions for estimating, 666t
double-precision floating-point format, 402–403, 402f, 402t–403t
drawnow function, 329
dsolve function, 358–359, 683
duodecimal system, 388
duration arrays, 825t

E

echo function, 226
editing functions, 819t
Editor, MATLAB, 34–35, 34f–35f
editor/debugger, in MATLAB, 225
code into sections, separation of, 229–231, 230f
debug mode with breakpoints, 231–233, 231f–233f
files/folders comparison, 233
message bar, 227–229
options of, 227–229, 227f–228f
programmatic tools for, 225–227
eigen/spectral decomposition, 435
eigenvalue problem, 468–472, 471f
eigenvector matrix, 448
eig function, 435, 468–472
EISPACK, 2
elementary math functions, 49
element-by-element multiplication, 99–101
engineering plots, requirements to, 326
equation solving functions, 356t
equationsToMatrix function, 357–358
errorbar plotting functions, 283t
error functions, 218
errors, 386
Euler method, 689, 689f
eval function, 359t, 363
excess-*M* method, 392–393
exist function, 213, 214t
expand function, 359t, 360
explicit loops, 188
exponential functions, 51t
exponential law relationship, 554

extrapolation, 549
ezplot function, 249t, 250–251, 250f, 251f

F

false-position method, 480–482, 481f–482f
Favorites, 32–34, 33f
fcnchk function, 221–222, 372–374
feather function, 284t
fft function, 546, 579, 600–604
Fibonacci numbers, 483
fig-file, 148
figure object, properties of, 264t–265t
files, MATLAB, 145–148
  DLL-files, 148
  fig-file, 148
  mat-files, 145–146
  mdl-files, 148
  mex-files, 148
  m-files, 146–148
  mlapp-files, 148
  mlx-files, 148
  p-files, 148
fillmissing function, 546, 606t
filloutliers function, 546, 606t
fimplicit function, 249t, 251, 251f
find functions, 106t, 181, 818t
finverse function, 355
first-order derivative, two-point difference approximations of, 612–613, 612f–613f
first-order ODEs, 449–451, 451f
fixed-point data type, 396–398, 396f–397f
fix (X) function, 52t
floating-point format, 398–401, 398f–400f
floor (X) function, 52t
fmaxbnd function, 202
fminbnd function, 202, 476, 508, 510, 526, 527t, 529
fmincon function, 476, 514
fminhj function, 513, 851–859
fminsearch function, 199, 202–203, 373, 476, 513
fminunc function, 476, 508, 511–512, 525t, 529, 530t, 533–540
fopen function, 169, 170
for loops, 188–189, 188f
formula (fun) function, 196t
formula manipulation and simplification functions, 359–363, 359t
FORTRAN, 2–3
Fourier approximation, 576–579, 576f
fplot function, 202, 249–251, 249t, 250f, 251f
fprintf function, 169–171
full-degree polynomial approximation, 557–561, 560f
function argument validation, 223–225
function-plotting functions, 318–320, 319f, 319t
functions, MATLAB, 49–56, 195
  anonymous, 197–201, 200f, 200t
  as argument, passing of, 221–223
  base workspace, 209–213, 209f–211f
  complex, 51t
  constants, 54–56, 55t
  elementary math, 49
  exponential, 51t
  hyperbolic trigonometric, 50t
  inline, 196–197, 196t
  libraries, 52–54
  local, 205–207, 206f
  main/primary, 205–207, 206f
  m-file, 201–205
  nested, 207–208, 208f
  rounding and remainder, 52t
  search order, 213–215, 214t
  trigonometric, 50t
  usage, 49–52
  workspace, 209–213, 209f–211f
fundamental frequency, 579
funtool function, 382–383
fzero function, 202, 218, 475–476, 515t–516t

G

Gauss/Gaussian elimination, 421–427, 425f
Gauss–Jordan elimination, 427–428
Gauss–Newton iterative method, 556
Gauss quadrature, 654–663, 657t, 658f, 660f, 660t, 661f
generalized Shur decomposition, 435
geographic data plotting functions, 292t, 293f
get function, 258–268
ginput function, 326
Global Optimization Toolbox, 476
global variables, 210
golden ratio, 483
golden-section method, 482–485, 482f, 484f, 508, 509f
gradient-based methods, 510–511
gradient function, 612, 630, 633–636
graphical user interface (GUI), 1, 4, 30–31, 148, 177
  to App Designer, 907–910, 908t, 909f–910f
  examples of, 899–904, 902f–904f
  laying out and programming, 892–899, 893f–895f, 897f–899f
  MATLAB apps, 904–905, 904f–905f
  standalone executables, 905–907, 905f
  for symbolic computations, 379t
  workflow for, 883–884, 883f
graphics handle, 259
graphics objects, 258–259, 258f, 266t
graphics root, 259

gridatan function, 546
griddata function, 546
griddedInterpolant function, 546
grid function, 252, 252t, 253f
GUI development environment (GUIDE), 883–884, 883f, 887–892
  vs. App Designer, 907–910, 908t, 909f–910f
  preferences window, 890f
  Quick Start window, 887f
  template, 887f
  toolbar, 889f, 889t
  UI options available in, 888t

H

harmonics, 579
help system, MATLAB, 46–49
  browser, 46–47, 46f
  documentation viewing, 48f
  Examples page, 49f
  functions, 47t
  options, 46
  search results viewing, 48f
Hermite interpolation, 571–573
Heun method, 692
higher-order polynomials, regression analysis with, 549–550, 549f
histograms function, 284t
homogeneous sets of LAEs, 446–448
Hooke–Jeeves algorithm, 513
Hooke–Jeeves minimization function, 851
  description, 851–852
  examples of, 858–859, 859f
  m-file function listing, 852–858
horner function, 359t, 361
hyperbolic trigonometric functions, 50t

I

if constructs, 184–186, 184f–185f
if-else construct, 186–187, 186f–187f
imag function, 364
imag (Z) function, 51t
implicit loops, 188
implicit trapezoidal rule, 712–714, 714t
importdata function, 158–159, 159t
imprecision, 386
imread function, 164
imshow function, 165–166
inaccuracy, 386
inconsistent sets of LAEs, 437f–439f, 438–439
inhomogeneous sets of LAEs, 421, 421f
  determinant of square matrix, 432–433
  Gauss/Gaussian elimination, 421–427, 425f
  Gauss–Jordan elimination, 427–428
  inverse of square matrix, 431–432, 432t
  lower-upper (LU) decomposition, 428–431
  solution, iterative improvement of, 433–434
initial-value problem (IVP), 683–684
  adaptive single-step algorithms, 698–701
  definitions, 684–687, 686f
  multistep methods, 701
    Adams–Bashforth–Moulton predictor–corrector method, 701–708, 703f, 706f, 707t, 708t
    backward differentiation method, 708–709, 708t
    numerical differentiation method, 709, 709t
  single-step methods, 687–688
    higher-order formulas, 695–698, 696f–697f
    low-order formulas, 688–695, 689f, 691f, 694f
  stiff problems, 710
    implicit trapezoidal rule, 712–714, 714t
    integration step size controlling, 715–716
    Jacobian matrix, incorporation of, 710–712
  using MATLAB IVP solvers, 716–730, 717t, 719f–721f, 723t–724t, 724f–725f, 727f, 729f–730f
    BVP solver, 733–737, 734t, 737f
    IVPs with delays, solvers for, 730–733, 733f
    PDE solver, 737–740, 740f
inline function, 196–197, 196t, 372–374
input arguments, enabling variable-length list of, 218–221
input/output, MATLAB, 172–177, 174t, 175f–177f
integer numbers, 391–395, 392f–393f, 394t–395t
integration step size controlling, 715–716
interactive plotting, 326–336, 327f–329f
interp1 functions, 590t
interpn functions, 590t
interpolation, 548–549
  with Bezier curves, 573–576, 574f–575f
  methods, 597t
inverse of square matrix, 431–432, 432t
inverse quadratic interpolation method, 495–498, 496f–498f
isempty function, 220
iskeyword, 23
isolate function, 359t, 360
isreal (X) function, 51t
istril (A), 84t
istriu (A), 84t
IVP. *See* Initial-value problem

J

Jacobian matrix, incorporation of, 710–712
join functions, 819t

K

Kronrod rule points, 660t, 675t

L

LAEs. *See* Linear algebraic equations
Lagrange interpolating polynomial, 558
LAPACK, 2
LaTeX
  equations in live editor, 803–805, 803f–805f
  math constructs, 811–814, 812t, 814f
  matrices and cases, 814–816, 814t, 816f
  representation of symbolic expressions, 366–367
  symbols, 806–811, 806t–809t, 811f
ldivide function, 99t
least significant bit (LSB), 391
least-squares regression, 546
  linear regression, 547–549, 547f
    error, 551–552
    to nonlinear relationship, 552–557
  multiple linear regression, 550–551
  regression analysis with higher-order polynomials, 549–550, 549f
least-squares solution, 548
left rectangles (LR) formula, 640, 641f
length (A), 102t
lhs function, 359t, 361–362
libraries, functions, 52–54
lighting functions functions, 312–313, 312f, 313t
line animation functions, 327t, 334t
linear algebra functions, 348, 349t
linear algebraic equations (LAEs), 419
  classification of, 420–421, 420f
  eigenvalue problem, 468–472, 471f
  homogeneous sets of, 446–448
  inconsistent sets of, 437f–439f, 438–439
  inhomogeneous sets of, 421–434, 421f
  linear differential equations, 468–472, 471f
  matrix decompositions, 434–437
  matrix eigenvalue problems of, 446–448
  for ODEs, 448–449
    first-order, 449–451, 451f
    solving systems of LAEs, 452–467, 453t, 460t, 462f, 464f, 466t, 467f
    state–space representation of, 451–452
  overdetermined sets of, 443–446, 444f–447f
  solving systems of, 452–467, 453t, 460t, 462f, 464f, 466t, 467f
  special cases of, 437–448, 437f
    homogeneous sets, 446–448
    inconsistent sets, 437f–439f, 438–439
    matrix eigenvalue problems, 446–448
    overdetermined sets, 443–446, 444f–447f
    underdetermined sets, 439–443, 440f–441f
  underdetermined sets of, 439–443, 440f–441f
linear differential equations, 468–472, 471f
linear indexing, 76
linear regression, 547–549, 547f
  error, 551–552
  to nonlinear relationship, 552–557
linear splines, 561–564, 562f
line function, 249t
line graphics properties, 256t
line object, properties of, 260t
LineSpec functions, 255–256, 255t, 256f
line specifications, 255–258, 255t, 256f–258f, 256t
LINPACK, 2
listdlg function, 176
Live Editor, 36–38, 36f–38f, 38t
load function, 150–151, 151t
local function/subfunction, 205–207, 206f
logarithmic functions, 283t
logical (Boolean) operators and functions, 181–184
  array type/content testing functions, 183t
  data type testing functions, 184t
loops, 188
  for, 188–189, 188f
  break statement, 191
  continue statement, 191
  explicit, 188
  implicit, 188
  while, 189–190, 190f
lower triangular matrix, 422
lower-upper (LU) decomposition, 428–431
lroot function, 501–503
LSB. *See* Least significant bit

M

[m, n] = size (A), 102t
Maclaurin series expansion, 410
main/primary functions, 205–207, 206f
makima function, 571
match functions, 820t
mat-files, 145–146
MathWork
  products, 5f, 8–9
    families, 8f, 9–14
    Simulink family, 14–18
  website, 2f, 6–7, 6f

MathWorks, Inc., 1
MATLAB, 1, 125
  Aerospace application tools, 7–8
  animation, 247–336
  arrays (*See* Arrays)
  capabilities, 4–7
  cell arrays, 129, 131–134
  coder *vs.* compiler, 14, 14t
  constants, 54–56, 55t
  creation, history of, 1–4
  data
    formatted, writing/displaying, 168–172
    importing/exporting of, 150–168
    types, 126–129
  default desktop of, 21–22, 22f
  development, history of, 1–4, 3t
  development environment, 21–46
    Command Window, 22–26, 24f
    Current Folder panel, 41–44, 42f–44f
    Desktop overview, 26–32, 26f, 28f–32f
    Editor, 34–35, 34f–35f
    Favorites, 32–34, 33f
    Live Editor, 36–38, 36f–38f, 38t
    search path setting, 44–46, 45f
    Variables Editor, 40–41, 40f–41f
    workspace panel, 39–40, 39f–40f
  external interface, 5
  files, 145–148
  FORTRAN, 2–3, 3t
  functions, 49–56
    complex, 51t
    constants, 54–56, 55t
    elementary math, 49
    exponential, 51t
    hyperbolic trigonometric, 50t
    libraries, 52–54
    rounding and remainder, 52t
    trigonometric, 50t
    usage, 49–52
  help system, 46–49
  importing/exporting of data, 150–168
  input/output, 172–177
  m-script, 6f
  Online, 21–46
  plotting capabilities, 247–336
  precedence, order of, 56–57
  predefined dialog boxes in, 174t
  preferences settings, 63–67, 64f–67f
  problem-solving methodology, 68–70
  products, 5–6
  programming, 179–245
  resources, 4–7
  scalar arithmetic operations, 26t
  session recording, 148–150
  and Simulink student version, 18–19
  storing workspace variables, 148–150
  structure arrays, 134–139
  tables, 139–145
  toolboxes, 9–14
    application deployment, 14t
    artificial intelligence, data science and statistics, 10t
    autonomous systems and aerospace, 11t
    code-generation and verification, 13t
    computational biology, 12t
    computational finance, 12t
    control system, 10t
    database access and reporting, 13t
    image processing and computer vision, 11t
    Math and optimization, 9t
    parallel and distributed computing, 13t
    signal processing, 10t
    test and measurement, 12t
    wireless communications, radio frequency, and mixed-signal, 11t
  user community, 6f
  uses for, 1
  version, 3–4
  work session, 57–63
MATLAB Compiler Runtime (MCR), 14
matlabFunctionBlock function, 793
matlabFunction function, 372, 374–377, 491
matrices, 73–74, 73f–74f
  formalism to handle polynomials, 110–114, 111t, 113t
  power operation, 98–99
  square matrices functions, 110t
  triangular, functions for creating, 84t, 85–86, 85f–86f
matrix decompositions, LAEs, 434–437
matrix eigenvalue problems, 446–448
mdl-files, 148
media data, import/export, 164–168, 164t, 165f
meshgrid function, 295–297, 295f–297
mesh plotting functions, 297t
mex-files, 148
m-file functions, 201–205
m-files, 146–148
midpoint method, 693, 694f
minus function, 95t
mlapp-files, 148
mldivide function, 95t, 421
mlx-files, 148
mod (a, m) function, 52t
most significant bit (MSB), 391
mpower function, 95t
mrdivide function, 95t, 421
MSB. *See* Most significant bit
mtimes function, 95t
multidimensional arrays, 73–74, 73f–74f

multidimensional optimization, 510–514, 511f–512f
multiple-application scheme, 652–654, 653f
multiple linear regression, 550–551
multiple-point approximations, 617–621, 621t
multiple roots bracketing, 501–503, 503f
MuPAD Notebooks, 340

N
namelengthmax function, 23
nargin function, 218–219
nargout function, 219
natural-number arrays, 83t
ndims (A), 102t
Nelder–Mead method, 513
nested functions, 207–208, 208f
Neural Net Fitting app, 30–31, 31f
Newton–Cotes formulas, 650–652, 651t
Newton's interpolating polynomial, 559
Newton's method, 490–494, 490f, 492f–493f, 510
nonlinear equations, 684
nonstandard arrays operations, 99–101, 99t
NOT operator, 181
numbers and precision group functions, 364
number theory group functions, 364, 366t
numden function, 359t, 361
numel (A), 102t
numerical differentiation, 611–612
  approximation formulas, 627–629
  first-order derivative, two-point difference approximations of, 612–613, 612f–613f
  MATLAB functions for, 630–637
  method, 709, 709t
  multiple-point approximations, 617–621, 621t
  partial derivatives, 629–630
  second- and higher-order derivatives, 621–626, 626t
  Taylor series expansion, error estimates from, 613–617
numerical errors, control of, 416–418
numerical integration, 639–640
  Gauss quadrature, 654–663, 657t, 658f, 660f, 660t, 661f
  multiple-application scheme, 652–654, 653f
  Newton–Cotes formulas, 650–652, 651t
  rectangle method, 640–641, 641f
  Simpson's 1/3 rule, 645–649, 646f, 647f
  Simpson's 3/8 rule, 649–650, 649f
  trapezoidal rule, 642–645, 642f, 644f
  using MATLAB functions, 663–680, 664f, 665t–666t, 669f, 670t, 672f, 674f, 675t, 676f–678f
numerical round-off errors, 387
numerical systems, 387–391, 390f
numeric and textual data, import/export, 150–162, 151t, 154t, 157f–158f, 159t, 162f
Nyquist–Shannon sampling theorem, 579

O
odefun function, 378–379, 717
ODEs. *See* Ordinary differential equations
one-dimensional (1-D) arrays, 72
open methods, 475, 485–486, 485f
  inverse quadratic interpolation method, 495–498, 496f–498f
  Newton's method, 490–494, 490f, 492f–493f
  secant method, 494–495, 495f
  simple fixed-point iteration, 486–490, 487f, 489f
open-source paradigm, 215–216, 216f, 217f
optimization
  functions, 524–541, 525t–527t, 526f, 528f, 530t–531t, 532f–536f, 538f–540f
  results, 541–542, 542f
optimoptions function, 524
optimset function, 789
ordinary differential equations (ODEs), 683
  higher-order into state space, 377–379, 378f–379f
  linear algebraic equations for, 448–449
    first-order, 449–451, 451f
    solving systems of LAEs, 452–467, 453t, 460t, 462f, 464f, 466t, 467f
    state–space representation of, 451–452
OR operator, 181
orthogonal columns, 97
orthogonal matrix, 97–98
output arguments, enabling variable-length list of, 218–221
overdetermined sets of LAEs, 443–446, 444f–447f

P
partial derivatives, 629–630
partial differential equations (PDEs), 683
  solver, 737–740, 740f
partial pivoting, 426–427
patch plotting functions, 307t, 308–310, 309f–310f
pause function, 226, 329
pchip function, 546, 571–572, 585–588
PDEs. *See* Partial differential equations
percent error, 408
perms (v), 86t
permutation functions, 86t
p-files, 148
PID Tuner app, 30–31, 31f

piecewise approximation, curve fitting, 561
cubic splines, 564–570, 564t–565t, 565f, 568t, 569t, 570f
cubic splines defined locally, 570–573
interpolation with Bezier curves, 573–576, 574f–575f
linear splines, 561–564, 562f
quadratic splines, 561–564, 562f, 563t
using MATLAB functions, 585–598, 587f, 587t–588t, 589f, 590t, 591f–596f, 597t, 598f
pie functions, 284t, 304–305, 304t, 305f
pinv function, 436
pivot, 422
Plot Catalog window, 89f
plot function, 249–250, 249t, 250f, 256
Plots tab gallery, 88f
plotting areas functions, 284t
plus function, 95t
polar plot functions, 283t
polyder function, 612, 630, 635–636, 635t
polyfit function, 546, 560–561, 580–585
polygon plotting functions, 307t, 309f–310f
polygons functions, 290t
polynomials
approximations, 580–585, 582f
deflation, 506
group functions, 363, 364t, 365t
roots of, 506–507
wiggle, 560
positional notation, 388
power function, 99t
precedence, order of, 56–57
precision of calculations/measurements, 385–386, 386f
predefined dialog boxes, 174t
prediction–correction method, 692
preferences settings, MATLAB, 63–67, 64f–67f
Colors, 65f
Command History, 64f
Command Window, 64f
editor/debugger, 66, 66f
figure copy options, 67f
toolbars, 65f
variables editor, 67f
preset colormaps, 316f, 317–318, 317t, 318f
private functions, 215
prod (. . .), 104t
profiler, 236–238, 237f
programming in MATLAB, 179
alarming behavior feedback, 218
Code Analyzer (M-Lint), 233–236, 234f, 236f
code designation comments, 218
comments for publishing, formatting, 238–245, 239f–240f, 241t–242t, 243f–244f
conditional statements, 184–187, 184f–187f
editing/debugging, 225
code into sections, separation of, 229–231, 230f
debug mode with breakpoints, 231–233, 231f–233f
files/folders comparison, 233
message bar, 227–229
options of, 227–229, 227f–228f
programmatic tools for, 225–227
function argument validation, 223–225
functions, 195
anonymous, 197–201, 200f, 200t
as argument, passing of, 221–223
base workspace, 209–213, 209f–211f
inline, 196–197, 196t
local, 205–207, 206f
main/primary, 205–207, 206f
m-file, 201–205
nested, 207–208, 208f
private, 215
search order, 213–215, 214t
workspace, 209–213, 209f–211f
input and output arguments, enabling variable-length list of, 218–221
logical (Boolean) operators and functions, 181–184, 182t, 183t, 184t
loops, 188
for, 188–189, 188f
break statement, 191
continue statement, 191
while, 189–190, 190f
open-source paradigm, 215–216, 216f, 217f
profiler, 236–238, 237f
relational operators, 180–181, 180t
script files, 193–195
switch construct, 191–193, 192f
psi function, 366

Q

QR decomposition, 434–435
qr function, 434
quad function, 653–654
quadgk function, 659, 668–669
quadratic interpolation method, 509–510, 509f
quadratic splines, 561–564, 562f, 563t
quasi-Newton methods, 512–513
questdlg function, 174
quiver function, 284t
qz function, 435–436

R

Ralston method, 694
rand (), 83t, 84
randn (), 83t, 84
randperm (n, k), 86t
rank function, 436
rate of convergence, 489
rdivide function, 99t
readcell function, 159
readmatrix function, 159
real function, 364
real (Z) function, 51t
recordblocking function, 168
rectangle function, 329
rectangle method, 640–641, 641f
regression analysis, 546
  with higher-order polynomials, 549–550, 549f
  multiple linear regression, 550–551
relational operators, 180–181, 180t
relative error, 408
rem (a, b) function, 52t
replace functions, 818t
repmat (A, [m, n]), 84t
repmat (A, n), 84t
rewrite function, 359t, 362
RGB values, 315t
rhs function, 359t, 361–362
right rectangles (RR) formula, 640, 641f
rng (), 84
root finding, 507–508
  functions, 514–524
  golden-section method for, 508, 509f
  Newton's method for, 510
  quadratic interpolation method for, 509–510, 509f
root multiplicity, handling of, 504–505, 504f
root polishing, 506
roots function, 475, 507
roots of polynomials, 506–507
rounding and remainder functions, 52t
round-off constant, 400
round-off errors, 403
  fixed-point *vs.* floating-point numbers, 403–404
  numerical, 387
  precision, 404–407, 406f–407f
  range, 404–407, 406f–407f
rounds (X) function, 52t
rsums function, 380–381
Runge–Kutta (RK) methods, 687–688
runtime errors, 225

S

save function, 149–150, 149t
scalar arithmetic operations, 26t
scalar/dot product, 95–96
scaled partial pivoting, 427
scatter charts functions, 290t
scatteredInterpolant function, 546
scatter plots functions, 310t, 311f, 312
script files, 193–195
search order function, 213–215, 214t
search path setting, 44–46, 45f
secant method, 494–495, 495f
second- and higher-order derivatives, 621–626, 626t
set function, 258–268
sexagesimal system, 388
shape functions, 305t, 306f–307f, 307
sign-and-magnitude, 391, 392f
sign (X) function, 52t
sim function, 789
simget function, 790
simple fixed-point method, 486–490, 487f, 489f
simplify function, 359t, 362
Simpson's 1/3 rule, 645–649, 646f, 647f
Simpson's 3/8 rule, 649–650, 649f
simset function, 789
Simulink, 743–744
  Blank Model window, 746f
  Block Libraries help topics, 747f
  blocksets, 14–18
    control systems, 15t
    event-based modeling, 15t
    physical modeling, 15t
    simulation graphics and reporting, 16t
    systems design, 16t
    systems engineering, 17t
    verification, validation, and testing, 18t
  code-generation tools, 17t
  continuous dynamics modeling, 750–751, 751f
    accelerating creating models, 761–771, 762f–770f, 763t
    integrator block configuration, 751–761, 752f, 754f–761f, 759f
    State–Space block, 773–774, 773f–774
    transfer function, 771–773, 772f
  development environment, 744–750, 745f–749f
  discrete-type systems modeling, 774
    continuous model discretization and controller design, 779–781, 780f–781f
    discrete library blocks to model simple systems, 774–779, 775f–779f
  integration of MATLAB and, 782–785, 783f–785f
    calling Simulink models, 789–791, 789f
    commands, 785–789, 786f–788f

MATLAB Function blocks generation from symbolic expressions, 793–794, 794f
publishing Simulink models, 791–793, 792f
library browser, 745–750, 746f
math operations library blocks, 747f
model reviewing and profiling, 798–801, 798f–801f
practical examples, 794–798, 795f–797f
Preferences window, 749f
real-time simulation and testing tools, 18t
signal lines and labels, 763t
sinks library blocks, 746f
sources library blocks, 746f
start page, 745f
student version, 18–19
Toolstrip, 748f
sin function, 199
single integral, functions for estimating, 665t
single-precision floating-point format, 401–402, 402f, 402t–403t
singular value decomposition (SVD), 436
size (A), 102t
SMT. *See* Symbolic Math Toolbox
solve function, 355, 357–359
sound function, 167
special arrays, functions for creating, 83t
special functions, 369t
special symbols, 61t, 269t
spline functions, 546, 561–573, 585–591
cubic, 564–570, 564t–565t, 565f, 568t, 569t, 570f
cubic splines defined locally, 570–573
linear, 561–564, 562f
quadratic, 561–564, 562f, 563t
split functions, 819t
sprintf function, 171
square matrix
determinant of, 432–433
functions, 110t
inverse of, 431–432, 432t
sscanf function, 171
stairs plotting functions, 283t
standalone executables, 905–907, 905f
standard array operations, 92–99, 95t, 96f
addition, 93–94
complex conjugate transpose, 92–93
cross function, 98
division, 99
functions for, 95t
matrix power operation, 98–99
multiplication, 94–96
subtraction, 94
standard deviation, 551
state–space representation, 377, 451–452
statistical functions, 105t
stem plotting functions, 283t
stopping criterion, 409
string arrays, 114–122
structure arrays, 134–139, 135f, 136t, 137f, 139f, 140t
sum (. . ., 'omitnan'), 104t
sum (A), 104t
sum (A, 'all'), 104t
sum (A, dim), 104t
sum (A, vecdim), 104t
superposition principle, 684
surface plotting functions, 297t, 298t, 299–300, 299f
SVD. *See* Singular value decomposition
switch construct, 191–193, 192f
symbolic calculators, 379–382, 379t, 380f–381f
symbolic expressions, SMT
LaTeX representation of, 366–367
symbolic expressions
plotting, 367–372, 368f, 369t, 370f–372f
Symbolic Math Toolbox (SMT), 339
algebraic equations, 355–359, 357t
areas supported by, 340t
assumptions functions, 342t
calculus functions, 348–355, 350t
differential equations, 355–359, 357t
equation solving group of, 355–359, 357t
flag options, 342t
formatted conversion to symbolic numbers, 343t
formula manipulation and simplification group of, 359–363, 359t
linear algebra functions, 348, 349t
numbers and precision group functions, 364
number theory group functions, 364, 366t
numerical operations, symbolic expressions in, 372–379
fcnchk function, 372–374
higher-order ODEs into state space, 377–379, 378f–379f
inline function, 372–374
matlabFunction function, 374–377
overview, 340
polynomials group functions, 363, 364t, 365t
symbolic calculators, 379–382, 379t, 380f–381f
symbolic expressions
LaTeX representation of, 366–367
in numerical operations, 372–379
plotting, 367–372, 368f, 369t, 370f–372f
symbolic numbers to numeric numbers, functions for converting, 343t
symbolic objects creation, 340–348
basic operations, 346, 347t

Symbolic Math Toolbox (SMT) (*Continued*)
expressions, 343–344, 344t
functions, 344–345
help, 346–348
variables, 341–342, 341t
transforms functions of, 354t
units of measurement, 382, 382f
symbolic numbers
formatted conversion to, 343t
to numeric numbers, functions for converting, 343t
synthetic vision approach, 506

T

tables, MATLAB, 139–145, 146t, 147t, 291t
Taylor series expansion, 410, 412t, 613–617
taylortool (f) function, 383
taylortool function, 383
ternary search algorithms, 485
testarg function, 219
texlabel function, 271
text functions, 253–254, 253t
textread function, 153–158
textscan function, 153–158
text strings, 268–271, 269t–270t, 270f
three-dimensional (3-D) arrays, 75f
three-dimensional (3-D) plotting, 247–248, 292, 294–310
function-plotting functions, 318–320, 319f, 319t
functions, 247
bar, 304–305, 304t, 305f
camera graphics, 314t, 315f
color-varying, 316t
contour, 303–304, 303t, 304f
lighting functions, 312–313, 312f, 313t
mesh plotting, 297t
patch plotting, 307t, 308–310, 309f–310f
pie, 304–305, 304t, 305f
polygon plotting, 307t, 309f–310f
scatter plots, 310t, 311f, 312
shape, 305t, 306f–307f, 307
surface plotting, 297t, 298t, 299–300, 299f
triangular surfaces and shapes, 301–303, 301t, 302f–303f
vector-type data, 303t, 304–305, 305f
volumetric data visualization, 310t, 311f, 312
plot editing, 320–326, 320f–322f, 321t
times function, 99t
timetable, 832–836
cleaning workflow, 837–842, 839f, 842t
data type handling, functions for, 838t
functions, 834t
tolerance, 409
toolboxes, MATLAB, 9–14
application deployment, 14t
artificial intelligence, data science and statistics, 10t
autonomous systems and aerospace, 11t
code-generation and verification, 13t
computational biology, 12t
computational finance, 12t
control system, 10t
database access and reporting, 13t
image processing and computer vision, 11t
Math and optimization, 9t
parallel and distributed computing, 13t
signal processing, 10t
test and measurement, 12t
wireless communications, radio frequency, and mixed-signal, 11t
transpose function, 95t
trapezoidal rule, 642–645, 642f, 644f
triangular matrices, functions for creating, 84t, 85–86, 85f–86f
triangular plots function, 290t
triangular surfaces and shapes functions, 301–303, 301t, 302f–303f
trigonometric functions, 50t
trigonometric series approximations, 598–605, 598f, 600f–601f, 603f, 605f
tril (B, k), 84t
triple integral, functions for estimating, 666t
triu (B, k), 84t
trivial solution, 446
true error, 408
truncation errors, 407
definitions, 408–410
formulas to fundamental arithmetic operations, 411–416, 413f
truth table, 181–182
two-dimensional (2-D) plotting, 247–248
anatomy of, 248–255, 248f
functions, 247
axis, 252t
basic, 249t
grid, 252t
modifying axis tick label parameters, 254t
text and annotation, 253–254, 253t
legends, 275–277, 276t, 277f
line specifications, 255–258, 255t, 256f–258f, 256t
multiple figures, 277–282, 280f, 282f
other types of, 282–292, 283t–284t, 285f–289f, 290t–292t, 291f
overlays, 271–275, 272f, 272t
plot properties, access of, 258–268, 258f, 267f
axes object, properties of, 262t–263t
figure object, properties of, 264t–265t

graphics objects, alternative ways of retrieving, 266t
line object, properties of, 260t
subplots, 277–282, 278f–279f
text strings, 268–271, 269t–270t, 270f
two's complement, 393–394
Tylor function, 353t

U

uigetfile function, 176
UI graphics objects, 884–887
uiputfile function, 176–177
uminus function, 95t
uncertainty. *See* Imprecision
underdetermined sets of LAEs, 439–443, 440f–441f
unwrap (X) function, 51t
uplus function, 95t
upper triangular matrix, 422

V

value-finding problem, 505
Vandermonde matrix, 557
Variable context menu, 89f
variables, 21–23
Variables Editor, MATLAB, 40–41, 40f–41f, 88f
vector, 72
vectorize (fun) function, 196t
vector-type data functions, 303t, 304–305, 305f
vector-valued anonymous function, 200–201
VideoReader function, 166–167
VideoWriter function, 167
view function, 292, 294–295, 294f
vigesimal system, 388
V-model, systems engineering process, 16f
volumetric data visualization functions, 310t, 311f, 312
vpa function, 364

W

waitbar function, 175
warndlg function, 175
warning functions, 218
weird number, 394
while loops, 189–190, 190f
word, 387
word clouds, 848–849, 848f
work session, MATLAB, 57–63, 60f
commands for, 62t
output display format, 58t
special symbols, 61t
workspace panel, MATLAB, 39–40, 39f–40f
writecell function, 159
writematrix function, 159
writetable function, 159
writetimetable function, 159

Z

zeroin algorithm, 475, 498–501, 499f–500f

Supporting Materials

To download supplemental material files, please go to AIAA's electronic library, Aerospace Research Central (ARC), at arc.aiaa.org. Use the menu bar at the top to navigate to Books > AIAA Education Series; then, sort alphanumerically by title to find the book's landing page.

A complete listing of titles in the AIAA Education Series is available from AIAA's electronic library, Aerospace Research Central (ARC), at arc.aiaa.org. Visit ARC frequently to stay abreast of product changes, corrections, special offers, and new publications.

AIAA is committed to devoting resources to the education of both practicing and future aerospace professionals. In 1996, the AIAA Foundation was founded. Its programs enhance scientific literacy and advance the arts and sciences of aerospace. For more information, please visit www.aiaafoundation.org.

ABOUT THE BOOK

Engineering Computations and Modeling in MATLAB®/Simulink®, Second Edition provides a broad overview of The MathWorks' MATLAB®/Simulink®/GUIDE/App Designer development and modeling environment. This textbook shows how to use these software tools effectively in a variety of engineering applications, including modeling real-world dynamic systems. This book is used as a textbook by undergraduate and graduate students taking both introductory and advanced courses in applied programming and numerical analysis, and also as a reference for working professionals using MATLAB®/Simulink®.

Since 2002 MathWorks has followed a twice-yearly general release schedule introducing new functions, features, concepts, and toolboxes. It is impossible to fully describe all the functions of MATLAB®, or keep pace with every new software release, in one book. Besides, such a book would only have a 6-month lifetime. However, being a proficient applied programmer means knowing the basics and understanding a variety of available resources so that, if needed, knowledge in certain areas can be built up fast, or new topics can be quickly incorporated. This book is intended to provide the user with exactly this type of firm grounding in the basics as well as the ability to understand and implement updates and changes going forward.

Topics Discussed:

- Major concepts of MATLAB®, Symbolic Math Toolbox™, Simulink®, and GUIDE / App Designer
- Programing techniques and development of effective foolproof m-file scripts and functions
- Data and time-stamped data import, storage, processing, and analysis
- 2-D and 3-D visualization and graphics object programming
- Symbolic manipulations in numerical analysis
- Accuracy and precision of digital computations
- Effective usage of numerical methods, coded in MATLAB® functions
- Tools for aerospace and dynamic systems modeling
- Building and deploying interactive user interfaces

ABOUT THE AUTHOR

OLEG A. YAKIMENKO is Distinguished Professor of Systems Engineering/Mechanical and Aerospace Engineering and Associate Dean of Research at the Naval Postgraduate School, Monterey, CA. He has been teaching the Engineering Computations course for over 30 years. He is the author or co-author of over 300 publications about modeling, guidance, navigation, and control of air, surface, and underwater vehicles, satellites, guided weapons, and parachutes. He holds two MS degrees in aerospace engineering and operations research from the Moscow Institute of Physics and Technologies and the Air Force Engineering Academy. He also holds two PhD degrees in these areas. He is a Chair of AIAA's Aerodynamic Decelerator Systems Technical Committee, Deputy Director of Education for the Western Region, and an Associate Fellow of AIAA.

American Institute of Aeronautics and Astronautics

aiaa.org

ISBN: 978-1-62410-515-9

ISBN 978-1-62410-515-9 $109.95

10995

9 781624 105159